Rust programozás C++-fejlesztőknek – a nulláról a mesterszintig

Gellai István

2026

Tartalomjegyzék

Impresszum **5**

I. RÉSZ – Indulás: eszközök, filozófia, első program **6**

1 Olvasói útmutató **7**
1.1 A könyv használata: C++-ból Rustba, mintaprojekt, olvasási utak 7

2 Eszközök és a Rust filozófiája **22**
2.1 Miért Rust? Biztonság, teljesítmény, és ami a C++-ban hiányzik 22
2.2 Eszközlánc: rustup, cargo, rustfmt, clippy, dokumentáció 32
2.3 Az rkv mintaprojekt indítása: Cargo-workspace és első CLI 43

II. RÉSZ – Az alapok C++-os szemmel **54**

3 Szintaxis, típusok, kifejezések **55**
3.1 Változók, mutabilitás, skalár típusok és konverziók 55
3.2 Kifejezések, utasítások és vezérlési szerkezetek 64
3.3 Tuple-ök, tömbök és az első pillantás a slice-okra 69

4 Structok, enumok és mintaillesztés **78**
4.1 Structok és metódusok: impl blokkok, asszociált függvények 78
4.2 Az enum mint összeg-típus és az Option a null helyett 87
4.3 Mintaillesztés: match, if let, let else és a minták 96

5 Modulok, crate-ek és láthatóság **108**
5.1 Modulok és láthatóság: header nélküli világ 108
5.2 Függőségek, feature-ök, verziózás és workspace-ek 119

III. RÉSZ – Ownership, borrowing, lifetime: a Rust szíve **130**

6 Ownership és mozgatás **131**
6.1 Ownership: minden értéknek egy tulajdonosa van 131
6.2 Copy, Clone és Drop: másolás és RAII Rustban 138

7 Borrowing **147**
7.1 Referenciák és a kölcsönzési szabály: sok olvasó XOR egy író 147
7.2 A borrow checker működése: non-lexical lifetimes és tipikus javítások 155

8 Élettartamok **166**
8.1 Élettartam-annotációk és az elision szabályai 166
8.2 Referenciát tartalmazó struktúrák, 'static és tervezési döntések 175

9 Slice-ok és stringek **185**
9.1 Slice-ok, String és &str, UTF-8 185

10 Smart pointerek és belső mutabilitás **194**
10.1 Box, Rc, Arc és Weak . . . 194
10.2 Belső mutabilitás: Cell, RefCell és a futásidejű kölcsönzés . . . 204

IV. RÉSZ – Absztrakciók: hibakezelés, trait-ek, generikusok, iterátorok **212**

11 Hibakezelés **213**
11.1 Result, a ? operátor és a panic helye . . . 213
11.2 Saját hibatípusok: enum-hibák, thiserror, anyhow . . . 221
11.3 I/O és szerializáció: fájlok, pufferelés, serde . . . 231

12 Generikusok és trait-ek **247**
12.1 Trait-ek és generikusok: sablonok és concepts helyett . . . 247
12.2 Asszociált típusok, koherencia, operátorok és derive . . . 256
12.3 A standard trait-ek: konverziók, formázás, Default, Deref, Borrow . . . 267
12.4 Trait objektumok és dinamikus diszpécs . . . 281

13 Closure-ök és iterátorok **292**
13.1 Closure-ök: Fn, FnMut, FnOnce és a capture . . . 292
13.2 Iterátorok: lusta adapterek és saját iterátor . . . 302

14 Gyűjtemények **311**
14.1 Vec, HashMap, BTreeMap és társaik; az Entry API . . . 311

15 Tesztelés és dokumentáció **321**
15.1 Unit-, integrációs és dokumentációs tesztek . . . 321
15.2 Property-alapú tesztelés és benchmark . . . 329

V. RÉSZ – Párhuzamosság és aszinkronitás **343**

16 Szálak és megosztott állapot **344**
16.1 Szálak, Send és Sync: data race fordítási időben kizárva . . . 344
16.2 Mutex, RwLock, csatornák és a megosztott rkv . . . 354
16.3 Atomikus műveletek és memóriarendezés . . . 367
16.4 Adatpárhuzamosság rayonnal . . . 378

17 Aszinkron Rust **390**
17.1 Future-ök és async/await: mi történik a motorháztető alatt . . . 390
17.2 Pin, Unpin, async trait-ek és a Send-határok . . . 400
17.3 Tokio: taskok, I/O, csatornák, select és megszakítás . . . 406
17.4 Az rkv hálózati szerver: TCP, protokoll, kecses leállás . . . 419
17.5 Naplózás és megfigyelhetőség: tracing . . . 446

VI. RÉSZ – Haladó Rust: unsafe, FFI, makrók, típusrendszer, teljesítmény **465**

18 Unsafe Rust **466**
18.1 Unsafe Rust: szuperképességek és felelősség . . . 466
18.2 Biztonságos absztrakció unsafe alapon . . . 477

19 FFI: C és C++ együttműködés **486**
19.1 C ABI: extern “C”, repr(C), bindgen és cbindgen . . . 486
19.2 C++-együttműködés a cxx crate-tel és a fokozatos migráció . . . 502

20 Makrók **518**
20.1 Deklaratív és procedurális makrók 518

21 Haladó típusrendszeri minták és API-tervezés **530**
21.1 Newtype, PhantomData, typestate és const generikusok 530
21.2 API-tervezés Rustban: irányelvek és ergonómia 543

22 Teljesítmény **554**
22.1 Teljesítmény: mérés, profilozás, allokáció, release-beállítások 554

23 Beágyazott Rust kitekintés **566**
23.1 no_std, embedded-hal és Embassy: Rust mikrovezérlőn 566

VII. RÉSZ – Záróprojekt és összefoglalás **582**

24 Záróprojekt és összefoglalás **583**
24.1 Záróprojekt: az rkv teljes rendszerként 583
24.2 Összefoglalás: C++→Rust fordítási kézikönyv és továbbtanulás 605

Függelékek **618**
Fogalomtár (magyar–angol) 618
Képlet- és jelölésgyűjtemény 620
Kompetenciatérkép és tanulási útvonal 622
Bibliográfia és forráskulcsok 624

Impresszum

- **Szerző:** Gellai István
- **Szerkesztő:** Gellai István
- **Kiadó:** Tananyag Simple
- **Kiadás:** 1.0.0 – 2026
- **Licenc:** MIT
- **Errata:** review/reports/STATUS.md
- **Forráskód:** chapters/ (a kódpéldák a fejezetekben)

A teljes licencszöveg a tároló gyökerének `LICENSE` fájljában található (a könyvkönyvtárból: `../../LICENSE`).

I. RÉSZ – Indulás: eszközök, filozófia, első program

1 Olvasói útmutató

1.1 A könyv használata: C++-ból Rustba, mintaprojekt, olvasási utak

1.1.1 Motiváció

A könyv célközönsége nem az a fejlesztő, aki most ismerkedik a programozással, hanem az, aki már több ezer sornyi modern C++-t írt: ismeri a RAII-t, a mozgatási szemantikát, a `unique_ptr` és `shared_ptr` belső működését, a sablonokat, az STL-t, a lambdákat és a `std::thread` alapú párhuzamosságot. Éppen ez a tudás jelenti a csapdát is: Rustban a legtöbb C++-reflex értelmezhető, de gyakran éppen az ellenkezője az idiomatikus megoldás. A C++-ban az alapértelmezett másolás, az aliasolt, módosítható állapot, a `nullptr` mint üres érték, a kivételek és az implementációs öröklés mindennaposak; a Rust ezeket a forrásait a legtöbb hibának szándékosan eltávolítja. Ez az útmutató arra készít fel, hogy a C++-tudásodat ne a nyelv „legyőzésére”, hanem a Rust-modell megértésére használd. Az rkv mintaprojekt pedig ugyanazt a tanulási utat járja be, mint egy valódi C++-ról Rustra migráló szolgáltatás: CLI-ből könyvtárrá fejlődik, hibakezeléssel, perzisztens tárolóval, párhuzamos tesztekkel, hálózati szerverrel, végül C++-ból hívható FFI-réteggel.

1.1.2 Fogalmak és szabályok

Definíció. A könyvben a *továbbvitt eredmény* olyan típus, trait, modul, függvény vagy mintaprojekt-döntés, amelyet az alfejezet kidolgozásakor pontosan meghatározott néven és viselkedéssel hozunk létre az rkv workspace-ben vagy a labor-könyvtárban, és amelyre a későbbi alfejezetek a megadott néven hivatkoznak. Ha egy későbbi alfejezet mégis módosítja a korábbi döntést, akkor a módosítás és annak indoklása maga is tananyag; a refaktorálás nem hiba, hanem a fejlődés természetes része.

Szabály. A könyv teljes tanulási íve egyetlen végigvitt mintaprojektet épít: az **rkv** kulcs–érték tárolót. Minden labor ennek egy darabját készíti el vagy dolgozza át. A labor-könyvtár a `~/rust-labor/` könyvtár, az rkv Cargo-workspace a `~/rust-labor/rkv/` alatt található. Két dolgot külön kell tartani: a Cargo-workspace **tagjait** (Rust-crate-ek, ezeket a `cargo metadata` listázza) és a repository **többi könyvtárát** (nem Rust-crate-ek, saját build-útvonallal).

A Cargo-workspace tagjai (a `members` lista elemei):

- `rkv-core` – könyvtár crate, a tároló, a parancsok, a hibatípusok és a `Storage` trait;
- `rkv-cli` – bináris crate, a parancssori felület clap alapú argumentumelemzéssel;
- `rkv-server` – bináris crate, a tokio TCP-szerver és a szöveges protokoll;
- `rkv-ffi` – C ABI-t exportáló könyvtár, amelyet C-ből és C++-ból lehet hívni;
- `rkv-cxx` – a `cxx` crate-tel épített híd, amely C++-ból teszi elérhetővé a tárolót (mérföldkő: C++-együttműködés a cxx crate-tel, a workspace-be itt kerül be);
- `rkv-derive` – procedurális makró crate, a `CommandName` derive implementációval.

A repository további, **nem** Cargo-tag elemei:

- `cpp-client/` – C++ klienskód (`~/rust-labor/rkv/cpp-client/`). Saját build-útvonala van: a `g++` (vagy CMake) a Rust oldal által előállított `target/release/librkv_cxx.a` statikus könyvtárra és a generált `lib.rs.h` fejlécre épül, tehát Rust-könyvtári függősége van, de a `cargo metadata` nem listázza;
- `Cargo.lock` és a gyökér `Cargo.toml` – a workspace közös fájljai; a `rust-toolchain.toml` a `~/rust-labor/` labor-gyökérben van, ezért minden testvér-labor örökli.

A záróprojekt (Záróprojekt: az rkv teljes rendszerként) ugyanezt a hat tagot és a külön `cpp-client/` könyvtárat használja; a `cargo metadata --no-deps` csak a hat Rust-crate-et mutatja.

Szabály. A C++-ismeretek három csoportba sorolhatók:

1. **Közvetlenül átvihető.** A RAII erőforrás-kezelés, az értékszemantika fogalma, a tulajdonjog-átadás gondolata (`unique_ptr`), a generikus programozás (sablonok → trait-ek), a smart pointerek, a `const`-helyesség alapgondolata, a memóriamodell, az atomikus műveletek és a nulla költségű absztrakció fogalma. Ezek a Rustban is hasonlóan működnek, bár szintaktikailag és a kényszerítő szabályokban eltérhetnek. A mozgatás külön figyelmet kér: a C++ `std::move` csak egy xvalue-konverzió, az állapotot a tényleges mozgató konstruktor vagy értékadás határozza meg, és a forrásobjektum *érvényes, de nem meghatározott* állapotban marad (a `unique_ptr`-nél garantáltan `nullptr`, amelynek dereferálása UB). A Rust mozgatása ezzel szemben *destruktív*: a forrás statikusan érvénytelenné válik. Az automatikusan generált mozgató műveletek és a Rule of Zero C++-ban közelebb viszik a két modellt, de a forrás használhatósága ott is futásidejű kérdés marad.
2. **Újratanulandó, gyakran az ellenkezője.** Rustban a mozgatás az alapértelmezett, nem a másolás; egyidejűleg vagy sok olvasó, vagy egy író lehet, aliasolt és módosítható állapot egyszerre nem engedett; nincs `null` referencia (a hiányt az `Option<T>` típus fejezi ki; nyers mutató lehet null, de csak `unsafe` blokkban dereferálható), nincs kivétel, nincs implementációs öröklés, nincs függvénytúlterhelés; a fordítási hibák nem akadályok, hanem a nyelv tanításának eszközei.
3. **Új, a C++-ban nincs közvetlen megfelelője.** A lifetime-annotációk, a borrow checker, a `Send`/`Sync` automatikus trait-ek, az async/await a `Future` típussal, a `Pin`, a `no_std` Rust és a makrórendszer ebben a formában.

1.1.3 Az rkv-lánc: kötelező kódblokkok, választható gyakorlás és reprodukálhatóság

A könyv nem mellékel nyilvános rkv-repositoryt: a mintaprojekt a fejezetek *Kódblokk* jelölésű blokkjaiból áll össze, a verziókat a `rust-toolchain.toml` és a `Cargo.lock` rögzíti. Ezért ajánlott a saját repositoryt azonnal létrehozni (`git init`), és minden alfejezet ellenőrzési pontja után commitolni és címkézni (például `git tag r034`), a `Cargo.lock`-kal együtt. Az egyes alfejezetek *Ellenőrzési pont* szakasza adja meg, milyen állapotban kell lennie a munkafának.

Három szintet különböztetünk meg. (1) A **kötelező lánc**: a *Kódblokk* jelölésű laborblokkok és azok a feladatmegoldások, amelyekre a következő fejezet kifejezetten épít. Ilyen például a `SharedStore` (a Szálak, Send és Sync 3. feladata, amelynek megoldása a Mutex, RwLock, csatornák és a megosztott rkv alapja): ezt a feladatot a megoldás kódja alapján mindenképp be kell vezetni. (2) A **választható gyakorlás**: a többi „Implementáció az rkv-ban” feladat, például a `Session` typestate az `rkv-server`ben; ennek hiánya nem töri meg a láncot, és a záróprojekt szervere nem használja. (3) A **külön próba**: az önálló kisprojektek (`~/rust-labor/rNNN/`), amelyek nem részei az rkv-nek.

Ahol egy blokk csak a változás helyét mutatja (`...` kommenttel), a *művelet* mező mondja meg, hogy teljes csere, részleges csere vagy külön próba; ha nem teljes fájlcsere, a blokkban a `...` jelölt részeket a korábbi állapotból kell megtartani.

1.1.4 Mit vállal a könyv, és mit nem (kompetenciahatárok)

A cím „zero to hero” ígérete a tapasztalt C++-fejlesztő számára a következő, ellenőrizhető kompetenciákra szól: (1) a tulajdonjog-, kölcsönzés- és élettartam-szabályok biztos alkalmazása, (2) trait-alapú, generikus API tervezése és hibakezelése, (3) szálak és `async` (tokio) használata korrekt leállással, (4) tesztelés, benchmark és dokumentáció, (5) C ABI-s és `cxx`-es együttműködés. **Nem** vállalja: az `unsafe` kód formális bizonyítását (a Miri és a `# Safety` szerződések gyakorlata van, nem teljes verifikáció), a hardveres/beágyazott célplatformon való tényleges futtatást, és több build-útvonal (Windows, macOS, keresztfordítás) teljes körű támogatását; ezek a megfelelő fejezetekben kitekintésként szerepelnek. A záró ellenőrzőlista a Záróprojekt: az rkv teljes rendszerként alfejezetben az rkv-ra konkrét; a záróértékelés a saját munkafára futtatott `scripts/accept.sh`.

1.1.5 Olvasási utak és előfeltételek

A könyv sorrendben épül, de a tapasztalt olvasó rövidebb utat is választhat. Minden nagy blokkhoz megadjuk, mit kell már tudnod, és mit tudsz utána.

Blokk	Fejezetek	Már tudnod kell	Utána tudod
Indulás	Olvasói útmutató–Eszközök és a Rust filozófiája	C++ alapok, parancssor	a Rust eszközláncát, az rkv workspace vázát
Alapok	Szintaxis, típusok, kifejezések–Modulok, crate-ek és láthatóság	szintaxis-szintű C++	structok, enumok, mintaillesztés, modulok
Ownership, borrowing	Ownership és mozgatás–Smart pointerek és belső mutabilitás	RAII, mozgatási szemantika (C++)	tulajdonjog, kölcsönzés, élettartam, smart pointerek
Absztrakciók	Hibakezelés–Tesztelés és dokumentáció	az ownership blokk	`Result`, trait-ek, generikusok, iterátorok, tesztek
Párhuzamosság	Szálak és megosztott állapot–Aszinkron Rust	a trait-ek és az ownership	szálak, `Send`/`Sync`, async, tokio
Haladó	Unsafe Rust–Beágyazott Rust kitekintés	a fentiek, kiválasztható sorrendben	`unsafe`, FFI, makrók, API-tervezés, teljesítmény, beágyazott
Záróprojekt	Záróprojekt és összefoglalás	a választott utak	a teljes rendszer átvétele

Lineáris alapút (ajánlott): Olvasói útmutató → Záróprojekt és összefoglalás sorrendben, az rkv-t építő *Kódblokk*-os laborokat végigcsinálva; az rkv mintaprojekt ezt a sorrendet feltételezi. Több fejezet önálló kísérleti crate-et készít (`~/rust-labor/rNNN/`), ezek nem kerülnek be a

továbbvitt projektbe: a fejezet *Továbbvitt eredmény* szakasza mondja meg, mi kerül át az rkv-ba.

Gyorsított út tapasztalt C++-fejlesztőnek: Olvasói útmutató–Eszközök és a Rust filozófiája végigolvasása, Szintaxis, típusok, kifejezések–Modulok, crate-ek és láthatóság átfutása a „C++ és Rust összevetés” szakaszokra koncentrálva, Ownership és mozgatás–Smart pointerek és belső mutabilitás (ownership) *alapos* elsajátítása, majd Hibakezelés–Gyűjtemények és Szálak és megosztott állapot–Aszinkron Rust. A haladó blokkból válassz: az **FFI-út** Unsafe Rust → FFI: C és C++ együttműködés → Makrók (unsafe, FFI, makrók), a **teljesítmény-út** Haladó típusrendszeri minták és API-tervezés → Teljesítmény, a **beágyazott kitekintés** Beágyazott Rust kitekintés opcionális. A záróprojekt (Záróprojekt és összefoglalás) előtt az rkv-labor lépéseit (a továbbvitt eredményeket) ilyenkor is el kell végezned, különben a záróprojekt nem épül.

Diagnosztikus kérdések. Egyes feladatok szándékosan előreutalnak (ilyen az alábbi 4. feladat a `Send`/`Sync`/`Mutex` témájával). Ezek *diagnosztikus* kérdések: most csak a gondolatmenetet vázold, a teljes megoldás a Szálak és megosztott állapot elolvasása után (a Szálak, Send és Sync és a Mutex, RwLock, csatornák és a megosztott rkv alfejezet után) válik megoldhatóvá, és akkor érdemes visszatérned hozzá.

1.1.6 C++ és Rust összevetés

A két nyelv filozófiáját egy kódpárral szemléltetjük, és három dolgot külön nevezünk meg: a **másolat költségét**, a **nyelvi érvényességet** és a **futásidejű hibát**. C++-ban egy érték szerinti átadás másolatot készít. Ez érvényes, hibátlan program, csak költsége van (a kupacmemória duplikálása):

```cpp
#include <vector>

void take_copy(std::vector<int> v) { /* v a hívó vektorának másolata */ }

int main() {
    std::vector<int> data{1, 2, 3};
    take_copy(data);        // érvényes: data megmarad, a másolat a költség
    // data tovább használható; a két vektor külön kupacmemóriát foglal
}
```

Rustban ugyanez a függvényhívás alapértelmezés szerint *mozgat*: nincs másolat (nincs költség), de a `data` többé nem használható a hívás után, és ezt a fordító **fordítási időben** kényszeríti ki:

```rust
fn take_owned(v: Vec<i32>) { /* v tulajdonba került */ }

fn main() {
    let data = vec![1, 2, 3];
    take_owned(data);       // Rust: data mozgatva, a kupacadat nem másolódik
    // println!("{data:?}"); // fordítási hiba: use of moved value (E0382)
}
```

A különbség a tulajdonjog (ownership) szabályából fakad: Rustban minden értéknek pontosan egy tulajdonosa van, és a függvényhívás a tulajdonjogot átadja. A C++ példa tehát nem hibás: a másolat költséges lehet, de helyes. A **valódi** hibák (use-after-free, lógó referencia, a `std::move` utáni használat) más példák, ezeket az Miért Rust? Biztonság, teljesítmény, és ami a C++-ban hiányzik alfejezet tárgyalja, külön a hibátlan másolástól. Az `std::move`-ot C++-ban az különbözteti meg a Rust-mozgatástól, hogy csak egy átalakítás (`xvalue`): a forrás a mozgató művelet után érvényes marad (a `std::vector` esetén üres, a `unique_ptr` esetén `nullptr`), és a használatát semmi nem tiltja.

1.1.7 Kidolgozott példa: az rkv mérföldkövei és a labor-könyvtár

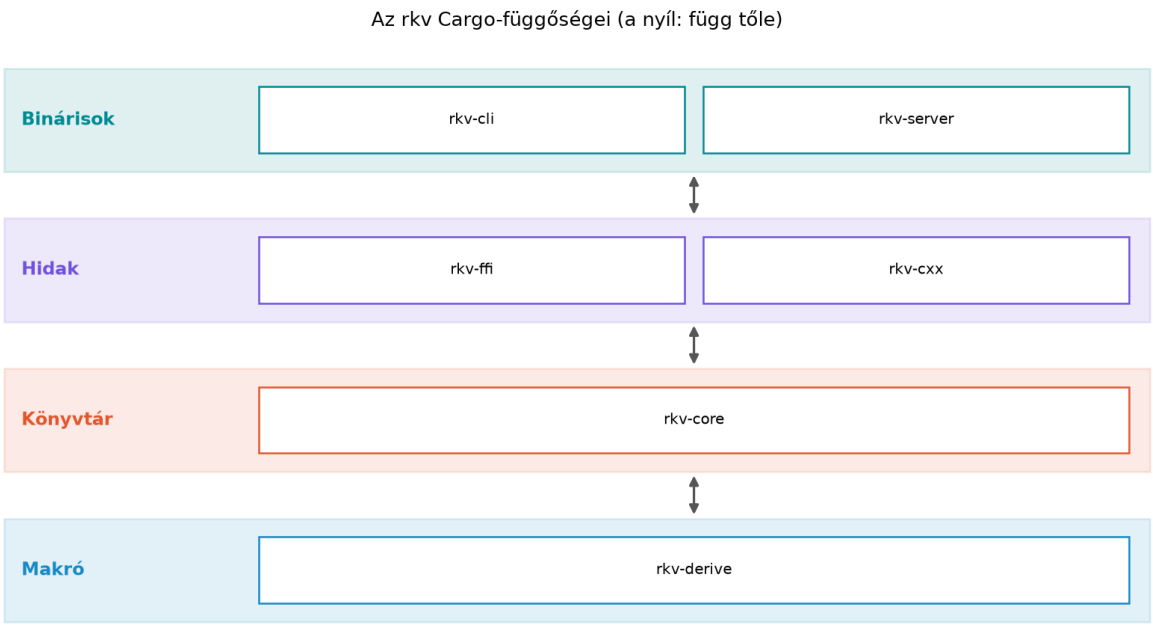
A könyv egyes részei az rkv projekt más-más mérföldkövét valósítják meg. A tanulási ív a következő:

- **Indulás (Olvasói útmutató–Eszközök és a Rust filozófiája):** az eszközlánc telepítése, a workspace létrehozása, az első CLI váz. A `~/rust-labor/` könyvtár és a Cargo-workspace is itt jön létre.
- **Alapok (Szintaxis, típusok, kifejezések–Modulok, crate-ek és láthatóság):** a Rust szintaxisa, típusrendszere, structok, enumok, mintaillesztés, modulok. Itt készül el az `rkv-core` első vázlata, a `Command` enum és a `Get`/`Set` parancsok.
- **Ownership és borrowing (Ownership és mozgatás–Smart pointerek és belső mutabilitás):** a Rust szíve. Itt dől el, hogyan kerül a `Store` típusba a kulcs–érték adat, és hogyan adjuk kölcsön anélkül, hogy a borrow checkiert „kijátszanánk”.
- **Hibakezelés és absztrakciók (Hibakezelés–Tesztelés és dokumentáció):** a `Result` és a `?` operátor, a saját `Error` enum, a `Storage` trait, a `MemStore` és `FileStore` implementációk, a tesztelés. Itt válik az rkv valódi, újrahasznosítható könyvtárrá.
- **Párhuzamosság és aszinkronitás (Szálak és megosztott állapot–Aszinkron Rust):** a `Storage` traitet szálbiztossá tesszük, az `Arc` és a `Mutex` segítségével megosztjuk a tárolót, majd a tokio szerverrel hálózati protokollt adunk hozzá.
- **Haladó témák (Unsafe Rust–Beágyazott Rust kitekintés):** unsafe Rust, C és C++ FFI, makrók, API-tervezés, teljesítményhangolás, beágyazott kitekintés. Az `rkv-ffi` a C ABI réteget, az `rkv-cxx` a `cxx`-hidat, az `rkv-derive` a makrókat, a `cpp-client/` a C++-oldali meghívást mutatja be.
- **Záróprojekt (Záróprojekt és összefoglalás):** az rkv teljes rendszerként, a részek integrálása és az összefoglalás.

A labor-könyvtár létrehozása egyszerű, de a teljes workspace beállítása a Eszközlánc: rustup, cargo, rustfmt, clippy, dokumentáció és Az rkv mintaprojekt indítása: Cargo-workspace és első CLI alfejezetben történik. Itt csak a várható szerkezetet mutatjuk be:

```
mkdir -p ~/rust-labor/rkv
cd ~/rust-labor/rkv
# Ezt követően a cargo new és a Cargo.toml szerkesztése az r003-ban.
```

A workspace rétegszerkezete:



1.1. ábra: Az rkv Cargo-függőségei (a nyíl: függ tőle) - A felső réteg a közvetlenül alatta lévőtől függ. rkv-core függ az rkv-derive-tól, nem fordítva.

A nyilak a **Cargo-függőség irányát** jelölik: a felső réteg a közvetlenül alatta lévőtől függ (`[dependencies]`). A `Cargo.toml` függőséglistája ebből egyértelműen felírható: `rkv-cli` → `rkv-core`; `rkv-server` → `rkv-core`; `rkv-ffi` → `rkv-core`; `rkv-cxx` → `rkv-core`; `rkv-core` → `rkv-derive` (path-függőség). A makró crate (`rkv-derive`) nem függ az `rkv-core`-tól: a derive által generált kód az `::rkv_core::CommandName` útvonalat nevezi meg, de ez a használó crate-ben oldódik fel, nem Cargo-függőség. A fizikai mappaszerkezet (a tagok egymás melletti könyvtárai) nem azonos ezzel a függőségi gráffal.

1.1.8 Hivatkozási térkép: az rNNN azonosítók

A szövegben az alfejezetekre két módon utalunk: **cím szerint** (kattintható hivatkozás, a cím mellett nincs szám) és a forrás **rNNN azonosítójával**, amely a fájlnevet, a labor-könyvtárat és a hivatkozás-célt adja. Az `rNNN` **nem** a nyomtatott fejezetszám: a nyomtatott (PDF) számozás `fejezet.alfejezet` alakú. Az alábbi térkép azonosítóból megadja a nyomtatott helyet, a címet és azt a könyvtárat, ahol a labor dolgozik. A `~/rust-labor/rkv/` az egyetlen, végigvitt rkv-workspace; az `~/rust-labor/rNNN/` önálló gyakorló-projekt, amely nem része a workspace-nek. A kereséshez használható a “Mintaillesztés” cím vagy az `r009` azonosító.

Azonosító	Nyomtatott hely	Alfejezet	Labor-könyvtár
`r000`	1.1	A könyv használata: C++-ból Rustba, mintaprojekt, olvasási utak	–

Azonosító	Nyomtatott hely	Alfejezet	Labor-könyvtár
`r001`	2.1	Miért Rust? Biztonság, teljesítmény, és ami a C++-ban hiányzik	–
`r002`	2.2	Eszközlánc: rustup, cargo, rustfmt, clippy, dokumentáció	`~/rust-labor/r002/`
`r003`	2.3	Az rkv mintaprojekt indítása: Cargo-workspace és első CLI	`~/rust-labor/rkv/`
`r004`	3.1	Változók, mutabilitás, skalár típusok és konverziók	–
`r005`	3.2	Kifejezések, utasítások és vezérlési szerkezetek	–
`r006`	3.3	Tuple-ök, tömbök és az első pillantás a slice-okra	–
`r007`	4.1	Structok és metódusok: impl blokkok, asszociált függvények	`~/rust-labor/rkv/`
`r008`	4.2	Az enum mint összeg-típus és az Option a null helyett	`~/rust-labor/rkv/`
`r009`	4.3	Mintaillesztés: match, if let, let else és a minták	`~/rust-labor/rkv/`
`r010`	5.1	Modulok és láthatóság: header nélküli világ	`~/rust-labor/rkv/`
`r011`	5.2	Függőségek, feature-ök, verziózás és workspace-ek	`~/rust-labor/rkv/`
`r012`	6.1	Ownership: minden értéknek egy tulajdonosa van	`~/rust-labor/r012/`
`r013`	6.2	Copy, Clone és Drop: másolás és RAII Rustban	`~/rust-labor/rkv/`
`r014`	7.1	Referenciák és a kölcsönzési szabály: sok olvasó XOR egy író	`~/rust-labor/r014/`
`r015`	7.2	A borrow checker működése: non-lexical lifetimes és tipikus javítások	`~/rust-labor/rkv/`

Azonosító	Nyomtatott hely	Alfejezet	Labor-könyvtár
r016	8.1	Élettartam-annotációk és az elision szabályai	–
r017	8.2	Referenciát tartalmazó struktúrák, 'static és tervezési döntések	~/rust-labor/rkv/
r018	9.1	Slice-ok, String és &str, UTF-8	~/rust-labor/r018-slices/
r019	10.1	Box, Rc, Arc és Weak	~/rust-labor/r019/
r020	10.2	Belső mutabilitás: Cell, RefCell és a futásidejű kölcsönzés	~/rust-labor/r020/
r021	11.1	Result, a ? operátor és a panic helye	~/rust-labor/rkv/
r022	11.2	Saját hibatípusok: enum-hibák, thiserror, anyhow	~/rust-labor/rkv/
r023	11.3	I/O és szerializáció: fájlok, pufferelés, serde	~/rust-labor/rkv/
r024	12.1	Trait-ek és generikusok: sablonok és concepts helyett	~/rust-labor/rkv/
r025	12.2	Asszociált típusok, koherencia, operátorok és derive	~/rust-labor/r025/
r026	12.3	A standard trait-ek: konverziók, formázás, Default, Deref, Borrow	~/rust-labor/rkv/
r027	12.4	Trait objektumok és dinamikus diszpécs	~/rust-labor/rkv/
r028	13.1	Closure-ök: Fn, FnMut, FnOnce és a capture	~/rust-labor/rkv/
r029	13.2	Iterátorok: lusta adapterek és saját iterátor	~/rust-labor/rkv/
r030	14.1	Vec, HashMap, BTreeMap és társaik; az Entry API	~/rust-labor/rkv/
r031	15.1	Unit-, integrációs és dokumentációs tesztek	~/rust-labor/rkv/
r032	15.2	Property-alapú tesztelés és benchmark	–

Azonosító	Nyomtatott hely	Alfejezet	Labor-könyvtár
r033	16.1	Szálak, Send és Sync: data race fordítási időben kizárva	~/rust-labor/r033/
r034	16.2	Mutex, RwLock, csatornák és a megosztott rkv	~/rust-labor/rkv/
r035	16.3	Atomikus műveletek és memóriarendezés	~/rust-labor/rkv/
r036	16.4	Adatpárhuzamosság rayonnal	~/rust-labor/rkv/
r037	17.1	Future-ök és async/await: mi történik a motorháztető alatt	–
r038	17.2	Pin, Unpin, async trait-ek és a Send-határok	–
r039	17.3	Tokio: taskok, I/O, csatornák, select és megszakítás	~/rust-labor/r039/
r040	17.4	Az rkv hálózati szerver: TCP, protokoll, kecses leállás	~/rust-labor/rkv/
r041	17.5	Naplózás és megfigyelhetőség: tracing	~/rust-labor/r041-tracing/, ~/rust-labor/rkv/
r042	18.1	Unsafe Rust: szuperképességek és felelősség	~/rust-labor/r042-unsafe-superpowers/
r043	18.2	Biztonságos absztrakció unsafe alapon	–
r044	19.1	C ABI: extern “C”, repr(C), bindgen és cbindgen	~/rust-labor/rkv/
r045	19.2	C++-együttműködés a cxx crate-tel és a fokozatos migráció	~/rust-labor/rkv/
r046	20.1	Deklaratív és procedurális makrók	~/rust-labor/rkv/
r047	21.1	Newtype, PhantomData, typestate és const generikusok	–
r048	21.2	API-tervezés Rustban: irányelvek és ergonómia	~/rust-labor/rkv/

Azonosító	Nyomtatott hely	Alfejezet	Labor-könyvtár
`r049`	22.1	Teljesítmény: mérés, profilozás, allokáció, release-beállítások	`~/rust-labor/rkv/`
`r050`	23.1	no_std, embedded-hal és Embassy: Rust mikrovezérlőn	`~/rust-labor/r050/`
`r051`	24.1	Záróprojekt: az rkv teljes rendszerként	`~/rust-labor/rkv/`
`r052`	24.2	Összefoglalás: C++→Rust fordítási kézikönyv és továbbtanulás	–

Az angol–magyar terminuspárok rövid fogalomtára az Összefoglalás: C++→Rust fordítási kézikönyv és továbbtanulás alfejezet végén található.

1.1.9 Tipikus hibák és átállási csapdák

1. **Az alapértelmezett másolás feltételezése.** C++-ban egy `std::vector` függvényargumentumként való átadása általában másolatot készít; Rustban a `Vec<T>` mozog. Ha a C++-os reflexből kiindulva `Clone`-t hívsz mindenhol, akkor gyakran felesleges allokációt és a tulajdonjogi modell megkerülését éred el. A helyes megoldás a borrow, az `&`-referencia, amikor csak olvasásra van szükség.

2. **Az aliasolt, módosítható állapot megszokása.** C++-ban teljesen természetes, hogy két mutató ugyanarra a módosítható objektumra mutat, és a fejlesztő felelőssége a szinkronizáció. Rustban a borrow checker ezt fordítási időben tiltja, hacsak nem használunk `Mutex`-et, `RefCell`-et vagy atomikus típusokat. A `Rc<RefCell<T>>` gyakori menekülőút, de a túlhasználata a C++-os „mindent megosztok” gondolkodás jele; Rustban gyakran jobb a tulajdonjogot átrendezni.

3. **A `null` mint hiányjel.** A C++-ban a `nullptr` gyakran jelenti azt, hogy „nincs érték”, és a dereferálása UB. Rustban a **referenciák** (`&T`, `&mut T`) soha nem null-ok; a hiányt az `Option<T>` típus fejezi ki. Ez nem azt jelenti, hogy nincs null mutató: a nyers mutató (`*const T`) lehet null, csak `unsafe` blokkban dereferálható. Az `Option` pedig nem akadályoz meg *minden* futási hibát, csak a hiány kezelését teszi láthatóvá, és a kezelés módja a programozó döntése:

```rust
fn first(v: &[i32]) -> Option<&i32> { v.first() }

fn main() {
    let empty: Vec<i32> = Vec::new();
    // 1. Teljes match: mindkét eset kifejezett.
    match first(&empty) {
        Some(x) => println!("első: {x}"),
        None => println!("üres"),
    }
    // 2. if let: a None ág csendben kimarad (nincs hiba, de nincs kezelés sem).
    if let Some(x) = first(&empty) {
        println!("első: {x}");
    }
```

```
    // 3. unwrap: a None esetén pánik, vagyis futásidejű hiba.
    // let x = first(&empty).unwrap(); // pánik
}
```

A `match` kimerítő, az `if let` a `None`-t eldobhatja, az `unwrap()` pedig pánikká alakítja: a három következménye különbözik.

4. **A kivételek helyettesítése a `Result` típussal.** C++-ban a hibákat kivételekkel dobjuk, és a hívó dönti el, hol kapja el. Rustban a hiba a `Result<T, E>` típus, és a `?` operátor expliciten továbbítja. A `unwrap()` / `expect()` hívások a fejlesztői kényelem eszközei, de könyvtári kódban kerülendők, mert panikot okoznak.

1.1.10 Feladatok

A feladatok felépítése. Minden feladatnál megadjuk a **nehézséget** (könnyű, közepes, nehéz), az **ellenőrizhető eredményt** – amiből a teljes megoldás megnyitása nélkül eldöntheted, hogy helyes-e a munkád –, valamint két fokozatos **súgót**: az első a gondolkodás irányát, a második a döntő lépést mondja ki. A megoldások a fejezet végén, külön **Megoldások** alfejezetben vannak, hogy előbb önállóan dolgozhass. A kötelező lánc kódja nem a megoldások között, hanem a *Kódblokk* jelölésű laborblokkokban található; a megoldások szemléltető példái nem lépnek a lánc helyébe.

1. **Lefordul-e, és miért?** (Fogalmi kérdés) Döntsd el, hogy a következő C++-kód Rust-megfelelője lefordulna-e, és indokold a válaszod anélkül, hogy lefuttatnád:

```
#include <memory>
void take(std::unique_ptr<int> p) {}
int main() {
    auto p = std::make_unique<int>(42);
    take(std::move(p));
    *p = 7; // C++: p a move után nullptr, a dereferálás UB
}
```

Mi lenne a Rust-megfelelő? Milyen hibát adna a fordító?

Nehézség: könnyű. *Ellenőrizhető eredmény:* meg tudod nevezni a mozgatás utáni használat hibáját (E0382), és megmondod, hogy a C++-ban ugyanez futásidejű UB, nem fordítási hiba. *1. súgó:* kövesd, kinél van a `Box` tulajdonjoga a `take` hívás után. *2. súgó:* a Rust mozgatása statikusan érvényteleníti a forrásváltozót.

2. **C++-kód átírása idiomatikus Rustra.** A következő C++ osztály egy egyszerű számlálót valósít meg. Írd át Rustra úgy, hogy a `value` soha ne lehessen `null`, és a mutáció explicit legyen:

```
class Counter {
    int* value;
public:
    Counter() : value(new int(0)) {}
    ~Counter() { delete value; }
    void increment() { ++(*value); }
    int get() const { return *value; }
};
```

Nehézség: közepes. *Ellenőrizhető eredmény:* a `Counter` lefordul, `increment()` után `get()` 1-et ad, és a kódban nincs `Box`, `unsafe`, `null` vagy kézi `delete`. *1. súgó:* a pointer itt csak a dinamikus allokációt demonstrálja; az érték lehet egyszerű mező. *2. súgó:* a mutációhoz `&mut self` kell, a kezdőértéket a `new()` állítja be.

3. **Implementációs feladat az rkv-ban.** Hozd létre a labor-könyvtárat a `~/rust-labor` alatt, és hozd létre az `rkv` mappa vázát. Írd le, milyen mappákat és fájlokat hoznál létre a workspace tagjaihoz. (A tényleges `cargo new` parancsok az rkv mintaprojekt indítása: Cargo-workspace és első CLI alfejezetben kerülnek sorra, itt a tervezés a feladat.)

   Nehézség: könnyű. *Ellenőrizhető eredmény:* felsorolod a hat Cargo-taget (`rkv-core`, `rkv-cli`, `rkv-server`, `rkv-ffi`, `rkv-cxx`, `rkv-derive`), és külön kimondod, hogy a `cpp-client/` nem tag; a későbbi `cargo metadata` listája ezt igazolja. *1. súgó:* a tagok listája a fejezet elején, a Fogalmak és szabályok szakaszban van. *2. súgó:* a C++-kliens saját build-útvonalat használ, nem Cargo-crate.

4. **Hibajavítás / tervezési döntés (diagnosztikus, később visszatérő kérdés).** Tegyük fel, hogy a `Storage` trait kezdeti definíciója így néz ki:

   ```
   trait Storage {
       fn get(&self, key: &str) -> Option<Vec<u8>>;
       fn set(&mut self, key: String, value: Vec<u8>);
       fn delete(&mut self, key: &str);
   }
   ```

   Milyen probléma merülhet fel, ha a `get` `&self`-et, a `set` viszont `&mut self`-et használ? Hogyan lehetne a trait-et szálbiztosan megosztani a Mutex, RwLock, csatornák és a megosztott rkv alfejezetben? Fogalmazd meg a tervezési döntést, és indokold. (Előretekintő kérdés: a `Send`, a `Sync` és a `Mutex` részleteit a Szálak, Send és Sync: data race fordítási időben kizárva és a Mutex, RwLock, csatornák és a megosztott rkv alfejezet vezeti be; most elég a gondolatmenet vázlata, a megoldást ezek után érdemes újraolvasni.)

   Nehézség: nehéz. *Ellenőrizhető eredmény:* kimondod, hogy (1) az `Arc<T>` önmagában csak megosztott `&T`-t ad, a kizárólagos `&mut T`-t futásidőben a `Mutex` guardja biztosítja, és (2) a `dyn Storage` trait-objektumot `+ Send` korláttal kell ellátni, különben nem vihető át szálra; az `Arc<Mutex<Box<dyn Storage>>>` példa E0277-tel nem fordul. *1. súgó:* válaszd szét a kizárólagos hozzáférés és a szálak közti átadhatóság kérdését. *2. súgó:* a `Mutex<T>` akkor `Send`/`Sync`, ha `T: Send`, a trait-objektum pedig alapból semmit nem ígér.

1.1.11 Megoldások

Az alábbi megoldások a feladatokhoz tartoznak, és nem részei a kötelező láncnak: a 4. feladat példája a `Storage` korai, szemléltető változatát használja, a könyv kanonikus `Storage` és `SharedStore` típusa pedig a későbbi *Kódblokk*-okban épül fel.

1. A C++-kód a `std::move` után a `p`-t használja: a `unique_ptr` mozgatás után garantáltan `nullptr`, a dereferálása pedig nem definiált viselkedés (UB). Rustban a mozgatás (move) fordítási időben érvényteleníti a változó használatát:

   ```
   fn take(p: Box<i32>) {
       // p tulajdonba került
   }

   fn main() {
       let p = Box::new(42);
       take(p);
       // *p = 7; // hibás: value moved here
   }
   ```

A hiba a Rustban a mozgatás utáni használat (error E0382, `borrow of moved value`). A C++-ban a `p` objektum a mozgatás után is létezik, érvényes, de `nullptr`-t tartalmaz, és a dereferálása UB; a Rustban a mozgatás statikusan garantálja, hogy az eredeti változó nem használható (a hiba fordítási idejű, nem UB).

2. A `Counter` idiomatikus Rust-megfelelője nem használ `Box`-ot, mert a `value` egyszerűen `i32` lehet, nem szükséges heap allokáció:

```
struct Counter {
    value: i32,
}

impl Counter {
    fn new() -> Self {
        Self { value: 0 }
    }

    fn increment(&mut self) {
        self.value += 1;
    }

    fn get(&self) -> i32 {
        self.value
    }
}
```

A C++-ban a `value` pointer azért volt, mert a dinamikus allokáció a RAII-t demonstrálta; Rustban a `value` egyszerűen a struct része, a mutációt az `&mut self` metódus biztosítja, és nincs `null` vagy memóriaszivárgás.

3. A labor-könyvtár létrehozása:

```
mkdir -p ~/rust-labor/rkv
cd ~/rust-labor/rkv
```

A tervezett mappák és fájlok:

- `rkv-core/` – `Cargo.toml`, `src/lib.rs`, `src/command.rs`, `src/error.rs`, `src/storage.rs`, `src/store.rs`;
- `rkv-cli/` – `Cargo.toml`, `src/main.rs`;
- `rkv-server/` – `Cargo.toml`, `src/main.rs`;
- `rkv-ffi/` – `Cargo.toml`, `src/lib.rs`;
- `rkv-cxx/` – `Cargo.toml`, `build.rs`, `src/lib.rs` (a C++-együttműködés a cxx crate-tel és a fokozatos migráció alfejezetben kerül a workspace-be);
- `rkv-derive/` – `Cargo.toml`, `src/lib.rs`;
- `cpp-client/` – nem Cargo-tag: `main.cpp` és a build-parancsok (g++ vagy CMake).

Ezek a fájlnevek a későbbi alfejezetekben is megjelennek, a továbbvitt eredmények hivatkozási alapjai.

4. A `get` `&self`-et, a `set` és a `delete` `&mut self`-et használ. Ez önmagában nem hiba, és nem is teszi a típust szálbiztossá vagy szálbiztonság nélkülivé: a `&mut self` csak azt mondja, hogy a hívás idejére *kizárólagos* hozzáférés kell. Két különálló kérdést kell szétválasztani:

- **Kizárólagos hozzáférés.** Egy `Arc<T>` csak megosztott `&T` hozzáférést ad, akkor is, ha `T: Sync`; rajta keresztül `&mut self` metódus nem hívható. A kizárólagosságot futásidőben egy zár adja: a `Mutex<T>::lock` egy guardot ad vissza, amelyen át `&mut T` érhető el.

- **Szálak közti átadhatóság.** A `Mutex<T>` akkor `Send` és `Sync`, ha `T: Send`. Egy sima `dyn Storage` trait-objektum erről semmit nem ígér (a mögötte lévő típus lehet például `Rc`-t tartalmazó, nem `Send` típus), ezért az `Arc<Mutex<Box<dyn Storage>>>` nem vihető át másik szálra. A trait-objektumnak a `+ Send` korláttal kell ezt megígérnie.

A tervezési döntés: a `Storage` trait maradjon egyszálú (`&mut self` a módosító műveleteken), a megosztást pedig `Arc<Mutex<Box<dyn Storage + Send>>>` adja. Teljes példa két szállal:

```
use std::collections::HashMap;
use std::sync::{Arc, Mutex};
use std::thread;

trait Storage {
    fn get(&self, key: &str) -> Option<Vec<u8>>;
    fn set(&mut self, key: String, value: Vec<u8>);
    fn delete(&mut self, key: &str);
}

#[derive(Default)]
struct MemStore {
    data: HashMap<String, Vec<u8>>,
}

impl Storage for MemStore {
    fn get(&self, key: &str) -> Option<Vec<u8>> {
        self.data.get(key).cloned()
    }
    fn set(&mut self, key: String, value: Vec<u8>) {
        self.data.insert(key, value);
    }
    fn delete(&mut self, key: &str) {
        self.data.remove(key);
    }
}

fn main() {
    let store: Arc<Mutex<Box<dyn Storage + Send>>> =
        Arc::new(Mutex::new(Box::new(MemStore::default())));

    let handles: Vec<_> = (0..2u8)
        .map(|i| {
            let store = Arc::clone(&store);
            thread::spawn(move || {
                // A lock kizárólagos hozzáférést ad: &mut self hívható.
                store.lock().unwrap().set(format!("k{i}"), vec![i]);
            })
        })
        .collect();
    for h in handles {
        h.join().unwrap();
    }

    let guard = store.lock().unwrap();
    assert_eq!(guard.get("k0"), Some(vec![0]));
    assert_eq!(guard.get("k1"), Some(vec![1]));
}
```

Ha a típusból elhagyjuk a `+ Send`-et (`Arc<Mutex<Box<dyn Storage>>>`), a `thread::spawn` hívás nem fordul le: az E0277-es hiba szerint a `dyn Storage` nem küldhető biztonságosan szálak között. A `Mutex` tehát nem „ad” `Sync`-et tetszőleges tartalomhoz; csak `Send` tartalomból készít `Sync` tárolót. A részleteket a Mutex, RwLock, csatornák és a megosztott rkv alfejezet tárgyalja.

1.1.12 Rövid összefoglaló

A könyv modern C++-tudású fejlesztőknek tanítja a Rustot, a C++-reflexek tudatos újraértelmezésével. A RAII, az értékszemantika és a generikus programozás átvihető; a másolás, az aliasolt állapot, a `null`, a kivételek és az öröklés viszont újratanulandó. A tananyag egy végigvitt mintaprojektet, az **rkv** kulcs–érték tárolót építi a `~/rust-labor/rkv/` Cargo-workspace-ben. A Cargo-tagok: `rkv-core`, `rkv-cli`, `rkv-server`, `rkv-ffi`, `rkv-cxx` és `rkv-derive`; a C++-kliens (`cpp-client/`) külön könyvtár, nem Cargo-tag. A mérföldkövek a könyv ívét követik: CLI-váz, core-könyvtár, hibakezelés, `Storage` trait, párhuzamosság, tokio-szerver, FFI. A **továbbvitt eredmények** elve biztosítja, hogy minden alfejezet a korábbiak pontos típusaira, neveire és döntéseire épít. A labor-könyvtár és a workspace tényleges létrehozása a Eszközlánc: rustup, cargo, rustfmt, clippy, dokumentáció és Az rkv mintaprojekt indítása: Cargo-workspace és első CLI alfejezetben történik.

1.1.13 Források és ellenőrzési hiányok

- The Rust Programming Language (a „Rust könyv”), 4. fejezet „Understanding Ownership”: https://doc.rust-lang.org/1.99.1/book/ch04-00-understanding-ownership.html
- A `Send` és a `Sync` trait, valamint a `std::sync::Mutex` dokumentációja: https://doc.rust-lang.org/1.99.1/std/marker/trait.Send.html, https://doc.rust-lang.org/1.99.1/std/marker/trait.Sync.html, https://doc.rust-lang.org/1.99.1/std/sync/struct.Mutex.html
- A fejezetben előforduló hibakódok közvetlen oldalai: https://doc.rust-lang.org/1.99.1/error_codes/E0382.html (mozgatás utáni használat) és https://doc.rust-lang.org/1.99.1/error_codes/E0277.html (hiányzó `Send` korlát); a teljes jegyzék: https://doc.rust-lang.org/1.99.1/error_codes/
- Rust Edition Guide, Rust 2024: https://doc.rust-lang.org/1.99.1/edition-guide/rust-2024/index.html

A hivatkozások a könyv rögzített toolchainjének (Rust 1.99.1, 2024-es edition) verziózott dokumentációjára mutatnak, nem a mozgó `stable` aliasra. Külső crate dokumentációját a crate pontos verziójával kell megnyitni a docs.rs-en (például `https://docs.rs/tokio/1.47.1/tokio/`), és a könyv által ténylegesen futtatott parancsoknál a `Cargo.lock`-ban rögzített verziót kell megőrizni. A könyv rögzített toolchainje és a crate-verziók rögzítése a Eszközlánc: rustup, cargo, rustfmt, clippy, dokumentáció alfejezetben található; a hibakódok és a hibaüzenetek erre a toolchainre vonatkoznak. Az alfejezet kódjai rövid szemléltetések; a teljes, lefordítható változatok a hivatkozott alfejezetekben vannak.

2 Eszközök és a Rust filozófiája

2.1 Miért Rust? Biztonság, teljesítmény, és ami a C++-ban hiányzik

Egy évtizedes C++-fejlesztőként pontosan ismeri azokat a hibákat, amelyek a leghosszabb éjszakákat okozzák: a lógó referenciát, amely egy lefutott függvény lokális változójára mutat; az iterátort, amelyet egy `push_back` hívás érvénytelenít; a két szálat, amelyek egyszerre ír egy megosztott egész változót. Ezek a hibák C++-ban többnyire lefordulnak, és csak futás közben – gyakran hetekkel később, éles környezetben – jelentkeznek. A Rust legfontosabb ígérete, hogy az ilyen hibaosztályokat a biztonságos részen már fordítási időben kizárja, méghozzá úgy, hogy a futási teljesítményt nem áldozza fel.

2.1.1 A C++-hibák, amelyeket a Rust eltüntet

A C++ a teljesítményért cserébe a programozóra bízza a memóriabiztonságot. A következő hibaosztályok mind nem definiált viselkedéshez (undefined behavior, UB) vagy logikai hibához vezethetnek, és a C++ fordító általában nem állítja meg őket:

1. **Lógó referencia** (dangling reference): egy változóra mutató referencia túléli magát a változót.
2. **use-after-free**: egy felszabadított memóriaterületet olvasunk vagy írunk.
3. **double-free**: ugyanazt a memóriát kétszer szabadítjuk fel.
4. **Iterator-invalidálás**: egy gyűjtemény módosítása után a korábban kapott iterátort vagy referenciát használjuk.
5. **Data race**: több szál szinkronizáció nélkül, egyszerre olvas és ír egy közös, nem atomi változót.
6. **Null pointer dereference**: `nullptr`-t derreferálunk.
7. **Buffer overflow**: egy tömb vagy puffer határain túl írunk.

A Rust ezeket a biztonságos kódban nem egyetlen eszközzel zárja ki: egy részüket a fordító statikusan, egy részüket a standard könyvtár futásidejű ellenőrzéssel, egy részüket pedig a típusrendszer azzal, hogy a hibás állapotot ki sem lehet fejezni.

Hibaosztály	Mechanizmus a safe Rustban	Jelleg
lógó referencia	élettartam-ellenőrzés (borrow checker)	statikus
use-after-free, double-free	ownership és mozgatás: egy tulajdonos, a mozgatott változó nem használható	statikus
iterátor-invalidálás	kölcsönzési szabály: amíg él egy `&` (például iterátor), nincs `&mut`	statikus
data race	`Send`/`Sync` trait-ek, illetve `&mut` kizárólagossága	statikus

Hibaosztály	Mechanizmus a safe Rustban	Jelleg
null pointer dereference	safe kódban a referencia nem lehet null; a hiányt az `Option<T>` fejezi ki, és a kezelés explicit (a nyers mutató lehet null, de csak `unsafe`-ban dereferálható)	statikus (típusrendszer)
buffer overflow	az indexelés és a slice-képzés határellenőrzést végez, túlindexeléskor *pánik*	futásidejű
`RefCell` kettős kölcsönzése	a `borrow`/`borrow_mut` számlálót ellenőriz, ütközéskor pánik	futásidejű
egész túlcsordulás	debug buildben pánik, release buildben körbefordulás (nem UB)	futásidejű / definiált

A határellenőrzés tehát **nem** a borrowing része: a kölcsönzési szabály a referenciák egyidejűségét korlátozza, azt nem, hogy egy index a tartományon belül van-e. A statikus mechanizmusok ára nulla futásidőben; a futásidejű ellenőrzéseké kicsi, de nem nulla (lásd lejjebb a határellenőrzést). A hibaosztály-lista nem csupán elmélet: a későbbi alfejezetekben mindegyikre konkrét nyelvi mechanizmust építünk.

2.1.2 A Rust válasza: ownership, borrowing, Send/Sync – előzetes

A Rust három alappillére adja a biztonságot:

1. **Tulajdonjog (ownership)**: minden értéknek pontosan egy tulajdonosa van; amikor a tulajdonos kikerül a hatókörből, az érték automatikusan eldobódik (RAII). Ez nem kivétel nélküli: `mem::forget`, `Box::leak`, `Rc`/`Arc`-referenciaciklus és `abort` esetén az eldobás elmarad, `Rc`/`Arc`-nál pedig a fogantyú eldobása csak az utolsó erős hivatkozás megszűnésekor dobja el a mögöttes értéket. Ez szünteti meg a use-after-free és a double-free lehetőségét.
2. **Kölcsönzés (borrowing)**: a referenciák vagy olvashatók (megosztott, `&T`), vagy írhatók (kizárólagos, `&mut T`), de egyszerre nem lehet több író vagy egy író és olvasó. Ez zárja ki az iterátor-invalidálást és a lógó referenciát; a buffer overflow ellen nem ez, hanem a futásidejű határellenőrzés véd.
3. **`Send` és `Sync`**: az előbbi azt jelöli, hogy egy érték átvihető egy másik szálra, az utóbbi azt, hogy egy referencia megosztható szálak között. A data race így fordítási időben kiszűrhető, mert a nem szálbiztos típusok nem léphetnek át szálhatáron.

Ezeket a fogalmakat itt még csak előzetesen vezetjük be; a részletes szabályokat a Ownership: minden értéknek egy tulajdonosa van, a Referenciák és a kölcsönzési szabály: sok olvasó XOR egy író és a Szálak, Send és Sync: data race fordítási időben kizárva alfejezetek fejtik ki.

Definíció. A *biztonságos Rust* (safe Rust) a nyelv `unsafe` nélküli része. Ebben a fordító statikus elemzése és a standard könyvtár futásidejű ellenőrzései együtt garantálják a memóriabiztonságot és az adatverseny-mentességet, *feltéve*, hogy a felhasznált `unsafe` kód (a standard könyvtárban, a függőségekben és az FFI-hívások mögött) helyes.

Definíció. A *nem definiált viselkedés* (undefined behavior, UB) olyan nyelvi szabálysértés, amelynek a szabvány nem ír elő semmilyen viselkedést; a program bármit tehet, akár „működik”, akár lefagy, akár biztonsági rést nyit.

Definíció (soundness). Egy `unsafe` kódot tartalmazó függvény, típus vagy könyvtár akkor *sound* (helyes), ha semmilyen safe kódból érkező hívássorozat nem tud vele nem definiált viselkedést kiváltani.

Szabály. Ha egy program saját kódja safe, és minden felhasznált könyvtár sound (az FFI-n hívott külső kód pedig betartja a szerződését), akkor a programban nem fordulhat elő nem definiált viselkedés; a fenti hibaosztályok (use-after-free, double-free, data race stb.) nem léphetnek fel. Ez feltételes garancia: egy unsound könyvtár safe hívása is okozhat UB-t.

Szabály. A Rust a veszélyes műveleteket az `unsafe` blokkokra korlátozza. Az `unsafe` kódban a fejlesztő felel a memóriabiztonsági invariánsok betartásáért, a safe kóddal való interfésznek pedig biztonságosnak kell lennie.

2.1.3 C++ és Rust összevetés: lógó referencia és iterator-invalidálás

Nézzük a lógó referencia klasszikus C++ példáját:

```cpp
#include <string>
#include <iostream>

std::string& dangling() {
    std::string s = "hello";
    return s; // UB: referencia a lefutott függvény lokális változójára
}

int main() {
    std::string& r = dangling();
    std::cout << r << '\n'; // olvashatatlan memória
}
```

Ez a kód C++-ban lefordul; a nyelvi szerződés nem tiltja, és a diagnózis nem kötelező. A GCC `-Wall`-lal és a Clang `-Wall`-lal tipikusan figyelmeztet a lokális változóra visszaadott referenciára (`-Wreturn-local-addr`, illetve `-Wreturn-stack-address`), de ez opcionális figyelmeztetés, kikapcsolható, és a program viselkedése ettől még UB marad. A Rust-példát egyetlen tanított hibára szűkítjük. (A paraméter nélküli `fn dangling() -> &String` már a függvény fejlécénél az `E0106` hibával, a hiányzó élettartam-megnevezéssel állna meg; azt az Élettartam-annotációk és az elision szabályai alfejezet tárgyalja. Itt azt akarjuk látni, hogy a *lokális változóra mutató referencia visszaadása* hibás, ezért a függvénynek van egy bemeneti referenciája, amelyből az élettartam kikövetkeztethető.) A Rust megfelelője nem fordul le:

```rust
fn dangling(_hint: &str) -> &String {
    let s = String::from("hello");
    &s // visszaadunk egy referenciát a lokális változóra
}

fn main() {
    let r = dangling("x");
    println!("{r}");
}
```

Ez a változat szándékosan nem fordul le. A fordító 1.99.1-es kimenete (a fenti forrásból, `src/main.rs`):

```
error[E0515]: cannot return reference to local variable `s`
 --> src/main.rs:3:5
  |
3 |     &s // visszaadunk egy referenciát a lokális változóra
  |     ^^ returns a reference to data owned by the current function
```

A különbség lényege: a Rust borrow checker ismeri az élettartamokat, és tudja, hogy az `s` változó a függvény végén felszabadul. A visszaadott referencia így érvénytelen lenne, ezért a kód már fordítási időben hibát ad, nem pedig futás közben.

Az iterator-invalidálás is hasonló. C++:

```cpp
#include <vector>
#include <iostream>

int main() {
    std::vector<int> v = {1, 2, 3};
    int& first = v[0];
    v.push_back(4); // ha a kapacitás elfogy, reallokál: a first érvénytelen
    std::cout << first << '\n'; // UB reallokáció esetén (a kapacitáson belüli push_back után a
    ↪  referencia érvényes maradhatna)
}
```

Rust:

```rust
fn main() {
    let mut v = vec![1, 2, 3];
    let first = &v[0];
    v.push(4); // megpróbálunk mutable kölcsönzést, miközben `first` él
    println!("{first}");
}
```

Ez a változat szándékosan nem fordul le:

```
error[E0502]: cannot borrow `v` as mutable because it is also borrowed as immutable
 --> src/main.rs:4:5
  |
3 |     let first = &v[0];
  |                  - immutable borrow occurs here
4 |     v.push(4); // megpróbálunk mutable kölcsönzést, miközben `first` él
  |     ^^^^^^^^^ mutable borrow occurs here
5 |     println!("{first}");
  |               ----- immutable borrow later used here
```

A kölcsönzési szabály (sok olvasó XOR egy író) tehát nemcsak a szálak közötti, hanem az egyszálú, moduláris kódban is kizárja az érvénytelen referenciákat.

2.1.4 Kidolgozott példa: use-after-move C++-ban és Rustban

A C++ 11 óta a mozgatási szemantika lehetővé teszi, hogy egy objektum erőforrásait átadjuk egy másik objektumnak. A moved-from objektum azonban C++-ban a standard könyvtár típusaira *érvényes, de nem specifikált állapotú* marad (ez a garancia nem terjed ki automatikusan minden felhasználói mozgató konstruktorra; a „nem specifikált” és az „implementációfüggő” a C++ terminológiában két külön kategória). Gyakori csapda, hogy a fejlesztő mégis használja, feltételezve az eredeti értéket:

```cpp
#include <string>
#include <iostream>

int main() {
    std::string s1 = "hello";
    std::string s2 = std::move(s1);
    std::cout << s1 << '\n'; // nem UB, de s1 tartalma nem specifikált (lehet üres, lehet
    ↪  "hello")
}
```

A Rust a mozgatást *destruktív mozgatásként* kezeli: a forrás változó a mozgatás után érvénytelenné válik, és a használata fordítási hibát okoz. Ez a kód szándékosan nem fordul le:

```rust
fn main() {
    let s1 = String::from("hello");
    let s2 = s1; // s1 mozgatva, innentől érvénytelen
    println!("{s1}"); // fordítási hiba: use of moved value
}
```

Hibaüzenet:

```
error[E0382]: borrow of moved value: `s1`
 --> src/main.rs:4:15
  |
2 |     let s1 = String::from("hello");
  |         -- move occurs because `s1` has type `String`,
  |            which does not implement the `Copy` trait
3 |     let s2 = s1;
  |              -- value moved here
4 |     println!("{s1}");
  |               ^^^^ value borrowed here after move
```

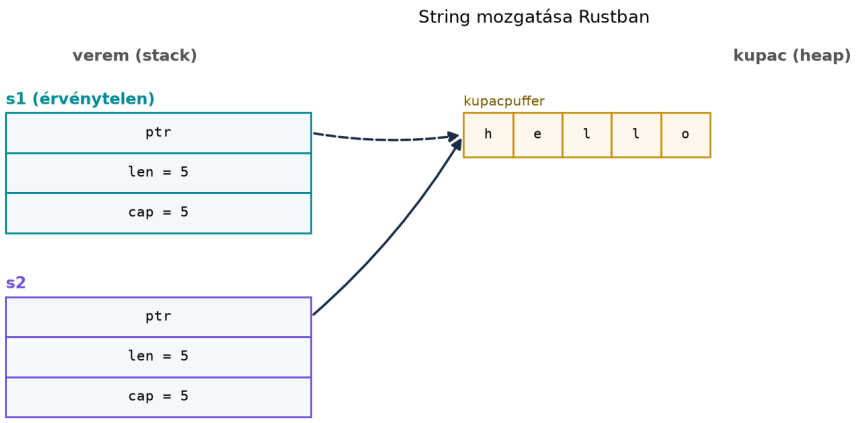
A javítás attól függ, mit akarunk. Ha az eredeti értékre már nincs szükségünk, használjuk az `s2`-t:

```rust
fn main() {
    let s1 = String::from("hello");
    let s2 = s1;
    println!("{s2}"); // "hello"
}
```

Ha viszont mindkét változót használni akarjuk, másolni kell:

```rust
fn main() {
    let s1 = String::from("hello");
    let s2 = s1.clone();
    println!("{s1} és {s2}"); // "hello és hello"
}
```

A mozgatás utáni állapotot a következő ábra szemlélteti. A `String` a veremben egy mutatót, hosszt és kapacitást tárol; a tényleges karakterek a kupacban vannak. Mozgatáskor a mutató átkerül az új tulajdonoshoz, a régi `s1` pedig érvénytelenné válik.



2.1. ábra: String mozgatása Rustban - A mozgatás után s1 érvénytelen; a kupacadat nem másolódik, a mutató átkerül s2-be.

A mozgatás tehát nem másolja a kupacadatot, csak a veremben lévő struktúrát másolja (bitenként), és a régi példányt a fordító statikusan érvényteleníti. Ez a *destruktív mozgatás* a Rust egyik kulcsfontosságú eltérése a C++-tól, ahol a mozgatás után a forrásobjektum érvényes, de nem meghatározott állapotú marad.

2.1.5 Zero-cost absztrakció mindkét nyelvben

A C++ és a Rust is büszke a zero-cost absztrakcióra: ha nem használsz egy absztrakciót, nem fizetsz érte; ha használod, nem tudnád kézzel gyorsabban megírni. A Rust generikusai – a C++ sablonokhoz hasonlóan – monomorfizálódnak, azaz a fordító minden konkrét típusra külön kódot generál. A trait-objektumok pedig a virtuális függvényekhez hasonló dinamikus diszpécset adnak, de erről később lesz szó.

A biztonsági ellenőrzések azonban némi futási költséggel járhatnak. A leggyakoribb példa a határellenőrzés:

```
let v = vec![1, 2, 3];
let x = v[5]; // panic: index out of bounds
```

A C++-ban `v[5]` UB, futás közben bármit okozhat. A Rust biztonságos kódban ezt futásidőben ellenőrzi, és túlindexeléskor *panikol*: ez nem UB, de futási költség és hibalehetőség. Ez tehát az a pont, ahol a Rust nem statikusan, hanem futásidejű ellenőrzéssel ad biztonságot. A gyakorlatban a Rust fordító sok esetben ki tudja optimalizálni a határellenőrzést, ha a határokat statikusan ismeri, vagy ha egy iterátorral eleve nem léphetünk túl. Ahol mégis megmarad, ott a biztonságért fizetünk.

Ugyanakkor a Rust az aliasing-információból profitál is. A `&mut T` *kizárólagos hozzáférési jogot* ad: amíg a kölcsönzés él, a hivatkozott memóriát csak ezen a referencián (és az abból levezetett, ún. reborrow-kon) keresztül lehet olvasni vagy írni. Ez nem azt jelenti, hogy sehol sem létezhet más mutató ugyanarra a címre: nyers mutató (`*mut T`) létezhet, de ha használjuk, miközben a `&mut` él és nem ő a forrása, az `unsafe` kódban UB (lásd az Unsafe Rust és a Slice-ok, String és &str alfejezet nyers mutatós példáit). A fordító arra az ígéretre építhet, hogy a `&mut` nélkül levezetett más elérési út nem nyúl az adathoz, és ezért agresszívebben optimalizálhat (például

gyorsítótárazhat értékeket vagy vektorizálhat). A reborrow az a mechanizmus, amellyel a `&mut` többször felhasználható anélkül, hogy elveszne:

```
fn bump(x: &mut i32) {
    *x += 1;
}

fn main() {
    let mut n = 0;
    let r = &mut n;
    bump(r); // implicit reborrow: &mut *r
    bump(r); // r továbbra is használható
    *r += 1;
    println!("{n}"); // 3
}
```

C++-ban a `restrict` (GCC/Clang: `__restrict`) hasonló célt szolgál, de nem része a szabványos C++-nak, és a fordító nem ellenőrzi.

2.1.6 Safe és unsafe Rust; az UB fogalma

A Rust alapértelmezett módja a **safe Rust**. Ebben a módban a fordító megakadályozza a memóriabiztonsági hibákat és a data race-t. Vannak azonban olyan alacsony szintű műveletek, amelyeket a fordító nem tud biztonságosnak bizonyítani: nyers mutatók dereferálása, `static mut` változók módosítása, nem biztonságos függvények hívása. Ezeket az `unsafe` blokkban lehet elvégezni:

```
fn main() {
    let mut x = 10;
    let ptr = &mut x as *mut i32;
    // SAFETY: a ptr érvényes, és nem aliasingolódik.
    unsafe {
        *ptr = 20;
    }
    println!("{x}");
}
```

Az `unsafe` kulcsszó nem teszi „rosszá” a kódot; csak azt jelzi, hogy a fejlesztő felel a további invariánsokért. A Rust ökoszisztéma filozófiája, hogy az `unsafe` kódot biztonságos absztrakció mögé rejtjük, így a felhasználó továbbra is safe kódot írhat. Erről részletesen a Unsafe Rust: szuperképességek és felelősség alfejezetben lesz szó.

A nem definiált viselkedés (UB) fogalma mindkét nyelvben ismert. A Rustban UB-t csak egy `unsafe` kód helytelensége okozhat, de a hiba *következménye* nem marad az `unsafe` blokk belsejében: ha egy `unsafe` blokk megsérti egy safe absztrakció invariánsát (például egy hibás hosszt állít be egy vektoron), a UB később, egy teljesen safe sorban jelentkezhet. Ezért nem az `unsafe` blokk szöveges helye a lényeg, hanem a felelősségi határ: az `unsafe` implementáció szerzője felel azért, hogy a köré épített safe interfész semmilyen hívással ne vezethessen UB-hez (lásd a soundness definícióját fent és a Biztonságos absztrakció unsafe alapon alfejezetet). Sound könyvtárak mellett a safe kódban memóriabiztonsági UB nem keletkezhet; a hibák kontrollált alakja a *panic* (a program leáll, vagy a hívó a `catch_unwind`-del elkaphatja). Ez nem a „legrosszabb” kimenetel: safe kód is okozhat végtelen ciklust, erőforrás-kimerülést és adatvesztést, és memóriahiánynál a folyamat abortálhat. A safe Rust tehát a memóriabiztonságot garantálja, nem a program általános helyességét: logikai hiba, holtpont, memóriaszivárgás (`mem::forget`, `Rc`-ciklus) safe kódban is lehetséges. Ez fontos garancia, mert a UB C++-ban gyakran csendes memóriakorrupciót okoz, amit nehéz reprodukálni.

2.1.7 Ökoszisztéma: Cargo, crates.io, editionök, stabilitási ígéret

A Rust nyelv nem csak szintaxisból áll: a **Cargo** a hivatalos build rendszer és csomagkezelő, a **crates.io** pedig a csomagregiszter. A C++-ban a CMake, a vcpkg, a Conan stb. külön eszközök; Rustban minden egy helyen van. A `Cargo.toml` deklarálja a függőségeket, a `cargo build` letölti és lefordítja őket, a `cargo test` futtatja a teszteket. A fejezet későbbi alfejezeteiben minden labor ezt használja.

A Rust **edition-ök** (2015, 2018, 2021, 2024) crate-enként, a `Cargo.toml` `edition` mezőjében választott *nyelvi kompatibilitási szintek*: egy edition a szintaxis és a nyelvi viselkedés egy összefüggő, a fordító által egységesen kezelt változatát jelöli. Három különböző dolgot kell megkülönböztetni: az **edition** a nyelvi szint (crate-enként), a **rustc verziója** a fordító kiadása (például 1.99.1), a **crate semver-verziója** pedig a csomag saját kiadási száma (`version = "0.1.0"`). Különböző editionű crate-ek egy programban együtt használhatók, mert az edition a crate-en belüli szintaxist és szemantikát érinti, az interfészeket nem. A stabilitási ígéret feltételes: ami stabil Rusttal lefordult, az az új stabil fordítókkal is lefordul, *kivéve a dokumentált eseteket* (hibajavítások a típusellenőrzésben, a beállítástól függő lintek, kiegészítések, amelyek metódusnév-ütközést okozhatnak). A `std` API-ja bővül, a stabilizált elemek nem tűnnek el, az új editionök pedig a régi kód működését nem törik, mert az átállás explicit (`cargo fix --edition`).

Továbbvitt eredmény. Ez az alfejezet rögzíti a hibaosztályok listáját, amelyre a későbbi fejezetek visszautalnak: az ownership (Ownership: minden értéknek egy tulajdonosa van) a use-after-free és a double-free ellen, a borrowing (Referenciák és a kölcsönzési szabály: sok olvasó XOR egy író) az iterator-invalidálás és az aliasing ellen, a `Send`/`Sync` (Szálak, Send és Sync: data race fordítási időben kizárva) a data race ellen, az `Option` (Az enum mint összeg-típus és az Option a null helyett) a null pointer ellen, a slice-ok (Slice-ok, String és &str, UTF-8) pedig a buffer overflow ellen. Ezt a listát a Összefoglalás: C++→Rust fordítási kézikönyv és továbbtanulás alfejezet is hivatkozni fogja.

2.1.8 Tipikus hibák és átállási csapdák

1. **A borrow checker „legyőzése” klónozással.** Amikor a C++-fejlesztő a kölcsönzési hibákat `.clone()`-nal oldja meg, valójában elkerüli a problémát, de felesleges másolással rontja a teljesítményt. Ilyenkor inkább a referencia-élettartamok átszervezésére vagy a típus kialakítására kell gondolni.
2. **Az `unsafe` használata a biztonsági ellenőrzés megkerülésére.** A C++-ból jövő fejlesztő hajlamos lehet `unsafe` blokkba csomagolni mindent, hogy „megszokott” nyers mutatókkal dolgozzon. Ez aláássa a Rust biztonsági ígéretét; a valódi cél az, hogy az `unsafe`-et minimális, jól dokumentált absztrakciókra korlátozzuk.
3. **A moved-from objektum használata.** C++-ban a moved-from objektum érvényes (ha nem specifikált), Rustban a mozgatás után a forrás érvénytelen. Ez a szokás gyakran E0382 hibát okoz, amíg meg nem szokjuk a destruktív mozgatást.

2.1.9 Feladatok

(a) Fogalmi kérdés. Sorolja fel legalább öt C++-hibaosztályt, és rendelje hozzá a Rust biztonsági mechanizmusát (ownership, borrowing, `Send`/`Sync`, `Option`, slice-határellenőrzés). Melyik hibaosztályt zárja ki az `Option`?

(b) C++-kód átírása. Az alábbi C++ kód lógó referenciát okoz. Írd át idiomatikus Rustra úgy, hogy ne legyen lógó referencia, és a program kinyomtassa az értéket.

```cpp
#include <string>
#include <iostream>

std::string& dangling() {
    std::string s = "hello";
    return s;
}

int main() {
    std::string& r = dangling();
    std::cout << r << '\n';
}
```

(c) Implementációs feladat. Hozz létre egy új Rust projektet `cargo new move_demo` paranccsal, és írd be a use-after-move hibás kódot (`let s2 = s1; println!("{s1}");`). Ellenőrizd a hibakódot (E0382), majd javítsd a kódot úgy, hogy lefordúljon és a várt kimenetet adja. Dokumentáld, miért nem szabad a mozgatás után a forrást használni.

(d) Tervezési döntés. Adott a következő C++ data race példa: két szál növel egy közös `int`-et szinkronizáció nélkül. Válassz Rust megoldást a probléma biztonságos kiküszöbölésére (például `Arc<Mutex<i32>>`, atomi típus vagy csatorna), indokold a választást, és írj egy lefordítható Rust vázlatot. Ez a feladat előkészíti a Szálak, Send és Sync: data race fordítási időben kizárva alfejezetet.

2.1.10 Megoldások

(a) A hibaosztályok és mechanizmusok: - use-after-free → ownership (automatikus felszabadítás a tulajdonos hatókörének végén) - double-free → ownership (a felszabadítás kizárólagos felelőssége) - lógó referencia → borrowing (élettartam-ellenőrzés) - iterator-invalidálás → borrowing (sok olvasó XOR egy író) - data race → `Send`/`Sync` (nem szálbiztos típus nem léphet át szálhatáron) - null pointer dereference → `Option` (nincs null referencia) - buffer overflow → slice-ok és határellenőrzés (panikolás a UB helyett)

Az `Option` a null *referencia* hibaosztályát zárja ki a safe kódban: a hiányzó értéket explicit `None` jelzi, a `match` és az `if let` láthatóvá teszi a kezelést. Nem akadályoz meg minden futási hibát (az `unwrap()` `None`-ra pánikol), és a nyers `*const T` mutató továbbra is lehet null, ezért a dereferálása `unsafe`.

(b) A javított Rust kód visszaadja a `String`-et, nem referenciát:

```rust
fn dangling() -> String {
    let s = String::from("hello");
    s
}

fn main() {
    let r = dangling();
    println!("{r}");
}
```

A `String` tulajdonjoga átkerül a hívóhoz, így nincs érvénytelen referencia.

(c) A lépések:

```
cargo new move_demo
cd move_demo
```

A `src/main.rs` hibás tartalma:

```rust
fn main() {
    let s1 = String::from("hello");
    let s2 = s1;
    println!("{s1}"); // E0382: borrow of moved value
}
```

A `cargo run` hibát ad a fenti E0382 hibakóddal. A javítás:

```rust
fn main() {
    let s1 = String::from("hello");
    let s2 = s1;
    println!("{s2}");
}
```

Várt kimenet: `hello`. A mozgatás után a forrás érvénytelen, mert a Rust a duplikált felszabadítás elkerülése érdekében a tulajdonjogot átadja; a forrás használata UB-t (double free) okozhatna, ezért a fordító tiltja.

(d) Egy biztonságos megoldás `Arc<Mutex<i32>>`-szel:

```rust
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let counter = Arc::new(Mutex::new(0));
    let mut handles = vec![];

    for _ in 0..2 {
        let counter = Arc::clone(&counter);
        let handle = thread::spawn(move || {
            let mut num = counter.lock().unwrap();
            *num += 1;
        });
        handles.push(handle);
    }

    for handle in handles {
        handle.join().unwrap();
    }

    println!("{}", *counter.lock().unwrap());
}
```

Az `Arc` biztosítja a megosztott tulajdonlást, a `Mutex` a kölcsönös kizárást. A `counter` típusa `Arc<Mutex<i32>>`, amely `Send` és `Sync`, így a closure átvihető a szálakra, és a data race kizárt.

2.1.11 Rövid összefoglaló

Ez az alfejezet bemutatta azokat a C++-hibaosztályokat, amelyeket a biztonságos Rust fordítási időben kizár: use-after-free, double-free, lógó referencia, iterator-invalidálás, data race, null pointer dereference és buffer overflow. A nyelv ezt az ownership, a borrowing, valamint a `Send`/`Sync` rendszerével éri el; a safe Rustban nincs nem definiált viselkedés, csak panik. A mozgatás destruktív, a moved-from változó használata E0382 fordítási hibát ad. A zero-cost absztrakció mindkét nyelvben érvényes, de a Rust határellenőrzése futási költséggel járhat, míg az aliasing-információból optimalizációs előnyt nyer. Az `unsafe` blokkokra korlátozott UB a fejlesztő felelőssége. A Cargo, a crates.io és az editionök adják az ökoszisztéma stabilitását. **Továbbvitt eredmény:** a hibaosztály-lista, amelyre az Ownership: minden értéknek egy tulajdonosa van, Referenciák és a kölcsönzési szabály: sok olvasó XOR egy író, Szálak, Send és Sync: data race fordítási időben kizárva és más alfejezetek visszautalnak.

2.1.12 Források és ellenőrzési hiányok

- The Rust Reference, „Behavior considered undefined”: https://doc.rust-lang.org/reference/behavior-considered-undefined.html
- The Rustonomicon, „Meet Safe and Unsafe” és „How Safe and Unsafe Interact” (a soundness fogalma): https://doc.rust-lang.org/nomicon/
- The Rust Programming Language, 4. fejezet (ownership, borrowing) és 16. fejezet (`Send`/`Sync`): https://doc.rust-lang.org/book/
- Rust compiler error index (E0382, E0502, E0515, E0277): https://doc.rust-lang.org/error_codes/

A hibakódok stabilak; a hibaüzenetek szövege a könyv rögzített toolchainjére (Rust 1.99.1, 2024-es edition, lásd Eszközlánc: rustup, cargo, rustfmt, clippy, dokumentáció) vonatkozik.

2.2 Eszközlánc: rustup, cargo, rustfmt, clippy, dokumentáció

Egy C++-projektben a buildrendszer, a függőségkezelés, a formázás és a statikus elemzés általában külön eszközökből áll össze: CMake vagy Make, vcpkg vagy Conan, clang-format, clang-tidy vagy cppcheck, és gyakran még egy CI-szkript, amely ezeket összefogja. A Rust ezt a négy területet egyetlen eszközláncba, a Cargo köré szervezi: a `rustup` a fordítót és a kiegészítőket telepíti, a `cargo` buildel, tesztel, függőséget tölt le és dokumentációt generál, a `rustfmt` formáz, a `clippy` pedig a nyelv saját statikus elemzője. Ez a fejezet ezt a munkafolyamatot alapozza meg; a későbbi rkv-laborok mind ezekre a parancsokra épülnek.

2.2.1 Fogalmak és szabályok

Definíció. A **rustup** a Rust toolchaineket kezelő eszköz. Egy toolchain a Rust fordítóból (`rustc`), a csomagkezelőből (`cargo`), a standard könyvtárból és választható komponensekből áll. A három fő csatorna:

- **stable**: a stabil kiadás, éles kódhoz ez az ajánlott;
- **beta**: a következő stable előzetese;
- **nightly**: a legfrissebb, de instabil funkciókat is tartalmazó build, kísérletezéshez.

Szabály. A könyv projektjei a **stable** toolchaint használják; nightly-only funkciót csak akkor említünk, ha azt külön jelöljük, és soha nem építünk rá éles kódot. Kivétel a Miri, amely csak nightly toolchainnel fut; ezt mindig `cargo +nightly miri ...` formában, a projekt toolchainjétől elkülönítve hívjuk.

A könyv rögzített tanulási környezete. A „stable” mozgó cél: hathetente új kiadás jön. Hogy a laborok tiszta könyvtárból is ugyanúgy álljanak elő, a könyv a következőket rögzíti:

Elem	Rögzített érték
toolchain	Rust **1.99.1** (stable), `rustfmt` és `clippy` komponenssel, a `~/rust-labor/rust-toolchain.toml` szerint (a labor-gyökérben, így az rkv és az összes testvér-labor örökli)
edition	**2024** minden crate-ben; a könyv nem vált vissza 2021-re

Elem	Rögzített érték
minimális Rust-verzió	`rust-version = "1.89"` (a 2024-es edition már az 1.85-től támogatott; az 1.89 azért a minimum, mert az I/O és szerializáció: fájlok, pufferelés, serde `FileStore`-ja a `File::try_lock`-ot használja)
függőségfeloldó	`resolver = "3"` a workspace gyökerében (a 2024-es edition alapértelmezése)
lockfájl	egyetlen közös `Cargo.lock` a workspace gyökerében, verziókezelés alatt
operációs rendszer	Linux x86-64 (a laborokat Ubuntu 24.04-en, bash alatt írtuk), C/C++ fordító: GCC 13 vagy Clang 18. A CMake-es build-integráció (C++-együttműködés a cxx crate-tel és a fokozatos migráció) **opcionális, nem ellenőrzött kitekintés**: CMake 3.28+ és Corrosion 0.6.1 kell hozzá, de a könyv ellenőrzött útvonala a kézi `cargo build` + `g++`

macOS-en és Windows/WSL2 alatt a Rust-laborok változtatás nélkül működnek; a C/C++ együttműködés (FFI, cxx) és a Linux-specifikus részek (például a könyvtár-`fsync`) ott eltérhetnek, ezeket a könyv nem támogatja külön.

A `rust-toolchain.toml` a **`~/rust-labor/` labor-gyökérbe** kerül (nem az rkv workspace-be): a `rustup` a munkakönyvtártól felfelé keresi, ezért az `rkv/`, a `hello/` és az összes `rNNN/` testvérkönyvtár ugyanazt a toolchaint kapja. Ha ilyen fájlt talál, a benne megadott toolchaint használja (és ha hiányzik, telepíti). A már meglévő felülbírálásokat ellenőrizd: `rustup override list` (könyvtáranként `rustup override set`), valamint a `RUSTUP_TOOLCHAIN` környezeti változó és a parancssori `cargo +<toolchain>` felülírja a fájlt. A tényleges toolchaint a `rustc --version` mutatja a projektkönyvtárban.

```
[toolchain]
channel = "1.99.1"
components = ["rustfmt", "clippy"]
profile = "minimal"
```

Egy újabb stable toolchain is szinte mindig lefordítja a könyv kódját, de a lintüzenetek és a hibaszövegek eltérhetnek. Ha a rögzítettnél újabb toolchainnel dolgozol, a `channel` értékét tudatosan írd át.

A `rustup` komponenseket is kezel: `rustfmt` (formázó), `clippy` (linter), `rust-docs` (helyi dokumentáció), `rust-analyzer` (nyelvi szerver). A `rustup component add <név>` paranccsal telepíthetők.

Definíció. A **Cargo** a Rust buildrendszere és csomagkezelője. Egy Cargo-projekt lelkét a `Cargo.toml` manifest adja, amely a csomag nevét, verzióját, editionjét és függőségeit tartalmazza. A `Cargo.lock` a feloldott függőségi gráf pontos verzióit rögzíti. A Cargo FAQ jelenlegi ajánlása szerint **minden** projekt lockfájlját érdemes verziókezelésbe tenni (a `cargo new` sem teszi már a `.gitignore`-ba), mert így a saját fejlesztői és CI-build reprodukálható; a lockfájl azonban csak a *saját* buildet rögzíti: egy közzétett könyvtár `Cargo.lock`-ját a fogyasztó nem veszi figyelembe, ő a saját gráfját oldja fel.

A Cargo alapvető parancsai:

- `cargo new <név>` – új projekt létrehozása;
- `cargo build` – debug profilú fordítás, alapértelmezés szerint a `target/debug/` könyvtárba;
- `cargo build --release` – optimalizált release fordítás a `target/release/` könyvtárba;
- `cargo run` – build, majd futtatás;
- `cargo test` – tesztfordítás és tesztek futtatása;
- `cargo check` – csak típusellenőrzés, kódgenerálás nélkül; gyorsabb, mint a teljes build;
- `cargo fmt` – a `rustfmt` futtatása a projektre;
- `cargo clippy` – a `clippy` linter futtatása;
- `cargo doc --open` – dokumentáció generálása és megnyitása böngészőben.

Szabály. A `target/` könyvtár mindig generált, nem kerül verziókezelésbe. A `cargo clean` törli, a `cargo build` újra létrehozza.

A **debug** profil gyors fordításra optimalizál, cserébe lassabb futást ad; a **release** profil lassabb fordításért cserébe agresszív optimalizációt (`opt-level = 3`) és kevesebb futásidejű ellenőrzést kapcsol be. A profilok a `Cargo.toml` `[profile.*]` szekcióiban hangolhatók, de az alapértelmezések a legtöbb projekthez megfelelők.

2.2.2 C++ és Rust összevetés

A C++-világban a buildrendszer és a csomagkezelő szét van választva; a Rustban a Cargo mindkettőt egyszerre végzi. Tekintsünk egy minimális C++ programot és annak CMake-buildjét, majd a Rust-megfelelőt.

```cpp
// main.cpp
#include <iostream>
int main() {
    std::cout << "Hello, C++!\n";
}
```

```
// CMakeLists.txt
cmake_minimum_required(VERSION 3.16)
project(hello_cpp)
add_executable(hello_cpp main.cpp)
```

```rust
// src/main.rs
fn main() {
    println!("Hello, Rust!");
}
```

```
# Cargo.toml
[package]
name = "hello_rust"
version = "0.1.0"
edition = "2024"

[dependencies]
```

A különbség nem csak szintaktikai. A CMake egy általános célú buildgenerátor, amely mögött Make, Ninja vagy Visual Studio áll; a Cargo közvetlenül hívja a `rustc`-t, és maga gondoskodik a függőségek letöltéséről, verziózásáról és a fordítási sorrendről. A vcpkg/Conan rendszerekben a függőségek gyakran rendszerszintű vagy félglobális könyvtárakba kerülnek; a Cargo a függőségeket a `~/.cargo/registry/` gyorsítótárba tölti, és projektenként, a `Cargo.lock` alapján reprodukálhatóan építi be őket.

A minőségi eszközök is szorosabban integráltak. A C++-ban a `clang-format` és a `clang-tidy` külön parancsok, gyakran egyedi konfigurációval; a Rustban a `cargo fmt` és a `cargo clippy` a

manifestben vagy külön konfigurációs fájlokban állítható, de működésük alapértelmezésben is konzisztens. A `rustdoc` a Doxygenhez hasonló szerepet tölt be, de a dokumentációs kommentek (`///`) a nyelv szintjén kapcsolódnak a kódhoz, és a generált HTML-be a dokumentációs tesztpéldák is bekerülnek.

2.2.3 Kidolgozott labor: a teljes eszközlánc működés közben

Munkakönyvtár: `~/rust-labor/r002/`

Kódblokk-jelölések. A könyv laborjaiban minden olyan kódblokk előtt, amelyet fájlba kell írni, egy *Kódblokk* sor áll. Ez megadja a fájl workspace-hez (vagy a labor munkakönyvtárához) viszonyított útját, a műveletet és az ellenőrzést. A műveletek:

- **új fájl** – a fájl még nem létezik, a blokk a teljes tartalma;
- **teljes csere** – a fájl meglévő tartalmát a blokk teljesen felváltja;
- **beillesztés** – a blokk a megnevezett helyre kerül, a fájl többi része változatlan marad;
- **önálló példa** – nem a projekt része, a `~/rust-labor/rNNN/` alatt egy friss `cargo new` projektben próbálható ki;
- **szándékosan hibás** – a blokk a megnevezett hibával (például `E0382`) *nem* fordul le, vagy a megnevezett figyelmeztetést adja; a javított változat külön blokkban következik.

A *Kódblokk* sor nélküli rövid kódrészletek a magyarázat illusztrációi, nem kell őket fájlba írni. Minden rkv-labor végén egy **Ellenőrzési pont** rögzíti a projekt elvárt állapotát és az azt igazoló parancsot; a következő labor ebből az állapotból indul.

A labor célja: telepíteni a stable toolchaint, létrehozni egy Cargo-projektet, lefuttatni a buildet, a teszteket, a formázót és a lintert, majd megnézni a generált dokumentációt. A labor kimenete egy `hello` nevű bináris crate lesz, amelyet később a Modulok és láthatóság: header nélküli világ alfejezetben könyvtári modullá bővítünk.

2.2.3.1 1. lépés: rustup telepítése (ha még nincs)

Linuxon a hivatalos telepítő:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

A telepítő a `~/.cargo/bin/` könyvtárba helyezi a binárisokat, és a `~/.cargo/env` fájlban elérhető környezeti beállítást hoz létre. Új terminálablakban vagy a `source ~/.cargo/env` kiadása után ellenőrizzük:

```
rustup --version
rustc --version
cargo --version
```

A `rustup show` megjeleníti az aktív toolchaint és a telepített komponenseket. Ha a stable toolchain még nincs telepítve:

```
rustup default stable
rustup component add rustfmt clippy rust-docs
```

A `rustup component list --installed` paranccsal ellenőrizhetjük, hogy minden komponens megvan-e. A pontos verziószámok a kiadástól függnek, ezért a kimenetet itt nem tüntetjük fel.

2.2.3.2 2. lépés: projekt létrehozása

```
mkdir -p ~/rust-labor/r002
cd ~/rust-labor/r002
cargo new hello
cd hello
```

A `cargo new` létrehozza a következő struktúrát:

```
hello/
|-- Cargo.toml
`-- src
    `-- main.rs
```

A generált `Cargo.toml` tartalma:

Kódblokk — fájl: `hello/Cargo.toml` · művelet: a `cargo new` generálta, nem kell szerkeszteni

```
[package]
name = "hello"
version = "0.1.0"
edition = "2024"

[dependencies]
```

Az `edition` a nyelvi kiadást jelöli; a 2024-es editiont a Rust 1.85-től támogatja, a rögzített 1.99.1-es toolchain `cargo new` parancsa ezt írja be. Régebbi toolchain a 2021-es editiont generálná; a könyv ezt nem támogatja, ezért ilyenkor a toolchaint kell frissíteni, nem az editiont visszaállítani. Az edition nem a toolchain verziója, hanem a nyelvi kompatibilitási szint; erről később, a Függőségek, feature-ök, verziózás és workspace-ek alfejezetben lesz részletesebben szó.

A generált `src/main.rs`:

Kódblokk — fájl: `hello/src/main.rs` · művelet: a `cargo new` generálta, nem kell szerkeszteni

```
fn main() {
    println!("Hello, world!");
}
```

2.2.3.3 3. lépés: build, futtatás, teszt

```
cargo build
```

A parancs lefordítja a programot, és létrehozza a `target/debug/hello` binárist. A `cargo run` újrafordít, ha szükséges, majd futtat:

```
cargo run
```

Várt kimenet:

```
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.02s
     Running `target/debug/hello`
Hello, world!
```

A `cargo check` csak a típushelyességet ellenőrzi, nem generál futtatható kódot; ezért gyorsabb, és akkor hasznos, ha csak a fordítási hibákat akarjuk látni:

```
cargo check
```

A teszteléshez adjunk egy egyszerű tesztet a `src/main.rs`-hez, vagy hozzunk létre egy `src/lib.rs`-t. Tiszta bináris crate-ben a tesztet a `main.rs` aljára tehetjük, de idiomatikusabb a könyvtári részt különválasztani. Készítsünk egy `src/lib.rs` fájlt:

Kódblokk — fájl: `hello/src/lib.rs` · művelet: új fájl · ellenőrzés: `cargo test` → 1 sikeres teszt

```
/// Visszaad egy üdvözlő üzenetet a megadott névvel.
pub fn greet(name: &str) -> String {
    format!("Hello, {name}!")
}

#[cfg(test)]
mod tests {
    use super::greet;

    #[test]
    fn greet_matches() {
        assert_eq!(greet("Rust"), "Hello, Rust!");
    }
}
```

Most a `cargo test` lefuttatja a tesztet:

```
cargo test
```

Várt kimenet (lerövidítve):

```
running 1 test
test tests::greet_matches ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

A `hello` crate így már bináris és könyvtári célpontot is tartalmaz. A `Cargo.toml` automatikusan érzékeli a `src/lib.rs` jelenlétét; ezt a Structok és metódusok alfejezetben fogjuk kihasználni.

2.2.3.4 4. lépés: rustfmt és clippy

A formázó futtatása:

```
cargo fmt
```

Ez átalakítja az összes `.rs` fájlt a hivatalos stílus szerint. Ellenőrzésre a `cargo fmt -- --check` parancs szolgál, amely hibával tér vissza, ha a kód nincs formázva.

A clippy futtatása:

```
cargo clippy
```

A clippy statikus elemzést végez, és idiomatikusabb, gyorsabb vagy biztonságosabb kódot javasol. Példaként szándékosan írjunk rosszabbul olvasható kódot a `src/lib.rs`-be:

Kódblokk — fájl: `hello/src/lib.rs` · művelet: beillesztés a fájl végére · **szándékosan rossz stílusú**, de lefordul · ellenőrzés: `cargo clippy` → `needless_return` és `needless_bool` figyelmeztetés

```rust
pub fn is_positive(n: i32) -> bool {
    if n > 0 {
        return true;
    } else {
        return false;
    }
}
```

Ez a változat lefordul, a `cargo build` nem jelez semmit; a clippy viszont két lintet is jelez: a `needless_return` szerint a `return true;` és a `return false;` felesleges, a `needless_bool` szerint pedig az egész `if`–`else`, amely csak `bool` literált ad vissza, az `n > 0` feltétellel helyettesíthető. A `cargo clippy` kimenetének eleje (Rust 1.99.1) **kivonat**: a `is_positive` függvényt egy önálló, üres `src/lib.rs`-be helyezve a sorok 2–3-ra esnek; a labor `lib.rs`-ének végére illesztve a sorszámok a fájl hosszával eltolódnak, a diagnózis tartalma és a lintnevek viszont ugyanazok.

```
warning: unneeded `return` statement
 --> src/lib.rs:3:9
  |
3 |         return true;
  |         ^^^^^^^^^^^
  |
  = note: `#[warn(clippy::needless_return)]` on by default
...
warning: this if-then-else expression returns a bool literal
 --> src/lib.rs:2:5
  |
  = note: `#[warn(clippy::needless_bool)]` on by default
```

Az alapértelmezett `cargo clippy` a figyelmeztetésekkel is **0-s kilépési kóddal** tér vissza: a warning nem sikertelen futás. A `-D warnings` ezen változtat: a `cargo clippy -- -D warnings` a clippy és a rustc minden figyelmeztetését hibává emeli, ezért erre a kódra három hibával (`needless_return` kétszer, `needless_bool` egyszer) és 101-es kilépési kóddal áll meg. Ezt használjuk CI-ban kapuként.

A javítás:

Kódblokk — fájl: `hello/src/lib.rs` · művelet: az előző `is_positive` cseréje · ellenőrzés: `cargo clippy` → nincs figyelmeztetés

```rust
pub fn is_positive(n: i32) -> bool {
    n > 0
}
```

Futtassuk újra a `cargo clippy`-t, és ellenőrizzük, hogy nincs több figyelmeztetés. Ha a clippy-figyelmeztetéseket hibává akarjuk alakítani (például CI-ban), használhatjuk a `-D warnings` kapcsolót:

```
cargo clippy -- -D warnings
```

Ez minden clippy-warningot fordítási hibává alakít. A munkafolyamatban ez hasznos, mert a figyelmeztetések nem maradnak észrevétlenek.

2.2.3.5 5. lépés: dokumentáció generálása

A `cargo doc` a `///` dokumentációs kommentekből HTML-dokumentációt generál a `target/doc/` könyvtárba. A `--open` kapcsoló megnyitja a böngészőben, de a laborban csak a generálást futtatjuk:

```
cargo doc
```

A generált oldal a `target/doc/hello/index.html` fájlban érhető el. A dokumentációs kommentekben lévő kódpéldák automatikusan tesztelődnek a `cargo test` során: a `///` kommentben elhelyezett kódblokkok dokumentációs tesztként (doctest) futnak le. A `greet` dokumentációjában még nincs kódpélda, ezért a `cargo test --doc` most nulla doctestet talál:

```
   Doc-tests hello

running 0 tests
```

Adjunk a `greet` dokumentációjához egy példát. A doctest külön crate-ként fordul, ezért a függvényre a crate nevével (`hello::greet`) hivatkozunk:

Kódblokk — fájl: `hello/src/lib.rs` · művelet: a `greet` feletti `///` komment cseréje · ellenőrzés: `cargo test --doc` → 1 sikeres doctest

```rust
/// Visszaad egy üdvözlő üzenetet a megadott névvel.
///
/// # Példa
///
/// ```
/// assert_eq!(hello::greet("Rust"), "Hello, Rust!");
/// ```
pub fn greet(name: &str) -> String {
    format!("Hello, {name}!")
}
```

A `cargo test --doc` kimenete most (a sorszám a kódblokk nyitó sorára utal a fájlban):

```
   Doc-tests hello

running 1 test
test src/lib.rs - greet (line 5) ... ok
```

A `rust-analyzer` egy nyelvi szerver, amely a szerkesztőbe integrálva valós idejű kódkiegészítést, hibajelzést és navigációt biztosít. Telepítése toolchain-komponensként lehetséges:

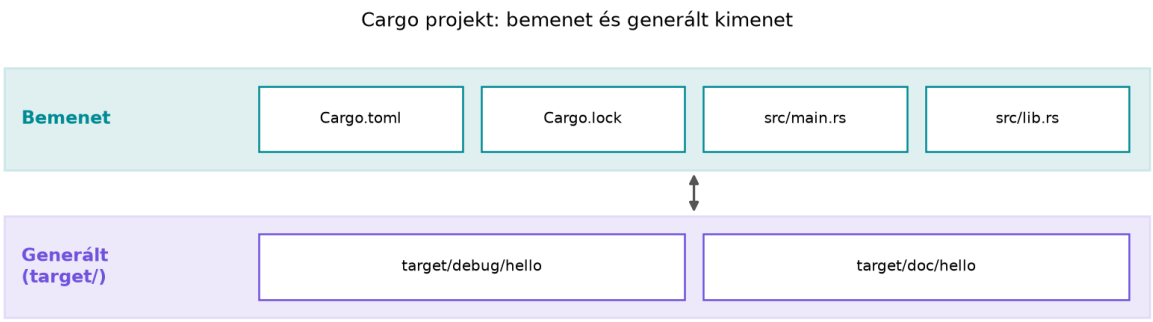
```
rustup component add rust-analyzer
```

A pontos IDE-integráció (VS Code, CLion, Neovim) a szerkesztőtől függ; a `rust-analyzer` a Cargo manifestjét és a `target/` könyvtár metaadatait használja a pontos elemzéshez.

Ellenőrzési pont. A `~/rust-labor/r002/hello` crate-ben van `src/main.rs` és `src/lib.rs` (`greet`, `is_positive`); a `cargo test` 1 sikeres unit tesztet és 1 sikeres doctestet futtat, a `cargo fmt --check` és a `cargo clippy -- -D warnings` hiba nélkül fut le.

2.2.4 A Cargo projekt rétegei

Az alábbi rétegábra a projekt fizikai szerkezetét mutatja.



2.2. ábra: Cargo projekt: bemenet és generált kimenet - A nyíl a build irányát jelöli (bemenet → kimenet), nem crate-függőséget. A Cargo a manifest és a forrás alapján generálja a target könyvtárat; a Cargo.lock a függőségek pontos verzióját rögzíti.

A `Cargo.toml` a projekt deklaratív leírása: mi a csomag neve, milyen függőségei vannak, milyen buildprofilokat használ. A `Cargo.lock` a feloldott függőségi fa pontos verzióit tartalmazza; binárisok esetén commitoljuk, hogy a build reprodukálható legyen. A `target/` mindig generált.

2.2.5 Tipikus hibák és átállási csapdák

1. **A `target/` könyvtár verziókezelése.** C++-projekteknél a build könyvtár (pl. `build/`) gyakran a repo része vagy a `.gitignore`-ban szerepel; Rustban a `target/` soha nem kerül be a verziókezelésbe. Ha véletlenül mégis, a `.gitignore`-hoz adjuk hozzá.
2. **A `cargo run` mindig debug módban fut.** A release módú futtatáshoz `cargo run --release` szükséges. A C++-ban a build típusát a CMake `CMAKE_BUILD_TYPE` változója szabályozza; Rustban a `--release` kapcsoló a Cargo parancs része, nem a manifesté.
3. **A `Cargo.lock` szerepének félreértése.** A régebbi tanács szerint könyvtáraknál a lockfájlt nem commitoljuk; a Cargo FAQ azóta azt javasolja, hogy a lockfájl legyen verziókezelés alatt minden projektben. Fontos különbség: a repositoryban tárolt lockfájl a *saját* build és CI reprodukálhatóságát szolgálja, a könyvtárat függőségként használó crate feloldását viszont nem befolyásolja, hiszen a fogyasztó a saját gráfját oldja fel. Az rkv teljes workspace-e egyetlen, a gyökérben lévő `Cargo.lock`-ot használ.
4. **A clippy figyelmeztetések figyelmen kívül hagyása.** A C++-ban a `clang-tidy` gyakran túl zajos, ezért sokan kikapcsolják; a Rust clippy azonban a nyelv idiomatikus használatát tanítja, és a figyelmeztetések gyakran valós tervezési hibákra utalnak. A `-D warnings` használata a CI-ban jó gyakorlat.

2.2.6 Feladatok

2.2.6.1 1. Fogalmi kérdés

Mi a különbség a `Cargo.toml` és a `Cargo.lock` között? Melyiket commitoljuk bináris, és melyiket könyvtár crate esetén, és miért?

2.2.6.2 2. C++-munkafolyamat átültetése

Adott egy C++-projekt, amely a következő elemekből áll: `main.cpp`, `CMakeLists.txt`, egy `format.sh` szkript (`clang-format -i`), és egy `lint.sh` (`clang-tidy`). Írd le, milyen Cargo-parancsokkal váltanád ki ezeket a Rustban.

2.2.6.3 3. Implementációs feladat

Hozz létre egy új Cargo-projektet `counter` néven a `~/rust-labor/r002/counter/` könyvtárban. A program tartalmazzon egy `src/lib.rs` fájlt egy `increment` függvénnyel, amely egy `i32` értékét eggyel növeli, és egy tesztet, amely ellenőrzi a működését. Futtasd a `cargo test` és a `cargo clippy` parancsokat.

2.2.6.4 4. Hibajavítás és refaktorálás

A laborban létrehozott `hello` projekt `src/lib.rs`-ében szándékosan hagyj clippy-figyelmeztetést (például `if n > 0 { true } else { false }`). Futtasd a `cargo clippy` parancsot, olvasd el a figyelmeztetést, javítsd a kódot, majd ellenőrizd, hogy a `cargo clippy -- -D warnings` már hibamentesen fut le.

2.2.7 Megoldások

2.2.7.1 1. Fogalmi kérdés megoldása

A `Cargo.toml` a manifest: a projekt neve, verziója, editionje és a függőségek leírása. A `Cargo.lock` a feloldott függőségi gráf pontos verzióit és ellenőrzőösszegeit tartalmazza. A Cargo FAQ jelenlegi ajánlása szerint minden projektnél, így könyvtáraknál is érdemes verziókezelés alá tenni, mert a saját build és a CI reprodukálható lesz tőle. A lockfájl azonban nem utazik a függőséggel: egy közzétett könyvtár lockfájlját a fogyasztó figyelmen kívül hagyja, és a saját gráfját oldja fel, ezért a könyvtár lockfájlja a fogyasztók buildjét nem befolyásolja.

2.2.7.2 2. C++-munkafolyamat átültetése megoldása

- `CMakeLists.txt` + `make` → `Cargo.toml` + `cargo build`
- `format.sh` → `cargo fmt`
- `lint.sh` → `cargo clippy`
- A tesztek futtatása C++-ban gyakran külön célpont, Rustban `cargo test`
- A dokumentáció generálása Doxygen helyett `cargo doc --open`

2.2.7.3 3. Implementációs feladat megoldása

```
mkdir -p ~/rust-labor/r002
cd ~/rust-labor/r002
cargo new counter --lib   # létrehozza a ~/rust-labor/r002/counter/ könyvtárat
cd counter
```

A `cargo new counter --lib` a `~/rust-labor/r002/counter/Cargo.toml`-t és a `src/lib.rs` vázat hozza létre (a feladatban kijelölt könyvtárban, nem egy beágyazott `counter/counter` alatt). A `src/lib.rs` tartalma:

```rust
/// Eggyel növeli a kapott számot.
pub fn increment(n: i32) -> i32 {
    n + 1
}

#[cfg(test)]
mod tests {
    use super::increment;

    #[test]
    fn increments() {
        assert_eq!(increment(41), 42);
    }
}
```

Futtatás:

```
cargo test
cargo clippy
```

A `cargo test` lefuttatja a tesztet, a `cargo clippy` pedig nem ad figyelmeztetést.

2.2.7.4 4. Hibajavítás és refaktorálás megoldása

A `src/lib.rs` rossz változata:

```rust
pub fn is_even(n: i32) -> bool {
    if n % 2 == 0 {
        true
    } else {
        false
    }
}
```

A `cargo clippy` figyelmeztet: `clippy::needless_bool` – a `if` kifejezés helyett közvetlenül a feltétel használandó. Javított változat:

```rust
pub fn is_even(n: i32) -> bool {
    n % 2 == 0
}
```

Futtatás ellenőrzéshez:

```
cargo clippy -- -D warnings
```

Nem érkezik hiba, a kód idiomatikus.

2.2.8 Rövid összefoglaló

Ez az alfejezet bemutatta a Rust eszközlánc alapjait. A **rustup** a toolchaineket és komponenseket kezeli; a **Cargo** a buildrendszer, csomagkezelő, tesztfuttató és dokumentációgeneráló. Megtanultuk a `cargo new`, `build`, `run`, `test`, `check`, `fmt`, `clippy` és `doc` parancsokat, valamint a `Cargo.toml` és `Cargo.lock` szerepét. A debug és release profilok a `target/debug/` és `target/release/` könyvtárakba építenek. A **rustfmt** a formázást, a **clippy** a statikus elemzést végzi; a **rustdoc** a `///` kommentekből generál dokumentációt, és a kódpéldák automatikusan tesztelődnek. A **Továbbvitt eredmény**: ez a Cargo-munkafolyamat az alapja az rkv mintaprojekt indítása: Cargo-workspace és első CLI alfejezetnek, ahol az rkv workspace-t hozzuk létre, a

Modulok és láthatóság: header nélküli világ alfejezetnek, ahol a modulokat szervezzük, és az Unit-, integrációs és dokumentációs tesztek alfejezetnek, ahol a tesztelést mélyítjük el.

2.2.9 Források és ellenőrzési hiányok

- The rustup book, „Overrides” fejezet (`rust-toolchain.toml`): https://rust-lang.github.io/rustup/overrides.html
- The Cargo Book: a manifest (`edition`, `rust-version`), a workspace és a függőségfeloldó (`resolver`): https://doc.rust-lang.org/cargo/reference/manifest.html, https://doc.rust-lang.org/cargo/reference/resolver.html
- Clippy dokumentáció: https://doc.rust-lang.org/clippy/
- Rust Edition Guide, Rust 2024 (stabil az 1.85-től): https://doc.rust-lang.org/edition-guide/rust-2024/index.html

A parancsok és a kimenetek a rögzített Rust 1.99.1 toolchainre vonatkoznak (ellenőrizve: 2026. október). Más toolchainnél a verziószámok és a lintüzenetek szövege eltérhet; a `rustup show` és a `cargo --version` mutatja, melyik toolchain aktív.

2.3 Az rkv mintaprojekt indítása: Cargo-workspace és első CLI

2.3.1 Motiváció

Egy C++ projektben, ha egy könyvtárat és több futtatható programot akarunk együtt fejleszteni, általában CMake-lel vagy valamilyen build-generátorral kell több könyvtárat összekapcsolnunk, include path-okat és linkelési beállításokat kezelnünk. A Rust ökoszisztéma ezt a problémát a **Cargo-workspace** fogalmával oldja meg: egyetlen gyökérkönyvtárból több, egymásra hivatkozó `package`-et (csomagot) kezelhetünk, és a fordító, a függőségfeloldás és a tesztelés egyszerre látja az egészet.

Az rkv mintaprojektet most indítjuk el: létrehozzuk a workspace-t, egy `rkv-core` nevű könyvtár-crate-et a tároló logikájával, és egy `rkv-cli` nevű bináris-crate-et, amely parancssori argumentumokból dolgozik. Az első verzió szándékosan egyszerű: memóriában tárol `String` kulcs és `Vec<u8>` érték párokat, a parancsokat kézzel elemezzük, és semmilyen hiba- vagy perzisztenciakezelést nem valósítunk meg. Ez a váz az egész könyv során fokozatosan átalakul majd: megtanuljuk a hibakezeléssel, a modulokkal, a perzisztenciával és a párhuzamos hozzáféréssel foglalkozó fejezetekben.

2.3.2 Fogalmak és szabályok

Definíció. A **Cargo-workspace** olyan könyvtár, amely egy gyökér `Cargo.toml`-ben felsorolja a hozzá tartozó `package`-eket (csomagokat). Minden package saját `Cargo.toml`-lel rendelkezik, és lehet könyvtár (`lib`), bináris (`bin`) vagy mindkettő.

Definíció. A **package** egy fordítási egység a Cargo számára; egy package több **crate**-et is tartalmazhat (például egy `lib`-et és több `bin`-t). A crate a tényleges fordítási egység, amelyre a Rust-kódban hivatkozunk.

Definíció. Az **útvonal-függőség** (path dependency) olyan függőség, amely nem regisztrált csomagforrásból (például crates.io), hanem a fájlrendszer egy relatív vagy abszolút útvonaláról származik. A workspace-en belül ezt a `path = "../rkv-core"` formában adjuk meg.

Szabály. A workspace gyökér `Cargo.toml`-je `[workspace]` szekciót tartalmaz, és `members` kulccsal sorolja fel a csomagok relatív elérési útjait. A tagok egymásra hivatkozhatnak path-függőséggel, és a `cargo` parancsok a gyökérből kiadva az összes tagot látják.

Szabály. A Rust programok parancssori argumentumait a `std::env::args()` függvény szolgáltatja iterátorként. Az első elem a program neve *úgy, ahogy az operációs rendszer átadta*; ez az `argv[0]`-nak felel meg, de nincs garantálva, hogy teljes útvonal, sőt a futtatás módjától függően relatív útvonal vagy csak a hívott név lehet, és az operációs rendszer üres vagy egyedi értéket is adhat. A többi elem a felhasználó argumentuma. Az `args()` pánikol, ha valamelyik argumentum nem érvényes Unicode; ha ez lehetséges, az `std::env::args_os()` `OsString` értékeket ad, panik nélkül. A `cargo run` a munkakönyvtár és a futtatás módja szerint más-más `args[0]`-t kaphat.

2.3.3 C++ és Rust összevetés

C++-ban a több binárist és könyvtárat tartalmazó projekt szerkezete CMake-könyvtárakkal és célokkal épül fel; a Rust ezt natívan, a Cargo workspace-szel oldja meg. A parancssori argumentumok kezelésében is eltér a filozófia: C++-ban az `argc` és `argv` kézzel indexelhető, a Rustban egy iterátorral.

Először nézzük meg, hogyan néz ki a C++-ban a feladat minimális váza:

```cpp
#include <iostream>
#include <map>
#include <string>
#include <vector>

int main(int argc, char* argv[]) {
    std::map<std::string, std::vector<unsigned char>> store;

    if (argc < 2) {
        std::cerr << "Használat: kv <parancs> ...\n";
        return 1;
    }

    std::string cmd = argv[1];
    if (cmd == "set" && argc >= 4) {
        std::string key = argv[2];
        std::string value = argv[3];
        store[key] = std::vector<unsigned char>(value.begin(), value.end());
        std::cout << "OK\n";
    } else if (cmd == "get" && argc >= 3) {
        auto it = store.find(argv[2]);
        if (it != store.end()) {
            std::cout.write(reinterpret_cast<const char*>(it->second.data()),
                            it->second.size());
            std::cout << "\n";
        } else {
            std::cout << "NOTFOUND\n";
        }
    }
}
```

A Rust változat ugyanazt a logikát a `HashMap` és az `args()` iterátor segítségével valósítja meg, de az adatokat típusbiztos módon, `String` kulccsal és `Vec<u8>` értékkel tárolja, és a lekérdezés `Option<&Vec<u8>>`-cal jelzi a hiányzó kulcsot. A C++-os `find()` + `end()` ellenőrzés itt beépül a típusba.

2.3.4 Kidolgozott példa és labor

Ebben a laborban a megadott munkakönyvtárban létrehozzuk az rkv workspace-t, a két csomagot, majd futtatjuk a CLI-t.

Munkakönyvtár: `~/rust-labor/rkv/` (a labor elején hozzuk létre); a kísérletekhez használt egyéb könyvtárak a `~/rust-labor/rNNN/` sémát követik.

2.3.4.1 1. lépés: A workspace gyökér létrehozása

Először hozzuk létre a gyökérkönyvtárat, és abban a gyökér `Cargo.toml`-t:

```
mkdir -p ~/rust-labor/rkv
cd ~/rust-labor/rkv
```

A gyökér `Cargo.toml` tartalma:

Kódblokk — fájl: `Cargo.toml` · művelet: új fájl (workspace gyökér) · ellenőrzés: a 3. lépés után `cargo build --workspace`

```
[workspace]
resolver = "3"
members = [
    "rkv-core",
    "rkv-cli",
]
```

A tagokat a `members` lista sorolja fel. A `resolver` a függőségfeloldó változata: egy csomagnál az edition határozza meg (a 2024-es editionnél ez a `"3"`), a csak `[workspace]` táblát tartalmazó, úgynevezett virtuális workspace-nél viszont nincs edition, ezért itt explicit meg kell adni. A `"3"` a `"2"`-höz képest annyiban több, hogy a függőségek kiválasztásánál a `rust-version` mezőt is figyelembe veszi. Az rkv minimális támogatott toolchainje `rust-version = "1.89"` (a tagok `Cargo.toml`-jában, a 2. lépésben); a `resolver = "3"` mellett a Cargo a függőségeknél is előnyben részesíti az ezt a Rust-verziót támogató kiadásokat, ezért a feloldás tudatosabb, mint a `"2"` mellett. A gyökérmanifest és az edition tehát egyezik: 2024-es edition, `resolver = "3"`.

Ugyanide, a workspace gyökerébe kerül a Eszközlánc: rustup, cargo, rustfmt, clippy, dokumentáció alfejezetben megadott `rust-toolchain.toml` is, így a workspace minden tagja ugyanazzal a toolchainnel fordul. A `cargo` a gyökérben egyetlen közös `Cargo.lock`-ot hoz létre; mivel a workspace binárisokat is tartalmaz, ezt verziókezelés alá vesszük.

2.3.4.2 2. lépés: Az `rkv-core` könyvtár-csomag létrehozása

Várt figyelmeztetés. A gyökér `Cargo.toml`-ban a `members` már az `rkv-cli`-t is felsorolja, de az csak a következő lépésben jön létre. Amíg a könyvtára hiányzik, a Cargo a workspace betöltésekor hibát vagy figyelmeztetést adhat a hiányzó tagra; ez nem a labor hibája. A `cargo new rkv-core` előtt vagy hagyjuk ki az `rkv-cli`-t a listából a 3. lépésig, vagy hozzuk létre mindkét tagot, mielőtt építünk.

A workspace-en belül hozzuk létre a könyvtár csomagot:

```
cargo new rkv-core --lib
cd rkv-core
```

A `cargo new` a rögzített toolchain alapértelmezett, 2024-es editionjét írja be; ezt egészítjük ki a minimális Rust-verzióval. A `Cargo.toml`:

Kódblokk — fájl: `rkv-core/Cargo.toml` · művelet: teljes csere · ellenőrzés: `cargo build -p rkv-core`

```
[package]
name = "rkv-core"
version = "0.1.0"
edition = "2024"
rust-version = "1.89"

[dependencies]
```

A `src/lib.rs` tartalmát cseréljük a következőre:

Kódblokk — fájl: `rkv-core/src/lib.rs` · művelet: teljes csere · ellenőrzés: `cargo build -p rkv-core` → hibátlan fordítás

```
use std::collections::HashMap;

/// Egyszerű memóriabeli kulcs-érték tároló.
///
/// A kulcs `String`, az érték `Vec<u8>`. Ez a választás az egész könyvben
/// érvényben marad: a kulcsok szövegesek, az értékek tetszőleges bájtok
/// lehetnek, ezért nem kényszerítjük őket UTF-8-ba.
#[derive(Debug, Default)]
pub struct Store {
    data: HashMap<String, Vec<u8>>,
}

impl Store {
    /// Üres tárolót hoz létre.
    pub fn new() -> Self {
        Self {
            data: HashMap::new(),
        }
    }

    /// Kulcs-érték pár beillesztése vagy felülírása.
    pub fn set(&mut self, key: String, value: Vec<u8>) {
        self.data.insert(key, value);
    }

    /// Kulcs alapján történő lekérdezés.
    ///
    /// Visszaad egy `Option<&Vec<u8>>` értéket:
    /// - `Some(&Vec<u8>)`, ha a kulcs létezik,
    /// - `None`, ha nem.
    pub fn get(&self, key: &str) -> Option<&Vec<u8>> {
        self.data.get(key)
    }

    /// Kulcs törlése; visszaadja a törölt értéket, ha létezett.
    pub fn delete(&mut self, key: &str) -> Option<Vec<u8>> {
        self.data.remove(key)
    }

    /// Az összes kulcs (rendezetlen) listája.
    pub fn keys(&self) -> Vec<String> {
        self.data.keys().cloned().collect()
    }
}
```

A `Store` típus és a hozzá tartozó `impl` blokk a Structok és metódusok: impl blokkok, asszociált függvények alfejezetben lesz részletesen kibontva. Most csak annyit jegyzünk meg: az `impl Store` blokk a Rust módja az osztálymetódusok definiálására; a `pub` a láthatóságot szabályozza, hasonlóan a C++ `public:` szekciójához, de itt minden elem alapból privát.

2.3.4.3 3. lépés: Az `rkv-cli` bináris-csomag létrehozása

Térjünk vissza a workspace gyökerébe, és hozzuk létre a CLI csomagot:

```
cd ..
cargo new rkv-cli --bin
cd rkv-cli
```

A `Cargo.toml`-ben adjuk meg a `rkv-core` függőséget path-függőségként:

Kódblokk — fájl: `rkv-cli/Cargo.toml` · művelet: teljes csere · ellenőrzés: `cargo build -p rkv-cli`

```
[package]
name = "rkv-cli"
version = "0.1.0"
edition = "2024"
rust-version = "1.89"

[dependencies]
rkv-core = { path = "../rkv-core" }
```

Fontos: a Cargo csomag nevében kötőjel van (`rkv-core`), de a Rust-kódban a crate hivatkozása alávonással történik: `rkv_core`. Ezt a Cargo automatikusan leképezi.

A `src/main.rs` tartalma:

Kódblokk — fájl: `rkv-cli/src/main.rs` · művelet: teljes csere · ellenőrzés: `cargo run -p rkv-cli -- set name Alice` → `OK`

```
use std::env;
use rkv_core::Store;

fn main() {
    let mut store = Store::new();
    let args: Vec<String> = env::args().collect();

    // args[0] a program neve, args[1] a parancs.
    if args.len() < 2 {
        eprintln!("Használat: kv <parancs> [argumentumok]");
        return;
    }

    let cmd = &args[1];
    match cmd.as_str() {
        "set" => {
            if args.len() < 4 {
                eprintln!("Használat: kv set <kulcs> <érték>");
                return;
            }
            let key = args[2].clone();
            let value = args[3].clone().into_bytes();
            store.set(key, value);
            println!("OK");
        }
        "get" => {
```

```
            if args.len() < 3 {
                eprintln!("Használat: kv get <kulcs>");
                return;
            }
            match store.get(&args[2]) {
                Some(bytes) => {
                    // Az érték tetszőleges bájt lehet; itt most szövegként
                    // próbáljuk kiírni, ami UTF-8-ban beválik.
                    match String::from_utf8(bytes.clone()) {
                        Ok(s) => println!("{s}"),
                        Err(_) => println!("BIN ({})", bytes.len()),
                    }
                }
                None => println!("NOTFOUND"),
            }
        }
        "del" => {
            if args.len() < 3 {
                eprintln!("Használat: kv del <kulcs>");
                return;
            }
            match store.delete(&args[2]) {
                Some(_) => println!("OK"),
                None => println!("NOTFOUND"),
            }
        }
        "list" => {
            for key in store.keys() {
                println!("{key}");
            }
        }
        _ => {
            eprintln!("Ismeretlen parancs: {cmd}");
        }
    }
}
```

A szándékosan egyszerű kódban a hiányzó argumentumoknál nem lépünk ki hibakóddal, és a `clone()` hívásokkal a tulajdonjogot kerüljük meg, amit a későbbi fejezetekben a borrowing és a hibakezelés tárgyalásakor javítunk. A `String::from_utf8` és a `bytes.clone()` jelzi, hogy az érték nem feltétlenül szöveg; a `BIN (n)` kiírás a lehetséges bináris tartalmat kezeli.

2.3.4.4 4. lépés: Fordítás és futtatás

A workspace gyökeréből futtassuk a programot:

```
cd ~/rust-labor/rkv
cargo run --bin rkv-cli -- set name Alice
```

Várható kimenet:

```
OK
```

Ezután lekérdezzük:

```
cargo run --bin rkv-cli -- get name
```

Várható kimenet:

```
NOTFOUND
```

Ez nem hiba a kódban, hanem a tároló élettartamának következménye. A `Store` a `main` lokális változója: minden `cargo run` új folyamatot indít, amely üres tárolóval (`Store::new()`) kezd, és kilépéskor a tároló a memóriával együtt megszűnik. Az előző folyamat `set` parancsa tehát a második folyamatban már nem látszik. Ugyanezért a

```
cargo run --bin rkv-cli -- set age 42
cargo run --bin rkv-cli -- list
```

parancspár második tagja üres listát ad: a `list` kimenete egyetlen sor sem. Egy folyamaton belül viszont a tároló működik; ezt a `Store` unit tesztjei a Structok és metódusok: impl blokkok, asszociált függvények alfejezetben ellenőrzik. A folyamatok közötti megmaradáshoz (perzisztencia) fájlba kell írni az állapotot; ezt a Copy, Clone és Drop: másolás és RAII Rustban alfejezet készíti elő, és az I/O és szerializáció: fájlok, pufferelés, serde alfejezet valósítja meg. A `list` kimenete, ha lesz benne kulcs, rendezetlen, mert a `HashMap` nem garantál bejárási sorrendet.

A `--bin rkv-cli` opcióval egyértelműen megadjuk, melyik binárist futtassuk; a `--` előtti részt a Cargo fogyasztja, utána a program argumentumai következnek.

Ellenőrzési pont. A workspace két tagja (`rkv-core`, `rkv-cli`) hiba nélkül fordul (`cargo build --workspace`). Az `rkv-core` a `Store` típust exportálja (`new`, `set`, `get`, `delete`, `keys`); a `cargo run -p rkv-cli -- set name Alice` kimenete `OK`, a külön folyamatban futtatott `get name` kimenete `NOTFOUND`.

2.3.4.5 5. lépés: Ábra a workspace-ről

A létrejött szerkezet rétegábrája:

```
workspace gyökér ([workspace], közös Cargo.lock)
    rkv-cli  (bin)
    rkv-core (lib)

Függőség (egyirányú):  rkv-cli   rkv-core
```

A nyíl a függőséget jelöli: az `rkv-cli` `Cargo.toml`-jában `rkv-core = { path = "../rkv-core" }` szerepel, az `rkv-core`-nak nincs függősége az `rkv-cli`-re. A gyökér `[workspace]` tábla nem függőség, csak a tagok közös gyűjtője.

A fenti szöveges ábrán a workspace gyökér a két tag közös gyűjtője (nem függőség); a nyíl egyirányú: az `rkv-cli` függ az `rkv-core`-tól, fordítva nem. Általánosan: a magasabb szintű bináris az alacsonyabb szintű könyvtárat használja.

2.3.5 Tipikus hibák és átállási csapdák

1. **A workspace gyökeréből kiadott `cargo run` nem találja, vagy nem tudja kiválasztani a binárist.** A Cargo a workspace célpontjai közül választ: ha a tagok között pontosan egy bináris cél van (mint az induló kétcsomagos workspace-ben, ahol csak az `rkv-cli` bináris), a gyökérből kiadott `cargo run -- ...` is működik. Ha nincs bináris cél, a Cargo `no bin target` jellegű hibát ad; ha kettő vagy több van, nem tudja, melyiket indítsa. Ilyenkor a `--bin rkv-cli` vagy a `-p rkv-cli` kapcsolóval nevezzük meg, ahogy a laborban tettük.
2. **A `std::env::args()` első elemével való elszámolás.** C++-ban az `argv[1]` az első felhasználói argumentum, a Rustban az `args().nth(1)` adja ugyanezt. Ha valaki elfelejti, hogy az iterátor első eleme a program neve, elcsúsznak az indexek, és hibás parancsértelmezést kapunk.

3. **A path-függőség kihagyása.** Ha a `rkv-cli` `Cargo.toml`-jéből hiányzik a `rkv-core = { path = "../rkv-core" }` bejegyzés, a `use rkv_core::Store;` sor fordítási hibát ad, mert a crate nem található. A Cargo pontos hibaüzenete a függőség hiányát jelzi, de a kezdő számára nem mindig egyértelmű, hogy a workspace-en belül is explicit path-függőséget kell bejegyezni.

4. **A `HashMap` sorrendjének feltételezése.** A `list` parancs a kulcsokat véletlenszerű sorrendben adhatja vissza, mert a `HashMap` a hash-alapú tárolás miatt nem őrzi meg a beillesztés vagy a rendezettség sorrendjét. Aki C++-ból `std::map`-hez van szokva, meglepődhet; ott a `std::map` rendezett, de a `std::unordered_map` nem. A Rustban a rendezett változat a `BTreeMap`, amelyet a Gyűjtemények: Vec, HashMap, BTreeMap és társaik; az Entry API alfejezet tárgyal.

2.3.6 Feladatok

2.3.6.1 1. Feladat (fogalmi)

Melyik állítás igaz a Cargo-workspace-re?

a) A workspace minden csomagja egyetlen közös `Cargo.toml`-ben definiálódik.

b) A workspace csak könyvtár-csomagokat tartalmazhat.

c) A workspace gyökér `Cargo.toml`-je felsorolja a tagokat, és minden tagnak saját `Cargo.toml`-je van, amelyben path-függőséggel hivatkozhat egymásra.

d) A workspace tagjai nem hivatkozhatnak egymásra.

Válasz: c.

2.3.6.2 2. Feladat (C++-kód átírása)

Adott egy egyszerű C++ kódrészlet:

```
void process(std::map<std::string, std::string>& store,
             const std::string& cmd, const std::string& key,
             const std::string& value) {
    if (cmd == "put") {
        store[key] = value;
    } else if (cmd == "fetch") {
        auto it = store.find(key);
        if (it != store.end()) {
            std::cout << it->second;
        }
    }
}
```

Írd át a fenti logikát úgy, hogy az általunk készített `Store` típust használja, és az értékeket `String` helyett `Vec<u8>`-ként tárolja. A `put` parancs `store.set(key, value.into_bytes())`-tel, a `fetch` pedig `store.get(&key)`-vel valósuljon meg; a hiányzó kulcs esetén írj ki `NOTFOUND`-ot.

2.3.6.3 3. Feladat (implementáció)

Bővítsd az `rkv-cli`-t egy `clear` paranccsal, amely törli az összes kulcs-érték párt. Ehhez először a `Store`-hoz adj egy `clear(&mut self)` metódust, amely kiüríti a `data` `HashMap`-et (használhatod a `HashMap::clear()`-t). A CLI-ben a `clear` parancs ne várjon argumentumot, törölje a tárolót, és írjon ki `OK`-t.

2.3.6.4 4. Feladat (hibajavítás/refaktorálás)

Az aktuális `main.rs` minden parancsot ugyanabban a függvényben kezel, és a hibás argumentumszámoknál nem ad vissza hibakódot. Refaktoráld a kódot úgy, hogy a parancsfeldolgozást kiemeled egy külön `process_command(store: &mut Store, args: &[String])` függvénybe, és a hibás argumentumszámoknál a függvény `Result<(), String>`-et adjon vissza, amelyet a `main` kiír, és kilépési kódot ad. (A `Result` típusról a Result, a ? operátor és a panic helye alfejezet szól majd részletesen; most elég, ha `Err`-rel jelzed a hibát.)

2.3.7 Megoldások

2.3.7.1 1. Megoldás

A helyes válasz c. A workspace gyökér `Cargo.toml`-je `[workspace]` szekcióval és `members` listával definiálja a tagokat; minden tagnak külön `Cargo.toml`-je van, és a path-függőségekkel kapcsolódnak.

2.3.7.2 2. Megoldás

A Rust megfelelő:

```
use rkv_core::Store;

fn process(store: &mut Store, cmd: &str, key: &str, value: &str) {
    match cmd {
        "put" => {
            store.set(key.to_string(), value.to_string().into_bytes());
        }
        "fetch" => {
            match store.get(key) {
                Some(bytes) => {
                    if let Ok(s) = String::from_utf8(bytes.clone()) {
                        println!("{s}");
                    } else {
                        println!("BIN ({})", bytes.len());
                    }
                }
                None => println!("NOTFOUND"),
            }
        }
        _ => {}
    }
}
```

A `store.set` a `String` kulcsot és a `Vec<u8>` értéket vár; a `value.into_bytes()` a `String`-et `Vec<u8>`-tá alakítja. A `get` `Option`-t ad, ezért a `match` a hiányzó kulcsot is kezeli.

2.3.7.3 3. Megoldás

A `Store` bővítése:

```
impl Store {
    // ... korábbi metódusok ...

    /// Minden kulcs-érték pár törlése.
    pub fn clear(&mut self) {
        self.data.clear();
    }
}
```

A `main.rs`-ben új ág a `match cmd.as_str()`-ben:

```
"clear" => {
    store.clear();
    println!("OK");
}
```

2.3.7.4 4. Megoldás

A `process_command` függvény:

```
fn process_command(store: &mut Store, args: &[String]) -> Result<(), String> {
    let cmd = args.get(0).ok_or("Hiányzó parancs")?;
    match cmd.as_str() {
        "set" => {
            let key = args.get(1).ok_or("Hiányzó kulcs")?;
            let value = args.get(2).ok_or("Hiányzó érték")?;
            store.set(key.clone(), value.clone().into_bytes());
            println!("OK");
        }
        "get" => {
            let key = args.get(1).ok_or("Hiányzó kulcs")?;
            match store.get(key) {
                Some(bytes) => {
                    if let Ok(s) = String::from_utf8(bytes.clone()) {
                        println!("{s}");
                    } else {
                        println!("BIN ({})", bytes.len());
                    }
                }
                None => println!("NOTFOUND"),
            }
        }
        "del" => {
            let key = args.get(1).ok_or("Hiányzó kulcs")?;
            match store.delete(key) {
                Some(_) => println!("OK"),
                None => println!("NOTFOUND"),
            }
        }
        "list" => {
            for key in store.keys() {
                println!("{key}");
            }
        }
        "clear" => {
            store.clear();
            println!("OK");
```

```
        }
        other => return Err(format!("Ismeretlen parancs: {other}")),
    }
    Ok(())
}
```

A `main` ekkor:

```
fn main() {
    let args: Vec<String> = env::args().collect();
    let mut store = Store::new();
    // Az args[1..] tartalmazza a parancsot és az argumentumait.
    if let Err(err) = process_command(&mut store, &args[1..]) {
        eprintln!("{err}");
        std::process::exit(1);
    }
}
```

Ez a kód a `Result` segítségével jelzi a hibákat, és a hibás parancsoknál a program kilépési kódja 1 lesz; a későbbi fejezetekben ezt a hibakezelést a `?` operátorral és saját hibatípussal finomítjuk.

2.3.8 Rövid összefoglaló

Létrehoztuk az rkv Cargo-workspace-t a `~/rust-labor/rkv/` könyvtárban, amely két csomagot tartalmaz: az `rkv-core` könyvtárat (crate név: `rkv_core`) és az `rkv-cli` binárist. A `Store` típus (fájl: `rkv-core/src/lib.rs`) egy `HashMap<String, Vec<u8>>`-at foglal magába, és `new`, `set`, `get`, `delete`, `keys`, valamint a feladatban hozzáadott `clear` metódusokat kínálja. A `rkv-cli` bináris (fájl: `rkv-cli/src/main.rs`) a `std::env::args()` segítségével elemzi a parancsokat (`set`, `get`, `del`, `list`, `clear`), és a `Store` API-t hívja. A workspace-hez a gyökér `Cargo.toml` `[workspace]` szekciója és a tagok `members` listája tartozik; a `rkv-cli` path-függőséggel hivatkozik a core csomagra. **Továbbvitt eredmények:** a `Store` struct és az `impl` blokk a Structok és metódusok: impl blokkok, asszociált függvények alfejezetben kerül mélyebb elemzésre; a modulok és láthatóság a Modulok és láthatóság: header nélküli világ alfejezetben finomodik; a hibakezelés hiányosságait a Result, a ? operátor és a panic helye alfejezet javítja; végül a `HashMap` használatát a Gyűjtemények: Vec, HashMap, BTreeMap és társaik; az Entry API alfejezet tanítja meg részletesen.

2.3.9 Források és ellenőrzési hiányok

- The Cargo Book, „Workspaces” és „Dependency Resolution” (a `resolver = "3"` az 1.84-től stabil, a 2024-es edition alapértelmezése): https://doc.rust-lang.org/cargo/reference/workspaces.html, https://doc.rust-lang.org/cargo/reference/resolver.html
- The Rust Programming Language, 14.3 „Cargo Workspaces” és 12.1 „Accepting Command Line Arguments”: https://doc.rust-lang.org/book/
- `std::env::args` és `std::collections::HashMap` dokumentáció: https://doc.rust-lang.org/std/env/fn.args.html, https://doc.rust-lang.org/std/collections/struct.HashMap.html. A `HashMap` bejárási sorrendje nem meghatározott, ezért a `list` parancs kimenete nem rendezett.

A hibaüzenetek szövege a rögzített Rust 1.99.1 toolchainre vonatkozik.

II. RÉSZ – Az alapok C++-os szemmel

3 Szintaxis, típusok, kifejezések

3.1 Változók, mutabilitás, skalár típusok és konverziók

3.1.1 Motiváció

Egy C++-fejlesztő számára a mutabilitás és az implicit konverzió a mindennapi rutin része: `int x = 10; x++;` magától értetődő, és a `uint8_t b = 300;` is lefordul (legfeljebb figyelmeztetéssel), csendben 44-re csonkolva, az `int` túlcsordulása pedig egyenesen definiálatlan viselkedés. A Rust ezeken a pontokon azonnal feszegeti a beidegződéseket: a kötés alapértelmezetten nem változtatható, a típusok között nincs néma átjárás, a túlcsordulás pedig nem „csak egy lehetséges hibafajta”, hanem szabályozott, ellenőrizhető művelet. Ez az alfejezet a nyelv legalapvetőbb építőköveit mutatja be úgy, hogy a C++-párhuzamot minden ponton explicit módon kimondja.

A cél nem a szintaxis bemagolása, hanem annak megértése, hogy a Rust miért kényszeríti ki a hosszabb kódot: mert a hosszabb kód a hibát a fordítás vagy a teszt idejére hozza előre, nem az üzembe helyezés utáni hibakeresésre.

3.1.2 Fogalmak és szabályok

3.1.2.1 `let`, `let mut` és shadowing

Definíció. A `let` kötés (binding) egy nevet és egy értéket kapcsol össze. Az így létrehozott kötés alapértelmezetten immutábilis, azaz a névhez rendelt érték a kötés élettartama alatt nem cserélhető. A `let mut` mutábilis kötést hoz létre, amelynél az `=` operátorral új érték rendelhető a névhez.

Szabály. A `let`/`let mut` azt dönti el, hogy a *kötés által elért tulajdonolt hely* közvetlenül újraírható-e. A C++ `const` a típus minősítője, és ott az állandóság a kivétel; Rustban az alapértelmezés a változtathatatlan kötés, és a változtatáshoz kifejezetten `let mut` kell. Három dolgot érdemes szétválasztani: (1) a **tulajdonolt hely** közvetlen újraírása (`x = 20`, `x += 1`) a `mut` kötést igényli; (2) a **referencia típusa** (`&T` vagy `&mut T`) azt mondja meg, hogy a hivatkozott értéket szabad-e módosítani; ehhez a `&mut` kölcsönzéshez maga az eredeti kötésnek is `mut`-nak kell lennie; (3) a **belső mutabilitás** (`Cell`, `RefCell`, `Mutex`, atomikus típusok; ezeket a Belső mutabilitás: Cell, RefCell és a futásidejű kölcsönzés és a Mutex, RwLock, csatornák és a megosztott rkv alfejezet tárgyalja) olyan kivétel, ahol megosztott (`&T`) hivatkozáson keresztül is változtatható az érték, mert a szabályt futásidőben vagy atomikus műveletekkel tartják be.

```
let x = 10;
// x = 20; // E0384: cannot assign twice to immutable variable

let mut y = 10;
y = 20; // rendben
```

A `&mut` hivatkozáson keresztüli módosításhoz is `mut` kötés kell az eredeti változón:

```rust
let mut x = 10;
let r = &mut x; // változtatható kölcsönzés: x-nek mut kötésnek kell lennie
*r += 1;
println!("{x}"); // 11
// let z = 5; let q = &mut z; // E0596: cannot borrow `z` as mutable
```

A shadowing nem összekeverendő a mutabilitással.

Definíció. A shadowing (leárnyékolás) ugyanazzal a névvel létrehozott új `let` kötés, amely a hatókörében eltakarja a korábbit: ugyanabban a scope-ban vagy egy belső blokkban is. Ugyanazon scope-ban a régi kötés a shadowing után már nem elérhető; belső blokkban a külső kötés a blokk után újra elérhető, az új kötés értéke és típusa is eltérhet. A shadowing nem módosítja a régi értéket, hanem új kötést hoz létre.

```rust
let width = 100;        // i32
let width = width as u8; // u8, új kötés

let mut value = 42;
value += 1;
let value = value;      // a továbbiakban immutábilis kötés
```

A `let mut` esetén a típus rögzített marad:

```rust
let mut counter = 0u32;
counter = 10u32;
// counter = 10u64; // típushiba, a mutabilitás nem típusváltás
```

A `const` és a `static` a C++ globális konstansainak és globálisainak felel meg, de a Rust-szabályok mások.

Szabály. A `const` elem fordítási idejű konstans, amelyet a felhasználási helyen beépít a fordító; a `static` elemnek rögzített memóriahelye van. A `const` deklaráció típusannotációt igényel, a `static` mutábilis változatát (`static mut`) az `unsafe` szabályai miatt el kell kerülni.

```rust
const MAX_RETRIES: u32 = 3;
static APP_NAME: &str = "rkv";
```

A C++-ban a `const int max_retries = 3;` és a `constexpr int max_retries = 3;` közötti különbség Rustban nincs jelen ebben a formában: a `const` maga a fordítási idejű konstans.

3.1.2.2 Skalár típusok

A Rust skalár típusai a C++ fejlesztő számára többnyire ismerősek, de a nevek és a `char` jelentése pontosan tisztázandó.

Rust	C++ közeli megfelelő	Megjegyzés
`bool`	`bool`	`true` és `false`
`i8`, `i16`, `i32`, `i64`, `i128`, `isize`	`std::int8_t`, …	`isize` pointer méretű előjeles típus
`u8`, `u16`, `u32`, `u64`, `u128`, `usize`	`std::uint8_t`, …	`usize` pointer méretű előjel nélküli típus
`f32`, `f64`	`float`, `double`	az alapértelmezett lebegőpontos típus `f64`

Rust	C++ közeli megfelelő	Megjegyzés
`char`	nincs közvetlen megfelelő	Unicode scalar value, nem `char` és nem `wchar_t`

Az egész literálok típusa alapértelmezés szerint `i32`, hacsak a kontextus vagy egy típusutótag mást nem diktál. A `42u8` utótaggal `u8` jön létre. A `usize` és `isize` pointer méretű, Linux 64 bites platformon 64 bites, ezért a gyűjtemények indexelése `usize` típust igényel; a C++ `size_t` és `ptrdiff_t` párja.

A Rust `char` négy bájtot foglal, és egy Unicode scalar value-t tárol. Ez nem a C++ `char`, amely egy bájt, és nem is a `wchar_t`, amely platformfüggő. A `char` például `'€'`, `' '` és `'a'` is lehet. A UTF-8 kódolás miatt ugyanazon karakter bájtokban mért hossza változó; a `char::len_utf8()` ezt adja meg.

3.1.2.3 Túlcsordulás: debug panic, release wrapping

A C++ előjeles egész túlcsordulása definiálatlan viselkedés, ezért a fordító optimalizálhat arra, hogy nem következik be. A Rust ezt a területet is szabályozza.

Szabály. Egész típusú túlcsordulás esetén a `+`, `-`, `*` és hasonló operátorok viselkedését a Cargo `overflow-checks` profilbeállítása dönti el. **Az alapértelmezett profilokban** a `dev` (debug) profilnál az ellenőrzés be van kapcsolva, ezért a túlcsordulás panicet okoz (`attempt to add with overflow`); a `release` profilnál ki van kapcsolva, ezért kétkomplements wrapping történik. Mindkettő *definiált* viselkedés (szemben a C++ előjeles túlcsordulásával), de a `Cargo.toml`-ban az `overflow-checks` átírható, tehát a szabály az alapértelmezett profilokra érvényes. Ha az érték már fordítási időben ismert, a fordító hibát ad, profiltól függetlenül (`arithmetic_overflow` lint). A viselkedés explicit módszerekkel buildtípustól függetlenül szabályozható.

A három eset külön jelölve (a kimenetek Rust 1.99.1-ből):

```
// 1. FORDÍTÁSI HIBA (debug és release is): error: this arithmetic operation will overflow
// let x = i32::MAX; let y = x + 1;
// let b: u8 = 256; // error: literal out of range for `u8`

// 2. FUTÁSI PANIC debugban, WRAPPING releaseben: a fordító nem tudja előre
fn bump(n: i32) -> i32 {
    n + 1 // n = i32::MAX esetén: debug panic, release -2147483648
}

// 3. DEFINIÁLT, build-független viselkedés explicit módszerrel
let m = i32::MAX;
let a = m.checked_add(1);    // None
let b = m.wrapping_add(1);   // -2147483648
let c = m.saturating_add(1); // 2147483647
```

Kifejezés	Eredmény
`200u8.checked_add(100)`	`None`
`200u8.wrapping_add(100)`	`44`
`200u8.saturating_add(100)`	`255`
`200u8.overflowing_add(100)`	`(44, true)`

A `checked_*` `Option<T>`-t ad vissza: `Some` eredményt, ha nincs túlcsordulás, különben `None`-t. A `wrapping_*` mindig kétkomplements wrappinget végez. A `saturating_*` a típus alsó vagy

felső határán telíti az eredményt. Az `overflowing_*` visszaadja a wrapped értéket és egy `bool`-t, amely jelzi a túlcsordulást. A `checked_*`, `wrapping_*`, `saturating_*` és `overflowing_*` minden egész típuson elérhetők, és a hívó dönti el, mi történjen túlcsorduláskor.

3.1.2.4 Nincs implicit konverzió

Szabály. A Rust nem végez implicit numerikus konverziót. Minden konverzió kifejezett: `as`, `From`/`Into` vagy `TryFrom`/`TryInto`.

Az `as` operátor C++-os értelemben explicit cast, és *veszteséges, nem ellenőrzött szűkítésre* is képes: szűkítésnél csonkol, előjelváltásnál kétkomplements átalakítást végez, lebegőpontos érték egész típusra való konverziójánál (`f64 as i32`) telít (a NaN `0` lesz); lebegőpontos típusok közötti `as` (például `f64 as f32`) kerekít, nem telít. Fontos: ez **nem** `unsafe` művelet. Az `as` teljesen safe Rust, használatához nincs szükség `unsafe` blokkra; a „nem biztonságos” itt a *számérték megőrzésére* vonatkozik, nem a memóriabiztonságra.

```
let x: i32 = 10;
let y: i64 = x as i64;        // rendben, kiterjesztés
let z: u8 = 300u16 as u8;     // 44, mert 300 - 256 = 44
let w: u32 = -1i32 as u32;    // 4294967295
```

A `From`/`Into` az olyan átalakításokra való, amelyek sosem hibáznak és az értéket megőrzik (értékmegőrző konverziók); a reprezentáció azért megváltozhat: az `f32::from(7i16)` létezik, mert minden `i16` pontosan ábrázolható `f32`-ben, az `f32::from(i32)` viszont nincs, mert nem minden `i32` az. A `From` tehát nem általános „matematikai veszteségmentesség”, hanem az adott típuspárra a standard könyvtár által vállalt, hibamentes értékmegőrzés. Fordítási időben eldől, hogy létezik-e az implementáció. A `TryFrom`/`TryInto` fallible konverziókra való, és `Result`-ot ad vissza. A 2021-es vagy újabb editionben a `TryFrom` és `TryInto` a prelude részét képezi, korábbi kiadásokban `use std::convert::TryFrom;` kell.

```
let big: u32 = u32::from(10u8);                 // szélesítés, mindig sikerül
let fits: Option<u16> = u16::try_from(300u32).ok(); // Some(300): belefér
let small: Option<u8> = u8::try_from(300u32).ok();  // None: u8-ba nem fér
```

A `From`/`Into`, valamint a `TryFrom`/`TryInto` igazi ereje a generikus határokban lesz; ezt a Trait-ek és generikusok: sablonok és concepts helyett alfejezetben fejtjük ki.

3.1.2.5 Típuskikövetkeztetés

A Rust típuskikövetkeztetése hasonlít a C++ `auto` mechanizmusára, de a numerikus literálok esetén van egy fontos különbség. A `let x = 42;` `i32`-t ad, a `let y = 3.14;` `f64`-et, mert ezek az alapértelmezett típusok. A literál típusát azonban a kontextus is befolyásolhatja:

```
let x: u8 = 10;
let y = x + 20; // y: u8, mert a kontextusból a 20 is u8-ra kényszerül
```

C++-ban az `auto x = 42;` mindig `int`-et ad, függetlenül a későbbi használattól. Rustban a literál típusa a kötés annotációjából, a függvényhatárokból és a műveletben részt vevő többi típusból vezethető le.

3.1.3 C++ és Rust összevetés

A kiinduló C++-kód természetesnek tűnik, de implicit konverziót, definiálatlan viselkedést és alulbecsült `const` használatot is tartalmaz:

```cpp
#include <cstdint>
#include <iostream>
#include <limits>

int main() {
    int max = 100;
    max = max + 1;                  // ok, de a mutabilitás alapértelmezett

    const int kMaxRetries = 3;      // az állandóság a kivétel

    uint8_t byte = 300;             // implicit szűkítés: 300 mod 256 = 44,
                                    // jól definiált, de csendes adatvesztés
    int overflow = std::numeric_limits<int>::max();
    overflow += 1;                  // előjeles túlcsordulás: definiálatlan viselkedés

    std::cout << byte << "\n";      // karakterként írja ki!
}
```

Három különböző szabály dolgozik itt, amelyeket C++-ban is érdemes szétválasztani. (1) **Konverzió előjel nélküli céltípusra:** a szabvány szerint az eredmény a forrásérték modulo 2^n, ahol n a céltípus bitszáma; a `uint8_t byte = 300` tehát garantáltan 44, se nem UB, se nem implementációfüggő (csak a fordító figyelmeztethet a szűkítésre). (2) **Előjeles aritmetikai túlcsordulás:** az `INT_MAX + 1` definiálatlan viselkedés. (3) **Egészpromóció:** a `int`-nél kisebb típusok (például `uint8_t`, `uint16_t`) aritmetikája előbb `int`-re léptetve történik, ezért két `uint16_t` összege maga nem csordul túl, csak a visszakonvertálás csonkol (az (1) szabály szerint). Az előjeles céltípusra végzett, nem ábrázolható konverzió C++20 óta szintén modulo-definiált; korábban implementációfüggő volt.

A Rust-megfelelő minden kockázatot fordítási vagy futási időben kezel:

```rust
fn main() {
    let max = 100;                  // i32, immutábilis kötés
    let max = max + 1;              // shadowing: új kötés, nem mutáció

    const MAX_RETRIES: i32 = 3;

    // let byte: u8 = 300;          // fordítási hiba: literal out of range
    let byte: u8 = 300u16 as u8;    // explicit csonkolás, 44
    println!("{byte}");             // 44, nem karakter

    let overflow = i32::MAX;
    // let overflow = overflow + 1; // FORDÍTÁSI HIBA: arithmetic_overflow (profiltól független)
    let overflow = overflow.saturating_add(1); // definiált: 2147483647
}
```

A leglényegesebb különbség három pontban fogalmazható meg: a Rustban az immutabilitás az alap, a konverzió explicit, a túlcsordulás pedig definiált viselkedés, amelyet a buildprofil és a választott metódus határoz meg.

3.1.4 Kidolgozott példa: labor a `r004` munkakönyvtárban

Ez a labor önállóan futtatható, nem módosítja az rkv workspace-t. A cél a shadowing, az explicit konverziók és a túlcsordulás-kezelő metódusok kipróbálása.

Munkakönyvtár:

```
mkdir -p ~/rust-labor
cd ~/rust-labor
cargo new r004
cd r004
```

A `src/main.rs` tartalma:

```
fn main() {
    // Shadowing: típusváltás új kötéssel, nem mutáció.
    let limit = 100;                // i32
    let limit = limit as u16;       // új kötés, u16
    println!("limit = {limit}");

    // Túlcsordulás ellenőrzött, wrapping, saturating módszerekkel.
    let a: u16 = u16::MAX;
    let wrapped = a.wrapping_add(1);
    let checked = a.checked_add(1);
    let saturated = 250u8.saturating_add(20);

    println!("wrapped = {wrapped}");
    println!("checked = {checked:?}");
    println!("saturated = {saturated}");

    // Explicit konverziók: as és TryFrom.
    let narrowed: u8 = 300u16 as u8;        // 44: csonkolás
    let fits = u16::try_from(300u32);       // Ok(300): a 300 belefér az u16-ba
    let too_big = u8::try_from(300u32);     // Err: az u8 legfeljebb 255
    let at_max = u16::try_from(u32::from(u16::MAX));     // Ok(65535)
    let over_max = u16::try_from(u32::from(u16::MAX) + 1); // Err

    println!("narrowed = {narrowed}");
    println!("fits = {fits:?}");
    println!("too_big = {too_big:?}");
    println!("at_max = {at_max:?}, over_max = {over_max:?}");

    // char = Unicode scalar value, nem bájt.
    let ch = '€';
    println!("{ch} UTF-8 hossza: {}", ch.len_utf8());
}
```

A futtatás és a várt kimenet:

```
cargo run
```

```
limit = 100
wrapped = 0
checked = None
saturated = 255
narrowed = 44
fits = Ok(300)
too_big = Err(TryFromIntError(PosOverflow))
at_max = Ok(65535), over_max = Err(TryFromIntError(PosOverflow))
€ UTF-8 hossza: 3
```

A `checked_add(1)` `None` lesz, mert `u16::MAX` után nincs értelmezhető `u16` eredmény. A `300u16 as u8` csonkolása 44. (A `TryFromIntError` `Debug`-kimenetének belső része, itt `PosOverflow`, toolchainenként eltérhet; a kimenet a Rust 1.99.1-é.) A `TryFrom` ugyanerre a szűkítésre (`u8::try_from(300u32)`) hibát ad ahelyett, hogy csonkolna, a 300 viszont belefér az `u16`-ba, ezért ott a konverzió sikeres. A határon az `u16::MAX` még átalakítható, az eggyel nagyobb érték

már nem. A `€` karakter Unicode scalar value-ként `char` típusú, UTF-8 kódolásban 3 bájt, ezért a `len_utf8()` 3-at ad.

A debug és release közötti különbséget a következő függvénnyel lehet kitapintani:

```
fn overflow_demo(a: u8) -> u8 {
    a + 100
}

fn main() {
    println!("{}", overflow_demo(200));
}
```

Debug buildben a `a + 100` kifejezés `200 + 100` esetén panicol: `attempt to add with overflow`. Release buildben ugyanez a hívás `44`-et ad, mert a `u8` túlcsordulás wrappinget végez. A C++ a `uint8_t` esetén hasonló csonkolást adna, de a Rust ezt a buildprofil alapján eltérően kezeli, és a két üzemmód közötti különbség szándékos.

3.1.5 Tipikus hibák és átállási csapdák

1. A `const`-reflex érvényesítése `let mut` nélkül. A C++ fejlesztő a Rustban is minden változót `let mut`-ként akar létrehozni, mert a C++-ban a `const` a kivétel. Ez a Rustban felesleges mutabilitást visz be, és elrejti a szándékot. A helyes gyakorlat: először `let`, és csak akkor `mut`, ha a kód ténylegesen írja a kötést.

2. Shadowing használata mutáció helyett. A `let x = 5; let x = x + 1;` nem ugyanaz, mint a `x += 1`. Az előbbi új kötést hoz létre, az utóbbi a meglévőt módosítja. Ha a szándék a „változó” fogalom, akkor `let mut` kell; ha a szándék a „új érték, új típus”, akkor shadowing.

3. Az `as` mint „biztonságos” implicit konverzió. A `as` nem biztonságos, mert csonkol és előjelváltásnál nem őrzi az előjelet. Például `-1i32 as u32` 4294967295 lesz, nem hiba. Ha a konverzió lehetetlen kimenetele is elfogadhatatlan, `TryFrom`-ot kell használni.

4. A release wrapping elfelejtése. Debug buildben a túlcsordulás panicet okoz, ezért a fejlesztő azt hiheti, hogy a kód soha nem csordul túl. Release buildben ugyanez a kód wrappinget ad, és csendes adatsérülést okozhat. A domain-invariánsokat `checked_*` vagy `saturating_*` hívásokkal kell kifejezni, nem a buildprofilra hagyatkozni.

5. A `char` bájtként kezelése. A `char` nem bájt, hanem Unicode scalar value. A `char` és a `u8` között nincs implicit konverzió; ha bájtokkal kell dolgozni, `u8` vagy `&str` / `String` kell, nem `char`.

3.1.6 Feladatok

1. Feladat (lefordul-e és miért?). Döntse el mindkét kódrészletről, hogy lefordul-e, és ha nem, miért:

```
let x = 10;
x = 20;

let x = 10;
let x = x + 1.5;
```

2. Feladat (C++-kód átírása idiomatikus Rustra). A következő C++ függvény implicit konverziót és hallgatólagos túlcsordulást rejt:

```cpp
#include <cstdint>

uint16_t add_pair(uint16_t a, uint16_t b) {
    return a + b; // implicit int promóció, majd narrowing
}
```

Írj Rust függvényt, amely `Option<u16>` visszatérési értékkel jelzi a túlcsordulást, és indokold, miért jobb ez a C++ csendes csonkolásánál.

3. Feladat (implementációs feladat: hosszmezős keret). Az rkv szöveges, soralapú protokollt használ (lásd Az rkv hálózati szerver: TCP, protokoll, kecses leállás), de sok bináris protokoll hosszmezővel keretez: a keret első bájtja a payload hossza, utána pontosan ennyi bájt következik. Készíts `fn frame(payload: &[u8]) -> Option<Vec<u8>>` függvényt, amely `u8` hosszmezős keretet épít, és `None`-t ad, ha a payload nem fér bele. Miért hibás itt a hossz telítése (`255`-re vágása)? Írj unit tesztet 255 és 256 bájtos payloadra.

4. Feladat (hibajavítás és tervezési döntés). Az alábbi Rust-kód nem fordul le:

```rust
let length: u32 = 1024;
let small: u16 = length; // típushiba
```

Javítsd a kódot kétféleképpen: `as` operátorral és `TryFrom`-mal. Melyik megoldást választanád biztonsági szempontból nem megbízható bemenet esetén, és miért?

3.1.7 Megoldások

1. Megoldás. Az első kódrészlet nem fordul le: `x` immutábilis kötés, ezért `x = 20;` E0384 hibát ad. A második kódrészlet szintén nem fordul le: `x i32` típusú, a `1.5 f64` típusú, és a `+` operátor nem konvertálja implicit módon a típusokat; mismatched types hibát ad.

2. Megoldás.

```rust
fn add_pair(a: u16, b: u16) -> Option<u16> {
    a.checked_add(b)
}
```

A `checked_add` a Rust szintjén ad vissza `None`-t túlcsorduláskor, ahelyett, hogy a hívó hallgatólagosan megkapná a csonkolt eredményt. A C++-ban az `a + b` előbb `int`-re lépteti mindkét operandust (egészpromóció), az összeg ott nem csordul túl; a visszatéréskor végzett `uint16_t`-re konverzió viszont modulo 2^{16} csonkol. Ez jól definiált, nem UB és nem implementációfüggő, de csendes adatvesztés: `add_pair(65535, 1)` eredménye 0. A Rustban a típus kényszeríti ki, hogy a hívó foglalkozzon a hibával.

3. Megoldás.

```rust
fn frame(payload: &[u8]) -> Option<Vec<u8>> {
    // Hibát jelző konverzió: 255 bájt fölött nincs keret.
    let len = u8::try_from(payload.len()).ok()?;
    let mut out = Vec::with_capacity(1 + payload.len());
    out.push(len);
    out.extend_from_slice(payload);
    Some(out)
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
```

```
    fn length_field_matches_payload() {
        let max = vec![7u8; 255];
        let f = frame(&max).unwrap();
        assert_eq!(f[0], 255);
        assert_eq!(f.len() - 1, 255); // a deklarált hossz = az elküldött bájtok
        assert_eq!(frame(&[7u8; 256]), None); // nincs csendes levágás
        assert_eq!(frame(&[]), Some(vec![0]));
    }
}
```

A telítés (`u8::try_from(len).unwrap_or(u8::MAX)`) itt hibás terv: egy 256 bájtos payloadnál a hosszmező 255-öt mondana, a keretben mégis 256 bájt következne, így a fogadó egy bájttal később kezdené a következő keretet, és a keretezés szétcsúszna. Hosszmezőnél a túl nagy üzenetet el kell utasítani (vagy nagyobb hosszmezőt, például `u32`-t kell választani). A telítés ott helyes, ahol a határérték maga is értelmes eredmény, például egy hangerő- vagy progresszjelzőnél. A `?` operátor az `Option`-on itt csak annyit tesz, hogy `None` esetén azonnal visszatér; részletesen a Result, a ? operátor és a panic helye alfejezet tárgyalja.

Továbbvitt eredmény. Az itt bemutatott explicit konverziós és túlcsordulás-kezelési minták alapozzák meg a Saját hibatípusok: enum-hibák, thiserror, anyhow alfejezetben az `Error` típusra épülő konverziókat, valamint a Trait-ek és generikusok: sablonok és concepts helyett alfejezetben a `From`/`Into` és `TryFrom`/`TryInto` implementációit. Az elv: ahol a csonkolás vagy a telítés adatot hamisítana, ott hibát jelző `TryFrom` kell.

4. Megoldás.

```
let length: u32 = 1024;

let small_as: u16 = length as u16;
let small_try: u16 = u16::try_from(length).unwrap_or(u16::MAX);
```

Az `as` verzió egyszerű, de csonkol; a `TryFrom` verzió explicit módon kezeli a hibát. Nem megbízható bemenet esetén a `TryFrom` a jobb választás, mert a hívó kénytelen dönteni a hiba lekezeléséről, ahelyett, hogy a csonkolt érték csendben tovább szivárogna. A `TryFrom` `Err` ága később a `?` operátorral a teljes hibatípusba is átfordítható.

3.1.8 Rövid összefoglaló

Az `r004` alfejezetben a Rust kötései, skalár típusai, túlcsordulási szabályai és konverziói kerültek terítékre. A `let` alapértelmezetten immutábilis, a `let mut` kivétel; a shadowing új kötést hoz létre, és típusváltozást is lehetővé tesz. Az egész típusok a `debug` buildben panicelnek túlcsordulláskor, `release` buildben kétkomplements wrappinget végeznek; a `checked_*`, `wrapping_*`, `saturating_*` és `overflowing_*` metódusok buildtípustól függetlenül vezérlik ezt a viselkedést. A Rust nem végez implicit numerikus konverziót: az `as` explicit, de nem biztonságos, a `From`/`Into` veszteségmentes, a `TryFrom`/`TryInto` fallible konverziót ad. A `char` Unicode scalar value, nem bájt. A laborban a `r004` crate mutatta be a műveleteket; a `frame` feladat azt mutatta meg, hogy hosszmezős keretezésnél a túl nagy értéket hibával kell elutasítani, nem telíteni. Ez előkészíti az `Error` és a generikus `From`/`TryFrom` implementációkat a későbbi fejezetekben.

3.1.9 Források és ellenőrzési hiányok

A fejezet a Rust Reference (https://doc.rust-lang.org/reference/) és a Rust Book (https://doc.rust-lang.org/book/) megfelelő fejezeteire támaszkodik. A túlcsordulás, a `let`/`let mut` és a konverziós szabályok stabil nyelvi viselkedések; a fordítói hibakódok stabilak, a hibaüzenetek szövege a

könyv rögzített toolchainjére (Rust 1.99.1) vonatkozik, más verzióban eltérhet. A `TryFrom` és a `TryInto` a 2021-es és újabb editionök preludejában van, régebbi editionben importálni kell. A debug/release túlcsordulási viselkedés a Cargo alapértelmezett profiljaiban érvényes, az `overflow-checks` beállítással felülírható.

3.2 Kifejezések, utasítások és vezérlési szerkezetek

3.2.1 Motiváció

Tapasztalt C++-fejlesztőként ismerős helyzet: egy `if`-ág eredményét változóba kell menteni, és a rövid `? :` operátor nem elég, mert mindkét ágban több utasítás kell. Ilyenkor előre deklarált változóhoz, vagy – modern C++-ban – azonnal meghívott lambda-kifejezéshez nyúlunk. A Rust ezt a feszültséget alapvetően oldja fel: a nyelvben szinte minden kifejezés – blokk, `if`, sőt `loop` is – képes értéket adni. Ez a kifejezés-orientált stílus az idiomatikus Rust alapja, és a későbbi mintaillesztés, valamint az iterátor-láncok is erre építenek.

3.2.2 Fogalmak és szabályok

Definíció (kifejezés). Egy Rust-szintaktikai egység kifejezés, ha kiértékelésekor értéket ad, és típusa van. Kifejezés például egy literál, egy változó, egy függvényhívás, egy blokk, egy `if` vagy egy `loop`.

Definíció (utasítás). Az utasítás *nyelvtani szerep*: olyan szintaktikai egység, amely egy blokkban önállóan állhat, és nem ad a blokk értékéhez hozzá. A Rust két fő utasításfajtája a deklaráció (`let`, elemdefiníció) és a kifejezés-utasítás: ha egy kifejezés után pontosvesszőt írunk (`expr;`), a kifejezést kiértékeljük, az értékét elvetjük, és az egész utasítássá válik. Mellékhatásra nincs szükség: a `42;` érvényes kifejezés-utasítás (a kiértékelés nem csinál semmit, a fordító figyelmeztethet is rá). Kifejezés viszont a *típusával és kiértékelésével* definiált: értéket ad.

Szabály (blokk értéke). A `{ ... }` blokk kifejezés. Értéke az utolsó, pontosvessző nélküli kifejezés (*tail expression*) értéke. Ha nincs ilyen, vagy az utolsó kifejezés után pontosvessző áll (az eredményt elvető kifejezés-utasítás), a blokk típusa `()`. A diverging (`!` típusú) kifejezések, például a `return` vagy a `panic!`, kifejezések maradnak: típusuk van, csak sosem adnak értéket.

Definíció (unit típus). A `()` az *egységtípus* (unit type): pontosan egy értéke van, maga a `()`, ezért nulla méretű (`size_of::<()>() == 0`). A C++ `void`-hoz hasonló szerepű, de valódi típus, ezért változóba menthető, visszaadható, összehasonlítható. Nem „üres” típus: az üres (értékkészlet nélküli) típus a `!`.

Definíció (never típus). A `!` *lakatlan* (uninhabited) típus: nincs értéke, ezért az olyan kifejezések típusa, amelyek soha nem térnek vissza: `panic!`, `return`, `break`, `continue`, illetve olyan `loop`, amelyből nincs `break`. Minden típusba koerciálható, így használható olyan helyeken, ahol egyébként konkrét érték kellene (`let v = if ok { 1 } else { panic!("hiba") };`). **Stabil használata** a Rust 1.99.1-ben: a divergáló kifejezések típusaként és függvény visszatérési típusaként (`fn f() -> !`). Általános típusparaméterként (például `Option<!>`) még nem stabil (`E0658`); ennek stabil megfelelője a `std::convert::Infallible`, egy változat nélküli enum, például `Result<T, Infallible>` a soha nem hibázó eredményre. A `()` és a `!` különbsége: az egyiknek egy értéke van, de vezérlést hagy folytatódni; a másiknak nincs értéke, és a vezérlés nem folytatódik.

A pontosvessző szerepe döntő. Nézzük:

```rust
let x = { 42 };
let y = { 42; };
```

Az első esetben a blokk értéke `42` (típusa `i32`), ezért `x: i32 = 42`. A másodikban a `42;` kifejezés-utasítás elveti az értéket, így a blokk értéke `()`, tehát `y: () = ()`.

3.2.3 C++ és Rust összevetés

A C++-ban az `if` utasítás, ezért kifejezésként csak a ternáris operátor jöhet szóba:

```cpp
int abs_val(int x) {
    return (x >= 0) ? x : -x;
}
```

Ha az ágakban több utasítás kell, a ternáris operátor elégtelenné válik. A Rustban az `if` maga kifejezés, így nincs is szükség ternáris operátorra:

```rust
fn abs_val(x: i32) -> i32 {
    if x >= 0 { x } else { -x }
}
```

Szélsőeset: `x == i32::MIN` esetén a `-x` nem ábrázolható `i32`-ben: debug módban pánik, az alap release profilban wrapping (az eredmény negatív marad); a C++ megfelelőjében UB. Biztonságos alak: `x.checked_abs()`, `x.wrapping_abs()` vagy `x.unsigned_abs()`.

Az `if` ágai blokkok, és az ágaknak kompatibilis típusúnak kell lenniük (a közös típus koercióval is kialakulhat, nem feltétlenül azonos már kezdetben). Ha mindkét ág `i32`-et ad, a kifejezés típusa `i32`; ha az egyik ág például `()`, a másik `i32`, akkor fordítási hiba keletkezik.

C++-ban gyakori a flag változóval vezérelt kilépés egymásba ágyazott ciklusokból:

```cpp
bool found = false;
for (int i = 0; i < rows; ++i) {
    for (int j = 0; j < cols; ++j) {
        if (matrix[i * cols + j] == target) {
            found = true;
            break;
        }
    }
    if (found) break;
}
```

A Rust címkézett `break`-et kínál:

```rust
'outer: for i in 0..rows {
    for j in 0..cols {
        if matrix[i * cols + j] == target {
            break 'outer;
        }
    }
}
```

A címke (`'outer`) a ciklus elé kerül, és a `break 'outer` a megjelölt ciklust töri, nem csak a belsőt.

3.2.4 Kidolgozott példa: a pontosvessző csapdája

A leggyakoribb átállási hiba, amikor a C++-os szokás szerint pontosvesszőt teszünk a blokk végére, nem gondolva arra, hogy ezzel a blokk értékét `()`-re változtatjuk.

Ez a változat szándékosan nem fordul le:

```
fn max(a: i32, b: i32) -> i32 {
    if a > b {
        a
    } else {
        b
    };
}
```

A `cargo build` hibaüzenetének lényege (a pontos szöveg verziófüggő, a hibakód `E0308`):

```
error[E0308]: mismatched types
 --> src/main.rs:1:28
  |
1 | fn max(a: i32, b: i32) -> i32 {
  |                           ^^^ expected `i32`, found `()`
  |
  = note: the final expression of a block is followed by a semicolon, which
    discards its value
```

A hiba oka: a pontosvessző az `if` kifejezés után áll, így annak értékét elveti, és az egész függvénytörzs értéke `()`. A javítás a pontosvessző elhagyása:

```
fn max(a: i32, b: i32) -> i32 {
    if a > b { a } else { b }
}
```

Ugyanez a szabály érvényes bármely blokkra, például:

```
fn main() {
    let x = {
        let y = 10;
        y + 5
    };
    println!("{x}"); // 15
}
```

Itt a belső blokk értéke `15`, mert az utolsó kifejezés `y + 5` után nincs pontosvessző.

3.2.5 Tipikus hibák és átállási csapdák

1. **Felesleges pontosvessző a visszatérési értéknél.** Ahogy a fenti példa mutatja, a `return` kulcsszó nélküli függvényeknél a függvénytörzs utolsó kifejezése adja a visszatérési értéket. Ha oda pontosvessző kerül, a függvény `()`-t ad vissza, és a típus nem egyezik. A `return` használata opcionális; idiomatikus Rustban általában az utolsó kifejezést használjuk, `return`-t csak korai kilépésre.

2. **A `while` ciklus értékként való használata.** A `while` feltétel miatt nem garantálható, hogy a törzs legalább egyszer lefut, ezért a `while` kifejezés mindig `()` típust ad, és a törzsében a `break` nem adhat értéket. Önmagában a `let x = while ... { ... };` szabályos: lefordul, csak az `x` típusa `()` lesz, ami ritkán szándékos. A hiba akkor jelentkezik, amikor értéket várunk tőle:

```rust
fn main() {
    let mut n = 0;
    let x = while n < 3 {
        n += 1;
    };
    println!("{x:?}"); // (): lefordul, de az x semmit nem hordoz
}
```

Kódblokk — önálló példa · **szándékosan hibás** · várt hiba: `E0308` (`expected i32, found ()`)

```rust
fn main() {
    let mut n = 0;
    let x: i32 = while n < 3 {
        n += 1;
    };
    println!("{x}");
}
```

Az első változat kimenete `()`, a második nem fordul le, mert a `while` kifejezés típusa `()`, az annotáció viszont `i32`-t kér. Ha értéket akarunk kihozni egy feltételes ismétlésből, használjunk `loop`-ot `break` értékkel: `let x = loop { n += 1; if n == 3 { break n; } };`.

3. **Az `if` ágak típusának inkompatibilitása.** Ha az egyik ág `i32`-et, a másik `()`-t ad, a fordítás meghiúsul. Például:

   ```rust
   let x = if cond { 1 } else { println!("nincs érték") };
   ```

   A `println!` típusa `()`, így az ágak típusa nem egyezik. A megoldás az, ha mindkét ág konkrét értéket ad, vagy ha a `()` az elfogadott típus.

4. **A `return` és a blokk értékének keverése.** Ha egy függvényben egy korai `return`-t használunk, az `!` típusú, így bármilyen visszatérési típussal kompatibilis; a függvény későbbi blokkja normál értéket adhat. Ezt nem szabad összekeverni azzal, hogy a `return` utáni kód halott.

3.2.6 Feladatok

3.2.6.1 1. Fogalmi kérdés

Mi lesz az `x` és `y` értéke, illetve típusa az alábbi kódban?

```rust
let x = { let a = 3; a * a };
let y = { 42; };
```

3.2.6.2 2. C++-kód átírása idiomatikus Rustra

Írjuk át az alábbi C++ függvényt Rustra úgy, hogy ne használjunk explicit `return`-t, és az `if` kifejezésként szerepeljen:

```cpp
int parity(int x) {
    if (x % 2 == 0) {
        return 0;
    } else {
        return 1;
    }
}
```

3.2.6.3 3. Implementáció az rkv előkészítéséhez

Írjunk egy `collatz` nevű függvényt, amely egy **pozitív** egész számra (`n >= 1`) kiszámolja, hány lépés alatt éri el az 1-et a Collatz-sejtés szerint. A függvény szignatúrája `fn collatz(mut n: u64) -> Option<u64>`: `None` jelzi az értelmezési tartományon kívüli bemenetet (`n == 0`) és a `u64`-ben túlcsorduló számítást. A ciklus használjon `loop`-ot, és az eredményt `break` adja át. A lépésszabály: ha `n` páros, akkor `n = n / 2`, különben `n = 3 * n + 1`. Határozd meg a viselkedést a `0`, az `1`, egy ismert sorozat (`6`, `27`) és a túlcsorduló nagy érték (`u64::MAX`) esetén.

3.2.6.4 4. Hibajavítás

Javítsuk ki az alábbi hibás kódot úgy, hogy a függvény a kisebbik számot adja vissza:

```
fn min(a: i32, b: i32) -> i32 {
    if a < b {
        a
    } else {
        b
    };
}
```

3.2.7 Megoldások

3.2.7.1 1. Fogalmi kérdés

Az `x` értéke `9`, típusa `i32`, mert a blokk utolsó, pontosvessző nélküli kifejezése `a * a`. Az `y` értéke `()`, típusa `()`, mert a `42;` utasítás elveti az értéket, és a blokk értéke a unit.

3.2.7.2 2. C++-kód átírása idiomatikus Rustra

```
fn parity(x: i32) -> i32 {
    if x % 2 == 0 { 0 } else { 1 }
}
```

Nincs szükség `return`-re; az `if` kifejezés értéke közvetlenül a függvény visszatérési értéke.

3.2.7.3 3. Implementáció az rkv előkészítéséhez

```
fn collatz(mut n: u64) -> Option<u64> {
    if n == 0 {
        return None; // nincs értelmezve: a 0 sosem éri el az 1-et
    }
    let mut steps = 0;
    loop {
        if n == 1 {
            break Some(steps);
        }
        n = if n % 2 == 0 {
            n / 2
        } else {
            n.checked_mul(3)?.checked_add(1)? // túlcsordulásnál None
        };
        steps += 1;
    }
}
```

Ellenőrzött kimenet: `collatz(0)` → `None`, `collatz(1)` → `Some(0)`, `collatz(6)` → `Some(8)` (6, 3, 10, 5, 16, 8, 4, 2, 1), `collatz(27)` → `Some(111)`, `collatz(u64::MAX)` → `None` (a `3 * n` túlcsordulna). Az előfeltételek nélküli `3 * n + 1` debugban pánikolna, releaseben pedig wrapping után akár soha nem érne véget; a `checked_*` ezt a két esetet kifejezetten kezeli. A mintát ezért nem a nyelv teszi általánosan biztonságossá: a bemeneti előfeltételt (pozitivitás) és az aritmetika határait a programozónak kell megadnia.

A `loop` a `break Some(steps);` kifejezéssel adja vissza a lépésszámot, a `loop` pedig a függvény utolsó kifejezése, ezért közvetlenül ez a visszatérési érték. A `mut n` kulcsszó jelzi, hogy a paraméter módosítható.

3.2.7.4 4. Hibajavítás

A hiba a pontosvessző az `if` kifejezés után. Javított változat:

```
fn min(a: i32, b: i32) -> i32 {
    if a < b { a } else { b }
}
```

3.2.8 Rövid összefoglaló

A Rust kifejezés-orientált nyelv: a blokkok, az `if` és a `loop` is értéket adnak. A pontosvessző a kifejezések értékét elveti, ezért a blokk utolsó kifejezésénél nem szabad kitenni, ha az a visszatérési érték. A `()` unit típus az üres értéket jelöli, a `!` never típus pedig a soha vissza nem térő kifejezéseket. A vezérlési szerkezetek közül a `while` és a `for` mindig `()`-t ad, a `loop` viszont `break` értékkel képes eredményt visszaadni. A címkézett `break`/`continue` lehetővé teszi a külső ciklusok közvetlen megszakítását anélkül, hogy flag változókat kellene bevezetni.

Továbbvitt eredmény. A kifejezés-orientált stílus – különösen a blokkok és az `if` mint érték – alapvető építőköve lesz a Mintaillesztés: match, if let, let else és a minták alfejezetnek, ahol a `match` kifejezés is ezt a mintát követi, és az Iterátorok: lusta adapterek és saját iterátor alfejezetnek, ahol az adapter-láncok szintén kifejezésként épülnek fel. Az itt megtanult pontosvessző-fegyelem nélkülözhetetlen az idiomatikus Rust-kód írásához.

3.2.9 Források és ellenőrzési hiányok

A fejezet a Rust hivatalos dokumentációjára épül: a Rust Reference kifejezés- és utasítás-definíciói, valamint a Rust Book vezérlési szerkezetekről szóló részei. A példában szereplő `E0308` hibakód stabil, a pontos hibaüzenet szövege a könyv rögzített toolchainjére (Rust 1.99.1) vonatkozik. A fejezet a `!` (never) típust a divergáló kifejezések (`return`, `panic!`, `break`, `continue`) típusaként használja; saját típuskénti, explicit annotálását nem tanítja.

3.3 Tuple-ök, tömbök és az első pillantás a slice-okra

3.3.1 Motiváció

C++-ban több visszatérési értéket általában `std::pair`-rel, `std::tuple`-pal vagy kimenő paraméterekkel adunk vissza; a strukturált kötés *(structured binding)* C++17 óta teszi ezt kényelmessé. A tömböknél a C++ fejlesztő hozzászokott ahhoz, hogy a `std::vector`/`std::array` `operator[]`-je általában nem ellenőrzi a határokat, és a hibás indexelés definiálatlan viselkedést okozhat.

Rust ezt a két területet alapvetően másként kezeli: a tuple és a destrukturálás nyelvi szintű, a tömbindexelés pedig mindig határellenőrzött, és a biztonságos alternatíva `Option`-t ad vissza. Ebben az alfejezetben ezeket az alapvető típusokat nézzük meg, majd bevezetjük a slice fogalmát, amely a Rust referenciavilág egyik legfontosabb eleme.

3.3.2 Fogalmak és szabályok

Definíció. A tuple heterogén, fix méretű érték, amely több különböző típusú értéket csomagol egybe. A típusa `(T1, T2, ...)`, például `(i32, String, bool)`. A mezők elérése nullától indexelt: `t.0`, `t.1`, stb. A tuple literálja `(érték1, érték2, ...)`. A zéró elemű tuple az egység *(unit)* típus: `()`, amely a C++ `void`-hoz hasonlítható, de értékként is létezik.

Definíció. A tömb típusa `[T; N]`, ahol `N` a típus része, és fordítási időben ismert. Például `[i32; 5]` öt darab `i32`-et tartalmaz. A méret nem változtatható futás közben; dinamikus méretű gyűjteményekhez `Vec<T>`-t használunk, amelyről később lesz szó.

Definíció. A *slice-típus* `[T]` egy folytonos, `T` elemekből álló sorozat, amelynek hossza fordítási időben nem ismert (*dinamikus méretű*, `!Sized` típus). Önmagában nem hozható létre változóként, mindig mögötte van valamilyen mutató. A leggyakoribb alakok:

Típus	Tulajdonol?	`Sized`?	Metaadat	Megjegyzés
`[T]`	– (típus, nem érték)	nem	–	dinamikus méretű; csak mutató mögött használható
`&[T]` / `&mut [T]`	nem, kölcsönzött nézet	igen (két gépi szó)	hossz (`len`)	*fat pointer*: mutató + hossz; a hossz az elemek száma, nem bájtok
`Box<[T]>`	igen	igen (két gépi szó)	hossz	kupacon tulajdonolt szelet, nincs kapacitás
`Vec<T>`	igen	igen (három gépi szó)	hossz + kapacitás	növelhető puffer
`[T; N]`	igen	igen	–	a méret a típus része

Az, hogy egy konkrét `&[T]` mögötti elemek a veremben vagy a kupacon vannak, a példa ténye, nem a típus tulajdonsága: ugyanaz a `&[i32]` mutathat tömbre (verem) és `Vec`-re (kupac) is.

Szabály. A tömb és slice indexelése a `[]` operátorral mindig határellenőrzött. Ha az index kívül esik a tartományon, a program *pánikba esik* (`panic!`); ez nem definiálatlan viselkedés, hanem jól meghatározott hiba. A pánik hatása a **panic stratégiától** függ: az alapértelmezett `panic = "unwind"` mellett a pánikoló szál verme lecsévélődik (a `Drop` implementációk lefutnak), a többi szál tovább fut, a szál `join()`-ja `Err`-t ad, a `main` szál pánikja pedig a folyamatot 101-es kilépési kóddal állítja le; `panic = "abort"` mellett az egész folyamat azonnal leáll, lecsévélés nélkül. (A részleteket a Result, a ? operátor és a panic helye és a Szálak, Send és Sync alfejezet tárgyalja ugyanezzel a két stratégiával.) A `get(index)` metódus `Option<&T>`-t ad vissza: `Some` a sikeres, `None` a hibás indexre.

Szabály. A `&[T]` slice nem tulajdonos; a mögöttes elemeket mindig egy tömb, `Vec<T>`, `Box<[T]>` vagy más tároló birtokolja. A `let s: [i32]` (a nyers `[T]` típus mint változó) nem fordul le, mert a méret nem ismert fordítási időben. A slice-ok élettartamát a kölcsönzés-ellenőrző garantálja, így nincs felszabadított memóriára mutató érvénytelen slice.

3.3.3 C++ és Rust összevetés

A több visszatérési érték C++-ban `std::tuple`-pal és strukturált kötéssel:

```cpp
#include <string>
#include <tuple>

std::tuple<int, std::string> parse(const std::string& line) {
    auto pos = line.find(':');
    if (pos == std::string::npos) {
        return {0, line};
    }
    return {
        std::stoi(line.substr(0, pos)),
        line.substr(pos + 1)
    };
}

// Használat:
auto [code, text] = parse("42:hello");
```

Rustban a tuple a nyelv része, és a destrukturálás közvetlenül `let`-ben történik:

```rust
use std::num::ParseIntError;

fn parse(line: &str) -> Result<(i32, &str), ParseIntError> {
    match line.split_once(':') {
        // Nincs elválasztó: a C++-változathoz hasonlóan (0, teljes sor).
        None => Ok((0, line)),
        // Van elválasztó: a szám rész hibája Err, nem pánik.
        Some((number, text)) => number.parse().map(|code| (code, text)),
    }
}

// Használat:
let (code, text) = parse("42:hello").unwrap(); // (42, "hello")
```

A `split_once(':')` az első `:` mentén két részre bont, és `None`-t ad, ha nincs elválasztó; így nincs kézi indexszámítás (`pos + 1`), amely elválasztó nélkül a sor végén túlra mutatna és pánikot okozna. A viselkedés **nem teljesen egyenértékű** a C++-változatéval: a `std::stoi` kivételt dob hibára (a Rust `Err`-t ad), ráadásul elfogadja a numerikus előtagot (`"42abc"` → 42) és a vezető szóközt, míg a Rust `parse` a teljes szövegre egyezést kér, és a vezető whitespace-t elutasítja (a 11.1 részletesen tárgyalja).

Bemenet	C++ `parse`	Rust `parse`
`"42:hello"`	`{42, "hello"}`	`Ok((42, "hello"))`
`"42"`	`{0, "42"}`	`Ok((0, "42"))`
`"hello"`	`{0, "hello"}`	`Ok((0, "hello"))`
`""`	`{0, ""}`	`Ok((0, ""))`
`"x:hi"`	`std::invalid_argument` kivétel	`Err(ParseIntError)`

A különbség: Rustban nincs `std::tuple` sablonosztály, mert a tuple beépített típus; a destrukturálás nem igényel C++17-es kiterjesztést, és a `&str` slice élettartamát a borrow checker követi. A használati példában az `.unwrap()` csak a rövidség miatt szerepel; a `Result` kezeléséről a Result, a ? operátor és a panic helye alfejezetben lesz szó.

A tömbindexelés C++-ban gyakran ellenőrizetlen, Rustban mindig ellenőrzött. Két külön esetet kell megkülönböztetni: azt, amelyet a fordító *fordításkor* kimutat, és azt, amelyet csak *futáskor* ellenőriz.

```cpp
#include <array>

std::array<int, 3> a{1, 2, 3};
int x = a[5]; // UB: definiálatlan viselkedés
```

```rust
let a = [1, 2, 3];
// let x = a[5]; // FORDÍTÁSI HIBA: this operation will panic at runtime
//               // (unconditional_panic lint, alapból deny): konstans index, ismert hossz
```

Ha az index csak futáskor derül ki, a fordító nem tudja eldönteni, és az ellenőrzés futáskor lép életbe:

```rust
let a = [1, 2, 3];
let i = std::env::args().count() + 4; // futásidőben kapott index (itt: 5)
let x = a[i]; // FUTÁSI PANIC: index out of bounds: the len is 3 but the index is 5
```

A biztonságos alternatíva Rustban:

```rust
let a = [1, 2, 3];
match a.get(5) {
    Some(v) => println!("{v}"),
    None => println!("nincs ilyen index"),
}
```

C++20-tól a `std::span<const int>` hasonló szerepet tölt be, mint a Rust `&[i32]`, de a Rust slice-ot a borrow checker életciklus-szabályai védik, és nem lehet null, valamint a hossza mindig érvényes a mutatójával együtt.

3.3.4 Kidolgozott példa

Vegyünk egy olyan függvényt, amely egy slice minimumát, maximumát és összegét adja vissza tuple-ben. A szándékosan naiv változat indexeléssel kezdődik, és üres bemenetre futásidejű hibát okoz:

```rust
fn min_max_sum(values: &[i32]) -> (i32, i32, i32) {
    let mut min = values[0]; // üres slice-nál panic!
    let mut max = values[0];
    let mut sum = 0;
    for &v in values {
        min = min.min(v);
        max = max.max(v);
        sum += v;
    }
    (min, max, sum)
}

fn main() {
    let arr = [15, -3, 22, 7, 10];
    let middle: &[i32] = &arr[1..4];
    println!("middle = {middle:?}");
    let (min, max, sum) = min_max_sum(middle);
```

```
    println!("min = {min}, max = {max}, sum = {sum}");
}
```

A `middle` kimenete `[-3, 22, 7]`, a statisztika pedig `min = -3, max = 22, sum = 26`. Ha a bemeneti slice üres, például `&arr[..0]`, a `values[0]` kifejezés pánikba esik: `panic! index out of bounds`. Ez hasonlít a C++ `std::array` ellenőrizetlen hozzáféréséhez, de itt nincs UB, hanem kontrollált leállás.

A naiv változatnak két hibája van: üres bemenetre pánikol (`values[0]`), és az `i32` összeg túlcsordulhat (debugban pánikkal, releaseben wrappinggel). A javított változat a két hibát **külön hibaértékként** adja vissza, ezért `Option` helyett `Result`-ot használ: a `None` ugyanis nem árulná el, hogy a bemenet üres volt, vagy az összeg túlcsordult. Az összeg típusa `i32` marad (a függvény előfeltétele: az összeg beleférjen), a túlcsordulást `checked_add` jelzi:

```
#[derive(Debug, PartialEq)]
enum StatsError {
    Empty,    // üres slice: nincs minimum, maximum
    Overflow, // az i32 összeg túlcsordult
}

fn min_max_sum(values: &[i32]) -> Result<(i32, i32, i32), StatsError> {
    let (first, rest) = values.split_first().ok_or(StatsError::Empty)?;
    let (mut min, mut max, mut sum) = (*first, *first, *first);
    for &v in rest {
        min = min.min(v);
        max = max.max(v);
        sum = sum.checked_add(v).ok_or(StatsError::Overflow)?;
    }
    Ok((min, max, sum))
}
```

A `checked_add` balról jobbra halad, ezért a *köztes* összegeknek is bele kell férniük `i32`-be: az `[i32::MAX, 1, -1]` is `Err(Overflow)`-t ad, bár a végösszeg belefér. Ha a cél csak a végösszeg beleférése, szélesebb akkumulátor kell (lásd a `describe` feladatot).

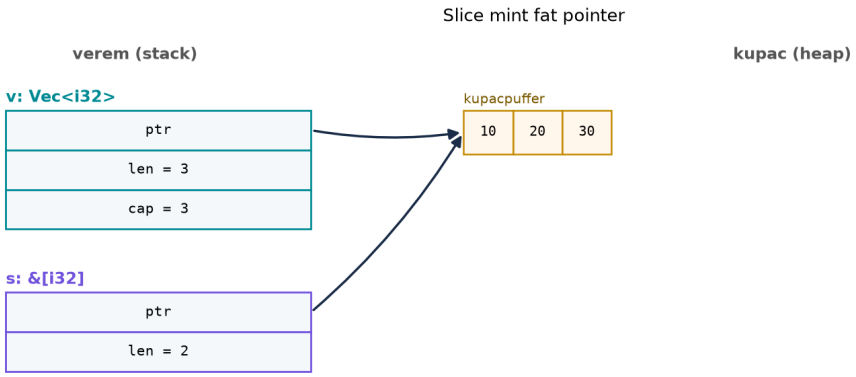
Meghatározott viselkedés (ellenőrizve): `[-3, 22, 7]` → `Ok((-3, 22, 26))`; üres slice → `Err(Empty)`; `[i32::MAX, 1]` → `Err(Overflow)`; negatív elemek: `[-5, 3]` → `Ok((-5, 3, -2))`. A `?` itt `Result`-ban működik: az `ok_or` az `Option`-t `Result`-tá alakítja, és a hibát azonnal továbbítja. A `main` mindkét esetet kezelni tudja:

```
match min_max_sum(&arr[..0]) {
    Ok((min, max, sum)) => println!("min = {min}, max = {max}, sum = {sum}"),
    Err(StatsError::Empty) => println!("A bemeneti slice üres."),
    Err(StatsError::Overflow) => println!("Az összeg túlcsordulna."),
}
```

A `min.max(v)` és `min.min(v)` metódusok a `std::cmp::Ord` trait által biztosítottak, és idiomatikusan fejezik ki a szándékot.

3.3.5 Memóriaábra: tömb és slice

A slice fat pointer felépítését a következő memóriaábra szemlélteti. Az egyszerűség kedvéért a cél egy `Vec`, mert annak a kupacpuffere jól ábrázolható; a tömb slice esetén a cél a stacken lenne, de a pointer célja ugyanolyan.



3.1. ábra: Slice mint fat pointer - A &(i32) két gépi szóból áll: ptr és len. Az ábrán a mutató a puffer elejére mutat; egy valós szeletnél a ptr a megfelelő elemre offsetelődik. A Vec a teljes pufferen osztozik.

A `v` egy `Vec<i32>`, amely a stacken három mezőt tart: a kupacpufferre mutató `ptr`-t, az aktuális elemszámot (`len = 3`) és a lefoglalt kapacitást (`cap = 3`). Az `s` slice szintén a stacken van, de csak két mezője van: `ptr` és `len = 2`. Nincs külön kapacitás, mert a slice nem birtokol; a mögöttes puffer életciklusát a borrow checker a kölcsönzés idejére biztosítja. A szöveges példában `&arr[1..4]` esetén a cél a stacken lévő tömb, a hossz pedig három, de a fat pointer elve ugyanaz.

Továbbvitt eredmény. A fat pointer gondolata – az, hogy egy referencia több adatot hordozhat, mint egyetlen gépi cím – a Slice-ok, String és &str, UTF-8 alfejezetben a `String`/`&str` párosnál, a Trait objektumok és dinamikus diszpécs alfejezetben pedig a dinamikus dispatch-nél tér vissza.

3.3.6 Tipikus hibák és átállási csapdák

1. **A `[]` határellenőrzés figyelmen kívül hagyása.** A C++ fejlesztő azt gondolhatja, hogy a Rust `a[i]` ugyanolyan olcsó, mint a C++ `a[i]` release módban. Bár a Rust határellenőrzése általában kicsi többletköltséggel jár, forró ciklusokban érdemes az iterátorokat vagy a `get`-et használni, mert az előbbieknél a fordító gyakran el tudja távolítani a fölösleges ellenőrzéseket. A `get` és a `map`/`and_then` kombinációja ráadásul kifejezőbb.

2. **A `get` után azonnali `unwrap`.** Az `arr.get(i).unwrap()` ugyanarra a hibásindex-viselkedésre vezet, mint az `arr[i]` indexelés – mindkettő pánikol –, de a diagnosztika eltér: az indexelés `index out of bounds: the len is 3 but the index is 5`, a `get(i).unwrap()` pedig `called \`Option::unwrap()\` on a \`None\` value` üzenetet ad. A stílus is rosszabb, mert az `Option` kezelését elrejti. Ha a hívó tudja, hogy az index érvényes, akkor a `[]` a megfelelő; ha nem tudja, akkor az `Option`-t kell kezelni `match`-csel, `if let`-tel vagy `?`-tel.

3. **A tuple mezők számozásának félreértése.** A Rust tuple mezői nullától kezdődnek, mint a C++ `std::get<0>` indexei. A `(1, 2, 3).3` nem fordul le, mert nincs negyedik mező. A destrukturálás megkerüli a `.0`/`.1` jelölést, és olvashatóbb kódot ad.

4. **A slice mint „pointer + hossz” félreértelmezése.** Bár a slice valóban fat pointer, a Rust típusrendszerében a hossz a stacken lévő érték része, és a mutató nem lehet null. A C++ `std::span`-hez képest a Rust slice-ot a borrow checker is védi, ezért nem fordul

elő felszabadított memóriára mutató slice, és a `&mut [T]` slice egyedüliségét a kölcsönzési szabály garantálja.

3.3.7 Feladatok

1. **Lefordul-e és miért?** Vizsgáljuk meg az alábbi kódot:

   ```
   let tuple = (1, 2, 3);
   let x = tuple.3;
   ```

   Fordul-e a kód? Ha nem, miért, és mi a javítás?

2. **C++-kód átírása idiomatikus Rustra.** Adott a következő C++ függvény, amely egy `std::array<int, 4>`-ből visszaadja az első és az utolsó elem különbségét, ha a tömb nem üres:

   ```
   #include <array>
   #include <optional>

   std::optional<int> first_last_diff(const std::array<int, 4>& a) {
       return a.front() - a.back(); // ha üres lenne, UB lenne
   }
   ```

   Írjunk Rust függvényt, amely `&[i32]` slice-ra `Option<i32>`-t ad vissza (`first - last`), és nem indexeléssel, hanem `first()` és `last()` metódussal dolgozik. Üres slice-ra és túlcsordulás esetén `None` legyen az eredmény. (A C++-változat `std::array<int, 4>`-e sosem üres, de egy `std::span`-t vagy `std::vector`-t fogadó változatnál a `front()` üres tárolóra UB, és az `int` kivonás is túlcsordulhat.)

3. **Implementációs feladat.** Hozz létre önálló `r006` crate-et a `~/rust-labor/r006/` könyvtárban (`cargo new r006 --bin`). A `main.rs` tartalmazzon egy `describe` függvényt, amely egy `&[i32]` slice-ra visszaad egy `(i32, i32, i64, f64)` tuple-t: minimum, maximum, összeg és átlag. Használj `split_first`-et és `Option`-t, hogy üres slice esetén ne pánikoljon, és `i64` akkumulátort, hogy az összeg ne csorduljon túl. A `main`-ben készíts slice-ot egy tömb egy részéből, és írd ki a tuple mezőit.

4. **Hibajavítás és refaktorálás.** Az alábbi kód `arr.get(i).unwrap()`-ot használ, ami pánikot okozhat:

   ```
   fn print_element(arr: &[i32], i: usize) {
       let v = arr.get(i).unwrap();
       println!("elem = {v}");
   }
   ```

   Refaktoráljuk a függvényt úgy, hogy ne pánikoljon, hanem `Option`-t adjon vissza, vagy `match`-csel írjon ki üzenetet. Melyik a jobb API-tervezés, és miért?

3.3.8 Megoldások

1. **Lefordul-e és miért?** A kód nem fordul le. A hiba lényege `no field '3' on type '(i32, i32, i32)'`; a tuple mezői nullától indexeltek, ezért a legnagyobb érvényes index a 2. A javítás a harmadik elemhez `let x = tuple.2;` vagy ezzel egyenértékűen `let (_, _, x) = tuple;`; mindkettőnél `x == 3`. (A `let (x, _, _) = tuple;` az *első* elemet adná, `x == 1`, tehát nem egyenértékű javítás; ha csak tetszőleges érvényes hozzáférés a cél, akkor `tuple.0` vagy `tuple.1` is megfelel.)

2. **C++-kód átírása idiomatikus Rustra.**

```rust
fn first_last_diff(a: &[i32]) -> Option<i32> {
    let first = a.first()?;
    let last = a.last()?;
    first.checked_sub(*last) // first - last, mint a C++ front() - back()
}

fn main() {
    assert_eq!(first_last_diff(&[1, 2, 3, 4]), Some(-3));
    assert_eq!(first_last_diff(&[]), None);
    assert_eq!(first_last_diff(&[i32::MIN, 1]), None); // túlcsordulna
    println!("ok");
}
```

A `first()` és `last()` metódusok a slice-API részei, és `Option<&i32>`-t adnak vissza. A `?` a `None` esetén azonnal visszatér, így az üres slice nem pánikol; a `checked_sub` a túlcsordulást is `None`-ként jelzi. A kivonás iránya a C++-éval egyezik: `front() - back()`, tehát `[1, 2, 3, 4]`-re `-3`. Egy `[i32; 4]` tömb is átadható, mert a `&[i32; 4]` automatikusan `&[i32]`-vé alakul.

3. **Implementációs feladat.** A `main.rs`:

```rust
fn describe(values: &[i32]) -> Option<(i32, i32, i64, f64)> {
    let (first, rest) = values.split_first()?;
    let mut min = *first;
    let mut max = *first;
    // i64 akkumulátor: az i32 elemek összege legfeljebb 2^32 elemig biztosan belefér;
    // a szignatúra nem korlátozza a szelet hosszát, ennél hosszabb szeletre checked_add
    ↪ vagy i128 kell.
    let mut sum = i64::from(*first);
    for &v in rest {
        min = min.min(v);
        max = max.max(v);
        sum += i64::from(v);
    }
    let avg = sum as f64 / values.len() as f64;
    Some((min, max, sum, avg))
}

fn main() {
    let arr = [8, 3, 12, 5, 9, 1];
    let part: &[i32] = &arr[1..5];
    match describe(part) {
        Some((min, max, sum, avg)) => {
            println!("min = {min}, max = {max}, sum = {sum}, átlag = {avg:.2}");
        }
        None => println!("A slice üres."),
    }
}
```

A `cargo run` kimenete: `min = 3, max = 12, sum = 29, átlag = 7.25`. A `part` slice a `[3, 12, 5, 9]` elemet fedi le.

4. **Hibajavítás és refaktorálás.** A `get(i).unwrap()` helyett a függvény `Option`-t ad vissza:

```rust
fn print_element(arr: &[i32], i: usize) -> Option<()> {
    let v = arr.get(i)?; // hibás indexnél itt None-nal kilép, nem ír ki semmit
    println!("elem = {v}");
    Some(()) // a siker jelzése
}
```

A visszatérési szerződés: `Some(())`, ha az elem létezett és kiírtuk, `None`, ha az index érvénytelen (ekkor semmi nem íródik ki). Az `()` itt azt jelenti, hogy a sikernek nincs további adata; a hívó csak azt tudja meg, hogy volt-e elem. Tisztább tervezés, ha a lekérdezést és a kiírást szétválasztjuk: a könyvtár csak `arr.get(i)`-t ad, a kiírásról a hívó dönt. Vagy a hívó oldalon történik a kezelés:

```
match arr.get(i) {
    Some(v) => println!("elem = {v}"),
    None => println!("nincs ilyen index"),
}
```

Az `Option`-t visszaadó változat jobb API-tervezés, mert a hívó döntheti el, hogyan kezeli a hibát, nem a könyvtár függvénye pánikol. A `match`-es változat a különálló side effect esetén is gyakori, de nem helyettesíti a hibaterjesztést.

3.3.9 Rövid összefoglaló

A tuple Rustban beépített heterogén, fix méretű típus; destrukturálása `let (a, b) = t;` alakban történik, és így a C++ `std::tuple`/`std::tie`/structured binding kényelmét nyújtja. A tömb `[T; N]` mérete a típus része. Az indexelés `[]` operátorral mindig határellenőrzött, hibás indexnél `panic!` lép fel, nem UB; a `get(i)` `Option<&T>`-t ad. A slice `&[T]` fat pointer: mutató és hossz, nem tulajdonos nézet, és a borrow checker garantálja az élettartamát. **Továbbvitt eredmény:** a fat pointer gondolatát a `&str` és `&[T]` részletes tárgyalása a Slice-ok, String és &str, UTF-8, valamint a dinamikus dispatch a Trait objektumok és dinamikus diszpécs alfejezetben viszi tovább.

3.3.10 Források és ellenőrzési hiányok

A tuple, a tömb, a `get` és a slice stabil nyelvi elemei; a hivatalos leírások a Rust Reference és a Rust Book dokumentációjában találhatók. A példákban szereplő pontos `panic!` üzenetek és a fordító hibaüzeneteinek szövege verziófüggő; ellenőrzésre érdemes a fejezet készítésekor használt toolchainen. A `split_first`, `first`, `last` és a `min`/`max` metódusok a standard könyvtár stabil API-jának részei, de a pontos szignatúrákat a docs.rs-en érdemes ellenőrizni.

4 Structok, enumok és mintaillesztés

4.1 Structok és metódusok: impl blokkok, asszociált függvények

4.1.1 Motiváció

C++-ban egy osztály (`class`) egyszerre csomagolja az adattagokat és a rajtuk dolgozó tagfüggvényeket, és ezt gyakran rejtett, implicit `this` mutatóval teszi: a tagfüggvényen belül `x` irányulhat a `this->x` mezőre, a `const` tagfüggvény pedig megígéri, hogy nem módosítja az objektumot. A konstruktorok, a túlterhelés és az öröklés mind része ennek a modellnek. A Rust radikálisan szétválasztja az adatot és a viselkedést, és nincs sem implicit `this`, sem konstruktor-szintaxis, sem implementációs öröklés. Ez a fejezet megmutatja, hogyan kell C++-os fejjel jól strukturálni Rust-struktúrákat és metódusaikat.

4.1.2 Fogalmak és szabályok

Definíció. A Rust `struct` olyan típus, amely heterogén, névvel ellátott (vagy pozícióalapú, vagy üres) mezőket foglal magába. A Rust a `struct` három alakját különbözteti meg:

- **Named-field struct**: a mezőknek nevük van, és a `struct Point { x: f64, y: f64 }` szintaxissal definiáljuk.
- **Tuple struct**: a mezőknek nincs nevük, csak pozíciójuk: `struct Rgb(u8, u8, u8);`.
- **Unit struct**: egyáltalán nincs mező, csak típusnév: `struct Marker;`.

Definíció. Az `impl` blokk egy típushoz tartozó asszociált elemeket definiál. Két dolgot tehetünk bele: asszociált függvényeket (amelyek nem vesznek fel `self`-et) és metódusokat (amelyek első paramétere `self`, `&self` vagy `&mut self`). Az adatot a `struct` (és az `enum`) írja le, a viselkedést az `impl` blokk; `class` kulcsszó nincs.

Szabály. A metódusok első paramétere (a *receiver*) leggyakrabban az alábbi három alak valamelyike; ez bevezető felsorolás, nem teljes lista:

- `self`: a hívó értéket **átveszi**: nem `Copy` típusnál mozgatás történik, és utána az eredeti változó nem használható; `Copy` típusnál másolat kerül a hívásba, ezért az eredeti tovább használható.
- `&self`: a hívó értéket **kölcsönzi** (shared borrow); ez a C++ `const` tagfüggvény Rust-megfelelője, de nem csak `const` szempontú, hanem aliasing-korlátozott is. A „nem módosítható” a *megosztott referencián keresztül* a mezők típusától függ: `Cell`, `RefCell`, `Mutex` és atomikus mezők megosztott (`&`) hivatkozáson át is módosíthatók (belső mutabilitás, lásd Belső mutabilitás: Cell, RefCell és a futásidejű kölcsönzés, Mutex, RwLock, csatornák és a megosztott rkv); a `&self` tehát nem általános „senki sem módosíthatja” garancia.
- `&mut self`: kizárólagos, módosító kölcsönzés (mutable borrow); ez a C++ nem-`const` tagfüggvény megfelelője.

Vannak további receiver-alakok is, amelyeket később, a megfelelő témánál tárgyalunk: a `mut self` (átvett, de módosítható érték), a `self: Box<Self>`, a `self: Rc<Self>` és a `self: Pin<&mut Self>` (Box, Rc, Arc és Weak, Pin, Unpin, async trait-ek).

Szabály. Ha egy függvény egy `impl` blokkban nem vesz fel `self`-et, akkor asszociált függvény; ilyen a `new` konvenció. A Rustban nincs nyelvi szintű konstruktor: a `new` csak egy megegyezés, és a típushoz kapcsolt asszociált függvény. Bármennyi ilyen függvényt írhatunk, de túlterhelés nincs — különböző nevek kellenek, vagy a `Default` trait és a builder minta használható.

Szabály. Nincs implementációs öröklés Rustban. Egy `struct` nem örökölhet más `struct`-ból. A viselkedés újrafelhasználása trait-ekkel, kompozícióval és generikusokkal történik, amelyeket a későbbi fejezetek tárgyalnak (lásd Trait-ek és generikusok).

Szabály. A `.` operátor automatikusan referenciát illeszt be a hívás helyén: ha a metódus `&self`-et vagy `&mut self`-et vár, és a hívó oldalon érték van, akkor a fordító automatikusan `&` vagy `&mut` kölcsönzést hoz létre. Ezért írhatunk `point.norm()`-ot annak ellenére, hogy `norm` `&self`-et kér.

A mezők inicializálásakor van egy hasznos rövid alak: ha a változó neve megegyezik a mező nevével, akkor a `x: x` helyett elegendő a `x` írása.

4.1.3 C++ és Rust összevetés

Tekintsünk egy egyszerű geometriai típust C++-ban és Rustban.

```cpp
// C++: osztály, konstruktor, const/non-const tagfüggvények
#include <cmath>

class Point {
    double x_, y_;
public:
    Point(double x, double y) : x_(x), y_(y) {}

    double norm() const {
        return std::sqrt(x_ * x_ + y_ * y_);
    }

    void scale(double factor) {
        x_ *= factor;
        y_ *= factor;
    }

    double x() const { return x_; }
    double y() const { return y_; }
};
```

A Rust megfelelő:

```rust
struct Point {
    x: f64,
    y: f64,
}

impl Point {
    // asszociált függvény; a `Self` a jelenlegi típus aliasa
    fn new(x: f64, y: f64) -> Self {
        Self { x, y }
    }

    // metódus, &self kölcsönöz, nem veszi át a tulajdont
```

```
    fn norm(&self) -> f64 {
        (self.x * self.x + self.y * self.y).sqrt()
    }

    // metódus, &mut self módosít
    fn scale(&mut self, factor: f64) {
        self.x *= factor;
        self.y *= factor;
    }

    // getterek: &self; a kisméretű `Copy` mezőt (itt f64) érték szerint adjuk vissza,
    // nagyobb vagy nem `Copy` mezőnél (például String) &T-t adnánk
    fn x(&self) -> f64 {
        self.x
    }

    fn y(&self) -> f64 {
        self.y
    }
}
```

A lényeges különbség: a Rustban nincs implicit `this`; a metódus paraméterlistája nyíltan kimondja, hogyan fér hozzá az objektumhoz. A C++ `const` tagfüggvény nem tudja módosítani az objektumot, de ez nem garantálja, hogy más szál nem módosítja eközben. A Rust `&self` ezen felül kizárja az egyidejű módosítást is a kölcsönzési szabályok által. A `scale` C++-ban hallgatólagosan módosít a `this`-en, míg Rustban a `&mut self` explicit.

4.1.4 Kidolgozott labor: az rkv `Store` struktúrája

Munkakönyvtár: `~/rust-labor/rkv/`

Az rkv mintaprojektben a `Store` a központi adatszerkezet, amely a kulcs–érték tárolást valósítja meg. Ebben a laborban a korábban (Az rkv mintaprojekt indítása: Cargo-workspace és első CLI) felvázolt `Store` struktúrát elemezzük, és rögzítjük a típusválasztás indoklását.

1. lépés: Nyissuk meg a `rkv-core/src/lib.rs` fájlt. A kiinduló állapot az Az rkv mintaprojekt indítása: Cargo-workspace és első CLI alfejezet végén elkészült `Store`. Az egyetlen API-változás az új `clear` metódus; a többi metódus szignatúrája változatlan, csak a dokumentációs kommentek bővülnek a típusválasztás indoklásával.

Kódblokk — fájl: `rkv-core/src/lib.rs` · művelet: teljes csere (az rkv mintaprojekt indítása: Cargo-workspace és első CLI alfejezetbeli `Store` + `clear`) · ellenőrzés: `cargo build --workspace` → az `rkv-cli` változatlanul fordul

```
use std::collections::HashMap;

/// A kulcs-érték tároló.
///
/// A kulcsok mindig `String` típusúak, mert a szöveges protokoll
/// (`GET`, `SET`, `DEL`, `LIST`) sorokkal dolgozik. Az értékek
/// `Vec<u8>` típusúak: a tároló (és a későbbi JSON-perzisztencia) tetszőleges
/// bájtsort veszteség nélkül tárol, nem korlátozzuk érvényes UTF-8-ra. A
/// hálózati protokoll szöveges rétege csak szöveg-biztos értékeket visz át;
/// tetszőleges bájtokhoz a későbbi `SETX`/`GETX` hexa réteg szolgál (r040).
#[derive(Debug, Default)]
pub struct Store {
    data: HashMap<String, Vec<u8>>,
}
```

```rust
impl Store {
    /// Üres tároló létrehozása.
    pub fn new() -> Self {
        Self {
            data: HashMap::new(),
        }
    }

    /// Kulcs-érték pár beillesztése. Felülírja a korábbi értéket.
    pub fn set(&mut self, key: String, value: Vec<u8>) {
        self.data.insert(key, value);
    }

    /// Egy kulcs értékének lekérdezése. `None`, ha nincs ilyen kulcs.
    /// A `get` `&str`-et fogad, mert így a hívó nem kényszerül fölösleges
    /// `String`-t létrehozni; a `HashMap` `String` kulcsa esetén
    /// `&str`-rel is lehet keresni.
    pub fn get(&self, key: &str) -> Option<&Vec<u8>> {
        self.data.get(key)
    }

    /// Kulcs törlése. Visszaadja a törölt értéket, ha létezett.
    pub fn delete(&mut self, key: &str) -> Option<Vec<u8>> {
        self.data.remove(key)
    }

    /// Az összes kulcs rendezetlen listája.
    pub fn keys(&self) -> Vec<String> {
        self.data.keys().cloned().collect()
    }

    /// Az összes kulcs törlése.
    pub fn clear(&mut self) {
        self.data.clear();
    }
}
```

2. lépés: Ellenőrizzük a típusválasztást. A `get` metódus `&str`-et fogad, holott a `HashMap` kulcsa `String`. Ez működik, mert a `HashMap::get` általános kulcsmintát vár, és a `String` kulcs esetén a `&str` is érvényes keresőkulcs. Ez hasonló a C++20 heterogén lookupjához, amely például `std::unordered_map<std::string, V> find` tagját `std::string_view`-val is meghívja. A C++ hagyományos `std::unordered_map` viszont csak `const std::string&`-et fogad, ami fölösleges másolást okozhat.

3. lépés: Futtassunk unit teszteket. A laborban a `cargo test -p rkv-core` paranccsal ellenőrizzük, hogy a `Store` a várt módon viselkedik. Ehhez adjunk a `lib.rs` végéhez egy tesztmodult.

Kódblokk — fájl: `rkv-core/src/lib.rs` · művelet: beillesztés a fájl végére · ellenőrzés: `cargo test -p rkv-core` → 2 sikeres teszt

```rust
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn set_and_get() {
        let mut store = Store::new();
        store.set("kulcs".to_string(), b"ertek".to_vec());
        assert_eq!(store.get("kulcs"), Some(&b"ertek".to_vec()));
    }
```

```
    #[test]
    fn delete_returns_old_value() {
        let mut store = Store::new();
        store.set("a".to_string(), vec![1, 2, 3]);
        let old = store.delete("a");
        assert_eq!(old, Some(vec![1, 2, 3]));
        assert!(store.get("a").is_none());
    }
}
```

Várt kimenet a `cargo test -p rkv-core` parancsra:

```
running 2 tests
test tests::set_and_get ... ok
test tests::delete_returns_old_value ... ok

test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

4. lépés: Ellenőrizzük a CLI-vel. A `cargo run -p rkv-cli -- set hello world` parancs kimenete `OK`. Egy ezt követő `cargo run -p rkv-cli -- get hello` viszont `NOTFOUND`-ot ad, mert minden folyamat új, üres `Store`-ral indul (lásd az rkv mintaprojekt indítása: Cargo-workspace és első CLI 4. lépését); a tároló helyes működését egy folyamaton belül a fenti unit tesztek igazolják. A `set` a kulcs utáni első argumentumot veszi értéknek; a szóközt tartalmazó érték idézőjelek közt adható meg (`set hello "szép világ"`), mert a shell ezt egyetlen argumentumként adja át.

Ellenőrzési pont. A `cargo build --workspace` hibátlan, a `cargo test -p rkv-core` 2 sikeres tesztet futtat. Az `rkv-core` API-ja: `Store::new`, `set(String, Vec<u8>)`, `get(&str) -> Option<&Vec<u8>>`, `delete(&str) -> Option<Vec<u8>>`, `keys() -> Vec<String>`, `clear()`; a `Store` `Debug` és `Default`.

Továbbvitt eredmény. Ebben a laborban rögzítettük a `Store` struktúrát (`rkv-core/src/lib.rs`): privát `data: HashMap<String, Vec<u8>>` mezővel, `new` asszociált függvénnyel, valamint `set`, `get`, `delete`, `keys` és `clear` metódusokkal. A `get` `Option<&Vec<u8>>`-t ad vissza, amit a Az enum mint összeg-típus és az Option a null helyett alfejezet tanít. A `set` `&mut self`-et kér, ami az Ownership: minden értéknek egy tulajdonosa van alfejezetben lesz a középpontban. A jelenlegi `set` és `delete` nem ad hibát, ezt a Result, a ? operátor és a panic helye alfejezet pontosítja.

4.1.5 Tipikus hibák és átállási csapdák

1. **`self` használata `&self` helyett.** C++-ban a tagfüggvény általában `this`-en dolgozik, de az objektumot a hívó nem „veszíti el”. Rustban a `self` átveszi a tulajdont, és a metódus hívása után az eredeti változó érvénytelen. Ha egy metódus csak olvassa az adatot, akkor `&self`-et kell használni.

```
// Ez a változat szándékosan nem fordul le:
struct S { x: i32 }
impl S {
    fn get_x(self) -> i32 { self.x }
}
fn main() {
    let s = S { x: 5 };
    println!("{}", s.get_x());
    println!("{}", s.x); // hiba: s mozgatva lett
}
```

A fordító hibakódja `E0382` (use of moved value). A javítás: `fn get_x(&self) -> i32 { self.x }`.

2. **Gettermásolás.** C++-ban a getter gyakran értéket ad vissza, mert a `const std::string&` visszaadása élettartam-problémákat okozhat. Rustban a `&str` vagy `&T` visszaadása biztonságos, mert a kölcsönzés szabályai garantálják, hogy a referenciák érvényesek maradnak. Ne klónozzunk minden getterben.
3. **Konstruktor-túlterhelés hiánya.** C++-ban természetes, hogy egy osztálynak több konstruktora van. Rustban nincs túlterhelés; a `new` konvenció egyetlen függvény, de írhatunk több, nevükben különböző asszociált függvényt: `from_config`, `with_capacity`, vagy a `Default` trait-et és a builder mintát használhatjuk.
4. **A `this` hiánya.** C++-ban a mezők elérése a tagfüggvényben automatikus. Rustban mindig `self.mező_neve` formát kell írni; a `mező_neve` önmagában a metódusban nem hivatkozik a struktúra mezőjére.

4.1.6 Feladatok

4.1.6.1 (a) Fordul-e, és miért?

```rust
struct Buffer {
    data: Vec<u8>,
}

impl Buffer {
    fn clear(self) {
        // törli a tartalmat
    }
}

fn main() {
    let mut buf = Buffer { data: vec![1, 2, 3] };
    buf.clear();
    println!("{}", buf.data.len());
}
```

Válaszolj: lefordul-e ez a kód? Mi a hiba, és hogyan javítanád?

4.1.6.2 (b) Írd át C++-ból Rustra

Az alábbi C++ osztállyal modellezz egy hőmérséklet-érzékelő mintát. Készítsd el a Rust megfelelőt strukturával, `impl` blokkal, asszociált `new` függvénnyel és a megfelelő `self`-fajtákkal.

```cpp
class TemperatureReading {
    double celsius_;
public:
    TemperatureReading(double celsius) : celsius_(celsius) {}

    double celsius() const { return celsius_; }
    double fahrenheit() const { return celsius_ * 9.0 / 5.0 + 32.0; }

    void adjust(double delta) { celsius_ += delta; }
};
```

4.1.6.3 (c) Bővítsd az rkv `Store`-t

Adj az `rkv-core/src/lib.rs` `Store` `impl` blokkjához egy `contains` metódust (`&self`, `&str` paraméter, `bool` visszatérés), valamint egy `replace` metódust, amely `&mut self`-et, `String` kulcsot és `Vec<u8>` értéket vár, és visszaadja a korábbi értéket `Option<Vec<u8>>`-ként. Futtass hozzá unit tesztet.

4.1.6.4 (d) Hibajavítás

Az alábbi kód szándékosan hibás. Azonosítsd a hibát, és javítsd úgy, hogy a `Counter` működjön.

```
struct Counter {
    count: u32,
}

impl Counter {
    fn new() -> Self {
        Self { count: 0 }
    }

    fn increment(self) {
        self.count += 1;
    }

    fn value(&self) -> u32 {
        self.count
    }
}

fn main() {
    let mut counter = Counter::new();
    counter.increment();
    counter.increment();
    println!("{}", counter.value());
}
```

4.1.7 Megoldások

4.1.7.1 (a)

Nem fordul le. A `clear` metódus `self`-et vár, ami átveszi a `buf` tulajdont. A hívás után `buf` mozgatott, érvénytelen, ezért a `buf.data.len()` hozzáférése `E0382` hibát okoz. Javítás: a metódus `&mut self`-et használjon, ha nem akarja megsemmisíteni a puffert:

```
struct Buffer {
    data: Vec<u8>,
}

impl Buffer {
    fn clear(&mut self) {
        self.data.clear();
    }
}
```

4.1.7.2 (b)

```
struct TemperatureReading {
    celsius: f64,
}

impl TemperatureReading {
    fn new(celsius: f64) -> Self {
        Self { celsius }
    }

    fn celsius(&self) -> f64 {
        self.celsius
    }

    fn fahrenheit(&self) -> f64 {
        self.celsius * 9.0 / 5.0 + 32.0
    }

    fn adjust(&mut self, delta: f64) {
        self.celsius += delta;
    }
}
```

A `celsius` és `fahrenheit` metódus csak olvas, ezért `&self`; az `adjust` módosít, ezért `&mut self`. A `new` asszociált függvény nem vesz fel `self`-et.

4.1.7.3 (c)

Az `rkv-core/src/lib.rs` bővítése:

```
impl Store {
    // ... korábbi metódusok ...

    pub fn contains(&self, key: &str) -> bool {
        self.data.contains_key(key)
    }

    pub fn replace(&mut self, key: String, value: Vec<u8>) -> Option<Vec<u8>> {
        self.data.insert(key, value)
    }
}
```

Unit teszt a `tests` modulban:

```
#[test]
fn contains_and_replace() {
    let mut store = Store::new();
    store.set("a".to_string(), vec![1]);
    assert!(store.contains("a"));
    assert!(!store.contains("b"));

    let old = store.replace("a".to_string(), vec![2, 3]);
    assert_eq!(old, Some(vec![1]));
    assert_eq!(store.get("a"), Some(&vec![2, 3]));
}
```

A `HashMap::insert` már eleve visszaadja a korábbi értéket, így a `replace` egyszerű továbbítás.

4.1.7.4 (d)

A hiba a `fn increment(self)` szignatúrában van. A `self` átveszi a `Counter` tulajdont, ezért az első `counter.increment()` hívás után a `counter` érvénytelen, és a második hívás `E0382` hibát okoz. Javítás: a metódus `&mut self`-et kapjon, és a hívónál a `counter` legyen `mut` – ez már így is megvan.

```
struct Counter {
    count: u32,
}

impl Counter {
    fn new() -> Self {
        Self { count: 0 }
    }

    fn increment(&mut self) {
        self.count += 1;
    }

    fn value(&self) -> u32 {
        self.count
    }
}
```

4.1.8 Rövid összefoglaló

A Rust `struct` három alakban létezik: named-field, tuple és unit struct. A viselkedés nem a struktúra része, hanem külön `impl` blokkban definiáljuk. A metódusok első paramétere explicit `self`, `&self` vagy `&mut self`, ami pontosan meghatározza, hogyan fér hozzá a hívó objektumhoz. Nincs konstruktor, nincs túlterhelés, nincs implementációs öröklés; a `new` asszociált függvény konvenció, a `Default` trait és a builder minta pedig helyettesíti a konstruktor-túlterhelést. A laborban rögzítettük az rkv `Store` típusát (`rkv-core/src/lib.rs`): privát `data: HashMap<String, Vec<u8>>` mezővel, `new` asszociált függvénnyel, és `set`, `get`, `delete`, `keys`, `clear` metódusokkal. Az értékek `Vec<u8>` típusúak, a kulcsok `String` típusúak. **Továbbvitt eredmény:** a `Store` a Az enum mint összeg-típus és az Option a null helyett alfejezetben az `Option` használata, a Ownership, borrowing, lifetime: a Rust szíve rész fejezeteiben a kölcsönzési szabályok, a Result, a ? operátor és a panic helye alfejezetben pedig a hibakezelés szempontjából kerül továbbfejlesztésre.

4.1.9 Források és ellenőrzési hiányok

A fejezet a Rust Book (https://doc.rust-lang.org/book/), a Rust Reference (https://doc.rust-lang.org/reference/) és a standard könyvtár dokumentációjának (https://doc.rust-lang.org/std/) stabil részeire épül: a `struct` és `impl` szintaxisra, a `self` paraméterekre és a `HashMap` alapvető API-jára. A `HashMap::get &str` kulccsal való használata a `Borrow` trait-nek köszönhető, amelyet a A standard trait-ek: konverziók, formázás, Default, Deref, Borrow alfejezet tárgyal.

4.2 Az enum mint összeg-típus és az Option a null helyett

4.2.1 Motiváció

Egy C++-fejlesztő számára a „típus és a hozzá tartozó adat” modellezése több eszköz között oszlik meg. Az `enum class` típusbiztos, de nem hordoz adatot: ha egy parancsfeldolgozó `Get`, `Set`, `Delete` és `List` műveleteket kezel, az `enum class` önmagában csak a diszkriminánst adja, a kulcsot és az értéket külön `std::optional` vagy `std::variant` mezőkben kell tárolni. Az öröklés-alapú megoldás szétteríti a logikát, a `std::visit` pedig kevésbé olvasható, mint egy kimerítő mintaillesztés. A Rust válasza az **összeg-típus (sum type)** nyelvi szinten: az `enum` variánsai közvetlenül hordoznak adatot, így a fordító biztosítja, hogy minden lehetséges alakot kezeljünk. Ugyanebből a családból jön az `Option<T>`, amely a hiányzó értéket nem null pointerként, hanem külön típusként modellezi.

4.2.2 Fogalmak és szabályok

Definíció. A Rust `enum` olyan zárt összeg-típus, amely véges számú variánst definiál. Minden variáns hordozhat adatot, akár név nélküli tuple-szerű mezőkkel, akár névvel ellátott mezőkkel; a variánshalmaz később (a modulon belül) nem bővíthető anélkül, hogy ne jelezné a fordító a hiányzó mintákat.

Szabály. Az enum variánsainak adatait csak mintaillesztéssel lehet elérni. A `match` kimerítő: ha egy `enum` értékét `match`-csel vizsgáljuk, minden variánst le kell fedni, vagy `_` ágat kell használnunk. A variánsok adatai a mintaillesztésben kötött változókkal bonthatók ki.

Definíció. Az `Option<T>` a standard könyvtár generikus enumja:

```
pub enum Option<T> {
    None,
    Some(T),
}
```

A `None` nem null pointer, hanem egy külön variáns; a `Some(T)` a jelen lévő értéket hordozza. Az `Option` használata kötelezővé teszi a hiányzó eset kezelését, mert a `T` és az `Option<T>` különböző típusok.

Szabály. Az `Option<T>` értékéből a belső `T` nem nyerhető ki implicit konverzióval: `Some(5)` nem adható át olyan helyre, ahol `i32`-et várunk. Az eléréshez `match`, `if let`, `unwrap_or`, `unwrap_or_else`, `map`, `and_then` vagy más explicit kombinator szükséges.

4.2.3 C++ és Rust összevetés

Vegyünk egy parancsot, amely egy kulcs–érték tárolóban `Get`, `Set`, `Delete` vagy `List` műveletet jelent. C++17-ben tipikus megoldás a `std::variant`:

```
#include <type_traits>
#include <variant>
#include <string>
#include <optional>
#include <vector>

struct Get { std::string key; };
struct Set { std::string key; std::string value; };
struct Delete { std::string key; };
struct List { std::optional<std::string> prefix; };
```

```cpp
using Command = std::variant<Get, Set, Delete, List>;
```

A feldolgozás `std::visit`-tel történik, ahol a látogató gyakran egy külön osztály vagy egy általános lambda:

```cpp
std::string to_line(const Command& cmd) {
    return std::visit([](const auto& c) -> std::string {
        using T = std::decay_t<decltype(c)>;
        if constexpr (std::is_same_v<T, Get>)
            return "GET " + c.key;
        else if constexpr (std::is_same_v<T, Set>)
            return "SET " + c.key + " " + c.value;
        else if constexpr (std::is_same_v<T, Delete>)
            return "DEL " + c.key;
        else
            return c.prefix ? "LIST " + *c.prefix : "LIST";
    }, cmd);
}
```

Rustban ugyanez egyetlen `enum`:

```rust
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum Command {
    Get(String),
    Set(String, String),
    Delete(String),
    List(Option<String>),
}

impl Command {
    pub fn to_line(&self) -> String {
        match self {
            Command::Get(key) => format!("GET {key}"),
            Command::Set(key, value) => format!("SET {key} {value}"),
            Command::Delete(key) => format!("DEL {key}"),
            Command::List(prefix) => match prefix {
                Some(p) => format!("LIST {p}"),
                None => "LIST".to_string(),
            },
        }
    }
}
```

A különbség lényege nem az, hogy a C++ `std::variant` „nem ismeri” az alternatíváit: ismeri (`index()`, `holds_alternative`), és a `std::visit` látogatója értékkel is visszatérhet (mint fent a `to_line`), valamint fordítási hibát ad, ha a látogató nem fedi le az összes alternatívát. A valós különbségek: (1) a Rust enum variánsai *névvel ellátott* nyelvi elemek (`Command::Get`), míg a `std::variant` alternatívái típusok vagy indexek, ezért azonos típusú alternatívákhoz burkolók kellenek; (2) a minta szintaxisa egyszerre bontja ki az adatot, beágyazva és őrfeltételekkel (`Some((k, v)) if ...`), a C++ `std::visit` ehhez `if constexpr` vagy túlterhelt látogató kell; (3) a Rust `match` kimerítőségét a nyelv ellenőrzi akkor is, ha beágyazott mintákról van szó, és új variáns felvételekor a fordító minden `match` helyet jelez.

4.2.4 Kidolgozott példa: `Option<T>` a hiányzó értékre

Tegyük fel, hogy egy konfigurációs térképet keresünk. C++-ban a hiányzó kulcsot gyakran null pointer vagy `std::optional` jelzi:

```cpp
#include <unordered_map>
#include <string>
#include <optional>

std::optional<std::string> find_value(
    const std::unordered_map<std::string, std::string>& map,
    const std::string& key)
{
    auto it = map.find(key);
    if (it == map.end()) return std::nullopt;
    return it->second;
}
```

A hívó könnyen elfelejtheti a `std::nullopt` ellenőrzését, vagy `*opt`-tal hozzáférhet anélkül, hogy a hiányzó esetet kezelné. Rustban a visszatérési típus `Option<&String>`:

```rust
use std::collections::HashMap;

fn find_value<'a>(map: &'a HashMap<String, String>, key: &str) -> Option<&'a String> {
    map.get(key)
}
```

A hívó nem tudja a `&String` értéket közvetlenül használni, mert az `Option<&String>` típusú. Az `Option` metódusai explicit kezelést kínálnak:

```rust
let mut map = HashMap::new();
map.insert("host".to_string(), "localhost".to_string());

// Alapértelmezett érték, ha hiányzik:
let host = map.get("host").map(|s| s.as_str()).unwrap_or("127.0.0.1");

// Láncolt transzformáció:
let port = map.get("port").and_then(|s| s.parse::<u16>().ok());

// Átalakítás Result-tá:
let value = map.get("key").cloned().ok_or("missing key");
```

Itt a `map` `Option<&String>`-et ad; a `.map()` átalakítja a belső `&String`-et `&str`-ré, az `unwrap_or` pedig a `None` esetet kezeli. Az `and_then` olyan függvényt vár, amely maga is `Option`-t ad vissza – a `.parse()` például `Result`-ot ad, amit `.ok()`-kal alakítunk `Option`-né. Az `ok_or` a `None`-t `Err`-ré alakítja, ami átvezet a Result, a `?` operátor és a panic helye alfejezethez. Ezt a részt itt csak előkészítjük, a részletes hibakezelés később következik.

Az `Option<&T>` reprezentációja különösen fontos. Mivel a Rust referenciái soha nem lehetnek nullok, a fordító a null mutatóértéket használhatja a `None` jelölésére. Ezt niche-optimalizációnak nevezzük, és bizonyos típusokra a dokumentáció *garantálja*: az `Option<&T>`, `Option<&mut T>`, `Option<Box<T>>` és `Option<NonNull<T>>` mérete és ABI-ja megegyezik a mögöttes mutatóéval, ha `T: Sized` (lásd az `std::option` modul „Representation” szakaszát). Sized `T`-re ez egy gépi szó, x86-64-en 8 bájt. Nem `Sized` célnál (`&str`, `&[T]`, `&dyn Trait`) a referencia *széles* (két gépi szó), és az `Option` mérete ilyenkor is a referenciáéval egyezik (x86-64-en 16 bájt), de ezt a dokumentáció már nem ABI-garanciaként, csak megfigyelt elrendezésként rögzíti. A `NULL` tehát nem a programozó kezébe kerül, hanem a típusrendszer belsejébe.

4.2.5 Labor: az rkv `Command` enumja

Munkakönyvtár: `~/rust-labor/rkv/`

Ebben a laborban létrehozzuk az rkv szöveges protokolljának parancsait reprezentáló `Command` enumot. A protokoll szerint a kliens a következő sorokat küldheti:

```
GET <kulcs>
SET <kulcs> <érték>
DEL <kulcs>
LIST [<prefix>]
```

A kulcs `String`. A `Set` variáns értéke is `String`, mert a parancs a *szöveges* protokoll egy sora: a hálózaton vagy a parancssorban érkező érték mindig szöveg. A tárolóban viszont az érték `Vec<u8>`, ahogy a Structok és metódusok: impl blokkok, asszociált függvények alfejezet rögzítette; a kettő közti átalakítás a parancs végrehajtásakor történik (`value.into_bytes()`, illetve kiíráskor `String::from_utf8`). Ez a kettősség az egész könyvben érvényes: a protokoll szöveges, a tároló bájtalapú.

1. Hozzuk létre a `rkv-core` könyvtárban a `command.rs` modult:

Kódblokk — fájl: `rkv-core/src/command.rs` · művelet: új fájl · ellenőrzés: a 2. lépés után `cargo build -p rkv-core`

```rust
//! A szöveges protokoll parancsai.

/// A kliens által küldött parancs.
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum Command {
    Get(String),
    Set(String, String),
    Delete(String),
    List(Option<String>),
}

impl Command {
    /// Visszaadja a parancs szöveges formáját.
    pub fn to_line(&self) -> String {
        match self {
            Command::Get(key) => format!("GET {key}"),
            Command::Set(key, value) => format!("SET {key} {value}"),
            Command::Delete(key) => format!("DEL {key}"),
            Command::List(prefix) => match prefix {
                Some(p) => format!("LIST {p}"),
                None => "LIST".to_string(),
            },
        }
    }
}
```

2. Deklaráljuk a modult a `rkv-core/src/lib.rs`-ben:

Kódblokk — fájl: `rkv-core/src/lib.rs` · művelet: beillesztés a fájl elejére, a `use` sorok elé · ellenőrzés: `cargo build -p rkv-core`

```rust
pub mod command;
```

3. Adjunk hozzá egységteszteket a `command.rs` fájl végéhez:

Kódblokk — fájl: `rkv-core/src/command.rs` · művelet: beillesztés a fájl végére · ellenőrzés: `cargo test -p rkv-core` → 4 sikeres teszt (a 2 új és a Structok és metódusok: impl blokkok, asszociált függvények alfejezetbeli 2)

```rust
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
```

```
    fn list_prefix_none_has_no_trailing_space() {
        let cmd = Command::List(None);
        assert_eq!(cmd.to_line(), "LIST");
    }

    #[test]
    fn set_formats_key_and_value() {
        let cmd = Command::Set("name".into(), "rust".into());
        assert_eq!(cmd.to_line(), "SET name rust");
    }
}
```

4. Futtassuk a teszteket:

```
cargo test -p rkv-core
```

Várt kimenet (rövidítve; a tesztek sorrendje változhat):

```
running 4 tests
test command::tests::list_prefix_none_has_no_trailing_space ... ok
test command::tests::set_formats_key_and_value ... ok
test tests::delete_returns_old_value ... ok
test tests::set_and_get ... ok

test result: ok. 4 passed; 0 failed; ...
```

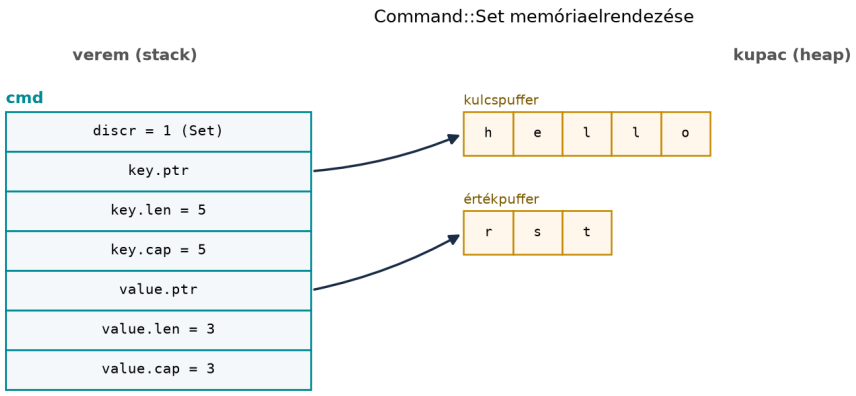
A `derive(Debug, Clone, PartialEq, Eq)` azt jelenti, hogy a parancs kiírható, klónozható és összehasonlítható. A `Clone` itt még kényelmes, de a későbbi alfejezetekben látni fogjuk, hogy a Rust alapértelmezésben mozgat, nem klónoz.

Továbbvitt eredmény. A `Command` enum a `rkv-core/src/command.rs` fájlban él. Ezt a típust használja a Mintaillesztés: match, if let, let else és a minták alfejezet a kimerítő minták bemutatására, a Result, a ? operátor és a panic helye alfejezet a protokoll-elemző hibakezelésére, valamint a Deklaratív és procedurális makrók alfejezet a derive-makrók működésének magyarázatára.

Ellenőrzési pont. Az `rkv-core` a `Store` mellett a `command::Command` enumot is exportálja (`Get`, `Set(String, String)`, `Delete`, `List(Option<String>)`, `to_line`); a `cargo test -p rkv-core` 4 sikeres tesztet futtat, az `rkv-cli` változatlan.

4.2.6 A memóriaelrendezés

Az alapértelmezett (`repr(Rust)`) enum elrendezését a nyelv szándékosan *nem* rögzíti: a fordító szabadon választhat diszkrimináns-méretet, mezősorrendet, és a diszkriminánst el is rejtheti egy mező tiltott bitmintái között (niche). Garantált elrendezés csak `#[repr(C)]`, `#[repr(u8)]` és hasonló attribútumokkal, illetve az `Option` fent felsorolt eseteire létezik. Az alábbi ábra ezért **szemléltető modell**: egy külön diszkriminánssal és a legnagyobb variáns adatterületével mutatja a `Command::Set(String, String)` értéket (kulcs `"hello"`, érték `"rst"`), mert így a legkönnyebb megérteni, mit tárol egy enum:



4.1. ábra: Command::Set memóriaelrendezése - A diszkrimináns megmondja, melyik variáns él; a String-ek ptr/len/cap triót tárolnak, a tényleges adat a kupacon van.

A modellben a `cmd` veremváltozó a variáns jelzését és a két `String` hármast (mutató, hossz, kapacitás) tartalmazza; a tényleges bájtsorozatok a kupacon vannak. A `Get` vagy `Delete` variáns csak egy `String` hármast használ, ezért az enum mérete legalább a legnagyobb variáns mérete.

A ténylegesen mért méret ettől eltérhet. Rust 1.99.1-gyel, `x86_64-unknown-linux-gnu` targeten a `std::mem::size_of` szerint a `String` 24, az `Option<String>` is 24, a `Command` pedig 48 bájt, vagyis pontosan két `String`: a fordító *nem* tárol külön diszkriminánst, hanem a `String` kapacitásmezőjének tiltott értékei (a kapacitás legfeljebb `isize::MAX` lehet) között kódolja a variánst és az `Option` `None`-ját. Ez a niche-optimalizáció megfigyelt eredménye, nem garancia: más toolchain vagy target más elrendezést adhat, ezért kódunk soha ne építsen rá. A `String` adatmutatója egyébként sosem null, üres `String` esetén sem (ilyenkor egy nem null, de nem dereferálható „dangling” címet tárol), tehát a `None` jelölésére nem null mutató szolgál.

4.2.7 Tipikus hibák és átállási csapdák

1. **`unwrap()` mindenhol az `Option`-ön.** A C++-fejlesztő, aki megszokta a null pointer ellenőrzésének elhagyását, hajlamos `unwrap()`-pel „lerövidíteni” a kódot. Ez visszahozza a null pointer hibát: `None` esetén pánik. Az idiomatikus megoldás a `match`, `if let`, vagy az `unwrap_or`-kombinatorok.

2. **`std::variant` szokás átvitele.** Aki `std::variant`-hoz van szokva, gyakran külön structokat definiál minden variánshoz, és az enum csak egy diszkriminánst tartalmaz. Ez felesleges: a Rust enum variánsai közvetlenül hordozhatják az adatokat, és a `match` automatikusan kibontja őket.

3. **A `None` és az üres érték összekeverése.** Az `Option<String>` esetén a `None` azt jelenti, hogy nincs érték; a `Some(String::new())` azt, hogy van egy üres sztring. A kettő szemantikailag különbözik, és a protokollban a `LIST` parancsnál a `None` a prefix nélküli listázást jelenti, míg a `Some("")` üres prefixet adna – ezt a labor `to_line` függvénye kezeli is.

4. **Klónozás a borrow checker „legyőzésére”.** Az `Option<&T>` helyett sokan `Option<T>`-t használnak, majd `clone()`-t hívnak, hogy a kölcsönzési hibákat megkerüljék. Ez felesleges allokációt okoz, és elkeni a tulajdonosi viszonyokat. Az `Option<&T>` gyakran elég, és a niche-optimalizáció miatt a mérete is azonos.

4.2.8 Feladatok

4.2.8.1 a) Lefordul-e és miért?

```
let x: Option<i32> = Some(5);
let y = x + 1;
```

Válaszold meg, hogy lefordul-e, és ha nem, mi a hiba lényege.

4.2.8.2 b) C++-kód átírása

Írd át Rust enummá a következő C++ `std::variant`-ot:

```
using Value = std::variant<std::monostate, int, double, std::string>;
```

Írj egy `describe` függvényt, amely `match`-csel kiírja, hogy az adott variáns milyen típusú.

4.2.8.3 c) Implementáció az rkv-ban

Bővítsd a `rkv-core/src/command.rs` fájlban lévő `Command` enumot egy `Ping` variánssal. Frissítsd a `to_line` metódust, hogy a `Ping` esetén `"PING"`-et adjon vissza. Adj hozzá egy tesztet is.

4.2.8.4 d) Hibajavítás

Az alábbi függvény pánikol, ha a kulcs nincs a térképben:

```
fn get_or_panic(map: &HashMap<String, String>, key: &str) -> String {
    map.get(key).unwrap().clone()
}
```

Refaktoráld a függvényt úgy, hogy `None` esetén `"missing"` értéket adjon vissza, és ne pánikoljon. Mutass be legalább két különböző megoldást (`match`, `unwrap_or`, `map` + `unwrap_or`).

4.2.9 Megoldások

4.2.9.1 a)

A kód nem fordul le. Az `Option<i32>` és az `i32` különböző típusok; a `+` operátor nem értelmezett `Option<i32>` és `i32` között. A hiba lényege: a `Some(5)` értéket előbb ki kell bontani, például `x.unwrap_or(0) + 1` vagy `match x { Some(n) => n + 1, None => 0 }` módon.

4.2.9.2 b)

```rust
#[derive(Debug, Clone, PartialEq)]
pub enum Value {
    Empty,
    Int(i32),
    Float(f64),
    Text(String),
}

impl Value {
    pub fn describe(&self) -> &'static str {
        match self {
            Value::Empty => "empty",
            Value::Int(_) => "int",
            Value::Float(_) => "float",
            Value::Text(_) => "string",
        }
    }
}
```

A `std::monostate` megfelelője a `Empty` variáns adat nélkül; a `int`, `double` és `std::string` a `Int`, `Float` és `Text` variánsok adattal.

4.2.9.3 c)

```rust
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum Command {
    Get(String),
    Set(String, String),
    Delete(String),
    List(Option<String>),
    Ping,
}

impl Command {
    pub fn to_line(&self) -> String {
        match self {
            Command::Get(key) => format!("GET {key}"),
            Command::Set(key, value) => format!("SET {key} {value}"),
            Command::Delete(key) => format!("DEL {key}"),
            Command::List(prefix) => match prefix {
                Some(p) => format!("LIST {p}"),
                None => "LIST".to_string(),
            },
            Command::Ping => "PING".to_string(),
        }
    }
}
```

Új teszt:

```rust
#[test]
fn ping_formats_as_ping() {
    let cmd = Command::Ping;
    assert_eq!(cmd.to_line(), "PING");
}
```

Ez a változat előkészíti a Mintaillesztés alfejezetet, ahol a `Ping` mintát is vizsgáljuk.

4.2.9.4 d)

Első megoldás `match`-csel:

```
fn get_or_missing(map: &HashMap<String, String>, key: &str) -> String {
    match map.get(key) {
        Some(v) => v.clone(),
        None => "missing".to_string(),
    }
}
```

Második megoldás `map` + `unwrap_or`:

```
fn get_or_missing(map: &HashMap<String, String>, key: &str) -> String {
    map.get(key).map(|s| s.clone()).unwrap_or_else(|| "missing".to_string())
}
```

Harmadik, tömörebb változat `cloned` + `unwrap_or`:

```
fn get_or_missing(map: &HashMap<String, String>, key: &str) -> String {
    map.get(key).cloned().unwrap_or_else(|| "missing".to_string())
}
```

Mindhárom ugyanarra a szerződésre épül: hiányzó kulcsra a `"missing"` szöveg, jelenlévő kulcsra az érték *tulajdonolt másolata*; ellenőrzött kimenet a `{a: "x", b: "missing"}` térképen: `a` → `x`, `b` → `missing`, `c` → `missing`. (Figyelem: a szerződés nem különbözteti meg a hiányzó kulcsot attól, amikor a kulcs értéke maga a `"missing"` szöveg, ahogy a `b` kulcs mutatja; ha a hívónak ezt tudnia kell, `Option<String>` visszatérés kell.) A `cloned()` az `Option<&String>`-et `Option<String>`-gé alakítja: a térkép marad az értékek tulajdonosa, a függvény pedig tulajdonolt `String`-et ad vissza, ezért a hivatkozott értéket le kell másolni (`String::clone`). Ugyanezt teszi a `map(|s| s.clone())`.

4.2.10 Rövid összefoglaló

A Rust `enum` zárt összeg-típus, amelynek variánsai adatot hordoznak; ez a C++ `enum class` + `std::variant` + öröklés kombinációját egyetlen nyelvi elemben egyesíti. A variánsadatokat `match`-csel érjük el, a kimerítőség garantálja a hiányzó esetek fordítási idejű jelzését. Az `Option<T>` (`Some(T)` / `None`) kiváltja a null pointert: a hiányzó érték külön típus, amelynek kezelése kötelező. Az `Option` kominatorai (`map`, `and_then`, `unwrap_or`, `ok_or`) a hiányzó esetek explicit kezelését teszik lehetővé. A laborban létrehozott `Command` enum a `rkv-core/src/command.rs` fájlban a `Get(String)`, `Set(String, String)`, `Delete(String)` és `List(Option<String>)` variánsokkal modellezi a szöveges protokollt; a `to_line` metódus a parancs szöveges alakját adja vissza. **Továbbvitt eredmény:** ezt a `Command` típust használja a Mintaillesztés, a Result és `?` és a Makrók alfejezet.

4.2.11 Források és ellenőrzési hiányok

- `std::option` modul, „Representation” szakasz (a garantált `Option`-elrendezések listája): https://doc.rust-lang.org/std/option/index.html#representation
- The Rust Reference, „Type layout” (a `repr(Rust)` elrendezése nem specifikált): https://doc.rust-lang.org/reference/type-layout.html
- The Rust Programming Language, 6. fejezet „Enums and Pattern Matching”: https://doc.rust-lang.org/book/ch06-00-enums.html

A méretadatok mérések (Rust 1.99.1, `x86_64-unknown-linux-gnu`, `std::mem::size_of`), nem nyelvi garanciák. A labor kimenetei a könyv rögzített toolchainjére vonatkoznak (lásd Eszközlánc: rustup, cargo, rustfmt, clippy, dokumentáció).

4.3 Mintaillesztés: match, if let, let else és a minták

4.3.1 Motiváció

A C++17 `switch` utasítása csak egész típusokon működik; enumok esetén is csak annyit tud, hogy a felsorolt értékekhez ugrik. Ha egy `enum class`-szal dolgozunk, a fordító legfeljebb figyelmeztet, ha egy új értéket kihagytunk, de nem tud típusos destrukturálást, kimerítőség-ellenőrzést vagy bonyolult feltételeket. A `std::variant` és a `std::visit` már közelebb visz a célhoz, de a látogató sablonok és a generikus lambda körülményesek, és a C++17 kódban a hibaüzenetek gyakran sablonmélységbe vezetnek.

A Rust `match` kifejezése éppen azt a hiányt pótolja, amit egy C++-fejlesztő a `switch`-ben és a `std::visit`ben keres: típusos, kimerítő, értékekre, mintákra és feltételekre egyaránt illeszthető, és kifejezésként használható. A fordító fordítási időben észleli, ha egy enum új variánsa után elfelejtettünk egy ágat frissíteni – ez a refaktorálás egyik legerősebb eszköze.

4.3.2 Fogalmak és szabályok

Definíció. A **mintaillesztés** (pattern matching) olyan szerkezet, amely egy értéket egy **mintával** (pattern) vet össze, és sikeres illeszkedés esetén a minta változóihoz kötött értékekkel végrehajt egy kódágat. A Rust `match` kifejezés minden ága `minta => kifejezés` alakú.

Definíció. A `match` **kimerítő** (exhaustive), ha a célérték típusának minden lehetséges értékéhez tartozik legalább egy illeszkedő minta. Enumoknál a fordító ezt ellenőrzi, és ha hiányzik egy variáns, hibát ad.

Szabály. A `match` kifejezés minden ága ugyanarra a típusra értékelődik ki. A minták (és szükség esetén a guardok) sorrendben kerülnek vizsgálatra, és az első illeszkedő ág hajtódik végre; az ágtörzsek közül csak a kiválasztott fut. A közös típus koercióval is kialakulhat, nem feltétlenül azonos már kezdetben.

Szabály. A `_` minta bármely értékre illeszkedik, de nem köt értéket. A `_` használata „mindent elnyelő” ágat ad, amely kimerítővé teszi a match-et; ezt azonban megfontoltan kell használni, mert elrejtheti az új variáns kezelését.

Definíció. A **guard** egy `if` feltétel a minta után, amely tovább szűkíti az illeszkedést: `Some(x) if x > 0 => ...`. Guard csak akkor fut le, ha a minta már illeszkedett, és ha a guard hamis, a következő ág próbálkozik.

Definíció. A **@ binding** lehetővé teszi, hogy egy részmintához tartozó teljes értéket elnevezzük: `pair @ (x, y) => ...` vagy `Some(n @ 1..=5) => ...`.

Definíció. A **refutable** minta olyan, amely nem feltétlenül illeszkedik (pl. `Some(x)`), az **irrefutable** minta mindig illeszkedik (pl. `x`, `_`, `(a, b)`). A `let` utasítás csak irrefutable mintát fogadhat el; a `let else` ezt a korlátot oldja fel azzal, hogy ha a minta nem illeszkedik, az `else` ág fut le.

4.3.3 C++ és Rust összevetés

Vegyünk egy C++17 példát, ahol egy `std::variant`-ot kell feldolgoznunk látogatóval:

```cpp
#include <variant>
#include <string>
#include <iostream>

using Value = std::variant<int, std::string>;

struct Printer {
    void operator()(int i) const { std::cout << "int: " << i << '\n'; }
    void operator()(const std::string& s) const { std::cout << "string: " << s << '\n'; }
};

int main() {
    Value v = 42;
    std::visit(Printer{}, v);
}
```

Rustban ugyanez `match`-csel:

```rust
enum Value {
    Int(i32),
    String(String),
}

fn main() {
    let v = Value::Int(42);
    match v {
        Value::Int(i) => println!("int: {i}"),
        Value::String(s) => println!("string: {s}"),
    }
}
```

A különbség: - A Rust `match` **kifejezés**, visszatérési értéke van; a C++ `std::visit` is képes értéket visszaadni (az eredményt a látogató adja, és minden ágnak azonos visszatérési típusúnak kell lennie, mint a `match` ágainak), a különbség a szintaxisban és a mintákban van. - A Rust enum variánsai közvetlenül tartalmazzák az értékeket, a minta pedig egyszerre bontja ki azokat; a C++ `std::variant` alternatíváit az indexek vagy a `std::get` kezeli, ami hiba esetén futásidejű kivételt dobhat. - Ha a Rust enumhoz új variánst veszünk fel, a `match` **fordítási hibát** ad, amíg nincs kezelve; a C++ `std::visit` a látogató minden alternatíváját lefedi, és ha egy overload hiányzik, a fordítás többnyire olvashatatlan, de fordítási hibával áll meg. A `switch` + `std::get_if` alapú kézi megközelítésnél viszont nincs nyelvi kimerítőség-ellenőrzés, csak külső eszköz (például `-Wswitch-enum` figyelmeztetés vagy statikus analízis) jelezheti a hiányzó ágat.

4.3.4 Kidolgozott példa: kimerítőség-ellenőrzés

Tekintsük a következő hibás Rust kódot, amely nem kezeli az összes variánst:

```rust
enum Color {
    Red,
    Green,
    Blue,
}

fn describe(c: Color) -> &'static str {
    match c {
        Color::Red => "piros",
```

```
        Color::Green => "zöld",
    }
}
```

Ez a változat szándékosan nem fordul le. A fordító hibaüzenetének lényege `E0004` (nem kimerítő minta):

```
error[E0004]: non-exhaustive patterns: `Color::Blue` not covered
 --> src/main.rs:8:11
  |
8 |     match c {
  |           ^ pattern `Color::Blue` not covered
```

A javítás: egészítsük ki a hiányzó ággal, vagy használjunk `_` ágat, ha tényleg minden más érték ugyanúgy kezelendő. Itt természetesebb a teljes felsorolás:

```
match c {
    Color::Red => "piros",
    Color::Green => "zöld",
    Color::Blue => "kék",
}
```

A kimerítőség-ellenőrzés a refaktorálás során válik igazán értékessé: ha később felveszünk egy `Color::Yellow` variánst, a fordító azonnal megmutatja az összes olyan `match` helyet, ahol frissíteni kell a logikát. Ez a C++ `switch`-ben csak jó esetben, figyelmeztetéssel működik.

4.3.5 Minták fajtái

A `match` ágak mintái sokfélék lehetnek:

- **Literálminta:** `1`, `"hello"`, `true`.
- **Tartományminta:** `1..=5` (zárt), `'a'..='z'`.
- **Tuple és struct destrukturálás:** `(x, y)`, `Point { x, y }`.
- **Enum variáns destrukturálás:** `Some(x)`, `Ok(v)`, `Err(e)`.
- **Alminták kombinálása:** `Some((x, y))`.
- **@ binding:** `n @ 1..=5` – az `n` a teljes értéket köti, miközben a tartományra is illeszkedik.
- **Guard:** `Some(n) if n % 2 == 0 => ...`.

Példa guard és @ binding használatára:

```
fn classify(n: i32) -> String {
    match n {
        n @ 0..=9 => format!("egyjegyű: {n}"),
        n if n < 0 => format!("negatív: {n}"),
        n => format!("nagy: {n}"),
    }
}
```

Itt az első ág a `n @ 0..=9` mintával egyszerre köti a teljes értéket és ellenőrzi a tartományt; a második ág guardot használ; a harmadik minden maradékot elkap.

4.3.6 if let, while let, let else

A `match` gyakran akkor is szükséges, ha csak egyetlen minta érdekel. Az `if let` kényelmes szintaktikai cukor, amely egyetlen refutable mintát ellenőriz:

```
let opt: Option<i32> = Some(5);
if let Some(x) = opt {
    println!("van érték: {x}");
} else {
    println!("nincs");
}
```

A `while let` hasonlóan működik, amíg a minta illeszkedik; tipikusan iterátoroknál használjuk:

```
let mut iter = vec![1, 2, 3].into_iter();
while let Some(x) = iter.next() {
    println!("{x}");
}
```

A `let else` lehetővé teszi, hogy refutable mintát használjunk egy `let`-ben, és ha nem illeszkedik, az `else` ág (amely nem térhet vissza normálisan, pl. `return`, `break`, `panic!`) fusson le:

```
let Some(config) = load_config() else {
    eprintln!("nincs konfiguráció");
    return;
};
// config itt már a kicsomagolt érték
```

4.3.7 Referencia-minták és binding-módok (előzetes)

Amikor a `match` egy referencia mögötti értéket vizsgál, a kötés alapértelmezés szerint **referencia marad**, nem másolódik. Például:

```
let opt = Some(String::from("hello"));
match &opt {
    Some(s) => println!("{s}"), // s típusa &String
    None => {}
}
```

Itt `opt` típusa `Option<String>`, de a match `&opt`-ra illeszkedik, ezért az `s` kötés `&String` típusú. Ha a belső értéket mozgatni akarnánk, a `match opt` kellene, de akkor az `opt` elveszne. A `ref` kulcsszóval explicitté tehetjük, hogy referenciát kössünk; a `&` a mintában pedig referenciát vár. Ez a téma a Borrowing fejezetben mélyül el; itt elég annyi, hogy a binding-módok határozzák meg, hogy a minta által kötött változó milyen módon (érték, `&`, `&mut`) kapja meg az értéket.

4.3.8 Labor: az rkv parancsfeldolgozója match-csel

Munkakönyvtár: `~/rust-labor/rkv/`

Ebben a laborban az Az enum mint összeg-típus és az Option a null helyett alfejezetben létrehozott `Command` enumhoz (`rkv-core/src/command.rs`) parancsértelmezőt írunk, és az `rkv-cli`-t átállítjuk a protokoll szerinti parancsokra. Új `Command` típust nem hozunk létre: a protokollnak egyetlen parancstípusa van. A kiinduló állapot az enum mint összeg-típus és az Option a null helyett ellenőrzési pontja.

A protokoll nyelvtana. Az értelmező ugyanazt az előírást követi, mint a könyv összes későbbi parsere (a Saját hibatípusok: enum-hibák, thiserror, anyhow és az Az rkv hálózati szerver: TCP, protokoll, kecses leállás alfejezeté):

- egy sor egy parancs, a sorvég `\n`; a parancsszó és az argumentumok között pontosan egy szóköz áll;
- a *kulcs* (és a `LIST` prefixe) nem üres, és nem tartalmaz szóközt, más whitespace-t vagy vezérlőkaraktert;
- `GET <kulcs>` és `DEL <kulcs>`: pontosan egy argumentum; a többletargumentum hiba;
- `SET <kulcs> <érték>`: az érték a kulcs utáni szóköztől a sor végéig tart, tehát tartalmazhat szóközt; üres nem lehet, és nem tartalmazhat sortörést (`\n`, `\r`);
- `LIST` vagy `LIST <prefix>`.

Szóköz és whitespace: közös táblázat. A parancsszó és a kulcs között *pontosan egy* ASCII szóköz az elválasztó; az érték a *kulcs utáni első szóköz után* kezdődik, ezért egy szóközzel kezdődő érték érvényes, nem elválasztóhiba (`SET k  x` → érték: `" x"`). A futtatott viselkedés (`Command::from_str`):

Bemenet	Eredmény
`SET k x`	`Set("k", "x")`
`SET k  x` (két szóköz)	`Set("k", " x")`: az érték vezető szóközzel kezdődik
`SET k x\ty`	`Set("k", "x\ty")`: a tab az *értékben* megengedett
`GET a\tb`, `GET a\u{a0}b` (NBSP)	hiba: a kulcsban whitespace és vezérlőkarakter tilos (a Unicode-whitespace is)
`GET  a` (két szóköz)	hiba: többletargumentum / üres kulcs
`LIST`	`List(None)`; `LIST` (üres prefix) és `LIST \u{a0}` hiba
`SET k` / `SET  x`	hiba: üres érték / üres kulcs
`SET k v\n`, `GET a\nb`, `GET a\r`	hiba: sortörés az értelmezőnek átadott sorban

A hálózati szerver a keretezéskor már levágja a sorvégi `\n`-t és a megelőző `\r`-t (CRLF), így a `nc`-ből jövő `\r\n` sor is érvényes; az értelmező maga a sorvég karaktereket tartalmazó sort hibának veszi.

Ezen a tartományon a `Command::to_line` és az értelmező egymás inverzei: `parse_command(&cmd.to_line()) == Ok(cmd)`. Ami kívül esik rajta (szóközös kulcs, sortöréses érték, többletargumentum), azt az értelmező *hibával* utasítja el, nem csonkolja csendben.

1. Nyissuk meg a `rkv-cli/src/main.rs` fájlt, és cseréljük le a tartalmát. Az rkv mintaprojekt indítása: Cargo-workspace és első CLI alfejezetbeli kézi argumentumkezelést a protokoll szerinti értelmező és egy `execute` függvény váltja fel; a `Store` továbbra is a `main` lokális változója.

Kódblokk — fájl: `rkv-cli/src/main.rs` · művelet: teljes csere · ellenőrzés: `cargo build -p rkv-cli` → hibátlan fordítás

```
use rkv_core::Store;
use rkv_core::command::Command;

/// Egy kulcs (vagy prefix) ellenőrzése a protokoll nyelvtana szerint.
fn parse_key(s: &str) -> Result<String, String> {
```

```
    if s.is_empty() || s.chars().any(|c| c.is_whitespace() || c.is_control()) {
        Err(format!("invalid key: {s:?}"))
    } else {
        Ok(s.to_string())
    }
}

/// A protokoll egy sorát `Command`-dá alakítja.
fn parse_command(line: &str) -> Result<Command, String> {
    let (verb, rest) = match line.split_once(' ') {
        Some((verb, rest)) => (verb, Some(rest)),
        None => (line, None),
    };
    match (verb, rest) {
        ("GET", Some(key)) => parse_key(key).map(Command::Get),
        ("DEL", Some(key)) => parse_key(key).map(Command::Delete),
        ("SET", Some(rest)) => match rest.split_once(' ') {
            Some((_, "")) => Err("SET requires a non-empty value".to_string()),
            Some((_, value)) if value.contains(['\n', '\r']) => {
                Err("SET value must not contain a line break".to_string())
            }
            Some((key, value)) => parse_key(key).map(|k| Command::Set(k, value.to_string())),
            None => Err("SET requires a key and a value".to_string()),
        },
        ("LIST", None) => Ok(Command::List(None)),
        ("LIST", Some(prefix)) => parse_key(prefix).map(|p| Command::List(Some(p))),
        ("GET" | "DEL", None) => Err(format!("{verb} requires a key")),
        ("SET", None) => Err("SET requires a key and a value".to_string()),
        ("", _) => Err("empty line".to_string()),
        (other, _) => Err(format!("unknown command: {other}")),
    }
}

/// Végrehajtja a parancsot a tárolón, és visszaadja a protokoll szerinti választ.
fn execute(store: &mut Store, command: Command) -> String {
    match command {
        Command::Get(key) => match store.get(&key) {
            Some(bytes) => match String::from_utf8(bytes.clone()) {
                Ok(text) => format!("VALUE {key} {text}"),
                Err(_) => format!("ERR value of {key} is not valid UTF-8"),
            },
            None => "NOTFOUND".to_string(),
        },
        Command::Set(key, value) => {
            store.set(key, value.into_bytes());
            "OK".to_string()
        }
        Command::Delete(key) => match store.delete(&key) {
            Some(_) => "OK".to_string(),
            None => "NOTFOUND".to_string(),
        },
        Command::List(prefix) => {
            let mut keys = store.keys();
            keys.retain(|k| match &prefix {
                Some(p) => k.starts_with(p.as_str()),
                None => true,
            });
            keys.sort();
            let mut out = String::new();
            for key in keys {
                if let Some(bytes) = store.get(&key) {
                    out.push_str(&format!("VALUE {key} {}\n", String::from_utf8_lossy(bytes)));
                }
```

```
            }
            out.push_str("OK");
            out
        }
    }
}

fn main() {
    // A parancssori argumentumokat egy protokollsorrá fűzzük össze:
    // `rkv-cli SET name "Alice Smith"` -> "SET name Alice Smith".
    let line = std::env::args().skip(1).collect::<Vec<_>>().join(" ");
    let mut store = Store::new();
    match parse_command(&line) {
        Ok(command) => println!("{}", execute(&mut store, command)),
        Err(e) => {
            eprintln!("ERR {e}");
            std::process::exit(1);
        }
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn round_trip_on_supported_domain() {
        let commands = [
            Command::Get("k".into()),
            Command::Set("k".into(), "hello world".into()),
            Command::Delete("k".into()),
            Command::List(None),
            Command::List(Some("app".into())),
        ];
        for cmd in commands {
            assert_eq!(parse_command(&cmd.to_line()), Ok(cmd));
        }
    }

    #[test]
    fn nothing_is_dropped_silently() {
        assert!(parse_command("GET k extra").is_err());
        assert!(parse_command("SET k").is_err());
        assert!(parse_command("SET k ").is_err());
        assert!(parse_command("SET k a\nb").is_err());
        assert!(parse_command("LIST a b").is_err());
        assert_eq!(
            parse_command("SET k hello world"),
            Ok(Command::Set("k".into(), "hello world".into()))
        );
    }
}
```

A `match (verb, rest)` egy tuple-ra illeszt: a minta egyszerre vizsgálja a parancsszót és azt, hogy van-e argumentum. Az `("GET" | "DEL", None)` ág a vagy-mintát, a `Some((_, ""))` ág a beágyazott literálmintát (üres érték), a `Some((_, value)) if value.contains(...)` ág pedig a guardot mutatja. (Az üres értékre is írhatnánk `if value.is_empty()` guardot, de a clippy `redundant_guard` lintje jogosan javasolja helyette a literálmintát.) A kimerítőséget a fordító ellenőrzi: az utolsó `(other, _)` ág minden más esetet lefed. A `parse_key(key).map(Command::Get)` a `Result` sikeres ágát alakítja át; a `?` operátort csak a Result, a ? operátor és a panic helye alfejezet vezeti be.

2. Ellenőrizzük a kódot, és futtassuk néhány példával. Minden `cargo run` új folyamat, üres tárolóval (lásd Az rkv mintaprojekt indítása: Cargo-workspace és első CLI), ezért a `GET` itt `NOTFOUND`-ot ad.

```
cargo test -p rkv-cli
cargo run -p rkv-cli -- SET name "Alice Smith"
cargo run -p rkv-cli -- GET name
cargo run -p rkv-cli -- GET name extra
cargo run -p rkv-cli -- FOO
```

Várt kimenet (a `cargo` saját sorai nélkül; a két utolsó sor a standard hibakimenetre kerül, a kilépési kód 1):

```
test tests::nothing_is_dropped_silently ... ok
test tests::round_trip_on_supported_domain ... ok
OK
NOTFOUND
ERR invalid key: "name extra"
ERR unknown command: FOO
```

Ellenőrzési pont. Az `rkv-core` változatlan (Az enum mint összeg-típus és az Option a null helyett); az `rkv-cli` a protokoll szerinti parancsokat fogadja (`GET`, `SET`, `DEL`, `LIST`), a `cargo test -p rkv-cli` 2 sikeres tesztet futtat, a `cargo build --workspace` hibátlan.

Továbbvitt eredmény. A `Command` enum (`rkv-core/src/command.rs`), a protokoll nyelvtana, valamint a `parse_command` és `execute` függvény (`rkv-cli/src/main.rs`) alapjait később a Result, a ? operátor és a panic helye alfejezet finomítja saját hibatípusra, a trait objektumos és haladó típusrendszeri minták pedig újra felhasználják a kimerítő `match` elvét.

4.3.9 Tipikus hibák és átállási csapdák

1. **Túl korai `_` ág.** A `_` kényelmes, de ha új variánst veszünk fel, a fordító nem figyelmeztet, mert a `_` mindent elnyel. Inkább soroljuk fel a tényleges variánsokat, és csak akkor használjunk `_`-t, ha valóban minden más értékre ugyanaz a viselkedés.
2. **`if let` túlhasználata.** Az `if let` elveszíti a kimerítőségi ellenőrzést. Ha egy enum több variánsát kell kezelni, a `match` a megfelelő eszköz; az `if let` csak akkor indokolt, ha tényleg csak egy ág érdekel, a többi pedig figyelmen kívül hagyandó.
3. **Guard logika erőltetése.** A guard `if` feltétel, nem minta. Ha a feltétel a minta részeként kifejezhető (pl. tartományminta), akkor azt használjuk: `1..=5` jobb, mint `n if n >= 1 && n <= 5`. A guard akkor hasznos, ha a feltétel nem minta (pl. `x % 2 == 0`).
4. **Binding-módok félreértése.** Amikor `match &opt`-ot írunk, a minta `Some(s)` a `s`-t `&String`-ként köti, nem `String`-ként. Ha a mozgatás szándékos, akkor `match opt` kell; ha a kölcsönzés szándékos referencia nélkül, akkor a `ref` vagy `ref mut` használható. Ez a hibás kód gyakran `cannot move out of borrowed content` hibaüzenethez vezet, amit a Borrowing fejezet tárgyal részletesen.

4.3.10 Feladatok

4.3.10.1 (a) Fogalmi kérdés: lefordul-e és miért?

Az alábbi kód szándékosan hibás:

```rust
enum Status {
    Pending,
    Running,
    Done,
}

fn describe(s: Status) -> &'static str {
    match s {
        Status::Pending => "várakozik",
        Status::Running => "fut",
    }
}
```

Mi a hiba, és hogyan javítható? Mondja ki a kimerítőségi szabályt.

4.3.10.2 (b) C++-kód átírása idiomatikus Rustra

Írd át a következő C++17 `std::variant`-os kódot Rustra `match`-csel, enumnal és destrukturálással:

```cpp
#include <iostream>
#include <string>
#include <type_traits>
#include <variant>

using Result = std::variant<int, std::string>;

int main() {
    Result r = 42;
    std::visit([](auto const& v) {
        if constexpr (std::is_same_v<std::decay_t<decltype(v)>, int>) {
            std::cout << "szám: " << v << '\n';
        } else {
            std::cout << "szöveg: " << v << '\n';
        }
    }, r);
}
```

4.3.10.3 (c) Implementációs feladat az rkv-ban

Bővítsd a `parse_command` függvényt egy `PING` paranccsal, amely visszaad egy `Command::Ping` variánst. Ehhez módosítsd a `Command` enumot is. A `PING` parancsnak nincs argumentuma. Teszteld a `cargo run -p rkv-cli -- "PING"` parancsot.

4.3.10.4 (d) Hibajavítás, refaktorálás

Az alábbi kód **helyes és minden esetet kezel** (a `Some`-ágban az `if`/`else`, a külső `else` a `None`-t), tehát ez nem hibajavítás, hanem *olvashatósági refaktorálás*. Írd át egyetlen `match`-csel, guarddal; opcionálisan készíts egy külön, `let else`-t használó változatot is, és vesd össze a kettőt:

```rust
fn process(opt: Option<i32>) {
    if let Some(x) = opt {
        if x > 0 {
            println!("pozitív: {x}");
        } else {
            println!("nem pozitív: {x}");
```

```
        }
    } else {
        println!("nincs érték");
    }
}
```

A cél: ugyanaz a kimenet `Some(5)`, `Some(-1)` és `None` bemenetre.

4.3.11 Megoldások

4.3.11.1 (a)

A hiba: a `match` nem kimerítő, mert a `Status::Done` variáns nincs lefedve. A javítás:

```
match s {
    Status::Pending => "várakozik",
    Status::Running => "fut",
    Status::Done => "kész",
}
```

A kimerítőségi szabály: minden enum-variánsnak rendelkeznie kell legalább egy illeszkedő mintával, különben a fordító `E0004` hibát ad.

4.3.11.2 (b)

Rust megfelelő:

```
enum Result {
    Number(i32),
    Text(String),
}

fn main() {
    let r = Result::Number(42);
    match r {
        Result::Number(n) => println!("szám: {n}"),
        Result::Text(s) => println!("szöveg: {s}"),
    }
}
```

A C++ `if constexpr` helyett a `match` enum variánsokra illeszkedik, és a destrukturálás automatikus.

4.3.11.3 (c)

A bővítés a meglévő `rkv-core/src/command.rs` `Command` enumját módosítja; új típust nem hozunk létre. A feladat egy kísérleti bővítés: a könyv kanonikus `Command` enumja a továbbiakban is négy variánsú marad (lásd lent).

Kódblokk — fájl: `rkv-core/src/command.rs` · művelet: módosításrészlet: új variáns és új `to_line` ág · ellenőrzés: `cargo build -p rkv-core`

```rust
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum Command {
    Get(String),
    Set(String, String),
    Delete(String),
    List(Option<String>),
    Ping,
}

// a to_line match-jébe:
            Command::Ping => "PING".to_string(),
```

Ha most `cargo build --workspace`-t futtatunk, a fordító az `rkv-cli execute` függvényében E0004-es hibát ad (`non-exhaustive patterns: Command::Ping not covered`): a kimerítőség-ellenőrzés pontosan megmutatja, hol kell az új variánst kezelni. A teljes, módosított `parse_command` (csak a két új ág tér el a laborbeli változattól) és az `execute` új ága:

Kódblokk — fájl: `rkv-cli/src/main.rs` · művelet: a `parse_command` cseréje és egy új ág az `execute` `match`-ében · ellenőrzés: `cargo run -p rkv-cli -- PING` → `PONG`

```rust
fn parse_command(line: &str) -> Result<Command, String> {
    let (verb, rest) = match line.split_once(' ') {
        Some((verb, rest)) => (verb, Some(rest)),
        None => (line, None),
    };
    match (verb, rest) {
        ("GET", Some(key)) => parse_key(key).map(Command::Get),
        ("DEL", Some(key)) => parse_key(key).map(Command::Delete),
        ("SET", Some(rest)) => match rest.split_once(' ') {
            Some((_, "")) => Err("SET requires a non-empty value".to_string()),
            Some((_, value)) if value.contains(['\n', '\r']) => {
                Err("SET value must not contain a line break".to_string())
            }
            Some((key, value)) => parse_key(key).map(|k| Command::Set(k, value.to_string())),
            None => Err("SET requires a key and a value".to_string()),
        },
        ("LIST", None) => Ok(Command::List(None)),
        ("LIST", Some(prefix)) => parse_key(prefix).map(|p| Command::List(Some(p))),
        ("PING", None) => Ok(Command::Ping),
        ("PING", Some(_)) => Err("PING takes no arguments".to_string()),
        ("GET" | "DEL", None) => Err(format!("{verb} requires a key")),
        ("SET", None) => Err("SET requires a key and a value".to_string()),
        ("", _) => Err("empty line".to_string()),
        (other, _) => Err(format!("unknown command: {other}")),
    }
}

// az execute match-jébe:
        Command::Ping => "PONG".to_string(),
```

Futtassuk:

```
cargo run -p rkv-cli -- PING
```

Várt kimenet:

```
PONG
```

A round-trip tesztbe a `Command::Ping` is felvehető. **Fontos:** a könyv kanonikus protokollja a négy parancsot (`GET`, `SET`, `DEL`, `LIST`) tartalmazza, a további laborok erre épülnek. Ha a `Ping`-et a saját projektedben megtartod, minden későbbi `match`-be (például a szerver `execute`

függvényébe) fel kell venned az ágát; a fordító mindegyik helyet megmutatja. Ha a könyv kódját változtatás nélkül szeretnéd követni, a feladat után állítsd vissza a két fájlt.

4.3.11.4 (d)

Refaktorált változat:

```
fn process(opt: Option<i32>) {
    match opt {
        Some(x) if x > 0 => println!("pozitív: {x}"),
        Some(x) => println!("nem pozitív: {x}"),
        None => println!("nincs érték"),
    }
}
```

A `match` mindhárom esetet egy helyen, kimerítően mutatja. Az opcionális `let else` változat a „korai kilépés” stílus, és akkor a legértékesebb, ha az `x` értékére a függvény további részében is szükség van:

```
fn process_let_else(opt: Option<i32>) {
    let Some(x) = opt else {
        println!("nincs érték");
        return;
    };
    if x > 0 {
        println!("pozitív: {x}");
    } else {
        println!("nem pozitív: {x}");
    }
}
```

A három változat (az eredeti, a `match` és a `let else`) ellenőrzötten azonos kimenetet ad: `pozitív: 5`, `nem pozitív: -1`, `nincs érték`. Itt a `match` elegendő és tömörebb; a `let else` hosszabb függvényben csökkenti a behúzást.

4.3.12 Rövid összefoglaló

Ebben az alfejezetben tanulmányoztuk a Rust `match` kifejezést, amely a C++ `switch`-nél és `std::visit`-nél kifejezőbb, kimerítő és típusos. A minták lehetnek literálok, tartományok, destrukturáló tuple/struct/enum minták, `@` bindingek és guardok. Az `if let`, `while let` és `let else` a gyakori egyágú esetek kényelmes formái. A kimerítőség-ellenőrzés a refaktorálás során garantálja, hogy új enum-variáns felvételekor minden `match` frissüljön. **Továbbvitt eredmények:** a protokoll nyelvtana, a `parse_command` és az `execute` függvény (`rkv-cli/src/main.rs`) a `rkv-core/src/command.rs` `Command` enumjára építve, valamint a kimerítő `match` elve, amelyet a Result és a ? operátor, a trait objektumok és a haladó típusrendszeri minták használnak tovább.

4.3.13 Források és ellenőrzési hiányok

A leírt nyelvi szabályok a Rust Reference (https://doc.rust-lang.org/reference/) és a Rust Book (https://doc.rust-lang.org/book/) stabil részein alapulnak. A hibaüzenet-szövegek a rögzített toolchainre (Rust 1.99.1, 2024-es edition) vonatkoznak, más verzióban eltérhetnek. A `Command` enum és a `parse_command` függvény a laborban bemutatott formát követi.

5 Modulok, crate-ek és láthatóság

5.1 Modulok és láthatóság: header nélküli világ

5.1.1 Motiváció: miért nincs header?

Egy tipikus C++-projektben egy új típushoz legalább két fájl tartozik: a `store.hpp` deklarációval, include guarddal vagy `#pragma once`-szal, és a `store.cpp` definícióval. A headerben ügyelni kell arra, hogy csak teljes típusokra mutató okos pointerek és referenciák szerepeljenek, ha el akarjuk kerülni a fölösleges include-okat; a ciklikus függőséghez előre deklaráció kell; a `namespace rkv_core` nyitó-záró blokkok pedig vizuálisan növelik a zajt. A helyes include-sorrend, a rejtett ODR-hiba és a sablonok headerbe kényszerítése mind a fejlesztő terhe.

A Rust ezt a problémát nyelvi szinten eltávolítja: **nincs header, nincs include guard, és nincs külön deklarációs és definíciós fázis.** A modulrendszer a crate-en belül hierarchikusan rendezi a kódot, a láthatóságot pedig a `pub` kulcsszó és a modulhatárok szabályozzák. A névterek szerepét maguk a modulok töltik be, de a láthatóság alapértelmezése éppen fordított, mint C++-ban: ami `pub`, az publikus, minden más privát.

5.1.2 Fogalmak és szabályok

Definíció (modul). A modul a Rust szervezési egysége: típusokat, függvényeket, konstansokat és más modulokat tartalmazó névtér. A modulok a `mod` kulcsszóval jönnek létre, és fizikailag fájlba vagy könyvtárba szervezhetők.

Definíció (crate gyökér). Minden crate-nek egyetlen gyökere van: könyvtári crate-nél a `src/lib.rs`, bináris crate-nél a `src/main.rs`. Ez a gyökér jelenti a modulhierarchia tetejét; a `crate::` útvonal innen indul.

Szabály (privát alapértelmezés). Ha egy elem – modul, típus, mező, függvény – elé nem írunk `pub`-ot, akkor az az adott modulban és annak leszármazott moduljaiban látható, más modulok számára nem.

Szabály (gyermekmodul hozzáférése). A gyermekmodul látja a szülő privát elemeit is; a privát határ nem a fájlok, hanem a modulok között húzódik. A szülő modul viszont nem látja a gyermekmodul privát elemeit.

Szabály (fordítási egység = crate). A Rust-fordító a crate-et fordítja le; a modulokra bontás nem hoz létre külön fordítási egységet. A `use` nem szúr be kódot, mint a `#include`, hanem csak rövidített elérési utat vezet be. Ennek következménye, hogy a modulok sorrendje és a fájlok közötti függőség nem befolyásolja a deklarációk elérhetőségét – a teljes crate „egyszerre” látható a fordító számára.

A láthatósági szintek:

- `pub` – mindenki számára látható, a crate-en kívül is;

- `pub(crate)` – csak az aktuális **crate** belsejében látható. Egy workspace több crate-ből áll (itt `rkv-core`, `rkv-cli`, `rkv-server`), és ezek egymás számára *külső* crate-ek: az `rkv-core` `pub(crate)` eleme az `rkv-cli`-ből és az `rkv-server`-ből nem látható;
- `pub(super)` – a szülőmodulban (és annak leszármazottaiban) látható;
- `pub(in path)` – csak a megadott, a környező modulok közül való útvonalon belül látható (például `pub(in crate::outer)`);
- nincs módosító – privát.

Három szintű modulhierarchián mutatjuk meg a szabályokat, konkrét engedett és tiltott útvonalakkal. A példa önálló (`rustc` vagy `cargo new`); a tiltott sorok kommentben állnak, a fordító üzenetével:

```
mod outer {
    pub struct Visible;           // pub típus, de a hatókörét a privát `outer` korlátozza
    fn secret() {}                // privát: `outer` és a gyermekei látják
    pub(super) fn for_parent() {} // pub(super): a szülő (itt a crate gyökér) is látja

    mod hidden {
        pub struct Exposed;       // pub típus egy PRIVÁT modulban
    }
    pub fn make_exposed() -> impl std::fmt::Debug {
        let _ = hidden::Exposed;  // `outer`-ből elérhető
        "Exposed"
    }

    pub mod middle {
        pub(in crate::outer) fn inner_only() {} // csak `outer`-en belül látható
        pub mod inner {
            pub fn call_up() {
                super::super::secret();     // OK: a gyermek látja a szülő privát elemét
                super::super::for_parent(); // OK
                super::inner_only();        // OK: pub(in crate::outer), és outer-en belül
                 ↪ vagyunk
            }
        }
    }
}

fn main() {
    outer::for_parent();             // OK: pub(super), a crate gyökér a szülő
    let _v = outer::Visible;         // OK: a gyökér látja a privát `outer`-t (testvérmodul),
     ↪ Visible pub
    outer::middle::inner::call_up(); // OK: minden köztes modul pub
    println!("{:?}", outer::make_exposed());
    // outer::secret();              // E0603: function `secret` is private
    // outer::middle::inner_only();  // E0603: function `inner_only` is private (main nincs
     ↪ outer-ben)
    // let _e = outer::hidden::Exposed; // E0603: module `hidden` is private, bár Exposed pub
}
```

Ellenőrzött viselkedés: a kód lefordul és a `"Exposed"` sort írja ki; a három kikommentezett sor egyenként `E0603` hibát ad. A tanulság: a `pub` elem hozzáférhetősége a *teljes útvonal* láthatóságán múlik (a privát `hidden` modulban lévő `pub` típus kívülről nem érhető el), a `pub(super)` a szülőtől látható, a `pub(in path)` pedig csak a megnevezett modulban.

A `pub(crate)` és a workspace határa külön fogalom:

Kódblokk — fájl: `core/src/lib.rs` és `cli/src/main.rs` (két crate-es próba-workspace, a `cli` path-függősége a `core`) · művelet: új fájlok · ellenőrzés: `cargo build` → `E0624` az `cli`-ben

```rust
// core/src/lib.rs
pub struct Store;
impl Store {
    pub fn new() -> Self { Store }
    pub(crate) fn apply(&mut self) {} // csak a `core` crate-en belül
}
pub fn internal(s: &mut Store) { s.apply(); } // azonos crate-ből hívható

// cli/src/main.rs
fn main() {
    let mut s = core::Store::new();
    s.apply(); // error[E0624]: method `apply` is private
}
```

A `use` és a `pub use` közötti különbség kulcsfontosságú: a `use store::Store;` csak a jelenlegi modulban hozza létre a rövid nevet, míg a `pub use store::Store;` a nevet a modul API-jának részeként kifelé is elérhetővé teszi. Ez utóbbit **újraexportálásnak** (re-exportnak) nevezzük, és az idiomatikus Rust gyakran így takarja el a belső modulszerkezetet.

5.1.3 C++ és Rust összevetés

A C++-oldalon a névtér és a header gondoskodik a szervezésről:

```cpp
// store.hpp
#pragma once
#include <string>
#include <unordered_map>

namespace rkv_core {
class Store {
public:
    void set(const std::string& key, const std::string& value);
private:
    std::unordered_map<std::string, std::string> data_;
};
}
```

A Rust-megfelelő egy modulfájl, amelyben a láthatóság minden elemnél explicit:

```rust
// src/store.rs
use std::collections::HashMap;

pub struct Store {
    data: HashMap<String, String>,
}

impl Store {
    pub fn set(&mut self, key: String, value: String) {
        self.data.insert(key, value);
    }
}
```

A legfontosabb különbség: a Rust-kódban a `Store` típus `pub`, ezért a crate-en kívül is elérhető; a `data` mező viszont privát, ezért kívülről nem lehet közvetlenül elérni. A C++ `private:` szakasz helyett itt a mező előtti `pub` hiánya jelzi ugyanezt. A Rust névtere a modulokhoz kötődik; ezek lehetnek külön fájlban, de inline is (`mod name { ... }` szabályos kapcsos blokkos modul), nem pusztán zárójel-párral jönnek létre, hanem a modulhatárokkal, és nem kell külön header és forrásfájl közötti deklaráció-egyezést karbantartani.

A `pub(crate)` tipikus belső API-khoz való: olyan segédfüggvényhez, amelyet a crate más moduljai használnak, de a könyvtár felhasználója nem hívhat:

```
// src/store.rs
impl Store {
    // Az rkv-core más moduljai (például a command) használják; az rkv-cli és az
    // rkv-server külön crate, ezért onnan nem látható. Ahhoz `pub` kellene.
    pub(crate) fn apply(&mut self, cmd: &crate::command::Command) -> Result<(),
    ↪  crate::error::Error> {
        // ...
    }
}
```

A `use` szintaxisa hasonlít a C++ `using`-ra, de nem hoz létre másolatot vagy láthatósági összefonódást:

```
using rkv_core::Store;   // C++: név bevezetése

use crate::store::Store; // Rust: rövid név bevezetése
```

5.1.4 Labor: rkv-core modulokra bontása

Munkakönyvtár: `~/rust-labor/rkv/`

A kiinduló állapot a Mintaillesztés: match, if let, let else és a minták ellenőrzési pontja: az `rkv-core/src/lib.rs` a `Store` típust és a teszteket tartalmazza, a `Command` már külön fájlban, a `rkv-core/src/command.rs`-ben él (Az enum mint összeg-típus és az Option a null helyett). A laborban a `Store`-t is saját modulba tesszük, és egy `error` modult nyitunk. Az API nem változik: a kulcs `String`, a tárolt érték `Vec<u8>` (Structok és metódusok: impl blokkok, asszociált függvények), a `Command` értéke `String` (Az enum mint összeg-típus és az Option a null helyett). Ez tiszta szerkezeti refaktorálás; az `rkv-cli`-hez nem kell nyúlni.

5.1.4.1 1. lépés: a modulfájlok létrehozása

Az `rkv-core/src/lib.rs` eddigi tartalmát (a `Store`-t, a `use std::collections::HashMap;` sort és a tesztmodult) változtatás nélkül áttesszük egy új `rkv-core/src/store.rs` fájlba. A `lib.rs` ezután csak a moduldeklarációkat és a re-exportokat tartalmazza:

Kódblokk — fájl: `rkv-core/src/lib.rs` · művelet: teljes csere · ellenőrzés: a lépés végén `cargo build -p rkv-core`

```
// rkv-core/src/lib.rs
pub mod command;
pub mod error;
pub mod store;

pub use command::Command;
pub use error::Error;
pub use store::Store;
```

A `pub mod` sorok azt mondják a fordítónak, hogy a modulok a crate gyökere alatt léteznek, és kifelé is láthatók. A `pub use` sorok a gyakran használt típusokat a crate gyökerébe emelik; így a külső kód `rkv_core::Store` néven éri el őket, nem csak `rkv_core::store::Store` néven. Mivel a `command` modul `pub`, az `rkv-cli` eddigi `use rkv_core::command::Command;` sora is változatlanul működik.

Kódblokk — fájl: `rkv-core/src/store.rs` · művelet: új fájl (a korábbi `lib.rs` tartalma, a tesztekkel együtt) · ellenőrzés: `cargo test -p rkv-core`

```
// rkv-core/src/store.rs
use std::collections::HashMap;

/// A kulcs-érték tároló. A kulcs `String`, az érték `Vec<u8>`.
///
/// A `data` mező privát: a külső kód csak a publikus API-n keresztül férhet hozzá.
#[derive(Debug, Default)]
pub struct Store {
    data: HashMap<String, Vec<u8>>,
}

impl Store {
    /// Üres tároló létrehozása.
    pub fn new() -> Self {
        Self {
            data: HashMap::new(),
        }
    }

    /// Kulcs-érték pár beillesztése. Felülírja a korábbi értéket.
    pub fn set(&mut self, key: String, value: Vec<u8>) {
        self.data.insert(key, value);
    }

    /// Egy kulcs értékének lekérdezése. `None`, ha nincs ilyen kulcs.
    pub fn get(&self, key: &str) -> Option<&Vec<u8>> {
        self.data.get(key)
    }

    /// Kulcs törlése. Visszaadja a törölt értéket, ha létezett.
    pub fn delete(&mut self, key: &str) -> Option<Vec<u8>> {
        self.data.remove(key)
    }

    /// Az összes kulcs rendezetlen listája.
    pub fn keys(&self) -> Vec<String> {
        self.data.keys().cloned().collect()
    }

    /// Az összes kulcs törlése.
    pub fn clear(&mut self) {
        self.data.clear();
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn set_and_get() {
        let mut store = Store::new();
        store.set("kulcs".to_string(), b"ertek".to_vec());
        assert_eq!(store.get("kulcs"), Some(&b"ertek".to_vec()));
    }

    #[test]
    fn delete_returns_old_value() {
        let mut store = Store::new();
        store.set("a".to_string(), vec![1, 2, 3]);
```

```
        let old = store.delete("a");
        assert_eq!(old, Some(vec![1, 2, 3]));
        assert!(store.get("a").is_none());
    }
}
```

A `rkv-core/src/command.rs` (Az enum mint összeg-típus és az Option a null helyett) változatlan marad. Az új `rkv-core/src/error.rs` egyelőre egy minimalista hibaséma, amelyet a Saját hibatípusok: enum-hibák, thiserror, anyhow alfejezet bővít tovább:

Kódblokk — fájl: `rkv-core/src/error.rs` · művelet: új fájl · ellenőrzés: `cargo build -p rkv-core`

```
// rkv-core/src/error.rs
use std::fmt;

/// Az rkv-core hibáinak közös típusa.
#[derive(Debug)]
pub enum Error {
    KeyNotFound,
    Io(std::io::Error),
}

impl fmt::Display for Error {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            Error::KeyNotFound => write!(f, "kulcs nem található"),
            Error::Io(e) => write!(f, "I/O hiba: {e}"),
        }
    }
}

impl std::error::Error for Error {}
```

A láthatóság itt három szintű: a `Store`, a `Command` és az `Error` `pub` (a workspace többi crate-je is látja), a `data` mező privát (csak a `store` modul látja). A harmadik szint a `pub(crate)`: az így jelölt elem a teljes `rkv-core` crate-en belül látható, de az `rkv-cli` számára nem. Ilyen belső segédfüggvényre a 3. feladat ad példát.

5.1.4.2 2. lépés: fordítás és ellenőrzés

```
cd ~/rust-labor/rkv
cargo build --workspace
cargo test -p rkv-core
```

Várt kimenet (rövidítve; a tesztek sorrendje változhat):

```
running 4 tests
test command::tests::list_prefix_none_has_no_trailing_space ... ok
test command::tests::set_formats_key_and_value ... ok
test store::tests::delete_returns_old_value ... ok
test store::tests::set_and_get ... ok

test result: ok. 4 passed; 0 failed; ...
```

A tesztek neve a modulútvonallal együtt jelenik meg (`store::tests::...`): ez is mutatja, hogy a `Store` most a `store` modulban él. A modulok láthatóságát egy külső crate-ből érdemes ellenőrizni; ehhez egy `examples/` fájl is elég, mert az a crate-et kívülről, a publikus API-n keresztül használja:

Kódblokk — fájl: `rkv-core/examples/use_core.rs` · művelet: új fájl · ellenőrzés: `cargo run -p rkv-core --example use_core`

```rust
// rkv-core/examples/use_core.rs
use rkv_core::Store;

fn main() {
    let store = Store::new();
    // A data mező itt nem érhető el: privát.
    // println!("{:?}", store.data); // fordítási hiba: E0616
    println!("Store létrehozva: {store:?}");
}
```

Futtatás:

```
cargo run -p rkv-core --example use_core
```

Várt kimenet:

```
Store létrehozva: Store { data: {} }
```

Ha a `store.data` sort visszatesszük, a fordító a privát mező elérését jelzi:

```
error[E0616]: field `data` of struct `Store` is private
```

5.1.4.3 3. ábra: a modulszerkezet

```
rkv-core (crate gyökér: lib.rs)
     mod store    (store.rs)
     mod command  (command.rs)
     mod error    (error.rs)

A három modul a crate gyökerének testvére (nem egymás almodulja);
a lib.rs re-exportjai (pub use) teszik elérhetővé a típusokat kívülről.
```

A modulfa azt mutatja, hogy a külső crate-ek a `lib.rs` re-exportjain keresztül érik el a típusokat; a modulok közötti belső kapcsolatot a `crate::` kezdetű útvonalak adják (például `use crate::error::Error`).

Ellenőrzési pont. Az `rkv-core` három modulból áll (`command`, `error`, `store`), a gyökérből a `Command`, `Error` és `Store` re-exportálva. A `cargo build --workspace` hibátlan, a `cargo test --workspace` az `rkv-core` 4 és az `rkv-cli` 2 tesztjét sikeresen futtatja; a `Store` API-ja ugyanaz, mint a Structok és metódusok: impl blokkok, asszociált függvények végén.

Továbbvitt eredmény. Ez a `store`, `command` és `error` modulokra épülő szerkezet az alapja a Saját hibatípusok: enum-hibák, thiserror, anyhow alfejezet `Error` bővítésének; a Trait-ek és generikusok alfejezet itt vezeti be a `Storage` traitet a `Store` fölé; a Unit-, integrációs és dokumentációs tesztek alfejezet pedig ezeket a modulokat teszteli külön fájlokban.

5.1.5 Tipikus hibák és átállási csapdák

1. **Minden `pub` C++-os beidegződésből.** A C++ `public:` osztályszakaszhoz szokott fejlesztő gyakran minden mezőt és metódust `pub`-ra állít, hogy „majd később elrejti”. Ezzel azonban a Rust éppen a fő előnyét veszíti el: a privát adat invariánsát a fordító kényszeríti ki. A helyes stratégia: kezdetben minden privát, és csak a valóban szükséges elem kapjon `pub`-ot. A `pub(crate)` a belső használatra való.

2. **A `use` és a `#include` összemosása.** A C++ `#include` a fájlt szó szerint beszúrja, és a sorrend befolyásolhatja a fordítást. A Rust `use` pusztán rövidítés: ha a `use`-t elhagyjuk, a teljes útvonal (`crate::store::Store`) ugyanúgy működik. Nincs szükség include guardra, és a modulok sorrendje sem számít – a fordító a crate minden modulját egyszerre látja.

3. **A moduldeklaráció és a modulfájl összekeverése.** Két különböző hiba tartozik ide, és a fordító mást mond rájuk. (a) Ha létrehozzuk a `src/store.rs` fájlt, de a `lib.rs`-be nem írjuk be a `mod store;` sort, a fordító a fájlt egyszerűen *nem tölti be*: a puszta létezése semmilyen hibát nem okoz. A hiba csak ott jelentkezik, ahol a modulra hivatkozunk: a `use crate::store::Store;` sor a következő diagnosztikát adja:

```
error[E0432]: unresolved import `crate::store`
```

(b) Fordítva, ha a `lib.rs`-ben ott a `pub mod store;`, de a fájl hiányzik (vagy elírtuk a nevét), a következő hibát kapjuk:

```
error[E0583]: file not found for module `store`
```

A fordító azt javasolja, hogy hozzuk létre a `src/store.rs` vagy a `src/store/mod.rs` fájlt. A C++-szal ellentétben tehát a fájlrendszer önmagában nem része a fordításnak: modul csak `mod` deklarációval lesz egy fájlból.

4. **Re-export nélküli, mélyen tagolt API.** Ha a külső kód minden típust a teljes útvonalon (`rkv_core::store::Store`) kénytelen elérni, a belső átszervezés minden felhasználót megtör. Az idiomatikus megoldás a `pub use` a crate gyökerében: a felhasználó a stabil `rkv_core::Store` nevet látja, a belső modulok pedig szabadon mozgathatók.

5.1.6 Feladatok

5.1.6.1 1. Feladat – Lefordul-e és miért?

Adott a következő kód:

```rust
mod service {
    pub struct Config {
        pub(crate) timeout_ms: u64,
    }

    pub fn create_config() -> Config {
        Config { timeout_ms: 1000 }
    }
}

fn main() {
    let cfg = service::create_config();
    println!("{}", cfg.timeout_ms);
}
```

Fordul-e a kód? Magyarázd el, hogy a `pub(crate)` láthatóság mit enged meg ebben a bináris crate-ben, és mi lenne a helyzet, ha a `pub(crate)` helyett `pub(super)` állna?

5.1.6.2 2. Feladat – C++-kód átírása idiomatikus Rustra

Alakítsd át a következő C++ fejlécet egyetlen Rust modullá (`src/message.rs`), ügyelve a láthatóságra és a privát alapértelmezésre:

```
#pragma once
#include <string>

namespace mail {
class Message {
public:
    explicit Message(std::string body);
    const std::string& body() const;
private:
    std::string body_;
};
}
```

A modul legyen `pub`, a típus `pub`, a mező privát. A `body()` visszatérési típusa Rustban `&str` legyen.

5.1.6.3 3. Feladat – rkv-bővítés: parancs-feldolgozó a command modulban

Az `rkv-core/src/command.rs` fájlban hozz létre egy `pub fn parse(line: &str) -> Result<Command, crate::error::Error>` függvényt, amely a szöveges protokoll egy sorát a Mintaillesztés: match, if let, let else és a minták alfejezetben rögzített nyelvtan szerint értelmezi; a hibát egy új `Error::Parse(String)` variáns jelezze. Ezzel az értelmező a CLI-ből a könyvtárba kerül, így a későbbi szerver is ugyanazt használhatja. A kulcsellenőrzést végző segédfüggvény legyen `pub(crate)`: a crate-en belül bárhol hívható, de az `rkv-core` publikus API-jának nem része.

5.1.6.4 4. Feladat – Hibajavítás: láthatóság és re-export

A következő kis crate nem fordul. Javítsd a kódot úgy, hogy a `main.rs` hozzáférjen a `Store` típushoz a gyökérből (`demo::Store` néven), a `data` mező viszont maradjon privát.

```
// src/lib.rs
mod store {
    pub struct Store {
        data: Vec<u8>,
    }

    impl Store {
        pub fn new() -> Self {
            Store { data: Vec::new() }
        }
    }
}

// src/main.rs
fn main() {
    let s = demo::Store::new();
    println!("{:?}", s.data);
}
```

5.1.7 Megoldások

5.1.7.1 1. Feladat megoldása

A kód lefordul. A `pub(crate)` láthatóság a teljes aktuális crate-en belül érvényes, tehát a `main` függvény (amely ugyanabban a bináris crate-ben van) eléri a `timeout_ms` mezőt. Ha `pub(super)` állna, a mező csak a `service` modul szülőjében, azaz a crate gyökerében lenne látható – mivel a `main` a crate gyökerében van, az is működne. Ha viszont a `service` modul egy gyermekmodulban lenne, a `pub(super)` már nem engedné a hozzáférést a gyermekmodulból.

5.1.7.2 2. Feladat megoldása

```rust
// src/message.rs
pub struct Message {
    body: String,
}

impl Message {
    pub fn new(body: String) -> Self {
        Self { body }
    }

    pub fn body(&self) -> &str {
        &self.body
    }
}
```

A moduldeklaráció a `lib.rs`-ben: `pub mod message;`. A `body` mező privát, a `body()` metódus publikusan ad olvasási hozzáférést.

5.1.7.3 3. Feladat megoldása

Az `error.rs` bővítése az új variánssal és a `Display` ágával:

Kódblokk — fájl: `rkv-core/src/error.rs` · művelet: módosításrészlet: új variáns az `Error`-ban, új ág a `Display`-ben

```rust
// rkv-core/src/error.rs, az Error enumban:
    Parse(String),

// a Display match-jében:
            Error::Parse(msg) => write!(f, "értelmezési hiba: {msg}"),
```

A `command.rs` új függvényei. A nyelvtan ugyanaz, mint a Mintaillesztés: match, if let, let else és a minták `parse_command`-jáé; a különbség csak a hibatípus. A `?` operátort itt még nem használjuk (azt a Result, a ? operátor és a panic helye vezeti be), helyette a `Result::map` és a `match` alakítja a hibát:

Kódblokk — fájl: `rkv-core/src/command.rs` · művelet: beillesztés az `impl Command` blokk után · ellenőrzés: `cargo test -p rkv-core`

```rust
// rkv-core/src/command.rs
use crate::error::Error;

/// Kulcs (vagy prefix) ellenőrzése a protokoll nyelvtana szerint.
/// `pub(crate)`: a crate-en belül újrahasznosítható, kívülről nem látszik.
pub(crate) fn parse_key(s: &str) -> Result<String, Error> {
```

```
    if s.is_empty() || s.chars().any(|c| c.is_whitespace() || c.is_control()) {
        Err(Error::Parse(format!("érvénytelen kulcs: {s:?}")))
    } else {
        Ok(s.to_string())
    }
}

/// A szöveges protokoll egy sorát `Command`-dá alakítja (nyelvtan: r009).
pub fn parse(line: &str) -> Result<Command, Error> {
    let (verb, rest) = match line.split_once(' ') {
        Some((verb, rest)) => (verb, Some(rest)),
        None => (line, None),
    };
    match (verb, rest) {
        ("GET", Some(key)) => parse_key(key).map(Command::Get),
        ("DEL", Some(key)) => parse_key(key).map(Command::Delete),
        ("SET", Some(rest)) => match rest.split_once(' ') {
            Some((_, value)) if value.is_empty() || value.contains(['\n', '\r']) => {
                Err(Error::Parse("a SET értéke üres vagy sortörést tartalmaz".into()))
            }
            Some((key, value)) => parse_key(key).map(|k| Command::Set(k, value.to_string())),
            None => Err(Error::Parse("SET érték nélkül".into())),
        },
        ("LIST", None) => Ok(Command::List(None)),
        ("LIST", Some(prefix)) => parse_key(prefix).map(|p| Command::List(Some(p))),
        ("GET" | "DEL" | "SET", None) => Err(Error::Parse(format!("{verb} argumentum nélkül"))),
        (other, _) => Err(Error::Parse(format!("ismeretlen parancs: {other:?}"))),
    }
}
```

Ez a függvény `pub`, tehát kívülről is hívható: az `rkv-cli` a saját `parse_command`-ja helyett `rkv_core::command::parse`-t hívhat. A `parse_key` `pub(crate)`, ezért az `rkv-cli` nem látja; ha megpróbálná, E0603 („function `parse_key` is private") hibát kapna. A `GET k extra` itt is hiba (a `"k extra"` kulcs szóközt tartalmaz), a `SET k hello world` értéke pedig a teljes `"hello world"`.

5.1.7.4 4. Feladat megoldása

A hiba oka: a `store` modul `mod` (privát) deklarációval jön létre, és a típus nincs re-exportálva a gyökérbe. Javítás:

```
// src/lib.rs
pub mod store {
    pub struct Store {
        data: Vec<u8>,
    }

    impl Store {
        pub fn new() -> Self {
            Store { data: Vec::new() }
        }
    }
}

pub use store::Store;
```

A `pub use` után a `demo::Store` név a crate gyökerében elérhető. A `data` mező továbbra is privát, így a `main.rs`-ben a `println!("{:?}", s.data);` sor továbbra is hibát ad; ezt a sort el kell távolítani, vagy egy publikus gettert kell bevezetni.

5.1.8 Rövid összefoglaló

A Rust modulrendszer header és include guard nélkül szervezi a kódot. A crate gyökere a `lib.rs` vagy `main.rs`; a `mod` deklaráció hozza létre a modulnévtérben a `store`, `command` és `error` egységeket. A láthatóság alapértelmezése privát; a `pub` kifelé, a `pub(crate)` a crate-en belül tesz elérhetővé, a `pub(super)` a szülőmodulra korlátoz. A `use` rövidített útvonal, a `pub use` újraexportálja a nevet a modul API-jába; ezért a külső kód `rkv_core::Store`, `rkv_core::Command` és `rkv_core::Error` néven éri el a típusokat. Az `rkv-core` moduljai: `store.rs` (`Store`, `pub(crate) fn apply`), `command.rs` (`Command`), `error.rs` (`Error`). **Továbbvitt eredmény:** ez a modulszerkezet az alapja a Saját hibatípusok, a Trait-ek és generikusok és a Tesztelés alfejezeteknek.

5.1.9 Források és ellenőrzési hiányok

A modulrendszer és a láthatósági szabályok a Rust Reference és a *The Rust Programming Language* stabil, jól dokumentált részei. A példákban szereplő `cargo build` kimenetek, a hibakódok és az edition-különbségek a rögzített toolchainre (Rust 1.99.1, 2024-es edition) vonatkoznak; a hibakódok magyarázata a `rustc --explain` paranccsal olvasható.

5.2 Függőségek, feature-ök, verziózás és workspace-ek

5.2.1 Motiváció

Egy C++-projektben a külső függőség többnyire három dologból áll: include-útvonalak, linker-kapcsolók és platformfüggő binárisok. A vcpkg, Conan vagy a CMake FetchContent megoldja a letöltést, de a build-rendszer továbbra is a te felelősséged marad: melyik fejléc, melyik könyvtárfájl, melyik ABI-verzió kerüljön a parancssorba. Az is gyakori, hogy ugyanannak a könyvtárnak két verziója él a gépen, és a hibás `find_package` csendben a rosszat választja.

A Rust ezt a problémát a Cargo manifestben rendezi: a `Cargo.toml` deklarálja a függőséget, a verziómegkötést, a fordítási lehetőségeket (feature-ök) és a Cargo gondoskodik a letöltésről, a tranzitív függőségek feloldásáról és a linkelésről. A cél nem az, hogy mindent „varázslatosan” kezeljen, hanem hogy a függőségi fa **reprodukálható** és **ellenőrizhető** legyen.

5.2.2 Fogalmak és szabályok

Definíció. A Cargo-manifest (`Cargo.toml`) a crate leíró állománya; a `[dependencies]` szekció a normál, a `[dev-dependencies]` a tesztekhez és példákhoz, a `[build-dependencies]` a build scripthez szükséges függőségeket sorolja fel.

Definíció. A szemantikus verziózás (semver) `MAJOR.MINOR.PATCH` alakú. A Cargo alapértelmezett verziómegkötése a caret (`^`), ami a bal szélső nem nulla komponenst tartja meg: `^1.2.3` jelentése `>=1.2.3, <2.0.0`, `^0.2.3` jelentése `>=0.2.3, <0.3.0`. A Cargo-ban a jel nélküli `1.2.3` *nem* pontos rögzítés: ugyanaz, mint a `^1.2.3`, tehát `>=1.2.3, <2.0.0`. Pontosan egy kiadást az egyenlőségjeles `=1.2.3` enged meg. A `*` bármely verziót elfogad – ezt kerüld, mert nem reprodukálható buildhez vezet. A manifest követelménye és a `Cargo.lock` két különböző dolog: a manifest a megengedett *tartományt* adja meg, a lockfájl pedig azt a konkrét verziót rögzíti, amelyet a feloldás ebből a tartományból kiválasztott (például `^1.2.3` mellett `1.4.0`-t). A reprodukálható buildet a commitolt `Cargo.lock` adja, nem az `=` megkötés; az `=`-t csak indokolt esetben, például egy ismert hibás kiadás kizárására használjuk.

Definíció. A feature opcionális, additív fordítási lehetőség; a crate-ek `[features]` szekcióban hivatkozhatnak más feature-ökre vagy opcionális függőségekre. Egy opcionális függőség akkor válik elérhetővé, ha `optional = true` jelölést kap, és egy feature bekapcsolja. Egy crate `default` feature-je azoknak a feature-öknek a listája, amelyek akkor is bekapcsolnak, ha a függőség deklarálásakor semmit sem kérünk (például a `std`). A `default-features = false` ezt a listát nem veszi figyelembe, ilyenkor csak az explicit `features = [...]` felsoroltak kerülnek be — a *saját* függőségi deklarációnkra nézve. A Cargo azonban a gráfban a feature-ket **egyesíti** (feature unification): ha egy másik függő ugyanahhoz a crate-hez további feature-t, akár a `default`-ot vagy a `std`-et is kéri, az bekerül; a `no_std` kompatibilitás tehát egyetlen `default-features = false` beállításból nem következik. Az explicit feature tehát additív kérés (a `clap` `derive` feature-je nem része a `clap` default készletének, ezért kérni kell); a `default-features = false` pedig kivonás (az alapértelmezett készlet nélkül indulunk).

Szabály. A Cargo.lock a feloldott függőségek **pontos** verzióit tárolja. A Cargo FAQ jelenlegi ajánlása szerint minden projekt lockfájlját érdemes verziókezelésbe tenni (alkalmazásnál és könyvtárnál egyaránt), mert így a saját fejlesztői és CI-build reprodukálható. Egy közzétett könyvtár lockfájlját a fogyasztó nem használja: ő a saját gráfját oldja fel. Az rkv workspace egyetlen, a gyökérben lévő `Cargo.lock`-ot használ.

Szabály. A függőségi higiénia része a fa és a biztonsági állapot ellenőrzése: `cargo tree` a megszűnt/duplikált verziókat, `cargo audit` az ismert sebezhetőségeket, `cargo deny` a licenceket és a nemkívánatos függőségeket mutatja. Ez utóbbi kettő külön telepítendő eszköz.

5.2.3 C++ és Rust összevetés

C++-ban egy CLI-argumentum feldolgozása gyakran kézi `argc`/`argv`-járás vagy `getopt`; a függőséget a build-rendszer külön kezeli:

```cpp
// C++: CMake FetchContent + getopt (rövidített vázlat)
#include <getopt.h>
#include <string>
#include <vector>

int main(int argc, char** argv) {
    std::string key, value;
    int opt;
    while ((opt = getopt(argc, argv, "k:v:")) != -1) {
        if (opt == 'k') key = optarg;
        else if (opt == 'v') value = optarg;
    }
    // ellenőrzés, hibaüzenet, súgó kézzel...
}
```

A megfelelő Rust-kód a clap derive API-jával:

```rust
use clap::{Parser, Subcommand};

#[derive(Parser)]
#[command(name = "rkv-cli", version, about = "rkv parancssori kliens")]
struct Cli {
    #[command(subcommand)]
    command: Command,
}

#[derive(Subcommand)]
enum Command {
    /// Kulcs lekérdezése
    Get { key: String },
```

```
    /// Kulcs-érték beállítása
    Set { key: String, value: String },
    /// Kulcs törlése
    Del { key: String },
    /// Kulcsok listázása opcionális prefixszel
    List { prefix: Option<String> },
}

fn main() {
    let cli = Cli::parse();
    match cli.command {
        Command::Get { key } => println!("GET {}", key),
        Command::Set { key, value } => println!("SET {} {}", key, value),
        Command::Del { key } => println!("DEL {}", key),
        Command::List { prefix } => match prefix {
            Some(p) => println!("LIST {}", p),
            None => println!("LIST"),
        },
    }
}
```

A különbség: a Rust-verzió a `Cargo.toml` egyetlen bejegyzéséből húzza be a clap-ot, a fordítás során a Cargo feloldja a tranzitív függőségeket, a `#[derive]` makró pedig a típusból generálja a parancssori elemzőt, a hibaüzeneteket és a súgót. Nincs külön include-útvonal, nincs linker-flag, nincs platformfüggő `.lib`/`.so` kézi kezelés.

5.2.4 Labor: clap a rkv-cliben

Munkakönyvtár: `~/rust-labor/rkv/` (a workspace gyökere). A kiinduló állapot a Modulok és láthatóság: header nélküli világ ellenőrzési pontja. A labor célja, hogy az `rkv-cli` kézzel írt argumentumkezelését a clap váltsa fel, a parancsok végrehajtása pedig továbbra is az `rkv-core` `Store`-ján történjen.

5.2.4.1 1. lépés – A jelenlegi állapot ellenőrzése

A workspace gyökér `Cargo.toml`-ja az rkv mintaprojekt indítása: Cargo-workspace és első CLI óta változatlan, két taggal:

```
[workspace]
resolver = "3"
members = [
    "rkv-core",
    "rkv-cli",
]
```

Az `rkv-server` tag még nem létezik: a Az rkv hálózati szerver: TCP, protokoll, kecses leállás alfejezet hozza létre `cargo new rkv-server` paranccsal, és csak akkor kerül a `members` listába. Egy nem létező tag felvétele már a workspace betöltését is meghiúsítaná („failed to load manifest for workspace member”).

Az `rkv-cli/src/main.rs` a Mintaillesztés: match, if let, let else és a minták alfejezetben írt változat: a `parse_key`, `parse_command` és `execute` függvény, valamint a `main`, amely az argumentumokat egy protokollsorrá fűzi össze. A `cargo test -p rkv-cli` 2 sikeres tesztet futtat.

Ha eltértél a könyvtől. Ha egy opcionális feladat (például a Mintaillesztés: match, if let, let else és a minták `PING` bővítése vagy a Modulok és láthatóság: header nélküli világ `parse`

függvénye) a fájljaidat megváltoztatta, két út van: vagy állítsd vissza őket a legutóbbi ellenőrzési pont állapotára, vagy tartsd meg a bővítést, és a lenti kódban kezeld az új elemet is (a fordító minden kimaradó `match`-ágat jelez). A könyv kódja mindig az ellenőrzési pontok állapotából indul.

5.2.4.2 2. lépés – A clap függőség felvétele

Kódblokk — fájl: `rkv-cli/Cargo.toml` · művelet: teljes csere · ellenőrzés: `cargo build -p rkv-cli`

```
[package]
name = "rkv-cli"
version = "0.1.0"
edition = "2024"
rust-version = "1.89"

[dependencies]
rkv-core = { path = "../rkv-core" }
clap = { version = "4", features = ["derive"] }
```

A `version = "4"` caret-megkötés, tehát bármely `4.x.y` elfogadott, de az 5-ös főverzió nem. A `derive` feature engedélyezi a `#[derive(Parser)]` makrót; e nélkül a clap csak a builder API-t kínálná.

5.2.4.3 3. lépés – A CLI típus implementálása

A clap saját `Command`-szerű enumot vár a részparancsokhoz. Hogy ne ütközzön az `rkv_core::Command`-dal, a clap-os enum neve `CliCommand`. A clap a parancssor *szerkezetét* ellenőrzi (melyik részparancs, hány argumentum); a kulcsok és értékek *tartalmát* továbbra is a protokoll nyelvtana dönti el, ezért a felépített parancsot a meglévő `parse_command`-dal visszaellenőrizzük. Így a nyelvtan egyetlen helyen él.

Kódblokk — fájl: `rkv-cli/src/main.rs` · művelet: beillesztés és csere: a `use` sorok és a `main` cseréje, a `parse_key`, `parse_command`, `execute` függvény és a tesztmodul változatlan · ellenőrzés: `cargo test -p rkv-cli` → 2 sikeres teszt

```
use clap::{Parser, Subcommand};
use rkv_core::{Command, Store};

/// rkv parancssori kliens
#[derive(Parser)]
#[command(name = "rkv-cli", version, about)]
struct Cli {
    #[command(subcommand)]
    command: CliCommand,
}

#[derive(Subcommand)]
enum CliCommand {
    /// Kulcs lekérdezése
    Get { key: String },
    /// Kulcs-érték beállítása
    Set { key: String, value: String },
    /// Kulcs törlése
    Del { key: String },
    /// Kulcs-érték párok listázása opcionális prefixszel
    List { prefix: Option<String> },
```

```
}

// ... parse_key, parse_command és execute változatlanul (r009) ...

fn main() {
    let cli = Cli::parse();
    let command = match cli.command {
        CliCommand::Get { key } => Command::Get(key),
        CliCommand::Set { key, value } => Command::Set(key, value),
        CliCommand::Del { key } => Command::Delete(key),
        CliCommand::List { prefix } => Command::List(prefix),
    };
    // A protokoll nyelvtana szerinti ellenőrzés: ugyanaz a parser dönt,
    // mint a protokollsorok esetén.
    let command = match parse_command(&command.to_line()) {
        Ok(command) => command,
        Err(e) => {
            eprintln!("ERR {e}");
            std::process::exit(1);
        }
    };
    let mut store = Store::new();
    println!("{}", execute(&mut store, command));
}
```

A `#[command(version, about)]` automatikusan hozzáadja a `--version` és `--help` kapcsolókat; az `about` szövegét a `Cli` feletti `///` komment adja, a részparancsok súgóját pedig a variánsok feletti `///` kommentek. A mezők (`key`, `value`, `prefix`) attribútum nélkül **pozicionális** argumentumok: `get foo`, nem `get --key foo`. Név szerinti kapcsolóhoz a mezőre `#[arg(long)]` kellene.

5.2.4.4 4. lépés – Futtatás és a függőségi fa

```
cd ~/rust-labor/rkv
cargo run -p rkv-cli -- set name "Alice Smith"
cargo run -p rkv-cli -- get foo
cargo run -p rkv-cli -- get "a b"
```

Várt kimenet (az utolsó sor a standard hibakimenetre kerül, kilépési kód 1):

```
OK
NOTFOUND
ERR invalid key: "a b"
```

A `get foo` azért `NOTFOUND`, mert minden futtatás új, üres tárolóval indul (Az rkv mintaprojekt indítása: Cargo-workspace és első CLI); a perzisztenciát az I/O és szerializáció: fájlok, pufferelés, serde adja hozzá. Próbáljuk ki a súgót:

```
cargo run -p rkv-cli -- --help
```

Várt kimenet (clap 4, Rust 1.99.1):

```
rkv parancssori kliens

Usage: rkv-cli <COMMAND>

Commands:
  get   Kulcs lekérdezése
  set   Kulcs-érték beállítása
  del   Kulcs törlése
  list  Kulcs-érték párok listázása opcionális prefixszel
  help  Print this message or the help of the given subcommand(s)
```

```
Options:
  -h, --help     Print help
  -V, --version  Print version
```

Nézzük meg a feloldott függőségi fát:

```
cargo tree -p rkv-cli
```

Közvetlen függőségek (valós kimenet, `--depth 1`; az `anyhow` az 5.2 mérföldkövének manifestjében még nincs, a Saját hibatípusok: enum-hibák, thiserror, anyhow vezeti be, ezért a kimenet a labor *végállapotát* mutatja; az 5.2 pontos mérföldkövénél csak a `clap` és az `rkv-core` jelenik meg):

```
cargo tree -p rkv-cli --depth 1
```

```
rkv-cli v0.1.0 (~/rust-labor/rkv/rkv-cli)
   anyhow v1.0.104
   clap v4.6.7
   rkv-core v0.1.0 (~/rust-labor/rkv/rkv-core)
```

A kérések feature-szinten (`-e features`): melyik feature-t ki kéri.

```
cargo tree -p rkv-cli -e features --depth 1
```

```
rkv-cli v0.1.0 (~/rust-labor/rkv/rkv-cli)
   anyhow feature "default"
   clap feature "default"
   clap feature "derive"
   rkv-core feature "default"
```

Az `rkv-cli` tehát a `clap` alapértelmezett készletét *és* az explicit `derive` feature-t kéri; ugyanezt fordítva is megnézhetjük: `cargo tree -p rkv-cli -e features -i clap` megmutatja, mely feature-t mi húzza be (a `derive` kérőjeként az `rkv-cli` jelenik meg).

A `clap_derive` proc-macro crate fordítási időben bővíti ki a `#[derive(Parser)]` annotációt.

5.2.4.5 5. lépés – Biztonsági és licenc-ellenőrzés

A `cargo audit` és a `cargo deny` nem része a Rust disztribúciónak. A könyv a `cargo-deny` **0.20.2**-es kiadásával dolgozott; a verziót rögzítve telepítjük:

```
cargo install cargo-deny --version 0.20.2 --locked
cargo deny --version   # cargo-deny 0.20.2
```

Minden workspace-tagra (az `rkv-core`, `rkv-cli`, és a később létrehozott `rkv-server`, `rkv-ffi`, `rkv-cxx`, `rkv-derive` manifestjére is) érvényes: ahol egy későbbi lépés teljes `Cargo.toml`-cserét ír elő, és a blokk a `license` és `publish = false` sorokat nem tartalmazza, a `[package]` táblában ezeket meg kell tartani vagy vissza kell adni, különben a `cargo deny check licenses` elbukik.

A `cargo deny` licencet kér minden crate-től, ezért a saját workspace-tagjainknak is kell `license` mező, és `publish = false` jelzi, hogy nem kerülnek a crates.io-ra (különben a path-függőségeket "wildcard"-nak minősítené). A tagok `Cargo.toml`-jának `[package]` táblájába vegyük fel:

Kódblokk — fájl: `rkv-core/Cargo.toml`, `rkv-cli/Cargo.toml` (és a többi tag) · művelet: két új sor a `[package]` táblában · ellenőrzés: `cargo deny check licenses`

```
license = "MIT OR Apache-2.0"
publish = false
```

A minimális, magyarázott konfiguráció a workspace gyökerében:

Kódblokk — fájl: `deny.toml` · művelet: új fájl (workspace gyökér) · ellenőrzés: `cargo deny check licenses bans` → `bans ok, licenses ok`

```
# Minimális cargo-deny konfiguráció az rkv workspace-hez (cargo-deny 0.20.2)
[graph]
all-features = true            # az összes feature-t vizsgálja

[licenses]
# Csak a függőségi fában ténylegesen előforduló, engedett licencek (cargo deny list).
allow = ["MIT", "Apache-2.0", "Apache-2.0 WITH LLVM-exception", "Unicode-3.0", "Unlicense",
↪  "Zlib"]

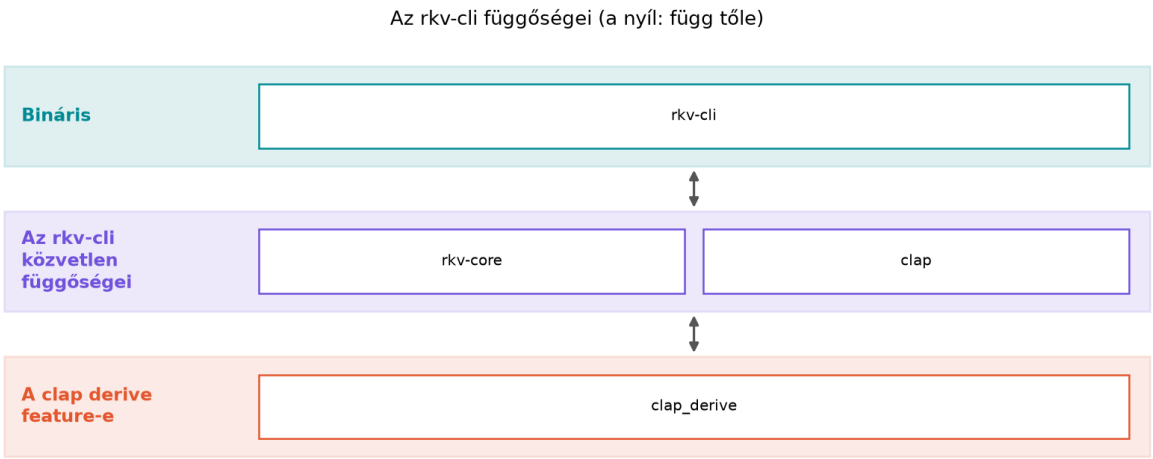
[bans]
multiple-versions = "warn"     # ugyanaz a crate több verzióban: figyelmeztetés
wildcards = "deny"             # "*" verziómegkötés tiltott
allow-wildcard-paths = true    # a workspace-tagok path-függősége nem számít wildcardnak
deny = [
    { crate = "openssl" },     # példa tiltott crate; az rkv nem használja
]
```

Futtatás a workspace gyökeréből:

```
cargo deny check licenses bans advisories
```

Ellenőrzött eredmény a labor workspace-én: `bans ok, licenses ok, advisories ok` (az advisory adatbázist a `cargo deny` hálózatról tölti le). A `bans` ellenőrzés `warn` szinten jelzi a duplikált verziókat (például a `syn` 2 és 3), de nem bukik meg tőlük. A `licenses` szekció `allow` listája a `cargo deny list` kimenetéből készült: pontosan azokat a licenceket tartalmazza, amelyek a fában előfordulnak. A `cargo audit` ugyanazt az advisory adatbázist használja önállóan: `cargo install cargo-audit --locked`, majd `cargo audit`.

A rétegábra a működő rendszer függőségi viszonyait mutatja:



5.1. ábra: Az rkv-cli függőségei (a nyíl: függ tőle) - Az rkv-cli függ az rkv-core-tól és a clap-től; a clap a derive feature miatt a clap_derive-ot is behúzza. Az rkv-core nem függ a clap-től.

A nyilak a függőség irányát mutatják. Az `rkv-cli` `Cargo.toml`-ja: `rkv-core` (path) és `clap` (`features = ["derive"]`); a `clap` a `derive` feature-rel a `clap_derive` crate-et is behúzza. Az `rkv-core` nem függ a `clap`-től.

Az ábra szavakkal: a `rkv-cli` a workspace-en belül a `rkv-core`-ra (path-függőség) és a külső clap-ra épül. A `clap` a `clap_derive` proc-macro crate-tel fordul, ami a `derive` feature bekapcsolásának következménye.

Ellenőrzési pont. A workspace két taggal (`rkv-core`, `rkv-cli`) hibátlanul fordul; az `rkv-cli` clap-alapú részparancsokat fogad (`get`, `set`, `del`, `list`), és a végrehajtást az `rkv-core` `Store`-ja végzi. A `cargo test --workspace` 6 tesztje (4 az `rkv-core`-ban, 2 az `rkv-cli`-ben) sikeres.

Továbbvitt eredmény. Ez a `Cli` típus és a `CliCommand` enum a `rkv-cli/src/main.rs` fájlban a clap-alapú parancssori felület alapja. A Hibakezelés: saját hibatípusok alfejezetben a `Command` feldolgozás hibatípusra köt majd rá, a Záróprojekt: az rkv teljes rendszerként alfejezetben a CLI lokális `Store`/`FileStore` klienseként marad; a hálózati szerverhez az `nc` és a C++-kliens csatlakozik, saját Rust TCP-kliens nem készül.

5.2.5 Tipikus hibák és átállási csapdák

1. **Wildcard verziómegkötés.** A `clap = "*"` azt mondja a Cargo-nak, hogy bármely verzió megfelel. Egy féléves projektben már nem reprodukálható a build, mert az új kiadás API-t törhet. Mindig adj meg legalább főverziós caret-megkötést, és a `Cargo.lock`-ot commitold.
2. **A `default-features = false` félreértése `no_std` környezetben.** Sok crate `default` feature-je a `std`-et is bekapcsolja, ami `no_std` célra (például `thumbv7em-none-eabihf`) fordítási hibát okoz. A `serde` jó példa: a `std`-et a `default` készlet tartalmazza, a `derive` viszont explicit feature. Ellenőrzött viselkedés: `serde = { version = "1", default-features = false, features = ["derive"] }` lefordul a `no_std` célra, a `serde = { version = "1", features = ["derive"] }` (az alapértelmezett készlettel) viszont `E0463: can't find crate for std` hibával áll meg. Fontos, hogy a `default-features = false` önmagában nem "kapcsolja ki a `std`-et általában", csak azokat a feature-öket hagyja ki, amelyeket az adott crate a `default` listájában felsorol; a `clap` `derive` feature-je például soha nem volt a default része, ezért mindig explicit kell (`features = ["derive"]`).
3. **A `cargo update` félreértése.** Az `cargo update` az összes függőséget frissíti a megengedett tartományon belül; ha csak egy crate-et akarsz, a `cargo update -p <crate>` a pontos eszköz. A `Cargo.lock` törlése nem „megjavítja", hanem megszünteti a reprodukálhatóságot.

5.2.6 Feladatok

5.2.6.1 (a) Fogalmi kérdés

Adott a `Cargo.toml` részlet: `foo = "0.3.8"`. Mely verziókat engedélyezi a Cargo? (Válaszd ki a helyeset, és indokold.)

1. `>=0.3.8, <1.0.0`
2. `>=0.3.8, <0.4.0`
3. `=0.3.8`

5.2.6.2 (b) C++-kód átírása

Írd át a következő C++-részletet idiomatikus Rustra clap derive API-val. A program egyetlen `--count N` opciót vár, alapértelmezett értéke 1, és annyiszor írja ki a `hello` szót.

```
#include <cstdlib>
#include <iostream>
#include <string>

int main(int argc, char** argv) {
    int count = 1;
    for (int i = 1; i < argc; ++i) {
        std::string arg = argv[i];
        if (arg == "--count" && i + 1 < argc) {
            count = std::atoi(argv[++i]);
        }
    }
    for (int i = 0; i < count; ++i) std::cout << "hello\n";
}
```

5.2.6.3 (c) Implementációs feladat az rkv-ban

Bővítsd a `rkv-cli/src/main.rs` `Cli` típusát egy `--verbose` kapcsolóval, amely `-v` rövid alakkal is elérhető, és ha be van kapcsolva, a program a `main` elején írja ki: `debug: verbose mód bekapcsolva`. A kapcsolót úgy add hozzá, hogy az minden alkalmazott parancsnál elérhető legyen (globális opció).

5.2.6.4 (d) Hibajavítás

Az alábbi `Cargo.toml` részlet nem fordul. Mi a hiba, és hogyan javítanád?

```
[dependencies]
clap = { version = "4", features = "derive" }
```

5.2.7 Megoldások

5.2.7.1 (a)

A helyes válasz: **2.** A `0.3.8` caret-megkötésben a bal szélső nem nulla komponens a `0` utáni `3`, tehát `>=0.3.8, <0.4.0`.

5.2.7.2 (b)

```
use clap::Parser;

#[derive(Parser)]
struct Cli {
    /// Ismétlések száma
    #[arg(short, long, default_value_t = 1)]
    count: u32,
}

fn main() {
    let cli = Cli::parse();
    for _ in 0..cli.count {
```

```
        println!("hello");
    }
}
```

A `default_value_t` használata a `u32` típus miatt biztonságos; a `#[arg(short, long)]` a `-c` és `--count` kapcsolókat generálja. A `for _ in 0..cli.count` a Rust tartományalapú iterációja.

5.2.7.3 (c)

A `Cli` típus kiegészítve:

```
use clap::{Parser, Subcommand};

#[derive(Parser)]
#[command(name = "rkv-cli", version, about = "rkv parancssori kliens")]
struct Cli {
    /// Verbose kiírás
    #[arg(short, long, global = true)]
    verbose: bool,

    #[command(subcommand)]
    command: Command,
}

#[derive(Subcommand)]
enum Command {
    Get { key: String },
    Set { key: String, value: String },
    Del { key: String },
    List { prefix: Option<String> },
}

fn main() {
    let cli = Cli::parse();
    if cli.verbose {
        println!("debug: verbose mód bekapcsolva");
    }
    match cli.command {
        Command::Get { key } => println!("GET {}", key),
        Command::Set { key, value } => println!("SET {} {}", key, value),
        Command::Del { key } => println!("DEL {}", key),
        Command::List { prefix } => match prefix {
            Some(p) => println!("LIST {}", p),
            None => println!("LIST"),
        },
    }
}
```

A `global = true` azt jelenti, hogy a kapcsolót nemcsak a főparancs után, hanem az alkalmazott parancs után is megadhatod (`rkv-cli get --verbose key`).

5.2.7.4 (d)

A `features` mezőnek **lista** típusúnak kell lennie, nem stringnek. A hibás sor:

```
features = "derive"
```

Helyesen:

```
features = ["derive"]
```

5.2.8 Rövid összefoglaló

A Cargo manifestben a `[dependencies]` szekció deklarálja a külső crate-eket, a caret (`^`) az alapértelmezett semver-megkötés. A feature-ök opcionális, additív fordítási lehetőségek; a `default-features = false` letiltja az alapértelmezett készletet. A `Cargo.lock` commitolandó a saját build reprodukálhatósága miatt (a fogyasztók feloldását egy könyvtár lockfájlja nem befolyásolja). A `cargo tree`, `cargo audit` és `cargo deny` a függőségi fa, a sebezhetőségek és a licencek ellenőrzését szolgálják. A laborban a `rkv-cli` crate-hez vettünk fel clap-ot (`version = "4"`, `features = ["derive"]`), és létrehoztuk a `Cli` típust és a `Command` enumot a `rkv-cli/src/main.rs` fájlban. Ez a **Továbbvitt eredmény** a Hibakezelés: saját hibatípusok és a Záróprojekt: az rkv teljes rendszerként alfejezetekben használatos.

5.2.9 Források és ellenőrzési hiányok

- A Cargo dokumentációja: https://doc.rust-lang.org/cargo/ – a verziómegkötések, feature-ök és Cargo.lock pontos szabályai.
- A Rust referencia: https://doc.rust-lang.org/reference/ – a nyelvi szintaxis ellenőrzése.
- A clap derive API pontos formátuma és a `#[arg]` attribútumok a docs.rs-en ellenőrizendők.
- A `cargo audit` és a `cargo deny` parancskészlete verziófüggő; a bemutatott hívások a telepítés utáni `--help` kimenetében is megtalálhatók.

III. RÉSZ – Ownership, borrowing, lifetime: a Rust szíve

6 Ownership és mozgatás

6.1 Ownership: minden értéknek egy tulajdonosa van

6.1.1 Motiváció

Egy C++-fejlesztő számára a mozgatási szemantika ismerős: `std::move` jelzi, hogy egy objektum erőforrását elvehetjük, a forrás pedig „érvényes, de nem meghatározott” állapotba kerül. A baj az, hogy a C++ fordító ezt nem ellenőrzi. A `std::move` valójában csak egy cast, a használat utáni hibát semmi sem akadályozza meg:

```cpp
#include <vector>
#include <string>
#include <iostream>

int main() {
    std::vector<int> v1 {1, 2, 3};
    std::vector<int> v2 = std::move(v1);
    // v1 most „valid but unspecified" - lehet üres, lehet nem.
    std::cout << v1.size() << '\n'; // lefordul, de a kimenet nem garantált
    v1.push_back(4);                // szintén lefordul, de logikai hiba lehet
}
```

A Rust válasza radikális: a mozgatás itt **destruktív**. Ha egy értéket áthelyezünk egy másik változóba, a régi kötés a továbbiakban **nem használható** (a mozgatott érték nem olvasható), és ezt a borrow checker fordítási időben garantálja. Nincs „valid but unspecified” moved-from állapot, nincs dupla felszabadítás, nincs használat utáni bizonytalanság.

6.1.2 Fogalmak és szabályok

Definíció. A Rustban minden értéknek van egy **tulajdonosa (owner)**: az a változó, amely az értéket birtokolja. A tulajdonos felelős az érték memóriájának felszabadításáért, amikor kilép a hatóköréből. Az érték egy másik változóhoz történő áthelyezése a tulajdonjog **mozgatása (move)**.

Definíció. A **destruktív mozgatás** azt jelenti, hogy a mozgatás során a régi tulajdonos **érvénytelenné válik**: a fordító a mozgatott kötést *inicializálatlannak* (uninitialized) tekinti. Fizikailag a Rust a verem tartalmát (például a `String` `ptr`, `len`, `cap` mezőit) bitenként átmásolhatja, a kupacbeli erőforrás nem duplikálódik, és a régi kötéshez tartozó destruktor nem fut le (a fordító nyomon követi, melyik kötés birtokol még értéket). A **név nem szűnik meg**: a kötés ugyanabban a hatókörben marad, csak a mozgatott értéke nem érhető el; a fizikai megvalósítás optimalizálástól függ, a szabály a fordító modelljére vonatkozik.

A mozgatott kötés újrainicializálható, ha `mut`, és a mozgatás lehet részleges is:

```rust
let mut s = String::from("a");
let t = s;                  // s inicializálatlan
s = String::from("b");      // újrainicializálás: szabályos
println!("{s} {t}");        // b a

struct Pair { a: String, b: String }
let p = Pair { a: "x".into(), b: "y".into() };
let a = p.a;                // részleges mozgatás: csak p.a inicializálatlan
println!("{a} {}", p.b);    // x y: p.b még használható
// println!("{}", p.a);     // E0382: borrow of moved value
// let q = p;              // E0382: p részben mozgatott
```

A részleges mozgatás csak olyan struktúrán megengedett, amely nem implementálja a `Drop` traitet (különben a destruktor a teljes értéket várná).

Szabály. Az ownership három alapszabálya:

1. Minden értéknek pontosan egy tulajdonosa van.
2. Amikor a tulajdonos kilép a hatókörből, az érték automatikusan felszabadul (`drop`).
3. A tulajdonjog átadása (hozzárendelés, függvényhívás, visszatérés) mozgatást jelent; a régi tulajdonos a továbbiakban **nem férhet hozzá** az értékhez.

Ezek a szabályok a teljes Rust memóriamodell alapját adják. Nincs kivétel, nincs rejtett másolás, hacsak a típus explicit módon nem engedi meg (lásd `Copy` a Copy, Clone és Drop alfejezetben).

6.1.3 C++ és Rust összevetés

A C++-ban a mozgatás **nem destruktív**: a moved-from objektum továbbra is érvényes, de a pontos állapota implementációfüggő. A Rustban a mozgatás **destruktív**, és a fordító garantálja, hogy a régi tulajdonost nem használjuk.

Nézzünk egy konkrét párost.

C++ változat – lefordul, de veszélyes:

```cpp
#include <string>
#include <iostream>

void take(std::string s) {
    std::cout << "birtokba vettem: " << s << '\n';
}

int main() {
    std::string s1 = "hello";
    std::string s2 = std::move(s1);
    std::cout << "s2: " << s2 << '\n';
    std::cout << "s1: " << s1 << '\n'; // lefordul, de a tartalom nem garantált
    take(std::move(s2));
    // s2 itt is valid but unspecified, használható, de logikailag kérdéses
}
```

Rust megfelelő – a mozgatott változó használata fordítási hiba:

```rust
fn take(s: String) {
    println!("birtokba vettem: {s}");
}

fn main() {
    let s1 = String::from("hello");
    let s2 = s1;            // s1 mozgatva s2-be
```

```
    println!("s2: {s2}");
    // println!("s1: {s1}"); // ez a sor fordítási hibát adna
    take(s2);
    // s2 itt már nem használható; a take utáni használat is hiba lenne
}
```

A lényeg: a Rust megakadályozza a mozgatott változó bármilyen használatát, még olvasást is. Ez a `use of moved value` hiba.

6.1.4 Kidolgozott példa

Írjuk meg a hibás Rust kódot, és nézzük meg a fordító hibaüzenetét.

Ez a változat szándékosan nem fordul le:

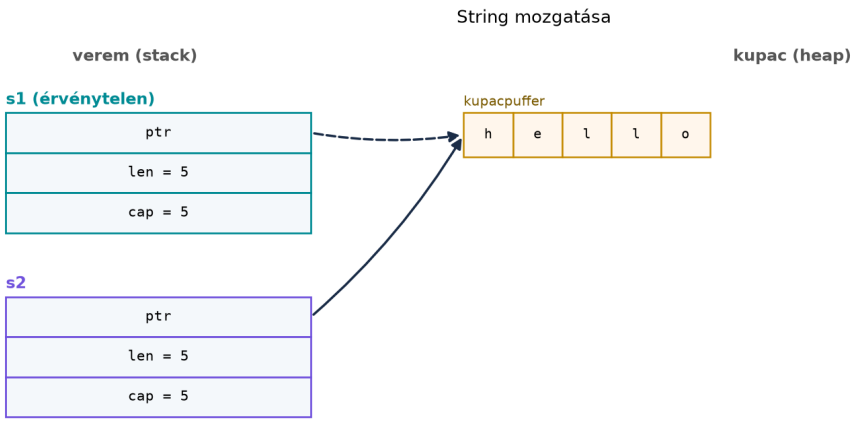
```
fn main() {
    let s1 = String::from("hello");
    let s2 = s1;
    println!("{s1}");
}
```

A `String` nem `Copy` típus, ezért a `let s2 = s1;` sor mozgatja a tulajdonjogot. A `println!("{s1}")` sorban a fordító hibát jelez:

```
error[E0382]: borrow of moved value: `s1`
 --> src/main.rs:4:15
  |
3 |     let s2 = s1;
  |              -- value moved here
4 |     println!("{s1}");
  |               ^^^^ value borrowed here after move
  |
  = note: move occurs because `s1` has type `String`, which does not implement the `Copy` trait
```

A hiba pontos szövege verziófüggő, de az `E0382` hibakód stabil. A javítás általában az, hogy nem használjuk a régi változót, vagy kifejezetten klónozzuk – de ezt tudatosan tegyük, mert a másolás költséges lehet.

A memóriaábra jól mutatja, mi történik a mozgatáskor. A `String` a veremben három mezőt tárol: egy mutatót a kupacbeli pufferre, a hosszt (`len`) és a kapacitást (`cap`). A mozgatás ezt a három értéket bitenként átmásolja az új változóba, a kupacpuffer nem mozdul. Ezután a régi változót a fordító érvénytelennek tekinti.



6.1. ábra: String mozgatása - Fogalmi modell: a mozgatás után s1 inicializálatlan, a kupacadat nem másolódik. A tényleges gépi kód optimalizálástól függ.

Az ábra *fogalmi modell*: a szaggatott nyíl azt jelzi, hogy az `s1` kötés inicializálatlan lett (a Rust nem feltétlenül nullázza ki a verem tartalmát, de a fordító tudja, hogy `s1` már nem birtokol semmit). A kupacpuffer ugyanaz marad, nem történik allokáció vagy felszabadítás.

6.1.5 Labor: a mozgatás megfigyelése önálló crate-ben

Ebben a laborban egy önálló kísérletet készítünk, amely bemutatja a mozgatást függvényhívásnál és visszatérésnél is.

Munkakönyvtár: `~/rust-labor/r012/`

1. Hozzuk létre a projektet:

```
mkdir -p ~/rust-labor/r012
cd ~/rust-labor/r012
cargo new ownership_demo
cd ownership_demo
```

2. Írjuk meg a `src/main.rs` tartalmát:

```
fn consume_and_return(s: String) -> String {
    println!("len = {}", s.len());
    s // visszaadjuk a tulajdonjogot
}

fn main() {
    let s1 = String::from("hello");
    let s2 = s1; // mozgatás
    // println!("{s1}"); // E0382 lenne, ha kikommentelnénk

    let s3 = consume_and_return(s2); // s2 mozgatva a függvénybe, majd vissza
    println!("s3 = {s3}");

    let s4 = s3.clone(); // explicit másolat; s3 továbbra is használható
    println!("s3 = {s3}, s4 = {s4}");
}
```

3. Futtassuk:

```
cargo run
```

Várt kimenet:

```
len = 5
s3 = hello
s3 = hello, s4 = hello
```

Próbáljuk ki a kommentezett `println!("{s1}")` sort is – a `cargo build` fordítási hibát fog adni az `E0382` hibakóddal.

Megfigyelések:

- A `let s2 = s1;` mozgatja a tulajdonjogot; a `s1` érvénytelenné válik.
- A `consume_and_return` függvény átveszi a `s2` tulajdonjogát, majd visszaadja azt `s3`-nak.
- A `clone()` explicit másolat, amelynek jelentését a típus határozza meg: a `String` esetén mély másolat (új kupacpuffer), ezért `s3` és `s4` függetlenek lesznek. Az `Rc` és az `Arc` `clone()`-ja ezzel szemben *megosztó* klón: nem másolja az adatot, csak a referenciaszámot növeli (lásd Box, Rc, Arc és Weak); ez nem kivétele a szabálynak, hanem a `Clone` szerződése: „explicit, típus által meghatározott másolatkészítés”.

6.1.6 Tipikus hibák és átállási csapdák

1. **A moved-from változó használatának megkísérlése.** A C++-ban megszoktuk, hogy a moved-from objektumot lehet használni (például újra feltölteni). Rustban ez fordítási hiba. A javítás: ne használjuk a régi változót, vagy explicit `clone()` ha másolatra van szükség.
2. **A mozgatás és a `clone()` összekeverése.** Gyakori hiba, hogy minden mozgatási hibát `clone()`-nal oldunk meg. Egy `String` vagy `Vec` klónja drága lehet (új kupacpuffer és elemenkénti másolás), egy `Rc`/`Arc` klónja olcsó (referenciaszám-növelés), a `clone()` költségét tehát a típus dönti el. Ez a „klónozás a borrow checker legyőzésére” anti-minta; a következő fejezetekben látni fogjuk, hogyan lehet a kölcsönzéssel többnyire elkerülni a felesleges másolatokat.
3. **A `Copy` típusok figyelmen kívül hagyása.** A skalár típusok (`i32`, `bool`, `char`) és a belőlük felépülő tuple-ök `Copy`-k, ezért a hozzárendelés nem mozgat, hanem másol. Ezt a Copy, Clone és Drop alfejezet tárgyalja részletesen. Az `String`, `Vec<T>` és a legtöbb kupacot használó típus nem `Copy`, így ezeknél a mozgatás az alapértelmezett.
4. **A destruktív mozgatás figyelmen kívül hagyása.** Ne várjuk el, hogy a mozgatott változó valamilyen üres, de használható állapotban marad. A Rustnál nincs „valid but unspecified” állapot: a kötés inicializálatlan, a mozgatott érték nem olvasható. A kötés újrainicializálható (`let mut s ...; s = ...;`), de addig nem használható.

6.1.7 Feladatok

6.1.7.1 (a) Lefordul-e és miért?

Az alábbi kód fordítási hibát ad-e? Ha igen, miért?

```rust
fn main() {
    let x = 42;
    let y = x;
    println!("{x} {y}");
}
```

6.1.7.2 (b) C++ kód átírása idiomatikus Rustra

Adott egy C++ függvény, amely egy `std::string`-et vesz át, kiírja, majd visszaadja. Írd át Rustra úgy, hogy a hívó a visszaadott értéket használja, de a mozgatás szabályait betartja.

```cpp
#include <string>
#include <iostream>

std::string process(std::string s) {
    std::cout << "received: " << s << '\n';
    s += "!";
    return s;
}

int main() {
    std::string value = "hello";
    std::string result = process(std::move(value));
    std::cout << "result: " << result << '\n';
}
```

6.1.7.3 (c) Implementációs feladat

Készíts egy függvényt `stats` néven, amely egy `Vec<i32>`-et vesz át, kiszámolja az elemek összegét és átlagát, majd kiírja ezeket. Határozd meg a tartományt: üres vektorra írja ki, hogy nincs adat (ne osszon nullával), és az összeg ne csorduljon túl. A függvény a tulajdonjogot megtartja, tehát nem adja vissza a vektort. A mainben hozz létre egy vektort, add át a függvénynek, majd próbáld használni a vektort a hívás után (ez fordítási hiba lesz). Írd meg a kommentezett sort, és magyarázd el, miért hibás.

6.1.7.4 (d) Hibajavítás

Az alábbi kód nem fordul le. Javítsd ki a legegyszerűbb módon, és indokold a választásodat.

```rust
fn main() {
    let name = String::from("Rust");
    let greeting = greet(name);
    println!("{name}!");
}

fn greet(s: String) -> String {
    format!("Hello, {s}")
}
```

6.1.8 Megoldások

6.1.8.1 (a) Megoldás

A kód lefordul, mert `i32` `Copy` típus. A `let y = x;` sor másolja az értéket, nem mozgatja, így `x` továbbra is használható marad.

6.1.8.2 (b) Megoldás

```rust
fn process(mut s: String) -> String {
    println!("received: {s}");
    s.push('!');
    s
}

fn main() {
    let value = String::from("hello");
    let result = process(value); // itt mozgatás történik
    println!("result: {result}");
}
```

A C++ `std::move`-hoz hasonlóan itt is átadjuk a tulajdonjogot, de a Rustban nincs szükség explicit `move`-ra; a hívás alapból mozgat. A hívó a visszaadott értéket használja, így a tulajdonjog visszakerül.

6.1.8.3 (c) Megoldás

```rust
fn stats(v: Vec<i32>) {
    if v.is_empty() {
        println!("nincs adat"); // az üres statisztika jelentése explicit
        return;
    }
    let sum: i64 = v.iter().map(|&x| i64::from(x)).sum(); // i64: az i32 elemek összege nem
    ↪  csordul túl
    let avg = sum as f64 / v.len() as f64;
    println!("sum = {sum}, avg = {avg}");
    // v itt felszabadul, mert kilép a hatókörből
}

fn main() {
    let numbers = vec![1, 2, 3, 4, 5];
    stats(numbers);
    // println!("{numbers:?}"); // Ez a sor E0382 hibát adna:
    // value moved here
    stats(Vec::new()); // üres bemenet: "nincs adat"
}
```

A `numbers` tulajdonjogát a `stats` függvény átveszi, és mivel nem adja vissza, a vektor a függvény végén felszabadul. A kommentezett sor használata fordítási hibát adna, mert `numbers` már nem birtokol semmit.

6.1.8.4 (d) Megoldás

```rust
fn main() {
    let name = String::from("Rust");
    let greeting = greet(name.clone()); // átmeneti megoldás: explicit másolat
    println!("{name}!");
    println!("{greeting}");
}

fn greet(s: String) -> String {
    format!("Hello, {s}")
}
```

Ez a legegyszerűbb javítás ebben a fejezetben, de a `clone` felesleges bemeneti másolatot készít. A jobb megoldás, amelyet a következő fejezet tanít (Referenciák és a kölcsönzési szabály), a kölcsönzés: a függvény `&str`-t vesz át, így a `name` tulajdonjoga és tartalma érintetlen marad, másolat nélkül:

```
fn main() {
    let name = String::from("Rust");
    let greeting = greet(&name); // kölcsönzés: name megmarad
    println!("{name}!");
    println!("{greeting}");
}

fn greet(s: &str) -> String {
    format!("Hello, {s}")
}
```

A két változat ugyanazt a kimenetet adja (`Rust!` és `Hello, Rust`). A döntés: ha a függvénynek valóban tulajdonolt értékre van szüksége, a hívó klónozhat; ha csak olvasni kell, kölcsönözzünk.

6.1.9 Rövid összefoglaló

A Rustban minden értéknek egy tulajdonosa van, és a tulajdonjog átadása mozgatást jelent, amely destruktív. A mozgatott változó használata fordítási időben `E0382` hibát ad, így nincs use-after-move, double-free vagy „valid but unspecified” állapot. A `String` példája megmutatja, hogy a mozgatás csak a veremmezőket (ptr, len, cap) másolja, a kupacpuffer nem duplikálódik. A függvényhívás és visszatérés is mozgat, a `clone()` pedig explicit, mély másolatot készít. **Továbbvitt eredmény:** ez a mozgatási modell alapozza meg a Copy, Clone és Drop alfejezetben a `Copy`/`Drop` szabályait, valamint a Referenciák és a kölcsönzési szabály és a Closure-ök: Fn, FnMut, FnOnce és a capture alfejezetekben a mozgatás és a capture közötti kapcsolatot.

6.1.10 Források és ellenőrzési hiányok

A fejezet a Rust Book ownership-fejezetére (https://doc.rust-lang.org/book/), a Rust Reference mozgatási szemantikára vonatkozó részére (https://doc.rust-lang.org/reference/) és a standard könyvtár dokumentációjára (https://doc.rust-lang.org/std/) támaszkodik. Az `E0382` hibakód stabil, a pontos hibaüzenet szövege a rögzített toolchainre (Rust 1.99.1) vonatkozik. A `String` és `Vec` viselkedése stabil API, a bemutatott példák a 2021-es és a 2024-es editionnel is fordulnak.

6.2 Copy, Clone és Drop: másolás és RAII Rustban

6.2.1 Motiváció

A C++-ban a másolás és a mozgatás vezérlése a programozó kezében van: másolási konstruktort, mozgatási konstruktort és értékadó operátorokat írhatunk, ezek döntik el, hogy egy értékből kettő vagy egy marad-e. A Rust ezt megfordítja: **a mozgatás az alapértelmezett**, és csak azok a típusok másolhatók implicit módon, amelyek ezt kifejezetten vállalják a `Copy` trait segítségével. A mély másolás sosem történik meg automatikusan; azt a `Clone` trait explicit hívása végzi. A RAII mindkét nyelv központi eleme, de Rustban a destruktor a `Drop` trait, amelynek szabályai és korlátai vannak – például **hibát nem tud visszaadni**, ez pedig alapvetően befolyásolja, hová tehetjük a perzisztenciát.

6.2.2 Fogalmak és szabályok

Definíció (Copy). A `Copy` trait egy olyan jelölő (marker) trait, amely azt mondja ki: a típus értéke **bitenkénti másolással** biztonságosan duplikálható úgy, hogy az eredeti érték érvényes marad. Ha egy típus implementálja a `Copy`-t, akkor a hozzárendelés és a függvényargumentum-átadás **másolást** végez, nem mozgatást.

Szabály. Egy típus akkor és csak akkor lehet `Copy`, ha minden mezője `Copy`, és a típus **nem** implementálja a `Drop` traitet. A `Drop` és a `Copy` kölcsönösen kizárják egymást, mert a bitenkénti másolás megkerülné a destruktor logikáját.

Definíció (Clone). A `Clone` trait explicit, felhasználó által hívott másolást valósít meg. A `clone()` metódus tetszőlegesen összetett lehet: végezhet mély másolást, foglalhat új memóriát, vagy akár olcsó is lehet. A Rust **nem hívja automatikusan** a `clone()`-t, ezért a drága művelet mindig látható a hívás helyén.

Definíció (Drop). A `Drop` trait a Rust destruktora. Aláírása `fn drop(&mut self)`: az egyetlen paramétere a `&mut self`, más argumentumot nem kap, és nem ad vissza értéket, így hibát sem tud jelezni. A `drop` metódus akkor fut le, amikor az érték kimegy a hatókörből, és a mezők automatikus eldobása előtt lehetőséget ad erőforrások lezárására. A `drop`-ot közvetlenül nem lehet hívni; korai eldobáshoz a `std::mem::drop` szolgál.

Szabály (drop-sorrend). Két külön szabály van, és a kettő ellentétes irányú:

1. A **lokális változók** a hatókör végén a deklaráció *fordított* sorrendjében dobódnak el: először az utoljára deklarált. Ez megegyezik a C++ automatikus változóinak destruktor-sorrendjével.
2. Egy **struktúra mezői** a deklaráció *sorrendjében* dobódnak el: először az első mező. Ez eltér a C++-tól, ahol a tagok fordított sorrendben semmisülnek meg. Ha a struktúrának saját `Drop` implementációja van, először a `Drop::drop` fut le (a mezők ekkor még érvényesek), és csak utána dobódnak el automatikusan a mezők.

```
struct Noisy(&'static str);

impl Drop for Noisy {
    fn drop(&mut self) {
        println!("drop {}", self.0);
    }
}

struct Pair {
    a: Noisy,
    b: Noisy,
}

impl Drop for Pair {
    fn drop(&mut self) {
        // A mezők itt még érvényesek.
        println!("drop Pair ({}, {})", self.a.0, self.b.0);
    }
}

fn main() {
    {
        let _a = Noisy("lokális a");
        let _b = Noisy("lokális b");
    }
    let _p = Pair { a: Noisy("mező a"), b: Noisy("mező b") };
}
```

A kimenet (Rust 1.99.1):

```
drop lokális b
drop lokális a
drop Pair (mező a, mező b)
drop mező a
drop mező b
```

Szabály (`std::mem` segédfüggvények). A `std::mem::drop` egy üres függvény, amely azonnal eldobja az argumentumát; a `std::mem::take` egy `&mut T`-ből kicseréli az értéket a `Default`-jára és visszaadja a régit; a `std::mem::replace` egy megadott értékre cseréli a régit; a `std::mem::swap` két `&mut T` értéket cserél meg.

6.2.3 C++ és Rust összevetés

C++-ban a másolás és mozgatás szabályait az osztály konstruktorai és értékadó operátorai határozzák meg:

```cpp
struct Buffer {
    char* data;
    size_t len;

    // Másolási konstruktor: mély másolás
    Buffer(const Buffer& other)
        : data(new char[other.len]), len(other.len) {
        std::copy(other.data, other.data + len, data);
    }

    // Mozgatási konstruktor: erőforrás átvétele
    Buffer(Buffer&& other) noexcept
        : data(other.data), len(other.len) {
        other.data = nullptr;
        other.len = 0;
    }

    ~Buffer() { delete[] data; }
};
```

A Rust-ban ugyanezt a `Clone` (explicit mély másolás) és a `Drop` (erőforrás-felszabadítás) fejezi ki, míg a mozgatás automatikus, bitenkénti:

```rust
struct Buffer {
    data: Vec<u8>,
}

impl Clone for Buffer {
    fn clone(&self) -> Self {
        Self { data: self.data.clone() }
    }
}

impl Drop for Buffer {
    fn drop(&mut self) {
        // A Vec automatikusan felszabadítja a kupacot,
        // itt csak akkor kell Drop, ha más erőforrást is kezelünk.
        println!("Buffer dropped");
    }
}
```

A kulcskülönbség: C++-ban a mozgatási konstruktor lehet kézzel írt (Rule of Five), de megfelelő feltételek mellett a fordító implicit módon is generálja (Rule of Zero típusoknál ez a szokásos), és a mozgatás után az eredeti objektum „kiürült”, érvényes de nem specifikált állapotban marad. Rustban a mozgatás bitenkénti másolás, de a mozgatott érték **statikusan érvénytelenné** válik, és nem férhetünk hozzá többé. A másolás viszont csak akkor történik, ha a típus `Copy`, vagy ha explicit `clone()`-t hívunk.

6.2.4 Labor: rkv `Store` mentése Drop-ban – és miért nem jó ötlet

Munkakönyvtár: `~/rust-labor/rkv/`

A labor célja, hogy a `rkv-core` crate `Store` típusához perzisztenciát adjunk, és megvizsgáljuk, miért nem a `Drop` a megfelelő hely a mentésre. A `Store` jelenleg csak memóriában tárol, de hamarosan fájlba is írni fog. A perzisztencia hibát okozhat (I/O hiba), a `Drop` viszont nem tud hibát visszaadni.

A kiinduló állapot a Függőségek, feature-ök, verziózás és workspace-ek ellenőrzési pontja: a `Store` a `rkv-core/src/store.rs`-ben, a hibatípus a `rkv-core/src/error.rs`-ben él (Modulok és láthatóság: header nélküli világ).

1. lépés: nyissuk meg a `rkv-core/src/store.rs` fájlt, és adjunk egy `dirty` mezőt a `Store`-hoz. A módosító metódusok (`set`, `delete`, `clear`) ezt igazra állítják; a `get` és a `keys` változatlan.

Kódblokk — fájl: `rkv-core/src/store.rs` · művelet: módosításrészlet: a `Store` definíciója, a `new`, `set`, `delete` és `clear` cseréje; a többi metódus és a tesztek változatlanok · ellenőrzés: `cargo test -p rkv-core` → 4 sikeres teszt

```rust
#[derive(Debug, Default)]
pub struct Store {
    data: HashMap<String, Vec<u8>>,
    /// Igaz, ha az utolsó mentés óta módosult a tároló.
    dirty: bool,
}

impl Store {
    pub fn new() -> Self {
        Self {
            data: HashMap::new(),
            dirty: false,
        }
    }

    pub fn set(&mut self, key: String, value: Vec<u8>) {
        self.data.insert(key, value);
        self.dirty = true;
    }

    pub fn delete(&mut self, key: &str) -> Option<Vec<u8>> {
        let old = self.data.remove(key);
        if old.is_some() {
            self.dirty = true;
        }
        old
    }

    pub fn clear(&mut self) {
        if !self.data.is_empty() {
            self.dirty = true;
```

```
        }
        self.data.clear();
    }

    // get és keys: változatlan
}
```

A `dirty` mező jelzi, hogy történt-e mentetlen módosítás.

2. lépés: kísérlet a `Drop` implementálására.

Egy naiv megoldás, hogy a `Drop`-ban megpróbáljuk menteni, ha `dirty` igaz:

```
impl Drop for Store {
    fn drop(&mut self) {
        if self.dirty {
            // Itt kellene kiírni a fájlt, de a hibát nem tudjuk kezelni!
            // eprintln!("Warning: unsaved changes in Store");
            // A flush() Result-ot adna, amit itt nem tudunk továbbadni.
        }
    }
}
```

Probléma: a `Drop::drop` nem adhat vissza `Result`-ot, így az esetleges I/O hibát nem tudjuk a hívónak jelenteni. Ha a mentés itt történne, a hiba elveszne, vagy a programnak panikálnia kellene, ami nem elfogadható könyvtári kódban. Ráadásul a `Drop` futásakor a program esetleg már nem is tud helyesen reagálni a hibára.

3. lépés: a helyes megoldás: explicit `flush` metódus.

A perzisztenciát a `Store` felhasználójának kell kezdeményeznie egy `flush` metódussal, amely `Result`-ot ad vissza. A hibatípus a Modulok és láthatóság: header nélküli világ alfejezetben létrehozott `crate::error::Error`, amelynek `Io` variánsa épp az I/O-hibák továbbadására való:

Kódblokk — fájl: `rkv-core/src/store.rs` · művelet: beillesztés: a fájl elejére a `use crate::error::Error;` sor, az `impl Store` blokkba a `flush` metódus · ellenőrzés: `cargo build -p rkv-core`

```
use crate::error::Error;

impl Store {
    pub fn flush(&mut self) -> Result<(), Error> {
        if self.dirty {
            // Itt jön majd a tényleges fájlírás (későbbi alfejezetekben).
            // Most csak szimuláljuk a mentést.
            // Például: std::fs::write("store.dat", serialize(&self.data))?;
            self.dirty = false;
        }
        Ok(())
    }
}
```

A `Drop` implementációt **teljesen elhagyjuk**, mert a `Store` nem birtokol olyan erőforrást, amelyet a memória felszabadításán túl kezelnie kellene. A `HashMap` maga felszabadítja a kupacot, így nincs szükség `Drop`-ra.

4. lépés: a labor kiegészítése `mem::take`-kel.

Ha a `Drop`-ban mégis szeretnénk valamit tenni, akkor a `std::mem::take` segítségével kiüresíthetjük a `data`-t, hogy a memória felszabadulása biztos legyen, de ez itt felesleges, mert a `HashMap` alapértelmezetten is felszabadul. A `mem::take` inkább akkor hasznos, ha egy `&mut T`-ből akarunk

értéket kivenni anélkül, hogy a típust `Default` nélkül kellene hagyni. A `flush` implementációban viszont használhatjuk a `mem::take`-et, ha a `data`-t ki akarjuk cserélni egy üres mapre:

```
pub fn flush(&mut self) -> Result<(), Error> {
    if self.dirty {
        let data = std::mem::take(&mut self.data);
        // data-val dolgozunk, majd betesszük vissza vagy mentjük.
        // Figyelem: ha itt `?`-es I/O áll és hibázik, a `self.data` üresen maradna
        // (a `take` után). A 7.2 ezt visszaállító őrrel vagy kölcsönzéssel kerüli el.
        self.data = data;
        self.dirty = false;
    }
    Ok(())
}
```

Ez a minta később, a hibakezelés és a párhuzamosság témakörében visszatér majd.

5. lépés: teszteljük a viselkedést. A teszt a `store.rs` meglévő tesztmoduljába kerül; a privát `dirty` mezőt azért érheti el, mert a `tests` modul a `store` modul gyermeke.

Kódblokk — fájl: `rkv-core/src/store.rs` · művelet: beillesztés a meglévő `mod tests` blokk végére · ellenőrzés: `cargo test -p rkv-core` → 5 sikeres teszt

```
    #[test]
    fn flush_clears_dirty_flag() {
        let mut store = Store::new();
        assert!(!store.dirty);
        store.set("key".to_string(), vec![1, 2, 3]);
        assert!(store.dirty);
        store.flush().unwrap();
        assert!(!store.dirty);
        assert!(store.delete("nincs").is_none());
        assert!(!store.dirty); // nem létező kulcs törlése nem módosítás
    }
```

Futtassuk a tesztet:

```
cargo test -p rkv-core
```

Várt kimenet (rövidítve; a tesztek sorrendje változhat):

```
running 5 tests
test command::tests::list_prefix_none_has_no_trailing_space ... ok
test command::tests::set_formats_key_and_value ... ok
test store::tests::delete_returns_old_value ... ok
test store::tests::flush_clears_dirty_flag ... ok
test store::tests::set_and_get ... ok

test result: ok. 5 passed; 0 failed; ...
```

A labor kimondja: a perzisztencia **explicit API**, a `Drop` nem alkalmas hibakezelésre. A `Store` típus `dirty` mezője és `flush` metódusa lesz az alap a későbbi perzisztencia-réteghez.

Ellenőrzési pont. A `Store` (`rkv-core/src/store.rs`) a `dirty` mezővel és a `flush(&mut self) -> Result<(), Error>` metódussal bővült, `Drop` implementáció nélkül; a `cargo test --workspace` 7 tesztje (5 + 2) sikeres, az `rkv-cli` változatlan.

Továbbvitt eredmény. A `Store` típus a `dirty` mezővel és az explicit `flush` metódussal a Borrowing: referenciák és a kölcsönzési szabály alfejezetben kap kölcsönzési mintákat; a `Result`-alapú hibakezelést a Result, a ? operátor és a panic helye alfejezet mélyíti el, a `mem::take` mintát pedig a Box, Rc, Arc és Weak alfejezet is használja.

6.2.5 Tipikus hibák és átállási csapdák

1. **A `clone()` használata a borrow checker „legyőzésére”.** Amikor a kölcsönzési hiba miatt a kód nem fordul, a kezdő Rust-fejlesztő gyakran `clone()`-t hív, hogy a fordítót kielégítse. Ez működhet, de felesleges allokációt és teljesítményromlást okoz. A helyes megoldás a kölcsönzési és lifetime-szabályok megértése, nem a másolás.
2. **A `Copy` és `Clone` összekeverése.** Sok fejlesztő azt várja, hogy a `derive(Clone, Copy)` minden típust automatikusan másolhatóvá tesz. A `Copy` csak bitenkénti másolásra alkalmas típusoknál működik, és csak akkor deriválható, ha a típus minden mezője `Copy`. A `String`, `Vec`, `HashMap` nem `Copy`, így az ilyen típusokat tartalmazó struktúra sem lehet az.
3. **A `Drop` implementálása hibakezelést igénylő műveletre.** Mivel a `Drop` nem adhat vissza `Result`-ot, a benne végzett I/O vagy hálózati művelet hibája elveszik. Ilyen műveleteket explicit metódusokba kell tenni, a `Drop` pedig csak csendes takarításra való.

6.2.6 Feladatok

1. **Fogalmi kérdés:** Miért nem lehet egyszerre egy típus `Copy` és `Drop`? Mondjon egy konkrét példát arra, mi történne, ha mégis megengedné a nyelv.
2. **C++ kód átírása:** Adott a következő C++ osztály, amely fájlmutatót birtokol, és másoláskor duplikálja a fájlleírót (a példa kedvéért `dup`-pal), destruktorkor pedig bezárja. Írd át Rustra a megfelelő idiomatikus kóddal, `Clone` és `Drop` használatával.

   ```
   struct File {
       int fd;
       File(int fd) : fd(fd) {}
       File(const File& other) : fd(dup(other.fd)) {}
       ~File() { close(fd); }
   };
   ```

   A Rust verzióban használj `File` típust a `std::fs`-ből, és magyarázd el, hogyan valósul meg a duplikálás és a lezárás.
3. **Implementációs feladat:** Készíts egy `CleaningBuffer` típust, amely egy `Vec<u8>`-ot tárol, és `Drop` implementációjában kiírja a puffer hosszát a standard hibára, majd üríti a puffert a `std::mem::take` segítségével. Teszteld, hogy a `Drop` akkor is lefut, ha a változót `std::mem::drop`-pal korán eldobjuk.
4. **Refaktorálás:** A laborban szereplő `Store` `flush` metódusa jelenleg szimulálja a fájlba írást. Egészítsd ki úgy, hogy a `data` mapet JSON formátumba szerializálja (például `serde_json` használatával), és írd ki egy fájlba. A `flush` hibát adjon vissza, ha a szerializálás vagy a fájlírás sikertelen. A `Drop`-ot ne módosítsd.

6.2.7 Megoldások

1. A `Copy` bitenkénti másolást jelent, ami megkerülné a `Drop::drop` logikáját. Ha egy típus `Copy` és `Drop` is lenne, akkor a másoláskor az erőforrás-birtoklás duplikálódna anélkül, hogy a destruktor tudomást szerezne róla, így a erőforrás kétszer szabadulna fel vagy sérülne. Például egy `File` típus, amely lezárja a fájlt a `Drop`-ban, nem lehet `Copy`, mert a bitmásolás után mindkét példány ugyanazt a fájlleírót birtokolná, és mindkettő lezárná.
2. A Rust megfelelő:

```rust
use std::fs::File;
use std::io;

struct MyFile {
    file: File,
}

impl MyFile {
    /// Hibázható duplikálás: a `dup` (itt `File::try_clone`) elbukhat
    /// (például elfogyott a fájlleíró), ezért `io::Result`-ot adunk vissza.
    fn try_clone(&self) -> io::Result<Self> {
        Ok(Self { file: self.file.try_clone()? })
    }
}

// A Drop nem szükséges, mert a File maga lezárja a fájlt a hatókör végén.
```

A `File` típus a `std::fs`-ben már implementálja a RAII-t: a hatókör végén automatikusan bezárja a fájlt. A `MyFile` szándékosan **nem** implementálja a `Clone` traitet, mert a `Clone::clone` aláírása nem hibázhat (`fn clone(&self) -> Self`), a fájlleíró duplikálása viszont igen; egy panikoló `unwrap` mögé rejtett hiba megtévesztő volna. Ezért a duplikálás `try_clone() -> io::Result<Self>`. Fontos következmény: a duplikált leíró *ugyanarra a megnyitott fájlleírásra* mutat, ezért a fájlpozíció közös. Ellenőrzött viselkedés: ha az eredeti fájlból 3 bájtot olvasunk az `abcdef` tartalomból (`abc`), a másolatból a következő olvasás `def`-et ad.

3. Példa implementáció:

```rust
struct CleaningBuffer {
    data: Vec<u8>,
}

impl CleaningBuffer {
    fn new(data: Vec<u8>) -> Self {
        Self { data }
    }
}

impl Drop for CleaningBuffer {
    fn drop(&mut self) {
        eprintln!("Dropping buffer of length {}", self.data.len());
        // Demonstráció: a mem::take kicseréli a mezőt egy üres Vec-re (Default),
        // és az eredeti tartalmat adja vissza. Manuális ürítésre a memória
        // felszabadításához nincs szükség: a Vec automatikus dropja elegendő.
        let old_data = std::mem::take(&mut self.data);
        eprintln!("old_data.len() = {}, self.data.len() = {}", old_data.len(),
        ↪  self.data.len());
        // old_data.len() == 4, self.data.len() == 0; az old_data itt dobódik el.
    }
}

fn main() {
    let buf = CleaningBuffer::new(vec![1, 2, 3, 4]);
    std::mem::drop(buf); // korai eldobás
    println!("End of main");
}
```

A `std::mem::drop` hatására a `drop` metódus azonnal lefut: a standard hibára `old_data.len() = 4, self.data.len() = 0` kerül (a `take` után az eredeti négy elem az `old_data`-ban van, a mezőben üres `Vec` marad), és csak ezután jelenik meg a “End

of main”. A `mem::take` itt szemléltetés: a `Vec` automatikus dropja a memóriát manuális ürítés nélkül is felszabadítaná.

4. A `flush` kibővítése:

```
use serde_json::json;

pub fn flush(&mut self) -> Result<(), Error> {
    if self.dirty {
        let json = json!(&self.data);
        let serialized = serde_json::to_string(&json)
            .map_err(|e| Error::Io(std::io::Error::new(std::io::ErrorKind::InvalidData,
            ↪  e)))?;
        std::fs::write("store.json", serialized)
            .map_err(Error::Io)?;
        self.dirty = false;
    }
    Ok(())
}
```

A `serde_json` a `Cargo.toml`-ben `serde_json = "1"` függőségként szerepel. A hibát `Error::Io`-ba csomagoljuk.

6.2.8 Rövid összefoglaló

A `Copy` trait bitenkénti másolást jelöl, és csak olyan típusoknál alkalmazható, amelyek minden mezője `Copy` és nincs `Drop` implementációjuk. Az implicit másolás sosem történik meg a `Clone` nélkül, így a drága mély másolás mindig látható. A `Drop` a RAII destruktora, amely nem adhat vissza hibát, ezért csak csendes takarításra való; a perzisztencia és más hibalehetőséget rejtő műveletek explicit metódusokba tartoznak. A laborban a `rkv-core::Store` típushoz `dirty` mezőt és `flush` metódust adtunk, amely `Result<(), Error>`-t ad vissza; a `Drop` implementációt elvetettük, mert a mentési hibát nem tudná továbbítani. A `std::mem::take`, `replace`, `swap` és `drop` segédfüggvények az erőforrás-kezelés fontos eszközei. A `Store` és a `flush` a további alfejezetekben (Referenciák és a kölcsönzési szabály: sok olvasó XOR egy író, Box, Rc, Arc és Weak, Result, a ? operátor és a panic helye) alapul szolgál.

7 Borrowing

7.1 Referenciák és a kölcsönzési szabály: sok olvasó XOR egy író

7.1.1 Motiváció: a C++-os aliasing-csapdák és a Rust válasza

Tapasztalt C++-fejlesztőként pontosan ismered az aliasing-problémákat. Klasszikus példa az iterátor-invalidálás, amikor egy `std::vector`-on iterálsz, és közben `push_back`-et hívsz:

```
#include <vector>
#include <iostream>

int main() {
    std::vector<int> v{1, 2, 3, 4};
    for (int& x : v) {
        if (x == 2) {
            v.push_back(42); // UB: push_back reallokálhat, x érvénytelenné válik
        }
    }
}
```

Ez nem fordítási hiba, hanem definiálatlan viselkedés (undefined behavior, UB): a `v.push_back(42)` reallokálhat, és a ciklus `x` referenciája, illetve az iterátor érvénytelenné válik. Vigyázat: nem minden C++-aliasolás hiba. Az, hogy egy `const T&` és egy `T&` ugyanarra az objektumra mutat, szabályos és (egyszálú kódban) jól definiált C++; a `const T&` csak azt ígéri, hogy *ezen a referencián át* nem írunk, azt nem, hogy az érték más úton nem változhat. A valódi hibák az érvénytelenített hivatkozás (mint fent) és a szálak közötti adatverseny.

A Rust válasza egy statikus szabály, a **kölcsönzési szabály**, amely a kód szintaktikai szerkezete alapján garantálja, hogy ilyen helyzet soha nem fordulhat elő. A szabály megszegése nem UB, hanem fordítási hiba, például `E0502`. Ez a szabály alapozza meg a nyelv memória- és szálbiztonsági garanciáit.

Továbbvitt eredmény. Az itt bevezetett kölcsönzési szabály a későbbi alfejezetek alapja: a A borrow checker működése: non-lexical lifetimes és tipikus javítások alfejezet a szabály hatókör-kezelését, a Élettartam-annotációk és az elision szabályai alfejezet az élettartamok explicit jelölését, a Belső mutabilitás: Cell, RefCell és a futásidejű kölcsönzés alfejezet a szabály megkerülésének ellenőrzött eseteit, a Szálak, Send és Sync: data race fordítási időben kizárva alfejezet pedig a szálbiztonságra való kiterjesztést tárgyalja.

7.1.2 Fogalmak és szabályok

Definíció. A **referencia** (reference) egy olyan típus, amely egy meglévő értékre mutat, anélkül hogy átvenné annak tulajdonjogát. A Rustban két referencia-típus van:

- `&T` – **megosztott referencia** (shared reference): olvasási hozzáférést ad; a hivatkozott érték nem módosítható ezen a referencián keresztül (kivéve a belső mutabilitást, lásd később).
- `&mut T` – **kizárólagos referencia** (mutable/exclusive reference): olvasási és írási hozzáférést ad; a kölcsönzés ideje alatt senki más nem férhet a hivatkozott értékhez.

Definíció. A **kölcsönzés** (borrowing) az a művelet, amikor egy érték tulajdonosa átmeneti hozzáférést ad egy referencián keresztül, de megtartja a tulajdonjogot. A kölcsönzés a `&` (kölcsönzés megosztott hivatkozásként) és a `&mut` (kölcsönzés kizárólagos hivatkozásként) operátorokkal történik.

Szabály. Egy adott értékre vonatkozóan egy időpillanatban:

- akárhány megosztott referencia `&T` létezhet, **vagy**
- pontosan egy kizárólagos referencia `&mut T` létezhet.

A két fajta referencia egyszerre nem lehet érvényben ugyanarra az értékre. Ezt a szabályt a **kölcsönzés-ellenőrző** (borrow checker) statikusan, fordítási időben ellenőrzi.

A referencia típusa soha nem lehet `null`. Ha hiányzó referenciát akarunk reprezentálni, az `Option<&T>` vagy `Option<&mut T>` típust használhatjuk, ahol a `None` a hiányzó eset.

A referencia nem élheti túl a hivatkozott értéket; ezt a szabályt a fordító az élettartamok (lifetimes) alapján ellenőrzi. Ebben az alfejezetben csak annyit használunk, hogy a kölcsönzés hatóköre a *referencia* utolsó használatáig tart (non-lexical lifetimes, NLL). Ez a referenciák kölcsönzési tartományára vonatkozik; a `Drop`-ot implementáló, erőforrást őrző értékek (például a `RefCell::borrow()` által visszaadott `Ref` vagy a `MutexGuard`) a hatókörük végéig élnek, függetlenül attól, hogy az utolsó olvasásuk hol volt (lásd A borrow checker működése). A részletes élettartam-annotációk a Élettartam-annotációk és az elision szabályai alfejezetben (8.1) jönnek.

7.1.3 C++ és Rust összevetés

Vegyünk egy egyszerű C++ példát, ahol egy függvény módosít egy egész számot, egy másik pedig csak olvassa.

```
void add_one(int& x) {
    ++x;
}

void print(const int& x) {
    std::cout << x << '\n';
}

int main() {
    int a = 10;
    add_one(a);   // nem-const referencia
    print(a);     // const referencia
    // Aliasing: szabályos és definiált C++. A const referencia csak a saját
    // elérési útról ígér csendet, az értéket más úton módosíthatja valaki:
    // int* p = &a;
    // add_one(*p);   // rendben, nincs UB (egyszálú kódban)
    // print(a);      // a már 12; a const ref nem garantálta az állandóságot
}
```

A Rust megfelelője:

```rust
fn add_one(x: &mut i32) {
    *x += 1;
}

fn print(x: &i32) {
    println!("{x}");
}

fn main() {
    let mut a = 10;
    add_one(&mut a);
    print(&a);

    // Ez lefordul (NLL): a p-t a módosítás után már nem használjuk,
    // így a megosztott kölcsönzés az add_one előtt véget ér.
    let p = &a;
    print(p);
    add_one(&mut a);
}
```

A kölcsönzési szabály akkor sérül, ha a megosztott referenciát a módosítás *után* is használnánk:

Kódblokk — önálló példa · **szándékosan hibás** · várt hiba: `E0502` (`cannot borrowaas mutable because it is also borrowed as immutable`)

```rust
fn add_one(x: &mut i32) {
    *x += 1;
}

fn main() {
    let mut a = 10;
    let p = &a;
    add_one(&mut a); // E0502: p még él, mert alább használjuk
    println!("{p}");
}
```

A két változat között egyetlen különbség van: az, hogy a `p` utolsó használata a `&mut a` előtt vagy után áll. A non-lexical lifetimes (NLL) miatt a kölcsönzés nem a blokk végéig, hanem az utolsó használatig tart.

A különbség lényege: C++-ban a `const int&` arra utal, hogy a hivatkozott objektumot az adott referencián keresztül nem módosíthatjuk, de más aliasok még módosíthatják. A Rust `&T` referencia ennél erősebb: a kölcsönzési szabály garantálja, hogy a kölcsönzés ideje alatt **senki** nem módosíthatja az értéket, mert `&mut T` nem létezhet mellette. Ez a statikus invariant teszi lehetővé, hogy a Rust kód biztonságosan párhuzamosítható legyen, és hogy a fordító agresszívan optimalizálhasson.

7.1.4 Kidolgozott példa: iterátor-invalidálás C++-ban vs. fordítási hiba Rustban

Készítsünk egy kis Rust projektet, amelyben megpróbáljuk lefordítani a C++ UB-s kód Rust megfelelőjét.

Munkakönyvtár: `~/rust-labor/r014/`

Hozd létre a projektet, és írd be a hibás Rust kódot.

```
mkdir -p ~/rust-labor
cd ~/rust-labor
cargo new r014 --bin
cd r014
```

A `src/main.rs` tartalma – ez a változat szándékosan nem fordul le:

```
fn main() {
    let mut v = vec![1, 2, 3, 4];

    for x in &v {
        if *x == 2 {
            v.push(42); // hiba: v kizárólagos kölcsönzése ütközik a &v megosztott kölcsönzéssel
        }
    }

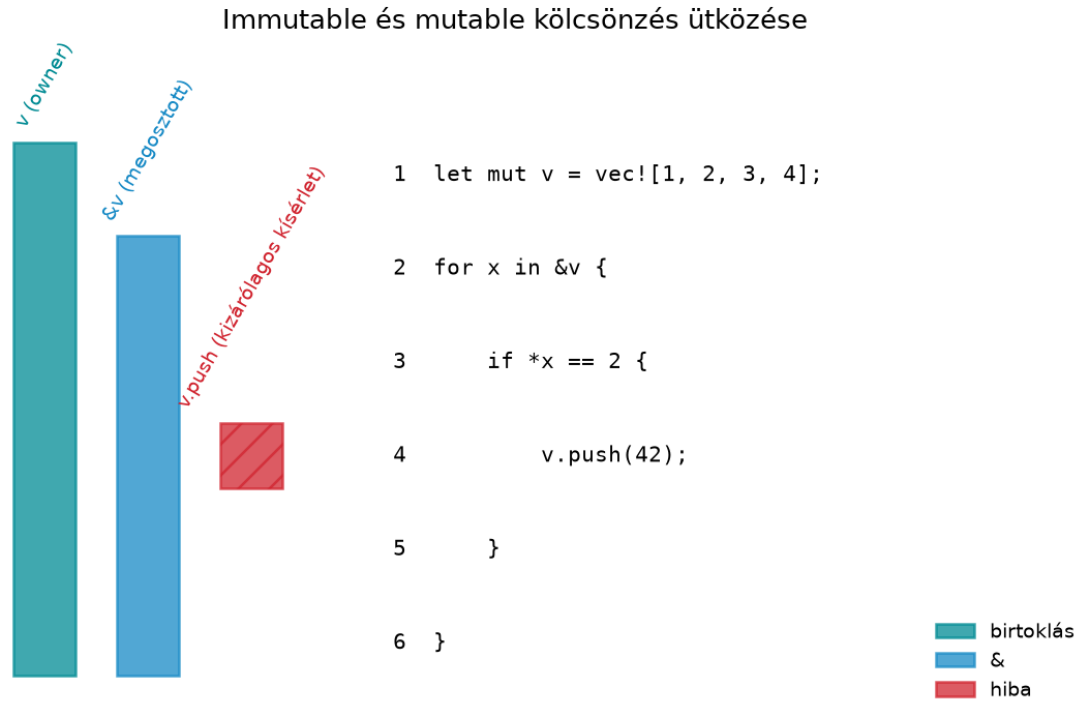
    println!("{v:?}");
}
```

A `cargo run` parancs kiadása után a fordító a következő lényegi hibát jelzi (a pontos szöveg verziófüggő):

```
error[E0502]: cannot borrow `v` as mutable because it is also borrowed as immutable
 --> src/main.rs:6:13
```

A hiba oka: a `for x in &v` megosztott kölcsönzést hoz létre `v`-re, amely a ciklus teljes futása alatt él. A ciklus törzsén belül a `v.push(42)` kizárólagos kölcsönzést próbál szerezni, ami ütközik a még élő megosztott kölcsönzéssel. Ez pontosan az a helyzet, amely C++-ban UB-t okozna; Rustban fordítási hibát kapunk.

A következő diagram mutatja a kölcsönzések érvényességi tartományát a hibás kódban.



7.1. ábra: Immutable és mutable kölcsönzés ütközése - A &v megosztott kölcsönzés a for ciklus teljes idejére kiterjed, ezért a cikluson belüli v.push(42) kizárólagos kölcsönzési kísérlete hibát okoz.

A megosztott kölcsönzés a ciklusfejben kezdődik (1. sor) és a ciklus végéig tart (5. sor). A `push` hívás a 3. sorban próbál kizárólagos hozzáférést, ezért a két sáv átfedi egymást, ami hiba.

Javítás. A cél az, hogy a kizárólagos hozzáférést csak a megosztott kölcsönzés lejárta után kérjük. Egy idiomatikus megoldás: először gyűjtsük össze, hogy szükség van-e a beszúrásra, majd a ciklus után módosítsuk a vektort.

```
fn main() {
    let mut v = vec![1, 2, 3, 4];

    let mut needs_push = false;
    for &x in &v {
        if x == 2 {
            needs_push = true;
        }
    }

    if needs_push {
        v.push(42);
    }

    println!("{v:?}");
}
```

Ez a változat legfeljebb *egy* `42`-t szúr be, akkor is, ha több `2`-es volt (a `needs_push` bool csak azt jegyzi, hogy volt-e). Ha minden `2` után kell egy `42`, a vektort új vektorba építjük: `let v: Vec<i32> = v.into_iter().flat_map(|x| if x == 2 { vec![x, 42] } else { vec![x] }).collect();` (a `[1, 2, 2, 4]`-ből `[1, 2, 42, 2, 42, 4]` lesz).

A `for &x in &v` ciklusban a megosztott kölcsönzés csak a ciklus végéig él. A ciklus után a `v.push(42)` már megkaphatja a kizárólagos kölcsönzést, mert az előző megosztott kölcsönzés lejárt. A program kimenete:

```
[1, 2, 3, 4, 42]
```

Ez a minta – ellenőrzés megosztott hozzáféréssel, majd módosítás kizárólagos hozzáféréssel – rendkívül gyakori idiomatikus Rust kódban. A hibaüzenetek (`E0502`) olvasása és a javítási stratégia a A borrow checker működése alfejezetben mélyebb tárgyalást kap.

7.1.5 Tipikus hibák és átállási csapdák

7.1.5.1 1. Klónozás a borrow checker „legyőzésére”

C++-ban megszokott, hogy ha egy függvény módosítani akar egy értéket, akkor referenciát vagy mutatót adunk át. Rustban sokan a `clone()`-hoz nyúlnak, hogy elkerüljék a kölcsönzési hibákat:

```
fn process(v: &Vec<i32>) {
    let mut copy = v.clone(); // a kölcsönzési hiba elkerülése
    copy.push(42);
    println!("{copy:?}");
}
```

Ez lefordul, de nem azt érjük el, amit szeretnénk: ha az eredeti `v`-t akartuk módosítani, a klón módosítása nem hat rá. A helyes megoldás az, hogy a függvény `&mut Vec<i32>` paramétert kap, és azon dolgozik. A felesleges klónozás teljesítmény- és karbantartási problémákat okoz, ráadásul elfedi a tervezési hibákat.

7.1.5.2 2. `Rc<RefCell<T>>` túlhasználata C++-os `shared_ptr` reflexből

A C++-fejlesztők gyakran a `std::shared_ptr`-hez hasonló megoldást keresnek, amikor megosztott, módosítható állapotra van szükségük. Rustban ez az `Rc<RefCell<T>>` vagy `Arc<Mutex<T>>` mintát jelenti. Azonban ezt csak akkor használd, ha valóban megosztott tulajdonra van szükség; a `RefCell` futásidejű kölcsönzés-ellenőrzést végez, így a fordítási hibák futásidejű pánikká alakulhatnak. A Belső mutabilitás: Cell, RefCell alfejezet ezt részletesen tárgyalja.

7.1.5.3 3. A kölcsönzés hatókörének félreértése

Sokan azt hiszik, hogy egy `&mut` kölcsönzés a teljes blokk végéig tart. Valójában a non-lexical lifetimes miatt a kölcsönzés csak az utolsó használatig él. Például:

```
fn main() {
    let mut v = vec![1, 2, 3];
    let r = &mut v[0];
    *r += 1;                    // r utoljára itt használt
    println!("{v:?}");          // ez lefordul, mert a &mut kölcsönzés lejárt
}
```

Régebbi Rust-verziókban ez hibát adott volna, de a NLL bevezetése óta a fordító megérti, hogy `r` már nem használt, így a kölcsönzés nem él a `println!`-ig. Ez a viselkedés fontos az idiomatikus Rust kód írásához; részletek a A borrow checker működése alfejezetben.

7.1.6 Feladatok

7.1.6.1 1. Lefordul-e és miért?

Az alábbi Rust kód vajon lefordul-e? Válaszodat indokold a kölcsönzési szabállyal.

```
fn main() {
    let mut s = String::from("hello");
    let r1 = &s;
    let r2 = &mut s;
    println!("{r1}");
}
```

7.1.6.2 2. C++ kód átírása idiomatikus Rustra

Adott a következő C++ kód, amely egy vektor páros elemeit megduplázza:

```
#include <vector>

int main() {
    std::vector<int> v{1, 2, 3, 4};
    for (int& x : v) {
        if (x % 2 == 0) {
            x *= 2;
        }
    }
}
```

Írd át ezt idiomatikus Rustra, amely lefordul, és ugyanazt az eredményt adja. Ügyelj arra, hogy a Rust kód a kölcsönzési szabálynak megfelelően módosítsa az elemeket.

7.1.6.3 3. Implementációs feladat: `append_unique` függvény

Írj egy `append_unique(v: &mut Vec<i32>, value: i32)` függvényt, amely a `value` értéket csak akkor fűzi hozzá a `v`-hez, ha az még nincs benne. A függvény ne klónozzon, ne használjon `contains`-t úgy, hogy közben módosítja a vektort, hanem használja ki a kölcsönzési szabályt helyesen. Gondolj arra, hogy a `contains` megosztott kölcsönzést vesz igénybe, ami a kifejezés végén lejár.

Ez a minta később az rkv `MemStore` implementációjában is felbukkan, amikor egy kulcs létezését ellenőrizzük a beszúrás előtt.

7.1.6.4 4. Hibajavítás: iterálás közbeni módosítás

Az alábbi Rust kód szándékosan nem fordul le (`E0502`):

```rust
fn double_even(v: &mut Vec<i32>) {
    for x in v.iter() {
        if x % 2 == 0 {
            v.push(*x); // hiba: az iterátor megosztott kölcsönzése még él
        }
    }
}
```

Javítsd ki a kódot úgy, hogy lefordúljon, a páros elemek másolatai a vektor végére kerüljenek, és a ciklus előre ismert hosszúságú maradjon (ne legyen végtelen). Magyarázd el, miért oldja meg a javítás a kölcsönzési hibát, és miért nem végtelen a ciklus.

7.1.7 Megoldások

7.1.7.1 1. Lefordul-e és miért?

A kód **nem fordul le**. A hiba oka, hogy az `r1` megosztott kölcsönzést hoz létre `s`-re, és ez a kölcsönzés a `println!`-ben való felhasználásig él. Az `r2` kizárólagos kölcsönzést próbál szerezni, miközben `r1` még érvényben van. A szabály szerint egyszerre nem létezhet `&T` és `&mut T`. Ha az `r1`-et nem használnánk az `r2` létrehozása után, akkor a non-lexical lifetimes miatt `r1` lejárhatna, és az `r2` létrehozása már legális lenne – de ebben a sorrendben nem.

7.1.7.2 2. C++ kód átírása idiomatikus Rustra

```rust
fn main() {
    let mut v = vec![1, 2, 3, 4];

    for x in &mut v {
        if *x % 2 == 0 {
            *x *= 2;
        }
    }

    println!("{v:?}");
}
```

A `for x in &mut v` kizárólagos kölcsönzést hoz létre a vektor egészére, így a cikluson belül módosíthatjuk az elemeket. Az `x` típusa `&mut i32`, ezért a dereferálással (`*x`) férünk hozzá az értékhez. A ciklus végén a kölcsönzés lejár, és a `println!` már megosztott kölcsönzést használhat. A kimenet: `[1, 4, 3, 8]`.

7.1.7.3 3. `append_unique` függvény

```rust
fn append_unique(v: &mut Vec<i32>, value: i32) {
    if !v.contains(&value) {
        v.push(value);
    }
}

fn main() {
    let mut v = vec![1, 2, 3];
    append_unique(&mut v, 4);
    append_unique(&mut v, 2);
    println!("{v:?}"); // [1, 2, 3, 4]
}
```

A `v.contains(&value)` megosztott kölcsönzést vesz igénybe, amely a kifejezés kiértékelése után azonnal lejár. A `v.push(value)` már kizárólagos kölcsönzést kér, ami ekkor már legális. Ez a minta – ellenőrzés, majd módosítás – idiomatikus, és elkerüli a klónozást és a `RefCell`-t.

7.1.7.4 4. Hibajavítás

A hiba oka: a `v.iter()` iterátora megosztott kölcsönzést tart a vektoron a teljes ciklus alatt, a `push` viszont kizárólagos kölcsönzést kér, és reallokációval érvényteleníthetné az iterátort. Ez pontosan a C++ iterátor-invalidálása, amelyet a Rust fordítási időben kiszűr. A javítás: index alapú ciklus, rögzített hosszal.

```rust
fn double_even(v: &mut Vec<i32>) {
    for i in 0..v.len() {
        if v[i] % 2 == 0 {
            v.push(v[i]);
        }
    }
}
```

Ez lefordul. A `0..v.len()` tartomány egyszer, a ciklus elején értékelődik ki, a `Range` értéke a későbbi `push`-októl nem változik, ezért a ciklus csak az eredeti elemeken megy végig. A `v.push(v[i])` pedig a *kétfázisú kölcsönzés* (two-phase borrow) miatt szabályos: a `push` implicit `&mut v` fogadója az argumentum kiértékelése alatt csak „foglalt”, ezért a `v[i]` olvasása (amely egy `i32`-t másol ki) még megengedett, és a kizárólagos kölcsönzés csak a hívás pillanatában aktiválódik. Ugyanez teszi lehetővé a hivatalos példát, a `v.push(v.len())`-t is (lásd A borrow checker működése: non-lexical lifetimes és tipikus javítások). Olvashatóság szempontjából egyenértékű alternatíva a `let val = v[i]; v.push(val);`, amely a másolást kifejezetten mutatja.

7.1.8 Rövid összefoglaló

A Rust két referencia-típusa a `&T` (megosztott) és a `&mut T` (kizárólagos). A **kölcsönzési szabály** szerint egy értékre egyszerre akárhány `&T` létezhet, vagy pontosan egy `&mut T`, de a kettő egyszerre nem. A szabály megszegése fordítási hibát okoz (leggyakrabban `E0502`), nem definiálatlan viselkedést. Ez a szabály zárja ki az olyan C++-os hibákat, mint az iterátor-invalidálás vagy az egyszerre konstans és nem-konstans hivatkozás ugyanarra az objektumra. A non-lexical lifetimes miatt a kölcsönzés hatóköre az utolsó használatig tart. A szabály alapozza meg a későbbi alfejezeteket: a A borrow checker működése a hatókör-kezelést, a Élettartam-annotációk az explicit élettartamokat, a Belső mutabilitás a kivételeket, a Szálak, Send és Sync pedig a szálbiztonsági kiterjesztést tárgyalja.

7.1.9 Források és ellenőrzési hiányok

A nyelv szabályait a Rust Book (https://doc.rust-lang.org/book/) és a Rust Reference (https://doc.rust-lang.org/reference/) írja le. A fordító hibaüzenetének szövege verziófüggő, ezért a fejezet a hibakódot (`E0502`) és a hiba lényegét közli. A non-lexical lifetimes viselkedése a Rust 2018-as edition óta stabil; a pontos hatókörmegállapítás részleteit a A borrow checker működése alfejezet tárgyalja. Ebben az alfejezetben külső crate-et nem használunk.

7.2 A borrow checker működése: non-lexical lifetimes és tipikus javítások

7.2.1 Motiváció

C++-ban a mutatók és referenciák aliasing-problémái gyakran csak futás közben, nemdeterminisztikus hibaként jelentkeznek. Egy `std::vector` elemének referenciáját megtartva a `push_back` hívása reallokációt válthat ki, és a referencia érvénytelenné válik; a program definiálatlan viselkedést (UB) mutat. A Rust válasza erre a kölcsönzés-ellenőrző (borrow checker), amely már fordítási időben elutasítja az ilyen mintát. A hibaüzenetek eleinte frusztrálóak lehetnek, de ha megértjük, hogyan dönt a fordító, és ismerjük a tipikus javítási mintákat, a hibák tanulságos iránytűvé válnak.

7.2.2 Fogalmak és szabályok

Definíció. *Non-lexical lifetime (NLL)*: a Rust 2018 edition óta a kölcsönzések élettartama nem a lexikális hatókör (blokk) végéig tart, hanem az adott referencia **utolsó használatáig**. A kölcsönzés-ellenőrző ezt az úgynevezett liveness-alapú elemzést használja, hogy pontosabban fogadjon el biztonságos kódokat.

Szabály (kölcsönzési szabály). Egy adott értékre egyszerre vagy tetszőleges számú megosztott (`&T`) referencia, vagy pontosan egy módosítható (`&mut T`) referencia lehet. A konfliktust az NLL figyelembevételével ellenőrzi a fordító: ha egy referencia már nem használt, nincs konfliktus.

Fontos különbség: kölcsönzési régió és drop-hatókör. Az NLL a *referencia* (`&T`, `&mut T`) kölcsönzési régióját az utolsó használatig rövidíti. Az olyan érték, amelynek van destruktora (`Drop`), például a `RefCell::borrow()` által visszaadott `Ref<T>` őrző, a *hatóköre végén* dobódik el, és az eldobás maga is használat: a kölcsönzési számláló csak ekkor csökken. Ezért egy már nem olvasott, de még élő `Ref` mellett a `borrow_mut()` futásidőben pánikol:

```
use std::cell::RefCell;

fn main() {
    let c = RefCell::new(vec![1]);
    let r = c.borrow();            // Ref guard: Drop-os érték
    println!("hossz: {}", r.len()); // r utolsó olvasása
    // c.borrow_mut().push(2);     // pánik: RefCell already borrowed, r még él
    println!("{}", c.try_borrow_mut().is_err()); // true
    drop(r);                       // explicit eldobás: a guard itt szűnik meg
    c.borrow_mut().push(2);        // most rendben

    // Sima referenciánál az NLL rövidít:
    let mut v = vec![1, 2, 3];
    let first = &v[0];
    println!("{first}");
```

```
    v.push(4);                    // rendben: first-et már nem használjuk
}
```

Előre megjósolható szabály: sima referencia → az utolsó használatig tart; `Drop`-os őrző → a hatókör végéig (vagy az explicit `drop`-ig). A `Cell` és a `RefCell` közti különbség ugyanebből adódik: a `Cell` megosztott `&Cell<T>`-ből sosem ad közönséges, ellenőrizetlen belső referenciát (a `get` másolatot ad, a `set` cserél; a `Cell::get_mut(&mut self)` létezik, de ahhoz kizárólagos hozzáférés kell), ezért nem kell őrző; a `RefCell` igen, ezért kell a futásidejű számláló és az őrző életciklusa.

Definíció. *Kétfázisú kölcsönzés (two-phase borrow)*: néhány, a fordító által *implicit módon* létrehozott `&mut` kölcsönzés két fázisban él. Ilyen a `&mut self` fogadójú metódushívás automatikus fogadó-kölcsönzése (autoref, például `v.push(...)`), az argumentumként átadott `&mut` implicit újrakölcsönzése és az összetett értékadó operátorok (`+=` és társai) túlterhelt változatának fogadója. A sorrend:

1. **foglalás:** a fogadó `&mut` kölcsönzése létrejön, de csak *foglalt* (reserved) állapotban; ilyenkor olvasásra még megosztható;
2. **argumentumok kiértékelése:** az argumentumok olvashatják ugyanazt az értéket (például `v.len()`);
3. **aktiválás:** a hívás pillanatában a kölcsönzés kizárólagossá válik; ekkor már egyetlen élő megosztott kölcsönzés sem lehet;
4. **hívás:** a metódus a kizárólagos referenciával fut.

Ezért fordul le a `v.push(v.len())`. Az *explicit* `&mut` kifejezésre a szabály nem vonatkozik: a `Vec::push(&mut v, v.len())` `E0502`-vel elutasul, mert a `&mut v` azonnal aktív kizárólagos kölcsönzés, mire a `v.len()` kiértékelődik.

A fordítási hibák leggyakoribb hibakódjai:

- `E0499`: cannot borrow `x` as mutable more than once at a time (többszörös `&mut`)
- `E0502`: cannot borrow `x` as mutable because it is also borrowed as immutable (`&` alatt `&mut`)
- `E0503`: cannot use `x` because it was mutably borrowed (az értéket *használjuk*, például kimásoljuk, miközben egy élő `&mut` mutat rá)

Az `E0502` mindkét irányt lefedi: `&` alatti `&mut`-ot és `&mut` alatti `&`-t is („cannot borrow `x` as immutable because it is also borrowed as mutable”). A hibakódok stabilak; a fenti üzenetszövegek a Rust 1.99.1-éi.

7.2.3 C++ és Rust összevetés

C++-ban az aliasing hiba néma UB lehet:

```
#include <vector>
#include <string>
#include <iostream>

int main() {
    std::vector<std::string> v = {"a", "b"};
    const std::string& first = v[0];
    v.push_back("c");          // reallokáció: first érvénytelen
    std::cout << first << '\n'; // UB: dangling reference
}
```

Rustban ugyanez a kísérlet fordítási hibát ad:

```rust
fn main() {
    let mut v = vec!["a".to_string(), "b".to_string()];
    let first = &v[0];
    v.push("c".to_string()); // HIBA: first aktív
    println!("{first}");
}
```

A hiba lényege (E0502): a `first` megosztott kölcsönzése aktív, mert a `println!`-ben használjuk, és a `v.push` módosítható kölcsönzést igényel. A különbség: C++ UB-t ad, Rust fordítási hibát.

7.2.4 Kidolgozott példa: iterálás közbeni módosítás

Tipikus hiba, amikor egy gyűjteményen iterálunk, és közben ugyanazt a gyűjteményt módosítani próbáljuk.

Ez a változat szándékosan nem fordul le:

```rust
fn main() {
    let mut v = vec![String::from("a"), String::from("b")];
    for s in &v {
        if s.is_empty() {
            v.push(String::from("x")); // HIBA: &v aktív
        }
    }
}
```

A hiba lényege `text` blokkban:

```text
error[E0502]: cannot borrow `v` as mutable because it is also borrowed as immutable
  --> src/main.rs:5:13
   |
4  |     for s in &v {
   |              -- immutable borrow occurs here
5  |         v.push(String::from("x"));
   |             ^^^^^^^^^^^^^^^^^^^^ mutable borrow occurs here
6  |     }
   |     - immutable borrow ends here
```

A `for s in &v` létrehoz egy megosztott kölcsönzést, amely a ciklus teljes idejére aktív, mert a `s`-t a ciklusmagban használjuk. A `v.push` viszont módosítható kölcsönzést igényel ugyanarra a `v`-re — ez ütközés.

7.2.4.1 Javítási minta: index használata

A legegyszerűbb javítás a ciklus iterátorának lecserélése indexre, így a megosztott kölcsönzés csak az aktuális elem eléréséig tart:

```rust
fn main() {
    let mut v = vec![String::from("a"), String::from("b")];
    for i in 0..v.len() {
        if v[i].is_empty() {
            v.push(String::from("x"));
        }
    }
    println!("{v:?}");
}
```

Itt a `v[i]` megosztott kölcsönzése az `if` feltétel kiértékelése után azonnal véget ér, így a `v.push` már nem ütközik. Fontos azonban, hogy az indexeléses megoldás nem mindig ideális: ha a ciklus közben törölünk vagy beszúrunk, az indexek elcsúszhatnak. Ilyenkor érdemes külön gyűjteni a műveleteket, vagy más mintát választani.

7.2.4.2 Javítási minta: hatókör szűkítése

Ha csak egy rövid részben van szükség a kölcsönzésre, zárjuk blokkba, hogy a kölcsönzés véget érjen:

```
fn main() {
    let mut v = vec![String::from("a"), String::from("b")];
    {
        let first = &v[0];
        println!("{first}");
    } // first kölcsönzése véget ér
    v.push(String::from("x"));
}
```

Az NLL ismeretében azonban a blokk nem mindig szükséges: ha a `first`-et nem használjuk a `push` után, a fenti kód explicit blokk nélkül is lefordul, mert a kölcsönzés az utolsó használatnál véget ér.

```
fn main() {
    let mut v = vec![String::from("a"), String::from("b")];
    let first = &v[0];
    println!("{first}");
    v.push(String::from("x"));
}
```

Ez az NLL közvetlen előnye: a régi lexikális szabály szerint ez a kód hibás lett volna.

7.2.4.3 Javítási minta: mem::take

A `std::mem::take(&mut x)` kiveszi az `x` értékét (a tulajdonjogával együtt), és a helyére a típus `Default` értékét teszi. Akkor van rá szükség, ha egy `&mut` mögött álló értékből *birtokolt* értéket kell készítenünk, mert a felhasznált API tulajdont kér, és nem akarunk klónozni. Ha csak helyben kell módosítani, a `iter_mut`/`values_mut` egyszerűbb (lásd a lenti labort); a `take` nem a kölcsönzési hibák általános feloldója.

Valós tulajdonosi igény: a `String::from_utf8` a `Vec<u8>`-t érték szerint veszi át. Az alábbi metódus a `Store` (`rkv-core/src/store.rs`) nem UTF-8 értékeit veszteséges UTF-8-ra cseréli, másolás nélkül:

Kódblokk — fájl: `rkv-core/src/store.rs` · művelet: beillesztés az `impl Store` blokkba · ellenőrzés: `cargo build -p rkv-core`

```
    /// A nem érvényes UTF-8 értékeket U+FFFD cserekarakteres változatra cseréli.
    /// Visszaadja a módosított értékek számát.
    pub fn normalize_utf8(&mut self) -> usize {
        let mut changed = 0;
        for value in self.data.values_mut() {
            // Tulajdon kell a from_utf8-hoz: kivesszük, a helyén üres Vec marad.
            let bytes = std::mem::take(value);
            *value = match String::from_utf8(bytes) {
                Ok(text) => text.into_bytes(), // érvényes: változatlanul vissza
                Err(e) => {
```

```
                    changed += 1;
                    String::from_utf8_lossy(e.as_bytes()).into_owned().into_bytes() //
                     ↪  érvénytelen bemenetnél új, cserekarakteres puffer: nem másolatmentes
                }
            };
        }
        if changed > 0 {
            self.dirty = true;
        }
        changed
    }
```

Helyreállítási szabály. A `take` után a mező átmenetileg a `Default` értéket tartalmazza; minden kilépési úton vissza kell írni a valódi értéket. Itt mindkét `match`-ág visszaír, és a kettő között nincs `?` vagy korai `return`. Ha mégis pánik következne be a két átmenet között, az érintett érték üres `Vec` maradna (a szokásos allokációs hiba nem pánik, hanem folyamatleállítás, tehát nem az a tipikus indok; a pánikbiztonság problémája egy közbeeső, pánikoló hívásra vonatkozik): a tároló érvényes, de adatot veszíthet, ezért ilyen kódban a `take` és a visszaírás közé ne kerüljön hibázó művelet.

7.2.5 Javítási minták katalógusa

Továbbvitt eredmény. Az itt összegyűjtött javítási mintákat a Élettartam-annotációk és az elision szabályai, a Vec, HashMap, BTreeMap és társaik; az Entry API és a Záróprojekt: az rkv teljes rendszerként alfejezetek használják fel.

Az alábbi katalógus a leggyakoribb borrow checker hibák feloldási mintáit foglalja össze. Mindegyik minta tudatos döntést igényel, nem a hibaüzenet „megkerülését” szolgálja. (A hatókör szűkítése a referenciák kölcsönzési régióján túl az őrző értékek eldobási pontját is érinti: `RefCell`-nél a blokkba zárás vagy az explicit `drop` rövidíti a guard életét.)

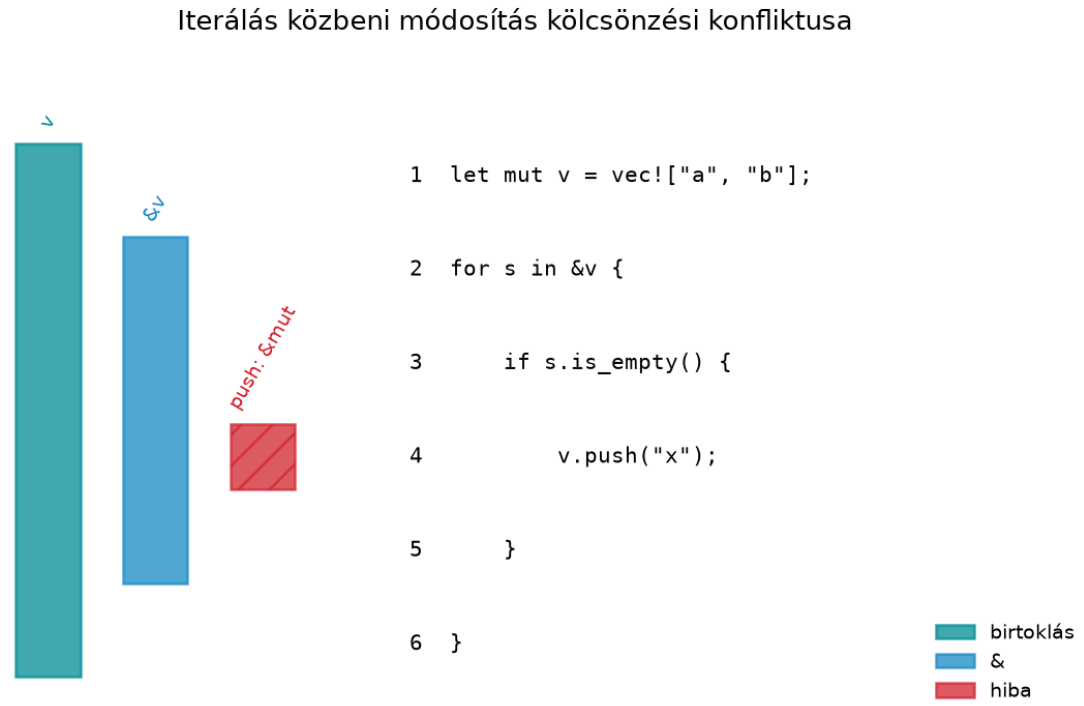
1. **Hatókör szűkítése.** Ha a kölcsönzésre csak rövid ideig van szükség, blokkba zárjuk, vagy átrendezzük a kódot, hogy az utolsó használat hamarabb következzen be. Ez a legolcsóbb, nem változtat a típusokon.
2. **Index használata.** Iterátor helyett explicit indexciklust írunk. A kölcsönzés rövid, de a gyűjtemény módosítása esetén ügyelni kell az indexek érvényességére.
3. **split_at_mut.** Ha egy mutábilis szeleten vagy tömbön két diszjunkt részre van szükségünk, a `split_at_mut` két `&mut` szeletet ad, amelyek nem fedik egymást. Példa:

```
let mut v = vec![1, 2, 3];
let (left, right) = v.split_at_mut(1);
left[0] += 10;
right[0] += 10;
```

4. **mem::take.** Egy `struct` mezőjének ideiglenes kicserélése alapértelmezett értékre. A mező tulajdonjoga lokális változóba kerül, megszüntetve a `self`-re vonatkozó kölcsönzést.
5. **Klónozás tudatosan.** Ha az adat kicsi, vagy a másolat költsége elfogadható, `clone`-ot használunk. Ez nem a borrow checker „legyőzése”, hanem explicit másolási döntés. Kerüljük a reflexszerű `clone`-ozást, mert elrejti a tervezési hibát.
6. **Belső mutabilitás (Cell, RefCell).** Futásidejű ellenőrzést végez, és lehetővé teszi a megosztott kölcsönzésen keresztüli módosítást. Csak akkor használjuk, ha a többi minta nem alkalmazható, mert a hiba futásidőbe kerül. Részletesen a Belső mutabilitás: Cell, RefCell és a futásidejű kölcsönzés alfejezet tárgyalja.

7.2.6 Ábra: kölcsönzési konfliktus idővonala

Az alábbi diagram az iterálás közbeni módosítás konfliktusát mutatja: a `&v` megosztott kölcsönzés a ciklus teljes ideje alatt aktív, ezért a `v.push` módosítható kölcsönzése hibát okoz.



7.2. ábra: Iterálás közbeni módosítás kölcsönzési konfliktusa - A &v megosztott kölcsönzés a ciklus teljes ideje alatt aktív, ezért a v.push(&mut) hívása E0502 hibát ad.

A diagram jól mutatja, hogy a `v` tulajdonos (owner) élettartama a teljes blokk, a `&v` megosztott kölcsönzés a `for` ciklus kezdetétől a ciklusmag végéig tart, a `v.push` pedig ugyanabban a tartományban igényel módosítható kölcsönzést — ez az átfedés okozza a hibát.

7.2.7 Labor: rkv frissítő műveletek borrow-hibáinak javítása

Munkakönyvtár: `~/rust-labor/rkv/`

Ebben a laborban a tényleges `rkv_core::Store`-hoz (`rkv-core/src/store.rs`, a Copy, Clone és Drop: másolás és RAII Rustban ellenőrzési pontjának állapota) adunk egy frissítő metódust, amely a megadott prefixszel kezdődő kulcsok értékeihez egy bájtot fűz, és kívülről, integrációs teszttel ellenőrizzük.

7.2.7.1 Lépések

1. Először írjuk meg a naiv változatot, amely a módosítást a meglévő `set` metódusra bízza.

Kódblokk — fájl: `rkv-core/src/store.rs` · művelet: beillesztés az `impl Store` blokkba · **szándékosan hibás** · várt hiba: `E0502`

```
    pub fn append_to_prefix(&mut self, prefix: &str, byte: u8) -> usize {
        let mut changed = 0;
        for (key, value) in self.data.iter() {
            if key.starts_with(prefix) {
                let mut new_value = value.clone();
                new_value.push(byte);
                self.set(key.clone(), new_value); // E0502
                changed += 1;
            }
        }
        changed
    }
```

A fordító elutasítja: `cannot borrow` *`self` as mutable because it is also borrowed as immutable. A `self.data.iter()` megosztott kölcsönzést tart a `self.data`-n a ciklus végéig, a `self.set` pedig az* egész* `self`-et kölcsönözné kizárólagosan. A C++-ban ugyanez a minta (beszúrás egy `std::unordered_map`-be bejárás közben) iterátor-invalidálást okozhat.

2. A javítás: a ciklus maga kérjen módosítható hozzáférést az értékekhez (`iter_mut`), és a módosítás helyben történjen. Klónozni sem kell.

Kódblokk — fájl: `rkv-core/src/store.rs` · művelet: az 1. lépés metódusának cseréje · ellenőrzés: `cargo build -p rkv-core`

```
    /// A `prefix`-szel kezdődő kulcsok értékeihez hozzáfűzi a `byte`-ot.
    /// Visszaadja a módosított értékek számát.
    pub fn append_to_prefix(&mut self, prefix: &str, byte: u8) -> usize {
        let mut changed = 0;
        for (key, value) in self.data.iter_mut() {
            if key.starts_with(prefix) {
                value.push(byte);
                changed += 1;
            }
        }
        if changed > 0 {
            self.dirty = true;
        }
        changed
    }
```

Az `iter_mut` a `self.data`-ra vesz fel kizárólagos kölcsönzést, és minden elemhez `(&String, &mut Vec<u8>)` párt ad; a kulcs nem módosítható (ez a `HashMap` invariánsát védi), az érték igen. A `self.dirty = true` a ciklus *után* áll, de a ciklusban is lefordulna, mert a `dirty` és a `data` különálló mezők, és a fordító a mezőnkénti kölcsönzéseket külön követi; egy `&mut self`-et kérő metódushívás (például `self.set`) viszont ott is ütközne.

3. Hozzunk létre egy integrációs tesztet. Az integrációs teszt külön crate-ként fordul, ezért csak a publikus API-t látja: a valódi `Store`-t importálja, saját másolatot nem tartalmaz.

Kódblokk — fájl: `rkv-core/tests/update_prefix.rs` · művelet: új fájl · ellenőrzés: `cargo test -p rkv-core --test update_prefix` → 1 sikeres teszt

```
use rkv_core::Store;

#[test]
fn append_to_prefix_changes_only_matching_keys() {
    let mut store = Store::new();
    store.set("alma".into(), vec![1]);
    store.set("alfa".into(), vec![]);
    store.set("korte".into(), vec![2]);
```

```
    assert_eq!(store.append_to_prefix("al", 9), 2);
    assert_eq!(store.get("alma"), Some(&vec![1, 9]));
    assert_eq!(store.get("alfa"), Some(&vec![9]));
    assert_eq!(store.get("korte"), Some(&vec![2]));
    assert_eq!(store.append_to_prefix("nincs", 9), 0);
}
```

4. Futtassuk a tesztet:

```
cargo test -p rkv-core --test update_prefix
```

Várt kimenet (rövidítve):

```
running 1 test
test append_to_prefix_changes_only_matching_keys ... ok
```

Ellenőrzési pont. A `Store` új publikus metódusai: `append_to_prefix(&mut self, &str, u8) -> usize` és `normalize_utf8(&mut self) -> usize` (mindkettő beállítja a `dirty` jelzőt); a `cargo test --workspace` az eddigi 7 unit tesztet és az új integrációs tesztet sikeresen futtatja.

7.2.8 Tipikus hibák és átállási csapdák

Kétszeri `&mut` ugyanarra az értékre. Amikor egy változóra két módosítható referenciát próbálunk szerezni, a fordító E0499 hibát ad. C++-ban ez gyakori (pl. két mutató ugyanarra az objektumra), de Rustban tilos. A javítás: szűkítsük a kölcsönzések hatókörét, vagy használjunk `split_at_mut`-ot, ha diszjunkt részekről van szó.

Reflexszerű klónozás. A kezdő Rust-fejlesztők gyakran `clone`-t hívnak minden borrow hibánál. Ez a kódot lassítja és elrejti a tervezési problémát. Mindig kérdezzük meg: tényleg szükség van a másolatra, vagy a kölcsönzés átrendezhető?

`Rc<RefCell<T>>` túlhasználat. A belső mutabilitás csábító megoldás, de elkerüli a fordítási idejű ellenőrzést, és futásidejű panikhez vezethet. Csak akkor használjuk, ha a statikus minták valóban nem alkalmazhatók.

Öröklés-szimuláció Deref-fel. A `Deref` trait automatikus konverziót tesz lehetővé, de nem öröklés. Ha egy típus `Deref`-et implementál, az félrevezető lehet, és nem helyettesíti a trait-eket.

Zár tartása `.await`-en át. Aszinkron kódban a `std::sync::MutexGuard` (és az `RwLock` guardjai) tartása `.await`-en keresztül a `Send` határok megsértéséhez vezet, mert ezek a guardok nem `Send`-ek; a tokio guardja `Send`, de hosszú zárat és holtpont-kockázatot jelent. Ezt a Send és Sync és az async alfejezetek tárgyalják.

7.2.9 Feladatok

7.2.9.1 (a) Fogalmi kérdés: lefordul-e és miért?

Az alábbi kód lefordul-e? Indokold az NLL szabályai alapján.

```
fn main() {
    let mut x = 10;
    let r = &mut x;
    *r += 1;
    println!("{x}");
}
```

7.2.9.2 (b) C++ kód átírása idiomatikus Rustra

Az alábbi C++ kód egy vektorban megkeresi az első üres stringet, majd hozzáfűz egy új elemet. A referencia érvénytelenné válhat a `push_back` miatt. Írd át biztonságos, idiomatikus Rustra.

```cpp
#include <vector>
#include <string>
#include <iostream>

int main() {
    std::vector<std::string> v = {"a", "", "b"};
    const std::string* first_empty = nullptr;
    for (const auto& s : v) {
        if (s.empty()) {
            first_empty = &s;
            break;
        }
    }
    if (first_empty) {
        v.push_back("x"); // first_empty érvénytelen lehet
        std::cout << *first_empty << '\n'; // UB
    }
}
```

7.2.9.3 (c) Implementációs feladat az rkv-ban

Valósítsd meg az `rkv-core` `Store` típusához (`rkv-core/src/store.rs`) a következő metódust a `mem::take` minta használatával:

```rust
pub fn remove_keys_with_prefix(&mut self, prefix: &str) -> Vec<String> {
    // Távolítsd el azokat a kulcsokat, amelyek a prefix-szel kezdődnek,
    // és add vissza a törölt kulcsok listáját.
}
```

A metódus nem használhat `clone`-t a kulcson; a törölt kulcsokat birtokolt `String`-ként kell visszaadnia. Itt a `take` valódi tulajdonosi igényt elégít ki: a kulcsokat a térképből *kivenni* kell, nem kölcsönözni.

7.2.9.4 (d) Hibajavítás

A következő kód fordítási hibát ad. Javítsd a `split_at_mut` minta használatával.

```rust
fn main() {
    let mut data = [1, 2, 3, 4];
    let left = &mut data[..2];
    let right = &mut data[2..];
    left[0] += 10;
    right[0] += 20;
}
```

Mi a hiba pontos oka, és hogyan oldja fel a `split_at_mut`?

7.2.10 Megoldások

7.2.10.1 (a)

Igen, lefordul. Az NLL szabály szerint a `r` módosítható kölcsönzésének élettartama az utolsó használatáig tart, ami a `*r += 1;` sor vége. A `println!("{x}")` már nem használja a `r`-t, így a kölcsönzés megszűnt, és a `x` megosztott kölcsönzése a `println!`-ben megengedett. Régi lexikális szabály szerint ez a kód hibás lett volna, mert a `r` a blokk végéig élt.

7.2.10.2 (b)

Idiomatikus Rust megoldás: először keressük meg az indexet, majd módosítsunk. Az indexelés elkerüli a hosszú kölcsönzést.

```
fn main() {
    let mut v = vec!["a".to_string(), String::new(), "b".to_string()];
    let first_empty = v.iter().position(|s| s.is_empty());
    if let Some(i) = first_empty {
        v.push("x".to_string());
        println!("{}", v[i]); // itt már biztonságos, mert nincs aktív kölcsönzés
    }
}
```

A `position` megosztott kölcsönzést használ, de az eredmény index, nem referencia, így a vektor módosítása után is érvényes.

7.2.10.3 (c)

```
impl Store {
    pub fn remove_keys_with_prefix(&mut self, prefix: &str) -> Vec<String> {
        let data = std::mem::take(&mut self.data);
        let mut removed = Vec::new();
        for (key, value) in data {
            if key.starts_with(prefix) {
                removed.push(key); // a kulcs tulajdona kerül a listába
            } else {
                self.data.insert(key, value); // visszaírás
            }
        }
        if !removed.is_empty() {
            self.dirty = true;
        }
        removed
    }
}
```

A `mem::take` után a `self.data` üres, a régi térkép a lokális `data`-ba kerül; ennek `into_iter`-e birtokolt `(String, Vec<u8>)` párokat ad, így a kulcsok klónozás nélkül kerülnek a `removed` listába. A ciklusban a `self.data.insert` nem ütközik semmivel, mert a lokális `data` már nem a `self` része. Helyreállítási szabály: minden meg nem tartott pár visszakerül, és a ciklusban nincs hibázó művelet. (A Rust 1.88 óta a `HashMap::extract_if` ugyanezt `take` nélkül is megoldja.)

7.2.10.4 (d)

A hiba: a `data[..2]` és `data[2..]` szeletek kölcsönzése két külön `&mut` referencia a `data`-ra, ami E0499 hibát ad, mert a fordító nem tudja statikusan bizonyítani, hogy a két tartomány diszjunkt. A `split_at_mut` explicit deklarálja a felosztást:

```
fn main() {
    let mut data = [1, 2, 3, 4];
    let (left, right) = data.split_at_mut(2);
    left[0] += 10;
    right[0] += 20;
}
```

A `split_at_mut` garantálja, hogy a két visszaadott `&mut` szelet nem fedi egymást, mert a felosztási pontot a hívó adja meg.

7.2.11 Rövid összefoglaló

A borrow checker NLL (non-lexical lifetimes) kiterjesztése óta a kölcsönzések az utolsó használatig tartanak, nem a blokk végéig. Ez pontosabbá teszi a hibadetektálást, és elfogad olyan biztonságos kódokat, amelyeket a régi lexikális szabály elutasított. A tipikus hibák (E0499, E0502, E0503) mögött általában aliasing-konfliktus áll; a javítási minták katalógusa: hatókör szűkítése, index használata, `split_at_mut`, `mem::take`, tudatos klónozás és végső esetben belső mutabilitás. A laborban a valódi `rkv_core::Store` kapott egy `append_to_prefix` metódust: a `self.data.iter()` + `self.set` naiv változat `E0502`-t ad, a helyes megoldás az `iter_mut`; a `mem::take` mintát (`normalize_utf8`, `remove_keys_with_prefix`) ott használjuk, ahol tényleg birtokolt érték kell, és mindig visszaírjuk a kivett értéket. Ezt a mintakatalógust a Élettartam-annotációk, a HashMap Entry API és a záróprojekt is használja.

7.2.12 Források és ellenőrzési hiányok

A Rust reference NLL-re vonatkozó fejezete és a The Rust Programming Language (Könyv) 4.2 és 10.3 alfejezetei a kiindulópontok. A hibaüzenetek pontos szövege verziófüggő; a hibakódok stabilak. A `std::mem::take` és a `split_at_mut` API-ja a stabil Rust része, de a pontos szignatúrák ellenőrizhetők a docs.rs-en. Ez a szöveg nem támaszkodik webes keresésre, a nyelvi szabályok a szerző stabil ismeretein alapulnak.

8 Élettartamok

8.1 Élettartam-annotációk és az elision szabályai

A C++-ban a `std::string_view` és a `std::span` rendkívül kényelmes: nulla költségű „nézet” egy meglévő pufferre. A modern C++-ban szinte mindenhol ezeket használjuk a `const std::string&` helyett, mert nem igényelnek másolást, és egységes felületet adnak a különböző konténerekre. A kényelemnek azonban ára van: a fordító nem ellenőrzi, hogy a nézet által hivatkozott adat tovább él-e. Lógó `string_view`-t létrehozni triviális, és a hiba csak futásidőben, gyakran rejtélyes memóriasérülésként jelentkezik. A Rust válasza erre a **élettartam-annotáció** (lifetime annotation): a függvény szignatúrája explicit szerződést tartalmaz arról, hogy a bemeneti referenciák és a kimeneti referencia között milyen érvényességi kapcsolat áll fenn. Ez a szerződés fordításkor ellenőrizhető, így a lógó referencia pontos típusú hibája már a fordítás során kiderül.

8.1.1 Fogalmak és szabályok

Definíció. *Élettartam-paraméternek* (lifetime parameter) nevezzük a függvény szignatúrájában szereplő `'a` alakú azonosítót, amely a referenciák érvényességi régiói közötti kapcsolatot nevezi meg. Az élettartam-paraméter **nem hoz létre** élettartamot, **nem hosszabbítja meg** egyetlen referencia érvényességét sem, és nincs futásidejű reprezentációja; kizárólag a fordító számára szolgáló statikus kapcsolatot fejez ki.

Szabály. Az élettartam-annotáció **régiókapcsolatot** ír le a szignatúrában szereplő referenciák között: azt mondja ki, hogy a megnevezett referenciák közös `'a` régióhoz kötöttek. Ebből az következik, hogy a kimeneti `&'a` referencia legfeljebb addig használható, ameddig az **azonos `'a`-t viselő** bemenetek érvényesek. Ha a kimenet *csak az egyik* bemenethez kötött, a többi bemenet élettartama független marad. Példa, két külön élettartammal és csak `x`-ből visszaadva:

```
fn first<'a, 'b>(x: &'a str, _y: &'b str) -> &'a str {
    x
}

fn main() {
    let s1 = String::from("hosszú életű");
    let result;
    {
        let s2 = String::from("rövid");
        result = first(&s1, &s2); // OK: az eredmény csak s1-hez kötött
    }
    println!("{result}"); // s2 már megszűnt, de az eredmény nem függ tőle
}
```

A `longest` ehhez képest egy *specifikus* eset: mindkét bemenet és a kimenet is ugyanazt az `'a`-t viseli, ezért a hívás oldalán az `'a` a két bemenet érvényességi tartományának **közös része** (metszete), vagyis a rövidebbik élettartam lesz. Ugyanez a belső blokkos hívás a `longest`-tel `E0597` hibát ad (`s2` does not live long enough), mert az eredmény `s2`-höz is kötött.

A Rustban az élettartam-annotációk elhagyhatóak, ha a fordító egyértelműen ki tudja következtetni őket. Ezt az egyszerűsítést **lifetime elision**-nek nevezzük. A Rust Reference három szabályt rögzít:

1. Minden bemeneti referencia saját, külön élettartam-paramétert kap. Például a `&T` bemenet `&'a T`-ként, a `&mut T` bemenet `&'a mut T`-ként értelmeződik.
2. Ha **pontosan egy** bemeneti élettartam-paraméter van, akkor a kimeneti referencia-élettartam ezt az egyet kapja. Ezért írható `fn first(s: &str) -> &str` explicit annotáció nélkül.
3. Ha több bemeneti élettartam-paraméter van, de az egyik `&self` vagy `&mut self` (metódus esetén), akkor a kimeneti referencia a `self` élettartamát kapja. Ezért a `fn get(&self, key: &str) -> &str` típusú metódusoknál sem kell explicite kiírni az élettartamot.

Ha a három szabály együttesen sem ad egyértelmű eredményt – például két önálló bemeneti referencia és egy kimeneti referencia esetén –, a fordító hibát jelez, és explicit annotációt kér.

Definíció. *Élettartam-altípusosság* (lifetime subtyping): ha `'long` élettartam legalább olyan hosszú, mint `'short`, akkor `&'long T` altípusa `&'short T`-nek. Ezt úgy is olvashatjuk, hogy a hosszabb élettartamú referencia „elfér” ott, ahol rövidebbet várnak. Ez az implicit rövidítés teszi lehetővé, hogy két, eltérő élettartamú referencia közös `'a` paraméterrel kerüljön egy függvénybe: a hosszabb referencia élettartama automatikusan a rövidebbikkel közös metszetre szűkül.

8.1.2 C++ és Rust összevetés

A leggyakoribb C++-os lógó nézet:

```cpp
#include <string>
#include <string_view>

std::string_view longest(std::string_view x, std::string_view y) {
    return x.size() > y.size() ? x : y;
}

int main() {
    std::string s1 = "alma";
    std::string_view result;
    {
        std::string s2 = "körte";
        result = longest(s1, s2);  // s2 megsemmisül a blokk végén
    }
    // result itt lógó nézet; a használata nem definiált viselkedés
    return 0;
}
```

A C++-fordító alapértelmezés szerint nem diagnosztizálja ezt; a `string_view` nem tud a hivatkozott adat tulajdonosáról, és a nyelvi szerződés nem ír elő ellenőrzést. Opcionálisan figyelmeztethet rá a GCC `-Wdangling-reference` kapcsolója (GCC 13-tól), a clang-tidy `bugprone-dangling-handle` ellenőrzése vagy az MSVC `/analyze` módja, és a futásidejű AddressSanitizer a stack-használat-utáni hibákat is képes jelezni; ezek azonban nem teljesek, nem kötelezőek, és a program UB-je figyelmeztetés nélkül is fennáll. A Rust megfelelő:

```rust
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
}

fn main() {
    let s1 = String::from("alma");
```

```
    let result;
    {
        let s2 = String::from("körte");
        result = longest(&s1, &s2); // fordítási hiba: s2 nem él elég sokáig
    }
    println!("{}", result);
}
```

A Rust-fordító a hívó oldalán ellenőrzi, hogy a közös `'a` élettartam lefedi-e a `result` használatát. Mivel `s2` a belső blokk végén megsemmisül, a közös élettartam nem terjed ki a `println!` soráig, ezért a program nem fordul le. A hibaüzenet lényege (verziófüggő szöveggel): a `s2` változó nem él elég sokáig.

8.1.3 Kidolgozott példa: a `longest` függvény élettartam-annotációja

Vegyük a `longest` függvényt annotáció nélkül:

```
// Ez a változat szándékosan nem fordul le:
fn longest(x: &str, y: &str) -> &str {
    if x.len() > y.len() { x } else { y }
}
```

A hibakód az E0106, a hibaüzenet lényege: a visszatérési típus kölcsönzött értéket tartalmaz, de a szignatúra nem mondja meg, hogy a referencia `x`-ből vagy `y`-ból származik-e. Pontosan ez az a helyzet, ahol az elision három szabálya nem alkalmazható: két bemeneti élettartam-paraméter van, és nincs `&self`.

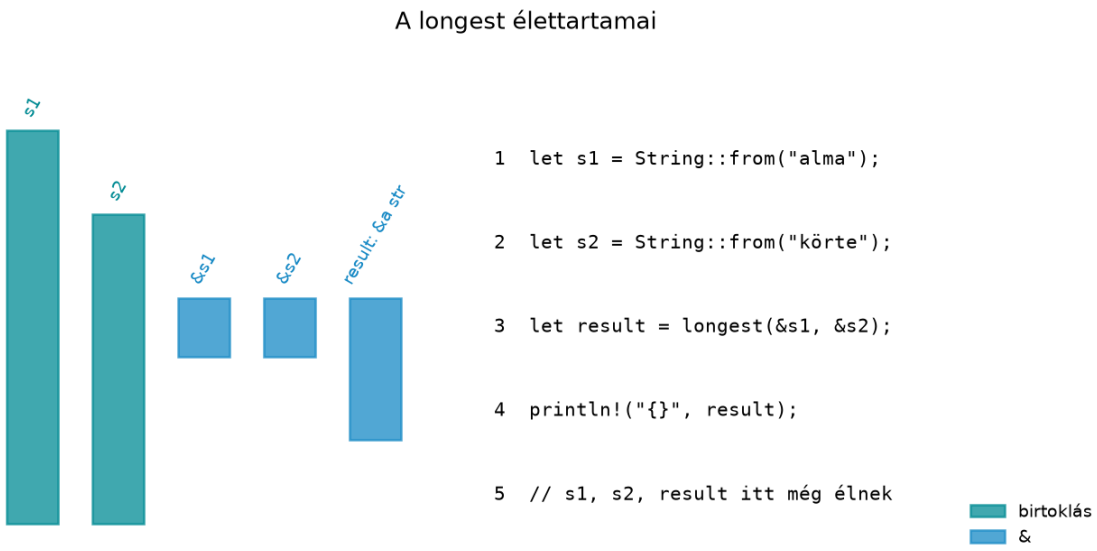
A javítás egyetlen `'a` paraméter bevezetése:

```
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
}

fn main() {
    let s1 = String::from("alma");
    let s2 = String::from("körte");
    let result = longest(&s1, &s2);
    println!("A hosszabb: {}", result);
}
```

A szignatúra kimondja: a visszaadott `&'a str` legfeljebb addig használható, ameddig mindkét bemeneti `&'a str` érvényes (a közös `'a` régió a kettő metszete). A hívó oldalán a borrow checker kiszámolja, hogy `s1` és `s2` közös élettartama a `main` függvény végéig tart, így a `result` használata biztonságos.

Az alábbi diagram a hívó oldal élettartamait mutatja:



8.1. ábra: A longest élettartamai - A hívó mindkét &str-je legalább a blokk végéig él, ezért a visszaadott referencia érvényes a println! sorában.

A diagramon a `result` sávja a `longest` hívásától a használatig tart, és mindkét bemeneti referencia sávjával közös metszetben van. Amint az egyik bemeneti élettartam megszűnik, a közös metszet is véget ér; a korábbi, hibás kódban a `s2` belső blokkja miatt ez a metszet a `println!` előtt lezárult.

8.1.4 Az elision szabályai a gyakorlatban

Az elision nem nyelvtani kivétel, hanem a szignatúrából egyértelműen következő írásforma. Tekintsük a következő eseteket:

```
// 2. szabály: egyetlen bemeneti élettartam.
fn first_word(s: &str) -> &str {
    s.split_whitespace().next().unwrap_or(s)
}

// 3. szabály: &self meghatározza a kimenetet.
struct Parser<'a> {
    text: &'a str,
}

impl<'a> Parser<'a> {
    // Explicit 'a: az eredmény a tárolt szöveghez kötött.
    fn text(&self) -> &'a str {
        self.text
    }
    // Elidált változat: az eredmény a &self kölcsönzéséhez kötött.
    fn text_elided(&self) -> &str {
        self.text
    }
}
```

A két metódus törzse azonos, a szignatúrájuk mégis **nem egyenértékű**. A harmadik elision-szabály szerint a `text_elided` eredménye a `&self` kölcsönzésének élettartamát kapja, tehát csak addig használható, amíg maga a `Parser` (és annak kölcsönzése) él. A `text` explicit `&'a str`

eredménye viszont a `Parser`-ben tárolt szövegre mutat, ezért a `Parser` eldobása után is érvényes marad. A különbséget az alábbi függvény mutatja, amely egy ideiglenes `Parser`-t hoz létre:

```
fn first_text<'a>(input: &'a str) -> &'a str {
    let p = Parser { text: input };
    p.text() // OK: &'a str, a p eldobása után is él
}

fn first_text_elided<'a>(input: &'a str) -> &'a str {
    let p = Parser { text: input };
    p.text_elided() // E0515: cannot return value referencing local variable `p`
}
```

Az első lefordul, a második `E0515`-tel elutasul, mert az elidált eredmény a lokális `p` kölcsönzéséhez kötött. Az explicit `'a` tehát nem felesleges: több kódot enged meg, mert a kimenetet a tárolt adathoz, nem a tároló objektumhoz köti.

Továbbvitt eredmény. Az itt megismert élettartam-annotációkat és elision-szabályokat alkalmazzuk a referenciát tartalmazó struktúrák tervezésekor a Referenciát tartalmazó struktúrák, 'static és tervezési döntések alfejezetben, a `String` és `&str` közötti konverziók és élettartamok elemzésénél a Slice-ok, String és &str, UTF-8 alfejezetben, valamint saját iterátorok `next()` metódusának szignatúrájánál a Closure-ök és iterátorok: lusta adapterek és saját iterátor alfejezetben.

8.1.5 Élettartam-altípusosság szemléletesen

Tegyük fel, hogy egy függvény `&'short str`-t vár, és egy hosszabb élettartamú `&'long str`-t adunk át. A hosszabb referencia automatikusan „rövidül”, mert a hívott függvény csak a rövidebb intervallumig használhatja. Ezért van az, hogy ugyanaz a `longest` függvény hívható különböző élettartamú bemenetekkel: mindkettőt a közös `'a`-ra szűkíti. A hosszabb élettartamú referencia altípusa a rövidebbnek, ezért az átadás biztonságos.

A `longest` szignatúrájában az `x` és `y` ugyanazt az `'a` paramétert viseli. Ha a hívónál `x` élettartama hosszabb, akkor a fordító implicit módon a rövidebb élettartamra szűkíti. Ez nem változtat a tényleges élettartamon, csupán a statikus ellenőrzésben használt intervallumot csökkenti. A visszaadott `&'a str` ezáltal a rövidebb élettartamot örökli, tehát a függvény után sem használhatjuk tovább annál, mint ami a rövidebb bemenetre érvényes.

8.1.6 A ritka `'static` szerepe

A `'static` élettartam azt jelenti, hogy a referencia az egész program futásáig érvényes. Ilyenek például a string-literálok (`"alma"` típusa `&'static str`), illetve a kódba égetett konstansok. A `'static` nem azt jelenti, hogy az érték a stack-en vagy a heap-en van, hanem kizárólag az élettartam hosszáról szól.

```
fn static_példa() -> &'static str {
    "ez a szöveg a binárisban él"
}

fn main() {
    let s = static_példa();
    println!("{}", s);
}
```

A `'static` annotáció négy különböző helyzetet fed le, amelyeket érdemes szétválasztani:

1. **Túl szigorú API-korlát.** Ha egy függvény `x: &'static str` paramétert kér, csak a program teljes futása alatt élő adat adható át; egy lokális `String` kölcsönzése nem. Ez korlátozza a hívót, de nem hoz létre futásidejű veszélyt.
2. **Fordítási hiba, nem futásidejű dangling.** Safe Rustban a `'static` megkövetelése azt eredményezi, hogy a fordító elutasítja a rövidebb élettartamú adatot (`E0597`); a `'static` annotáció soha nem tolja át a lógó referencia hibáját futásidőbe.
3. **Szándékos leak.** A `Box::leak(String::from("konfig").into_boxed_str())` `&'static mut str`-t ad: a referencia érvényes marad a program végéig, de a memória sosem szabadul fel. Ez memóriaszivárgás, nem lógó referencia, és csak tudatosan (például a folyamat teljes életére szóló konfigurációra) használjuk.
4. **Unsafe lifetime-hamisítás.** Az egyetlen módja a valódi lógó `'static` referenciának az `unsafe` kód (például `transmute`-tal bővített élettartam). Ez nem definiált viselkedés; a Unsafe Rust alfejezet szabályai szerint kerülendő.

Ellenőrzött viselkedés: a `longest_static("alma", "körte")` lefordul (a literálok `'static`-ok), a `longest_static(&s, "körte")` egy lokális `String`-gel viszont `E0597`-et ad (`s` does not live long enough). Ezért nem szabad minden élettartam-problémát `'static`-kal „megoldani”: a fordító így nem a hibát oldja meg, hanem a hívót korlátozza.

8.1.7 Tipikus hibák és átállási csapdák

1. Explicit annotáció elhagyása, amikor a fordító kéri. A leggyakoribb hiba, hogy a fejlesztő a C++-os szignatúra-logikát viszi tovább: `fn combine(x: &str, y: &str) -> &str`. A Rust-fordító azonnal E0106 hibával jelzi, hogy nem tudja, a kimeneti referencia melyik bemenetből származik. A megoldás a `'a` paraméter bevezetése és a kapcsolat kimondása.

2. Az élettartam meghosszabbításának téves hite. Gyakori tévhit, hogy a `'a` paraméter meghosszabbítja a reference érvényességét. Valójában nincs ilyen hatása: ha a bemeneti referencia érvényessége lejár, a kimeneti referencia is érvénytelenné válik, amely a fordító hibájaként jelenik meg. Az „élettartam meghosszabbítás” csak nagyon korlátozott, nyelv által szabályozott esetekben fordul elő (például ideiglenes értékek kötése bizonyos referencia-kontextusokban), de az annotáció ezt nem befolyásolja.

3. Túlzott `'static` használata. Amikor egy függvény visszaad egy referenciát, amely valójában csak a bemenetek élettartamához kötött, a `'static` ráírásával a kód lefordul, de a hívók csak `'static` adatot adhatnak át (a lokális `String` kölcsönzése `E0597`-et ad); ez túl szigorú API-korlát, futásidejű veszélyt nem okoz, és a dizájn memóriaszivárgás felé terelhet. Például:

```
// Ez a változat lefordul, de elrejti a tényleges kapcsolatot.
fn longest_static(x: &'static str, y: &'static str) -> &'static str {
    if x.len() > y.len() { x } else { y }
}
```

Ez a függvény csak `'static` élettartamú bemenetekkel hívható, ami a gyakorlatban szinte soha nem szükséges, és a hívókat arra kényszeríti, hogy `'static` értékeket használjanak, ahelyett hogy a kölcsönzött `String`-eket adnák át.

4. Felesleges annotáció, amikor az elision elegendő. Az `fn first<'a>(s: &'a str) -> &'a str` tökéletesen lefordul, de az elision szerint a `fn first(s: &str) -> &str` ekvivalens, és idiomatikusabb. A túlzott annotáció nem hiba, de zajossá teszi az API-t, és csökkenti az olvashatóságot. Rust kódban az a jó gyakorlat, hogy csak akkor írunk ki élettartam-annotációt, ha a fordító kéri, vagy ha a kapcsolat nem egyértelmű.

8.1.8 Feladatok

a) Lefordul-e és miért?

Vizsgáld meg a következő szignatúrákat **elision szempontjából**! A `f1`–`f3` trait-deklarációban szerepel, mert a törzs nélküli `fn` szignatúra önmagában nem érvényes Rust; a `f3` így metódus. Melyik fordul le, és melyik hibás? Indokold az elision szabályai alapján.

```
trait F1 {
    fn f1(s: &str) -> &str;
}

trait F2 {
    fn f2(x: &str, y: &str) -> &str;
}

trait F3 {
    fn f3(&self, key: &str) -> &str;
}

fn f4<'a>(x: &'a str, _y: &str) -> &'a str { x }
```

b) C++-kód átírása idiomatikus Rustra

A következő C++ függvény lógó nézetet eredményezhet. Írd át Rustra úgy, hogy a fordító garantálja az élettartam érvényességét, és magyarázd el, miért nem fordulna le a C++-logika szerinti szignatúra.

```
std::string_view first_or_second(std::string_view x, std::string_view y) {
    return x.empty() ? y : x;
}
```

c) Implementáció az rkv-ban

Diagnosztikus előretekintés: a `Storage` trait a 12.1 alfejezetben jön létre; itt a trait-metódus előre jelzett szignatúráját vizsgáljuk (még nem vezettük be az rkv-ban):

```
fn get(&self, key: &str) -> Result<Option<Vec<u8>>, Error>;
```

Magyarázd el, miért nincs szükség explicit élettartam-annotációra ebben a metódusban! Ezután írj egy segédfüggvényt az rkv-core crate-be `longest_key` néven, amely két `&str` kulcs közül a hosszabbat adja vissza, explicit annotációval. A függvényt illeszd be a `lib.rs` modulba, és írj hozzá egy egységtesztet.

d) Hibajavítás és refaktorálás

Az alábbi kód nem fordul le. Javítsd ki, majd refaktoráld idiomatikusabb formára, ha lehet:

```
fn pick<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x < y { x } else { y }
}

fn main() {
    let s1 = String::from("a");
    let result;
    {
        let s2 = String::from("b");
        result = pick(&s1, &s2);
    }
    println!("{}", result);
}
```

8.1.9 Megoldások

a)

- `f1` lefordul: egyetlen bemeneti élettartam van, az elision 2. szabálya érvényesül.
- `f2` nem fordul le: két bemeneti élettartam és kimeneti referencia; az elision nem tudja eldönteni.
- `f3` lefordul: trait-metódus, a 3. szabály a `&self` élettartamát adja a kimenetnek.
- `f4` lefordul, és összhangban van az általános szabállyal: a kimenet csak az `'a`-t viselő `x`-hez kötött, az `y` külön, független élettartamot kap (az `fn first<'a, 'b>` példa ugyanez). A közös metszet szabálya csak azokra a bemenetekre vonatkozik, amelyek a kimenettel *azonos* paramétert viselik, ahogy a `longest`-nél.

b)

C++-logika szerinti szignatúra `fn first_or_second(x: &str, y: &str) -> &str` nem fordul le E0106 hibával. A Rust-megfelelő:

```
fn first_or_second<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.is_empty() { y } else { x }
}

fn main() {
    let s1 = String::from("");
    let s2 = String::from("érték");
    let result = first_or_second(&s1, &s2);
    println!("{}", result);
}
```

A `'a` kimondja, hogy a visszaadott referencia legfeljebb addig használható, ameddig mindkét bemenet érvényes, vagyis a rövidebb élettartamú bemenet határáig (a közös `'a` régió a kettő metszete). A C++-ban a `string_view` nem hordoz ilyen kapcsolatot, ezért a fordító nem tudja ellenőrizni.

c)

A `get` metódushoz azért nem kell élettartam-annotáció, mert a visszatérési típusa (birtokolt `Result`) egyáltalán nem tartalmaz referenciát: kimeneti élettartam híján nincs mit hozzárendelni a bemenetekhez, így az elision szabályaira sincs szükség. A bemeneti referenciák (`&self`, `key: &str`) az 1. szabály szerint külön élettartamot kapnak, és csak a hívás idejére kötöttek.

A `longest_key` segédfüggvény az rkv-core `lib.rs` fájljában:

```
pub fn longest_key<'a>(a: &'a str, b: &'a str) -> &'a str {
    if a.len() >= b.len() { a } else { b }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn longest_key_returns_longer() {
        let a = "short";
        let b = "longer";
        assert_eq!(longest_key(a, b), "longer");
    }
}
```

d)

A hiba oka, hogy a `pick` a közös `'a` élettartamot használja, de a hívónál a `s2` a belső blokkban megsemmisül, ezért a `result` értékére a blokk után hivatkozni nem lehet. A javítás: a `result` használatát a belső blokkba kell hozni.

```
fn pick<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x < y { x } else { y }
}

fn main() {
    let s1 = String::from("a");
    {
        let s2 = String::from("b");
        let result = pick(&s1, &s2);
        println!("{}", result);
    }
}
```

A refaktorálás során az explicit `'a` annotáció szükséges, mert két bemeneti referencia van, és az eredmény bármelyikből származhat; az elision ilyenkor nem dönt. Ha a függvény birtokolt értéket (`String`) adna vissza, annotáció sem kellene, de akkor másolni kellene.

8.1.10 Rövid összefoglaló

Az élettartam-annotáció (`'a`) kizárólag statikus kapcsolatot ír le a bemeneti és kimeneti referenciák között; nem hosszabbítja meg az élettartamot, és nincs futásidejű reprezentációja. A `longest(x, y)` klasszikus példája két bemeneti referenciát és egy kimeneti referenciát köt össze, ezért explicit `'a` paramétert igényel, mert az elision szabályai nem tudják egyértelműen meghatározni. Az elision három szabálya: (1) minden bemeneti referencia külön élettartamot kap, (2) egyetlen bemeneti élettartam esetén az output ezt örökli, (3) `&self` vagy `&mut self` esetén az output a `self` élettartamát kapja. A `'static` kizárólag a program teljes élettartamát jelöli, és nem szabad általános megoldásként használni. Az élettartam-altípusosság szemléletesen azt jelenti, hogy a hosszabb élettartamú referencia ott is használható, ahol rövidebbet várnak, így a `longest` hívásánál a hosszabb bemenet automatikusan a rövidebb metszetére szűkül. **Továbbvitt eredmény:** az itt megismert annotációk és elision-szabályok alapjait használjuk a referenciát tartalmazó struktúrák (`r017`), a `String`/`&str` konverziók (`r018`) és a saját iterátorok (`r029`) tervezésekor.

8.1.11 Források és ellenőrzési hiányok

A Rust élettartam- és elision-szabályai stabil nyelvi elemek, a Rust Reference „Lifetime elision” szakaszában és a Rust Book generikus élettartamokról szóló fejezetében találhatók. A hibaüzenetek pontos szövege Rust-verziónként változhat, ezért a példákban a hibakódot (E0106) és a hiba lényegét adtuk meg. A `std::string_view` lógó referencia-viselkedése C++17-ben érvényes; a példa az undefined behavior illusztrálására szolgál, nem megengedett kód. A crate-verziókra és a docs.rs API-ellenőrzésre vonatkozó megjegyzések az rkv-projekt kontextusában a workspace `Cargo.toml` fájljában találhatók.

8.2 Referenciát tartalmazó struktúrák, 'static és tervezési döntések

8.2.1 Motiváció

C++-ban gyakori, hogy egy osztály referenciát vagy nézetet tárol, például `std::string_view` taggal, hogy elkerülje a felesleges másolást. Ez a minta viszont a programozóra bízza az élettartam-kezelést: ha a hivatkozott `std::string` hamarabb felszabadul, a nézet érvénytelenné válik, és a hiba csak futásidőben, gyakran nehezen reprodukálható módon jelentkezik. A Rust ebben a helyzetben is a fordítóra bízza az ellenőrzést: ha egy struktúra referenciát tartalmaz, a struktúra típusában explicit élettartam-paraméterrel kell rögzíteni a kapcsolatot. Ebben az alfejezetben azt vizsgáljuk, hogyan tárolunk referenciát struktúrában, mit jelent a `'static`, és milyen tervezési döntéseket hozzunk birtokolt és kölcsönzött adatok között.

8.2.2 Fogalmak és szabályok

Definíció. *Élettartam-paraméteres struktúra*: olyan `struct`, amely legalább egy referencia típusú mezőt tartalmaz. A `struct` deklarációjában fel kell venni egy élettartam-paramétert, és a referencia típusában használni kell:

```
struct Parser<'a> {
    input: &'a str,
}
```

Az `'a` itt nem egy konkrét, ismert élettartam, hanem egy *paraméter*, amely a `Parser` példányosításakor kötődik ahhoz az élettartamhoz, ameddig a kölcsönzött `input` érvényes. Ugyanezt az élettartamot az `impl` blokkon is fel kell tüntetni:

```
impl<'a> Parser<'a> {
    fn new(input: &'a str) -> Self {
        Self { input }
    }
}
```

Szabály. A struktúra élettartama nem haladhatja meg a benne tárolt referenciák élettartamát; a borrow checker ezt a struktúra típusában szereplő élettartam-paraméter segítségével ellenőrzi. Ha a struktúrát olyan helyen hozod létre, ahol a kölcsönzött adat már nem él elég sokáig, a kód nem fordul le.

Definíció. A `'static` élettartam a teljes program futásidejére érvényes élettartam. Két gyakori jelentését különböztessük meg:

1. `&'static str`: olyan referencia, amely a program teljes élettartama alatt érvényes, például a forráskódba írt string-literalek (`"hello"`).
2. `T: 'static` korlát: azt jelenti, hogy a `T` típus *nem tartalmaz* nem-`'static` referenciákat, tehát minden benne tárolt referencia `'static` élettartamú, vagy a típus teljesen birtokolja az adatait. Fontos: ez **nem** azt jelenti, hogy az adott érték örökké él; egy `String` példány átadható `T: 'static` korlátú függvénynek, pedig a függvény visszatérése után felszabadul. A korlát csak a típusban lévő esetleges referenciákra vonatkozik.

A `T: 'static` korlát akkor hasznos, ha egy értéket hosszú életű háttérszálra vagy aszinkron taskra akarunk átadni; ezt a Send és Sync és a future-ök alfejezetekben fogjuk használni.

Önhivatkozó struktúrák. Előfordulhat, hogy egy struktúra egyik mezője a másik mezőjére mutatna. Maga a típusdeklaráció szabályos, lefordul:

```rust
struct SelfRef<'a> {
    data: String,
    view: &'a str,
}
```

A gond az *értékek* létrehozásánál jelentkezik. A `view`-nak a `data`-ra kellene mutatnia, de a `data`-t a struktúrába mozgatjuk, a referenciát pedig a mozgatás előtt kellene felvenni:

Kódblokk — önálló példa · **szándékosan hibás** · várt hiba: `E0505` és `E0515`

```rust
struct SelfRef<'a> {
    data: String,
    view: &'a str,
}

fn make<'a>() -> SelfRef<'a> {
    let data = String::from("GET kulcs");
    let view = &data[..3];
    SelfRef { data, view } // E0505: a data-t mozgatnánk, miközben a view kölcsönzi
                           // E0515: a visszaadott érték lokális változóra hivatkozik
}
```

Három különálló probléma van itt, amelyeket érdemes szétválasztani. (1) **Ownership és aliasing:** a `view` a `data` kölcsönzése, ezért a `data` a kölcsönzés alatt nem mozgatható a struktúrába, és a borrow checker a struktúra élettartam-paraméterét (`'a`) semmilyen lokális változóhoz nem tudja kötni, amely a struktúrát túlélné. (2) **Inline mező címe:** ha egy mező a struktúra *saját* memóriájára mutatna (például egy `[u8; 16]` tömbre), a struktúra minden mozgatásakor az a cím megváltozna, és a mutató lógó lenne. (3) **Kupacpuffer címe:** a `String` adatai a kupacon vannak, és a `String` mozgatásakor ez a cím *nem* változik; a gyakorlatban egy ilyen nézet a mozgatást túlélné, de a fordító ezt nem tudja bizonyítani, és a `data` módosítása (például `push_str` reallokációval) a nézetet így is érvénytelenítené.

A stabil megoldás általában az, hogy a nézetet nem tároljuk, hanem mindig újra kiszámítjuk, vagy birtokolt adatot és indexet használunk:

```rust
struct Parsed {
    data: String,
    verb_end: usize, // a view helyett: a data[..verb_end] tartomány
}

impl Parsed {
    fn verb(&self) -> &str {
        &self.data[..self.verb_end]
    }
}
```

A haladó `Pin`-alapú megoldást később, az aszinkron fejezetben tárgyaljuk, itt csak a problémát említjük.

8.2.3 C++ és Rust összevetés

A következő C++ kód egy `std::string_view`-t tároló osztályt mutat:

```cpp
#include <string>
#include <string_view>

struct Parser {
    std::string_view input;
```

```
    explicit Parser(const std::string& s) : input(s) {}
};

Parser make_parser() {
    std::string local = "GET key";
    return Parser(local); // veszély: local felszabadul, input függő lesz
}
```

A Rust-megfelelő:

```
struct Parser<'a> {
    input: &'a str,
}

impl<'a> Parser<'a> {
    fn new(input: &'a str) -> Self {
        Self { input }
    }
}

// 1. HIÁNYZÓ ÉLETTARTAM: nincs bemeneti referencia, amelyhez a kimenetet kötni lehetne.
// error[E0106]: missing lifetime specifier
fn make_parser() -> Parser<'_> {
    let local = String::from("GET key");
    Parser::new(&local)
}

// 2. LOKÁLIS ADAT VISSZAKÖLCSÖNZÉSE: van bemeneti referencia (így az élettartam megvan),
// de a visszaadott érték a függvény lokális változójára hivatkozik.
// error[E0515]: cannot return value referencing local variable `local`
fn make_parser_from(_hint: &str) -> Parser<'_> {
    let local = String::from("GET key");
    Parser::new(&local)
}
```

A két hiba különböző okú, ezért külön hibakódot kap (a kimenetek Rust 1.99.1-ből): bemeneti referencia nélkül a fordító már az aláírásnál megáll (`E0106`, a javaslat: `'static` vagy tulajdonolt érték visszaadása); ha van bemenet, de a visszaadott `Parser` a lokális `local`-ra mutat, `E0515` jelzi, hogy a lokális változó a függvény végén megszűnik. Az `E0597` egy harmadik eset: a kölcsönzött érték egy *belső blokkban* ér véget, miközben a referencia a blokk után is él:

```
let p;
{
    let local = String::from("GET key");
    p = Parser { input: &local }; // error[E0597]: `local` does not live long enough
}
println!("{}", p.input);
```

A különbség: a C++-fordító nem figyelmeztet a függő nézetre, a Rust-fordító viszont már a `make_parser` definíciójakor hibát ad. A három hibakód (`E0106`, `E0515`, `E0597`) nem cserélhető fel egymással: mindegyik más hibát jelöl (hiányzó élettartam, lokális visszakölcsönzés, rövidebb hatókör), a pontos üzenetszöveg viszont verziófüggő.

8.2.4 Kidolgozott példa: parancssor-tokenizáló &str nézetekkel

Munkakönyvtár: `~/rust-labor/rkv/`

A szöveges parancsok (`GET <kulcs>`, `SET <kulcs> <érték>`, stb.) soronkénti feldolgozásának szemléltetésére szolgál egy olyan tokenizáló, amely a bemeneti sort nem birtokolja, csak nézeteket

ad vissza. Ez a labor egy `Tokenizer<'a>` struktúrát valósít meg az `rkv-core` crate-ben, amelyet később a Tokio-alapú szerver használ.

Továbbvitt eredmény. Ez a `Tokenizer<'a>` típus az `rkv-core/src/tokenizer.rs` fájlban jelenik meg, a nézet-alapú (nem birtokló) tokenizálás szemléltetésére. **Nem része a hálózati szerver feldolgozási útjának:** az rkv hálózati szerver: TCP, protokoll, kecses leállás szerver a `Command FromStr`-jét használja, mert a tokenizáló whitespace-összevonása nem felel meg a szigorú protokollnak (a `SET` értéke szóközt tartalmazhat, és az elválasztó pontosan egy szóköz).

8.2.4.1 Lépések

1. Hozzuk létre az `rkv-core/src/tokenizer.rs` fájlt:

Kódblokk — fájl: `rkv-core/src/tokenizer.rs` · művelet: új fájl · ellenőrzés: a 2. lépés után `cargo build -p rkv-core`

```rust
/// Szöveges parancssorokat tokenizáló nézet.
///
/// A `Tokenizer` nem birtokolja a bemenetet, csupán egy `&str`-re
/// mutató nézet. A `next_token` hívások során a belső `rest`
/// mezőt léptetjük a bemenetben.
#[derive(Debug)]
pub struct Tokenizer<'a> {
    /// A bemenet fennmaradó része.
    rest: &'a str,
}

impl<'a> Tokenizer<'a> {
    /// Új tokenizálót hoz létre a megadott bemeneti sorra.
    pub fn new(input: &'a str) -> Self {
        Self { rest: input }
    }

    /// Visszaadja a következő, szóközzel elválasztott tokent,
    /// vagy `None`-t, ha nincs több.
    ///
    /// A visszaadott token élettartama a bemenet (`'a`) élettartamához
    /// kötött, nem a `&mut self` kölcsönzéséhez, így a tokenizáló
    /// tovább használható a token független feldolgozása közben is.
    pub fn next_token(&mut self) -> Option<&'a str> {
        self.rest = self.rest.trim_start();
        if self.rest.is_empty() {
            return None;
        }
        let end = self
            .rest
            .find(char::is_whitespace)
            .unwrap_or(self.rest.len());
        let token = &self.rest[..end];
        self.rest = &self.rest[end..];
        Some(token)
    }
}
```

2. Adjuk hozzá a modult az `rkv-core/src/lib.rs` fájlhoz:

Kódblokk — fájl: `rkv-core/src/lib.rs` · művelet: beillesztés a többi `pub mod` sor mellé · ellenőrzés: `cargo build -p rkv-core`

```rust
pub mod tokenizer;
```

3. Írjunk egy egyszerű tesztet az `rkv-core/tests/tokenizer.rs` fájlba:

Kódblokk — fájl: `rkv-core/tests/tokenizer.rs` · művelet: új fájl · ellenőrzés: `cargo test -p rkv-core --test tokenizer` → 2 sikeres teszt

```
use rkv_core::tokenizer::Tokenizer;

#[test]
fn tokenizálja_a_parancsot() {
    let mut tok = Tokenizer::new("SET kulcs ertek");
    assert_eq!(tok.next_token(), Some("SET"));
    assert_eq!(tok.next_token(), Some("kulcs"));
    assert_eq!(tok.next_token(), Some("ertek"));
    assert_eq!(tok.next_token(), None);
}

#[test]
fn kezeli_a_szóközöket() {
    let mut tok = Tokenizer::new("   LIST   prefix  ");
    assert_eq!(tok.next_token(), Some("LIST"));
    assert_eq!(tok.next_token(), Some("prefix"));
    assert_eq!(tok.next_token(), None);
}
```

4. Futtassuk a teszteket:

```
cargo test -p rkv-core
```

Várható kimenet: mindkét teszt sikeres, mert a tokenizáló a `trim_start` és a `find(char::is_whitespace)` segítségével átugorja a vezető szóközöket, és a tokenhatárt az első szóköznél állapítja meg.

A `next_token` szignatúrájában fontos az explicit `'a` a visszatérési típusban. Ha ezt elhagynánk, az elision szabály a `&mut self` élettartamához kötné a visszaadott referenciát, ami túlságosan szigorú lenne: a token nem maradhatna használatban, miközben a tokenizálót tovább akarjuk hívni. Az explicit annotáció megőrzi a bemeneti élettartamot.

8.2.4.2 A Tokenizer élettartama ábrán

A kölcsönzés a tényleges utolsó használatig tart (NLL). Ha a `tok`-ot és a `token`-t a `drop(input)` előtt használjuk utoljára, a bemenet eldobása szabályos:

Kódblokk — önálló példa (az `rkv-core`-ra függőségként építő crate-ben) · ellenőrzés: lefordul, kimenete `Some("GET")`

```
use rkv_core::tokenizer::Tokenizer;

fn main() {
    let input = String::from("GET kulcs");
    let mut tok = Tokenizer::new(&input);
    let token = tok.next_token();
    println!("{token:?}");
    drop(input); // OK: a tok és a token utolsó használata már megtörtént
}
```

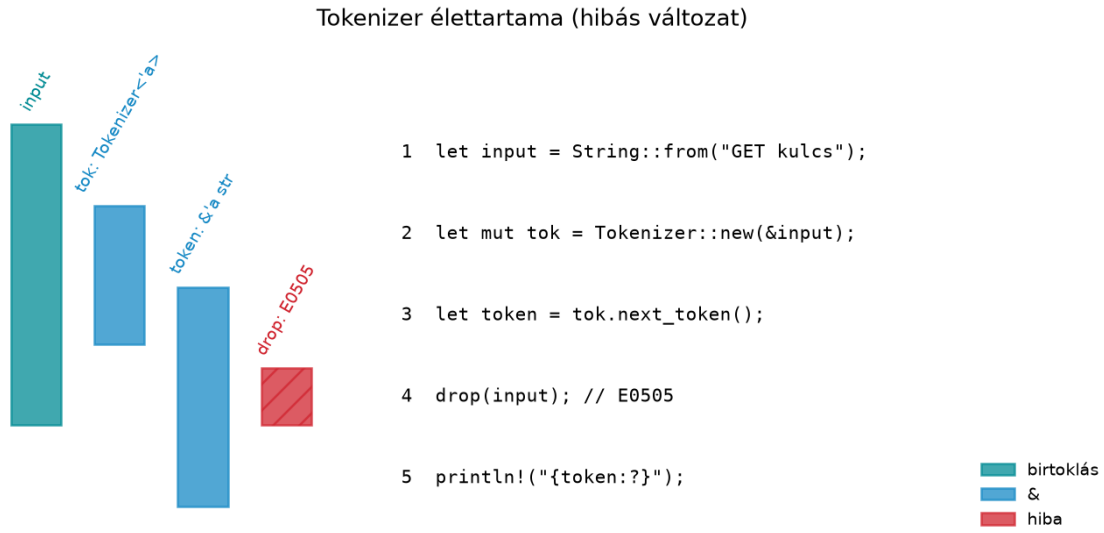
Ha viszont a tokent az eldobás *után* is használnánk, a bemenetet még kölcsönzik, amikor mozgatni (eldobni) akarjuk:

Kódblokk — önálló példa · **szándékosan hibás** · várt hiba: `E0505` (`cannot move out ofinputbecause it is borrowed`)

```rust
use rkv_core::tokenizer::Tokenizer;

fn main() {
    let input = String::from("GET kulcs");
    let mut tok = Tokenizer::new(&input);
    let token = tok.next_token();
    drop(input); // E0505: a token még él
    println!("{token:?}");
}
```

Az ábra a hibás változatot mutatja; a sávok a kölcsönzések tényleges használati tartományát jelölik. A `tok` kölcsönzése a 3. sorban véget ér (utolsó használat), a `token`-é viszont az 5. sorig tart, ezért ütközik a 4. sor `drop`-jával:



8.2. ábra: Tokenizer élettartama (hibás változat) - A kölcsönzés az utolsó használatig tart: a token az 5. sorban még kell, ezért a 4. sor drop(input) hívása E0505 hibát ad. Ha a println a drop elé kerül, a program lefordul.

A `Tokenizer` és a belőle kinyert tokenek is az `input` élettartamához kötöttek. A `drop(input)` csak akkor hibás, ha utána még használjuk a tokenizálót vagy egy tokent; a puszta létezésük (használat nélkül) nem akadályozza meg az eldobást.

8.2.5 Birtokolt vs. kölcsönzött: API-ergonómia és teljesítmény

Amikor struktúrában adatot tárolunk, alapvető tervezési döntés a birtokolt és a kölcsönzött forma közötti választás. A két lehetőség:

```rust
// Birtokolt: a struktúra a saját adatát tárolja.
struct OwnedConfig {
    name: String,
}

// Kölcsönzött: a struktúra csak nézet a máshol tárolt adatra.
struct BorrowedConfig<'a> {
    name: &'a str,
}
```

Birtokolt (`String`) előnyei: - A struktúra önállóan, a bemenettől függetlenül élhet; könnyű visszaadni, eltárolni gyűjteményben, átadni más szálakra. - Nincs élettartam-paraméter a típusban, így az API egyszerűbb.

Hátránya: az adat *tulajdonolt másolata* kell hozzá. A másolás költsége azonban nem szükségszerű, hanem a tényleges átalakítástól függ: ha a hívónak már van `String`-je, átadhatja a tulajdonjogot (`String` paraméter, `impl Into<String>`), másolat nélkül; ha csak `&str`-je van, a másolat az az ár, amelyet a bemenettől független tulajdonért fizetünk.

Kölcsönzött (`&'a str`) előnyei: - Nincs másolás, gyors, különösen nagy bemeneteknél vagy gyakori hívásoknál. - A hívó ellenőrzését a borrow checker garantálja, így nincs dangling pointer.

Hátránya: a struktúra élettartama a bemenethez kötött. A kölcsönzött adat eltárolása *nem tilos*, csak az eltárolt érték élettartama kötött: a `Parser<'a>` addig él, ameddig a bemenet, tehát visszaadható és eltárolható egy olyan hatókörben, amelynél a bemenet hosszabb ideig él. A hosszú életű, bemenettől független tulajdonhoz (például szerver-kapcsolati állapothoz) birtokolt adat kell.

Az API-irány konzisztens az API-tervezés Rustban paramétertípus-szabályával: a konstruktor `&str`-t fogad, ha a típus nézet (`Parser::new(input: &'a str)`), és tulajdonolt értéket (`String` vagy `impl Into<String>`), ha a típus önálló tulajdonos (`OwnedParser::new(input: impl Into<String>)`):

```
struct Parser<'a> { input: &'a str }
impl<'a> Parser<'a> {
    fn new(input: &'a str) -> Self { Self { input } }   // nézet: nincs másolat
}

struct OwnedParser { input: String }
impl OwnedParser {
    fn new(input: impl Into<String>) -> Self {          // tulajdon: a hívó dönt az átadásról
        Self { input: input.into() }
    }
}
```

A gyakorlati irányelv: **ha a struktúra rövid életű nézet, akkor kölcsönzött; ha hosszú életű, önálló érték, akkor birtokolt.** Parser, tokenizáló, konfiguráció-validátor esetén a kölcsönzött forma idiomatikus. Egy szerver kapcsolati állapotát viszont általában birtokolt adattal érdemes tárolni, mert a kapcsolat hosszan él.

8.2.6 Tipikus hibák és átállási csapdák

1. **Hiányzó élettartam-paraméter a struktúrán.** Ha a `struct Parser { input: &str }` deklarációt használod, a fordító E0106 hibát ad: „missing lifetime specifier”. A javítás: `struct Parser<'a> { input: &'a str }`.
2. **A `'static` félreértése.** A `T: 'static` korlát nem követeli meg, hogy az érték örökké éljen. Például a következő kód lefordul:

```
fn take_static<T: 'static>(_x: T) {}

let s = String::from("hello");
take_static(s); // OK, mert a String birtokol, nincs benne nem-'static referencia
```

3. **Túl szigorú élettartam a metódus visszatérési típusában.** Ha a `next_token` szignatúrája `fn next_token(&mut self) -> Option<&str>`, a visszakapott token élettartama a `&mut self` kölcsönzéséhez kötődik, nem a bemenetéhez. Ez a különbség akkor okoz gondot, ha a tokent a tokenizáló további hívása után is használni szeretnénk. A megoldás az explicit `'a` a visszatérési típusban.

4. **Klónozás mindenhol.** A borrow checker hibáit sok kezdő Rust-fejlesztő azzal kerüli meg, hogy mindenhol `String`-et klónoz. Ez felesleges másolást és gyengébb teljesítményt eredményez. Törekedj a kölcsönzött nézetek használatára, ahol a struktúra rövid életű.

8.2.7 Feladatok

8.2.7.1 (a) Lefordul-e és miért?

Vizsgáld meg a következő kódot:

```
struct View<'a> {
    s: &'a str,
}

fn create_view() -> View<'_> {
    let text = String::from("data");
    View { s: &text }
}
```

Válaszodban indokold, melyik szabály sérül, és mi a hibaüzenet lényege (E0597 vagy hasonló).

8.2.7.2 (b) C++ kód átírása idiomatikus Rustra

Az alábbi C++ tokenizáló `std::string_view`-t használ:

```
#include <string_view>

struct Tokenizer {
    std::string_view rest;

    explicit Tokenizer(std::string_view input) : rest(input) {}

    std::string_view next_token() {
        while (!rest.empty() && rest.front() == ' ') rest.remove_prefix(1);
        if (rest.empty()) return {};
        size_t end = 0;
        while (end < rest.size() && rest[end] != ' ') ++end;
        auto token = rest.substr(0, end);
        rest.remove_prefix(end);
        return token;
    }
};
```

Írd át ezt a Rust `Tokenizer<'a>`-ra, és magyarázd el, miért biztonságosabb a Rust-verzió élettartam-kezelése.

8.2.7.3 (c) Implementációs feladat az rkv-ban

Valósítsd meg a laborban bemutatott `Tokenizer<'a>`-t az `rkv-core` crate-ben. Egészítsd ki egy `next_token` metódussal, amely a szóközöket és tabulátorokat is elválasztóként kezeli, és írj hozzá unit teszteket: üres bemenet, csak szóközök, többszörös szóközök, tabulátorok, valamint egy `GET kulcs` parancs tokenizálása. Ellenőrizd a `cargo test -p rkv-core` paranccsal.

8.2.7.4 (d) Tervezési döntés: birtokolt vagy kölcsönzött puffer?

Az rkv-szerver egy kapcsolatánál a bejövő sorokat egy pufferben dolgozzuk fel. Tegyük fel, hogy egy `Connection` struktúrát tervezel:

- Ha a puffer a kapcsolat teljes élettartama alatt egyetlen `String`, akkor a parancs-tokenizáló nézeteket használhat.
- Ha viszont a `Connection`-t gyakran mozgatják, vagy a puffert más struktúrákba kell átadni, milyen érvek szólnak a birtokolt `String` mellett?

Indokold a választásodat a birtokolt vs. kölcsönzött szempontok alapján, és írd le, milyen élettartam-következményei lennének a kölcsönzött megoldásnak.

8.2.8 Megoldások

8.2.8.1 (a)

A kód nem fordul le. A `create_view` függvény egy lokális `String`-re mutató referenciát próbál visszaadni a `View`-ban. A `text` élettartama a függvény blokkjára korlátozódik, míg a visszatérési `View<'_>` élettartama a hívó által elvárt tetszőleges élettartam. A borrow checker E0597 hibát ad: a kölcsönzött érték nem él elég sokáig. A javítás az lenne, ha a `View` birtokolná az adatot (`String` mező), vagy a függvény paraméterként kapná a bemenetet.

8.2.8.2 (b)

A Rust-megfelelő a laborban bemutatott `Tokenizer<'a>`. A fő különbség: a C++-osztály nem rögzíti az élettartamot a típusban, és a hívó felelőssége, hogy a `std::string_view` mögötti adat éljen. A Rust-struktúra az `<'a>` paraméterrel kikényszeríti, hogy a `Tokenizer` ne élje túl a bemenetet, és a `next_token` visszatérési típusában explicit `'a` jelzi, hogy a token is ugyanahhoz az élettartamhoz kötött.

8.2.8.3 (c)

A megoldás megegyezik a laborban közölt kóddal. A `trim_start` alapértelmezetten a Unicode whitespace-t is kezeli, beleértve a tabulátorokat. A tesztek a következőképpen egészíthetők ki:

```
#[test]
fn kezeli_a_tabulátorokat() {
    let mut tok = Tokenizer::new("DEL\tkulcs");
    assert_eq!(tok.next_token(), Some("DEL"));
    assert_eq!(tok.next_token(), Some("kulcs"));
    assert_eq!(tok.next_token(), None);
}
```

```rust
#[test]
fn üres_bemenet() {
    let mut tok = Tokenizer::new("");
    assert_eq!(tok.next_token(), None);
}
```

8.2.8.4 (d)

A `Connection` puffert célszerű birtokolt `String`-ként tárolni, mert a kapcsolat hosszú életű, és a puffert többször is módosítani, kiüríteni, valamint akár más struktúráknak átadni szükséges. A kölcsönzött `&str` tárolása azt jelentené, hogy a `Connection` élettartama a bemeneti adat élettartamához kötött, ami a szerver esetében a socketből olvasott adatokra nem alkalmazható, mert azok a feldolgozás után felszabadulnak. A birtokolt puffer mellett az egyes sorokat feldolgozás előtt `Tokenizer::new(&buffer)` módon nézetként lehet tokenizálni, így a tokenizálás gyors, másolásmentes marad.

8.2.9 Rövid összefoglaló

Ebben az alfejezetben megismerkedtünk az élettartam-paraméteres struktúrákkal: ha egy `struct` referenciát tárol, a típusdeklarációban `<'a>` paramétert kell használni, és a referencia típusában jelölni kell. A `'static` két jelentését tisztáztuk: `&'static str` statikus referencia, míg `T: 'static` típuskorlát, amely a típusban lévő referenciákra vonatkozik, nem az érték élettartamára. Megvizsgáltuk az önhivatkozó struktúrák problémáját, és a birtokolt versus kölcsönzött adatok közötti tervezési döntéseket. A laborban létrehoztuk az `rkv-core/src/tokenizer.rs` fájlban a `Tokenizer<'a>` típust, amely a szöveges parancsokat `&str` nézetekkel tokenizálja, és ezt a hálózati szerver használja majd. A `T: 'static` korlátot a Send és Sync és a future-ök alfejezetekben fogjuk alkalmazni. A legfontosabb szabály: a struktúra élettartama nem haladhatja meg a benne tárolt referencia élettartamát, és a metódusok visszatérési típusánál explicit élettartam-annotációval őrizzük meg a bemeneti élettartamot.

8.2.10 Források és ellenőrzési hiányok

- Rust Reference: Lifetime elision – a pontos szabályok ellenőrzéséhez.
- The Rust Book: Validating References with Lifetimes – a `'static` és a struktúrákban tárolt referenciák hátteréhez.
- A hibaüzenetek pontos szövege (E0106, E0597, E0505) verziófüggő; a példák a lényeget mutatják, a szövegek a rögzített toolchainre (Rust 1.99.1) vonatkoznak. A `rkv-core` crate- és modulneveit a könyv korábbi alfejezetei rögzítik.

9 Slice-ok és stringek

9.1 Slice-ok, String és &str, UTF-8

9.1.1 Motiváció

A C++-ban a `std::string_view` megjelenése óta természetes, hogy string-függvények nem birtokló nézetet fogadnak, nem teljes `std::string` másolatot. A `std::string_view` azonban nem véd meg a *dangling pointertől*: ha az a `std::string`, amelyre mutat, kikerül a hatóköréből, a program viselkedése definiálatlan. Rustban a `&str` ugyanezt a koncepcionális szerepet tölti be, de a Élettartam-annotációk és az elision szabályai alfejezetben tanult lifetime-rendszerrel a fordító fordítási időben kizárja, hogy a nézet túlélje a birtokolt puffert. Ebben az alfejezetben a slice-ok, a `String`/`&str` páros és az UTF-8 kezelés C++-os megfelelőit járjuk körbe.

9.1.2 Fogalmak és szabályok

9.1.2.1 Definíciók

Definíció (slice). A `&[T]` típusú slice *fat pointer*: egy adatmutató és egy hossz. Nem tulajdonos, nem allokál; csak egy meglévő, egybefüggő `[T]` tömbre mutat. A `&mut [T]` mutálható nézet, de a hossza ekkor is rögzített.

Definíció (`&str`). A `&str` string slice: egy UTF-8 kódolású bájtsorozatra mutató fat pointer (mutató + bájthossz). Garantáltan érvényes UTF-8, és nem tulajdonos. A `&str` nem null-végű – a hosszt a fat pointer tárolja.

Definíció (`String`). A `String` birtokolt, heap-en tárolt, UTF-8 kódolású szövegpuffer. A `Vec<u8>`-hoz hasonlóan három szóból álló tulajdonos: mutató, hossz, kapacitás. A `String` tulajdonosa a puffernek, és a `Drop` implementáció felszabadítja.

9.1.2.2 Szabályok

Szabály (paramétertípus-választás). Függvényparaméterben szöveget `&str`-ként, bájtpuffert pedig `&[T]`-ként fogadjunk, ha a függvény csak olvassa az adatot, vagy ha az eltárolt értéke a bemenet élettartamához kötött marad (nézet-típus, például `Parser<'a>`). A `String` vagy `Vec<T>` paraméter arra való, ha a függvény birtokba veszi az értéket, és a bemenettől független, hosszabb életű tulajdonként tárolja vagy módosítja. A másolás költsége a tényleges átalakításhoz kötött: tulajdonjog-átadásnál (`String`) nincs másolat, `&str`-ből tulajdon készítésekor van.

Továbbvitt eredmény. Ezt a paramétertípus-választási döntést a API-tervezés Rustban: irányelvek és ergonómia alfejezetben általános API-elvként, a C ABI: extern “C”, repr(C), bindgen és cbindgen alfejezetben pedig az FFI-határokra alkalmazva visszük tovább.

Szabály (deref coercion). Ahol `&str` várt, `&String` is átadható, mert a `String` automatikusan kölcsönözhető `&str`-ként. A részletes mechanizmust (a `Deref` traitet) a A standard trait-ek:

konverziók, formázás, Default, Deref, Borrow alfejezet tárgyalja; itt annyit jegyzünk meg, hogy a paraméter típusa legyen `&str`, és a hívás `&my_string`-gel is működik.

Szabály (UTF-8 indexelés). A `String` és a `&str` nem indexelhető `[]` operátorral egyetlen számmal, mert a `s[i]` bájtot adna, nem karaktert. A `s[i..j]` szeletelés **bájtindexekkel** dolgozik, és pánikol, ha a határok nem Unicode skalárérték-határok. A „karakter” szónak három különböző egysége van:

Egység	Mit jelent	Eszköz	Példa: `é` előre komponálva (`U+00E9`)	Példa: `e` + kombináló ékezet (`U+0065 U+0301`)
bájt	UTF-8 kódolt bájt	`len()`, `bytes()`, `s[a..b]`	2	3
Unicode skalárérték	a `char` típus egy értéke	`chars()`, `char_indices()`	1	2
grapheme cluster	a felhasználó által látott karakter	külső crate (`unicode-segmentation`)	1	1

A két `é` megjelenése azonos, a bájtsorozatuk és skalárértékeik száma viszont eltér, és `==`-vel nem egyenlők. A `chars()` és a `char_indices()` **skalárértéket**, nem látható karaktert kezelnek: egy kombinált ékezetes szövegen a `chars().take(3)` három skalárértéket ad, ami akár két látható karakter is lehet. Látható karakterekhez grapheme-szegmentálás kell, amelyet a standard könyvtár nem tartalmaz.

9.1.3 C++ és Rust összevetés

C++-ban a `std::string_view` a nem birtokló nézet; a programozó felelőssége, hogy ne lógjon a levegőben:

```cpp
#include <string>
#include <string_view>
#include <iostream>

std::string_view first_word(std::string_view s) {
    auto pos = s.find(' ');
    return pos == std::string_view::npos ? s : s.substr(0, pos);
}

int main() {
    std::string text = "hello world";
    std::string_view word = first_word(text);
    std::cout << word << '\n'; // hello
    // Ha text kikerül a hatókörből, a word dangling lesz.
}
```

A Rust-megfelelő ugyanazt a nem birtokló nézetet használja, de az élettartamot a fordító ellenőrzi:

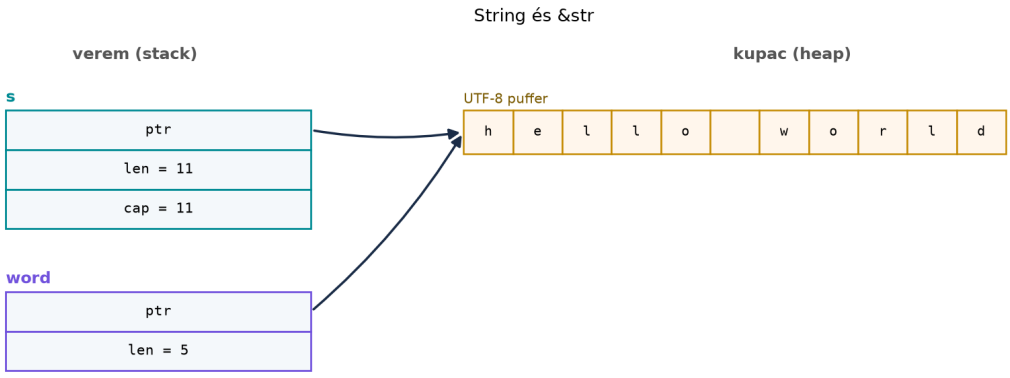
```rust
fn first_word(s: &str) -> &str {
    match s.find(' ') {
        Some(pos) => &s[..pos],
        None => s,
    }
```

```
}

fn main() {
    let text = String::from("hello world");
    let word = first_word(&text);
    println!("{word}"); // hello
    // A borrow checker garantálja, hogy word nem éli túl textet.
}
```

A különbség: a C++ kódban a `string_view` élettartama a programozón múlik; a Rust kódban a `&str` visszatérési típusához az elision szabályok alapján a bemenő `&str` élettartama kötődik, így a hívó `String` nem mozdulhat el, amíg a `word` él. A `&s[..pos]` kifejezés pedig *slice-ot* készít: ugyanarra a pufferre mutató fat pointert, nem másolatot.

A `String` és a `&str` viszonyát a memóriában az alábbi ábra szemlélteti:



9.1. ábra: String és &str - A String birtokolja a kupacpuffert, a &str csak mutató és hossz, nem tulajdonos.

A `word` ugyanarra a kupacpufferre mutat, mint a `s`, de a hossza csak 5, tehát a `"hello"` előtagot látja. A kupacpuffer adatai nem másolódnak; a `String` és a `&str` is ugyanazt a memóriát használja.

9.1.4 Kidolgozott labor: UTF-8 határok és &str

Munkakönyvtár: `~/rust-labor/r018-slices`.

Hozzuk létre az önálló kísérleti crate-et:

```
cargo new r018-slices
cd r018-slices
```

A `src/main.rs` tartalma:

```
fn first_word(s: &str) -> &str {
    match s.find(' ') {
        Some(pos) => &s[..pos],
        None => s,
    }
}

fn print_chars(s: &str) {
```

```
    println!("String: {s:?}, byte hossz: {}", s.len());
    for (byte_idx, ch) in s.char_indices() {
        println!("  byte {byte_idx}: {ch:?}");
    }
}

fn main() {
    let text = String::from("hello world");
    let word = first_word(&text);
    println!("first word: {word}");

    let s = String::from("árvíz");
    print_chars(&s);

    // Fél-nyílt bájttartományok: a vég kizárt, és karakterhatárra kell esnie.
    for range in [0..2, 0..1] {
        match s.get(range.clone()) {
            Some(sub) => println!("{range:?} mint str: {sub}"),
            None => println!("{range:?} nem karakterhatár"),
        }
    }
}
```

Futtatás:

```
cargo run
```

Várt kimenet:

```
first word: hello
String: "árvíz", byte hossz: 7
  byte 0: 'á'
  byte 2: 'r'
  byte 3: 'v'
  byte 4: 'í'
  byte 6: 'z'
0..2 mint str: á
0..1 nem karakterhatár
```

Magyarázat: az `árvíz` string hossza 7 bájt, de 5 karakter. A `char_indices()` a karakterek *kezdő bájtindexét* adja: 0, 2, 3, 4, 6, a szöveg vége pedig a 7. Egy `a..b` tartomány fél-nyílt: a `b` indexű bájt már nem tartozik bele, és mindkét végpontnak e határok (0, 2, 3, 4, 6, 7) egyikére kell esnie. A `0..2` ezért érvényes, és az `"á"` két bájtját adja; a `0..1` viszont az `"á"` közepén végződne, így a `get` `None`-t ad. A `&s[0..1]` ugyanerre pánikolna; a `get` változat biztonságos. A `char_indices()` bájt-indexeket ad vissza karakterenként, így láthatjuk, hogy a karakterek nem egyenlő szélességűek.

9.1.5 Slice-szerű nézetek a szövegen túl: OsStr, Path, CStr, Cow

A `&str` nem az egyetlen nem birtokló szövegnézet. A Rust standard könyvtára a következő típusokat kínálja, amikor a `String`/`&str` nem megfelelő:

- **`OsStr`/`OsString`**: platformfüggetlen, *nem feltétlenül UTF-8* szövegnézet. Unixon tetszőleges bájtsor; Windowson a belső reprezentáció platformfüggő implementációs részlet (a Windows API-val széles karakteres, UTF-16 átalakítással kommunikál), ezért az `OsStr` belső kódolása nem hordozható szerződés. Fájlnevekhez és környezeti változókhoz használjuk, mert a fájlrendszer nem garantál UTF-8-at.

- **`Path`/`PathBuf`**: `OsStr`-ön alapuló elérésiút-típusok. A `Path` a nem birtokló, a `PathBuf` a birtokolt változat.
- **`CStr`/`CString`**: null-végű bájtsorozat, belső nullbájt nélkül, és nem feltétlenül UTF-8. A `&CStr` kölcsönzött, a `CString` birtokolt változat. *Szerepük* a C-s `const char*`-é, de maguk nem C ABI-típusok: az `extern "C"` függvények aláírásában mindig nyers mutató (`*const c_char`, illetve `*mut c_char`) áll, a `CStr`/`CString` a Rust-oldali biztonságos kezelést adja. A kapcsolat: a `CString::new` ellenőrzi, hogy nincs belső nullbájt (különben `NulError`); az `as_ptr()` egy `*const c_char` mutatót ad a hívás idejére (a `CString`-nek addig élnie kell); a C-től kapott mutatóból az `unsafe` `CStr::from_ptr` készít kölcsönzött nézetet; a `to_str()` ellenőrzi az UTF-8-at. Ha a tulajdonjog átkerül a C-oldalra, a `CString::into_raw` adja át a puffert, és csak a `CString::from_raw` veheti vissza (a Rust allokátorával kell felszabadítani, nem C `free`-vel). A C ABI: extern “C”, repr(C), bindgen és cbindgen alfejezet ugyanezt a szerződést használja.

A C++-os párok: a `std::filesystem::path` megfelelője a `PathBuf`/`Path`, a `const char*` megfelelője (szerepében) a `&CStr` (az ABI-n `*const c_char`), a `std::string` megfelelője a `String`. A választási szempont mindig az: ha a szöveg UTF-8 és a funkció nem igényel C ABI-t, használjunk `String`-et/`&str`-t; fájlrendszer-műveletnél `Path`-ot; C-FFI-nál a határon nyers `*const c_char`-t, a Rust-oldalon `CStr`/`CString`-et.

A `std::borrow::Cow<'a, str>` a *clone-on-write* típus. Akkor hasznos, ha egy függvény néha birtokolt `String`-et, néha kölcsönzött `&str`-t akar visszaadni anélkül, hogy mindig allokálna:

Szerződés: az `expand_tabs` a bemeneti szövegben minden tabulátort négy szóközre cserél. Ha nincs tabulátor, a bemenetet változtatás nélkül, allokáció nélkül adja vissza (`Cow::Borrowed`); ha van, új bájtokat kell előállítani, ezért `Cow::Owned`-ot ad. (Egy `trim_start()` nem alkalmas példa: az is `&str` szeletet ad, új bájtok nélkül.)

```
use std::borrow::Cow;

fn expand_tabs(s: &str) -> Cow<'_, str> {
    if s.contains('\t') {
        Cow::Owned(s.replace('\t', "    "))
    } else {
        Cow::Borrowed(s)
    }
}
```

Ellenőrzött viselkedés: `expand_tabs("alma")` → `Cow::Borrowed("alma")` (nincs allokáció); `expand_tabs("a\tb")` → `Cow::Owned("a    b")`.

Ha nincs elvégzendő munka, a függvény allokáció nélkül adja vissza a bemenetet; ha új bájtokat kell előállítani, akkor allokál. Ezt a Teljesítmény: mérés, profilozás, allokáció, release-beállítások alfejezetben visszük tovább.

Továbbvitt eredmény. A `Cow<'a, str>` típus használata a Teljesítmény: mérés, profilozás, allokáció, release-beállítások alfejezetben jelenik meg újra, ahol az allokáció elkerülését mérjük és hangoljuk.

9.1.6 Tipikus hibák és átállási csapdák

9.1.6.1 1. `&String` paraméter használata `&str` helyett

```
fn bad(s: &String) {
    println!("{s}");
}
```

```rust
fn good(s: &str) {
    println!("{s}");
}

fn main() {
    let s = String::from("hello");
    bad(&s);       // működik, mert &String
    // bad("hello"); // NEM fordul: &str nem &String
    good(&s);      // működik, deref coercion
    good("hello"); // működik
}
```

A `bad` függvényt nem lehet `&str`-rel hívni; ezért a paraméter általában `&str`. Ez irányelv, nem nyelvi tiltás: ha a függvénynek `String`-specifikus API kell (például `s.capacity()`, amely `&str`-en nem létezik), az `&String` indokolt, és a `clippy::ptr_arg` sem jelzi. A cél a hívó rugalmassága, nem a típus kerülése.

9.1.6.2 2. Byte-szeletelés karakter közepén

```rust
let s = String::from("árvíz");
let sub = &s[0..1]; // pánikol: end byte index 1 is not a char boundary
```

Ez a kód lefordul, de futás közben pánikol, mert az 1. bájtindex az `"á"` karakter (0–1. bájt) közepére esik; a `&s[0..2]` viszont szabályos, és `"á"`-t ad. A pontos üzenet verziófüggő, de a lényeg mindig a karakterhatár-sértés. Javítás: használjunk `get`-et és `Option`-t, vagy karakter-alapú iterátorokat.

9.1.6.3 3. `clone()` a borrow checker „legyőzésére”

```rust
fn takes_string(s: String) {
    println!("{s}");
}

fn main() {
    let text = String::from("hello");
    takes_string(text.clone()); // felesleges allokáció
    // Helyesebb: fn takes_str(s: &str) és takes_str(&text)
}
```

A `clone()` megkerüli a borrow checkert, de felesleges allokációt és másolást végez. A helyes megoldás a kölcsönzés (`&str`), ha a függvény nem veszi birtokba az értéket.

9.1.7 Feladatok

9.1.7.1 a) Fogalmi kérdés

Melyik függvényparaméter a jobb, és miért: `fn f(s: &String)` vagy `fn f(s: &str)`? Mi a különbség hívásnál?

9.1.7.2 b) C++ kód átírása idiomatikus Rustra

Fordítsd le a következő C++ kódot idiomatikus Rustra. A `std::span<const int>` a nem birtokló nézet, amely a Rust `&[i32]` slice-nak felel meg:

```
#include <span>
#include <iostream>

void print_first(std::span<const int> xs) {
    if (!xs.empty()) {
        std::cout << xs[0] << '\n';
    }
}
```

9.1.7.3 c) Implementációs feladat az rkv előkészítésére

Írj egy `parse_command(line: &str) -> Option<(&str, &str)>` függvényt, amely a szöveges protokoll sorait bontja *parancsnévre és maradékra*. Szerződés: az elválasztó egyetlen ASCII szóköz (tabulátor nem); a `GET` pontosan egy, nem üres, szóközt nem tartalmazó argumentumot kér; a `SET` nem üres kulcsot és nem üres értéket (az érték szóközt tartalmazhat); a visszaadott maradék a `GET` esetén a kulcs, a `SET` esetén a `kulcs érték` rész; minden más bemenet (üres sor, ismeretlen parancs, hiányzó vagy többletargumentum, tabulátoros sor) `None`. A visszatérési típus `&str` legyen, ne `String`, és használj `split_once`-t. (Ez bontás, nem teljes protokoll-elemző: az értékvalidálást a Mintaillesztés: match, if let, let else és a minták parsere végzi.) (Ezt a paramétertípus-választást visszük tovább a API-tervezés Rustban: irányelvek és ergonómia és a C ABI: extern “C”, repr(C), bindgen és cbindgen alfejezetben.)

9.1.7.4 d) Hibajavítás

Az alábbi függvény pánikolhat egyes bemenetekre. Adj meg egy ilyen bemenetet, és javítsd a függvényt úgy, hogy `Option<&str>`-t adjon vissza (`Some`, ha legalább három Unicode skalárérték van, egyébként `None`), és ne pánikoljon. A név jelezze, hogy skalárértékről, nem látható karakterről van szó:

```
fn first_three_chars(s: &str) -> &str {
    &s[0..3] // hibás: byte, nem karakter
}
```

9.1.8 Megoldások

9.1.8.1 a) Megoldás

Az `&str` a jobb választás, mert a `&String`-et csak `&String`-gel lehet hívni, míg az `&str`-t `&String` és `&str` értékkel is (deref coercion révén). Ha a függvénynek nincs szüksége a tulajdonjogra, `&str`-t fogadjon.

9.1.8.2 b) Megoldás

```rust
fn print_first(xs: &[i32]) {
    if let Some(x) = xs.first() {
        println!("{x}");
    }
}

fn main() {
    let v = vec![1, 2, 3];
    print_first(&v);
}
```

A `&v &Vec<i32>`-ből `&[i32]`-re deref coercion történik. A C++ `std::span<const int>` a `&[i32]`-nek felel meg; az ürességvizsgálat Rustban a `first()` `Option`-jával történik.

9.1.8.3 c) Megoldás

```rust
fn parse_command(line: &str) -> Option<(&str, &str)> {
    let (cmd, rest) = line.split_once(' ')?;
    match cmd {
        "GET" if !rest.is_empty() && !rest.contains(' ') => Some((cmd, rest)),
        "SET" => {
            let (key, value) = rest.split_once(' ')?;
            if key.is_empty() || value.is_empty() { None } else { Some((cmd, rest)) }
        }
        _ => None,
    }
}
```

Ellenőrzött kimenetek: `""` → `None`; `"GET"` → `None`; `"GET a"` → `Some(("GET", "a"))`; `"GET a b"` → `None`; `"SET k"` → `None`; `"SET k v"` → `Some(("SET", "k v"))`; `"SET k v w"` → `Some(("SET", "k v w"))`; `"GET\ta"` (tabulátor) → `None`; `"FOO x"` → `None`; `"GET "` → `None`. A `split_once` `&str`-eket ad vissza, nem allokál. A valódi protokoll elemzésénél a `DEL` és `LIST` parancsokat is kezelni kell; ez a gyakorlat a `&str` alapú bontás mintáját mutatja.

9.1.8.4 d) Megoldás

Pánikoló bemenet például `"éé"` (két darab 2 bájtos karakter, hossz 4 bájt), ahol a 3. bájt a második karakter közepén van, így `&s[0..3]` pánikol.

Javítás (a név jelzi, hogy skalárértékekről van szó):

```rust
fn first_three_scalars(s: &str) -> Option<&str> {
    // a skalárértékek kezdő bájtindexei + a szöveg vége
    let mut ends = s.char_indices().map(|(i, _)| i).chain(std::iter::once(s.len()));
    let end = ends.nth(3)?; // None, ha 3-nál kevesebb skalárérték van
    s.get(..end)
}
```

Ellenőrzött viselkedés: `first_three_scalars("árvíz")` → `Some("árv")`; `first_three_scalars("ab")` → `None`; `first_three_scalars("")` → `None`; `first_three_scalars("éé")` → `None` (csak két skalárérték). A függvény **skalárértékeket** vág, nem látható karaktereket: a kombinált ékezetes `"e\u{301}ab"` szövegre `Some("e\u{301}a")`-t ad, ami három skalárérték, de csak két látható karakter. Pontosan három látható karakterhez grapheme-szegmentálás kellene. Az `s.get(..end)` biztonságosan ellenőrzi a karakterhatárt, ezért nem pánikol.

9.1.9 Rövid összefoglaló

A `&[T]` és a `&str` *fat pointer*: adatmutató és hossz, nem tulajdonos nézet. A `String` birtokolt, heap-en tárolt UTF-8 puffer, a `Vec<u8>`-hoz hasonló tulajdonosi szereppel. A függvényparaméterekben szöveget mindig `&str`-ként, bájtsorozatot `&[T]`-ként fogadjunk, hacsak a függvény nem szerzi meg a tulajdonjogot; ezt a döntést a API-tervezés Rustban: irányelvek és ergonómia és a C ABI: extern “C”, repr(C), bindgen és cbindgen alfejezet viszi tovább. A `String` nem indexelhető bájtonként, mert a karakterek több bájtosak lehetnek; használjunk `chars()`, `bytes()`, `char_indices()` vagy `get()` metódusokat. A `Cow<'a, str>` clone-on-write típus, amelyet a Teljesítmény: mérés, profilozás, allokáció, release-beállítások alfejezetben vizsgálunk tovább.

9.1.10 Források és ellenőrzési hiányok

A slice-ok, a `String`/`&str`, a `char_indices`, a `split_once` és a `Cow` a Rust standard könyvtár stabil API-jának részei; a hivatalos leírások a Rust Bookban és a `std` dokumentációjában találhatók. A példákban szereplő pontos pánik- és hibaüzenetek szövege verziófüggő; érdemes ellenőrizni a fejezet készítésekor használt toolchainen. A `cargo new` és a `cargo run` parancsok stabilak, a bemutatott kód a 2021-es és 2024-es edition szerint is lefordítható.

10 Smart pointerek és belső mutabilitás

10.1 Box, Rc, Arc és Weak

10.1.1 Motiváció

C++-ban a nyers mutatók és a `new`/`delete` korszaka után a smart pointerek lettek az erőforrás-kezelés alapeszközei: `std::unique_ptr` a kizárólagos tulajdonláshoz, `std::shared_ptr` a megosztotthoz, `std::weak_ptr` a ciklusok megtöréséhez. A Rust tulajdonlási rendszere miatt azonban ezeknek a szerepeknek más a súlya: a `Box<T>` sokkal ritkábban kell, mint a `unique_ptr`, mert a Rust alapértelmezett mozgatási szemantikája sok esetben kupacallokáció nélkül is elég; az `Rc<T>` és `Arc<T>` pedig gyakran elkerülhető a kölcsönzéssel. Ebben az alfejezetben megtanulja, mikor és miért kell okos pointert használni Rustban, és hogyan térnek el a C++ megfelelőiktől.

10.1.2 Box<T>: kizárólagos kupactulajdon

Definíció. A `Box<T>` egy `T` értéket az allokátor által adott memóriában (a gyakorlatban a kupacon) tároló, egyetlen tulajdonos mutató: az egyik leggyakoribb *tulajdonló indirekció*, de nem az egyetlen. A `Box` maga egy veremben tárolt mutató, amely a kupacon lévő értékre mutat; amikor a `Box` kikerül a hatókörből, a `Drop` implementációja felszabadítja a memóriát. A `Box` nem lehet `null`. A `Box<T>` **nem `Copy`**, a mozgatása a tulajdonjogot adja át; `Clone`-nal viszont klónozható, ha `T: Clone`, és ekkor *új allokációval* mély másolatot készít (`Copy` = implicit bitmásolás, `Clone` = explicit, típus által meghatározott másolat). Az, hogy az érték a kupacon van, a példa tulajdonsága (a `Box<()>` például nem is allokál), nem a típus lényege: a döntés arról szól, kell-e fix méretű indirekció, tulajdonos-trait-objektum vagy nagy érték mozgatásának olcsósága.

Szabály. A `Box<T>` a `std::unique_ptr<T>`-hez hasonlóan kizárólagos tulajdonlást valósít meg, de a Rust mozgatási szemantikája miatt a mozgatás implicit: nincs szükség `std::move`-ra. Rekurzív típusokhoz *valamilyen* fix méretű indirekció kell, mert a Rust nem enged végtelen méretű típusokat: a `Node` mezője nem lehet közvetlenül `Node`, mert a fordító nem tudná kiszámítani a méretét. Erre a leggyakoribb eszköz a `Box<Node>` (egyetlen mutatót tartalmaz), de ugyanezt adja a `Vec<Node>`, az `Rc<Node>` vagy a hivatkozás is, ahogyan a későbbi `Rc`-alapú fa és a `Box<dyn Trait>` példák mutatják: a `Box` tehát nem kötelező, hanem a kizárólagos tulajdon esetén kézenfekvő választás.

Rekurzív enum lista `Box`-szal:

```
enum List<T> {
    Nil,
    Cons(T, Box<List<T>>),
}

impl<T> List<T> {
    fn len(&self) -> usize {
        match self {
            List::Nil => 0,
```

```
            List::Cons(_, tail) => 1 + tail.len(),
        }
    }
}
```

C++-ban a `unique_ptr`-es láncolt lista hasonló:

```
#include <memory>

template <typename T>
struct Node {
    T value;
    std::unique_ptr<Node> next;
    Node(T v, std::unique_ptr<Node> n = nullptr)
        : value(std::move(v)), next(std::move(n)) {}
};
```

A különbség: Rustban a `List::Cons` második mezője `Box<List<T>>`, azaz a `Box` jelzi a kupacfoglalást; nincs `nullptr`, a végét a `Nil` variáns reprezentálja. A `unique_ptr` mozgatása után a forrás élő, *üres* objektum marad (null mutató, használható), míg a `Box` mozgatása után a Rust-forrás inicializálatlanná válik, és a fordító tiltja a használatát; ez a nyelv alapértelmezése, nem külön művelet.

10.1.3 Rc<T> és Arc<T>: megosztott tulajdon

Definíció. Az `Rc<T>` (reference counted) egy egyszálú, nem atomikus referenciaszámlálóval rendelkező okos pointer. A `Rc::clone` lemásolja a mutatót, és a belső számlálót eggyel növeli; a `Drop` csökkenti, és nulla erős számlálónál eldobja a `T` értéket; a vezérlőblokk (a számlálókkal) csak az utolsó `Weak` megszűnésekor szabadul fel. Az `Arc<T>` (atomically reference counted) ugyanezt a modellt valósítja meg atomikus műveletekkel, ezért biztonságosan használható több szál között, feltéve, hogy `T: Send + Sync`.

Szabály. Az `Rc<T>` és `Arc<T>` a C++ `std::shared_ptr`-ének felel meg, de Rustban a klónozás explicit: a `Rc::clone` hívás világossá teszi, hogy a referenciaszámláló nő. Az `Rc` nem biztonságos többszálú környezetben, mert a számlálója nem atomikus; a fordító ezt a `Send` és `Sync` trait hiányával jelzi. Az `Arc` atomikus számlálása drágább, ezért csak akkor használjuk, ha valóban szükséges a szálak közötti megosztás.

Példa `Rc`-re:

```
use std::rc::Rc;

let a = Rc::new(String::from("hello"));
let b = Rc::clone(&a);
println!("{} {}", a, b);
println!("strong: {}", Rc::strong_count(&a));
```

Várt kimenet:

```
hello hello
strong: 2
```

C++ `shared_ptr`-rel ugyanez:

```cpp
#include <memory>
#include <string>
#include <iostream>

auto a = std::make_shared<std::string>("hello");
auto b = a; // a számláló nő
std::cout << *a << " " << *b << "\n";
std::cout << "count: " << a.use_count() << "\n";
```

Fontos különbség, hogy a C++ `shared_ptr` másolása implicit, a Rust `Rc` klónozása viszont a `Rc::clone` hívással explicit. A `Rc::strong_count` a `use_count()` Rust-megfelelője.

10.1.4 Weak<T>: ciklusok megtörése

Definíció. A `Weak<T>` egy gyenge referenciát tárol az `Rc`-ben (vagy `Arc`-ban) lévő értékre. Nem növeli az erős számlálót, így nem akadályozza meg az érték felszabadítását. A `Weak` nem ad közvetlen hozzáférést az adathoz; az `upgrade()` metódus `Option<Rc<T>>`-t ad vissza: ha az érték még él, `Some`, ha már felszabadult, `None`.

Szabály. A `Weak<T>` a C++ `std::weak_ptr`-nek felel meg, de biztonságosabb használatra kényszerít: az `upgrade()` `Option`-t ad vissza, így nem lehet érvénytelen pointert dereferálni. Referenciaciklusok esetén a gyenge mutatóval megtörhetjük a ciklust: a szülő erős `Rc`-t tart a gyerekre, a gyerek `Weak`-t a szülőre, így az erős számláló nem alkot kört.

Példa egy fa csomópontra, ahol a gyerek `Weak`-kel mutat a szülőre:

```rust
use std::rc::{Rc, Weak};

#[derive(Debug)]
struct Node {
    value: String,
    parent: Option<Weak<Node>>,
    children: Vec<Rc<Node>>,
}

impl Node {
    fn new(value: &str) -> Rc<Node> {
        Rc::new(Node {
            value: value.to_string(),
            parent: None,
            children: Vec::new(),
        })
    }

    fn with_parent(value: &str, parent: &Rc<Node>) -> Rc<Node> {
        Rc::new(Node {
            value: value.to_string(),
            parent: Some(Rc::downgrade(parent)),
            children: Vec::new(),
        })
    }

    fn parent_value(&self) -> Option<String> {
        self.parent
            .as_ref()
            .and_then(|weak| weak.upgrade())
            .map(|p| p.value.clone())
    }
}
```

Ebben a kódban a `Weak` megtöri a szülő–gyerek ciklust: a szülő `Vec<Rc<Node>>`-ja erős referenciákat tart, a gyerek `Weak`-je nem növeli az erős számlálót. Ha a szülő `Rc`-je kikerül a hatókörből, az erős számláló az esetleges gyerek referenciák ellenére is csökkenhet, mert a gyerekek csak `Weak`-et tartanak a szülőre. A `parent_value` metódus az `upgrade()`-et használja, hogy biztonságosan hozzáférjen a szülőhöz, ha az még él.

A C++ `weak_ptr` hasonló:

```cpp
#include <memory>
#include <string>
#include <iostream>
#include <vector>

struct Node {
    std::string value;
    std::weak_ptr<Node> parent;
    std::vector<std::shared_ptr<Node>> children;
};

int main() {
    auto parent = std::make_shared<Node>();
    parent->value = "root";
    auto child = std::make_shared<Node>();
    child->value = "child";
    child->parent = parent;
    parent->children.push_back(child);
    if (auto p = child->parent.lock()) {
        std::cout << p->value << "\n";
    }
}
```

A különbség: Rustban az `upgrade()` `Option`-t ad, a `lock()` pedig `shared_ptr`-t, amely null lehet; a `weak_ptr` dereferálása előtt mindig ellenőrizni kell a `lock()` eredményét, a Rust `Option` kezelése pedig mintamegyeztetéssel vagy `?`-vel kötelező.

10.1.5 Deref és deref coercion

Definíció. A `Deref` trait lehetővé teszi a `*` operátor használatát, és a *deref coercion* (kényszerítés) automatikusan konvertálja a `&Box<T>`-t (vagy `&Rc<T>`-t, `&Arc<T>`-t) `&T`-vé, ha a referenciát `&T`-t váró helyre (például függvényargumentumként) adjuk át; metódushívásnál pedig az autoderef keresi meg a belső típus metódusát. A `Box<T>`, `Rc<T>` és `Arc<T>` implementálják a `Deref`-et, így a belső érték metódusai közvetlenül hívhatók.

Szabály. A deref coercion fordítási idejű, implicit konverzió: a fordító automatikusan `Deref::deref` hívásokat illeszt be a láncba (`&Box<String>` → `&str`); beépített típusoknál ez elhanyagolható, de felhasználói `Deref` implementációnál az ott írt kód ténylegesen lefut. Ez megkönnyíti az okos pointerek használatát, mert nem kell explicit `*` vagy `as_ref()` hívás.

Példa:

```rust
let b: Box<String> = Box::new(String::from("hello"));
println!("{}", b.len()); // Deref coercion: &Box<String> -> &String, len() a String-en
```

C++-ban az `operator*` és `operator->` explicit hívást igényelne hasonló helyzetben, bár a `->` operátor automatikusan át van terhelve. Két különböző mechanizmus dolgozik itt, és érdemes őket szétválasztani:

- **Metódushívásnál autoref és autoderef:** a `b.len()` hívásnál a fordító maga képez referenciát (`&b`), és addig dereferál (`Box<String>` → `String`), amíg megtalálja a `len` metódust. Itt a hívó semmit nem ír ki.
- **Függvényargumentumnál deref coercion:** egy már meglévő *referencia* típusa alakul át, `&Box<String>` → `&String` (→ `&str`). A referenciát viszont a hívónak kell képeznie; érték nem alakul át automatikusan referenciává.

```
fn len(s: &String) -> usize {
    s.len()
}

fn main() {
    let b: Box<String> = Box::new(String::from("hello"));
    println!("{}", b.len());  // metódushívás: autoref + autoderef
    println!("{}", len(&b));  // deref coercion: &Box<String> -> &String
    // println!("{}", len(b)); // E0308: expected `&String`, found `Box<String>`
}
```

A `len(&b)` tehát nem igényel explicit `&*b`-t, de a `&`-t igen; a `len(b)` nem fordul le, mert a `Box<String>` érték, nem referencia, és a coercion csak referenciákon dolgozik.

10.1.6 Labor: Box, Rc és Weak gyakorlása

Munkakönyvtár: `~/rust-labor/r019/`

Hozz létre egy önálló Cargo-projektet, és írd meg a következő kódot a `src/main.rs` fájlba:

```
cargo new r019
cd r019
```

`src/main.rs`:

```
use std::rc::{Rc, Weak};

enum List<T> {
    Nil,
    Cons(T, Box<List<T>>),
}

impl<T> List<T> {
    fn len(&self) -> usize {
        match self {
            List::Nil => 0,
            List::Cons(_, tail) => 1 + tail.len(),
        }
    }
}

#[derive(Debug)]
struct Node {
    value: String,
    parent: Option<Weak<Node>>,
    children: Vec<Rc<Node>>,
}

impl Node {
    fn new(value: &str) -> Rc<Node> {
        Rc::new(Node {
            value: value.to_string(),
            parent: None,
```

```
            children: Vec::new(),
        })
    }

    fn add_child(parent: &Rc<Node>, child_value: &str) -> Rc<Node> {
        // A szülő children vektorát itt még nem tudjuk módosítani, mert a Node
        // mezői immutable-ok; ezt a r020 alfejezetben RefCell-lel oldjuk meg.
        Rc::new(Node {
            value: child_value.to_string(),
            parent: Some(Rc::downgrade(parent)),
            children: Vec::new(),
        })
    }
}

fn main() {
    let list = List::Cons(1, Box::new(List::Cons(2, Box::new(List::Nil))));
    println!("list length: {}", list.len());

    let root = Node::new("root");
    let child = Node::add_child(&root, "child");
    println!("child parent: {:?}", child.parent.as_ref().and_then(|w| w.upgrade()));

    let shared = Rc::new(String::from("shared"));
    let weak = Rc::downgrade(&shared);
    drop(shared);
    assert!(weak.upgrade().is_none());
    println!("weak after drop: None");
}
```

Fordítás és futtatás:

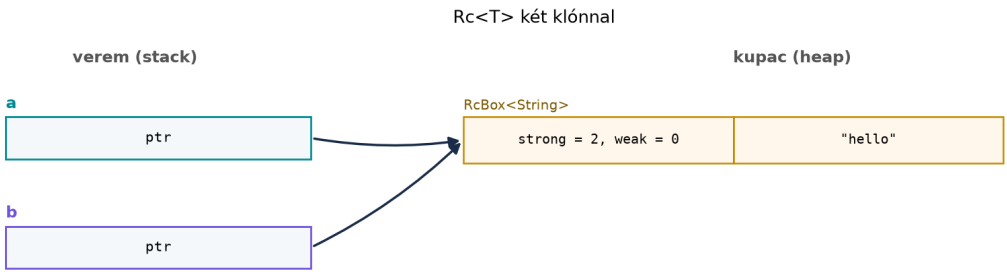
```
cargo run
```

Várt kimenet. A `child.parent` egy `Weak<Node>`, ezért az `upgrade()`-del élő `Rc`-vé alakítjuk, hogy a még élő szülő *tartalmát* lássuk, ne a gyenge fogantyú diagnosztikáját (a `Weak` `Debug`-ja csak azt jelzi, hogy gyenge referencia). A kimenet a `Node` `Debug` implementációjától függ:

```
list length: 2
child parent: Some(Node { value: "root", parent: None, children: [] })
weak after drop: None
```

A laborban a `List` rekurzív enum a `Box` használatát, a `Node` a `Weak`-et, a `Rc` pedig a megosztott tulajdonlást mutatja be. A `Node::add_child` metódus jelenleg nem módosítja a szülő `children` vektorát, mert a `Node` mezői változtathatatlanok; ezt a hiányosságot a Belső mutabilitás: Cell, RefCell alfejezetben pótoljuk, ahol a `RefCell`-t bevonva a fa teljesen felépíthető lesz.

Az ábra a memóriaelrendezést mutatja két `Rc` klónnal:



10.1. ábra: Rc<T> két klónnal - A kupacblokk a referenciaszámlálót és az értéket is tartalmazza; a két Rc ugyanarra a blokkra mutat.

A `Rc` a kupacon egy belső `RcBox` struktúrát hoz létre, amely tartalmazza a számlálókat (erős és gyenge) és az értéket. A veremben lévő `a` és `b` csak egy-egy mutatót tárol, mindkettő ugyanarra a kupacblokkra mutat. A gyenge számláló akkor nő, amikor `Weak`-et hozunk létre; az erős számláló nulla elérésekor az érték felszabadul, a gyenge számláló nulla elérésekor pedig a `RcBox` is megszűnik.

10.1.7 Tipikus hibák és átállási csapdák

1. **Klónozás a borrow checker „legyőzésére”.** A `Rc::clone` szórása az egész kódban gyakran rossz válasz a kölcsönzési hibákra. Ha egy értéket csak olvasni akarunk egy másik hatókörben, a `&T` referenciával egyszerűbb és hatékonyabb. Példa rossz kód:

```
use std::rc::Rc;

fn len(s: Rc<String>) -> usize {
    s.len()
}

fn main() {
    let s = Rc::new(String::from("hello"));
    // Ez lefordul, de feleslegesen növeli a számlálót.
    println!("{}", len(Rc::clone(&s)));
}
```

Javítás: `fn len(s: &str) -> usize` és `len(&s)`.

2. **`Rc` küldése szálakba.** Az `Rc` nem implementálja a `Send` traitet, ezért fordítási hibát kapunk, ha egy szálba próbáljuk mozgatni. Helyette `Arc`-ot kell használni:

```
use std::rc::Rc;

fn main() {
    let s = Rc::new(String::from("hello"));
    // Ez a sor szándékosan nem fordul:
    // let handle = std::thread::spawn(move || println!("{}", s));
}
```

A hiba lényege: `Rc<String>` nem `Send`, mert a referenciaszámláló nem atomikus. A javítás az `Arc` használata.

3. **`Weak::upgrade()` eredményének `unwrap`-elése.** Ha az utolsó `Rc` már eldobásra került, az `upgrade()` `None`-t ad, és az `unwrap` panikol. Használj `match`-et, `if let`-et vagy a `?` operátort.

4. **Felesleges egymásba ágyazott okos pointerek.** Az `Arc<Rc<T>>` vagy `Rc<Arc<T>>` szinte soha nem indokolt; válasszon egyet, és a többi réteget kölcsönzéssel oldja meg.

10.1.8 Feladatok

1. **Lefordul-e és miért?** Az alábbi kódban `Rc`-t küldünk egy szálba. Mi lesz a fordítási hiba lényege, és hogyan javítható?

```rust
use std::rc::Rc;
use std::thread;

fn main() {
    let s = Rc::new(String::from("hello"));
    let handle = thread::spawn(move || println!("{}", s));
    let _ = handle.join();
}
```

2. **C++-kód átírása idiomatikus Rustra.** Az alábbi C++ kód egy bináris fa csomópontját `unique_ptr`-rel definiálja. Írd át Rustra `Box` vagy `Option` használatával, és add meg a rekurzív `height` metódust.

```cpp
#include <algorithm>
#include <memory>

struct Node {
    int value;
    std::unique_ptr<Node> left;
    std::unique_ptr<Node> right;

    int height() const {
        int lh = left ? left->height() : 0;
        int rh = right ? right->height() : 0;
        return 1 + std::max(lh, rh);
    }
};
```

3. **Implementációs feladat.** Készíts egy `Tree` típust, amelyben a szülő `Rc<Tree>`-t tart a gyerekekről, a gyerek pedig `Weak<Tree>`-t a szülőről. A fa csomópontjai tartalmazzanak egy `String` értéket. Írj egy `parent_value(&self) -> Option<String>` metódust, amely visszaadja a szülő értékét, ha a szülő még él. A szülőt és a gyerekeit egyetlen lépésben, az `Rc::new_cyclic` segítségével hozd létre, úgy, hogy a gyerekek ténylegesen szerepeljenek a szülő `children` listájában. Ellenőrizd a `parent_value`-t élő és már eldobott szülőre is.

4. **Hibajavítás.** Az alábbi kód egyszálú kontextusban `Arc`-ot használ, és `unwrap`-eli az `upgrade` eredményét. Refaktorálja idiomatikus `Rc` és `Option`-kezeléssé.

```rust
use std::sync::{Arc, Weak};

fn main() {
    let a = Arc::new(String::from("hello"));
    let w: Weak<String> = Arc::downgrade(&a);
    drop(a);
    // Ez panikol, ha a gyenge mutató nem érvényes.
    let s = w.upgrade().unwrap();
    println!("{}", s);
}
```

10.1.9 Megoldások

10.1.9.1 1. Lefordul-e?

Válasz: Nem fordul. Az `Rc<String>` nem implementálja a `Send` traitet, mert a referenciaszámlálója nem atomikus, ezért nem biztonságos egy másik szálba mozgatni. A hiba lényege: `Rc<String>` nem `Send`. Javítás `Arc`-ra cserélve:

```
use std::sync::Arc;
use std::thread;

fn main() {
    let s = Arc::new(String::from("hello"));
    let handle = thread::spawn(move || println!("{}", s));
    let _ = handle.join();
}
```

10.1.9.2 2. C++-kód átírása

A Rust-megfelelő `Box`-szal és `Option`-nal:

```
#[derive(Debug)]
struct Node {
    value: i32,
    left: Option<Box<Node>>,
    right: Option<Box<Node>>,
}

impl Node {
    fn new(value: i32) -> Box<Node> {
        Box::new(Node {
            value,
            left: None,
            right: None,
        })
    }

    fn height(&self) -> usize {
        let lh = self.left.as_ref().map(|n| n.height()).unwrap_or(0);
        let rh = self.right.as_ref().map(|n| n.height()).unwrap_or(0);
        1 + lh.max(rh)
    }
}
```

A különbség: a C++ `unique_ptr`-rel a `left` és `right` közvetlenül a `Node`-ban van, és a null értékeket a `unique_ptr` null típusával jelzi; Rustban az `Option<Box<Node>>` kombináció explicit en kezeli a hiányzó gyereket. A `height` metódusban a `Box`-ra a deref coercion automatikusan alkalmazza a `Node` metódusait.

10.1.9.3 3. Tree típus

Egy `Rc<Tree>` tartalma közös, ezért utólag nem módosítható: a szülő `children` vektorába a létrehozás után már nem tehetünk gyereket (ehhez belső mutabilitás, például `RefCell<Vec<Rc<Tree>>>` kellene, lásd Belső mutabilitás: Cell, RefCell és a futásidejű kölcsönzés). A tyúk–tojás problémát (a gyereknek a szülőre mutató `Weak` kell, a szülőnek pedig a kész gyerekek) az `Rc::new_cyclic` oldja meg: a lezárás egy `Weak`-et kap a még épülő szülőre, és ezt a gyerekek létrehozásakor már felhasználhatjuk.

```rust
use std::rc::{Rc, Weak};

#[derive(Debug)]
struct Tree {
    value: String,
    parent: Option<Weak<Tree>>,
    children: Vec<Rc<Tree>>,
}

impl Tree {
    /// A szülőt és a megadott értékű gyerekeit egy lépésben hozza létre.
    fn with_children(value: &str, child_values: &[&str]) -> Rc<Tree> {
        Rc::new_cyclic(|me: &Weak<Tree>| Tree {
            value: value.to_string(),
            parent: None,
            children: child_values
                .iter()
                .map(|v| {
                    Rc::new(Tree {
                        value: v.to_string(),
                        parent: Some(me.clone()), // gyenge hivatkozás a szülőre
                        children: Vec::new(),
                    })
                })
                .collect(),
        })
    }

    fn parent_value(&self) -> Option<String> {
        let parent = self.parent.as_ref()?.upgrade()?;
        Some(parent.value.clone())
    }
}

fn main() {
    let root = Tree::with_children("root", &["a", "b"]);
    assert_eq!(root.children.len(), 2);
    assert_eq!(root.parent_value(), None); // a gyökérnek nincs szülője

    let child = Rc::clone(&root.children[0]);
    assert_eq!(child.value, "a");
    assert_eq!(child.parent_value(), Some("root".to_string()));
    assert_eq!(Rc::strong_count(&root), 1); // a gyerekek nem tartják életben
    assert_eq!(Rc::weak_count(&root), 2);

    drop(root); // a szülő felszabadul, a "b" gyerekkel együtt
    assert_eq!(child.parent_value(), None); // az upgrade már None
    println!("ok");
}
```

Kimenet: `ok`. Az `Rc::strong_count(&root) == 1` mutatja, hogy nincs referenciaciklus: a gyerekek csak gyenge hivatkozást tartanak. A `drop(root)` után a gyökér felszabadul; a `child` azért él tovább, mert a `main` egy saját erős hivatkozást (`Rc::clone`) tart rá, a szülője viszont már nincs, ezért a `parent_value` `None`-t ad.

10.1.9.4 4. Hibajavítás

Javított változat:

```rust
use std::rc::{Rc, Weak};
```

```rust
fn main() {
    let a = Rc::new(String::from("hello"));
    let w: Weak<String> = Rc::downgrade(&a);
    drop(a);
    match w.upgrade() {
        Some(s) => println!("{}", s),
        None => println!("már nincs"),
    }
}
```

Az `Arc`-ról `Rc`-re váltottunk, mert egyszálú környezetben nincs szükség atomikus számlálásra; az `unwrap` helyett `match`-et használunk, hogy kezeljük a `None` esetet.

10.1.10 Rövid összefoglaló

A `Box<T>` a C++ `unique_ptr` Rust-megfelelője: kupacra allokált, kizárólagos tulajdon, rekurzív típusokhoz nélkülözhetetlen. Az `Rc<T>` a `shared_ptr` egyszálú, nem atomikus változata, az `Arc<T>` pedig az atomikus, szálak között is biztonságos. Mindkettő explicit `clone`-t igényel a számláló növeléséhez. A `Weak<T>` a `weak_ptr` Rust-megfelelője; az `upgrade()` `Option`-t ad, és megtöri a szülő–gyerek referenciaciklust. A `Deref` és a deref coercion lehetővé teszi, hogy a `Box`, `Rc` és `Arc` a belső érték metódusait közvetlenül meghívja. A laborban a `List<T>` rekurzív enumot és a `Weak` mechanizmust gyakoroltuk a `~/rust-labor/r019/` projektben. **Továbbvitt eredmény.** A `Rc` és a `Deref` a Belső mutabilitás: Cell, RefCell alfejezetben az `Rc<RefCell<T>>` minta alapját képezi; az `Arc` a Szálak, Send és Sync alfejezetben a szálak közötti megosztásnál tér vissza; a `Deref` a Trait objektumok és dinamikus diszpécs alfejezetben a `Box<dyn Trait>` és `Rc<dyn Trait>` használatakor válik kulcsfontosságúvá.

10.1.11 Források és ellenőrzési hiányok

A fejezet a `std::boxed::Box`, `std::rc::Rc`, `std::sync::Arc` és `std::rc::Weak` stabil API-jára épül (https://doc.rust-lang.org/std/). A `Debug` formátum a `Weak`-re és az `upgrade` viselkedése a rögzített Rust 1.99.1-re vonatkozik. Az alfejezetben nincs külső crate-függőség.

10.2 Belső mutabilitás: Cell, RefCell és a futásidejű kölcsönzés

10.2.1 Motiváció

Egy C++-os osztályban a `const` tagfüggvény fő szabálya, hogy nem módosíthatja az objektum megfigyelhető állapotát. Mégis gyakori, hogy egy `const` metódus gyorsítótárazni szeretne, vagy számlálnia kell a hívásait. Erre a C++ a `mutable` kulcsszót adja:

```cpp
struct Cache {
    mutable std::unordered_map<std::string, std::string> cache;

    std::string get(const std::string& key) const {
        auto it = cache.find(key);
        if (it != cache.end()) return it->second;
        std::string value = expensive(key);
        cache.emplace(key, value);
        return value;
    }
};
```

A `get` tehát `const`, mégis írja a `cache` tagot. A Rust ugyanezt alapból megtiltja: egy `&self` metódus nem kaphat `&mut self`-et. A probléma természetesen nem a cache-elés, hanem az, hogy a megosztott referencián keresztüli módosítás megsértheti a kölcsönzési szabályt. A Rust a **belső mutabilitás** (interior mutability) eszközeivel ad szabályozott, típusos kivételt ez alól.

10.2.2 Fogalmak és szabályok

Definíció (belső mutabilitás). A belső mutabilitás olyan minta, amelyben egy `&T` megosztott referencián keresztül elérhető érték *belsejét* megváltoztathatjuk anélkül, hogy magára az értékre `&mut T` kizárólagos referenciát vennénk.

Szabály. A `&T` referencián keresztül a Rust típusrendszere alapból tiltja a mutálást. A `Cell<T>` és a `RefCell<T>` típusok `UnsafeCell<T>`-re épülve biztonságos, ellenőrzött felületet adnak erre. A biztonságot két különböző módon érik el: a `Cell` *megszorított API-val* (soha nem ad referenciát a belsejére, ezért nincs futásidejű ellenőrzés), a `RefCell` *futásidejű kölcsönzés-számlálóval.*

A két alaptípus közötti választást a következő táblázat foglalja össze:

Kérdés	Választás
`T` `Copy`, és csak egész értéket cserélünk vagy olvasunk	`Cell<T>`
Nagy vagy összetett `T`, és referenciát akarunk kiadni a belső adatra	`RefCell<T>`
Több szálból akarjuk módosítani	sem `Cell`, sem `RefCell`; később `Mutex`/`RwLock`

Definíció (`Cell<T>`). A `Cell<T>` a belső értéket csak teljes egészében engedi kiolvasni vagy lecserélni, referenciát a belsejére nem ad ki. A `get` metódus megköveteli a `T: Copy` határt, mert másolatot ad. Ez a feltétel **csak a `get`-re vonatkozik**: a `set`, a `replace` és a `swap` bármilyen `T`-vel működik, a `take` pedig `T: Default`-ot kér (a helyére az alapértéket teszi). A `Cell<String>` tehát nem `get`-elhető, de `replace`-elhető és `take`-elhető: `c.replace(String::from("körte"))` visszaadja a régi `String`-et, `c.take()` pedig kiveszi az értéket és üres `String`-et hagy. Mivel a `Cell` sosem ad referenciát a belsejére, nem keletkezhet aliasolás a belső értékre; a szabályt az API-ja tartja be, nem futásidejű ellenőrzés.

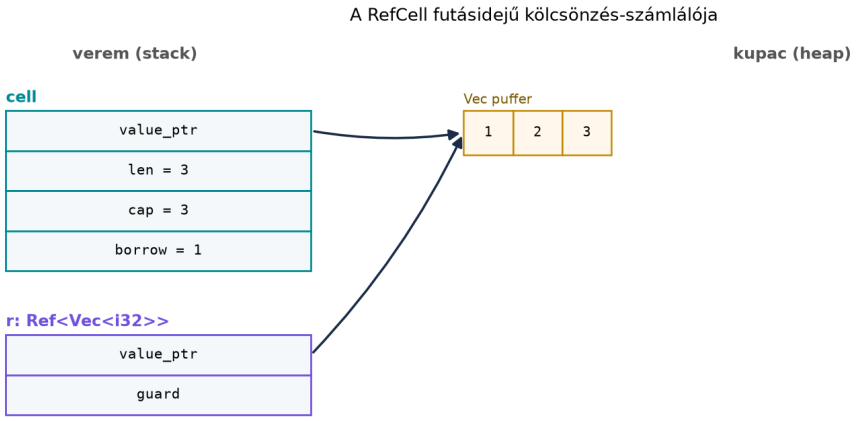
Definíció (`RefCell<T>`). A `RefCell<T>` futásidejű kölcsönzés-számlálót tart nyilván. A `borrow()` metódus `Ref<T>`, a `borrow_mut()` pedig `RefMut<T>` guardot ad. Ha a szabályt megsértjük – például élő `Ref` mellett kérünk `RefMut`-ot –, akkor a program **pánikol**, tipikusan `already borrowed` üzenettel. A pánikmentes változatok a `try_borrow()` és `try_borrow_mut()`, amelyek `Result`-ot adnak vissza.

Megosztott referenciából safe castal bármilyen típusnál lehet `*mut T` mutatót képezni; a mutató létrehozása azonban nem ad írási jogot, és az írás nem `UnsafeCell` mögötti adaton UB. Az `UnsafeCell<T>` a nyelv egyetlen olyan építőköve, amelynek a szerződése megengedi a megosztott referencia mögötti módosítást (a mutató `UnsafeCell::get`-tel kapható). Két dolgot kell szétválasztani: a mutató *létrehozása* (`UnsafeCell::get`) safe művelet, a mutató *dereferálása* viszont `unsafe`, és az aliasing-szabályok betartásáért ilyenkor a programozó felel. A `Cell` és a `RefCell` ezt az `unsafe` alapot csomagolják biztonságos API-ba; a `UnsafeCell` közvetlen használatát a Unsafe Rust: szuperképességek és felelősség alfejezet tárgyalja.

Továbbvitt eredmény. A `Cell` és `RefCell` belső mutabilitási mintái egyetlen szálon kijelölik azokat a határokat, amelyeket a Mutex, RwLock, csatornák és a megosztott rkv alfejezet a

szálbiztos `Mutex`/`RwLock` guardokkal, a Unsafe Rust: szuperképességek és felelősség pedig az `UnsafeCell` közvetlen, `unsafe` használatával visznek tovább.

A következő memóriaábra a `RefCell` futásidejű flagjét mutatja:



10.2. ábra: A RefCell futásidejű kölcsönzés-számlálója - A cell a value mellett egy borrow flaget tárol. Amíg a Ref guard él, a borrow = 1, ezért a borrow_mut pánikol.

A `Ref` guard tehát nem maga a `Vec`, hanem egy „kölcsönzés-jegy”, amely a lefutás végéig fenntartja a megosztott állapotot. Amikor a `Ref` eldobódik, a `borrow` flag visszaáll nullára.

10.2.3 C++ és Rust összevetés

A C++ `mutable` kulcsszóval egy `const` objektumon belül is írhatunk taget. Ehhez a Rustban a `Cell` vagy a `RefCell` használható. A bevezető cache példa Rustban így néz ki:

```
use std::cell::{Cell, RefCell};
use std::collections::HashMap;

struct Cache {
    store: RefCell<HashMap<String, String>>,
    hits: Cell<u64>,
}

impl Cache {
    fn get(&self, key: &str) -> String {
        if let Some(value) = self.store.borrow().get(key) {
            self.hits.set(self.hits.get() + 1); // csak valódi cache-találatnál
            return value.clone();
        }

        let value = format!("érték: {key}");
        self.store.borrow_mut().insert(key.to_string(), value.clone());
        value
    }
}
```

A különbség nem csak szintaktikai. A C++ `mutable` taghez a `const` metódus is `T&`-ként fér hozzá, tehát a fordító semmilyen szabályt nem ellenőriz arra, hogy a referenciát meddig tartja valaki. A Rust `RefCell` ezzel szemben guardot ad: a `borrow()` eredménye nem élhet túl a

`Ref` guardot, és a `borrow_mut()` futásidőben ellenőrzi, hogy nincs-e élő megosztott kölcsönzés. A C++ `mutable` tag szálbiztonságot sem ad. A Rust `Cell` és `RefCell` két külön tulajdonság miatt viselkedik másképp: `Send` (egy szálba *átmozgatható*, ha a `T: Send`), de nem `Sync` (több szálból *nem osztható meg* referenciával). Ezért egy `RefCell` `move` closure-rel egy másik szálba áthelyezhető, az `Arc<RefCell<T>>` viszont nem küldhető át (`E0277: RefCell<i32>` cannot be shared between threads safely). A szálbiztos megfelelőjük a `Mutex` és az `RwLock`.

10.2.4 Kidolgozott példa: egyszálú hit cache

Munkakönyvtár: `~/rust-labor/r020/`

Hozzuk létre a kísérleti projektet:

```
mkdir -p ~/rust-labor
cd ~/rust-labor
cargo new r020
cd r020
```

A `src/main.rs` teljes tartalma:

```
use std::cell::{Cell, RefCell};
use std::collections::HashMap;

struct Cache {
    store: RefCell<HashMap<String, String>>,
    hits: Cell<u64>,
}

impl Cache {
    fn new() -> Self {
        Self {
            store: RefCell::new(HashMap::new()),
            hits: Cell::new(0),
        }
    }

    fn get(&self, key: &str) -> String {
        if let Some(value) = self.store.borrow().get(key) {
            self.hits.set(self.hits.get() + 1); // csak valódi cache-találatnál
            return value.clone();
        }

        let value = format!("érték: {key}");
        self.store.borrow_mut().insert(key.to_string(), value.clone());
        value
    }
}

fn main() {
    let cache = Cache::new();
    println!("{}", cache.get("alma"));
    println!("{}", cache.get("alma"));
    println!("találatok: {}", cache.hits.get());
}
```

Várt kimenet:

```
érték: alma
érték: alma
találatok: 1
```

A `hits` tag `Cell<u64>`-ként `&self` alatt is növelhető, mert a `Cell::get` `u64`-t másol, a `set` pedig lecseréli az értéket. A számláló csak a valódi cache-találatot számolja: az első `get("alma")` kihagyás (kiszámolja és beszúrja), a második találat, ezért `találatok: 1`. A `store` `RefCell<HashMap>` mert a `HashMap`-ba a `borrow_mut()` segítségével referencián keresztül szúrunk be.

A következő rossz változat **futásidőben pánikol**:

```
let c = RefCell::new(vec![1, 2, 3]);
let r = c.borrow();
c.borrow_mut().push(4); // futásidejű pánik
println!("{r:?}");
```

A hiba lényege: `r` életben tart egy `Ref` guardot, ezért a `borrow_mut()` második kizárólagos kölcsönzése megsérti a futásidejű számlálót. A pontos üzenet verziófüggő, de a jelenség `already borrowed: BorrowMutError` formában jelenik meg.

Egyik javítás, ha a megosztott kölcsönzést korábban elengedjük:

```
let len = c.borrow().len();
c.borrow_mut().push(4);
println!("{len}");
```

Másik javítás pánikmentes próbálkozással:

```
let r = c.borrow();
if let Ok(mut v) = c.try_borrow_mut() {
    v.push(4);
} else {
    println!("még kölcsönözve");
}
drop(r);
c.borrow_mut().push(4);
```

Itt a `try_borrow_mut()` `Result<RefMut<_>, BorrowMutError>` értéket ad, így elkerülhető a pánik.

10.2.5 Tipikus hibák és átállási csapdák

1. **A `Ref` guard túl sokáig él.** Ha a `borrow()` eredményét változóba kötjük, és később `borrow_mut()`-ot hívunk, a program pánikol. A javítás: blokkba zárni a megosztott kölcsönzést, `drop(r)`-t hívni, vagy az értéket klónozva kivenni, mielőtt a mutáló kölcsönzést kérnénk.
2. **Az `Rc<RefCell<T>>` minta túlhasználata.** Az `Rc<RefCell<T>>` kényelmes megosztott, egyszálú, módosítható állapotot ad, és gyakran használjuk fák, gráfok vagy cache-ek megosztására. Azonban minden `borrow_mut()` futásidejű pánikforrás, és a ciklusok `Rc` erős számlálóval memóriaszivárgáshoz vezetnek. Ha nincs valódi megosztottság, érdemes az ownershipet átrendezni, és a `RefCell`-t csak a legszűkebb helyen tartani.
3. **`Cell` és `RefCell` több szálból.** Mindkét típus `!Sync`, ezért referenciával nem osztható meg szálak között: az `Arc<RefCell<T>>` nem küldhető át szálba (`E0277: RefCell<T>` cannot be shared between threads safely). A típusok viszont `Send`-ek (ha `T: Send`), tehát egyetlen szálnak *átadhatók* `move` closure-rel; ilyenkor a szál kizárólagos tulajdonosuk. A szálbiztos megosztott állapot a későbbi `Mutex`/`RwLock` feladata.

4. **`Cell::get` Copy nélküli típusra.** A `String` például nem `Copy`, ezért a `Cell<String>::get()` fordítási hibát ad; a `Cell<String>` más műveletei viszont működnek: a `replace(új)` visszaadja a régi értéket, a `take()` kiveszi az értéket és `String::default()`-ot hagy. Ha referenciát kell kiadni a belsejére, `RefCell<String>` kell.

10.2.6 Feladatok

1. **Lefordul-e és miért?** Az alábbi kód lefordul-e, és ha nem, mi történik?

```
let c = Cell::new(String::from("alma"));
let s = c.get();
println!("{s}");
```

Adj választ a `Cell` `Copy` korlátjával indokolva.

2. **C++→Rust átírás.** Adott a következő C++ osztály:

```
struct Counter {
    mutable int count = 0;
    int get() const {
        return ++count;
    }
};
```

Írd át idiomatikus Rustra belső mutabilitással.

3. **Implementációs feladat.** Hozz létre a `~/rust-labor/r020/` crate-ben egy `Memo` típust, amely egy `RefCell<HashMap<String, usize>>` mezőt tartalmaz, és egy `get_or_insert(&self, key: &str, f: impl FnOnce() -> usize) -> usize` metódust biztosít. A metódus a gyorsítótárból adja vissza az értéket, vagy kiszámolja, eltárolja, majd visszaadja. A típust oszd meg `Rc`-vel, és mutasd be, hogy két `Rc` klónból is ugyanazt a cache-t érik el.

4. **Hibajavítás/refaktoring.** Az alábbi kód pánikolhat. Magyarázd el, mikor és miért, majd írd át úgy, hogy ne pánikoljon:

```
fn bad(cache: &Cache) {
    let guard = cache.store.borrow();
    if !guard.contains_key("x") {
        cache.store.borrow_mut().insert("x".into(), "y".into());
    }
    println!("{}", guard.len());
}
```

10.2.7 Megoldások

1. **Nem fordul le.** A `Cell::get` megköveteli a `T: Copy` határt, mert nem adhat referenciát a belső adatra. A `String` nem `Copy`, ezért a fordítási hiba lényege: `the methodgetexists for structCell<String>, but its trait bounds were not satisfied.` A javítás `Cell<String>` helyett például `RefCell<String>` használata, vagy ha cserélni kell, `take`/`replace`.

2. A `Cell<i32>` idiomatikus megoldás:

```rust
use std::cell::Cell;

struct Counter {
    count: Cell<i32>,
}

impl Counter {
    fn get(&self) -> i32 {
        self.count.set(self.count.get() + 1);
        self.count.get()
    }
}
```

3. A `Memo` típus teljes megvalósítása:

```rust
use std::cell::RefCell;
use std::collections::HashMap;
use std::rc::Rc;

struct Memo {
    cache: RefCell<HashMap<String, usize>>,
}

impl Memo {
    fn new() -> Self {
        Self {
            cache: RefCell::new(HashMap::new()),
        }
    }

    fn get_or_insert(&self, key: &str, f: impl FnOnce() -> usize) -> usize {
        if let Some(&value) = self.cache.borrow().get(key) {
            return value;
        }

        let value = f();
        self.cache.borrow_mut().insert(key.to_string(), value);
        value
    }
}

fn main() {
    let memo = Rc::new(Memo::new());
    let clone = Rc::clone(&memo);

    println!("{}", memo.get_or_insert("a", || 10));
    println!("{}", clone.get_or_insert("a", || 20));
    println!("{}", clone.get_or_insert("b", || 30));
}
```

Várt kimenet:

```
10
10
30
```

4. A `bad` függvényben a `guard` `Ref` guard életben marad a függvény végéig, ezért a `borrow_mut()` pánikol, ha a `cache.store` `RefCell`-jén már van megosztott kölcsönzés. Javítás: a `guard`-ot elengedjük, mielőtt a mutáló kölcsönzést kérnénk:

```rust
fn good(cache: &Cache) {
    if !cache.store.borrow().contains_key("x") {
        cache.store.borrow_mut().insert("x".into(), "y".into());
```

```
    }
    println!("{}", cache.store.borrow().len()); // a módosítás UTÁN kérdezzük a méretet
}
```

Itt minden `borrow()` guard csak a kifejezés végéig él (az `if` feltételében keletkező ideiglenes `Ref` a feltétel végén megszűnik), a `borrow_mut()` pedig már nem talál élő `Ref`-et. A méretet a módosítás után kérdezzük le, mint az eredeti `bad`: üres cache-en a beszúrás után `1`-et ír ki (ellenőrzött kimenet).

10.2.8 Rövid összefoglaló

A belső mutabilitás arra való, hogy megosztott referencián keresztül is módosíthassunk állapotot, szabályozott módon. A `Cell<T>` `Copy` értékek egészének cseréjére való, és nem ad ki referenciát; a `RefCell<T>` futásidejű kölcsönzés-számlálóval `borrow()`/`borrow_mut()` guardokat ad, és szabálysértéskor pánikol. Az `Rc<RefCell<T>>` minta egyszálú megosztott, módosítható állapotot ad, de a pánikok és a ciklikus referenciák miatt takarékosan használandó. A `UnsafeCell<T>` az alap, amelyre a `Cell` és a `RefCell` épül. A C++ `mutable` taghez képest a Rust a belső mutabilitást típussal és futásidejű guardokkal ellenőrzi. **Továbbvitt eredmény:** a `Cell`/`RefCell` mintát a Mutex, RwLock, csatornák és a megosztott rkv alfejezet a szálbiztos `Mutex`/`RwLock` irányba, a Unsafe Rust: szuperképességek és felelősség pedig az `UnsafeCell` közvetlen használata felé viszi tovább.

10.2.9 Források és ellenőrzési hiányok

A `Cell`, `RefCell` és `UnsafeCell` stabil API-ját a `std::cell` dokumentáció (https://doc.rust-lang.org/std/cell/), a belső mutabilitás nyelvi szabályait a Rust Reference írja le. A `try_borrow`/`try_borrow_mut` visszatérési típusai (`Result<Ref<'_, T>, BorrowError>`, illetve `Result<RefMut<'_, T>, BorrowMutError>`) stabilak, de a pontos hibaszöveg verziófüggő. A `Cell` és `RefCell` `!Sync` tulajdonsága a stabil nyelvi szerződés része; a szálbiztos megfelelőket a későbbi alfejezetek tárgyalják.

IV. RÉSZ – Absztrakciók: hibakezelés, trait-ek, generikusok, iterátorok

11 Hibakezelés

11.1 Result, a ? operátor és a panic helye

11.1.1 Motiváció

A C++ kivételmodellben a hibakezelés kétarcú: a szignatúra gyakran nem mutatja, hogy egy függvény dobhat-e, a hívó pedig csak akkor tudja meg, ha a kivétel ténylegesen elrepül. Egy `std::stoi` hívás például `std::invalid_argument` vagy `std::out_of_range` kivételt dobhat, de a fejlécben ez a `noexcept` hiányán kívül nem látszik. Ennek következménye a „catch-amit-elfelejtettünk” hiba, a bizonytalan exception-safety és az, hogy a vezérlési folyamat láthatatlanul ugrik.

A Rust ezt a helyreállítható hibák esetén a **típusrendszerbe emeli**: a hibát visszaadó függvény `Result`-ot ad vissza, és ezt a hívónak explicite kell kezelnie, így a helyreállítható hibák nem láthatatlan kivételként, hanem visszatérési értékként jelennek meg (a `panic!` külön, másféle mechanizmus). A `?` operátor a fejezetben tárgyalt esetekben (`Result`-t vagy `Option`-t visszaadó függvényben) használható.

11.1.2 Fogalmak és szabályok

Definíció. A `Result<T, E>` a standard prelude-ban található összeg-típus:

```
pub enum Result<T, E> {
    Ok(T),
    Err(E),
}
```

Az `Ok(T)` a sikeres érték, az `Err(E)` a hiba. A típus `#[must_use]` attribútummal van ellátva: ha egy `Result` értéket egyszerűen eldobunk, a fordító *figyelmeztet* (`unused_must_use` lint), hogy a hibát nem kezeltük. Ez figyelmeztetés, nem nyelvi tiltás: a `let _ = f();`, a `.ok()` vagy az `unwrap()` megkerüli, és olyan függvény esetén sem véd, amelyet egyáltalán nem hívunk. A C++-ban gyakori „megnéztük-e a hibakódot” hibák ellen tehát erős segítség, de nem teljes kényszer.

Szabály. Helyreállítható hibához `Result`-ot használunk, és a hibát `?`-lel továbbítjuk. A `panic!` nem helyreállítható hibát, programozói hibát vagy sérült invariánst jelez; könyvtári kódban a `panic`-et a legszűkebb körre kell korlátozni.

A `Result` leggyakoribb kombinátorai:

- `map_err`: a hibát alakítja át, például kontextus hozzáadásához;
- `ok_or` / `ok_or_else`: `Option<T>`-ből csinál `Result<T, E>`-t;
- `and_then`: sikeres értéken újabb `Result`-ot visszaadó műveletet hajt végre.

A `?` operátor lényegében egy rövid korai visszatérés:

```
let value = expr?;
```

körülbelül ennek felel meg:

```
let value = match expr {
    Ok(v) => v,
    Err(e) => return Err(From::from(e)),
};
```

A `?` tehát nem „dobja” a hibát, hanem az aktuális függvényből `Err`-rel visszatér. A `From::from(e)` konverzió lehetővé teszi, hogy a belső hibatípus automatikusan a külső függvény hibatípusává alakuljon, ha létezik rá `From` implementáció. Ezt a `From` traitet részletesen a A standard trait-ek: konverziók, formázás, Default, Deref, Borrow alfejezet tárgyalja.

Szabály. A fejezetben tárgyalt esetekben a `?` olyan függvényben használható, amely `Result`-ot vagy `Option`-t ad vissza (és a `main`-ben, ha az `Result`-ot ad). Ha a függvény visszatérési típusa `Result<T, E>`, akkor a `Result`-on használt `?` a belső hibatípust `From`-mal alakítja `E`-vé, vagy azonosnak kell lennie vele. A `?` egy `Option`-ön csak `Option`-t visszaadó függvényben működik; `Result`-ot visszaadó függvényben az `Option`-t előbb `ok_or`/`ok_or_else`-szel kell `Result`-tá alakítani. (A háttérben a `Try` és `FromResidual` trait-ek állnak; a standard típusokon kívüli saját implementációjuk a Rust 1.99.1-ben még instabil (`E0658`, `try_trait_v2`), ezért a könyv ezt nem használja.)

11.1.3 C++ és Rust összevetés

C++-ban a kivétel és a hibakód hagyományosan két külön világ. A C++23 `std::expected<T, E>` már a Rust `Result`-hoz hasonló célú típus, de a C++-ökoszisztéma jelentős része továbbra is kivételalapú.

```
#include <expected>
#include <string>

std::expected<int, std::string> parse(const std::string& s) noexcept;
```

```
fn parse(s: &str) -> Result<i32, String> {
    s.parse::<i32>().map_err(|e| format!("nem szám: {s}: {e}"))
}
```

A két függvény **nem egyenértékű**: a Rust `str::parse::<i32>` szigorúbb, mint a C++ `std::stoi`, és ez szándékos szigorítás. Ellenőrzött viselkedés négy bemenetre:

Bemenet	C++ `std::stoi`	Rust `s.parse::<i32>()`	Rust `s.trim().parse()`	C++ `std::from_chars` (teljes egyezés)
`"42"`	42	`Ok(42)`	`Ok(42)`	42
`" 42"` (vezető szóköz)	42 (átugorja)	`Err(InvalidDigit)`	`Ok(42)`	hiba
`"42abc"`	42 (a maradékot figyelmen kívül hagyja)	`Err(InvalidDigit)`	`Err(InvalidDigit)`	hiba (a `ptr` nem a végén áll)
`"42 7"`	42	`Err(InvalidDigit)`	`Err(InvalidDigit)`	hiba

A Rust `parse` a *teljes* szöveget értelmezi, szóközök és maradék nélkül. A `std::stoi` viselkedését Rustban az első szó parszolása közelíti (`s.split_whitespace().next()`), a Rust-hoz hasonló szigorú C++ megfelelő a `<charconv>` `std::from_chars`-ja azzal a kiegészítéssel, hogy az `ptr` a

szöveg végére kell mutasson. Ne állítsunk tehát automatikus szemantikai egyenértékűséget: a konverzió helyességét a bemeneti szerződés (milyen szövegeket fogadunk el) határozza meg.

A legfontosabb különbség nem a típus létezése, hanem a **nyelvi kultúra és a fordítói garancia**. A Rust `Result` esetén a hiba mindig része a szignatúrának, és a `#[must_use]` figyelmeztet, ha a hívó figyelmen kívül hagyja az eredményt. A C++ `std::expected` is hasonló célt szolgál, de a korábbi kódokban a kivételek, hibakódok és `std::optional` keverednek, a `noexcept` pedig nem írja le a hibatípust.

A Rust `panic` párhuzamba állítható a C++ `assert`/`terminate` viselkedésével, nem pedig a C++ kivételekkel. A kivétel C++-ban gyakran helyreállítható hibát jelez; a Rust `panic` az alapértelmezett `panic = "unwind"` stratégiával lecsévéli a vermet, futtatja a `Drop` implementációkat, majd az aktuális szálat leállítja (a `main` szál pánikja a folyamatot állítja le); `panic = "abort"` mellett az egész folyamat azonnal leáll. A `std::panic::catch_unwind`-dal a pánik elkapható ugyan, de ez nem általános hibakezelési eszköz, hanem elsősorban FFI- vagy szálhatárok védelmére való.

11.1.4 Kidolgozott példa: `parse_command` az `rkv-cli`-ben

Munkakönyvtár: `~/rust-labor/rkv`

A korábbi alfejezetekben elkészült a `Command` enum az Az enum mint összeg-típus és az Option a null helyett alfejezetben, és a kimerítő `match` mintáját a Mintaillesztés: match, if let, let else és a minták alfejezetben alkalmaztuk. Most a `parse_command` függvényt úgy alakítjuk át, hogy hibás bemenet esetén ne pánikoljon, hanem `Result`-ot adjon vissza.

A kiinduló állapot a Copy, Clone és Drop: másolás és RAII Rustban és az A borrow checker működése: non-lexical lifetimes és tipikus javítások ellenőrzési pontja: az `rkv-cli/src/main.rs` `parse_command` függvénye (Mintaillesztés: match, if let, let else és a minták) már `Result<Command, String>`-et ad, de a `?` operátor híján minden hibát `match`-csel és `Result::map`-pel kellett továbbítania. Először nézzük meg, milyen lenne egy *törékeny*, pánikoló értelmező (ez csak szemléltetés, nem a projekt kódja):

Kódblokk — önálló példa · **szándékosan rossz tervezésű**: lefordul, de hiányos bemenetre pánikol

```
use rkv_core::Command;

fn parse_command_fragile(input: &str) -> Command {
    let mut parts = input.split_whitespace();
    match parts.next() {
        Some("GET") => {
            let key = parts.next().unwrap(); // üres GET esetén panic
            Command::Get(key.to_string())
        }
        Some("SET") => {
            let key = parts.next().unwrap();
            let value = parts.next().unwrap(); // a további szavak elvesznek
            Command::Set(key.to_string(), value.to_string())
        }
        // A többi parancs hasonlóan...
        _ => panic!("ismeretlen parancs: {input}"),
    }
}
```

Ez a változat `panic!`-kel áll le, ha a felhasználó hiányos parancsot ad, és a `split_whitespace` miatt a `SET k hello world` értékéből csendben csak a `hello` marad. A Mintaillesztés: match,

if let, let else és a minták alfejezetbeli értelmező mindkét hibát elkerüli; most a `?` operátorral írjuk át, rövidebbre, *ugyanazzal* a viselkedéssel és nyelvtannal:

Kódblokk — fájl: `rkv-cli/src/main.rs` · művelet: a `parse_command` függvény cseréje (a `parse_key`, az `execute`, a `main` és a tesztek változatlanok) · ellenőrzés: `cargo test -p rkv-cli` → a 2 meglévő teszt változatlanul sikeres

```
fn parse_command(line: &str) -> Result<Command, String> {
    let (verb, rest) = match line.split_once(' ') {
        Some((verb, rest)) => (verb, Some(rest)),
        None => (line, None),
    };
    match (verb, rest) {
        ("GET", Some(key)) => Ok(Command::Get(parse_key(key)?)),
        ("DEL", Some(key)) => Ok(Command::Delete(parse_key(key)?)),
        ("SET", Some(rest)) => {
            let (key, value) = rest
                .split_once(' ')
                .ok_or_else(|| "SET requires a key and a value".to_string())?;
            if value.is_empty() {
                return Err("SET requires a non-empty value".to_string());
            }
            if value.contains(['\n', '\r']) {
                return Err("SET value must not contain a line break".to_string());
            }
            Ok(Command::Set(parse_key(key)?, value.to_string()))
        }
        ("LIST", None) => Ok(Command::List(None)),
        ("LIST", Some(prefix)) => Ok(Command::List(Some(parse_key(prefix)?))),
        ("GET" | "DEL", None) => Err(format!("{verb} requires a key")),
        ("SET", None) => Err("SET requires a key and a value".to_string()),
        ("", _) => Err("empty line".to_string()),
        (other, _) => Err(format!("unknown command: {other}")),
    }
}
```

A `parse_key(key)?` `Ok` esetén kibontja a kulcsot, `Err` esetén azonnal visszatér a hibával; az `ok_or_else(...)?` az `Option`-t alakítja `Result`-tá, majd ugyanígy továbbít. A függvény szerződése nem változott, ezért a Mintaillesztés: match, if let, let else és a minták round-trip és „semmi sem vész el csendben” tesztjei változatlanul sikeresek:

```
cargo test -p rkv-cli
cargo run -p rkv-cli -- get "a b"
```

Várt kimenet (a második parancs a standard hibakimenetre ír, kilépési kód 1):

```
test tests::nothing_is_dropped_silently ... ok
test tests::round_trip_on_supported_domain ... ok
ERR invalid key: "a b"
```

A `String` hibatípus itt tanulási célú egyszerűsítés. A következő, Saját hibatípusok: enum-hibák, thiserror, anyhow alfejezetben a könyvtár oldalán strukturált `Error` enumot vezetünk be, amely implementálja a `std::error::Error` traitet, és így kompatibilis a standard hibakezelő eszközökkel.

Ellenőrzési pont. Az `rkv-cli` `parse_command`-ja a `?` operátort használja; a viselkedés, a `cargo test --workspace` eredménye és az `rkv-core` változatlan.

Továbbvitt eredmény. A `parse_command` függvény (`Result<Command, String>`) a `?` operátorral továbbítja a hibákat, a Mintaillesztés: match, if let, let else és a minták nyelvtana szerint. Ezt a `?` + `From` mintát használja tovább a Saját hibatípusok: enum-hibák, thiserror, anyhow,

az Iterátorok: lusta adapterek és saját iterátor és a Future-ök és async/await: mi történik a motorháztető alatt alfejezet.

11.1.5 A `?` és a `From` konverzió

A `?` akkor a leghasznosabb, ha a belső hibatípus nem egyezik a külső hibatípussal. A konverziót a `From` trait biztosítja. Például egy fájlt olvasó függvény `Box<dyn std::error::Error>` hibát ad vissza:

```
fn first_line(path: &str) -> Result<String, Box<dyn std::error::Error>> {
    let text = std::fs::read_to_string(path)?;
    Ok(text.lines().next().unwrap_or("").to_string())
}
```

Itt a `std::fs::read_to_string` `std::io::Error`-t ad vissza, de a `?` automatikusan `Box<dyn std::error::Error>`-rá konvertálja, mert a standard könyvtárban létezik `impl From<std::io::Error> for Box<dyn std::error::Error>`. A függvény hibatípusa tehát egységes marad, a `?` pedig nem igényel kézi `map_err`-t.

Ha a cél `String` hibatípus, az `std::io::Error`-ból nincs ilyen `From` implementáció, ezért ott a `map_err` a megfelelő eszköz:

```
fn parse_positive(s: &str) -> Result<i32, String> {
    let n: i32 = s.parse()
        .map_err(|e| format!("nem szám: {s}: {e}"))?;
    if n > 0 {
        Ok(n)
    } else {
        Err("nem pozitív szám".to_string())
    }
}
```

11.1.6 `unwrap` és `expect`: mikor elfogadható

Az `unwrap` és az `expect` az `Ok` értéket csomagolja ki, és `Err` esetén pánikot okoz. Különbségük, hogy az `expect` egyéni hibaüzenetet kap, ami a pánik okának lokalizálását segíti:

```
let cmd = parse_command("SET k v").expect("a beépített parancs érvényes");
```

Szabály. `unwrap`/`expect` elsősorban példában, tesztben, prototípusban, illetve olyan helyen elfogadható, ahol egy külső invariáns garantálja, hogy az érték nem lehet `Err`. Könyvtári kódban a hibát inkább `Result`-tal vagy a `?`-lel kell továbbítani. Az `expect` mindig előnyösebb az `unwrap`-nál, mert legalább a programozó szándékát dokumentálja.

A pánik viselkedését a `Cargo.toml` profiljai is befolyásolják. Alapértelmezés szerint `panic = "unwind"`: a verem lecsévélődik, a `Drop` implementációk lefutnak, majd az aktuális szál leáll. A `panic = "abort"` beállítás az egész folyamatot azonnal leállítja; ez kisebb kódot eredményez, és például beágyazott rendszereken vagy speciális telepítésekben hasznos. A pontos profilbeállítás a Rust Cargo-referenciájában ellenőrizhető.

11.1.7 Tipikus hibák és átállási csapdák

11.1.7.1 `unwrap` mindenhol a `?` helyett

A C++-fejlesztő ösztönösen `unwrap`-pel akarja „kicsomagolni” az értéket, mert a kivétel esetén úgysem kell minden szinten kezelni. Ez azonban könyvtári kódban pánikot okoz, és a hibát kivonja a típusrendszerből:

```
let key = parts.next().unwrap(); // hibás bemenetnél panic
```

Helyette:

```
let key = parts.next().ok_or_else(|| "GET kulcs nélkül".to_string())?;
```

11.1.7.2 `?` használata nem `Result`-ot visszaadó függvényben

A `?` nem nyelvi „kivételdobás”, hanem korai `return`. Ez a változat szándékosan nem fordul le:

```
fn main() {
    let cmd = parse_command("SET k v")?;
    println!("{}", cmd.to_line());
}
```

A fordító hibakódja `E0277`, a lényege pedig az, hogy a `?` csak `Result`-ot vagy `Option`-t visszaadó függvényben használható. A javítás:

```
fn main() -> Result<(), String> {
    let cmd = parse_command("SET k v")?;
    println!("{}", cmd.to_line());
    Ok(())
}
```

11.1.7.3 `catch_unwind` mint kivételkezelő

C++-os reflex lehet a pánikok `catch_unwind`-dal való elkapása, mintha azok kivételek lennének. Ez Rustban nem idiomatikus. A `catch_unwind` elsősorban FFI-határokra, szálkezelésre és tesztkörnyezetekre való; a helyreállítható hibákat `Result`-tal, a programozói hibákat pedig javítással, nem elkapással kell kezelni.

11.1.8 Feladatok

11.1.8.1 1. Fogalmi kérdés

Lefordul-e a következő Rust-kód, és ha nem, miért?

```
fn f() -> Result<i32, String> {
    let x: Option<i32> = Some(42);
    Ok(x?)
}
```

11.1.8.2 2. C++-kód átírása idiomatikus Rustra

Írd át a következő C++ függvényt Rustra `Result`/`Option` felhasználásával, ne kivétellel:

```cpp
#include <string>

int parse_or_default(const std::string& s, int default_value) noexcept {
    try {
        return std::stoi(s);
    } catch (...) {
        return default_value;
    }
}
```

11.1.8.3 3. Implementáció az `rkv-cli`-ben

Írj egy `fn parse_script(text: &str) -> Result<Vec<Command>, String>` függvényt az `rkv-cli/src/main.rs`-be, amely egy többsoros szöveg minden nem üres sorát a `parse_command`-dal értelmezi, és az első hibás sornál a sor számát is tartalmazó hibával áll meg. Használd a `?` operátort, és írj hozzá unit tesztet.

11.1.8.4 4. Hibajavítás

Az alábbi függvény üres sztring esetén pánikol. Írd át `Result<String, &'static str>` visszatérésűre úgy, hogy a hiba a hívóhoz kerüljön:

```rust
fn third_word(s: &str) -> String {
    s.split_whitespace().nth(2).unwrap().to_string()
}
```

11.1.9 Megoldások

11.1.9.1 1. megoldás

Nem fordul le (`E0277`: the `?` operator can only be used on `Results`, not `Options`, in a function that returns `Result`). A `?` egy `Option`-t csak `Option`-t visszaadó függvényben tud kibontani; `Result`-ot visszaadó függvényben nincs automatikus átalakítás, és nem is `From` implementációval javítható. A tanított átalakítás az `ok_or`/`ok_or_else`, amely a hiányzó értékhez hibát rendel. Helyesen:

```rust
fn f() -> Result<i32, String> {
    let x: Option<i32> = Some(42);
    Ok(x.ok_or_else(|| "hiányzó érték".to_string())?)
}
```

11.1.9.2 2. megoldás

```rust
fn parse_or_default(s: &str, default_value: i32) -> i32 {
    s.parse::<i32>().unwrap_or(default_value)
}
```

A `parse` `Result<i32, ParseIntError>`-t ad vissza, az `unwrap_or` a `Result` `Err` ágában a megadott alapértéket adja. Ha a `ParseIntError` értékre szükségünk lenne, `map_err`-rel kontextust adhatnánk a hibához.

11.1.9.3 3. megoldás

Kódblokk — fájl: `rkv-cli/src/main.rs` · művelet: beillesztés a `parse_command` után, a teszt a `mod tests` blokkba · ellenőrzés: `cargo test -p rkv-cli`

```
fn parse_script(text: &str) -> Result<Vec<Command>, String> {
    let mut commands = Vec::new();
    for (index, line) in text.lines().enumerate() {
        if line.is_empty() {
            continue;
        }
        let command = parse_command(line).map_err(|e| format!("{}. sor: {e}", index + 1))?;
        commands.push(command);
    }
    Ok(commands)
}

// a mod tests blokkba:
    #[test]
    fn script_reports_line_number() {
        assert_eq!(parse_script("SET a 1\n\nGET a").map(|c| c.len()), Ok(2));
        assert_eq!(
            parse_script("SET a 1\nGET a b"),
            Err("2. sor: invalid key: \"a b\"".to_string())
        );
        // A maradék tokeneket minden egyargumentumos parancsnál a `parse_key` utasítja el:
        assert!(parse_script("GET k extra").is_err());
        assert!(parse_script("LIST p extra").is_err());
        assert!(parse_script("DEL k extra").is_err());
        // A SET értéke a sor maradéka, ezért a többletszó az érték része:
        assert_eq!(parse_script("SET k v extra").map(|c| c.len()), Ok(1));
    }
```

A `map_err` a hibát a sor számával egészíti ki, a `?` pedig az első hibánál kilép. A „maradék token” ellenőrzés a nyelvtan része: `GET k extra`, `DEL k extra` és `LIST p extra` is `Err`, mert a kulcs a sor maradéka, és a `parse_key` szóközt nem fogad el (ez már a Mintaillesztés: match, if let, let else és a minták parser szerződése, nem ennek a feladatnak az újdonsága); a `SET` értéke viszont szándékosan a sor maradéka, ezért ott a többletszó az érték része. A `main`-hez nem kell nyúlni; a következő alfejezet egy `--script` kapcsolóval köti be a parancsfájlt.

11.1.9.4 4. megoldás

```
fn third_word(s: &str) -> Result<String, &'static str> {
    s.split_whitespace()
        .nth(2)
        .map(|w| w.to_string())
        .ok_or("nincs harmadik szó")
}
```

Az `Option<String>` `ok_or`-ral `Result<String, &'static str>`-rá alakul, így a hiba nem pánikot, hanem `Err` értéket ad.

11.1.10 Rövid összefoglaló

A `Result<T, E>` a helyreállítható hibák típusba emelt formája: `Ok(T)` a siker, `Err(E)` a hiba. A `?` operátor `Ok` esetén kibontja az értéket, `Err` esetén pedig `return Err(From::from(e))`-ként viselkedik; ezért a `?` a `From` konverzión keresztül egységesíti a hibatípusokat, de csak

`Result`/`Option` visszatérésű függvényben használható. Az `unwrap`/`expect` pánikot okoznak; példában, tesztben és garantált invariáns mellett elfogadhatók, könyvtári kódban viszont `Result`-ot használunk. A `panic!` nem a C++-kivételek megfelelője: nem általános hibakezelésre való, hanem programozói hibák és sérült invariánsok jelzésére. A `parse_command` az `rkv-cli/src/main.rs` fájlban `Result<Command, String>` visszatérésű lett; a `String` hibatípust a következő alfejezet strukturált `Error` enumra cseréli.

11.1.11 Források és ellenőrzési hiányok

A nyelvi szabályok a Rust Reference (https://doc.rust-lang.org/reference/) és a Rust Book (https://doc.rust-lang.org/book/) hibakezelésről szóló részein alapulnak. A hibaüzenet-szövegek a rögzített toolchainre (Rust 1.99.1) vonatkoznak. A `std::error::Error`, a `From` implementációk és a `panic` profilbeállítások részletei a standard könyvtár és a Cargo Book dokumentációjában találhatók (https://doc.rust-lang.org/std/error/trait.Error.html, https://doc.rust-lang.org/cargo/reference/profiles.html).

11.2 Saját hibatípusok: enum-hibák, thiserror, anyhow

11.2.1 Motiváció

C++-ban a hibakezelés gyakran kivételhierarchiákon alapul: `std::runtime_error`, `std::ios_base::failure`, saját `store_error` osztályok. A hívó fél számára azonban nem mindig világos, hogy egy függvény pontosan milyen kivételeket dobhat, és a `catch` blokkok sorrendje, illetve a `catch (...)` gyakran elfedi a hibák valódi típusát. Ha egy `Store::load` függvény fájlművelet közben `std::ios_base::failure`-t, rossz adatformátum esetén `std::runtime_error`-t dob, a hívó kód csak a `what()` szövegére hagyatkozhat, vagy bonyolult `dynamic_cast` vizsgálatokat írhat.

A Rust ezt a problémát a hibák **típusként való modellezésével** oldja meg: egy függvény `Result<T, E>`-t ad vissza, ahol `E` általában egy saját `enum`. A hívó `match` segítségével pontosan meg tudja különböztetni az egyes hibaeseteket, a fordító pedig ellenőrzi, hogy minden lehetséges variánst kezelünk-e. A `?` operátorral a hibák automatikusan konvertálhatók a saját típusunkra, ha biztosítjuk a megfelelő `From` implementációkat. Ebben az alfejezetben megtervezzük az rkv könyvtár saját `Error` típusát, és bevezetjük az alkalmazásoldali `anyhow`-t a kontextusgazdag hibakezeléshez.

11.2.2 Fogalmak és szabályok

Definíció (hiba trait). A Rust standard könyvtárában a `std::error::Error` trait jelöli azokat a típusokat, amelyek hibaként használhatók: `pub trait Error: Debug + Display`. A `Debug` és a `Display` tehát *szupertrait*-korlátok, nem az `Error` metódusai: egy típus csak akkor lehet `Error`, ha a `Debug` traitet (diagnosztikai kiírás, általában `#[derive(Debug)]`) és a `Display` traitet (az ember által olvasható leírás) is implementálja. Magának az `Error` traitnek a fontos, felülírható metódusa a `source()`, amely a hibát kiváltó alacsonyabb szintű hibát adja vissza, ha van ilyen; alapértelmezett implementációja `None`.

Definíció (hibalánc, source-lánc). Ha egy magasabb szintű hiba egy alacsonyabb szintű hibát burkol be – például egy fájlműveleti hiba egy `io::Error`-t –, akkor a két hibát láncba fűzhetjük. A Rust `Error` trait `source()` metódusa ezt a láncot teszi bejárhatóvá: minden hiba

visszaadhatja az őt kiváltó okot. A `Display` implementáció általában a közvetlen hibát írja le, a lánc további elemeit a leképező eszközök (pl. `anyhow {:#}` formázása) jelenítik meg.

Szabály (könyvtár vs. alkalmazás hibastratégiája). Könyvtári kódban **saját, strukturált hibatípust** (enum) használjunk, amely pontosan felsorolja a lehetséges hibaeseteket, és `From` implementációkkal támogatja a `?` operátort. Alkalmazásban (CLI, szerver) gyakran elegendő az `anyhow::Result<T>` (vagy `anyhow::Error`), amely típuserázott hibákat kezel, és kontextust adhatunk hozzájuk. A kettő közötti határ a hívó fél szerződése: ha a hívónak programozottan kell döntenie a hibáról, használjunk enumot; ha csak emberi olvasásra és naplózásra kell, az `anyhow` praktikusabb.

Szabály (a `?` operátor From-konverziója). A `?` operátor egy `Result<T, E>` értéken meghívva a hiba esetén megkísérli a hibát `From<E> for F` segítségével konvertálni a környező függvény `F` hibatípusára. Ezért ha a hibatípusunkra implementáljuk a `From<io::Error>`-t, akkor a `File::open(...)?` kifejezés automatikusan a mi hibánkká alakítja az I/O hibát.

11.2.3 C++ és Rust összevetés

Tekintsünk egy C++ függvényt, amely egy fájlból olvas be egy egész számot, és különböző hibákat dob:

```cpp
#include <fstream>
#include <stdexcept>
#include <string>

class parse_error : public std::runtime_error {
public:
    explicit parse_error(const std::string& msg) : std::runtime_error(msg) {}
};

int load_count(const std::string& path) {
    std::ifstream file(path);
    if (!file.is_open()) {
        throw std::ios_base::failure("cannot open file");
    }
    int value = 0;
    file >> value;
    if (file.fail()) {
        throw parse_error("invalid integer");
    }
    return value;
}
```

A hívónak két különböző kivételt kell elkapnia, ráadásul ezek típusa nem derül ki a függvény szignatúrájából. (Figyelem, a két változat nem azonos szerződésű: a C++ `file >> value` az első szóközzel elválasztott szót olvassa, a maradékot figyelmen kívül hagyja, tehát a `"42abc"` és a `"42 7"` tartalmú fájl is 42-t ad; a lenti Rust változat a teljes, `trim`-melt tartalmat értelmezi, ezért ugyanazokat `ParseIntError`-ral utasítja el. Ez szándékos szigorítás; az első szó olvasásához `content.split_whitespace().next()` kell — de a `parse` a `"42abc"`-t ezután is elutasítja; a C++ prefix-olvasás Rust-megfelelője a számjegyek előzetes kiválasztása (`take_while(char::is_ascii_digit)`), majd `parse`.) A Rust megfelelője saját hibát enummal írja le:

```rust
use std::{fmt, fs, io, num::ParseIntError};

#[derive(Debug)]
enum LoadError {
    Io(io::Error),
```

```
    Parse(ParseIntError),
}

impl fmt::Display for LoadError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            LoadError::Io(e) => write!(f, "I/O hiba: {e}"),
            LoadError::Parse(e) => write!(f, "számformátum hiba: {e}"),
        }
    }
}

impl std::error::Error for LoadError {
    fn source(&self) -> Option<&(dyn std::error::Error + 'static)> {
        match self {
            LoadError::Io(e) => Some(e),
            LoadError::Parse(e) => Some(e),
        }
    }
}

// A ? operátor ezeken keresztül alakítja át a két alacsonyabb szintű hibát.
impl From<io::Error> for LoadError {
    fn from(e: io::Error) -> Self {
        LoadError::Io(e)
    }
}

impl From<ParseIntError> for LoadError {
    fn from(e: ParseIntError) -> Self {
        LoadError::Parse(e)
    }
}

fn load_count(path: &str) -> Result<i32, LoadError> {
    let content = fs::read_to_string(path)?;   // io::Error -> LoadError
    let value = content.trim().parse::<i32>()?; // ParseIntError -> LoadError
    Ok(value)
}
```

A `?` operátor automatikusan konvertálja a hibákat, mert a `LoadError` implementálja a két `From`-ot; ha bármelyiket elhagynánk, a megfelelő `?` sor `E0277`-tel (`the trait `From<io::Error>` is not implemented for `LoadError``) nem fordulna le. A hívó `match`-csel pontosan kezelheti a két hibautat:

```
fn main() {
    match load_count("count.txt") {
        Ok(n) => println!("érték: {n}"),
        Err(LoadError::Io(e)) => eprintln!("nem olvasható: {e}"),
        Err(LoadError::Parse(e)) => eprintln!("rossz formátum: {e}"),
    }
}
```

A fenti kézzel írt `Display`, `Error` és `From` implementáció azonban sok boilerplate; a `thiserror` derive ezt jelentősen lerövidíti, amint a laborban látni fogjuk.

11.2.4 Labor: az rkv `Error` típusa és anyhow a CLI-ben

Ebben a laborban az `rkv-core` könyvtár hibatípusát (`rkv-core/src/error.rs`, Modulok és láthatóság: header nélküli világ) a `thiserror` crate-tel a végleges, *kanonikus* alakjára hozzuk, és

bevezetjük az `anyhow`-t a `rkv-cli` binárisban. A kiinduló állapot a Result, a ? operátor és a panic helye ellenőrzési pontja.

Munkakönyvtár: `~/rust-labor/rkv`

Migráció. A Modulok és láthatóság: header nélküli világ alfejezetbeli `Error` két variánsa (`KeyNotFound`, `Io`) helyett a kanonikus `Error` a következő: `Io(io::Error)`, `KeyNotFound(String)` (most már hordozza a hiányzó kulcsot), `InvalidCommand(String)` (a Modulok és láthatóság: header nélküli világ 3. feladatának `Parse` variánsa ezen a néven él tovább). Az I/O és szerializáció: fájlok, pufferelés, serde alfejezet egyetlen variánst ad hozzá (`Serialization`); ettől kezdve az `Error` nem változik, és minden crate ezt használja. A meglévő kódban az `Error`-t egyedül a `Store::flush` említi, és csak a típusnevén, ezért a csere más fájlt nem érint. (Aki megoldotta a Modulok és láthatóság: header nélküli világ 3. feladatát, a `command.rs`-ben az `Error::Parse(...)` hívásokat `Error::InvalidCommand(...)`-ra cseréli.)

11.2.4.1 1. lépés: Függőségek hozzáadása

Kódblokk — fájl: `rkv-core/Cargo.toml` · művelet: beillesztés a `[dependencies]` szakaszba · ellenőrzés: `cargo build -p rkv-core`

```
thiserror = "2"
```

Kódblokk — fájl: `rkv-cli/Cargo.toml` · művelet: beillesztés a `[dependencies]` szakaszba · ellenőrzés: `cargo build -p rkv-cli`

```
anyhow = "1"
```

A könyv a `thiserror` 2-es főverziójával készült (a `Cargo.lock` a pontos verziót rögzíti); az itt használt derive-attribútumok az 1-es főverzióban is azonosak.

11.2.4.2 2. lépés: Az `rkv-core` hibatípusa

Kódblokk — fájl: `rkv-core/src/error.rs` · művelet: teljes csere · ellenőrzés: `cargo build -p rkv-core`

```
// rkv-core/src/error.rs
use std::io;
use thiserror::Error;

/// Az rkv tároló hibáinak típusa.
#[derive(Debug, Error)]
pub enum Error {
    /// Fájl- vagy egyéb I/O művelet hibája.
    #[error("I/O hiba: {0}")]
    Io(#[from] io::Error),

    /// A kért kulcs nem található a tárolóban.
    #[error("kulcs nem található: {0}")]
    KeyNotFound(String),

    /// Érvénytelen parancs vagy protokollüzenet.
    #[error("érvénytelen parancs: {0}")]
    InvalidCommand(String),
}

/// Az rkv-core műveleteinek eredménytípusa.
pub type Result<T> = std::result::Result<T, Error>;
```

A `#[derive(Error)]` implementálja a `std::error::Error` traitet, a `#[error("...")]` attribútumok a `Display`-et. A `#[from]` az `Io` variánson generálja a `From<io::Error> for Error` implementációt, így a `?` operátor közvetlenül használható minden `io::Result<T>`-t visszaadó hívásnál, és a `source()` is az eredeti `io::Error`-t adja vissza, tehát a hibalánc megmarad. A `Result<T>` alias a gyakori `Result<T, Error>` rövidítése; a crate gyökerében is exportáljuk:

Kódblokk — fájl: `rkv-core/src/lib.rs` · művelet: a `pub use error::Error;` sor cseréje · ellenőrzés: `cargo build --workspace`

```
pub use error::{Error, Result};
```

Így a crate-en belül `crate::Result<T>`, kívülről `rkv_core::Result<T>` használható.

11.2.4.3 3. lépés: Kontextus az anyhow-val a CLI-ben

A CLI eddig minden futtatásnál egyetlen parancsot hajtott végre egy üres tárolón. Most egy `--script <fájl>` kapcsolót adunk hozzá: a fájl minden sora egy protokollparancs, amelyet a CLI a parancssori parancs *előtt*, ugyanazon a tárolón hajt végre. Így egy folyamaton belül több parancs is futhat, és két valódi hibaút is keletkezik: a fájl olvasása (`io::Error`) és egy hibás sor (`Error::InvalidCommand`). Mindkettőhöz az `anyhow` ad kontextust.

Kódblokk — fájl: `rkv-cli/src/main.rs` · művelet: a `use` sorok, a `Cli` struktúra és a `main` cseréje, új `run_script` függvény; a `CliCommand`, `parse_key`, `parse_command`, `execute` és a tesztek változatlanok · ellenőrzés: `cargo test -p rkv-cli`

```
use std::path::{Path, PathBuf};

use anyhow::{Context, Result};
use clap::{Parser, Subcommand};
use rkv_core::{Command, Error, Store};

/// rkv parancssori kliens
#[derive(Parser)]
#[command(name = "rkv-cli", version, about)]
struct Cli {
    /// Parancsfájl: soronként egy protokollparancs, a megadott parancs előtt fut le
    #[arg(long)]
    script: Option<PathBuf>,
    #[command(subcommand)]
    command: CliCommand,
}

// ... CliCommand, parse_key, parse_command és execute változatlanul ...

/// Végrehajtja a parancsfájl sorait; az üres sorokat átugorja.
fn run_script(store: &mut Store, path: &Path) -> Result<()> {
    let text = std::fs::read_to_string(path)
        .with_context(|| format!("a(z) {} parancsfájl olvasása", path.display()))?;
    for (index, line) in text.lines().enumerate() {
        if line.is_empty() {
            continue;
        }
        let command = parse_command(line)
            .map_err(Error::InvalidCommand)
            .with_context(|| format!("{}:{}", path.display(), index + 1))?;
        println!("{}", execute(store, command));
    }
    Ok(())
}
```

```
fn main() -> Result<()> {
    let cli = Cli::parse();
    let mut store = Store::new();
    if let Some(path) = &cli.script {
        run_script(&mut store, path)?;
    }
    let command = match cli.command {
        CliCommand::Get { key } => Command::Get(key),
        CliCommand::Set { key, value } => Command::Set(key, value),
        CliCommand::Del { key } => Command::Delete(key),
        CliCommand::List { prefix } => Command::List(prefix),
    };
    let command = parse_command(&command.to_line())
        .map_err(Error::InvalidCommand)
        .context("érvénytelen parancssori argumentum")?;
    println!("{}", execute(&mut store, command));
    Ok(())
}
```

A `with_context` (és a `context`) az `anyhow::Context` traitből származik: ha a `Result` `Err`, a hibát `anyhow::Error`-rá alakítja, és elé fűzi a megadott szöveges kontextust; az eredeti hiba a lánc következő eleme (`source`) marad. A `with_context` lusta: a formázott üzenet csak hiba esetén készül el. A `main` `anyhow::Result<()>`-t ad vissza; ha `Err`-rel tér vissza, a program a hibát és a teljes láncát `Error: ... Caused by: ...` formában a standard hibakimenetre írja, és 1-es kilépési kóddal áll le.

Próbáljuk ki egy működő és két hibás úttal:

```
printf 'SET name Alice\nSET city Szeged\n' > init.txt
cargo run -q -p rkv-cli -- --script init.txt get name
cargo run -q -p rkv-cli -- --script nincs.txt get name
printf 'SET name Alice\nGET name extra\n' > rossz.txt
cargo run -q -p rkv-cli -- --script rossz.txt get name
```

Várt kimenet (a `cargo` saját sorai nélkül; az `Error:` kezdetű blokkok a standard hibakimenetre kerülnek, a kilépési kód 1; az `os error 2` szövege rendszerfüggő):

```
OK
OK
VALUE name Alice
Error: a(z) nincs.txt parancsfájl olvasása

Caused by:
    No such file or directory (os error 2)
OK
Error: rossz.txt:2

Caused by:
    érvénytelen parancs: invalid key: "name extra"
```

Az első futtatás mutatja, hogy egy folyamaton belül a `SET` után a `GET` már látja az értéket. A második és a harmadik futtatás a két hibautat: a kontextus megmondja, *mit* csináltunk (melyik fájlt olvastuk, melyik sorban jártunk), a lánc következő eleme pedig, hogy *mi* romlott el.

11.2.4.4 4. lépés: Teszt a hibatípushoz

Kódblokk — fájl: `rkv-core/src/error.rs` · művelet: beillesztés a fájl végére · ellenőrzés: `cargo test -p rkv-core` → 7 sikeres teszt

```rust
#[cfg(test)]
mod tests {
    use super::*;
    use std::error::Error as _;

    #[test]
    fn io_error_converted_to_error() {
        let io_err = io::Error::new(io::ErrorKind::NotFound, "file not found");
        let err: Error = io_err.into();
        assert!(matches!(err, Error::Io(_)));
        // A forráslánc megmarad: a source az eredeti io::Error.
        assert_eq!(err.source().unwrap().to_string(), "file not found");
    }

    #[test]
    fn display_contains_message() {
        let err = Error::KeyNotFound("k1".into());
        let text = err.to_string();
        assert!(text.contains("kulcs nem található"));
        assert!(text.contains("k1"));
    }
}
```

Ellenőrzési pont. Az `rkv-core` kanonikus `Error` típusa (`Io`, `KeyNotFound(String)`, `InvalidCommand(String)`) és a `Result<T>` alias a crate gyökeréből is elérhető. Az `rkv-cli` minden parancsa működik, a `--script` kapcsoló egy folyamaton belül több parancsot futtat, és a hibák kontextussal együtt jelennek meg. A `cargo test --workspace` $7 + 1 + 2$ tesztje (az `rkv-core` unit tesztjei, az `update_prefix` integrációs teszt és az `rkv-cli` tesztjei) sikeres.

11.2.5 Tipikus hibák és átállási csapdák

1. **`unwrap`/`expect` a könyvtárban.** C++-ban megszokott, hogy hibás fájlműveletnél kivételt dobunk; Rustban a `File::open(path).expect("nem nyitható")` pánikot okoz a könyvtári kódban, ami a hívó alkalmazás összeomlásához vezet. Ehelyett minden felhasználói bemenettől függő műveletnek `Result`-ot kell visszaadnia, és a `?` operátort kell használni. A pánik csak ténylegesen helyrehozhatatlan programozói hibánál elfogadható (pl. belső invariáns megsértése).

2. **`anyhow` használata könyvtári API-ban.** Az `anyhow::Error` típuserázott: a hívó nem tud enumvariánsokra statikusan, kimerítően `match`-elni. Programozottan dönteni mégis lehet `downcast_ref`-fel és a hibalánc bejárásával (`chain()`), de ez futásidejű, nem ellenőrzi a fordító. Ezért könyvtári API-ban a saját enum a megfelelő alapértelmezés (nem kivétel nélküli szabály); az alkalmazásban az `anyhow` már elfogadható, mert ott a hibák emberi fogyasztásra és naplózásra valók.

3. **`Box<dyn Error>` örökölt reflexe.** C++-ban a `std::exception` polimorfikus hivatkozás a természetes; Rustban a `Box<dyn Error>` hasonló, de típuserázott: a `match` nem tud variánsokra illeszkedni. A konkrét típus a `downcast_ref::<T>()`-tel *utólag* kideríthető: `e.downcast_ref::<io::Error>()` egy `Option<&io::Error>`-t ad, amelyből a `map(io::Error::kind)` a hibakategóriát (`Some(ErrorKind::NotFound)`) nyeri; a `downcast_ref::<MyErr>()` `None`-t ad, ha a hiba nem ilyen típusú. Ez futásidejű típusvizsgálat: csak a feltételezett típusokat lehet végigpróbálni, és a fordító nem ellenőrzi a teljességet. Ezért a könyvtár a strukturált enumot részesíti előnyben: az enum pontos szerződés (a fordító jelzi, ha egy variánst nem kezelünk), a `Box<dyn Error>` és az `anyhow` pedig az alkalmazás szintjére való.

4. **A `map_err` és a `source()` láncának félreértése.** A `source()` lánc nem a `map_err`-től vagy a `From`-tól függ, hanem attól, hogy a hibatípus `Error::source` metódusa visszaadja-e a burkolt hibát. Ugyanarra az `Error::Io` variánsra `map_err(Error::Io)` és `From` + `?` használatával azonos `source()` lánc adódik (ellenőrzött kimenet mindkettőre: `source = nincs fájl`). A `thiserror` `#[from]` attribútuma a `From` implementációt generálja, a `#[source]` (és a `#[from]` implicit módon) pedig a `source()` metódust.

11.2.6 Feladatok

11.2.6.1 a) Fogalmi kérdés

Egy könyvtár fejlesztője a következő függvényt írja:

```
fn load_config(path: &str) -> Result<String, Box<dyn std::error::Error>> {
    let content = std::fs::read_to_string(path)?;
    Ok(content)
}
```

Miért nem ideális ez a hibatípus egy olyan könyvtárban, ahol a hívónak meg kell különböztetnie a „fájl nem található” és a „JSON formátum hiba” eseteket? Hogyan lehetne jobbá tenni? Fogalmazd meg a választ a könyvtár vs. alkalmazás stratégia alapján.

11.2.6.2 b) C++-kód átírása idiomatikus Rustra

Adott a C++ függvény:

```
#include <stdexcept>
#include <fstream>

int read_seed(const std::string& path) {
    std::ifstream file(path);
    if (!file) throw std::runtime_error{"cannot open seed file"};
    int seed;
    file >> seed;
    if (file.fail()) throw std::runtime_error{"invalid seed format"};
    return seed;
}
```

Írd át Rustra a laborban bemutatott `Error` enum mintájára: hozz létre egy `SeedError` enumot a `thiserror` használatával, amely `io::Error`-t és `ParseIntError`-t tud burkolni, majd írd meg a `read_seed` függvényt `Result<i32, SeedError>` visszatéréssel. Használd a `?` operátort.

11.2.6.3 c) Implementációs feladat az rkv-ban

Bővítsd az `rkv-core` `Error` enumját egy `Serialization` variánssal, amely `serde_json::Error`-t fogad be `#[from]`-mal. Ehhez vedd fel a `serde_json = "1"` függőséget az `rkv-core/Cargo.toml`-be. Írj egy függvényt `parse_record(data: &str) -> Result<(), Error>` néven, amely a `serde_json::from_str::<serde_json::Value>(data)?` kifejezést használja, és sikeres parse esetén `Ok(())`-t ad. Futtasd a teszteket.

11.2.6.4 d) Hibajavítás és refaktorálás

Tekintsd a következő Rust kódot:

```rust
fn process_file(path: &str) -> Result<(), Box<dyn std::error::Error>> {
    let data = std::fs::read_to_string(path)?;
    let value: i32 = data.trim().parse()?;
    println!("{value}");
    Ok(())
}
```

Refaktoráld a `process_file` függvényt úgy, hogy a laborban definiált `rkv_core::Error`-t használja, és adj hozzá kontextust az `anyhow` segítségével egy CLI-hívásban: ha a fájl nem nyitható, a hibaüzenet tartalmazza a fájl elérési útját. (Tipp: a `process_file`-t hívó kód `anyhow::Result<()>`-t adjon vissza, és használja a `with_context` metódust.)

11.2.7 Megoldások

11.2.7.1 a) Megoldás

A `Box<dyn Error>` típuserázott: a hívó a fordítási időben nem tudja, hogy a hibát `io::Error` vagy `serde_json::Error` okozta-e. A `match` nem tud különbséget tenni; a konkrét típus csak a `downcast_ref::<T>()` futásidejű próbálgatásával, illetve a `source()` lánc bejárásával derül ki, ami körülményes és nem teljességellenőrzött. Könyvtárban saját enumot kell használni, például:

```rust
#[derive(Debug, thiserror::Error)]
pub enum ConfigError {
    #[error("I/O hiba: {0}")]
    Io(#[from] std::io::Error),
    #[error("JSON hiba: {0}")]
    Json(#[from] serde_json::Error),
}
```

A hívó így `match`-csel pontosan kezelheti a variánsokat. Az `anyhow::Error` csak alkalmazásban ajánlott, ahol nincs szükség programozott döntésre.

11.2.7.2 b) Megoldás

```rust
use std::{fs, io, num::ParseIntError};
use thiserror::Error;

#[derive(Debug, Error)]
enum SeedError {
    #[error("I/O hiba a seed fájlnál: {0}")]
    Io(#[from] io::Error),

    #[error("érvénytelen seed formátum: {0}")]
    Parse(#[from] ParseIntError),
}

fn read_seed(path: &str) -> Result<i32, SeedError> {
    let content = fs::read_to_string(path)?;
    let seed = content.trim().parse::<i32>()?;
    Ok(seed)
}
```

A `?` operátor automatikusan konvertálja az `io::Error`-t és a `ParseIntError`-t a `SeedError` megfelelő variánsaira a generált `From`-ok révén.

11.2.7.3 c) Megoldás

Az `rkv-core/Cargo.toml`:

```toml
[dependencies]
thiserror = "2"
serde_json = "1"
```

Az `rkv-core/src/error.rs` frissített változata:

```rust
use std::io;
use thiserror::Error;

#[derive(Debug, Error)]
pub enum Error {
    #[error("I/O hiba: {0}")]
    Io(#[from] io::Error),

    #[error("kulcs nem található: {0}")]
    KeyNotFound(String),

    #[error("érvénytelen parancs: {0}")]
    InvalidCommand(String),

    #[error("szerializációs hiba: {0}")]
    Serialization(#[from] serde_json::Error),
}
```

A `parse_record` függvény az `rkv-core/src/lib.rs`-ben vagy egy új modulban:

```rust
pub fn parse_record(data: &str) -> Result<(), Error> {
    // A `?` automatikusan Error::Serialization-vá alakítja a serde_json hibát.
    let _: serde_json::Value = serde_json::from_str(data)?;
    Ok(())
}
```

A `cargo test -p rkv-core` lefuttatása után minden tesztnek sikeresnek kell lennie.

11.2.7.4 d) Megoldás

A könyvtári függvény saját hibával:

```rust
// rkv-core/src/lib.rs (vagy egy megfelelő modulban)
pub fn process_file(path: &str) -> Result<(), Error> {
    let data = std::fs::read_to_string(path)?;
    let value: i32 = data.trim().parse().map_err(|_| Error::InvalidCommand("nem szám".into()))?;
    println!("{value}");
    Ok(())
}
```

A parse hiba `ParseIntError`, amit egyelőre `InvalidCommand`-ra képezünk le, mert az `Error`-ban nincs parse variáns. (A későbbi fejezetekben ezt finomíthatjuk.)

A CLI-ben:

```rust
use anyhow::Context;
use rkv_core::process_file;

fn main() -> anyhow::Result<()> {
    let path = "/tmp/rkv-seed.txt";
    process_file(path).with_context(|| format!("fájl feldolgozása: {path}"))?;
```

```
    Ok(())
}
```

Ha a fájl nem nyitható, az `anyhow` a láncban megőrzi az eredeti `rkv_core::Error`-t, és a kiíráskor a kontextus is megjelenik.

11.2.8 Rövid összefoglaló

Az rkv könyvtár saját hibatípusa az `rkv_core::Error` enum, amelyet a `thiserror` derive-tal definiáltunk az `rkv-core/src/error.rs` fájlban. A variánsok: `Io(#[from] io::Error)`, `KeyNotFound(String)`, `InvalidCommand(String)`, és a feladatban hozzáadott `Serialization(#[from] serde_json::Error)`. A `#[from]` attribútumok generálják a `From` implementációkat, így a `?` operátor automatikusan konvertálja az alacsonyabb szintű hibákat, és a `source()` lánc is megmarad. A `rkv-cli` binárisban bevezettük az `anyhow::Result`-et és a `with_context` metódust, és a `--script` kapcsolóval két valódi hibautat (fájlolvasás, hibás sor) kontextussal mutattunk be; a CLI minden parancsa működik, `todo!()` nélkül. A könyvtár és az alkalmazás közötti határ: könyvtárban strukturált enum, alkalmazásban `anyhow` a rugalmasságért. **Továbbvitt eredmény:** az `Error` típus a későbbi fejezetekben a tényleges I/O-műveletek, a `Storage` trait, a hálózati szerver és az FFI hibakezelésének alapja.

11.2.9 Források és ellenőrzési hiányok

- A `std::error::Error` trait (szupertrait-korlátok, `source`): https://doc.rust-lang.org/std/error/trait.Error.html
- The Rust Programming Language, 9. fejezet „Error Handling”: https://doc.rust-lang.org/book/ch09-00-error-handling.html
- `thiserror` (2.x): https://docs.rs/thiserror/2 és `anyhow` (1.x): https://docs.rs/anyhow/1 — a `Context`, a `with_context` és a `main`-ből visszaadott hiba `Error: … Caused by: …` kiírása.

A kódot és a kimeneteket a könyv rögzített toolchainjével (Rust 1.99.1), `thiserror` 2 és `anyhow` 1 főverzióval ellenőriztük; a pontos crate-verziókat a workspace `Cargo.lock`-ja rögzíti. Az I/O-hibák szövege (például `No such file or directory (os error 2)`) operációsrendszer-függő.

11.3 I/O és szerializáció: fájlok, pufferelés, serde

11.3.1 Motiváció

C++-ban az iostream-réteg (`std::ifstream`, `std::ofstream`, `std::cin`) pufferel, formáz és kényelmes operátorokat ad, de a hibakezelés gyakran „utólagos”: a streamek hibazászlókat állítanak, és a kódban ez a rész gyakran ellenőrizetlen marad. Rustban minden I/O-művelet `Result`-ot ad; a `?` operátorral a hibaterjedés explicit, a fordító pedig figyelmeztet, ha egy `Result`-ot figyelmen kívül hagynánk. A `std::io` traitjei ráadásul nem fájlra vagy socketre szabottak: ugyanaz a `Read`/`Write` implementáció működik fájllal, hálózattal, pufferrel vagy éppen egy teszt-`Vec`-kel.

Ebben az alfejezetben ezeket a trait-eket használjuk, bemutatjuk a `serde`-alapú szerializációt, és egy valódi, hibatűrő perzisztenciaréteget építünk az rkv számára: snapshot plusz hozzáfűzéses napló. Ez lesz a `FileStore`, amelyet később a `Storage` trait mögé emelünk, és a hálózati szerver is használ majd.

11.3.2 Fogalmak és szabályok

Definíció. A `std::io::Read` trait egy bájtsorozat olvasását modellezi. A központi metódusa:

```
fn read(&mut self, buf: &mut [u8]) -> io::Result<usize>;
```

A `read` **legfeljebb** `buf.len()` bájtot olvas be, és a ténylegesen beolvasott bájtok számát adja vissza. A `0` érték a fájl végét (EOF) jelenti normál esetben. A trait kényelmi metódusokat is ad: `read_to_end`, `read_to_string`, `read_exact`.

Szabály. Soha ne feltételezd, hogy egyetlen `read` hívás teleolvassa a puffert. A `read_exact` garantálja a teljes puffer beolvasását, a `read_to_end` pedig a teljes bemenetet gyűjti be. Fájl esetén a gyakorlatban gyakran működik a teli puffer, de hálózaton, pipe-on csomagokban érkezhet az adat.

Definíció. A `std::io::Write` trait a kimeneti bájtsorozatot modellezi:

```
fn write(&mut self, buf: &[u8]) -> io::Result<usize>;
fn flush(&mut self) -> io::Result<()>;
```

A `write` a puffer egy részét is kiírhatja; a teljes puffer garantált kiírásához a `write_all` kényelmi metódust vagy a `writeln!` makrót használjuk. A `flush` a pufferelt adatot azonnal a háttérbe küldi.

Definíció. A `std::io::BufRead` a `Read`-et belső pufferrel bővíti, és sorolvasó kényelmi metódusokat ad: `read_line`, `lines`, `split`. A `BufReader<T>` egy `Read` implementáló típus köré húz puffert; a `BufWriter<T>` egy `Write` implementáló típus köré.

Definíció. A `Path` nem birtokló, platformfüggetlen nézet egy fájlrendszerbeli elérési útra; a `PathBuf` ennek birtokló, módosítható változata. Ez hasonló a `std::string_view` és a `std::string` viszonyához, csak itt fájlrendszeri utakra.

Definíció (atomikus csere). Ha egy állapotfájlt úgy mentünk, hogy először egy ideiglenes fájlt írunk tele, `flush`-elünk és `sync_all`-lal a lemezre kényszerítjük, majd `std::fs::rename`-nel a végleges névre nevezzük át, a meglévő állapotfájl soha nem lesz félkész. Linuxon a `rename` azonos fájlrendszeren atomikus csereként működik; ahhoz, hogy maga a csere is túlélje az áramkimaradást, a szülőkönyvtárat is szinkronizálni kell. A `flush` csak a Rust-oldali puffert üríti az operációs rendszer felé, a lemezre írást nem garantálja. A Rust `File::create` a célfájlt lenullázva nyitja meg, ezért önmagában nem alkalmas törésbiztos mentésre.

Definíció (hozzáfűzéses napló). Az append-only log minden mutációs műveletet a fájl végéhez fűz. Újraindításkor a legutóbbi snapshotot betöltjük, majd a naplót sorról sorra visszajátsszuk. A `compact` művelet tartós snapshotot készít, és csak ezután nullázza le a naplót.

11.3.3 C++ és Rust összevetés

C++-ban a fájl olvasása és a hiba ellenőrzése gyakran így néz ki:

```
#include <fstream>
#include <iostream>
#include <string>

int main() {
    std::ifstream in("input.txt");
    if (!in) {
        std::cerr << "open failed\n";
        return 1;
    }
```

```
    std::string line;
    while (std::getline(in, line)) {
        std::cout << line << '\n';
    }
    if (in.bad()) return 2; // sokszor elmarad
}
```

A Rust-változat minden lépés `Result`-ját továbbadja:

```
use std::fs::File;
use std::io::{self, BufRead, BufReader};

fn main() -> io::Result<()> {
    let file = File::open("input.txt")?;
    for line in BufReader::new(file).lines() {
        println!("{}", line?);
    }
    Ok(())
}
```

A különbség: a `?` nem hagyja, hogy a `File::open` vagy a sorolvasás hibája észrevétlen maradjon. A `lines()` a sor elejét és végét leválasztja, de a sortörést nem adja vissza; a C++ `getline` ugyanezt teszi eltérő vonalazási részletekkel. A Rust `File` nem pufferelt, ezért a hatékony sorolvasáshoz `BufReader` kell; a C++ streamek alapból pufferelnek.

11.3.4 Kidolgozott példa: generikus Read/Write

A `Read` és `Write` trait-ek ereje abban van, hogy nem kötődnek fájlhoz. Ugyanaz a függvény működik fájllal, bájttömbbel vagy hálózati streammel:

```
use std::io::{self, Read, Write};

fn copy_all<R: Read, W: Write>(mut from: R, mut to: W) -> io::Result<u64> {
    io::copy(&mut from, &mut to)
}
```

Szerializációhoz a `serde` a deriválható `Serialize` és `Deserialize` trait-eket adja. Az `rkv-core` `Cargo.toml`-jába a következő függőségek kerülnek:

Kódblokk — fájl: `rkv-core/Cargo.toml` · művelet: beillesztés a `[dependencies]` szakaszba · ellenőrzés: `cargo build -p rkv-core`

```
serde = { version = "1", features = ["derive"] }
serde_json = "1"
```

Példa JSON-mentésre:

```
use serde::{Deserialize, Serialize};
use std::collections::BTreeMap;
use std::fs::File;
use std::io::{BufWriter, Write}; // a Write trait kell a flush() híváshoz

#[derive(Serialize, Deserialize)]
struct Config {
    port: u16,
    entries: BTreeMap<String, Vec<u8>>,
}

fn save_config(config: &Config, path: &str) -> std::io::Result<()> {
    let file = File::create(path)?;
```

```
    let mut writer = BufWriter::new(file);
    serde_json::to_writer(&mut writer, config)?;
    writer.flush()?;
    Ok(())
}
```

A `serde_json::to_writer` bármely `Write` implementáló típusra dolgozik, ezért a `BufWriter` pufferezését egyszerűen kihasználjuk. A `flush` a `Write` trait metódusa, ezért a traitet importálni kell (`use std::io::Write`); e nélkül a hívás `E0599`-cel nem fordul le. A `?` itt a `serde_json::Error`-t is `io::Error`-rá alakítja, mert a `serde_json` biztosít ilyen `From` implementációt. Bináris formátumhoz a `bincode` crate használható; az rkv nem használja, és mivel az 1.x és a 2.x főverzió API-ja jelentősen eltér, a használatához a választott főverzió docs.rs-oldalát kell követni.

A `serde_json` olvasható és könnyen debuggolható, ezért az rkv naplójához és snapshotjához ezt használjuk. A bináris `bincode` kisebb, de olvashatatlan; beágyazott vagy nagy forgalmú rendszereknél jobb választás lehet.

11.3.5 Labor: rkv `FileStore` snapshot + napló

Munkakönyvtár: `~/rust-labor/rkv/`

A labor a `rkv-core` könyvtári crate-ben valósítja meg a `FileStore`-t. A cél: a tároló állapota túlélje a processz újraindítását. A mentés kettős: egy JSON snapshot plusz egy JSON-soronkénti hozzáfűzéses napló. A `FileStore` még konkrét típus; a közös `Storage` traitet a Trait-ek és generikusok: sablonok és concepts helyett alfejezet vezeti be.

11.3.5.0.1 1. Függőségek és hibatípus

A `rkv-core/Cargo.toml` kiegészül a fenti `serde` és `serde_json` függőséggel. A kanonikus `Error` (Saját hibatípusok: enum-hibák, thiserror, anyhow) egyetlen új variánst kap a (de)szerializációs hibákhoz; a `Result<T>` alias és a crate-gyökérbeli `pub use error::{Error, Result};` már a Saját hibatípusok: enum-hibák, thiserror, anyhow óta létezik:

Kódblokk — fájl: `rkv-core/src/error.rs` · művelet: beillesztés az `Error` enum utolsó variánsa után · ellenőrzés: `cargo build -p rkv-core`

```
    /// Snapshot vagy naplórekord (de)szerializációs hibája.
    #[error("szerializációs hiba: {0}")]
    Serialization(#[from] serde_json::Error),
```

Ettől kezdve az `Error` nem változik: `Io`, `KeyNotFound`, `InvalidCommand`, `Serialization`.

11.3.5.0.2 2. Modul és FileStore

A `store` modul eddig egyetlen fájl volt (`rkv-core/src/store.rs`, Modulok és láthatóság: header nélküli világ). Mivel most almodult kap, a fájlt könyvtármodullá alakítjuk: **költöztessük** változatlan tartalommal a `rkv-core/src/store/mod.rs` helyre (például `git mv rkv-core/src/store.rs rkv-core/src/store/mod.rs`). A régi fájl nem maradhat meg: ha a `store.rs` és a `store/mod.rs` egyszerre létezik, a fordító `E0761`-es hibát ad (`file for modulestorefound at both ...`). A `Store` definíciója és tesztjei a `mod.rs`-ben maradnak; ehhez adjuk hozzá az új almodult:

Kódblokk — fájl: `rkv-core/src/store/mod.rs` · művelet: beillesztés a fájl elejére (a költöztetés után) · ellenőrzés: a következő blokkok után `cargo build -p rkv-core`

```
mod file_store;
pub use file_store::FileStore;
```

A crate gyökerében a `FileStore`-t is re-exportáljuk, így a `Store` és a `FileStore` egyszerre elérhető (`rkv_core::Store`, `rkv_core::FileStore`):

Kódblokk — fájl: `rkv-core/src/lib.rs` · művelet: a `pub use store::Store;` sor cseréje · ellenőrzés: `cargo build --workspace`

```
pub use store::{FileStore, Store};
```

A `FileStore` implementációja:

Kódblokk — fájl: `rkv-core/src/store/file_store.rs` · művelet: új fájl · ellenőrzés: `cargo build -p rkv-core` → hibátlan fordítás

```
use std::collections::HashMap;
use std::fs::{self, File, OpenOptions, TryLockError};
use std::io::{self, BufRead, BufReader, BufWriter, Write};
use std::path::{Path, PathBuf};
use serde::{Deserialize, Serialize};

use crate::{Error, Result};

#[derive(Debug, Clone, Serialize, Deserialize)]
struct Snapshot {
    data: HashMap<String, Vec<u8>>,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
enum LogEntry {
    Set { key: String, value: Vec<u8> },
    Delete { key: String },
}

pub struct FileStore {
    data: HashMap<String, Vec<u8>>,
    log_writer: BufWriter<File>,
    snapshot_path: PathBuf,
    /// A snapshotra tett kizárólagos zár (külön `.lock` fájl); a FileStore eldobásakor oldódik.
    _lock: File,
    /// Egy sikertelen naplóírás után igaz: a tároló nem fogad több írást.
    poisoned: bool,
}

impl FileStore {
    pub fn open(snapshot_path: impl AsRef<Path>, log_path: impl AsRef<Path>) -> Result<Self> {
        let snapshot_path = snapshot_path.as_ref().to_path_buf();
        let log_path = log_path.as_ref();

        // A három segédfájl neve a snapshot nevéből származik, ezért a snapshot,
        // a napló, az ideiglenes fájl és a zárfájl útvonalai nem eshetnek egybe.
        let tmp_path = sibling(&snapshot_path, ".tmp");
        let lock_path = sibling(&snapshot_path, ".lock");
        for (a, name) in [(&tmp_path, "ideiglenes"), (&lock_path, "zár")] {
            if same_path(log_path, a) {
                return Err(Error::Io(io::Error::new(
                    io::ErrorKind::InvalidInput,
                    format!("a napló útvonala ütközik a snapshot {name} fájljával"),
                )));
            }
        }
        if same_path(log_path, &snapshot_path) {
            return Err(Error::Io(io::Error::new(
                io::ErrorKind::InvalidInput,
                "a napló és a snapshot útvonala nem lehet azonos",
```

```
            )));
        }

        // Kizárólagos használat: a *snapshotra* (nem a naplóra) tett zár miatt
        // egy snapshotot egyszerre csak egy FileStore nyithat meg, akárhány
        // naplóútvonallal. A zár a FileStore eldobásakor oldódik fel.
        let lock_file = OpenOptions::new()
            .create(true)
            .write(true)
            .truncate(false)
            .open(&lock_path)?;
        match lock_file.try_lock() {
            Ok(()) => {}
            Err(TryLockError::WouldBlock) => {
                return Err(Error::Io(io::Error::new(
                    io::ErrorKind::WouldBlock,
                    "a tárolót már egy másik FileStore használja",
                )));
            }
            Err(TryLockError::Error(e)) => return Err(e.into()),
        }

        let mut data = HashMap::new();
        if snapshot_path.exists() {
            let file = File::open(&snapshot_path)?;
            let reader = BufReader::new(file);
            let snap: Snapshot = serde_json::from_reader(reader)?;
            data = snap.data;
        }

        let log_file = OpenOptions::new()
            .create(true)
            .append(true)
            .open(log_path)?;
        // Az újonnan létrehozott napló fájlneve is tartós legyen (nem csak a tartalma).
        sync_parent_dir(log_path)?;
        let valid_len = replay_log(log_path, &mut data)?;
        if log_file.metadata()?.len() > valid_len {
            // Csonka (nem commitolt) utolsó rekord levágása, hogy az új
            // bejegyzések ne félkész sor mögé kerüljenek.
            log_file.set_len(valid_len)?;
            log_file.sync_all()?;
        }
        let log_writer = BufWriter::new(log_file);

        Ok(Self {
            data,
            log_writer,
            snapshot_path,
            _lock: lock_file,
            poisoned: false,
        })
    }

    pub fn get(&self, key: &str) -> Option<&Vec<u8>> {
        self.data.get(key)
    }

    pub fn set(&mut self, key: String, value: Vec<u8>) -> Result<()> {
        let entry = LogEntry::Set { key: key.clone(), value: value.clone() };
        self.append_entry(&entry)?;
        // Csak a tartós naplórekord után módosul a memóriabeli állapot.
        self.data.insert(key, value);
```

```
    Ok(())
}

pub fn delete(&mut self, key: &str) -> Result<()> {
    // A mérgezett tároló minden további írást elutasít, hiányzó kulcs törlését is.
    self.ensure_healthy()?;
    if !self.data.contains_key(key) {
        return Ok(());
    }
    let entry = LogEntry::Delete { key: key.to_string() };
    self.append_entry(&entry)?;
    self.data.remove(key);
    Ok(())
}

fn ensure_healthy(&self) -> Result<()> {
    if self.poisoned {
        return Err(std::io::Error::other(
            "a napló egy korábbi írási hiba óta sérült; nyisd meg újra a tárolót",
        )
        .into());
    }
    Ok(())
}

/// Commit-szerződés: a teljes rekord (JSON + '\n') kiírása, flush és
/// sync_data. Csak Ok esetén szabad a memóriabeli állapotot módosítani.
fn append_entry(&mut self, entry: &LogEntry) -> Result<()> {
    self.ensure_healthy()?;
    let mut line = serde_json::to_vec(entry)?;
    line.push(b'\n');
    let res = self
        .log_writer
        .write_all(&line)
        .and_then(|()| self.log_writer.flush())
        .and_then(|()| self.log_writer.get_ref().sync_data());
    if let Err(e) = res {
        // A pufferben vagy a fájlban félkész rekord maradhatott:
        // vakon nem folytatjuk a naplózást.
        self.poisoned = true;
        return Err(e.into());
    }
    Ok(())
}

pub fn list(&self, prefix: Option<&str>) -> Vec<(String, Vec<u8>)> {
    let mut items: Vec<_> = self
        .data
        .iter()
        .filter(|(k, _)| prefix.is_none_or(|p| k.starts_with(p)))
        .map(|(k, v)| (k.clone(), v.clone()))
        .collect();
    items.sort_by(|a, b| a.0.cmp(&b.0));
    items
}

pub fn save_snapshot(&mut self) -> Result<()> {
    self.ensure_healthy()?;
    let tmp_path = sibling(&self.snapshot_path, ".tmp");
    // A File::create az ideiglenes fájlt nullázza le, a régi snapshotot nem.
    let file = File::create(&tmp_path)?;
    let mut writer = BufWriter::new(file);
    serde_json::to_writer(&mut writer, &Snapshot { data: self.data.clone() })?;
```

```
        writer.flush()?;
        writer.get_ref().sync_all()?; // 1. az új tartalom a lemezen van
        drop(writer);
        fs::rename(&tmp_path, &self.snapshot_path)?; // 2. atomikus csere
        sync_parent_dir(&self.snapshot_path)?; // 3. maga a csere is tartós
        Ok(())
    }

    pub fn compact(&mut self) -> Result<()> {
        self.save_snapshot()?; // 1-3. lépés: az új snapshot tartós
        // 4. Csak ezután dobható el a napló.
        let log = self.log_writer.get_mut();
        log.set_len(0)?;
        log.sync_all()?;
        Ok(())
    }
}

/// A snapshot mellé, a nevéhez fűzött utótaggal képzett útvonal (`state.json` →
↪  `state.json.tmp`).
fn sibling(path: &Path, suffix: &str) -> PathBuf {
    let mut name = path.file_name().unwrap_or_default().to_os_string();
    name.push(suffix);
    path.with_file_name(name)
}

/// Két útvonal azonos-e (létező fájlnál a kanonikus alak, különben a szülő kanonikus alakja +
↪  név).
fn same_path(a: &Path, b: &Path) -> bool {
    fn norm(p: &Path) -> PathBuf {
        let parent = p
            .parent()
            .filter(|d| !d.as_os_str().is_empty())
            .unwrap_or(Path::new("."));
        match (fs::canonicalize(parent), p.file_name()) {
            (Ok(d), Some(n)) => d.join(n),
            _ => p.to_path_buf(),
        }
    }
    norm(a) == norm(b)
}

/// POSIX-on a rename a szülőkönyvtár szinkronizálása után tartós.
#[cfg(unix)]
fn sync_parent_dir(path: &Path) -> Result<()> {
    let dir = match path.parent() {
        Some(p) if !p.as_os_str().is_empty() => p,
        _ => Path::new("."),
    };
    File::open(dir)?.sync_all()?;
    Ok(())
}

#[cfg(not(unix))]
fn sync_parent_dir(_path: &Path) -> Result<()> {
    // Nem POSIX-platformon könyvtár így nem szinkronizálható; lásd a
    // tartóssági modellt a labor után.
    Ok(())
}

/// Visszajátssza a naplót, és visszaadja az utolsó teljes rekord végének
/// bájtpozícióját.
fn replay_log(path: &Path, data: &mut HashMap<String, Vec<u8>>) -> Result<u64> {
```

```
    let file = match File::open(path) {
        Ok(f) => f,
        Err(e) if e.kind() == std::io::ErrorKind::NotFound => return Ok(0),
        Err(e) => return Err(e.into()),
    };
    let mut reader = BufReader::new(file);
    let mut line = Vec::new();
    let mut valid_len = 0u64;
    loop {
        line.clear();
        let n = reader.read_until(b'\n', &mut line)?;
        if n == 0 || line.last() != Some(&b'\n') {
            // Fájl vége, vagy sorvégjel nélküli csonka rekord: ez utóbbi
            // sosem kapott Ok-t, mert az append_entry a teljes sort írja.
            break;
        }
        valid_len += n as u64;
        if line.trim_ascii().is_empty() {
            continue;
        }
        // A csak whitespace-ből álló sort tudatosan átugorjuk (a naplóíró ilyet nem állít elő).
        // A sorvégjellel lezárt, de hibás rekord valódi sérülés: hiba.
        let entry: LogEntry = serde_json::from_slice(&line)?;
        match entry {
            LogEntry::Set { key, value } => {
                data.insert(key, value);
            }
            LogEntry::Delete { key } => {
                data.remove(&key);
            }
        }
    }
    Ok(valid_len)
}
```

A felhasznált újabb std API-k: a `File::try_lock` az 1.89-től, a `trim_ascii` az 1.80-tól, az `io::Error::other` az 1.74-től stabil; a könyv rögzített toolchainje (1.99.1) mindhármat tartalmazza.

11.3.5.0.3 3. Unit tesztek

A tesztek a `file_store.rs` alján kapnak helyet, és a perzisztencia egy-egy esetét külön ellenőrzik: csak snapshotból, csak naplóból, illetve snapshotból és az utána írt naplóból való újranyitást; a törlést és a felülírást; azt, hogy a commitolt rekord az eldobás előtt már a fájlban van; a csonka utolsó rekordot; a hibázó naplóírást; és a kettős megnyitás elutasítását. Ha a `replay_log` hívását kikapcsolnánk, legalább a `log_only_reopen` teszt elbukna: a tartósságot így nem puszta fájllétezéssel, hanem a visszaolvasott állapottal igazoljuk. A `failed_append_leaves_committed_state` a modulon belüli privát mezőhöz fér hozzá, és a naplóírót egy csak olvasásra megnyitott fájlra cseréli: ez a `Write` implementáció biztosan hibával tér vissza.

Kódblokk — fájl: `rkv-core/src/store/file_store.rs` · művelet: beillesztés a fájl végére · ellenőrzés: `cargo test -p rkv-core file_store` → 8 sikeres teszt

```
#[cfg(test)]
mod tests {
    use super::*;

    /// Tesztenként egyedi, üres könyvtár (a név a tesztet, a pid a futást különíti el).
    fn fresh_dir(name: &str) -> (PathBuf, PathBuf) {
        let dir = std::env::temp_dir().join(format!(
            "rkv-file-store-{name}-{}",
```

```
        std::process::id()
    ));
    let _ = std::fs::remove_dir_all(&dir);
    std::fs::create_dir_all(&dir).unwrap();
    (dir.join("state.json"), dir.join("append.log"))
}

fn get(store: &FileStore, key: &str) -> Option<Vec<u8>> {
    store.get(key).cloned()
}

#[test]
fn snapshot_only_reopen() {
    let (snap, log) = fresh_dir("snap");
    {
        let mut store = FileStore::open(&snap, &log).unwrap();
        store.set("a".into(), b"1".to_vec()).unwrap();
        store.set("b".into(), b"2".to_vec()).unwrap();
        store.compact().unwrap();
    }
    assert_eq!(std::fs::metadata(&log).unwrap().len(), 0); // a napló üres
    let store = FileStore::open(&snap, &log).unwrap();
    assert_eq!(get(&store, "a"), Some(b"1".to_vec()));
    assert_eq!(get(&store, "b"), Some(b"2".to_vec()));
}

#[test]
fn log_only_reopen() {
    let (snap, log) = fresh_dir("log");
    {
        let mut store = FileStore::open(&snap, &log).unwrap();
        store.set("a".into(), b"1".to_vec()).unwrap();
        store.set("b".into(), b"2".to_vec()).unwrap();
        store.delete("a").unwrap();
        store.set("b".into(), b"3".to_vec()).unwrap(); // felülírás
    }
    assert!(!snap.exists()); // csak a napló hordozza az állapotot
    let store = FileStore::open(&snap, &log).unwrap();
    assert_eq!(get(&store, "a"), None);
    assert_eq!(get(&store, "b"), Some(b"3".to_vec()));
}

#[test]
fn snapshot_plus_new_log() {
    let (snap, log) = fresh_dir("both");
    {
        let mut store = FileStore::open(&snap, &log).unwrap();
        store.set("a".into(), b"1".to_vec()).unwrap();
        store.compact().unwrap();
        store.set("b".into(), b"2".to_vec()).unwrap();
        store.delete("a").unwrap();
    }
    let store = FileStore::open(&snap, &log).unwrap();
    assert_eq!(get(&store, "a"), None);
    assert_eq!(get(&store, "b"), Some(b"2".to_vec()));
}

#[test]
fn commit_is_in_file_before_drop() {
    let (snap, log) = fresh_dir("commit");
    let mut store = FileStore::open(&snap, &log).unwrap();
    store.set("k".into(), b"v".to_vec()).unwrap();
    // A store még él, nem futott Drop és nem hívtunk flush-t:
```

```
    // az Ok után a rekordnak már a fájlban kell lennie.
    let text = std::fs::read_to_string(&log).unwrap();
    assert!(text.ends_with('\n') && text.contains("\"k\""));
}

#[test]
fn torn_tail_is_dropped() {
    let (snap, log) = fresh_dir("torn");
    {
        let mut store = FileStore::open(&snap, &log).unwrap();
        store.set("a".into(), b"1".to_vec()).unwrap();
    }
    // Összeomlás közbeni, félbemaradt rekord szimulálása.
    let mut f = OpenOptions::new().append(true).open(&log).unwrap();
    f.write_all(br#"{"Set":{"key":"b","val"#).unwrap();
    drop(f);

    let mut store = FileStore::open(&snap, &log).unwrap();
    assert_eq!(get(&store, "a"), Some(b"1".to_vec()));
    assert_eq!(get(&store, "b"), None);
    store.set("c".into(), b"3".to_vec()).unwrap();
    drop(store);

    let store = FileStore::open(&snap, &log).unwrap();
    assert_eq!(get(&store, "c"), Some(b"3".to_vec()));
}

#[test]
fn failed_append_leaves_committed_state() {
    let (snap, log) = fresh_dir("fail");
    let mut store = FileStore::open(&snap, &log).unwrap();
    store.set("a".into(), b"1".to_vec()).unwrap();

    // Szándékosan hibázó író: csak olvasásra nyitott fájl.
    store.log_writer = BufWriter::new(File::open(&log).unwrap());
    assert!(store.set("b".into(), b"2".to_vec()).is_err());
    assert_eq!(get(&store, "b"), None); // a memória nem változott
    assert!(store.set("c".into(), b"3".to_vec()).is_err()); // mérgezett
    assert!(store.compact().is_err());
    drop(store);

    let store = FileStore::open(&snap, &log).unwrap();
    assert_eq!(get(&store, "a"), Some(b"1".to_vec()));
    assert_eq!(get(&store, "b"), None);
    assert_eq!(get(&store, "c"), None);
}

#[test]
fn second_open_is_rejected() {
    let (snap, log) = fresh_dir("lock");
    let first = FileStore::open(&snap, &log).unwrap();
    assert!(matches!(FileStore::open(&snap, &log), Err(Error::Io(_))));
    drop(first); // a zár feloldódik
    assert!(FileStore::open(&snap, &log).is_ok());
}

#[test]
fn same_snapshot_with_different_logs_is_rejected() {
    let (snap, log) = fresh_dir("lock2");
    let other_log = log.with_file_name("other.log");
    let first = FileStore::open(&snap, &log).unwrap();
    // Ugyanaz a snapshot másik naplóval sem nyitható meg.
    assert!(matches!(
```

```
            FileStore::open(&snap, &other_log),
            Err(Error::Io(_))
        ));
        drop(first);
    }

    #[test]
    fn colliding_paths_are_rejected() {
        let (snap, _log) = fresh_dir("collide");
        let tmp = snap.with_file_name("state.json.tmp");
        let lock = snap.with_file_name("state.json.lock");
        for bad in [&snap, &tmp, &lock] {
            let r = FileStore::open(&snap, bad);
            assert!(
                matches!(r, Err(Error::Io(ref e)) if e.kind() ==
                ↪  std::io::ErrorKind::InvalidInput),
                "{bad:?}"
            );
        }
    }

    #[test]
    fn compact_is_idempotent() {
        let (snap, log) = fresh_dir("idem");
        let mut store = FileStore::open(&snap, &log).unwrap();
        store.set("a".into(), b"1".to_vec()).unwrap();
        store.compact().unwrap();
        store.compact().unwrap();
        drop(store);
        let store = FileStore::open(&snap, &log).unwrap();
        assert_eq!(get(&store, "a"), Some(b"1".to_vec()));
    }
}
```

Futtatás:

```
cargo test -p rkv-core file_store
```

Várt kimenet (rövidítve, a tesztek neve szerepel; a sorrend változhat):

```
running 8 tests
test store::file_store::tests::commit_is_in_file_before_drop ... ok
test store::file_store::tests::compact_is_idempotent ... ok
test store::file_store::tests::failed_append_leaves_committed_state ... ok
test store::file_store::tests::log_only_reopen ... ok
test store::file_store::tests::second_open_is_rejected ... ok
test store::file_store::tests::snapshot_only_reopen ... ok
test store::file_store::tests::snapshot_plus_new_log ... ok
test store::file_store::tests::torn_tail_is_dropped ... ok
```

Commit-szerződés. A `set` és a `delete` csak akkor tér vissza `Ok`-val, ha a teljes, sorvégjellel lezárt naplórekord kiíródott, és a `sync_data` is sikerült; a memóriabeli állapotot csak ezután módosítjuk. `Err` esetén két dolog garantált: a memóriabeli állapot nem tartalmazza a műveletet, és a tároló mérgezett lesz, vagyis minden további írás és `compact` hibát ad, amíg újra nem nyitjuk. A művelet tartós kimenetele viszont `Err` után *bizonytalan*: ha a rekord kiíródott, de a `sync_data` hibázott, az újranyitás a műveletet már tartalmazhatja. Ez minden naplózó adatbázis közös tulajdonsága; a hívó ilyenkor újranyitással és olvasással tudja meg az eredményt.

Tartóssági modell. Az alábbi garanciák Linuxon, helyi POSIX-fájlrendszeren (ext4, XFS) érvényesek, feltéve, hogy a háttértár a gyorsítótár-ürítési kéréseket betartja. Hálózati fájlrendszerre (NFS, SMB) és nem POSIX-platformra a könyv nem ad garanciát. Windowson a könyvtár

nem szinkronizálható a fenti módon, ott a `MoveFileExW` `MOVEFILE_WRITE_THROUGH` kapcsolója a megfelelő eszköz (ellenőrzési igény). A `compact` lépései és az összeomlás utáni helyreállítás:

Összeomlás pontja	Lemezen maradó állapot	Újranyitás eredménye
1. az ideiglenes snapshot írása vagy `sync_all`-ja közben	régi snapshot + teljes napló (+ félkész `.tmp`)	a régi snapshot + napló visszajátszása: teljes állapot
2. a `rename` előtt vagy közben	régi vagy új snapshot, mindkét esetben teljes napló	a csere atomikus, a teljes állapot visszaáll
3. a `rename` után, a könyvtár `sync_all`-ja előtt	a régi vagy az új könyvtárbejegyzés tartós	mint a 2. pontban
4. a napló csonkolása közben (a `set_len(0)` után, a `sync_all` előtt vagy hibájakor)	új snapshot + teljes vagy üres napló	a napló a snapshotra visszajátszva ugyanazt az állapotot adja (a `Set`/`Delete` rekordok idempotens újrajátszása), tehát mindkét állapot helyes; ezért a `compact` hibája után is biztonságos tovább írni: az új snapshot már tartós (1–3. lépés), a naplócsonkolás csak helyet szabadít fel

A 4. sor azért helyes, mert a napló `Set`/`Delete` rekordjainak sorrendben való újrajátszása a snapshot állapotára ugyanazt a végállapotot adja: a snapshot már a napló végének állapota, az újrajátszás pedig csak ugyanezeket az értékeket írja felül. A napló eldobása ezért csak a 3. lépés után biztonságos.

Kizárólagos használat és ideiglenes fájl. A `FileStore::open` a *snapshotot* azonosító külön zárfájlra (`<snapshot>.lock`) tesz kizárólagos zárat (`File::try_lock`), ezért ugyanazt a snapshotot egyszerre csak egy `FileStore` nyithatja meg, akárhány naplóútvonallal (különben két író közös snapshotot írna felül, és az egyik napló tartalma elveszhetne a közös kompaktálásnál); a második megnyitás `Error::Io` (`WouldBlock`) hibát kap. Az ideiglenes fájl (`<snapshot>.tmp`) és a zárfájl neve a snapshot nevéből származik, ezért egyetlen író használja őket; az `open` visszautasítja (`InvalidInput`), ha a napló útvonala a snapshotéval, az ideiglenes vagy a zárfájléval azonos (különben a `save_snapshot` a naplót nullázná le, a `compact` pedig a snapshotot csonkolná). A zár *tanácsadó* (advisory) jellegű: csak a szintén zárat kérő folyamatokat tartja távol, egy a fájlt zár nélkül író külső program ellen nem véd.

Mit igazol és mit nem a teszt. A fenti unit tesztek a bezárás-újranyitást (snapshotból, naplóból és a kettőből), azt, hogy a commitolt rekord az eldobás előtt már a fájlban van, a csonka utolsó rekordot, a hibázó írót és a kettős megnyitást ellenőrzik. **Nem** igazolják az áramkimaradás-tűrést: a processz leállása után az operációs rendszer lapgyorsítótára megmarad, ezért az `fsync` hiánya ilyen teszttel nem mutatható ki. A fázisonkénti hibainjektálási tervhez minden táblázatsor elé egy hibapont kerül (például egy `fail_at: Option<Phase>` mező csak tesztbuildben, amely a megadott fázisban `Err`-t ad vagy `std::process::abort()`-ot hív), utána újranyitás és állapot-összevetés következik. Valódi áramkimaradást virtuális gépben, a lemez hirtelen leválasztásával, vagy a Linux `dm-flakey` / `dm-log-writes` eszközével lehet modellezni; ez a könyv laborján kívül esik.

A napló megnyitásánál az `OpenOptions::new().append(true).create(true)` biztosítja, hogy a fájl létezzen, és minden írás a végére kerüljön. A `replay_log` a hiányzó naplófájlt elfogadja első indításkor, a tényleges I/O-hibát viszont továbbadja.

Továbbvitt eredmény. A `FileStore` perzisztenciáját a Trait-ek és generikusok: sablonok és concepts helyett alfejezet a `Storage` trait mögé emeli, a Az rkv hálózati szerver: TCP, protokoll, kecses leállás alfejezetben a `rkv-server` használja, a Záróprojekt: az rkv teljes rendszerként alfejezetben pedig a teljes rendszer része lesz.

Ellenőrzési pont. Az `rkv-core` a `Store` mellett a `FileStore`-t is exportálja (`rkv_core::FileStore`, a `store/file_store.rs`-ben), az `Error` a `Serialization` variánssal teljes. A `cargo test -p rkv-core` a 7 korábbi unit teszt mellett a 8 `file_store` tesztet is sikeresen futtatja (Rust 1.99.1, `serde` 1, `serde_json` 1, `thiserror` 2). Az `rkv-cli` változatlanul fordul (`cargo build --workspace`); a CLI-t a Trait-ek és generikusok: sablonok és concepts helyett alfejezet köti a `FileStore`-hoz, amikor a két tároló közös `Storage` trait mögé kerül.

11.3.6 Tipikus hibák és átállási csapdák

1. **Egyetlen `read` hívásra alapozni.** A `read` nem garantálja, hogy a puffer tele lesz; a `read_to_end`, `read_to_string` vagy `read_exact` használata a helyes. C++-os beidegződés, hogy a `istream::read` teljes rekordot vár; Rustban ez explicit.
2. **A `writeln!` eredményét elhagyni.** A `writeln!` `io::Result`-ot ad; ha nem használjuk a `?`-et, a kód lefordulhat `#[must_use]` figyelmeztetéssel, de a hiba elveszik. A könyvtári kódban mindig `?`.
3. **A `BufWriter` Drop-jára hagyatkozni.** A `BufWriter` Drop-implementációja megpróbálja ugyan a puffert kiüríteni, de a hibát elnyeli. Ha a lemez megtelt, az adatvesztés némán következhet be. A fontos kimeneteket explicit `flush`-sel zárjuk; ha tartósság is kell, utána `sync_all`/`sync_data` következik.
4. **A `File::create` atomikus cserével való összekeverése.** A `File::create` azonnal lenullázza a célfájlt. Ha a mentés közben a processz leáll, az eredeti állapot is elveszik. A helyes minta: ideiglenes fájl, `flush`, `sync_all`, `rename`, majd a szülőkönyvtár szinkronizálása.
5. **A napló visszajátszásának túlzott szigora.** Ha a processz a naplósor felénél áll le, a `serde_json` parse hiba lehet. A szigorú `?` ilyenkor a teljes indítást megakadályozza. Az rkv ezért különbséget tesz: a sorvégjel nélküli, csonka *utolsó* rekordot eldobja és levágja (az soha nem kapott `Ok`-t), a sorvégjellel lezárt, de hibás rekordot viszont sérülésként jelzi, és megtagadja az indítást.

11.3.7 Feladatok

1. **Fogalmi kérdés.** A `Read::read` miért adhat vissza `Ok(0)`-t, és mit jelent ez? Hogyan olvasnád be egy fájl teljes tartalmát anélkül, hogy előre ismernéd a méretét?
2. **C++-kód átírása.** Az alábbi C++ program soronként beolvas egy fájlt, és a sorok elé írja a sorszámot. Írd át idiomatikus Rustra `?` operátorral és `BufReader`-rel.

```cpp
#include <fstream>
#include <iostream>
#include <string>

int main() {
    std::ifstream in("input.txt");
    std::string line;
    int n = 1;
    while (std::getline(in, line)) {
```

```
        std::cout << n++ << ": " << line << '\n';
    }
}
```

3. **rkv-bővítés.** Egészítsd ki a `FileStore`-t egy `compact` utáni megerősített lezárással: a `compact` végén a `sync_all` mellett ellenőrizd, hogy a snapshot fájl tényleg létezik és olvasható. Írj hozzá tesztet, amely két `compact` hívást hajt végre, és ellenőrzi az állapotot.

4. **Hibajavítás.** Az alábbi kód szándékosan hibásan feltételezi, hogy egyetlen `write` mindig kiírja a teljes puffert, és a Drop elintézi a flush-t. Javítsd a kódot úgy, hogy a hiba ne vesszen el.

```
use std::fs::File;
use std::io::Write;

fn save(path: &str, data: &[u8]) -> std::io::Result<()> {
    let mut f = File::create(path)?;
    f.write(data)?;
    Ok(())
}
```

11.3.8 Megoldások

1. Az `Ok(0)` normál esetben a fájl végét jelenti. A teljes tartalom beolvasásához `read_to_end` (bájtok) vagy `read_to_string` (UTF-8 szöveg) kell; ezek tetszőleges hosszúságú adatot kezelnek.

2. Rust változat:

```
use std::fs::File;
use std::io::{self, BufRead, BufReader};

fn main() -> io::Result<()> {
    let file = File::open("input.txt")?;
    for (i, line) in BufReader::new(file).lines().enumerate() {
        println!("{}: {}", i + 1, line?);
    }
    Ok(())
}
```

3. A `compact` kiegészítése:

```
pub fn compact(&mut self) -> Result<()> {
    self.save_snapshot()?;
    // Megerősítés a napló eldobása előtt: a snapshot létezik, olvasható,
    // és pontosan a memóriabeli állapotot tartalmazza.
    let file = File::open(&self.snapshot_path)?;
    let snap: Snapshot = serde_json::from_reader(BufReader::new(file))?;
    if snap.data != self.data {
        return Err(std::io::Error::other("a mentett snapshot eltér az állapottól").into());
    }
    let log = self.log_writer.get_mut();
    log.set_len(0)?;
    log.sync_all()?;
    Ok(())
}
```

Az ellenőrzés a napló csonkolása *előtt* fut: ha hibát talál, a napló érintetlen marad, így a régi snapshot és a napló továbbra is helyreállítható. Teszt két `compact` hívással: a második hívásnak idempotensnek kell lennie, és a `FileStore::open` utáni `get` értékek nem változhatnak. Ez a teszt is csak a logikai helyességet igazolja, az áramkimaradás-tűrést nem (lásd a tartóssági modellt).

4. Javított kód:

```
use std::fs::File;
use std::io::{BufWriter, Write};

fn save(path: &str, data: &[u8]) -> std::io::Result<()> {
    let file = File::create(path)?;
    let mut writer = BufWriter::new(file);
    writer.write_all(data)?;
    writer.flush()?;
    Ok(())
}
```

A `write` helyett `write_all`, a pufferelt kimenetnél pedig explicit `flush` biztosítja a hibák terjedését.

11.3.9 Rövid összefoglaló

Ebben az alfejezetben megismerkedtünk a Rust I/O traitjeivel. A `Read`, `Write` és `BufRead` generikus absztrakciót adnak fájlok, hálózati streamek és pufferek fölé; a `BufReader` és `BufWriter` explicit pufferelést biztosít. A `std::io::Result` és a `?` minden I/O-hibát explicit terjedést kényszerít ki. A `serde` `Serialize`/`Deserialize` deriválásokkal a `serde_json` JSON formátumba szerializál; bináris formátumhoz a `bincode` crate használható verzióellenőrzéssel. Elkészítettük az rkv `FileStore` típusát a `rkv-core/src/store/file_store.rs` fájlban, amely JSON snapshotot és hozzáfűzéses naplót használ. A `FileStore::open`, `set`, `delete`, `list`, `save_snapshot` és `compact` metódusok a `Result` hibakezeléssel dolgoznak. Az atomikus csere ideiglenes fájllal, `sync_all`-lal, `fs::rename`-nel és a könyvtár szinkronizálásával védi az állapotfájlt; a naplóírás commit-szerződése szerint a memória csak a tartós naplórekord után változik. **Továbbvitt eredmény:** a `FileStore` perzisztenciája a Trait-ek és generikusok: sablonok és concepts helyett, a Az rkv hálózati szerver: TCP, protokoll, kecses leállás és a Záróprojekt: az rkv teljes rendszerként alfejezetek alapja.

11.3.10 Források és ellenőrzési hiányok

A fejezet a Rust standard könyvtár stabil `std::io` és `std::fs` API-jára épít. A `serde` és a `serde_json` főverziója `1`, a pontos verziót a `Cargo.lock` rögzíti. A `bincode` 1.x és 2.x API-ja jelentősen eltér (`bincode::serialize` szemben a `bincode::serde::encode_to_vec`-kel), ezért a fejezet a `serde_json`-t használja. Az atomikus `rename` Linuxon azonos fájlrendszeren atomikus; a tartóssági modell a fejezetben megadott Linux/POSIX környezetre érvényes, Windows és macOS viselkedése eltérhet.

12 Generikusok és trait-ek

12.1 Trait-ek és generikusok: sablonok és concepts helyett

12.1.1 Motiváció

C++-ban egy sablonos függvény deklarációja alig mond valamit arról, mit vár el a típusparamétertől. A használat helyén derül ki, hogy a `T`-nek kell-e `operator+`-a, `operator<`-e, vagy éppen `size()` tagfüggvénye. Ez a duck typing időnként nagyon rugalmas, de a hibaüzenetek a példányosítás mélyén keletkeznek, és gyakran több száz sorból kell kihámozni, hogy mi hiányzott. A C++20 concepts ezt orvosolja, de a nyelv továbbra is megengedi a korlát nélküli sablonokat, és a hibák továbbra is a sablon *használatakor* és nem a definíciójakor jelentkeznek.

A Rust ezzel szemben a generikus kódot már a definíció helyén, a típusparaméterre kimondott **trait boundok** alapján ellenőrzi. Ha a típusparaméter nem elégíti ki a boundot, a fordító a definícióban mutatja a hibát – nem a hívó fél kódjában. Ez a különbség nem csupán kényelem: a Rust generikusoknál a függvény törzse csak a bound által garantált műveleteket használhatja, ezért a generikus API már a deklarációkor dokumentált és ellenőrzött.

12.1.2 Fogalmak és szabályok

Definíció. A **trait** olyan viselkedési interfész, amely metódusok szignatúráját írja elő, és opcionálisan törzsüket is tartalmazhatja. A traitet egy típusra az `impl Trait for Type` blokkal implementáljuk.

Definíció. A **trait bound** egy generikus paraméterre kimondott elvárás: `T: TraitName`. Azt jelenti, hogy a generikus függvény vagy típus törzsében a `T`-re csak a `TraitName` által garantált metódusok hívhatók.

Szabály. Generikus függvényben vagy `impl` blokkban a Rust fordító a törzset a trait boundok alapján, a konkrét típus ismerete nélkül ellenőrzi. Ha a bound nem ad elég műveletet, a hiba a definíció sorára mutat.

Például egy Rust trait definíciója és megvalósítása:

```
trait Summary {
    fn summarize(&self) -> String;

    // default metódus, amelyet az implementáló felülírhat
    fn short_summary(&self) -> String {
        self.summarize().chars().take(40).collect()
    }
}

struct Article {
    title: String,
    author: String,
}
```

```rust
impl Summary for Article {
    fn summarize(&self) -> String {
        format!("{} by {}", self.title, self.author)
    }
}
```

Egy generikus függvény bounddal:

```rust
fn print_summary<T: Summary>(item: &T) {
    println!("{}", item.summarize());
}
```

Bonyolultabb esetben a `where` klauzula javítja az olvashatóságot:

```rust
fn print_both<T, U>(x: &T, y: &U)
where
    T: Summary,
    U: std::fmt::Debug,
{
    println!("{}; {y:?}", x.summarize());
}
```

Definíció. A **monomorfizálás** az a fordítási folyamat, amelyben a generikus kód minden használt konkrét típuskombinációra lefordul. Az `article_f64` például `print_both::<Article, f64>` konkrét kódot kap, akárcsak a C++ sablonpéldányosítás. (Az `i32` csak akkor lehetne a `T` helyén, ha implementálnánk rá a `Summary`-t; a példában a már definiált `Article` tölti be ezt a szerepet.) Ez futásidőben nulla absztrakciós költséget eredményez, cserébe növelheti a bináris méretét.

```
Generikus definíció                    Példányosított kód
+--------------------------+           +-----------------------+
| fn largest<T: PartialOrd>|  fordítás  | largest::<i32>        |
| (a: T, b: T) -> T        | ---------> | largest::<f64>        |
| { if a > b { a } else    |            | largest::<&str>       |
|     { b } }              |            +-----------------------+
+--------------------------+           C++-ban ugyanez a sablon
                                       példányosítás; Rustban a
                                       boundot a definíció előtt
                                       ellenőrzi a fordító.
```

Az `impl Trait` szintaktikai egyszerűsítés. Argumentumpozícióban:

```rust
fn print_summary2(item: &impl Summary) {
    println!("{}", item.summarize());
}
```

Ez pontosan ugyanaz, mint a `T: Summary` paraméter, csak a `T` nevet nem köti. Visszatérési pozícióban viszont más a szerepe: azt mondja, hogy a függvény visszatér egy olyan típussal, amelyik implementálja a traitet, de a konkrét típust a fordító választja meg – **egyetlen** konkrét típust. Az alábbi nem fordul le:

```rust
// Ez a változat szándékosan nem fordul le:
fn make_bool_or_int(flag: bool) -> impl std::fmt::Display {
    if flag {
        42
    } else {
        "hello"
    }
}
```

A hiba lényege: a függvény két különböző konkrét típust adna vissza (`i32` és `&str`), az `impl Display` pedig nem `dyn Display`, ezért nem lehet több típus egyetlen visszatérési pozícióban. A pontos hibaüzenet verziófüggő, de a lényeg mindig ez.

12.1.3 C++ és Rust összevetés

C++ sablon duck typinggal:

```cpp
#include <iostream>
#include <string>

template <typename T>
T max_value(T a, T b) {
    return (a > b) ? a : b; // T-nek operator>-t kell adnia
}

int main() {
    std::cout << max_value(3, 5) << '\n';   // ok: 5
    const char* p = max_value("x", "y");    // lefordul, de nem azt teszi, amit várnánk
    std::cout << p << '\n';
    return 0;
}
```

Ez a program lefordul, és ez a tanulság. A `T` a `"x"` és `"y"` literálból `const char*`-nak adódik, és a `const char*`-ra van beépített `operator>`: a sablon a két *pointer címét* hasonlítja össze, nem a szövegeket. Az eredmény a fordító és a linker által elhelyezett literálok címétől függ (nem specifikált sorrend), tehát szemantikailag hibás, de semmilyen fordítási hibát nem kapunk. A javítás C++-ban a `std::string` vagy `std::string_view` használata: `max_value(std::string("x"), std::string("y"))`, illetve `max_value<std::string_view>("x", "y")` már lexikografikusan hasonlít. A sablon hibája tehát nem hiányzó operátor, hanem az, hogy a duck typing semmit nem mond arról, *milyen jelentéssel* működik a művelet.

Rust megfelelő:

```rust
fn max_value<T: PartialOrd>(a: T, b: T) -> T {
    if a > b { a } else { b }
}

fn main() {
    println!("{}", max_value(3, 5));
    // A "x" és "y" &str típusú; a &str PartialOrd, és az összehasonlítás
    // a szöveg tartalmát (lexikografikusan) vizsgálja, nem a címét.
    println!("{}", max_value("x", "y"));
}
```

A Rust a `PartialOrd` boundot a definícióban ellenőrzi: a `>` művelet ebből a boundból következik. Ha a `T` nem `PartialOrd`, a hiba a definícióban vagy a hívásnál jelezné, hogy a bound nem teljesül, de a függvény törzse már a definícióban érvényes.

C++20 concepts ugyanezt a szándékot fejezi ki, de a másik irányból: a sablon fejében mondjuk ki, hogy `T` total orderinget vár. A Rust ezt nem opcionálisan teszi, hanem a generikusok alapvető működéseként.

12.1.4 Kidolgozott példa

A lenti generikus Rust függvény szándékosan hibás, mert a `T`-nek nincs boundja:

```
// Ez a változat szándékosan nem fordul le:
fn add_one<T>(x: T) -> T {
    x + 1
}

fn main() {
    let n = add_one(41);
    println!("{n}");
}
```

A hiba lényege: a binary `+` műveletet nem lehet alkalmazni a `T` típusra, mert a `T`-ről semmit sem tud a fordító. Ezt a hibaüzenet általában `E0369` kóddal jelzi, és a definíción belül arra a sorra mutat, ahol a `+` szerepel. A javítás:

```
use std::ops::Add;

fn add_one<T>(x: T) -> T
where
    T: Add<Output = T> + From<u8>,
{
    x + T::from(1)
}

fn main() {
    let n = add_one(41);
    println!("{n} {} {}", add_one(1.5f64), add_one(250u16));
}
```

A bound minden eleme szükséges a törzshöz: az `Add<Output = T>` (a `Rhs` alapértelmezetten `Self`) teszi lehetővé az `x + ...` kifejezést, és azt, hogy az eredmény ismét `T`; a `From<u8>` adja a `T::from(1)`-et, amely az `1` literálból `T`-t csinál (a `u8` azért jó választás, mert az `i32`, `u16`, `f64` és a többi szám is megvalósítja). A kimenet `42 2.5 251`; az `add_one(41)` az `i32`-re, az `add_one(1.5f64)` az `f64`-re monomorfizálódik. (Egy `Add<i32, Output = T>` bound csak azokra a `T` típusokra működne, amelyek `i32`-t is összeadhatnak, ezért szűkebb.)

12.1.5 Labor: az rkv `Storage` traitje

Munkakönyvtár: `~/rust-labor/rkv`

A kiinduló állapot az I/O és szerializáció: fájlok, pufferelés, serde ellenőrzési pontja: az `rkv-core` két konkrét tárolót exportál, a memóriabeli `Store`-t (`store/mod.rs`) és a naplózó, snapshotot író `FileStore`-t (`store/file_store.rs`). Ezek metódusai eltérnek (a `Store::set` nem ad `Result`-ot, a `FileStore::delete` nem mondja meg, létezett-e a kulcs). A laborban egy közös `Storage` traitet vezetünk be, és **a meglévő típusokra implementáljuk delegálással**: sem a `FileStore` fájlformátuma, sem a korábbi inherens API nem változik, tehát az I/O és szerializáció: fájlok, pufferelés, serde alfejezetben létrehozott adatbázis a labor után is ugyanúgy megnyitható. A memóriabeli implementáció könyvbeli neve `MemStore`; ez a `Store` típus aliasa (`pub type MemStore = Store;`), nem új típus.

Kanonikus `Storage` trait (a könyv további részeiben ezt használjuk):

- `get(&self, key: &str) -> Result<Option<Vec<u8>>>`
- `set(&mut self, key: String, value: Vec<u8>) -> Result<()>`

- `delete(&mut self, key: &str) -> Result<bool>` (`true`, ha a kulcs létezett)
- `list(&self, prefix: Option<&str>) -> Result<Vec<(String, Vec<u8>)>>`, kulcs szerint rendezve

A `Result` az `rkv_core::Result` alias (Saját hibatípusok: enum-hibák, thiserror, anyhow).

Továbbvitt eredmény. A `Storage` traitet a Asszociált típusok, koherencia, operátorok és derive alfejezet bővíti, a Trait objektumok és dinamikus diszpécs tárgyalja `dyn Storage`-ként, a Unit-, integrációs és dokumentációs tesztek teszteli, a Mutex, RwLock, csatornák és a megosztott rkv pedig szálbiztossá teszi.

Lépések:

1. A `Cargo.toml`-ban nincs új függőség: a `serde`, `serde_json` és `thiserror` a Saját hibatípusok: enum-hibák, thiserror, anyhow–I/O és szerializáció: fájlok, pufferelés, serde óta szerepel.
2. Hozd létre a `rkv-core/src/storage.rs` fájlt. A `Command` típust nem definiáljuk újra: az `rkv_core::Command` (Az enum mint összeg-típus és az Option a null helyett) az egyetlen parancstípus, a `Set` értéke szöveg, amit végrehajtáskor bájtokká alakítunk.

Kódblokk — fájl: `rkv-core/src/storage.rs` · művelet: új fájl · ellenőrzés: a 3. lépés után `cargo test -p rkv-core storage`

```
use crate::{Command, FileStore, MemStore, Result};

/// Kulcs-érték tároló absztrakció: a parancsok végrehajtásához szükséges
/// műveletek. A kulcs `String`, az érték `Vec<u8>` (r007).
pub trait Storage {
    fn get(&self, key: &str) -> Result<Option<Vec<u8>>>;
    fn set(&mut self, key: String, value: Vec<u8>) -> Result<()>;
    /// `true`, ha a kulcs létezett.
    fn delete(&mut self, key: &str) -> Result<bool>;
    /// A `prefix`-szel kezdődő párok kulcs szerint rendezve.
    fn list(&self, prefix: Option<&str>) -> Result<Vec<(String, Vec<u8>)>>;
}

// A memóriabeli tároló delegálása az inherens metódusokra. A teljesen
// minősített hívás (`MemStore::get(self, ..)`) egyértelművé teszi, hogy az
// inherens, nem a trait-metódust hívjuk, így nincs rekurzió.
impl Storage for MemStore {
    fn get(&self, key: &str) -> Result<Option<Vec<u8>>> {
        Ok(MemStore::get(self, key).cloned())
    }

    fn set(&mut self, key: String, value: Vec<u8>) -> Result<()> {
        MemStore::set(self, key, value);
        Ok(())
    }

    fn delete(&mut self, key: &str) -> Result<bool> {
        Ok(MemStore::delete(self, key).is_some())
    }

    fn list(&self, prefix: Option<&str>) -> Result<Vec<(String, Vec<u8>)>> {
        let mut items: Vec<_> = MemStore::keys(self)
            .into_iter()
            .filter(|k| prefix.is_none_or(|p| k.starts_with(p)))
            .filter_map(|k| MemStore::get(self, &k).map(|v| (k, v.clone())))
            .collect();
        items.sort_by(|a, b| a.0.cmp(&b.0));
```

```
        Ok(items)
    }
}

// A naplózó FileStore delegálása: a fájlformátum és a commit-szerződés (r023)
// változatlan, a trait csak egységes felületet ad.
impl Storage for FileStore {
    fn get(&self, key: &str) -> Result<Option<Vec<u8>>> {
        Ok(FileStore::get(self, key).cloned())
    }

    fn set(&mut self, key: String, value: Vec<u8>) -> Result<()> {
        FileStore::set(self, key, value)
    }

    fn delete(&mut self, key: &str) -> Result<bool> {
        let existed = FileStore::get(self, key).is_some();
        FileStore::delete(self, key)?;
        Ok(existed)
    }

    fn list(&self, prefix: Option<&str>) -> Result<Vec<(String, Vec<u8>)>> {
        Ok(FileStore::list(self, prefix))
    }
}

/// A parancsok végrehajtásának eredménye.
#[derive(Debug, PartialEq, Eq)]
pub enum Response {
    Ok,
    Value(Vec<u8>),
    NotFound,
    Entries(Vec<(String, Vec<u8>)>),
}

/// Generikus parancsvégrehajtás bármely `Storage` implementációra.
///
/// A `S: Storage` bound garantálja, hogy a `Command` minden ágában a
/// `Storage` négy metódusa hívható. A `?Sized` feloldja az implicit `Sized`
/// kötést, így a függvény `&mut dyn Storage` (trait-objektum) értékkel is hívható
/// (a 16. fejezet `SharedStore` ezt használja).
pub fn apply<S: Storage + ?Sized>(store: &mut S, command: &Command) -> Result<Response> {
    Ok(match command {
        Command::Get(key) => match store.get(key)? {
            Some(value) => Response::Value(value),
            None => Response::NotFound,
        },
        Command::Set(key, value) => {
            store.set(key.clone(), value.clone().into_bytes())?;
            Response::Ok
        }
        Command::Delete(key) => {
            if store.delete(key)? {
                Response::Ok
            } else {
                Response::NotFound
            }
        }
        Command::List(prefix) => Response::Entries(store.list(prefix.as_deref())?),
    })
}

#[cfg(test)]
```

```rust
mod tests {
    use super::*;

    /// Szerződésteszt: minden `Storage` implementációnak ugyanígy kell viselkednie.
    fn contract<S: Storage>(store: &mut S) {
        store.set("k".into(), b"v".to_vec()).unwrap();
        assert_eq!(store.get("k").unwrap(), Some(b"v".to_vec()));
        assert!(store.delete("k").unwrap());
        assert_eq!(store.get("k").unwrap(), None);
        assert!(!store.delete("k").unwrap());

        store.set("beta".into(), b"2".to_vec()).unwrap();
        store.set("alpha".into(), b"1".to_vec()).unwrap();
        let all = store.list(None).unwrap();
        assert_eq!(all[0].0, "alpha"); // rendezett
        assert_eq!(
            store.list(Some("a")).unwrap(),
            vec![("alpha".to_string(), b"1".to_vec())]
        );
        assert_eq!(
            apply(store, &Command::Get("beta".into())).unwrap(),
            Response::Value(b"2".to_vec())
        );
    }

    #[test]
    fn mem_store_satisfies_contract() {
        contract(&mut MemStore::new());
    }

    #[test]
    fn file_store_satisfies_contract() {
        let dir = std::env::temp_dir().join(format!("rkv-storage-{}", std::process::id()));
        let _ = std::fs::remove_dir_all(&dir);
        std::fs::create_dir_all(&dir).unwrap();
        contract(&mut FileStore::open(dir.join("state.json"), dir.join("append.log")).unwrap());
    }

    #[test]
    fn existing_database_still_opens() {
        // Az r023 inherens API-jával írt adatbázis a Storage-on át is olvasható.
        let dir = std::env::temp_dir().join(format!("rkv-compat-{}", std::process::id()));
        let _ = std::fs::remove_dir_all(&dir);
        std::fs::create_dir_all(&dir).unwrap();
        let (snap, log) = (dir.join("state.json"), dir.join("append.log"));
        {
            let mut store = FileStore::open(&snap, &log).unwrap();
            store.set("a".into(), b"1".to_vec()).unwrap(); // inherens set
            store.compact().unwrap();
        }
        let store = FileStore::open(&snap, &log).unwrap();
        assert_eq!(Storage::get(&store, "a").unwrap(), Some(b"1".to_vec()));
    }
}
```

3. A `rkv-core/src/lib.rs`-ben vedd fel a modult, a `MemStore` aliast és a re-exportokat:

Kódblokk — fájl: `rkv-core/src/lib.rs` · művelet: beillesztés a `pub mod` sorok közé, illetve a `pub use` sorok után · ellenőrzés: `cargo build --workspace`

```rust
pub mod storage;

pub use storage::{apply, Response, Storage};
```

```rust
/// A memóriabeli `Storage`-implementáció könyvbeli neve.
pub type MemStore = Store;
```

4. Fordítsd le és futtasd a teszteket:

```
cd ~/rust-labor/rkv
cargo test -p rkv-core storage
```

Várt kimenet (rövidítve; a sorrend változhat):

```
running 3 tests
test storage::tests::existing_database_still_opens ... ok
test storage::tests::file_store_satisfies_contract ... ok
test storage::tests::mem_store_satisfies_contract ... ok
```

Ellenőrzési pont. Az `rkv-core` a `Storage` traitet, az `apply` függvényt, a `Response` enumot és a `MemStore` aliast exportálja; a `Store` és a `FileStore` mindkettő implementálja a `Storage`-t, a korábbi inherens API-jukkal együtt. A `cargo test --workspace` a korábbi tesztek mellett a 3 új tesztet is sikeresen futtatja, az `rkv-cli` változatlan.

12.1.6 Tipikus hibák és átállási csapdák

1. **A C++-os duck typing elvárása.** Ha egy Rust generikus függvényben a `T`-re `x.doSomething()`-et hívsz, a boundot ki kell mondanod. A hiba nem a hívóban, hanem a definícióban jelenik meg; sok C++-fejlesztő a hívás helyére görgetve keresi a hibát, és meglepődik.
2. **A `clone()` mindenütt a borrow checker „legyőzésére” és a generikusok egyszerűbbé tételére** nem idiomatikus. Ha a `Storage` `get` metódusa miatt klónozni kell a `Vec<u8>`-at, az lehet elfogadható, de gyakran jobb visszaadni `&[u8]`-ot vagy `Option<&Vec<u8>>`-at, ha az élettartam engedi. A laborban a `MemStore::get cloned()`-ot használ, mert a `Storage` trait a belső adattól független `Option<Vec<u8>>`-at ad vissza; ez konzisztens, de a forró úton némi másolással jár.
3. **Az `impl Trait` félreértése visszatérési típusként.** Az `impl Trait` nem hoz létre heterogén gyűjteményt. Ha egy függvény két különböző konkrét típust adna vissza, a kód nem fordul le; ilyenkor `dyn Trait` kell (lásd később).
4. **A `where` klauzula elkerülése.** Hosszú, egymásba ágyazott boundok esetén a `where` szakasz növeli az olvashatóságot, és közelebb áll a C++ concepts `requires` deklarációjához.

12.1.7 Feladatok

1. **(a) Lefordul-e és miért?** Az alábbi Rust kód lefordul-e? Ha nem, mi a hiba lényege?

```rust
fn print_debug<T>(x: T) {
    println!("{:?}", x);
}
```

2. **(b) C++-kód átírása idiomatikus Rustra.** Írd át az alábbi C++ sablont Rustra trait bounddal:

```cpp
template <typename T>
T add_twice(T a, T b) {
    return (a + b) + b;
}
```

Vigyázz: a Rust `+` művelet a `std::ops::Add` traitből jön; a `T`-nek nem elég `Add`-ot adni, az `Add Output` típusa is `T` kell legyen.

3. **(c) Implementációs feladat az rkv-ban.** Bővítsd a `Storage` traitet egy `contains` metódussal, amely igazat ad, ha a kulcs létezik. Implementáld `MemStore`-ra és `FileStore`-ra is. A `contains` hívja a `get`-et, és a `map(|v| v.is_some())` értéket adja vissza, vagy közvetlenül a `HashMap::contains_key`-et.

4. **(d) Hibajavítás / refaktorálás.** A jelenlegi `FileStore` a memóriában tartja a teljes adatbázist, és minden `set`/`delete` után teljesen újraírja a fájlt. Tervezd meg szövegesen, hogyan vezetnéd be a **snapshot + append-only log** perzisztenciát a `FileStore`-ba úgy, hogy a `Storage` trait szignatúrája ne változzon. Milyen hibamódokra kellene felkészülnöd?

12.1.8 Megoldások

1. **Nem fordul le.** A `println!("{:?}", x)` a `T`-re a `Debug` traitet követeli meg. Mivel a `T`-re nincs bound, a fordító a definícióban jelzi, hogy a `T` nem kielégítően korlátozott. A javítás: `fn print_debug<T: std::fmt::Debug>(x: T)` vagy `fn print_debug(x: &impl std::fmt::Debug)`.

2. **Rust megoldás:**

```rust
use std::ops::Add;

fn add_twice<T>(a: T, b: T) -> T
where
    T: Add<Output = T> + Clone,
{
    (a + b.clone()) + b
}

// Ha a típus Copy, a klónozás is elhagyható:
fn add_twice_copy<T>(a: T, b: T) -> T
where
    T: Add<Output = T> + Copy,
{
    (a + b) + b
}
```

Az `Add::add(self, rhs)` mindkét operandust **érték szerint** veszi át. A `(a + b) + b` kifejezésben az `a` egyszer fogy el, a `b` viszont kétszer szerepel: az első összeadásban elmozdul, a másodikban újra kellene. Ezért `a`-hoz nem kell semmilyen másolhatósági bound, a `b`-hez viszont vagy `Copy` (implicit másolat), vagy `Clone` (kifejezett `b.clone()`) szükséges. Ha mindkettő hiányzik, a fordító `E0382`-vel (use of moved value: `b`) áll meg.

3. **A `Storage` trait bővítése:**

```rust
pub trait Storage {
    // ... eddigi metódusok ...
    fn contains(&self, key: &str) -> Result<bool, Error> {
        Ok(self.get(key)?.is_some()) // default metódus
    }
}
```

A default metódust `MemStore` és `FileStore` örökli, de ha szükséges, felülírhatják.

4. **Tervezési válasz** (részletes kóddal itt nem kivitelezve). A `FileStore` a memóriabeli `HashMap`-et kiegészítené egy snapshot-fájllal és egy naplófájllal. A `set`/`delete` során a parancsot append-only módon naplóba írná (`SET`/`DEL` bejegyzések, sorozatszámmal), és a memóriabeli állapotot frissítené. A `persist` a napló kiírása után ritkán (pl. méret- vagy időalapú küszöb átlépésekor) új snapshotot készítene, és a naplót csonkolná. A fájlcserét atomikusan kellene megoldani: a snapshotot ideiglenes fájlba írjuk, majd `rename`-mel cseréljük, hogy a folyamat közbeni összeomlás ne hagyjon félkész állapotot. A betöltés során a napló bejegyzéseit a snapshotra játszva állítanánk helyre a memóriabeli állapotot. A `Storage` trait szignatúrája nem változna, mert a perzisztencia a `FileStore` belső ügye.

12.1.9 Rövid összefoglaló

Ebben az alfejezetben bevezettük a traitet mint viselkedési interfészt és a generikusokat trait boundokkal. A `trait` definíciójából és az `impl Trait for Type` blokkból kiindulva megmutattuk, hogy a boundok a generikus törzset már a definícióban ellenőrzik, szemben a C++ sablonok példányosításkori duck typingjával. A monomorfizálás Rustban ugyanúgy konkrét típusonként fordít, mint a C++ sablonpéldányosítás, de a fordító a boundok miatt előre garantálja az interfészt. Az `impl Trait` argumentumként a névtelen generikus paramétert, visszatérésként egyetlen konkrét típust jelent. A laborban létrehoztuk a `rkv-core/src/storage.rs`-ben a `Storage` traitet (`get`, `set`, `delete`, `list` metódusokkal), valamint a `MemStore` és `FileStore` implementációkat. A generikus `apply` függvény a `Command` enumot bármely `S: Storage` típusra futtatja. A `Storage` traitet a későbbi alfejezetek tovább bővítik és tesztelik.

12.1.10 Források és ellenőrzési hiányok

A fejezet a Rust Book (https://doc.rust-lang.org/book/) és a Rust Reference (https://doc.rust-lang.org/reference/) trait-, generikus-, trait bound-, monomorfizálás- és `impl Trait`-szakaszaira támaszkodik, amelyek stabil nyelvi fogalmak. A fordítói hibaüzenetek szövege a rögzített toolchainre (Rust 1.99.1) vonatkozik. A laborban szereplő `Error` típus a Saját hibatípusok: enum-hibák, thiserror, anyhow–I/O és szerializáció: fájlok, pufferelés, serde alfejezetek `thiserror`-alapú hibatípusa.

12.2 Asszociált típusok, koherencia, operátorok és derive

12.2.1 Motiváció

C++-ban a generikus kód gyakran a típus „belsejében” lévő típusokat használ: az `std::vector<T>::value_type`, az iterátor `value_type`-ja vagy egy konténer `key_type`-ja. Ezek konvenciók, amelyeket a template paraméterből kell kitalálni, és a fordító csak akkor tudja ellenőrizni, ha egy concept vagy `std::iterator_traits` segít. Rustban ezt a szerepet az **asszociált típus** tölti be: a trait részeként deklaráljuk, hogy az implementáló típus milyen típust ad hozzá. Így a generikus függvény egyértelműen hivatkozhat `T::Item`-re, anélkül hogy a hívó oldalán paraméterezni kellene. Ugyanez a filozófia vezet a koherencia-szabályhoz is: a Rust (az alap esetben) nem enged idegen trait idegen típusra való implementálását, mert így garantálja, hogy egy `(trait, típus)` párhoz globálisan egyetlen implementáció tartozik. Ez elsőre korlátozásnak tűnik, valójában a generikus API-k kiszámíthatóságát védi.

12.2.2 Asszociált típusok

Definíció. Az *asszociált típus* (associated type) egy traitben deklarált típusnév, amelyet az implementáló típus ad meg. A trait törzsében `type Item;` formában szerepel, az implementációban pedig `type Item = KonkrétTípus;`.

A standard könyvtár legismertebb példája az `Iterator`:

```
trait Iterator {
    type Item;
    fn next(&mut self) -> Option<Self::Item>;
}
```

Az asszociált típus előnye, hogy a generikus kód nem `Iterator<u32>`-t ír, hanem egyszerűen `I: Iterator`, és a függvényben `I::Item`-ként hivatkozik az elem típusára. Ha a `Iterator` generikus paramétert használna, akkor egy típushoz több `Iterator<u32>`, `Iterator<String>` implementáció is tartozhatna, ami a legtöbb iterátornál értelmetlen: egy iterátor egyfajta elemet ad.

Szabály. Ha egy típushoz természeténél fogva egyetlen kapcsolódó típus tartozik, asszociált típust használj. Ha ugyanarra a típusra több, egymástól független implementációra van szükséged, amelyek csak a paraméterben térnek el, generikus paramétert használj.

A különbség szemléltetése:

```
// Generikus paraméter: több implementáció lehetséges.
trait Parser<T> {
    fn parse(&self, input: &str) -> Option<T>;
}

impl Parser<i32> for MyParser { /* ... */ }
impl Parser<String> for MyParser { /* ... */ }

// Asszociált típus: egy implementáció, egy Output.
trait Parser {
    type Output;
    fn parse(&self, input: &str) -> Option<Self::Output>;
}

impl Parser for MyParser {
    type Output = i32;
    // ...
}
```

C++-ban hasonló célt szolgál a tag-típus vagy az `std::iterator_traits`:

```
template <typename T>
struct my_iterator {
    using value_type = T;
    // ...
};

template <typename T>
concept an_iterator = requires(T it) {
    typename T::value_type;
    { it.next() } -> std::same_as<typename T::value_type>;
};
```

A Rust fordító azonban nem csak jelzi, hanem a trait definíciójakor kötelezi az implementálót, hogy az asszociált típust is megadja. Ha lemarad, a hibakód jellemzően E0046: a trait nem teljes, mert hiányzik az `Item` asszociált típus.

12.2.3 Koherencia és az orphan rule

Definíció. Az *orphan rule* (árvaszabály) egyszerű bevezető formában azt mondja ki: idegen traitet idegen típusra nem implementálhatsz. A *koherencia* azt jelenti, hogy egy adott, teljesen megadott `(trait paraméterekkel, típus)` párhoz (például `Add<Rhs>` vagy `From<T>` konkrét `Rhs`-sel, illetve `T`-vel és a `Self` típussal) legfeljebb egy implementáció tartozhat; ugyanarra a típusra különböző paraméterekkel több implementáció is lehet (`From<A>` és `From<B>`), így nem kerülhetnek ütközésbe különböző crate-ek impl-jei.

Pontosabb szabály. Egy `impl Trait<T1, ..., Tn> for T0` akkor megengedett, ha vagy a trait lokális az aktuális crate-ben, vagy a `T0, T1, ..., Tn` típusok közül *legalább az egyik lokális* típus (és az ezt megelőző típusok között nincs "fedetlen" típusparaméter). A saját típus tehát nem csak `Self` lehet, hanem a trait típusparamétere is. Két megengedett ellenpélda az egyszerű szabályra:

```
use std::ops::Add;
struct Meter(f64);

// Idegen trait (From), idegen Self (String), de a trait paramétere saját típus: szabályos.
impl From<Meter> for String {
    fn from(m: Meter) -> String { format!("{} m", m.0) }
}

// Ugyanez az operátor-trait-ekkel: idegen Add, idegen Self (f64), saját Rhs: szabályos.
impl Add<Meter> for f64 {
    type Output = f64;
    fn add(self, rhs: Meter) -> f64 { self + rhs.0 }
}
```

(A „fundamental" típusok, például a `Box<T>` és a `&T`, átlátszóak: `impl Display for Box<Meter>` szabályos, mert a `Box<Meter>` a saját típust burkolja.) Két tiltott ellenpélda, a hibakódokkal (a kimenetek Rust 1.99.1-ből):

```
// error[E0117]: only traits defined in the current crate can be implemented for types defined
↪  outside of the crate
// A Vec idegen; a benne lévő saját típus nem teszi a Vec<Meter>-t lokálissá.
impl std::fmt::Display for Vec<Meter> { /* ... */ }

// error[E0210]: type parameter `T` must be used as an argument to some local type
// Blanket impl idegen traitre: a T "fedetlen" típusparaméter.
impl<T> std::fmt::Display for T { /* ... */ }
```

Ez a C++-tól eltérő modell. C++-ban a `std::hash` teljes specializálása user-defined típusra megengedett a `std` névtérben:

```
struct MyType { int x; };

namespace std {
    template<>
    struct hash<MyType> {
        size_t operator()(const MyType& t) const noexcept {
            return std::hash<int>{}(t.x);
        }
    };
}
```

Rustban nem nyitogathatjuk a standard könyvtárat. Ha viszont a saját típusunkra akarunk idegen traitet implementálni, az szabályos (a saját típus `Self`):

```rust
struct MyType { x: i32 }

impl std::fmt::Display for MyType {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        write!(f, "MyType({})", self.x)
    }
}
```

A tipikusan tiltott eset az, amikor mindkettő idegen:

```rust
// Ez a változat szándékosan nem fordul le:
impl std::fmt::Display for Vec<i32> {
    // ...
}
```

A hibakód E0117, lényege: a `Display` trait és a `Vec` típus is idegen; a Rust megtagadja az implementációt, mert az egy másik crate-ben okozhatna ütközést. A gyakori megkerülés a **newtype** minta: becsomagoljuk az idegen típust egy lokális típusba. A newtype tehát gyakori eszköz, nem kötelező kerülőút: ha a trait paramétere saját típus (mint a `From<Meter> for String`), nincs szükség burkolóra.

```rust
struct WrappedVec(Vec<i32>);

impl std::fmt::Display for WrappedVec {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        write!(f, "{:?}", self.0)
    }
}
```

Szabály. Ha idegen típusra kell idegen trait képessége, hozz létre egy lokális `struct Wrapper(InnerType)` típust, és arra implementáld a traitet. A wrapper mezőt ne fedd fel automatikusan `Deref` implementációval, mert az elrejtheti a típushatárokat; a későbbi Newtype, PhantomData, typestate alfejezet ezt részletesen tárgyalja.

12.2.4 Operátor-túlterhelés trait-ekkel

Rustban nincs szabad `operator+` függvény, hanem a standard könyvtár `std::ops` moduljának trait-jeit implementáljuk. A `+` kifejezés az `Add` trait `add` metódusává íródik át, a `[]` az `Index`/`IndexMut` hívása.

```rust
struct Point { int x, y; };

Point operator+(Point a, Point b) {
    return {a.x + b.x, a.y + b.y};
}
```

Rustban:

```rust
#[derive(Debug, Clone, PartialEq, Eq)]
struct Point { x: i32, y: i32 }

impl std::ops::Add for Point {
    type Output = Point;

    fn add(self, rhs: Self) -> Self::Output {
        Self { x: self.x + rhs.x, y: self.y + rhs.y }
    }
}
```

Lényeges különbség: az eredmény típusa nem a függvény szignatúrájában van, hanem az asszociált `Output` típus adja. Ezzel például `Vector + Vector -> Vector` és `Vector * Vector -> f64` is kifejezhető, de egy típusra egy `Add` implementáció egy `Rhs` paraméterre csak egy `Output`-tal rendelkezhet.

Az `Index` szintén trait, és a visszatérési érték kötelezően referencia:

```
impl std::ops::Index<usize> for Point {
    type Output = i32;

    fn index(&self, index: usize) -> &Self::Output {
        match index {
            0 => &self.x,
            1 => &self.y,
            _ => panic!("Point index out of range"),
        }
    }
}
```

C++-ban a `const` és nem `const operator[]` különbözteti meg az olvasást és az írást; Rustban az `Index` csak olvas, az `IndexMut` pedig `&mut Self::Output`-ot ad. Fontos, hogy az `Index` szerződése pánik hibás index esetén, nem ad `Option`-t. Ha az indexelés hibázhat, írj inkább `get` metódust `Option`/`Result` visszatéréssel.

Az egyenlőséghez és rendezéshez a `PartialEq`, `Eq`, `PartialOrd`, `Ord` trait-ek tartoznak. A `PartialEq` lehet részleges egyenlőség: például `f32::NAN != f32::NAN` igaz, ezért a lebegőpontos típusok nem `Eq`. Az `Eq` marker trait, amely a teljes reflexivitást ígéri, vagyis minden `a`-ra `a == a`. A `PartialOrd`/`Ord` ugyanezt a részleges/teljes megkülönböztetést hozza a rendezésre. A `Hash` trait pedig arra való, hogy egyenlő értékek hash-e megegyezzen, de a hash nem titkosítási eszköz.

12.2.5 Derive-lok és szemantikai következményeik

A `#[derive(...)]` arra utasítja a fordítót, hogy ismert szabályok alapján implementálja a felsorolt traiteket. A leggyakoribb derive-ok:

```
#[derive(Debug, Clone, PartialEq, Eq, Hash, Default, PartialOrd, Ord)]
struct Command {
    name: String,
    priority: u32,
}
```

Szabály. A derive-ok mezőnkénti (strukturális) szemantikát adnak. A generált kód minden mezőre megköveteli ugyanazt a traitet, így ha egy mező nem `Eq`, a `Eq` derive sem fordul le.

A fontosabb szemantikai következmények:

- `Debug`: fejlesztői, `{:?}` formátum; nem a felhasználói `Display`.
- `Clone`: explicit, potenciálisan drága másolás; soha nem implicit.
- `PartialEq`/`Eq`: mezőnkénti strukturális egyenlőség. Ha lebegőpontos mező van, az `Eq` derive hibás, mert `f64` nem `Eq`.
- `Hash`: a mezők hash-ét kombinálja, konzisztens a `PartialEq`-vel; ha ketten egyenlők, a hash-ük is azonos. Nem kriptográfiai hash.
- `Default`: minden mezőre a `Default` értékét használja, így a mezőknek is `Default`-nak kell lenniük.

- `PartialOrd`/`Ord`: a mezősorrendben lexikografikus rendezést ad. Ez gyakran nem a természetes üzleti rendezés; ha a mezők sorrendje véletlenszerű, a rendezés is kiszámíthatatlan. Ilyenkor inkább kézzel implementáld a traitet.

12.2.6 Blanket implementációk röviden

A *blanket implementáció* (blanket implementation) olyan generikus impl, amely egy egész típuscsaládra vonatkozik. A standard könyvtárban ilyen például:

```
impl<T: fmt::Display> ToString for T {
    fn to_string(&self) -> String {
        // ...
    }
}
```

Minden `Display`-et implementáló típushoz ad egy `ToString` implementációt. Ugyanígy működik a `From`/`Into` páros: ha `T: From<U>`, akkor `U: Into<T>` is automatikusan teljesül. A blanket impl-ek azért fontosak, mert csökkentik a boilerplate-et, de a koherencia miatt a fordító ellenőrzi, hogy ne ütközzenek kézzel írt impl-ekkel. Az orphan rule különösen szigorú blanket impl-eknél, mert egy idegen blanket impl sok típusra terjedhet ki.

12.2.7 Kidolgozott példa: labor az asszociált típusokkal és operátorokkal

Munkakönyvtár: `~/rust-labor/r025/`

Hozd létre a labort, és írd meg a `src/main.rs` fájlt:

```
cargo new r025
cd r025
```

`Cargo.toml`:

```
[package]
name = "r025"
version = "0.1.0"
edition = "2024"
```

Ha a telepített toolchain nem támogatja a 2024-es editiont, ne térj vissza a 2021-re: a könyv minden kódja és szabálya (például a `gen` foglalt kulcsszó, a `unsafe extern`, az `impl Trait` rögzítési szabályai) a 2024-es editionre épül, ezért a megoldás a toolchain frissítése a Eszközlánc: rustup, cargo, rustfmt, clippy, dokumentáció szerint (`rust-toolchain.toml`: 1.99.1).

`src/main.rs`:

```
use std::fmt;

/// A `start..=end` zárt tartomány bejárása. A `done` jelző kell, mert a
/// `u32::MAX` végpontnál a `current += 1` túlcsordulna (debug: pánik,
/// release: 0-ra fordul, és a bejárás újrakezdődne).
struct Counter {
    current: u32,
    end: u32,
    done: bool,
}

impl Counter {
    fn new(start: u32, end: u32) -> Self {
        Self { current: start, end, done: start > end }
```

```
    }
}

impl Iterator for Counter {
    type Item = u32;

    fn next(&mut self) -> Option<Self::Item> {
        if self.done {
            return None;
        }
        let value = self.current;
        match value.checked_add(1) {
            Some(next) if value < self.end => self.current = next,
            _ => self.done = true, // utolsó elem vagy túlcsordulás: vége
        }
        Some(value)
    }
}

#[derive(Debug, Clone, PartialEq, Eq, Hash, Default, PartialOrd, Ord)]
struct Point {
    x: i32,
    y: i32,
}

impl std::ops::Add for Point {
    type Output = Point;

    fn add(self, rhs: Self) -> Self::Output {
        Self {
            x: self.x + rhs.x,
            y: self.y + rhs.y,
        }
    }
}

impl std::ops::Index<usize> for Point {
    type Output = i32;

    fn index(&self, index: usize) -> &Self::Output {
        match index {
            0 => &self.x,
            1 => &self.y,
            _ => panic!("Point index out of range"),
        }
    }
}

struct WrappedVec(Vec<i32>);

impl fmt::Display for WrappedVec {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "[")?;
        for (i, value) in self.0.iter().enumerate() {
            if i > 0 {
                write!(f, ", ")?;
            }
            write!(f, "{value}")?;
        }
        write!(f, "]")
    }
}
```

```rust
fn main() {
    let sum = Point { x: 1, y: 2 } + Point { x: 3, y: 4 };
    println!("sum = {sum:?}");

    let p = Point::default();
    println!("p = {p:?}, p[0] = {}", p[0]);

    let counter = Counter::new(1, 3).collect::<Vec<_>>();
    println!("counter = {counter:?}");

    let wrapped = WrappedVec(vec![10, 20, 30]);
    println!("wrapped = {wrapped}");
}
```

Futtasd:

```
cargo run
```

Várt kimenet:

```
sum = Point { x: 4, y: 6 }
p = Point { x: 0, y: 0 }, p[0] = 0
counter = [1, 2, 3]
wrapped = [10, 20, 30]
```

A `Counter` zárt tartományt jár be, ezért a végpont is eleme. A `u32::MAX` végpontnál a naiv `self.current += 1` túlcsordulna (debug buildben pánik, release-ben 0-ra fordulás és végtelen bejárás), ezért a `done` jelző és a `checked_add` kell. Az ellenőrző tesztek:

```rust
#[cfg(test)]
mod tests {
    use super::Counter;

    #[test]
    fn max_endpoint_yields_exactly_one_element_then_none() {
        let mut c = Counter::new(u32::MAX, u32::MAX);
        assert_eq!(c.next(), Some(u32::MAX));
        assert_eq!(c.next(), None);
        assert_eq!(c.next(), None); // tartósan None
    }

    #[test]
    fn ranges_match_std() {
        assert_eq!(Counter::new(1, 3).collect::<Vec<_>>(), vec![1, 2, 3]);
        assert_eq!(Counter::new(4, 3).count(), 0);
        let near_max: Vec<u32> = Counter::new(u32::MAX - 1, u32::MAX).collect();
        assert_eq!(near_max, vec![u32::MAX - 1, u32::MAX]);
    }
}
```

A példa egyszerre mutatja az asszociált típust (`Iterator::Item`), a derive-olt strukturális egyenlőséget és rendezést, az `Add`/`Index` operátor-trait-eket, valamint a newtype-mintát az orphan rule megkerülésére.

Továbbvitt eredmény. Az itt bemutatott `Iterator::Item` asszociált típus és a derive-készlet a(z) A standard trait-ek: konverziók, formázás, Default, Deref, Borrow alfejezetben a `Display`, `Deref` és `Borrow` implementációk, a(z) Iterátorok: lusta adapterek és saját iterátor alfejezetben a saját iterátor, a(z) Vec, HashMap, BTreeMap és társaik; az Entry API alfejezetben a hash-kulcsok `Hash`/`Eq` követelményei, a(z) Deklaratív és procedurális makrók alfejezetben a derive-makrók, valamint a(z) Newtype, PhantomData, typestate és const generikusok alfejezetben a newtype-minta alapjául szolgál.

12.2.8 Tipikus hibák és átállási csapdák

1. **Generikus paraméter asszociált típus helyett.** Ha egy típushoz egyértelműen egyetlen kapcsolódó típus tartozik, a generikus paraméter fölösleges `impl`-ekhez és turbofish-zűrzavarhoz vezet. Például `trait Storage<K, V>` helyett inkább `type Key` és `type Value` asszociált típus, ha a típus természetesen meghatározza a kulcsot és az értéket.
2. **`Eq`, `Ord`, `Hash` derive lebegőpontos mezőkkel.** Az `f32`/`f64` típusok nem implementálják az `Eq`, `Ord` és `Hash` traiteket, mert a `NaN` és a `-0.0`/`+0.0` szemantikája sértené a szerződést. Ha ilyen mező van, ne erőltesd a derive-ot.
3. **`Index` hibakezelésre.** Az `Index` szerződése pánik; ha a kulcs hiányozhat, a `HashMap`-nél használd a `get` metódust, és add vissza `Option`-t. Az `Index` csak akkor való, ha a hibás index programhiba.
4. **Newtype és `Deref` automatikus kitevése.** Sokan `Deref`-fel teszik láthatóvá a belső típust, hogy ne kelljen delegálni a metódusokat. Ez elrejti a típushatárt, és a későbbi override-ok meglepő viselkedést okozhatnak. A newtype célja éppen a határok megőrzése.
5. **`PartialOrd` derive mezősorrendre bízva.** A mezősorrend lexikografikus rendezése gyakran nem a kívánt üzleti rendezés. Ha a rendezésnek van jelentése, írj explicit `impl`-et, vagy dokumentáld a mezősorrendet.

12.2.9 Feladatok

12.2.9.1 a) Lefordul-e és miért?

```rust
impl std::fmt::Display for Vec<i32> {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        write!(f, "{self:?}")
    }
}
```

Válaszolj: lefordul-e ez a kód? Ha nem, melyik szabályt sérti, és hogyan javítanád?

12.2.9.2 b) C++-kód átírása idiomatikus Rustra

Add meg a Rust megfelelőjét a következő C++ típusnak, beleértve az összeadást és az egyenlőségvizsgálatot:

```cpp
struct Size {
    int w;
    int h;
};

Size operator+(Size a, Size b) {
    return {a.w + b.w, a.h + b.h};
}

bool operator==(Size a, Size b) {
    return a.w == b.w && a.h == b.h;
}
```

12.2.9.3 c) Implementációs feladat az rkv-ban

Az `rkv-core` crate `Command` enumjára add a `Debug`, `Clone`, `PartialEq`, `Eq`, `Hash` derive-okat. Az enum a `Get(String)`, `Set(String, Vec<u8>)`, `Delete(String)`, `List(Option<String>)` változatokat tartalmazza. Indokold meg, miért nem adunk `PartialOrd`-ot `Ord`-dal.

12.2.9.4 d) Hibajavítás

Az alábbi derive nem fordul le. Miért? Javítsd ki, és indokold a választott megoldást.

```
#[derive(Debug, Clone, PartialEq, Eq, PartialOrd, Ord, Hash)]
struct Celsius(f64);
```

12.2.10 Megoldások

12.2.10.1 a)

Nem fordul le. A `Display` trait és a `Vec<i32>` típus is idegen, ezért az orphan rule (hibakód E0117) tiltja az implementációt. Megoldás a newtype:

```
struct ListVec(Vec<i32>);

impl std::fmt::Display for ListVec {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        write!(f, "{:?}", self.0)
    }
}
```

12.2.10.2 b)

```
#[derive(Debug, Clone, PartialEq, Eq)]
struct Size {
    w: i32,
    h: i32,
}

impl std::ops::Add for Size {
    type Output = Size;

    fn add(self, rhs: Self) -> Self::Output {
        Self {
            w: self.w + rhs.w,
            h: self.h + rhs.h,
        }
    }
}
```

Az egyenlőséget a `PartialEq/Eq` derive generálja; az összeadást az `Add` trait asszociált `Output` típusával adjuk meg.

12.2.10.3 c)

```rust
#[derive(Debug, Clone, PartialEq, Eq, Hash)]
pub enum Command {
    Get(String),
    Set(String, Vec<u8>),
    Delete(String),
    List(Option<String>),
}
```

A `String` és `Vec<u8>` mindegyik felsorolt traitet implementálja, így a derive lefordul. `PartialOrd`-ot és `Ord`-ot szándékosan nem adunk, mert a parancsok között nincs természetes rendezés; a sorrend bevezetése félrevezető lenne a protokoll-feldolgozásban és a későbbi bővítéseknél.

12.2.10.4 d)

A `f64` típus nem implementálja az `Eq`, `Ord` és `Hash` traiteket a `NaN` és az előjeles nulla problémái miatt, ezért a derive nem fordul le. Javítás: távolítsd el az `Eq`, `Ord`, `Hash` derive-okat, vagy ha szükség van teljes egyenlőségre és hash-re, tárold a biteket például `u64`-ben, és ott kezeld a kanonikus `-0.0` problémát. A legegyszerűbb változat:

```rust
#[derive(Debug, Clone, PartialEq, PartialOrd)]
struct Celsius(f64);
```

12.2.11 Rövid összefoglaló

Az asszociált típus a trait részeként deklarált típusnév, amelyet az implementáló típus ad meg; a generikus paraméter helyett akkor használjuk, ha egy típushoz egyetlen kapcsolódó típus tartozik, mint az `Iterator::Item` esetében. Az orphan rule kimondja, hogy idegen traitet idegen típusra nem implementálhatunk; a megoldás a newtype, például `struct WrappedVec(Vec<i32>)`. Az operátorok trait-ek: az `Add` asszociált `Output` típussal, az `Index` pánikkal, míg a `PartialEq`/`Eq`, `PartialOrd`/`Ord` és `Hash` a részleges/teljes szemantikát rögzítik. A `#[derive(Debug, Clone, PartialEq, Eq, Hash, Default, PartialOrd, Ord)]` mezőnkénti strukturális implementációt generál, ezért a mezőknek is teljesíteniük kell a megfelelő traiteket. A blanket impl-ek, mint az `impl<T: Display> ToString for T`, a koherencia szabályai mellett nyújtanak automatikus implementációkat. **Továbbvitt eredmények:** az `Iterator::Item` asszociált típus és a derive-készlet alapozza meg a(z) A standard trait-ek: konverziók, formázás, Default, Deref, Borrow, a(z) Iterátorok: lusta adapterek és saját iterátor, a(z) Vec, HashMap, BTreeMap és társaik; az Entry API, a(z) Deklaratív és procedurális makrók és a(z) Newtype, PhantomData, typestate és const generikusok alfejezeteket.

12.2.12 Források és ellenőrzési hiányok

A Rust nyelvi szabályok a stabil referenciadokumentációból származnak: https://doc.rust-lang.org/book/, https://doc.rust-lang.org/reference/, https://rust-lang.github.io/api-guidelines/. A pontos hibaüzenetek szövege verziófüggő; a hibakódok (E0046, E0117) stabilak. A 2024-es edition elérhetősége az adott stable toolchainben ellenőrzendő. A lebegőpontos típusok `Eq`/`Ord`/`Hash` hiánya a Rust stabil szabálya.

12.3 A standard trait-ek: konverziók, formázás, Default, Deref, Borrow

12.3.1 Motiváció

A C++-ban a konverziók, a formázás és a smart pointerek viselkedése gyakran implicit: az egyargumentumú konstruktorok konvertálnak, az `operator<<` túlterhelhető, a `shared_ptr` a `->` operátort túlterheli, a `std::string` és a `const char*` között szabad az átjárás. A Rust ezeket a műveleteket explicit, jól meghatározott trait-ekkel szabályozza. A C++-fejlesztőnek újra kell tanulnia, hogy a konverziókat kérni kell, a formázás a `Display` és `Debug` trait-ekkel történik, a pointerek használatát pedig a `Deref` trait biztosítja. Ez a fejezet a standard könyvtár leggyakrabban használt, az idiómát meghatározó traitjeit veszi sorra.

12.3.2 Fogalmak és szabályok

12.3.2.1 From/Into és TryFrom/TryInto

Definíció. A `From<T>` trait a `T` típusból való konverziót írja le: `fn from(value: T) -> Self`. Az `Into<U>` ennek a fordítottja: ha `U` implementálja a `From<T>`-t, akkor a `T` típus automatikusan implementálja az `Into<U>`-t. A `TryFrom` és `TryInto` ugyanez, de a konverzió hibát adhat vissza: `fn try_from(value: T) -> Result<Self, Self::Error>`.

Szabály. Saját típushoz mindig a `From`-ot vagy a `TryFrom`-ot implementáljuk, mert az `Into`/`TryInto` automatikusan elérhetővé válik. A `From` implementációnak mindig teljesnek és hibamentesnek kell lennie; ha a konverzió hibázhat, használjunk `TryFrom`-et. A `?` operátor a `From`-ot használja a hibakonverzióhoz: ha a hiba típusa `E1`, a függvény hibatípusa `E2`, és `E2` implementálja a `From<E1>`-et, akkor a `?` automatikusan hívja az `E2::from(E1)`-et.

12.3.2.2 AsRef, Borrow, ToOwned

Definíció. Az `AsRef<T>` lehetővé teszi egy típusból `&T` referencia készítését: `fn as_ref(&self) -> &T`. A `Borrow<T>` hasonló, de szemantikailag erősebb: a kölcsönzött értéknek ugyanazt a hash- és egyenlőség-relációt kell képviselnie, mint az eredeti típus. A `ToOwned` a `Borrow` fordítottja: `fn to_owned(&self) -> Owned`, ahol `Owned` egy olyan típus, amely `Borrow<Self>`-et implementál.

Szabály. Az `AsRef`-et gyakran használjuk függvényparaméterekben, hogy a hívó rugalmasan adhasson át `String`, `&str` vagy más típust. A `Borrow`-t elsősorban a `HashMap` kulcstípusainál használjuk: a `HashMap<K, V>` `get` metódusa `&Q` paramétert vár, ahol `K: Borrow<Q>`. Ez lehetővé teszi, hogy `String` kulcsú mapet `&str`-rel keressünk.

12.3.2.3 Display vs. Debug

Definíció. A `Display` trait a felhasználóbarát, humán olvasásra szánt formátumot írja le: `fn fmt(&self, f: &mut Formatter<'_>) -> fmt::Result`. A `Debug` trait a fejlesztői, diagnosztikai formátumot szolgálja. A `Debug` automatikusan `derive`-olható, a `Display`-et kézzel kell írni.

Szabály. A `Display` implementálása után a típus automatikusan elérhető a `{}` formátummal, a `Debug` pedig a `{:?}`-val. A formázási makrók (`println!`, `format!`, stb.) a `format_args!`

makrót használják, és a formátum-specifikátorok a `std::fmt` modulban vannak. A `Debug` minden típushoz legyen elérhető, a `Display`-et csak akkor írjuk, ha valódi, felhasználóbarát szöveges reprezentáció létezik.

12.3.2.4 Default és FromStr

Definíció. A `Default` trait az „üres” vagy alapértelmezett értéket adja: `fn default() -> Self`. A `FromStr` a szövegből való parsingot írja le: `fn from_str(s: &str) -> Result<Self, Self::Err>`. A `str::parse` metódus használja ezt a traitet.

Szabály. A `Default`-ot gyakran `#[derive(Default)]`-vel implementáljuk, ha minden mezőnek van `Default`-ja. A `FromStr`-t a szöveges protokollok, parancssori argumentumok feldolgozásához használjuk, és a hiba típusának egyértelműnek kell lennie.

12.3.2.5 Deref: smart pointerek, nem öröklés

Definíció. A `Deref` trait definiálja a `*` operátor viselkedését, és lehetővé teszi a deref coercion-t: ha a `T` implementálja a `Deref<Target=U>`-t, akkor a `&T` automatikusan `&U`-vá konvertálható ott, ahol a `&U` típus elvárt.

Szabály. A `Deref`-et csak smart pointerekhez és ahhoz hasonló „transzparens” burkolókhoz implementáljuk, például `Box<T>`, `Rc<T>`, `String` (Deref a `str`-re). A `Deref` nem használható öröklés-szimulációra: nem helyettesíti a trait-eket, és a deref coercion nem jelent automatikus altípus-relációt. Ha a `Deref`-et túlhasználjuk, a kód nehezen olvashatóvá válik, mert a metódushívások látszólag független típusokon futnak.

12.3.3 C++ és Rust összevetés

12.3.3.1 Implicit konverziók vs. From/Into

C++-ban egy egyargumentumú konstruktor implicit konverzióként működik, hacsak nem `explicit`:

```cpp
struct Port {
    int value;
    Port(int v) : value{v} {} // implicit
};

void print_port(Port p) {
    std::cout << p.value << '\n';
}

print_port(8080); // működik, az int Port-tá konvertálódik
```

Rustban nincs implicit konverzió; a konverziót explicit módon kell kérni:

```rust
struct Port { value: u16 }

impl From<u16> for Port {
    fn from(value: u16) -> Self {
        Port { value }
    }
}

fn print_port(p: Port) {
```

```
    println!("{}", p.value);
}

print_port(8080.into()); // explicit konverzió
```

A különbség: a Rust `Into` használata világosan jelzi a konverziót, és a koherencia-szabályok megakadályozzák a kétértelmű vagy nem szándékos konverziókat.

12.3.3.2 Az `operator<<` túlterhelése vs. Display

C++-ban a `std::ostream& operator<<` túlterhelése a formázás szokásos módja:

```
struct Port {
    int value;
    friend std::ostream& operator<<(std::ostream& os, const Port& p) {
        return os << "port " << p.value;
    }
};
```

Rustban a `Display` traitet implementáljuk:

```
impl std::fmt::Display for Port {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        write!(f, "port {}", self.value)
    }
}
```

A Rust megoldás nincs az áramlási operátorra építve, és a formátum-specifikátorok egységesek.

12.3.3.3 `std::hash` és kulcsok vs. Borrow

C++-ban a `std::map<std::string, int>` keresésekor `std::string` kulcsot kell adni, vagy a C++14 óta heterogén keresést kell használni. A Rust `HashMap<String, V>` `get` metódusa `&str`-t is elfogad a `Borrow` trait miatt:

```
use std::collections::HashMap;

let mut map = HashMap::new();
map.insert(String::from("key"), 42);
// A get &str-t vár, mert a String Borrow<str>.
let value = map.get("key"); // &str
```

12.3.3.4 Öröklés vs. Deref

C++-ban az osztályhierarchiák természetesek:

```
class Base {
public:
    virtual void foo() {}
};

class Derived : public Base {};
```

Rustban nincs implementációs öröklés; a `Deref` nem pótolja:

```rust
struct Base;
impl Base {
    fn foo(&self) {}
}

struct Derived { base: Base }

impl std::ops::Deref for Derived {
    type Target = Base;
    fn deref(&self) -> &Base {
        &self.base
    }
}
// Derived.foo() működik a deref coercion révén, de ez nem öröklés:
// a polimorfizmusra trait objektumok valók (lásd [Trait objektumok és dinamikus
↪  diszpécs](#r027)).
```

12.3.4 Kidolgozott példa: HostPort típus

A következő példában egy `HostPort` típust készítünk, amely egy host:port párt reprezentál. Ez egyszerű szövegpár, nem hálózati validáció: a parser elfogad üres hostot (`:8080`), a `Default` üres hostot és 0 portot ad, a `Display` pedig nem rögzít IPv6-szabályt (az IPv6-címhez szögletes zárójel kellene); a név a konverziós traitek bemutatására való, valódi címzéshez a `std::net::SocketAddr` használandó. Megmutatjuk a `From`, `TryFrom`, `Display`, `Debug`, `Default`, `FromStr`, `AsRef` implementálását.

```rust
use std::fmt;
use std::str::FromStr;

#[derive(Debug, Clone, PartialEq, Eq, Hash, Default)]
pub struct HostPort {
    pub host: String,
    pub port: u16,
}

impl HostPort {
    pub fn new(host: impl Into<String>, port: u16) -> Self {
        HostPort { host: host.into(), port }
    }

    pub fn host(&self) -> &str {
        &self.host
    }
}

impl fmt::Display for HostPort {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{}:{}", self.host, self.port)
    }
}

impl From<(String, u16)> for HostPort {
    fn from((host, port): (String, u16)) -> Self {
        HostPort { host, port }
    }
}

impl From<(&str, u16)> for HostPort {
    fn from((host, port): (&str, u16)) -> Self {
        HostPort { host: host.to_string(), port }
    }
```

```
}

impl TryFrom<&str> for HostPort {
    type Error = HostPortParseError;

    fn try_from(s: &str) -> Result<Self, Self::Error> {
        let (host, port_str) = s
            .rsplit_once(':')
            .ok_or(HostPortParseError::MissingColon)?;
        let port: u16 = port_str
            .parse()
            .map_err(|_| HostPortParseError::InvalidPort(port_str.to_string()))?;
        Ok(HostPort::new(host, port))
    }
}

impl FromStr for HostPort {
    type Err = HostPortParseError;

    fn from_str(s: &str) -> Result<Self, Self::Err> {
        HostPort::try_from(s)
    }
}

impl AsRef<str> for HostPort {
    fn as_ref(&self) -> &str {
        &self.host
    }
}

#[derive(Debug, Clone, PartialEq, Eq)]
pub enum HostPortParseError {
    MissingColon,
    InvalidPort(String),
}

impl fmt::Display for HostPortParseError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            HostPortParseError::MissingColon => write!(f, "missing ':' in host:port"),
            HostPortParseError::InvalidPort(s) => write!(f, "invalid port: {s}"),
        }
    }
}
```

Ez a kód egy teljes, lefordítható példa. A `HostPort` implementálja a `From`-ot tuple-ből, a `TryFrom`-et `&str`-ből, a `FromStr`-öt, a `Display`-t, a `Debug`-ot derive-dal, a `Default`-ot derive-dal, és az `AsRef<str>`-et. A `TryFrom` és a `FromStr` ugyanazt a hibát használja. A példa használata:

```
fn main() {
    let p = HostPort::from(("localhost", 8080));
    println!("{}", p); // Display
    println!("{:?}", p); // Debug

    let parsed: HostPort = "localhost:8080".parse().unwrap();
    assert_eq!(p, parsed);

    let s = format!("Host: {}", p.as_ref()); // AsRef<str>
    println!("{s}");
}
```

12.3.5 Labor: FromStr és Display az rkv-core-ban

Munkakönyvtár: `~/rust-labor/rkv/`

A szerver és a CLI ugyanazt a parancsformátumot használja, ezért a formázást és az értelmezést a `Command` enumra bízzuk, a `Display` és a `FromStr` trait implementálásával. A kiinduló állapot a Trait-ek és generikusok: sablonok és concepts helyett ellenőrzési pontja: a `Command` az `rkv-core/src/command.rs`-ben él (`Get(String)`, `Set(String, String)`, `Delete(String)`, `List(Option<String>)`, `to_line`), az `rkv-cli` pedig saját `parse_command` függvénnyel értelmezi a sorokat (Mintaillesztés: match, if let, let else és a minták). Új `Command` típust nem hozunk létre, és új hibatípust sem: az értelmező a kanonikus `rkv_core::Error::InvalidCommand(String)` változatot adja vissza (Saját hibatípusok: enum-hibák, thiserror, anyhow).

A támogatott tartomány. A `Display` és a `FromStr` a Mintaillesztés: match, if let, let else és a minták nyelvtanát valósítja meg: a kulcs nem üres és nem tartalmaz szóközt, más whitespace-t vagy vezérlőkaraktert; a `SET` értéke a kulcs utáni szóköztől a sor végéig tart, nem üres, és nem tartalmaz sortörést. Ezen a tartományon `s.parse::<Command>() == Ok(cmd)` minden `cmd`-re, ha `s = cmd.to_string()`. A tartományon kívüli parancsot (például szóközös kulcsot) a `FromStr` hibával utasítja el, a `Display` viszont nem ellenőrzi: a kiírt sor ilyenkor nem olvasható vissza. A **tetszőleges bájtsorozat** (`Vec<u8>`) értékeit a szöveges protokoll nem viszi át, ezért ehhez külön, veszteségmentes kódolást adunk (hexadecimális `encode_value` / `decode_value`); a protokoll szövegértékei és a bináris értékek így két külön, egyenként kerek úton járnak.

Lépések:

1. Cseréljük le a `command.rs`-ben az `impl Command` blokkot (a `to_line`-t) a `Display`-re, a `FromStr`-re és a bájtkódolásra. A `to_line` megmarad, de a `Display`-re épül, hogy a két forma ne térjen el.

Kódblokk — fájl: `rkv-core/src/command.rs` · művelet: az `impl Command { ... }` blokk cseréje (az enum és a `#[cfg(test)] mod tests` változatlan) · ellenőrzés: `cargo build -p rkv-core`

```
use std::fmt;
use std::str::FromStr;

use crate::Error;

impl Command {
    /// A parancs protokollsora (a `Display` kimenete).
    pub fn to_line(&self) -> String {
        self.to_string()
    }
}

impl fmt::Display for Command {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            Command::Get(key) => write!(f, "GET {key}"),
            Command::Set(key, value) => write!(f, "SET {key} {value}"),
            Command::Delete(key) => write!(f, "DEL {key}"),
            Command::List(Some(prefix)) => write!(f, "LIST {prefix}"),
            Command::List(None) => write!(f, "LIST"),
        }
    }
}

fn invalid(msg: impl Into<String>) -> Error {
    Error::InvalidCommand(msg.into())
```

```
}

/// Kulcs (vagy prefix) ellenőrzése a protokoll nyelvtana szerint.
fn parse_key(s: &str) -> Result<String, Error> {
    if s.is_empty() || s.chars().any(|c| c.is_whitespace() || c.is_control()) {
        Err(invalid(format!("invalid key: {s:?}")))
    } else {
        Ok(s.to_string())
    }
}

/// Pontosan egy argumentum: a szóközt tartalmazó maradék felesleges argumentum.
fn one_argument(verb: &str, rest: &str) -> Result<String, Error> {
    if rest.contains(' ') {
        Err(invalid(format!("{verb} takes exactly one argument")))
    } else {
        parse_key(rest)
    }
}

impl FromStr for Command {
    type Err = Error;

    fn from_str(line: &str) -> Result<Self, Error> {
        let (verb, rest) = match line.split_once(' ') {
            Some((verb, rest)) => (verb, Some(rest)),
            None => (line, None),
        };
        match (verb, rest) {
            ("GET", Some(rest)) => one_argument("GET", rest).map(Command::Get),
            ("DEL", Some(rest)) => one_argument("DEL", rest).map(Command::Delete),
            ("SET", Some(rest)) => match rest.split_once(' ') {
                Some((_, "")) => Err(invalid("SET requires a non-empty value")),
                Some((_, value)) if value.contains(['\n', '\r']) => {
                    Err(invalid("SET value must not contain a line break"))
                }
                Some((key, value)) => parse_key(key).map(|k| Command::Set(k,
                ↪  value.to_string())),
                None => Err(invalid("SET requires a key and a value")),
            },
            ("LIST", None) => Ok(Command::List(None)),
            ("LIST", Some(rest)) => one_argument("LIST", rest).map(|p| Command::List(Some(p))),
            ("GET" | "DEL", None) => Err(invalid(format!("{verb} requires a key"))),
            ("SET", None) => Err(invalid("SET requires a key and a value")),
            ("", _) => Err(invalid("empty line")),
            (other, _) => Err(invalid(format!("unknown command: {other}"))),
        }
    }
}

/// Tetszőleges bájtsorozat veszteségmentes, szóköz- és sortörésmentes kódolása.
pub fn encode_value(bytes: &[u8]) -> String {
    bytes.iter().map(|b| format!("{b:02x}")).collect()
}

/// Az `encode_value` inverze; páratlan hosszra vagy nem hexa karakterre hiba.
pub fn decode_value(text: &str) -> Result<Vec<u8>, Error> {
    if !text.len().is_multiple_of(2) || !text.is_ascii() {
        return Err(invalid("invalid hex value"));
    }
    (0..text.len())
        .step_by(2)
        .map(|i| u8::from_str_radix(&text[i..i + 2], 16).map_err(|_| invalid("invalid hex
        ↪  value")))
```

```
        .collect()
}
```

2. Tesztek: egy külön modul a `command.rs` végén. A bájtkódolás körútját tetszőleges bájtokra, köztük `0xFF`-re, üres értékre és whitespace-bájtokra is ellenőrizzük.

Kódblokk — fájl: `rkv-core/src/command.rs` · művelet: új modul a fájl végére · ellenőrzés: `cargo test -p rkv-core command` → 6 sikeres teszt (a 2 korábbi és a 4 új)

```
#[cfg(test)]
mod parse_tests {
    use super::*;

    #[test]
    fn every_variant_round_trips() {
        let commands = [
            Command::Get("k".into()),
            Command::Set("k".into(), "hello world".into()),
            Command::Delete("k".into()),
            Command::List(None),
            Command::List(Some("app".into())),
        ];
        for cmd in commands {
            assert_eq!(cmd.to_string().parse::<Command>().unwrap(), cmd);
        }
    }

    #[test]
    fn argument_count_errors_are_distinct() {
        let msg = |s: &str| s.parse::<Command>().unwrap_err().to_string();
        assert!(msg("GET").contains("requires a key"));
        assert!(msg("GET a b").contains("exactly one argument"));
        assert!(msg("LIST a b").contains("exactly one argument"));
        assert!(msg("SET k").contains("key and a value"));
        assert!(msg("SET k ").contains("non-empty value"));
        assert!(msg("").contains("empty line"));
        assert!(msg("FOO x").contains("unknown command"));
    }

    #[test]
    fn out_of_domain_is_rejected_not_truncated() {
        assert!("SET k a\nb".parse::<Command>().is_err());
        assert!("GET a\tb".parse::<Command>().is_err());
        // A Display nem ellenőriz: a tartományon kívüli parancs sora nem olvasható vissza.
        let bad = Command::Get("a b".into());
        assert!(bad.to_string().parse::<Command>().is_err());
    }

    #[test]
    fn arbitrary_bytes_round_trip_through_hex() {
        let all: Vec<u8> = (0..=255).collect(); // 0x00..0xFF, whitespace-bájtok is
        for bytes in [vec![], vec![0xFF], b" \n\t".to_vec(), all] {
            let text = encode_value(&bytes);
            assert!(!text.contains(char::is_whitespace));
            assert_eq!(decode_value(&text).unwrap(), bytes);
        }
        assert!(decode_value("abc").is_err()); // páratlan hossz
        assert!(decode_value("zz").is_err()); // nem hexa
    }
}
```

3. Az `rkv-cli` saját értelmezője (`parse_command`, `parse_key`) helyett mostantól a `FromStr`-t használjuk, így a nyelvtan egyetlen helyen él. Törölhető a CLI `parse_key` függvénye; a

`parse_command` vékony burkoló marad, mert a CLI `String` hibákkal dolgozik:

Kódblokk — fájl: `rkv-cli/src/main.rs` · művelet: a `parse_key` törlése és a `parse_command` cseréje; a tesztek változatlanok · ellenőrzés: `cargo test --workspace`

```
/// A protokoll egy sorát `Command`-dá alakítja: az `rkv-core` `FromStr`-je végzi.
fn parse_command(line: &str) -> Result<Command, String> {
    line.parse::<Command>().map_err(|e| match e {
        Error::InvalidCommand(msg) => msg,
        other => other.to_string(),
    })
}
```

A változás egy megfigyelhető következménye: a felesleges argumentum (`get "a b"`) mostantól `ERR GET takes exactly one argument` választ ad, nem `ERR invalid key: "a b"`-t, mert a külön argumentumszámhiba az érvénytelen kulcs elé kerül. Aki a Modulok és láthatóság: header nélküli világ 3. feladatát megoldotta, a `command::parse` függvényét is cserélje `FromStr`-re.

4. Futtassuk a teszteket:

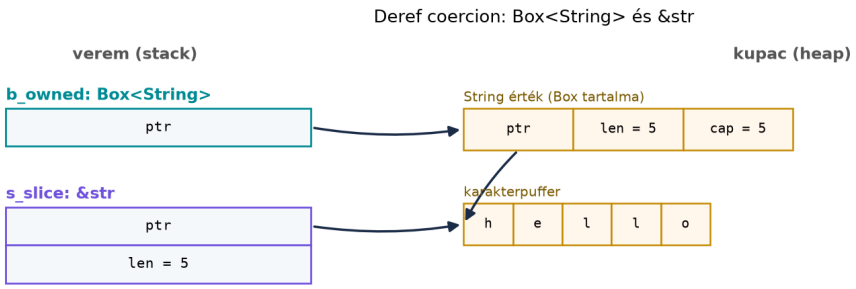
```
cargo test --workspace
```

Várt kimenet: az `rkv-core` unit tesztjei között a négy új `parse_tests::...` is sikeres, az `rkv-cli` két tesztje változatlanul zöld.

Továbbvitt eredmény. Ez a `Command` `FromStr` és `Display` implementáció az Az rkv hálózati szerver: TCP, protokoll, kecses leállás alfejezetben a szöveges protokoll értelmezésének alapja; a hibát a kanonikus `Error::InvalidCommand` hordozza. A bináris értékek hexa kódolása (`encode_value` / `decode_value`) a szöveges protokoll és a `Vec<u8>` tárolt értékek közti híd. A `From`/`TryFrom` konverziókat a C ABI: extern “C”, repr(C), bindgen és cbindgen alfejezet a FFI-határon használja, az `AsRef`/`Borrow` mintákat pedig az API-tervezés Rustban: irányelvek és ergonómia alfejezet viszi tovább.

Ellenőrzési pont. Az `rkv-core` `Command` típusa `Display` + `FromStr` (hibája `Error::InvalidCommand`), valamint `encode_value` / `decode_value` függvényekkel bővült; az `rkv-cli` a `FromStr`-t használja. A `cargo test --workspace` mind a korábbi, mind az új tesztekkel sikeres.

A memóriaábra a `Deref` coercion mechanizmusát szemlélteti. Az ábra *szemléltető modell*: a mezők sorrendjét és az elrendezést a nyelv nem rögzíti, a lényeg a mutatók útvonala. A `Box<String>` két indirekciót jelent: a `b_owned` pointere a kupacon lévő `String` értékre (mutató–hossz–kapacitás hármas) mutat, és ennek a `String`-nek a saját pufferpointere mutat a karakterekre. A `&str` szelet közvetlenül a karakterpufferre mutat:



12.1. ábra: Deref coercion: Box<String> és &str - Szemléltető modell. A b_owned a kupacon lévő String értékre mutat (Box); a String ptr mezője a karakterpufferre. A deref coercion (&Box<String> -> &String -> &str) után az s_slice közvetlenül a karakterpufferre mutat.

Az ábra mellett: a `&Box<String>` deref coercion révén `&String`-gé, majd `&str`-ré alakul, anélkül, hogy a tulajdonjog vagy a kupacadat másolódna. A `Deref` trait teszi ezt lehetővé, de csak smart pointerek esetén indokolt.

12.3.6 Tipikus hibák és átállási csapdák

1. **Deref öröklés-szimulációra.** C++-fejlesztők gyakran próbálják a `Deref` segítségével utánozni az osztályhierarchiát: `struct Derived { base: Base }` és `impl Deref for Derived { type Target = Base; ... }`. Ez működik a metódushívásoknál, de nem ad valódi polimorfizmust, nem működik a trait-objektumokkal, és a deref coercion rejtett függőségeket okoz. Ilyenkor inkább kompozíciót és trait-eket használjunk, vagy ha dinamikus diszpécsre van szükség, `dyn Trait`-et.
2. **Túlzott AsRef használat generikus függvényekben.** Az `AsRef<str>` gyakran hasznos, de ha egy függvény minden paraméterét `AsRef`-re vesszük, a generikus kód feleslegesen bonyolulttá válik, és a fordítási idő nő. Egyszerű esetben fogadjunk el konkrét típust (pl. `&str`), és a hívónál alkalmazzunk `.as_ref()`-t.
3. **Debug és Display összekeverése.** A `{}` formátum a `Display`-et, a `{:?}` a `Debug`-ot használja. Ha egy típus nem implementálja a `Display`-et, a `{}` nem működik, és a fordító hibát jelez (E0277, verziófüggő). Ne írjunk `Display`-et csak azért, hogy a `println!` működjön; ha nincs emberi olvasásra szánt reprezentáció, használjunk `Debug`-ot.
4. **Hibás From-impl koherencia.** A koherencia (orphan rule) miatt idegen traitet idegen típusra nem implementálhatunk. A `From<T> for Saját` viszont szabályos idegen `T`-vel is (a céltípus saját), és a `From<Saját> for String` is az, mert a trait-paraméter lokális típus. Tilos az idegen `From<Idegen> for Idegen`. A konverziókat ezért a saját típusunkra (vagy saját típust paraméterként használva) implementáljuk.

12.3.7 Feladatok

12.3.7.1 (a) Lefordul-e és miért?

Az alábbi kódban a `Foo` típus implementálja a `Deref`-et a `String`-re. Lefordul-e a `println!` hívás? Miért?

```rust
use std::ops::Deref;

struct Foo {
    s: String,
}

impl Deref for Foo {
    type Target = String;
    fn deref(&self) -> &String {
        &self.s
    }
}

fn main() {
    let f = Foo { s: String::from("hello") };
    println!("{}", f);
}
```

12.3.7.2 (b) C++-kód átírása idiomatikus Rustra

Írd át a következő C++ kódot idiomatikus Rustra, használva a `From`/`TryFrom`, `Display` és `Default` traiteket:

```cpp
#include <string>
#include <iostream>

class Config {
public:
    std::string name;
    int threads;
    Config() : name("default"), threads(1) {}
    Config(std::string n, int t) : name(std::move(n)), threads(t) {}
};

void print_config(const Config& c) {
    std::cout << c.name << " (" << c.threads << " threads)\n";
}

int main() {
    Config c1;
    Config c2{"server", 4};
    print_config(c1);
    print_config(c2);
}
```

12.3.7.3 (c) Implementációs feladat az rkv-ban

Az rkv-core kanonikus `Error` enumja (Saját hibatípusok: enum-hibák, thiserror, anyhow–I/O és szerializáció: fájlok, pufferelés, serde) már tartalmazza az `Io(#[from] io::Error)` és a `Serialization(#[from] serde_json::Error)` változatot. Gyakorlásként bővítsd egy `Utf8(#[from] std::string::FromUtf8Error)` változattal, hogy a `String::from_utf8(bytes)?` hívás `rkv_core::Result`-ot visszaadó függvényben közvetlenül használható legyen. Írj hozzá teszteket, amelyek ellenőrzik a `?` általi konverziót és a `Display` szöveget. (A bővítés gyakorlat; a továbbvitt `Error` négy változatú marad.)

12.3.7.4 (d) Refaktorálás

A `Store::get(&self, key: &str)` metódus `&str`-t vár, ezért tulajdonolt `String`-gel csak `&`-lel hívható (`store.get(&key)`). Adj a `Store`-hoz egy `get_by` kényelmi metódust, amely a kulcsot `&` nélkül is elfogadja (`&str`, `String`, `&String`, `Cow<str>`). Teszteld mind a négy hívási alakot. Majd magyarázd el: miért nem kell magát a `get`-et `Borrow`-ra átírni, és mi a különbség az `AsRef<str>` és a `Borrow<str>` között? (Ne írj saját `Borrow<str>` implementációt `String`-re: az a standard könyvtárban már létezik.)

12.3.8 Megoldások

12.3.8.1 (a)

Nem fordul le, mert a `Foo` típus nem implementálja a `Display`-et, és a `println!` a `{}` formátummal a `Display` traitet igényli. A `Deref` a `String`-re nem ad automatikusan `Display` implementációt. Javítás: `println!("{:?}", f);` ha a `Debug` implementálva van, vagy implementáljuk a `Display`-et.

12.3.8.2 (b)

```rust
#[derive(Debug, Clone)]
pub struct Config {
    pub name: String,
    pub threads: u32,
}

impl Config {
    pub fn new(name: impl Into<String>, threads: u32) -> Self {
        Config { name: name.into(), threads }
    }
}

// Kézi Default: a #[derive(Default)] name = "" és threads = 0 értéket adna,
// a C++ alapértelmezett konstruktor viszont name = "default", threads = 1.
impl Default for Config {
    fn default() -> Self {
        Config::new("default", 1)
    }
}

impl std::fmt::Display for Config {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        write!(f, "{} ({} threads)", self.name, self.threads)
    }
}

// Ha szükséges, From<(&str, u32)> implementálható.
impl From<(&str, u32)> for Config {
    fn from((name, threads): (&str, u32)) -> Self {
        Config::new(name, threads)
    }
}

fn main() {
    let c1 = Config::default();
    let c2 = Config::from(("server", 4));
    println!("{}", c1);
```

```
    println!("{}", c2);
}
```

Várt kimenet:

```
default (1 threads)
server (4 threads)
```

Szándékos tartományváltozás: a C++ `int threads` negatív is lehet, a Rust `u32` nem. Ha a szál-számot előjeles forrásból kapjuk, a konverzió hibázhat, ezért validált `TryFrom`-ot írunk, nem `as`-t (az `as u32` a `-1`-et 4 294 967 295-re csonkolná):

```
impl TryFrom<(&str, i32)> for Config {
    type Error = String;

    fn try_from((name, threads): (&str, i32)) -> Result<Self, String> {
        let threads = u32::try_from(threads)
            .ok()
            .filter(|&t| t >= 1)
            .ok_or_else(|| format!("érvénytelen szálszám: {threads}"))?;
        Ok(Config::new(name, threads))
    }
}
```

A `Config::try_from(("x", -1))` `Err("érvénytelen szálszám: -1")`, a `Config::try_from(("x", 2))` pedig `Ok`.

12.3.8.3 (c)

Kódblokk — fájl: `rkv-core/src/error.rs` · művelet: új változat az `Error` enumba · ellenőrzés: `cargo test -p rkv-core error`

```
    /// Érvénytelen UTF-8 bájtsorozat (gyakorlat; nem része a továbbvitt Error-nak).
    #[error("érvénytelen UTF-8: {0}")]
    Utf8(#[from] std::string::FromUtf8Error),
```

A `#[from]` attribútum a `thiserror`-ben automatikusan `From`-ot generál, ezért a `?` operátor a `FromUtf8Error`-t `Error::Utf8`-ra alakítja. Teszt (az `error.rs` `mod tests` blokkjába):

```
    fn decode(bytes: Vec<u8>) -> Result<String> {
        Ok(String::from_utf8(bytes)?) // FromUtf8Error -> Error::Utf8
    }

    #[test]
    fn utf8_error_is_converted_by_question_mark() {
        assert_eq!(decode(b"abc".to_vec()).unwrap(), "abc");
        let err = decode(vec![0xFF]).unwrap_err();
        assert!(matches!(err, Error::Utf8(_)));
        assert!(err.to_string().starts_with("érvénytelen UTF-8"));
    }
```

12.3.8.4 (d)

A `get` a `HashMap::get`-re támaszkodik, amelynek szignatúrája már `Borrow`-alapú: `fn get<Q>(&self, k: &Q) -> Option<&V> where K: Borrow<Q>, Q: Hash + Eq + ?Sized`. A `String: Borrow<str>` a standard könyvtárban létezik, ezért a `Store::get(&str)` a `HashMap` felé már az általános `Q = str` mintát használja; ezt nem kell megismételni. Ami hiányzik, az hívási kényelem: a `get_by` az `AsRef<str>` bemeneti általánosságot adja.

Kódblokk — fájl: `rkv-core/src/store/mod.rs` · művelet: beillesztés az `impl Store` blokkba · ellenőrzés: `cargo test -p rkv-core get_by`

```
    /// Kényelmi keresés: a kulcs `&str`, `String`, `&String` vagy `Cow<str>` is lehet.
    pub fn get_by<K: AsRef<str>>(&self, key: K) -> Option<&Vec<u8>> {
        self.get(key.as_ref())
    }

    #[test]
    fn get_by_accepts_all_key_forms() {
        use std::borrow::Cow;
        let mut store = Store::new();
        store.set("k".to_string(), vec![1]);
        let owned = String::from("k");
        assert_eq!(store.get_by("k"), Some(&vec![1]));
        assert_eq!(store.get_by(&owned), Some(&vec![1]));
        assert_eq!(store.get_by(owned.clone()), Some(&vec![1]));
        assert_eq!(store.get_by(Cow::Borrowed("k")), Some(&vec![1]));
        assert_eq!(store.get_by("x"), None);
    }
```

A különbség: az `AsRef<str>` csak annyit mond, hogy egy `&str` nézet előállítható; semmit nem ígér arról, hogy a nézet hash-e és egyenlősége megegyezik az eredetiével. A `Borrow<Q>` ezt *garantálja* (a `Hash` és az `Eq` kompatibilis kell legyen), ezért a `HashMap`/`BTreeMap` kulcskeresés `Borrow`-ra épül. Egy kis- és nagybetűt nem megkülönböztető kulcs-újtípus például `AsRef<str>` lehetne, de `Borrow<str>` nem, mert más egyenlőséget használ, és a keresés hibásan működne. Ezért a függvényparaméter kényelmét az `AsRef`, a gyűjtemény-keresés általánosságát a `Borrow` adja.

12.3.9 Rövid összefoglaló

Az idiomatikus Rust-kód a konverziókat, formázást és pointer-viselkedést a standard trait-ekkel szabályozza. A `From`/`Into` és `TryFrom`/`TryInto` a hibamentes, illetve hibás konverziókat írja le; a `?` operátor a `From`-on keresztül konvertálja a hibákat. Az `AsRef` rugalmas paraméterátadást tesz lehetővé, a `Borrow` a `HashMap` kulcsainál garantálja a hash- és egyenlőség-konzisztenciát. A `Display` a felhasználói, a `Debug` a diagnosztikai formátum; a `Default` az alapértelmezett értéket ad, a `FromStr` szövegből parse-ol. A `Deref` smart pointerekhez való, nem öröklés-szimulációra. A laborban a `Command` enum `Display` és `FromStr` implementációja készült el az `rkv-core/src/command.rs` fájlban (hibája a kanonikus `Error::InvalidCommand`), valamint a veszteségmentes `encode_value`/`decode_value` bájtkódolás. **Továbbvitt eredmény.** A `Command` FromStr kódját a Az rkv hálózati szerver: TCP, protokoll, kecses leállás alfejezet használja, a `From`/`TryFrom` konverziók az API-tervezés Rustban és a C ABI alfejezetekben térnek vissza.

12.3.10 Források és ellenőrzési hiányok

A fejezet a standard könyvtár stabil traitjeire épül: `From`, `Into`, `TryFrom`, `TryInto`, `AsRef`, `Borrow`, `ToOwned`, `Display`, `Debug`, `Default`, `FromStr`, `Deref`. A `thiserror` crate `#[from]` attribútumának működése a crate dokumentációjában olvasható (https://docs.rs/thiserror); a verziót a `Cargo.toml` rögzíti. A `Command` enum és a hibatípus definíciója az rkv `rkv-core` crate-jében található. A példakód a 2021-es és a 2024-es editionnel is lefordul.

12.4 Trait objektumok és dinamikus diszpécs

12.4.1 Motiváció

C++-ban a futásidőben kiválasztott viselkedés klasszikus eszköze az absztrakt osztály virtuális függvényekkel. Például egy `Storage` interfészt implementáló `MemStorage` és `FileStorage` közül a konfiguráció alapján választunk, és a hívó csak az absztrakt osztályt látja:

```
struct Storage {
    virtual ~Storage() = default;
    virtual std::optional<std::string> get(const std::string& key) const = 0;
    virtual void set(const std::string& key, const std::string& value) = 0;
};

std::unique_ptr<Storage> create_storage(const std::string& kind) {
    if (kind == "mem") return std::make_unique<MemStorage>();
    if (kind == "file") return std::make_unique<FileStorage>();
    throw std::runtime_error("unknown kind");
}
```

A Rust válasza erre a `dyn Trait`: a trait-objektum. Ugyanúgy futásidőben hívódik a megfelelő implementáció, de nincs virtuális öröklés, nincs kézzel írt virtuális destruktor, és a típusrendszer továbbra is kikényszeríti a biztonsági szabályokat.

12.4.2 Fogalmak és szabályok

Definíció. A trait-objektum (`trait object`) egy olyan dinamikus típus, amelyet a `dyn Trait` jelöl. Fordítási időben nem ismerjük a konkrét típust, csak azt, hogy az adott traitet implementálja. Mivel a `dyn Trait` mérete nem ismert fordítási időben, ezért a Rust-ban *unsized* (DST) típus: csak mutatón vagy referencián keresztül használható, például `&dyn Trait`, `&mut dyn Trait`, `Box<dyn Trait>`, `Rc<dyn Trait>` vagy `Arc<dyn Trait>` formában.

Definíció. A `&dyn Trait` és a `Box<dyn Trait>` úgynevezett kövér mutató (fat pointer): két gépi szóból áll. Az első mutató a konkrét adatra, a második a vtable-re mutat. A vtable a trait dyn-kompatibilis metódusaira és a drop-rutinra mutató függvénymutatók táblája. A hívás tehát egy indirekción keresztül történik, ez a dinamikus diszpécs.

Szabály. Csak *dyn-kompatibilis* (korábbi nevén object safe) traitből lehet trait-objektumot készíteni. Egy trait akkor dyn-kompatibilis (Rust Reference, „Dyn compatibility”), ha

1. minden szupertraitje is dyn-kompatibilis, és a `Sized` nem szupertraitje (ezért nem az a `Clone`, a `Copy` és a `Default`, és a `Self` típusparaméterű `PartialEq` sem);
2. nincs asszociált konstansa;
3. nincs típusparaméteres asszociált típusa (GAT);
4. minden asszociált függvénye vagy *dispatcholható* a trait-objektumon, vagy kifejezetten kizárt a `where Self: Sized` megkötéssel.

Dispatcholható az a metódus, amelynek - nincs típusparamétere (élettartam-paraméter lehet); - van receivere, és az `&Self`, `&mut Self`, `Box<Self>`, `Rc<Self>`, `Arc<Self>` vagy ezek `Pin`-elt változata; - a `Self` típus a receiveren kívül sem a paraméterei, sem a visszatérési típusa között nem szerepel; - nem `async fn` és nem `-> impl Trait` (mindkettő rejtett, metódusonként más konkrét típust ad vissza); - nem kapott `where Self: Sized` megkötést.

Az asszociált típus megengedett, de a trait-objektumban meg kell adni (`dyn Iterator<Item = u32>`). A `where Self: Sized` megkötésű (és ezért a vtable-ből kikerülő) metódusok trait-objektumon nem hívhatók, de a trait dyn-kompatibilis marad. A `fn f(self)` (érték szerinti receiver) szintén nem dispatcholható trait-objektumon, de a trait emiatt még dyn-kompatibilis; ha `Box<dyn Trait>`-en akarjuk hívni, `self: Box<Self>` kell.

A hibás esetek rövid példái (mindegyik `E0038`, kivéve az asszociált típust, ahol `E0191`):

```
trait A { fn make(&self) -> Self; }            // Self a visszatérésben
trait B { fn cmp_with(&self, o: &Self); }      // Self a paraméterben
trait C { fn map<U>(&self, u: U); }            // típusparaméter
trait D { const N: usize; }                    // asszociált konstans
trait E { async fn fetch(&self); }             // async fn
trait F: Sized { fn f(&self); }                // Sized szupertrait
trait G { type Out; fn out(&self) -> Self::Out; }  // dyn G nem elég: dyn G<Out = u32>
```

A könyv traitjeinek besorolása: a `Storage` (Trait-ek és generikusok: sablonok és concepts helyett; csak `&self`/`&mut self`, nincs generikus), a `Summary` (Trait-ek és generikusok: sablonok és concepts helyett), az `Animal` és a `Handler` dyn-kompatibilis; az `Iterator` dyn-kompatibilis, ha megadjuk az `Item`-et; a `Clone` nem az (`Sized`), és a `Loader` (lásd a feladatot) sem, mert a `Self`-et adja vissza.

Szabály. A modern Rustban mindig írjuk ki a `dyn` kulcsszót. A régi, `dyn` nélküli `&Trait` forma a 2021-es és a könyv 2024-es editionjében fordítási hiba (a 2018-as editionben még figyelmeztetés volt).

12.4.3 C++ és Rust összevetés

A C++ virtuális függvény és a Rust trait-objektum párhuzama jól látható egy egyszerű példán:

```
#include <iostream>
#include <memory>
#include <string>

struct Animal {
    virtual ~Animal() = default;
    virtual std::string speak() const = 0;
};

struct Dog : Animal {
    std::string speak() const override { return "Vau"; }
};

struct Cat : Animal {
    std::string speak() const override { return "Miau"; }
};

void describe(const Animal& a) {
    std::cout << a.speak() << "\n";
}

int main() {
    std::unique_ptr<Animal> p = std::make_unique<Dog>();
    describe(*p);
}
```

Rustban ugyanez kompozícióval és trait-objektummal:

```rust
trait Animal {
    fn speak(&self) -> String;
}

struct Dog;
struct Cat;

impl Animal for Dog {
    fn speak(&self) -> String { "Vau".to_string() }
}

impl Animal for Cat {
    fn speak(&self) -> String { "Miau".to_string() }
}

fn describe(a: &dyn Animal) {
    println!("{}", a.speak());
}

fn main() {
    let p: Box<dyn Animal> = Box::new(Dog);
    describe(&*p);
}
```

A különbség nem csak szintaktikai. C++-ban a `Dog` a `Animal`-ból öröklődik, a vtable az osztály része, és a virtuális destruktorért a fejlesztő felel. Rustban a trait implementáció teljesen különválik a struktúrától; a `Dog` nem „tudja”, hogy `Animal` traitet implementál, és a `Box<dyn Animal>` dropja a trait-objektum vtable-jén keresztül hívja meg a konkrét típus dropját. Nincs tehát virtuális destruktor-probléma.

A statikus és dinamikus diszpécs közötti döntés hasonló a C++ template és virtuális függvény közötti döntéshez:

Szempont	Rust statikus	Rust dinamikus
Eszköz	generikus `fn f<S: Storage>(s: &S)`	`fn f(s: &dyn Storage)`
C++ megfelelő	`template<class S>`	virtuális függvény
Döntés ideje	fordítási idő (monomorfizálás)	futásidő (vtable-hívás)
Kódméret	nagyobb lehet	kisebb
Hívási költség	általában kisebb, jobban inline-olható	indirekt, kevésbé inline-olható
Típusok halmaza	nyitott, de a hívó ismeri a konkrét típust	nyitott, a hívónak elég a trait

A generikus `fn save<S: Storage>(s: &S, ...)` minden konkrét `S`-re külön kódot generál. A `fn save(s: &dyn Storage, ...)` egyetlen kódot generál, és a megfelelő implementációt a vtable-n keresztül éri el. Hot path esetén a statikus változat gyakran gyorsabb, plugin-szerű bővíthetőség esetén viszont a dinamikus a természetes.

12.4.4 Kidolgozott példa: object safety javítása

Az alábbi trait nem dyn-kompatibilis, mert a `clone_box` visszatérési típusa `Self`:

```rust
// Ez a változat szándékosan nem fordul le:
trait Cloneable {
```

```
    fn clone_box(&self) -> Self;
}

fn clone_all(items: &[Box<dyn Cloneable>]) -> Vec<Box<dyn Cloneable>> {
    items.iter().map(|item| item.clone_box()).collect()
}
```

A fordító E0038 hibakód körüli üzenetet ad: a trait nem hozható létre trait-objektumként, mert a metódus visszatérési típusa `Self`. A pontos szöveg verziófüggő.

A javítás az, hogy a metódus ne konkrét `Self`-et adjon vissza, hanem `Box<dyn Cloneable>`-t:

```
trait Cloneable {
    fn clone_box(&self) -> Box<dyn Cloneable>;
}

#[derive(Clone)]
struct Point { x: i32, y: i32 }

impl Cloneable for Point {
    fn clone_box(&self) -> Box<dyn Cloneable> {
        Box::new(self.clone())
    }
}
```

Így a vtable-ben a `clone_box` visszatérési típusa már ismert, és a hívó `Box<dyn Cloneable>`-t kap. Ha viszont egy metódust csak a konkrét típuson akarunk meghívni, akkor `where Self: Sized` megkötéssel kizárhatjuk a vtable-ből:

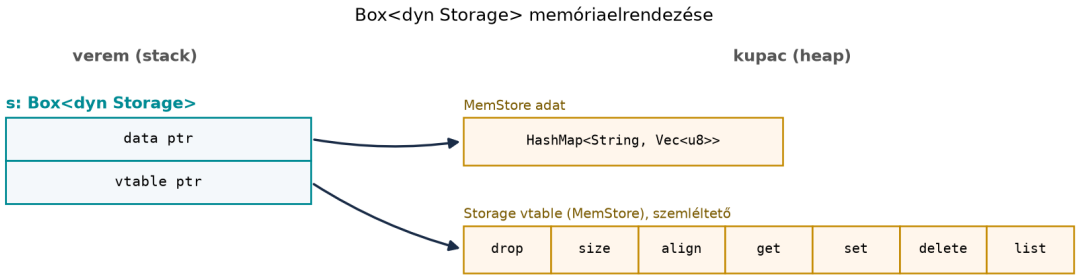
```
trait Animal {
    fn speak(&self) -> String;

    /// 'static objektum: a konkrét típus nem tartalmazhat nem 'static kölcsönzést.
    fn into_box(self) -> Box<dyn Animal>
    where
        Self: Sized + 'static,
    {
        Box::new(self)
    }

    /// Kölcsönzött adatot is tartalmazhat: az objektum élettartama `'a`.
    fn into_box_in<'a>(self) -> Box<dyn Animal + 'a>
    where
        Self: Sized + 'a,
    {
        Box::new(self)
    }
}
```

Az `into_box` nem hívható `&dyn Animal`-on (a `Self: Sized` kizárja a vtable-ből), de a trait dyn-kompatibilis marad. A `Self: 'static` korlát azért kell, mert a `Box<dyn Animal>` típus *alapértelmezett objektum-élettartama* `'static`: a `Box<dyn Animal>` valójában `Box<dyn Animal + 'static>`. A korlát nélkül a fordító `E0310`-et ad („the parameter type `Self` may not live long enough”). A hivatkozások esetén az alapértelmezés más: az `&'a dyn Animal` jelentése `&'a (dyn Animal + 'a)`. Ha az implementáló típus kölcsönzést tartalmaz (például `struct Echo<'a>(&'a str)`), az `into_box` nem hívható rá (`E0597`), az `into_box_in` viszont igen, mert az objektum élettartamát is paraméterezi. A választás tehát tervezési döntés: `'static` objektum (szálak között, hosszú életű tárolókban) vagy kölcsönzést engedő, rövidebb életű objektum.

A `Box<dyn Trait>` memóriaelrendezését mutatja az ábra. A veremben a `s` változó két szót foglal: az első a kupacon lévő konkrét adatra mutat, a második a vtable-re. A vtable a konkrét típushoz tartozó függvénymutatókat tartalmazza.



12.2. ábra: Box<dyn Storage> memóriaelrendezése - A Box<dyn Storage> kövér mutató: adatmutató és vtable-mutató. A vtable a drop-rutint, a típus méretét és igazítását, valamint a dyn-kompatibilis metódusok mutatóit tárolja; a pontos elrendezés nem specifikált, az ábra szemléltető.

64 bites platformon a `Box<dyn Storage>` tipikusan 16 bájt, szemben a közönséges `Box<T>` 8 bájtjával. Ezért is érdemes elkerülni a felesleges dinamikus diszpécset: a fat pointer tárolása és a vtable-indirekció is költség.

12.4.5 Labor: futásidőben választott `Box<dyn Storage>`

Munkakönyvtár: `~/rust-labor/rkv/`

Ebben a laborban az rkv tárolóját tesszük futásidőben választhatóvá. A kiinduló állapot a Asszociált típusok, koherencia, operátorok és derive és a A standard trait-ek: konverziók, formázás, Default, Deref, Borrow ellenőrzési pontja: a `Storage` trait az `rkv-core/src/storage.rs`-ben él (Trait-ek és generikusok: sablonok és concepts helyett), a `MemStore` (a `Store` aliasa) és a `FileStore` is implementálja. A traitet nem írjuk át: a gyártófüggvény a kanonikus szerződésre épül, és egy `Box<dyn Storage + Send>`-ot ad vissza. A `Send` korlát azért kell már most, mert a Mutex, RwLock, csatornák és a megosztott rkv alfejezet szálak közé helyezi a tárolót.

A `Storage` dyn-kompatibilis: minden metódusa `&self` vagy `&mut self` receivert használ, nincs típusparaméteres metódusa, és a `Self` sehol nem szerepel a szignatúrákban (lásd a fenti szabálylistát). Ezt a fordító is ellenőrzi, amint a gyártófüggvény `Box<dyn Storage + Send>`-ot hoz létre.

Kódblokk — fájl: `rkv-core/src/storage.rs` · művelet: az elején a `use` sor cseréje, a fájl végére (a `mod tests` elé) a gyártófüggvény · ellenőrzés: `cargo build -p rkv-core`

```rust
use std::path::Path;

use crate::{Command, Error, FileStore, MemStore, Result};

/// Futásidőben választott háttér. `"file"` esetén a `dir` könyvtárban
/// a `state.json` snapshot és az `append.log` napló él (r023).
pub fn create_store(kind: &str, dir: &Path) -> Result<Box<dyn Storage + Send>> {
    match kind {
        "mem" => Ok(Box::new(MemStore::new())),
        "file" => {
```

```
            let store = FileStore::open(dir.join("state.json"), dir.join("append.log"))?;
            Ok(Box::new(store))
        }
        other => Err(Error::InvalidCommand(format!("unknown store kind: {other}"))),
    }
}
```

Az ismeretlen háttér konfigurációs hiba, de az rkv `Error` típusa négy változatú marad (I/O és szerializáció: fájlok, pufferelés, serde), ezért az `InvalidCommand` hordozza; egy külön változat bevezetése esetén minden `match`-et és `From`-ot együtt kell frissíteni. A crate gyökerében exportáljuk a függvényt:

Kódblokk — fájl: `rkv-core/src/lib.rs` · művelet: a `pub use storage::{...}` sor cseréje · ellenőrzés: `cargo build --workspace`

```
pub use storage::{apply, create_store, Response, Storage};
```

A dinamikus diszpécset az alábbi integrációs teszt ellenőrzi. A *közös szerződésteszt* (`contract`) egyetlen `&mut dyn Storage` paramétert kap, ezért ugyanaz a tesztsor fut a memóriabeli és a fájl-alapú háttéren is; új háttér hozzáadásakor elég a listába felvenni:

Kódblokk — fájl: `rkv-core/tests/dyn_store.rs` · művelet: új fájl · ellenőrzés: `cargo test -p rkv-core --test dyn_store` → 2 sikeres teszt

```
use std::path::PathBuf;

use rkv_core::{create_store, Error, Storage};

fn fresh_dir(name: &str) -> PathBuf {
    let dir = std::env::temp_dir().join(format!("rkv-dyn-{name}-{}", std::process::id()));
    let _ = std::fs::remove_dir_all(&dir);
    std::fs::create_dir_all(&dir).unwrap();
    dir
}

/// Közös szerződésteszt: minden háttérnek ugyanígy kell viselkednie.
fn contract(store: &mut dyn Storage) {
    store.set("kulcs".to_owned(), "érték".as_bytes().to_vec()).unwrap();
    assert_eq!(store.get("kulcs").unwrap(), Some("érték".as_bytes().to_vec()));
    assert!(store.delete("kulcs").unwrap());
    assert!(!store.delete("kulcs").unwrap());
    store.set("b".into(), b"2".to_vec()).unwrap();
    store.set("a".into(), b"1".to_vec()).unwrap();
    let keys: Vec<String> = store.list(None).unwrap().into_iter().map(|(k, _)| k).collect();
    assert_eq!(keys, ["a", "b"]); // rendezett
}

#[test]
fn every_backend_satisfies_the_contract() {
    for kind in ["mem", "file"] {
        let mut store = create_store(kind, &fresh_dir(kind)).unwrap();
        contract(&mut *store); // Box<dyn Storage + Send> -> &mut dyn Storage
    }
}

#[test]
fn unknown_store_kind_is_an_error() {
    let result = create_store("postgres", &fresh_dir("unknown"));
    assert!(matches!(result, Err(Error::InvalidCommand(ref m)) if m.contains("postgres")));
}
```

Futtatás:

```
cargo test -p rkv-core --test dyn_store
```

Várt kimenet (rövidítve):

```
running 2 tests
test every_backend_satisfies_the_contract ... ok
test unknown_store_kind_is_an_error ... ok
```

A `create_store("mem", ..)` egy `Box<dyn Storage + Send>`-ot ad vissza, amely mögött `MemStore` áll, és a `set`/`get` hívások a vtable-n keresztül érik el a konkrét implementációt. A teszt a hívó szemszögéből dolgozik: a konkrét típust nem ismeri.

Ellenőrzési pont. Az `rkv-core` a `create_store(kind, dir) -> Result<Box<dyn Storage + Send>>` függvényt exportálja; a `mem` és a `file` háttér ugyanazt a közös szerződéstesztet teljesíti. A `cargo test --workspace` az eddigi tesztek mellett az új integrációs tesztet is sikeresen futtatja.

Továbbvitt eredmény. A `create_store` gyártófüggvény és a `Box<dyn Storage + Send>` dinamikus tároló beépül a későbbi fejezetekbe: a Szálak, Send és Sync: data race fordítási időben kizárva alfejezetben a tárolót szálak közé kell elhelyezni, ahol a `dyn Storage + Send` határ kulcsszerepet kap; a Az rkv hálózati szerver: TCP, protokoll, kecses leállás ezen a gyártófüggvényen keresztül választja ki futásidőben a memóriabeli vagy fájl-alapú hátteret; a Záróprojekt: az rkv teljes rendszerként pedig ezt a dinamikus API-t fűzi teljes rendszerré.

12.4.6 Enum vs. trait objektum: zárt és nyitott halmaz

A `Box<dyn Storage>` mellett a másik gyakori megoldás egy `enum`, amely a háttereket zárt halmazként sorolja fel, és maga is implementálja a `Storage`-ot:

```
enum StorageBackend {
    Mem(MemStore),
    File(FileStore),
}

impl Storage for StorageBackend {
    fn get(&self, key: &str) -> Result<Option<Vec<u8>>> {
        match self {
            StorageBackend::Mem(m) => Storage::get(m, key),
            StorageBackend::File(f) => Storage::get(f, key),
        }
    }
    // set, delete, list: ugyanígy, minden ágban a megfelelő háttérre delegálva
}
```

A kettő közötti döntés a zárt–nyitott halmaz elvén alapul. Az `enum` fordítási időben rögzíti az összes lehetséges változatot: ha új hátteret adunk hozzá, az összes `match` frissítését a fordító kikényszeríti, ami kimerítőségi hibákkal jár. A `dyn Trait` viszont nyitott halmazt ad: más crate-ek is implementálhatják a `Storage` traitet, és a kódunk változatlanul tudja használni a trait-objektumot. C++-ban ez a `std::variant` és az absztrakt osztály közötti választásnak felel meg.

Az rkv esetében a külső bővíthetőség és a szerver futásidejű konfigurációja miatt a `dyn Storage` a hasznosabb határfelület, de a parancsok feldolgozásához például a `Command` enum továbbra is zárt halmaz marad.

12.4.7 Tipikus hibák és átállási csapdák

1. **Minden absztrakciót trait-objektummá tenni.** A Rust nem kényszerít virtuális diszpécsre; a generikus `fn f<S: Storage>(s: &S)` gyakran gyorsabb és legalább ilyen rugalmas. Akkor használj `dyn Trait`-et, ha a konkrét típus valóban csak futásidőben ismert, vagy ha heterogén kollekciót kell tárolni.
2. **Nem figyelni az object safetyre.** C++-ban egy absztrakt osztály metódusa visszaadhat `std::unique_ptr<Base>`-t, és ez rendben van; Rustban a `Self` visszatérés nem object-safe. A szokásos javítás `Box<dyn Trait>` visszatérés vagy `where Self: Sized`.
3. **A fat pointer méretének figyelmen kívül hagyása.** A `Box<dyn Storage>` nem 8 bájt, hanem tipikusan 16 bájt 64 bites rendszereken. Nagy trait-objektum-vektoroknál ez számít, és a `std::mem::size_of` használata meglepő lehet.
4. **A `dyn Trait` és a konkrét típus keverése.** A `Box<dyn Storage>` nem `Box<MemStore>`. Ha le kell kérdezni a konkrét típust, az `Any` downcastingot igényelne, de idiomatikus Rustban erre ritkán van szükség; ehelyett a trait felületét érdemes bővíteni.

12.4.8 Feladatok

12.4.8.1 1. Lefordul-e és miért?

```
trait Loader {
    fn load(&self) -> Self;
}

fn load_all(items: &[Box<dyn Loader>]) -> Vec<Box<dyn Loader>> {
    items.iter().map(|i| i.load()).collect()
}
```

Válaszold meg, hogy lefordul-e, és ha nem, hogyan javítanád.

12.4.8.2 2. C++-kód átírása

Írd át Rustra a következő C++ absztrakt osztályt és a hívást:

```
struct Handler {
    virtual ~Handler() = default;
    virtual int handle(int x) const = 0;
};

void run(const Handler& h) { std::cout << h.handle(42); }
```

Készíts `Box<dyn Handler>`-t egy konkrét típussal, és hívd a `run` Rust-megfelelőjét.

12.4.8.3 3. Továbbvitt eredmény: hibakezelés a gyártóban

Egészítsd ki a `create_store` függvényt egy harmadik, `"readonly"` háttérrel: egy olyan `Storage`-implementációval, amely a `get` és a `list` hívásokat egy `MemStore`-ra delegálja, a `set` és a `delete` hívásokra pedig `Err(Error::Io(..))` hibát ad (`io::ErrorKind::PermissionDenied`). Vedd fel a `"readonly"` nevet a `dyn_store` teszt listájába úgy, hogy a közös szerződésteszt helyett egy külön teszt ellenőrizze: az írás hibát ad, az olvasás működik.

12.4.8.4 4. Refaktorálás

Adott két függvény:

```
fn process_mem(store: &MemStore, key: &str) -> Result<Option<Vec<u8>>> { Storage::get(store,
↪  key) }
fn process_file(store: &FileStore, key: &str) -> Result<Option<Vec<u8>>> { Storage::get(store,
↪  key) }
```

Helyettesítsd őket egyetlen, `&dyn Storage` paraméterű függvénnyel. Milyen módosítások szükségesek a hívóknál?

12.4.9 Megoldások

12.4.9.1 1.

Nem fordul le, mert a `loader` visszatérési típusa `Self`, ezért a trait nem dyn-kompatibilis. Javítás:

```
trait Loader {
    fn load(&self) -> Box<dyn Loader>;
}
```

12.4.9.2 2.

```
trait Handler {
    fn handle(&self, x: i32) -> i32;
}

struct AddOne;
impl Handler for AddOne {
    fn handle(&self, x: i32) -> i32 { x + 1 }
}

fn run(h: &dyn Handler) {
    println!("{}", h.handle(42));
}

fn main() {
    let h: Box<dyn Handler> = Box::new(AddOne);
    run(&*h);
}
```

A `&*h` a `Box<dyn Handler>`-ből `&dyn Handler`-t hoz létre a deref coercion segítségével.

12.4.9.3 3.

Kódblokk — fájl: `rkv-core/src/storage.rs` · művelet: új típus és új ág a `create_store`-ban · ellenőrzés: `cargo test -p rkv-core --test dyn_store`

```
/// Csak olvasható háttér: az írások hibát adnak.
pub struct ReadOnly(MemStore);

fn denied() -> Error {
    Error::Io(std::io::Error::new(
        std::io::ErrorKind::PermissionDenied,
        "a tároló csak olvasható",
```

```
    ))
}

impl Storage for ReadOnly {
    fn get(&self, key: &str) -> Result<Option<Vec<u8>>> {
        Storage::get(&self.0, key)
    }
    fn set(&mut self, _key: String, _value: Vec<u8>) -> Result<()> {
        Err(denied())
    }
    fn delete(&mut self, _key: &str) -> Result<bool> {
        Err(denied())
    }
    fn list(&self, prefix: Option<&str>) -> Result<Vec<(String, Vec<u8>)>> {
        Storage::list(&self.0, prefix)
    }
}

// a create_store match-ében, az "other" ág elé:
        "readonly" => Ok(Box::new(ReadOnly(MemStore::new()))),
```

Teszt a `dyn_store.rs`-be:

```
#[test]
fn readonly_backend_rejects_writes_but_reads() {
    let mut store = create_store("readonly", &fresh_dir("ro")).unwrap();
    assert!(matches!(store.set("k".into(), b"v".to_vec()), Err(Error::Io(_))));
    assert!(matches!(store.delete("k"), Err(Error::Io(_))));
    assert_eq!(store.get("k").unwrap(), None);
    assert!(store.list(None).unwrap().is_empty());
}
```

A `ReadOnly` nem teljesíti a közös szerződéstesztet (az `set`-et elutasítja), ezért kapott külön tesztet; ez a szerződés kifejezett kivétele, nem rejtett eltérés.

12.4.9.4 4.

A közös függvény:

```
fn process(store: &dyn Storage, key: &str) -> Result<Option<Vec<u8>>> {
    store.get(key)
}
```

A hívóknál az `&MemStore` és `&FileStore` helyett elég `&dyn Storage`-ot átadni; az `&MemStore` és az `&FileStore` automatikusan `&dyn Storage`-gá kényszerül (unsized coercion), egy `Box<dyn Storage + Send>`-ból pedig `&*s` készít ilyet, például `let s = create_store("mem", dir)?; process(&*s, "kulcs")`. A hívó oldalán így nem kell két külön függvényt karbantartani.

12.4.10 Rövid összefoglaló

A trait-objektum (`dyn Trait`) a Rust dinamikus diszpécs eszköze, a C++ virtuális függvények párja. A `&dyn Trait` és a `Box<dyn Trait>` kövér mutató: adatmutatóból és vtable-mutatóból áll. Csak dyn-kompatibilis traitből hozható létre trait-objektum: nincs `Sized` szupertrait, asszociált konstans vagy GAT, a metódusok pedig vagy dispatcholhatók (nincs típusparaméter, `Self` csak a receiverben, nem `async`), vagy `where Self: Sized` megkötéssel kizártak. A `Box<dyn Trait>` alapértelmezett objektum-élettartama `'static`. A statikus generikus (`impl Trait`/`<T: Trait>`) fordítási időben monomorfizál, a `dyn Trait` futásidőben, vtable-n keresztül hív. Az

enum zárt, a trait-objektum nyitott halmazt modellez. A laborban az rkv-hoz hozzáadtuk a `create_store(kind, dir)` gyártófüggvényt, amely `Result<Box<dyn Storage + Send>>`-t ad vissza, és a `dyn_store` integrációs tesztet, amelyben minden háttér ugyanazt a közös szerződéstesztet teljesíti. Ez a dinamikus `Storage` API a Szálak, Send és Sync, a Az rkv hálózati szerver és a Záróprojekt alapja.

12.4.11 Források és ellenőrzési hiányok

Az alfejezet a trait-objektumok, a `dyn` szintaxis, a dyn-kompatibilitás és a vtable-alapú diszpécs stabil nyelvi részeire épül, amelyeket a Rust Reference és a *The Rust Programming Language* ír le. A hibaüzenetek szövege a rögzített toolchainre (Rust 1.99.1) vonatkozik. A labor a korábbi rkv-laborokban definiált `Storage`, `MemStore`, `FileStore` és `Error` típusokat használja; a `FileStore::open` a snapshot- és naplóútvonalat veszi át. Az rkv-crate-en kívül új külső függőség nem szükséges.

13 Closure-ök és iterátorok

13.1 Closure-ök: Fn, FnMut, FnOnce és a capture

13.1.1 Motiváció

A C++-fejlesztő a lambda-kifejezéseknél megszokta az explicit capture-listát: `[&]`, `[=]`, `[x]`, `[x = std::move(x)]`. Ez a felsorolás pontosan megmondja, hogy a lambda mit és hogyan rögzít a környezetéből. A Rust ezt a döntést nem a programozóra bízza, hanem a closure törzséből vezeti le: a borrow checker látja, hogy a closure csak olvas, módosít, vagy tulajdonba vesz egy értéket, és ennek megfelelően rendeli hozzá a három lehetséges trait egyikét. Ez eleinte kényelmetlenül szokatlan, de a gyakorlatban megszünteti az elavult lambda-fogásokból adódó hibák egész osztályát. A C++-ban egy `[&x]` lambda túlélheti az `x` élettartamát, és futás közben dangling hivatkozást használhat; Rustban ugyanez fordítási időben hibát ad. Ez a fejezet azt mutatja meg, hogyan válasszunk a `Fn`, `FnMut` és `FnOnce` trait-ek, valamint a `move` kulcsszó között.

13.1.2 Fogalmak és szabályok

Definíció. A *closure* (zárvány) olyan névtelen függvényérték, amely rögzítheti a környezetében lévő változókat. A Rust closure szintaxisa `|paraméterek| törzs`, és a capture módját nem deklaráljuk explicit módon: a fordító a törzsben használt változók alapján állapítja meg.

Definíció. A Rust standard könyvtár három closure-traitet definiál, amelyek supertrait-lánccal kapcsolódnak: `Fn` `FnMut` `FnOnce` (az `Fn` az `FnMut` alatt, az pedig az `FnOnce` alatt áll). A különbségüket a *hívás receiverje* adja:

- `FnOnce`: `call_once(self)`, a closure **egyszer** hívható, és a hívás elfogyaszthatja (kimozgathatja) a rögzített értékeket. Ez a legkevésbé szigorú követelmény: minden closure megvalósítja.
- `FnMut`: `call_mut(&mut self)`, **többször** hívható, a hívás módosíthatja a rögzített állapotot. Minden `Fn` closure egyben `FnMut` is.
- `Fn`: `call(&self)`, **többször** hívható anélkül, hogy a rögzített állapotot módosítaná vagy kimozgatná; a hívás csak megosztott hozzáférést használ. Ez a három közül a legszigorúbb követelmény (a legkevesebb closure teljesíti).

A capture módját és a hívhatósági traitet külön kell tartani. A capture mód (megosztott hivatkozás, `&mut` hivatkozás, vagy tulajdonba vétel `move`-val) azt dönti el, *hogyan rögzít* a closure; a trait azt, hogy a **törzs mit csinál a rögzített értékekkel hívás közben**. Ezért egy `move` closure is lehet `Fn`: ha a törzs csak olvassa a rögzített értéket. Ellenőrzött példák:

```
use std::cell::Cell;

fn call_fn<F: Fn()>(f: F) {
    f();
    f();
}
```

```rust
fn main() {
    let s = String::from("hello");
    let printer = move || println!("{s}"); // move rögzítés, a törzs csak olvas: Fn
    call_fn(printer);                      // kimenet: hello, hello

    let counter = Cell::new(0);
    let tick = || counter.set(counter.get() + 1); // megosztott rögzítés + belső mutabilitás: Fn
    call_fn(&tick);                               // kétszer hívjuk
    tick();                                       // egyszer még
    println!("{}", counter.get());                // 3
}
```

A `move` closure (`printer`) tehát `Fn`, a belső mutabilitással számláló `tick` is `Fn` (nem `FnMut`!), mert nem módosít rögzített *értéket* a hívás receiverén át, csak a `Cell` belsejét egy megosztott hivatkozáson keresztül. Az igazán `FnOnce` closure az, amely a törzsben kimozgat egy rögzített értéket (`move || t`): ha ezt `Fn() -> String` boundú függvénynek adjuk, a fordító `E0525`-tel jelzi, hogy a closure csak `FnOnce`.

Szabály. A closure által implementált traitet a törzs használati módja határozza meg. Ha a closure: - a rögzített értékeket csak megosztott hozzáféréssel használja (olvassa, vagy belső mutabilitással írja), akkor `Fn` (és ezzel `FnMut` és `FnOnce` is); - módosítja a rögzített változót (`&mut` hozzáférés), de nem mozgatja ki, akkor `FnMut` (és `FnOnce`); - kimozgat egy rögzített értéket a hívás során (visszaadja vagy eldobja), akkor csak `FnOnce`.

Szabály. A `move` kulcsszó a closure elején minden, a törzsben használt változót *by value* rögzít. Ha a változó típusa `Copy`, akkor a `move` másolatot készít; ha nem `Copy`, akkor a tulajdonjog átkerül a closure-be. A `move` nélküli closure a lehető legszűkebb hivatkozást használja: `&T`-t, ha elég, `&mut T`-t, ha módosít, és csak akkor mozgat, ha elkerülhetetlen.

Szabály. A trait-hierarchia miatt egy `Fn` closure átadható olyan függvénynek, amely `FnMut` vagy `FnOnce` boundot vár. Egy `FnMut` closure átadható `FnOnce` paraméternek, de `Fn`-t váró paraméternek nem. A fordítási hibák ilyenkor a trait-követelmények megsértését jelzik.

13.1.3 C++ és Rust összevetés

A C++ lambda capture-listája és a Rust implicit capture közötti különbség jól látható egy egyszerű példán.

```cpp
#include <iostream>

int main() {
    int x = 10;
    auto f = [&x]() { return x + 1; };          // hivatkozás
    auto g = [x]() mutable { return ++x; };     // másolat, módosítható
    auto h = [x = std::move(x)]() { return x; }; // move capture
    std::cout << f() << ' ' << g() << ' ' << h() << '\n';
}
```

A Rust-megfelelő:

```rust
fn main() {
    let mut x = 10;                     // mut kell: a g saját másolata módosul
    let f = || x + 1;                   // &x, Fn
    let mut g = move || { x += 1; x };  // x Copy, move másol, FnMut
    let h = move || x;                  // x Copy, move másol, Fn
    println!("{} {} {}", f(), g(), h());
}
```

Kimenet: `11 11 10`, a C++-példával megegyezően. A két `mut` két különböző dolgot tesz módosíthatóvá. A `let mut x` azért kell, mert a `move` closure az `x` *másolatát* a closure-be viszi, és ennek módosítása csak akkor megengedett, ha az eredeti kötés `mut` (különben `E0594`: „cannot assign to `x`, as it is not declared as mutable"). A `let mut g` pedig azért kell, mert egy `FnMut` closure meghívása `&mut self`-et igényel, vagyis maga a `g` kötés módosítható kell legyen. Mivel a `g` a saját másolatán dolgozik, az eredeti `x` értéke (10) nem változik, ezért a `h()` is 10-et ad.

A legfontosabb különbség az, hogy a Rustban a `[&x]`-nek megfelelő hivatkozás az alapértelmezett, és nincs szükség `&` jelölésre. A C++ `[x]` másoló capture-jéhez a Rustban `move` kell, mert a Rust closure alapból nem másol, hanem hivatkozik. A `mutable` kulcsszó a Rustban nem létezik ilyen formában; a módosító closure automatikusan `FnMut` lesz, és a változót `mut`-tal kell deklarálni. A `move` closure nem Copy típus esetén felel meg a C++ move capture-nek.

A `FnOnce` viselkedésére a legjobb példa, amikor a closure egy nem `Copy` értéket mozgat ki magából.

```
let s = String::from("hello");
let consume = move || s;   // s mozgatva a closure-be
println!("{}", consume()); // s kikerül a closure-ből
// consume();             // hiba: use of moved value
```

C++-ban a lambda újrahívása szabályos és nem definiálatlan viselkedés: a `[s = std::move(s)]() mutable { return std::move(s); }` lambda második hívásánál az `s` *moved-from* állapotú `std::string`, amely érvényes, de nem meghatározott tartalmú (általában üres: ellenőrzött kimenet `a=hello b=[]`). Rustban a kimozgató closure `FnOnce`: a második hívás fordítási hiba, így az érvénytelen állapot el sem érhető.

13.1.4 Kidolgozott példa

A leggyakoribb feltörekvő hiba, amikor egy closure-t vissza akarunk adni egy függvényből, de a closure a függvény paraméterére hivatkozik.

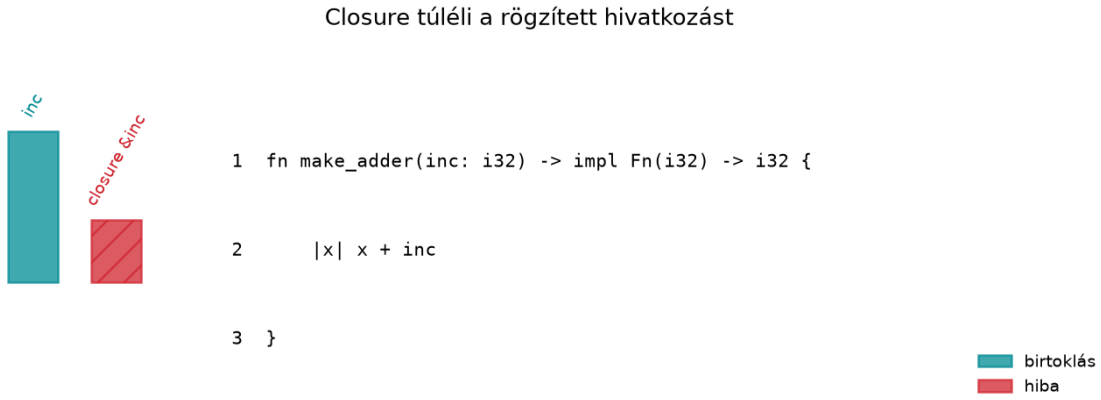
Ez a változat szándékosan nem fordul le:

```
fn make_adder(inc: i32) -> impl Fn(i32) -> i32 {
    |x| x + inc
}
```

A hiba lényege:

```
E0373: closure may outlive the current function, but it borrows `inc`,
which is owned by the current function
```

A `make_adder` visszatérő closure túléli a függvény veremkeretét, tehát az `inc`-re mutató hivatkozás érvénytelen lenne. Ezt a `lifetimes` diagram is szemlélteti.



13.1. ábra: Closure túléli a rögzített hivatkozást - A make_adder visszatérő closure hivatkozik az inc-re, de az inc veremkerete felszabadul.

A javítás a `move` kulcsszóval:

```
fn make_adder(inc: i32) -> impl Fn(i32) -> i32 {
    move |x| x + inc
}

fn main() {
    let add_five = make_adder(5);
    println!("{}", add_five(10)); // 15
}
```

Mivel az `i32` `Copy` típus, a `move` a closure saját másolatát rögzíti az `inc`-ből, így a visszaadott closure önálló tulajdonú, és nincs élettartam-probléma.

A `FnMut` működését egy `apply_twice` függvénnyel mutatjuk be:

```
fn apply_twice<F: FnMut() -> i32>(mut f: F) -> i32 {
    f() + f()
}

fn main() {
    let mut x = 0;
    let r = apply_twice(|| {
        x += 1;
        x
    });
    println!("r = {r}, x = {x}");
}
```

A várt kimenet:

```
r = 3, x = 2
```

A closure `FnMut`, mert módosítja az `x`-et. A `apply_twice` kétszer hívja, ezért a zárt változó értéke kétszer növekszik.

13.1.5 Labor: rkv parancs-callbackek closure-rel

Munkakönyvtár: `~/rust-labor/rkv/`

Ebben a laborban egy callback-alapú parancsdiszpécsert készítünk az `rkv-core` könyvtárban. A cél egy olyan függvény, amely egy `Command` slice-on megy végig, és minden parancshoz meghív egy closure-t. Ez a függvény később az iterátor-alapú feldolgozásban (Iterátorok: lusta adapterek és saját iterátor) is újra felhasználható.

A kiinduló állapot az Trait objektumok és dinamikus diszpécs ellenőrzési pontja. A `Command` enum az `rkv-core/src/command.rs`-ben él, *tuple-variánsokkal*: `Get(String)`, `Set(String, String)`, `Delete(String)`, `List(Option<String>)` (Az enum mint összeg-típus és az Option a null helyett). A callback-kód és a tesztek ehhez igazodnak; a variánsok szerkezetét nem változtatjuk.

13.1.5.1 1. lépés: ellenőrizzük a workspace állapotát

```
cd ~/rust-labor/rkv
cargo test -p rkv-core
```

Ennek sikeresen le kell futnia, mielőtt továbblépünk.

13.1.5.2 2. lépés: hozzuk létre a `callback` modult

Kódblokk — fájl: `rkv-core/src/lib.rs` · művelet: beillesztés a `pub mod` sorok közé · ellenőrzés: `cargo build -p rkv-core`

```
pub mod callback;
```

Hozd létre az `rkv-core/src/callback.rs` fájlt a következő tartalommal:

Kódblokk — fájl: `rkv-core/src/callback.rs` · művelet: új fájl · ellenőrzés: `cargo build -p rkv-core`

```
use crate::command::Command;

/// Végigjárja a parancsokat, és mindegyikre meghívja a callbacket.
pub fn visit_commands<F>(commands: &[Command], mut f: F)
where
    F: FnMut(&Command),
{
    for command in commands {
        f(command);
    }
}
```

A függvény `FnMut(&Command)` boundot használ, mert a callback módosíthat külső állapotot (például számlálót), de magát a `Command` értéket nem kell módosítania. A `&Command` paraméter lehetővé teszi, hogy a closure olvassa a parancs adatait anélkül, hogy tulajdonba venné.

13.1.5.3 3. lépés: írjunk unit tesztet a closure használatára

Az `rkv-core/src/callback.rs` aljára illeszd be a következő tesztmodult:

Kódblokk — fájl: `rkv-core/src/callback.rs` · művelet: beillesztés a fájl végére · ellenőrzés: `cargo test -p rkv-core callback` → 1 sikeres teszt ebben a lépésben (a törzslabor végén 2)

```
#[cfg(test)]
mod tests {
    use super::*;

    fn sample() -> Vec<Command> {
        vec![
            Command::Set("a".to_string(), "1".to_string()),
            Command::Get("b".to_string()),
            Command::Set("c".to_string(), "2".to_string()),
        ]
    }

    #[test]
    fn count_set_commands() {
        let commands = sample();
        let mut count = 0;
        visit_commands(&commands, |cmd| {
            if matches!(cmd, Command::Set(..)) {
                count += 1;
            }
        });
        assert_eq!(count, 2);
    }
}
```

Figyeljük meg, hogy itt nincs `move` a closure előtt, tehát a closure a `count` változót `&mut` hivatkozással rögzíti. A `visit_commands` hívása után a `count` továbbra is elérhető, és az `assert_eq!` ellenőrizheti.

13.1.5.4 4. lépés: a `move` closure két arca

A `count` egy `i32`, vagyis `Copy`. Egy `move` closure ezért **másolatot** rögzít, a külső `count` pedig érintetlen marad. Ez a változat *lefordul*, de nem azt teszi, amit várnánk:

Kódblokk — fájl: `rkv-core/src/callback.rs` · művelet: beillesztés a `mod tests` blokkba · **rossz megfigyelt állapot**: lefordul, a külső `count` 0 marad · ellenőrzés: `cargo test -p rkv-core callback`

```
    #[test]
    #[allow(unused_assignments)] // szándékos: a másolat értékét senki nem olvassa
    fn move_closure_counts_a_copy() {
        let commands = sample();
        let mut count = 0;
        visit_commands(&commands, move |cmd| {
            if matches!(cmd, Command::Set(..)) {
                count += 1; // a closure saját másolatát növeli
            }
        });
        assert_eq!(count, 0); // a külső count nem változott
    }
```

A `count += 1` a closure belsejében a másolatot növeli (a fordító ezt maga is jelzi: „value captured by `count` is never read” és „value assigned to `count` is never read”, vagyis `unused_assignments` figyelmeztetés; a tesztben ezért van `#[allow(unused_assignments)]`, hogy a workspace `clippy -D warnings` mellett is tiszta maradjon), a külső `count` így 0 marad, míg a `move` nélküli, `&mut`-kölcsönzést használó változatban a végeredmény 2 volt. A hiba tehát nem fordítási, hanem **logikai**: a számláló csendben rossz.

Valódi `E0382` akkor keletkezik, ha a rögzített érték **nem `Copy`**, és a closure tulajdonba veszi:

Kódblokk — önálló példa · **szándékosan hibás** · várt hiba: `E0382` (`borrow of moved value:`seen“)

```
let commands = sample();
let mut seen: Vec<String> = Vec::new();
visit_commands(&commands, move |cmd| seen.push(cmd.to_line()));
println!("{seen:?}"); // E0382: a seen a closure-be került
```

A javítás ismét a `move` elhagyása (a closure `&mut seen`-t rögzít), vagy az eredmény visszaadása a closure-ből.

13.1.5.5 5. lépés: futtassuk a tesztet

```
cargo test -p rkv-core callback
```

A várt kimenet (a sorrend változhat; a **törzslabor** 2 tesztet tartalmaz, a harmadik a c) feladat megoldása után jelenik meg):

```
running 2 tests
test callback::tests::count_set_commands ... ok
test callback::tests::move_closure_counts_a_copy ... ok
```

Ellenőrzési pont a c) feladat után. A `visit_commands_mut` megoldásával a `cargo test -p rkv-core callback` már 3 tesztet futtat; a harmadik a külön `tests_mut` modulban van:

```
running 3 tests
test callback::tests::count_set_commands ... ok
test callback::tests::move_closure_counts_a_copy ... ok
test callback::tests_mut::convert_delete_to_get ... ok
```

Ezzel elkészült a `visit_commands` függvény, amely closure-t fogad paraméterként. **Továbbvitt eredmény:** ez a callback-minta az iterátoroknál (Iterátorok: lusta adapterek és saját iterátor) az `Iterator::for_each`-hez hasonló logikát készíti elő, a `move` closure-ök pedig a párhuzamos (Szálak, Send és Sync: data race fordítási időben kizárva) és aszinkron (Future-ök és async/await: mi történik a motorháztető alatt) kódban lesznek elengedhetetlenek.

13.1.6 Tipikus hibák és átállási csapdák

1. **A C++ `[=]` mintájára másolást várunk.** A Rust closure alapból hivatkozik, nem másol. Ha a closure-t el akarjuk tárolni, és a környezeténél tovább kell élnie, akkor `move` kell. Ha a tulajdonost meg akarjuk őrizni, és a closure-t is el kell tárolni, a másolatot a closure *előtt* készítjük (`let c = x.clone();`), és azt `move`-oljuk a closure-be. A `clone()` a törzsben `move` nélkül nem segít: a closure ilyenkor is az eredetit kölcsönzi, `move`-val pedig az eredeti már a closure-é lenne.

2. **`FnMut` closure átadása `Fn`-t váró paraméternek.** Gyakori hiba, amikor egy módosító closure-t próbálunk átadni egy olyan függvénynek, amely csak olvasó callbacket vár. A fordítási hiba egyértelmű: a closure nem teljesíti az `Fn` boundot, mert `&mut`-tal rögzíti a változót. A megoldás vagy a függvény boundjának lazítása `FnMut`-ra, vagy a closure áttervezése.

3. **A `move` closure után a külső változó használata.** Amikor egy nem `Copy` értéket `move`-val rögzítünk, a külső változó érvénytelenné válik. Ez a hiba különösen a szálaknál (Szálak, Send és Sync: data race fordítási időben kizárva) válik gyakorivá, ahol a `move` elengedhetetlen, de a külső változó elérése már nem lehetséges. Ilyenkor `Arc` vagy `Clone` szükséges.

4. **A `Rc<RefCell<T>>` túlhasználata.** A C++ `shared_ptr` tapasztalat alapján csábító lehet minden closure-t `Rc<RefCell<_>>`-kel körbevenni, hogy a borrow checker „ne zavarjon”. Ez azonban futásidejű kölcsönzési hibákhoz vezet, és megkerüli a Rust tervezési előnyeit. A closure-öket elsősorban a természetes capture-szabályokkal érdemes használni.

13.1.7 Feladatok

13.1.7.1 a) Lefordul-e és miért?

```rust
fn call<F: Fn()>(f: F) {
    f();
}

fn main() {
    let mut x = 0;
    call(|| x += 1);
}
```

13.1.7.2 b) C++-kód átírása idiomatikus Rustra

```cpp
int count = 0;
auto inc = [&count]() { return ++count; };
auto total = inc() + inc();
```

13.1.7.3 c) rkv implementációs feladat

Bővítsd az `rkv-core/src/callback.rs` modult egy `visit_commands_mut` függvénnyel, amely `&mut Command`-ot ad át a closure-nek, és írj tesztet, amely egy `Command::Delete` parancsot `Command::Get`-re módosít a callbackben. (A `Command` tuple-variánsait használd: `Delete(String)`, `Get(String)`.)

13.1.7.4 d) Hibajavítás

Az alábbi kód nem fordul le. Javítsd úgy, hogy a külső `s` elérhető maradjon a closure hívása után is, és indokold a választást.

```rust
let s = String::from("hello");
let printer = move || println!("{s}");
printer();
println!("{s}");
```

13.1.8 Megoldások

13.1.8.1 a)

Nem fordul le. A `|| x += 1` closure módosítja az `x`-et, ezért `FnMut` traitet implementál, nem `Fn`-t. A `call` függvény `Fn` boundot vár, így a hívás típushibát ad. A javítás:

```
fn call<F: FnMut()>(mut f: F) {
    f();
}

fn main() {
    let mut x = 0;
    call(|| x += 1);
}
```

13.1.8.2 b)

```
fn main() {
    let mut count = 0;
    let mut inc = || {
        count += 1;
        count
    };
    let total = inc() + inc();
    println!("{total}");
}
```

A closure `FnMut`, mert módosítja a `count`-ot. A `mut` a closure deklarációjánál szükséges, mert a `FnMut` hívás `&mut self`-et igényel.

13.1.8.3 c)

```
// rku-core/src/callback.rs: a meglévő `use crate::command::Command;` sor mellé,
// a `tests` modulon kívülre (a teszt a meglévő `mod tests` blokkba kerül).
pub fn visit_commands_mut<F>(commands: &mut [Command], mut f: F)
where
    F: FnMut(&mut Command),
{
    for command in commands {
        f(command);
    }
}

#[cfg(test)]
mod tests_mut {
    use super::*;

    #[test]
    fn convert_delete_to_get() {
        let mut commands = vec![Command::Delete("k".to_string())];
        visit_commands_mut(&mut commands, |cmd| {
            if let Command::Delete(key) = cmd {
                *cmd = Command::Get(key.clone());
            }
        });
        assert_eq!(commands[0], Command::Get("k".to_string()));
    }
}
```

A `visit_commands_mut FnMut(&mut Command)` closure-t vár, így a callback módosíthatja a parancsot.

13.1.8.4 d)

A `move` closure tulajdonba veszi az `s`-t, ezért a külső `s` a hívás után érvénytelen. Két lehetséges javítás:

Ha az élettartam engedi, egyszerűen ne használjunk `move`-ot:

```
let s = String::from("hello");
let printer = || println!("{s}");
printer();
println!("{s}");
```

A closure ekkor `&s`-t rögzít, és `s` a hívás után is elérhető. A puszta eltárolás önmagában nem követel `move`-ot vagy klónt: egy kölcsönzést rögzítő closure ugyanabban a hatókörben tárolható (például egy `Vec`-ben), amíg a kölcsönzött adat elég sokáig él. `move` akkor kell, ha a closure túléli a hatókört (visszaadás, `thread::spawn`), `clone()` pedig akkor, ha emellett a külső értéket is tovább használni akarjuk: az önálló másolatot a closure létrehozása *előtt* klónozzuk, és azt mozgatjuk bele:

```
let s = String::from("hello");
let s2 = s.clone();
let printer = move || println!("{s2}");
printer();
println!("{s}");
```

A `move` a `s2` másolatot rögzíti, így az eredeti `s` érintetlen marad.

13.1.9 Rövid összefoglaló

A Rust closure-ök automatikusan rögzítik a környezetüket, és a rögzített változók használati módja határozza meg, hogy a `Fn`, `FnMut` vagy `FnOnce` trait-et implementálják. A `Fn` closure többször hívható és csak olvas, az `FnMut` módosíthat, az `FnOnce` pedig felemésztheti a rögzített értékeket. A `move` kulcsszó minden használt változót by value rögzít, ami elengedhetetlen, ha a closure túléli a környezetét, például szálakban (Szálak, Send és Sync: data race fordítási időben kizárva) vagy aszinkron taskokban (Future-ök és async/await: mi történik a motorháztető alatt). A laborban létrehoztuk az `rkv-core/src/callback.rs` modult a `visit_commands` függvénnyel, amely `FnMut(&Command)` closure-t fogad. Ez a callback-minta az iterátorok (Iterátorok: lusta adapterek és saját iterátor) adaptereinél köszön vissza, ahol a closure-ök hasonlóan rugalmasan kezelik az elemeket.

13.1.10 Források és ellenőrzési hiányok

A fejezet a Rust Book closure-fejezetére (https://doc.rust-lang.org/book/), a Rust Reference closure-típusokról szóló részére (https://doc.rust-lang.org/reference/) és a standard könyvtár `Fn`, `FnMut` és `FnOnce` trait-dokumentációjára támaszkodik. A `Command` enum elérési útja `rkv_core::command::Command`. Az `E0373` és `E0382` hibakódok stabilak, a pontos hibaüzenet szövege a rögzített toolchainre (Rust 1.99.1) vonatkozik.

13.2 Iterátorok: lusta adapterek és saját iterátor

13.2.1 Motiváció

C++-ban egy vektor elemeinek szűrése és transzformálása gyakran kézi ciklussal történik: indexelünk, `push_back`-et hívunk, ellenőrizzük a határokat, és közben könnyen elkövetünk off-by-one hibát vagy véletlenül módosítjuk a bejárt konténert. A C++20 `std::ranges` már kínál lusta view-kat, de a Rust iterátorai ezt a szemléletet a nyelv szerves részeként, trait-alapú, egységes felülettel valósítják meg. A Rust iterátorlánc kifejező, lusta, és a fordító a borrow checkerrel garantálja, hogy az iteráció közben nem érvénytelenítjük a gyűjteményt. Ebben az alfejezetben megismerjük az `Iterator` trait-et, a három alapvető iterátortípust, az adaptereket, és saját iterátort írunk az rkv kulcsainak bejárására.

13.2.2 Fogalmak és szabályok

Definíció. Az `Iterator` trait egyetlen kötelező metódust ír elő: a `next`-et, amely `Option<Self::Item>`-et ad vissza. Az `Item` asszociált típus az iterátor által szolgáltatott elem típusa. A trait számos további metódust (adaptereket és fogyasztókat) default implementációval biztosít, amelyek a `next`-re épülnek.

```
pub trait Iterator {
    type Item;
    fn next(&mut self) -> Option<Self::Item>;
    // map, filter, fold, collect, ...
}
```

Definíció. A *lusta adapter* (lazy adapter) olyan iterátor-transzformáció, amely a `next` hívásakor végzi el a számítást. A *fogyasztó* (consumer) olyan metódus, amely végighúzza az iterátort, és értéket, gyűjteményt vagy mellékhatást állít elő. A `map`, `filter`, `enumerate`, `zip`, `take`, `skip`, `flat_map` adapterek; a `collect`, `fold`, `count`, `sum`, `for_each` és a `for` ciklus fogyasztók. Amíg nincs fogyasztó, az adapterlánc egyetlen elemet sem dolgoz fel.

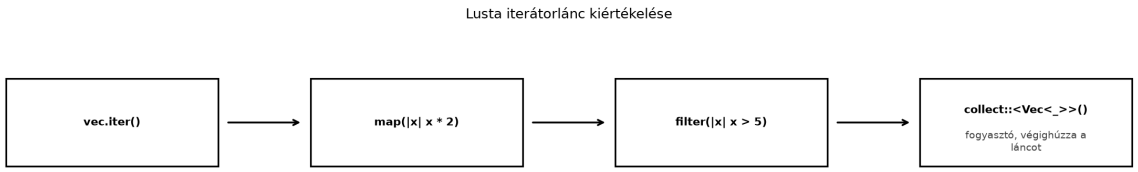
Szabály. A `for ciklus` szintaktikai cukor: a `for x in iterable` kifejezés az `iterable` értéken meghívja az `IntoIterator::into_iter`-et, majd addig hívja a kapott iterátor `next`-jét, amíg az `Some`-ot ad. A `&v` és `&mut v` is implementálja az `IntoIterator`-t, így a `for &x in &v` a `v.iter()`-nek, a `for x in &mut v` a `v.iter_mut()`-nak felel meg.

Szabály. Három alapvető iterátortípus létezik:

- `iter()`: `&T` elemeket ad, a gyűjteményt nem fogyasztja, a kollekció kölcsönzött marad.
- `iter_mut()`: `&mut T` elemeket ad, a gyűjteményt nem fogyasztja. A kölcsönzési szabály a *gyűjteményre* vonatkozik (amíg az iterátor él, a gyűjtemény kizárólagosan kölcsönzött); az elemek `&mut` referenciái viszont különböző elemekre mutatnak, ezért egyszerre élhetnek, mint a `let refs: Vec<&mut i32> = v.iter_mut().collect();` példában. Ez nem szabálysértés.
- `into_iter()`: a jelentése a **receiver típusától** függ. A `Vec<T>` értéken hívott `v.into_iter()` `T` elemeket ad, és a gyűjteményt elfogyasztja (utána `v` nem használható); a `(&v).into_iter()` ugyanaz, mint a `v.iter()`: `&T` elemeket ad, és `v` megmarad; a `(&mut v).into_iter()` pedig `&mut T` elemeket.

Szabály. A `collect` az iterátort `B` típusú gyűjteménnyé alakítja, ha `B` implementálja a `FromIterator<Self::Item>` trait-et. Ha az elemek `Result<T, E>` típusúak, akkor a `Result<Vec<T>, E>` is implementálja a `FromIterator`-t: a gyűjtés az első `Err`-nél leáll, és `Err`-t

ad; ha minden elem `Ok`, akkor `Ok(Vec<T>)` lesz az eredmény. Hasonlóan működik `Option` esetén is.



13.2. ábra: Lusta iterátorlánc kiértékelése - Az adapterek nem hajtódnak végre, amíg egy fogyasztó (collect) meg nem hívja a next()-et.

A fenti ábra azt mutatja, hogy az adapterek csak egy leírást építenek fel; a tényleges munka a `collect` hívásakor történik meg, amikor a fogyasztó sorban meghívja a `next`-et a lánc végén.

13.2.3 C++ és Rust összevetés

C++20 ranges segítségével lusta view-kat fűzhetünk össze:

```cpp
#include <iostream>
#include <ranges>
#include <vector>

int main() {
    std::vector<int> v = {1, 2, 3, 4, 5, 6};
    auto result = v
        | std::views::filter([](int x) { return x % 2 == 0; })
        | std::views::transform([](int x) { return x * x; });

    for (int x : result) {
        std::cout << x << ' ';
    }
}
```

Rustban ugyanez idiomatikusan:

```rust
fn main() {
    let v = vec![1, 2, 3, 4, 5, 6];
    let result: Vec<_> = v
        .iter()
        .filter(|&&x| x % 2 == 0)
        .map(|&x| x * x)
        .collect();
    println!("{result:?}");
}
```

A különbség lényeges: a C++ ranges view-k a `begin()`/`end()` és a koncepciók köré épülnek, míg a Rust az `Iterator` trait egységes `next()` metódusára. A Rust verzióban a closure-ök explicit mintát kapnak: `v.iter()` `&i32` elemeket ad, a `filter` predikátuma ezért `&&i32`-et kap (`&&x`), a `map` pedig `&x`-et. A C++-ban a lambda paramétere `int` (a view értékként adja át az elemet). A Rust `collect` típusinferencia a turbofish (`::<Vec<_>>`) segítségével egyértelmű, míg a C++-ban C++23-ig explicit konstruktor vagy `std::ranges::to` kellett. Mindkét megoldás zero-cost absztrakció: a fordító inline-eli a hívásokat, és a generált kód gyakorlatilag megegyezik a kézi ciklusával.

13.2.4 Kidolgozott példa: lustaság és a `Result` gyűjtése

A lustaság megértéséhez nézzünk egy hibás kódot, amelyben nincs fogyasztó:

```
fn main() {
    let v = vec![1, 2, 3, 4, 5];
    v.iter().map(|x| {
        println!("{x}");
        x * 2
    });
}
```

Ez a program **nem ír ki semmit**. A `map` csak egy új iterátort ad vissza, de a `next`-et soha nem hívja meg senki, mert nincs fogyasztó. A lánc eredményét el kellene tárolni, vagy meg kell hívni a `collect`-et.

Helyes változat:

```
fn main() {
    let v = vec![1, 2, 3, 4, 5];
    let doubled: Vec<_> = v.iter().map(|&x| x * 2).collect();
    println!("{doubled:?}");
}
```

Most a `collect` végighúzza az iterátort, és a `map` lefut minden elemen. Várt kimenet: `[2, 4, 6, 8, 10]`.

A `Result` elemeket tartalmazó iterátor gyűjtése gyakori hibaforrás. Tekintsük a következő függvényt:

```
fn parse_all(items: Vec<&str>) -> Result<Vec<i32>, std::num::ParseIntError> {
    items
        .iter()
        .map(|s| s.parse::<i32>())
        .collect::<Result<Vec<_>, _>>()
}

fn main() {
    let ok = parse_all(vec!["1", "2", "3"]);
    let err = parse_all(vec!["1", "x", "3"]);
    println!("{ok:?}");
    println!("{err:?}");
}
```

A `collect` a `FromIterator` implementációt használja: ha minden `parse` `Ok`-t ad, akkor `Ok(vec![1, 2, 3])`; ha az egyik `Err`, akkor a gyűjtés leáll, és `Err(ParseIntError)` lesz az eredmény. Itt a turbofish **nem kötelező**: a függvény visszatérési típusa (`Result<Vec<i32>, ParseIntError>`) már meghatározza a célt, így a rövidebb `items.iter().map(|s| s.parse::<i32>()).collect()` alak is lefordul (ellenőrzött kimenet: `Ok([1, 2, 3])` és `Err(ParseIntError { kind: InvalidDigit })`). Típusannotációt akkor kell adni, ha a cél típusát semmi sem rögzíti:

```
// error[E0283]: type annotations needed
let result = [1, 2, 3].iter().map(|x| x + 1).collect();
```

Itt a `collect` célja kétértelmű (`Vec<i32>`, `HashSet<i32>`, `VecDeque<i32>` stb. is lehetne), ezért a turbofish (`collect::<Vec<_>>()`) vagy a változó típusának annotálása kell.

13.2.5 Labor: saját iterátor az rkv kulcsainak bejárására

Ebben a laborban saját iterátort implementálunk az `rkv-core` crate-ben, amely a memóriabeli tároló kulcsainak kölcsönző bejárását teszi lehetővé. A cél egy `Keys` típus, amely az `Iterator` trait-et implementálja, és egy új `Store::iter_keys()` metódus ezt adja vissza. A már létező `Store::keys() -> Vec<String>` (Az rkv mintaprojekt indítása: Cargo-workspace és első CLI) viselkedése nem változik: továbbra is minden kulcsot ad, az üres értékűeket is.

Munkakönyvtár: `~/rust-labor/rkv` (a workspace gyökere). A kiinduló állapot a Trait objektumok és dinamikus diszpécs és a Closure-ök: Fn, FnMut, FnOnce ellenőrzési pontja. A `Store` (a `MemStore` alias mögötti típus) az `rkv-core/src/store/mod.rs`-ben él, belső tárolója `data: HashMap<String, Vec<u8>>`; új típust nem hozunk létre, a meglévőt bővítjük. A `Store::new()` az rkv mintaprojekt indítása: Cargo-workspace és első CLI óta létezik.

A `Keys` struct egy meglévő `hash_map::Keys` iterátort csomagol be, hogy saját, `&str` elemű típust adjon:

Kódblokk — fájl: `rkv-core/src/store/mod.rs` · művelet: beillesztés az `impl Store` blokk elé; a fájl elejére `use std::collections::hash_map;` · ellenőrzés: `cargo build -p rkv-core`

```
/// A tároló kulcsainak kölcsönző bejárása (minden kulcs, az üres értékűek is).
pub struct Keys<'a> {
    inner: hash_map::Keys<'a, String, Vec<u8>>,
}

impl<'a> Iterator for Keys<'a> {
    type Item = &'a str;

    fn next(&mut self) -> Option<Self::Item> {
        self.inner.next().map(|k| k.as_str())
    }
}
```

A `hash_map::Keys` iterátor `next` metódusa `Option<&String>`-et ad; a `map` ezt alakítja `Option<&str>`-ré. Az élettartam `'a` a `Store` belső `HashMap`-éhez kötődik, így a visszaadott referenciák addig érvényesek, amíg a tároló kölcsönzése él.

Az `iter_keys` metódus az iterátort adja, a régi `keys` pedig mostantól erre épül (a viselkedése azonos marad):

Kódblokk — fájl: `rkv-core/src/store/mod.rs` · művelet: az `impl Store` blokkba új `iter_keys`, a meglévő `keys` törzsének cseréje · ellenőrzés: `cargo test -p rkv-core store`

```
    /// Kulcsok kölcsönző bejárása, allokáció nélkül.
    pub fn iter_keys(&self) -> Keys<'_> {
        Keys {
            inner: self.data.keys(),
        }
    }

    /// Az összes kulcs (rendezetlen) listája: a `Keys` iterátorra épül.
    pub fn keys(&self) -> Vec<String> {
        self.iter_keys().map(str::to_owned).collect()
    }
```

Az `iter_keys` visszatérési típusa `Keys<'_>`: az elision szabályai szerint a visszatérési élettartam az `&self` élettartamához kötődik.

Most használjuk az iterátort a `Storage` trait `list` metódusának memóriabeli megvalósításában (Trait-ek és generikusok: sablonok és concepts helyett). A korábbi, `Vec<String>`-et gyártó közbenső lépés helyett a lánc közvetlenül a kölcsönző iterátoron fut:

Kódblokk — fájl: `rkv-core/src/storage.rs` · művelet: a `Storage for MemStore` `list` metódusának cseréje · ellenőrzés: `cargo test -p rkv-core storage`

```
    fn list(&self, prefix: Option<&str>) -> Result<Vec<(String, Vec<u8>)>> {
        let mut items: Vec<_> = self
            .iter_keys()
            .filter(|k| prefix.is_none_or(|p| k.starts_with(p)))
            .filter_map(|k| MemStore::get(self, k).map(|v| (k.to_owned(), v.clone())))
            .collect();
        items.sort_by(|a, b| a.0.cmp(&b.0));
        Ok(items)
    }
```

Ez egy tipikus iterátorlánc: szűrés, transzformáció, gyűjtés. A `filter` csak a prefixet tartalmazó kulcsokat engedi tovább, a `filter_map` kulcs–érték párt épít, a `collect` pedig `Vec`-et; a végén rendezünk, mert a `HashMap` bejárási sorrendje nem meghatározott.

Végül tesztek az `iter_keys`-re és arra, hogy az üres értékű kulcs is szerepel a `keys` és a `list` eredményében:

Kódblokk — fájl: `rkv-core/src/store/mod.rs` · művelet: beillesztés a meglévő `mod tests` blokkba · ellenőrzés: `cargo test -p rkv-core iter_keys`

```
    #[test]
    fn iter_keys_iterates_all_keys_including_empty_values() {
        let mut store = Store::new();
        store.set("alpha".to_string(), b"1".to_vec());
        store.set("beta".to_string(), Vec::new()); // üres értékű kulcs
        let mut keys: Vec<_> = store.iter_keys().collect();
        keys.sort();
        assert_eq!(keys, vec!["alpha", "beta"]);

        let mut owned = store.keys();
        owned.sort();
        assert_eq!(owned, vec!["alpha".to_string(), "beta".to_string()]);
    }

// a storage.rs tesztmoduljába (a Storage list is megtartja az üres értékű kulcsot):
    #[test]
    fn list_keeps_keys_with_empty_values() {
        let mut store = MemStore::new();
        Storage::set(&mut store, "beta".into(), Vec::new()).unwrap();
        let keys: Vec<String> = Storage::list(&store, None).unwrap().into_iter().map(|(k, _)|
         ↪  k).collect();
        assert_eq!(keys, ["beta"]);
    }
```

A teszt futtatása:

```
cd ~/rust-labor/rkv
cargo test -p rkv-core
```

Várt kimenet: minden teszt sikeres, a `keys`, az `iter_keys` és a `list` egyaránt visszaadja az üres értékű kulcsot is.

Ellenőrzési pont. Az `rkv-core/src/store/mod.rs`-ben megjelent a `Keys<'a>` iterátor és a `Store::iter_keys()` metódus; a `Store::keys()` és a `Storage::list` viselkedése változatlan

(minden kulcs). Ezt az iterátort később a gyűjteményekkel foglalkozó alfejezetben, a párhuzamos iterátoroknál és a teljesítményhangolásnál is használjuk.

13.2.6 Tipikus hibák és átállási csapdák

1. **A lustaság elfelejtése.** A `v.iter().map(...)` önmagában nem végez semmilyen munkát; a lánc csak akkor fut le, ha egy fogyasztót (például `collect`, `for_each` vagy `fold`) is meghívunk. Ez a hiba csendben maradhat, mert a program lefordul, de nem csinálja a várt műveletet.

2. **A `collect` típusának elhagyása.** A `let result = iterator.collect();` típusannotáció nélkül `E0283`-mal (type annotations needed) áll meg, mert a fordító nem tudja megállapítani a célgyűjteményt. A megoldás a turbofish (`collect::<Vec<_>>()`) vagy a változó típusának annotálása; ha a célt a környezet (például a függvény visszatérési típusa) már rögzíti, az annotáció elhagyható.

3. **`iter_mut` és `iter` keverése.** Ha `iter()`-rel kapott `&T` referenciát próbálunk módosítani, a fordító hibát jelez. Hasonlóan, ha `iter_mut`-ot használunk, miközben más shared referenciák élnek a gyűjteményre, a borrow checker elutasítja a kódot. A megoldás a kölcsönzési szabályok betartása: vagy csak `iter`, vagy csak `iter_mut`, de nem egyszerre.

4. **`into_iter` tulajdonjogvesztés.** A `v.into_iter()` (érték receiver) elfogyasztja a gyűjteményt; ha utána még hivatkozunk rá, fordítási hiba lép fel. A `(&v).into_iter()` és a `for x in &v` viszont kölcsönöz (nem fogyaszt). A lusta adapterek sem garantáltan allokálás- és indexelésmentesek: a closure, a `flat_map` és egy saját `next` tetszőleges munkát végezhet. Ha szükségünk van a gyűjteményre az iteráció után, használjunk `iter`-t vagy `iter_mut`-ot.

13.2.7 Feladatok

1. **Lefordul-e és miért?** Adott a következő kód:

```
let v = vec![1, 2, 3];
let it = v.iter().map(|x| x + 1);
println!("{}", v[0]);
```

Lefordul-e? Magyarázd meg az `iter` és az `into_iter` közötti különbséget ebben a kontextusban!

2. **C++-kód átírása idiomatikus Rustra.** Az alábbi C++ kód egy vektor összes páros elemének négyzetét számítja ki kézi ciklussal:

```
std::vector<int> result;
for (int x : v) {
    if (x % 2 == 0) {
        result.push_back(x * x);
    }
}
```

Írd át idiomatikus Rust iterátorláncra! Ügyelj a `collect` típusának megadására.

3. **Implementációs feladat az rkv-ban.** A `Store::keys()` és az `iter_keys()` az összes kulcsot adja, az üres értékűeket is, és ez így marad. Írj egy *külön* `NonEmptyKeys<'a>` iterátort és egy `Store::nonempty_keys()` metódust, amely csak a nem üres értékű kulcsokat adja vissza (az értékük hossza nagyobb, mint 0). Az iterátor a belső `hash_map::Iter`-re

épüljön, és a `next` az első nem üres értékű kulcsig lépkedjen. Írj tesztet, amely megmutatja a különbséget: egy üres értékű kulcs szerepel az `iter_keys`-ben, de nem szerepel a `nonempty_keys`-ben.

4. **Hibajavítás/refaktorálás.** A következő kód szándékosan hibás:

```
fn sum_of_squares(v: &Vec<i32>) -> i32 {
    let squares = v.iter().map(|x| x * x);
    squares.sum()
}

fn main() {
    let v = vec![1, 2, 3];
    println!("{}", sum_of_squares(&v));
}
```

Ez a kód lefordul és működik, de van egy stilisztikai és egy API-javaslat. A `squares` lokális kötés összevonása stílusválasztás (a teljesítményen nem változtat: az adapterlánc ugyanaz), a `&Vec<i32>` paraméter lecserélése `&[i32]`-re viszont valódi API-javítás. Módosítsd a függvényt, és magyarázd el, miért működik a `sum` fogyasztó a lusta `map` után!

13.2.8 Megoldások

1. **Lefordul-e?** Igen, lefordul. A `v.iter()` a vektort nem fogyasztja, csak kölcsönzi; az `it` egy lusta iterátor, amely a `next` hívásakor dolgozik. A `v[0]` kifejezés a vektor indexelése, amihez nem szükséges a tulajdonjog. Ha azonban `v.into_iter()`-et használtunk volna, a `v` tulajdonjoga átkerült volna az iterátorba, és a `v[0]` hivatkozás E0382 hibát adna, mert a `v` már érvénytelen. Az `iter` tehát nem fogyasztó, az `into_iter` pedig igen.

2. **C++ → Rust átírás.** Az idiomatikus Rust iterátorlánc:

```
let result: Vec<i32> = v
    .iter()
    .filter(|&&x| x % 2 == 0)
    .map(|&x| x * x)
    .collect();
```

A `v.iter()` `&i32` elemeket ad; a `filter` predikátuma `&&x`-et kap, mert a `filter` az elem referenciáját adja a closure-nek. A `map` a páros elemek négyzetét számítja, a `collect` pedig `Vec<i32>`-t épít. A típus annotálása a `result` változón egyértelműsíti a `collect` célgyűjteményét.

3. **Implementációs feladat megoldása.** A szűrt bejárás külön típus és külön metódus; a `Keys` és az `iter_keys` változatlan marad.

Kódblokk — fájl: `rkv-core/src/store/mod.rs` · művelet: beillesztés az `impl Store` blokk elé, illetve az `impl Store` blokkba · ellenőrzés: `cargo test -p rkv-core nonempty`

```
pub struct NonEmptyKeys<'a> {
    inner: hash_map::Iter<'a, String, Vec<u8>>,
}

impl<'a> Iterator for NonEmptyKeys<'a> {
    type Item = &'a str;

    fn next(&mut self) -> Option<Self::Item> {
        for (k, v) in self.inner.by_ref() {
            if !v.is_empty() {
                return Some(k.as_str());
            }
```

```
        }
        None
    }
}

// az impl Store blokkba:
    pub fn nonempty_keys(&self) -> NonEmptyKeys<'_> {
        NonEmptyKeys {
            inner: self.data.iter(),
        }
    }

    // a tests modulba:
    #[test]
    fn nonempty_keys_skips_empty_values_but_iter_keys_does_not() {
        let mut store = Store::new();
        store.set("full".to_string(), b"x".to_vec());
        store.set("empty".to_string(), Vec::new());
        let nonempty: Vec<_> = store.nonempty_keys().collect();
        assert_eq!(nonempty, vec!["full"]);
        assert_eq!(store.iter_keys().count(), 2);
    }
```

A `hash_map::Iter` a `HashMap` iterátora, amely `(k, v)` párokat ad. A `next` metódus addig lép (`by_ref()`: az iterátort kölcsönvéve járjuk be, hogy a következő híváskor onnan folytassa), amíg nem talál egy nem üres értéket; ha nincs több elem, `None`-t ad. Az élettartam-annotációk biztosítják, hogy a visszaadott `&str` referenciák a `self.data`-hoz kötődnek. (Ugyanez rövidebben: `self.data.iter().filter(|(_, v)| !v.is_empty()).map(|(k, _)| k.as_str())`, `impl Iterator` visszatéréssel; a saját típus azért tanulságos, mert az `Iterator` trait minden részletét kézben tartja.)

4. **Hibajavítás/refaktorálás.** A függvény működik. A `squares` kötés elhagyása stílusválasztás (a lokális kötés nem jár mért költséggel), a `&[i32]` paraméter viszont tényleges API-javítás. Idiomatikusabb változat:

```
fn sum_of_squares(v: &[i32]) -> i32 {
    v.iter().map(|x| x * x).sum()
}
```

A `sum` fogyasztó a `map` által visszaadott lusta iterátoron hívódik; a `sum` végighúzza a láncot, kiszámítja a négyzetek összegét. A `&Vec<i32>` paramétert `&[i32]`-re cseréltük, mert a slice általánosabb: `Vec`-re, tömbre és más slice-okra is működik (a clippy `ptr_arg` lintje is ezt javasolja). Ez az igazi javítás; a `squares` kötés összevonása csak olvashatósági kérdés. A `sum` a `x * x` értékeket `i32`-ként adja össze, mert az `Iterator` `Item`-je `i32` (a `map` closure `&i32`-et kap, de `x * x` már érték).

13.2.9 Rövid összefoglaló

Az `Iterator` trait a `next` metódusra épül, és az `Item` asszociált típussal adja meg az elem típusát. A `for` ciklus az `IntoIterator`-t használja, a `iter`, `iter_mut` és `into_iter` pedig a kölcsönzési és tulajdonjogi szemantikát különbözteti meg. Az adapterek (`map`, `filter`, `enumerate`, `zip`, `take`, `skip`, `flat_map`) lusták: csak akkor futnak le, ha egy fogyasztó (`collect`, `sum`, `fold`, `for_each`) végighúzza a láncot. A `collect::<Result<Vec<_>, _>>()` rövidre zár az első hibánál. A laborban az `rkv-core/src/store/mod.rs` fájlban létrehoztuk a `Keys<'a>` iterátort és a `Store::iter_keys()` metódust, amely a kulcsokat `&str` elemekként adja (a `Store::keys()` változatlanul minden kulcsot ad); a szűrt `NonEmptyKeys` külön típus. Ezt az iterátort a későbbi alfejezetek – a gyűjtemények (Vec, HashMap, BTreeMap és társaik; az Entry API), a párhuzamos

iterátorok (Adatpárhuzamosság rayonnal) és a teljesítményoptimalizálás (Teljesítmény: mérés, profilozás, allokáció, release-beállítások) – használják majd.

13.2.10 Források és ellenőrzési hiányok

A nyelvi szabályok a Rust Reference (https://doc.rust-lang.org/reference/) és a Rust Book (https://doc.rust-lang.org/book/) iterátorokról szóló részein alapulnak; az `Iterator` és a `FromIterator` pontos API-ja a standard könyvtár dokumentációjában (https://doc.rust-lang.org/std/iter/) található. A hibaüzenet-szövegek a rögzített toolchainre (Rust 1.99.1) vonatkoznak. A laborban a `Store` belső tárolója `HashMap<String, Vec<u8>>`; a Vec, HashMap, BTreeMap és társaik; az Entry API alfejezet áttér a rendezett `BTreeMap`-re.

14 Gyűjtemények

14.1 Vec, HashMap, BTreeMap és társaik; az Entry API

A C++ STL-ben a konténerválasztás sokszor rutinszerű: `std::vector` az általános dinamikus tömb, `std::map` a rendezett asszociatív tároló, `std::unordered_map` a hash-alapú. Rustban a `Vec`, a `BTreeMap` és a `HashMap` hasonló szerepet töltenek be, de az alapértelmezések eltérnek, és a Rust-idiómák más megoldásokat kínálnak a gyakori frissítési mintákra. Ez az alfejezet a legfontosabb std-gyűjteményeket, a választási szempontokat és az Entry API-t tárgyalja, majd az rkv mintaprojektet rendezett tárolóra állítja át.

14.1.1 Fogalmak és szabályok

Definíció. A Rust std könyvtárának gyűjteményei (collections) a C++ STL-hez hasonlóan adatszerkezeteket valósítanak meg, de a tulajdonlási és kölcsönzési szabályokkal összhangban. A főbb típusok:

- `Vec<T>`: folytonos, dinamikusan növekvő tömb; a `std::vector<T>` megfelelője.
- `VecDeque<T>`: két végén bővíthető sor (deque); a `std::deque<T>`-hez hasonló, de Rustban egyetlen gyűrűpuffer, nem blokkokból áll.
- `LinkedList<T>`: láncolt lista; a `std::list<T>` megfelelője, de ritkán használjuk.
- `HashMap<K, V>`: hash-tábla, rendezetlen; a `std::unordered_map<K, V>`-nek felel meg.
- `BTreeMap<K, V>`: B-fa alapú, kulcs szerint rendezett asszociatív tároló; a `std::map<K, V>` megfelelője.
- `HashSet<K>` és `BTreeSet<K>`: halmazok; a `std::unordered_set` és `std::set` megfelelői.

Szabály. A Rust gyűjtemények komplexitási garanciái hasonlóak az STL-hez, de mindegyik mögött előfeltétel áll:

- `Vec`: indexelés O(1); `push` amortizált O(1) (az újrafoglalás alkalmanként O(n)); `insert`/`remove` a közepén O(n), mert az utána álló elemeket mozgatni kell;
- `VecDeque`: `push`/`pop` mindkét végén amortizált O(1), indexelés O(1); a közepén beszúrás O(n);
- `HashMap`: átlagosan O(1) keresés, beszúrás és törlés, ha a kulcs `Hash + Eq` konzisztens és az eloszlás jó; legrosszabb esetben O(n). A kulcs `Hash`-ének és `Eq`-ének összhangban kell lennie (`k1 == k2` azonos hash);
- `BTreeMap`: a kulcs `Ord`, a beszúrás, keresés és törlés O(log n); a **tartománybejárás** (`range`) O(log n + k), ahol `k` a találatok száma (a kezdőpont megkeresése O(log n), utána k lépés); a teljes bejárás O(n);
- a `BTreeMap` iterációja kulcs szerint rendezett, a `HashMap` iterációja tetszőleges sorrendű.

Szabály. A `HashMap` alapértelmezett hasherje a Rustban a `SipHash` családba tartozik, amely véletlenszerű kulcsot használ, és DoS-támadások ellen véd. Ez biztonságosabb, de általában lassabb, mint a C++ `std::unordered_map` gyakori implementációi. Ha a teljesítmény kritikus, és a kulcsok nem támadhatók, a `BuildHasher` paraméterrel más hasher (például a `FxHashMap` a `rustc-hash` crate-ből) állítható be.

14.1.2 C++ és Rust összevetés

A legfontosabb különbség: a C++-ban a rendezett `std::map` a klasszikus asszociatív tároló, a Rustban a `HashMap` az alapértelmezett, a `BTreeMap` a rendezett alternatíva. Ezt tükrözi az alábbi táblázat:

C++ STL típus	Rust típus	Rendezett	Komplexitás
`std::vector<T>`	`Vec<T>`	nem	O(1) index; amortizált O(1) `push_back`
`std::deque<T>`	`VecDeque<T>`	nem	O(1) beszúrás elején és végén
`std::list<T>`	`LinkedList<T>`	nem	O(1) `push`/`pop` mindkét végén és `append`; tetszőleges pozíció elérése O(n); nincs O(1) index
`std::unordered_map<K,V>`	`HashMap<K,V>`	nem	átlagos O(1)
`std::map<K,V>`	`BTreeMap<K,V>`	igen, kulcs szerint	O(log n)
`std::unordered_set<K>`	`HashSet<K>`	nem	átlagos O(1)
`std::set<K>`	`BTreeSet<K>`	igen	O(log n)

A C++-ban a `std::map` rendezett, ezért a Rust fejlesztők gyakran tévesen a `HashMap`-et tekintik rendezettnek. Rustban a rendezetlen tároló a `HashMap`, a rendezett a `BTreeMap`.

A `Vec` és a `std::vector` működése nagyon hasonló, de a kapacitás-API-ban van egy lényeges különbség: a Rust `Vec::reserve(additional)` a *jelenlegi hossz fölé* kér további `additional` elemnyi helyet, míg a C++ `vector::reserve(new_cap)` a kívánt *teljes* kapacitást adja meg. A `capacity()`, `with_capacity()` és `shrink_to_fit()` hasonlóan viselkedik. A különbség a mozgatásban rejlik: ha a `Vec` újrafoglaláskor mozgatja az elemeket, a `Copy` típusok másolódnak, a nem `Copy` típusok destruktív mozgatást szenvednek el, ami után a régi elemek nem érvényesek. Ez a Rust mozgatási szemantikájának következménye.

A `LinkedList` ritkán használjuk, mert rossz a cache-lokalitása, és nem biztosít O(1) indexelést. Az O(1) beszúrás/törlés csak a két végén (és az `append`-nél) igaz; egy tetszőleges pozíció megtalálása O(n), mert a standard `LinkedList` stabil API-ja nem ad pozíciót megtartó kurzort (a C++ `std::list` iterátora ezt megteszi). A Rust közösség javaslata szerint a legtöbb esetben `Vec` vagy `VecDeque` jobb választás.

14.1.3 Az Entry API

Definíció. Az Entry API a `HashMap` és `BTreeMap` asszociatív tárolók frissítési mintáját egyszerűsíti. Az `entry(key)` metódus egy `Entry` enumot ad vissza, amelynek két variánsa: `Occupied` (a kulcs létezik) és `Vacant` (a kulcs nem létezik). Az `Entry` objektum lehetővé teszi, hogy egyetlen kereséssel beszúrjunk vagy módosítsunk egy értéket, elkerülve a dupla keresést és a kölcsönzési konfliktusokat.

Szabály. A `HashMap::entry` és `BTreeMap::entry` metódusok `&mut self`-et igényelnek, és `Entry` objektumot adnak vissza. Az `or_insert(default)` beszúrja a `default` értéket, ha a kulcs hiányzik, és visszaad egy `&mut V`-t; az `or_insert_with(f)` lustán hívja az `f` függvényt; az `and_modify(closure)` csak meglévő érték esetén futtatja a closure-t.

A C++-ban a tipikus frissítési minta:

```cpp
std::unordered_map<std::string, int> counters;
for (const auto& word : words) {
    auto it = counters.find(word);
    if (it == counters.end()) {
        counters.emplace(word, 1);
    } else {
        ++it->second;
    }
}
```

Rustban ugyanez az Entry API-val:

```rust
use std::collections::HashMap;

let mut counters: HashMap<String, i32> = HashMap::new();
for word in words {
    let count = counters.entry(word).or_insert(0);
    *count += 1;
}
```

A Rust verzió egyetlen keresést végez, és a kölcsönzés-ellenőrző számára is biztonságos, mert az `entry` egyetlen `&mut` kölcsönzést tart fenn a teljes művelet alatt.

14.1.4 Kölcsönzési szabály és gyűjteménykezelés

A `get_mut` metódus **mutable** referenciát ad vissza, amely a térkép kölcsönzését jelenti. Ha ugyanabban a hatókörben ezután a térképet újra módosítani próbáljuk, a borrow checker hibát ad:

```rust
let mut map = HashMap::new();
map.insert("alma", 1);
let value = map.get_mut("alma").unwrap();
map.insert("korte", 2);   // hiba: a map-et már mutable-ként kölcsönöztük
*value += 1;
```

Ez a kód szándékosan nem fordul le:

```
error[E0499]: cannot borrow `map` as mutable more than once at a time
```

A pontos hibaüzenet verziófüggő, de a hibakód stabil. A probléma az, hogy a `get_mut` visszaadott `&mut` referenciájának élettartama a `map` kölcsönzéséhez kötött, és az `insert` újabb `&mut`-ot igényel. Az Entry API ezt a problémát elegánsan megoldja, mert a teljes frissítési művelet egyetlen kölcsönzésen belül marad:

```rust
map.entry("korte").or_insert(2);
if let Some(value) = map.get_mut("alma") {
    *value += 1;
}
```

14.1.5 Labor: rkv BTreeMap-re váltása rendezett listázáshoz és tartomány-lekérdezéshez

Munkakönyvtár: `~/rust-labor/rkv/`

A korábbi fejezetekben az `rkv-core` crate `Store` típusa `HashMap<String, Vec<u8>>`-ot használt belső tárolóként. Most áttérünk `BTreeMap`-re, hogy a kulcsok bejárása rendezett legyen, és tartomány-lekérdezést is támogassunk. A kiinduló állapot az Closure-ök: Fn, FnMut, FnOnce és az Iterátorok: lusta adapterek és saját iterátor ellenőrzési pontja. A `Store` az `rkv-core/src/store/mod.rs`-ben él, a belső mező neve `data`; a `FileStore` (I/O és szerializáció: fájlok, pufferelés, serde) külön típus, a saját `HashMap`-jével, ezt nem érintjük.

A migráció teljes listája. A tárolótípus cseréje öt helyet érint, a többi kód változatlan:

Hely	Változás
`use` sorok	`HashMap`, `hash_map` helyett `BTreeMap`, `btree_map`
`Store::data` mező és `Store::new`	`BTreeMap<String, Vec<u8>>`, `BTreeMap::new()`
`Keys` és `NonEmptyKeys` belső mezője (Iterátorok: lusta adapterek és saját iterátor)	`btree_map::Keys`, illetve `btree_map::Iter`
`Store::keys`, `iter_keys`, `Storage::list` tesztek	kódváltozás nélkül rendezettek lesznek a `keys`/`iter_keys` tesztjei futnak tovább; új: sorrend, `range`, `range_prefix`

Az iterátortípusok *burkoló* structok (`Keys<'a>`, `NonEmptyKeys<'a>`), nem a `std` iterátorainak közvetlen visszaadásai. Ezért a `Store` publikus API-ja (`iter_keys() -> Keys<'_>`) a belső tároló cseréjekor nem változik, és a hívók kódja érintetlen marad: ez a burkolás értéke. (Ha a burkoló nélküli `impl Iterator` visszatérést választottuk volna, ugyanez a stabilitás érhető el, de a konkrét típus neve nem lenne kimondható.)

1. Cseréljük le az importokat és a mező típusát. A `Store` struct és a `new` így néz ki:

Kódblokk — fájl: `rkv-core/src/store/mod.rs` · művelet: a `use std::collections::{HashMap, hash_map};`-hoz hasonló sorok cseréje, a `Store` struct és a `Store::new` módosítása · ellenőrzés: `cargo build -p rkv-core`

```
use std::collections::{btree_map, BTreeMap};

// A korábbi `#[derive(Debug, Default)]` megmarad: a korábbi példák és szerződések erre épülnek.
#[derive(Debug, Default)]
pub struct Store {
    data: BTreeMap<String, Vec<u8>>,
    dirty: bool,
}

impl Store {
    pub fn new() -> Self {
        Self {
            data: BTreeMap::new(),
            dirty: false,
        }
    }
    // ...
}
```

2. A `Keys` és a `NonEmptyKeys` belső mezőjének típusa követi a tároló típusát. A `next` törzse változatlan:

Kódblokk — fájl: `rkv-core/src/store/mod.rs` · művelet: a két struct mezőtípusának cseréje · ellenőrzés: `cargo build -p rkv-core`

```
pub struct Keys<'a> {
    inner: btree_map::Keys<'a, String, Vec<u8>>,
}

pub struct NonEmptyKeys<'a> {
    inner: btree_map::Iter<'a, String, Vec<u8>>,
}
```

3. A `Store::keys()` (és az `iter_keys()`) ettől kezdve kulcs szerint rendezett sorrendben adja a kulcsokat, a `Store` saját API-ja nem igényel külön rendezést. (A `Storage::list` és a CLI `execute LIST` ága a Trait-ek és generikusok: sablonok és concepts helyett óta maga is `sort_by`-t hív, mert a `Storage` szerződés rendezett eredményt ír elő; a `BTreeMap`-pel ez fölöslegessé válik, de a fejezet nem törli: opcionális egyszerűsítés.) Ellenőrizzük egy teszttel:

Kódblokk — fájl: `rkv-core/src/store/mod.rs` · művelet: beillesztés a meglévő `mod tests` blokkba · ellenőrzés: `cargo test -p rkv-core store`

```
    #[test]
    fn keys_are_sorted_without_sorting() {
        let mut store = Store::new();
        for k in ["c", "a", "b"] {
            store.set(k.to_string(), vec![]);
        }
        assert_eq!(store.keys(), ["a", "b", "c"]);
        assert_eq!(store.iter_keys().collect::<Vec<_>>(), ["a", "b", "c"]);
    }
```

4. Adjunk hozzá egy `range` metódust a `Store`-hoz, amely a `BTreeMap::range`-re támaszkodik. A `BTreeMap::range` **pánikol**, ha a `start` nagyobb az `end`-nél, ezért a metódus ilyenkor üres bejárást ad:

Kódblokk — fájl: `rkv-core/src/store/mod.rs` · művelet: beillesztés az `impl Store` blokkba · ellenőrzés: `cargo test -p rkv-core range`

```
    /// A `start..end` (fél-nyílt) tartomány kulcs-érték párjai, rendezve.
    /// Ha `start > end`, a tartomány üres (a `BTreeMap::range` ilyenkor pánikolna).
    pub fn range<'a>(
        &'a self,
        start: &str,
        end: &str,
    ) -> impl Iterator<Item = (&'a str, &'a [u8])> + 'a {
        let range = (start <= end).then(|| self.data.range(start.to_owned()..end.to_owned()));
        range
            .into_iter()
            .flatten()
            .map(|(k, v)| (k.as_str(), v.as_slice()))
    }
```

A `range` a `start` és `end` kulcsok közötti tartományt adja vissza (a `start` zárt, az `end` nyitott intervallum). A `start.to_owned()..end.to_owned()` ideiglenes `Range<String>`, amelyet a `BTreeMap::range` azonnal feldolgoz.

5. A **prefix-lekérdezés** a rendezett kulcsok egy összefüggő sávja. A helyes megoldás nem próbál felső határt számolni a prefixből, hanem alulról zárt, felül nyitott tartománnyal indul a prefixnél, és addig olvas, amíg a kulcsok a prefixszel kezdődnek:

Kódblokk — fájl: `rkv-core/src/store/mod.rs` · művelet: beillesztés az `impl Store` blokkba · ellenőrzés: `cargo test -p rkv-core prefix`

```
    /// A `prefix`-szel kezdődő kulcsok, rendezve. Az üres prefix minden kulcsot ad.
    pub fn range_prefix<'a>(
        &'a self,
        prefix: &str,
    ) -> impl Iterator<Item = (&'a str, &'a [u8])> + 'a {
        let prefix = prefix.to_owned();
        self.data
            .range(prefix.clone()..) // alulról zárt, felül nyitott
            .take_while(move |(k, _)| k.starts_with(&prefix))
            .map(|(k, v)| (k.as_str(), v.as_slice()))
    }
```

Edition 2024 és a visszatérő `impl Trait`. A 2024-es editionben a visszatérési `impl Trait` alapértelmezésben **minden** hatókörbeli élettartamot befog: a `&'a self`-ét és az argumentumok elidált élettartamait is (itt `start`, `end`, `prefix`). A `+ 'a` ezért nem a befogást írja elő, hanem egy outlives-korlát (a visszaadott típusnak legalább `'a`-ig élnie kell); a befogás pontos szabályozásához a *precise capturing* szintaxis (`+ use<'a>`) való. Mivel a `range` és a `range_prefix` az argumentumokból tulajdonolt másolatot készít (`to_owned`), az iterátor nem hivatkozik rájuk, és a `use<'a>` ezt a szerződést ki is mondja. Mindkét függvényt együtt, 2024 editionnel ellenőriztük, `+ 'a` és `+ use<'a>` alakban is; lefordulnak, és olyan hívóval is használhatók, amelynek `start`/`end`/`prefix` argumentuma már megszűnt, mielőtt az iterátort használjuk:

```
let (r, p);
{
    let (s, e, pre) = (String::from("a"), String::from("b"), String::from("ap"));
    r = store.range(&s, &e);        // -> impl Iterator<...> + use<'a>
    p = store.range_prefix(&pre);   // -> impl Iterator<...> + use<'a>
} // s, e, pre itt megszűnik; r és p továbbra is használható
```

Miért nem jó a `prefix + '\u{10FFFF}'` mint felső határ? A `char::MAX` nem „a lexikografikus rendezés maximuma”: a `prefix\u{10FFFF}x` kulcs (a prefix, a legnagyobb kódpont, majd még egy karakter) *nagyobb*, mint a `prefix\u{10FFFF}`, de még a prefixszel kezdődik, ezért egy `prefix..=prefix\u{10FFFF}` tartomány kihagyná. A `take_while` megoldás nem feltételez ilyen felső korlátot. A matematikailag helyes lexikografikus utód („a prefix utolsó karakterének növelése”) bonyolultabb (a `\u{10FFFF}`-ra és a szurrogát-tartományra külön szabály kell), ezért a tankönyvi megoldás a `take_while`. (A naiv `prefix + 'z'` vagy `prefix + char::MAX` felső határ hibás; ebből azonban nem következik, hogy helyes lexikografikus utód ne volna számolható — megvalósítható, csak a kiválasztott egyszerűbb megoldás nem igényli.)

6. Tesztek: a prefix önmaga, szokásos folytatása, `prefix + U+10FFFF`, `prefix + U+10FFFF + még egy karakter`, a nem illeszkedő szomszédok és az üres prefix:

Kódblokk — fájl: `rkv-core/src/store/mod.rs` · művelet: beillesztés a meglévő `mod tests` blokkba · ellenőrzés: `cargo test -p rkv-core store` → minden teszt sikeres

```
    #[test]
    fn range_returns_ordered_subset() {
        let mut store = Store::new();
        store.set("a".into(), vec![1]);
        store.set("b".into(), vec![2]);
        store.set("c".into(), vec![3]);
```

```
    let got: Vec<_> = store.range("a", "c").map(|(k, v)| (k.to_string(),
    ↪ v.to_vec())).collect();
    assert_eq!(got, vec![("a".to_string(), vec![1]), ("b".to_string(), vec![2])]);
    assert_eq!(store.range("c", "a").count(), 0); // start > end: üres, nem pánik
}

#[test]
fn range_prefix_covers_the_whole_prefix_band() {
    let max = '\u{10FFFF}';
    let mut store = Store::new();
    for k in [
        "ap".to_string(),
        "app".to_string(),
        "app::1".to_string(),
        format!("app{max}"),
        format!("app{max}x"),
        "apq".to_string(),
        "b".to_string(),
    ] {
        store.set(k, vec![]);
    }
    let got: Vec<&str> = store.range_prefix("app").map(|(k, _)| k).collect();
    assert_eq!(
        got,
        vec!["app", "app::1", &format!("app{max}"), &format!("app{max}x")]
    );
    assert_eq!(store.range_prefix("").count(), 7); // üres prefix: minden kulcs
    assert_eq!(store.range_prefix("zzz").count(), 0);
}
```

7. Futtassuk a teszteket, és ellenőrizzük, hogy a teljes workspace fordul:

```
cargo test -p rkv-core
cargo build --workspace
```

Várható kimenet: minden teszt sikeres; az `rkv-cli` és az eddigi `Keys`-tesztek változtatás nélkül futnak.

Eredmény: Az `rkv-core` `Store` típusa mostantól `BTreeMap<String, Vec<u8>>`-ot használ, a kulcsbejárások rendezettek, és a `Store` rendelkezik `range` és `range_prefix` metódussal.

Ellenőrzési pont. A `Store` rendezett (`BTreeMap`) belső tárolóval, `range(start, end)` és `range_prefix(prefix)` metódussal; a `Keys` és a `NonEmptyKeys` publikus típusa változatlan. A `cargo test --workspace` az eddigi tesztek mellett az újakat is sikeresen futtatja.

Továbbvitt eredmény. Az `rkv-core` `Store` típusa ezen a ponton rendezett tárolót használ, és új `range`/`range_prefix` metódust kap. Ezt a rendezett tárolót teszi szálbiztossá a Mutex, RwLock, csatornák és a megosztott rkv alfejezet, a teljesítményét méri a Teljesítmény: mérés, profilozás, allokáció, release-beállítások alfejezet, és a Záróprojekt: az rkv teljes rendszerként is erre épít.

14.1.6 Tipikus hibák és átállási csapdák

1. **Klónozás a borrow checker "legyőzésére".** Amikor egy `get_mut` hívás után a térképet módosítani kell, a fejlesztő klónozza a kulcsot, hogy kikerülje a kölcsönzési konfliktust. Ez felesleges allokációhoz vezet. Az Entry API a helyes megoldás: `map.entry(key).and_modify(...).or_insert(...)`.

2. **`LinkedList` használata `std::list`-et keresve.** A C++ fejlesztők gyakran választanak `std::list`-et, mert a `std::vector` beszúrása a közepén O(n). Rustban a `Vec` ugyanez, de a legtöbb esetben a `Vec` vagy `VecDeque` jobb a cache-lokalitás miatt. A `LinkedList` ritkán indokolt.

3. **`HashMap` alapértelmezett hasher lassúságának figyelmen kívül hagyása.** A Rust `HashMap` alapértelmezett hasherje DoS-rezisztens, de lassabb, mint a C++ `std::unordered_map` gyakori implementációi. Ha a kulcsok nem támadhatók, érdemes más hashert használni.

4. **`unwrap` a gyűjtemény-hozzáféréseknél.** Az indexelés (`map[&key]`) panikolhat, ha a kulcs hiányzik. Használj `get`-et vagy az Entry API-t, amely explicit módon kezeli a hiányzó kulcsot.

14.1.7 Feladatok

1. **Lefordul-e és miért?** Az alábbi Rust kód lefordul-e? Ha nem, mi a hiba oka, és hogyan javítanád idiomatikusan?

```rust
let mut map = HashMap::new();
map.insert("alma", 1);
let value = map.get_mut("alma").unwrap();
map.insert("korte", 2);
*value += 1;
```

2. **C++-kód átírása idiomatikus Rustra.** Írd át az alábbi C++ kódot Rustra, használva az Entry API-t:

```cpp
std::map<std::string, int> counters;
for (const auto& word : words) {
    auto it = counters.find(word);
    if (it == counters.end()) {
        counters.emplace(word, 1);
    } else {
        ++it->second;
    }
}
```

3. **Implementációs feladat az rkv-ban.** Adj egy `last_key_with_prefix(&self, prefix: &str) -> Option<&str>` metódust az `rkv-core` `Store` típusához, amely a megadott prefixszel kezdődő *legnagyobb* kulcsot adja vissza. Építs a labor `range_prefix` metódusára, és írj tesztet a prefix nélküli találatra (`None`) és az üres prefixre is.

4. **Tervezési döntés.** Az rkv szerverének naplózásához időbélyeg szerint rendezett eseményeket kell tárolni, és gyakran kell tartomány-lekérdezést végezni; azonos időpontú esemény is előfordulhat, és egyet sem szabad elveszíteni. Milyen Rust gyűjteményt választanál (és milyen kulccsal), és mennyi a tartománylistázás költsége?

14.1.8 Megoldások

1. A kód nem fordul le, mert a `get_mut` `&mut` kölcsönzést ad a `map`-re, és ez a kölcsönzés a `value` változó élettartamáig tart. Az `insert` újabb `&mut`-ot igényel, ami ütközik. A hiba `E0499` (cannot borrow `map` as mutable more than once at a time). Javítás: szűkítsd a `value` hatókörét:

```
{
    let value = map.get_mut("alma").unwrap();
    *value += 1;
}
map.insert("korte", 2);
```

Vagy használd az Entry API-t:

```
map.entry("korte").or_insert(2);
if let Some(value) = map.get_mut("alma") {
    *value += 1;
}
```

2. Az idiomatikus Rust megoldás:

```
use std::collections::BTreeMap; // vagy HashMap

let mut counters: BTreeMap<String, i32> = BTreeMap::new();
for word in words {
    *counters.entry(word).or_insert(0) += 1;
}
```

3. A `last_key_with_prefix` metódus megvalósítása a labor `range_prefix`-ére építve:

```
pub fn last_key_with_prefix(&self, prefix: &str) -> Option<&str> {
    self.range_prefix(prefix).map(|(k, _)| k).last()
}
```

Teszt (a `mod tests` blokkba):

```
#[test]
fn last_key_with_prefix_finds_the_greatest() {
    let mut store = Store::new();
    for k in ["app", "app::1", "app::2", "b"] {
        store.set(k.to_string(), vec![]);
    }
    assert_eq!(store.last_key_with_prefix("app"), Some("app::2"));
    assert_eq!(store.last_key_with_prefix(""), Some("b")); // üres prefix: a legnagyobb kulcs
    assert_eq!(store.last_key_with_prefix("zzz"), None);
}
```

A `last()` végigjárja a találati sávot (O(log n + k)). Ha csak a legnagyobb kulcs kell, és a prefix felső határa számítható, a `BTreeMap::range(..bound).next_back()` O(log n)-ben is megoldja; a `take_while` megoldás előnye, hogy nem feltételez felső korlátot.

4. A `BTreeMap` a megfelelő választás, mert kulcs szerint rendezett, a tartománylistázás költsége **O(log n + k)** (`k` a találatok száma), az iteráció pedig rendezett. A kulcsra figyelni kell: egy `BTreeMap<u64, Esemény>` az azonos időbélyegű eseményeket felülírná (a második `insert` az elsőt elveszti). Két megoldás veszteség nélkül:

```
use std::collections::BTreeMap;
use std::ops::Bound::{Excluded, Included};

// 1. Összetett kulcs: (időbélyeg, sorszám); az azonos időpontú események sorszámmal
↪ különböznek.
let mut events: BTreeMap<(u64, u32), &str> = BTreeMap::new();
events.insert((100, 0), "a");
events.insert((100, 1), "b");
events.insert((101, 0), "c");
let hits: Vec<_> = events.range((Included((100, 0)), Excluded((101, 0)))).map(|(_, v)|
↪ *v).collect(); // ["a", "b"]
```

```rust
// 2. Időbélyeg -> Vec<Esemény>: egy időponthoz több esemény tartozik.
let mut grouped: BTreeMap<u64, Vec<&str>> = BTreeMap::new();
grouped.entry(100).or_default().push("a");
grouped.entry(100).or_default().push("b");
grouped.entry(101).or_default().push("c");
let n: usize = grouped.range(100..101).map(|(_, v)| v.len()).sum(); // 2
```

Ellenőrzött kimenet: a naiv `BTreeMap<u64, _>` két azonos időbélyegű beszúrás után 1 elemet tárol (az egyik esemény elveszett); az összetett kulcs mindkét eseményt megtartja (`["a", "b"]`), a csoportosított változat 2 eseményt számol a `100..101` tartományban. A `HashMap` nem garantál rendezettséget, ezért a naplóhoz tartománylekérdezéshez nem alkalmas.

14.1.9 Rövid összefoglaló

A Rust std gyűjteményei közül a `Vec<T>` a `std::vector`, a `VecDeque<T>` a `std::deque`, a `HashMap<K,V>` a `std::unordered_map`, a `BTreeMap<K,V>` az `std::map`, a halmazok pedig a `HashSet`/`BTreeSet` megfelelői. A legfontosabb különbség, hogy a Rust `HashMap` rendezetlen, a rendezett tároló a `BTreeMap`. Az Entry API (`entry`, `or_insert`, `or_insert_with`, `and_modify`) egyetlen kereséssel oldja meg a beszúrást és frissítést, elkerülve a dupla keresést és a kölcsönzési konfliktusokat. A laborban az `rkv-core` `Store` típusát `BTreeMap<String, Vec<u8>>`-ra állítottuk át, így a `List` parancs rendezett, és új `range` metódust adtunk hozzá. **Továbbvitt eredmény:** a rendezett `Store` és a `range` metódus alapja a későbbi szálbiztos megosztott rkv-nak, a teljesítménymérésnek és a záróprojektnek.

14.1.10 Források és ellenőrzési hiányok

A fejezet a Rust std dokumentációjára (doc.rust-lang.org/std) és a The Rust Programming Language könyv 8. fejezetére támaszkodik. A pontos API-k (pl. `BTreeMap::range`, `HashMap::entry`) a stabil Rust részei, de a szignatúrák ellenőrizhetők a docs.rs-en. A hibaüzenetek pontos szövege verziófüggő; a hibakódok stabilak. A `BTreeMap` belső implementációja B-fa, de a pontos invariánsok nem része a stabil szerződésnek. A `Vec` újrafoglalási stratégiája nem garantált, bár gyakorlatban duplázó. Az rkv labor feltételezi a korábbi fejezetekben kialakított `Store` struktúrát; ha az érték típusa korábban `String` volt, akkor a `Vec<u8>`-ra váltást is jelezni kell.

15 Tesztelés és dokumentáció

15.1 Unit-, integrációs és dokumentációs tesztek

15.1.1 Motiváció

A C++-os világban a tesztelés szinte mindig külső keretrendszer kérdése: GoogleTest, Catch2 vagy Boost.Test. A buildrendszerben külön célt kell létrehozni, a tesztfájlokat kézzel kell regisztrálni, a tesztelendő privát fehér dobozt gyakran `#define private public` trükkel vagy friend osztályokkal nyitjuk meg. A Rust ezt a kérdést a nyelvbe és a Cargo-ba építette: a `#[test]` attribútummal jelölt függvényeket a `cargo test` automatikusan futtatja, az egységtesztek a forráskód mellett, az integrációs tesztek a `tests/` könyvtárban, a dokumentációs példák pedig magukban a doc-kommentekben élnek.

A cél most az, hogy az rkv-core `Storage` absztrakcióját egység-, integrációs és dokumentációs tesztekkel fedjük le, és közben kialakítsunk egy generikus tesztkészletet, amelyet mind a `MemStore`, mind a `FileStore` implementációra futtathatunk.

15.1.2 Fogalmak és szabályok

Definíció. A *unit teszt* egyetlen modul, függvény vagy típus viselkedését ellenőrzi, és a forráskódban, általában egy `#[cfg(test)] mod tests` blokkban helyezkedik el. Az *integrációs teszt* a crate nyilvános API-ját a `tests/` könyvtárból, tehát külső fogyasztóként ellenőrzi. A *doc-teszt* pedig a `///` dokumentációs kommentben lévő, “‘‘`rust jelölésű kódblokk, amelyet` a`rustdoc‘ lefordít és futtat.

Szabály. A `#[test]` függvényeket a `cargo test` párhuzamosan, minden CPU-magon futtatja. A teszt akkor sikeres, ha a függvény visszatér, és akkor bukik, ha pánikol. Az `assert!`, `assert_eq!`, `assert_ne!` makrók a pánik mechanizmusára épülnek: hamis feltétel esetén `panic!`-t váltanak ki. A `#[should_panic(expected = "részlet")]` attribútum azt ellenőrzi, hogy a függvény pánikol-e, és a pániküzenet tartalmazza-e a megadott részt.

Szabály. A `#[cfg(test)]` modulban a tesztkód hozzáfér a szülőmodul privát elemeihez is, mert a modul a crate-en belül marad. Az integrációs tesztek ezzel szemben kizárólag a nyilvános API-t látják, mert a `tests/` könyvtár külön crate-ként fordul.

Szabály. A tesztfüggvény visszatérési típusa lehet `()` vagy `Result<(), E>`, ahol az `E` típusra a tesztkeret csak a `Debug` implementációt kéri (a `Termination` trait `Result<T, E>`-re `T: Termination, E: Debug` mellett él); az `E` nem kell, hogy a `std::error::Error` traitet is implementálja, így a `Result<(), String>` is működő teszt. Ekkor a teszt a `?` operátorral futtathat hibás műveleteket: ha `Err` keletkezik, a teszt elbukik, ha `Ok`, akkor sikeres.

15.1.3 C++ és Rust összevetés

C++ oldalon a GoogleTest keretrendszerrel:

```cpp
#include <gtest/gtest.h>
#include "mem_store.h"

TEST(StorageTest, SetGet) {
    MemStore store;
    store.set("k", std::vector<uint8_t>{'v'});
    EXPECT_EQ(store.get("k"), std::vector<uint8_t>{'v'});
}
```

Rustban ugyanez az `rkv-core` `Storage` traitjén keresztül. Fontos részlet: a `MemStore` (a `Store` aliasa) inherens `set` metódusa nem ad `Result`-ot, ezért a *trait* metódusát teljesen minősítve hívjuk (`Storage::set(&mut store, ..)`), vagy generikus kódból, ahol csak a trait látszik:

```rust
#[test]
fn memstore_set_get_roundtrip() {
    let mut store = MemStore::new();
    Storage::set(&mut store, "k".into(), b"v".to_vec()).unwrap();
    assert_eq!(Storage::get(&store, "k").unwrap(), Some(b"v".to_vec()));
}
```

A különbség nem szintaktikai csupán. A Rust teszt a source fájlban él, a `cargo test` minden regisztráció nélkül megtalálja, és a `#[test]` attribútum a függvényt a teszt-binárisba irányítja. A GoogleTest makrók egy külső könyvtárat és egy külön build targetet igényelnek. Ráadásul a Rust teszt a `Result`-ot visszaadva képes a `?` operátorral kezelni a hibákat, ami C++-ban külön `ASSERT_NO_THROW` vagy `std::expected` minta kérdése.

15.1.4 Kidolgozott labor: unit, integrációs és doc-tesztek az rkv-core-hoz

Munkakönyvtár: `~/rust-labor/rkv/`

A labor az rkv-core `Storage` trait megvalósításait teszteli három szinten. A kiinduló állapot a Vec, HashMap, BTreeMap és társaik; az Entry API ellenőrzési pontja: a `Storage` trait (Trait-ek és generikusok: sablonok és concepts helyett) kanonikus szerződése (`get`, `set(String, Vec<u8>)`, `delete -> bool`, `list(Option<&str>)`), a `MemStore` alias és a `FileStore::open(snapshot, log)` (I/O és szerializáció: fájlok, pufferelés, serde). A teszteléshez ezt a meglévő API-t használjuk, és egy új generikus tesztkészletet hozunk létre, amely mindkét implementációra futtatható. A `FileStore` tesztjei dev-függőségként a `tempfile` crate-et használják:

Kódblokk — fájl: `rkv-core/Cargo.toml` · művelet: új szakasz a fájl végére · ellenőrzés: `cargo build -p rkv-core --tests`

```toml
[dev-dependencies]
tempfile = "3"
```

15.1.4.1 Unit tesztek a `storage.rs`-ben

Először adjunk egy privát normalizáló függvényt a `storage.rs`-hez, és teszteljük azt a `#[cfg(test)] mod tests` blokkban. A privát függvény a kulcsot trimeli és kisbetűssé alakítja:

Kódblokk — fájl: `rkv-core/src/storage.rs` · művelet: beillesztés a `Response` típus elé; a `#[allow(dead_code)]` csak azért kell, mert a függvényt kizárólag a teszt hívja · ellenőrzés: `cargo build -p rkv-core`

```
#[allow(dead_code)]
fn normalize_key(key: &str) -> String {
    key.trim().to_lowercase()
}
```

A `storage.rs` meglévő `mod tests` blokkjába:

Kódblokk — fájl: `rkv-core/src/storage.rs` · művelet: beillesztés a meglévő `mod tests` blokkba · ellenőrzés: `cargo test -p rkv-core storage` → a 3 korábbi és a 3 új teszt sikeres

```
    #[test]
    fn normalize_trims_and_lowercases() {
        assert_eq!(normalize_key("  Hello  "), "hello");
    }

    #[test]
    fn memstore_lists_keys_sorted() {
        let mut store = MemStore::new();
        Storage::set(&mut store, "b".into(), b"2".to_vec()).unwrap();
        Storage::set(&mut store, "a".into(), b"1".to_vec()).unwrap();
        let pairs = Storage::list(&store, None).unwrap();
        assert_eq!(pairs.len(), 2);
        assert_eq!(pairs[0].0, "a");
        assert_eq!(pairs[0].1, b"1".to_vec());
    }

    #[test]
    #[should_panic(expected = "index out of bounds")]
    fn index_out_of_bounds_panics() {
        let v = [1];
        let i = v.len() + 1; // futásidejű index: nem fordítási idejű hiba
        let _ = v[i];
    }
```

A `normalize_key` privát, de a `tests` modul a `use super::*;` miatt látja. (Megjegyzés: a függvényt egyetlen teszt használja, a termék nem, ezért kell az `#[allow(dead_code)]`; a tároló korábbi kulcsai kis- és nagybetűt megkülönböztetők, ezért a `normalize_key` itt önálló szemléltető példa, nem az integrált kulcsszemantika része.) A `#[should_panic]` példa mesterséges, de jól mutatja a szintaxist: ha a pánik üzenete nem tartalmazná az `expected` részt, a teszt elbukna.

15.1.4.2 Integrációs tesztek a `tests/` könyvtárban

Hozzuk létre a `tests/common/mod.rs` fájlt. Ez a modul a generikus tesztkészletet tartalmazza, amely a `Storage` traiten keresztül bármely implementációra lefuttatható. Mivel itt csak a trait látszik, a `set` és a `delete` a trait (hibát jelző) változata:

Kódblokk — fájl: `rkv-core/tests/common/mod.rs` · művelet: új fájl · ellenőrzés: a következő két fájl után `cargo test -p rkv-core --tests`

```
use rkv_core::Storage;

pub fn assert_storage_contract<S: Storage>(mut store: S) {
    assert_eq!(store.get("missing").unwrap(), None);
    store.set("k".into(), b"v".to_vec()).unwrap();
    assert_eq!(store.get("k").unwrap(), Some(b"v".to_vec()));
    store.set("k".into(), b"v2".to_vec()).unwrap();
```

```
    assert_eq!(store.get("k").unwrap(), Some(b"v2".to_vec()));
    assert_eq!(store.list(None).unwrap().len(), 1);
    assert!(store.delete("k").unwrap()); // létezett
    assert!(!store.delete("k").unwrap()); // már nem létezik
    assert_eq!(store.get("k").unwrap(), None);
}
```

A `tests/memstore.rs` fájlban csak ennyi kell:

Kódblokk — fájl: `rkv-core/tests/memstore.rs` · művelet: új fájl · ellenőrzés: `cargo test -p rkv-core --test memstore`

```
mod common;

use rkv_core::MemStore;

#[test]
fn memstore_satisfies_storage_contract() {
    common::assert_storage_contract(MemStore::new());
}
```

A `tests/filestore.rs` `Result`-ot visszaadó teszt, és a `tempfile::tempdir()` által létrehozott, a hívásonként egyedi könyvtárat használja (a `TempDir` a `Drop`-ra törli a tartalmát):

Kódblokk — fájl: `rkv-core/tests/filestore.rs` · művelet: új fájl · ellenőrzés: `cargo test -p rkv-core --test filestore`

```
mod common;

use rkv_core::{Error, FileStore};

#[test]
fn filestore_satisfies_storage_contract() -> Result<(), Error> {
    let dir = tempfile::tempdir()?; // egyedi könyvtár, a Drop törli
    let store = FileStore::open(dir.path().join("state.json"), dir.path().join("append.log"))?;
    common::assert_storage_contract(store);
    Ok(())
}
```

A `FileStore::open` az I/O és szerializáció: fájlok, pufferelés, serde alfejezetben rögzített módon `Result<Self>`-t ad, a `?` tehát automatikusan konvertálja az `io::Error`-t a `thiserror` `#[from]` implementációján keresztül. Ha a könyvtár létrehozása vagy a megnyitás sikertelen, a teszt elbukik.

A futtatáshoz a workspace gyökeréből:

```
cargo test -p rkv-core
```

Várt kimenet: a `cargo test` **tesztcélonként** (unit tesztek, minden `tests/` fájl külön bináris, doc-tesztek) külön `running N tests` blokkot ír, saját darabszámmal; a számok a crate összes tesztjétől függenek. Az itt létrehozott két integrációs bináris blokkja:

```
     Running tests/filestore.rs (target/debug/deps/filestore-...)
running 1 test
test filestore_satisfies_storage_contract ... ok
test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out

     Running tests/memstore.rs (target/debug/deps/memstore-...)
running 1 test
test memstore_satisfies_storage_contract ... ok
test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

Egyetlen tesztcél külön futtatásához: `cargo test -p rkv-core --test memstore`, a doc-tesztekhez `cargo test -p rkv-core --doc`, a `storage.rs` unit teszteihez `cargo test -p rkv-core --lib storage`.

15.1.4.3 Doc-tesztek

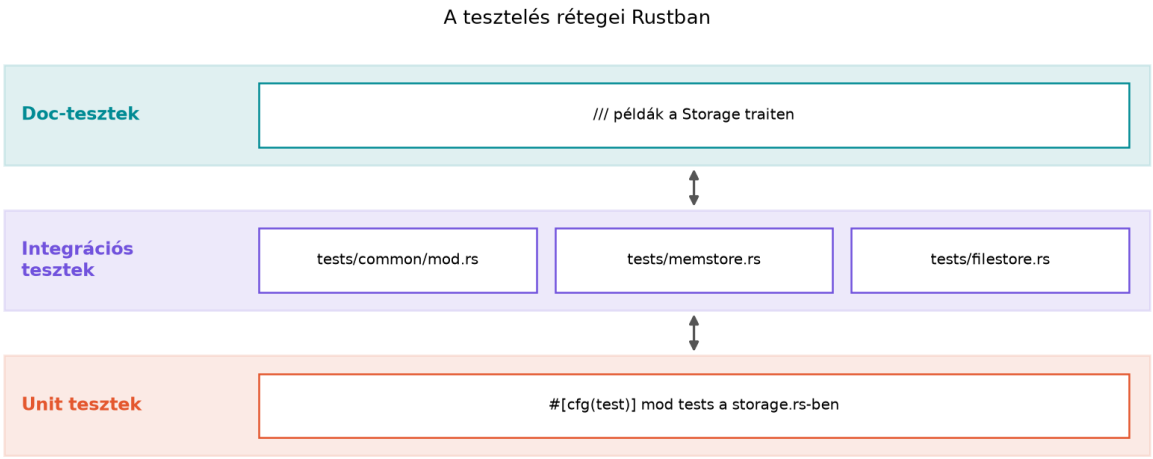
A `Storage` trait dokumentációjában tegyünk egy futtatható példát a `storage.rs` fájlban. A doc-teszt külön crate-ként fordul, ezért a publikus útvonalakat (`rkv_core::...`) használja, és a trait metódusait teljesen minősítve hívja:

Kódblokk — fájl: `rkv-core/src/storage.rs` · művelet: a `pub trait Storage` feletti doc-komment cseréje · ellenőrzés: `cargo test --doc -p rkv-core` → 1 sikeres doctest

```
/// Kulcs-érték tároló absztrakció.
///
/// # Példa
///
/// ```
/// use rkv_core::{MemStore, Storage};
///
/// let mut store = MemStore::new();
/// Storage::set(&mut store, "k".into(), b"v".to_vec()).unwrap();
/// assert_eq!(Storage::get(&store, "k").unwrap(), Some(b"v".to_vec()));
/// ```
pub trait Storage { /* a négy metódus változatlan (r024) */ }
```

A `cargo test --doc -p rkv-core` parancs a `rustdoc` segítségével lefordítja és futtatja ezt a kódblokkot. A doc-tesztben az `unwrap` elfogadható, mert a cél a példa tömörsége, nem a hibakezelés.

A tesztelési rétegeket a következő ábra foglalja össze:



15.1. ábra: A tesztelés rétegei Rustban - Az egységtesztek a forráskód mellett, az integrációs tesztek a tests/ könyvtárban, a doc-tesztek a dokumentációs kommentekben élnek.

A három szint elkülönül, de közös bennük, hogy a `cargo test` egyetlen paranccsal futtatja mindet, és a `Storage` trait az a szerződés, amelyet az integrációs generikus tesztkészlet egységesen ellenőriz.

Továbbvitt eredmény. A `tests/common/mod.rs` fájlban lévő `assert_storage_contract<S: Storage>` generikus tesztkészlet a későbbi alfejezetek alapja lesz: a Property-alapú tesztelés és benchmark erre épít property-teszteket, a Szálak, Send és Sync a megosztott `rkv` szálbiztosságát, a Záróprojekt pedig a teljes rendszer integrációs tesztjeit futtatja ezzel a szerződéssel.

15.1.5 Tipikus hibák és átállási csapdák

1. **Párhuzamos tesztek, közös állapot.** A `cargo test` alapértelmezésben párhuzamosít. Ha a tesztek ugyanarra a fájlra írnak, vagy környezeti változóra támaszkodnak, versenyszerű hibákat kaphatunk. Az integrációs `FileStore` tesztben ezért `tempfile::tempdir()`-rel minden hívás saját, egyedi könyvtárat kap (a folyamatazonosító önmagában nem elég: ugyanabban a folyamatban futó tesztek, vagy egy korábbi, megszakadt futás maradéka ugyanazt a nevet kapná). Ha több tesztnek kell közös erőforrás, használjunk `Mutex`-et, vagy inkább egyedi erőforrást tesztesetenként.
2. **`#[should_panic]` expected nélkül.** Ha nem adunk meg `expected` paramétert, a teszt bármilyen pánik esetén sikeres lesz, akár a tesztelt kód egy tőle független helyen is pánikolt. Mindig adjuk meg a pániküzenet egy egyedi részletét.
3. **Az integrációs teszt nem látja a privát elemeket.** Ha egy típust vagy függvényt csak integrációs teszttel akarunk ellenőrizni, akkor annak nyilvánosnak kell lennie. Ez nem hiba, hanem tervezési jel: ha a belső logikát nem tudjuk a publikus API-n át tesztelni, akkor vagy unit teszt kell a modulban, vagy az API nem elég teljes.
4. **Klónozás a borrow checker „legyőzésére”.** A tesztekben gyakori kísértés, hogy a kölcsönzési hibák elkerülésére mindent `.clone()`-ozzunk. A generikus tesztkészletben a `Storage` trait bound elegendő, nincs szükség klónozásra; a `mut store` paraméter és a `&self`/`&mut self` metódusok a borrow checkerrel biztonságosak.

15.1.6 Feladatok

1. **Lefordul-e és miért?** Alapállapot: a labor `storage.rs` fájljában a `normalize_key` **privát** (`fn normalize_key`, nincs `pub`), a `storage` modul pedig nyilvános (`pub mod storage;`). Az alábbi teszt a `tests/` könyvtárban van, és ezt a függvényt próbálja hívni. Lefordul-e? Indokold válaszod.

```
#[test]
fn test_normalize() {
    assert_eq!(rkv_core::storage::normalize_key("A"), "a");
}
```

2. **C++-kód átírása idiomatikus Rustra.** Az alábbi GoogleTest stílusú tesztet írd át Rust `#[test]` függvénnyé, és használd a `?` operátort a hibakezelésre. (A Rust `Storage::delete` `Result<bool>`-t ad: a bool azt jelzi, létezett-e a kulcs.)

```
TEST(StorageTest, DeleteMissingKey) {
    MemStore store;
    EXPECT_NO_THROW(store.delete("missing"));
    EXPECT_EQ(store.get("missing"), std::nullopt);
}
```

3. **Implementációs feladat az rkv-ban.** Bővítsd a `tests/common/mod.rs` fájlban lévő `assert_storage_contract` függvényt egy prefix-ellenőrzéssel: állítsd be a `one` és `two` kulcsokat, majd ellenőrizd, hogy a `list(None)` mindkettőt visszaadja, a `list(Some("t"))` csak a `two`-t, a `list(Some("x"))` pedig semmit.

4. **Hibajavítás, refaktorálás.** Egy korábbi változat az ideiglenes fájl nevét a folyamatazonosítóból képezte (`rkv-test-<pid>`). Ha ugyanabban a folyamatban két teszt is létrehoz ilyet, ütköznek, és egy korábbi, megszakadt futás maradéka is ott maradhat. Írj egy `TempDir` segédtípust, amely tényleg egyedi könyvtárat készít (nem keveri más teszt vagy korábbi futás állapotával), és `Drop`-ban törli. Írd le, mikor *nem* törlődik: a `Drop` best effort, `panic = "abort"` profilnál, `std::process::abort()`-nál vagy a folyamat kilövésekor nem fut le.

15.1.7 Megoldások

1. **Nem fordul le.** A `normalize_key` privát (nincs `pub`), és a `tests/` könyvtár külön crate-ként csak a `pub` elemeket látja. A fordító `E0603` hibát ad („function `normalize_key` is private”; a függvény privát, a modul nyilvános). Privát elem a modul `#[cfg(test)] mod tests` blokkjában tesztelhető; ha a nyilvános API-n át nem érhető el, az vagy unit tesztet kíván, vagy azt jelzi, hogy az API nem elég teljes. (A `pub(crate)` sem segít: az is csak a crate-en belül látható, az integrációs teszt viszont külső crate.)

2. Rust megoldás:

```rust
use rkv_core::Storage;

#[test]
fn delete_missing_key() -> Result<(), rkv_core::Error> {
    let mut store = rkv_core::MemStore::new();
    assert!(!Storage::delete(&mut store, "missing")?); // nem létezett: false, nem hiba
    assert_eq!(Storage::get(&store, "missing")?, None);
    Ok(())
}
```

A `delete` és a `get` is `Result`-ot ad, a `?` továbbítja a hibát, a teszt pedig `Result`-tal jelzi a keretrendszernek. A trait metódusait teljesen minősítve hívjuk, mert a `MemStore` inherens, nem `Result`-ot adó `delete`-je egyébként elnyomná őket.

3. Bővített generikus tesztkészlet:

```rust
pub fn assert_storage_contract<S: Storage>(mut store: S) {
    assert_eq!(store.get("missing").unwrap(), None);
    store.set("k".into(), b"v".to_vec()).unwrap();
    assert_eq!(store.get("k").unwrap(), Some(b"v".to_vec()));
    store.set("k".into(), b"v2".to_vec()).unwrap();
    assert_eq!(store.get("k").unwrap(), Some(b"v2".to_vec()));
    assert_eq!(store.list(None).unwrap().len(), 1);
    assert!(store.delete("k").unwrap());
    assert!(!store.delete("k").unwrap());
    assert_eq!(store.get("k").unwrap(), None);

    store.set("one".into(), b"1".to_vec()).unwrap();
    store.set("two".into(), b"2".to_vec()).unwrap();
    assert_eq!(store.list(None).unwrap().len(), 2);
    let prefixed = store.list(Some("t")).unwrap();
    assert_eq!(prefixed.len(), 1);
    assert_eq!(prefixed[0].0, "two");
    assert!(store.list(Some("x")).unwrap().is_empty());
}
```

4. Egyedi könyvtár `Drop`-os törléssel. A kulcs a `create_dir`: ez *atomikusan* hibát ad, ha a könyvtár már létezik, ezért sosem kerülünk idegen előállapotú könyvtárba; ütközéskor a számláló következő értékét próbáljuk.

```rust
use std::path::{Path, PathBuf};
use std::sync::atomic::{AtomicUsize, Ordering};

static NEXT: AtomicUsize = AtomicUsize::new(0);

struct TempDir {
    path: PathBuf,
}

impl TempDir {
    fn new(prefix: &str) -> Self {
        loop {
            let n = NEXT.fetch_add(1, Ordering::Relaxed);
            let path = std::env::temp_dir()
                .join(format!("{prefix}-{}-{n}", std::process::id()));
            match std::fs::create_dir(&path) {
                Ok(()) => return Self { path },
                // Már létezik (korábbi futás maradéka): a következő szám.
                Err(e) if e.kind() == std::io::ErrorKind::AlreadyExists => continue,
                Err(e) => panic!("nem hozható létre ideiglenes könyvtár: {e}"),
            }
        }
    }

    fn path(&self) -> &Path {
        &self.path
    }
}

impl Drop for TempDir {
    fn drop(&mut self) {
        let _ = std::fs::remove_dir_all(&self.path); // best effort
    }
}

#[test]
fn filestore_satisfies_storage_contract() -> Result<(), Error> {
    let dir = TempDir::new("rkv-test");
    let store = FileStore::open(dir.path().join("state.json"), dir.path().join("append.log"))?;
    common::assert_storage_contract(store);
    Ok(())
}

#[test]
fn two_temp_dirs_in_one_process_are_distinct() {
    let (a, b) = (TempDir::new("rkv-test"), TempDir::new("rkv-test"));
    assert_ne!(a.path(), b.path());
}
```

A `TempDir::drop` a teszt végén, pánik esetén (a verem visszacsévélésekor) is lefut, tehát a könyvtár ilyenkor is törlődik. Ez *best effort*: a `remove_dir_all` hibáját elnyeljük, és a `Drop` egyáltalán nem fut `panic = "abort"` profilnál, `std::process::abort()`-nál, a folyamat kilövésekor (`SIGKILL`) vagy áramkimaradáskor. Ezért az egyediség a `create_dir` atomikus hibáján múlik, nem a törlésen. A gyakorlatban a `tempfile::TempDir` ugyanezt a mintát (véletlen név, atomikus létrehozás, `Drop`-os törlés) adja, ezért a labor integrációs tesztje azt használja.

15.1.8 Rövid összefoglaló

A Rust tesztelési eszközei beépítettek: a `#[test]` attribútum a forráskódban, a `tests/` könyvtár az integrációs tesztek, a `///` dokumentációs példák pedig a doc-tesztek színhelye. A `cargo test`

egy paranccsal futtatja mindhárom szintet, párhuzamosan. A unit tesztek a `#[cfg(test)] mod tests` blokkon keresztül érik el a privát elemeket, az integrációs tesztek viszont kizárólag a publikus API-t látják. A tesztek `assert!`, `assert_eq!` makrókat használnak, pánik esetén buknak, `#[should_panic(expected = "...")]` attribútummal a pánikot ellenőrizhetjük, `Result` visszatéréssel pedig a `?` operátort használhatjuk. A laborban létrehoztuk a `tests/common/mod.rs` generikus tesztkészletet `assert_storage_contract<S: Storage>` néven; ezt a `MemStore` és `FileStore` integrációs tesztek egyaránt használják, a `FileStore` teszt pedig a `tempfile` által adott, hívásonként egyedi könyvtárat. **Továbbvitt eredmény:** a `tests/common/mod.rs` generikus tesztkészlet a Property-alapú tesztelés, a Szálak, Send és Sync és a Záróprojekt alfejezetek alapja. A `Storage` trait doc-tesztje a `cargo test --doc` paranccsal futtatható.

15.1.9 Források és ellenőrzési hiányok

- The Rust Programming Language, 11. fejezet „Writing Automated Tests”: https://doc.rust-lang.org/book/ch11-00-testing.html
- The Rustdoc Book, „Documentation tests”: https://doc.rust-lang.org/rustdoc/write-documentation/documentation-tests.html
- A Cargo Book „Cargo Targets / Tests” szakasza (`tests/` könyvtár, integrációs tesztek): https://doc.rust-lang.org/cargo/reference/cargo-targets.html#tests
- `tempfile` (3.x): https://docs.rs/tempfile/3

A kódot és a kimeneteket a könyv rögzített toolchainjével (Rust 1.99.1) ellenőriztük. A `libtest` kimenetének pontos formátuma és a párhuzamos futtatás részletei toolchain-verziótól függenek.

15.2 Property-alapú tesztelés és benchmark

15.2.1 Motiváció

C++-ban a tesztelés gyakran kézzel kiválasztott példákkal történik, legfeljebb néhány véletlen értékkel. A property-alapú tesztelés – vagyis olyan tulajdonságok megfogalmazása, amelyek minden lehetséges bemenetre igazak – csak külső könyvtárakkal érhető el. A mikrobenchmarkoknál hasonló a helyzet: a fejlesztő `std::chrono`-val mér, de a fordító optimalizálása és a mérési zaj miatt a kapott szám gyakran félrevezető. A Rust ökoszisztéma ezt a két problémát a `proptest` és a `criterion` crate-ekkel oldja meg: a property-teszt generátorokkal és zsugorítással találja meg a hibás bemeneteket, a criterion pedig statisztikailag megalapozott mérést ad.

15.2.2 Fogalmak és szabályok

Definíció. A *property-alapú tesztelés* (property-based testing) során nem konkrét bemenetekre írunk tesztet, hanem egy *invariánst* fogalmazunk meg, amelynek a bemenetek egy meghatározott tartományán teljesülnie kell. A keretrendszer ebből a tartományból véletlen mintákat generál (alapértelmezés szerint tesztenként 256-ot), ellenőrzi az invariánst, és ha talál egy ellenpéldát, *zsugorítással* (shrinking) a generátor stratégiáján belül egyszerűbb ellenpéldát keres. Ez erős hibakereső eszköz, de nem bizonyítás: a minta mérete és eloszlása korlátozza, mit talál meg.

Szabály. A `proptest` a `proptest!` makróval definiált tesztfüggvényeket futtatja. A generátorokat a `Strategy` trait írja le; a leggyakoribb beépített stratégia az `any::<T>()`. Ha egy teszt elbukik, a `proptest` automatikusan menti a hibás bemenetet egy regressziós fájlba (alapértelmezés szerint a `proptest-regressions` könyvtárba), hogy a hiba reprodukálható legyen.

Definíció. A *benchmark* egy kód futási idejének ismételt, statisztikailag értelmezhető mérése. A `criterion` (0.8) a futási időre `time: [alsó becslés felső]` sort ír: ez a 95%-os konfidenciaintervallum alsó és felső határa és a pontbecslés (a minták átlagos futási idejére, bootstrap alapján). A 95% ismételt mintavételi lefedettséget jelent: ha a mérést sokszor megismételnénk, az így számolt intervallumok kb. 95%-a tartalmazná a valódi értéket; nem azt, hogy egy adott intervallumban 95% valószínűséggel van az érték. A riport emellett jelzi a kilógó mérések számát (`Found N outliers`) és az előző futáshoz képesti változást (`change:`).

Szabály. A benchmarkban a mért kód bemenetét és kimenetét `black_box`-ba kell csomagolni, különben a fordító kioptimalizálhatja a mért számítást, így a mérés hamis eredményt ad. A `std::hint::black_box` az 1.66 óta stabil; a `criterion::black_box` a `criterion` 0.8-ban elavult (a fordító `use of deprecated function criterion::black_box` figyelmeztetést ad), ezért a `std::hint::black_box`-ot használjuk. A `cargo bench` alapértelmezés szerint release módban fut, ami a valós produkciós teljesítményt közelíti.

15.2.3 C++ és Rust összevetés

C++-ban a standard könyvtár nem tartalmaz property-alapú tesztelést; a leggyakoribb kézi megoldás csak néhány beépített értéket használ, és nem ad automatikus zsugorítást. Harmadik féltől származó keretrendszerek (például a rapidcheck vagy a Google FuzzTest) viszont kínálnak generálást és zsugorítást is, tehát a hiány nem nyelvi, hanem az alapkönyvtár szintjén jelentkezik:

```cpp
#include <cassert>
#include <random>
#include <unordered_map>
#include <string>

int main() {
    std::mt19937 rng(42);
    std::uniform_int_distribution<int> dist(0, 9);
    std::unordered_map<std::string, std::string> map;
    for (int i = 0; i < 100; ++i) {
        auto key = "k" + std::to_string(dist(rng));
        auto value = "v" + std::to_string(dist(rng));
        map[key] = value;
        assert(map[key] == value);
    }
    return 0;
}
```

Ez a teszt ugyanazt a beszúrás–visszaolvasás tulajdonságot ellenőrzi néhány kézzel választott értékre (egy invariáns példa-alapú vizsgálata); a különbség nem az, hogy a C++ teszt nem fogalmaz meg tulajdonságot, hanem hogy a *beépített* eszköztárban nincs automatikus generálás, zsugorítás és regressziókezelés – ezt C++-ban külső könyvtárral (rapidcheck, FuzzTest) lehet pótolni –, így a kézi változat hibája esetén nem segít megtalálni a minimális ellenpéldát. A Rust `proptest` ezzel szemben deklaratívan írja le az invariánst, és automatikusan zsugorít:

```rust
use proptest::prelude::*;
use rkv_core::{MemStore, Storage};

proptest! {
    #[test]
    fn set_then_get_returns_value(
        key in any::<String>(),
        value in prop::collection::vec(any::<u8>(), 0..64),
    ) {
```

```
        let mut store = MemStore::new();
        Storage::set(&mut store, key.clone(), value.clone()).unwrap();
        prop_assert_eq!(Storage::get(&store, &key).unwrap(), Some(value));
    }
}
```

A `prop_assert_eq!` makró ugyanazt ellenőrzi, mint az `assert_eq!`, de a `proptest` képes a generált bemeneteket és a hibaüzenetet a teszt kontextusába illeszteni. A kulcsot az `any::<String>()` stratégia generálja, az értéket (a tároló `Vec<u8>` értéktípusához igazodva) bájtvektor-stratégia; a stratégia hiba esetén mindkettőt zsugorítja. A `Storage::set(&mut store, ..)` teljesen minősített hívás azért kell, mert a `MemStore` inherens `set`-je nem ad `Result`-ot, és elnyomná a trait metódusát.

A benchmark terén is hasonló a különbség. C++-ban a standard könyvtárban csak a `std::chrono` áll rendelkezésre, a kézi mérés:

```
#include <chrono>
#include <unordered_map>
#include <string>

int main() {
    using Clock = std::chrono::high_resolution_clock;
    auto start = Clock::now();
    std::unordered_map<std::string, std::string> map;
    for (int i = 0; i < 1000; ++i) {
        map[std::to_string(i)] = "value";
    }
    auto end = Clock::now();
    auto ns = std::chrono::duration_cast<std::chrono::nanoseconds>(end - start).count();
    // Probléma: a fordító optimalizálhat, a mérés túl rövid, a cache-hatás torzít.
}
```

A Rust `criterion` ezt megoldja a statisztikai elemzéssel és a `black_box`-szal:

```
use std::hint::black_box;

use criterion::{criterion_group, criterion_main, Criterion};
use rkv_core::{MemStore, Storage};

fn bench_get(c: &mut Criterion) {
    let mut store = MemStore::new();
    Storage::set(&mut store, "key".to_string(), b"value".to_vec()).unwrap();
    c.bench_function("MemStore::get (hit)", |b| {
        b.iter(|| black_box(Storage::get(&store, black_box("key")).unwrap()));
    });
}

criterion_group!(benches, bench_get);
criterion_main!(benches);
```

A `black_box` (a `std::hint::black_box`, stabil az 1.66 óta) megakadályozza, hogy a fordító kioptimalizálja a `get` hívását vagy az eredmény felhasználását. A C++ oldalon a Google Benchmark nyújt hasonló statisztikai elemzést és `DoNotOptimize`-ot, de ez külső könyvtár, nem része a nyelvi szerződésnek vagy a standardnak.

15.2.4 Kidolgozott labor: proptest és criterion az rkv-ban

A laborban az rkv workspace `rkv-core` crate-jéhez adunk property-tesztet és benchmarkot. A kiinduló állapot a Unit-, integrációs és dokumentációs tesztek ellenőrzési pontja; az API

a kanonikus `Storage` trait (Trait-ek és generikusok: sablonok és concepts helyett): az érték `Vec<u8>`, a `delete` `Result<bool>`, a `list` rendezett `(String, Vec<u8>)` párokat ad. A tesztek a trait metódusait teljesen minősítve hívják (`Storage::set(&mut store, ..)`), mert a `MemStore` inherens metódusai egyébként elnyomnák őket.

15.2.4.1 1. lépés: függőségek

Kódblokk — fájl: `rkv-core/Cargo.toml` · művelet: beillesztés a `[dev-dependencies]` szakaszba, új `[[bench]]` szakasz a fájl végére · ellenőrzés: `cargo build -p rkv-core --tests --benches`

```
proptest = "1"
criterion = "0.8"

[[bench]]
name = "benchmark"
harness = false
```

A `harness = false` azt jelenti, hogy a benchmarkot nem a beépített tesztkeret futtatja, hanem a `criterion` saját `criterion_main!` belépési pontja. (A `[dev-dependencies]` szakasz már létezik az Unit-, integrációs és dokumentációs tesztek alfejezetből; ide kerül a két új sor.)

15.2.4.2 2. lépés: property-teszt

Hozzuk létre az `rkv-core/tests/storage_proptest.rs` fájlt:

Kódblokk — fájl: `rkv-core/tests/storage_proptest.rs` · művelet: új fájl (a 4. lépés kiegészíti) · ellenőrzés: `cargo test -p rkv-core --test storage_proptest`

```
use std::collections::BTreeMap;

use proptest::prelude::*;
use proptest::test_runner::TestRunner;
use rkv_core::{MemStore, Result, Storage};

fn value_strategy() -> impl Strategy<Value = Vec<u8>> {
    // Bináris értékek: tetszőleges bájtok, az üres is.
    prop::collection::vec(any::<u8>(), 0..64)
}

proptest! {
    #[test]
    fn set_then_get_returns_value(key in any::<String>(), value in value_strategy()) {
        let mut store = MemStore::new();
        Storage::set(&mut store, key.clone(), value.clone()).unwrap();
        prop_assert_eq!(Storage::get(&store, &key).unwrap(), Some(value));
    }

    #[test]
    fn delete_after_set_removes_the_key(key in any::<String>(), value in value_strategy()) {
        let mut store = MemStore::new();
        Storage::set(&mut store, key.clone(), value).unwrap();
        prop_assert!(Storage::delete(&mut store, &key).unwrap()); // létezett
        prop_assert_eq!(Storage::get(&store, &key).unwrap(), None);
        prop_assert!(!Storage::delete(&mut store, &key).unwrap()); // már nem
    }
}
```

A `set_then_get_returns_value` invariáns a beillesztés és a visszaolvasás konzisztenciáját fejezi ki; a `delete_after_set_removes_the_key` a törlés helyességét ellenőrzi, a `Result<bool>` mindkét értékével (létezett / már nem).

Futtassuk a tesztet:

```
cd ~/rust-labor/rkv
cargo test -p rkv-core --test storage_proptest
```

Várható kimenet (rövidítve):

```
running 2 tests
test set_then_get_returns_value ... ok
test delete_after_set_removes_the_key ... ok

test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

A `proptest` alapértelmezés szerint tesztenként 256 véletlen esetet próbál ki. A generált minta *széles* (az `any::<String>()` sokféle hosszú és összetételű, többbájtos Unicode szöveget állít elő, az értékek 0 és 63 bájt közötti tetszőleges bájtsorozatok), de ez mintavétel, nem teljes lefedés és nem bizonyítás: a 256 eset a bemeneti tér elenyésző része, és csak az invariánsok megsértését keresi.

Többbájtos Unicode kulcsokra külön is érdemes tesztet írni (a `\PC` reguláris stratégia nem vezérlő karaktereket generál), és néhány kritikus esetet rögzített példával is ellenőrzünk:

Kódblokk — fájl: `rkv-core/tests/unicode_keys.rs` · művelet: új fájl · ellenőrzés: `cargo test -p rkv-core --test unicode_keys` → 2 sikeres teszt

```
use proptest::prelude::*;
use rkv_core::{MemStore, Storage};

proptest! {
    /// Többbájtos Unicode kulcsok is: a `\PC` a nem vezérlő karaktereket generálja.
    #[test]
    fn multibyte_keys_round_trip(key in "\\PC{1,8}", value in prop::collection::vec(any::<u8>(),
    ↪  0..16)) {
        let mut store = MemStore::new();
        Storage::set(&mut store, key.clone(), value.clone()).unwrap();
        prop_assert_eq!(Storage::get(&store, &key).unwrap(), Some(value));
        prop_assert!(Storage::list(&store, Some(&key)).unwrap().iter().any(|(k, _)| *k == key));
    }
}

#[test]
fn fixed_multibyte_keys() {
    // Előre komponált és kombinált ékezet, CJK és emoji (4 bájtos): mind más kulcs.
    let keys = ["árvíztűrő", "  ", " ", "\u{e9}", "e\u{301}"];
    let mut store = MemStore::new();
    for (i, key) in keys.iter().enumerate() {
        Storage::set(&mut store, (*key).to_string(), vec![i as u8]).unwrap();
    }
    assert_eq!(Storage::list(&store, None).unwrap().len(), keys.len());
    for (i, key) in keys.iter().enumerate() {
        assert_eq!(Storage::get(&store, key).unwrap(), Some(vec![i as u8]));
    }
}
```

15.2.4.3 3. lépés: a zsugorítás bemutatása szándékos hibával

A zsugorítás szemléltetéséhez egy ideiglenes tárolótípus kell, amely **típusszinten teljesíti ugyanazt a `Storage` traitet**, és csak a tanítani kívánt viselkedésben tér el: a `get` a 10 bájtnál hosszabb kulcsra mindig `None`-t ad. Ezt a tesztfájlba tesszük, nem az rkv kódjába. A teszt lefordul, és **futás közben** bukik meg ellenpéldával; hogy a csomag tesztjei zöldek maradjanak, `#[ignore]`-ral jelöljük, és külön kérésre futtatjuk.

Kódblokk — fájl: `rkv-core/tests/buggy.rs` · művelet: új fájl · **szándékosan hibás viselkedés** (futásidejű bukás) · ellenőrzés: `cargo test -p rkv-core --test buggy -- --ignored` → 1 bukott teszt, zsugorított ellenpéldával

```rust
use std::collections::HashMap;

use proptest::prelude::*;
use rkv_core::{Result, Storage};

/// Szándékosan hibás tároló: csak a `get` hibás (11+ bájtos kulcsra mindig `None`).
struct BuggyStore {
    map: HashMap<String, Vec<u8>>,
}

impl Storage for BuggyStore {
    fn get(&self, key: &str) -> Result<Option<Vec<u8>>> {
        if key.len() > 10 {
            Ok(None) // szándékos hiba
        } else {
            Ok(self.map.get(key).cloned())
        }
    }
    fn set(&mut self, key: String, value: Vec<u8>) -> Result<()> {
        self.map.insert(key, value);
        Ok(())
    }
    fn delete(&mut self, key: &str) -> Result<bool> {
        Ok(self.map.remove(key).is_some())
    }
    fn list(&self, prefix: Option<&str>) -> Result<Vec<(String, Vec<u8>)>> {
        let mut items: Vec<_> = self
            .map
            .iter()
            .filter(|(k, _)| prefix.is_none_or(|p| k.starts_with(p)))
            .map(|(k, v)| (k.clone(), v.clone()))
            .collect();
        items.sort();
        Ok(items)
    }
}

proptest! {
    #[test]
    #[ignore = "szándékosan hibás: futás közben ellenpéldával bukik"]
    fn buggy_get_fails_for_long_keys(
        key in any::<String>(),
        value in prop::collection::vec(any::<u8>(), 0..16),
    ) {
        let mut store = BuggyStore { map: HashMap::new() };
        store.set(key.clone(), value.clone()).unwrap();
        prop_assert_eq!(store.get(&key).unwrap(), Some(value));
    }
}
```

A futtatás parancsa:

```
cargo test -p rkv-core --test buggy -- --ignored
```

A hibás küszöb **bájtban** értendő: a `key.len() > 10` miatt minden legalább 11 bájtos kulcs hibás. A `proptest` talál egy ilyen kulcsot, majd a zsugorító a generátor stratégiáján belül egyszerűbb ellenpéldát keres, amíg a hiba még előáll. A zsugorítás nem garantál globális minimumot, és a talált ellenpélda nem szükségszerűen 11 ASCII karakter (a többbájtos karakterek kevesebb karakterből is 11 bájtot adnak). A kimenet egy futás eredménye (Rust 1.99.1, `proptest` 1.12):

```
thread 'buggy_get_fails_for_long_keys' panicked at rkv-core/tests/buggy.rs:38:1:
Test failed: assertion failed: `(left == right)` ...
minimal failing input: key = " 0  A\u{c46}", value = []
test result: FAILED. 0 passed; 1 failed; ...
```

A zsugorított kulcs konkrét karakterei futásonként eltérhetnek (itt többbájtos karakterek szerepelnek, mert a feltétel a *bájthosszra* vonatkozik); a lényeg, hogy az ellenpélda egyszerűbb lett, és az érték `[]`-ra zsugorodott. A `proptest` a hibás esetet a `proptest-regressions` fájlba menti, és a következő futásnál először azt játssza vissza.

15.2.4.4 4. lépés: prefix-property referenciamodellel

A `list(Some(prefix))` helyességét nem elég azzal ellenőrizni, hogy „ha a kulcs szerepel, akkor illeszkedik”: az üres eredményt visszaadó tároló is átmenne. Egy egyszerű **referenciamodellhez** (`BTreeMap`) hasonlítjuk a teljes eredményhalmazt; a modell szűrt, rendezett tartalmával való egyenlőség egyszerre ellenőrzi a teljességet, a helyességet és a duplikációmentességet. A generátor szűk ábécét (`[a-c]`) használ, hogy a prefixek gyakran illeszkedjenek, de gyakran ne is.

Kódblokk — fájl: `rkv-core/tests/storage_proptest.rs` · művelet: beillesztés a fájl végére · ellenőrzés: `cargo test -p rkv-core --test storage_proptest` → 4 sikeres teszt

```rust
/// Referenciamodell: egy egyszerű `BTreeMap`. A `list(Some(prefix))` eredményének
/// pontosan a modell szűrt, rendezett tartalmának kell lennie: így a teljesség,
/// a helyesség és a duplikációmentesség egyetlen egyenlőségben ellenőrződik.
fn check_prefix<S: Storage>(store: &mut S, keys: &[String], prefix: &str) ->
↪  std::result::Result<(), TestCaseError> {
    let mut model = BTreeMap::new();
    for (i, key) in keys.iter().enumerate() {
        let value = vec![i as u8];
        Storage::set(store, key.clone(), value.clone()).unwrap();
        model.insert(key.clone(), value);
    }
    let expected: Vec<(String, Vec<u8>)> =
        model.into_iter().filter(|(k, _)| k.starts_with(prefix)).collect();
    prop_assert_eq!(Storage::list(store, Some(prefix)).unwrap(), expected);
    Ok(())
}

/// Szűk ábécé: így a prefixek gyakran illeszkednek, de gyakran nem is.
fn prefix_inputs() -> impl Strategy<Value = (Vec<String>, String)> {
    (prop::collection::vec("[a-c]{0,3}", 0..8), "[a-c]{0,2}")
}

proptest! {
    #[test]
    fn list_with_prefix_matches_the_model((keys, prefix) in prefix_inputs()) {
        check_prefix(&mut MemStore::new(), &keys, &prefix)?;
    }
}
```

```rust
/// Hamis tárolók: a property-nek mindkettőn el kell buknia.
struct AlwaysEmpty(MemStore);
struct AlwaysAll(MemStore);

macro_rules! delegate_except_list {
    ($ty:ty) => {
        impl Storage for $ty {
            fn get(&self, key: &str) -> Result<Option<Vec<u8>>> { Storage::get(&self.0, key) }
            fn set(&mut self, key: String, value: Vec<u8>) -> Result<()> { Storage::set(&mut
             ↪  self.0, key, value) }
            fn delete(&mut self, key: &str) -> Result<bool> { Storage::delete(&mut self.0, key)
             ↪  }
            fn list(&self, prefix: Option<&str>) -> Result<Vec<(String, Vec<u8>)>> {
             ↪  self.list_impl(prefix) }
        }
    };
}
impl AlwaysEmpty { fn list_impl(&self, _p: Option<&str>) -> Result<Vec<(String, Vec<u8>)>> {
 ↪  Ok(Vec::new()) } }
impl AlwaysAll { fn list_impl(&self, _p: Option<&str>) -> Result<Vec<(String, Vec<u8>)>> {
 ↪  Storage::list(&self.0, None) } }
delegate_except_list!(AlwaysEmpty);
delegate_except_list!(AlwaysAll);

fn property_fails_for<S: Storage>(make: fn() -> S) -> bool {
    TestRunner::default()
        .run(&prefix_inputs(), |(keys, prefix)| check_prefix(&mut make(), &keys, &prefix))
        .is_err()
}

#[test]
fn the_prefix_property_rejects_wrong_implementations() {
    assert!(property_fails_for(|| AlwaysEmpty(MemStore::new())));
    assert!(property_fails_for(|| AlwaysAll(MemStore::new())));
}
```

A `the_prefix_property_rejects_wrong_implementations` teszt kimondja, hogy a property **értékes**: a mindig üres és a prefixet figyelmen kívül hagyó, mindent visszaadó hamis tárolón is elbukik (a `TestRunner::run` `Err`-t ad). Az utóbbi két típus csak a `list`-ben tér el, a többi metódus a `MemStore`-ra delegál; a makró a delegálás ismétlését takarítja meg.

15.2.4.5 5. lépés: criterion benchmark

Hozzuk létre az `rkv-core/benches/benchmark.rs` fájlt. A mérés tisztasága két döntésen múlik. (1) A `delete` csak akkor mér törlést, ha a kulcs a mért híváskor létezik: az `iter_batched` az előkészítést (a tároló feltöltését) a mérésen *kívül* végzi el, minden iterációhoz friss tárolóval. (2) A hiányzó kulcs törlése (`miss`) külön eset, külön névvel, mert egészen más költségű.

Kódblokk — fájl: `rkv-core/benches/benchmark.rs` · művelet: új fájl · ellenőrzés: `cargo bench -p rkv-core --bench benchmark`

```rust
use std::hint::black_box;

use criterion::{criterion_group, criterion_main, BatchSize, Criterion};
use rkv_core::{MemStore, Storage};

fn bench_set(c: &mut Criterion) {
    // Új kulcs beszúrása egy üres tárolóba (a tároló összeállítása és eldobása nem mért).
    c.bench_function("MemStore::set (új kulcs, üres tároló)", |b| {
```

```
        b.iter_batched(
            // Előkészítés (nem mért): friss tároló, tulajdonolt kulcs és érték.
            || (MemStore::new(), "key".to_string(), b"value".to_vec()),
            |(mut store, key, value)| {
                Storage::set(&mut store, black_box(key), black_box(value)).unwrap();
                store // a Drop költsége kívül esik a mérésen
            },
            BatchSize::SmallInput,
        );
    });
}

fn bench_get(c: &mut Criterion) {
    let mut store = MemStore::new();
    Storage::set(&mut store, "key".to_string(), b"value".to_vec()).unwrap();
    c.bench_function("MemStore::get (hit)", |b| {
        b.iter(|| black_box(Storage::get(&store, black_box("key")).unwrap()));
    });
}

fn bench_delete(c: &mut Criterion) {
    // Találat: minden mért törlés előtt létezik a kulcs (iter_batched előkészítés).
    c.bench_function("MemStore::delete (hit)", |b| {
        b.iter_batched(
            || {
                let mut store = MemStore::new();
                Storage::set(&mut store, "key".to_string(), b"value".to_vec()).unwrap();
                store
            },
            |mut store| {
                let deleted = black_box(Storage::delete(&mut store, black_box("key")).unwrap());
                (store, deleted) // a tároló visszaadva: a Drop költsége kívül esik a mérésen
            },
            BatchSize::SmallInput,
        );
    });
    // Melléfogás: a hiányzó kulcs törlése külön eset, külön névvel.
    let mut empty = MemStore::new();
    c.bench_function("MemStore::delete (miss)", |b| {
        b.iter(|| black_box(Storage::delete(&mut empty, black_box("key")).unwrap()));
    });
}

criterion_group!(benches, bench_set, bench_get, bench_delete);
criterion_main!(benches);
```

Futtassuk a benchmarkot:

```
cargo bench -p rkv-core --bench benchmark
```

Mit mér a `get` benchmark? A `Storage::get` `Vec<u8>`-ot másol (a visszaadott `Option<Vec<u8>>` allokációja és az eldobása is a mért `b.iter` részét képezi), ezért a `MemStore::get (hit)` idő a *másoló* `Storage::get` költsége, nem a kölcsönző inherens `MemStore::get` tiszta keresési költsége.

A `criterion` melegítési fázist tart, több száz mintát gyűjt, és statisztikát számol. A kimenet gépfüggő, mért adat; a lent szereplő számok egyetlen futásból valók (`Rust 1.99.1`, egy konkrét Linux x86_64 gép, a `Cargo.lock` szerinti `criterion` verzió; nyers riport a futás `target/criterion/` könyvtárában keletkezik), tehát *illusztratívak*, nem általános értékek, és másik gépen eltérnek:

```
MemStore::set (új kulcs, üres tároló)
                        time:   [27.6 ns 27.7 ns 27.8 ns]
```

```
MemStore::get (hit)       time:   [18.2 ns 18.3 ns 18.5 ns]
MemStore::delete (hit)    time:   [69.1 ns 70.6 ns 71.8 ns]
MemStore::delete (miss)   time:   [3.00 ns 3.03 ns 3.06 ns]
```

A szögletes zárójelben a 95%-os konfidenciaintervallum alsó határa, a pontbecslés és a felső határ szerepel (a lefedettség ismételt mintavételre értendő, lásd fent). Egy benchmark címe és előkészítése mondja meg, mit mér: a `set` itt kizárólag az *új kulcs beszúrását egy üres tárolóba* méri (az `iter_batched` előkészítése és a tároló eldobása kívül esik); a meglévő kulcs felülírását, az allokációk számát és a nagyobb tárolóban végzett műveleteket a Teljesítmény: mérés, profilozás, allokáció, release-beállítások alfejezet méri külön. A `get (hit)` és a `delete (miss)` a lekérdezést, illetve a sikertelen törlést méri. A `hit` és a `miss` nem összehasonlítható egymással műveletenként (a `delete (hit)` esetben a mért idő a törlést méri, a visszaadott tároló eldobásának költsége kívül esik): az előbbi egy tényleges bejegyzést távolít el, az utóbbi csak egy sikertelen keresést mér.

15.2.5 Ábra



15.2. ábra: Property-teszt iterációja - A proptest generál, ellenőriz, hiba esetén zsugorít és ment.

Az ábra lépései szövegesen:

1. **Generál**: a generátor véletlen kulcsot és értéket állít elő.
2. **Ellenőriz**: a teszt ellenőrzi az invariánst.
3. **Sikeres?**: ha igen, jöhet a következő eset (tesztenként 256-ig).
4. **Zsugorít**: hiba esetén a zsugorító egyszerűbb ellenpéldát keres.
5. **Ment**: a hibás eset a `proptest-regressions` fájlba kerül.

Az ábra a `proptest` folyamatát mutatja: a generátor véletlen bemenetet állít elő, a teszt ellenőrzi az invariánst, hiba esetén a zsugorító minimalizálja az ellenpéldát, majd a hiba bekerül a regressziós fájlba. A következő futtatáskor a `proptest` először ezeket a mentett eseteket ellenőrzi, így a korábban megtalált hibák nem vesznek el.

15.2.6 Tipikus hibák és átállási csapdák

1. **Túl szűk generátor.** Ha a property-tesztben csak rövid, fix hosszúságú `String`-eket generálunk (például `"[a-z]{1,5}"`), a hosszú kulcsokkal kapcsolatos hibák sosem derülnek ki. Az `any::<String>()` széles teret fed le, de ha a hibát speciális bemenet váltja ki, szükség lehet egyéni `Strategy`-re, például `prop::collection::vec(any::<u8>(), 0..100)`.
2. **Benchmark `black_box` nélkül.** Ha a mért függvény kimenetét nem csomagoljuk `black_box`-ba, a Rust (és a C++ is) kioptimalizálhatja a teljes számítást. A `b.iter` closure visszatérési értékét mindig `black_box`-szal kell elrejteni, és a bemeneteket is célszerű `black_box`-ba tenni, hogy az optimalizáló ne tudjon konstans-számítást végezni.

3. **A benchmark eredményét abszolút igazságként kezelni.** A mikrobenchmarkok a CPU gyorsítótárától, az operációs rendszer ütemezőjétől és a linkelési sorrendtől is függnek. A criterion konfidenciaintervalluma segít az ingadozás számszerűsítésében, de a mérések mindig gépfüggőek. A teljesítményt a Teljesítmény: mérés, profilozás, allokáció, release-beállítások alfejezetben tárgyaljuk részletesen.

15.2.7 Feladatok

15.2.7.1 (a) Fogalmi kérdés

Lefordul-e a következő `proptest` teszt, és ha nem, mi a hiba?

```
use proptest::prelude::*;
use rkv_core::{MemStore, Storage};

proptest! {
    #[test]
    fn list_returns_same_keys(key in any::<String>(), value in
    ↪  prop::collection::vec(any::<u8>(), 0..8)) {
        let mut store = MemStore::new();
        Storage::set(&mut store, key.clone(), value).unwrap();
        let keys = Storage::list(&store, None).unwrap();
        prop_assert!(keys.contains(&key));
    }
}
```

Válaszodat indokold!

15.2.7.2 (b) C++ kód átírása Rustra

Írd át idiomatikus `proptest` tesztre a következő C++ tesztet, amely azt ellenőrzi, hogy két különböző kulcs beszúrása után a `list` mindkét kulcsot visszaadja:

```
#include <cassert>
#include <unordered_map>
#include <string>

int main() {
    std::unordered_map<std::string, std::string> map;
    map["a"] = "1";
    map["b"] = "2";
    auto count = map.count("a") + map.count("b");
    assert(count == 2);
    return 0;
}
```

15.2.7.3 (c) Implementációs feladat az rkv-ban

Írj property-tesztet az `rkv-core` crate-hez, amely azt az invariánst ellenőrzi, hogy a `list(None)` eredménye kulcs szerint *szigorúan növekvő* (tehát rendezett és duplikációmentes), és a hossza megegyezik a beszúrt kulcsok közül az egymástól különbözők számával. Használj véletlen kulcslistát (`prop::collection::vec`), és `prop_assert!`/`prop_assert_eq!` makrókat.

15.2.7.4 (d) Benchmark refaktorálás

A következő benchmarkot egy fejlesztő `black_box` nélkül, és egyetlen előkészítéssel írta meg:

```
use criterion::{criterion_group, criterion_main, Criterion};
use rkv_core::{MemStore, Storage};

fn bench_delete(c: &mut Criterion) {
    let mut store = MemStore::new();
    Storage::set(&mut store, "key".to_string(), b"value".to_vec()).unwrap();
    c.bench_function("MemStore::delete", |b| {
        b.iter(|| {
            Storage::delete(&mut store, "key").unwrap();
        });
    });
}

criterion_group!(benches, bench_delete);
criterion_main!(benches);
```

Mi a probléma (két különböző is)? Írd át a benchmarkot helyesen, és magyarázd el, miért fontos az `iter_batched` és a `black_box`.

15.2.8 Megoldások

15.2.8.1 (a)

A teszt **nem fordul le**. A `Storage::list` visszatérési típusa `Result<Vec<(String, Vec<u8>)>>`, tehát a `keys` elemei kulcs–érték párok, a `keys.contains(&key)` pedig `&String`-et keres: az `E0308` („mismatched types: expected `&(String, Vec<u8>)`, found `&String`”) hibát kapjuk. Javítás: a párok kulcsait kell vizsgálni, például `prop_assert!(keys.iter().any(|(k, _)| *k == key));`. A tanulság: a generikus teszt a trait *tényleges* szignatúrájához kötött, és a fordító azonnal jelzi az eltérést; ezért tesz jót, ha a tesztek a kanonikus API-ra épülnek.

15.2.8.2 (b)

```
use proptest::prelude::*;
use rkv_core::{MemStore, Storage};

proptest! {
    #[test]
    fn two_keys_are_listed(
        key1 in any::<String>(),
        key2 in any::<String>(),
        value1 in prop::collection::vec(any::<u8>(), 0..8),
        value2 in prop::collection::vec(any::<u8>(), 0..8),
    ) {
        prop_assume!(key1 != key2);
        let mut store = MemStore::new();
        Storage::set(&mut store, key1.clone(), value1).unwrap();
        Storage::set(&mut store, key2.clone(), value2).unwrap();
        let keys: Vec<String> = Storage::list(&store, None)
            .unwrap()
            .into_iter()
            .map(|(k, _)| k)
            .collect();
        prop_assert!(keys.contains(&key1));
        prop_assert!(keys.contains(&key2));
```

```
    }
}
```

A `prop_assume!` makró elutasítja azokat a generált eseteket, ahol a két kulcs egyenlő, így a teszt csak különböző kulcsokra fut.

15.2.8.3 (c)

```
use proptest::prelude::*;
use rkv_core::{MemStore, Storage};
use std::collections::BTreeSet;

proptest! {
    #[test]
    fn list_is_strictly_increasing_and_complete(
        keys in prop::collection::vec("[a-c]{0,3}", 0..10),
    ) {
        let mut store = MemStore::new();
        for k in &keys {
            Storage::set(&mut store, k.clone(), vec![]).unwrap();
        }
        let listed: Vec<String> = Storage::list(&store, None)
            .unwrap()
            .into_iter()
            .map(|(k, _)| k)
            .collect();
        // Szigorúan növekvő: rendezett és duplikációmentes.
        prop_assert!(listed.windows(2).all(|w| w[0] < w[1]));
        // A hossz az eltérő kulcsok száma.
        let distinct: BTreeSet<_> = keys.iter().collect();
        prop_assert_eq!(listed.len(), distinct.len());
    }
}
```

A szűk ábécé (`[a-c]{0,3}`) gyakori ütközést (ismétlődő kulcsot) eredményez, így a duplikációmentességet a teszt ténylegesen terheli.

15.2.8.4 (d)

Két különböző hiba van. (1) Az első iteráció után a kulcs már nem létezik, tehát a további iterációk a *hiányzó* kulcs törlését mérik (nagyságrendekkel olcsóbb), nem a sikeres törlést. (2) Sem a bemenet, sem az eredmény nincs `black_box`-ba csomagolva, így a fordító kioptimalizálhat belőle. A javítás az `iter_batched`: az előkészítés minden iterációhoz friss, feltöltött tárolót ad (a mérésen kívül), a hiányzó kulcs esete külön benchmark:

```
use std::hint::black_box;

use criterion::{criterion_group, criterion_main, BatchSize, Criterion};
use rkv_core::{MemStore, Storage};

fn bench_delete(c: &mut Criterion) {
    c.bench_function("MemStore::delete (hit)", |b| {
        b.iter_batched(
            || {
                let mut store = MemStore::new();
                Storage::set(&mut store, "key".to_string(), b"value".to_vec()).unwrap();
                store
            },
            |mut store| {
```

```
                let deleted = black_box(Storage::delete(&mut store, black_box("key")).unwrap());
                (store, deleted) // a tároló visszaadva: a Drop költsége kívül esik a mérésen
            },
            BatchSize::SmallInput,
        );
    });
    let mut empty = MemStore::new();
    c.bench_function("MemStore::delete (miss)", |b| {
        b.iter(|| black_box(Storage::delete(&mut empty, black_box("key")).unwrap()));
    });
}

criterion_group!(benches, bench_delete);
criterion_main!(benches);
```

A `black_box(result)` megakadályozza, hogy a fordító eltávolítsa a hívás hatását; a `black_box("key")` pedig azt biztosítja, hogy a bemenet ne legyen fordítási időben ismert konstans. Az `iter_batched` az előkészítés költségét kihagyja a mérésből; a `BatchSize::SmallInput` kis, olcsón másolható bemenetet jelez, így a criterion nagyobb kötegben készíti elő az iterációkat. Ha az előkészítés költsége *is* a mérendő rész lenne, a sima `b.iter` a helyes választás, de ekkor a név legyen egyértelmű (például „create+delete”).

15.2.9 Rövid összefoglaló

A `proptest` crate lehetővé teszi, hogy az rkv `Storage` szerződésére invariánsokat fogalmazzunk meg generátorokkal és automatikus zsugorítással. A laborban létrehoztuk az `rkv-core/tests/storage_proptest.rs` fájlt, amely a `set`–`get` konzisztenciát, a `delete` `Result<bool>` szerződését és a prefix-szűrést ellenőrzi referenciamodellel (`BTreeMap`); a szándékosan hibás tároló (`tests/buggy.rs`) futás közben, zsugorított ellenpéldával bukik. Egy külön teszt igazolja, hogy a prefix-property a mindig üres és a mindent visszaadó hamis tárolót is elutasítja. A criterion benchmarkot az `rkv-core/benches/benchmark.rs` fájlban adtuk hozzá: a `set`, a `get` és a `delete` (találat és melléfogás külön) `black_box`-szal és `iter_batched`-del mér. A konfidenciaintervallum értelmezése és a tiszta mérési elrendezés elengedhetetlen a megbízható méréshez. **Továbbvitt eredmény.** Ez a benchmark-módszertan (release mód, `black_box`, `iter_batched`, konfidenciaintervallum) alapozza meg a Teljesítmény: mérés, profilozás, allokáció, release-beállítások alfejezetben tárgyalt haladó méréseket.

15.2.10 Források és ellenőrzési hiányok

- A `proptest` (1.x) és a `criterion` (0.8.x) crate-ek API-ja a docs.rs-en érhető el: https://docs.rs/proptest/1, https://docs.rs/criterion/0.8. A kódot a könyv rögzített toolchainjével (Rust 1.99.1), `proptest` 1.12 és `criterion` 0.8.2 verzióval ellenőriztük; a `criterion` főverziói között a `black_box` helye változott, ezért a könyv a `std::hint::black_box`-ot használja.
- A `proptest` regressziós fájlok pontos elnevezése és a `prop_assume!` pontos szemantikája verziófüggő.
- A fejezetben bemutatott futási idők illusztratívak, gépfüggőek, és nem tekinthetők mért értéknek.
- A C++ példákban a `std::chrono` mérés korlátai jól ismertek; a Google Benchmark `DoNotOptimize` használata analóg a Rust `black_box`-szal, de ennek részletei nem részei ennek az alfejezetnek.

V. RÉSZ – Párhuzamosság és aszinkronitás

16 Szálak és megosztott állapot

16.1 Szálak, Send és Sync: data race fordítási időben kizárva

16.1.1 Motiváció

A C++ `std::thread` használatakor a fordító nem tudja megmondani, hogy egy megosztott objektumhoz való hozzáférés biztonságos-e. A data race definiálatlan viselkedés (UB), amelyet a szabvány csak tilt, de a fordítási folyamat nem ellenőriz. A Rust ezzel szemben a **típusrendszerbe** építi a szálbiztonságot: a `Send` és `Sync` marker trait-ek segítségével a fordító már fordítási időben kizárja a data race-hez vezető leggyakoribb mintákat. A garancia a leforduló *safe* Rustra vonatkozik, feltéve, hogy a felhasznált `unsafe` kód, köztük a kézi `Send`/`Sync` implementációk és az FFI helyes (lásd a 2.1 alfejezetet). Ez a „félelem nélküli párhuzamosság” (fearless concurrency) alapja.

16.1.2 Fogalmak és szabályok

Definíció. A `std::thread::spawn` egy új operációsrendszer-szálat indít, és `JoinHandle<T>`-t ad vissza. A szignatúrája (egyszerűsítve):

```
pub fn spawn<F, T>(f: F) -> JoinHandle<T>
where
    F: FnOnce() -> T,
    F: Send + 'static,
    T: Send + 'static,
```

A `F: FnOnce() -> T` azt jelenti, hogy a closure egyszer lefuttatható, és a benne lévő értékeket elkölti. A `'static` korlát azt követeli meg, hogy a closure **nem tartalmazhat** nem-`'static` referenciát; a szál a fő szál élettartamától függetlenül futhat. Ezért a closure-t általában `move` kulcsszóval írjuk, hogy a környezetéből **tulajdonba vegye** a szükséges értékeket, ne csak kölcsönözze őket. A korlát kikényszeríti, hogy a szál a saját adatával dolgozzon, vagy olyan adattal, amelynek a teljes élettartama garantáltan elég hosszú.

Definíció. A `Send` és `Sync` úgynevezett **auto trait-ek**: a fordító automatikusan implementálja őket egy típusra, ha minden mezőjük is `Send`/`Sync`. A szabályok:

- `Send`: a típus értéke **tulajdonjoggal átadható** egy másik szálnak. Más szóval a mozgatás szálhatáron keresztül biztonságos.
- `Sync`: a típusra mutató `&T` **megosztható** több szál között, azaz a referencia használata párhuzamosan biztonságos.

Ezek az auto trait-ek a fordító számára ismert belső szabályokból származnak. Például:

- A primitív típusok (`i32`, `bool`, `char`, `&str`, stb.), valamint a `String`, `Vec<T>`, `Box<T>` általában `Send + Sync`, ha a belső típus is az.
- Az `Rc<T>` **nem** `Send` és **nem** `Sync`, mert a referenciaszámláló növelése nem atomikus; két szál egyszerre módosíthatná a számlálót, ami data race.

- Az `Arc<T>` (atomic reference counted) `Send + Sync`, ha `T: Send + Sync`, mert a számláló atomikus. Az `Arc` a *megosztást* oldja meg, a módosítást nem: egy `Arc<i32>` helyesen megosztható szálak között, de közvetlenül nem módosítható (az `Arc` csak `&T`-t ad). A módosításhoz belső mutabilitás kell, szálbiztosan például `Arc<Mutex<T>>` vagy atomikus típus.
- A `RefCell<T>` `Send`, ha `T: Send`, de **nem** `Sync`, mert a futásidejű kölcsönzés-számláló nem szálbiztos.
- A `Mutex<T>`, `RwLock<T>`, `AtomicUsize` és társai tipikusan `Send + Sync`, ha a belső típus megfelelő; ezek a szálbiztos szinkronizációs primitívek.
- A nyers mutatók (`*const T`, `*mut T`) **nem** `Send` és **nem** `Sync`, mert a memória hozzáférés szinkronizálása a programozó felelőssége.

Szabály. A Rust adatverseny-mentességi garanciája: ha a kód `safe` Rustban lefordul, akkor a data race kizárt. A `Send`/`Sync` auto trait-ek a típusrendszeren keresztül biztosítják, hogy egy érték ne jusson el olyan szálhoz, ahol a memóriához való hozzáférés nincs szinkronizálva. A szabály megszegése fordítási hibát okoz, nem UB-t.

16.1.3 C++ és Rust összevetés

Tekintsünk egy tipikus C++ data race példát: két szál egyszerre növel egy közös `int`-et, szinkronizáció nélkül.

```cpp
#include <thread>
#include <iostream>

int main() {
    int counter = 0;
    std::thread t1([&] { for (int i = 0; i < 1'000'000; ++i) ++counter; });
    std::thread t2([&] { for (int i = 0; i < 1'000'000; ++i) ++counter; });
    t1.join();
    t2.join();
    std::cout << counter << '\n'; // UB: data race
}
```

A C++ fordító ezt lefordítja, de a viselkedés definiálatlan – a kimenet nem garantált. A Rustban a kulcs nem az `i32` típusa (az `i32` `Send` és `Sync`), hanem a *hozzáférések átfedése*. Egyetlen író szál szabályos, mert a scope garantálja, hogy a szál a scope vége előtt befejeződik, így a `&mut counter` kölcsönzés nem él tovább a scope-nál:

Kódblokk — önálló példa · szabályos változat (egyetlen író) · kimenet: `1000000`

```rust
use std::thread;

fn main() {
    let mut counter = 0;
    thread::scope(|s| {
        s.spawn(|| {
            for _ in 0..1_000_000 {
                counter += 1;
            }
        });
    });
    println!("{counter}");
}
```

Ha viszont két szál is módosítaná ugyanazt a változót, a két `&mut counter` kölcsönzés átfedne, és a fordító elutasítja (ez a C++ data race-ének fordítási idejű megfelelője):

Kódblokk — önálló példa · **szándékosan hibás** · várt hiba: `E0499` (`cannot borrowcounteras mutable more than once at a time`)

```rust
use std::thread;

fn main() {
    let mut counter = 0;
    thread::scope(|s| {
        s.spawn(|| {
            for _ in 0..1_000_000 {
                counter += 1;
            }
        });
        s.spawn(|| {
            for _ in 0..1_000_000 {
                counter += 1; // E0499: a másik closure már kizárólag kölcsönzi
            }
        });
    });
    println!("{counter}");
}
```

A hibaüzenet a kölcsönzési szabályra, nem a `Send`/`Sync` traitekre hivatkozik. A szinkronizált megosztott módosítás helyes Rust-megoldása `Mutex` vagy atomikus típus, amit az Mutex, RwLock, csatornák és a megosztott rkv alfejezet tárgyal.

A különbség: a C++ UB-t hagy a programban, a Rust pedig a kölcsönzési szabállyal (és a nem scoped szálaknál a `Send`/`Sync` traitekkel) kikényszeríti a szinkronizációt a típusrendszeren keresztül.

16.1.4 Kidolgozott példa: szálak indítása move closure-rel és Rc vs. Arc

A következő laborban a szálak indítását, a `move` closure-t és az `Rc`/`Arc` különbséget gyakoroljuk.

Munkakönyvtár: `~/rust-labor/r033/`

Hozzuk létre a projektet:

```
cd ~/rust-labor
cargo new r033
cd r033
```

Nyissuk meg a `src/main.rs` fájlt, és írjuk be az alábbi, **szándékosan hibás** kódot:

```rust
use std::rc::Rc;
use std::thread;

fn main() {
    let data = Rc::new(vec![1, 2, 3, 4, 5]);

    let handle = thread::spawn(move || {
        println!("Az adat hossza: {}", data.len());
    });

    handle.join().unwrap();
}
```

Futtassuk a `cargo run` parancsot. A fordító hibát ad, a hibakód jellemzően `E0277: Rc<Vec<i32>> cannot be sent between threads safely`. A hiba oka, hogy az `Rc` nem `Send`, ezért a `move` closure nem teljesíti a `thread::spawn F: Send` korlátját. Az `Rc` referenciaszámlálója nem atomikus, ezért két szál egyszerre módosíthatná, ami data race-hez vezetne.

Javítás: cseréljük le az `Rc`-t `Arc`-ra:

```
use std::sync::Arc;
use std::thread;

fn main() {
    let data = Arc::new(vec![1, 2, 3, 4, 5]);

    let handle = thread::spawn(move || {
        println!("Az adat hossza: {}", data.len());
    });

    handle.join().unwrap();
}
```

Futtassuk újra a `cargo run` parancsot. Várt kimenet:

```
Az adat hossza: 5
```

Ez a példa azt mutatja, hogy az `Arc` atomikus referenciaszámlálóval rendelkezik, ezért `Send + Sync`, míg az `Rc` nem az. Az `Arc` klónozása szálak között is biztonságos:

```
use std::sync::Arc;
use std::thread;

fn main() {
    let data = Arc::new(vec![1, 2, 3, 4, 5]);

    let mut handles = vec![];
    for _ in 0..4 {
        let data_clone = Arc::clone(&data);
        let handle = thread::spawn(move || {
            println!("Hossz: {}", data_clone.len());
        });
        handles.push(handle);
    }

    for handle in handles {
        handle.join().unwrap();
    }
}
```

Szál pánikja. A szál pánikjának hatása a panic stratégiától függ. Az alapértelmezett `panic = "unwind"` mellett csak a pánikoló szál áll le, a többi szál és a `main` tovább fut, a `join()` pedig `Err`-t ad vissza, amelyből a pánik oka kiolvasható:

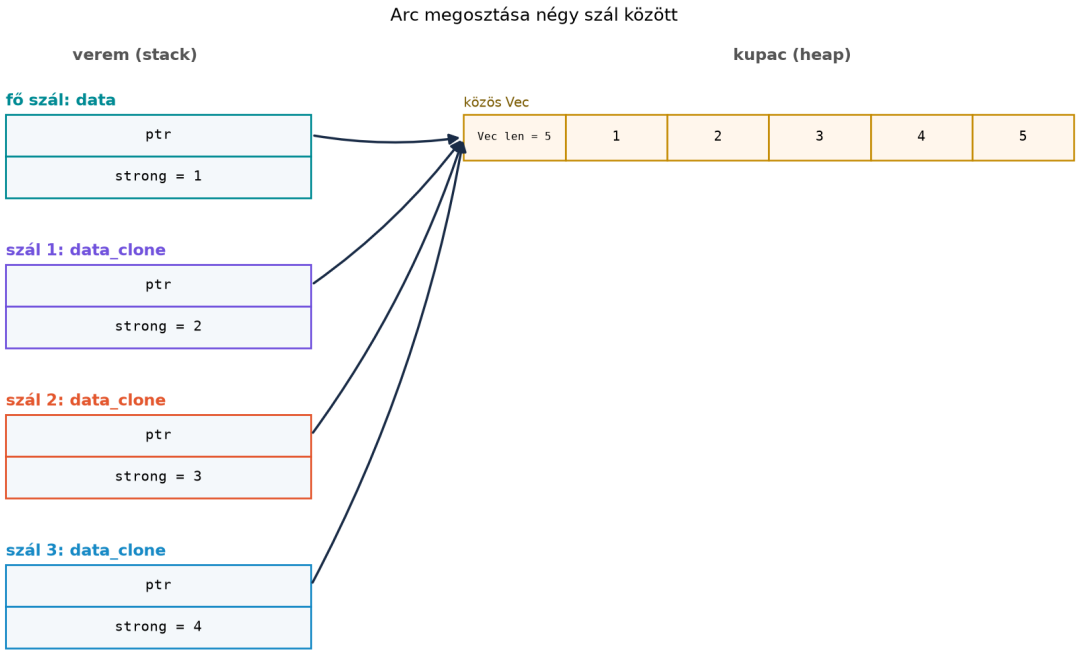
```
use std::thread;

fn main() {
    let h = thread::spawn(|| {
        let v: Vec<i32> = Vec::new();
        v[std::hint::black_box(3)] // index out of bounds -> pánik ebben a szálban
    });
    let r = h.join();
    println!("join().is_err() = {}", r.is_err()); // true; a main szál tovább fut
}
```

`panic = "abort"` mellett viszont az egész folyamat azonnal leáll, a többi szállal együtt; ott a `join()` nem kap `Err`-t. A könyv alapértelmezett profilja `unwind`.

Figyeljük meg: a `move` closure-rel minden szál saját `Arc` klónt kap, és a számláló atomikusan növekszik.

Ábra: Az `Arc` megosztása szálak között.



16.1. ábra: Arc megosztása négy szál között - Az Arc atomikus számlálója biztosítja a biztonságos megosztást; minden szál saját Arc-mutatójával hivatkozik a közös kupacobjektumra.

Az ábra azt mutatja, hogy az `Arc` másolatai ugyanarra a kupacobjektumra mutatnak, és a referenciaszámláló atomikusan növekszik. Az `Rc` ugyan hasonló elrendezésű, de a számláló növelése nem atomikus, ezért nem `Send`.

16.1.5 Scoped threads: kölcsönzés szálakba

A `thread::spawn 'static` korlátja miatt nem tudunk kölcsönzött referenciákat átadni a szálnak. A `std::thread::scope` ezt a korlátot oldja fel: a scope garantálja, hogy a benne indított szálak **befejeződnek**, mielőtt a scope-ból kilépünk. Így a szálak kölcsönözhetnek olyan adatokat, amelyek a scope-on kívül élnek, de nem `'static`.

Kódblokk — önálló példa · **szándékosan hibás** · várt hiba: `E0502` (`cannot borrowdataas mutable because it is also borrowed as immutable`)

```
use std::thread;

fn main() {
    let mut data = vec![1, 2, 3, 4, 5];

    thread::scope(|s| {
        s.spawn(|| {
```

```
            println!("Első szál: {:?}", data); // megosztott kölcsönzés
        });
        s.spawn(|| {
            data.push(6); // E0502: módosító kölcsönzés, miközben az előző él
        });
    });

    println!("A scope után: {:?}", data);
}
```

Ez a program **nem fordul le**, ezért nincs futási kimenete: az első closure `&data`-t, a második `&mut data`-t rögzít, és a két kölcsönzés a scope-on belül egyszerre él. A `scope` a `'static` korlátot váltja ki a scope élettartamával, de a kölcsönzési szabály továbbra is érvényes. Négy szabályos megoldás van, attól függően, mit akarunk.

Csak olvasó szálak. Tetszőleges számú megosztott kölcsönzés megengedett; a szálak értéket is visszaadhatnak a `join`-on keresztül:

```
use std::thread;

fn main() {
    let data = vec![1, 2, 3, 4, 5];

    let (sum, max) = thread::scope(|s| {
        let sum = s.spawn(|| data.iter().sum::<i32>());
        let max = s.spawn(|| *data.iter().max().unwrap());
        (sum.join().unwrap(), max.join().unwrap())
    });
    println!("összeg = {sum}, maximum = {max}");
}
```

Várt kimenet:

```
összeg = 15, maximum = 5
```

Diszjunkt szeletek. A `split_at_mut` két nem átfedő `&mut` szeletet ad (lásd Biztonságos absztrakció unsafe alapon), így két szál egyszerre módosíthat egy vektort:

```
use std::thread;

fn main() {
    let mut data = vec![1, 2, 3, 4, 5, 6];

    let (left, right) = data.split_at_mut(3);
    thread::scope(|s| {
        s.spawn(|| left.iter_mut().for_each(|x| *x *= 10));
        s.spawn(|| right.iter_mut().for_each(|x| *x += 100));
    });
    println!("{data:?}");
}
```

Várt kimenet (a két szál nem fed át, ezért determinisztikus):

```
[10, 20, 30, 104, 105, 106]
```

Külön scope-ok. Az olvasó és az író scope-ja egymás után fut, így a kölcsönzések nem fednek át (ez gyakorlatilag szekvenciális, de szabályos).

Zár. Ha a szálaknak ténylegesen osztozniuk kell egy módosítható adaton, `Mutex` kell; a push-ok sorrendje ekkor nem meghatározott, ezért a kimenetet rendezve érdemes ellenőrizni:

```rust
use std::sync::Mutex;
use std::thread;

fn main() {
    let data = Mutex::new(vec![1, 2, 3, 4, 5]);

    thread::scope(|s| {
        s.spawn(|| data.lock().unwrap().push(6));
        s.spawn(|| data.lock().unwrap().push(7));
    });

    let mut v = data.into_inner().unwrap();
    v.sort(); // a két push sorrendje nem meghatározott
    println!("{v:?}");
}
```

Várt kimenet:

```
[1, 2, 3, 4, 5, 6, 7]
```

16.1.6 Tipikus hibák és átállási csapdák

1. **`Rc` használata szálak között.** A leggyakoribb hiba, amikor a C++-os `shared_ptr` mintájára `Rc`-t próbálunk megosztani. A `Rc` nem `Send` és nem `Sync`; a megoldás `Arc`. A hibaüzenet (`E0277`) világos: a típus nem teljesíti a `Send` korlátot. **Továbbvitt eredmény.** Ez a különbségtétel az Mutex, RwLock, csatornák és a megosztott rkv alfejezetben az `Arc<Mutex<Store>>` mintához vezet; a Send-határok az aszinkron kódban, a C ABI pedig az FFI-ban használja a `Send`/`Sync` fogalmát.

2. **`move` closure elfelejtése.** Ha a closure nem `move`, a környezetéből kölcsönzött értékek nem `'static`, ezért a `thread::spawn` hibát ad. A hibakód `E0373` vagy `E0521` jellemzően. A megoldás a `move` használata, vagy ha a kölcsönzés szükséges, a `thread::scope` alkalmazása.

3. **`RefCell<T>` megosztása.** A `RefCell` futásidejű kölcsönzést biztosít, de nem szálbiztos. Ha több szálból akarunk hozzáférni, a `Mutex` vagy `RwLock` a megfelelő választás. Ez a hiba gyakran akkor jelentkezik, amikor az `Rc<RefCell<T>>` mintát próbáljuk több szálra kiterjeszteni; a helyettesítés `Arc<Mutex<T>>`.

4. **`'static` korlát figyelmen kívül hagyása.** A `thread::spawn` által megkövetelt `'static` azt jelenti, hogy a closure-nek nem lehet nem-`'static` referenciája. Ha a fő szál adatait szeretnénk látni, a `move` closure-rel tulajdonba kell venni az értéket, vagy `Arc`-ot használni.

16.1.7 Feladatok

16.1.7.1 1. Feladat: Lefordul-e és miért?

A következő kód lefordul-e? Ha nem, miért és mi a javítás?

```rust
use std::rc::Rc;
use std::thread;

fn main() {
    let s = Rc::new(String::from("hello"));
    let handle = thread::spawn(move || {
        println!("{}", s);
    });
```

```
    handle.join().unwrap();
}
```

16.1.7.2 2. Feladat: C++ kód átírása idiomatikus Rustra

Írd át az alábbi C++ kódot idiomatikus Rustra úgy, hogy a szálak közötti adatmegosztás `Arc`-cal történjen, és a kimenet tartalma ugyanaz legyen:

```cpp
#include <iostream>
#include <thread>
#include <vector>

int main() {
    std::vector<int> v = {1, 2, 3, 4, 5};
    std::thread t([&v] {
        for (int x : v) std::cout << x << ' ';
    });
    t.join();
}
```

16.1.7.3 3. Feladat: Implementáció az rkv-ban

Az `rkv-core` meglévő tárolóit (`MemStore`, `FileStore`) tedd szálak között megosztható típussá. Az `rkv-core/src/shared.rs` fájlban definiálj egy `SharedStore` típust, amely bármely `Storage + Send` implementációt burkol egy `Arc<Mutex<Box<dyn Storage + Send>>>` mezőben. Legyen `Clone`, legyen `new(store: impl Storage + Send + 'static)` konstruktora, `set(&self, ..)` és `get(&self, ..)` metódusa, valamint egy `report(&self) -> Result<String>` metódusa, amely a tárolt kulcsok számát és listáját adja. Írj tesztet, amely fordítási időben ellenőrzi a `Send + Sync` tulajdonságot, két szálból ír, a fő szálból olvas. Magyarázd el, miért elég a `Mutex<Box<dyn Storage + Send>>` (miért nem kell a trait-objektumnak `Sync`-nek lennie), és miért nem működne közvetlenül egy `Arc<MemStore>`.

16.1.7.4 4. Feladat: Hibajavítás és refaktorálás

Az alábbi kód szándékosan hibás. Javítsd ki úgy, hogy a `thread::scope`-ot használd a kölcsönzött adat megosztására, és a kód lefordúljon:

```rust
use std::thread;

fn main() {
    let data = vec![10, 20, 30];
    let handle = thread::spawn(|| {
        println!("{:?}", data);
    });
    handle.join().unwrap();
}
```

16.1.8 Megoldások

16.1.8.1 1. Feladat megoldása

A kód nem fordul le, mert `Rc` nem `Send`. A hibaüzenet jellemzően `E0277`. Javítás: cserélje `Rc`-t `Arc`-ra:

```rust
use std::sync::Arc;
use std::thread;

fn main() {
    let s = Arc::new(String::from("hello"));
    let handle = thread::spawn(move || {
        println!("{}", s);
    });
    handle.join().unwrap();
}
```

16.1.8.2 2. Feladat megoldása

Idiomatikus Rust-átírás:

```rust
use std::sync::Arc;
use std::thread;

fn main() {
    let v = Arc::new(vec![1, 2, 3, 4, 5]);
    let v_clone = Arc::clone(&v);
    let handle = thread::spawn(move || {
        for x in v_clone.iter() {
            print!("{} ", x);
        }
    });
    handle.join().unwrap();
}
```

A kimenet: `1 2 3 4 5`.

16.1.8.3 3. Feladat megoldása

Kódblokk — fájl: `rkv-core/src/shared.rs` · művelet: új fájl · ellenőrzés: `cargo build -p rkv-core`

```rust
use std::sync::{Arc, Mutex};

use crate::{Result, Storage};

/// Szálak között megosztható tároló: bármely `Storage` implementációt burkol.
#[derive(Clone)]
pub struct SharedStore {
    inner: Arc<Mutex<Box<dyn Storage + Send>>>,
}

impl SharedStore {
    pub fn new(store: impl Storage + Send + 'static) -> Self {
        Self {
            inner: Arc::new(Mutex::new(Box::new(store))),
        }
    }

    pub fn set(&self, key: String, value: Vec<u8>) -> Result<()> {
        self.inner.lock().unwrap().set(key, value)
    }

    pub fn get(&self, key: &str) -> Result<Option<Vec<u8>>> {
        self.inner.lock().unwrap().get(key)
    }
```

```
    /// A tárolt kulcsok száma és listája (egyetlen zárfoglalással).
    pub fn report(&self) -> Result<String> {
        let guard = self.inner.lock().unwrap();
        let keys: Vec<String> = guard.list(None)?.into_iter().map(|(k, _)| k).collect();
        Ok(format!("A tárolóban {} kulcs van: {:?}", keys.len(), keys))
    }
}
```

Kódblokk — fájl: `rkv-core/src/lib.rs` · művelet: beillesztés a `pub mod` sorok közé és a `pub use` sorok után · ellenőrzés: `cargo build --workspace`

```
pub mod shared;

pub use shared::SharedStore;
```

Kódblokk — fájl: `rkv-core/tests/shared.rs` · művelet: új fájl · ellenőrzés: `cargo test -p rkv-core --test shared`

```
use std::thread;

use rkv_core::{MemStore, SharedStore};

fn assert_send_sync<T: Send + Sync>() {}

#[test]
fn shared_store_works_across_threads() {
    assert_send_sync::<SharedStore>(); // fordítási idejű ellenőrzés

    let store = SharedStore::new(MemStore::new());
    let handles: Vec<_> = (0..2u8)
        .map(|i| {
            let store = store.clone();
            thread::spawn(move || store.set(format!("k{i}"), vec![i]).unwrap())
        })
        .collect();
    for h in handles {
        h.join().unwrap();
    }

    assert_eq!(store.get("k1").unwrap(), Some(vec![1]));
    assert_eq!(store.report().unwrap(), r#"A tárolóban 2 kulcs van: ["k0", "k1"]"#);
}
```

Az `Arc` a megosztást adja (a `SharedStore` klónjai ugyanazt a tárolót látják), a `Mutex` pedig a belső mutabilitást: a `set` `&self`-et vesz, mégis `&mut dyn Storage`-ot ér el a zár alatt. A `Mutex<T>` akkor `Send + Sync`, ha `T: Send`; ezért elég, hogy a trait-objektum `Send` (`Box<dyn Storage + Send>`), `Sync` nem kell. Egy `Arc<MemStore>` ezzel szemben nem módosítható (az `Arc` csak `&MemStore`-t ad, a `set` pedig `&mut self`-et kér). A `report` egyetlen zárfoglalással dolgozik, és a tároló bármely háttérrel (`MemStore`, `FileStore`) működik. A Mutex, RwLock, csatornák és a megosztott rkv ezt a típust bővíti tovább.

16.1.8.4 4. Feladat megoldása

A hibát a `thread::spawn 'static` korlátja okozza. Javítás `thread::scope`-pal:

```
use std::thread;

fn main() {
    let data = vec![10, 20, 30];
```

```
    thread::scope(|s| {
        s.spawn(|| {
            println!("{:?}", data);
        });
    });
}
```

A `thread::scope` garantálja, hogy a szál befejeződik, mielőtt a `data` kimegy a hatókörből.

16.1.9 Rövid összefoglaló

A Rust `std::thread::spawn` egy `'static` korlátos, `Send` closure-t vár el, és `JoinHandle<T>`-t ad vissza, amelynek `join` metódusa várja a szál végét. A `Send` és `Sync` auto trait-ek fordítási időben garantálják, hogy egy típus értéke átadható (`Send`) vagy referenciája megosztható (`Sync`) szálak között. Az `Rc<T>` nem `Send`, mert nem atomikus referenciaszámlálót használ; az `Arc<T>` viszont `Send + Sync`, ha `T` is az. A `thread::scope` lehetővé teszi nem-`'static` referenciák kölcsönzését szálakba, mert a scope garantálja a szálak befejeződését. A C++-ban a data race UB, a Rustban a `Send`/`Sync` rendszer kizárja a versenyhelyzetet a safe kódban. **Továbbvitt eredmény.** A `Send`/`Sync` fogalma alapozza meg a Mutex, RwLock, csatornák és a megosztott rkv alfejezet `Arc<Mutex<Store>>` mintáját, a Future-ök és async/await `Send`-határait, és a C ABI FFI-biztonsági megfontolásait.

16.1.10 Források és ellenőrzési hiányok

A nyelv szabályait a `std::thread` modul dokumentációja és a Rust Reference írja le. A hibaüzenetek (`E0277`, `E0373`, `E0502`) szövege verziófüggő, ezért a fejezet a hibakódot és a hiba lényegét közli. A `thread::scope` a Rust 1.63 óta stabil; a `Send`/`Sync` auto trait-ek viselkedése a nyelv alapvető része. A `SharedStore` típust a Mutex, RwLock, csatornák és a megosztott rkv alfejezet használja. Ebben az alfejezetben külső crate-et nem használunk.

16.2 Mutex, RwLock, csatornák és a megosztott rkv

16.2.1 Motiváció

C++-ban a szálbiztos megosztott állapot kézi fegyelmet igényel: a `std::mutex` és a védett adat külön tagváltozók, és a fordító nem tudja, melyik zár melyik adatot védi. Ha elfelejtünk `lock_guard`-ot írni, adatversenyt kapunk; ha rossz sorrendben zárunk, holtpontot. A Rust ezt a típust egyesíti: a `Mutex<T>` és az `RwLock<T>` **birtokolja** a védett értéket, és a zár megszerzése egy RAII guardot ad, amelyen keresztül csak a zár élettartama alatt férhetünk az adathoz. Ez a Belső mutabilitás: Cell, RefCell és a futásidejű kölcsönzés alfejezetben megismert `RefCell<T>` szálbiztos párja: a zár futásidejű kölcsönzést valósít meg, de ott nem pánikol kölcsönzési szabálysértéskor, hanem blokkol.

16.2.2 Fogalmak és szabályok

Definíció. A `std::sync::Mutex<T>` kölcsönös kizárást (mutual exclusion) biztosító szinkronizációs primitív. A `T` érték a zár mögött él; a `lock()` metódus `LockResult<MutexGuard<'_, T>>` értéket ad vissza. A `MutexGuard<'_, T>` egy RAII guard, amely `Deref<T>`-et és `DerefMut<T>`-et

implementál, tehát `&T`-ként vagy `&mut T`-ként használható. Amikor a guard eldobódik (például kimegy a hatókörből), a zár automatikusan felszabadul.

Szabály. A `Mutex<T>` akkor `Send` és `Sync`, ha `T: Send` (a zár kizárólagossága miatt a `T`-nek nem kell `Sync`-nek lennie: egy `Mutex<Cell<i32>>` is `Sync`). A `RwLock<T>` ezzel szemben csak akkor `Sync`, ha `T: Send + Sync`, mert az olvasók egyszerre több szálon osztoznak a `&T`-n. A `std::sync::MutexGuard<'_, T>` **nem** `Send` (a zárat azon a szálon kell elengedni, amelyik megszerezte), viszont `Sync`, ha `T: Sync`. Ugyanez igaz az `RwLockReadGuard`-ra és az `RwLockWriteGuard`-ra. A tokio `tokio::sync::MutexGuard` ezzel szemben `Send` (ha `T: Send`) és `Sync` (ha `T: Send + Sync`), mert az aszinkron zár nincs szálhoz kötve. Következmény: egy `std` guardot tartó `async` blokk `Send` future-je nem lehet, ha a guard az `.await`-en át él (a `tokio::spawn` ilyet elutasít), a tokio guardé igen; ez utóbbi a *típusrendszer* szerint megengedett, de hosszú kritikus szakaszt, áteresztőképesség-romlást és holtpont-kockázatot jelent (lásd az Future-ök és async/await: mi történik a motorháztető alatt alfejezetet).

Definíció. A `std::sync::RwLock<T>` többszörös olvasó / egy író zárat biztosít. A `read()` metódus `RwLockReadGuard`-ot, a `write()` metódus `RwLockWriteGuard`-ot ad. Egyszerre több `read` guard élhet, de `write` guard csak egyedül. A `RwLock` nem támogatja a read-guard upgradejét write-guard-ra, ezért a read lock alatt nem szabad write-ot kérni ugyanarra a zárra, mert holtpontot kaphatunk.

Szabály. Ha egy szál pánikol, miközben a `Mutex` guardját tartja, a zár **mérgezetté** (poisoned) válik. Az `RwLock` csak akkor mérgeződik, ha az **író** (`write()` guard) pánikol; egy olvasó (`read()` guard) pánikja nem mérgezi a zárat (ellenőrzött: olvasói pánik után `is_poisoned() == false`, írói pánik után `true`). A következő `lock()`/`read()`/`write()` hívás mérgezett zárnál `Err(PoisonError<_>)` értéket ad vissza; a belső adat a `PoisonError::into_inner()` metódussal nyerhető ki (a `Mutex` példánk `5` értéket adott vissza a pánik után is). Gyakran `unwrap()`-ot használunk, mert a pánik már megtörtént, és a konzisztens állapotot amúgy sem garantálhatjuk; ha az invariánsok védettek, az `into_inner()` folytatást enged.

Definíció. Az `std::sync::mpsc` modul **több termelő, egy fogyasztó** (multi-producer, single-consumer) csatornát ad, két alakban. Az `mpsc::channel()` **korlátlan** csatornát hoz létre: a `send()` sosem blokkol (a puffer a memória erejéig nő, ellenőrzött: 100 000 üzenet küldése olvasó nélkül is lefut), ezért gyors termelő és lassú fogyasztó esetén a memóriahasználat nőhet. Az `mpsc::sync_channel(n)` **korlátos** csatornát hoz létre `n` elemű pufferrel: a `send()` blokkol, ha a puffer tele van (a `try_send()` ilyenkor `TrySendError::Full` hibát ad), `n = 0` esetén pedig a küldő és a fogadó találkozásáig (rendezvous). A `Sender<T>` (és a `SyncSender<T>`) klónozható, a `Receiver<T>` nem. A `recv()` blokkol, ha a csatorna üres; ha az összes küldő eldobódik, a `recv()` `Err(RecvError)` értékkel tér vissza, és a `Receiver` feletti `for` ciklus automatikusan leáll.

Röviden. A `std::sync::Condvar` feltételváltozót ad, amely egy `MutexGuard`-ot vár a `wait()` hívásakor, elengedi a zárat, amíg értesítik, majd újra megszerzi. A `wait()` **hamis ébredéseket** (spurious wakeup) is adhat, ezért kizárólag **predikátumhurokkal** használjuk:

```
use std::sync::{Arc, Condvar, Mutex};
use std::thread;

let pair = Arc::new((Mutex::new(false), Condvar::new()));
let p2 = Arc::clone(&pair);
let h = thread::spawn(move || {
    let (m, c) = &*p2;
    *m.lock().unwrap() = true;
    c.notify_one();
});
let (m, c) = &*pair;
let mut ready = m.lock().unwrap();
while !*ready {
```

```
    ready = c.wait(ready).unwrap(); // a feltételt újra ellenőrizzük
}
drop(ready);
h.join().unwrap();
```

Sok esetben a csatornák vagy magasabb szintű absztrakciók kiváltják a `Condvar`-t. A külső `parking_lot` crate gyorsabb, de **nem API-azonos** alternatíva: a `lock()` közvetlenül a guardot adja (nincs `Result`, nincs mérgezés), ezért a `std`-ről `parking_lot`-ra váltáskor el kell hagyni az `unwrap()`-ot és a mérgezéskezelést, a `Condvar::wait` pedig a guardot `&mut`-ként veszi át. A `crossbeam` csatornái több fogyasztót is támogatnak (lásd az ellenőrzési hiányokat).

16.2.3 C++ és Rust összevetés

A C++-ban a zár és az adat külön van; a védett hozzáférést minden metódusban kézzel kell biztosítani:

```
#include <mutex>
#include <map>
#include <string>
#include <optional>

class Store {
    std::mutex mtx;
    std::map<std::string, std::string> data;
public:
    void set(std::string key, std::string value) {
        std::lock_guard<std::mutex> lock(mtx);
        data[std::move(key)] = std::move(value);
    }
    std::optional<std::string> get(const std::string& key) {
        std::lock_guard<std::mutex> lock(mtx);
        auto it = data.find(key);
        if (it != data.end()) return it->second;
        return std::nullopt;
    }
};
```

Rustban a `Mutex<HashMap<...>>` magában foglalja a zárat és az adatot, és a metódusok `&self`-et vesznek, mert a zár belső mutabilitást ad (a példa `LockedMap` típusa szemléltető; az rkv saját szálbiztos típusa a `SharedStore`):

```
use std::collections::HashMap;
use std::sync::Mutex;

pub struct LockedMap {
    data: Mutex<HashMap<String, String>>,
}

impl LockedMap {
    pub fn new() -> Self {
        Self { data: Mutex::new(HashMap::new()) }
    }

    pub fn set(&self, key: String, value: String) {
        let mut guard = self.data.lock().unwrap();
        guard.insert(key, value);
    }

    pub fn get(&self, key: &str) -> Option<String> {
        let guard = self.data.lock().unwrap();
```

```
        guard.get(key).cloned()
    }
}
```

A kulcskülönbség: a Rust-fordító típusrendszerrel kényszeríti ki, hogy a `HashMap`-et csak a `MutexGuard`-on keresztül érjük el. A C++-ban semmi sem akadályozza meg, hogy a `data`-t zár nélkül olvassuk vagy írjuk.

16.2.4 Kidolgozott példa: szálbiztos számláló

A legegyszerűbb szálbiztos megosztott állapot egy `Arc<Mutex<i32>>`. Több szál növeli a számlálót, és a `MutexGuard` RAII-ja biztosítja a zár felszabadítását:

```
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let counter = Arc::new(Mutex::new(0));
    let mut handles = vec![];

    for _ in 0..10 {
        let counter = Arc::clone(&counter);
        let handle = thread::spawn(move || {
            for _ in 0..1000 {
                let mut guard = counter.lock().unwrap();
                *guard += 1;
            }
        });
        handles.push(handle);
    }

    for handle in handles {
        handle.join().unwrap();
    }

    println!("végső érték: {}", *counter.lock().unwrap());
}
```

Várt kimenet (gépfüggő sorrendű, de a végső érték determinisztikus): `végső érték: 10000`. A `Mutex` nélkül az `Arc<i32>` ugyan megosztható (`Send + Sync`), de **közvetlenül nem módosítható**: az `Arc` csak `&i32`-t ad, ezért a `*counter += 1` fordítási hibát okozna (`E0594`: „cannot assign to data in an `Arc`"). A módosításhoz belső mutabilitás kell, a szálbiztos változata a `Mutex` (vagy egy atomikus típus). Az `Arc<Cell<i32>>` például nem küldhető szálnak, mert a `Cell` nem `Sync`; így a `Send`/`Sync` határok a kétféle hibát (megosztás, módosítás) külön kezelik.

16.2.5 Labor: szálbiztos rkv `SharedStore` és csatornás munkásszál

Munkakönyvtár: `~/rust-labor/rkv/`. A kiinduló állapot a Szálak, Send és Sync: data race fordítási időben kizárva 3. feladatának megoldása: az `rkv-core/src/shared.rs`-ben a `SharedStore` (`new`, `set`, `get`, `report`) és a `tests/shared.rs`. (Aki a feladatot kihagyta, a kódot a Szálak, Send és Sync: data race fordítási időben kizárva megoldásai között találja.) A labor célja, hogy ezt a **meglévő** szálbiztos burkolót kiegészítsük, és minden meglévő `Storage`-implementációt (`MemStore`, `FileStore`) együtt, ugyanazzal a tesztsorral vizsgáljunk többszálú használatban; majd egy csatornás munkásszál-mintát építünk rá. Új `Store` vagy `Storage` típust nem hozunk létre, és a `Storage` trait szignatúrája nem változik: a metódusok továbbra is `&mut self`-et vesznek, a zár ad a burkolón belül kizárólagos hozzáférést.

16.2.5.1 1. lépés: zár választása

Miért `Mutex` és nem `RwLock`? Mert a tároló `Box<dyn Storage + Send>`: a `Mutex<T>` `Sync`, ha `T: Send`, az `RwLock<T>` viszont csak akkor, ha `T: Send + Sync`, és a trait-objektum nem `Sync`. Ráadásul a `get` is `&self`-et kér ugyan, de a `FileStore` és a `MemStore` nincs olvasásra optimalizálva, ezért az `RwLock` előnye (párhuzamos olvasók) itt nem érné meg a `Sync` korlát miatti átalakítást. A zárat mérgezés (poisoning) esetén is kezelni kell: ha egy szál a zár alatt pánikol, a `lock()` `Err`-t ad. A `SharedStore` alapértelmezése a *hiba*: a `lock()` `Err`-t ad (`Error::Io`), mert egy tetszőleges `Storage` invariánsait a pánik megsérthette; automatikus továbbfolytatás (`PoisonError::into_inner`) csak olyan háttérre indokolt, amelynek panic-biztonsága igazolt, és ezt explicit, külön helyreállító API-nak kell vállalnia.

16.2.5.2 2. lépés: a `SharedStore` kiegészítése

A `shared.rs` teljes, kiegészített változata: a `lock()` segéd, a `delete`, a `list` és az `apply` (egy parancs végrehajtása egyetlen zárfoglalással) kerül hozzá, a meglévő metódusok pedig a segédet használják:

Kódblokk — fájl: `rkv-core/src/shared.rs` · művelet: teljes csere · ellenőrzés: `cargo build -p rkv-core`

```rust
use std::sync::{Arc, Mutex, MutexGuard};

use crate::{apply, Command, Error, Response, Result, Storage};

/// Szálak között megosztható tároló: bármely `Storage` implementációt burkol.
#[derive(Clone)]
pub struct SharedStore {
    inner: Arc<Mutex<Box<dyn Storage + Send>>>,
}

impl SharedStore {
    pub fn new(store: impl Storage + Send + 'static) -> Self {
        Self {
            inner: Arc::new(Mutex::new(Box::new(store))),
        }
    }

    /// A zár megszerzése. Ha egy korábbi szál pánikolt a zár alatt, a zár mérgezett.
    /// Az absztrakció alapértelmezése a *hiba*: tetszőleges `Storage` invariánsait
    /// egy pánik megsérthette (például egy félig lefutott, nem naplózott módosítás),
    /// ezért nem folytatunk automatikusan. Helyreállításhoz a hívónak új tárolót kell
    /// nyitnia (a `FileStore` a naplóból visszaépíti az állapotát).
    fn lock(&self) -> Result<MutexGuard<'_, Box<dyn Storage + Send>>> {
        self.inner.lock().map_err(|_| {
            Error::Io(std::io::Error::other(
                "a megosztott tároló mérgezett: egy szál pánikolt a zár alatt",
            ))
        })
    }

    pub fn set(&self, key: String, value: Vec<u8>) -> Result<()> {
        self.lock()?.set(key, value)
    }

    pub fn get(&self, key: &str) -> Result<Option<Vec<u8>>> {
        self.lock()?.get(key)
    }
```

```
    pub fn delete(&self, key: &str) -> Result<bool> {
        self.lock()?.delete(key)
    }

    pub fn list(&self, prefix: Option<&str>) -> Result<Vec<(String, Vec<u8>)>> {
        self.lock()?.list(prefix)
    }

    /// Egy parancs végrehajtása egyetlen zárfoglalással.
    pub fn apply(&self, command: &Command) -> Result<Response> {
        let mut guard = self.lock()?;
        apply(&mut **guard, command)
    }

    /// A tárolt kulcsok száma és listája (egyetlen zárfoglalással).
    pub fn report(&self) -> Result<String> {
        let keys: Vec<String> = self.list(None)?.into_iter().map(|(k, _)| k).collect();
        Ok(format!("A tárolóban {} kulcs van: {:?}", keys.len(), keys))
    }
}
```

Az `apply` a Trait-ek és generikusok: sablonok és concepts helyett alfejezet `apply` függvényét hívja; ez a `&mut **guard` kifejezéssel kapott `&mut (dyn Storage + Send)` értékkel is működik, mert az `apply S: Storage + ?Sized` korlátot használ (a `?Sized` engedi a trait-objektumot).

16.2.5.3 3. lépés: többszálú teszt minden háttérre

Ugyanaz a tesztsor fut a memóriabeli és a fájl-alapú háttéren is; egy új háttér hozzáadásakor elég egy újabb `#[test]`, amely a `hammer`-t hívja:

Kódblokk — fájl: `rkv-core/tests/shared_backends.rs` · művelet: új fájl · ellenőrzés: `cargo test -p rkv-core --test shared_backends` → 2 sikeres teszt

```
use std::thread;

use rkv_core::{FileStore, MemStore, SharedStore};

/// Közös szerződés: minden háttér ugyanazt kapja, ugyanazzal a többszálú tesztsorral.
fn hammer(store: SharedStore) {
    let handles: Vec<_> = (0..4)
        .map(|i| {
            let store = store.clone();
            thread::spawn(move || {
                for j in 0..100 {
                    let key = format!("kulcs-{i}-{j}");
                    store.set(key, format!("ertek-{i}-{j}").into_bytes()).unwrap();
                }
            })
        })
        .collect();
    for h in handles {
        h.join().unwrap();
    }

    let items = store.list(Some("kulcs-")).unwrap();
    assert_eq!(items.len(), 400);
    assert!(items.windows(2).all(|w| w[0].0 < w[1].0)); // rendezett
    assert_eq!(store.get("kulcs-3-99").unwrap(), Some(b"ertek-3-99".to_vec()));
    assert!(store.delete("kulcs-0-0").unwrap());
    assert!(!store.delete("kulcs-0-0").unwrap());
}
```

```rust
#[test]
fn mem_backend_survives_concurrent_writers() {
    hammer(SharedStore::new(MemStore::new()));
}

#[test]
fn file_backend_survives_concurrent_writers() {
    let dir = tempfile::tempdir().unwrap();
    let store = FileStore::open(dir.path().join("state.json"),
    ↪  dir.path().join("append.log")).unwrap();
    hammer(SharedStore::new(store));
}
```

16.2.5.4 4. lépés: csatornás munkásszál

A közvetlenül megosztott `SharedStore` helyett gyakran kényelmesebb egy munkásszálat (worker) indítani, amely egy csatornán fogadja a kéréseket. Itt **minden kérés a saját válaszcsatornáját hordozza**, ezért több kliens esetén sem keveredhetnek a válaszok: a kliens a sajátját várja. Ez nem a blokkolás elkerülése: a `call` szinkron `recv()`-en várja a worker válaszát, a worker pedig sorosítja a tárolóműveleteket, ugyanúgy, mint a `Mutex`. A parancs- és választípus nem új: az `rkv_core::Command` (Az enum mint összeg-típus és az Option a null helyett) és a `Response` (Trait-ek és generikusok: sablonok és concepts helyett), a hibát a kanonikus `Error` hordozza.

Kódblokk — fájl: `rkv-core/src/worker.rs` · művelet: új fájl; a `lib.rs`-be `pub mod worker;` · ellenőrzés: `cargo build -p rkv-core`

```rust
use std::sync::mpsc::{self, Receiver, Sender};
use std::thread::{self, JoinHandle};

use crate::{Command, Error, Response, Result, SharedStore};

/// Egy kérés: a parancs és a csatorna, amelyen a kliens a válaszát várja.
pub struct Request {
    pub command: Command,
    pub reply: Sender<Result<Response>>,
}

/// Elindítja a munkásszálat. A szál addig fut, amíg van élő `Sender<Request>`.
pub fn spawn(store: SharedStore) -> (Sender<Request>, JoinHandle<()>) {
    let (tx, rx) = mpsc::channel::<Request>();
    let handle = thread::spawn(move || run(&store, &rx));
    (tx, handle)
}

fn run(store: &SharedStore, rx: &Receiver<Request>) {
    for request in rx {
        let response = store.apply(&request.command);
        // Ha a kliens már nem vár a válaszra, a küldés hibáját figyelmen kívül hagyjuk.
        let _ = request.reply.send(response);
    }
}

/// Kliens oldali segéd: parancs küldése és a saját válasz megvárása.
pub fn call(tx: &Sender<Request>, command: Command) -> Result<Response> {
    let (reply, wait) = mpsc::channel();
    tx.send(Request { command, reply }).map_err(|_| closed())?;
    wait.recv().map_err(|_| closed())?
}
```

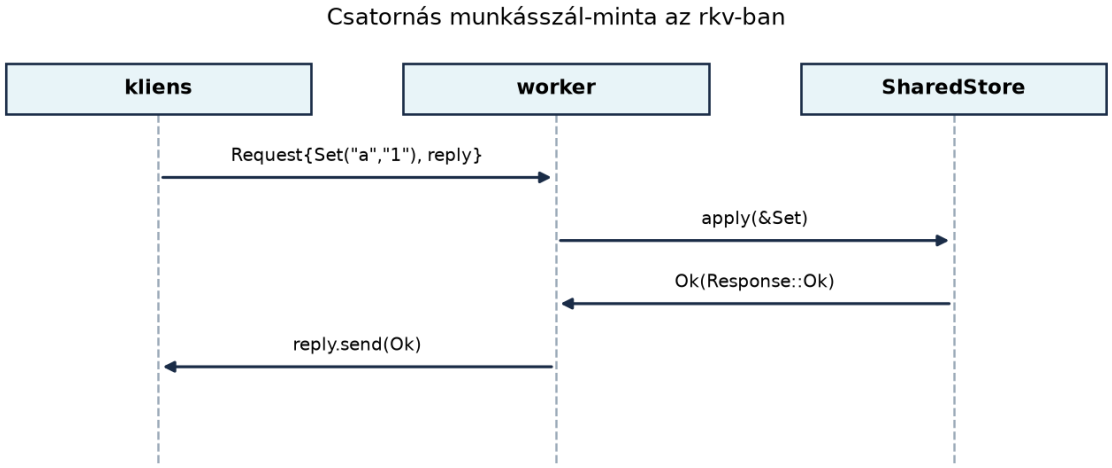
```
fn closed() -> Error {
    Error::Io(std::io::Error::new(
        std::io::ErrorKind::BrokenPipe,
        "a munkásszál leállt",
    ))
}
```

Kódblokk — fájl: `rkv-core/src/lib.rs` · művelet: beillesztés a `pub mod` sorok közé · ellenőrzés: `cargo build -p rkv-core`

```
pub mod worker;
```

A `spawn` elindítja a munkásszálat, és visszaadja a kéréseket fogadó `Sender<Request>`-et és a szál `JoinHandle`-jét. A `for request in rx` ciklus automatikusan leáll, ha az összes `Sender` eldobódik, ami a **szabályos leállítás**: a kliensek eldobják a `Sender`-eiket, majd a `join` megvárja a szálat. A `call` segéd egy kérést küld, és megvárja a saját válaszát; ha a munkásszál már leállt, `Error::Io` (`BrokenPipe`) a hiba. A munkásszál soros feldolgozása a `SharedStore` zárát nem teszi fölöslegessé (a többi kód közvetlenül is használhatja a tárolót), de a kliensek nem blokkolnak a záron.

A következő diagram a kliens és a munkásszál közötti üzenetváltást mutatja:



16.2. ábra: Csatornás munkásszál-minta az rkv-ban - A kliens a Request-ben a parancsot és a saját válaszcsatornáját küldi; a worker a SharedStore-on végrehajtja, majd a kérés válaszcsatornáján válaszol.

A hívó oldal használata egy futtatható példaként (a `Debug`-ot a `Response` deriválja; a `worker` modult a `lib.rs` `pub mod worker;` sora teszi elérhetővé):

Kódblokk — fájl: `rkv-core/examples/worker_demo.rs` · művelet: új fájl · ellenőrzés: `cargo run -p rkv-core --example worker_demo`

```
use rkv_core::worker::{call, spawn};
use rkv_core::{Command, MemStore, Response, SharedStore};

fn main() {
    let (tx, handle) = spawn(SharedStore::new(MemStore::new()));

    let set = call(&tx, Command::Set("nev".into(), "ertek".into())).unwrap();
    println!("SET -> {set:?}");
```

```
    if let Response::Value(bytes) = call(&tx, Command::Get("nev".into())).unwrap() {
        println!("GET -> {}", String::from_utf8_lossy(&bytes));
    }

    drop(tx);
    handle.join().unwrap();
}
```

Várt kimenet:

```
SET -> Ok
GET -> ertek
```

A teszt több klienssel ellenőrzi, hogy minden kliens a saját válaszát kapja, és hogy a leállítás után a `call` hibát ad:

Kódblokk — fájl: `rkv-core/tests/worker.rs` · művelet: új fájl · ellenőrzés: `cargo test -p rkv-core --test worker` → 2 sikeres teszt

```
use std::thread;

use rkv_core::worker::{call, spawn};
use rkv_core::{Command, MemStore, Response, SharedStore};

#[test]
fn worker_serves_many_clients_without_mixing_replies() {
    let (tx, handle) = spawn(SharedStore::new(MemStore::new()));

    let clients: Vec<_> = (0..4)
        .map(|i| {
            let tx = tx.clone();
            thread::spawn(move || {
                for j in 0..50 {
                    let key = format!("worker-{i}-{j}");
                    let reply = call(&tx, Command::Set(key.clone(), format!("v{j}"))).unwrap();
                    assert_eq!(reply, Response::Ok);
                    // Minden kliens a saját kérésére kapja a saját választ.
                    let reply = call(&tx, Command::Get(key)).unwrap();
                    assert_eq!(reply, Response::Value(format!("v{j}").into_bytes()));
                }
            })
        })
        .collect();
    for c in clients {
        c.join().unwrap();
    }

    match call(&tx, Command::List(Some("worker-".into()))).unwrap() {
        Response::Entries(items) => assert_eq!(items.len(), 200),
        other => panic!("váratlan válasz: {other:?}"),
    }

    drop(tx); // az utolsó Sender eldobása leállítja a munkásszálat
    handle.join().unwrap();
}

#[test]
fn call_fails_after_the_worker_has_stopped() {
    let (tx, handle) = spawn(SharedStore::new(MemStore::new()));
    let tx2 = tx.clone();
    drop(tx);
    drop(tx2); // minden Sender eldobva: a szál kilép
```

```
    handle.join().unwrap();
    // Új Sender már nem szerezhető; a leállást a hívó oldali csatornahiba jelzi.
    let (dead_tx, _rx) = std::sync::mpsc::channel();
    drop(_rx);
    let err = call(&dead_tx, Command::List(None)).unwrap_err();
    assert!(err.to_string().contains("leállt"));
}
```

16.2.5.5 5. lépés: futtatás

A módosítások után ellenőrizzük a teljes workspace-t:

```
cargo test --workspace
cargo clippy --workspace --all-targets -- -D warnings
```

Várt kimenet: minden teszt zöld, és nincs figyelmeztetés. (A `-D warnings` mellett is tiszta marad, mert a szándékosan mutatott Closure-ök: Fn, FnMut, FnOnce és a capture alfejezetbeli figyelmeztetést a tesztben `#[allow]` nyeli el.)

Ellenőrzési pont. Az `rkv-core` a `SharedStore`-t (`new`, `set`, `get`, `delete`, `list`, `apply`, `report`) és a `worker` modult (`Request`, `spawn`, `call`) exportálja; a `MemStore` és a `FileStore` is ugyanazzal a többszálú tesztsorral fut; a `cargo test --workspace` és a `clippy -D warnings` hibátlan.

Továbbvitt eredmény. A szálbiztos `SharedStore` (`rkv-core/src/shared.rs`) és a `Worker` csatornás mintája (`rkv-core/src/worker.rs`) alapot ad a Atomikus műveletek és memóriarendezés alfejezetnek (ott kap a `SharedStore` statisztikát), a hálózati szervernek (Az rkv hálózati szerver: TCP, protokoll, kecses leállás) és a Záróprojekt: az rkv teljes rendszerként alfejezetnek.

16.2.6 Tipikus hibák és átállási csapdák

1. **A guard referencia kiengedése a zárból.** Ha a `MutexGuard`-ra hivatkozó referenciát próbálunk kivinni a guard hatóköréből, a borrow checker hibázik. Például:

```
let guard = mutex.lock().unwrap();
let reference = &guard;
drop(guard);
println!("{}", *reference); // E0505: cannot move out of guard because it is borrowed
```

A hibaüzenet (E0505) arra figyelmeztet, hogy a guard eldobása után a referencia érvénytelen. A helyes megoldás, hogy a hozzáférést a guard hatókörén belül végezzük, vagy az adatot klónozzuk.

2. **Read és write lock keverése ugyanazon a záron.** A `RwLock` read guardja mellett nem lehet ugyanazon a szálon write guardot szerezni, mert az holtponthoz vezethet. Például:

```
let read_guard = rwlock.read().unwrap();
let write_guard = rwlock.write().unwrap(); // itt a szál örökre vár
```

A megoldás: vagy ne szerezzünk write-ot read alatt, vagy használjunk `try_write()`-ot, és ha kell, engedjük el a read guardot előtte.

3. **Poisoning elrejtése `unwrap()`-pel.** Ha egy szál pánikol a zár alatt, a `lock().unwrap()` pánikot dob, ami könnyen hibás hibakezeléshez vezet. A jobb megoldás a mérgezés hibává alakítása (`map_err`), vagy — ha a védett adat invariánsa igazoltan a pánik után is érvényes — a `unwrap_or_else(PoisonError::into_inner)` explicit, indokolt használata.

4. **Csatorna lezárás nem kezelése.** Ha az összes `Sender` eldobódik, a `recv()` `Err`-t ad. A `for` ciklus automatikusan leáll, de a kézi `recv()` hívásoknál a hiba elágazást kezelni kell, különben a program pánikolhat.

16.2.7 Feladatok

16.2.7.1 (a) Lefordul-e és miért?

Tekintsük a következő Rust kódot. Lefordul-e? Ha nem, mi a hibakód, és mi a baj?

```
use std::sync::{Arc, Mutex};

fn main() {
    let data = Arc::new(Mutex::new(vec![1, 2, 3]));
    let data_clone = Arc::clone(&data);

    let handle = std::thread::spawn(move || {
        let guard = data_clone.lock().unwrap();
        println!("{:?}", *guard);
    });

    let guard = data.lock().unwrap();
    println!("{:?}", *guard);
    handle.join().unwrap();
}
```

16.2.7.2 (b) C++ kód átírása idiomatikus Rustra

Az alábbi C++ osztály egy szálbiztos gyűjtőt valósít meg. Írd át Rustra `Arc<Mutex<Vec<T>>>` használatával úgy, hogy a `push` és a `get_all` metódusok szálbiztosak legyenek, és a típus `Clone`-olható legyen.

```
#include <mutex>
#include <vector>
#include <string>

class Collector {
    std::mutex mtx;
    std::vector<std::string> items;
public:
    void push(std::string item) {
        std::lock_guard<std::mutex> lock(mtx);
        items.push_back(std::move(item));
    }
    std::vector<std::string> get_all() {
        std::lock_guard<std::mutex> lock(mtx);
        return items;
    }
};
```

16.2.7.3 (c) Implementációs feladat az rkv-ban

Egészítsd ki az `rkv-core/tests/worker.rs` tesztet egy harmadik esettel: a munkásszál `Command::Delete` kérésére `Response::Ok`-t, a már nem létező kulcs ismételt törlésére `Response::NotFound`-ot adjon. A teszt több kliens szálat indítson, és minden kliens a *saját* kulcsait törölje; a végén a `Command::List` üres eredményt adjon.

16.2.7.4 (d) Hibajavítás és tervezési döntés

A következő kód holtpontot okozhat két `RwLock`-kal. Magyarázza el, miért, és adjon két lehetséges javítást: (1) zársorrend rögzítése, (2) `try_write` használata visszalépéssel.

```
use std::sync::{Arc, RwLock};

fn transfer(from: &RwLock<i32>, to: &RwLock<i32>) {
    let mut a = from.write().unwrap();
    let mut b = to.write().unwrap();
    *a -= 1;
    *b += 1;
}

fn main() {
    let a = Arc::new(RwLock::new(10));
    let b = Arc::new(RwLock::new(20));
    // két szál: transfer(&a, &b) és transfer(&b, &a)
}
```

16.2.8 Megoldások

16.2.8.1 (a)

A kód lefordul, de **holtpontba futhat**, tehát nem helyes. A főszál `guard` változója a `main` blokkjának végéig él: egy `MutexGuard` a hatókör végén dobódik el, nem az utolsó használatánál. Az NLL csak a kölcsönzések élettartamát rövidíti, a `Drop`-ot nem hozza előre. A rossz ütemezés:

1. A főszál előbb szerzi meg a zárat (`data.lock()`), és kiírja a vektort.
2. A munkásszál a `data_clone.lock()` hívásnál blokkol, mert a zárat a főszál tartja.
3. A főszál a `handle.join()`-nál a munkásszálra vár, miközben a `guard` még él.

Mindkét szál a másikra vár: holtpont. Ha a munkásszál kapja meg előbb a zárat, a program lefut, ezért a hiba csak időnként jelentkezik. A javítás: a főszál zárját a `join` előtt fel kell szabadítani, külön blokkal vagy explicit `drop(guard)`-dal:

```
use std::sync::{Arc, Mutex};

fn main() {
    let data = Arc::new(Mutex::new(vec![1, 2, 3]));
    let data_clone = Arc::clone(&data);

    let handle = std::thread::spawn(move || {
        let guard = data_clone.lock().unwrap();
        println!("{:?}", *guard);
    });

    {
        let guard = data.lock().unwrap();
        println!("{:?}", *guard);
    } // a guard itt dobódik el, a join előtt
    handle.join().unwrap();
}
```

A javított változat minden ütemezésben befejeződik; a kimenet sorrendje gépfüggő. A `Mutex` miatt nincs adatverseny. Az eredeti változat holtpontja tesztben időkorláttal reprodukálható: a `main` törzsét külön szálon futtatjuk, és a tesztszál `mpsc::Receiver::recv_timeout`-tal vár a befejezésre, így a tesztfuttató nem blokkol véglegesen.

16.2.8.2 (b)

```rust
use std::sync::{Arc, Mutex};

#[derive(Clone)]
pub struct Collector {
    items: Arc<Mutex<Vec<String>>>,
}

impl Collector {
    pub fn new() -> Self {
        Self {
            items: Arc::new(Mutex::new(Vec::new())),
        }
    }

    pub fn push(&self, item: String) {
        let mut guard = self.items.lock().unwrap();
        guard.push(item);
    }

    pub fn get_all(&self) -> Vec<String> {
        let guard = self.items.lock().unwrap();
        guard.clone()
    }
}
```

A `Clone` az `Arc` klónozásával megosztja a belső állapotot, így minden klón ugyanazt a gyűjtőt látja.

16.2.8.3 (c)

Kódblokk — fájl: `rkv-core/tests/worker.rs` · művelet: beillesztés a fájl végére · ellenőrzés: `cargo test -p rkv-core --test worker` → 3 sikeres teszt

```rust
#[test]
fn clients_delete_their_own_keys() {
    let (tx, handle) = spawn(SharedStore::new(MemStore::new()));

    let clients: Vec<_> = (0..4)
        .map(|i| {
            let tx = tx.clone();
            thread::spawn(move || {
                for j in 0..25 {
                    let key = format!("own-{i}-{j}");
                    call(&tx, Command::Set(key.clone(), "v".into())).unwrap();
                    assert_eq!(call(&tx, Command::Delete(key.clone())).unwrap(), Response::Ok);
                    // A második törlés már nem talál kulcsot.
                    assert_eq!(call(&tx, Command::Delete(key)).unwrap(), Response::NotFound);
                }
            })
        })
        .collect();
    for c in clients {
        c.join().unwrap();
    }

    assert_eq!(
        call(&tx, Command::List(None)).unwrap(),
        Response::Entries(Vec::new())
    );
```

```
    drop(tx);
    handle.join().unwrap();
}
```

Minden kérés a saját válaszcsatornáján kapja a választ, ezért a kliensek párhuzamos futása sem keveri össze a `Ok` / `NotFound` válaszokat, és a munkásszál sorosan dolgozza fel a kéréseket.

16.2.8.4 (d)

A holtpont oka: ha az egyik szál `from`-ként `a`-t, `to`-ként `b`-t zárol, a másik pedig fordítva, mindkettő megkapja az első zárat, és a másodikra vár; egyik sem engedi el az elsőt. Megoldás 1: minden szál rögzített sorrendben zár, például a `RwLock` pointerek címe szerint, vagy azonos sorrendű paraméterekkel. Megoldás 2: `try_write()` hívás, és ha sikertelen, az első zár eldobása után újra próbálkozás.

16.2.9 Rövid összefoglaló

A `Mutex<T>` és `RwLock<T>` a Rust szálbiztos belső mutabilitás alapjai: a zár birtokolja a védett adatot, a guard RAII-ja biztosítja a felszabadítást. A `Mutex<T>` `Sync`, ha `T: Send`; az `RwLock<T>` csak ha `T: Send + Sync`. A `std::sync::MutexGuard` nem `Send`, a tokio guardja `Send`. Az `mpsc` csatornák több termelő / egy fogyasztó üzenetküldést adnak. A laborban a Szálak, Send és Sync: data race fordítási időben kizárva alfejezetben bevezetett `SharedStore`-t (`rkv-core/src/shared.rs`) egészítettük ki `delete`, `list`, `apply` metódusokkal és a mérgezés kezelésével, és a `MemStore`-t és a `FileStore`-t egyaránt, közös többszálú tesztsorral ellenőriztük. A `worker` modul (`Request`, `spawn`, `call`) csatornás munkásszál-mintát valósít meg, kérésenkénti válaszcsatornával. **Továbbvitt eredmény:** a `SharedStore` és a `worker` a Atomikus műveletek és memóriarendezés, a Az rkv hálózati szerver és a Záróprojekt alfejezetek alapja.

16.2.10 Források és ellenőrzési hiányok

A `std::sync::Mutex`, `RwLock` és `mpsc` stabil API-ját a `std` dokumentációja írja le (https://doc.rust-lang.org/std/sync/). A `parking_lot` (`0.12`) és a `crossbeam` (`0.8`) crate-ek API-ja a docs.rs-en olvasható. A `RwLock` fairness-viselkedése platformfüggő; a `lock()`/`read()`/`write()` `unwrap()` mintája a poisoning kezelésére csak akkor biztonságos, ha a pánik utáni állapot nem okoz további inkonzisztenciát. A `Response` enum `#[derive(Debug)]`-ot kapott (Traitek és generikusok: sablonok és concepts helyett), ezért a `println!("{:?}")` használható. A hibaüzenetek szövege (pl. E0505) verziófüggő. A kódot és a kimeneteket a könyv rögzített toolchainjével (Rust 1.99.1) ellenőriztük.

16.3 Atomikus műveletek és memóriarendezés

16.3.1 Motiváció

C++-ban a `std::atomic<T>` és a `std::memory_order` évek óta ismerős, mégis a legtöbb kódban `memory_order_seq_cst` marad, mert „az a legerősebb”. Ez sokszor fölöslegesen drága, máskor pedig a `relaxed` használata vezet finom, csak ritkán jelentkező adatversenyhez. A Rust ugyanazt a memóriamodellt használja, mint a C++20, de az atomikus típusai a biztonságos Rust részei: a `fetch_add` és a `load` nem igényel `unsafe` blokkot. Ez nem jelenti azt, hogy a memóriarendezés kérdése eltűnik; a `Ordering` érték kiválasztása ugyanúgy tervezési döntés, mint C++-ban.

16.3.2 Fogalmak és szabályok

Definíció (atomikus típus). Az `AtomicBool`, `AtomicUsize`, `AtomicU64` és hasonló típusok olyan, belsőleg szinkronizált értékek, amelyeken az egyes műveletek megszakíthatatlanul hajtódnak végre, és a fordító a megadott `Ordering` korlátai szerint rendezheti át őket más memóriaműveletekkel. A Rustban ezek a `std::sync::atomic` modulban élnek.

Definíció (happens-before). Ha egy `Release` rendezésű tárolás szinkronizál egy `Acquire` rendezésű betöltéssel ugyanazon az atomikus objektumon, akkor a tárolás előtt végrehajtott összes írás *happens-before* a betöltés utáni összes olvasáshoz képest. A szinkronizáció feltétele, hogy az `Acquire` betöltés **pontosan azt az értéket figyelje meg, amelyet a `Release` tárolás írt** (vagy annak *release sequence*-éből egy későbbit); ha a betöltés még a régi értéket látja, nincs happens-before, és az olvasó nem tehet fel semmit az író korábbi írásairól. A happens-before reláció tranzitív, és ez teremti meg a több szál közötti megfigyelhető sorrendet.

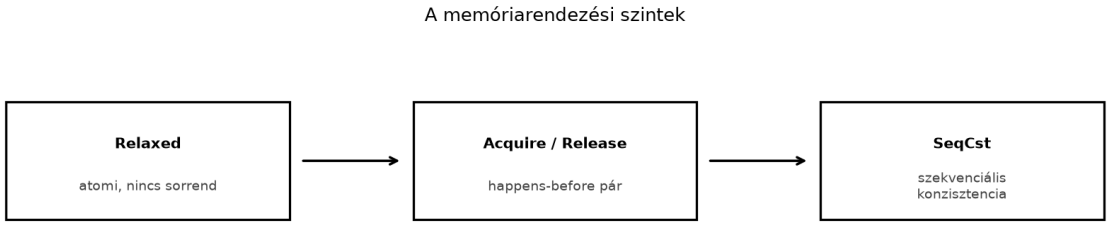
Adatverseny és versenyhelyzet. Ne keverjük össze a két fogalmat. Az *adatverseny* (data race) szinkronizálatlan, ütköző (legalább az egyik író) nem atomi memóriahozzáférés: C++-ban UB, safe Rustban a típusrendszer kizárja. A *versenyhelyzet* (race condition) logikai hiba, amelyben az eredmény az események sorrendjétől függ; safe Rustban atomikusokkal vagy zárakkal is előfordulhat. A fordító az atomikus *típusát* ellenőrzi, a `Ordering` helyességét nem: egy helyes típusú, de rossz rendezésű algoritmus (például `Relaxed` zászló egy másik adat publikálására) lefordul, és csak futás közben, ritkán hibázik, tehát nem fordításkor kizárt hiba.

Szabály. A Rust memóriamodell a C++20 memóriamodellt követi. A `std::sync::atomic::Ordering` értékei a következőképpen felelnek meg a C++ `std::memory_order` értékeinek:

Rust `Ordering`	C++ `memory_order`	Alkalmazható műveletek	Garancia
`Relaxed`	`memory_order_relaxed`	bármely	atomikusság, nincs sorrendi korlát
`Acquire`	`memory_order_acquire`	`load`	a load utáni műveletek nem mozognak elé
`Release`	`memory_order_release`	`store`	a store előtti műveletek nem mozognak mögé
`AcqRel`	`memory_order_acq_rel`	`fetch_add`, `compare_exchange` stb.	Release és Acquire együtt

A táblázat a tipikus párosítást mutatja. A `fetch_add` és a `compare_exchange` (read-modify-write) `Acquire`, `Release`, `AcqRel` és `Relaxed` rendezést is fogadhat; ilyenkor a művelet egyik fele (például `Release` RMW-nél a betöltés) `Relaxed` jellegű. A `compare_exchange` külön *failure ordering* paramétert kap (nem lehet `Release`/`AcqRel`). | `SeqCst` | `memory_order_seq_cst` | bármely | szekvenciális konzisztencia |

A C++ `memory_order_consume` megfelelője a stabil Rustban nincs; a C++-ban is kerülendő, mert a szemantikája a gyakorlatban bizonytalan.



16.3. ábra: A memóriarendezési szintek - A Relaxed növeléshez elég; az Acquire/Release pár szinkronizálja a függő adatokat; a SeqCst a legerősebb, de drágább.

A fenti ábra a három gyakorlatban használt rendezési szintet mutatja. A `Relaxed` csak azt garantálja, hogy a művelet atomikus, tehát például egy számláló növelése nem veszíti el a másik szál által ugyanabban a pillanatban elvégzett növelést. Az `Acquire`/`Release` pár már sorrendet teremt a tárolás előtti és a betöltés utáni műveletek között. A `SeqCst` minden szál számára egyetlen közös sorrendet állít fel, és ezért a legkönnyebben érvelhető, de gyakran fölöslegesen erős.

Szabály. Atomikus számlálóhoz, ahol csak a végső érték számít, a `Relaxed` rendezés elegendő. Ha az atomikus érték egy másik, nem atomikus adat láthatóságát jelzi, `Release` tárolást és `Acquire` betöltést kell párosítani.

16.3.3 C++ és Rust összevetés

A legegyszerűbb atomikus számláló C++-ban:

```cpp
#include <atomic>
#include <thread>
#include <cassert>

std::atomic<int> counter{0};

void increment(int n) {
    for (int i = 0; i < n; ++i) {
        counter.fetch_add(1, std::memory_order_relaxed);
    }
}

int main() {
    std::thread t1(increment, 10'000);
    std::thread t2(increment, 10'000);
    t1.join();
    t2.join();
    assert(counter.load(std::memory_order_relaxed) == 20'000);
}
```

A Rust megfelelő:

```rust
use std::sync::atomic::{AtomicUsize, Ordering};
use std::thread;

fn main() {
    let counter = AtomicUsize::new(0);

    thread::scope(|s| {
        s.spawn(|| {
```

```
            for _ in 0..10_000 {
                counter.fetch_add(1, Ordering::Relaxed);
            }
        });
        s.spawn(|| {
            for _ in 0..10_000 {
                counter.fetch_add(1, Ordering::Relaxed);
            }
        });
    });

    assert_eq!(counter.load(Ordering::Relaxed), 20_000);
}
```

A különbség itt nem a memóriamodell, hanem a nyelvi környezet. A Rust `atomic` műveletei biztonságosak, míg C++-ban az `std::atomic` használatával a programozó felel azért, hogy minden memóriaműveletet a megfelelő szinkronizáció védjen. A `thread::scope` a Rust-ban garantálja, hogy a szálak a scope végére befejeződnek, ezért nem kell `join()` hívás; a C++-ban a `std::jthread` hasonló célt szolgál.

16.3.4 Kidolgozott példa: Release és Acquire pár

A `Relaxed` rendezés önmagában nem teremt happens-before-t, ezért nem elegendő egy másik adat láthatóságának jelzésére. A következő kód lefordul, de a `Relaxed` rendezésű `ready` zászló miatt a `payload` értéke nem garantáltan 42:

```
use std::sync::atomic::{AtomicBool, AtomicUsize, Ordering};
use std::thread;

fn main() {
    let ready = AtomicBool::new(false);
    let payload = AtomicUsize::new(0);

    thread::scope(|s| {
        s.spawn(|| {
            payload.store(42, Ordering::Relaxed);
            ready.store(true, Ordering::Relaxed); // helytelen: Relaxed zászló
        });
        s.spawn(|| {
            while !ready.load(Ordering::Relaxed) {
                std::hint::spin_loop();
            }
            // Nem garantált, hogy a payload 42 látszik.
            assert_eq!(payload.load(Ordering::Relaxed), 42);
        });
    });
}
```

A hiba oka: a `ready` Relaxed tárolása és betöltése nem hoz létre happens-before-t, így a `payload` Relaxed írása a másik szál szerint a `ready` betöltése után is végrehajtódhat. A javítás:

```
use std::sync::atomic::{AtomicBool, AtomicUsize, Ordering};
use std::thread;

fn main() {
    let ready = AtomicBool::new(false);
    let payload = AtomicUsize::new(0);

    thread::scope(|s| {
        s.spawn(|| {
```

```
            payload.store(42, Ordering::Relaxed);
            ready.store(true, Ordering::Release); // Release
        });
        s.spawn(|| {
            while !ready.load(Ordering::Acquire) { // Acquire
                std::hint::spin_loop();
            }
            // Most már garantált: a Release-Acquire pár miatt
            // a payload.store happens-before a payload.load előtt.
            assert_eq!(payload.load(Ordering::Relaxed), 42);
        });
    });
}
```

Itt a `ready` Release tárolása szinkronizál az Acquire betöltéssel, ezért az író szál `payload.store`-ja happens-before az olvasó szál `payload.load`-jához képest. A `payload` maga maradhat Relaxed, mert a sorrendet a zászló garantálja.

A `compare_exchange` művelet akkor hasznos, ha egy atomikus értéket csak akkor akarunk megváltoztatni, ha az aktuális értéke egyezik egy elvárt értékkel. Két feladatot kell azonban szétválasztani: az **egyszeri munka kiválasztását** (ki inicializál) és a **befejezett inicializálás publikálását** (mikor láthatják mások az eredményt). A következő zászló az elsőt oldja meg:

```
use std::sync::atomic::{AtomicBool, Ordering};

static CLAIMED: AtomicBool = AtomicBool::new(false);

fn try_claim() -> bool {
    // Pontosan egy szál kap `true`-t: ő végzi az egyszeri munkát.
    CLAIMED.compare_exchange(false, true, Ordering::AcqRel, Ordering::Acquire).is_ok()
}
```

Ez a zászló **nem publikálja a kész inicializálást**: a többi szál `true`-t láthat, mielőtt a nyertes befejezte a munkát, tehát késznek jelölt, de még nem inicializált adatot olvashatna. Valódi egyszeri értékhez a standard könyvtár `OnceLock`-ja való, amely a befejezett inicializálást megfelelő `Release`/`Acquire` párral publikálja, és mindenkinek ugyanazt a kész értéket adja (ellenőrzött: több szálról is csak egyszer fut le az inicializáló):

```
use std::sync::OnceLock;

static CONFIG: OnceLock<String> = OnceLock::new();

fn config() -> &'static str {
    CONFIG.get_or_init(|| "alapérték".to_string()) // egyszer fut, a többiek megvárják az
↪   eredményt
}
```

Kézi állapotgép alternatívája három állapot (`0` = nincs, `1` = folyamatban, `2` = kész), ahol a végső `2`-t `Release` tárolással írja a nyertes az adat elkészülte után, a többiek pedig `Acquire` betöltéssel várják a `2`-t.

A `compare_exchange_weak` az `Ok` ágon megváltoztatja az értéket, ha az még a várt `current`; az `Err` ágon visszaadja az aktuális értéket, és a ciklus újrapróbálja. A gyenge változat néha akkor is hibázhat, ha az érték egyezik, ezért mindig ciklusban kell használni.

16.3.5 Labor: rkv statisztika-számlálók atomikusokkal

Munkakönyvtár: `~/rust-labor/rkv`

Az rkv szervernek hamarosan szüksége lesz műveleti statisztikákra, például arra, hogy hány `GET`, `SET`, `DEL` és `LIST` parancs érkezett. A `Mutex` túlzás lenne egyetlen számlálóhoz; atomikus típusokkal a `Stats` típus `Sync` lesz. A kiinduló állapot a Mutex, RwLock, csatornák és a megosztott rkv ellenőrzési pontja: a szálbiztos burkoló a **meglévő `SharedStore`** (`rkv-core/src/shared.rs`), amely bármely `Storage`-implementációt (`MemStore`, `FileStore`) burkol; a statisztikát ehhez adjuk hozzá, nem egy új tárolótípushoz.

1. Hozd létre az `rkv-core/src/stats.rs` fájlt:

```
use std::sync::atomic::{AtomicU64, Ordering};

#[derive(Debug, Default)]
pub struct Stats {
    get_count: AtomicU64,
    set_count: AtomicU64,
    delete_count: AtomicU64,
    list_count: AtomicU64,
}

impl Stats {
    pub(crate) fn record_get(&self) {
        self.get_count.fetch_add(1, Ordering::Relaxed);
    }

    pub(crate) fn record_set(&self) {
        self.set_count.fetch_add(1, Ordering::Relaxed);
    }

    pub(crate) fn record_delete(&self) {
        self.delete_count.fetch_add(1, Ordering::Relaxed);
    }

    pub(crate) fn record_list(&self) {
        self.list_count.fetch_add(1, Ordering::Relaxed);
    }

    pub fn get_count(&self) -> u64 {
        self.get_count.load(Ordering::Relaxed)
    }

    pub fn set_count(&self) -> u64 {
        self.set_count.load(Ordering::Relaxed)
    }

    pub fn delete_count(&self) -> u64 {
        self.delete_count.load(Ordering::Relaxed)
    }

    pub fn list_count(&self) -> u64 {
        self.list_count.load(Ordering::Relaxed)
    }
}
```

Itt a `Relaxed` rendezés elegendő, mert a számlálók önmagukban atomikusak, és nincs más adat, amelynek a láthatóságát a számlálóval kellene szinkronizálni.

2. Vedd fel a modult az `rkv-core/src/lib.rs`-ben:

Kódblokk — fájl: `rkv-core/src/lib.rs` · művelet: beillesztés a `pub mod` sorok közé és a `pub use` sorok után · ellenőrzés: `cargo build -p rkv-core`

```rust
pub mod stats;

pub use stats::Stats;
```

3. Add a `SharedStore`-hoz a `stats` mezőt, és minden művelet (a `set`, `get`, `delete`, `list` és az `apply`) növelje a megfelelő számlálót. A `Stats` `Arc`-ban van, mert a `SharedStore` klónjainak közösen kell számolniuk; a `stats()` getter egy `Arc<Stats>`-ot ad, amelyet a későbbi naplózó és lekérdező réteg megtarthat. A számláló a művelet *megkísérlésekor* nő (a sikertelen műveletek is számítanak), a zár megszerzése előtt, így a számlálás a zárat nem hosszabbítja. Mivel a `SharedStore` bármely háttérre épül, a `FileStore` is automatikusan statisztikát kap, külön migráció nélkül.

Kódblokk — fájl: `rkv-core/src/shared.rs` · művelet: teljes csere (a Mutex, RwLock, csatornák és a megosztott rkv alfejezetbeli változat a `stats` mezővel, a `stats()` getterrel és a számlálókkal bővítve) · ellenőrzés: `cargo build -p rkv-core`

```rust
use std::sync::{Arc, Mutex, MutexGuard};

use crate::{apply, Command, Error, Response, Result, Stats, Storage};

/// Szálak között megosztható tároló: bármely `Storage` implementációt burkol.
#[derive(Clone)]
pub struct SharedStore {
    inner: Arc<Mutex<Box<dyn Storage + Send>>>,
    stats: Arc<Stats>,
}

impl SharedStore {
    pub fn new(store: impl Storage + Send + 'static) -> Self {
        Self {
            inner: Arc::new(Mutex::new(Box::new(store))),
            stats: Arc::new(Stats::default()),
        }
    }

    /// A zár megszerzése. Ha egy korábbi szál pánikolt a zár alatt, a zár mérgezett.
    /// Az absztrakció alapértelmezése a *hiba*: tetszőleges `Storage` invariánsait
    /// egy pánik megsérthette (például egy félig lefutott, nem naplózott módosítás),
    /// ezért nem folytatunk automatikusan. Helyreállításhoz a hívónak új tárolót kell
    /// nyitnia (a `FileStore` a naplóból visszaépíti az állapotát).
    fn lock(&self) -> Result<MutexGuard<'_, Box<dyn Storage + Send>>> {
        self.inner.lock().map_err(|_| {
            Error::Io(std::io::Error::other(
                "a megosztott tároló mérgezett: egy szál pánikolt a zár alatt",
            ))
        })
    }

    /// A műveleti statisztika (a klónok közösen számolnak).
    pub fn stats(&self) -> Arc<Stats> {
        Arc::clone(&self.stats)
    }

    pub fn set(&self, key: String, value: Vec<u8>) -> Result<()> {
        self.stats.record_set();
        self.lock()?.set(key, value)
    }

    pub fn get(&self, key: &str) -> Result<Option<Vec<u8>>> {
        self.stats.record_get();
        self.lock()?.get(key)
```

```
    }

    pub fn delete(&self, key: &str) -> Result<bool> {
        self.stats.record_delete();
        self.lock()?.delete(key)
    }

    pub fn list(&self, prefix: Option<&str>) -> Result<Vec<(String, Vec<u8>)>> {
        self.stats.record_list();
        self.lock()?.list(prefix)
    }

    /// Egy parancs végrehajtása egyetlen zárfoglalással.
    pub fn apply(&self, command: &Command) -> Result<Response> {
        match command {
            Command::Get(_) => self.stats.record_get(),
            Command::Set(..) => self.stats.record_set(),
            Command::Delete(_) => self.stats.record_delete(),
            Command::List(_) => self.stats.record_list(),
        }
        let mut guard = self.lock()?;
        apply(&mut **guard, command)
    }

    /// A tárolt kulcsok száma és listája (egyetlen zárfoglalással).
    pub fn report(&self) -> Result<String> {
        let keys: Vec<String> = self.list(None)?.into_iter().map(|(k, _)| k).collect();
        Ok(format!("A tárolóban {} kulcs van: {:?}", keys.len(), keys))
    }
}
```

4. Futtasd a teszteket:

```
cd ~/rust-labor/rkv
cargo test -p rkv-core
```

Várt kimenet: a meglévő tesztek mellett az új statisztikai teszt is zöld:

Kódblokk — fájl: `rkv-core/tests/stats.rs` · művelet: új fájl · ellenőrzés: `cargo test -p rkv-core --test stats`

```
use std::thread;

use rkv_core::{Command, MemStore, SharedStore};

#[test]
fn stats_count_operations_across_clones_and_threads() {
    let store = SharedStore::new(MemStore::new());

    let handles: Vec<_> = (0..4)
        .map(|i| {
            let store = store.clone();
            thread::spawn(move || {
                for j in 0..250 {
                    store.set(format!("k{i}-{j}"), vec![1]).unwrap();
                }
            })
        })
        .collect();
    for h in handles {
        h.join().unwrap();
    }
```

```
    store.get("k0-0").unwrap();
    store.delete("k0-0").unwrap();
    store.list(None).unwrap();
    store.apply(&Command::Get("k1-1".into())).unwrap(); // az apply is számol

    let stats = store.stats();
    assert_eq!(stats.set_count(), 1000);
    assert_eq!(stats.get_count(), 2);
    assert_eq!(stats.delete_count(), 1);
    assert_eq!(stats.list_count(), 1);
}
```

A számlálók jelentése. A számlálókat a `SharedStore` metódusai növelik az eredmény *előtt*: a közvetlen `SharedStore::set/get` és az `apply` hívások is számítanak, a `report` belső `list`-je is növeli a `list_count`-ot. Ezért nem hálózaton érkezett parancsok száma és nem sikerességi metrika.

Továbbvitt eredmény. Az itt bevezetett `Stats` típus atomikus számlálókkal (`get_count`, `set_count`, `delete_count`, `list_count`) készül. Ezt a típust fogja használni a Naplózás és megfigyelhetőség: tracing alfejezet a műveleti metrikák kiírásához, és az Unsafe Rust: szuperképességek és felelősség alfejezetben is hivatkozunk rá, amikor az atomikus műveletek és a biztonsági határok közötti kapcsolatot vizsgáljuk.

16.3.6 Tipikus hibák és átállási csapdák

Minden atomikus műveletet `SeqCst`-tel írni. A `SeqCst` a legerősebb rendezés, és a legtöbb platformon drágább, mint a `Relaxed` vagy az `Acquire`/`Release`. Ha csak egy számlálót növelsz, a `Relaxed` elég; ha zászlóval szinkronizálsz, a `Release`/`Acquire` pár a megfelelő.

`Relaxed` használata függő adatokhoz. Amikor egy atomikus zászló jelzi, hogy egy másik adat készen áll, a zászlón `Release` tárolást és `Acquire` betöltést kell használni. A `Relaxed`-del az olvasó a zászlót késznek láthatja, miközben a másik adat írása még nem látszik: ez safe Rustban (ahol a másik adat is atomikus, mint a fenti példában) logikai versenyhelyzet, `unsafe`-fal megosztott nem atomi adatnál pedig adatverseny, azaz UB.

Atomikusokkal kiváltani a `Mutex`-et összetett invariánsoknál. Ha egy invariáns több mezőt vagy objektumot érint, a `Mutex` sokkal egyszerűbb és gyakran nem is lassabb. Az atomikus típusok akkor jók, ha az állapot egyetlen, jól elkülöníthető érték.

A `compare_exchange` hibás újrapróbálási ciklusa. A `compare_exchange_weak` esetén a `current` változót frissíteni kell az `Err` ágon, különben a következő próba ugyanazzal az elavult értékkel dolgozik.

16.3.7 Feladatok

1. **Fogalmi kérdés.** Milyen `Ordering`-et választanál egy olyan atomikus számlálóhoz, amelyet csak teljesítménymetrikák gyűjtésére használsz, és miért?
2. **C++-kód átírása.** A fenti C++ `std::atomic<int>` példát írd át idiomatikus Rustra úgy, hogy a szálak `thread::scope`-ot használjanak, és a számláló `Relaxed` rendezéssel növekedjen.
3. **Implementáció az rkv-ban.** Bővítsd a `Stats` típust egy `total()` metódussal, amely a négy számláló összegét adja, és írj tesztet, amely több szálból futtatott műveletek után a `SharedStore::stats().total()` értékét ellenőrzi. Gondold végig, miért nem garantálja

az összeg pontosságát egy másik szálban futó, közben történő kiolvasás (a `Relaxed` rendezés csak az egyes számlálók atomi voltát ígéri).

4. **Hibajavítás.** Az alábbi kódban a `ready` zászló `Relaxed`, és a `payload` atomikus `Relaxed` tárolása és betöltése nem látszik garantáltan. Javítsd úgy, hogy a `payload` értéke mindig helyesen látható legyen, és indokold, miért elegendő a `payload`-t `Relaxed`-nek hagyni.

```
use std::sync::atomic::{AtomicBool, AtomicUsize, Ordering};
use std::thread;

fn main() {
    let ready = AtomicBool::new(false);
    let payload = AtomicUsize::new(0);

    thread::scope(|s| {
        s.spawn(|| {
            payload.store(99, Ordering::Relaxed);
            ready.store(true, Ordering::Relaxed);
        });
        s.spawn(|| {
            while !ready.load(Ordering::Relaxed) {
                std::hint::spin_loop();
            }
            assert_eq!(payload.load(Ordering::Relaxed), 99);
        });
    });
}
```

16.3.8 Megoldások

1. A `Relaxed` rendezést érdemes választani, mert a számláló önmagában atomikus, és nem szinkronizál más adatokat. A `Relaxed` csak a megszakíthatatlan növelést garantálja, ami a metrika pontos összegéhez elég; a sorrendi korlátok hiánya itt nem jelent hibát.

2. A Rust változat:

```
use std::sync::atomic::{AtomicUsize, Ordering};
use std::thread;

fn main() {
    let counter = AtomicUsize::new(0);

    thread::scope(|s| {
        s.spawn(|| {
            for _ in 0..10_000 {
                counter.fetch_add(1, Ordering::Relaxed);
            }
        });
        s.spawn(|| {
            for _ in 0..10_000 {
                counter.fetch_add(1, Ordering::Relaxed);
            }
        });
    });

    assert_eq!(counter.load(Ordering::Relaxed), 20_000);
}
```

A `thread::scope` biztosítja, hogy a closure-ökből a `counter` kölcsönözhető legyen `'static` nélkül, és a scope végére mindkét szál befejeződik.

3. A `total()` metódus a négy getter összege:

```
impl Stats {
    /// A négy számláló összege. Nem pillanatkép: párhuzamos műveletek közben
    /// a négy `load` egymás után fut, ezért az összeg átmeneti állapotot tükrözhet.
    pub fn total(&self) -> u64 {
        self.get_count() + self.set_count() + self.delete_count() + self.list_count()
    }
}
```

A teszt: négy szál 250–250 `set`-et futtat, a szálak befejezése (`join`) után a fő szál `store.stats().total()` értéke pontosan 1000. A `join` happens-before kapcsolatot létesít a szálak és a fő szál között, ezért a `Relaxed` számlálók végső értéke ekkor már biztosan látszik. Egy *még futó* szál mellett kiolvasott összeg viszont nem pillanatkép: a négy atomi `load` közt a számlálók változhatnak, így az összeg átmeneti, soha nem létezett állapotot is mutathat. A `Relaxed` rendezés az egyes számlálók atomi voltát ígéri, nem a közöttük lévő összefüggést. A `Stats` mezői `AtomicU64` típust használnak, a `record_*` metódusok `fetch_add`-dal növelnek, a getterek `load`-dal olvasnak.

4. A javított kód:

```
use std::sync::atomic::{AtomicBool, AtomicUsize, Ordering};
use std::thread;

fn main() {
    let ready = AtomicBool::new(false);
    let payload = AtomicUsize::new(0);

    thread::scope(|s| {
        s.spawn(|| {
            payload.store(99, Ordering::Relaxed);
            ready.store(true, Ordering::Release);
        });
        s.spawn(|| {
            while !ready.load(Ordering::Acquire) {
                std::hint::spin_loop();
            }
            assert_eq!(payload.load(Ordering::Relaxed), 99);
        });
    });
}
```

A `ready` Release tárolása és Acquire betöltése happens-before-t hoz létre, így a `payload.store` a másik szál `payload.load`-ja előtt biztosan végrehajtódik. A `payload` azért maradhat `Relaxed`, mert a sorrendet már a zászló garantálja; a `payload` műveletei közötti szinkronizációt nem a saját rendezésük hozza létre.

16.3.9 Rövid összefoglaló

Az atomikus típusok (`AtomicUsize`, `AtomicBool`, `AtomicU64`) a Rust biztonságos nyelvi felületébe tartoznak, és a C++20 memóriamodelljével azonos szabályokat követnek. A `Relaxed` rendezés atomikus, de sorrendet nem garantál; az `Acquire`/`Release` pár happens-before-t teremt, és zászlóval történő adatszinkronizáláshoz való; a `SeqCst` a legerősebb, de sokszor fölösleges. A `fetch_add` és a `compare_exchange` a leggyakoribb atomikus RMW műveletek. A `Mutex` továbbra is a helyes választás összetett invariánsokhoz; az atomikusok egyetlen, jól elkülöníthető állapothoz hatékonyak. Az rkv laborban bevezettük a `Stats` típust az `rkv-core/src/stats.rs` fájlban: `get_count`, `set_count`, `delete_count` és `list_count` mezők `AtomicU64` típussal, valamint a

`Store` mezőjeként `Arc<Stats>`. A `Store` metódusai most már `record_*` hívásokkal frissítik a számlálókat, és a `stats()` getter elérhető a későbbi megfigyelhetőségi réteg számára.

16.3.10 Források és ellenőrzési hiányok

- Az atomikus típusok és a `Ordering` leírása: std::sync::atomic.
- Mara Bos, *Rust Atomics and Locks*: marabos.nl/atomics (a hivatkozás megnyitható).
- A C++ szabvány `[intro.races]` és `[atomics.order]` szakaszai tartalmazzák a memóriamodell formális leírását.
- A `thread::scope` a Rust 1.63 óta stabil.
- A `std::hint::spin_loop` stabilitását és az atomikus API-k stabilizálási állapotát a `std::sync::atomic` dokumentációjában érdemes megerősíteni.

16.4 Adatpárhuzamosság rayonnal

A párhuzamos programozás klasszikus C++-os útja az `std::thread`, az `std::async` vagy a C++17 `std::execution` policy. Mindegyiknél a programozó dönti el, hogyan osztja fel a munkát, hogyan csatlakozik a szálakhoz, és ami a legfontosabb: ő felel azért, hogy ne legyen data race. A Rust rayon könyvtár ezt a terhet jelentősen csökkenti: egy meglévő iterátorláncot gyakran egyetlen szó – `par_iter` – átírásával párhuzamosíthatunk, miközben a típusrendszer továbbra is garantálja a memóriabiztonságot.

16.4.1 Motiváció: a párhuzamos iterátorok hiánya a szabványos C++-ban

Tegyük fel, hogy az rkv snapshot-fájlja több tíz megabájtos, és minden mentésnél ellenőrzőösszeget akarunk számolni. A C++17 `std::transform_reduce` `std::execution::par` policyvel elvileg párhuzamosan végzi a map-reduce lépést, de a szabvány nem garantálja, hogy ténylegesen párhuzamos lesz: a policy csak *engedély*, nem parancs. A Rustban a rayon munkaellopásos (work-stealing) ütemezőt használ, de a tényleges párhuzamosítás itt sem szerződés: függ a pool méretétől (egyszálú pool is érvényes), a célplatformtól, az ütemezéstől és a munka méretétől, és a párhuzamos iterátorok a megszokott `Iterator` API-ra hasonlítanak, így a kód olvasható marad.

Az rkv esetében a korábbi Iterátorok: lusta adapterek és saját iterátor alfejezetben megírtuk a `Keys<'a>` iterátort, és láttuk, hogyan fűzhetők adapterek a lusta láncba. A rayon ugyanezt a szemléletet terjeszti ki több szálra, de a párhuzamos iterátoroknak szigorúbb követelményei vannak, mert az elemek feldolgozása más-más szálon történhet.

16.4.2 Fogalmak és szabályok

Definíció. A rayon *párhuzamos iterátor* (parallel iterator) a szekvenciális `Iterator`-hoz hasonló absztrakció, amely a gyűjtemény elemeit több szálon, felosztva dolgozza fel. A három fő belépési pont:

- `par_iter()` – párhuzamosan iterál kölcsönzött elemeken (`&T`),
- `par_iter_mut()` – párhuzamosan iterál változtatható kölcsönzéseken (`&mut T`),
- `into_par_iter()` – párhuzamosan fogyasztja el a gyűjteményt, tulajdonba véve az elemeket.

Szabály. A párhuzamos iterátorok minden closure-jének, amelyet map, filter, for_each vagy hasonló adapter kap, `Send` és `Sync` követelménynek kell megfelelnie. Ennek oka, hogy a rayon a munkát más szálakra költöztetheti, és a closure-ök által befogott értékeknek is át kell tudniuk lépni a szálhatárokat. A rayon ezt a `ParallelIterator` trait boundjaiban rögzíti: a closure `Fn + Send + Sync` kell, hogy legyen. Ez a C++-ban nem létezik: ott a programozónak kell ellenőriznie, hogy a lambda által használt adatok biztonságosak-e.

Definíció. A *munkaellopás* (work stealing) olyan párhuzamos ütemezési stratégia, amelyben minden szál rendelkezik egy saját feladatsorral; ha egy szál kifogy a munkából, más szálak sorából „lop” feladatokat. Ez kiegyensúlyozott terhelést ad akkor is, ha a feladatok mérete egyenetlen. A rayon alapértelmezett ütemezője munkaellopást használ.

Szabály. A rayon API **lehetővé teszi** a párhuzamos végrehajtást, de a tényleges hardveres párhuzamosságot a szálkészlet mérete és a terhelés dönti el: egyetlen szálra konfigurált készlettel (`ThreadPoolBuilder::new().num_threads(1)`) ugyanaz az API és ugyanaz az eredmény, szekvenciálisan fut (ellenőrzött: minden elem ugyanazon a szálon dolgozódik fel). A `par_iter().map(...).collect::<Vec<_>>()` megtartja az eredeti sorrendet, mert a `collect` gondoskodik a helyes pozícióba helyezésről, de a `for_each` mellékhatásainak sorrendje és a részeredmények kombinálásának **zárójelezése** az ütemezéstől függ.

Szabály (reduce). A `reduce(identity, op)` helyességéhez két feltétel kell: az `identity` az `op` semleges eleme, és az `op` **asszociatív**. Kommutativitás nem kell: a rayon a részeredményeket sorrendben kombinálja, ezért egy asszociatív, de nem kommutatív művelet (például szövegek összefűzése) is helyes eredményt ad (ellenőrzött: 1000 elem összefűzése azonos a szekvenciálissal). A nem asszociatív művelet (például kivonás) eredménye függ a zárójelezéstől, tehát az ütemezéstől.

Lebegőpontos összegzés. Az `f64` összeadás **nem asszociatív**: `((1e16 + 1.0) + -1e16) + 1.0 = 1`, míg `(1e16 + 1.0) + (-1e16 + 1.0) = 0`. A rayon `sum` részösszegeket kombinál, ezért a párhuzamos és a szekvenciális `f64` összeg az utolsó bitekben eltérhet (ellenőrzött: egymillió elemen `42822.895499999584` szemben `42822.895500000035`-tel, nem azonos bitek), és futásonként a szétosztás miatt is változhat. Ne ígérjünk bitpontos, sorrendfüggetlen float-eredményt; relatív hibahatárral összehasonlítsunk, vagy rögzített összeadási sorrendet használjunk.

Amdahl-törvény. Ha a program P hányada párhuzamosítható, a fennmaradó $1-P$ hányad pedig soros, akkor N szál esetén az elérhető maximális gyorsulás:

$$S = \frac{1}{(1-P) + \frac{P}{N}}.$$

Például ha a kód 90%-a párhuzamosítható ($P = 0{,}9$), akkor végtelen sok szálon is legfeljebb $1/(0{,}1) = 10$-szeres gyorsulás érhető el; ha csak 50% párhuzamosítható, a maximum 2-szeres. A valóságban a párhuzamosításnak fix overheadje is van (szinkronizáció, feladatütemezés, cache-koherencia), így kis munkák esetén a párhuzamos változat lassabb lehet a szekvenciálisnál.

16.4.3 C++ és Rust összevetés

A C++17 `std::transform_reduce` párhuzamos policyvel map-reduce szerűen összegezhetünk:

```
#include <execution>
#include <numeric>
#include <vector>

long long sum_of_squares(const std::vector<int>& v) {
```

```
    return std::transform_reduce(
        std::execution::par,
        v.begin(), v.end(),
        0LL,
        std::plus<>{},
        [](int x) { return static_cast<long long>(x) * x; }
    );
}
```

Rustban ugyanez a rayonnal:

```
use rayon::prelude::*;

fn sum_of_squares(v: &[i32]) -> i64 {
    v.par_iter()
        .map(|&x| i64::from(x) * i64::from(x))
        .sum()
}
```

A két kód felületesen hasonló, de a különbség lényeges. A C++-ban a `std::execution::par` nem garantál tényleges párhuzamosságot, és a lambda által befogott állapotra semmilyen típusbiztonsági ellenőrzés nincs; a data race-t elkerülő fegyelem a programozóé. A Rustban a `par_iter` closure-je csak akkor fordul, ha a befogott értékek `Send` és `Sync`, így a fordító kizárja a megosztott állapotból eredő adatversenyeket. Ráadásul a rayon munkaellopásos ütemezője garantáltan párhuzamosít, ha a munka mérete indokolja.

16.4.4 Kidolgozott példa és labor: párhuzamos snapshot-ellenőrzőösszeg az rkv-ban

Munkakönyvtár: `~/rust-labor/rkv/`

A labor célja, hogy snapshot-szerű bájtsorozatból párhuzamosan számoljunk ellenőrzőösszeget, és azt a `save_with_checksum` mentéskor oldalfájlba (`.sum`) el is tároljuk, a `verify_file` pedig visszaolvasáskor ellenőrizze. **Hatókör:** ez egy önálló segédmodul; a `FileStore::save_snapshot` és `open` a végleges projektben nem hívja (a `FileStore` épségét az atomikus csere és a JSON-értelmezés hibája védi), tehát az ellenőrzőösszeg itt külön, opcionális réteg. A snapshot bytesorozat, amelyet nagy méret esetén érdemes több szálon feldolgozni. Az rkv-core crate-hez hozzáadjuk a rayon függőséget, létrehozunk egy `checksum` modult, és példaprogrammal összehasonlítjuk a szekvenciális és a párhuzamos változatot.

A specifikáció. A checksum egy *kétszintű hash* (hash-lista, a Merkle-fa egyszerűsített alakja): (1) a bemenetet rögzített, `CHUNK_SIZE = 64 KiB` méretű darabokra osztjuk, és minden darabra FNV-1a hash-t számolunk; (2) a darabhash-eket **sorrendben** összefűzzük a teljes bemenet hosszával együtt, és az összefűzött bájtokra újra FNV-1a hash-t számolunk. A szekvenciális és a párhuzamos változat **ugyanazt a függvényt** számolja, azonos darabhatárokkal: csak az (1) lépés fut más módon (iterátorral, illetve párhuzamosan), a (2) lépés közös. Ezért az eredményük bitre azonos. Az FNV-1a nem kriptográfiai hash: véletlen sérülés (bitbillenés, csonka fájl) észlelésére való, szándékos módosítás ellen nem véd; ahhoz SHA-256 kell.

Miért nem XOR a kombináció? Kézenfekvő ötlet a darabhash-eket XOR-ral kombinálni (`reduce(|| 0, |a, b| a ^ b)`). Az XOR asszociatív és kommutatív, így a párhuzamos reduce sorrendje nem számít, de épp ezért *érzéketlen* a darabok sorrendjére és a duplikált darabpárokra (`b ^ b = 0`): két darab felcserélése, vagy egy darabpár kettőzése ugyanazt az értéket adná. Integritás-ellenőrzéshez ez nem elfogadható. A sorrendérzékenységet itt a rendezett összefűzés adja; a párhuzamosság ennek ellenére megmarad, mert a darabhash-ek számítása a költséges lépés, az összefűzés pedig csak a darabonként 8 bájtot dolgozza fel. A `collect` az indexelt párhuzamos iterátor elemeit az eredeti sorrendben gyűjti.

16.4.4.1 1. Függőség hozzáadása

Kódblokk — fájl: `rkv-core/Cargo.toml` · művelet: beillesztés a `[dependencies]` szakaszba · ellenőrzés: `cargo build -p rkv-core`

```
rayon = "1"
```

(A tesztek a már meglévő `tempfile` dev-függőséget használják.)

16.4.4.2 2. A `checksum` modul

Hozzuk létre az `rkv-core/src/checksum.rs` fájlt:

Kódblokk — fájl: `rkv-core/src/checksum.rs` · művelet: új fájl (az 5. lépés tesztmodulja kerül a végére) · ellenőrzés: `cargo build -p rkv-core`

```
use std::io;
use std::path::{Path, PathBuf};

use rayon::prelude::*;

/// A darabméret. A checksum definíciójának része: más darabméret más értéket ad.
pub const CHUNK_SIZE: usize = 64 * 1024;

/// FNV-1a, 64 bit: gyors, nem kriptográfiai hash. Véletlen sérülés
/// (bitbillenés, csonka fájl) észlelésére alkalmas. Szándékos módosítás ellen
/// semmilyen sima hash (SHA-256 sem) nem véd, ha a támadó az adatot és a mellette
/// tárolt összeget is átírhatja; ahhoz kulcsos hitelesítés (HMAC) vagy aláírás kell.
fn fnv1a(bytes: &[u8]) -> u64 {
    let mut hash = 0xcbf2_9ce4_8422_2325u64;
    for &b in bytes {
        hash ^= u64::from(b);
        hash = hash.wrapping_mul(0x100_0000_01b3);
    }
    hash
}

/// A gyökérhash: a teljes hossz és a darabhash-ek *sorrendben* összefűzve, újra FNV-1a-val.
/// Az összefűzés sorrend- és hosszérzékeny, ezért darabcsere, duplikált darab
/// vagy elhagyott darab a gyakorlatban megváltoztatja az eredményt (az XOR-kombináció
/// nem). Véges hashérték mellett azonban léteznek ütközések: a tesztek néhány konkrét
/// esetet igazolnak, nem általános garanciát.
fn combine(len: usize, chunk_hashes: &[u64]) -> u64 {
    let mut bytes = Vec::with_capacity(8 + chunk_hashes.len() * 8);
    bytes.extend_from_slice(&(len as u64).to_le_bytes());
    for hash in chunk_hashes {
        bytes.extend_from_slice(&hash.to_le_bytes());
    }
    fnv1a(&bytes)
}

/// Szekvenciális ellenőrzőösszeg: darabonkénti FNV-1a, majd `combine`.
pub fn sequential_checksum(data: &[u8]) -> u64 {
    let hashes: Vec<u64> = data.chunks(CHUNK_SIZE).map(fnv1a).collect();
    combine(data.len(), &hashes)
}

/// Párhuzamos ellenőrzőösszeg: a darabhash-eket párhuzamosan számítja, de a
/// `collect` megtartja a darabok sorrendjét, a `combine` pedig ugyanaz a
/// függvény: a két változat bitre azonos eredményt ad.
pub fn parallel_checksum(data: &[u8]) -> u64 {
```

```
    let hashes: Vec<u64> = data.par_chunks(CHUNK_SIZE).map(fnv1a).collect();
    combine(data.len(), &hashes)
}

fn sum_path(path: &Path) -> PathBuf {
    let mut name = path.as_os_str().to_owned();
    name.push(".sum");
    PathBuf::from(name)
}

/// Elmenti az adatot és mellé az ellenőrzőösszeget (`<fájl>.sum`, 16 hexa számjegy).
pub fn save_with_checksum(path: &Path, data: &[u8]) -> io::Result<u64> {
    let checksum = parallel_checksum(data);
    std::fs::write(path, data)?;
    std::fs::write(sum_path(path), format!("{checksum:016x}\n"))?;
    Ok(checksum)
}

/// Visszaolvasáskor újraszámolja az összeget; `Ok(false)`, ha eltér a mentettől.
pub fn verify_file(path: &Path) -> io::Result<bool> {
    let data = std::fs::read(path)?;
    let saved = std::fs::read_to_string(sum_path(path))?;
    let saved = u64::from_str_radix(saved.trim(), 16)
        .map_err(|e| io::Error::new(io::ErrorKind::InvalidData, e))?;
    Ok(parallel_checksum(&data) == saved)
}
```

A `par_chunks` a `ParallelSlice` trait metódusa, amely párhuzamosan iterál a szelet darabjain. A `map` minden darabra kiszámítja az FNV-1a hash-t, a `collect` pedig a darabok sorrendjében `Vec<u64>`-be gyűjti őket. A `save_with_checksum` az adat mellé egy `<fájl>.sum` fájlba menti az összeget, a `verify_file` visszaolvasáskor újraszámolja és összeveti; hiányzó vagy olvashatatlan összegfájl **hiba** (nem „rendben”). (A tartós, atomikus mentés lépései, az `fsync` és a könyvtár szinkronizálása, az I/O és szerializáció: fájlok, pufferelés, serde alfejezetben szerepelnek; itt az ellenőrzőösszegen van a hangsúly.)

16.4.4.3 3. Modul regisztrálása

Kódblokk — fájl: `rkv-core/src/lib.rs` · művelet: beillesztés a `pub mod` sorok közé · ellenőrzés: `cargo build -p rkv-core`

```
pub mod checksum;
```

16.4.4.4 4. Példaprogram a gyorsulás megfigyelésére

Hozzuk létre az `rkv-core/examples/checksum_bench.rs` fájlt:

Kódblokk — fájl: `rkv-core/examples/checksum_bench.rs` · művelet: új fájl · ellenőrzés: `cargo run --release -p rkv-core --example checksum_bench`

```
use std::time::Instant;

use rkv_core::checksum::{parallel_checksum, sequential_checksum};

fn main() {
    let data = vec![0xABu8; 64 * 1024 * 1024]; // 64 MiB

    let start = Instant::now();
    let seq = sequential_checksum(&data);
```

```
    let seq_time = start.elapsed();

    let start = Instant::now();
    let par = parallel_checksum(&data);
    let par_time = start.elapsed();

    println!("szekvenciális: {seq_time:?}, checksum = {seq:016x}");
    println!("párhuzamos:    {par_time:?}, checksum = {par:016x}");
    assert_eq!(seq, par);
}
```

A példa 64 MiB adatot generál, és mindkét függvényt lefuttatja; az `assert_eq!` a két eredmény azonosságát ellenőrzi. A kimenet gépfüggő; egy tipikus futás szerkezete (az időket egy gép egyetlen futása adta, illusztratívak, a checksum az adott bemenetre determinisztikus):

```
szekvenciális: 65.9ms, checksum = f5008e7ce99e7039
párhuzamos:    5.48ms, checksum = f5008e7ce99e7039
```

Nagyobb gépen a párhuzamos változat általában gyorsabb, de a különbség nem garantált, és kis adaton a thread pool felépítése miatt a párhuzamos változat lassabb is lehet. A mért időt mérési jegyzőkönyvben érdemes rögzíteni, de következtetést csak megismételhető benchmarkból szabad levonni; az Teljesítmény: mérés, profilozás, allokáció, release-beállítások alfejezetben a Criterion benchmark-crate-tel fogjuk ezt a függvényt mérni. **A benchmark előtt** az egyenértékűséget tesztek igazolják.

16.4.4.5 5. Tesztek: egyenértékűség és hibaészlelés

A teszteknek két kérdésre kell válaszolniuk: ugyanazt számolja-e a két változat (üres, egybájtos, a darabhatáron és azon túl átnyúló, véletlen bemeneten), és észreveszi-e a checksum a darabcserét, az elhagyott és a duplikált darabot, valamint a mentett fájl egyetlen bitnyi sérülését. Az `xor_combination_is_blind_to_duplicates` teszt az eredeti, XOR-os kombináció hibáját is megmutatja.

Kódblokk — fájl: `rkv-core/src/checksum.rs` · művelet: beillesztés a fájl végére · ellenőrzés: `cargo test -p rkv-core checksum` → 5 sikeres teszt

```
#[cfg(test)]
mod tests {
    use super::*;

    /// Determinisztikus álvéletlen bemenet (LCG), a külső crate nélkül.
    fn pseudo_random(len: usize, seed: u64) -> Vec<u8> {
        let mut state = seed;
        (0..len)
            .map(|_| {
                state =
↪   state.wrapping_mul(6364136223846793005).wrapping_add(1442695040888963407);
                (state >> 33) as u8
            })
            .collect()
    }

    #[test]
    fn both_impls_agree_on_boundaries_and_random_input() {
        let sizes = [0, 1, 2, CHUNK_SIZE - 1, CHUNK_SIZE, CHUNK_SIZE + 1, 3 * CHUNK_SIZE, 3 *
        ↪ CHUNK_SIZE + 17];
        for (i, &len) in sizes.iter().enumerate() {
            let data = pseudo_random(len, i as u64 + 1);
            assert_eq!(parallel_checksum(&data), sequential_checksum(&data), "len = {len}");
```

```
        }
    }

    #[test]
    fn empty_and_single_byte_differ() {
        assert_ne!(sequential_checksum(b""), sequential_checksum(b"\0"));
        assert_ne!(sequential_checksum(b"a"), sequential_checksum(b"b"));
    }

    #[test]
    fn swapped_duplicated_and_dropped_chunks_are_detected() {
        let a = vec![1u8; CHUNK_SIZE];
        let b = vec![2u8; CHUNK_SIZE];
        let c = vec![3u8; CHUNK_SIZE];
        let cat = |parts: &[&Vec<u8>]| parts.iter().flat_map(|p|
         ↪  p.iter().copied()).collect::<Vec<u8>>();

        let abc = sequential_checksum(&cat(&[&a, &b, &c]));
        // Darabcsere:
        assert_ne!(abc, sequential_checksum(&cat(&[&b, &a, &c])));
        // Elhagyott darab:
        assert_ne!(abc, sequential_checksum(&cat(&[&a, &c])));
        // Duplikált darabpár: az XOR-kombináció ezt nem látná (b ^ b == 0),
        // a sorrendérzékeny összefűzés igen.
        assert_ne!(sequential_checksum(&cat(&[&a, &b, &b, &c])), sequential_checksum(&cat(&[&a,
         ↪  &c])));
    }

    #[test]
    fn xor_combination_is_blind_to_duplicates() {
        // Az eredeti (hibás) kombináció demonstrációja: a duplikált pár kiesik.
        let xor_combine = |chunks: &[&[u8]]| chunks.iter().fold(0, |acc, c| acc ^ fnv1a(c));
        let (a, b, c) = ([1u8; 8], [2u8; 8], [3u8; 8]);
        assert_eq!(xor_combine(&[&a, &b, &b, &c]), xor_combine(&[&a, &c]));
    }

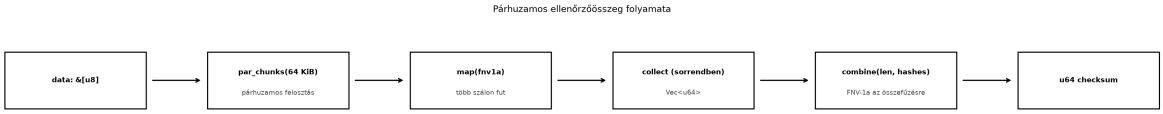
    #[test]
    fn saved_checksum_detects_corruption() {
        let dir = tempfile::tempdir().unwrap();
        let path = dir.path().join("snapshot.bin");
        let data = pseudo_random(2 * CHUNK_SIZE + 5, 99);
        let saved = save_with_checksum(&path, &data).unwrap();
        assert_eq!(saved, sequential_checksum(&data));
        assert!(verify_file(&path).unwrap());

        let mut corrupted = data.clone();
        corrupted[CHUNK_SIZE + 3] ^= 0x01; // egyetlen bit
        std::fs::write(&path, &corrupted).unwrap();
        assert!(!verify_file(&path).unwrap());

        std::fs::remove_file(sum_path(&path)).unwrap();
        assert!(verify_file(&path).is_err()); // hiányzó összeg: hiba, nem "ok"
    }
}
```

A `both_impls_agree_on_boundaries_and_random_input` a `CHUNK_SIZE` körüli hosszakat (`CHUNK_SIZE - 1`, `CHUNK_SIZE`, `CHUNK_SIZE + 1`) is lefedi, ahol a darabolás hibái jellemzően megjelennek.

16.4.4.6 6. A folyamat ábrája



16.4. ábra: Párhuzamos ellenőrzőösszeg folyamata - A bemeneti bájtsorozatot 64 KiB-os darabokra osztjuk; a darabhash-eket párhuzamosan számítjuk, sorrendben gyűjtjük, majd a hosszal együtt újra hash-eljük.

A párhuzamos iterátorlánc elemei ugyanazok, mint a szekvenciális `Iterator`-nál: egy adatforrás, egy leképezés és egy fogyasztó. A különbség az, hogy a `par_chunks` által előállított darabokat a rayon ütemezője szétosztja a worker szálak között, a `collect` pedig az elemeket az eredeti sorrendben gyűjti.

Ellenőrzési pont. Az `rkv-core` a `checksum` modult exportálja (`CHUNK_SIZE`, `sequential_checksum`, `parallel_checksum`, `save_with_checksum`, `verify_file`); a két checksum-változat azonos darabhatárokkal, azonos hash-listával és azonos kombinációs függvénnyel dolgozik, a mentett összeg visszaolvasáskor ellenőrződik. A `cargo test --workspace` az új teszteket is sikeresen futtatja.

Továbbvitt eredmény. Az itt létrehozott `checksum` modul `parallel_checksum` és `sequential_checksum` függvényeit a Teljesítmény: mérés, profilozás, allokáció, release-beállítások alfejezet fogja benchmarkolni Criterionnal, és ott döntjük el, hogy a snapshot-mentésben melyik változatot érdemes használni.

16.4.5 Tipikus hibák és átállási csapdák

1. A párhuzamosítás minden áron. A rayon párhuzamos iterátorai szálakat és feladatokat hoznak létre, ami fix overheaddel jár. Egy néhány elemből álló tömbön a `par_iter` gyakran lassabb, mint a szekvenciális `iter`. A párhuzamosítást csak akkor érdemes bevezetni, ha az elemenkénti munka elég nagy, vagy ha az elemszám elég nagy ahhoz, hogy a munkaellopásos ütemező nyeresége meghaladja az overheadet. A laborban ezért használtunk 64 MiB tesztadatot.

2. `Rc<RefCell<T>>` párhuzamos closure-ben. A `Rc` nem `Send` és nem `Sync`, mert a referenciaszámláló nem atomi. Ha egy `par_iter` closure megpróbálja *befogni*, a fordító hibát ad:

Kódblokk — önálló példa · **szándékosan hibás** · várt hiba: `E0277` (`Rc<RefCell<i32>>` cannot be shared between threads safely)

```
use rayon::prelude::*;
use std::cell::RefCell;
use std::rc::Rc;

fn main() {
    let shared = Rc::new(RefCell::new(0));
    let data = vec![1, 2, 3];
    data.par_iter().for_each(|x| {
        *shared.borrow_mut() += x; // Rc nem Send
    });
}
```

A hibaüzenet lényege: `Rc<RefCell<i32>>` cannot be shared between threads safely, mert a closure befogott `&Rc<...>` referenciája szálak közt osztozna, ehhez viszont az `Rc`-nek `Sync`-nek kellene lennie. A javítás az, ha valóban megosztott, szálbiztos mutációra van szükség, akkor `Arc<Mutex<T>>` vagy `Arc<AtomicI32>` használunk. A rayonnal azonban gyakran jobb megoldás a `reduce` vagy a `map` + `collect`, amely nem igényel megosztott állapotot.

3. Nem asszociatív kombináló művelet. Ha a `reduce` kombináló művelete nem asszociatív (például kivonás, vagy `f64` összeadás, amely csak közelítőleg az), a párhuzamos eredmény eltérhet a szekvenciálistól. Az asszociatív és kommutatív műveletek (XOR, egész összeadás) sorrendérzéketlenek, ami hash-kombinációnál hiba: a laborban ezért nem XOR-t használtunk (felcserélt vagy duplikált darabot nem venne észre). A kommutativitás hiánya önmagában nem hiba: egy asszociatív, nem kommutatív művelet a rayon `reduce`-ával is helyes. Sorrendfüggő eredményhez a darabonkénti részeredményeket explicit `collect`-tel gyűjtjük (ez megtartja a sorrendet), és utána sorrendben kombináljuk, ahogy a `checksum` modul teszi.

4. A `par_iter_mut` nem enged átfedő írást. A `par_iter_mut` párhuzamosan ad `&mut` referenciákat az elemekre, de a párhuzamos iterátor belsőleg felosztja a szeletet, így egyszerre soha nem ad két `&mut` referenciát ugyanarra az elemre. Ez azonban azt is jelenti, hogy a closure-ön belül nem férhetünk hozzá a gyűjtemény egészéhez; ha például egy `Vec`-be akarunk közben pusholni, az adatversenyhez vezetne, és a fordító megakadályozza. A megoldás a `collect` vagy a `rayon::prelude::ParallelIterator::collect` használata.

16.4.6 Feladatok

16.4.6.1 1. Fogalmi kérdés

Lefordul-e a következő két kód? Ha valamelyik nem, miért, és mi a javítás?

```rust
use rayon::prelude::*;
use std::rc::Rc;

fn main() {
    let owner = Rc::new(vec![1, 2, 3]);
    let sum: i32 = owner.par_iter().sum();
    println!("{sum}");
}
```

```rust
use rayon::prelude::*;
use std::rc::Rc;

fn main() {
    let owner = Rc::new(vec![1, 2, 3]);
    let sum: i32 = (0..3).into_par_iter().map(|i| owner[i]).sum();
    println!("{sum}");
}
```

16.4.6.2 2. C++-kód átírása idiomatikus Rustra

Az alábbi C++ kód párhuzamosan számolja ki egy `std::vector<double>` elemeinek átlagát (a `std::execution::par` feltételezett valós párhuzamossággal). Írd át Rustra a rayon használatával, és magyarázd el, hogy a Rust változat miért biztonságosabb.

```cpp
#include <execution>
#include <numeric>
#include <vector>

double average(const std::vector<double>& v) {
    double sum = std::reduce(std::execution::par, v.begin(), v.end(), 0.0);
    return sum / v.size();
}
```

16.4.6.3 3. Implementációs feladat az rkv-ban

Egészítsd ki az `rkv-core/src/checksum.rs`-t egy `parallel_max` függvénnyel, amely párhuzamosan megkeresi egy `&[u32]` maximális elemét. Használd a `par_iter` `max` metódusát, és írj tesztet, amely összehasonlítja a szekvenciális `iter().max()` eredményével. A függvény szignatúrája:

```rust
pub fn parallel_max(data: &[u32]) -> Option<u32>;
```

16.4.6.4 4. Refaktorálás vagy tervezési döntés

A snapshot-mentés jelenleg szekvenciálisan írja a fájlt. Tervezd meg, hogyan lehetne a `parallel_checksum` függvényt integrálni a mentési folyamatba anélkül, hogy a fájlírás párhuzamossá válna. Írd le a tervezett hívási helyet pszeudokódban vagy Rust-vázlatban, és indokold, miért érdemes a checksumot a fájl bezárása előtt vagy után számítani.

16.4.7 Megoldások

16.4.7.1 1. Fogalmi kérdés

Az **első kód lefordul**, és `6`-ot ír ki. A `owner.par_iter()` az `Rc` automatikus dereferálásával a `Vec<i32>` elemeire mutató szeletet (`&[i32]`) kölcsönöz, és *ez* kerül át a worker szálakra. Az `i32` `Sync`, tehát a szelet szálak közt megosztható; maga az `Rc` (a referenciaszámláló) a fő szálon marad, és semmilyen szálhatárt nem lép át. A **második kód nem fordul le** (`E0277`: `Rc<Vec<i32>>` cannot be shared between threads safely): a `map` closure befogja az `owner`-t, vagyis egy `&Rc<Vec<i32>>` referenciát, a closure-nek viszont `Send + Sync`-nek kell lennie. Itt tehát *az `Rc` maga* lépné át a szálhatárt, és a nem atomi számláló adatversenyes lehetne. Javítás: `Arc` használata (vagy a szelet kivétele a szálak indítása előtt):

```rust
use rayon::prelude::*;
use std::sync::Arc;

fn main() {
    let owner = Arc::new(vec![1, 2, 3]);
    let sum: i32 = (0..3).into_par_iter().map(|i| owner[i]).sum();
    println!("{sum}");
}
```

A tanulság: nem a gyűjtemény *típusa* (`Rc<Vec<_>>`), hanem az számít, hogy melyik érték kerül a párhuzamos closure-be.

16.4.7.2 2. C++-kód átírása

```rust
use rayon::prelude::*;

fn average(v: &[f64]) -> f64 {
    let sum: f64 = v.par_iter().sum();
    sum / v.len() as f64
}
```

A Rust változat nem engedi, hogy a `par_iter` closure-je megosztott állapotot adatversennyel módosítson; a `sum` egy fogyasztó, amely belsőleg kezeli a részösszegek kombinálását. Az `f64` összegnél a párhuzamos és a szekvenciális eredmény az utolsó bitekben eltérhet, mert a lebegőpontos összeadás nem asszociatív. A C++ `std::reduce` párhuzamos policyje esetén a programozónak kell biztosítania, hogy a művelet asszociatív legyen (a lebegőpontos `std::plus` csak közelítőleg az), és hogy a vektor elemeihez ne férjen hozzá másik szál közben. A Rust típusrendszer az utóbbit a `Send` és `Sync` boundokkal kényszeríti ki.

16.4.7.3 3. Implementációs feladat

```rust
use rayon::prelude::*;

pub fn parallel_max(data: &[u32]) -> Option<u32> {
    data.par_iter().copied().max()
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn max_matches_sequential() {
        let data = vec![3, 7, 1, 9, 2, 8, 5];
        assert_eq!(parallel_max(&data), data.iter().copied().max());
    }

    #[test]
    fn empty_input_yields_none() {
        assert_eq!(parallel_max(&[]), None);
    }
}
```

A `par_iter()` `&u32` elemeket ad, ezért a `copied()` adapterrel `u32` értékeket készítünk, amit a `max` fogyasztó összehasonlít.

16.4.7.4 4. Refaktorálás

A checksumot a mentés előtt a memóriabeli pufferből számoljuk, és a fájl mellé mentjük (`save_with_checksum`); ez a `checksum` modul része:

```rust
pub fn save_with_checksum(path: &Path, data: &[u8]) -> io::Result<u64> {
    let checksum = parallel_checksum(data); // a pufferből, párhuzamosan
    std::fs::write(path, data)?;            // a fájlírás sorosan marad
    std::fs::write(sum_path(path), format!("{checksum:016x}\n"))?;
    Ok(checksum)
}
```

A checksumot a fájl írása előtt a memóriabeli pufferből számoljuk, mert így nem kell a fájlt újra beolvasni, és a párhuzamos feldolgozás a `data` szeletet használhatja; a fájlírás közben nem változik az adat. Ha a fájl bezárása után számítanánk, újabb I/O-t igényelne, és a párhuzamosítás a háttértár sebességétől függne. A visszaolvasáskor a `verify_file` mindig újraszámol, és az eredményt a mentettel veti össze. A függvény `io::Result`-ot ad vissza, így a hibakezelés a hívónál marad. Az igazán tartós mentéshez (ideiglenes fájl, `sync_all`, `rename`, könyvtár-szinkronizálás) lásd az I/O és szerializáció: fájlok, pufferelés, serde alfejezetet.

16.4.8 Rövid összefoglaló

A rayon párhuzamos iterátorokkal (`par_iter`, `par_iter_mut`, `into_par_iter`) egy meglévő iterátorlánc egy sor módosításával párhuzamosítható, feltéve, hogy a closure-ök `Send` és `Sync`, és a kombináló művelet asszociatív. A munkaellopásos ütemező kiegyensúlyozza a terhelést, a `reduce` és `fold` pedig determinisztikus eredményt ad, ha a művelet kommutatív. A laborban az `rkv-core/src/checksum.rs` modulban létrehoztuk a `parallel_checksum` és `sequential_checksum` függvényeket, amelyek 64 KiB-os darabokra bontva darabonként FNV-1a hash-t számolnak, majd a darabhash-eket a bemenet hosszával együtt, sorrendben újra hash-elik (sorrend- és hosszérzékeny kombináció); a `save_with_checksum` és a `verify_file` a mentett összeget ellenőrzi visszaolvasáskor. A párhuzamos változatot a Teljesítmény: mérés, profilozás, allokáció, release-beállítások alfejezetben fogjuk benchmarkolni. A párhuzamosítás csak kellően nagy munkánál kifizetődő; az Amdahl-törvény és az overhead határt szab a gyorsulásnak.

16.4.9 Források és ellenőrzési hiányok

A nyelvi szabályok a Rust Reference és a *The Rust Programming Language* dokumentációján alapulnak. A rayon API-ja (`par_chunks`, `reduce`, `max`) a crate dokumentációjában található (https://docs.rs/rayon). A C++17 `std::execution` policy tényleges párhuzamossága implementációfüggő; az összevetés csak az API-szintű különbséget mutatja. A laborban használt rayon főverziója `1`, a pontos verziót a `Cargo.lock` rögzíti. A gyorsulási adatok gépfüggők, ezért a példában nem szerepelnek konkrét számok.

17 Aszinkron Rust

17.1 Future-ök és async/await: mi történik a motorháztető alatt

17.1.1 Motiváció

Ha C++-ban tíz TCP-kapcsolatot akarunk egyszerre kiszolgálni, a hagyományos megoldás gyakran tíz szál. Minden szál saját veremmel, saját ütemezési állapottal rendelkezik, és a kernel egy-egy blokkoló `read` hívás közben felfüggeszti. Ez tíz szál esetén még elfogadható; százezer kapcsolatnál már memória- és ütemezési katasztrófa. A C++20 bevezette a korutinokat, a Rust pedig a `Future`-alapú async/await modellt: a cél mindkét nyelvben az, hogy egy feladat felfüggesztéséhez ne kelljen külön szál, hanem egy kisméretű állapotgép tárolja, hol tart a művelet.

A Rust megközelítés lényege: az `async fn` hívása **nem hajtja végre** a függvény törzsét. Ehelyett egy olyan értéket ad vissza, amelyet egy úgynevezett **executor** polloz. Ez a fejezet azt mutatja meg, mi történik a motorháztető alatt: mit tartalmaz a `Future`, mi a `poll`, mi a `Waker`, és miért van szükség `Pin`-re.

17.1.2 Fogalmak és szabályok

Definíció. A `Future` trait egy olyan értéket ír le, amely egy később rendelkezésre álló eredményt képvisel. A szabványos definíció leegyszerűsítve:

```
pub trait Future {
    type Output;
    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>;
}

pub enum Poll<T> {
    Ready(T),
    Pending,
}
```

A `poll` hívás az egyetlen mód, ahogy a future haladást érhet el. A `Ready(value)` azt jelenti, hogy az eredmény elkészült, a `Pending` pedig azt, hogy a future még nem tud befejeződni; az executor később újra fogja hívni a `poll`-t.

Szabály. Az `async fn` hívása nem fut le azonnal: a hívás egy `impl Future<Output = T>` értéket hoz létre, amely a függvény állapotgépét tartalmazza. A függvény törzse csak akkor kezd el futni, amikor valaki meghívja a `poll` metódust.

A `Waker` szerepe: amikor egy future `Pending` értéket ad vissza, gondoskodnia kell arról, hogy valaki később értesítse az executort. Három modellt kell megkülönböztetni:

- **Levél-future** (például egy socket olvasása): a `poll` a `Context`-ből kapott `Waker`-t eltárolja az eseményforrásnál; amikor az esemény bekövetkezik (a socket olvashatóvá válik), a forrás meghívja a `Waker::wake()`-et, és az executor újra ütemezi a taskot. Minden `poll`-nál a *legutóbbi* wakert kell tárolni (a régit lecserélve, szükség esetén a `will_wake` összehasonlítással), mert az executor a taskot közben másik wakerrel is pollozhatja.
- **Összetett future** (az `async fn` állapotgépe): nem tárol saját wakert. A `.await` pontokon a generált `poll` ugyanazt a `cx`-et adja tovább a belső (child) future-nek, így a wakerfelelősség a levélig *delegálódik*; az executor ezért az összetett future mellé sem tárol külön wakert, csak a task `Waker`-ét adja át a `poll`-nak.
- **Már elindított task** (`tokio::spawn`): a runtime a taskot magától futtatja, az `await`-elt `JoinHandle` pedig egy külön future, amely akkor lesz kész, amikor a task befejeződik; a `JoinHandle` eldobása a taskot nem szakítja meg (leválasztja róla).

A `poll` szerződése: egy `Ready(v)` után a future-t többé nem szabad pollozni (az `async fn` állapotgépe ekkor pánikol: `async fn resumed after completion`), egy `Pending` után pedig csak akkor érdemes újrapollozni, ha a waker jelzett. A `Waker` tehát az a csatorna, amely lehetővé teszi, hogy a future ne pörögjön folyamatosan, hanem várjon.

Szabály. A Rust standard könyvtár **nem** tartalmaz beépített async runtime-et. Az `async fn` és a `Future` csak a nyelvi és könyvtári alapszint; az executor és a reaktor — az I/O eseményeket figyelő komponens — külső crate-ekből, jellemzően a Tokio-ról származik.

17.1.3 C++ és Rust összevetés

C++20-ban egy korutin vázlata:

```
#include <coroutine>
#include <iostream>

struct Task {
    struct promise_type {
        Task get_return_object() { return {}; }
        std::suspend_never initial_suspend() { return {}; }
        std::suspend_never final_suspend() noexcept { return {}; }
        void return_void() {}
        void unhandled_exception() {}
    };
};

Task compute() {
    co_await std::suspend_never{};
    std::cout << "hello from coroutine\n";
}
```

A Rust megfelelő gondolati párja:

```
async fn compute() {
    println!("hello from async fn");
}
```

A látszólagos hasonlóság mögött fontos különbségek vannak. A C++ korutinoknál a visszatérési típus `promise_type`-ja határozza meg a korutin viselkedését, és a felfüggesztés a `co_await` kifejezéssel történik. A Rust-ban az `async fn` egy `Future` értéket ad, és a felfüggesztési pont a `.await` posztfix művelet. A C++ korutin frame-je gyakran dinamikusan allokálódik, bár a fordító elideálhatja; a Rust async fn állapotgépe általában inline tárolódik a future értékben, és akár nulla allokációval is használható. A legfontosabb különbség a végrehajtási modell: a Rust future passzív, az executor húzza a `poll`-on keresztül; a C++ korutinok is külső resumálásra

várnak, de a nyelv szabadabban engedi a `coroutine_handle` közvetlen folytatását, és a `co_await` általánosabb felfüggesztési mechanizmust takar.

17.1.4 Kidolgozott példa: kézzel pollozott Future

Először nézzük a leggyakoribb kezdő hibát:

```
async fn greet() -> String {
    "hello".into()
}

fn main() {
    let s = greet();
    println!("{s}");
}
```

Ez a változat szándékosan nem fordul le: a `greet()` hívás eredménye `impl Future<Output = String>`, nem `String`, ezért a `println!` nem tudja formázni. A hibaüzenet lényege: „`greet()` returns `impl Future<Output = String>`, which does not implement `Display`”. A javítás egy executor és egy `.await` hozzáadása:

```
#[tokio::main]
async fn main() {
    let s = greet().await;
    println!("{s}");
}
```

Ahhoz, hogy lássuk, mit csinál az executor, implementáljunk egy nagyon egyszerű future-t. A következő `Remind` az első pollozásnál még `Pending`, a másodiknál `Ready` lesz, és közben értesíti a wakert:

```
use std::future::Future;
use std::pin::Pin;
use std::task::{Context, Poll, Waker};

struct Remind {
    ready: bool,
}

impl Future for Remind {
    type Output = &'static str;

    fn poll(mut self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<&'static str> {
        if self.ready {
            Poll::Ready("megvan!")
        } else {
            self.ready = true;
            // Értesítsük az executort, hogy érdemes újra pollozni.
            cx.waker().wake_by_ref();
            Poll::Pending
        }
    }
}

fn main() {
    let future = Remind { ready: false };
    let waker = Waker::noop();
    let mut cx = Context::from_waker(&waker);
    let mut pinned = Box::pin(future);
```

```
    match pinned.as_mut().poll(&mut cx) {
        Poll::Pending => println!("első poll: Pending"),
        Poll::Ready(s) => println!("első poll: Ready({s})"),
    }
    match pinned.as_mut().poll(&mut cx) {
        Poll::Pending => println!("második poll: Pending"),
        Poll::Ready(s) => println!("második poll: Ready({s})"),
    }
}
```

A `Waker::noop()` a standard könyvtárban stabil (a rögzített Rust 1.99.1-ben elérhető). A `poll` `Pin<&mut Self>` paramétert vár, de ehhez a `Box::pin` **nem szükséges**: a `Remind` csak egy `bool`-t tartalmaz, tehát `Unpin`, ezért a `Pin::new(&mut future)` is megfelel (lásd a következő szakaszt). A példa a `Box::pin`-t a kupacon tartott, könnyen továbbadható `Pin<Box<F>>` miatt használja. Várt kimenet:

```
első poll: Pending
második poll: Ready(megvan!)
```

A `Remind` példa megmutatja a modell lényegét: a future nem fut, amíg pollozva nincs; `Pending` esetén a wakerrel jelzi az újrapollozási igényét; a `Ready` pedig a végeredményt tartalmazza.

17.1.5 Mi van egy async fn állapotgépében?

Egy `async fn` lefordításakor a Rust fordító egy olyan struktúrát generál, amely a függvény lokális változóit tárolja, és egy `Future` implementációt ad hozzá. A `.await` pontoknál a generált `poll` meghívja a belső future `poll` metódusát; ha az `Pending`, akkor a külső future is `Pending`-et ad vissza. Ha a belső `Ready`, akkor a külső folytatja a következő utasítással.

Tekintsük ezt a vázlatot:

```
async fn example(a: u32) -> u32 {
    let x = a + 1;
    let y = read_value().await;
    x + y
}
```

A generált állapotgép nagyjából így gondolkodik:

1. **Állapot 0:** `x = a + 1`, hívja `read_value().poll(...)`.
2. Ha `read_value` `Ready(y)`, **állapot 1:** visszaadja `Ready(x + y)`.
3. Ha `read_value` `Pending`, a külső future is `Pending`-et ad vissza; a wakert nem ő tárolja: ugyanazt a `cx`-et adta tovább, így a belső (levél) future regisztrálta.

Az állapotgép tehát a `.await` pontokon felfüggeszti magát, és a `poll` legközelebbi hívásakor ott folytatja, ahol abbahagyta.

Miért nincs beépített runtime? A Rust standard könyvtár csak a `Future`, a `poll` és a `Waker` alapfogalmát definiálja. A valódi ütemezés — mikor, melyik szálon, hány future-t pollozzon az executor — tudatosan a felhasználóra van bízva. Ez lehetővé teszi, hogy különböző környezetben különböző executor fusson: operációs rendszeren (szerveren, asztali gépen) a Tokio, mikrovezérlőn pedig `no_std` executor, például az Embassy (lásd no_std, embedded-hal és Embassy). A Tokio operációs rendszert (szálakat, I/O-eseménykezelőt) feltételez, ezért nem mikrovezérlő-runtime. A C++-hoz hasonlóan a Rustban sincs „egyetlen igaz” async futtató, de a Tokio a de facto standard.

17.1.6 Pin: az önhivatkozó állapotgép problémája

Az `async fn`-ek egyik elsőre meglepő tulajdonsága, hogy miért kell `Pin`-t használni. A C++-ban megszokott mozgatási szemantikából tudjuk: egy objektum memóriabeli mozgatása után a rá mutató nyers címek érvénytelenné válnak. Ha egy `async fn` törzse egy `.await` pont felett tart életben egy referenciát, például:

```
async fn read_to_end(&mut self) -> std::io::Result<()> { // töredék: a `self` környezete nincs
↪  megadva
    while !self.buf_full() {
        let n = self.reader.read(&mut self.buf).await?;
        // ...
    }
    Ok(())
}
```

akkor a generált állapotgép önhivatkozóvá válhat: az egyik mezője referencia egy másik mezőjére mutat. Ha a future-t ezután megmozgatjuk, a referencia érvénytelen lesz. A `Pin<&mut Self>` azt garantálja, hogy a future nem mozdul el a pollozás során. A `Future::poll` ezért vesz `Pin<&mut Self>`-et, és a tipikus használat `Box::pin` vagy a `tokio::pin!` makró.

A `Pin` tehát nem misztikum: típusrendszerbeli garancia arra, hogy egy önhivatkozó értéket ne lehessen biztonságos kódból elmozdítani, miután pinelték. A pontos szerződés négy részből áll:

- **Pinelés előtt és után.** Pinelés előtt az érték szabadon mozgatható (átadható, `Vec`-be tehető). A `Pin<P>` létrehozása után az, amire a pointer mutat (a *pointee*), a drop-ig nem mozdulhat el, és a memóriája nem szabadítható fel, nem használható fel újra.
- **Pointer és pointee.** A `Pin` a *pointee* címét rögzíti, nem a pointerét. Egy `Pin<Box<F>>` ezért szabadon mozgatható (csak a `Box` pointer költözik), miközben a benne lévő `F` címe állandó. Egy `Pin<&mut F>` esetén az `F`-nek kell helyben maradnia.
- **Drop guarantee.** A pinelt pointee memóriája a `Drop::drop` lefutásáig érvényes marad, ezért az önhivatkozó mutatók a `drop`-ban is használhatók.
- **Safe és unsafe határ.** Az `Unpin` típusoknál (a legtöbb szokásos típus) a pinelés semmit nem korlátoz: a `Pin::new` biztonságosan hívható, és a `Pin<&mut T>`-ből `&mut T` nyerhető. `!Unpin` típust (például az `async fn` állapotgépét, vagy egy `PhantomPinned` mezőt tartalmazó structot) csak `Box::pin`-nel vagy a `pin!` makróval lehet pinelni; a mezőkhöz való hozzáférés (*projekció*) biztonságosan csak szerkezetileg nem pinelt mezőre lehetséges, a pinelt mezőre `unsafe` kell (`Pin::get_unchecked_mut`), amit a `pin-project` crate fed el biztonságos felülettel.

Az alábbi program a három tipikus esetet mutatja egymás mellett (kimenete: `ok`):

```
use std::future::Future;
use std::marker::PhantomPinned;
use std::pin::{pin, Pin};
use std::task::{Context, Poll, Waker};

/// Unpin: egyetlen bool mező, a Pin::new elég.
struct Remind {
    ready: bool,
}

impl Future for Remind {
    type Output = &'static str;
    fn poll(mut self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<&'static str> {
        if self.ready {
            Poll::Ready("megvan!")
        } else {
```

```
            self.ready = true;
            cx.waker().wake_by_ref();
            Poll::Pending
        }
    }
}

/// !Unpin: a PhantomPinned jelzi, hogy a pinelt érték címe nem változhat.
struct Fixed(PhantomPinned);

fn main() {
    let mut cx = Context::from_waker(Waker::noop());

    // 1. Unpin future: a Pin::new elég, nincs szükség kupacra.
    let mut remind = Remind { ready: false };
    let mut pinned = Pin::new(&mut remind);
    assert!(pinned.as_mut().poll(&mut cx).is_pending());
    assert!(pinned.as_mut().poll(&mut cx).is_ready());

    // 2. A pinelés *előtt* a future szabadon mozgatható.
    let fut = Remind { ready: false };
    let moved = fut;

    // 3. Pin<Box<F>>: a Box mozgatható, a benne lévő F címe állandó.
    let boxed: Pin<Box<Remind>> = Box::pin(moved);
    let address_before = &*boxed as *const Remind;
    let moved_box = boxed; // csak a pointer költözik
    assert_eq!(address_before, &*moved_box as *const Remind);

    // 4. pin!: a verem-lokális érték helyben pinelődik (Pin<&mut F>). A `Remind` `Unpin`
    //    (csak egy bool), ezért ebből a `Pin`-ből `&mut`-ot ki lehet nyerni; a
    ↪  mozgathatatlanság
    //    kikényszerítése a `!Unpin` típusnál (lásd 5.) számít.
    let local = pin!(Remind { ready: false });
    let _ = local;

    // 5. !Unpin típusra a Pin::new nem hívható; csak Box::pin vagy pin!.
    let _fixed = pin!(Fixed(PhantomPinned));
    println!("ok");
}
```

Ha az utolsó pontban `Pin::new(&mut fixed)`-et írnánk, a fordító `E0277` hibát ad (`PhantomPinned` cannot be unpinned). A `Box::pin` tehát nem „a `Pin` kötelező kelléke”, hanem egyike a pinelés módjainak: `Unpin` típusnál a `Pin::new`, verem-lokális `!Unpin` értéknél a `pin!`, kupacon élő és továbbadandó futurenél a `Box::pin` a megfelelő. A következő, Pin, Unpin, async trait-ek és a Send-határok alfejezet ezt mélyebben ki fogja fejteni.

17.1.7 Ábra: a Future életciklusa az executorban



17.1. ábra: Egy Future életciklusa - Az async fn hívása csak létrehozza a future-t; a futás pollozással kezdődik, Pending esetén a waker biztosítja az újrapollozást.

Az ábra lépései szövegesen:

1. **Hívás**: az `async fn` meghívása.
2. **Létrejön**: a `Future<Output = T>` létrejön, de még nem fut.
3. **poll**: az executor pollozza.
4. **Pending**: a future nem kész; regisztrálja a wakert.
5. **wake()**: esemény érkezik, a waker jelez az executornak.
6. **Újrapoll**: az executor újra pollozza.
7. **Ready(T)**: a future kész, az eredmény elérhető.

Az ábra azt hangsúlyozza, hogy a future életét az executor pollozása vezérli. A `Pending` nem hiba: a future ilyenkor adja át magát az ütemezőnek, és csak akkor jelentkezik újra, ha a waker jelzi, hogy a várt erőforrás elérhető.

17.1.8 Async vs. szálak: döntési tábla

Szempont	Async/Future	Szál
Sok rövid, I/O-igényes feladat	Kiváló: kis memórianyom, gyors váltás	Sok szál nagy veremköltséget jelent
CPU-igényes, hosszú számítás	Rossz, ha egy executor szálat blokkolja	Jó, ha a feladat tényleg párhuzamos
Egyszerű szinkron kód	`.await` fertőzés, executor kell	Egyszerű `thread::spawn`
Hibaelkülönítés	Egy pánik a taskot veszi le, de a runtime marad	A szál pánikja a szálat öli meg
Memória	Kis állapotgépek	2–8 MiB verem szálanként
Megfelelő Rust-megoldás	Tokio `spawn`	`std::thread` vagy rayon

A döntési szabály: ha a feladat döntően várakozás — hálózat, fájlművelet, időzítő —, az async jobb. Ha a feladat ténylegesen számol, használjunk szálat vagy a rayon crate-et, és ne blokkoljunk async taskot hosszú CPU-munkával.

17.1.9 Tipikus hibák és átállási csapdák

1. **Az `async fn` hívása `.await` nélkül.** A visszatérési érték `impl Future`, nem a várt `String` vagy `u32`. A fordító ilyenkor `Future` típusú értéket mutat, és a hibaüzenetben a `Display` hiánya vagy a típuseltérés jelenik meg.

2. **`.await` szinkron kódban.** A `.await` csak `async fn`-ben vagy `async` blokkban használható. A következő nem fordul le, a hiba lényege `E0728`:

```
fn main() {
    async { 1 + 1 }.await;
}
```

3. **Blokkoló hívás async taskban.** A `std::thread::sleep` vagy egy nagy `read_to_string` egy `async fn`-ben blokkolja az executor egy szálát, tönkretéve a többi task ütemezését. Helyette `tokio::time::sleep`-et és async I/O-t kell használni.

4. **A `Waker` elfelejtése.** Ha egy `Future` `Pending`-et ad, de nem regisztrálja a wakert, az executor nem fogja tudni, mikor érdemes újrapollozni; a future örökre felfüggesztve marad. Saját `Future` implementációban erre különösen figyelni kell.

17.1.10 Feladatok

17.1.10.1 1. Fogalmi ellenőrzés

Mit jelent a `Future::poll` `Pending` visszatérési értéke? Mi a különbség aközött, hogy egy `async fn` hívása `Ready`-vel tér vissza, és aközött, hogy a hívás maga `Ready` értéket ad?

17.1.10.2 2. C++-kód átírása

Írd át a következő C++20 `std::async` alapú kódot idiomatikus Rust async kódra Tokio `spawn` használatával:

```cpp
#include <future>
#include <iostream>

int compute() { return 40 + 2; }

int main() {
    auto fut = std::async(std::launch::async, compute);
    std::cout << fut.get() << "\n";
}
```

17.1.10.3 3. Implementáció: saját `DelayedReady` future

Készíts egy önálló crate-et a `~/rust-labor/r037/` munkakönyvtárban (`cargo new r037`). Implementálj egy `DelayedReady` struktúrát, amely az első `poll`-nál `Pending`-et ad, eltárolja a wakert, és egy külső `wake()` hívás után a második `poll`-nál `Ready(42)`-t ad vissza. A `main`-ben kézzel pollozd.

17.1.10.4 4. Hibajavítás: blokkoló hívás async taskban

Adott a következő async függvény:

```rust
async fn wait_a_second() {
    std::thread::sleep(std::time::Duration::from_secs(1));
}
```

Miért problémás ez a Tokio executor szempontjából? Javítsd `tokio::time::sleep` használatára, és magyarázd el a különbséget.

17.1.11 Megoldások

17.1.11.1 1. Fogalmi ellenőrzés

A `Pending` nem azt jelenti, hogy a future fut, hanem azt, hogy jelenleg nem tud befejeződni; a future-nek ilyenkor regisztrálnia kell a wakert, hogy az executor később újra pollozza. Amikor egy `async fn` **hívása** egy future-t ad vissza, az egy érték, amelynek a típusa `impl Future<Output = T>`; a függvény törzse még nem futott. A `Ready` a `poll` visszatérési értéke, és azt jelzi, hogy a future kész.

17.1.11.2 2. C++-kód átírása

```rust
#[tokio::main]
async fn main() {
    let fut = tokio::spawn(compute());
    let result = fut.await.unwrap();
    println!("{result}");
}

async fn compute() -> i32 {
    40 + 2
}
```

A `tokio::spawn` egy `JoinHandle`-t ad vissza, amely maga is `Future`. A `compute()` azonnal visszaad egy future-t, amelyet a Tokio runtime egy külön taskként ütemez. A `.await` a `JoinHandle`-on várja meg a task eredményét.

17.1.11.3 3. Implementáció: `DelayedReady`

```rust
use std::future::Future;
use std::pin::Pin;
use std::task::{Context, Poll, Waker};

struct DelayedReady {
    waker: Option<Waker>,
    woke: bool,
}

impl DelayedReady {
    fn wake(&mut self) {
        self.woke = true;
        if let Some(waker) = self.waker.take() {
            waker.wake();
        }
    }
}

impl Future for DelayedReady {
    type Output = i32;

    fn poll(mut self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<i32> {
        if self.woke {
            Poll::Ready(42)
        } else {
            self.waker = Some(cx.waker().clone());
            Poll::Pending
        }
    }
}

fn main() {
    let future = DelayedReady { waker: None, woke: false };
    let waker = Waker::noop();
    let mut cx = Context::from_waker(&waker);
    let mut pinned = Box::pin(future);

    assert!(pinned.as_mut().poll(&mut cx).is_pending());

    // Kívülről jelezzük a készenlétet.
    pinned.as_mut().get_mut().wake();
```

```
    // A Pin::get_mut kifejezetten `T: Unpin` feltételt kér: a DelayedReady (Option<Waker> +
    ↪ bool)
    // Unpin, ezért a safe get_mut hívható. Egy !Unpin (például önhivatkozó async) future-re
    // ez E0277 fordítási hiba lenne, így nem lehet véletlenül kimozgatni.

    match pinned.as_mut().poll(&mut cx) {
        Poll::Ready(v) => println!("ready: {v}"),
        Poll::Pending => println!("még mindig pending"),
    }
}
```

Ez a példa előkészíti a tokio taskok és a `JoinHandle` alfejezetet, ahol a valódi executor végzi el ugyanezt a pollozási és waker-kezelési feladatot.

17.1.11.4 4. Hibajavítás: blokkoló hívás

A `std::thread::sleep` blokkolja az aktuális szálat. A Tokio executor egy szálon több száz vagy több ezer async taskot futtat; ha egy task blokkolja a szálat, az összes többi task is megáll. Ráadásul a blokkolás alatt a timer vagy I/O események sem tudnak lefutni. A javítás:

```
async fn wait_a_second() {
    tokio::time::sleep(std::time::Duration::from_secs(1)).await;
}
```

A `tokio::time::sleep` egy olyan future-t ad, amely a Tokio timer-reaktorjában regisztrálja a wakert, és azonnal `Pending`-et ad; a szál így szabadon pollozhat más taskokat. Ez az alapja a Tokio: taskok, I/O, csatornák, select és megszakítás alfejezetnek.

17.1.12 Rövid összefoglaló

A `Future` trait a Rust aszinkron modelljének alapja: a `poll` metódus `Ready`-t vagy `Pending`-et ad, a `Waker` pedig lehetővé teszi az executor értesítését. Az `async fn` hívása egy `impl Future` állapotgépet hoz létre, de nem futtatja a függvényt; a futás csak a pollozással kezdődik. A standard könyvtár nem tartalmaz runtime-et, ezért a Tokio biztosítja az executort és a reaktort. A `Pin` az önhivatkozó async állapotgép mozgatását akadályozza meg. A C++20 korutinokkal szemben a Rust `Future` poll-alapú, passzív, és a `.await` a felfüggesztési pont. A döntési szabály: sok várakozó I/O-ra async, hosszú CPU-munkára szál vagy rayon. Ez a modell készíti elő a Pin, Unpin, async trait-ek és Send-határok, a Tokio taskok és csatornák, valamint az rkv hálózati szerver alfejezeteket.

17.1.13 Források és ellenőrzési hiányok

A `Future`, `Poll`, `Waker` és `Pin` fogalmai a Rust Reference és a standard könyvtár `std::future`/`std::task`/`std::pin` dokumentációján alapulnak (https://doc.rust-lang.org/std/). A `Waker::noop()` a rögzített Rust 1.99.1-ben stabil. A hibaüzenetek pontos szövege verziófüggő. A C++20 korutinok viselkedésének részletei, például az `initial_suspend` futtatási sorrendje, a C++ szabvány és a használt fordító dokumentációjában ellenőrizhetők.

17.2 Pin, Unpin, async trait-ek és a Send-határok

17.2.1 Motiváció: az önhivatkozó future-ök és a mozgatás problémája

Tegyük fel, hogy egy C++ osztály az *objektumon belül tárolt* mezőre mutató pointert tárol. Ha az objektumot áthelyezzük (move), a pointer a régi helyre mutat, hacsak a move konstruktor nem frissíti:

```
struct SelfRef {
    int data;
    int* p; // az objektumon belüli mezőre mutat
    explicit SelfRef(int x) : data{x}, p{&data} {}
};

SelfRef a{1};
SelfRef b = std::move(a);  // az implicit move bitenként másol: b.p még &a.data
// b.p != &b.data: elavult hivatkozás
```

Fontos a különbség egy *kupacpuffer birtokosa* és egy *objektumon belüli cím* között. Ha a `data` egy `std::vector<int>` lenne, és a `p` az elemére mutatna (`&data[0]`), a vektor move-ja a kupacpuffer címét **nem** változtatná meg (csak a birtokló struktúra költözik), ezért az ilyen `p` a move után is érvényes marad; az invalidálás ott a vektor növekedésekor (reallokáció) lép fel. Az önhivatkozás problémája az objektumon *belül* tárolt (inline) mezőre mutató cím, mert az az objektummal együtt elmozdul. A javított C++ osztály kézzel írt move konstruktora ezt frissíti:

```
SelfRefFixed(SelfRefFixed&& other) noexcept : data{other.data}, p{&data} {}
```

A Rust async/await modelljében a fordító által generált future-ök gyakran hasonlóan önhivatkozóak: a future egy belső állapotgépet tartalmaz, amelyben a helyi változók egymásra hivatkozhatnak. Ha ezt a future-t megmozgatnánk, a belső referenciák érvénytelenné válnának. A Rust ezt a `Pin` típussal és az `Unpin` automatikus traittel zárja ki.

17.2.2 Fogalmak és szabályok

Definíció. A `Pin<P>` egy olyan wrapper, ahol `P` pointer típus – leggyakrabban `&mut T` vagy `Box<T>`. Ha `T` **nem** `Unpin`, a `Pin<P>` garantálja, hogy az általa mutatott `T` érték attól kezdve, hogy pinned lett, **a pointee eldobásáig nem mozdul el** (a `Pin<&mut T>` fogantyú eldobása ezt a kötelezettséget nem oldja fel) a memóriában. A garancia azt jelenti, hogy a `Pin<&mut T>`-ből nem kaphatunk közvetlen `&mut T` hozzáférést, így nem hívhatunk olyan műveletet, amely a `T`-t mozgatná (például `mem::swap`, `mem::replace`).

Definíció. Az `Unpin` egy automatikus trait. Ha egy típus `Unpin`, akkor biztonságosan mozgatható pinned állapotban is. A legtöbb primitív és standard típus (`i32`, `String`, `Vec<T>`) `Unpin`. A generált async future-k általában **nem** `Unpin`, mert önhivatkozóak lehetnek. Ha `T: Unpin`, akkor `Pin<&mut T>` egyszerűen `&mut T`-re dereferálható.

Szabály. A `Future` trait `poll` metódusának aláírása:

```
fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>;
```

A `self: Pin<&mut Self>` azt jelenti, hogy a poll implementáció csak pinned referencián keresztül kapja meg a future-t. Ezért a jövőbeni `poll` hívások között a future nem mozgatható, és a belső önhivatkozások érvényesek maradnak.

A `Pin` létrehozásának leggyakoribb módjai:

- `Box::pin(future)` – heapre pin-neli a future-t, `Pin<Box<F>>` típust ad vissza. Ez akkor hasznos, ha a future-t hosszabb ideig szeretnénk tárolni, például egy `tokio::spawn`-hoz.
- `std::pin::pin!` makró – stackre pin-neli az értéket, `Pin<&mut T>`-t ad.
- `unsafe Pin::new_unchecked` – csak akkor, ha a fejlesztő garantálja, hogy a `T` (a `!Unpin` pointee) a **pointee eldobásáig** nem mozdul és nem érvényteleníthető; a `Pin` fogantyú eldobása nem szünteti meg a kötelezettséget.

A `Pin` projekció a pinned struktúra mezőinek elérését jelenti. Ha egy `!Unpin` típusba beágyazott mezőt szeretnénk elérni, nem kapunk közvetlen hozzáférést, mert a mező mozgatása sértheti a garanciát. Ezt `unsafe` blokkal vagy erre szolgáló crate-ekkel (például `pin-project`) lehet megoldani; a részletes projekció kívül esik a jelen alfejezet hatókörén.

17.2.3 C++ és Rust összevetés

C++-ban az önhivatkozó objektumok mozgatása a fejlesztő felelőssége, és a move konstruktorban kell frissíteni a belső mutatókat. Rustban ezt a kölcsönzés-ellenőrző és a `Pin` együttese szabályozza. Az async függvény által generált future egy olyan struktúra, amely a helyi változókat tartalmazza, és a köztük lévő kölcsönzéseket a generált kódban referencia-mezőkként reprezentálja. A Rust fordító ezt a future-t `!Unpin`-ként jelöli, így a poll-hoz csak `Pin<&mut Self>`-et használhatunk, ami kizárja a mozgatást.

```
async fn example() {
    let mut data = vec![1, 2, 3];
    let r = &data;
    tokio::time::sleep(std::time::Duration::from_millis(100)).await;
    println!("{:?}", r);
}
```

Ebben a példában a `r` referencia a `data`-ra mutat, és az `await` alatt is él. Ha a future-t megmozgatnánk, a `r` elavulna. A `Pin` garancia miatt ez nem történhet meg.

17.2.4 Kidolgozott példa: `Rc` a future-ben és a Send-határ

A `tokio::spawn` megköveteli, hogy a future `Send + 'static` legyen, mert a tokio más szálakra is mozgathatja a taskot. A future akkor `Send`, ha minden benne birtokolt érték `Send`. Az `Rc` nem `Send`, mert a referencia-számláló nem atomi. Ez a leggyakoribb hiba, amikor egy async blokkban `Rc`-t használunk, majd megpróbáljuk spawnolni.

Ez a változat szándékosan nem fordul le:

```
use std::rc::Rc;
use std::time::Duration;
use tokio::time::sleep;

async fn bad_future() {
    let data = Rc::new(vec![1, 2, 3]);
    sleep(Duration::from_millis(100)).await;
    println!("len = {}", data.len());
}

#[tokio::main]
async fn main() {
    tokio::spawn(bad_future());
}
```

A hiba lényege (a pontos szöveg verziófüggő): `future cannot be sent between threads safely`, és a diagnosztika megjelöli az `Rc`-t mint nem `Send` típust. A javítás: `std::sync::Arc` használata.

```
use std::sync::Arc;
use std::time::Duration;
use tokio::time::sleep;

async fn good_future() {
    let data = Arc::new(vec![1, 2, 3]);
    sleep(Duration::from_millis(100)).await;
    println!("len = {}", data.len());
}

#[tokio::main]
async fn main() {
    let handle = tokio::spawn(good_future());
    // A JoinHandle megvárása nélkül a main visszatérhetne a task befejezése előtt.
    if let Err(e) = handle.await {
        eprintln!("a task hibával állt le: {e}"); // JoinError: pánik vagy megszakítás
    }
}
```

Az `Arc` atomi számlálót használ, ezért `Send + Sync`, így a future megfelel a követelménynek. A `tokio::spawn` maga pin-neli a future-t a heapre, ezért nincs szükség explicit `Box::pin`-re. A `main` megvárja a `JoinHandle`-t, különben a runtime a program kilépésekor megszakíthatná a még futó taskot, és a példa futáskor nem írná ki a `len = 3` sort; a `JoinError` a task pánikját vagy megszakítását jelzi. (A tokio az operációs rendszeres szerver-környezet runtime-ja; mikrovezérlőn `no_std` executort, például az Embassyt használunk, lásd no_std, embedded-hal és Embassy.) Ha a futurében `Rc` marad, és mégis egy szálon akarjuk futtatni, a `tokio::task::LocalSet` és a `spawn_local` való, mert az nem követel `Send`-et.

Hasonló probléma lép fel, ha a **`std::sync::MutexGuard`**-ot tartjuk egy `.await`-en át. A `std` guard **nem `Send`** (a zárat azon a szálon kell elengedni, amelyik megszerezte), ezért az a future, amelyben az `.await`-en át él, nem `Send`, és a `tokio::spawn` elutasítja:

Kódblokk — önálló példa · **szándékosan hibás** · várt hiba: `future cannot be sent between threads safely` (`the trait Send is not implemented for std::sync::MutexGuard<'_, i32>`)

```
use std::sync::{Arc, Mutex};
use std::time::Duration;
use tokio::time::sleep;

async fn process(mutex: Arc<Mutex<i32>>) {
    let guard = mutex.lock().unwrap();
    sleep(Duration::from_millis(50)).await; // a std guard az await-en át él
    println!("value = {}", *guard);
}

#[tokio::main]
async fn main() {
    tokio::spawn(process(Arc::new(Mutex::new(1)))).await.unwrap();
}
```

A javítás: szűkítsük a lock élettartamát egy belső blokkra, hogy a guard az `.await` előtt eldobódjon (futtatva: `value = 1`):

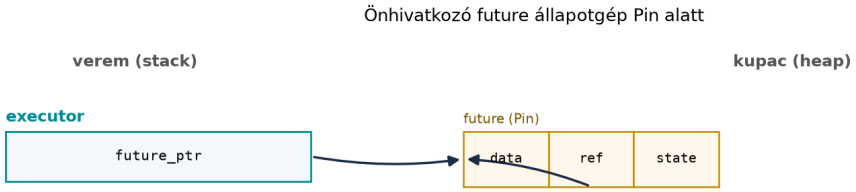
```
use std::sync::{Arc, Mutex};
use std::time::Duration;
use tokio::time::sleep;
```

```rust
async fn process(mutex: Arc<Mutex<i32>>) {
    {
        let guard = mutex.lock().unwrap();
        println!("value = {}", *guard);
    } // a guard az await előtt eldobódik
    sleep(Duration::from_millis(50)).await;
}

#[tokio::main]
async fn main() {
    tokio::spawn(process(Arc::new(Mutex::new(1)))).await.unwrap();
}
```

A `tokio::sync::MutexGuard` viszont `Send` (ha a védett típus `Send`) és `Sync` (ha `Send + Sync`): az aszinkron zár nincs szálhoz kötve, ezért az `.await`-en át tartott tokio guard **nem teszi** a future-t `!Send`-dé, a `tokio::spawn` elfogadja. Ez *típus szerint* megengedett, de nem ingyenes: a zár az egész várakozás alatt foglalt marad, a többi task blokkol rajta, és ha a várakozott művelet maga is ugyanezt a zárat kérné, holtpont keletkezik. Ezért a tokio guardot is csak akkor tartjuk `.await`-en át, ha a zár *szándékosan* védi a várakozás alatti állapotot (például egy kapcsolat kizárólagos írása).

17.2.5 Önhivatkozó future memóriaábra



17.2. ábra: Önhivatkozó future állapotgép Pin alatt - A future a heapen, Pin-nelt állapotban. A belső ref mező a saját data mezőre mutat; a Pin megakadályozza a future mozgatását, így a mutató érvényes marad.

Az ábra azt mutatja, hogy a future a heapre kerül, és a `Pin` miatt a címe rögzített. A future struct egyik mezője (`data`) a tényleges adatot tartalmazza, egy másik mezője (`ref`) a `data`-ra mutat. Ha a future-t megmozgatnánk, a `ref` elavulna; a `Pin` ezt akadályozza meg.

17.2.6 async fn traitekben és dyn-kompatibilitás

A Rust stabilizálta az async fn-eket trait-ekben (Async Fn in Trait, AFIT). Ez lehetővé teszi, hogy egy trait metódusa async legyen. Fontos azonban: a `dyn` trait objektum nem tud közvetlenül async metódust hívni, mert az async fn visszatérési típusa `impl Future<Output = ...>`, ami opak típus, és ezért nem dyn-kompatibilis. A dyn Trait-ekhez vagy az `async_trait` crate-et szokták használni (amely `Box<dyn Future + Send>`-t állít elő egy makróval), vagy enum-alapú dispatch-et. A stabilizálás pontos részletei és a dyn-kompatibilis async trait-ek esetleges későbbi kiterjesztései verziófüggőek, ezért a konkrét Rust-verzió dokumentációját ellenőrizni kell.

Az `async fn` trait-ekben a Rust 1.75-től stabil nyelvi képesség (nem az edition függvénye: edition 2021-ben is elérhető). Statikus generikus kódban használható (például `fn f<T: MyTrait>(t: T)`), de a `dyn MyTrait` objektumokon továbbra is korlátok vannak. Ezért a gyakorlatban a trait-objektumos async kódnál gyakran az `async_trait`-et alkalmazzák.

17.2.7 Tipikus hibák és átállási csapdák

1. **`Rc` a future-ben.** Ha Rc-t használunk egy async blokkban, a future `!Send` lesz, és `tokio::spawn`-nal nem futtatható. **Javítás:** használjunk `Arc`-ot, vagy ne tartsuk az `Rc`-t az `.await`-en át, ha lehetséges.
2. **`MutexGuard` tartása `.await` alatt.** A `std::sync::MutexGuard` nem `Send`; ha az async blokkban a guard az `await`-ig él, a future nem lesz `Send`, és a `tokio::spawn` elutasítja. A `tokio::sync::MutexGuard` `Send`, így a future `Send` marad, de a zár a várakozás alatt foglalt, ami teljesítmény- és holtpontkockázat. **Javítás:** zárjuk blokkba a kritikus szekciót, és a guardot az `await` előtt engedjük el.
3. **A `Pin` félreértelmezése.** A `Pin` nem a `T` belső változását tiltja, hanem a `T` mozgatását. Nem akadályozza meg például, hogy `&mut T`-n keresztül módosítsuk az adatot, ha a típus `Unpin`. Csak `!Unpin` típusoknál korlátozza a mozgató műveleteket.
4. **`Box::pin` és `pin!` helytelen választása.** Ha a future-t rövid ideig, egy adott scope-ban használjuk, a `pin!` elegendő; ha tárolni kell vagy vissza kell adni, akkor `Box::pin`. A `Pin<Box<F>>` `Send` lehet, ha `F: Send`, így spawnolható.

Továbbvitt eredmény. A `Send`-határok és a `Pin` ismerete alapvető lesz a Tokio: taskok, I/O, csatornák, select és megszakítás alfejezetben, ahol spawn-olt taskokat készítünk, és a Az rkv hálózati szerver alfejezetben, ahol a handler future-öknek `Send`-nek kell lenniük, és a megosztott állapotot `Arc<Mutex<...>>`-tal vagy `Arc<tokio::sync::RwLock<...>>`-kal oldjuk meg. A `Pin` garanciáját a `Future::poll` szignatúra betartásakor használjuk.

17.2.8 Feladatok

1. **Fogalmi kérdés.** Miért nem `Unpin` a legtöbb generált async future? Mi a különbség a `Pin` és az `Unpin` között?
2. **C++-ból Rustba.** Adott a fenti C++ `SelfRef` osztály, amelynek implicit move-ja nem frissíti a belső pointert (a javított változatban a kézzel írt move konstruktor igen). Magyarázd el, hogyan kerüli el a Rust ugyanezt a hibát a `Pin` segítségével az async future-ökben.
3. **Implementáció.** Írj egy async függvényt, amely `Rc<Vec<i32>>`-ot használ, és próbáld `tokio::spawn`-nal futtatni. Javítsd `Arc`-ra, és indokold, miért lesz a future `Send`.
4. **Hibajavítás.** Adott a következő kód:

```
use std::sync::{Arc, Mutex};
use tokio::time::{sleep, Duration};

async fn process(mutex: Arc<Mutex<i32>>) {
    let guard = mutex.lock().unwrap();
    sleep(Duration::from_millis(50)).await;
    println!("value = {}", *guard);
}
```

A `tokio::spawn(process(..))` nem fordul le. Miért? Módosítsd úgy, hogy a `guard` ne élje túl az `await`-et, és a future `Send` legyen. Mi változna, ha a `std::sync::Mutex` helyett a `tokio::sync::Mutex`-et használnád (`lock().await`)? Elég-e ez a csere a `Send` követelményhez, és mit nyersz vagy veszítesz vele?

5. **Tervezési döntés.** Az rkv hálózati szerverben minden klienshez egy task tartozik, és a `Store`-t több task is eléri. Milyen kombinációt választanál a megosztott `Store`-hoz, hogy a handler future-ök `Send`-ek legyenek? (Gondolj `Arc`, tokio `Mutex` vagy `RwLock` használatára, és a lock élettartamának minimalizálására.)

17.2.9 Megoldások

1. A generált async future azért nem `Unpin`, mert a fordító a helyi változókat és a köztük lévő referenciákat egy structba csomagolja; ha ezt a structot mozgatnánk, a belső referenciák elavulnának. Az `Unpin` automatikus trait, amely azt jelzi, hogy a típus pinned állapotban is mozgatható; a `Pin` pedig olyan wrapper, amely a `!Unpin` típusok mozgatását akadályozza meg.
2. C++-ban a move konstruktor felel a pointer frissítéséért; Rustban az async future-ök `!Unpin`-ek, és a `Future::poll` csak `Pin<&mut Self>`-et kap, így a future nem mozgatható az élettartama alatt. Ez garantálja, hogy a belső referenciák mindig érvényesek maradnak, anélkül hogy a fejlesztőnek manuálisan frissítenie kellene őket.
3. A kód:

```rust
use std::rc::Rc;
use std::time::Duration;
use tokio::time::sleep;

async fn bad() {
    let data = Rc::new(vec![1, 2, 3]);
    sleep(Duration::from_millis(100)).await;
    println!("{:?}", data);
}

#[tokio::main]
async fn main() {
    tokio::spawn(bad()); // hiba: future cannot be sent between threads safely
}
```

Javítás `Arc`-cal:

```rust
use std::sync::Arc;
use std::time::Duration;
use tokio::time::sleep;

async fn good() {
    let data = Arc::new(vec![1, 2, 3]);
    sleep(Duration::from_millis(100)).await;
    println!("{:?}", data);
}

#[tokio::main]
async fn main() {
    tokio::spawn(good());
}
```

4. A hiba oka: a `std::sync::MutexGuard` nem `Send`, és az `.await`-en át él, ezért a future nem `Send` (`tokio::spawn` elutasítja). A javított kód:

```rust
async fn process(mutex: Arc<Mutex<i32>>) {
    {
        let guard = mutex.lock().unwrap();
        println!("value = {}", *guard);
    } // a guard az await előtt eldobódik
```

```
    sleep(Duration::from_millis(50)).await;
}
```

A `tokio::sync::Mutex` (`lock().await`) önmagában is `Send` guardot ad, tehát a ***`Send`** követelményt* a csere megoldaná, a kritikus szakasz viszont továbbra is 50 ms-ig tartana: a többi task ennyit várna a zárra. Hosszú, `.await`-et tartalmazó kritikus szakaszhoz a tokio mutex való, rövid, `.await`-mentes szakaszhoz a (gyorsabb) `std` mutex a blokkba szűkített guarddal.

5. Az rkv szerverben a `Store`-hoz `Arc<tokio::sync::RwLock<Store>>` használata javasolt. A `RwLock` lehetővé teszi, hogy több olvasó egyszerre férjen hozzá, a tokio async lock nem blokkolja a szálat, és a `Store` így `Send + Sync` lesz. A lock-ot mindig minimalizáljuk: ne tartsuk `.await`-en át, és a kritikus szakasz után azonnal engedjük el.

17.2.10 Rövid összefoglaló

A `Pin` a Rust válasza az önhivatkozó future-ök mozgatásából eredő memóriabiztonsági problémára. A `Future::poll` szignatúrája `self: Pin<&mut Self>`, így a future nem mozgatható a poll hívások között. Az `Unpin` automatikus trait jelzi, hogy egy típus pinned állapotban is mozgatható; a generált async future-k általában `!Unpin`. `Box::pin` és a `pin!` makró pin-nelt future-öket hoz létre. A `tokio::spawn` megköveteli, hogy a future `Send + 'static` legyen; az `Rc` és a `tokio::sync::MutexGuard` nem `Send`, ezért helyettük `Arc`-ot és rövid élettartamú lock-okat használunk. Az async fn traitekben (AFIT) stabil, de dyn-kompatibilis használata korlátozott; ilyenkor `async_trait`-et vagy enum-dispatchet alkalmazunk. **Továbbvitt eredmény:** a Send-határok és a Pin ismerete elengedhetetlen a Tokio: taskok, I/O, csatornák, select és megszakítás és a Az rkv hálózati szerver alfejezetekhez, ahol spawn-olt taskok és megosztott állapotok kezelik ezeket.

17.2.11 Források és ellenőrzési hiányok

- Rust Reference: Pin
- The Rust Async Book: Pinning
- Tokio dokumentáció: Spawning
- Az async fn traitekben (AFIT) az 1.75-ös Rustban stabilizálódott; a dyn-kompatibilitás korlátai a Reference-ben olvashatók. A `pin!` makró a Rust 1.68 óta stabil. A `tokio::sync::MutexGuard` `Send`-tulajdonságát a tokio dokumentációja rögzíti; a `std::sync::MutexGuard` nem `Send`.

17.3 Tokio: taskok, I/O, csatornák, select és megszakítás

17.3.1 Motiváció: C++-os async-ból a tokio taskokig

A C++-fejlesztő a párhuzamosságot leggyakrabban `std::thread`, `std::async` és `std::future` segítségével kezeli. A szálak azonban drágák: minden szálhoz saját verem, kernel-ütemezés és gyakran rengeteg szinkronizáció tartozik. Ha több ezer egyidejű kérést kell kiszolgálni, a szálankénti blokkolás hamar skálázódási problémát okoz. A `std::async` elindít egy feladatot, de nem tudja garantálni, hogy az külön szálon, szálkészletben vagy éppen a hívó szálon fut-e; a `std::future` pedig blokkoló `.get()`-et ad, nem pedig valódi aszinkron kompozíciót. Az aszinkron I/O-hoz a C++-ban könyvtárakra (Boost.Asio, libuv) vagy platformfüggő `epoll`/`io_uring` kódra van szükség.

A Rust az aszinkronitást nyelvi szinten a `async`/`await` párossal kezeli, de a végrehajtáshoz szükség van egy runtime-ra. A **tokio** a legelterjedtebb ilyen runtime, amely taskokra, aszinkron I/O-ra, csatornákra és időzítőkre épít. Ez az alfejezet a tokio alapjait adja: taskok indítását, csatornákat, `select!`-et, időkorlátokat és a megszakítás-biztonságot. Ezt az alapot közvetlenül felhasználjuk az rkv hálózati szerverben, ahol a TCP-protokollt és a kecses leállást valósítjuk meg, valamint a naplózás és megfigyelhetőség alfejezetben is.

17.3.2 Async runtime, taskok és a `#[tokio::main]`

Definíció. A **runtime** (futtatókörnyezet) az a komponens, amely az aszinkron feladatokat (future-öket) ütemezi és futtatja. A **task** egy olyan, a runtime által ütemezett future, amely önállóan futhat a többi taskkal párhuzamosan. A **JoinHandle** egy olyan fogantyú, amely a task befejeződését várja, és opcionálisan annak eredményét adja vissza.

A tokio runtime-ot legegyszerűbben a `#[tokio::main]` attribútummal indíthatjuk el:

```
#[tokio::main]
async fn main() {
    println!("Hello, async!");
}
```

Ez az attribútum lényegében a következőre fordul:

```
fn main() {
    tokio::runtime::Runtime::new()
        .expect("hiba a runtime indításakor")
        .block_on(async {
            println!("Hello, async!");
        });
}
```

A `#[tokio::main]` alapértelmezetten **multi-threaded** runtime-ot hoz létre, amely egy szálkészleten futtatja a taskokat. Minden taskot a `tokio::spawn` függvénnyel indíthatunk:

```
let handle = tokio::spawn(async {
    // Task törzse
});
```

A `tokio::spawn` megköveteli, hogy a taskban lévő future `Send + 'static` legyen. A `'static` megkötés miatt a task nem tarthat referenciát például a hívó veremére; a megosztott állapotot `Arc`-cal vagy csatornákkal kell átadni. A `Send` megkötés garantálja, hogy a task biztonságosan átkerülhet egy másik szálra. Ha egy adat nem `Send`, akkor a task nem futtatható a tokio multi-threaded runtime-ján.

A `JoinHandle`-t `await`-elve megvárhatjuk a task befejezését:

```
let handle = tokio::spawn(async { 42 });
let result = handle.await.unwrap(); // 42
```

Itt az `unwrap` a task `JoinError`-ját kezeli, amely akkor keletkezik, ha a task pánikba fulladt vagy megszakadt. A Rust `std::thread::JoinHandle` analógiájára a tokio `JoinHandle` is a task „csatlakoztatására” szolgál, de nem blokkol, hanem egy future-t ad vissza.

17.3.3 C++ és Rust összevetés

A C++ `std::async` és a tokio `spawn` alapvetően más modellt követ.

```cpp
// C++: std::async feladat
#include <future>
#include <iostream>

int main() {
    auto fut = std::async(std::launch::async, [] {
        return 42;
    });
    std::cout << fut.get() << '\n';
}
```

```rust
// Rust: tokio::spawn task
#[tokio::main]
async fn main() {
    let handle = tokio::spawn(async {
        42
    });
    let result = handle.await.expect("a task pánikba fulladt");
    println!("{result}");
}
```

A különbség: a C++ `std::async` viselkedése a **launch policytől** függ. A `std::launch::async` külön szálon indítja a feladatot (és a visszaadott `future` destruktora vár a befejezésére); a `std::launch::deferred` késleltetve, az első `get()`/`wait()` hívó szálon futtatja; az alapértelmezett policy (`async | deferred`) implementációfüggő. A `get()` a hívó szálat blokkolja. A tokio `spawn` azonnal ütemezi a taskot, és egy `JoinHandle`-t ad vissza (ez maga is future); a task akkor is lefut, ha a handle-t nem `await`-eljük, a runtime saját ütemezője kezeli. A `tokio::spawn` `Send + 'static` future-t követel, mert a task bármely munkásszálra kerülhet; a `tokio::task::spawn_local` és a `LocalSet` ezzel szemben `!Send` futurekhez való, és ugyanazon a szálon futtat. a taskok száma nem feltétlenül egyezik a szálak számával, mert a tokio **work-stealing** módon osztja el a taskokat a szálkészleten.

A `std::promise`/`std::future` páros egyértelmű Rust-megfelelője a `oneshot` csatorna, amelyet lentebb tárgyalunk.

17.3.4 Async I/O: a blokkolás tilalma

Szabály. Az aszinkron task ne tartsa sokáig lefoglalva a runtime szálát `.await` pont nélkül. A gond a **hosszú vagy blokkoló** munka (például `std::thread::sleep`, `std::fs::read`, `std::net::TcpStream::read`, zárra várás, nagy CPU-számítás): ez az egész munkásszálat blokkolja, és megakadályozza a többi task ütemezését. Ezt a munkát izoláljuk: a `tokio::task::spawn_blocking` külön szálkészleten futtatja; aszinkron I/O-ra a `tokio::fs`, `tokio::net` és azok `async` metódusai valók. Ez nem a szinkron kód általános tiltása: a rövid, nem blokkoló szinkron kód (egy sor értelmezése, számolás) az async taskban is rendben van. Az rkv szerver `process_line` függvénye is szinkron, de mivel a `FileStore` fájlműveleteket végez (blokkolhat), a szerver `spawn_blocking`-ben hívja.

Példa a helyes I/O-ra:

```rust
use tokio::fs;

async fn read_config() -> std::io::Result<String> {
    fs::read_to_string("/etc/rkv/config.toml").await
}
```

Ha CPU-köteles vagy blokkoló szinkron I/O-műveletet kell végezni, használjunk `spawn_blocking`-et:

```
let result = tokio::task::spawn_blocking(|| {
    // Blokkoló művelet, pl. std::fs::read
    std::fs::read_to_string("/etc/rkv/config.toml")
})
.await
.expect("a blocking task pánikba fulladt")?;
```

A `spawn_blocking` egy külön szálkészleten futtatja a lezárt closure-t, így nem akadályozza az async feladatokat.

17.3.5 Async csatornák: mpsc, oneshot, broadcast

A tokio a következő async csatornákat nyújtja:

- **`mpsc`** (multi-producer, single-consumer): több küldő, egy fogadó. Ez a leggyakoribb munkamegosztási eszköz.
- **`oneshot`**: egyetlen érték egyszeri küldése. A C++ `std::promise`–`std::future` pár megfelelője.
- **`broadcast`**: több fogadó, mindenki ugyanazt az üzenetet kapja meg.

Példa `mpsc` csatornára:

```
use tokio::sync::mpsc;

let (tx, mut rx) = mpsc::channel::<i32>(8);
tx.send(10).await?;
let value = rx.recv().await.unwrap();
```

A csatornák alapértelmezetten **bounded** (korlátos) csatornák, ami a C++ `std::vector`/`std::queue` alapú saját implementációkhoz képest beépített **backpressure**-t ad: ha a puffer megtelik, a `send` future addig vár, amíg a fogadó fel nem szabadít helyet. Az `unbounded_channel` korlátlan puffert ad, de ezt ritkán érdemes használni, mert a termelő túlcsordulhat.

A `oneshot` csatorna a `std::promise` közvetlen Rust-megfelelője:

```
use tokio::sync::oneshot;

let (tx, rx) = oneshot::channel::<String>();
tx.send(String::from("hello")).unwrap();
let value = rx.await.unwrap();
```

A `broadcast` csatorna egyszerre több fogadónak juttatja el az üzenetet:

```
use tokio::sync::broadcast;

let (tx, _rx) = broadcast::channel::<i32>(16);
let mut rx1 = tx.subscribe();
let mut rx2 = tx.subscribe();
tx.send(1).unwrap();
assert_eq!(rx1.recv().await.unwrap(), 1);
assert_eq!(rx2.recv().await.unwrap(), 1);
```

17.3.6 `select!` és `timeout`

A `tokio::select!` makró lehetővé teszi, hogy több future-t versenyeztessünk: az elsőként befejeződő ág kiválasztódik, a többi future automatikusan dropolódik (megszakad). Ez a C++ `select`/`poll` rendszerhívások async megfelelője, de a C++-ban erre a nyelv szintjén nincs beépített megoldás.

A `timeout` függvény egy future-t és egy időkorlátot kapcsol össze:

```
use tokio::time::{timeout, Duration};

async fn slow_work() -> String {
    tokio::time::sleep(Duration::from_secs(5)).await;
    String::from("kész")
}

#[tokio::main]
async fn main() {
    let result = timeout(Duration::from_millis(100), slow_work()).await;
    match result {
        Ok(value) => println!("Időben kész: {value}"),
        Err(_) => println!("Időkorlát lejárt."),
    }
}
```

A `timeout` tulajdonképpen egy `select!`-re bontható:

```
tokio::select! {
    value = slow_work() => {
        println!("Időben kész: {value}");
    }
    _ = tokio::time::sleep(Duration::from_millis(100)) => {
        println!("Időkorlát lejárt.");
    }
}
```

Ez a példa egyben bemutatja a `select!` szintaxisát is. A `select!` képes a `Future` trait `poll` mechanizmusára építve hatékonyan várni több future-re anélkül, hogy szálanként blokkolnánk.

A `select!`-ben az egyes ágak teljes kifejezések, és minden ágban egy jövőbeli (future) kifejezés szerepel, amelynek eredményét mintázni lehet. A kiválasztott ág lefut, a többi ág future-je megszakad (dropolódik).

17.3.7 Megszakítás-biztonság (cancellation safety)

Definíció. Egy aszinkron művelet **megszakításbiztos** (cancel-safe) a tokio dokumentációja értelmében, ha az `.await` ponton való eldobása (például egy `select!` vesztes ágaként) után a dokumentált szerződése szerint nem vész el adat, és a művelet újrapróbálható vagy folytatható. Ez kétféle lehet: a művelet vagy teljesen elvégződött, vagy nincs megfigyelhető hatása; vagy a részleges haladás dokumentáltan a hívónál marad (például az `AsyncBufReadExt::read_until` a már beolvasott bájtokat a hívó `Vec`-jében hagyja, és új híváskor folytatható). **Ez nem általános visszagörgetési garancia.** A megszakítás nem vonja vissza a future *korábbi* mellékhatásait, és a megszakításbiztosság metódusonként dokumentált tulajdonság, nem a `select!` adománya. Mindig a konkrét metódus dokumentációjára támaszkodunk.

A tokio dokumentációja szerint például megszakításbiztos az `mpsc::Receiver::recv`, a `broadcast::Receiver::recv`, a `oneshot::Receiver` `await`-je, a `TcpListener::accept` és az `AsyncReadExt::read`; **nem** megszakításbiztos az `AsyncWriteExt::write_all` (részlegesen

írhat), az `AsyncReadExt::read_exact`, a `read_to_end` és minden olyan összetett `async` blokk, amely több egymást követő mellékhatást hajt végre. Az `mpsc::Sender::send` megszakításbiztos abban az értelemben, hogy megszakítás esetén az üzenet *nem lett elküldve*, de **az üzenet, amelyet a future birtokolt, elvész** (eldobódik).

A jellegzetes hiba tehát két alakban jelentkezik. (1) Az összetett blokk részleges sikere:

A részleges siker kontrolláltan előidézhető: a `tx2` csatornát előre megtöltjük, így a második küldés `Pending` lesz, és a timeout nyer. A blokkot `Ok::<(), SendError<i32>>(())`-val zárjuk, hogy a `?` típusa egyértelmű legyen (futtatható program, kimenete alább):

Kódblokk — önálló példa · **negatív viselkedési példa**: lefordul, és megmutatja az első sikeres, második megszakított küldést

```
use tokio::sync::mpsc::{self, error::SendError};
use tokio::time::{sleep, Duration};

#[tokio::main]
async fn main() {
    let (tx1, mut rx1) = mpsc::channel::<i32>(8);
    let (tx2, mut rx2) = mpsc::channel::<i32>(1);
    tx2.send(0).await.unwrap(); // a tx2 telített: a második küldés Pending lesz

    tokio::select! {
        r = async {
            tx1.send(42).await?;
            tx2.send(24).await?; // itt vár, amíg a timeout nem nyer
            Ok::<(), SendError<i32>>(())
        } => println!("kész: {r:?}"),
        _ = sleep(Duration::from_millis(50)) => println!("időkorlát"),
    }

    println!("rx1: {:?}", rx1.try_recv()); // Ok(42): az első küldés megtörtént
    println!("rx2: {:?}", rx2.recv().await); // Some(0): csak a régi elem
    println!("rx2 tovább: {:?}", rx2.try_recv()); // Err(Empty): a 24 sosem érkezett meg
}
```

Kimenet:

```
időkorlát
rx1: Ok(42)
rx2: Some(0)
rx2 tovább: Err(Empty)
```

Az `rx1` megkapta a `42`-t, az `rx2` viszont soha nem kapja meg a `24`-et: a két csatorna állapota eltér. (2) Tulajdonolt érték elvesztése telített csatornán. Egy bounded `mpsc` csatorna megtelik, a `send` vár, és a `timeout` a vesztes future-rel együtt a küldendő értéket is eldobja:

```
let (tx, mut rx) = mpsc::channel::<String>(1);
tx.send("elso".into()).await.unwrap(); // a csatorna megtelt
let r = timeout(Duration::from_millis(30), tx.send("masodik".into())).await;
assert!(r.is_err());                   // Elapsed
assert_eq!(rx.recv().await.as_deref(), Some("elso"));
assert!(rx.try_recv().is_err());       // a "masodik" elveszett
```

A megoldás az, hogy a tulajdon a megszakítás után is a miénk marad. Két mód van rá, mindkettő futtatható teszttel ellenőrzött. A `send_timeout` az értéket hibában visszaadja:

```
let err = tx.send_timeout("masodik".into(), Duration::from_millis(30)).await.unwrap_err();
match err {
    mpsc::error::SendTimeoutError::Timeout(v) => assert_eq!(v, "masodik"), // megmaradt
```

```
    other => panic!("{other:?}"),
}
```

A `reserve` pedig először csak helyet foglal (egy `Permit`-et), és az értéket csak az sikeres foglalás után, szinkron küldi el; a foglalás megszakítása nem veszít adatot:

```
let value = String::from("masodik");
tokio::select! {
    permit = tx.reserve() => { permit.unwrap().send(value); }
    _ = sleep(Duration::from_millis(30)) => {
        // timeout: a value továbbra is a miénk, újrapróbálható vagy naplózható
        assert_eq!(value, "masodik");
    }
}
```

Külön eset a **`broadcast`** csatorna lassú fogadója. A küldő soha nem blokkol; ha a fogadó lemarad, és a puffer (kapacitás) megtelik, a legrégebbi üzenetek felülíródnak, a fogadó pedig a következő `recv`-nél `Err(RecvError::Lagged(n))` hibát kap, ahol `n` az elveszett üzenetek száma, majd folytatja a megmaradtakkal:

```
let (tx, mut rx) = broadcast::channel::<i32>(2);
for i in 0..4 { tx.send(i).unwrap(); }
assert_eq!(rx.recv().await, Err(broadcast::error::RecvError::Lagged(2)));
assert_eq!(rx.recv().await, Ok(2));
assert_eq!(rx.recv().await, Ok(3));
```

A megszakítás-biztonság a drop-alapú cancellation következménye. A Rust async modellben nincs explicit cancellation API a future-re; a future elhagyása (drop) szakítja meg a műveletet a következő `.await` pontnál. Ez az egyszerű és hatékony stratégia, de fegyelmet igényel a fejlesztőtől.

17.3.7.1 Mit szakít meg a drop, és mit nem?

Négy különböző eset van, és mindegyik másképp viselkedik:

- **Lokális future dropja.** Ha egy közvetlenül birtokolt future-t eldobunk (például a `timeout` lejár), az állapotgép azonnal eldobódik, a művelet nem fut tovább. (Ez más, mint a spawnolt task `abort`-ja: az együttműködő, a következő ütemezési ponton lép érvénybe, és egy azonnal `Ready` `await` nem feltétlenül ad vissza vezérlést.)
- **Spawnolt task.** A `tokio::spawn` külön taskot indít; a `JoinHandle` eldobása a taskot *leválasztja*, nem állítja le. Egy `timeout(dur, handle)` lejártakor tehát a task fut tovább. Leállításhoz `handle.abort()` kell, ami a következő `.await` pontnál lép érvénybe (és a `handle.await` ekkor `JoinError::is_cancelled()` hibát ad).
- **Együttműködő megszakítás.** Ha a task a leállítási jelet maga figyeli (például `CancellationToken` vagy csatorna), akkor szabályosan, a saját rendezett lépéseivel áll le; ez az rkv hálózati szerver kecses leállásának alapja.
- **Blokkoló munka.** A `spawn_blocking` closure-je, ha elindult, nem szakítható meg: az `abort` csak azt akadályozza meg, hogy elinduljon.

A különbség tesztelhető: egy 20 ms-onként számláló future-t 50 ms-os `timeout` alatt közvetlenül futtatva a számláló a lejárat után nem nő tovább (legfeljebb 3 növekedés), ugyanez a munka spawnolt taskként, a `JoinHandle`-re tett `timeout` után végigfut (5 növekedés), `abort` után pedig megáll.

17.3.8 Kidolgozott labor: taskok, csatorna, select és timeout

Munkakönyvtár: `~/rust-labor/r039/`

Ebben a laborban létrehozunk egy önálló kísérleti programot, amely bemutatja a taskokat, az `mpsc` csatornát, a `select!`-et és az időkorlátot. Az itteni ismereteket a későbbi `rkv-server` laborban a szerver TCP-protokollja hasznosítja.

Hozzuk létre a projektet és adjuk hozzá a tokio függőséget:

```
mkdir -p ~/rust-labor && cd ~/rust-labor && cargo new r039
cd r039
```

A `Cargo.toml` tartalma:

```
[package]
name = "r039"
version = "0.1.0"
edition = "2024"

[dependencies]
tokio = { version = "1", features = ["full"] }
```

Írjuk a `src/main.rs`-t. A consumer **egyetlen, teljes 1 másodperces határidőt** kap a ciklus egészére: a `sleep` future-t a ciklus előtt hozzuk létre és pineljük (`tokio::pin!`), a `select!` pedig a `&mut deadline` referenciát versenyezteti. (Ha a `sleep(...)`-et a ciklusban, a `select!`-ben írnánk, az minden iterációban újraindulna, így csak *inaktivitási* időkorlátot adna, lásd alább.)

```
use std::time::Duration;
use tokio::sync::mpsc;
use tokio::time::{sleep, Instant};

#[tokio::main]
async fn main() {
    let (tx, mut rx) = mpsc::channel::<u32>(8);

    // Producer: 5 értéket küld 50 ms-onként, majd a tx eldobásával lezárja a csatornát.
    let producer = tokio::spawn(async move {
        for i in 0..5 {
            tx.send(i).await.expect("a consumer él");
            sleep(Duration::from_millis(50)).await;
        }
        println!("Producer: befejeztem.");
        // `tx` itt kerül eldobásra -> a csatorna lezárul.
    });

    // Consumer: egyetlen, a ciklus egészére érvényes 1 s-os határidő.
    let consumer = tokio::spawn(async move {
        let deadline = sleep(Duration::from_secs(1));
        tokio::pin!(deadline);
        let start = Instant::now();
        loop {
            tokio::select! {
                msg = rx.recv() => match msg {
                    Some(v) => println!("Consumer: fogadva: {v}"),
                    None => {
                        println!("Consumer: a csatorna lezárult, leállás.");
                        break;
                    }
                },
                _ = &mut deadline => {
                    println!("Consumer: a határidő lejárt ({:?}), leállás.", start.elapsed());
```

```
                    break;
                }
            }
        }
    });

    producer.await.expect("a producer nem pánikolt");
    consumer.await.expect("a consumer nem pánikolt");
}
```

Futtassuk a programot:

```
cargo run
```

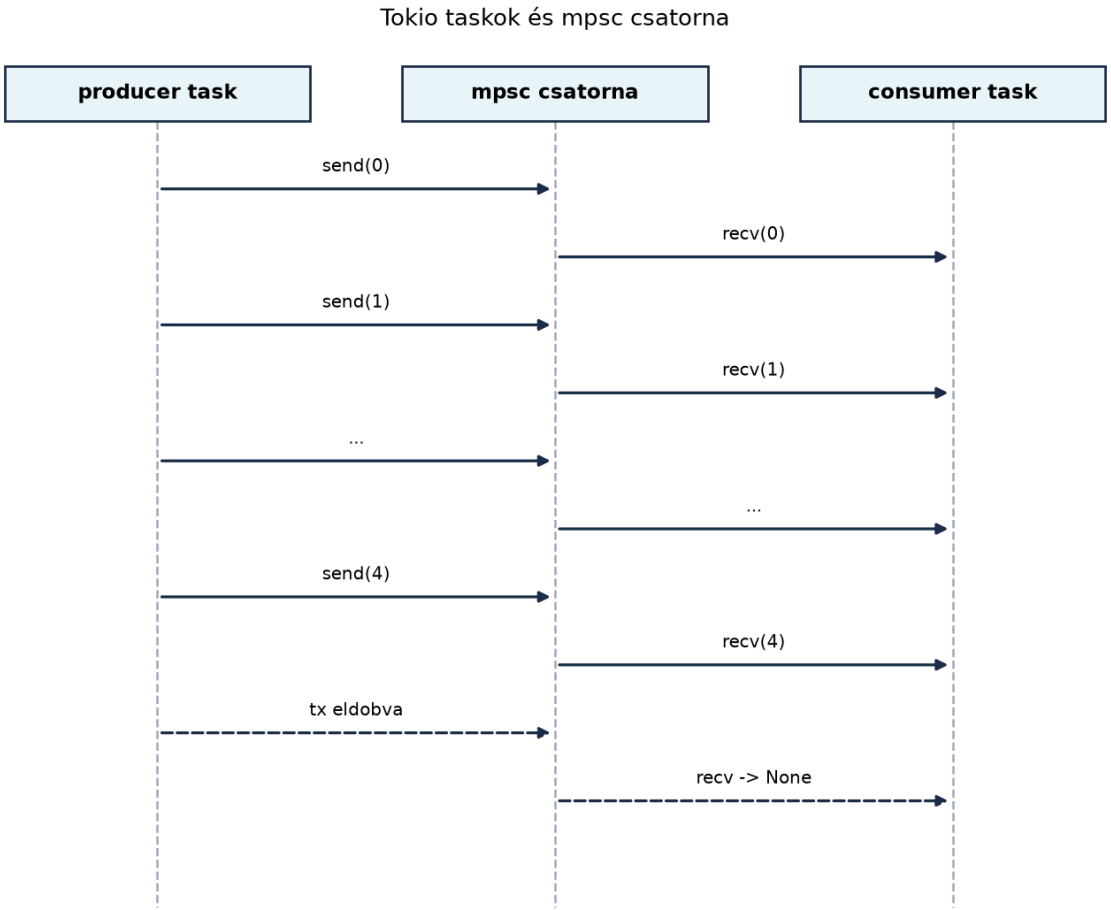
Várt kimenet (futtatva, `tokio` 1.x; az időzítés miatt a két utolsó sor sorrendje elvileg eltérhet, de a `Producer` a csatorna lezárása *előtt* ír):

```
Consumer: fogadva: 0
Consumer: fogadva: 1
Consumer: fogadva: 2
Consumer: fogadva: 3
Consumer: fogadva: 4
Producer: befejeztem.
Consumer: a csatorna lezárult, leállás.
```

A producer 5 értéket küld összesen 250 ms alatt, majd a taskja véget ér, és a `tx` eldobása lezárja a csatornát. A consumer `rx.recv()` hívása ekkor `None`-t ad, ami a ciklus **normál vége**: a consumer azonnal leáll, nem vár szükségtelenül további másodpercet. A határidő-ág (`&mut deadline`) csak akkor nyer, ha a termelés a teljes határidő alatt nem ér véget.

Teljes határidő és inaktivitási időkorlát. A két fogalmat érdemes külön nevezni. A *teljes határidő* (deadline) a ciklus összes futására vonatkozik: folyamatos termelés mellett is az 1. másodpercnél leáll. Az *inaktivitási időkorlát* (idle timeout) azt méri, mennyi ideig nem érkezett semmi; ezt kapjuk, ha a `sleep` minden iterációban újraindul. A tesztek ezt mutatják: egy 10 ms-onként termelő forrással a pinelt határidős consumer a 300 ms-os határidőnél áll le (290–450 ms közt), míg az iterációnként újraindított 100 ms-os `sleep` a 400 ms-ig tartó termelést végig kivárja, mert egyszer sem jár le.

A következő ábra a taskok és a csatorna közötti üzenetváltást mutatja:



17.3. ábra: Tokio taskok és mpsc csatorna - A producer task 5 üzenetet küld az mpsc csatornán, a consumer task fogadja azokat; a producer befejezése lezárja a csatornát (recv -> None), ezért a consumer a határidő lejárta előtt szabályosan leáll.

A producer task aszinkron módon küldi az értékeket, a consumer task aszinkron módon fogadja őket. A határidő egy belső időzítő, amely a consumer `select!`-ében versenyez a `recv`-vel; ha a termelés tovább tartana az 1 másodpercnél, a határidő-ág nyerne, a `recv` future megszakadna, és a consumer kilépne a ciklusból.

17.3.9 Tipikus hibák és átállási csapdák

1. **Hosszú vagy blokkoló hívás async taskban.** Ha például `std::thread::sleep`-et vagy `std::fs::read`-et hívunk `async fn`-ben, az blokkolja a tokio runtime-szálát, és a többi task nem futhat. Az elv: a hosszú vagy blokkoló munkát izoláljuk (`spawn_blocking`, vagy az async megfelelő `tokio::time::sleep`, `tokio::fs`), a rövid, nem blokkoló szinkron kód viszont maradhat inline.
2. **A `select!`-ben több mellékhatás.** A megszakítás-biztonság hiánya gyakori hiba. Ha egy `select!`-ágban két egymást követő `send` hívás van, a timeout megszakíthatja a műveletet az első send után, ami részleges állapotot hagy. Tulajdonolt érték küldése telített csatornán `timeout` alatt az értéket is elveszti. A megoldás: egyetlen, dokumentáltan megszakításbiztos műveletet tegyünk az ágba (például `reserve`, `send_timeout`), vagy a többlépéses munkát állapotgéppel/külön taskkal végezzük, és a részleges siker kezelését explicit megtervezzük.

3. **`Rc` és nem-`Send` adatok a taskban.** A `tokio::spawn` `Send + 'static` future-t igényel. Ha egy taskban `Rc`-t vagy olyan referenciát használunk, amely nem `'static`, fordítási hibát kapunk. Ilyenkor `Arc`-ra kell váltani, vagy a data ownership-et csatornán átadni. Ez a Rust szigorúságának egyik előnye: a hibát már fordítás előtt elkapjuk.

4. **Az `await` elfelejtése a `JoinHandle`-n.** Ha a `JoinHandle`-t nem `await`-eljük, a task akkor is lefut, de a főprogram kiléphet, mielőtt a task befejeződne. A tokio runtime a kilépéskor leállítja a taskokat. A `JoinHandle` `await`-elése biztosítja a kecses leállást.

17.3.10 Feladatok

17.3.10.1 (a) Fogalmi kérdés

Miért követeli meg a `tokio::spawn`, hogy a future `Send + 'static` legyen? Miért nem elég a `'static`, és miért okoz problémát, ha a future nem `Send` a multi-threaded runtime-ban?

17.3.10.2 (b) C++ kód átírása

Írd át a következő C++ kódot idiomatikus Rust async kódra tokio-val:

```cpp
#include <future>
#include <iostream>
#include <chrono>
#include <thread>

int main() {
    auto fut1 = std::async(std::launch::async, [] {
        std::this_thread::sleep_for(std::chrono::milliseconds(500));
        return 1;
    });
    auto fut2 = std::async(std::launch::async, [] {
        std::this_thread::sleep_for(std::chrono::milliseconds(300));
        return 2;
    });
    // Az elsőként befejeződő eredményét írjuk ki.
    // (Egyszerűen megvárjuk mindkettőt, de a válaszban használj select!-et.)
    std::cout << fut1.get() << ' ' << fut2.get() << '\n';
}
```

17.3.10.3 (c) Implementációs feladat

Írj egy `select_timeout` nevű async generikus függvényt, amely egy future-t és egy `Duration` időkorlátot kap, és `Result<F::Output, TimeoutError>` típust ad vissza, ahol a `TimeoutError` **saját** hibatípus (a `tokio::time::error::Elapsed`-nek nincs publikus konstruktora). A függvény belül `tokio::select!`-et használjon, ne a `tokio::time::timeout`-ot. Hasonlítsd össze a viselkedést a beépített `timeout`-tal.

17.3.10.4 (d) Hibajavítás és tervezési döntés

Az alábbi `select!` kód nem megszakításbiztos. Magyarázd el, mi a baj, és javítsd ki úgy, hogy az időkorlát lejártakor se vesszen el a küldendő érték, és a két csatornán ne maradjon kétértelmű részleges állapot. Nevezd meg, mit nem ad a megoldás (például nem atomikus tranzakció).

```rust
let (tx1, mut rx1) = mpsc::channel::<i32>(8);
let (tx2, _rx2) = mpsc::channel::<i32>(8);

tokio::select! {
    _ = async {
        tx1.send(42).await?;
        tx2.send(24).await?;
    } => { /* ... */ }
    _ = sleep(Duration::from_secs(1)) => { /* timeout */ }
}
```

17.3.11 Megoldások

17.3.11.1 (a)

A `'static` megkötés azért szükséges, mert a task elvileg tetszőleges ideig futhat, így minden benne lévő referenciának érvényesnek kell lennie a task teljes élettartama alatt. A `'static` nem azt jelenti, hogy az adat soha nem szabadul fel, csak azt, hogy a hivatkozott adat nem függ a hívó veremétől. A `Send` megkötés azért kell, mert a multi-threaded runtime a taskot átmozgathatja egyik szálról a másikra; a `Send` tulajdonság garantálja, hogy az adat biztonságosan átvihető szálak között. Ha a future nem `Send`, akkor nem lehet a taskot másik szálon futtatni, ami versenyhelyzetet okozhatna. A megoldás ilyenkor a `LocalSet` használata (csak azon a szálon futó taskok), de ez ritkábban szükséges.

17.3.11.2 (b)

```rust
use tokio::time::{sleep, Duration};

#[tokio::main]
async fn main() {
    let fut1 = async {
        sleep(Duration::from_millis(500)).await;
        1
    };
    let fut2 = async {
        sleep(Duration::from_millis(300)).await;
        2
    };

    let result = tokio::select! {
        n1 = fut1 => n1,
        n2 = fut2 => n2,
    };
    println!("Elsőként: {result}");
}
```

A `select!` az elsőként befejeződő eredményt adja vissza, a másik future megszakad. Ha mindkettőt meg akarjuk várni, használhatunk `tokio::join!`-ot is.

17.3.11.3 (c)

```rust
use std::future::Future;
use tokio::time::{sleep, Duration};

/// Saját hibatípus: a tokio `Elapsed` értékének nincs publikus konstruktora,
/// ezért nem hozhatjuk létre magunk.
```

```rust
#[derive(Debug, PartialEq, Eq)]
pub struct TimeoutError;

pub async fn select_timeout<F>(fut: F, dur: Duration) -> Result<F::Output, TimeoutError>
where
    F: Future,
{
    tokio::select! {
        output = fut => Ok(output),
        _ = sleep(dur) => Err(TimeoutError),
    }
}

#[tokio::main]
async fn main() {
    let result = select_timeout(
        async { sleep(Duration::from_millis(100)).await; 42 },
        Duration::from_secs(1),
    )
    .await;
    println!("{result:?}"); // Ok(42)
}
```

A beépített `timeout` ugyanezt teszi, de `tokio::time::error::Elapsed` hibát ad; ennek az értékét csak a `timeout` állíthatja elő, ezért a saját függvény saját hibatípust ad vissza (a másik lehetőség a feladat átfogalmazása a publikus `tokio::time::timeout` használatára). Mindkét változat a lejáratkor eldobja a future-t, tehát a lezáratlan tulajdonolt értékek elvesznek (lásd a megszakítás-biztonság szakaszt). A fenti kód a rögzített tokio 1.x verzióval fordul és fut, a viselkedést teszt igazolja: sikeres befejezésre `Ok(42)`, lassú future-re `Err(TimeoutError)`, ahogy a beépített `timeout` is `Err`-t ad.

17.3.11.4 (d)

A probléma az, hogy a `select!`-ben az `async` blokk két `send` hívást tartalmaz, és a `timeout` ág bármikor megszakíthatja. Ha a timeout nyer az első `send` után, a `tx1`-re elküldött érték már a csatornában van, a `tx2`-re szánt érték viszont soha nem lesz elküldve; ráadásul egy telített `tx1` mellett a még el nem küldött `42` is elveszik. A fogadó `rx1` értéket kap, `rx2` nem.

A javítás első lépése, hogy az érték a megszakítás után is a miénk maradjon: `reserve`-vel előbb *helyet* foglalunk mindkét csatornán (a foglalás megszakítása nem veszít adatot), és csak ha mindkét `Permit` megvan, küldünk, szinkron módon, megszakítási pont nélkül:

```rust
let (tx1, mut rx1) = mpsc::channel::<i32>(8);
let (tx2, mut rx2) = mpsc::channel::<i32>(8);

let sent = async {
    let p1 = tx1.reserve().await.ok()?;
    let p2 = tx2.reserve().await.ok()?;
    p1.send(42);   // innen nincs .await: a két küldés együtt, megszakítás nélkül megy végbe
    p2.send(24);
    Some(())
};

tokio::select! {
    r = sent => println!("elküldve: {}", r.is_some()),
    _ = sleep(Duration::from_secs(1)) => println!("Időkorlát: semmi nem lett elküldve, az
↪  értékek a miénk."),
}
```

Ha az időkorlát a foglalás közben jár le, egyik érték sem lett elküldve (a `Permit`-ek eldobása visszaadja a helyet); ha mindkét foglalás sikerült, a két `send` egyetlen `.await`-mentes szakaszban fut le. Ez **nem atomikus tranzakció** a két csatorna között: ha a fogadó oldal leállt, vagy a csatorna zárt (`reserve` hiba), a részleges siker továbbra is kezelendő (itt: `None`, nincs küldés). Ha a két küldést háttértaskba tennénk, az a megszakítás kérdését csak áthelyezné: a `JoinHandle` eldobása a taskot nem állítja le, és a shutdown is külön kezelendő (lásd az előző szakasz négy esetét).

17.3.12 Rövid összefoglaló

A tokio runtime feladata a future-ök ütemezése. A `#[tokio::main]` multi-threaded runtime-ot indít, a `tokio::spawn` `Send + 'static` future-öket futtat taskokként, és `JoinHandle`-t ad vissza. Az async I/O-hoz a `tokio::fs` és `tokio::net` modulok async függvényeit kell használni, blokkoló kódhoz pedig a `spawn_blocking`-et. A tokio csatornák (`mpsc`, `oneshot`, `broadcast`) async kommunikációt biztosítanak; a `mpsc` bounded csatornája backpressure-t ad. A `select!` makró több future-t versenyeztet, a `timeout` pedig időkorlátot ad egy future-höz. A kiválasztatlan ágak future-jei megszakadnak, ezért az ágak műveleteinek dokumentáltan megszakításbiztosnak kell lenniük (`recv`, `reserve`, `send_timeout`), a tulajdonolt értéket pedig a megszakítás után is tartsuk meg. A `JoinHandle` eldobása a taskot leválasztja, nem állítja le; az időzítő-ágat egy pinelt `sleep` adja, ha teljes határidőre van szükség. Ezek az ismeretek közvetlenül felhasználásra kerülnek az rkv hálózati szerver TCP-protokolljában és a naplózás alfejezetben.

17.3.13 Források és ellenőrzési hiányok

- A `tokio::select!` makró és a megszakításbiztosság (cancellation safety) metódusonkénti listája: https://docs.rs/tokio/1/tokio/macro.select.html
- `tokio::sync::mpsc` (`reserve`, `send_timeout`) és `tokio::sync::broadcast` (`RecvError::Lagged`): https://docs.rs/tokio/1/tokio/sync/
- `tokio::time::timeout` és `sleep`: https://docs.rs/tokio/1/tokio/time/

A kódot a rögzített toolchainnel (Rust 1.99.1) és tokio 1.x főverzióval ellenőriztük; a `tokio::time::error::Elapsed` értéknek nincs publikus konstruktora, ezért a saját `select_timeout` saját hibatípust használ.

17.4 Az rkv hálózati szerver: TCP, protokoll, kecses leállás

17.4.1 Motiváció

Egy C++-ban írt TCP-szerver tipikusan a következő mintát követi: egy `accept` ciklus, minden kapcsolatnak egy `std::thread`, és egy osztott, mutexszel védett állapot. A kecses leállás (graceful shutdown) azonban gyakran utólag kerül be, és ilyenkor jönnek a jellegzetes hibák: a szálak nem tudnak egy atomi `bool` leállítási flagre időben reagálni, a `join` a blokkoló `recv` miatt lefagy, a nyitott kapcsolatokra nem lehet hatni. A Rust async modellben a leállítás is elsőrendű állampolgár: a futó feladatok (taskok) megszakíthatók, ahelyett hogy egy blokkoló hívásban ragadnának. Ebben az alfejezetben az rkv-hoz építünk egy tokio-alapú TCP-szervert, amely soronkénti szöveges protokollt beszél, és **graceful shutdownnal** áll le: új kapcsolatokat nem fogad, a folyamatban lévő parancsokat befejezi, és a klienseket hagyja kilépni.

17.4.2 Fogalmak és szabályok

Definíció. A *keretezés* (framing) az a szabály, amely meghatározza, hol kezdődik és végződik egy üzenet egy bájtfolyamban. A mi szerverünknél a keretezés egy sor vége (`\n`): minden parancs egyetlen sorból áll, a válasz is soronkénti (a `LIST` kivételével, ahol több `VALUE` sor és egy záró `OK` sor érkezik).

Definíció. A *kecses leállás* (graceful shutdown) olyan leállítási folyamat, amely során a szerver nem szakítja meg azonnal az aktív kapcsolatokat, hanem: (1) leállítja az új kapcsolatok fogadását, (2) jelzi a klienseknek, hogy a folyamatban lévő művelet befejezése után lépjenek ki, és (3) megvárja a futó taskok befejeződését.

Szabály. Ha egy `MutexGuard`-ot `.await` ponton át tartunk, a kritikus szakasz akár tetszőlegesen hosszú ideig zárva maradhat, ami csökkenti az áteresztőképességet, és `Mutex`-függő holtpontot okozhat. Async kódban a zárat csak rövid, nem `.await`-elő szakaszra szerezhetjük meg, és a `.await` előtt el kell engednünk.

A tokio két választ ad a megosztott állapotra: a `std::sync::Mutex` (vagy `RwLock`) és a `tokio::sync::Mutex`. A szerver a tárolót a `SharedStore`-on keresztül éri el (Mutex, RwLock, csatornák és a megosztott rkv), amely belül `std` mutexet használ, és a zárat **soha nem tartja `.await`-en át**: minden tárolóművelet szinkron. A művelet viszont **blokkolhat** (a `FileStore` fájlírást és `fsync`-et végez), ezért a szerver nem az executor szálán futtatja, hanem `tokio::task::spawn_blocking`-be teszi (lásd lent). A `tokio::sync::Mutex` akkor jön jól, ha a védett szakasz maga is `.await`-el.

Szabály (erőforráskorlátok). Egy hálózatra nyitott szerver három erőforrását korlátoznia kell: (1) a *kérés mérete* (`max_frame`): sorvég nélküli, végtelen bemenet nem növelheti a memóriát; (2) az *egyidejű kapcsolatok száma* (`max_connections`): a többlet kontrollált hibát (`ERR server busy`) kap; (3) a *blokkoló munka* nem foglalhatja le az executor szálait. A korlát túllépése mindig explicit, protokoll szerinti hibaválasz, nem csendes csonkolás.

Definíció. A `tokio_util::sync::CancellationToken` egy olyan jelzés, amelyet `cancel()` hívással lehet bekapcsolni, és a váró taskok `.cancelled()`-re ébrednek. Jól használható a leállítási jel terjesztésére: minden task kap egy klónt, és `select!`-ben versenyezteti a normál eseménnyel.

17.4.3 C++ és Rust összevetés

Egy C++ szerver vázlata szálakkal és leállítási flaggel:

```
#include <atomic>
#include <iostream>
#include <thread>
#include <vector>
#include <sys/socket.h>
#include <netinet/in.h>

std::atomic<bool> stop{false};

void handle_client(int fd) {
    char buf[1024];
    while (!stop) {
        ssize_t n = recv(fd, buf, sizeof(buf), 0);
        if (n <= 0) break;
        // feldolgozás...
    }
    close(fd);
```

```
}

int main() {
    int listener = socket(AF_INET, SOCK_STREAM, 0);
    // bind, listen...
    std::vector<std::thread> threads;
    while (!stop) {
        int fd = accept(listener, nullptr, nullptr);
        threads.emplace_back(handle_client, fd);
    }
    // leállítás: stop = true; majd join...
}
```

A probléma: ha a kliens nem küld adatot, a `recv` blokkol, és a `stop` flaget soha nem veszi észre; a `join` a blokkoló szálon függ. Ezt jellemzően `poll`-lal, `shutdown` hívással vagy timeouttal kell megkerülni. A Rust/tokio szerver ehelyett `select!`-ben figyeli a megszakítást, és a task kilép a futó `.await`-ből akkor is, ha épp sorra várt.

A Rust szerver fő hurka (vázlat; a teljes, `JoinSet`-es változat a labor `lib.rs`-ében van):

```
use rkv_core::SharedStore;
use tokio::net::TcpListener;
use tokio_util::sync::CancellationToken;

async fn run_server(listener: TcpListener, store: SharedStore, token: CancellationToken) {
    loop {
        tokio::select! {
            _ = token.cancelled() => break,
            res = listener.accept() => {
                let (socket, addr) = match res {
                    Ok(v) => v,
                    Err(e) => { eprintln!("accept error: {e}"); continue; }
                };
                let store = store.clone();
                let token = token.clone();
                tokio::spawn(async move {
                    if let Err(e) = handle_client(socket, store, token).await {
                        eprintln!("client {addr}: {e}");
                    }
                });
            }
        }
    }
}
```

A különbség: a `select!` a listener `accept` és a leállítási token között választ; ha a token megszakad, a ciklus kilép anélkül, hogy új kapcsolatot fogadna. Ez a vázlat csak a leállás első fázisát mutatja: a `tokio::spawn`-nal indított klienstaskokat senki nem várja meg, és a `main` visszatérése a runtime-mal együtt félbeszakítaná őket. A labor teljes `run_server` függvénye ezért `JoinSet`-ben tartja nyilván a klienstaskokat, és a megszakítás után határidővel megvárja őket.

17.4.4 A protokoll és a válaszformátum

Az rkv-szerver protokollja soralapú. A kliens egyetlen sora egy parancsot tartalmaz:

```
GET <kulcs>
SET <kulcs> <érték>
DEL <kulcs>
LIST [<prefix>]
```

A szerver válaszai:

```
OK                  - SET és DEL sikeres; LIST lezáró sora
VALUE <kulcs> <érték>  - GET találat vagy LIST elem
NOTFOUND            - GET/DEL nem létező kulcs esetén
ERR <üzenet>        - hibás parancs vagy belső hiba
```

A szöveges réteg csak *szöveg-biztos* kulcsokra és értékekre való. Bármilyen bájtsor átviteléhez (a `[0xFF, 0x0A, 0x0D]`-tól az üres értékig) külön, hexa kódolt bináris réteg van:

```
SETX <hexkulcs> <hexérték>   - a kulcs és az érték hexa; az üres érték jele: -
GETX <hexkulcs>              - válasz: VALUEX <hexkulcs> <hexérték>  vagy NOTFOUND
```

A `[0,255,10,13]` érték így `SETX 62696e 00ff0a0d`, az üres érték `SETX 62696e -`; a `GETX 62696e` pontosan ugyanazt a hexát adja vissza. A szöveges `SET k ff` ezzel szemben az `ff` két ASCII karaktert tárolja, nem a `0xFF` bájtot: a két réteg elkülönül.

A `LIST` válasza: minden egyező kulcshoz egy-egy `VALUE` sor, majd egy `OK` sor. Egy munkamenet `nc`-vel:

```
$ nc localhost 8080
SET hello world
OK
GET hello
VALUE hello world
LIST he
VALUE hello world
OK
DEL hello
OK
GET hello
NOTFOUND
```

A protokoll nyelvtanát az Az enum mint összeg-típus és az Option a null helyett–Mintaillesztés: match, if let, let else és a minták alfejezet rögzítette, és az `rkv-core` `Command` típusának `FromStr` implementációja valósítja meg (A standard trait-ek: konverziók, formázás, Default, Deref, Borrow): a kulcs nem tartalmazhat szóközt, a `SET` értéke a kulcs utáni szóköztől a sor végéig tart (tartalmazhat szóközt), a felesleges argumentum hiba. A hibás kérés `ERR <üzenet>` választ kap, a kapcsolat pedig használható marad. A szöveges rétegben egy bejegyzés akkor *szöveg-biztos*, ha a kulcs nem üres és nem tartalmaz szóközt vagy vezérlőkaraktert, az érték pedig nem üres, érvényes UTF-8, és nem tartalmaz vezérlőkaraktert (sortörést sem). Az `rkv-core`, az FFI és a cxx réteg tetszőleges UTF-8 kulcsot és bájtértéket tárolhat, ezért a válaszformázás védekezik: a nem szöveg-biztos érték `GET`-re `ERR value is not text-safe, use GETX` választ ad (nem csonkolódik, és nem tud új válaszsort kezdeni, például egy `OK`-t tartalmazó értékkel), a `LIST` pedig az ilyen bejegyzéseket `VALUEX <hexkulcs> <hexérték>` sorban adja, így a válasz mindig soronként értelmezhető, és `OK`-ra végződik. A kérés mérete korlátos (lásd a `ServerConfig`-ot).

17.4.5 Keretezés: korlátos soralapú olvasás

A bejövő adatfolyam bájtok sorozata; a keretezést a sorvég (`\n`) adja. A kézenfekvő `BufReader::lines()` / `read_line` viszont **korlátlan**: egy sorvég nélküli, végtelen bemenet addig növeli a sor pufferét, amíg a memória el nem fogy. Ezért a szerver saját `read_frame` függvénnyel olvas, amely a `take(max_frame + 1)` adapterrel korlátozza a pufferbe kerülő bájtokat. Három kimenete van: teljes sor, túl hosszú sor (a puffer meghaladja a `max_frame`-et) és fájlvég:

```rust
async fn read_frame<R>(reader: &mut R, buf: &mut Vec<u8>, max_frame: usize) -> io::Result<Frame>
where
    R: AsyncBufRead + Unpin,
{
    buf.clear();
    let n = reader
        .take(max_frame as u64 + 1)
        .read_until(b'\n', buf)
        .await?;
    Ok(if n == 0 {
        Frame::Eof
    } else if buf.len() > max_frame {
        Frame::TooLong
    } else if buf.ends_with(b"\n") {
        Frame::Line
    } else {
        Frame::Eof // csonka, sorvég nélküli maradék: a kliens lezárt
    })
}
```

Túl hosszú sornál a szerver `ERR line too long` választ küld, és lezárja a kapcsolatot. Ez tudatos házirend, nem elvi kényszer: a keretezés a következő `\n`-ig tartó eldobással (drain-until-LF) elvileg újraszinkronizálható volna, de ehhez a szervernek a maradék bemenetet előírt bájt- és időkeretig kellene olvasnia és eldobnia (például 64 KiB és 1 s), majd a sornál újrapróbálnia. Ez a korlátos erőforrás-védelem mellett is bonyolítja a protokollt, és a lassú vagy rosszindulatú kliens miatt a kapcsolat hosszú ideig foglalt maradhat; a lezárás egyszerűbb, kiszámítható és a támadási felületet is csökkenti. A választott korlátot a `ServerConfig::max_frame` adja. (A tokio `AsyncBufReadExt::read_until` dokumentáltan megszakításbiztos: a már beolvasott bájtok a hívó `Vec`-jében maradnak. A *saját* `read_frame` wrapper viszont a hívás elején `buf.clear()`-t végez, ezért a megszakítás után egy újabb hívás eldobná a részleges adatot. Ez itt elfogadható, mert a megszakítás csak a leállításkor történik, és a kapcsolat ekkor úgyis lezárul; ha az olvasást folytatni kellene, a `clear()`-t csak teljes keret feldolgozása után szabadna végezni. Lásd az Tokio: taskok, I/O, csatornák, select és megszakítás alfejezetet.)

A `socket.into_split()` az olvasó- és íróoldalt különválasztja. A `BufReader` a `read_half`-ból olvas, a `BufWriter` a `write_half`-ra pufferel. A klienst kezelő függvény szerkezete: a ciklus a beolvasást a leállítási tokennel versenyezteti, a már beolvasott parancsot viszont mindig végrehajtja és megválaszolja:

```rust
loop {
    let frame = tokio::select! {
        _ = token.cancelled() => break,
        frame = read_frame(&mut reader, &mut buf, max_frame) => frame?,
    };
    match frame {
        Frame::Eof => break,
        Frame::TooLong => {
            reply(&mut writer, "ERR line too long").await?;
            break;
        }
        Frame::Line => { /* végrehajtás és válasz, lásd a labort */ }
    }
}
writer.shutdown().await
```

A válasz írását szándékosan *nem* tesszük `select!`-be: a már beolvasott parancs válaszát a leállás alatt is teljesen elküldjük.

Két részlet a `select!`-hez. (1) **A `break` típusa.** A `select!` minden ágának azonos típusú kifejezést kell adnia. Az `_ = token.cancelled() => break` ág `break` kifejezése `!` (never)

típusú, amely tetszőleges típusra koerciálható, ezért illeszkedik a másik ág `Frame` típusához (a `frame?` szintén `Frame`). (2) **Ági elsőbbség.** Alapértelmezésben a `select!` a *kész* ágak közül véletlenszerűen választ (fairség), és ha egy ág már kész, nem ad elsőbbséget a leállításnak: ha a token már megszakadt, és közben egy teljes sor is olvasható, bármelyik ág nyerhet. Ha a kódnak ezt az elsőbbséget kell kikényszerítenie, a `select!` elejére írt `biased;` rögzített, fentről lefelé haladó sorrendet ad (a `cancelled()` ág kerüljön az első helyre), vagy a leállítás állapotát explicit ellenőrizzük a ciklus elején (`if token.is_cancelled() { break; }`). A fenti kód ezt az elsőbbséget nem ígéri: a már beolvasott parancs végrehajtása és megválaszolása a leállás alatt is megtörténik.

Szabály (flush). A tokio `BufWriter` a kis válaszokat a belső pufferében tartja, amíg az meg nem telik vagy `flush`-t nem hívunk. Ha minden válasz után nincs `flush().await`, a kliens a válaszra vár, a szerver a következő parancsra: a kapcsolat beragad. A tokio `BufWriter` eldobása ráadásul *nem* üríti a puffert (aszinkron ürítést `Drop` nem tud végezni), a bent maradt bájtok elvesznek. Ezért minden teljes protokollválasz után `flush().await?`, kilépéskor pedig `shutdown().await?` következik; ez utóbbi üríti a puffert és lezárja a küldő irányt.

A `tokio_util::codec` modul kifinomultabb keretezést ad: például hosszprefixes vagy szigorúbb szöveges protokollt (`LinesCodec::new_with_max_length`), és a `Framed` típus `Stream`-ként kezeli a bejövő kereteket. A mi protokollunknál a saját `read_frame` elegendő; a codecalapú megoldást nagyobb rendszereknél érdemes megfontolni.

17.4.6 Megosztott állapot: `SharedStore` és blokkoló munka

A szerver több klienssel dolgozik, ezért a tárolót minden tasknak el kell érnie. A típus a Mutex, RwLock, csatornák és a megosztott rkv alfejezet `SharedStore`-ja: klónozható, `Send + Sync`, és bármely `Storage`-ot (`MemStore`, `FileStore`) burkol, így a szerver nem tudja, mi áll mögötte. A zár a `SharedStore` metódusain belül van; a szerver kódja sehol nem tart zárat `.await`-en át.

A kritikus rész egy kérés feldolgozása. A `process_line` szinkron függvény: értelmezi a sort (`FromStr`), végrehajtja a parancsot (`SharedStore::apply`), és a protokoll szerinti választ formázza:

```
pub fn process_line(line: &str, store: &SharedStore) -> String {
    let command = match line.parse::<Command>() {
        Ok(command) => command,
        Err(e) => return format!("ERR {e}"),
    };
    let response = match store.apply(&command) {
        Ok(response) => response,
        Err(e) => return single_line(&format!("ERR {e}")),
    };
    match (&command, response) {
        (_, Response::Ok) => "OK".to_string(),
        (_, Response::NotFound) => "NOTFOUND".to_string(),
        (Command::Get(key), Response::Value(bytes)) => match String::from_utf8(bytes) {
            Ok(text) => format!("VALUE {key} {text}"),
            Err(_) => format!("ERR value of {key} is not valid UTF-8"),
        },
        // ... Entries (LIST) és a többi ág: lásd a labor teljes lib.rs-ét
    }
}
```

Mivel a `FileStore` fájlműveletei (írás, `fsync`) blokkolnak, a `process_line` **nem az executor szálán fut**: a kliens task `tokio::task::spawn_blocking`-gel indítja, és a `JoinHandle`-t `await`-eli. Így egy lassú fájlművelet csak a blokkoló szálkészlet egy szálát köti le, a többi task (új

kapcsolatok fogadása, más kliensek olvasása/írása) tovább halad. A tároló soros szerződése (a `SharedStore` mutexe) megmarad: két lassú írás egymásra vár, de az executort nem állítja meg. A shutdown szerződés: a `spawn_blocking` closure-je, ha elindult, nem szakítható meg, ezért a leállítás megvárja a futó tárolóműveletet (legfeljebb `shutdown_grace` ideig).

A `process_line` a zárat egyetlen `apply` hívásra szerzi meg, és a válasz formázása már zár nélkül történik. Ha a zárat az `.await`-ig kiterjesztenénk, a tárolót minden kliens megvárná; ezt mutatja a következő, szándékosan rossz minta (a `std` guard nem `Send`, ezért a `tokio::spawn` el is utasítaná):

```
// Ez a változat szándékosan rossz tervezési minta:
async fn handle_client_bad(socket: TcpStream, store: Arc<std::sync::Mutex<dyn Storage + Send>>)
 ↪ {
    let mut lines = BufReader::new(socket).lines();
    while let Ok(Some(line)) = lines.next_line().await {
        let mut guard = store.lock().unwrap();           // a zár az .await-en át él
        let response = process(&mut **guard, &line);
        reply(&response).await;                           // .await a guarddal a kezünkben
    }
}
```

17.4.7 Kecses leállás CancellationToken-nel

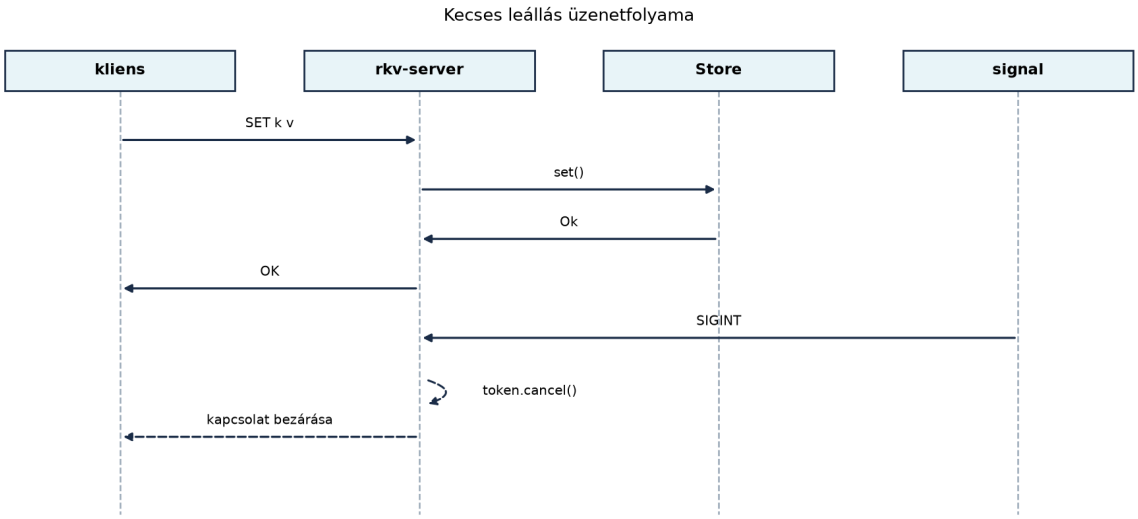
A leállítás folyamata a következő: 1. A `main` függvény létrehoz egy `CancellationToken` példányt. 2. Egy külön task figyeli a `Ctrl-C` jelet `tokio::signal::ctrl_c().await` segítségével, majd meghívja a `token.cancel()`-t. 3. **1. fázis:** a listener `select!`-ben figyeli a `token.cancelled()`-et, így az `accept` ciklus kilép, a listener eldobódik, új kapcsolat nem jön létre. 4. **2. fázis:** a klienstaskok a következő sor olvasásakor észreveszik a megszakítást. A már beolvasott parancsot végrehajtják, a választ kiírják és `flush`-elik, majd `shutdown`-nal lezárják a kapcsolatot. 5. A `run_server` a klienstaskokat egy `JoinSet`-ben tartja nyilván, és a megszakítás után megvárja őket, legfeljebb `shutdown_grace` ideig. A határidő után a még futó taskokat (például egy lassan olvasó kliens miatt beragadt írást) megszakítja (`abort`). 6. A `main` csak a `run_server` visszatérése után írja ki a `shutdown complete` sort; ekkor már minden klienstask véget ért *vagy megszakadt*. A `main` visszatérése leállítja a runtime-ot, ezért a nyilvántartott taskokat *előtte* kell megvárni. 7. **Három külön szerződés.** (a) *Kapcsolatlezárás:* a `shutdown_grace` ideig vár a szerver a futó parancsokra, utána a klienstaskot megszakítja (`JoinSet::shutdown`). (b) *Blokkoló tárolóművelet:* az elindult `spawn_blocking` munka nem szakítható meg (egy `fsync` sem), a task megszakítása csak a `JoinHandle`-t dobja el; a művelet tovább futhat, és a `run_server` visszatérése *nem* garantálja a befejezését. (c) *Folyamatkilépés:* a runtime eldobása a futó blokkoló munkát korlátlanul megvárná, ezért a `main` a runtime-ot `shutdown_timeout`-tal állítja le; ha a határidő lejár, a folyamat a háttérben futó művelet befejezése nélkül lép ki. Nem megszakítható `fsync`-re tehát nem ígérünk kemény leállási határt.

A főprogram (`main.rs`) rövid: konfiguráció és indítás, a kiszolgáló logika a könyvtárban (`lib.rs`) van. Teljes kódja a laborban szerepel; lényege:

```
let store = SharedStore::new(MemStore::new());
let token = CancellationToken::new();
// ... egy task a Ctrl-C-re: token.cancel() ...
run_server(listener, store, token).await;
println!("shutdown complete"); // a kapcsolatok lezárultak; a blokkoló munka vége külön kérdés
```

A `tokio::signal::ctrl_c()` a Unix `SIGINT` jelet aszinkron módon várja. A haladóbb alkalmazások `tracing`-gel naplózhatják az eseményeket; ezt a Naplózás és megfigyelhetőség alfejezetben tesszük hozzá.

A leállítás és a klienskezelés kapcsolatát foglalja össze az alábbi ábra:



17.4. ábra: Kecses leállás üzenetfolyama - A Ctrl-C hatására a token megszakad, a listener leáll, a kliens befejezi az aktuális parancsot.

Az ábrán látható, hogy a `SET` parancs feldolgozása közben érkező `SIGINT` nem szakítja félbe a tároló műveletet; a token megszakad, de a kliens csak a válasz elküldése és `flush`-e után lép ki. Ezt a lenti teljes kód garantálja: a válaszírás nincs `select!`-ben, a `run_server` pedig megvárja a klienstaskokat. A garancia korlátja a `shutdown_grace` határidő: ha egy kliens addig sem olvassa ki a választ, a task megszakad, és a válasz elveszhet. Ez a kecses leállás lényege: a vállalt munka befejeződik, de a leállás nem tarthat korlátlanul.

17.4.8 Labor: az rkv-server crate

Munkakönyvtár: `~/rust-labor/rkv/`

A kiinduló állapot a Tokio: taskok, I/O, csatornák, select és megszakítás és a megelőző alfejezetek ellenőrzési pontja: az `rkv-core` exportálja a `Command`, `Storage`, `MemStore`, `FileStore`, `SharedStore` típusokat, a `Command` `FromStr`/`Display`-jét és az `apply` függvényt. A labor célja az `rkv-server` bináris *és* könyvtár létrehozása, amely az `rkv-core` `SharedStore`-ját használja. A kiszolgáló logika a `src/lib.rs`-ben van (így az integrációs tesztek importálhatják), a `src/main.rs` pedig rövid indítóprogram: egy bináris crate `pub` függvényét más crate nem tudja importálni.

17.4.8.1 1. lépés: a crate létrehozása és a workspace-be illesztése

```
cd ~/rust-labor/rkv
cargo new rkv-server
```

(A `cargo new` binárist hoz létre; a `src/lib.rs`-t kézzel adjuk hozzá a 2. lépésben, a Cargo mindkettőt felismeri: a `rkv_server` könyvtár- és az `rkv-server` bináris célt.)

Kódblokk — fájl: `Cargo.toml` (workspace gyökér) · művelet: az `"rkv-server"` felvétele a `members` listába · ellenőrzés: `cargo metadata --no-deps` listázza a tagot

```toml
[workspace]
resolver = "3"
members = [
    "rkv-core",
    "rkv-cli",
    "rkv-server",
]
```

Kódblokk — fájl: `rkv-server/Cargo.toml` · művelet: teljes csere · ellenőrzés: `cargo build -p rkv-server`

```toml
[package]
name = "rkv-server"
version = "0.1.0"
edition = "2024"
rust-version = "1.89"

[dependencies]
rkv-core = { path = "../rkv-core" }
tokio = { version = "1", features = ["full"] }
tokio-util = { version = "0.7", features = ["rt"] }
anyhow = "1"

[dev-dependencies]
tempfile = "3"
```

A `tokio-util` `rt` feature a `CancellationToken` használatához szükséges. (Az `rkv-core` a Miért Rust? Biztonság, teljesítmény, és ami a C++-ban hiányzik óta ismert; a `tempfile` csak a tesztekben kell.)

17.4.8.2 2. lépés: a kiszolgáló logika: `rkv-server/src/lib.rs`

Kódblokk — fájl: `rkv-server/src/lib.rs` · művelet: új fájl · ellenőrzés: `cargo build -p rkv-server`

```rust
use std::io;
use std::sync::Arc;
use std::time::Duration;

use rkv_core::{Command, Response, SharedStore};
use tokio::io::{AsyncBufRead, AsyncBufReadExt, AsyncReadExt, AsyncWriteExt, BufReader,
↪  BufWriter};
use tokio::net::{TcpListener, TcpStream};
use tokio::sync::Semaphore;
use tokio::task::JoinSet;
use tokio_util::sync::CancellationToken;

/// A szerver erőforráskorlátai.
#[derive(Debug, Clone)]
pub struct ServerConfig {
    /// Egy kérés (a sorvéggel együtt) legfeljebb ennyi bájt lehet: a kulcs és az érték együtt.
    pub max_frame: usize,
    /// Egyidejű kapcsolatok felső határa; a többletet "ERR server busy" válasszal utasítjuk el.
    pub max_connections: usize,
    /// A leállás 2. fázisának határideje: eddig várunk a futó klienstaskokra.
    pub shutdown_grace: Duration,
    /// Egy válasz legfeljebb ennyi bájt (a `LIST` miatt); a nagyobbat `ERR response too large`
    ↪  váltja fel.
    pub max_response: usize,
}
```

```rust
impl Default for ServerConfig {
    fn default() -> Self {
        Self {
            max_frame: 64 * 1024,
            max_connections: 1024,
            shutdown_grace: Duration::from_secs(5),
            max_response: 1024 * 1024,
        }
    }
}

/// A `max_frame` felső határa: a `take(max_frame + 1)` és a puffer lefoglalása ennél nagyobbra
↪  nem értelmes.
pub const MAX_FRAME_LIMIT: usize = 64 * 1024 * 1024;

impl ServerConfig {
    /// A belépési ponton is érvényesített ellenőrzés: a publikus mezők a builderen kívül is
    /// beállíthatók, ezért a `run_server_with` ezt hívja. A `Semaphore` kapacitáshatárát is
    ↪  betartja.
    pub fn validate(&self) -> Result<(), ConfigError> {
        if self.max_frame == 0 {
            return Err(ConfigError::Zero("max_frame"));
        }
        if self.max_frame > MAX_FRAME_LIMIT {
            return Err(ConfigError::TooLarge("max_frame"));
        }
        if self.max_connections == 0 {
            return Err(ConfigError::Zero("max_connections"));
        }
        if self.max_connections > Semaphore::MAX_PERMITS {
            return Err(ConfigError::TooLarge("max_connections"));
        }
        if self.max_response == 0 {
            return Err(ConfigError::Zero("max_response"));
        }
        Ok(())
    }
}

/// A `ServerConfig` érvénytelen korlátja (a `validate` és a builder `build` hibája).
#[derive(Debug, PartialEq, Eq)]
#[non_exhaustive]
pub enum ConfigError {
    /// A megnevezett korlát nulla, ami a szervert használhatatlanná tenné.
    Zero(&'static str),
    /// A megnevezett korlát a megengedett felső határnál nagyobb.
    TooLarge(&'static str),
}

impl std::fmt::Display for ConfigError {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        match self {
            ConfigError::Zero(field) => write!(f, "a(z) {field} korlát nem lehet nulla"),
            ConfigError::TooLarge(field) => write!(f, "a(z) {field} korlát túl nagy"),
        }
    }
}

impl std::error::Error for ConfigError {}


/// Egy beolvasott keret eredménye.
#[derive(Debug, PartialEq, Eq)]
```

```
enum Frame {
    /// Teljes, `\n`-nel lezárt sor van a pufferben.
    Line,
    /// A sor hosszabb a megengedettnél (a puffer legfeljebb `max_frame + 1` bájtot tartalmaz).
    TooLong,
    /// A kapcsolat lezárult (egy csonka, sorvég nélküli maradékot eldobunk).
    Eof,
}

/// Egy sort olvas legfeljebb `max_frame` bájtig. A `take` korlátozza a pufferbe kerülő
/// bájtok számát, így sorvég nélküli, végtelen bemenet sem növeli a memóriát.
async fn read_frame<R>(reader: &mut R, buf: &mut Vec<u8>, max_frame: usize) -> io::Result<Frame>
where
    R: AsyncBufRead + Unpin,
{
    buf.clear();
    let n = reader
        .take(max_frame as u64 + 1)
        .read_until(b'\n', buf)
        .await?;
    Ok(if n == 0 {
        Frame::Eof
    } else if buf.len() > max_frame {
        Frame::TooLong
    } else if buf.ends_with(b"\n") {
        Frame::Line
    } else {
        Frame::Eof
    })
}

/// Egy kérés feldolgozása: értelmezés, végrehajtás, válasz. Szinkron és blokkolhat
/// (a `FileStore` fájlműveleteket végez), ezért a hívó `spawn_blocking`-ben futtatja.
///
/// Két réteg van: a szöveges `GET/SET/DEL/LIST` az olvasható, szöveg-biztos
/// kulcsokra és értékekre való; a `GETX/SETX` hexa kulccsal és értékkel bármilyen
/// bájtsort átvisz (az üres értéket a `-` jelöli).
pub fn process_line(line: &str, store: &SharedStore) -> String {
    process_line_with(line, store, ServerConfig::default().max_response)
}

/// Mint a `process_line`, de a válasz legfeljebb `max_response` bájt: a nagyobb `LIST`-et
/// `ERR response too large` váltja fel (a részleges lista félrevezető volna). A tároló
/// `list` hívása még a teljes találati halmazt másolja; a valódi folyamatos küldéshez
/// iterátor-alapú `Storage` API kellene.
pub fn process_line_with(line: &str, store: &SharedStore, max_response: usize) -> String {
    let reply = process_line_inner(line, store);
    if reply.len() > max_response {
        "ERR response too large".to_string()
    } else {
        reply
    }
}

fn process_line_inner(line: &str, store: &SharedStore) -> String {
    if let Some(reply) = process_binary(line, store) {
        return reply;
    }
    let command = match line.parse::<Command>() {
        Ok(command) => command,
        Err(e) => return single_line(&format!("ERR {e}")),
    };
    let response = match store.apply(&command) {
```

```
        Ok(response) => response,
        Err(e) => return single_line(&format!("ERR {e}")),
    };
    match (&command, response) {
        (_, Response::Ok) => "OK".to_string(),
        (_, Response::NotFound) => "NOTFOUND".to_string(),
        (Command::Get(key), Response::Value(bytes)) => match text_safe(key, &bytes) {
            Some(text) => format!("VALUE {key} {text}"),
            None => "ERR value is not text-safe, use GETX".to_string(),
        },
        (_, Response::Entries(items)) => {
            let mut out = String::new();
            for (key, bytes) in items {
                match text_safe(&key, &bytes) {
                    Some(text) => out.push_str(&format!("VALUE {key} {text}\n")),
                    // Ami szövegként nem ábrázolható, az hexa alakban jön: VALUEX <hexkulcs>
                    ↪  <hexérték>
                    None => out.push_str(&format!(
                        "VALUEX {} {}\n",
                        to_hex(key.as_bytes()),
                        to_hex(&bytes)
                    )),
                }
            }
            out.push_str("OK");
            out
        }
        (_, Response::Value(_)) => "ERR internal error".to_string(),
    }
}

/// Szöveg-biztos-e a bejegyzés: a kulcs érvényes protokollkulcs (nem üres, nincs benne
/// szóköz vagy vezérlőkarakter), az érték érvényes UTF-8 és nincs benne vezérlőkarakter
/// (így nem tud új választ kezdeni).
fn text_safe<'a>(key: &str, bytes: &'a [u8]) -> Option<&'a str> {
    let key_ok = !key.is_empty() && !key.chars().any(|c| c.is_whitespace() || c.is_control());
    let text = std::str::from_utf8(bytes).ok()?;
    (key_ok && !text.is_empty() && !text.chars().any(char::is_control)).then_some(text)
}

fn to_hex(bytes: &[u8]) -> String {
    if bytes.is_empty() {
        return "-".to_string();
    }
    bytes.iter().map(|b| format!("{b:02x}")).collect()
}

fn from_hex(text: &str) -> Result<Vec<u8>, String> {
    if text == "-" {
        return Ok(Vec::new());
    }
    if text.is_empty() || !text.len().is_multiple_of(2) || !text.is_ascii() {
        return Err("invalid hex".to_string());
    }
    (0..text.len())
        .step_by(2)
        .map(|i| u8::from_str_radix(&text[i..i + 2], 16).map_err(|_| "invalid hex".to_string()))
        .collect()
}

/// `SETX <hexkulcs> <hexérték|->` és `GETX <hexkulcs>`; más sorra `None`.
fn process_binary(line: &str, store: &SharedStore) -> Option<String> {
    let (verb, rest) = line.split_once(' ').unwrap_or((line, ""));
```

```
    let result = match verb {
        "SETX" => {
            let (k, v) = rest.split_once(' ')?;
            decode_key(k).and_then(|key| {
                let value = from_hex(v)?;
                store.set(key, value).map_err(|e| e.to_string())?;
                Ok("OK".to_string())
            })
        }
        "GETX" => {
            decode_key(rest).and_then(|key| match store.get(&key).map_err(|e| e.to_string())? {
                Some(bytes) => Ok(format!(
                    "VALUEX {} {}",
                    to_hex(key.as_bytes()),
                    to_hex(&bytes)
                )),
                None => Ok("NOTFOUND".to_string()),
            })
        }
        _ => return None,
    };
    Some(result.unwrap_or_else(|e| single_line(&format!("ERR {e}"))))
}

fn decode_key(hex: &str) -> Result<String, String> {
    let bytes = from_hex(hex)?;
    match String::from_utf8(bytes) {
        Ok(key) if !key.is_empty() => Ok(key),
        _ => Err("key must be non-empty UTF-8".to_string()),
    }
}

/// A protokoll soronkénti: a hibaüzenetekből eltávolítjuk a sortörést.
fn single_line(text: &str) -> String {
    text.replace(['\n', '\r'], " ")
}

async fn reply<W>(writer: &mut W, response: &str) -> io::Result<()>
where
    W: AsyncWriteExt + Unpin,
{
    writer.write_all(response.as_bytes()).await?;
    writer.write_all(b"\n").await?;
    writer.flush().await
}

async fn handle_client(
    socket: TcpStream,
    store: SharedStore,
    token: CancellationToken,
    max_frame: usize,
    max_response: usize,
) -> io::Result<()> {
    let (read_half, write_half) = socket.into_split();
    let mut reader = BufReader::new(read_half);
    let mut writer = BufWriter::new(write_half);
    let mut buf = Vec::new();

    loop {
        // A beolvasás megszakítható (a leállítás kilép a ciklusból); a már beolvasott
        // parancsot viszont mindig végrehajtjuk és megválaszoljuk.
        let frame = tokio::select! {
            _ = token.cancelled() => break,
```

```
            frame = read_frame(&mut reader, &mut buf, max_frame) => frame?,
        };
        match frame {
            Frame::Eof => break,
            Frame::TooLong => {
                // A keretezés nem állítható vissza: hibaüzenet, majd lezárás.
                reply(&mut writer, "ERR line too long").await?;
                break;
            }
            Frame::Line => {
                let line = buf.strip_suffix(b"\n").unwrap_or(&buf);
                let line = line.strip_suffix(b"\r").unwrap_or(line);
                let response = match std::str::from_utf8(line) {
                    Ok(line) => {
                        let (line, store) = (line.to_owned(), store.clone());
                        // A tárolóművelet blokkolhat: nem az executor szálán fut.
                        tokio::task::spawn_blocking(move || {
                            process_line_with(&line, &store, max_response)
                        })
                        .await
                        .unwrap_or_else(|_| "ERR internal error".to_string())
                    }
                    Err(_) => "ERR request is not valid UTF-8".to_string(),
                };
                reply(&mut writer, &response).await?;
            }
        }
    }
    // A puffer ürítése és a küldő irány lezárása kilépés előtt.
    writer.shutdown().await
}

/// A szerver az alapértelmezett korlátokkal.
pub async fn run_server(listener: TcpListener, store: SharedStore, token: CancellationToken) {
    run_server_with(listener, store, token, ServerConfig::default()).await;
}

/// A szerver megadott korlátokkal. Visszatérése előtt minden klienstaskot megvár (vagy határidő
↪  után megszakít).
pub async fn run_server_with(
    listener: TcpListener,
    store: SharedStore,
    token: CancellationToken,
    config: ServerConfig,
) {
    config.validate().expect("érvénytelen ServerConfig");
    const REJECT_LIMIT: usize = 64;
    let reject_slots = Arc::new(Semaphore::new(REJECT_LIMIT));
    let slots = Arc::new(Semaphore::new(config.max_connections));
    let mut clients = JoinSet::new();
    loop {
        tokio::select! {
            _ = token.cancelled() => break,
            res = listener.accept() => {
                let (mut socket, addr) = match res {
                    Ok(v) => v,
                    Err(e) => { eprintln!("accept error: {e}"); continue; }
                };
                let Ok(permit) = Arc::clone(&slots).try_acquire_owned() else {
                    // Betelt a kapcsolatkeret: rövid, korlátos idejű elutasítás, korlátos számú
                    // párhuzamos elutasító taskkal; a többi többlet-kapcsolat azonnal lezárul.
                    if let Ok(reject_permit) = Arc::clone(&reject_slots).try_acquire_owned() {
                        clients.spawn(async move {
```

```
                            let _reject_permit = reject_permit;
                            let _ = tokio::time::timeout(
                                Duration::from_millis(200),
                                socket.write_all(b"ERR server busy\n"),
                            ).await;
                        });
                    }
                    continue;
                };
                let (store, token) = (store.clone(), token.clone());
                clients.spawn(async move {
                    // A permit a task végéig él: a kapcsolat bezárásakor felszabadul.
                    let _permit = permit;
                    if let Err(e) = handle_client(socket, store, token, config.max_frame,
                    ↪  config.max_response).await {
                        eprintln!("client {addr}: {e}");
                    }
                });
            }
            // A befejeződött taskok eredményét folyamatosan begyűjtjük.
            Some(res) = clients.join_next(), if !clients.is_empty() => {
                if let Err(e) = res {
                    eprintln!("client task: {e}");
                }
            }
        }
    }

    // 1. fázis: új kapcsolatot nem fogadunk.
    drop(listener);

    // 2. fázis: a futó parancsok befejezése és a válaszok ürítése, határidővel.
    let drain = async {
        while let Some(res) = clients.join_next().await {
            if let Err(e) = res {
                eprintln!("client task: {e}");
            }
        }
    };
    if tokio::time::timeout(config.shutdown_grace, drain).await.is_err() {
        eprintln!("shutdown grace expired; aborting {} client(s)", clients.len());
        clients.shutdown().await;
    }
}
```

A `ServerConfig` három korlátot rögzít: `max_frame` (egy kérés bájtban, a sorvéggel együtt), `max_connections` (a `Semaphore` permitjei; egy permit a kapcsolat teljes élettartama alatt foglalt, a task végén felszabadul), `shutdown_grace` (a leállás 2. fázisának határideje). A túllépés protokoll szerinti hiba: `ERR line too long`, illetve `ERR server busy`.

17.4.8.3 3. lépés: az indítóprogram: `rkv-server/src/main.rs`

Kódblokk — fájl: `rkv-server/src/main.rs` · művelet: teljes csere · ellenőrzés: `cargo build -p rkv-server`

```
use anyhow::Result;
use rkv_core::{FileStore, MemStore, SharedStore};
use rkv_server::run_server;
use tokio::net::TcpListener;
use tokio_util::sync::CancellationToken;
```

```rust
fn main() -> Result<()> {
    let rt = tokio::runtime::Runtime::new()?;
    let outcome = rt.block_on(serve());
    // A runtime eldobása a futó `spawn_blocking` munkát korlátlanul megvárná: időkorlátot
    ↪  adunk.
    // Ha lejár, a folyamat a háttérben futó tárolóművelet befejezése nélkül lép ki.
    rt.shutdown_timeout(std::time::Duration::from_secs(5));
    outcome
}

async fn serve() -> Result<()> {
    // Használat: rkv-server [cím] [--data-dir <könyvtár>]
    // `--data-dir` nélkül memóriabeli tároló; vele a FileStore (snapshot + napló) fut.
    let mut addr = "127.0.0.1:8080".to_string();
    let mut data_dir: Option<std::path::PathBuf> = None;
    let mut args = std::env::args().skip(1);
    while let Some(arg) = args.next() {
        if arg == "--data-dir" {
            data_dir = Some(
                args.next()
                    .ok_or_else(|| anyhow::anyhow!("a --data-dir érték hiányzik"))?
                    .into(),
            );
        } else {
            addr = arg;
        }
    }
    let listener = TcpListener::bind(&addr).await?;
    println!("rkv-server listening on {addr}");

    let store = SharedStore::new(MemStore::new());
    let token = CancellationToken::new();
    let shutdown_token = token.clone();

    tokio::spawn(async move {
        if let Err(e) = tokio::signal::ctrl_c().await {
            eprintln!("ctrl_c failed: {e}");
            return;
        }
        println!("shutdown signal received");
        shutdown_token.cancel();
    });

    run_server(listener, store, token).await;
    println!("shutdown complete");
    Ok(())
}
```

A szerver `--data-dir <könyvtár>` kapcsolóval tartós háttérrel indul (`cargo run -p rkv-server -- 127.0.0.1:8080 --data-dir ~/rkv-adat`): ilyenkor a `FileStore` a könyvtár `state.json` snapshotját és `append.log` naplóját használja, kapcsoló nélkül memóriabeli tároló fut. A `tokio::signal::ctrl_c()` a Unix `SIGINT` jelet aszinkron módon várja. A `tracing`-es naplózást a Naplózás és megfigyelhetőség: tracing alfejezetben tesszük hozzá.

17.4.8.4 4. lépés: fordítás és kézi teszt

```
cd ~/rust-labor/rkv
cargo run -p rkv-server
```

Egy másik terminálon (az `nc` a sorokat a szerver `\n` keretezéséhez igazítja):

```
printf 'SET hello world\nGET hello\nLIST he\nDEL hello\nGET hello\n' | nc -q1 localhost 8080
```

Várt kimenet:

```
OK
VALUE hello world
VALUE hello world
OK
OK
NOTFOUND
```

A `LIST he` két sort ad: a `hello` kulcs `VALUE` sorát és a záró `OK`-t. Az `nc -q1` kapcsolója (Linux, netcat-openbsd) a bemenet vége után egy másodpercig vár a válaszokra.

17.4.8.5 5. lépés: integrációs tesztek

A teszteket a tokio saját tesztkerete futtatja, és **minden hálózati várakozásnak időkorlátja van**: egy hiányzó `flush` vagy egy el nem induló leállítás így hibás tesztet ad, nem végtelenül váró tesztfuttatót. Az első fájl a protokollt, a hibás kéréseket és a korlátokat ellenőrzi:

Kódblokk — fájl: `rkv-server/tests/server.rs` · művelet: új fájl · ellenőrzés: `cargo test -p rkv-server --test server` → 5 sikeres teszt

```
use std::time::Duration;

use rkv_core::{MemStore, SharedStore};
use rkv_server::{run_server, run_server_with, ServerConfig};
use tokio::io::{AsyncBufReadExt, AsyncReadExt, AsyncWriteExt, BufReader};
use tokio::net::{TcpListener, TcpStream};
use tokio::time::timeout;
use tokio_util::sync::CancellationToken;

const T: Duration = Duration::from_secs(5);

/// Elindít egy szervert egy szabad porton; visszaadja a címet, a tokent és a szerver taskját.
async fn start(config: Option<ServerConfig>) -> (std::net::SocketAddr, CancellationToken,
↪  tokio::task::JoinHandle<()>) {
    let listener = TcpListener::bind("127.0.0.1:0").await.unwrap();
    let addr = listener.local_addr().unwrap();
    let store = match &data_dir {
        Some(dir) => {
            std::fs::create_dir_all(dir)?;
            SharedStore::new(FileStore::open(
                dir.join("state.json"),
                dir.join("append.log"),
            )?)
        }
        None => SharedStore::new(MemStore::new()),
    };
    let token = CancellationToken::new();
    let server = match config {
        Some(config) => tokio::spawn(run_server_with(listener, store, token.clone(), config)),
        None => tokio::spawn(run_server(listener, store, token.clone())),
    };
    (addr, token, server)
}

async fn stop(token: CancellationToken, server: tokio::task::JoinHandle<()>) {
    token.cancel();
    timeout(Duration::from_secs(10), server)
        .await
```

```
        .expect("a szerver nem állt le határidőn belül")
        .unwrap();
}

#[tokio::test]
async fn set_get_delete() {
    let (addr, token, server) = start(None).await;

    let stream = TcpStream::connect(addr).await.unwrap();
    let (read_half, mut write_half) = stream.into_split();
    let mut lines = BufReader::new(read_half).lines();

    // Egy kérés, egy válasz: nyitva tartott kapcsolaton is megérkezik, EOF nélkül.
    write_half.write_all(b"SET hello world\n").await.unwrap();
    let reply = timeout(T, lines.next_line()).await.expect("nincs válasz").unwrap();
    assert_eq!(reply.as_deref(), Some("OK"));

    for (request, expected) in [
        ("GET hello\n", "VALUE hello world"),
        ("DEL hello\n", "OK"),
        ("GET hello\n", "NOTFOUND"),
    ] {
        write_half.write_all(request.as_bytes()).await.unwrap();
        let reply = timeout(T, lines.next_line()).await.expect("nincs válasz").unwrap();
        assert_eq!(reply.as_deref(), Some(expected));
    }

    stop(token, server).await;
}

#[tokio::test]
async fn list_ends_with_ok_line() {
    let (addr, token, server) = start(None).await;
    let stream = TcpStream::connect(addr).await.unwrap();
    let (read_half, mut write_half) = stream.into_split();
    let mut lines = BufReader::new(read_half).lines();

    for k in ["app::1", "app::2", "other"] {
        write_half.write_all(format!("SET {k} v\n").as_bytes()).await.unwrap();
        timeout(T, lines.next_line()).await.unwrap().unwrap();
    }
    write_half.write_all(b"LIST app\n").await.unwrap();
    let mut got = Vec::new();
    loop {
        let line = timeout(T, lines.next_line()).await.expect("nincs válasz").unwrap().unwrap();
        if line == "OK" { break; }
        got.push(line);
    }
    assert_eq!(got, ["VALUE app::1 v", "VALUE app::2 v"]);
    stop(token, server).await;
}

#[tokio::test]
async fn oversized_line_without_newline_is_rejected_with_bounded_memory() {
    let config = ServerConfig { max_frame: 1024, ..ServerConfig::default() };
    let (addr, token, server) = start(Some(config)).await;

    let mut stream = TcpStream::connect(addr).await.unwrap();
    // 1 MiB sorvég nélkül: a szerver a határnál elutasít, nem tárolja el az egészet.
    let junk = vec![b'a'; 1 << 20];
    let _ = stream.write_all(&junk).await; // lehet, hogy a szerver már lezárta
    let mut reply = String::new();
    let _ = timeout(T, BufReader::new(&mut stream).read_line(&mut reply)).await.expect("nincs
    ↪ válasz");
```

```
    assert_eq!(reply.trim_end(), "ERR line too long");
    // A kapcsolatot a szerver lezárta: a maradék olvasása véget ér.
    let mut rest = Vec::new();
    let _ = timeout(T, stream.read_to_end(&mut rest)).await.expect("nem zárult le");

    stop(token, server).await;
}

#[tokio::test]
async fn connection_limit_rejects_the_excess_client() {
    let config = ServerConfig { max_connections: 2, ..ServerConfig::default() };
    let (addr, token, server) = start(Some(config)).await;

    let a = TcpStream::connect(addr).await.unwrap();
    let b = TcpStream::connect(addr).await.unwrap();
    // Mindkét kapcsolat kiszolgált, tehát mindkét permit foglalt.
    let mut a_lines = BufReader::new(a);
    a_lines.get_mut().write_all(b"SET k v\n").await.unwrap();
    let mut line = String::new();
    timeout(T, a_lines.read_line(&mut line)).await.unwrap().unwrap();
    assert_eq!(line.trim_end(), "OK");
    let mut b_lines = BufReader::new(b);
    b_lines.get_mut().write_all(b"GET k\n").await.unwrap();
    line.clear();
    timeout(T, b_lines.read_line(&mut line)).await.unwrap().unwrap();
    assert_eq!(line.trim_end(), "VALUE k v");

    // A harmadik kliens elutasítást kap.
    let c = TcpStream::connect(addr).await.unwrap();
    let mut c_lines = BufReader::new(c);
    line.clear();
    timeout(T, c_lines.read_line(&mut line)).await.expect("nincs válasz").unwrap();
    assert_eq!(line.trim_end(), "ERR server busy");

    // Egy kapcsolat bezárása után újra van hely.
    drop(a_lines);
    let mut accepted = false;
    for _ in 0..50 {
        let d = TcpStream::connect(addr).await.unwrap();
        let mut d_lines = BufReader::new(d);
        d_lines.get_mut().write_all(b"GET k\n").await.unwrap();
        line.clear();
        timeout(T, d_lines.read_line(&mut line)).await.unwrap().unwrap();
        if line.trim_end() == "VALUE k v" { accepted = true; break; }
        tokio::time::sleep(Duration::from_millis(20)).await;
    }
    assert!(accepted, "a felszabadult hely nem lett újra használható");

    drop(b_lines);
    stop(token, server).await;
}

#[tokio::test]
async fn invalid_requests_get_err_replies() {
    let (addr, token, server) = start(None).await;
    let stream = TcpStream::connect(addr).await.unwrap();
    let (read_half, mut write_half) = stream.into_split();
    let mut lines = BufReader::new(read_half).lines();

    for request in [&b"FOO x\n"[..], b"GET\n", b"SET k\n", b"\xff\xfe\n"] {
        write_half.write_all(request).await.unwrap();
        let reply = timeout(T, lines.next_line()).await.expect("nincs
        ↪ válasz").unwrap().unwrap();
```

```
        assert!(reply.starts_with("ERR "), "{reply}");
    }
    // A kapcsolat a hibák után is használható.
    write_half.write_all(b"SET k v\n").await.unwrap();
    assert_eq!(timeout(T, lines.next_line()).await.unwrap().unwrap().as_deref(), Some("OK"));
    stop(token, server).await;
}
```

A második fájl a leállítást és a lassú tárolót vizsgálja. A `SlowStore` blokkolja a szálat egy `set` közben (mint egy lassú fájlművelet). Az első teszt *egyszálú* (`current_thread`) runtime-on fut: ha a művelet az executor szálán futna, a `ticker` task megállna, és a tickek száma nem érné el a 15-öt; a `spawn_blocking` miatt halad. A második teszt a leállítás menetét igazolja: a folyamatban lévő parancs válasza megérkezik, a tétlen kliens EOF-ot kap, és a szerver csak ezután tér vissza:

Kódblokk — fájl: `rkv-server/tests/shutdown.rs` · művelet: új fájl · ellenőrzés: `cargo test -p rkv-server --test shutdown` → 3 sikeres teszt

```
use std::sync::atomic::{AtomicUsize, Ordering};
use std::sync::Arc;
use std::time::Duration;

use rkv_core::{MemStore, Result, SharedStore, Storage};
use rkv_server::run_server;
use tokio::io::{AsyncBufReadExt, AsyncReadExt, AsyncWriteExt, BufReader};
use tokio::net::{TcpListener, TcpStream};
use tokio::time::timeout;
use tokio_util::sync::CancellationToken;

const T: Duration = Duration::from_secs(5);

/// Lassú háttér: a `set` blokkolja a szálat (mint egy lassú fájlművelet).
struct SlowStore(MemStore);

impl Storage for SlowStore {
    fn get(&self, key: &str) -> Result<Option<Vec<u8>>> { Storage::get(&self.0, key) }
    fn set(&mut self, key: String, value: Vec<u8>) -> Result<()> {
        std::thread::sleep(Duration::from_millis(400));
        Storage::set(&mut self.0, key, value)
    }
    fn delete(&mut self, key: &str) -> Result<bool> { Storage::delete(&mut self.0, key) }
    fn list(&self, prefix: Option<&str>) -> Result<Vec<(String, Vec<u8>)>> {
     ↪  Storage::list(&self.0, prefix) }
}

async fn start(store: SharedStore) -> (std::net::SocketAddr, CancellationToken,
 ↪  tokio::task::JoinHandle<()>) {
    let listener = TcpListener::bind("127.0.0.1:0").await.unwrap();
    let addr = listener.local_addr().unwrap();
    let token = CancellationToken::new();
    let server = tokio::spawn(run_server(listener, store, token.clone()));
    (addr, token, server)
}

// Egyszálas (current_thread) runtime: ha a lassú tárolóművelet az executor szálán futna,
// a `ticker` task megállna.
#[tokio::test(flavor = "current_thread")]
async fn slow_storage_does_not_stall_the_executor() {
    let (addr, token, server) = start(SharedStore::new(SlowStore(MemStore::new()))).await;

    let ticks = Arc::new(AtomicUsize::new(0));
    let ticker = {
        let ticks = Arc::clone(&ticks);
```

```
        tokio::spawn(async move {
            loop {
                tokio::time::sleep(Duration::from_millis(10)).await;
                ticks.fetch_add(1, Ordering::SeqCst);
            }
        })
    };

    let mut client = BufReader::new(TcpStream::connect(addr).await.unwrap());
    let before = ticks.load(Ordering::SeqCst);
    client.get_mut().write_all(b"SET k v\n").await.unwrap(); // 400 ms-ig blokkol a háttérben
    let mut line = String::new();
    timeout(T, client.read_line(&mut line)).await.unwrap().unwrap();
    assert_eq!(line.trim_end(), "OK");
    let during = ticks.load(Ordering::SeqCst) - before;
    assert!(during >= 15, "az executor megállt a lassú művelet alatt: {during} tick");

    ticker.abort();
    token.cancel();
    timeout(Duration::from_secs(10), server).await.unwrap().unwrap();
}

#[tokio::test]
async fn shutdown_waits_for_the_inflight_command_and_closes_idle_clients() {
    let (addr, token, server) = start(SharedStore::new(SlowStore(MemStore::new()))).await;

    let idle = TcpStream::connect(addr).await.unwrap();
    let busy = TcpStream::connect(addr).await.unwrap();
    let (busy_read, mut busy_write) = busy.into_split();
    let mut busy_lines = BufReader::new(busy_read).lines();

    busy_write.write_all(b"SET k v\n").await.unwrap(); // 400 ms-ig tart
    tokio::time::sleep(Duration::from_millis(100)).await; // a parancs már folyamatban van
    token.cancel(); // leállítási jel a parancs közben

    // A folyamatban lévő parancs válasza megérkezik, utána a kapcsolat EOF-fal zárul.
    let reply = timeout(T, busy_lines.next_line()).await.expect("nincs válasz").unwrap();
    assert_eq!(reply.as_deref(), Some("OK"));
    assert_eq!(timeout(T, busy_lines.next_line()).await.unwrap().unwrap(), None);

    // A tétlen kliens EOF-ot kap.
    let mut idle = idle;
    let mut rest = Vec::new();
    assert_eq!(timeout(T, idle.read_to_end(&mut rest)).await.unwrap().unwrap(), 0);

    // A szerver csak a kliensek lezárása után tér vissza.
    timeout(Duration::from_secs(10), server).await.expect("a szerver nem állt le").unwrap();
}

/// Lassú tároló, amely jelzi, hogy a `set` befejeződött.
struct FlagStore {
    inner: MemStore,
    done: Arc<std::sync::atomic::AtomicBool>,
}

impl Storage for FlagStore {
    fn get(&self, key: &str) -> Result<Option<Vec<u8>>> {
        Storage::get(&self.inner, key)
    }
    fn set(&mut self, key: String, value: Vec<u8>) -> Result<()> {
        std::thread::sleep(Duration::from_millis(500));
        let r = Storage::set(&mut self.inner, key, value);
        self.done.store(true, Ordering::SeqCst);
```

```
        r
    }
    fn delete(&mut self, key: &str) -> Result<bool> {
        Storage::delete(&mut self.inner, key)
    }
    fn list(&self, prefix: Option<&str>) -> Result<Vec<(String, Vec<u8>)>> {
        Storage::list(&self.inner, prefix)
    }
}

// A szerződés őszinte határa: a `shutdown_grace` a kapcsolatok lezárására szól. A már
// elindult blokkoló tárolóművelet nem szakítható meg, ezért a `run_server` visszatérése
// után is futhat; a befejezése külön esemény.
#[tokio::test]
async fn run_server_may_return_before_the_blocking_store_operation_ends() {
    use rkv_server::{ServerConfig, run_server_with};
    use std::sync::atomic::AtomicBool;

    let done = Arc::new(AtomicBool::new(false));
    let store = SharedStore::new(FlagStore {
        inner: MemStore::new(),
        done: Arc::clone(&done),
    });
    let listener = TcpListener::bind("127.0.0.1:0").await.unwrap();
    let addr = listener.local_addr().unwrap();
    let token = CancellationToken::new();
    let config = ServerConfig::builder()
        .shutdown_grace(Duration::from_millis(50))
        .build()
        .unwrap();
    let server = tokio::spawn(run_server_with(listener, store, token.clone(), config));

    let mut client = TcpStream::connect(addr).await.unwrap();
    client.write_all(b"SET k v\n").await.unwrap(); // 500 ms-ig blokkol a háttérben
    tokio::time::sleep(Duration::from_millis(100)).await;
    token.cancel();

    // 1. a run_server a határidő után visszatér (a kapcsolatot megszakították) ...
    timeout(Duration::from_millis(400), server)
        .await
        .expect("a run_server nem tért vissza a határidő után")
        .unwrap();
    // 2. ... de a tárolóművelet ekkor még fut,
    assert!(!done.load(Ordering::SeqCst), "a művelet még nem végzett");
    // 3. és a befejezése külön, később következik be.
    tokio::time::sleep(Duration::from_millis(700)).await;
    assert!(done.load(Ordering::SeqCst), "a művelet végül befejeződik");
}
```

Futtassuk az összes tesztet:

```
cargo test -p rkv-server
```

Várt kimenet (rövidítve):

```
test result: ok. 5 passed; 0 failed; ...
test result: ok. 2 passed; 0 failed; ...
```

17.4.8.6 6. lépés: leállítási teszt kézzel

Indítsuk el a szervert, majd küldjünk egy `Ctrl-C`-t. A terminálban a kimenet:

```
rkv-server listening on 127.0.0.1:8080
^Cshutdown signal received
shutdown complete
```

Ha egy kliens éppen kapcsolódva van, és nem küld parancsot, a klienstask a token megszakadásakor kilép a sorolvasásból, a `shutdown` lezárja a kapcsolatot, és a kliens fájlvéget (EOF) kap. Ha a kliens parancsa már beérkezett, előbb megkapja rá a választ. A `shutdown complete` csak az összes klienstask befejeződése (vagy a `shutdown_grace` utáni megszakítása) után jelenik meg. A négy leállási eset (tétlen kliens, folyamatban lévő `SET`, lassú olvasó, több kapcsolat) közül az első kettőt a `shutdown.rs`, a harmadikat a `shutdown_grace` korlát, a negyediket a `JoinSet` biztosítja.

A bináris réteg és a korlátok tesztjei külön fájlokba kerülnek:

Kódblokk — fájl: `rkv-server/tests/binary.rs` · művelet: új fájl · ellenőrzés: `cargo test -p rkv-server --test binary` → 2 sikeres teszt

```rust
use rkv_core::{SharedStore, Store};
use rkv_server::process_line;

fn store() -> SharedStore {
    SharedStore::new(Store::new())
}

#[test]
fn binary_value_round_trip() {
    let s = store();
    // [0, 255, 10, 13] és az üres érték; a kulcs "bin" (hexa: 62696e).
    assert_eq!(process_line("SETX 62696e 00ff0a0d", &s), "OK");
    assert_eq!(process_line("GETX 62696e", &s), "VALUEX 62696e 00ff0a0d");
    assert_eq!(s.get("bin").unwrap().unwrap(), vec![0, 255, 10, 13]);
    assert_eq!(process_line("SETX 656d707479 -", &s), "OK");
    assert_eq!(process_line("GETX 656d707479", &s), "VALUEX 656d707479 -");
    assert_eq!(s.get("empty").unwrap().unwrap(), Vec::<u8>::new());
    assert!(process_line("SETX zz 00", &s).starts_with("ERR"));
}

#[test]
fn unsafe_entries_cannot_break_the_text_reply() {
    let s = store();
    // Érvényes UTF-8 értékben sortörés + OK, szóközös kulcs: a szöveges válasz nem torzulhat.
    s.set("a b".into(), b"x".to_vec()).unwrap();
    s.set("k".into(), b"v\nOK".to_vec()).unwrap();
    s.set("ok".into(), b"fine".to_vec()).unwrap();
    assert_eq!(
        process_line("GET k", &s),
        "ERR value is not text-safe, use GETX"
    );
    let list = process_line("LIST", &s);
    let lines: Vec<&str> = list.split('\n').collect();
    // Három bejegyzés + záró OK: pontosan 4 sor, a veszélyes sorok VALUEX alakban.
    assert_eq!(lines.len(), 4, "{list:?}");
    assert_eq!(lines.last(), Some(&"OK"));
    assert!(lines.contains(&"VALUE ok fine"));
    assert_eq!(lines.iter().filter(|l| l.starts_with("VALUEX ")).count(), 2);
}
```

Kódblokk — fájl: `rkv-server/tests/limits.rs` · művelet: új fájl · ellenőrzés: `cargo test -p rkv-server --test limits` → 2 sikeres teszt

```rust
use rkv_core::{MemStore, SharedStore};
use rkv_server::{ConfigError, ServerConfig, process_line_with};
```

```rust
#[test]
fn config_validation_rejects_huge_values_even_with_public_fields() {
    let huge = ServerConfig {
        max_frame: usize::MAX,
        ..ServerConfig::default()
    };
    assert_eq!(huge.validate(), Err(ConfigError::TooLarge("max_frame")));
    let many = ServerConfig {
        max_connections: usize::MAX,
        ..ServerConfig::default()
    };
    assert_eq!(
        many.validate(),
        Err(ConfigError::TooLarge("max_connections"))
    );
    assert!(
        ServerConfig::builder()
            .max_frame(usize::MAX)
            .build()
            .is_err()
    );
    assert!(ServerConfig::default().validate().is_ok());
}

#[test]
fn oversized_list_reply_is_replaced_by_an_error() {
    let store = SharedStore::new(MemStore::new());
    for i in 0..50 {
        store.set(format!("key{i:03}"), vec![b'x'; 100]).unwrap();
    }
    assert_eq!(
        process_line_with("LIST", &store, 1_000),
        "ERR response too large"
    );
    assert!(process_line_with("LIST", &store, 1_000_000).ends_with("\nOK"));
}
```

Ellenőrzési pont. Az `rkv-server` crate könyvtár (`run_server`, `run_server_with`, `ServerConfig`, `process_line`) és bináris; a `cargo test --workspace` az $5 + 3 + 4$ szerverteszttel együtt sikeres, a `cargo clippy --workspace --all-targets -- -D warnings` hibátlan.

17.4.9 Tipikus hibák és átállási csapdák

1. **Zár tartása `.await`-en át.** A `std::sync::MutexGuard` nem `Send`, ezért `.await`-en át tartva a handler future `!Send`, és a `tokio::spawn` elutasítja; a tokio `MutexGuard` `Send`, de `.await`-en át tartva a task várakozás közben birtokolja a zárat, a többi task áll, és holtpont is kialakulhat. A megoldás a fenti `process_line`-minta: a zár hatóköre rövid, szinkron blokk, és a válaszírást zár nélkül végezzük.
2. **A leállítási jel elfelejtése a kliensben.** Ha a kliens task nem figyeli a `CancellationToken`-t, a szerver leállási kérése után a kliens a `next_line()`-ra vár, és a szerver futó taskjai sosem érnek véget. Minden `.await`-et – különösen a hosszú I/O-t – `select!`-ben kell a `token.cancelled()`-tel párhuzamosítani.
3. **`unwrap` mindenhol a záron.** A mérgezés (poisoning) kezelését a `SharedStore` központosítja (`lock()` hibát ad, nem folytat), a szerver kódjában nincs `lock().unwrap()`. A `process_line` `String`-et ad, ezért a hibák `ERR ...` szöveggé alakítása a protokoll része.

4. **A `LIST` válaszának rossz keretezése.** Ha minden `VALUE` sor után a `reply` külön `\n`-t írna, a `LIST` belső sortörései és a hozzáfűzés dupla sortörést eredményezne. A `process_line` egyetlen stringet épít, benne a szükséges `\n`-ekkel, és a `reply` csak a válasz végére tesz egyet.
5. **Korlátlan olvasás.** A `lines()`/`read_line` sorvég nélküli bemenetre korlátlanul növeli a puffert; hálózatra nyitott szervernél mindig korlátozott olvasót (`take`, `LinesCodec::new_with_max_length`) használjunk, és a túllépést protokollhibával jelezzük.
6. **Blokkoló tárolóművelet az executor szálán.** Ha a `process_line`-t közvetlenül a kliens taskban hívjuk, egy lassú `FileStore`-írás az executor szálát köti le, és az összes többi kapcsolat megáll. A `spawn_blocking` ezt elkerüli.

17.4.10 Feladatok

17.4.10.1 a) Lefordul-e és miért?

Az alábbi tokio kliens task `select!`-ben a `token.cancelled()`-et és a korlátos beolvasást versenyezteti:

```
let frame = tokio::select! {
    _ = token.cancelled() => break,
    frame = read_frame(&mut reader, &mut buf, max_frame) => frame?,
};
```

Miért nem okoz ez lefordulási hibát? Mi történik, ha a token már a `select!` előtt meg volt szakítva? Mi a baj azzal, hogy a `read_frame` wrapper nem folytatható megszakítás után (holott a `read_until` maga megszakításbiztos), és miért elfogadható itt?

17.4.10.2 b) C++ kód átírása idiomatikus Rustra

Adott egy egyszerű C++ echo-szerver, amely minden bejövő sort visszaküld. Írd át a kapcsolatkezelőt Rust/tokio `select!`-tel és `CancellationToken`-nel kecses leállással. A szerver globális leállási flag helyett tokent használjon.

17.4.10.3 c) Implementálási feladat az rkv-ban

Írj integrációs tesztet, amely igazolja, hogy a `LIST` válasza a kulcsok szerint rendezett, és mindig `OK` sorral zárul: üres tárolóra is, és három, fordított sorrendben beszúrt kulcsra is. (A rendezést a `Storage::list` szerződése garantálja, Trait-ek és generikusok: sablonok és concepts helyett; a teszt a szerver végpontjáig igazolja.)

17.4.10.4 d) Refaktorálás vagy tervezési döntés

A leállítás során a kliens, amelyik épp a következő sor olvasására vár, csak a token megszakadásakor lép ki. Ha a kliens nem küld adatot, a leállás azonnal megtörténik, de mi van akkor, ha egy kliens folyamatosan küld parancsokat? Tervezz egy időkorlátot: a leállás után legfeljebb 5 másodpercig engedjük a folyamatban lévő parancsok befejezését, utána kényszerítsük a kilépést. Milyen tokio eszközöket használnál?

17.4.11 Megoldások

17.4.11.1 a) Megoldás

A kód lefordul. A `select!` minden ága egy `Future`-t vár: a `token.cancelled()` és a `read_frame(..)` is az. A `break` kifejezés típusa `!` (soha nem tér vissza), amely tetszőleges típusra kényszeríthető, így a `let frame = ...` kötés típusa a másik ág típusa (itt a `frame?` után `Frame`) marad, és a `break` ágban nem keletkezik érték. Ha a token már a `select!` előtt meg volt szakítva, a `cancelled()` azonnal kész. Ha ilyenkor az olvasás ága is kész (például már van beérkezett sor), a `select!` alapértelmezésben **véletlenszerűen** választ a kész ágak között; a sorrendet a `biased;` kulcsszóval lehet rögzíteni (ekkor a felsorolás sorrendje dönt). Ezért nem szabad arra építeni, hogy egy megszakított token mellett pontosan a `break` ág fut.

A tokio `read_until` megszakításbiztos (a beolvasott bájtok a hívó pufferében maradnak), de a `read_frame` wrapper nem folytatható: ha a `select!` a `cancelled()` ágat választja, a `read_frame` future eldobódik, és a puffer a következő hívás elején `clear()`-rel törlődne, tehát a részleges kérés elveszne. Itt ez elfogadható, mert a megszakítás után a kliens task nem olvas többet: a kapcsolat lezárul, és a megszakadt, nem teljes kérést a kliens úgysem kapja meg. Ha a megszakítás után folytatni kellene az olvasást, a `read_until`-t nem szabadna `select!`-be tenni.

17.4.11.2 b) Megoldás

A C++ echo-szerver kapcsolatkezelőjének Rust megfelelője:

```rust
async fn echo_client(socket: TcpStream, token: CancellationToken) -> Result<()> {
    let (read_half, write_half) = socket.into_split();
    let mut lines = BufReader::new(read_half).lines();
    let mut writer = BufWriter::new(write_half);

    loop {
        let line = tokio::select! {
            _ = token.cancelled() => break,
            res = lines.next_line() => match res? {
                Some(l) => l,
                None => break,
            },
        };
        writer.write_all(line.as_bytes()).await?;
        writer.write_all(b"\n").await?;
        writer.flush().await?; // a BufWriter nélküle visszatartaná a választ
    }
    writer.shutdown().await?;
    Ok(())
}
```

17.4.11.3 c) Megoldás

Kódblokk — fájl: `rkv-server/tests/list_order.rs` · művelet: új fájl · ellenőrzés: `cargo test -p rkv-server --test list_order`

```rust
use std::time::Duration;

use rkv_core::{MemStore, SharedStore};
use rkv_server::run_server;
use tokio::io::{AsyncBufReadExt, AsyncWriteExt, BufReader};
```

```rust
use tokio::net::{TcpListener, TcpStream};
use tokio::time::timeout;
use tokio_util::sync::CancellationToken;

async fn list(lines: &mut tokio::io::Lines<BufReader<tokio::net::tcp::OwnedReadHalf>>) ->
↪  Vec<String> {
    let mut got = Vec::new();
    loop {
        let line = timeout(Duration::from_secs(5),
        ↪  lines.next_line()).await.unwrap().unwrap().unwrap();
        if line == "OK" {
            return got;
        }
        got.push(line);
    }
}

#[tokio::test]
async fn list_is_sorted_and_always_ends_with_ok() {
    let listener = TcpListener::bind("127.0.0.1:0").await.unwrap();
    let addr = listener.local_addr().unwrap();
    let token = CancellationToken::new();
    let server = tokio::spawn(run_server(listener, SharedStore::new(MemStore::new()),
    ↪  token.clone()));

    let (read_half, mut write_half) = TcpStream::connect(addr).await.unwrap().into_split();
    let mut lines = BufReader::new(read_half).lines();

    // Üres tároló: nincs VALUE sor, csak a záró OK.
    write_half.write_all(b"LIST\n").await.unwrap();
    assert!(list(&mut lines).await.is_empty());

    for key in ["b", "c", "a"] {
        write_half.write_all(format!("SET {key} 1\n").as_bytes()).await.unwrap();
        timeout(Duration::from_secs(5), lines.next_line()).await.unwrap().unwrap();
    }
    write_half.write_all(b"LIST\n").await.unwrap();
    assert_eq!(list(&mut lines).await, ["VALUE a 1", "VALUE b 1", "VALUE c 1"]);

    token.cancel();
    timeout(Duration::from_secs(10), server).await.unwrap().unwrap();
}
```

A rendezés nem a szerverben van: a `Storage::list` szerződése kulcs szerint rendezett eredményt ad (a `MemStore` `BTreeMap`-ből, a `FileStore` rendezéssel), a `process_line` pedig a `LIST` válaszát mindig `OK` sorral zárja. Az üres tárolóra adott válasz csak a záró `OK`.

17.4.11.4 d) Megoldás

A tervezési döntés: a tokio `timeout`-ja és a `CancellationToken` kombinálható; ezt a labor teljes `run_server` függvénye már meg is valósítja. A klienstaskokat `JoinSet::spawn` indítja; a token megszakítása után a fő task egy `timeout(shutdown_grace)`-n belül egyenként megvárja őket. Ha a határidő lejár, a `JoinSet::shutdown().await` minden még futó taskot megszakít (`abort`), és a megszakítás befejeződését is megvárja. A `JoinSet` eldobása szintén abortálja a taskokat, de nem várja meg őket; a `main` visszatérése előtt ezért explicit `shutdown` kell. Egy másik lehetőség a `tokio::time::timeout` minden egyes sor olvasására a kliensben, hogy a leállás után ne várjunk új parancsra.

Példa a leállítási szakaszra:

```
let drain = async {
    while let Some(res) = join_set.join_next().await {
        if let Err(e) = res {
            eprintln!("task error: {e}");
        }
    }
};
if tokio::time::timeout(Duration::from_secs(5), drain).await.is_err() {
    join_set.shutdown().await; // a határidőn túl futó taskok megszakítása
}
```

17.4.12 Rövid összefoglaló

Ebben az alfejezetben megépítettük az `rkv-server` tokio-alapú TCP-szervert, amely soralapú szöveges protokollt beszél: `GET`, `SET`, `DEL`, `LIST` parancsokat fogad, és `OK`, `VALUE`, `NOTFOUND`, `ERR` válaszokat ad. A kiszolgáló logika az `rkv-server/src/lib.rs`-ben van (`run_server`, `run_server_with`, `ServerConfig`, `process_line`), a `main.rs` rövid indítóprogram, így az integrációs tesztek importálhatják a könyvtárat. A keretezés korlátos soralapú olvasás (`read_frame`, `max_frame`), a kapcsolatok száma `Semaphore`-ral korlátozott, a hiba protokollválasz (`ERR line too long`, `ERR server busy`). A megosztott állapot az `rkv-core` `SharedStore`-ja; a blokkoló tárolóműveletek `spawn_blocking`-ben futnak, hogy a lassú fájlművelet ne állítsa meg az executort. A kecses leállást `tokio_util::sync::CancellationToken` valósítja meg: a listener kilép, a klienstaskok `JoinSet`-ben futnak, a már beolvasott parancsot végrehajtják és `flush`-elik, a határidő (`shutdown_grace`) után a maradékot megszakítja. **Továbbvitt eredmény:** az `rkv-server` könyvtár, a `run_server`, a `ServerConfig` és a `process_line` a Naplózás és megfigyelhetőség és a Záróprojekt alapja.

17.4.13 Források és ellenőrzési hiányok

- `tokio::net::TcpListener`, `tokio::io::AsyncBufReadExt::read_until` (megszakításbiztos), `AsyncReadExt::take`: https://docs.rs/tokio/1/tokio/io/
- `tokio::task::spawn_blocking` és `JoinSet`: https://docs.rs/tokio/1/tokio/task/
- `tokio::sync::Semaphore` (`try_acquire_owned`): https://docs.rs/tokio/1/tokio/sync/struct.Semaphore.html
- `tokio_util::sync::CancellationToken`: https://docs.rs/tokio-util/0.7/tokio_util/sync/struct.CancellationToken.html

A kódot és a teszteket a könyv rögzített toolchainjével (Rust 1.99.1), `tokio` 1.x és `tokio-util` 0.7 főverzióval ellenőriztük (`cargo test --workspace`, `cargo clippy --workspace --all-targets -- -D warnings`). A hálózati tesztek helyi (`127.0.0.1`) portokat használnak, az időzítéses tesztek (a lassú tároló, a határidők) bőséges tartalékkal íródtak, de nagyon terhelt gépen a küszöbök módosítást igényelhetnek.

17.5 Naplózás és megfigyelhetőség: tracing

17.5.1 Motiváció

Egy C++-ban írt TCP-szerver naplózása gyakran `std::cerr`-re írt sorokból vagy egy `spdlog`-hoz hasonló könyvtárból áll. A kérések nyomon követéséhez kézzel kell egyedi azonosítókat (`request_id`) generálni, és minden logüzenetbe beilleszteni. A konkurrens kérések esetén a

naplóbejegyzések összekeverednek, és utólag nehéz rekonstruálni, hogy melyik sor melyik kéréshez tartozott. A szűrés szintje és modulja gyakran fordítási időben rögzült, vagy bonyolult konfigurációt igényel.

A Rust `tracing` crate-je ezt a problémát strukturált naplózással és **span**-ekkel oldja meg. Egy span egy műveleti hatókör (például egy kérés feldolgozása), amelybe a benne lévő események automatikusan beletartoznak. Így egyetlen `#[instrument]` attribútummal minden kapcsolathoz (nem kérés-szinten) egy span jön létre, amelynek belső azonosítóját a `tracing` kezeli (ez a formázott kimenetben alapból nem jelenik meg, lásd lent), és a napló szűrése futás közben, környezeti változóval állítható. Az alapformázó a span nevét és a rajta rögzített mezőket írja ki; a belső span-`Id` és az átfutási idő csak külön beállítással (például `FmtSpan::CLOSE`, illetve saját időmérés) kerül a kimenetbe. Az rkv-server már rendelkezik a `Stats` típusú atomikus számlálókkal a Atomikus műveletek és memóriarendezés alfejezetből; ebben a laborban ezeket tesszük megfigyelhetővé.

17.5.2 Fogalmak és szabályok

Definíció. A `tracing` crate három alapfogalma:

- **Event** (`tracing::info!`, `debug!`, `warn!`, `error!`, `trace!`): egy időpontban bekövetkező esemény, hasonló a klasszikus naplóbejegyzéshez, de strukturált mezőket tartalmazhat.
- **Span** (`tracing::span!` vagy a `#[instrument]` attribútum): egy időintervallumot átfogó, hierarchikus kontextus. A span-ek egymásba ágyazhatók, és az események a legbelső aktív span-hez tartoznak.
- **Subscriber**: a naplófogyasztó, amely a span-eket és eventeket megkapja, és formázva kiírja, szűri vagy továbbítja. A `tracing-subscriber` a leggyakoribb implementáció.

Definíció. Az `EnvFilter` a `tracing-subscriber` szűrője, amely környezeti változóból (`RUST_LOG`) olvassa be a naplózási szinteket, például:

```
RUST_LOG=info
RUST_LOG=rkv_server=debug
RUST_LOG=info,tokio=warn
```

Ez modulonként (crate-enként) állítja be a minimális szintet.

Szabály. A `#[tracing::instrument]` attribútum egy függvényre írva automatikusan span-t nyit a függvény belépésekor, és bezárja kilépéskor. Alapértelmezés szerint a span neve a függvény neve, és a paramétereket strukturált mezőkként rögzíti (ha azok implementálják a `Debug` trait-et; a `skip` paraméterrel kihagyhatók).

Szabály (async kontextus). Async függvényekben a span-eknek át kell ívelniük az `.await` pontokon. A `tracing` modern verziói (0.1.x) a `tracing-futures` integrációval és az `Instrument` trait-tel ezt automatikusan kezelik; a `#[instrument]` async függvényekre is működik, mert a proc-macro a jövőt instrumentálja.

17.5.3 C++ és Rust összevetés

C++-ban egy szokásos időmérő és naplózó scope-segéd:

```cpp
#include <spdlog/spdlog.h>
#include <chrono>

class ScopedLog {
public:
    ScopedLog(const std::string& name)
        : name_(name), start_(std::chrono::steady_clock::now()) {
        spdlog::info("{} start", name_);
    }
    ~ScopedLog() {
        auto ms = std::chrono::duration_cast<std::chrono::milliseconds>(
            std::chrono::steady_clock::now() - start_).count();
        spdlog::info("{} end ({} ms)", name_, ms);
    }
private:
    std::string name_;
    std::chrono::steady_clock::time_point start_;
};

void handle_request(const Request& req) {
    ScopedLog log("handle_request");
    // ...
}
```

A Rust megfelelője ugyanez, de nyelvi szinten és async-tudatosan:

```rust
#[tracing::instrument]
async fn handle_request(socket: TcpStream) {
    // ...
}
```

A különbség: a Rust `#[instrument]` nem csupán időt mér, hanem egy span-t hoz létre, amely a naplófogyasztó számára hierarchiát jelent. C++-ban ehhez nem nyelvi eszköz kell, hanem könyvtári megoldás: az OpenTelemetry C++ spanjei, a Boost.Log scope-olt attribútumai vagy egy thread-local kontextus (például spdlog-formázóval) hasonló hierarchiát adnak. A `request_id` kézi átadása csak az egyik, könyvtár nélküli lehetőség; a lényegi különbség, hogy C++-ban a követhetőség szerződését a választott naplózó könyvtárnak kell meghatároznia és a hívóknak követniük, míg a Rust attribútum a függvény szignatúrájához kötött instrumentálást generál.

17.5.4 Kidolgozott példa: tracing alapok

Először egy önálló kis programot írunk, hogy lássuk a span-ek és az `EnvFilter` működését.

Munkakönyvtár: `~/rust-labor/r041-tracing/`

```
cargo new r041-tracing
cd r041-tracing
```

`Cargo.toml`:

```toml
[dependencies]
tracing = "0.1"
tracing-subscriber = { version = "0.3", features = ["env-filter"] }
```

`src/main.rs`:

```rust
use tracing::{debug, info, warn};
use tracing_subscriber::fmt::format::FmtSpan;
use tracing_subscriber::EnvFilter;

#[tracing::instrument]
fn compute(value: u64) -> u64 {
    debug!(value, "számítás indul");
    let result = value * 2;
    info!(result, "számítás kész");
    result
}

fn main() {
    // RUST_LOG hiányában INFO szint; a span lezárását is eseményként kiírjuk.
    let filter = EnvFilter::try_from_default_env().unwrap_or_else(|_| EnvFilter::new("info"));
    tracing_subscriber::fmt()
        .with_env_filter(filter)
        .with_span_events(FmtSpan::CLOSE)
        .without_time() // a mintakimenet determinisztikus legyen
        .with_ansi(false) // szín nélküli, másolható kimenet
        .with_target(false)
        .init();

    warn!("példa indul");
    compute(21);
}
```

Az `EnvFilter::from_default_env()` önmagában **nem** ad INFO alapértelmezést: ha a `RUST_LOG` nincs beállítva (vagy üres), csak az `ERROR` szintet engedi át, így a példa semmit nem írna ki. Ezért használjuk az `EnvFilter::try_from_default_env()`-et explicit tartalékkal: ha a változó hiányzik, `EnvFilter::new("info")` lép életbe. A `#[tracing::instrument]` önmagában nem ír ki sort a span lezárásakor; ehhez a formázót kell beállítani (`with_span_events(FmtSpan::CLOSE)`). A mezők a span nevében jelennek meg (`compute{value=21}`), az eseményekhez fűzött mezők (`result=42`) a szöveg után.

Futtatás `RUST_LOG` nélkül (és ugyanígy `RUST_LOG=info` mellett):

```
cargo run
```

Kimenet (Rust 1.99.1, `tracing-subscriber` 0.3, a `.without_time()` és a `.with_ansi(false)` miatt determinisztikus és szín nélküli):

```
 WARN példa indul
 INFO compute{value=21}: számítás kész result=42
 INFO compute{value=21}: close
```

A `debug!` üzenet nem jelenik meg, mert az alapértelmezett szint `INFO`. A `RUST_LOG=debug cargo run` kimenete:

```
 WARN példa indul
DEBUG compute{value=21}: számítás indul value=21
 INFO compute{value=21}: számítás kész result=42
 INFO compute{value=21}: close
```

Szemben ezzel, ha a tartalékot elhagyjuk, és csak az `EnvFilter::from_default_env()`-et használjuk, a `RUST_LOG` nélküli futás **nem ír ki semmit**, `RUST_LOG=warn` mellett pedig csak a `WARN` sort.

Span-azonosító és `request_id`. A `tracing` subscriberje belsőleg számon tart egy *span-azonosítót* (`Id`), de ez a szokásos formázott kimenetben nem jelenik meg, és folyamatok vagy

gépek között nem követhető. Ha egy kérést a naplókban végigkövetni kell, azt explicit mezőként vesszük fel (`request_id`), amely a span nevében minden eseménynél megjelenik.

17.5.5 Labor: naplózás és metrikák az rkv-serverben

Munkakönyvtár: `~/rust-labor/rkv/`

A kiinduló állapot az Az rkv hálózati szerver: TCP, protokoll, kecses leállás ellenőrzési pontja: az `rkv-server` könyvtár (`run_server`, `ServerConfig`) és bináris, amely a `SharedStore`-t használja. A Atomikus műveletek és memóriarendezés alfejezetben a `SharedStore` már tartalmaz egy `Arc<Stats>` mezőt atomikus számlálókkal, amelyet a `SharedStore::stats()` ad. Most ezt tesszük megfigyelhetővé: a **tényleges szervert** módosítjuk, és a metrikákat a `Stats` publikus getterein keresztül olvassuk.

17.5.5.1 1. Függőségek hozzáadása

Kódblokk — fájl: `rkv-server/Cargo.toml` · művelet: beillesztés a `[dependencies]` szakaszba · ellenőrzés: `cargo build -p rkv-server`

```
tracing = "0.1"
tracing-subscriber = { version = "0.3", features = ["env-filter"] }
```

17.5.5.2 2. Aktív kapcsolatok: RAII őr a `Stats`-ban

Az aktív kapcsolatok számlálója csak akkor megbízható, ha a csökkentés *minden* kilépési úton lefut: normál zárásnál, hibánál (`?`) és akkor is, ha a kapcsolatot kezelő taskot megszakítják (`abort`, vagy a leállítási határidő lejár). Ezt a `Drop`-alapú őr garantálja: a `Stats::connection_opened` egy `ConnectionGuard`-ot ad, amelynek eldobása csökkenti a számlálót.

Kódblokk — fájl: `rkv-core/src/stats.rs` · művelet: az `active_connections` mező és az új `impl`/típus/tesztek hozzáadása az Atomikus műveletek és memóriarendezés-ös fájlhoz · ellenőrzés: `cargo test -p rkv-core stats`

```
// a Stats struktúrába: active_connections: AtomicU64,
// a fájl elejére: use std::sync::Arc;
impl Stats {
    /// Az éppen nyitott kapcsolatok száma.
    pub fn active_connections(&self) -> u64 {
        self.active_connections.load(Ordering::Relaxed)
    }

    /// Új kapcsolatot jegyez be. A visszaadott őr `Drop`-ja csökkenti a számlálót,
    /// ezért normál kilépésnél, hibánál és a task megszakításakor (abort) is lefut.
    pub fn connection_opened(self: &Arc<Self>) -> ConnectionGuard {
        self.active_connections.fetch_add(1, Ordering::Relaxed);
        ConnectionGuard(Arc::clone(self))
    }
}

/// RAII őr: a kapcsolat élettartama alatt él, eldobáskor csökkenti az aktív kapcsolatok számát.
#[derive(Debug)]
pub struct ConnectionGuard(Arc<Stats>);

impl Drop for ConnectionGuard {
    fn drop(&mut self) {
        self.0.active_connections.fetch_sub(1, Ordering::Relaxed);
```

```
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn guard_decrements_on_drop() {
        let stats = Arc::new(Stats::default());
        let guard = stats.connection_opened();
        assert_eq!(stats.active_connections(), 1);
        drop(guard);
        assert_eq!(stats.active_connections(), 0);
    }

    #[test]
    fn guard_decrements_on_panic_unwind() {
        let stats = Arc::new(Stats::default());
        let s = Arc::clone(&stats);
        let result = std::panic::catch_unwind(move || {
            let _guard = s.connection_opened();
            panic!("hiba a kapcsolat közben");
        });
        assert!(result.is_err());
        assert_eq!(stats.active_connections(), 0);
    }
}
```

(A `Stats` struktúra kiegészül egy `active_connections: AtomicU64` mezővel, a `lib.rs`-ben pedig a `pub use stats::{ConnectionGuard, Stats};` sor exportálja az új típust.)

17.5.5.3 3. A szerver instrumentálása

A `handle_client` függvényre `#[tracing::instrument]` kerül: a span minden kapcsolathoz hozzáadja a `peer` mezőt, és a függvény minden eseménye ebben a kontextusban jelenik meg. A `skip_all` kikapcsolja a paraméterek automatikus naplózását (a socket és a tároló nem `Debug`-barát). A függvény elején jegyezzük be a kapcsolatot, az őr a függvény végéig él. A `spawn_blocking` külön szálon fut, ezért a `tracing` szál-lokális span-kontextusa **nem** öröklődik automatikusan: a `Span::current()`-et a closure előtt el kell menteni, és a szinkron munkát `in_scope`-ban kell futtatni, különben a `process_line_with` eseményei span nélkül jelennek meg. Ez egyben a megfigyelhetőség határa is: a blokkoló szálon belül minden esemény az átadott span alá kerül, de a `spawn_blocking` saját életciklus-eseményei (indítás, befejezés) nem. A következő részlet a függvény elejét mutatja:

```
#[tracing::instrument(skip_all, fields(peer = %peer))]
async fn handle_client(
    socket: TcpStream,
    peer: SocketAddr,
    store: SharedStore,
    token: CancellationToken,
    max_frame: usize,
    max_response: usize,
) -> io::Result<()> {
    // Az őr a függvény végéig él: normál kilépésnél, hibánál (`?`) és
    // a task megszakításakor is csökkenti a számlálót.
    let _guard = store.stats().connection_opened();
    info!("kapcsolat nyitva");
    // ... a korábbi ciklus, a TooLong ágban warn!(max_frame, ...), a kérésnél debug!(bytes =
    ↪  ..., "kérés") ...
```

```
    info!("kapcsolat zárul");
    writer.shutdown().await
}
```

A `run_server_with` a `peer` címet átadja a `handle_client`-nek, és az `eprintln!` hívások helyett `warn!`-ot használ (`warn!(%addr, "kliens hiba: {e}")`). A teljes, módosított `lib.rs`:

Kódblokk — fájl: `rkv-server/src/lib.rs` · művelet: teljes csere (az rkv hálózati szerver: TCP, protokoll, kecses leállás alfejezetbeli változat a `tracing` hívásokkal, a `peer` mezővel, az őrrel és a `metrics_loop`-pal) · ellenőrzés: `cargo build -p rkv-server`

```
use std::io;
use std::net::SocketAddr;
use std::sync::Arc;
use std::time::Duration;

use rkv_core::{Command, Response, SharedStore, Stats};
use tokio::io::{AsyncBufRead, AsyncBufReadExt, AsyncReadExt, AsyncWriteExt, BufReader,
↪  BufWriter};
use tokio::net::{TcpListener, TcpStream};
use tokio::sync::Semaphore;
use tokio::task::JoinSet;
use tokio_util::sync::CancellationToken;
use tracing::{debug, info, warn};

/// A szerver erőforráskorlátai.
#[derive(Debug, Clone)]
pub struct ServerConfig {
    /// Egy kérés (a sorvéggel együtt) legfeljebb ennyi bájt lehet: a kulcs és az érték együtt.
    pub max_frame: usize,
    /// Egyidejű kapcsolatok felső határa; a többletet "ERR server busy" válasszal utasítjuk el.
    pub max_connections: usize,
    /// A leállás 2. fázisának határideje: eddig várunk a futó klienstaskokra.
    pub shutdown_grace: Duration,
    /// Egy válasz legfeljebb ennyi bájt (a `LIST` miatt); a nagyobbat `ERR response too large`
    ↪  váltja fel.
    pub max_response: usize,
}

impl Default for ServerConfig {
    fn default() -> Self {
        Self {
            max_frame: 64 * 1024,
            max_connections: 1024,
            shutdown_grace: Duration::from_secs(5),
            max_response: 1024 * 1024,
        }
    }
}

/// A `max_frame` felső határa: a `take(max_frame + 1)` és a puffer lefoglalása ennél nagyobbra
↪  nem értelmes.
pub const MAX_FRAME_LIMIT: usize = 64 * 1024 * 1024;

impl ServerConfig {
    /// A belépési ponton is érvényesített ellenőrzés: a publikus mezők a builderen kívül is
    /// beállíthatók, ezért a `run_server_with` ezt hívja. A `Semaphore` kapacitáshatárát is
    ↪  betartja.
    pub fn validate(&self) -> Result<(), ConfigError> {
        if self.max_frame == 0 {
            return Err(ConfigError::Zero("max_frame"));
        }
        if self.max_frame > MAX_FRAME_LIMIT {
```

```
            return Err(ConfigError::TooLarge("max_frame"));
        }
        if self.max_connections == 0 {
            return Err(ConfigError::Zero("max_connections"));
        }
        if self.max_connections > Semaphore::MAX_PERMITS {
            return Err(ConfigError::TooLarge("max_connections"));
        }
        if self.max_response == 0 {
            return Err(ConfigError::Zero("max_response"));
        }
        Ok(())
    }
}

/// A `ServerConfig` érvénytelen korlátja (a `validate` és a builder `build` hibája).
#[derive(Debug, PartialEq, Eq)]
#[non_exhaustive]
pub enum ConfigError {
    /// A megnevezett korlát nulla, ami a szervert használhatatlanná tenné.
    Zero(&'static str),
    /// A megnevezett korlát a megengedett felső határnál nagyobb.
    TooLarge(&'static str),
}

impl std::fmt::Display for ConfigError {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        match self {
            ConfigError::Zero(field) => write!(f, "a(z) {field} korlát nem lehet nulla"),
            ConfigError::TooLarge(field) => write!(f, "a(z) {field} korlát túl nagy"),
        }
    }
}

impl std::error::Error for ConfigError {}

/// Egy beolvasott keret eredménye.
#[derive(Debug, PartialEq, Eq)]
enum Frame {
    /// Teljes, `\n`-nel lezárt sor van a pufferben.
    Line,
    /// A sor hosszabb a megengedettnél (a puffer legfeljebb `max_frame + 1` bájtot tartalmaz).
    TooLong,
    /// A kapcsolat lezárult (egy csonka, sorvég nélküli maradékot eldobunk).
    Eof,
}

/// Egy sort olvas legfeljebb `max_frame` bájtig. A `take` korlátozza a pufferbe kerülő
/// bájtok számát, így sorvég nélküli, végtelen bemenet sem növeli a memóriát.
async fn read_frame<R>(reader: &mut R, buf: &mut Vec<u8>, max_frame: usize) -> io::Result<Frame>
where
    R: AsyncBufRead + Unpin,
{
    buf.clear();
    let n = reader
        .take(max_frame as u64 + 1)
        .read_until(b'\n', buf)
        .await?;
    Ok(if n == 0 {
        Frame::Eof
    } else if buf.len() > max_frame {
        Frame::TooLong
    } else if buf.ends_with(b"\n") {
```

```
        Frame::Line
    } else {
        Frame::Eof
    })
}

/// Egy kérés feldolgozása: értelmezés, végrehajtás, válasz. Szinkron és blokkolhat
/// (a `FileStore` fájlműveleteket végez), ezért a hívó `spawn_blocking`-ben futtatja.
///
/// Két réteg van: a szöveges `GET/SET/DEL/LIST` az olvasható, szöveg-biztos
/// kulcsokra és értékekre való; a `GETX/SETX` hexa kulccsal és értékkel bármilyen
/// bájtsort átvisz (az üres értéket a `-` jelöli).
pub fn process_line(line: &str, store: &SharedStore) -> String {
    if let Some(reply) = process_binary(line, store) {
        return reply;
    }
    let command = match line.parse::<Command>() {
        Ok(command) => command,
        Err(e) => return single_line(&format!("ERR {e}")),
    };
    let response = match store.apply(&command) {
        Ok(response) => response,
        Err(e) => return single_line(&format!("ERR {e}")),
    };
    match (&command, response) {
        (_, Response::Ok) => "OK".to_string(),
        (_, Response::NotFound) => "NOTFOUND".to_string(),
        (Command::Get(key), Response::Value(bytes)) => match text_safe(key, &bytes) {
            Some(text) => format!("VALUE {key} {text}"),
            None => "ERR value is not text-safe, use GETX".to_string(),
        },
        (_, Response::Entries(items)) => {
            let mut out = String::new();
            for (key, bytes) in items {
                match text_safe(&key, &bytes) {
                    Some(text) => out.push_str(&format!("VALUE {key} {text}\n")),
                    // Ami szövegként nem ábrázolható, az hexa alakban jön: VALUEX <hexkulcs>
                    ↪  <hexérték>
                    None => out.push_str(&format!(
                        "VALUEX {} {}\n",
                        to_hex(key.as_bytes()),
                        to_hex(&bytes)
                    )),
                }
            }
            out.push_str("OK");
            out
        }
        (_, Response::Value(_)) => "ERR internal error".to_string(),
    }
}

/// Szöveg-biztos-e a bejegyzés: a kulcs érvényes protokollkulcs (nem üres, nincs benne
/// szóköz vagy vezérlőkarakter), az érték érvényes UTF-8 és nincs benne vezérlőkarakter
/// (így nem tud új választ kezdeni).
fn text_safe<'a>(key: &str, bytes: &'a [u8]) -> Option<&'a str> {
    let key_ok = !key.is_empty() && !key.chars().any(|c| c.is_whitespace() || c.is_control());
    let text = std::str::from_utf8(bytes).ok()?;
    (key_ok && !text.is_empty() && !text.chars().any(char::is_control)).then_some(text)
}

fn to_hex(bytes: &[u8]) -> String {
    if bytes.is_empty() {
```

```
        return "-".to_string();
    }
    bytes.iter().map(|b| format!("{b:02x}")).collect()
}

fn from_hex(text: &str) -> Result<Vec<u8>, String> {
    if text == "-" {
        return Ok(Vec::new());
    }
    if text.is_empty() || !text.len().is_multiple_of(2) || !text.is_ascii() {
        return Err("invalid hex".to_string());
    }
    (0..text.len())
        .step_by(2)
        .map(|i| u8::from_str_radix(&text[i..i + 2], 16).map_err(|_| "invalid hex".to_string()))
        .collect()
}

/// `SETX <hexkulcs> <hexérték|->` és `GETX <hexkulcs>`; más sorra `None`.
fn process_binary(line: &str, store: &SharedStore) -> Option<String> {
    let (verb, rest) = line.split_once(' ').unwrap_or((line, ""));
    let result = match verb {
        "SETX" => {
            let (k, v) = rest.split_once(' ')?;
            decode_key(k).and_then(|key| {
                let value = from_hex(v)?;
                store.set(key, value).map_err(|e| e.to_string())?;
                Ok("OK".to_string())
            })
        }
        "GETX" => {
            decode_key(rest).and_then(|key| match store.get(&key).map_err(|e| e.to_string())? {
                Some(bytes) => Ok(format!(
                    "VALUEX {} {}",
                    to_hex(key.as_bytes()),
                    to_hex(&bytes)
                )),
                None => Ok("NOTFOUND".to_string()),
            })
        }
        _ => return None,
    };
    Some(result.unwrap_or_else(|e| single_line(&format!("ERR {e}"))))
}

fn decode_key(hex: &str) -> Result<String, String> {
    let bytes = from_hex(hex)?;
    match String::from_utf8(bytes) {
        Ok(key) if !key.is_empty() => Ok(key),
        _ => Err("key must be non-empty UTF-8".to_string()),
    }
}

/// A protokoll soronkénti: a hibaüzenetekből eltávolítjuk a sortörést.
fn single_line(text: &str) -> String {
    text.replace(['\n', '\r'], " ")
}

async fn reply<W>(writer: &mut W, response: &str) -> io::Result<()>
where
    W: AsyncWriteExt + Unpin,
{
    writer.write_all(response.as_bytes()).await?;
```

```
    writer.write_all(b"\n").await?;
    writer.flush().await
}

#[tracing::instrument(skip_all, fields(peer = %peer))]
async fn handle_client(
    socket: TcpStream,
    peer: SocketAddr,
    store: SharedStore,
    token: CancellationToken,
    max_frame: usize,
    max_response: usize,
) -> io::Result<()> {
    // Az őr a függvény végéig él: normál kilépésnél, hibánál (`?`) és a task megszakításakor is
    ↪  csökkenti a számlálót.
    let _guard = store.stats().connection_opened();
    info!("kapcsolat nyitva");
    let (read_half, write_half) = socket.into_split();
    let mut reader = BufReader::new(read_half);
    let mut writer = BufWriter::new(write_half);
    let mut buf = Vec::new();

    loop {
        // A beolvasás megszakítható (a leállítás kilép a ciklusból); a már beolvasott
        // parancsot viszont mindig végrehajtjuk és megválaszoljuk.
        let frame = tokio::select! {
            _ = token.cancelled() => break,
            frame = read_frame(&mut reader, &mut buf, max_frame) => frame?,
        };
        match frame {
            Frame::Eof => break,
            Frame::TooLong => {
                warn!(max_frame, "túl hosszú kérés, a kapcsolat lezárul");
                // A keretezés nem állítható vissza: hibaüzenet, majd lezárás.
                reply(&mut writer, "ERR line too long").await?;
                break;
            }
            Frame::Line => {
                let line = buf.strip_suffix(b"\n").unwrap_or(&buf);
                let line = line.strip_suffix(b"\r").unwrap_or(line);
                debug!(bytes = line.len(), "kérés");
                let response = match std::str::from_utf8(line) {
                    Ok(line) => {
                        let (line, store) = (line.to_owned(), store.clone());
                        // A tárolóművelet blokkolhat: nem az executor szálán fut.
                        // A span nem öröklődik a blokkoló szálra, ezért explicit átadjuk.
                        let span = tracing::Span::current();
                        tokio::task::spawn_blocking(move || {
                            span.in_scope(|| process_line_with(&line, &store, max_response))
                        })
                        .await
                        .unwrap_or_else(|_| "ERR internal error".to_string())
                    }
                    Err(_) => "ERR request is not valid UTF-8".to_string(),
                };
                reply(&mut writer, &response).await?;
            }
        }
    }
    // A puffer ürítése és a küldő irány lezárása kilépés előtt.
    info!("kapcsolat zárul");
    writer.shutdown().await
}
```

```rust
/// A szerver az alapértelmezett korlátokkal.
pub async fn run_server(listener: TcpListener, store: SharedStore, token: CancellationToken) {
    run_server_with(listener, store, token, ServerConfig::default()).await;
}

/// A szerver megadott korlátokkal. Visszatérése előtt minden klienstaskot megvár (vagy határidő
↪  után megszakít).
pub async fn run_server_with(
    listener: TcpListener,
    store: SharedStore,
    token: CancellationToken,
    config: ServerConfig,
) {
    config.validate().expect("érvénytelen ServerConfig");
    const REJECT_LIMIT: usize = 64;
    let reject_slots = Arc::new(Semaphore::new(REJECT_LIMIT));
    let slots = Arc::new(Semaphore::new(config.max_connections));
    let mut clients = JoinSet::new();
    loop {
        tokio::select! {
            _ = token.cancelled() => break,
            res = listener.accept() => {
                let (mut socket, addr) = match res {
                    Ok(v) => v,
                    Err(e) => { warn!("accept hiba: {e}"); continue; }
                };
                let Ok(permit) = Arc::clone(&slots).try_acquire_owned() else {
                    // Betelt a kapcsolatkeret: rövid, korlátos idejű elutasítás, korlátos számú
                    // párhuzamos elutasító taskkal; a többi többlet-kapcsolat azonnal lezárul.
                    if let Ok(reject_permit) = Arc::clone(&reject_slots).try_acquire_owned() {
                        clients.spawn(async move {
                            let _reject_permit = reject_permit;
                            let _ = tokio::time::timeout(
                                Duration::from_millis(200),
                                socket.write_all(b"ERR server busy\n"),
                            ).await;
                        });
                    }
                    continue;
                };
                let (store, token) = (store.clone(), token.clone());
                clients.spawn(async move {
                    // A permit a task végéig él: a kapcsolat bezárásakor felszabadul.
                    let _permit = permit;
                    if let Err(e) = handle_client(socket, addr, store, token, config.max_frame,
                    ↪  config.max_response).await {
                        warn!(%addr, "kliens hiba: {e}");
                    }
                });
            }
            // A befejeződött taskok eredményét folyamatosan begyűjtjük.
            Some(res) = clients.join_next(), if !clients.is_empty() => {
                if let Err(e) = res {
                    warn!("klienstask: {e}");
                }
            }
        }
    }

    // 1. fázis: új kapcsolatot nem fogadunk.
    drop(listener);
```

```
    // 2. fázis: a futó parancsok befejezése és a válaszok ürítése, határidővel.
    let drain = async {
        while let Some(res) = clients.join_next().await {
            if let Err(e) = res {
                warn!("klienstask: {e}");
            }
        }
    };
    if tokio::time::timeout(config.shutdown_grace, drain).await.is_err() {
        warn!(clients = clients.len(), "a leállítási határidő lejárt, a maradék kliens
        ↪ megszakad");
        clients.shutdown().await;
    }
}

/// Periodikus metrikajelentés: a `Stats` publikus getterein keresztül olvas.
pub async fn metrics_loop(stats: Arc<Stats>, every: Duration) {
    let mut interval = tokio::time::interval(every);
    interval.tick().await; // az első tick azonnali: kihagyjuk
    loop {
        interval.tick().await;
        info!(
            get = stats.get_count(),
            set = stats.set_count(),
            delete = stats.delete_count(),
            list = stats.list_count(),
            active = stats.active_connections(),
            "metrikák"
        );
    }
}
```

A `metrics_loop` a `Stats` publikus getterein keresztül olvas (`get_count()`, `set_count()`, … `active_connections()`), nem a mezőkhöz nyúl; a getterek `Relaxed` rendezést használnak, ami a metrikákhoz elegendő (Atomikus műveletek és memóriarendezés).

17.5.5.4 4. Subscriber inicializálása és a metrikatask indítása

A subscriber-t a `serve` függvény elején, a `block_on` által a **runtime-on futtatott aszinkron törzsben** inicializáljuk, a szerver indítása előtt. (Ez már a tokio runtime *belsejében* van, nem előtte.) A subscriber a runtime-tól független, ezért ez a hely megfelelő; fontos, hogy csak egyszer hívjuk.)

Kódblokk — fájl: `rkv-server/src/main.rs` · művelet: teljes csere · ellenőrzés: `cargo build -p rkv-server`

```
use std::time::Duration;

use anyhow::Result;
use rkv_core::{FileStore, MemStore, SharedStore};
use rkv_server::{metrics_loop, run_server};
use tokio::net::TcpListener;
use tokio_util::sync::CancellationToken;
use tracing::info;
use tracing_subscriber::EnvFilter;

fn main() -> Result<()> {
    let rt = tokio::runtime::Runtime::new()?;
    let outcome = rt.block_on(serve());
    // A runtime eldobása a futó `spawn_blocking` munkát korlátlanul megvárná: időkorlát.
```

```
    rt.shutdown_timeout(Duration::from_secs(5));
    outcome
}

async fn serve() -> Result<()> {
    // RUST_LOG hiányában INFO szint: az `EnvFilter::from_default_env()` hiányzó
    // változónál csak az ERROR szintet engedné át.
    let filter = EnvFilter::try_from_default_env().unwrap_or_else(|_| EnvFilter::new("info"));
    tracing_subscriber::fmt().with_env_filter(filter).init();

    // Használat: rkv-server [cím] [--data-dir <könyvtár>]
    // `--data-dir` nélkül memóriabeli tároló; vele a FileStore (snapshot + napló) fut.
    let mut addr = "127.0.0.1:8080".to_string();
    let mut data_dir: Option<std::path::PathBuf> = None;
    let mut args = std::env::args().skip(1);
    while let Some(arg) = args.next() {
        if arg == "--data-dir" {
            data_dir = Some(
                args.next()
                    .ok_or_else(|| anyhow::anyhow!("a --data-dir érték hiányzik"))?
                    .into(),
            );
        } else {
            addr = arg;
        }
    }
    let listener = TcpListener::bind(&addr).await?;
    info!(%addr, "rkv-server indul");

    let store = SharedStore::new(MemStore::new());
    let token = CancellationToken::new();
    let shutdown_token = token.clone();

    tokio::spawn(async move {
        if let Err(e) = tokio::signal::ctrl_c().await {
            tracing::error!("ctrl_c hiba: {e}");
            return;
        }
        info!("leállítási jel érkezett");
        shutdown_token.cancel();
    });

    let metrics = tokio::spawn(metrics_loop(store.stats(), Duration::from_secs(10)));
    run_server(listener, store, token).await;
    metrics.abort();
    info!("leállás kész");
    Ok(())
}
```

17.5.5.5 5. Szűrés környezeti változóval

A szerver futtatása előtt beállíthatjuk a kívánt szintű naplózást. `RUST_LOG` nélkül a `main` fenti tartaléka miatt INFO szint érvényes:

```
cargo run -p rkv-server
RUST_LOG=info cargo run -p rkv-server
```

Ha csak a saját crate naplóit szeretnénk `DEBUG` szinten, a külső crate-eket (például a tokio belső üzeneteit) pedig `WARN` szinten:

```
RUST_LOG=rkv_server=debug,tokio=warn cargo run -p rkv-server
```

A `RUST_LOG` támogatja a többszörös modulszűrést vesszővel elválasztva. Az `info` szinten a szerver indulása, a kapcsolatok nyitása/zárása és a metrikák jelennek meg; a `debug` szint a kérésenkénti `kérés` eseményeket is hozzáadja. A kimenet pontos formátuma (időbélyeg, színek, a span mezőinek megjelenítése) a `tracing-subscriber` beállításától függ.

17.5.5.6 6. Teszt: az aktív kapcsolatok száma visszatér a kiinduló értékre

Kódblokk — fájl: `rkv-server/tests/metrics.rs` · művelet: új fájl · ellenőrzés: `cargo test -p rkv-server --test metrics` → 2 sikeres teszt

```
use std::time::Duration;

use rkv_core::{MemStore, SharedStore};
use rkv_server::run_server;
use tokio::io::{AsyncBufReadExt, AsyncWriteExt, BufReader};
use tokio::net::{TcpListener, TcpStream};
use tokio::time::{sleep, timeout};
use tokio_util::sync::CancellationToken;

/// Addig vár, amíg a feltétel igaz nem lesz (legfeljebb 5 s).
async fn eventually(mut condition: impl FnMut() -> bool) {
    timeout(Duration::from_secs(5), async {
        while !condition() {
            sleep(Duration::from_millis(10)).await;
        }
    })
    .await
    .expect("a feltétel 5 s alatt sem teljesült");
}

#[tokio::test]
async fn active_connections_return_to_the_baseline() {
    let store = SharedStore::new(MemStore::new());
    let stats = store.stats();
    let listener = TcpListener::bind("127.0.0.1:0").await.unwrap();
    let addr = listener.local_addr().unwrap();
    let token = CancellationToken::new();
    let server = tokio::spawn(run_server(listener, store, token.clone()));

    let baseline = stats.active_connections();
    assert_eq!(baseline, 0);

    let mut client = BufReader::new(TcpStream::connect(addr).await.unwrap());
    client.get_mut().write_all(b"SET k v\n").await.unwrap();
    let mut line = String::new();
    timeout(Duration::from_secs(5), client.read_line(&mut line)).await.unwrap().unwrap();
    assert_eq!(line.trim_end(), "OK");
    assert_eq!(stats.active_connections(), baseline + 1); // nyitott kapcsolat
    assert_eq!(stats.set_count(), 1);

    drop(client); // bezárás
    eventually(|| stats.active_connections() == baseline).await;

    token.cancel();
    timeout(Duration::from_secs(10), server).await.unwrap().unwrap();
}

#[tokio::test]
async fn guard_is_released_when_the_task_is_aborted() {
    let store = match &data_dir {
        Some(dir) => {
```

```
            std::fs::create_dir_all(dir)?;
            SharedStore::new(FileStore::open(
                dir.join("state.json"),
                dir.join("append.log"),
            )?)
        }
        None => SharedStore::new(MemStore::new()),
    };
    let stats = store.stats();

    let held = stats.clone();
    let task = tokio::spawn(async move {
        let _guard = held.connection_opened();
        std::future::pending::<()>().await; // a kapcsolat "örökké" tart
    });
    eventually(|| stats.active_connections() == 1).await;

    task.abort(); // a task megszakítása eldobja a future-t, vele az őrt
    assert!(task.await.unwrap_err().is_cancelled());
    assert_eq!(stats.active_connections(), 0);
}
```

Az első teszt egy kapcsolat megnyitása után egyet, bezárása után újra a kiinduló értéket olvassa; a második a task-megszakítási utat ellenőrzi: az `abort` eldobja a future-t, és vele az őrt, így a számláló nem marad „beragadva”.

A kapcsolatkezelés során a span-ek a formázott kimenetben a `handle_client{peer=127.0.0.1:...}` előtaggal jelennek meg, jelezve a hierarchiát.

Ellenőrzési pont. Az `rkv-server` instrumentált (`tracing`), a `Stats` az aktív kapcsolatokat is számolja (`ConnectionGuard`), a `metrics_loop` a getterekből jelent; a `cargo test --workspace` és a `clippy -D warnings` hibátlan.

17.5.6 Tipikus hibák és átállási csapdák

1. **A subscriber többszöri inicializálása.** A kód lefordul: a hiba **futási** hiba. Ha például egy tesztmodulban minden teszt lefutásakor meghívjuk a `tracing_subscriber::fmt().init()`-et, a második hívás **pánikol** (`Unable to install global subscriber: SetGlobalDefaultError(...)`, ellenőrzött viselkedés). A két indító metódus szerződése eltér: az `init()` pánikkal jelzi, ha már van globális subscriber; a `try_init()` `Result<(), TryInitError>`-t ad vissza, a második hívásra `Err`-t, pánik nélkül. Megoldás: a subscriber-t egyszer, a `main`-ben vagy egy `Once`/`OnceLock` segítségével inicializáljuk, vagy a tesztekben a `try_init()`-et használjuk, és az `Err` eredményt tudatosan eldobjuk (`let _ = ...`), mert a már működő subscriber elfogadható.

2. **A `span.enter()` guard tartása `.await`-en át.** Az `enter()` guardja a *szálhoz* köti az aktív spant; ha a guard az `.await`-en át él, a task felfüggesztésekor a span a szálon marad, és egy másik task eseményei rossz (beágyazott) span alá kerülnek. Két váltakozva futó task hibás naplója:

```
INFO kérés{id=1}: kezdet
INFO kérés{id=1}:kérés{id=2}: kezdet
INFO kérés{id=1}:kérés{id=2}: vég
INFO kérés{id=1}: vég
```

A helyes megoldás a teljes future instrumentálása (`future.instrument(span)`, vagy `#[tracing::instrument]` async függvényen), amely a spant pollolásonként lépteti be és ki; ekkor mindkét task a saját spanje alatt naplóz (`kérés{id=1}`, `kérés{id=2}`). Await nélküli

szinkron szakaszra a `span.in_scope(|| ...)` való. Az `.instrument(info_span!("kérés", id))` változat kimenete (Rust 1.99.1, `tokio::join!` egyszálú runtime-on):

```
 INFO kérés{id=1}: kezdet
 INFO kérés{id=2}: kezdet
 INFO kérés{id=2}: vég
 INFO kérés{id=1}: vég
```

3. **Túlzott strukturálás.** Minden kis lépést span-be csomagolni zajossá teszi a naplót. A span-eket műveleti határokhoz (például kérés, parancs) rendeljük, ne minden egyes függvényhíváshoz.

4. **A `RUST_LOG` figyelmen kívül hagyása, és a rejtett ERROR alapértelmezés.** Ha fix szintet adunk meg `EnvFilter` helyett, a szint csak újrafordítással változtatható; az `EnvFilter` a `RUST_LOG`-ot az induláskor olvassa be, tehát az sem változik futás közben. Élő újraszűréshez külön `tracing_subscriber::reload` réteg kell (a `reload::Layer` egy kezelőt ad, amellyel a szűrő cserélhető); ezt a könyv nem tárgyalja. Az `EnvFilter::from_default_env()` tartalék nélkül változó hiányában csak az `ERROR` szintet engedi át, ezért a „semmi nem jelenik meg” hibakereséskor ez az első gyanú. Használjunk explicit tartalékot (`try_from_default_env().unwrap_or_else(|_| EnvFilter::new("info"))`).

17.5.7 Feladatok

1. **Fogalmi kérdés.** Mi történik, ha a `compute` függvényben a `#[tracing::instrument]` attribútumot elhagyjuk, de a függvényen belül továbbra is van `info!` makró? A kimenetben mi változik a span nélkül?

2. **C++ átírás.** A következő C++ kódrészlet egy kérést naplóz `request_id`-vel. Írd át Rustra `tracing` span-nel úgy, hogy a `request_id` ne legyen kézzel átadott paraméter, hanem a span automatikusan rögzítse.

```cpp
void handle_request(int request_id, const Request& req) {
    spdlog::info("request {}: start", request_id);
    // ...
    spdlog::info("request {}: end", request_id);
}
```

3. **Implementáció.** Adj a szerverhez egy új metrikát: az elutasított kapcsolatok számát (`ERR server busy`). Használj `AtomicU64`-et a `Stats` struktúrában, növeld az elutasításkor, add hozzá a `metrics_loop` kimenetéhez, és írj tesztet, amely a `max_connections = 1` korlát túllépése után ellenőrzi a számláló értékét.

4. **Hibajavítás.** A következő kód lefordul, de **futáskor pánikol**, mert a subscriber-t kétszer inicializálja (az `init()` a második hívásnál pánikol). Javítsd úgy, hogy a második inicializálás ne okozzon hibát.

```rust
fn init_logging() {
    tracing_subscriber::fmt().init();
}

fn main() {
    init_logging();
    init_logging(); // hibát okoz
    info!("hello");
}
```

17.5.8 Megoldások

1. **Fogalmi kérdés.** A span nélkül az `info!` event-ek továbbra is megjelennek, de nem tartoznak egyetlen műveleti kontextushoz sem. A formázott kimenetben nem lesz span-azonosító, és a `compute` függvény belépését/kilépését nem látjuk automatikusan. Az event-ek lapos listát alkotnak, és nem lehet őket összekapcsolni.

2. **C++ átírás.** A Rust változat span-t használ:

```
#[tracing::instrument(skip(req))]
fn handle_request(req: &Request) {
    info!("feldolgozás kezdődik");
    // ...
    info!("feldolgozás kész");
}
```

A `request_id`-t nem kell explicit paraméterként átadni: a span a saját mezőjeként rögzítheti, és a `tracing-subscriber` alapformázója a span mezőit az események prefixében megjeleníti (`handle_request{request_id=7}: feldolgozás kezdődik`). Fontos, hogy a `tracing` belső span-Id-ja ettől külön dolog: az alapformázó azt nem írja ki, ezért a kérés követhetőségéhez a saját `request_id` mező (vagy egy erre felkészített subscriber-réteg) kell. Ha a hívást blokkoló szálra adjuk át (`spawn_blocking`), a szál-lokális span-kontextus nem öröklődik automatikusan: a `Span::current()`-et át kell adni a closure-nek, és `in_scope`-ban kell futtatni. Például:

```
use std::sync::atomic::{AtomicU64, Ordering};
use tracing::Span;

static NEXT_REQUEST_ID: AtomicU64 = AtomicU64::new(1);

#[tracing::instrument(skip(req), fields(request_id = NEXT_REQUEST_ID.fetch_add(1,
↪  Ordering::Relaxed)))]
fn handle_request(req: &Request) {
    info!("feldolgozás kezdődik");
    // ...
    info!("feldolgozás kész");
}

fn offload(req: Request) {
    let span = Span::current();
    std::thread::spawn(move || span.in_scope(|| handle_request(&req)));
}
```

3. **Implementáció.** A `Stats`-ba új mező, getter és egy `pub(crate)` növelő kerül, a `SharedStore` pedig egy publikus metódust ad a szervernek (a számláló mezői privátak, a szerver csak a publikus API-t látja):

```
// rkv-core/src/stats.rs
rejected_connections: AtomicU64, // új mező a Stats-ban

impl Stats {
    pub fn rejected_connections(&self) -> u64 {
        self.rejected_connections.load(Ordering::Relaxed)
    }

    /// Elutasított kapcsolat bejegyzése (a szerver hívja, ezért publikus).
    pub fn connection_rejected(&self) {
        self.rejected_connections.fetch_add(1, Ordering::Relaxed);
    }
}
```

A szerverben, az elutasító ágban (`let Ok(permit) = ... else { ... }`) a `store.stats().connection_reje` hívás növeli a számlálót, a `metrics_loop`-ba pedig a `rejected = stats.rejected_connections()` mező kerül. A teszt a `max_connections = 1` konfigurációval két kapcsolatot nyit: a másodikra `ERR server busy` érkezik, és a `stats.rejected_connections()` 1. A növelés a kapcsolat *elfogadásakor* történik (nem őrrel), mert az elutasított kapcsolat nem kezdett el élni.

4. **Hibajavítás.** Az eredeti kód futási pánikot okoz a második `init()`-nél. A `try_init` `Result`-ot ad vissza, így a második hívás `Err`-t kap, pánik nélkül; az eredményt itt tudatosan eldobjuk (`let _ =`), mert a már beállított subscriber megfelelő:

```
fn init_logging() {
    let _ = tracing_subscriber::fmt().try_init();
}

fn main() {
    init_logging();
    init_logging(); // a második try_init() Err-t ad, de nem pánikol
    info!("hello");
}
```

17.5.9 Rövid összefoglaló

A `tracing` crate strukturált naplózást és hierarchikus span-eket biztosít. Az eventek (`info!`, `debug!`, `warn!`, `error!`) klasszikus naplóbejegyzések, míg a span-ek műveleti hatókörök, amelyeket a `#[tracing::instrument]` attribútum automatikusan nyit és zár. A `tracing-subscriber` `EnvFilter`-je a `RUST_LOG` környezeti változóval modulonként és szintenként szűri a naplót; tartalék nélkül (`from_default_env()`) a változó hiányában csak az ERROR szintet engedi át, ezért az `rkv-server` explicit INFO tartalékot ad. A span lezárásának kiírásához `FmtSpan::CLOSE` kell. Async kódban a `#[instrument]` a teljes future-t instrumentálja. Az rkv-serverben a tényleges szervert instrumentáltuk (`handle_client`, `peer` mezővel), a `Stats` megkapta az aktív kapcsolatok számlálóját és a `Drop`-alapú `ConnectionGuard`-ot (normál kilépésnél, hibánál és task-megszakításnál is csökkent), a `metrics_loop` pedig a publikus getterekből jelent. **Továbbvitt eredmény.** A megfigyelhető szerver (`rkv-server/src/lib.rs`, `main.rs`) és a `Stats` biztosítja a Záróprojekt: az rkv teljes rendszerként alfejezetben a rendszer állapotának áttekintését.

17.5.10 Források és ellenőrzési hiányok

- A `tracing` crate dokumentációja: https://docs.rs/tracing (`#[instrument]`, `info_span!`, mezők).
- A `tracing-subscriber` `EnvFilter` dokumentációja: https://docs.rs/tracing-subscriber/0.3/tracing_subscriber/filter/struct.EnvFilter.html (`from_default_env`, `try_from_default_env`, a `RUST_LOG` szintaxis) és a `fmt::format::FmtSpan` típus.
- A tokio tutorial a `tracing` integrációról: https://tokio.rs/tokio/topics/tracing

A kimeneteket és a teszteket a rögzített toolchainnel (Rust 1.99.1), `tracing` 0.1 és `tracing-subscriber` 0.3 főverzióval ellenőriztük; a formázott kimenet részletei (időbélyeg, színek, a mezők sorrendje) a `tracing-subscriber` beállításaitól és verziójától függenek. Az rkv-server korábbi kódját az Az rkv hálózati szerver: TCP, protokoll, kecses leállás alfejezetben hoztuk létre; a `Stats` típust a Atomikus műveletek és memóriarendezés alfejezetben vezettük be.

VI. RÉSZ – Haladó Rust: unsafe, FFI, makrók, típusrendszer, teljesítmény

18 Unsafe Rust

18.1 Unsafe Rust: szuperképességek és felelősség

18.1.1 Motiváció

A C++-ban a nyers mutató és a `reinterpret_cast` a nyelv alaprétegéhez tartozik. Amikor FFI-t írunk, egyedi memóriaelrendezést készítünk, vagy a fordító által nem látható invariánst tartunk fenn, gyakran nincs más választásunk: `void*`-ot castolunk, `char*`-on keresztül nyúlunk objektumokhoz, és reméljük, hogy a kód a gyakorlatban működik. A modern C++ ezt a réteget a `std::launder`, a `std::bit_cast`, a `restrict` és a memóriamodell szabályaival próbálja megregulázni, de a hibák továbbra is gyakran csak futásidőben, és gyakran csak „ritkán reprodukálhatóan” jelennek meg.

A Rust ezt a határvonalat nyelvi szintre emeli. A biztonságos Rust nem engedi meg a nyers mutatók dereferálását, a `union` mezők elérését vagy a `static mut` olvasását – ezeket explicit `unsafe` blokkba kell tenni. Az `unsafe` nem a „veszélyes üzemmód” kapcsolója, hanem egy **szerződés**: a felelősség áthelyeződik a fordítóról a programozóra. Aki pedig ezt a szerződést megszegi, nem definiált viselkedést (UB – undefined behaviour) kaphat, amit a Rust a C++-hoz hasonlóan, de bizonyos pontokon még szigorúbban kezel. Ebben az alfejezetben az unsafe öt képességét, a nyers mutatókat, a tipikus UB-forrásokat és a Miri detektort tanuljuk meg. A cél nem az, hogy mindent `unsafe`-ben írjunk, hanem hogy megértsük, milyen invariánsokat kell fenntartani ahhoz, hogy a ritka, valóban szükséges unsafe kód helyes legyen – és hogy a köré épített biztonságos absztrakció ne hazudjon a hívónak.

18.1.2 Az unsafe öt képessége

Definíció (unsafe blokk). Az `unsafe { ... }` szintaktikai jelölés, amelyben a következő öt művelet elvégezhető. Maga a blokk semmilyen ellenőrzést nem kapcsol ki; a kódváltoztatást nem hajtja végre, csupán a programozó nyilatkozata arról, hogy a benne foglalt kód kielégíti a nyelv által megkövetelt invariánsokat.

Definíció (unsafe fn). Az `unsafe fn` olyan függvény, amelyet biztonságos kódból csak `unsafe` blokkon belül lehet hívni; a `# Safety` dokumentációs szakaszban meg kell adni a hívóra háruló előfeltételeket. A törzsében a veszélyes műveletek a **lintszinttől** függnek: a 2021-es editionben az `unsafe fn` törzse implicit `unsafe` környezet; a 2024-es editionben (ezt használja a könyv) az `unsafe_op_in_unsafe_fn` lint alapértelmezése `warn`, tehát a törzsben a veszélyes műveletet külön `unsafe { ... }` blokkba kell tenni, különben figyelmeztetést kapunk (`E0133` a nyers mutató dereferálására). Ha a projekt ezt kötelezővé teszi, konfigurációként emeljük `deny`-ra: `#![deny(unsafe_op_in_unsafe_fn)]` a crate gyökerében, vagy a `Cargo.toml [lints.rust]` szakaszában.

Definíció (unsafe trait és impl). Az `unsafe trait` olyan trait, amelynek implementálása további, a fordító által nem ellenőrizhető invariánsokat követel (például `Send`, `Sync`, `GlobalAlloc`). Az ilyen trait-et `unsafe impl`-lel lehet csak implementálni. Az `unsafe impl` nem *művelet* a függvénytörzsben, hanem deklaráció: az implementáló vállalja az invariánst.

Szabály (az unsafe öt képessége). A következő öt dolog igényel `unsafe` jelölést; érdemes külön kategóriába sorolni őket, mert a fordító különbözőképpen kezeli:

1. nyers mutató (`*const T` vagy `*mut T`) **dereferálása** (művelet, `unsafe` blokkban);
2. `unsafe fn` vagy `unsafe` metódus **meghívása** (művelet, `unsafe` blokkban; az `unsafe fn` *törzsében* a 2024-es editionben a lint dönt, lásd fent);
3. `unsafe trait` **implementálása** (`unsafe impl`): deklaráció, nem művelet;
4. `static mut` változó **elérése** (olvasás és írás, a 2024-es editionben a referenciaképzés is tiltott lint);
5. `union` mezőjének **olvasása** (művelet, `unsafe` blokkban). A `union` mezőjének **írása** biztonságos (ha a mező `Copy` vagy `ManuallyDrop`): `u.f = 2.0;` `unsafe` nélkül fordul, az olvasás (`u.i`) nem.

Fontos: az `unsafe` **nem** kapcsolja ki a kölcsönzés-ellenőrzőt (borrow checker). Ha egy `unsafe` blokkban biztonságos referenciákat (`&T` / `&mut T`) hozunk létre, azokra továbbra is érvényesek a Borrowing és az Élettartamok szabályai; a nagy `unsafe` blokk sem kapcsol ki egyetlen biztonságos ellenőrzést sem. Az `unsafe` csak a fenti öt nyelvi határig terjed; minden más, így a mintaillesztés, a move-szemantika, a generikusok ellenőrzése és a `Result` kezelés változatlanul működik.

18.1.3 Nyers mutatók: *const T*, mut T

A Rust nyers mutatói a C++ `const T*` / `T*` párnak felelnek meg: típusos memóriacímek, ahol a `*const T` a pointee konstansságát, a `*mut T` a pointee módosíthatóságát jelzi. A mutatóváltozó saját konstanssága (C++-ban `T* const`) nem a mutató típusában, hanem a `let` kötésben jelenik meg. Szemantikájukban annyiban különböznek a C++ mutatóktól, hogy **nincs tulajdonjoguk, nincs élettartam-információjuk, és nem ellenőrzi őket a borrow checker**; a típusuk azonban megmarad, ezért nem `void*`-ként kell kezelni őket. A `*const T` és a `*mut T` közti különbség a szándék dokumentálása és a `const`/`mut` megfeleltetés: a `*const T` a `&T`, a `*mut T` a `&mut T` nyers megfelelője (`*const T`-ből `cast_mut()`-tal lehet `*mut T`-t kapni, de az eredeti `&T`-ből származó mutatón keresztüli írás UB). Egy mutató önmagában nem garantál semmit: a dereferálás szerződése a *provenance* (a mutató melyik allokációból/referenciából származik), a *határok* (az allokáción belül), az *inicializáltság*, a *hozzáférési jog* (írás csak írható származtatásból), az *aliasing* és az *élettartam*.

C++	Rust	Mit jelent
`const T*`	`*const T`	a pointee nem módosítható ezen a mutatón át
`T*`	`*mut T`	a pointee módosítható
`T* const`	`let p: *mut T`	a mutatóváltozó nem írható át, a pointee igen
átírható mutató (`T*`)	`let mut p: *mut T`	a mutatóváltozó is átírható

Nyers mutatót biztonságos kódban is létre lehet hozni, csak dereferálni nem:

```
let x = 42;
let p: *const i32 = &x;          // biztonságos: referencia → nyers mutató
let q: *const i32 = std::ptr::null(); // biztonságos: null mutató
let r = p as usize;              // biztonságos: cím egész számmá alakítása
// let y = *p;                   // HIBA: dereferálás csak unsafe blokkban
```

A mutató-aritmetika a C++ `p + i` műveletének felel meg, de explicit metódusokkal történik:

- `add(n)` / `sub(n)`: előre / hátra lép `n` elemmel; az eredménynek ugyanabban az allokációban (vagy közvetlenül utána) kell lennie, különben UB.
- `offset(n)`: a `add`/`sub` alacsony szintű megfelelője; a hívó felel az érvényességért.
- `wrapping_add(n)` / `wrapping_sub(n)`: túlcsordulás-biztos lépés; a kapott mutató akár érvénytelen is lehet, de a dereferálás csak akkor szabályos, ha érvényes.
- `read()` / `write()`: egyetlen elem olvasása / írása a mutató címén. A `write` **nem droppolja** a régi értéket (felülírja, mintha az nem létezne), tehát droppot igénylő típusnál a régi érték elszivárog, hacsak előtte ki nem vettük (`read`). A `read` bitenkénti másolatot készít, **nem** mozgatja ki az értéket: `Drop`-os típusnál az eredeti és a másolat is úgy hiszi, hogy övé a tulajdon, ami dupla dropot okoz, ha mindkettőt eldobjuk.
- `ptr::copy(src, dst, n)` (a C `memmove` megfelelője): blokkmásolás, a forrás és a cél **átfedhet**. `ptr::copy_nonoverlapping(src, dst, n)` (a C `memcpy` megfelelője): a tartományok **nem fedhetnek át**, különben UB; gyorsabb lehet, de csak akkor használjuk, ha a nem-átfedés garantált. Mindkettőre igaz: a `src` mutatóra `n` elemre olvasható, a `dst`-re `n` elemre írható tartomány kell, mindkettő igazított és nem null; **a forrás és a cél külön allokációban is lehet** (a `copy_nonoverlapping` tipikus használata éppen két külön puffer), de mindkettőnek a *saját* allokációján belül kell maradnia. Ez más feltétel, mint az `add`/`offset` mutatóaritmetikáé. A metódusforma: `p.copy_to(dst, n)` / `p.copy_from(src, n)` (a `copy` családé), `copy_to_nonoverlapping` / `copy_from_nonoverlapping`.

Szabály (nyers mutató dereferálása). Egy nyers mutató dereferálása akkor definiált, ha a mutató (a) nem null, (b) érvényes `T` típusú objektumra mutat, (c) a címe megfelelően igazított (aligned) a `T` típushoz, és (d) a `T` típus értékreprezentációja érvényes. A dereferálás csak `unsafe` blokkban lehetséges, és a hívónak ezeket az előfeltételeket dokumentálnia kell.

Példa nyers mutatóval végzett írásra:

```
let mut v = vec![1, 2, 3];
let p: *mut i32 = v.as_mut_ptr();
// SAFETY: p az első elemre mutat, a Vec legalább 3 elemű, az i32 igazítása
// a Vec<i32> allokátorában garantált.
unsafe {
    p.add(1).write(42);
}
assert_eq!(v, vec![1, 42, 3]);
```

Átfedő tartományra a `copy` a helyes eszköz: egy tömb elemeit egy hellyel jobbra csúsztatjuk (Miri alatt hibátlan, `[1, 1, 2, 3, 4]`):

```
use std::ptr;

let mut v = [1, 2, 3, 4, 5];
let p = v.as_mut_ptr();
// SAFETY: a p..p+4 és a p+1..p+5 tartomány a v-n belül van, igazított és inicializált;
// a tartományok átfedők, ezért ptr::copy (memmove-szerű) kell, nem copy_nonoverlapping.
unsafe { ptr::copy(p, p.add(1), 4) };
assert_eq!(v, [1, 1, 2, 3, 4]);
```

Ugyanez `copy_nonoverlapping`-gel UB lenne (a Miri jelzi). `Drop`-os érték cseréjénél a régi értéket előbb ki kell venni (`read`), különben a `write` elszivárogtatja; a kivett példányt pedig el kell dobni vagy vissza kell írni, hogy a tulajdon ne duplikálódjon.

A `Vec::as_mut_ptr` a belső puffer kezdőcímét adja; a `write` felülírja a második elemet. Ez a példa jól mutatja, hogy a nyers mutató **nem véd** a hibák ellen: ha például `p.add(3).write(0)`-t írnánk, az a `Vec` allokáción kívülre írna, ami UB. A biztonságos alternatíva ugyanerre a feladatra a `&mut [i32]` slice és az indexelés.

18.1.4 Rust UB: a leggyakoribb csapdák

Definíció (UB). Nem definiált viselkedés (undefined behaviour): olyan programállapot, amelyre a nyelvi specifikáció nem ír elő semmilyen viselkedést. A fordító joggal feltételezi, hogy UB soha nem következik be, és ez alapján optimalizálhat; ha mégis bekövetkezik, az egész program viselkedése érvénytelenné válik – nemcsak az adott utasítás, hanem az azt megelőző és követő kód is.

A Rust UB-katalógusának legfontosabb elemei a következők:

1. **Aliasing megsértése.** A Rust szigorúbb referencia-aliasing szabályokat használ, mint a C++ `restrict` kulcsszava. Egy `&mut T` referencia kizárólagos, egyedi hozzáférést jelent: amíg a `&mut` él, semmilyen más `&mut T` vagy `&T` sem mutathat ugyanarra a memóriára, és olyan nyers mutatón keresztül sem szabad hozzáférni, amely *nem ebből a `&mut`-ból származik*. Az `&mut`-ból származtatott nyers mutatóval (és az abból készített reborrow-val) a hozzáférés megengedett, amíg a származtatott mutatót az eredeti `&mut` ismételt használata előtt befejezzük (lásd a pozitív példát). Ez a szabály a biztonságos kódban a borrow checker által automatikusan teljesül, de nyers mutatóval megsérthető:

Két példa mutatja a különbséget. A pozitívban a nyers mutató az `r`-ből származik, és az `r`-t csak azután használjuk újra, hogy a `p`-t már nem használjuk (a Miri, Stacked Borrows modellben, hibátlannak találja, kimenet `12`):

```
let mut x = 10;
let r: &mut i32 = &mut x;
let p: *mut i32 = r; // r-ből származtatott nyers mutató
// SAFETY: p az r-ből származik, x él, és a két elérést nem váltogatjuk.
unsafe { *p += 1 };
*r += 1; // az r újbóli használata érvényes: p-t már nem használjuk
```

A negatívban a szülő (`r`) használata érvényteleníti a belőle származtatott `p`-t, és a `p` későbbi használata UB. Ez a program nem futtatandó normál módon (nincs elvárt kimenete), csak Miri alatt vizsgáljuk. A Miri a `p` használatánál hibát jelez (a címke száma futásonként eltérhet): `error: Undefined Behavior: attempting a read access using <338> at alloc…, but that tag does not exist in the borrow stack for this location`, és hozzáteszi, hogy a Stacked Borrows szabályai még kísérleti jellegűek:

```
let mut x = 10;
let r: &mut i32 = &mut x;
let p: *mut i32 = r;
*r += 1; // az r használata érvényteleníti a belőle származtatott p-t
// SAFETY: HIBÁS - p már érvénytelen.
unsafe { *p += 1 };
```

A Miri a Stacked Borrows modell szerint ellenőriz. Ez nem a Rust végleges, hivatalos aliasing-modellje: a Rust Reference is jelzi, hogy az aliasing szabályai még nem teljesen rögzítettek, ezért a „UB” minősítés egy adott modellre vonatkozik, nem végleges nyelvi szabályra; a pozitív minta (származtatott mutató, felváltva nem használva) viszont minden szóba jövő modellben elfogadott.

2. **Érvénytelen értékreprezentáció.** A Rust típusai nem minden bitmintát engednek meg. A `bool` például csak a `0` és `1` bájtot fogadja el; a `char` csak érvényes Unicode skalárértéket; az `enum` esetén a diszkriminánsnak a deklarált változatok valamelyikét kell jelölnie. Ha egy nyers mutatón keresztül ilyen típusként értelmezünk egy memóriát, amely nem érvényes reprezentációt tartalmaz, UB-t kapunk. Ezt a C++ `bool`/`enum` szabályai is tiltják, de a Rust a `char` és az `enum` esetén gyakran még szigorúbb.

3. **Igazítás (alignment) megsértése.** Minden típusnak van igazítási követelménye: az `i32` általában 4 bájtra, a `u128` akár 16 bájtra igazított kell legyen. Ha egy mutató olyan címet jelöl, amely nem felel meg a `T::align`-nek, a dereferálás UB. A `Vec<u8>` pufferéből például nem szabad önkényes címre `*const i32`-et castolni, mert a `u8` puffer igazítása 1, így a kapott cím lehet, hogy nem 4-re igazított.

4. **Adatverseny (data race).** Ugyanazt a memóriát több szál egyidejűleg éri el, legalább az egyik írás, és nincs köztük megfelelő szinkronizáció. Ez a szabály azonos a C++-beli adatverseny definíciójával; a Rust a Send és Sync és az Atomikus műveletek révén a biztonságos kódban kizárja, de nyers mutatókkal és nem szinkronizált hozzáféréssel ez a korlát megkerülhető.

5. **Dangling pointer dereferálása.** Olyan memóriára mutató mutató dereferálása, amely már felszabadult, vagy amelynek az élettartama lejárt. A Rust a biztonságos kódban ezt a borrow checkerrel és a lifetime-okkal akadályozza meg, de nyers mutatóval ezt a szabályt is kézzel kell betartani. A C++-hoz hasonlóan a dangling pointer használata UB.

18.1.5 C++ és Rust összevetés

A C++-ban a `reinterpret_cast` és a nyers mutatók szabadon használhatók; a hibákat a strict aliasing szabály, a `restrict` és a memóriamodell próbálja korlátozni, de a felelősség nagyrészt a programozóé. A Rust ezt a nyers mutatók dereferálásának `unsafe` blokkba kényszerítésével és a referencia-aliasing szigorúbb szabályával teszi explicité.

```cpp
#include <cstdint>

bool read_bool_from_u8(const std::uint8_t& byte) {
    // UB, ha byte értéke nem 0 vagy 1;
    // a C++ szabvány szerint a bool tárolt értéke csak false/true lehet.
    return *reinterpret_cast<const bool*>(&byte);
}
```

A Rust megfelelő hibás változata:

```rust
unsafe fn read_bool_from_u8(byte: &u8) -> bool {
    let p: *const bool = byte as *const u8 as *const bool;
    // SAFETY: a hívó szerződése szerint byte értéke 0 vagy 1; ez a példa
    // ezt a szerződést nem ellenőrzi, így a hívó hibája esetén UB.
    unsafe { *p }
}
```

A különbség nem abban van, hogy a Rust észreveszi-e a hibát – nem veszi észre –, hanem abban, hogy a Rust a `&mut` kizárólagosságát a teljes optimalizálási csővezetékben feltételezi, míg a C++ csak a `restrict` minősítővel ad ilyen garanciát. A Rust aliasing-szabályai tehát **szigorúbbak**, mint a C++ általános aliasing-szabályai: a `&mut T` a C++ `T* restrict`-hez hasonló, de a nyelv minden `&mut` referenciára automatikusan alkalmazza, és a szabály megsértése a borrow checker megkerülésével azonnal UB, nem csak akkor, ha a fordító véletlenül kihasználja.

18.1.6 Kidolgozott példa: Miri és az érvénytelen bool

A Miri egy interpretáló alapú eszköz, amely a `rustc` MIR (mid-level intermediate representation) szintjén futtatja a kódot, és közben ellenőrzi a memória-hozzáféréseket, az igazítást, az érvénytelen bitmintákat és az aliasing-szabályokat. A Miri a Rust projekt része, de a pontos parancs és a nightly-toolchain követelmény verziófüggő; ellenőrizzük a hivatalos `miri` dokumentációt.

Munkakönyvtár: `~/rust-labor/r042-unsafe-superpowers/`

Hozzuk létre a projektet, és írjuk meg a hibás kódot:

```
cargo new --bin r042-unsafe-superpowers
cd r042-unsafe-superpowers
```

`src/main.rs` – szándékosan hibás változat:

```
unsafe fn read_bool_from_u8(byte: &u8) -> bool {
    let p: *const bool = byte as *const u8 as *const bool;
    // SAFETY: a hívó szerződése szerint byte  {0, 1}; itt a hívó megszegi ezt.
    unsafe { *p }
}

fn main() {
    let b: u8 = 2;
    let value = unsafe { read_bool_from_u8(&b) };
    println!("{value}");
}
```

A szokásos futás (`cargo run`) lefordul, és gépfüggő módon `true`-t vagy `false`-t írhat ki; a `b` 2-es értéke nem érvényes `bool` reprezentáció, tehát a program UB-t követ el. Ennek a programnak nincs elvárt kimenete, ezért normál módon nem futtatjuk, csak Miri alatt. A Miri nightly toolchaint és a `miri` komponenst igényel:

```
rustup +nightly component add miri
cargo +nightly miri run
```

A Miri a Rust 1.99.1 mellett kipróbált nightly-vel ezt jelezte: `error: Undefined Behavior: constructing invalid value of type bool: encountered 0x02, but expected a boolean`. A pontos szöveg verziófüggő, a lényeg (érvénytelen `bool`, `0x02`) nem.

A Miri korlátai. A Miri csak a ténylegesen lefutott utakat ellenőrzi, tehát a tesztelt bemeneteken kívüli UB-t nem találja meg. Idegen (C) függvényt nem tud végrehajtani: `error: unsupported operation: can't call foreign function`. A natív FFI-kódot ezért C/C++ tesztekkel és sanitizerekkel (ASan, UBSan) kell ellenőrizni, a Miri a Rust oldali burkolót vizsgálja.

Javítás. A biztonságos, idiomatikus megoldás egyetlen összehasonlítás:

```
fn byte_to_bool(byte: u8) -> bool {
    byte != 0
}

fn main() {
    let b: u8 = 2;
    let value = byte_to_bool(b);
    println!("{value}"); // true
}
```

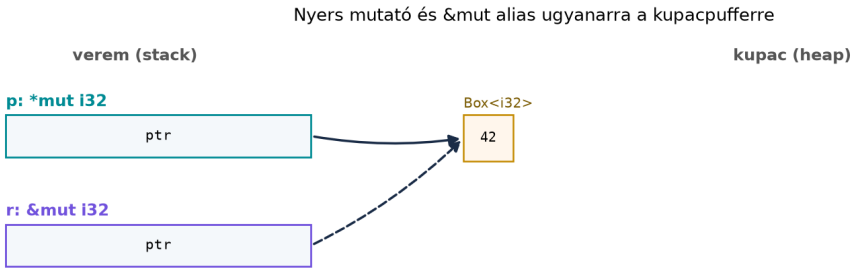
Ha valamilyen FFI-követelmény miatt mégis szükség lenne a nyers mutatóra, a szerződést explicit ellenőrzéssel kell betartani:

```
unsafe fn read_bool_from_u8(byte: &u8) -> bool {
    // SAFETY: a hívótól elvárjuk, hogy a *byte memória érvényes u8 legyen;
    // a függvény maga ellenőrzi a bool érvényes reprezentációját.
    if *byte > 1 {
        panic!("invalid bool representation: {}", *byte);
    }
    let p: *const bool = byte as *const u8 as *const bool;
    // SAFETY: p nem null, a byte igazítása 1, ami a bool-nak is megfelel,
    // és a byte értéke 0 vagy 1, tehát érvényes bool.
```

```
    unsafe { *p }
}
```

A második változat már nem UB, mert az érvénytelen bitmintát a cast előtt kiszűrjük; a Miri ezt már nem jelzi hibaként. A megoldás rávilágít az unsafe szerződések alapelvére: **az invariánst vagy ellenőrizzük a határon, vagy szigorúan dokumentáljuk a hívó felé, de soha nem hagyjuk kimondatlanul.**

Az alábbi ábra azt az esetet mutatja, amikor egy nyers mutató és egy `&mut` referencia egyszerre mutat ugyanarra a kupacpufferre. A `&mut` érvényessége alatt a nyers mutatón keresztüli olvasás vagy írás megsérti a kizárólagosságot, tehát UB.



18.1. ábra: Nyers mutató és &mut alias ugyanarra a kupacpufferre - A p nyers mutató és az r &mut egyszerre mutat ugyanarra a memóriára; ha r érvényessége alatt p-n keresztül hozzáférünk, az &mut aliasing megsértése UB.

A `p` nyers mutató és az `r` `&mut` referencia ugyanarra a kupacblokkra mutat. Mivel `r` kizárólagos referenciaként viselkedik, a `p`-n keresztüli hozzáférés a `r` érvényességi tartományában UB-t okoz; a `dashed` nyíl ezt a tiltott, aliasoló mutatót jelöli.

18.1.7 Tipikus hibák és átállási csapdák

1. **Túl nagy unsafe blokk.** Az `unsafe` blokk nem kapcsolja ki a borrow checkert és a típusellenőrzést, a biztonságos részeket a fordító nagy blokkban is ellenőrzi. A nagy `unsafe` blokk baja más: romlik a kézi bizonyítás, az auditálhatóság és az invariánsok követhetősége (nehéz megmondani, melyik művelet melyik előfeltételre támaszkodik). A jó gyakorlat az, hogy az `unsafe` blokk a lehető legkisebb legyen, és mindig `// SAFETY:` komment előzze meg a hívó által fenntartott invariánsokkal. Az `unsafe fn` esetén a törzs implicit unsafe, ezért a függvény elején dokumentálni kell az előfeltételeket.

2. **Nyers mutatóval megkerülni a borrow checkert.** A tipikus kísérlet: „ha a borrow checker nem engedi, csinálok nyers mutatót”. Ez az `&mut` aliasing UB-hoz vezet, még akkor is, ha a kód látszólag működik. A Miri és a release módban agresszívabb optimalizálás gyakran leleplezi az ilyen kódot. A helyes megoldás a Belső mutabilitás vagy a Mutex, RwLock, csatornák használata.

3. **`transmute` érvénytelen bitmintákra.** A `std::mem::transmute` by-value bitmásolást végez, és a fordító fordítási időben megköveteli, hogy a forrás és a cél típus **mérete egyezzen**. Ez azonban csak szükséges, nem elégséges feltétel; három dolgot külön kell igazolni: (1) a kapott bitminta **érvényes reprezentációja** a cél típusnak (`bool`, `char`, `enum`, `NonZero*` esetén szigorú szabályok vannak); (2) ha a cél referencia vagy mutató, a

mutatott memória érvényessége, igazítása és élettartama külön feltétel; (3) az igazítás nem önmagában a by-value `transmute` akadálya – a cél lokális a saját típusa szerint igazított –, hanem a mutatóvá/referenciává alakításnál és a mutatott memória értelmezésénél számít. Ha az átalakítás nem garantáltan érvényes, használjunk `From`/`TryFrom` implementációt vagy explicit ellenőrzést; `unsafe`-beli `transmute` előtt a fenti három feltételt a `// SAFETY:` kommentben kell igazolni.

4. **Unsafe kód élettartam-hazugsága.** Ha egy `unsafe` függvény egy nyers mutatóból `&'a T`-t készít, a hívónak garantálnia kell, hogy a mögöttes adat legalább `'a` ideig él. Ellenkező esetben a kapott referencia dangling lesz, és a hibát a borrow checker nem fogja észrevenni, mert a nyers mutatónál nincs élettartam-információ.

18.1.8 Feladatok

18.1.8.1 1. Fogalmi kérdés: mit kapcsol ki az unsafe?

Igaz vagy hamis? „Az `unsafe` blokk kikapcsolja a borrow checkert, ezért a biztonságos referenciákra ugyanúgy érvényesek a szabályok, mint máshol.” Indokolj egy mondatban, és sorold fel, melyik öt művelet engedélyezett `unsafe`-ben.

18.1.8.2 2. C++ kód átírása idiomatikus Rustra

Adott a következő C++ függvény:

```cpp
// Visszaadja a vector első n elemének összegét.
int sum_first(const std::vector<int>& v, std::size_t n) {
    const int* p = v.data();
    int sum = 0;
    for (std::size_t i = 0; i < n; ++i) {
        sum += *p;
        ++p;
    }
    return sum;
}
```

Írd át a függvényt Rustra (a) biztonságos slice-műveletekkel, (b) nyers mutatókkal `unsafe` blokkban. Melyik változat az idiomatikus, és miért?

18.1.8.3 3. Implementáció: UB-katalógus az rkv workspace-ben

Készítsd elő a Biztonságos absztrakció unsafe alapon és a C ABI: extern “C”, repr(C), bindgen és cbindgen alfejezet számára egy UB-katalógust! Hozz létre az `rkv-core` crate-ben egy `docs/unsafe-ub.md` fájlt, amely táblázatosan felsorolja a leggyakoribb UB-forrásokat (aliasing, érvénytelen bool/enum, igazítás, adatverseny, dangling pointer). Minden sorhoz írj egy rövid, ellenőrizhető példakódot és azt a szabályt, amelyet a hívónak be kell tartania. A fájl legyen hivatkozható a későbbi laborokból.

18.1.8.4 4. Hibajavítás: érvénytelen enum

A következő kód egy C-stílusú hibakódot olvas be `u8`-ból Rust `enum`-má, és hibásan `transmute`-öt használ:

```
#[repr(u8)]
enum ErrorCode { Ok = 0, NotFound = 1, Internal = 2 }

fn read_error_code(raw: u8) -> ErrorCode {
    // SAFETY: a hívó szerint raw mindig érvényes diszkrimináns.
    unsafe { std::mem::transmute::<u8, ErrorCode>(raw) }
}
```

Mi a baj ezzel a megoldással, ha a hívó `3`-at ad át? Javítsd a függvényt `TryFrom` implementációval és `match`-csel, hogy ne lehessen UB-t okozni. Mutasd meg a teljes javított kódot.

18.1.9 Megoldások

18.1.9.1 1. Fogalmi kérdés

Az állítás hamis. Az `unsafe` blokk **nem** kapcsolja ki a borrow checkert; a biztonságos referenciákra ugyanazok a kölcsönzési és élettartamszabályok érvényesek. Amit az `unsafe` engedélyez: (1) nyers mutató dereferálása, (2) `unsafe fn` hívása, (3) `unsafe trait` impl, (4) `static mut` elérése, (5) `union` mező elérése.

18.1.9.2 2. C++ → Rust átírás

Biztonságos változat:

```
/// Az első `n` elem összege. `None`, ha `n > v.len()`, vagy ha az összeg
/// túlcsordulna az `i32` tartományán.
fn sum_first(v: &[i32], n: usize) -> Option<i32> {
    v.get(..n)?.iter().try_fold(0i32, |acc, &x| acc.checked_add(x))
}
```

Ez a változat a túl nagy `n`-et explicit hibaként (`None`) jelzi, nem némán `0`-ként, hiszen a `0` egy érvényes összeg is lehet. A túlcsordulás jelentését is rögzítjük: a sima `iter().sum()` debug buildben pánikol, release buildben körbefordul, ezért a `checked_add` mindkét esetben `None`-t ad. Ha a körbeforduló összeg a szándék, `wrapping_add` használandó, és ezt a dokumentációban ki kell mondani.

Nyers mutatós változat. Egy safe függvény nem válthat ki UB-t semmilyen bemenetre, ezért a tartományt az `unsafe` blokk *előtt* ellenőrizzük:

```
fn sum_first_raw(v: &[i32], n: usize) -> Option<i32> {
    if n > v.len() {
        return None; // e nélkül a safe hívó UB-t válthatna ki
    }
    let p: *const i32 = v.as_ptr();
    let mut sum: i32 = 0;
    for i in 0..n {
        // SAFETY: fent ellenőriztük, hogy n <= v.len(), ezért i < v.len();
        // p.add(i) a slice érvényes, igazított, inicializált elemére mutat.
        let x = unsafe { *p.add(i) };
        sum = sum.checked_add(x)?;
    }
```

```
    Some(sum)
}
```

A másik lehetőség, hogy a feltételt a hívóra hárítjuk: ekkor a függvény maga `unsafe fn`, és a `# Safety` szakasz rögzíti a szerződést.

```rust
/// Az első `n` elem összege, körbeforduló (wrapping) összeadással.
///
/// # Safety
///
/// A hívónak garantálnia kell, hogy `n <= v.len()`.
unsafe fn sum_first_unchecked(v: &[i32], n: usize) -> i32 {
    let p: *const i32 = v.as_ptr();
    let mut sum: i32 = 0;
    for i in 0..n {
        // SAFETY: a hívó szerződése szerint i < n <= v.len().
        sum = sum.wrapping_add(unsafe { *p.add(i) });
    }
    sum
}
```

Az eredeti, ellenőrzés nélküli, de safe `fn sum_first_raw(v: &[i32], n: usize) -> i32` *unsound* volt: üres slice-szal és `n = 1`-gyel egy teljesen safe hívás is UB-t (határon túli olvasást) váltott ki. Ez pontosan az, amit a Biztonságos absztrakció unsafe alapon alfejezet safe-absztrakciós szerződése tilt. A javított változatok a helyes tartományon (`n <= v.len()`, túlcsordulás nélkül) ugyanazt az eredményt adják, mint a safe `sum_first`; a gyakorlatban mégis a slice-műveletes változat az idiomatikus megoldás.

18.1.9.3 3. UB-katalógus

Példa `rkv-core/docs/unsafe-ub.md`:

```
# Unsafe UB-katalógus

Minden példa önálló `src/bin/<név>.rs` program a `miri-ub` scratch crate-ben; futtatás: `cargo
↪  +nightly miri run --bin <név>`. A példáknak nincs elvárt kimenete, a Miri hibaüzenete az
↪  eredmény.

| UB-forrás | Önálló példa (`src/bin/`) | Miri-üzenet (kivonat) | Hívó invariánsa |
|---|---|---|---|
| &mut aliasing | `alias_bad.rs`: `let p: *mut i32 = r; *r += 1; *p += 1;` | `attempting a read
↪  access using <…>, but that tag does not exist in the borrow stack` | `&mut` érvényessége
↪  alatt nincs más hozzáférés |
| érvénytelen bool | `bool_bad.rs`: `*(&2u8 as *const u8 as *const bool)` | `constructing
↪  invalid value of type bool: encountered 0x02` | a memória csak 0/1 lehet |
| érvénytelen enum | `enum_bad.rs`: `transmute::<u8, ErrorCode>(3)` | `constructing invalid
↪  value of type ErrorCode: at .<enum-tag>, encountered 0x03, but expected a valid enum tag` |
↪  diszkrimináns érvényes |
| igazítás | `align_bad.rs`: `*(bytes.as_ptr().add(1) as *const u32)` | `accessing memory based
↪  on pointer with alignment 1, but alignment 4 is required` | cím igazított a T-hez |
| adatverseny | `race_bad.rs`: két szál ír egy `i32`-t szinkronizáció nélkül | `Data race
↪  detected between (1) non-atomic write on thread main and (2) non-atomic write on thread
↪  unnamed-1` | szinkronizáció vagy atomikus típus |
| dangling pointer | `dangle_bad.rs`: `drop(Box::from_raw(p)); *p` | `memory access failed:
↪  alloc… has been freed, so this pointer is dangling` | pointer az élettartamon belül marad |
```

A Miri-üzenetek a nightly verziójától függően eltérhetnek; a sorok a Rust 1.99.1-gyel egy időben telepített nightly Mirivel futtatva adták ezeket. A Miri csak a lefutó utat ellenőrzi, idegen C függvényt nem hajt végre.

A katalógus a Biztonságos absztrakció unsafe alapon alfejezetben a belépési pontok ellenőrzéséhez, a C ABI fejezetben pedig a C-ből érkező adatok validálásához nyújt referenciát.

18.1.9.4 4. Hibajavítás

A hiba: ha a `raw` értéke `3`, a `transmute` érvénytelen diszkriminánst hoz létre, ami UB. A javítás `TryFrom` implementációval:

```
#[repr(u8)]
#[derive(Debug, PartialEq, Eq)]
enum ErrorCode {
    Ok = 0,
    NotFound = 1,
    Internal = 2,
}

impl TryFrom<u8> for ErrorCode {
    type Error = &'static str;

    fn try_from(raw: u8) -> Result<Self, Self::Error> {
        match raw {
            0 => Ok(ErrorCode::Ok),
            1 => Ok(ErrorCode::NotFound),
            2 => Ok(ErrorCode::Internal),
            _ => Err("invalid error code"),
        }
    }
}

fn read_error_code(raw: u8) -> ErrorCode {
    ErrorCode::try_from(raw).unwrap_or(ErrorCode::Internal)
}
```

Így a hívó soha nem kap érvénytelen `ErrorCode` példányt; a `3` esetén a `read_error_code` `Internal`-t ad vissza, de a `TryFrom` explicit hibaesetet is lehetővé tesz.

18.1.10 Rövid összefoglaló

Az `unsafe` nyelvi határai öt képességet engednek meg: nyers mutató dereferálása, `unsafe fn` hívása, `unsafe trait` impl, `static mut` elérése és `union` mezők használata. Az `unsafe` nem kapcsolja ki a borrow checkert, és a nyers mutatók (`*const T`, `*mut T`) dereferálásához a hívónak nem-null, igazított, érvényes értékreprezentációjú memóriát kell garantálnia. A leggyakoribb UB-források a `&mut` aliasing megsértése, az érvénytelen `bool`/`enum` bitminták, az igazítás megsértése, az adatverseny és a dangling pointer dereferálás. A Miri interpretáló futás közben detektálja ezeket, ezért unsafe kód tesztelésekor futtassuk `cargo +nightly miri test` paranccsal (a pontos parancs verziófüggő). **Továbbvitt eredmény:** az `rkv-core/docs/unsafe-ub.md` UB-katalógus a Biztonságos absztrakció unsafe alapon alfejezetben az invariánsok azonosításához, a C ABI: extern “C”, repr(C), bindgen és cbindgen alfejezetben pedig a C-ből érkező adatok validálásához használható.

18.1.11 Források és ellenőrzési hiányok

A fejezet a Rust Reference `unsafe` fejezetére, a Rustonomiconra (https://doc.rust-lang.org/nomicon/) és az Unsafe Code Guidelines tárgykörére épül. A Miri telepítése és a `cargo +nightly miri run` parancs a Miri dokumentációjában olvasható (https://github.com/rust-lang/miri);

a fent közölt hibaüzenet-leírás tájékoztató jellegű, a pontos szöveg verziófüggő. Az `unsafe` öt képessége és a nyers mutatók szabályai a stabil nyelvi szerződés részei; a `bool`/`enum` érvényes reprezentációjára vonatkozó UB-definíció a Rust Reference-ben és a Nomiconban található.

18.2 Biztonságos absztrakció unsafe alapon

18.2.1 Motiváció: a biztonságos felszín és az unsafe mag

A C++-ban a `std::vector` is biztonságosnak tűnő API-t ad, de a háttérben nyers mutatókkal, allokátorokkal és kézi élettartam-kezeléssel dolgozik. Ha egy C++-programozó saját konténert, hatékony szeletelő függvényt vagy lock-free adatszerkezetet ír, a felelősség általában szétszóródik az egész kódban: a fordító nem jelzi, ha egy `reinterpret_cast` vagy pointer-aritmetika megszeg egy aliasing-szabályt. A Rust ezt a határt szándékosan a modulhatárra teszi. A `Vec`, a `slice::split_at_mut` és a `Mutex` mind biztonságos publikus felületet adnak, de a belsejükben célzott `unsafe` blokkok vannak. Ez az alfejezet azt mutatja meg, hogyan kell ilyen absztrakciót építeni: nem „minél kevesebb unsafe”, hanem az unsafe blokkokat körülvevő, pontosan dokumentált invariánsok a lényeg.

18.2.2 Fogalmak és szabályok

Definíció (biztonságos absztrakció, safe abstraction). Egy publikus API biztonságos, ha *minden* lehetséges, csak biztonságos Rust-kóddal történő hívása mentes az undefined behaviourtől (UB). Az UB megelőzése kizárólag az API mögötti, ellenőrzött `unsafe` blokkok feladata.

Definíció (soundness). Egy függvény akkor *sound*, ha nincs olyan bemenet vagy programállapot, amelyre a biztonságos hívást követően UB következhet. Egy `unsafe fn` esetén a dokumentált előfeltételek betartása a hívó felelőssége; egy biztonságos `fn` esetén a hívó semmilyen módon nem szegheti meg az invariánst.

Szabály. Minden `unsafe` blokkban a programozó felel azért, hogy a blokkban hívott `unsafe` műveletek dokumentált előfeltételei teljesüljenek. Ezt a blokk előtt `// SAFETY:` kommenttel kell indokolni. A kommentnek a konkrét invariánsokra kell hivatkoznia, nem elég annyi, hogy „biztonságos”.

Szabály. A `safe fn` nem teheti a hívót UB-ért felelőssé. Ha egy függvény előfeltétele a hívó által betartandó (például az index egy érvénytelenített pointerre mutat), akkor a függvényt `unsafe fn`-ként kell deklarálni, és a dokumentációban `# Safety` szakaszban fel kell tüntetni a feltételeket. A `split_at_mut` viszont biztonságos, mert a `mid <= len` feltételt az `assert!` ellenőrzi. Az `unsafe fn` tehát csak az olyan ellenőrizetlen biztonsági előfeltételekre való, amelyeket a függvény maga nem tud kikényszeríteni.

A `split_at_mut` határeseteit Miri alatt is lefuttattuk: üres slice, `mid == 0`, `mid == len`, nulla méretű elemtípus rendben lefutott; `mid > len` esetén biztonságos panic keletkezik, `mid <= slice.len()` üzenettel (nem UB).

A clippy `undocumented_unsafe_blocks` lintje (alapértelmezés szerint nem aktív, `#![warn(clippy::undocument` kel kapcsolható be) figyelmeztet, ha egy `unsafe` blokk előtt hiányzik a `SAFETY` komment; a 2024-es editionben az `unsafe fn`-ek belsejében is explicit `unsafe` blokk szükséges az `unsafe_op_in_unsafe_fn` lint miatt (ellenőrzési igénnyel: a pontos viselkedés edition- és verziófüggő).

18.2.3 C++ és Rust összevetés

A C++20 `std::span` segítségével is készíthetünk szeletelő függvényt, de a biztonsági garancia más:

```
#include <cassert>
#include <span>
#include <utility>
#include <vector>

template<class T>
std::pair<std::span<T>, std::span<T>>
split(std::vector<T>& v, std::size_t mid) {
    assert(mid <= v.size());
    std::span<T> s(v);
    return {s.subspan(0, mid), s.subspan(mid)};
}
```

Ez a kód határellenőrzést végez, de a visszaadott `span`-ek mögött a `v` továbbra is elérhető és módosítható. Ha a hívó a `split` után `v.push_back`-et hív, a spánok invalidálódhatnak, és a fordító nem szól. A Rust-megfelelő:

```
pub fn split_at_mut<T>(slice: &mut [T], mid: usize) -> (&mut [T], &mut [T]) {
    assert!(mid <= slice.len());
    let len = slice.len();
    let ptr = slice.as_mut_ptr();
    unsafe {
        // SAFETY: mid <= len, ezért a [0, mid) és [mid, len) intervallumok
        // diszjunktak és lefedik a teljes slice-ot. A ptr a slice első
        // elemére mutat, a slice elemei inicializáltak, és a &mut slice
        // kizárólagos hozzáférést ad a függvény hívása alatt.
        let left = std::slice::from_raw_parts_mut(ptr, mid);
        let right = std::slice::from_raw_parts_mut(ptr.add(mid), len - mid);
        (left, right)
    }
}
```

A Rust-változat lefoglalja az eredeti `&mut [T]`-t: a visszaadott két kölcsönzés élettartama alatt a fordító megakadályozza, hogy bárki újra elérje az eredeti slice-ot. Ez az aliasing-szabály nem a fejlesztő fegyelmén, hanem a típusrendszeren alapul.

18.2.4 Kidolgozott példa: `split_at_mut` újraimplementálása

A standard könyvtár `slice::split_at_mut` je pontosan ezt a mintát követi. Nézzük először a hibás változatot:

```
pub fn split_at_mut_bad<T>(slice: &mut [T], mid: usize) -> (&mut [T], &mut [T]) {
    let len = slice.len();
    let ptr = slice.as_mut_ptr();
    unsafe {
        // Nincs ellenőrzés: ha mid > len, ptr.add(mid) túlmutat, len - mid
        // alulcsordul, a from_raw_parts_mut érvénytelen tartományt kap.
        let left = std::slice::from_raw_parts_mut(ptr, mid);
        let right = std::slice::from_raw_parts_mut(ptr.add(mid), len - mid);
        (left, right)
    }
}
```

Ez a változat lefordul, de `mid > len` esetén UB-t okoz. A biztonságos függvényben az `assert!` ezt kizárja. A javított kódot fentebb már láttuk; a kulcsmondat a `// SAFETY:` kommentben az, hogy a `mid <= len` feltétel miatt a két rész diszjunkt, ezért nem sérül az egyidejű `&mut` kizárólagosság.

Így tesztelhetjük egy laborban:

```
mkdir -p ~/rust-labor/r043 && cd ~/rust-labor/r043
cargo new split_lab && cd split_lab
```

A `src/main.rs` tartalma:

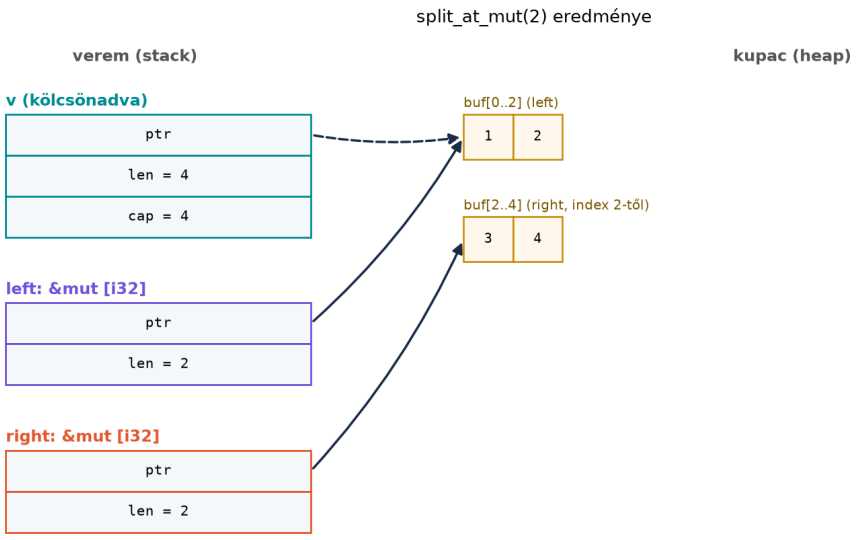
```
fn split_at_mut<T>(slice: &mut [T], mid: usize) -> (&mut [T], &mut [T]) {
    assert!(mid <= slice.len());
    let len = slice.len();
    let ptr = slice.as_mut_ptr();
    unsafe {
        // SAFETY: az assert! miatt mid <= len, így a [0, mid) és [mid, len)
        // diszjunkt, a slice-on belüli tartományok; ptr a &mut slice
        // kizárólagos hozzáférésén alapul, az elemek inicializáltak.
        let left = std::slice::from_raw_parts_mut(ptr, mid);
        let right = std::slice::from_raw_parts_mut(ptr.add(mid), len - mid);
        (left, right)
    }
}

fn main() {
    let mut v = vec![1, 2, 3, 4];
    let (a, b) = split_at_mut(&mut v, 2);
    a[0] += 10;
    b[1] *= 2;
    println!("{a:?} {b:?}");
}
```

Várt kimenet:

```
[11, 2] [3, 8]
```

Az alábbi ábra *szemléltető modell*: egyetlen kupacpuffert mutat két szomszédos tartományra bontva, mert a `left` és a `right` ugyanannak a négyelemű pufferének két, nem átfedő részére mutat. A `[1, 2, 3, 4]` adaton `split_at_mut(2)` után `left = [1, 2]` (az `0..2` indexek), `right = [3, 4]` (a `2..4` indexek); a `right` pointere tehát a *harmadik* elemre (a 2-es indexre) mutat, nem a puffer elejére:



18.2. ábra: split_at_mut(2) eredménye - Szemléltető modell: egy négyelemű kupacpuffer két szomszédos, nem átfedő tartománya. A left a 0..2, a right a 2..4 indexű elemekre mutat. A v kölcsönadott, a kölcsönzés alatt nem használható.

A két tartomány diszjunkt, ezért mindkettőre egyszerre adható `&mut` referencia. A `v`-t a kölcsönzés idejére érvénytelenné teszi a borrow checker (szaggatott nyíl).

18.2.5 Esettanulmány: egy egyszerű `MyVec` vázlat

A `Vec` teljes implementációja túlmutat ezen az alfejezeten, de egy egyszerű vázlat jól mutatja, hogyan kerülnek az invariánsok a `unsafe` blokkok köré. A `MyVec<T>` három mezővel bír: egy `NonNull<T>` pointer, egy `len` és egy `cap`.

```
use std::alloc::{self, Layout};
use std::mem;
use std::ptr::{self, NonNull};

pub struct MyVec<T> {
    ptr: NonNull<T>,
    len: usize,
    cap: usize,
}

impl<T> MyVec<T> {
    pub fn new() -> Self {
        // Nulla méretű típusnál (ZST) nincs mit allokálni: a kapacitás
        // „végtelen", a pointer végig dangling marad.
        let cap = if mem::size_of::<T>() == 0 { usize::MAX } else { 0 };
        Self { ptr: NonNull::dangling(), len: 0, cap }
    }

    pub fn push(&mut self, value: T) {
        if self.len == self.cap {
            self.grow();
        }
        unsafe {
            // SAFETY: len < cap, ezért a ptr.offset(len) az allokált pufferen
```

```
            // belül van; a célhely inicializálatlan, így write biztonságos.
            self.ptr.as_ptr().add(self.len).write(value);
        }
        self.len += 1;
    }

    fn grow(&mut self) {
        // ZST-nél cap == usize::MAX, így ide csak len == usize::MAX esetén
        // jutunk: ez kapacitás-túlcsordulás, nem allokációs igény.
        assert!(mem::size_of::<T>() != 0, "capacity overflow");
        let new_cap = if self.cap == 0 { 4 } else { self.cap.checked_mul(2).expect("capacity
        ↪ overflow") };
        // Az unwrap pánikol, ha a bájtméret meghaladná az isize::MAX-ot.
        let new_layout = Layout::array::<T>(new_cap).unwrap();
        let new_ptr = if self.cap == 0 {
            unsafe {
                // SAFETY: size_of::<T>() > 0 (fent ellenőriztük) és new_cap >= 4,
                // ezért new_layout.size() > 0: az alloc nulla méretű kéréssel
                // nem hívható.
                let p = alloc::alloc(new_layout) as *mut T;
                if p.is_null() {
                    alloc::handle_alloc_error(new_layout);
                }
                p
            }
        } else {
            let old_layout = Layout::array::<T>(self.cap).unwrap();
            unsafe {
                // SAFETY: a self.ptr ezzel a old_layout-tal lett allokálva;
                // a realloc megtartja az elemeket, a régi pointert nem
                // használjuk többé.
                let p = alloc::realloc(
                    self.ptr.as_ptr() as *mut u8,
                    old_layout,
                    new_layout.size(),
                ) as *mut T;
                if p.is_null() {
                    alloc::handle_alloc_error(new_layout);
                }
                p
            }
        };
        self.ptr = NonNull::new(new_ptr).unwrap();
        self.cap = new_cap;
    }
}

impl<T> Drop for MyVec<T> {
    fn drop(&mut self) {
        unsafe {
            // SAFETY: a [0, len) elemek érvényesek, inicializáltak, és
            // mindegyik pontosan egyszer kerül dropolásra.
            ptr::drop_in_place(ptr::slice_from_raw_parts_mut(self.ptr.as_ptr(), self.len));
        }
        // ZST-nél soha nem allokáltunk, ezért nincs mit felszabadítani.
        if self.cap != 0 && mem::size_of::<T>() != 0 {
            let layout = Layout::array::<T>(self.cap).unwrap();
            unsafe {
                // SAFETY: a pointert ezzel a layouttal allokáltuk, az elemek
                // már droppolva vannak, a dealloc felszabadítja a memóriát.
                alloc::dealloc(self.ptr.as_ptr() as *mut u8, layout);
            }
        }
```

```
    }
}
```

A `MyVec` belső invariánsai:

- ha `size_of::<T>() > 0`: `ptr` vagy dangling (`cap == 0`), vagy egy `Layout::array::<T>(cap)` elrendezéssel kapott, élő allokáció kezdőcíme;
- ha `T` nulla méretű típus (ZST, például `()`): `cap == usize::MAX`, a `ptr` mindig dangling, allokáció és `dealloc` soha nem történik;
- `len <= cap`;
- a `[0, len)` tartomány inicializált elemeket tartalmaz, a többi memória inicializálatlan; a `Drop` pontosan ezt a `len` darab elemet droppolja, egyszer. ZST-nél is: a `drop_in_place` a dangling, de igazított pointeren `len` darab nulla méretű elemet droppol, ami megengedett.

A ZST-k külön kezelése nem szőrszálhasogatás. A `Layout::array::<()>(4).size()` értéke `0`, a `GlobalAlloc::alloc` szerződése pedig tiltja a nulla méretű kérést: a `new_cap` puszta növelése nem ad pozitív bájtméretet. Az ellenőrzés nélküli változatban egy teljesen safe `MyVec::<()>::new().push(())` hívás UB-t váltana ki. A választott megoldás ugyanaz, mint a std `Vec`-é (lásd a Rustonomicon „Handling Zero-Sized Types” fejezetét). A másik sound lehetőség az, hogy a `new` elején `assert!(size_of::<T>() != 0)` utasítja el a ZST-t; ekkor ezt a dokumentációban rögzíteni kell. Az elfogadási teszt: `MyVec::<()>` push/pop, valamint egy `Drop`-számláló ZST-vel a droppolások száma, `cargo +nightly miri test` alatt is. A vázlatot `String` elemekkel (push, többszöri növekedés, drop) és ZST-vel is lefuttattuk `cargo +nightly miri test`-tel, a három teszt hibátlanul lefutott.

A vázlat korlátai. A `MyVec` nem teljesen panic-biztos, de a két hibaforrást külön kell választani. (1) **Növekedési hiba:** a `grow` a `checked_mul`, a `Layout::array` vagy az allokáció hibájánál pánikol, illetve `handle_alloc_error`-rel leáll; ekkor még egyetlen elem sem került eldobásra, és a régi puffer tulajdonjoga változatlanul a `self`-nél van, ezért a `Drop` a normál úton felszabadítja – itt nincs memóriaszivárgás. (2) **`T::drop` pánikja:** a veszélyes pont a `Drop::drop`-ban a `drop_in_place`: ha egy elem destruktora pánikol, a `Drop` unwindol, a `dealloc` már nem fut le, és a még el nem dobott elemek is elszivárognak. A teljes `Vec` ezért őrszerkezetet (például `SetLenOnDrop`/`DropGuard`) használ, amely pánik közben is felszabadítja a puffert; ez a vázlat nem. A két eset külön tesztelhető:

```rust
#[cfg(test)]
mod panic_tests {
    use super::*;

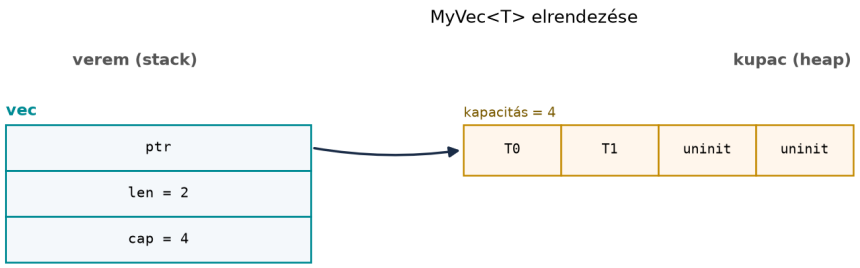
    #[test]
    fn capacity_overflow_panics_before_allocation() {
        let mut v: MyVec<u8> = MyVec::new();
        // A következő növelés már nem fér el: a checked_mul None-t ad,
        // a grow az allokáció előtt pánikol, a puffer a self-nél marad.
        v.cap = usize::MAX / 2 + 1;
        v.len = v.cap;
        let result = std::panic::catch_unwind(std::panic::AssertUnwindSafe(|| v.push(0)));
        assert!(result.is_err());
        std::mem::forget(v); // a teszt szándékosan inkonzisztens állapotot hagyott hátra
    }

    struct PanicOnDrop(u8);
    impl Drop for PanicOnDrop {
        fn drop(&mut self) {
            let _ = self.0; // a mező használata, hogy ne legyen holt kód
            panic!("szándékos pánik a Drop-ban");
        }
    }
```

```
    #[test]
    fn drop_panic_leaks_the_buffer() {
        let mut v = MyVec::new();
        v.push(PanicOnDrop(0));
        let result = std::panic::catch_unwind(std::panic::AssertUnwindSafe(|| drop(v)));
        assert!(result.is_err());
        // A vázlatban a puffer és a még el nem dobott elemek elszivárognak;
        // ezért ezt a tesztet nem futtatjuk Miri alatt (a leak check elbukna).
    }
}
```

A `push` növekedésnél `realloc`-ot használ, ami bitmásolásként mozgatja az elemeket, ha a blokk költözik. Ez azért érvényes, mert a `T` Rustban mozgatható; a régi helyet nem droppoljuk külön. A `Drop` megvalósításban a `drop_in_place` a teljes `[0, len)` szeletre droppol, a `dealloc` pedig a memóriát szabadítja fel.

Az alábbi ábra a `MyVec` memóriafoglalását mutatja:



18.3. ábra: MyVec<T> elrendezése - A ptr a kupacon lévő, cap T méretű pufferre mutat; len <= cap.

Továbbvitt eredmény. A soundness és a `// SAFETY:` dokumentáció itt bemutatott mintája az alapja a C ABI: extern “C”, repr(C), bindgen és cbindgen alfejezetnek, ahol a C-ből hívott függvények előfeltételeit kell dokumentálni, valamint a no_std, embedded-hal és Embassy: Rust mikrovezérlőn alfejezet hardverközeli absztrakcióinak.

18.2.6 Tipikus hibák és átállási csapdák

1. **`unsafe` a függvény aláírásában, amikor nem a hívó felel.** A `MyVec::push` biztonságos, mert a hívó bármilyen `T`-vel – nulla méretű típussal is – hívhatja; a kapacitást és a ZST-esetet a típus maga kezeli. Ha a hívónak kellene betartania valamilyen előfeltételt, akkor `unsafe fn` és `# Safety` szakasz kell.
2. **Üres `SAFETY` komment.** A `// SAFETY: biztonságos` nem dokumentál semmit. Minden kommentnek a konkrét invariánsra kell hivatkoznia: például „`mid <= len`, ezért a két tartomány diszjunkt”, vagy „a célhely inicializálatlan”.
3. **`unsafe` blokkok túlméretezése.** Ha egy `unsafe` blokk sok műveletet tartalmaz, nehéz ellenőrizni az invariánst. A cél az, hogy a blokk pontosan a `unsafe` hívás köré szűküljön.
4. **A C++-ból hozott `memcpy`-gondolkodás.** Rustban a `ptr::copy_nonoverlapping` és a `ptr::read`/`write` is `unsafe`; a C++-ból ismert `memcpy` bitmásolás nem mindig elég, ha a típusnak `Drop` implementációja van, mert a régi helyet nem szabad újra droppolni.

18.2.7 Feladatok

1. (a) Fogalmi kérdés: Miért biztonságos a `split_at_mut` annak ellenére, hogy belül `unsafe` blokk van? Mi történne, ha `unsafe fn`-ként deklarálnánk?

2. (b) Írd át a fenti C++ `split` függvényt idiomatikus Rustra úgy, hogy a hívó a visszaadott két szeletet egyidejűleg módosíthassa, és az állapot-invalidálás fordítási időben ki legyen zárva.

3. (c) Egészítsd ki a `MyVec<T>` vázlatot egy `pop` metódussal, amely `Option<T>`-t ad vissza, és egy `len` metódussal. Írj hozzá `SAFETY` kommentet.

4. (d) Tervezési döntés: Tegyük fel, hogy a `MyVec`-hez publikus `set_len(&mut self, new_len: usize)` metódust akarunk adni. Miért nem lehet ezt biztonságos függvényként kitenni? Hogyan kellene `unsafe fn`-ként dokumentálni a `# Safety` szakaszban?

18.2.8 Megoldások

1. (a) A `split_at_mut` biztonságos, mert a `mid <= len` feltételt az `assert!` ellenőrzi, így a hívó semmilyen módon nem tud UB-t előidézni. Ha `unsafe fn`-ként deklarálnánk, a hívónak kellene bizonyítania a feltételt, és a hívás helyén `unsafe` blokk kellene. Ez rontaná az ergonómiát, és feleslegesen terjesztené az unsafe-et.

2. (b) A Rust-megoldás a fenti `split_at_mut`; a hívó a `let (a, b) = split_at_mut(&mut v, 2);` után `a`-t és `b`-t módosíthatja, de `v`-t nem használhatja, amíg `a` és `b` él. A fordító ezt enforce-olja.

3. (c) A `pop` implementáció:

```rust
pub fn pop(&mut self) -> Option<T> {
    if self.len == 0 {
        None
    } else {
        self.len -= 1;
        unsafe {
            // SAFETY: a self.len csökkentése után a korábbi len-1 indexű elem
            // érvényes, inicializált, és a MyVec invariáns szerint a MyVec
            // birtokolja. A read bitmásolja az értéket, a régi helyet nem
            // droppoljuk, mert a len már nem tartalmazza.
            Some(self.ptr.as_ptr().add(self.len).read())
        }
    }
}
```

A `len` metódus triviálisan `self.len`. 4. (d) A publikus `set_len` veszélyes, mert a hívó `new_len > cap` értéket is beállíthat, ami későbbi olvasásnál inicializálatlan memóriához vagy deallokáció utáni hozzáféréshez vezet. Ha mégis szükséges, `unsafe fn`-ként kell kitenni:

```rust
/// Beállítja a vektort `new_len` eleműre.
///
/// # Safety
///
/// A hívó köteles biztosítani, hogy:
/// - `new_len <= self.cap`, és
/// - a `[len, new_len)` tartomány minden eleme inicializált.
pub unsafe fn set_len(&mut self, new_len: usize) {
    self.len = new_len;
}
```

18.2.9 Rövid összefoglaló

A biztonságos absztrakció lényege, hogy a `safe fn` soha nem okozhat UB-t, függetlenül a hívótól. Az UB-ért felelős kódot célzott `unsafe` blokkokba zárjuk, és minden blokk előtt `// SAFETY:` kommenttel indokoljuk, melyik invariáns miért teljesül. A `split_at_mut` példája megmutatja, hogy egy `assert!` hogyan teszi biztonságossá a pointer-aritmetikát; a `MyVec<T>` vázlat pedig bemutatja, hogyan tartja nyilván a `len <= cap` és az inicializáltság invariánsait a `push` és a `Drop` implementációja. A soundness fogalma és a `# Safety` dokumentáció továbbvitelre kerül a C ABI-hoz és a beágyazott absztrakciókhoz.

18.2.10 Források és ellenőrzési hiányok

- A Rust Referencia `unsafe` blokkokra és UB-listára vonatkozó fejezetei a `doc.rust-lang.org`-on elérhetők, de a pontos szakaszok verziófüggők.
- A `std::alloc` és `std::slice::from_raw_parts_mut` dokumentációja a docs.rs-en ellenőrizhető a konkrét stable toolchainre.
- A clippy `undocumented_unsafe_blocks` lintje alapértelmezés szerint nem aktív, bekapcsolása `#![warn(clippy::undocumented_unsafe_blocks)]` vagy a `clippy.toml`/parancssor útján történik; az `unsafe_op_in_unsafe_fn` a 2024-es editionben figyelmeztet. A példakódok a rögzített Rust 1.99.1 (2024-es edition) szerint készültek.

19 FFI: C és C++ együttműködés

19.1 C ABI: extern “C”, repr(C), bindgen és cbindgen

19.1.1 Motiváció

Egy évtizedes C++-kódbázis ritkán cserélhető le egyik napról a másikra Rustra. A gyakorlatban a két nyelv ugyanabban a folyamatban él: meglévő C vagy C++ könyvtárat hívunk Rustból, vagy egy új Rust komponenst teszünk elérhetővé a régi C++ kliens számára. A közös nevező a C ABI: a C függvényhívási konvenció, a C adatszerkezetek és a C stringek. A Rust nem beszéli a C++ osztályok, túlterhelt függvények, kivételek vagy sablonok nyelvét, de a C ABI-n keresztül bármilyen C++ kód elérhető, ha `extern "C"` függvényekbe csomagoljuk.

Az rkv mintaprojektben most azt a hidat építjük meg, amelyen a későbbi C++ kliens átkelhet: az `rkv-ffi` crate egy C API-t exportál, amelyet a C++-együttműködés a cxx crate-tel és a fokozatos migráció alfejezet használ tovább, a záróprojektben pedig a teljes rendszer része lesz.

19.1.2 Fogalmak és szabályok

Definíció. Az *ABI* (application binary interface) az a bináris szerződés, amely meghatározza, hogyan adjuk át az argumentumokat, hogyan rendezzük a struktúrákat, és hogyan nevezzük a szimbólumokat. A Rust alapértelmezett ABI-ja nem stabil, ezért C-vel való együttműködésnél mindig explicit `extern "C"` blokkot használunk.

Definíció. A `#[repr(C)]` attribútum C-kompatibilis memóriaelrendezést ad a Rust típusoknak: a mezők sorrendje megegyezik a deklaráció sorrendjével, nincs újrarendezés, és a padding a C szabályait követi. A `#[repr(Rust)]` (az alapértelmezett) esetén a fordító szabadon átrendezheti a mezőket, ezért C-nek átadott struktúrákhoz mindig `#[repr(C)]` kell.

Két külön kérdés. Egy FFI-határ tervezésekor két dolgot kell szétválasztani. (1) Az **ABI-kompatibilitás** azt dönti el, hogy a két oldal ugyanúgy értelmezi-e a bájtokat: `#[repr(C)]` elrendezés, C-kompatibilis mezőtípusok (`c_int`, mutatók, `repr(C)` structok, nem `String`/`Vec`/`&T`), a hívási konvenció (`extern "C"`). (2) Az **értékérvényességi és ownership szerződés** azt, hogy az átadott bájtok érvényes értéket jelentenek-e, ki szabadítja fel őket, meddig élnek, és használhatók-e egyszerre több szálról. Egy `*const c_char` ABI-szempontból tökéletes, a szerződés (NUL-végű? UTF-8? meddig él?) viszont nincs a típusában: ezt dokumentálni és ellenőrizni kell. Fordítva, egy `repr(C)` struct mezői csak akkor C-kompatibilisek, ha maguk is azok.

Ajánlás (nyers mutatók a határon). A Rust referencia (`&T`, `&mut T`) a C ABI-ban hívási szinten mutatóként adódna át, de a Rust-oldali érvényességi garanciái (nem null, igazított, él, kizárólagos vagy megosztott) C-ből nem kényszeríthetők ki. Ezért az egyszerű határ-API-ban nyers mutatókat (`*const T`, `*mut T`) használunk, és a függvény belsejében ellenőrizzük őket (null, hossz), mielőtt referenciát képzünk belőlük. Az ownership-átadásról a hívóval egyértelműen megállapodunk.

Szabály (panic a határon). Egy `extern "C"` függvényből kilépő Rust pánik **nem** terjedhet a C hívóba. A Rust 1.81 óta az `extern "C"` függvényből kilépő pánik a folyamat **leállítását**

(abort) okozza (korábban UB volt). Három külön eset van: (a) *Rust pánik* az `extern "C"` Rust-exportban: abort, hacsak a függvény törzsét `catch_unwind`-del nem védjük; (b) *idegen kivétel* (például C++ `throw`), amely Rust frame-eken át terjedne: ez más eset, mint a Rust pánik. A nem unwind ABI-n (`extern "C"`) Rustba belépő idegen unwind **nem definiált viselkedés** lehet; a Rust a pánikra ad abort-garanciát, az idegen kivételre nem (a Rustonomicon külön tárgyalja). A szabály ezért: C++ kivétel nem hagyhatja el azt a C++ függvényt, amelyet Rustból (vagy Rust-exportból C++-ból) `extern "C"` határon hívunk; a C++ oldalon `try { ... } catch (...) { return RKV_ERR; }` alakú védelem kell. Ha a terjedést szándékosan engedjük, az `extern "C-unwind"` ABI kell mindkét oldalon; (c) a `panic = "abort"` profil: ekkor nincs unwind, a `catch_unwind` nem fog el semmit, a pánik azonnal leállítja a folyamatot.

Az `rkv-ffi` explicit policyja: a crate `panic = "unwind"` profillal készül (ez a Cargo alapértelmezése `dev` és `release` profilban is), és minden státuszkódot adó exportált függvény (`rkv_set`, `rkv_get`, `rkv_delete`) törzse a `guarded` segédfüggvényen megy át (a `new`, `free` és `buffer_free` életciklus-függvények nem adnak státuszt, és nem tartalmaznak pánikolható logikát), amely a pánikot `RKV_PANIC` státuszkóddá alakítja. A `catch_unwind` itt a *ténylegesen pánikolható határ* védelme, nem mindent felfogó háló: a szokásos hibákat (`Result`) továbbra is státuszkóddá alakítjuk, a `catch_unwind` csak a váratlan pánikot kezeli. Ha a projekt `panic = "abort"` profilt választ, a dokumentáció ezt vállalja: a pánik leállítja a folyamatot, `RKV_PANIC` nem keletkezik.

Szabály. A `#[no_mangle]` attribútum megakadályozza a szimbólum átnevezését, így a függvény neve a C linker számára látható marad. A Rust 2024-es editionben ez az attribútum az unsafe attribútumok közé került, ezért `#[unsafe(no_mangle)]` formában kell írni; a 2021-es editionben a `#[no_mangle]` továbbra is működik. *(Ellenőrzési igény: a pontos szintaxis a telepített toolchain verziójától függ.)*

19.1.3 C hívása Rustból: extern blokk és bindgen

Egy meglévő C könyvtár legegyszerűbb hívása kézzel írt `extern` blokkal történik. A 2024-es editionben az extern blokk `unsafe extern "C"`, a C `int` Rust-megfelelője pedig a platformfüggetlen `c_int` (`std::ffi::c_int`), nem az `i32`:

```c
// mylib.h
int add(int a, int b);
```

```rust
use std::ffi::c_int;

// Kézzel írt kötés (2024 edition: unsafe extern)
unsafe extern "C" {
    fn add(a: c_int, b: c_int) -> c_int;
}

fn main() {
    // SAFETY: a C függvénynek nincs előfeltétele, és a hívás nem sért Rust-invariánst.
    let sum = unsafe { add(2, 3) };
    println!("{sum}");
}
```

A linkeléshez a `build.rs`-ben meg kell adni a könyvtárat. Nagyobb C fejléchez a kézi fordítás hibalehetőség; erre való a `bindgen`, amely a C fejlécből generálja a Rust kötést. Az alábbi teljes, ellenőrzött példa a C forrást is lefordítja (a `cc` crate), a kötést az `OUT_DIR`-be generálja (nem a forrásfába), és onnan `include!`-olja. A `Cargo.toml`-ban `[build-dependencies]` közé `cc` és `bindgen` kerül; a `bindgen` a **libclang** könyvtárat igényli (Ubuntu: `libclang-dev`), amely a *natív toolchain* része.

Kódblokk — fájl: `build.rs` · művelet: új fájl egy önálló `cargo new` projektben (mellé `mylib.c`, `mylib.h`, `wrapper.h`, ahol a `wrapper.h` a `#include "mylib.h"` sort tartalmazza) · ellenőrzés: `cargo run` → 5

```
use std::env;
use std::path::PathBuf;

fn main() {
    println!("cargo:rerun-if-changed=mylib.c");
    println!("cargo:rerun-if-changed=mylib.h");
    println!("cargo:rerun-if-changed=wrapper.h");

    // A C forrás fordítása és statikus könyvtárba csomagolása (libmylib.a, a linkelést a cc
    ↪  crate kéri a Cargótól).
    cc::Build::new().file("mylib.c").compile("mylib");

    // A kötés generálása az OUT_DIR-be (nem a forrásfába).
    let bindings = bindgen::Builder::default()
        .header("wrapper.h")
        .generate()
        .expect("bindgen hiba");
    let out = PathBuf::from(env::var("OUT_DIR").unwrap());
    bindings.write_to_file(out.join("bindings.rs")).expect("a bindings írása nem sikerült");
}
```

Kódblokk — fájl: `src/main.rs` · művelet: teljes csere · ellenőrzés: `cargo run` → 5

```
#![allow(non_upper_case_globals, non_camel_case_types, non_snake_case, dead_code)]
include!(concat!(env!("OUT_DIR"), "/bindings.rs"));

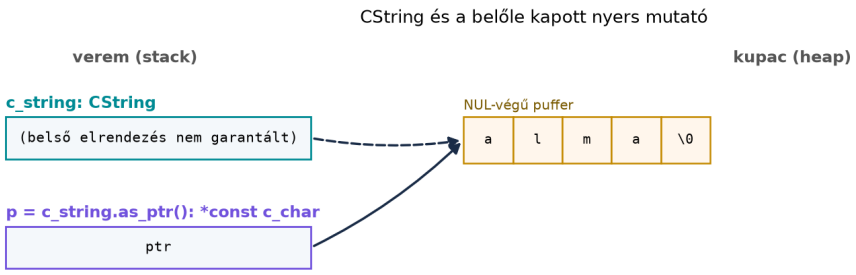
fn main() {
    // SAFETY: a C függvénynek nincs előfeltétele, és a hívás nem sért Rust-invariánst.
    let sum = unsafe { add(2, 3) };
    println!("{sum}");
}
```

A bindgen a függvényeket `unsafe extern "C"` blokkba generálja (a kimenet edition-kompatibilis), és a `c_int` típust használja. A generált fájl edition-függő részeit (`allow` attribútumok) a fogadó modul kezeli. A bindgen verziója az eszköz kimenetét befolyásolja, ezért a `Cargo.lock` rögzítse.

19.1.4 Stringek a határon: CStr és CString

A C string null-terminált bájtsorozat; a Rust `String` heap-en allokált, hossz- és kapacitásmezőkkel ellátott UTF-8 puffert. A kettő között a `std::ffi` modul típusai konvertálnak:

- `CStr`: kölcsönzött C string, mint a `&str`, de bájt-alapú és null-terminált.
- `CString`: birtokolt C string, mint a `String`, de null-terminált, és nem tartalmazhat belső null bájtot.



19.1. ábra: CString és a belőle kapott nyers mutató - Szemléltető modell. A CString belső elrendezése nem specifikált és nem stabil ABI; csak az as_ptr() által adott *const c_char nyers mutató (NUL-végű puffer) része a szerződésnek. A CStr egy kölcsönzött nézet ugyanerre a pufferre.

A `CStr::from_ptr` unsafe függvény, mert a mutatónak érvényes, null-terminált stringre kell mutatnia. A `CString::new` ellenőrzi, hogy nincs-e belső null bájt, és `NulError`-t ad vissza, ha van. A C++-ban a `std::string::c_str()` automatikusan null-terminált mutatót ad; a Rustban ezt explicit konverzióval és hibakezeléssel kell megtenni.

```cpp
#include <string>
extern "C" void c_library_set(const char* key, const char* value, size_t value_len);

void set_value(const std::string& key, const std::string& value) {
    c_library_set(key.c_str(), value.data(), value.size());
}
```

```rust
use std::ffi::{CString, c_char};

unsafe extern "C" {
    fn c_library_set(key: *const c_char, value: *const c_char, value_len: usize);
}

fn set_value(key: &str, value: &str) -> Result<(), std::ffi::NulError> {
    let key = CString::new(key)?;
    let value = CString::new(value)?;
    // SAFETY: a CString érvényes, null-terminált puffert ad, és a hívás alatt él.
    unsafe {
        c_library_set(key.as_ptr(), value.as_ptr(), value.as_bytes().len());
    }
    Ok(())
}
```

A különbség: a C++ automatikusan kezeli a null-terminálást, de a Rustban a `CString` explicit, és a belső null bájt hibát okoz. Bináris adatoknál (mint az rkv `Vec<u8>` értékei) ezért nem null-terminált stringet, hanem mutató + hossz párt adunk át, és a Rust oldalon `std::slice::from_raw_parts`-szel rekonstruáljuk a slice-ot.

19.1.5 Ownership a határon

A C ABI-n nincs Rust tulajdonjog: a hívó és a hívott közötti megállapodás határozza meg, ki szabadít fel. A leggyakoribb minták:

Az alapszabály: **minden memóriát ugyanaz az allokátor szabadít fel, amelyik lefoglalta.** A Rust globális allokátora és a C `malloc`/`free` párosa nem keverhető, akkor sem, ha egy adott platformon véletlenül ugyanaz a `malloc` van alattuk.

- A Rust oldal allokál, és a C hívó engedi el: `Box::into_raw` vagy `CString::into_raw` a Rust oldalon; a C hívó *nem* `free`-t hív, hanem a Rust által exportált, dedikált felszabadító függvényt (például `rkv_buffer_free`), amely `Box::from_raw`-val (illetve `CString::from_raw`-val) veszi vissza és dobja el a puffert.
- A C oldal allokál (`malloc`, `strdup`), és a Rust oldal használja: a Rust `CStr::from_ptr`-rel kölcsönzött nézetként olvassa, szükség esetén `to_owned()`-dal saját másolatot készít. Ha a szerződés szerint a Rust-é a felszabadítás, azt a C `free`-vel (például a `libc::free` hívásával) kell végeznie, nem `CString` drop-pal.

Fontos csapda: a `CString::from_raw` csak olyan mutatót fogadhat, amelyet a `CString::into_raw` hozott létre. Ha a C kód `malloc`-kal allokált mutatót ad át, és a Rust `CString::from_raw`-t hív rá, az undefined behavior. A C-ből érkező mutatót a Rust oldalon `CStr::from_ptr`-ként kezeljük, és nem szabadítjuk fel, kivéve ha a szerződés szerint a Rust-é a felszabadítás.

A `Box::into_raw` elveszi a Rust `Box`-ot, és visszaadja a nyers mutatót; a memória felszabadításához `Box::from_raw` kell. Ez a minta biztosítja, hogy a Rust allocator párosuljon a Rust deallokátorral.

19.1.6 Panic és hibakódok

A pánik-policy a „Fogalmak” szakaszban szerepel. A gyakorlatban a státuszkódot egy `int`-ként adjuk vissza, és a C oldal a `RKV_*` állandókkal vizsgálja. A visszatérési kód **egyértelműen megkülönbözteti** a lehetséges kimeneteleket: siker (`RKV_OK`), hiányzó kulcs (`RKV_NOT_FOUND`, ez nem hiba), érvénytelen argumentum (`RKV_INVALID_ARG`: null mutató, nem UTF-8 kulcs), belső hiba (`RKV_ERROR`) és pánik (`RKV_PANIC`). A `catch_unwind` feltételezi, hogy a profilban a pánik nem `abort`; a viselkedést **a kiadási profillal külön ellenőrizzük**: a `guarded` segéd egységtesztje `cargo test` és `cargo test --release` alatt is lefut.

```
fn guarded(f: impl FnOnce() -> c_int) -> c_int {
    catch_unwind(AssertUnwindSafe(f)).unwrap_or(RKV_PANIC)
}
```

19.1.7 Labor: rkv-ffi C ABI-val és cbindgen headerrel

Munkakönyvtár: `~/rust-labor/rkv/`

A kiinduló állapot az Az rkv hálózati szerver–Naplózás és megfigyelhetőség ellenőrzési pontja: az `rkv-core` exportálja a `SharedStore`-t és a `MemStore`-t. Ebben a laborban létrehozzuk az `rkv-ffi` crate-et, amely a `SharedStore`-ra épülő C API-t exportál, `cbindgen`-nel generáljuk a C headert, és C, valamint C++ klienst fordítunk rá.

A szerződés (mind a Rust-dokumentációban, mind a generált headerben szerepel):

Kérdés	Szerződés
Kezelő	`RkvStore *`, átlátszatlan; a `rkv_store_new` adja, a `rkv_store_free` szabadítja fel, kizárólagosan és egyszer, minden más hívás után
Párhuzamosság	a kezelő több szálról egyszerre használható (belül `Mutex`), a `free` kivételével
Kulcs	NUL-végű, érvényes UTF-8 C string
Érték	tetszőleges bájtsorozat, mutató + hossz; üres érték: `value == NULL`, `len == 0` engedett
Kimeneti puffer	`rkv_get` adja; a hívó a `rkv_buffer_free(ptr, len)`-nel szabadítja fel, egyszer; az üres érték hossza 0, a mutatója nem null
Státusz	`RKV_OK`, `RKV_NOT_FOUND`, `RKV_INVALID_ARG`, `RKV_ERROR`, `RKV_PANIC`

1. **Hozzuk létre az `rkv-ffi` crate-et és vegyük fel a workspace-be:**

```
cd ~/rust-labor/rkv
cargo new --lib rkv-ffi
```

Kódblokk — fájl: `Cargo.toml` (workspace gyökér) · művelet: `"rkv-ffi"` felvétele a `members` listába · ellenőrzés: `cargo metadata --no-deps`

```
[workspace]
resolver = "3"
members = ["rkv-core", "rkv-cli", "rkv-server", "rkv-ffi"]
```

2. **Az `rkv-ffi/Cargo.toml`:**

Kódblokk — fájl: `rkv-ffi/Cargo.toml` · művelet: teljes csere · ellenőrzés: `cargo build -p rkv-ffi`

```
[package]
name = "rkv-ffi"
version = "0.1.0"
edition = "2024"
rust-version = "1.89"

[lib]
crate-type = ["cdylib", "staticlib", "rlib"]

[dependencies]
rkv-core = { path = "../rkv-core" }
```

A `cdylib` dinamikus, a `staticlib` statikus C könyvtárat állít elő; az `rlib` a Rust-oldali tesztekhez kell (a `cargo test` ehhez linkel). A pánik-policy: a profil nem állítja `abort`-ra a `panic`-et (az alapértelmezés `unwind`).

3. **A `src/lib.rs`.** Minden exportált függvény `# Safety` szerződést kap (ez kerül a headerbe is), a kezelőből csak megosztott referenciát (`&RkvStore`) képzünk (a `SharedStore` metódusai `&self`-et vesznek, a zárat belül tartja), a törzs pedig a `guarded` segédfüggvényen megy át:

Kódblokk — fájl: `rkv-ffi/src/lib.rs` · művelet: teljes csere · ellenőrzés: `cargo build -p rkv-ffi`

```rust
//! C ABI az rkv tárolóhoz.
//!
//! Szerződés (a generált `rkv_ffi.h`-ban is szerepel):
//! * A műveleti függvények (`rkv_set`, `rkv_get`, `rkv_delete`) `int` státuszkódot adnak
//!   (`RKV_OK`, `RKV_NOT_FOUND`, `RKV_INVALID_ARG`, `RKV_ERROR`, `RKV_PANIC`), és a törzsük a
//!   `guarded` segéden megy át. Az életciklus-függvények nem adnak státuszt és nem guardedek:
//!   `rkv_store_new` kezelőt (hibánál nem tér vissza: az allokációs hiba abortot okoz),
//!   `rkv_store_free` és `rkv_buffer_free` `void`-ot ad; nincs bennük pánikolható logika.
//! * A kulcs NUL-végű, érvényes UTF-8 C string; az érték tetszőleges bájtsorozat (pointer +
↪  hossz).
//! * A `RkvStore` kezelő több szálról egyszerre is használható (belül zár védi), kivéve a
//!   `rkv_store_free`-t: az kizárólagos, egyszeri, és minden más hívás után következhet.
//! * A Rust által lefoglalt kimeneti puffert a `rkv_buffer_free(ptr, len)` szabadítja fel,
↪  pontosan egyszer.
use std::ffi::{CStr, c_char, c_int};
use std::panic::{AssertUnwindSafe, catch_unwind};
use std::ptr;

use rkv_core::{MemStore, SharedStore};

pub const RKV_OK: c_int = 0;
pub const RKV_NOT_FOUND: c_int = 1;
pub const RKV_INVALID_ARG: c_int = 2;
pub const RKV_ERROR: c_int = 3;
pub const RKV_PANIC: c_int = 4;

/// Átlátszatlan kezelő: a C oldal csak mutatót lát (`typedef struct RkvStore RkvStore;`).
pub struct RkvStore {
    inner: SharedStore,
}

/// Minden exportált függvény törzse ezen megy át: a pánikot státuszkóddá alakítja.
/// Ez csak `panic = "unwind"` profilban működik; `panic = "abort"` mellett a pánik
/// leállítja a folyamatot (ezt a könyv nem vállalja el: a csomag profilja `unwind`).
fn guarded(f: impl FnOnce() -> c_int) -> c_int {
    catch_unwind(AssertUnwindSafe(f)).unwrap_or(RKV_PANIC)
}

/// Új, memóriabeli tároló. A kezelőt a `rkv_store_free`-vel kell felszabadítani.
#[unsafe(no_mangle)]
pub extern "C" fn rkv_store_new() -> *mut RkvStore {
    let store = RkvStore { inner: SharedStore::new(MemStore::new()) };
    Box::into_raw(Box::new(store))
}

/// A kezelő felszabadítása.
///
/// # Safety
/// `store` null, vagy a `rkv_store_new` által adott, még fel nem szabadított kezelő.
/// A hívás kizárólagos: közben más szál nem használhatja a kezelőt, és utána senki sem.
#[unsafe(no_mangle)]
pub unsafe extern "C" fn rkv_store_free(store: *mut RkvStore) {
    if !store.is_null() {
        // SAFETY: a szerződés szerint a mutató a Box::into_raw eredménye, egyszer szabadítjuk.
        drop(unsafe { Box::from_raw(store) });
    }
}

/// Segéd: a kezelő megosztott referenciája (nincs `&mut`, a zár a SharedStore-ban van).
///
/// # Safety
/// `store` null vagy érvényes, élő kezelő.
unsafe fn handle<'a>(store: *const RkvStore) -> Option<&'a RkvStore> {
```

```
    // SAFETY: a hívó garantálja az érvényességet és az élettartamot.
    unsafe { store.as_ref() }
}

/// Segéd: NUL-végű UTF-8 kulcs.
///
/// # Safety
/// `key` null vagy érvényes, NUL-végű C string, amely a hívás alatt él.
unsafe fn key_str<'a>(key: *const c_char) -> Option<&'a str> {
    if key.is_null() {
        return None;
    }
    // SAFETY: a hívó szerződése szerint érvényes, NUL-végű.
    unsafe { CStr::from_ptr(key) }.to_str().ok()
}

/// Érték beállítása. Az érték tetszőleges bájtsorozat; `value` lehet null, ha `value_len == 0`.
///
/// # Safety
/// `store` érvényes kezelő; `key` NUL-végű C string; `value` legalább `value_len` olvasható
↪  bájtra mutat.
#[unsafe(no_mangle)]
pub unsafe extern "C" fn rkv_set(
    store: *const RkvStore,
    key: *const c_char,
    value: *const u8,
    value_len: usize,
) -> c_int {
    guarded(|| {
        // SAFETY: lásd a függvény # Safety szakaszát.
        let (Some(store), Some(key)) = (unsafe { handle(store) }, unsafe { key_str(key) }) else
        ↪  {
            return RKV_INVALID_ARG;
        };
        let bytes = if value_len == 0 {
            Vec::new()
        } else if value.is_null() {
            return RKV_INVALID_ARG;
        } else {
            // SAFETY: value_len olvasható bájt, a szerződés szerint.
            unsafe { std::slice::from_raw_parts(value, value_len) }.to_vec()
        };
        match store.inner.set(key.to_owned(), bytes) {
            Ok(()) => RKV_OK,
            Err(_) => RKV_ERROR,
        }
    })
}

/// Érték lekérdezése. Siker (`RKV_OK`) esetén `*out_value` egy Rust által lefoglalt puffer
/// (`*out_len` bájt; az üres érték hossza 0, a mutató ekkor is nem null), amelyet a
/// `rkv_buffer_free(*out_value, *out_len)` szabadít fel. Hiányzó kulcsra `RKV_NOT_FOUND`,
/// a kimenetek `NULL` és `0`.
///
/// # Safety
/// A függvény *normál hibakezelésként* ellenőrzi és `RKV_INVALID_ARG`-ra képezi a `NULL`
/// `store`, `key`, `out_value` és `out_len` mutatót, valamint a nem UTF-8 kulcsot. Az alábbi
/// feltételeket viszont **nem tudja ellenőrizni**, ezek a hívó felelőssége:
/// * `store` nem `NULL` esetén a `rkv_store_new` által adott, még fel nem szabadított kezelő,
///   és a hívás ideje alatt senki nem hívja a `rkv_store_free`-t (a kezelő egyébként több
///   szálról használható);
/// * `key` a NUL-ig olvasható, teljes, élő tartományra mutat, és a hívás alatt nem módosul;
/// * `out_value` és `out_len` érvényes, igazított, írható `*mut u8`, illetve `usize` tárolóra
```

```rust
///   mutat, egymással és a `key` tartományával nem fedik át egymást, és a hívás alatt más szál
///   nem fér hozzájuk.
#[unsafe(no_mangle)]
pub unsafe extern "C" fn rkv_get(
    store: *const RkvStore,
    key: *const c_char,
    out_value: *mut *mut u8,
    out_len: *mut usize,
) -> c_int {
    guarded(|| {
        if out_value.is_null() || out_len.is_null() {
            return RKV_INVALID_ARG;
        }
        // SAFETY: a kimeneti mutatók írhatók, a szerződés szerint.
        unsafe {
            *out_value = ptr::null_mut();
            *out_len = 0;
        }
        // SAFETY: lásd a függvény # Safety szakaszát.
        let (Some(store), Some(key)) = (unsafe { handle(store) }, unsafe { key_str(key) }) else
        ↪ {
            return RKV_INVALID_ARG;
        };
        match store.inner.get(key) {
            Ok(Some(value)) => {
                let boxed = value.into_boxed_slice();
                let len = boxed.len();
                // SAFETY: a kimeneti mutatók írhatók.
                unsafe {
                    *out_len = len;
                    *out_value = Box::into_raw(boxed).cast::<u8>();
                }
                RKV_OK
            }
            Ok(None) => RKV_NOT_FOUND,
            Err(_) => RKV_ERROR,
        }
    })
}

/// Törlés. `RKV_OK`, ha a kulcs létezett és törlődött; `RKV_NOT_FOUND`, ha nem létezett.
///
/// # Safety
/// `store` érvényes kezelő; `key` NUL-végű C string.
#[unsafe(no_mangle)]
pub unsafe extern "C" fn rkv_delete(store: *const RkvStore, key: *const c_char) -> c_int {
    guarded(|| {
        // SAFETY: lásd a függvény # Safety szakaszát.
        let (Some(store), Some(key)) = (unsafe { handle(store) }, unsafe { key_str(key) }) else
        ↪ {
            return RKV_INVALID_ARG;
        };
        match store.inner.delete(key) {
            Ok(true) => RKV_OK,
            Ok(false) => RKV_NOT_FOUND,
            Err(_) => RKV_ERROR,
        }
    })
}

/// A `rkv_get` által adott puffer felszabadítása.
///
/// # Safety
```

```
/// `ptr` és `len` pontosan a `rkv_get` által adott `*out_value` és `*out_len`, még fel nem
↪  szabadítva;
/// `ptr == NULL` esetén a hívás nem tesz semmit.
#[unsafe(no_mangle)]
pub unsafe extern "C" fn rkv_buffer_free(ptr: *mut u8, len: usize) {
    if !ptr.is_null() {
        // SAFETY: a puffert Box<[u8]>-ból adtuk ki pontosan ekkora hosszal.
        drop(unsafe { Box::from_raw(ptr::slice_from_raw_parts_mut(ptr, len)) });
    }
}
```

A hibakezelés négy külön ellenőrizhető ágat ad: a hiányzó kulcs `RKV_NOT_FOUND` (nem hiba), a null mutató vagy nem UTF-8 kulcs `RKV_INVALID_ARG`, a tároló hibája `RKV_ERROR`, a pánik `RKV_PANIC`. Az érték tetszőleges bájtsorozat, belső `0` és nem UTF-8 bájt is: ezért mutató + hossz a szerződés, nem NUL-végű string.

4. **Tesztek.** A tesztek a szerződés minden esetét külön vizsgálják: üres érték, belső `0`, nem UTF-8 érték, hiányzó kulcs, hibás kulcskódolás, `NULL` argumentumok, törlés (volt / nem volt), valamint a pánik-védelmet. A pánik-policy a *termék* profiljától függ, a tesztcélpontot azonban a Cargo mindig unwindinggel fordítja, ezért a `cargo test --release` önmagában **nem** igazolja a termék pánikvédelmét: csak a `guarded` logikáját ellenőrzi. A termék profilját külön, közvetlenül ellenőrizzük (az eredmény `panic="unwind"` kell legyen): `cargo rustc -p rkv-ffi --release --lib -- --print cfg | grep panic`; ezen felül a C kliens a kész `cdylib`/`staticlib` ellen futó átvételi tesztje a végső bizonyíték. A tesztet debug és release módban is futtatjuk:

Kódblokk — fájl: `rkv-ffi/src/lib.rs` · művelet: beillesztés a fájl végére · ellenőrzés: `cargo test -p rkv-ffi && cargo test --release -p rkv-ffi` → 2–2 sikeres teszt

```
#[cfg(test)]
mod tests {
    use super::*;
    use std::ffi::CString;

    #[test]
    fn guard_turns_a_panic_into_a_status() {
        assert_eq!(guarded(|| panic!("hiba")), RKV_PANIC);
        assert_eq!(guarded(|| RKV_OK), RKV_OK);
    }

    #[test]
    fn all_cases_are_distinct_and_checkable() {
        let store = rkv_store_new();
        let k = CString::new("k").unwrap();
        let mut out: *mut u8 = ptr::null_mut();
        let mut len = 0usize;
        unsafe {
            // hiányzó kulcs
            assert_eq!(rkv_get(store, k.as_ptr(), &mut out, &mut len), RKV_NOT_FOUND);
            assert!(out.is_null() && len == 0);
            // üres érték (value == NULL, len == 0)
            assert_eq!(rkv_set(store, k.as_ptr(), ptr::null(), 0), RKV_OK);
            assert_eq!(rkv_get(store, k.as_ptr(), &mut out, &mut len), RKV_OK);
            assert!(!out.is_null() && len == 0);
            rkv_buffer_free(out, len);
            // belső 0 és nem UTF-8 bájt
            let bin = [0u8, 0xFF, 0, 7];
            assert_eq!(rkv_set(store, k.as_ptr(), bin.as_ptr(), bin.len()), RKV_OK);
            assert_eq!(rkv_get(store, k.as_ptr(), &mut out, &mut len), RKV_OK);
            assert_eq!(std::slice::from_raw_parts(out, len), bin);
```

```
            rkv_buffer_free(out, len);
            // hibás kulcskódolás (nem UTF-8) és null kulcs
            let bad = CString::new(vec![0xFFu8, b'x']).unwrap();
            assert_eq!(rkv_set(store, bad.as_ptr(), bin.as_ptr(), 1), RKV_INVALID_ARG);
            assert_eq!(rkv_set(store, ptr::null(), bin.as_ptr(), 1), RKV_INVALID_ARG);
            // null érték nem nulla hosszal
            assert_eq!(rkv_set(store, k.as_ptr(), ptr::null(), 3), RKV_INVALID_ARG);
            // törlés: létezett / nem létezett
            assert_eq!(rkv_delete(store, k.as_ptr()), RKV_OK);
            assert_eq!(rkv_delete(store, k.as_ptr()), RKV_NOT_FOUND);
            rkv_store_free(store);
        }
    }
}
```

5. **A `cbindgen` konfigurációja.** A `cpp_compat = true` beállítás a deklarációkat `extern "C" { ... }` blokkba teszi, ha a headert C++ fordító olvassa (`#ifdef __cplusplus`), a `documentation = true` pedig a `///` és `# Safety` kommenteket a headerbe másolja.

Kódblokk — fájl: `rkv-ffi/cbindgen.toml` · művelet: új fájl · ellenőrzés: a következő lépés

```
language = "C"
include_guard = "RKV_FFI_H"
cpp_compat = true          # extern "C" { ... } burkolás C++ fordításhoz
documentation = true       # a /// és # Safety kommentek a headerbe kerülnek
style = "type"
usize_is_size_t = true
```

6. **Generáljuk a C headert** (a `cbindgen` egy külön telepített eszköz, a verziót érdemes rögzíteni):

```
cargo install cbindgen
cd ~/rust-labor/rkv/rkv-ffi
cbindgen --config cbindgen.toml --crate rkv-ffi --output ../cpp-client/rkv_ffi.h
```

A generált `rkv_ffi.h` az `RkvStore` átlátszatlan típust előre deklarálja (`typedef struct RkvStore RkvStore;`: a Rust struct nem `repr(C)`, a C oldal csak mutatót lát), a státuszkódokat `#define`-ként adja, és a függvények dokumentációját átveszi. Részlet:

```
#ifndef RKV_FFI_H
#define RKV_FFI_H

#include <stdarg.h>
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include <stdlib.h>

#define RKV_OK 0

#define RKV_NOT_FOUND 1

#define RKV_INVALID_ARG 2

#define RKV_ERROR 3

#define RKV_PANIC 4

/**
 * Átlátszatlan kezelő: a C oldal csak mutatót lát (`typedef struct RkvStore RkvStore;`).
 */
typedef struct RkvStore RkvStore;
```

```
#ifdef __cplusplus
extern "C" {
#endif // __cplusplus
...
```

7. **Fordítsuk le a Rust könyvtárat:**

```
cd ~/rust-labor/rkv
cargo build --release -p rkv-ffi
```

Ez a `target/release/` alatt `librkv_ffi.a` (statikus) és `librkv_ffi.so` (dinamikus; macOS-en `.dylib`) fájlokat készít.

8. **Írjunk C klienst** a `~/rust-labor/rkv/cpp-client/` könyvtárba. A hibaágon is pontosan egyszer szabadítunk (`goto done`), és a `rkv_get` pufferét a kapott hosszal engedjük el:

Kódblokk — fájl: `cpp-client/main.c` · művelet: új fájl · ellenőrzés: a következő lépés

```
#include "rkv_ffi.h"
#include <stdio.h>

int main(void) {
    RkvStore *store = rkv_store_new();
    if (store == NULL) {
        fprintf(stderr, "store létrehozási hiba\n");
        return 1;
    }
    int rc = 1;

    if (rkv_set(store, "alma", (const uint8_t *)"piros", 5) != RKV_OK) {
        fprintf(stderr, "SET hiba\n");
        goto done; /* a hibaágon is pontosan egyszer szabadítunk */
    }

    uint8_t *value = NULL;
    size_t len = 0;
    int status = rkv_get(store, "alma", &value, &len);
    if (status == RKV_OK) {
        /* A puffer tetszőleges bájtokat tartalmazhat: belső 0, nem UTF-8.
           A printf("%.*s") csak NUL-mentes szövegre jó, ezért itt a
           hosszal írunk: a tartalom és a hossz is megmarad. */
        fwrite("VALUE ", 1, 6, stdout);
        fwrite(value, 1, len, stdout);
        fputc('\n', stdout);
        rkv_buffer_free(value, len); /* pontosan egyszer, a kapott hosszal */
    } else if (status == RKV_NOT_FOUND) {
        printf("NOTFOUND\n");
    } else {
        fprintf(stderr, "GET hiba: %d\n", status);
        goto done;
    }

    for (int i = 0; i < 2; i++) { /* az első törlés OK, a második NOTFOUND */
        int del = rkv_delete(store, "alma");
        if (del == RKV_OK) {
            printf("DEL OK\n");
        } else if (del == RKV_NOT_FOUND) {
            printf("DEL NOTFOUND\n");
        } else { /* RKV_ERROR, RKV_INVALID_ARG, RKV_PANIC: nem "nem találtam" */
            fprintf(stderr, "DEL hiba: %d\n", del);
            goto done;
        }
```

```
    }
    rc = 0;
done:
    rkv_store_free(store);
    return rc;
}
```

9. **Fordítsuk és linkeljük a C klienst.** A linker a parancssorban *balról jobbra* old fel: a **forrás/object fájl előzze meg a `-lrkv_ffi`-t**, különben a szimbólumok feloldatlanok maradhatnak. Statikus linkelésnél a Rust standard könyvtár rendszerfüggőségeit is meg kell adni (Linux: `-lpthread -ldl -lm`; a pontos listát a `cargo rustc -p rkv-ffi --release -- --print native-static-libs` mutatja). Dinamikus linkelésnél a futásidejű betöltő keresési útja platformfüggő: Linuxon a `-Wl,-rpath,<könyvtár>` (vagy futáskor az `LD_LIBRARY_PATH`), macOS-en a `DYLD_LIBRARY_PATH` / `-Wl,-rpath` az `@rpath` install name-mel.

```
cd ~/rust-labor/rkv/cpp-client
# statikus
gcc -Wall -Wextra -o client_static main.c ../target/release/librkv_ffi.a -lpthread -ldl -lm
./client_static
# dinamikus (Linux): a forrás előzze meg a -l kapcsolót
gcc -Wall -Wextra -o client_dyn main.c -L../target/release -lrkv_ffi
 ↪  -Wl,-rpath,"$PWD/../target/release"
./client_dyn
```

Várt kimenet:

```
VALUE piros
DEL OK
DEL NOTFOUND
```

Ez a kimenet a szöveges `piros` értékre vonatkozik. Belső `0`-t vagy `0xFF`-et tartalmazó értéknél a `fwrite` a nyers bájtokat írja ki; az ellenőrzéshez a hosszt és a bájtokat kell összevetni (például `xxd`-vel), nem a szöveges megjelenítést. A `printf("%.*s")` erre nem alkalmas: a belső `0`-nál megállna, és a hossz sem ellenőrizhető belőle.

10. **C++ kliens.** A header `cpp_compat` beállítása miatt ugyanaz a fájl C++-ként is fordítható (a deklarációk `extern "C"` blokkban vannak, a C++ nem manglingeli őket). A kezelőt `std::unique_ptr` deleterrel RAII-ra bízzuk, így a `free` pontosan egyszer fut le:

Kódblokk — fájl: `cpp-client/main.cpp` · művelet: új fájl · ellenőrzés: `g++ -std=c++17 -Wall -Wextra -o client_cpp main.cpp ../target/release/librkv_ffi.a -lpthread -ldl -lm && ./client_cpp` → `len=3`

```
#include "rkv_ffi.h"
#include <iostream>
#include <memory>

int main() {
    // RAII: a kezelőt a deleter pontosan egyszer szabadítja fel.
    std::unique_ptr<RkvStore, decltype(&rkv_store_free)> store(rkv_store_new(),
     ↪  &rkv_store_free);
    const uint8_t v[] = {'k', 0, 'v'}; // belső 0 bájt is átmegy
    if (rkv_set(store.get(), "kulcs", v, sizeof v) != RKV_OK) return 1;
    uint8_t *out = nullptr;
    size_t len = 0;
    if (rkv_get(store.get(), "kulcs", &out, &len) != RKV_OK) return 1;
    std::cout << "len=" << len << '\n';
    rkv_buffer_free(out, len);
```

```
    return 0;
}
```

A kimenet `len=3`: a belső `0` bájtot tartalmazó érték sértetlenül átmegy a határon.

Ellenőrzési pont. Az `rkv-ffi` crate (`rkv_store_new`, `rkv_store_free`, `rkv_set`, `rkv_get`, `rkv_delete`, `rkv_buffer_free`) a `SharedStore`-ra épül; a `cargo test` debug és release módban sikeres, a generált header C-ként és C++-ként is fordul, a C és a C++ kliens statikusan és dinamikusan is linkelődik és fut.

19.1.8 Tipikus hibák és átállási csapdák

1. **Rust `String` átadása C-nek.** A `String` belső elrendezése nem C-kompatibilis; közvetlenül soha ne adjuk át. Szöveghez `CString` + `as_ptr`/`into_raw`, bináris adathoz mutató + hossz.
2. **A hívó felszabadításának elmulasztása.** A `rkv_get` által adott puffert a C hívónak kell a `rkv_buffer_free(ptr, len)`-nel felszabadítania. Ha elmulasztja, memóriaszivárgás; ha kétszer hívja, double free; ha rossz hosszat ad, a Rust oldali rekonstrukció hibás.
3. **Panik átengedése a határon.** `catch_unwind` nélkül a pánik az `extern "C"` határán leállítja a folyamatot (Rust 1.81 óta abort). A `guarded` segéd `RKV_PANIC`-ot ad, de csak `panic = "unwind"` profilban; `panic = "abort"` mellett a leállás a vállalt viselkedés.
4. **Rossz felszabadító.** A `rkv_get` eredményére a C hívó nem hívhat `free()`-t, csak `rkv_buffer_free()`-t (a Rust allokátorával foglalt memóriát csak a Rust szabadíthatja). Fordítva ugyanígy: C `malloc`-kal foglalt mutatót nem szabad `Box::from_raw`/`CString::from_raw` útján eldobni.
5. **Nem `#[repr(C)]` struktúra átadása C-nek.** Az `rkv-ffi`-ben az `RkvStore` átlátszatlan típus, ezért nem kell `#[repr(C)]`, de ha a C kód hozzáfér a mezőkhöz, akkor kötelező, és a mezők típusának is C-kompatibilisnek kell lennie.
6. **Linkelési sorrend.** `gcc -lrkv_ffi main.c` (a könyvtár a forrás előtt) feloldatlan szimbólumot adhat; a forrás/object mindig előzze meg a `-l` kapcsolót.
7. **A `free` és egy másik hívás versenye.** A `rkv_store_free` kizárólagos: ha egy másik szál közben még használja a kezelőt, use-after-free keletkezik. A C hívónak kell biztosítania, hogy a `free` az összes többi hívás befejezése után fut.

19.1.9 Feladatok

1. **Fogalmi kérdés:** Miért nem lehet a Rust `&str`-et közvetlenül C függvénynek átadni? Milyen típust használunk helyette, és miért? Mi a különbség az ABI-kompatibilitás és az értékérvényességi szerződés között?
2. **C++ kód átírása:** Írj C++ függvényt, amely `std::string`-et fogad, és egy C könyvtárnak adja át `char*`-ként, a könyvtár pedig a pufferrel a saját `free()`-jével szabadít. Írd meg a Rust megfelelőjét `unsafe extern "C"` deklarációval, és indokold, miért nem jó itt a `CString::into_raw`.
3. **Implementációs feladat:** Egészítsd ki az `rkv-ffi` crate-et egy `rkv_exists(store, key)` függvénnyel, amely `RKV_OK`-t ad, ha a kulcs létezik, `RKV_NOT_FOUND`-ot, ha nem. Generáld újra a headert, és teszteld a Rust egységtesztben.
4. **Tervezési döntés:** Tervezze meg egy `rkv_list` függvény szerződését, amely sok kulcsot ad vissza. Milyen kimeneti paraméterek kellenek, ki foglal és ki szabadít, hogyan kezelendő az üres lista és a bináris/nem UTF-8 kulcs?

19.1.10 Megoldások

1. A `&str` nem null-terminált, és fat pointer (adatmutató + hossz). A C egy `char*`-ot vár, null-terminált bájtsorozatot. A `CString` biztosítja a null-terminált, birtokolt puffert; a `&CStr` a kölcsönzött nézetet. Az **ABI-kompatibilitás** azt jelenti, hogy a bájtok elrendezése és a hívási konvenció egyezik (például `*const c_char`); az **értékérvényességi szerződés** azt, hogy a mutató mögötti bájtok jelentése is rögzített (NUL-végű? UTF-8? ki szabadítja? meddig él?). A mutató típusa az elsőt garantálja, a másodikat dokumentálni és ellenőrizni kell.
2. C++ oldal: `std::string s = "alma"; c_library_set_takes_ownership(strdup(s.c_str()));` — itt a hívó ad át tulajdonjogot. Rust oldal:

```rust
use std::ffi::{CString, c_char};

unsafe extern "C" {
    fn c_library_set_takes_ownership(key: *mut c_char);
}

fn set_value(key: &str) -> Result<(), std::ffi::NulError> {
    let key = CString::new(key)?;
    // A C könyvtár a kapott puffert később free()-vel engedi el, ezért azt a
    // C allokátorral kell lefoglalni; a CString::into_raw puffere ide nem jó.
    let bytes = key.as_bytes_with_nul();
    // SAFETY: a malloc-ot a C allokátor szolgálja ki; null esetén nem írunk.
    let buf = unsafe { libc::malloc(bytes.len()) } as *mut c_char;
    if buf.is_null() {
        std::alloc::handle_alloc_error(std::alloc::Layout::array::<u8>(bytes.len()).unwrap());
    }
    // SAFETY: buf legalább bytes.len() bájtos, friss, a forrással nem átfedő puffer.
    unsafe { std::ptr::copy_nonoverlapping(bytes.as_ptr().cast::<c_char>(), buf, bytes.len()) };
    // SAFETY: a malloc-kal foglalt puffer tulajdonjogát átadjuk; a C könyvtár a saját
    ↪  free()-jével szabadítja fel, pontosan egyszer.
    unsafe { c_library_set_takes_ownership(buf) };
    Ok(())
}
```

A `libc::malloc` a `libc` crate-et igényli (`libc = "0.2"`). A `CString::into_raw` pointerét **nem** szabad a C `free`-nek átadni: az a Rust allokátorával készült, ezért csak a `CString::from_raw` veheti vissza. Ha a C API-t mi tervezzük, jobb megoldás, ha a könyvtár egy felszabadító callbacket vagy egy saját `free` függvényt is kap, vagy ha a hívás csak kölcsönveszi a stringet (`const char*`), és saját másolatot készít.

3. A `rkv_exists` implementációja (az `rkv-ffi/src/lib.rs`-be):

Kódblokk — fájl: `rkv-ffi/src/lib.rs` · művelet: beillesztés a `rkv_buffer_free` elé · ellenőrzés: `cargo test -p rkv-ffi`

```rust
/// Létezik-e a kulcs? `RKV_OK`, ha igen; `RKV_NOT_FOUND`, ha nem.
///
/// # Safety
/// `store` érvényes kezelő; `key` NUL-végű C string.
#[unsafe(no_mangle)]
pub unsafe extern "C" fn rkv_exists(store: *const RkvStore, key: *const c_char) -> c_int {
    guarded(|| {
        // SAFETY: lásd a függvény # Safety szakaszát.
        let (Some(store), Some(key)) = (unsafe { handle(store) }, unsafe { key_str(key) }) else
        ↪  {
            return RKV_INVALID_ARG;
        };
        match store.inner.get(key) {
```

```
            Ok(Some(_)) => RKV_OK,
            Ok(None) => RKV_NOT_FOUND,
            Err(_) => RKV_ERROR,
        }
    })
}
```

A tesztbe a törlés előtt `assert_eq!(rkv_exists(store, k.as_ptr()), RKV_OK);`, a törlés után `RKV_NOT_FOUND` ellenőrzés kerül. A headert újra kell generálni, és a C kliensben `rkv_exists(store, "alma") == RKV_OK` formában hívható.

4. A `rkv_list` szerződése. A Rust foglalja a kimenetet, és egyetlen felszabadító függvény engedi el; a kulcsok mutató + hossz párok (nem NUL-végű), így a belső `0` és a nem UTF-8 bájt sem csonkít. Egy lehetséges alak: a `rkv_list(store, prefix, &mut out_items, &mut out_count)` egy `RkvEntry { const uint8_t *key; size_t key_len; ... }` tömböt (`repr(C)` struct, kizárólag C-kompatibilis mezőkkel) és annak elemszámát adja; az üres lista `RKV_OK`, `count == 0`, a mutató `NULL`. A hívó a `rkv_list_free(items, count)`-tal szabadítja fel az egészet (a tömböt és a benne lévő kulcspuffereket együtt, hogy ne kelljen mindegyiket külön követni). A szerződésnek ki kell mondania a szálbiztonságot (a lista pillanatkép, a tároló közben módosulhat) és hogy a `free` pontosan egyszer hívható.

19.1.11 Rövid összefoglaló

A C ABI a Rust és C/C++ együttműködés alapja. A Rust `unsafe extern "C"` blokkokkal hív C függvényeket (a C `int` megfelelője a `c_int`), amelyeket a `bindgen` generálhat C fejlécekből (az `OUT_DIR`-be írva, `include!`-dal beillesztve); a `#[unsafe(no_mangle)]` (2024 editionben) és a `pub extern "C" fn` exportálja a Rust függvényeket C számára. A határon két dolgot választunk szét: az ABI-kompatibilitást (`repr(C)`, C-kompatibilis típusok) és az értékérvényességi/ownership szerződést (NUL-végű? ki szabadít? szálbiztos?). A pánik-policy: a pánik az `extern "C"` határán nem terjedhet tovább (Rust 1.81 óta abort); az `rkv-ffi` `panic = "unwind"` profillal készül, és a `guarded` segéd `RKV_PANIC`-ká alakítja. Az `rkv-ffi` kezelője átlátszatlan `RkvStore`, az érték mutató + hossz, a státusz külön kód (`RKV_OK`, `RKV_NOT_FOUND`, `RKV_INVALID_ARG`, `RKV_ERROR`, `RKV_PANIC`), a kimeneti puffert a `rkv_buffer_free` szabadítja. A `cbindgen` `cpp_compat` beállítással C és C++ számára is használható headert ad. **Továbbvitt eredmény:** ez a C API (`rkv-ffi`, `rkv_ffi.h`) az alapja a C++-együttműködés a cxx crate-tel és a fokozatos migráció és a Záróprojekt: az rkv teljes rendszerként alfejezeteknek.

19.1.12 Források és ellenőrzési hiányok

- A Rust Reference „External blocks” és „Functions / ABI”, a Rustonomicon „FFI” fejezete; a `std::ffi::CStr/CString`, `Box::into_raw/from_raw`, `catch_unwind` dokumentációja. A pánik `extern "C"` határán való abortja a Rust 1.81 kiadási jegyzetében szerepel.
- A `bindgen` (0.7x+) és a `cbindgen` (0.29) eszközök: https://rust-lang.github.io/rust-bindgen/ és https://github.com/mozilla/cbindgen. A kódot és a parancsokat Rust 1.99.1-gyel, GCC 13 és libclang 18 környezetben ellenőriztük (Linux).
- A `#[unsafe(no_mangle)]` és az `unsafe extern "C"` szintaxis a 2024-es editionhez tartozik (Rust 1.82-től stabil).
- A macOS- és Windows-specifikus betöltési utak (`DYLD_LIBRARY_PATH`, `@rpath`, DLL-keresés) nem lettek kipróbálva; a labor Linux-only.

19.2 C++-együttműködés a cxx crate-tel és a fokozatos migráció

19.2.1 Motiváció: a C ABI alacsony szintű, a cxx biztonságos hidat épít

Az előző alfejezetben (C ABI: extern “C”, repr(C), bindgen és cbindgen) láttuk, hogy a Rust és C/C++ közötti átjárás a C ABI-n keresztül kézzel írt, `unsafe` blokkokkal tűzdelt, hibalehetőségekkel terhelt kódot jelent. A mutatók átadása, a stringek és vektorok kézi konvertálása, a hibakezelés `errno` vagy saját hibakódok formájában mind fokozott figyelmet igényel. Ráadásul a C++ osztályok, kivételek, `std::string` és `std::unique_ptr` közvetlenül nem férnek hozzá a Rust oldalról.

A `cxx` crate ezt a problémát oldja meg azzal, hogy **biztonságos, típusos C++-hidat** generál. A `#[cxx::bridge]` modulban deklaráljuk a C++ és Rust közötti interfészt, és a `cxx-build` eszköz legenerálja mindkét oldal wrapper kódját. Ennek köszönhetően a gyakori típusok (`std::string`, `std::vector`, `std::unique_ptr`, Rust `String`, `Vec`, `Box`) automatikusan konvertálódnak, a C++ kivételek Rust `Result`-tá alakulnak, a Rust `Result` pedig C++ kivétellé. A híd használata a támogatott típusok körében **nem igényel `unsafe`-et**.

Továbbvitt eredmény. Az itt elkészített C++-integrációt a Záróprojekt: az rkv teljes rendszerként alfejezet használja fel, ahol a C++-kliens a Rust tárolóval együttműködve mutatja be a fokozatos migráció eredményét.

19.2.2 Fogalmak és szabályok

> **Definíció.** A `#[cxx::bridge]` egy olyan modul, amely a Rust és C++ közötti interfészt írja le. A modul belsejében `extern "Rust"` blokkban a Rust oldalon implementált, C++-ból hívható függvényeket és típusokat deklaráljuk; `extern "C++"` blokkban pedig a C++ oldalon definiált, Rustból hívható elemeket. A `cxx-build` crate a bridge modul alapján generálja a szükséges C++ és Rust wrapper kódot (a `lib.rs.h` fejlécet és a `lib.rs.cc` forrást).

Szabály 1: támogatott típusok. A bridge-ben a következő típusok szerepelhetnek:

Rust oldal	C++ oldal	Megjegyzés
`&str`	`rust::Str`	nem tulajdonló, UTF-8 nézet
`String`	`rust::String`	tulajdonló
`&[T]`	`rust::Slice<const T>`	nem tulajdonló nézet
`Vec<T>`	`rust::Vec<T>`	tulajdonló
`Box<T>`	`rust::Box<T>`	tulajdonló, átlátszatlan Rust típushoz
`UniquePtr<T>`	`std::unique_ptr<T>`	C++ típus Rust oldali tulajdonosa
`&CxxString`	`const std::string &`	C++ `std::string` Rust oldali nézete
saját `struct`	ugyanaz a mezőkkel generált `struct`	**megosztott struct**, a bridge-ben adjuk meg
`type T;` az `extern "Rust"` blokkban	`rust::Box<T>`-ként kezelhető	**átlátszatlan Rust típus**
`type T;` az `extern "C++"` blokkban	`T` (osztály)	**átlátszatlan C++ típus**, csak referencián vagy `UniquePtr`-en át érhető el

A saját struct tehát nem tiltott: ha mezői is támogatott típusok, a bridge-ben megosztott structként deklaráljuk, és a `cxx` mindkét oldalon legenerálja. A nem támogatott típusnál a `cxx-build` fordítási hibát ad (`unsupported type`).

Szabály 2: `Result` a bridge-ben. A bridge-ben a hibatípust **nem** írjuk ki: a deklaráció `Result<T>`, nem `Result<T, E>`.

- *Rust függvény a hídon.* A bridge-ben `fn get(…) -> Result<Vec<u8>>;` szerepel, a Rust implementáció viszont tetszőleges `Result<Vec<u8>, E>`-t adhat vissza, **ha `E: Display`** (a gyakorlatban `std::error::Error`-t is implementáló saját hibatípus). C++ oldalon a függvény `rust::Vec<std::uint8_t>`-t ad vissza, hiba esetén pedig `rust::Error` kivételt dob, amelynek `what()` szövege az `E` `Display` kimenete.
- *C++ függvény a hídon.* A bridge-ben `fn may_throw(fail: bool) -> Result<()>;` szerepel, a C++ implementáció pedig `void`-ot ad vissza, és hibát kivétellel jelez. Rust oldalon ez `Result<(), cxx::Exception>`, a kivétel szövege az `Exception::what()` metódussal olvasható.

A `Result<T, String>` forma tehát fordítási hiba, és a `String` hibatípus sem szükséges: a `Display`-t implementáló saját hiba közvetlenül használható.

Szabály 3: namespace. A `#[cxx::bridge(namespace = "rkv")]` a generált C++ elemeket (és a C++ oldalon deklarált típusokat) az `rkv` névtérbe teszi. A C++ fejlécben és implementációban ugyanabba a névtérbe kell írni a `Logger`-t és a függvényeket.

19.2.3 C++ és Rust összevetés

A cxx előnye a kézzel írt C ABI-val szemben szembetűnő. Hasonlítsuk össze egy értéket visszaadó, hibázhat függvény két megvalósítását.

C ABI (C ABI: extern “C”, repr(C), bindgen és cbindgen):

```
// C++ oldal
extern "C" {
    int rkv_get(RkvStore *store, const char *key, uint8_t **out, size_t *len);
    void rkv_buffer_free(uint8_t *ptr, size_t len);
}
```

A hívónak kezelnie kell az `int` státuszkódot (a `RKV_*` makrókkal), a kimeneti mutatót és a felszabadítást, a Rust oldalon pedig `unsafe` blokk és `catch_unwind` szükséges. Az `RkvStatus` típus nem része a generált 19.1-es headernek; a visszatérési típus minden státuszkódos függvénynél `int`.

cxx-szel:

Kódblokk — fájl: `src/lib.rs` · művelet: részlet (a teljes fájl a labor 2. lépésében) · ellenőrzés: `cargo build -p rkv-cxx`

```
#[cxx::bridge(namespace = "rkv")]
mod ffi {
    extern "Rust" {
        type Db;
        fn get(self: &Db, key: &str) -> Result<Vec<u8>>;
    }
}

// C++ oldal
rust::Vec<std::uint8_t> value = db->get("key"); // hiba esetén rust::Error kivétel
```

A különbség: a cxx generálja a konverziós és memóriakezelési kódot, a típusok ellenőrzöttek, és a hívó nem férhet rossz mutatóhoz. A Rust oldalon nincs `unsafe`.

19.2.4 Kidolgozott példa: C++ osztály használata Rustból

A cxx nemcsak C++-ból Rust hívását teszi lehetővé, hanem fordítva is. Tegyük fel, hogy van egy meglévő C++ `Logger` osztályunk. A generált `rust::Str` argumentumot a Rust `&str` képezi le, a `log` metódus pedig `const`, mert a Rust oldalon `&Logger` receiverként jelenik meg.

Kódblokk — fájl: `rkv-cxx/include/logger.h` · művelet: új fájl · ellenőrzés: a labor 5. lépésében a `cargo build` lefordítja

```
#pragma once
#include <memory>
#include <string>
#include "rust/cxx.h"

namespace rkv {

class Logger {
public:
    explicit Logger(std::string prefix) : prefix_(std::move(prefix)) {}
    // A cxx a Rust &str-t rust::Str-ként adja át.
    void log(rust::Str message) const;

private:
    std::string prefix_;
};

// Gyártófüggvény: a cxx nem tud C++ konstruktort hívni, ezért std::unique_ptr-t adunk vissza.
std::unique_ptr<Logger> new_logger(rust::Str prefix);

// C++ függvény, amely kivételt dobhat: Rust oldalon Result<(), cxx::Exception>.
void may_throw(bool fail);

} // namespace rkv
```

Kódblokk — fájl: `rkv-cxx/src/logger.cc` · művelet: új fájl · ellenőrzés: a labor 5. lépésében a `cargo build` lefordítja

```
#include "rkv-cxx/include/logger.h"
#include <iostream>
#include <stdexcept>

namespace rkv {

void Logger::log(rust::Str message) const {
    std::cerr << prefix_ << ": " << std::string(message) << "\n";
}

std::unique_ptr<Logger> new_logger(rust::Str prefix) {
    return std::make_unique<Logger>(std::string(prefix));
}

void may_throw(bool fail) {
    if (fail) throw std::runtime_error("C++ hiba");
}

} // namespace rkv
```

A bridge-ben az `unsafe extern "C++"` blokk jelzi, hogy a deklarált C++ elemek tényleges létezését és az aláírások egyezését a programozó garantálja; a hívás maga biztonságos. A `Logger` átlátszatlan C++ típus: Rustból csak `&Logger` referencián vagy `UniquePtr<Logger>`-en át érhető el.

Kódblokk — fájl: `rkv-cxx/src/lib.rs` · művelet: részlet (a bridge `extern "C++"` blokkja) · ellenőrzés: `cargo test -p rkv-cxx` (4 teszt zöld)

```
unsafe extern "C++" {
    include!("rkv-cxx/include/logger.h");

    type Logger;
    fn log(self: &Logger, message: &str);
    fn new_logger(prefix: &str) -> UniquePtr<Logger>;
    /// A C++ kivétel Rust oldalon `Err(cxx::Exception)`.
    fn may_throw(fail: bool) -> Result<()>;
}
```

Rust oldalról a használat:

```
let logger = ffi::new_logger("rkv"); // UniquePtr<Logger>
logger.log("Hello from Rust");       // UniquePtr<Logger> deref-el &Logger-ré
```

19.2.5 Labor: C++ CLI, amely az rkv-t használja cxx-szel

Munkakönyvtár: `~/rust-labor/rkv/`

Ebben a laborban létrehozunk egy `rkv-cxx` crate-et, amely cxx-híddal teszi elérhetővé az rkv tároló alapműveleteit C++-ból. A C++ oldalon egy parancssori eszközt (`cpp-client/cxx-cli`) készítünk. Mivel minden CLI-hívás **külön folyamat**, a memóriatároló állapota hívások között elveszne; ezért a híd a fájl-alapú `FileStore`-t nyitja meg egy megadott adatkönyvtárban, és az állapot a folyamatok között megmarad.

19.2.5.1 1. lépés: az `rkv-cxx` crate létrehozása

Kódblokk — fájl: terminál (workspace gyökér) · művelet: új crate · ellenőrzés: `ls rkv-cxx/Cargo.toml`

```
cargo new --lib rkv-cxx
```

Kódblokk — fájl: `rkv-cxx/Cargo.toml` · művelet: teljes csere; a workspace `members` listájába vegyük fel az `"rkv-cxx"`-et is · ellenőrzés: `cargo metadata --no-deps`

```
[package]
name = "rkv-cxx"
version = "0.1.0"
edition = "2024"
rust-version = "1.89"

[lib]
crate-type = ["staticlib", "rlib"]

[dependencies]
cxx = "1"
rkv-core = { path = "../rkv-core" }

[build-dependencies]
cxx-build = "1"
```

```
[dev-dependencies]
tempfile = "3"
```

A `staticlib` hozza létre a C++ kliens által linkelhető `librkv_cxx.a` archívumot; az `rlib` kell a Rust-oldali `cargo test`-hez.

19.2.5.2 2. lépés: a bridge modul és a Rust implementáció

A híd az rkv-core valódi API-jára épül: a `SharedStore` a `Storage` trait mögé rejtett tárolót zár `Mutex`-be, a `FileStore::open(snapshot, napló)` pedig a tartós tárolót nyitja meg. A hiányzó kulcs a core-ban `Option::None`; a hídon ezt explicit `NotFound` hibává alakítjuk, hogy a C++ kód kivételt kapjon, ne üres értéket.

Kódblokk — fájl: `rkv-cxx/src/lib.rs` · művelet: teljes csere · ellenőrzés: `cargo test -p rkv-cxx` → `4 passed`

```
use std::fmt;
use std::path::Path;

use rkv_core::{FileStore, MemStore, SharedStore};

// A Logger-példa függvényeit a lib csak a tesztekben hívja.
#[allow(dead_code)]
#[cxx::bridge(namespace = "rkv")]
mod ffi {
    /// Megosztott struct: mindkét oldalon ugyanaz a mezőkkel megadott típus.
    struct Entry {
        key: String,
        value: Vec<u8>,
    }

    extern "Rust" {
        /// Átlátszatlan Rust típus: C++-ban `rust::Box<rkv::Db>`.
        type Db;

        fn open_mem() -> Box<Db>;
        /// Fájl-alapú tároló: az állapot a folyamatok között megmarad.
        fn open_file(dir: &str) -> Result<Box<Db>>;

        fn set(self: &Db, key: &str, value: &[u8]) -> Result<()>;
        /// Hiányzó kulcsra hiba (`NotFound`), nem üres érték.
        fn get(self: &Db, key: &str) -> Result<Vec<u8>>;
        /// Hiányzó kulcsra hiba (`NotFound`).
        fn del(self: &Db, key: &str) -> Result<()>;
        fn list(self: &Db, prefix: &str) -> Result<Vec<Entry>>;
        fn count(self: &Db, prefix: &str) -> Result<usize>;
        /// Rust függvény, amely C++ objektumot használ.
        fn set_logged(self: &Db, logger: &Logger, key: &str, value: &[u8]) -> Result<()>;
    }

    unsafe extern "C++" {
        include!("rkv-cxx/include/logger.h");

        type Logger;
        fn log(self: &Logger, message: &str);
        fn new_logger(prefix: &str) -> UniquePtr<Logger>;
        /// A C++ kivétel Rust oldalon `Err(cxx::Exception)`.
        fn may_throw(fail: bool) -> Result<()>;
    }
}
```

```rust
/// Saját hibatípus: `Display`-t implementál, ezért a cxx `rust::Error` kivételt dob belőle.
#[derive(Debug)]
pub enum DbError {
    NotFound(String),
    Store(rkv_core::Error),
}

impl fmt::Display for DbError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            DbError::NotFound(key) => write!(f, "a kulcs nem található: {key}"),
            DbError::Store(e) => write!(f, "tárolóhiba: {e}"),
        }
    }
}

impl std::error::Error for DbError {
    fn source(&self) -> Option<&(dyn std::error::Error + 'static)> {
        match self {
            // A hiányzó kulcs nem hibaok, nincs mögötte alacsonyabb szintű hiba.
            DbError::NotFound(_) => None,
            // A tárolóhiba lánca az rkv-core eredeti hibájáig vezet.
            DbError::Store(e) => Some(e),
        }
    }
}

impl From<rkv_core::Error> for DbError {
    fn from(e: rkv_core::Error) -> Self {
        DbError::Store(e)
    }
}

pub struct Db {
    store: SharedStore,
}

fn open_mem() -> Box<Db> {
    Box::new(Db {
        store: SharedStore::new(MemStore::new()),
    })
}

fn open_file(dir: &str) -> Result<Box<Db>, DbError> {
    let dir = Path::new(dir);
    std::fs::create_dir_all(dir).map_err(rkv_core::Error::from)?;
    let store = FileStore::open(dir.join("state.json"), dir.join("append.log"))?;
    Ok(Box::new(Db {
        store: SharedStore::new(store),
    }))
}

impl Db {
    fn set(&self, key: &str, value: &[u8]) -> Result<(), DbError> {
        Ok(self.store.set(key.to_owned(), value.to_vec())?)
    }

    fn get(&self, key: &str) -> Result<Vec<u8>, DbError> {
        self.store.get(key)?.ok_or_else(|| DbError::NotFound(key.to_owned()))
    }

    fn del(&self, key: &str) -> Result<(), DbError> {
```

```
        if self.store.delete(key)? {
            Ok(())
        } else {
            Err(DbError::NotFound(key.to_owned()))
        }
    }

    fn list(&self, prefix: &str) -> Result<Vec<ffi::Entry>, DbError> {
        let items = self.store.list(Some(prefix).filter(|p| !p.is_empty()))?;
        Ok(items
            .into_iter()
            .map(|(key, value)| ffi::Entry { key, value })
            .collect())
    }

    fn count(&self, prefix: &str) -> Result<usize, DbError> {
        Ok(self.list(prefix)?.len())
    }

    fn set_logged(&self, logger: &ffi::Logger, key: &str, value: &[u8]) -> Result<(), DbError> {
        logger.log(&format!("SET {key} ({} bájt)", value.len()));
        self.set(key, value)
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn contract_holds_on_the_rust_side() {
        let db = open_mem();
        db.set("a", b"1").unwrap();
        db.set("b", b"").unwrap(); // üres érték
        assert_eq!(db.get("a").unwrap(), b"1");
        assert_eq!(db.get("b").unwrap(), b"");
        assert!(matches!(db.get("nincs"), Err(DbError::NotFound(_))));
        assert_eq!(db.count("").unwrap(), 2);
        assert_eq!(db.count("a").unwrap(), 1);
        db.del("a").unwrap();
        assert!(matches!(db.del("a"), Err(DbError::NotFound(_))));
        assert_eq!(db.list("").unwrap().len(), 1);
    }

    #[test]
    fn file_store_persists_between_opens() {
        let dir = tempfile::tempdir().unwrap();
        let path = dir.path().to_str().unwrap();
        open_file(path).unwrap().set("k", b"v").unwrap();
        assert_eq!(open_file(path).unwrap().get("k").unwrap(), b"v");
    }

    #[test]
    fn a_cpp_exception_becomes_a_rust_err() {
        assert!(ffi::may_throw(false).is_ok());
        let err = ffi::may_throw(true).unwrap_err();
        assert!(err.what().contains("C++ hiba")); // cxx::Exception
    }

    #[test]
    fn rust_calls_a_cpp_class() {
        let logger = ffi::new_logger("rkv");
        let db = open_mem();
```

```
        db.set_logged(&logger, "k", b"vv").unwrap();
        assert_eq!(db.get("k").unwrap(), b"vv");
    }
}
```

Néhány döntés, amelyet érdemes tudatosítani:

- A híd **nem globális tárolót** használ: a `Db` a `SharedStore`-t birtokolja, a C++ oldal pedig `rust::Box<rkv::Db>`-t tart. Így nincs `static`, és a tárolók függetlenek.
- A `list` üres prefixre minden bejegyzést visszaad (`None`), a `count` ennek hosszát számolja, így nincs szükség külön, a core-ban nem létező `len` API-ra.
- A `Db` metódusai `&self`-et vesznek, mert a belső `Mutex` adja a változtathatóságot; ez felel meg a bridge `self: &Db` receiverének.

19.2.5.3 3. lépés: a build.rs

Kódblokk — fájl: `rkv-cxx/build.rs` · művelet: új fájl · ellenőrzés: `cargo build -p rkv-cxx --release`

```
fn main() {
    cxx_build::bridge("src/lib.rs")
        .file("src/logger.cc")
        .std("c++17")
        .compile("rkv-cxx-glue");
    println!("cargo:rerun-if-changed=src/lib.rs");
    println!("cargo:rerun-if-changed=src/logger.cc");
    println!("cargo:rerun-if-changed=include/logger.h");
}
```

A `cxx_build::bridge("src/lib.rs")` **beolvassa** a Rust forrást, és abból generálja a C++ oldali wrappert; a Rust fájlt tehát soha nem adjuk `.file(…)` hívással a C++ fordítónak. A `.file(…)` kizárólag valódi C++ implementációkat (itt a `logger.cc`-t) kap.

19.2.5.4 4. lépés: a C++ CLI

Kódblokk — fájl: `rkv-cxx/../cpp-client/cxx_cli.cpp` (azaz `cpp-client/cxx_cli.cpp` a workspace gyökerében) · művelet: új fájl (a 19.1-es C ABI-s `cpp-client/main.cpp`-től elkülönítve, hogy mindkét kliens megmaradjon) · ellenőrzés: az 5. lépés futtatása

```
#include "rkv-cxx/src/lib.rs.h"
#include <cstdint>
#include <iostream>
#include <string>

static void usage() {
    std::cerr << "Használat: cxx-cli <adatkönyvtár> "
                 "<SET kulcs érték | GET kulcs | DEL kulcs | COUNT [prefix] | LIST [prefix]>\n";
}

int main(int argc, char** argv) {
    if (argc < 3) {
        usage();
        return 2;
    }
    const std::string cmd = argv[2];
    try {
        // Fájl-alapú tároló: a külön folyamatokban kiadott parancsok ugyanazt az állapotot
        ↪ látják.
```

```
        rust::Box<rkv::Db> db = rkv::open_file(argv[1]);
        if (cmd == "SET" && argc == 5) {
            const std::string value = argv[4];
            const auto* bytes = reinterpret_cast<const std::uint8_t*>(value.data());
            db->set(argv[3], rust::Slice<const std::uint8_t>(bytes, value.size()));
            std::cout << "OK\n";
        } else if (cmd == "GET" && argc == 4) {
            rust::Vec<std::uint8_t> value = db->get(argv[3]);
            std::cout << "VALUE " << std::string(value.begin(), value.end()) << "\n";
        } else if (cmd == "DEL" && argc == 4) {
            db->del(argv[3]);
            std::cout << "OK\n";
        } else if (cmd == "COUNT" && (argc == 3 || argc == 4)) {
            std::cout << db->count(argc == 4 ? argv[3] : "") << "\n";
        } else if (cmd == "LIST" && (argc == 3 || argc == 4)) {
            for (const rkv::Entry& e : db->list(argc == 4 ? argv[3] : "")) {
                std::cout << std::string(e.key) << "\n";
            }
        } else {
            usage();
            return 2;
        }
    } catch (const rust::Error& e) {
        std::cerr << "ERR " << e.what() << "\n";
        return 3;
    }
    return 0;
}
```

A `rust::Str` paraméterek `const char*`-ból és `std::string`-ből implicit módon épülnek, a `rust::Slice<const std::uint8_t>` bájtnézet kézzel készül. A `rust::Error` kivétel a Rust `Err` esetén keletkezik; ezt a `main` elkapja, így a program nem terminál kezeletlen kivétellel.

19.2.5.5 5. lépés: build és futtatás

Kódblokk — fájl: terminál (workspace gyökér) · művelet: a híd és a statikus könyvtár építése, majd a C++ CLI fordítása · ellenőrzés: a `librkv_cxx.a` és a `cxx-cli` létrejön

```
cargo build -p rkv-cxx --release
ls target/release/librkv_cxx.a target/cxxbridge/rkv-cxx/src/lib.rs.h
g++ -std=c++17 -Wall -Wextra -I target/cxxbridge -I . \
    cpp-client/cxx_cli.cpp target/release/librkv_cxx.a \
    -lpthread -ldl -lm -o cpp-client/cxx-cli
```

Az include-útvonalak magyarázata: a `-I target/cxxbridge` adja a generált `rkv-cxx/src/lib.rs.h` és a `rust/cxx.h` fejlécet, a `-I .` pedig a `lib.rs.h`-ban szereplő `#include "rkv-cxx/include/logger.h"` sort oldja fel a workspace gyökeréből. A natív linkfüggőségek (`-lpthread -ldl -lm`) a Rust standard könyvtár statikus linkeléséhez kellenek.

Futtatás (minden parancs külön folyamat, az állapotot a megadott könyvtár őrzi):

Kódblokk — fájl: terminál · művelet: parancssorozat · ellenőrzés: a lenti kimenet

```
D=/tmp/rkv-cxx-demo
./cpp-client/cxx-cli $D SET alma piros
./cpp-client/cxx-cli $D SET alga zold
./cpp-client/cxx-cli $D GET alma
./cpp-client/cxx-cli $D COUNT al
./cpp-client/cxx-cli $D LIST
./cpp-client/cxx-cli $D DEL alga
```

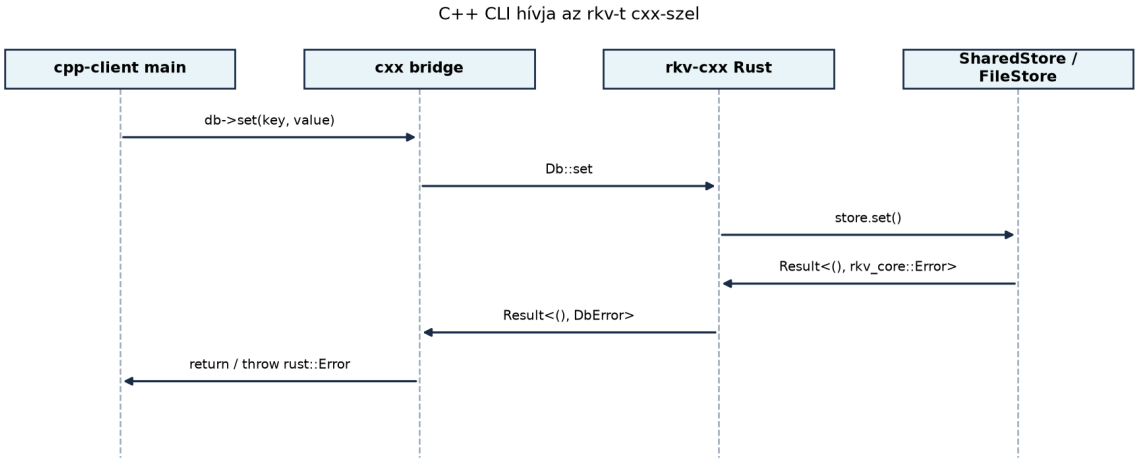
```
./cpp-client/cxx-cli $D GET alga; echo "kilépési kód: $?"
./cpp-client/cxx-cli $D COUNT
```

Várt kimenet (a fenti parancsokat kipróbáltuk, az eredmény ez):

```
OK
OK
VALUE piros
2
alga
alma
OK
ERR a kulcs nem található: alga
kilépési kód: 3
1
```

Ellenőrzési pont. `cargo test -p rkv-cxx` négy tesztje zöld (Rust-oldali szerződés, tartósság két nyitás között, C++ kivétel mint `Err`, Rust által hívott C++ osztály), a `cxx-cli` pedig a fenti kimenetet adja. A hiányzó kulcs `rust::Error` kivételt okoz (kilépési kód 3), nem üres értéket.

Az ábra a C++ CLI, a cxx bridge és a Rust tároló közötti hívásláncot mutatja:



19.2. ábra: C++ CLI hívja az rkv-t cxx-szel - A C++ main a generált wrapperen keresztül hívja a Rust függvényeket; a Result hibája rust::Error kivétellé alakul.

19.2.6 Build-integráció: CMake + Cargo

A laborban kézzel fordítottuk a C++ oldalt; ez a könyv ellenőrzött útvonala. Valódi projektben a build rendszer integrálható, de ez **opcionális, a szerzői környezetben nem ellenőrzött kitekintés**: a **Corrosion 0.6.1** CMake-modulja lehetővé teszi Cargo crate-ek beillesztését a CMake buildbe: a CMake meghívja a Cargo-t, és a generált staticlib-et automatikusan linkeli. Az alábbi vázlat a Corrosion 0.6.1 dokumentált API-jára épül, nem lefuttatott eredmény:

```
# CMakeLists.txt (vázlat)
find_package(Corrosion REQUIRED)
corrosion_import_crate(MANIFEST_PATH ../rkv-cxx/Cargo.toml)
add_executable(cxx-cli cpp-client/cxx_cli.cpp)
target_include_directories(cxx-cli PRIVATE ${CMAKE_SOURCE_DIR}
 ↪  ${CMAKE_BINARY_DIR}/cargo/build/target/cxxbridge)
target_link_libraries(cxx-cli PRIVATE rkv_cxx)
```

Mit ellenőriztünk, és mit nem. A fenti vázlatot **nem futtattuk le**: a szerzői környezetben nincs telepítve a CMake (a rögzített környezet csak GCC 13/Clang 18-at ír elő, lásd Eszközlánc), ezért a Corrosion 0.6.1 pontos target- és include-útvonalait (különösen a `cxxbridge` könyvtár helyét a CMake build könyvtárán belül) a saját környezetben kell ellenőrizni. A kézi `cargo build` + `g++` lánc viszont lefutott (lásd az 5. lépést), és ez a CMake-es változat referenciája: ugyanazt az include-készletet (`target/cxxbridge`, a workspace gyökere) és ugyanazt a három natív könyvtárat kell kapnia. Ha a CMake-es utat választod, előbb egy friss buildkönyvtárban futtasd a `cmake -S . -B build && cmake --build build` parancsot, és csak a sikeres configure/build után tekintsd működőnek.

19.2.7 Migrációs stratégia: perem-modulok, új kód Rustban, tesztlefedettség

A cxx ideális eszköz egy meglévő C++-kódbázis fokozatos Rustra migrálásához. A stratégia lépései:

1. **Perem-modulok azonosítása.** Keressük meg azokat a modulokat, amelyek jól elkülöníthetők, és interfészük stabil. Ilyen például az rkv esetében a tárolóréteg, a parancsértelmezés vagy a naplózás.
2. **Új kód Rustban.** A kiválasztott modult Rustban újraimplementáljuk, és a cxx bridge-en keresztül tesszük elérhetővé a C++ oldal számára. A régi C++ hívások változatlanul működnek, mert a bridge ugyanazt az interfészt nyújtja.
3. **Tesztlefedettség megőrzése.** A meglévő C++ unit teszteket változatlanul futtatjuk a cxx hídon keresztül, így azonnal észleljük a viselkedésbeli eltéréseket. Emellett a Rust oldalon is írunk teszteket (lásd Unit-, integrációs és dokumentációs tesztek).
4. **Fokozatos csere.** Miután egy modul stabilan működik Rustban, a C++ implementációt eltávolíthatjuk, vagy a hívókat átirányíthatjuk közvetlenül a Rust API-ra. A cxx lehetővé teszi, hogy a C++ oldal `std::unique_ptr`-t, `std::string`-et és más típusokat kapjon, így a migráció nem igényel nagy átírásokat.
5. **Határ felügyelete.** A határon átívelő típusokat érdemes egyszerű, stabil formában tartani, hogy a bridge karbantartása minimális legyen.

19.2.8 Teljesítmény a határon

A cxx híd teljesítménye általában közel van a kézzel írt C ABI-hoz, mert a generált kód nem végez felesleges másolást a megosztott típusoknál. A `rust::Vec<T>` és `rust::String` a Rust oldali memóriát látják, így a C++ oldal közvetlenül olvashatja, módosíthatja őket. A `Box<T>` és `std::unique_ptr` mozgatással adódnak át, nem történik mély másolás.

A határon való átlépés alapesetben nem inlinelhető (a generált thunk és a Rust függvény külön fordítási egység), ezért minden hívás függvényhívási költséggel jár; ez fordítási beállítástól függ, nem abszolút tilalom: ha a két oldal ugyanazzal az LLVM-mel, LTO-val (például `-Clinker-plugin-lto` és `-flto`) épül, az inlinelés elvileg lehetséges, de ezt a saját buildünkön kell mérni. A `Result` hibája C++ oldalon `rust::Error` kivétel, és a kivétel dobása-elkapása jellemzően nagyságrendekkel drágább a normál visszatérésnél, mert a C++ kivételkezelés a hibaágon táblavezérelt visszacsévélést végez. Az rkv-ben ezen a környezeten kivételköltséget **nem mértünk**, ezért számot nem közlünk; a döntés elve: ami gyakori és várt (hiányzó kulcs), ne kivétel legyen.

Gyakori `NotFound` esetén ezért adjunk nem dobó, explicit státuszos megosztott típust. A `cxx` megosztott structja `bool` és `Vec<u8>` mezőt is hordozhat:

```rust
#[cxx::bridge(namespace = "rkv")]
mod ffi {
    /// Nem dobó keresési eredmény: a hiányzó kulcs gyakori eset, nem kivétel.
    struct Lookup {
        found: bool,
        value: Vec<u8>,
    }

    extern "Rust" {
        type Db;
        /// Hiányzó kulcsra `found == false`, nem kivétel.
        fn try_get(self: &Db, key: &str) -> Result<Lookup>;
    }
}

impl Db {
    fn try_get(&self, key: &str) -> Result<ffi::Lookup, DbError> {
        let value = self.store.get(key)?;
        Ok(ffi::Lookup { found: value.is_some(), value: value.unwrap_or_default() })
    }
}
```

C++ oldalon: `auto r = db->try_get("k"); if (r.found) { /* r.value */ }`. A `Result` marad a valódi hibákra (I/O, poisoned tároló). A `try_get` tesztje (`missing_key_is_a_flag_not_an_error`) a `cargo test -p rkv-cxx` futásban zöld, és `clippy -D warnings`-szel is tiszta; a C++ oldali hívást nem fordítottuk le. A teljesítményt mindig mérni kell, a saját terhelésünkön.

19.2.9 Tipikus hibák és átállási csapdák

1. **A generált header include útvonalának elfelejtése.** A `cxx-build` a generált headert a `target/cxxbridge/<crate>/src/lib.rs.h` útvonalra teszi, és a benne lévő `#include "rkv-cxx/include/logger.h"` a workspace gyökeréhez képest oldódik fel. Ha a C++ oldal nem találja a fejlécet, a `-I target/cxxbridge` és a `-I .` hiányzik.
2. **A kivételkezelés elhanyagolása a C++ oldalon.** Ha a Rust függvény `Result`-ot ad vissza, a C++ hívás `rust::Error` kivételt dobhat. Ha nincs `try/catch`, a program `std::terminate`-tel leáll.
3. **`Result<T, E>` a bridge-ben.** A bridge-ben a hibatípus nem szerepelhet (`unsupported type`); a bridge-deklaráció `Result<T>`, a Rust implementáció hibatípusa pedig bármilyen `Display`-t implementáló típus lehet.
4. **Rust forrás átadása a C++ fordítónak.** A `.file("src/lib.rs")` hiba: a Rust fájlt a `bridge(…)` hívás olvassa be, a `.file(…)` csak `.cc`/`.cpp` forrásokat kap.
5. **Állapot a folyamatok között.** A memóriatároló egy folyamat élettartamáig él; külön CLI-hívások között csak a `FileStore` (vagy a hálózati szerver) őrzi meg az adatot.
6. **Az élettartamok figyelmen kívül hagyása.** A `&str` és `&[T]` a bridge-ben csak addig érvényes, amíg a mögöttes adat él. A C++ oldalon a `rust::Str` vagy `rust::Slice` nem tárolható el a függvényhíváson túl anélkül, hogy a Rust oldali adatot is életben tartanánk.

19.2.10 Feladatok

19.2.10.1 1. feladat (fogalmi kérdés)

Lefordul-e az alábbi bridge modul? Ha nem, miért? Javítsd, és add meg a hiányzó `get_data` implementációt úgy, hogy a `MyError` C++ oldalon `rust::Error` kivételként jelenjen meg.

```rust
#[cxx::bridge]
mod ffi {
    extern "Rust" {
        fn get_data() -> Result<Vec<u8>, MyError>;
    }
}

#[derive(Debug)]
struct MyError;
impl std::fmt::Display for MyError {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        write!(f, "MyError")
    }
}
```

19.2.10.2 2. feladat (C++ kód átírása)

Írd át az alábbi C ABI-s Rust függvényt cxx bridge-re. A függvény egy stringet kap, és megfordítva adja vissza; üres bemenetre hibát jelez (a C ABI-s változat `nullptr`-t adna).

```rust
#[unsafe(no_mangle)]
pub unsafe extern "C" fn reverse(s: *const c_char) -> *mut c_char {
    // ...
}
```

19.2.10.3 3. feladat (implementációs feladat)

Bővítsd az `rkv-cxx` crate-et egy `exists(key)` függvénnyel, amely `bool`-t ad vissza arról, hogy a kulcs létezik-e (hiányzó kulcsra **nem** hiba). Add hozzá a bridge-hez, implementáld a Rust oldalon, tesztelj, és egészítsd ki a C++ CLI-t `EXISTS` paranccsal.

19.2.10.4 4. feladat (refaktorálás)

A `Db` metódusai közvetlenül a `SharedStore`-t hívják a bridge-ben. Szervezd át úgy, hogy a tárolóműveletek tiszta Rust függvényekként külön `ops` modulba kerüljenek, és a bridge-metódusok csak delegáljanak. Írj egy tesztet, amely a C++ réteg nélkül, csak az `ops` modulon át ellenőrzi a szerződést.

19.2.11 Megoldások

19.2.11.1 1. feladat megoldása

Nem fordul le: a cxx bridge-ben a `Result` hibatípusát nem lehet kiírni, a cxx-build `unsupported type` hibát ad. A deklaráció `Result<Vec<u8>>`; a Rust implementáció hibatípusa a `Display`-t implementáló `MyError` maradhat (a `std::error::Error` implementálása ajánlott). A hiba C++ oldalon `rust::Error` kivétel, `what()` szövege `MyError`.

```rust
use std::fmt;

#[cxx::bridge]
mod ffi {
    extern "Rust" {
        fn get_data() -> Result<Vec<u8>>;
    }
```

```
}

#[derive(Debug)]
struct MyError;

impl fmt::Display for MyError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "MyError")
    }
}

impl std::error::Error for MyError {}

fn get_data() -> Result<Vec<u8>, MyError> {
    Err(MyError)
}
```

(Kipróbálva egy külön scratch crate-ben: az eredeti `Result<Vec<u8>, MyError>` deklaráció `unsupported type` hibával áll meg, a fenti forma lefordul.)

A fordított irány: ha a C++ függvény `void may_throw(bool)` és a bridge-ben `fn may_throw(fail: bool) -> Result<()>;`, akkor a dobott `std::runtime_error` Rust oldalon `Err(cxx::Exception)`, amelynek `what()` metódusa az eredeti szöveget adja (lásd a labor `a_cpp_exception_becomes_a_rust_err` tesztjét).

19.2.11.2 2. feladat megoldása

```
use std::fmt;

#[cxx::bridge]
mod ffi {
    extern "Rust" {
        fn reverse(s: &str) -> Result<String>;
    }
}

#[derive(Debug)]
struct EmptyInput;

impl fmt::Display for EmptyInput {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "üres bemenet")
    }
}

impl std::error::Error for EmptyInput {}

fn reverse(s: &str) -> Result<String, EmptyInput> {
    if s.is_empty() {
        Err(EmptyInput)
    } else {
        Ok(s.chars().rev().collect())
    }
}
```

A C++ oldalon a függvény `rust::String`-et ad vissza (`std::string(…)`-gé alakítható), és hiba esetén `rust::Error` kivételt dob. Nincs `unsafe`, nincs kézi felszabadítás.

19.2.11.3 3. feladat megoldása

A bridge bővítése az `extern "Rust"` blokkban:

```
fn exists(self: &Db, key: &str) -> Result<bool>;
```

Implementáció a `Db` blokkban:

```
fn exists(&self, key: &str) -> Result<bool, DbError> {
    Ok(self.store.get(key)?.is_some())
}
```

Teszt:

```
#[test]
fn exists_is_not_an_error() {
    let db = open_mem();
    assert!(!db.exists("a").unwrap());
    db.set("a", b"").unwrap();
    assert!(db.exists("a").unwrap());
}
```

A C++ CLI-ben:

```
} else if (cmd == "EXISTS" && argc == 4) {
    std::cout << (db->exists(argv[3]) ? "true" : "false") << "\n";
}
```

19.2.11.4 4. feladat megoldása

Kódblokk — fájl: `rkv-cxx/src/lib.rs` · művelet: új `ops` modul, a `Db` metódusai delegálnak · ellenőrzés: `cargo test -p rkv-cxx`

```
mod ops {
    use super::DbError;
    use rkv_core::SharedStore;

    pub fn get(store: &SharedStore, key: &str) -> Result<Vec<u8>, DbError> {
        store.get(key)?.ok_or_else(|| DbError::NotFound(key.to_owned()))
    }

    pub fn del(store: &SharedStore, key: &str) -> Result<(), DbError> {
        if store.delete(key)? {
            Ok(())
        } else {
            Err(DbError::NotFound(key.to_owned()))
        }
    }
}

impl Db {
    fn get(&self, key: &str) -> Result<Vec<u8>, DbError> {
        ops::get(&self.store, key)
    }

    fn del(&self, key: &str) -> Result<(), DbError> {
        ops::del(&self.store, key)
    }
}
```

Az `ops` függvények nem ismerik sem a cxx-et, sem a `Db`-t, ezért egy `SharedStore::new(MemStore::new())` értékkel C++ nélkül tesztelhetők; a bridge-metódusok vékony delegálók maradnak. Ez a szétválasztás előkészíti a Záróprojekt tesztelhetőségét.

19.2.12 Rövid összefoglaló

A `cxx` crate biztonságos, típusos hidat generál Rust és C++ között. A `#[cxx::bridge]` modulban `extern "Rust"` és `extern "C++"` blokkokkal deklaráljuk a határon átívelő függvényeket és típusokat; a `cxx-build` a bridge-et tartalmazó Rust fájlt olvassa be, és legenerálja a wrapper kódot, a `.file(…)` pedig csak valódi C++ forrásokat kap. A bridge-ben a hibát `Result<T>` jelzi: a Rust implementáció `Display`-t implementáló hibatípusa C++ oldalon `rust::Error` kivétel, a C++ kivétel Rust oldalon `Err(cxx::Exception)`. Megosztott structok és átlátszatlan típusok is használhatók, a C++ osztályokat gyártófüggvényen és `UniquePtr`-en keresztül érjük el. A labor az `rkv-cxx` crate-et a valódi `SharedStore`/`FileStore` API-ra építi, a CLI külön folyamatai pedig a fájl-alapú tárolón át osztoznak az állapoton. A CMake/corrosion build vázlat, nem futtatott. **Továbbvitt eredmény:** a C++-integráció az r051 záróprojektben teljes rendszerként működik együtt a Rust tárolóval.

19.2.13 Források és ellenőrzési hiányok

- Ellenőrizve (Rust 1.99.1, `cxx` 1, g++ 13): a `cargo test -p rkv-cxx` négy tesztje, a `cargo build --release`, a `cxx-cli` fordítása és a labor kimenete, valamint a 2. feladat és az 1. feladat javított/hibás bridge-formája.
- Nem ellenőrizve: a CMake + corrosion build, mert a szerzői környezetben nincs CMake; a vázlat útvonalait a saját környezetben kell igazítani.
- A `cxx` crate API-ja és a generált fájlok elnevezése a crate verziójától függően változhat; a pontos részletek a docs.rs-en találhatók.

20 Makrók

20.1 Deklaratív és procedurális makrók

20.1.1 Motiváció

A C++-fejlesztő jól ismeri a preprocessor makrókat: `#define`, szöveges behelyettesítés, zárójelezési paranoia és néha rejtélyes hibák. A Rust makrók első ránézésre hasonlónak tűnnek, de alapvetően más elven dolgoznak: a fordító szintaktikai elemzésén belül, szintaktikai egységeken, úgynevezett tokenfán operálnak, és a higiénia miatt nem okozzák a C-preprocessor klasszikus névütközéseit. Ebben az alfejezetben a két makrófajtát, a deklaratív `macro_rules!`-t és a procedurális makrókat, egy konkrét rkv-s példán, a `#[derive(CommandName)]` elkészítésén keresztül tanuljuk meg.

A deklaratív makrók a gyakori mintákat rövidítik le, a procedurális makrók pedig tetszőleges Rust-kódot generálhatnak a forrásból. A kettő közötti választás az absztrakció megfelelő szintjének megtalálása. A cél nem az, hogy mindent makróval írjunk, hanem az, hogy akkor használjuk őket, amikor valóban nyelvi szintű kódgenerálásra van szükség.

20.1.2 Fogalmak és szabályok

Definíció. A *makró* olyan fordítási idejű kódgeneráló eszköz, amely bemenő szintaktikai egységekből állít elő Rust-kódot, még a név- és típusellenőrzés előtt. A C-preprocessorral ellentétben a Rust-makró nem szöveges szinten, hanem *tokenfákon* (token tree) dolgozik: a bemenet már a lexer által felismert tokenekből áll, és a makró a tokenek szerkezetét látja, nem a nyers karaktereket.

Két külön ellenőrzés. A makró *mintaillesztése* és a *kiterjesztett kód* ellenőrzése két különböző lépés. A fragmentspecifikátor (`expr`, `literal`, `ty`, …) csak a bemenet *szintaxisát* köti meg: a `$x:expr` bármilyen kifejezést elfogad, a `$x:literal` bármilyen literált (`5`, `true`, `"szöveg"`), típusról egyik sem tud. A típusellenőrzés csak a kiterjesztés *után*, a hívás helyén fut le; ezért a típushiba a kiterjesztett kódra mutat, nem a makró mintájára.

Elnevezés. A makróhívásnak három formája van, és csak a függvényszerű makró neve zárul felkiáltójellel: (1) függvényszerű makró: `vec!`, `println!`, `vec_of_strings!(...)`; (2) derive makró: `#[derive(Debug, Clone)]`; (3) attribútum makró: `#[instrument]`, `#[serde(...)]`. A `!` segít megkülönböztetni a függvényhívástól, és jelzi, hogy fordítási idejű kódgenerálás történik; a `#[...]` forma elemet vagy attribútumot jelöl, nem hívást.

Definíció. A *higiénia* (hygiene) a makrórendszer azon tulajdonsága, hogy a makró által bevezetett nevek (például lokális változók, `let`-kötések) nem ütköznek a hívás helyén lévő azonos nevű nevekkel, és nem szivárognak ki a makróból. A makró paraméterei természetesen a hívó által megadott kódot helyettesítik, de a makró törzsében deklarált segédváltozók a makró saját, higiénikus teréhez tartoznak.

A higiénia hatóköre. A `macro_rules!` ún. *mixed-site* higiéniát használ: a lokális változók és a címkék a makró definíciós helyén oldódnak fel (ezért nem ütköznek a hívóéval), minden más név

(típus, függvény, modul, trait) viszont a hívás helyén. Emiatt a `macro_rules!` törzsében a külső elemekre abszolút útvonallal hivatkozzunk (`::std::string::String`), a saját crate elemeire pedig a `$crate::elem` alakkal, amely a hívó crate-ben átnevezett importnál is a definiáló crate-re mutat. A procedurális makró ezzel szemben *tokeneket* állít elő, és a `quote!` alapértelmezésben a hívás helyének (`call_site`) spanjét adja a tokeneknek: a generált azonosítók **nem** higiénikusak, ütközhetnek a hívó neveivel, és a generált útvonalak a hívó crate nevei szerint oldódnak fel. Ezért a generált kód minden külső elemre teljes útvonalat használ.

Szabály. A Rust-makrók higiénikusak a lokális változók és címkék tekintetében. A C-preprocessor nem higiénikus: a makróban deklarált `tmp` változó ütközhet a hívó ugyanolyan nevű változójával, a makró által használt nevek pedig véletlenül elkaphatják a hívó scope-jában lévő neveket.

A `macro_rules!` deklaratív makrókat a következő szintaxissal definiáljuk:

```
macro_rules! makro_nev {
    (minta1) => { kiterjesztés1 };
    (minta2) => { kiterjesztés2 };
    // ...
}
```

A minták *fragmentspecifikátorokat* használnak: `$valtozo:fragspec` formában. A leggyakoribb fragmentspecifikátorok:

Fragmentspecifikátor	Mit fogad el
`expr`	kifejezés
`ident`	azonosító
`ty`	típus
`literal`	literál
`pat`	minta
`stmt`	utasítás (statement)
`item`	elem (fn, struct, trait, …)
`path`	útvonal (pl. `std::collections::HashMap`)
`tt`	tetszőleges tokenfa
`block`	blokk-kifejezés
`vis`	láthatósági szint (`pub`, `pub(crate)`, stb.)
`meta`	meta-attribútum (pl. `derive` tartalma)

A mintákban ismétlést is használhatunk:

- `$(...)*` nulla vagy több ismétlés;
- `$(...)+` egy vagy több ismétlés;
- `$(...)?` nulla vagy egy előfordulás.

Szeparátor megadható az ismétlésen belül: `$($x:expr),*` azt jelenti, hogy a kifejezéseket vessző választja el. A vessző az ismétlés része, nem a kiterjesztésé.

20.1.3 C++ és Rust összevetés

A klasszikus C++ swap-makró a higiénia hiányát mutatja. A makró belső `tmp` változója a hívó nevével csak akkor ütközik, ha a hívó **ugyanazt a nevet adja át argumentumként**:

Kódblokk — fájl: `swap.cpp` · művelet: új fájl · ellenőrzés: `g++ -std=c++17 -Wall -Wextra swap.cpp`

```cpp
#include <iostream>
#define SWAP(a, b) do { auto tmp = (a); (a) = (b); (b) = tmp; } while (0)

int main() {
    int tmp = 1, y = 2;
    SWAP(tmp, y); // kiterjesztve: auto tmp = (tmp); ... a belső tmp eltakarja a hívóét
    std::cout << tmp << " " << y << "\n";
}
```

A GCC 13 itt `error: use of 'tmp' before deduction of 'auto'` hibát ad; ha a makró `auto` helyett explicit típust (`int tmp = (a);`) használna, a hiba csendes, inicializálatlan érték lenne. Ez a *névelkapás*. Ha a hívó `tmp` változója nem szerepel az argumentumok között (`SWAP(x, y)`), a kód lefordul és helyes, mert a makró a saját blokkjában deklarál.

A Rust deklaratív makró tokenfán dolgozik, és a `tmp` név higiénikus: mindkét hívás helyes.

Kódblokk — fájl: `src/main.rs` (önálló `cargo new` projekt) · művelet: teljes csere · ellenőrzés: `cargo run` → `2 1 999` és `2 1`

```rust
macro_rules! swap {
    ($a:expr, $b:expr) => {
        let tmp = $a;
        $a = $b;
        $b = tmp;
    };
}

fn main() {
    let mut x = 1;
    let mut y = 2;
    let tmp = 999;
    swap!(x, y);
    println!("{x} {y} {tmp}"); // 2 1 999

    let mut tmp2 = 1;
    let mut z = 2;
    swap!(tmp2, z);
    println!("{tmp2} {z}"); // 2 1
}
```

A makró által bevezetett `tmp` más szintaktikai környezetbe tartozik, mint a hívó `tmp`-je, ezért a `let tmp` új, higiénikus kötést hoz létre. A Rust makró sem varázslat: a `$a` és `$b` kifejezéseket a törzs többször is beilleszti, ezért mellékhatásos argumentumnál (`swap!(v.pop(), …)`) ugyanúgy óvatosnak kell lenni, mint a C++ makróknál. (A higiénia a lokális változókra és címkékre vonatkozik, a függvény-, típus- és modulnevekre nem.)

Egy gyakorlatiasabb deklaratív makró, amely ismétlést használ, a `vec_of_strings!`:

```rust
macro_rules! vec_of_strings {
    ($($x:expr),* $(,)?) => {
        vec![$($x.to_string()),*]
    };
}

fn main() {
    let v = vec_of_strings!["alma", "körte", "szilva"];
    let w = vec_of_strings![42, 7]; // "42", "7"
    println!("{v:?} {w:?}");
}
```

A minta `$($x:expr),*` nulla vagy több vesszővel elválasztott kifejezést fogad el. A `$(,)?` opcionális záró vesszőt enged meg. A kiterjesztésben ugyanaz a `$()`-ismétlés szerepel, így a makró minden bemenő kifejezésre legenerálja a `.to_string()` hívást. Ez a minta a C++ variadic makrók logikáját idézi, de a tokenfák miatt nem kell a `__VA_ARGS__` trükkjeit használni.

20.1.4 Kidolgozott példa: derive makró az rkv `Command` enumra

A procedurális makrók külön, `proc-macro = true` crate-ben élnek, mert a fordító a makrót a célkód fordításától elkülönítve, a gazdaprogramban hajtja végre; ez a szétválasztás a higiéniát is érinti (lásd fent). Három fajtájuk van:

- **derive makró**: `#[derive(TraitNev)]`, adott típusra implementál egy traitet;
- **attribútum makró**: `#[my_attribute(...)]`, tetszőleges elemhez adható;
- **függvényszerű makró**: `my_macro!(...)`, úgy hívható, mint a deklaratív makró, de tetszőleges Rust-kódot futtathat.

A derive makró azért fontos, mert a standard `#[derive(Debug)]` is derive makró, csak az nem külön felhasználói proc-macro crate, hanem a fordító beépített, a standard könyvtárhoz tartozó implementációja; a mi derive-unk ugyanazt a felületet használja, de külső crate-ből. A célunk most egy saját derive, a `#[derive(CommandName)]`, amely az `rkv-core` `Command` enum minden variánsához protokoll-parancsnevet rendel, és az `impl CommandName for Command` blokkot generálja le. Az alapértelmezett név a variánsnév nagybetűs alakja (`Get` → `GET`), de a protokoll neve nem mindig egyezik a variánsnévvel: a `Delete` variáns a protokollban `DEL`, ezt a `#[command_name = "DEL"]` segédattribútummal adjuk meg. Így a meglévő szöveges protokoll (`DEL kulcs`) nem változik. Ehhez a `syn` crate parse-olja a típus AST-ját, a `quote` crate pedig Rust-kódot generál.

Munkakönyvtár: `~/rust-labor/rkv`.

A labor lépései:

1. Hozzuk létre a `rkv-derive` proc-macro crate-et:

Kódblokk — fájl: terminál (workspace gyökér) · művelet: új crate · ellenőrzés: `ls rkv-derive/Cargo.toml`

```
cd ~/rust-labor/rkv
cargo new --lib rkv-derive
```

2. A `rkv-derive/Cargo.toml` tartalma:

Kódblokk — fájl: `rkv-derive/Cargo.toml` · művelet: teljes csere · ellenőrzés: `cargo metadata --no-deps`

```
[package]
name = "rkv-derive"
version = "0.1.0"
edition = "2024"
rust-version = "1.89"

[lib]
proc-macro = true

[dependencies]
syn = "2"
quote = "1"
proc-macro2 = "1"
```

A `proc-macro = true` jelöli, hogy ez a crate proc-macro, vagyis a kimenete `TokenStream`, nem normál könyvtár. A crate a workspace többi tagjával megegyező 2024-es editiont használ.

3. Adjuk hozzá a crate-et a workspace tagjaihoz a gyökér `Cargo.toml`-ben, az `rkv-core`-hoz pedig függőségként:

Kódblokk — fájl: `Cargo.toml` (workspace gyökér) · művelet: `"rkv-derive"` felvétele a `members` listába · ellenőrzés: `cargo metadata --no-deps`

```
[workspace]
resolver = "3"
members = [
    "rkv-core",
    "rkv-cli",
    "rkv-server",
    "rkv-ffi",
    "rkv-cxx",
    "rkv-derive",
]
```

4. Vegyük fel az `rkv-core/Cargo.toml` függőségei közé:

Kódblokk — fájl: `rkv-core/Cargo.toml` · művelet: új sor a `[dependencies]` szakaszban · ellenőrzés: `cargo build -p rkv-core`

```
[dependencies]
rkv-derive = { path = "../rkv-derive" }
```

5. Írjuk meg a derive makrót. A szerkezet: a belépési pont (`derive_command_name`) a `syn::Result`-ot adó `expand` függvényt hívja, és a hibát `compile_error!`-ként adja vissza, így a fordító a felhasználó kódjára mutat, nem a makró belsejére. A segédattribútumot a `attributes(command_name)` jegyzi be; enélkül a `#[command_name = "…"] cannot find attribute` hibát ad.

Kódblokk — fájl: `rkv-derive/src/lib.rs` · művelet: teljes csere · ellenőrzés: `cargo build -p rkv-derive`

```
use proc_macro::TokenStream;
use quote::quote;
use syn::{Data, DeriveInput, Expr, ExprLit, Fields, Lit, Meta, Variant, parse_macro_input};

/// `#[derive(CommandName)]`: a `CommandName::command_name` a variáns protokollneve.
/// Alapérték a variánsnév nagybetűs alakja; `#[command_name = "DEL"]` felülírja.
#[proc_macro_derive(CommandName, attributes(command_name))]
pub fn derive_command_name(input: TokenStream) -> TokenStream {
    let input = parse_macro_input!(input as DeriveInput);
    expand(&input)
        .unwrap_or_else(syn::Error::into_compile_error)
        .into()
}

fn expand(input: &DeriveInput) -> syn::Result<proc_macro2::TokenStream> {
    let Data::Enum(data_enum) = &input.data else {
        return Err(syn::Error::new_spanned(
            input,
            "a CommandName derive csak enumokra alkalmazható",
        ));
    };
    if !input.generics.params.is_empty() {
        return Err(syn::Error::new_spanned(
            &input.generics,
```

```
            "a CommandName derive nem támogatja a generikus enumokat",
        ));
    }

    let name = &input.ident;
    let arms = data_enum
        .variants
        .iter()
        .map(|variant| {
            let ident = &variant.ident;
            let pattern = match &variant.fields {
                Fields::Unnamed(_) => quote! { Self::#ident(..) },
                other => {
                    return Err(syn::Error::new_spanned(
                        other,
                        "a CommandName derive csak tuple-variánsokat támogat",
                    ));
                }
            };
            let protocol_name = protocol_name(variant)?;
            Ok(quote! { #pattern => #protocol_name, })
        })
        .collect::<syn::Result<Vec<_>>>()?;

    Ok(quote! {
        impl ::rkv_core::CommandName for #name {
            fn command_name(&self) -> &'static str {
                match self {
                    #(#arms)*
                }
            }
        }
    })
}

/// A variáns protokollneve: a `#[command_name = "..."]` értéke, ennek hiányában a nagybetűs
↪ variánsnév.
fn protocol_name(variant: &Variant) -> syn::Result<String> {
    let mut found: Option<String> = None;
    for attr in variant
        .attrs
        .iter()
        .filter(|a| a.path().is_ident("command_name"))
    {
        if found.is_some() {
            return Err(syn::Error::new_spanned(
                attr,
                "a command_name attribútum csak egyszer adható meg",
            ));
        }
        let Meta::NameValue(name_value) = &attr.meta else {
            return Err(syn::Error::new_spanned(
                attr,
                "várt alak: #[command_name = \"NÉV\"]",
            ));
        };
        let Expr::Lit(ExprLit {
            lit: Lit::Str(text),
            ..
        }) = &name_value.value
        else {
            return Err(syn::Error::new_spanned(
                &name_value.value,
```

```
                "a command_name értéke szöveges literál legyen",
            ));
        };
        let value = text.value();
        if value.is_empty() || value.chars().any(|c| c.is_whitespace() || c.is_control()) {
            return Err(syn::Error::new_spanned(
                text,
                "a protokollnév nem lehet üres, és nem tartalmazhat szóközt vagy
                 ↪  vezérlőkaraktert",
            ));
        }
        found = Some(value);
    }
    Ok(found.unwrap_or_else(|| variant.ident.to_string().to_ascii_uppercase()))
}
```

A makró beolvassa a típus AST-ját, ellenőrzi, hogy generikus paraméter nélküli enum-e, és minden tuple-variánsra generál egy `Self::Var(..) => "NÉV"` kart. A `#pattern`, `#protocol_name` és `#name` a megfelelő értéket interpolálja, a `#(#arms)*` ismétli a karokat. A `protocol_name` a `variant.attrs` között keresi a `command_name` attribútumot, és a syn 2 szerkezetével (`Meta::NameValue` → `Expr::Lit` → `Lit::Str`) olvassa ki az értékét. Hiányzó attribútum esetén a nagybetűs variánsnév az alapérték. A generált kód a trait teljes útvonalát (`::rkv_core::CommandName`) használja, így a derive a trait importálása nélkül, bármely olyan crate-ben működik, amelynek függőségei között az `rkv-core` az `rkv_core` néven szerepel, és bármely modulból használható. Amit a generált kód a környezetétől vár: (1) az `rkv_core` nevű extern crate létezik; (2) benne a `CommandName` trait elérhető a gyökéren. Átnevezett függőségnél (`kv = { package = "rkv-core", … }`) a derive nem fordul le: a scratch crate-ben ezt kipróbáltuk, `error[E0433]: cannot findrkv_corein the list of imported crates` a hiba. Egy proc-macro nem látja, hogyan nevezte át a hívó a függőséget; az újraexportálás **önmagában nem segít**, mert a generált kód továbbra is `::rkv_core`-ra hivatkozik, ami az átnevezett crate-ben nem létezik. Valódi megoldás a derive konfigurálható crate-útvonala (`#[command_name(crate = "kv")]`, ahogy a serde `#[serde(crate = "…")]`-je), vagy a `proc-macro-crate` crate, amely a hívó `Cargo.toml`-jából oldja fel a csomagnév → crate-név leképezést. Ez a labor egyiket sem valósítja meg: a `rkv_core` nevet rögzített szerződésnek tekinti, és ezt a korlátot a dokumentációban kimondja. A `macro_rules!` ugyanezt `$crate`-tel oldja meg, a proc-macrónak nincs `$crate`-megfelelője.

6. Módosítsuk az `rkv-core/src/command.rs` fájlt: vezessük be a `CommandName` traitet, alkalmazzuk a derive-ot, és használjuk a `Display`-ben a protokollnevet. A `FromStr` és a hibakezelés változatlan marad.

Kódblokk — fájl: `rkv-core/src/command.rs` · művelet: a fájl elejének (a `Command` definíciója) és a `Display` implementáció cseréje · ellenőrzés: `cargo test -p rkv-core --lib command`

```
/// A parancs protokollneve; a `Command`-nál a `rkv_derive::CommandName` generálja.
pub trait CommandName {
    fn command_name(&self) -> &'static str;
}

/// A kliens által küldött parancs.
#[derive(Debug, Clone, PartialEq, Eq, rkv_derive::CommandName)]
pub enum Command {
    Get(String),
    Set(String, String),
    #[command_name = "DEL"]
    Delete(String),
    List(Option<String>),
}
```

```
impl fmt::Display for Command {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            Command::Get(key) | Command::Delete(key) => write!(f, "{} {key}",
            ↪  self.command_name()),
            Command::Set(key, value) => write!(f, "{} {key} {value}", self.command_name()),
            Command::List(Some(prefix)) => write!(f, "{} {prefix}", self.command_name()),
            Command::List(None) => f.write_str(self.command_name()),
        }
    }
}
```

A `rkv-core/src/lib.rs`-ben két sor kell: a trait újraexportálása és az `extern crate self`, amely miatt a generált `::rkv_core::CommandName` útvonal magában az `rkv-core` crate-ben is feloldódik:

Kódblokk — fájl: `rkv-core/src/lib.rs` · művelet: a fájl elejére és a `pub use command::…` sor cseréje · ellenőrzés: `cargo build -p rkv-core`

```
extern crate self as rkv_core;

pub use command::{Command, CommandName};
```

A `Command` értékei (`Set(String, String)`) változatlanok: a derive csak a nevet adja, a tárolt `Vec<u8>` értékek továbbra is hexa kódolással (`encode_value`/`decode_value`) mennek át a szöveges protokollon.

7. Ellenőrizzük teszttel: minden variánsra a protokollnév, a formázott sor első szava és a visszaolvasás (`FromStr`). A teszt a `command.rs` meglévő `tests` moduljába kerül:

Kódblokk — fájl: `rkv-core/src/command.rs` · művelet: új teszt a `tests` modulba · ellenőrzés: `cargo test -p rkv-core --lib command`

```
#[test]
fn every_variant_has_its_protocol_name() {
    let names = [
        (Command::Get("a".into()), "GET"),
        (Command::Set("a".into(), "b".into()), "SET"),
        (Command::Delete("a".into()), "DEL"), // attribútum: nem DELETE
        (Command::List(None), "LIST"),
        (Command::List(Some("p".into())), "LIST"),
    ];
    for (cmd, name) in names {
        assert_eq!(cmd.command_name(), name);
        // A Display első szava a protokollnév, és a sor visszaolvasható.
        assert_eq!(cmd.to_string().split(' ').next(), Some(name));
        assert_eq!(cmd.to_string().parse::<Command>().unwrap(), cmd);
    }
}
```

Futtassuk a parancs-teszteket:

```
cargo test -p rkv-core --lib command
```

Várható kimenet (a `command` szűrőre illeszkedő összes teszt, köztük az új):

```
running 8 tests
test command::tests::every_variant_has_its_protocol_name ... ok
...
test result: ok. 8 passed; 0 failed
```

Ellenőrzési pont. A teljes `cargo test -p rkv-core --lib` zöld (45 teszt), a `cargo clippy -p rkv-core -p rkv-derive --all-targets -- -D warnings` figyelmeztetés nélkül lefut, és a `Delete` formázása `DEL kulcs`.

A derive hibás használatát a fordító a megjelölt helyen jelzi (az alábbi kimeneteket a lab crate-jein ellenőriztük):

Hibás használat	Fordítási hiba
`#[command_name = "A"] #[command_name = "B"]`	`a command_name attribútum csak egyszer adható meg`
`#[command_name = 5]`	`a command_name értéke szöveges literál legyen`
`#[command_name = ""]` vagy `"A B"`	`a protokollnév nem lehet üres, és nem tartalmazhat szóközt vagy vezérlőkaraktert`
`#[command_name]` vagy `#[command_name("LST")]`	`várt alak: #[command_name = "NÉV"]`
unit vagy struct-variáns	`a CommandName derive csak tuple-variánsokat támogat`
`enum C<T>`	`a CommandName derive nem támogatja a generikus enumokat`
`struct`	`a CommandName derive csak enumokra alkalmazható`

Továbbvitt eredmény. Az itt elkészített `rkv-derive` proc-macro crate és a `CommandName` derive makró a **formázás** (Display) egyetlen forrása lesz a Záróprojekt: az rkv teljes rendszerként alfejezetben. A beolvasás (`FromStr`) továbbra is kézzel írt `match` a parancsnevekkel, ezért a protokollnév megváltoztatásakor a parser karjait is át kell írni; a fenti `every_variant_has_its_protocol_name` teszt a Display és a FromStr körkörös egyezését ellenőrzi, így az inkonzisztens átírást elkapja. Teljes egyforrásúsághoz a derive-nek a parser karjait is generálnia kellene (vagy mindkét oldalnak egy közös `const` táblából kellene dolgoznia); ezt a labor nem valósítja meg. A `CommandName` trait és a `Command Display` implementációja (`to_line` ezt hívja) a hálózati réteg szöveges protokolljának formázója marad; a `DEL` név nem változik.

20.1.5 Tipikus hibák és átállási csapdák

1. **Makró használata ott, ahol függvény is elég.** A C++-ban a preprocessor gyakran kényszerűség, a Rustban a generikus függvények, trait-ek és closure-ök sok esetet lefednek. Ha a feladat megoldható `fn`-nel, ne írjunk makrót, mert a makró megnehezíti a hibakeresést és a dokumentálást. Például a `vec_of_strings!` helyett egy `fn vec_of_strings<S: ToString>(items: impl IntoIterator<Item = S>) -> Vec<String>` is működik, ha a hívó az elemeket egy tömbbe vagy iterátorba gyűjti. A makró akkor indokolt, ha szintaktikai mintát kell rövidíteni, például a `println!` formázási argumentumait.

2. **A C-preprocessor szokások átvétele.** Ne használjunk makrót konstansokhoz (`#define MAX_SIZE 1024` helyett `const MAX_SIZE: usize = 1024;`), és ne használjunk makrót függvények helyettesítésére. A Rust `const` és `#[inline]` függvények típusbiztosabbak. A fragmentspecifikátor nem típusmegkötés: a `$x:literal` mindössze azt írja elő, hogy az argumentum literál (`5`, `true` és `"szöveg"` egyaránt), azt nem, hogy `bool`. Típusmegkötéshez generikus függvény vagy a kiterjesztett kódba írt típusannotáció kell (`let b: bool = $x;`).

3. **A derive makró nem kezeli a generikus típusokat.** A fenti `CommandName` derive a generikus enumot és a nem tuple-variánsokat érthető fordítási hibával visszautasítja. Ha a jövőben `Command<T>` lenne, a generált kódban `impl<T> CommandName for Command<T>` és a `T` megkötései is kellenének; a `syn` API-ban `input.generics.split_for_impl()` adja ezt. A példa az rkv konkrét `Command` típusára szabott.

4. **A proc makróban `panic!`-kel jelezni a hibát** a felhasználó felé nyers, nehezen olvasható fordítási hibát ad. Használjunk `syn::Error::new_spanned` és `to_compile_error` (vagy `into_compile_error`) párost, ahogy a példában is tettük, hogy a hiba a pontos helyre mutasson.

5. **Be nem jegyzett segédattribútum.** Ha a `#[proc_macro_derive(CommandName)]` mellől hiányzik az `attributes(command_name)`, a `#[command_name = "DEL"]` sor `cannot find attributecommand_namein this scope` hibát ad, még akkor is, ha a derive kódja helyes.

20.1.6 Feladatok

1. **Lefordul-e és miért?** Tekintsd a következő Rust-kódot:

```
macro_rules! add_one {
    ($x:expr) => { $x + 1 };
}

fn main() {
    let s = add_one!("hello");
    println!("{s}");
}
```

Lefordul-e a program? Tipp: a makró mintaillesztése csak a szintaxist nézi, a típusellenőrzés a kiterjesztés után fut.

2. **C++-kód átírása idiomatikus Rustra.** A C++-ban gyakran használnak ilyen makrót:

```
#define TO_STRING(x) #x
```

Írd át ezt Rustra! Használj `stringify!` beépített makrót, és mutasd be, miért nincs szükség saját makróra.

3. **Implementációs feladat az rkv-ban.** Bővítsd a `CommandName` derive makrót úgy, hogy az enum unit és struct-variánsait is támogassa, ne csak a tuple-szerkezetűeket. Ellenőrizd egy külön próba-enummal, hogy a `command_name` mindhárom alakra működik.

4. **Hibás attribútumok.** A lab derive-ja a `#[command_name = "…"]` segédattribútumot már kezeli. Próbáld ki, mit mond a fordító, ha az attribútum üres, ismételt, nem szöveges értékű, vagy `#[command_name("LST")]` alakú, és mit, ha a `proc_macro_derive` nem jegyzi be a segédattribútumot. Mutasd meg, hol (melyik tokenre) mutat a hibaüzenet.

20.1.7 Megoldások

1. A program **nem fordul le.** A makró mintaillesztése sikeres (`"hello"` kifejezés), a kiterjesztés viszont `"hello" + 1`, és a típusellenőrzés itt bukik el: `error[E0369]: cannot add{integer}to&str‘. A hiba a makróban lévő$x + 1kifejezésre és a hívás helyére mutat. Tanulság: a makró nem ellenőrzi az argumentum típusát, a hiba a kiterjesztés után jön, és csak azadd_one!(41)‘` hívásból lesz 42.

2. A C++ stringizáló makró helyett a `stringify!` beépített makrót használjuk:

```
fn main() {
    let s = stringify!(alma + 1);
    println!("{s}"); // alma + 1
}
```

A `stringify!` a tokeneket string literállá alakítja, nincs szükség saját makróra. A C++-ban a preprocessor `#` operátora végezte ezt; Rustban ez nyelvi szintű beépített eszköz, és a tokenfára épít, nem a nyers szövegre (a kimenetben a tokenek közé szóköz kerül).

3. A bővített derive-ban a `Fields` mindhárom alakját kezeljük:

```
use syn::Fields;

let pattern = match &variant.fields {
    Fields::Unit => quote! { Self::#ident },
    Fields::Unnamed(_) => quote! { Self::#ident(..) },
    Fields::Named(_) => quote! { Self::#ident { .. } },
};
```

A `command_name` nem függ a mezőktől, ezért elég a mezőket figyelmen kívül hagyó minta. A próba-enum, amelyen kipróbáltuk:

```
#[derive(rkv_derive::CommandName)]
enum Probe {
    Ping,
    Pair(u8, u8),
    #[command_name = "LST"]
    Named { a: u8 },
}
// Probe::Ping.command_name() == "PING", Probe::Pair(1, 2) → "PAIR", Probe::Named { a: 1 } →
↪  "LST"
```

4. A fordító a hibás használatot az attribútumra (vagy az értékére) mutatva jelzi; az üzenetek a labor táblázatában szerepelnek. A lényeg:

- ismételt attribútum: a **második** `#[command_name …]` tokenjeire mutat;
- `= 5`: az `5` literálra mutat, `a command_name értéke szöveges literál legyen`;
- `= ""` vagy `"A B"`: a szöveges literálra mutat;
- `#[command_name]` és `#[command_name("LST")]`: az egész attribútumra mutat, `várt alak: #[command_name = "NÉV"]`;
- ha az `attributes(command_name)` hiányzik a `proc_macro_derive`-ból: `cannot find attributecommand_namein this scope`, az attribútum nevére mutat.

Ha a `#[command_name("LST")]` alakot szeretnénk elfogadni, a `Meta::List` ágat kell kezelni, és `attr.parse_args::<syn::LitStr>()`-rel kiolvasni az értéket; a két alakot egyszerre nem érdemes támogatni, a könyvben a `= "NÉV"` alak az egyetlen elfogadott.

20.1.8 Rövid összefoglaló

A deklaratív `macro_rules!` makrók tokenfa-mintákkal, fragmentspecifikátorokkal (`expr`, `ident`, `ty`, `tt`, `item`, `pat`, `path`, `stmt`, `block`, `vis`, `meta`, `literal`) és ismétlésekkel (`$(...)*`, `$(...)+`, `$(...)?`) generálnak kódot. A fragmentspecifikátor csak a bemenet szintaxisát köti meg; a kiterjesztett kód típusellenőrzése utána, a hívás helyén történik. A higiénia a lokális változókra és címkékre garantálja, hogy a makróban bevezetett nevek nem ütköznek a hívó neveivel; a C++ makró ezzel szemben névelkapásra hajlamos. A procedurális makrók külön, `proc-macro`

`= true` crate-ben élnek; a derive, attribútum és függvényszerű formákat támogatja. A `syn` parse-olja a típus AST-ját, a `quote` generál kódot. A laborban létrehoztuk az `rkv-derive` crate-et, megírtuk a `#[proc_macro_derive(CommandName, attributes(command_name))]` makrót, amely a variánsnevet nagybetűssé alakítja, a `#[command_name = "DEL"]` attribútummal pedig felülírható, és az `rkv-core` `Command` enumján alkalmaztuk. A generált `impl CommandName for Command` a `command_name()` metódust adja, amelyet a `Display` használ. **Továbbvitt eredmények:** az `rkv-derive` crate és a `CommandName` derive makró; a `Command` formázása ezt a nevet használja (`Delete` → `DEL`), és ez a protokoll-konzisztencia alapja a formázás oldalán a Záróprojekt: az rkv teljes rendszerként alfejezetben (a `FromStr` parser karjai továbbra is kézzel írtak, a körkörös teszt ellenőrzi az egyezést).

20.1.9 Források és ellenőrzési hiányok

- Ellenőrizve (Rust 1.99.1, `syn` 2, `quote` 1, g++ 13): a labor `cargo test -p rkv-core --lib` (45 teszt) és `clippy -D warnings`; az `add_one!` hibája (`E0369`); a `swap!` kimenetei; a C++ `SWAP` névelkapásos hibája; a derive hibaüzenetei az összes felsorolt hibás alakra; a unit/struct-variánsokat támogató bővítés.
- A Rust-makrók nyelvi szabályai a Rust Reference makró-fejezetében találhatók: https://doc.rust-lang.org/reference/macros.html. A `syn` és `quote` pontos API-ja a docs.rs-en ellenőrizhető: https://docs.rs/syn, https://docs.rs/quote.
- A fordítói hibaüzenetek szövege a rustc és a g++ verziójától függően változhat.

21 Haladó típusrendszeri minták és API-tervezés

21.1 Newtype, PhantomData, typestate és const generikusok

21.1.1 Motiváció

A C++ típusrendszere sok esetben túl megengedő. Ha egy függvény métert és másodpercet vár, akkor is elfogad két `double` értéket, ha véletlenül felcseréljük őket. Hasonlóképpen egy `std::string` azonosító és egy felhasználónév ugyanaz a típus, ezért a fordító nem jelez, ha a kettőt összekeverjük. A sablonok és a `std::array` méretei fordítási időben ismertek, de a típusok közötti kapcsolatokat – például azt, hogy egy mátrixszorzás csak akkor lefordítható, ha a belső dimenziók egyeznek – nem mindig lehet természetesen kifejezni. Végül a C++ `std::true_type`/`std::false_type` tag dispatch technikája működik, de zajos és nem közvetlenül az állapotátmeneteket modellezi.

A Rust válasza: a típusrendszer aktív tervezési eszköz. A `newtype` minta új, egymással nem összekeverhető típusokat hoz létre; a `typestate` minta az objektum állapotát a típusparaméterbe kódolja; a `PhantomData` a birtoklási és variancia-információkat teszi explicitté; a `const generikusok` pedig a méreteket teszik a típus részévé. Ezáltal a hibás állapotátmenetek, a mértékegység-keverések és a rossz méretű mátrixok már fordítási időben kizárhatók.

21.1.2 Fogalmak és szabályok

21.1.2.1 Newtype

Definíció. A *newtype* olyan tuple-struct, amely pontosan egy mezőt tartalmaz, és célja, hogy a belső típusból új, elkülönült típust hozzon létre anélkül, hogy futásidőben extra költséget jelentene.

Szabály. A newtype típus nem örökli a belső típus trait-implementációit; a kívánt viselkedést `derive`-tal vagy kézi delegálással kell biztosítani. A newtype megkerüli az orphan rule-t, mert a saját típusunkra idegen traitet is implementálhatunk.

A mértékegység-biztonság érdekében a newtype a leghatékonyabb eszköz: `Meters(f64)` és `Seconds(f64)` külön típusok, így az `m + s` kifejezés fordítási hibát ad.

21.1.2.2 Typestate

Definíció. A *typestate* olyan programtervezési minta, amelyben az objektum állapota a típus paramétere, és az állapotváltó műveletek új típusú értéket adnak vissza, miközben a régi értéket elfogyasztják.

Szabály. A `Connection<S>` generikus típus `S` paramétere jelöli az állapotot (`Closed`, `Open`). Az `impl Connection<Closed>` blokkban definiált `open` metódus `Connection<Open>`-t ad vissza, az `impl Connection<Open>` blokkban definiált `close` pedig `Connection<Closed>`-t. A `send` csak

a `Connection<Open>` impl blokkban létezik, így a `Connection<Closed>`-on való hívása fordítási hibát ad (E0599, „method not found”).

21.1.2.3 PhantomData

Definíció. A `PhantomData<T>` zéró méretű marker típus, amely azt jelzi a fordítónak, hogy a struktúra úgy viselkedik, mintha `T` típusú értéket birtokolna – anélkül, hogy valóban tárolná.

Szabály. Ha egy `struct` típusparamétert vagy lifetime-ot deklarál, de az nem jelenik meg a mezőiben, akkor `PhantomData` kell. A választott alak három dolgot dönt el egyszerre: a *varianciát* (milyen altípusképzés megengedett), a *birtoklási szándékot* (a drop check szempontjából a típus birtokolja-e a `T`-t), és az *auto trait-eket* (`Send`, `Sync`).

Variancia röviden. Az `'a: 'b` jelentése: az `'a` legalább addig él, mint a `'b`. *Kovariáns* típusnál a hosszabb élettartamú érték használható rövidebb helyett (`&'long str` → `&'short str`); *kontravariánsnál* fordítva; *invariánsnál* egyik irány sem megengedett.

Típus	Variancia `T` felett	Birtoklás (drop check szándék)	`Send` / `Sync`
`PhantomData<T>`	kovariáns	igen, birtokolja a `T`-t	pontosan akkor, ha `T` az
`PhantomData<&'a T>`	kovariáns (`'a` és `T`)	nem	`Send`/`Sync`, ha `T: Sync`
`PhantomData<&'a mut T>`	`'a` felett kovariáns, `T` felett **invariáns**	nem	`Send`, ha `T: Send`; `Sync`, ha `T: Sync`
`PhantomData<*const T>`	kovariáns	nem	sem `Send`, sem `Sync`
`PhantomData<*mut T>`	**invariáns**	nem	sem `Send`, sem `Sync`
`PhantomData<fn() -> T>`	kovariáns	nem	mindig `Send` és `Sync`
`PhantomData<fn(T)>`	**kontravariáns**	nem	mindig `Send` és `Sync`
`PhantomData<fn(T) -> T>`	**invariáns**	nem	mindig `Send` és `Sync`
`PhantomData<Cell<T>>`	**invariáns**	igen (a `Cell` tartalmazza a `T`-t)	`Send`, ha `T: Send`; **nem** `Sync`

Gyakori tévedés, hogy a `PhantomData<fn(T)>` invariáns: a függvény *paramétere* kontravariáns pozíció, csak a `fn(T) -> T` (paraméter és visszatérés együtt) invariáns. Ha invariáns jelölő kell, amely nem korlátozza az auto trait-eket, a `PhantomData<fn(T) -> T>` a szokásos választás.

A táblázat variancia- és `Send`/`Sync`-oszlopát lefordítással ellenőriztük. A pozitív eset (a hosszabb élettartam rövidebbként használható) kovariáns típusra fordul, az invariánsra, a kontravariánsra és a fordított irányra nem:

Kódblokk — fájl: `src/main.rs` (önálló `cargo new` projekt) · művelet: teljes csere · ellenőrzés: `cargo build` (a fordítónak hibátlanul le kell fordulnia)

```
use std::marker::PhantomData;

struct Cov<T>(PhantomData<T>);
struct Contra<T>(PhantomData<fn(T)>);
```

```rust
// Pozitív: kovariáns típus hosszabb élettartama rövidebbként is használható.
fn shorten<'long: 'short, 'short>(x: Cov<&'long str>) -> Cov<&'short str> {
    x
}

// Pozitív: kontravariáns típusnál az ellenkező irány megengedett.
fn lengthen<'long: 'short, 'short>(x: Contra<&'short str>) -> Contra<&'long str> {
    x
}

fn main() {}
```

Negatív esetek: ugyanez a `shorten` az `Inv<T>(PhantomData<fn(T) -> T>)`-re, a `Contra`-ra, a `lengthen` a `Cov`-ra, valamint a `*mut T` mezőt tartalmazó struktúrákra is `error: lifetime may not live long enough` hibával áll meg. A birtoklás oszlopot a szándékot jelző `PhantomData<T>` adja: a drop check hatása stabil Rustban csak a `Drop`-ot implementáló típusokon látszik, a `#[may_dangle]` instabil, ezért ezt a táblázatban nem tesztelhető állításként, hanem a Nomicon szándéka szerint közöljük.

21.1.2.4 Const generikusok

Definíció. A *const generikus* olyan generikus paraméter, amely értéket (pl. `usize` méretet) vesz fel, nem típust. A Rust a `const` kulcsszóval jelöli, például `struct Matrix<T, const R: usize, const C: usize>`.

Szabály. A const generikusok a típus részévé teszik a méretet, így a fordító ellenőrizni *tudja* a méretkompatibilitást, de csak akkor, ha az műveletek aláírása ugyanazt a const paramétert használja az összekapcsolódó helyeken (például a szorzás belső dimenziója mindkét operandusban `K`). Például a `[[T; C]; R]` mezőben `R` és `C` a típusparaméterek, így minden `(R, C)` kombináció külön típust alkot.

21.1.3 C++ és Rust összevetés

21.1.3.1 Strong typedef és newtype

C++-ban a *strong typedef* általában egy wrapper osztályt jelent:

```cpp
class Meters {
    double v_;
public:
    explicit Meters(double v) : v_(v) {}
    double value() const { return v_; }
};

Meters operator+(Meters a, Meters b) {
    return Meters(a.value() + b.value());
}
```

Rustban ugyanez a newtype minta:

```rust
#[derive(Debug, Clone, Copy, PartialEq)]
struct Meters(f64);

impl std::ops::Add for Meters {
    type Output = Self;
    fn add(self, rhs: Self) -> Self {
        Meters(self.0 + rhs.0)
```

```
    }
}
```

A két minta lényegében egyenértékű: a C++ `explicit` konstruktor is megakadályozza az implicit konverziót, és a fordító által generált másoló konstruktor a wrapper osztályt is másolhatóvá teszi. A különbség inkább az, hogy a Rust-ban a `derive` explicit módon, egy sorban kéri a `Clone`, `Copy`, `PartialEq` képességeket, és a `Meters` alapból nem kap aritmetikát, azt trait-implementációval (`Add`) adjuk meg; a C++ operátort szabad függvényként is lehet írni.

A reprezentációról: egy egymezős tuple struct ábrázolása a gyakorlatban megegyezik a mezőével, de a nyelv ezt csak `#[repr(transparent)]` esetén **garantálja** (méret, igazítás és ABI azonos a mezőével; ezért FFI-n át is átadható). Ezért érdemes a newtype-ra kitenni: `#[repr(transparent)] struct Meters(f64);`. Ellenőriztük: `size_of::<Meters>() == size_of::<f64>()` és az igazítás is egyezik.

21.1.3.2 Tag dispatch és typestate

C++-ban a tag dispatch funkciótúlterheléssel és tag típusokkal oldjuk meg:

```cpp
template <typename T>
void process(T x, std::true_type) { /* integral */ }

template <typename T>
void process(T x, std::false_type) { /* not integral */ }

template <typename T>
void process(T x) {
    process(x, std::is_integral<T>{});
}
```

Rustban a typestate minta az állapotokat típusparaméterként kezeli, és a metódusok rendelkezésre állása az állapottól függ. Ez közvetlenebbül fejezi ki az állapotgépet, és a hibás állapotátmenet nem fordítható le.

21.1.3.3 Const generikusok és `std::array`

C++-ban a méretet sablonparaméterként adjuk meg:

```cpp
template <typename T, std::size_t R, std::size_t C>
struct Matrix {
    std::array<std::array<T, C>, R> data;
};
```

Rustban a const generikus ugyanezt a célt szolgálja:

```
struct Matrix<T, const R: usize, const C: usize> {
    data: [[T; C]; R],
}
```

A Rust verzióban a `Matrix<f64, 3, 4>` és a `Matrix<f64, 4, 3>` különböző típus, de ettől a szorzás még nem lesz ellenőrzött: a méretkompatibilitást a `mul` **aláírása** kényszeríti ki, ha a közös (belső) dimenzió ugyanaz a const paraméter a két operandusban. A C++ sablonnal ugyanez megírható; a különbség nem a lehetőségben, hanem abban van, hogy a Rustban a `where`-/const-határok a hibát a definíciónál, érthetőbb üzenettel jelzik.

Kódblokk — fájl: `src/main.rs` (önálló `cargo new` projekt) · művelet: teljes csere · ellenőrzés: `cargo run` kimenete `[[58.0, 64.0], [139.0, 154.0]]`

```rust
use std::ops::Mul;

#[derive(Debug, Clone, Copy, PartialEq)]
struct Matrix<const R: usize, const C: usize> {
    data: [[f64; C]; R],
}

// (R×K) · (K×C) = (R×C): a K a két operandusban ugyanaz a típusparaméter.
impl<const R: usize, const K: usize, const C: usize> Mul<Matrix<K, C>> for Matrix<R, K> {
    type Output = Matrix<R, C>;

    fn mul(self, rhs: Matrix<K, C>) -> Matrix<R, C> {
        let mut data = [[0.0; C]; R];
        for i in 0..R {
            for j in 0..C {
                for k in 0..K {
                    data[i][j] += self.data[i][k] * rhs.data[k][j];
                }
            }
        }
        Matrix { data }
    }
}

fn main() {
    let a = Matrix::<2, 3> { data: [[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]] };
    let b = Matrix::<3, 2> { data: [[7.0, 8.0], [9.0, 10.0], [11.0, 12.0]] };
    let c: Matrix<2, 2> = a * b;
    println!("{:?}", c.data);
}
```

A rossz belső dimenziójú szorzás (`a * a`, ahol `a` 2×3-as) nem fordul le: `error[E0308]: mismatched types … expected3, found2‘(expected struct Matrix<3, _>,found struct Matrix<2, 3>‘). Ezt kipróbáltuk, a hiba a hívás helyén jelenik meg, nem futásidőben.

21.1.4 Kidolgozott példa: typestate Connection

Vegyünk egy hálózati kapcsolatot, amely lehet zárt (`Closed`) vagy nyitott (`Open`). Zárt állapotban az `open` hívható, de a `send` nem; nyitott állapotban a `send` és a `close` hívható.

```rust
use std::marker::PhantomData;

struct Closed;
struct Open;

struct Connection<State> {
    addr: String,
    state: PhantomData<State>,
}

impl Connection<Closed> {
    pub fn new(addr: impl Into<String>) -> Self {
        Connection {
            addr: addr.into(),
            state: PhantomData,
        }
    }

    pub fn open(self) -> Result<Connection<Open>, std::io::Error> {
        // A valóságban itt TCP-kapcsolatot nyitnánk.
        Ok(Connection {
```

```
            addr: self.addr,
            state: PhantomData,
        })
    }
}

impl Connection<Open> {
    pub fn send(&self, data: &[u8]) -> Result<usize, std::io::Error> {
        // Itt küldenénk a hálózaton keresztül.
        Ok(data.len())
    }

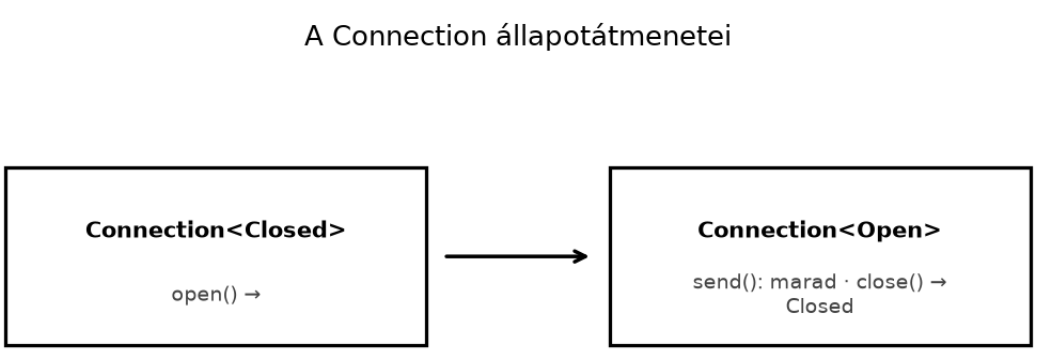
    pub fn close(self) -> Connection<Closed> {
        Connection {
            addr: self.addr,
            state: PhantomData,
        }
    }
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let conn = Connection::new("localhost:8080");
    // conn.send(b"hello")?; // Ez a sor fordítási hibát ad!
    let conn = conn.open()?;
    let n = conn.send(b"hello")?;
    println!("{n} bájt elküldve");
    let _conn = conn.close();
    Ok(())
}
```

A hibás sor kommentezve van. Ha ténylegesen beírjuk, a fordító a következőhöz hasonló hibát ad (a pontos szöveg verziófüggő):

```
error[E0599]: no method named `send` found for struct `Connection<Closed>` in the current scope
 --> src/main.rs:...
  |
  |     conn.send(b"hello")?;
  |          ^^^^ method not found in `Connection<Closed>`
```

A javítás: először `open`-nel nyissuk meg a kapcsolatot, és csak utána hívjuk a `send`-et, ahogy a fenti kódban tesszük.



21.1. ábra: A Connection állapotátmenetei - Zárt állapotból az open() nyit; nyitott állapotban a send() az állapotot nem változtatja, a close() visszavisz Closed-ba.

Az ábra szöveges megfelelője: `Closed` –`open()`→ `Open`; `Open` –`send()`→ `Open`; `Open` –`close()`→ `Closed`. A `Connection` típus állapotátmeneteit mutatja. A `send` csak az `Open` állapotban létezik, ezért a zárt állapotban való hívása fordítási hibát ad. Ez a typestate minta lényege: az állapotgép érvénytelen átmenetei a típusrendszer segítségével kizárhatók.

21.1.5 Tipikus hibák és átállási csapdák

1. **Új típus létrehozása `Deref` implementációval a belső típusra.** Bár a `Deref` kényelmes lehet, hogy a newtype gyakorlatilag úgy viselkedjen, mint a belső típus, ez megtöri az absztrakciót, és visszahozza az implicit konverzió problémáját. Helyette explicit metódusokat vagy a `From`/`Into` konverziókat használjunk, ha szükséges.
2. **A `PhantomData` elfelejtése.** Ha egy struktúra típusparamétert deklarál, de nem használja azt egyetlen mezőben sem, a fordító a „parameter `T` is never used” hibát ad. A megoldás egy `PhantomData<T>` mező felvétele. Hasonlóan, ha a lifetime paraméter nem jelenik meg, a `PhantomData<&'a ()>` jelzi az élettartam-kötést.
3. **A const generikusok túlzott használata.** Ha minden méretet const generikussá teszünk, a monomorfizáció miatt sok különböző típus jön létre, ami növeli a fordítási időt és a bináris méretét. Csak akkor használjunk const generikust, ha a méret valóban a típus részét kell képezze (pl. mátrixműveletek, fix méretű pufferek), és ne akkor, ha a méret csak futásidőben ismert.
4. **A `typestate` és az `enum` összekeverése.** Bár az `enum` is modellezhet állapotokat, a typestate minta előnye, hogy a hibás metódushívás már fordítási időben kiderül, míg az `enum` esetén a `match` futásidőben kezeli az állapotokat. A typestate különösen akkor hasznos, ha az állapotokhoz eltérő metóduskészletek tartoznak.

21.1.6 Feladatok

21.1.6.1 a) Lefordul-e és miért?

Az alábbi kód szándékosan tartalmaz egy hibát. Fordulna-e le? Ha nem, miért? Mi a pontos hibakód?

```
use std::marker::PhantomData;

struct Closed;
struct Open;

struct Connection<S> {
    state: PhantomData<S>,
}

impl Connection<Closed> {
    fn new() -> Self {
        Connection { state: PhantomData }
    }
    fn open(self) -> Connection<Open> {
        Connection { state: PhantomData }
    }
}

impl Connection<Open> {
    fn send(&self) {}
    fn close(self) -> Connection<Closed> {
```

```
        Connection { state: PhantomData }
    }
}

fn main() {
    let conn = Connection::<Closed>::new();
    conn.send();
}
```

21.1.6.2 b) C++ kód átírása idiomatikus Rustra

Adott a következő C++ `strong typedef` osztály:

```
class UserId {
    int value_;
public:
    explicit UserId(int v) : value_(v) {}
    int value() const { return value_; }
    bool operator==(UserId rhs) const { return value_ == rhs.value_; }
};
```

Írj Rust newtype-ot, amely `UserId(u32)` néven definiálja a típust, `Copy`, `Clone`, `Debug`, `PartialEq`, `Eq`, `Hash` és `Ord` tulajdonságokkal, és adj hozzá egy `From<UserId> for u32` és `From<u32> for UserId` konverziót! Mutasd meg, hogyan lehet `UserId` értéket `HashMap` kulcsaként használni.

21.1.6.3 c) Implementálj Session typestate-et az rkv-serverben

Önálló typestate-gyakorlat. A `Session` nem része a hálózati szerver (Az rkv hálózati szerver: TCP, protokoll, kecses leállás) feldolgozási útjának: a `handle_client` közvetlenül a `SharedStore`-t hívja, így a záróprojekt szervere hitelesítés nélkül fogad parancsot. A feladat a típusállapotok tervezését gyakoroltatja; a protokoll átvezetése (hitelesítő parancs, a végrehajtás minden útja a `Session<Authenticated>`-en át) külön, nem kötelező bővítés.

Az `rkv-server` crate-ben hozz létre egy `session` modult és benne egy `Session<State>` típust, amely egy `tokio::net::TcpStream`-et és egy állapotot tárol. Az állapotok: `Anonymous` és `Authenticated { user: String }`. Az `Anonymous` állapotban a `new` és az `authenticate` metódusok, az `Authenticated` állapotban a `user`, a `run_command` és a `logout` metódusok érhetők el. Az `authenticate` ellenőrizze a jelszót (konstans felhasználónév/jelszó pár elég), sikerrel `Session<Authenticated>`-et adjon; hibás jelszó esetén a névtelen munkamenetet a hibával együtt adja vissza, hogy a kliens újra próbálkozhasson. A `run_command` a megosztott `SharedStore`-on hajtsa végre a parancsot. Gondoskodj arról, hogy a `logout` után a `user` hívása fordítási hiba legyen, és ezt `compile_fail` doctesttel igazold. Ez a feladat előkészíti az API-tervezés Rustban alfejezetben tárgyalt ergonomikus API-kat.

21.1.6.4 d) Refaktorálás: PhantomData hozzáadása

Adott egy nyers mutatót tároló struktúra, amelyből hiányzik a `PhantomData`:

```
struct RawBuffer<T> {
    ptr: *mut T,
    len: usize,
}
```

Ez a kód lefordul? Ha nem, miért? Ha igen, milyen problémák merülhetnek fel a varianciával és a birtoklással kapcsolatban? Írd át a struktúrát úgy, hogy a típus helyesen jelezze a birtoklást és a varianciát! Milyen `PhantomData` típust választanál, ha a `RawBuffer` a `T`-t birtokolja, és miért?

21.1.7 Megoldások

21.1.7.1 a) Lefordul-e és miért?

Nem fordul le. A `Connection<Closed>` típuson nincs `send` metódus, mert az csak a `Connection<Open>` impl blokkban van definiálva. A hiba kódja E0599 (method not found). A megoldás az, hogy előbb `open`-nel nyissuk meg a kapcsolatot, és csak utána hívjuk a `send`-et, vagy távolítsuk el a hibás hívást.

21.1.7.2 b) C++ kód átírása idiomatikus Rustra

```rust
use std::collections::HashMap;

#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, PartialOrd, Ord)]
struct UserId(u32);

impl From<UserId> for u32 {
    fn from(id: UserId) -> Self {
        id.0
    }
}

impl From<u32> for UserId {
    fn from(v: u32) -> Self {
        UserId(v)
    }
}

fn main() {
    let mut map = HashMap::new();
    let id = UserId::from(42);
    map.insert(id, "Alice");
    let raw: u32 = id.into();
    println!("{raw}");
    assert_eq!(map.get(&UserId(42)), Some(&"Alice"));
}
```

A `derive(Eq, Hash, Ord)` lehetővé teszi a `HashMap` kulcsként való használatot. A `From` implementációk explicit konverziót biztosítanak a newtype és a belső típus között.

21.1.7.3 c) Session typestate az rkv-serverben

A munkamenet a szerver tokio-s típusaira épül: a socket `tokio::net::TcpStream`, a parancsot a megosztott `SharedStore` hajtja végre. Mivel a hitelesítés itt csak típusállapot-átmenet, a példa szinkron `authenticate`-et használ; az olvasás és az írás a meglévő async szerverkódban marad.

Kódblokk — fájl: `rkv-server/src/session.rs` · művelet: új fájl · ellenőrzés: `cargo test -p rkv-server` (a két `compile_fail` doctest is)

```rust
//! Kapcsolat-állapot típusként (typestate): névtelen és hitelesített munkamenet.

use std::fmt;
use std::io;
use std::net::SocketAddr;

use rkv_core::{Command, Response, SharedStore};
use tokio::net::TcpStream;

/// Állapotjelző: a kliens még nem azonosította magát.
pub struct Anonymous;

/// Állapotjelző: a kliens hitelesített, a felhasználónév az állapot része.
pub struct Authenticated {
    user: String,
}

/// Egy klienskapcsolat; az `State` típusparaméter az állapotot kódolja.
pub struct Session<State> {
    conn: TcpStream,
    state: State,
}

// Oktatási célú, rögzített hitelesítő adatok: valódi rendszerben tárolt, sózott hash kell.
const USER: &str = "admin";
const PASSWORD: &str = "secret";

impl Session<Anonymous> {
    pub fn new(conn: TcpStream) -> Self {
        Session { conn, state: Anonymous }
    }

    /// Siker esetén `Session<Authenticated>`. Hiba esetén a névtelen munkamenet
    /// visszajár a hibával együtt, így a kliens újra próbálkozhat.
    pub fn authenticate(
        self,
        user: &str,
        password: &str,
    ) -> Result<Session<Authenticated>, (Session<Anonymous>, AuthError)> {
        if user == USER && password == PASSWORD {
            Ok(Session {
                conn: self.conn,
                state: Authenticated { user: user.to_owned() },
            })
        } else {
            Err((self, AuthError::InvalidCredentials))
        }
    }
}

impl Session<Authenticated> {
    pub fn user(&self) -> &str {
        &self.state.user
    }

    /// A parancsot a megosztott tárolón hajtja végre; csak hitelesített munkamenet hívhatja.
    pub fn run_command(&self, store: &SharedStore, command: &Command) ->
     ↪  rkv_core::Result<Response> {
        store.apply(command)
    }

    /// Elfogyasztja a hitelesített munkamenetet, és névtelent ad vissza.
    pub fn logout(self) -> Session<Anonymous> {
```

```
        Session { conn: self.conn, state: Anonymous }
    }
}

impl<State> Session<State> {
    pub fn peer_addr(&self) -> io::Result<SocketAddr> {
        self.conn.peer_addr()
    }

    /// A mögöttes socket visszaadása (például a válasz megírásához).
    pub fn into_stream(self) -> TcpStream {
        self.conn
    }
}

#[derive(Debug, PartialEq, Eq)]
#[non_exhaustive]
pub enum AuthError {
    InvalidCredentials,
}

impl fmt::Display for AuthError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            AuthError::InvalidCredentials => write!(f, "invalid credentials"),
        }
    }
}

impl std::error::Error for AuthError {}

/// ```compile_fail,E0599
/// use rkv_server::session::{Authenticated, Session};
///
/// fn demo(session: Session<Authenticated>) {
///     let anonymous = session.logout();
///     anonymous.user(); // a `Session<Anonymous>`-nek nincs `user` metódusa
/// }
/// ```
///
/// ```compile_fail,E0382
/// use rkv_server::session::{Authenticated, Session};
///
/// fn demo(session: Session<Authenticated>) {
///     let _anonymous = session.logout();
///     session.user(); // a `logout` elfogyasztotta a hitelesített munkamenetet
/// }
/// ```
#[cfg(doctest)]
struct _TypestateMisuse;
```

A `#[cfg(doctest)]` jelölésű üres struktúra csak a dokumentációs példák hordozója: a `compile_fail` doctestek fordítási hibát várnak, és csak `cargo test --doc` alatt fordulnak.

A modult a `lib.rs` elején kapcsoljuk be, hogy a doctestek és az integrációs tesztek is elérjék:

Kódblokk — fájl: `rkv-server/src/lib.rs` · művelet: új sor a fájl elejére · ellenőrzés: `cargo build -p rkv-server`

```
pub mod session;
```

Az állapotátmenetek tesztje valódi loopback-kapcsolaton fut. A hibás jelszó a névtelen munkamenetet adja vissza, amellyel újra lehet próbálkozni:

Kódblokk — fájl: `rkv-server/src/session.rs` · művelet: új tesztmodul a fájl végére · ellenőrzés: `cargo test -p rkv-server --lib session`

```
#[cfg(test)]
mod tests {
    use super::*;
    use tokio::net::TcpListener;

    async fn connected() -> (TcpStream, TcpStream) {
        let listener = TcpListener::bind("127.0.0.1:0").await.unwrap();
        let addr = listener.local_addr().unwrap();
        let (client, server) = tokio::join!(TcpStream::connect(addr), listener.accept());
        (client.unwrap(), server.unwrap().0)
    }

    #[tokio::test]
    async fn wrong_password_returns_the_anonymous_session() {
        let (_client, server) = connected().await;
        let session = Session::new(server);
        let (session, err) = match session.authenticate("admin", "nope") {
            Ok(_) => panic!("a hibás jelszót el kellett utasítani"),
            Err(pair) => pair,
        };
        assert_eq!(err, AuthError::InvalidCredentials);
        // Ugyanazzal a (névtelen) munkamenettel újra lehet próbálkozni.
        let session = session.authenticate("admin", "secret").ok().unwrap();
        assert_eq!(session.user(), "admin");
    }

    #[tokio::test]
    async fn authenticated_session_runs_commands_and_logs_out() {
        let (_client, server) = connected().await;
        let store = SharedStore::new(rkv_core::MemStore::new());
        let session = Session::new(server).authenticate("admin", "secret").ok().unwrap();
        let set = Command::Set("k".into(), "v".into());
        assert!(matches!(session.run_command(&store, &set), Ok(Response::Ok)));
        assert_eq!(store.get("k").unwrap(), Some(b"v".to_vec()));
        let anonymous = session.logout();
        assert!(anonymous.peer_addr().is_ok());
    }
}
```

Ellenőrzési pont. `cargo test -p rkv-server` zöld (a `session` két tesztje és a két `compile_fail` doctest), a `cargo clippy -p rkv-server --all-targets -- -D warnings` figyelmeztetés nélkül lefut. A két doctestet külön lefordítva a várt hibát kapjuk: `E0599` (`no method nameduserfound for structSession<Anonymous>)`, `illetve` `` `E0382` (`borrow of moved value: `session`). A típusrendszer tehát garantálja az állapotok sorrendjét: `user` csak hitelesített állapotban hívható, a `logout` pedig elfogyasztja a hitelesített munkamenetet.

21.1.7.4 d) Refaktorálás: PhantomData hozzáadása

Az eredeti `RawBuffer<T>` **lefordul**: a `T` megjelenik a `*mut T` mező típusában, így nincs „unused parameter” hiba. Hiányzik viszont a birtoklási szándék, és a viselkedése a táblázatból levezethető: a `*mut T` **invariáns** `T` felett (nem kovariáns, ahogy gyakran hiszik), ezért a `RawBuffer<&'long str>` nem használható `RawBuffer<&'short str>`-ként, és a nyers mutató miatt a típus sem `Send`, sem `Sync`. Ha a `RawBuffer` birtokolja a `T`-t, a megfelelő szerkezet a `Vec`-é: `NonNull<T>` (kovariáns, mint a `*const T`) és `PhantomData<T>` (birtoklási szándék; a drop check és az auto trait-ek `T`-hez kötése).

Kódblokk — fájl: `src/lib.rs` · művelet: a `RawBuffer` cseréje · ellenőrzés: `cargo build`

```
use std::marker::PhantomData;
use std::ptr::NonNull;

struct RawBuffer<T> {
    ptr: NonNull<T>,
    len: usize,
    _owns: PhantomData<T>, // birtokolja a T-t; kovariáns T felett
}

// Pozitív: a hosszabb élettartam rövidebbként is használható.
fn shorten<'long: 'short, 'short>(b: RawBuffer<&'long str>) -> RawBuffer<&'short str> {
    b
}
```

A `NonNull<T>` a `*mut T`-vel ellentétben kovariáns, ezért a `shorten` lefordul (ellenőrizve). A `*mut T` mezővel írt változatnál ugyanez `lifetime may not live long enough` hibát ad. A `NonNull` miatt a típus továbbra sem `Send`/`Sync` (a fordító `NonNull<u8> cannot be sent between threads safely` hibát ad); ha a szándék az, hogy a `RawBuffer<T> Send` legyen, ha `T: Send`, azt `unsafe impl<T: Send> Send for RawBuffer<T> {}` mondja ki, és az `unsafe` kötelezettségét a szerző vállalja. A `PhantomData<Cell<T>>` ezzel szemben invariánssá tenné a típust és kizárná a `Sync`-et; akkor érdemes, ha a belső mutabilitás miatt éppen ez a cél.

21.1.8 Rövid összefoglaló

Ebben az alfejezetben a Rust típusrendszerének haladó eszközeit tanulmányoztuk. A `newtype` minta tuple-struct egyetlen mezővel új, típusbiztos típust hoz létre, amely megkerüli az orphan rule-t, és nem örökli a belső típus traiteket. A `typestate` minta az állapotot típusparaméterként kezeli: a `Connection<Closed>` és `Connection<Open>` típusokra külön impl blokkok biztosítják a megfelelő metódusokat, így a hibás állapotátmenetek fordítási hibát adnak (E0599). A `PhantomData<T>` zéró méretű marker, amely a birtoklást, az élettartamot és a varianciát jelzi, a választott alak dönti el a varianciát (`PhantomData<T>` kovariáns, `PhantomData<fn(T)>` kontravariáns, `PhantomData<fn(T) -> T>` és `PhantomData<Cell<T>>` invariáns) és az auto trait-eket. A const generikusok, mint a `Matrix<T, const R: usize, const C: usize>`, a méreteket a típus részévé teszik, így a mátrixműveletek méretkompatibilitása fordítási időben ellenőrizhető. **Továbbvitt eredmények:** a `typestate` minta és a `Session<State>` típus (`rkv-server/src/session.rs`) előkészíti az API-tervezés Rustban alfejezet ergonomikus API-irányelveit, valamint az no_std, embedded-hal és Embassy beágyazott állapotgépeit.

21.1.9 Források és ellenőrzési hiányok

- Ellenőrizve (Rust 1.99.1): a variancia-táblázat variancia- és `Send`/`Sync`-oszlopa lefordítással (pozitív és negatív `shorten`/`lengthen` esetek); az `rkv-server session` modulja `cargo test` és `clippy -D warnings` alatt, a két `compile_fail` doctest és a várt `E0599`/`E0382` hibák.
- Nem tesztelhető stabil Rustban: a `PhantomData<T>` drop check hatása (a `#[may_dangle]` instabil); ezt a Nomicon leírása szerint közöljük.
- A nyelvi szabályok forrása a Rust Reference variancia-fejezete és a Nomicon; a hibaüzenetek szövege verziófüggő, a hibakódok (E0599, E0382) stabilak.

21.2 API-tervezés Rustban: irányelvek és ergonómia

A C++-ban az API tervezését gyakran a fejlécekben deklarált függvénytúlterhelések, az implicit konverziók és a default paraméterek alakítják. A hívó oldal kényelmét szolgálják, de könnyen rejtett költségekhez, félreérthető túlterhelési feloldáshoz vagy véletlen másolásokhoz vezetnek. A Rust ezeket az eszközöket szándékosan elhagyja: nincs függvénytúlterhelés, nincs implicit konverzió, nincs default paraméter. Helyette a nyelv és a közösségi irányelvek explicit, kiszámítható mintákat adnak: konverziós trait-ek, builder típusok, kölcsönzésbarát paraméterek. Ez az alfejezet az rkv-core nyilvános API-ját vizsgálja felül ezen irányelvek szerint, és előkészíti a Záróprojekt: az rkv teljes rendszerként alfejezetet.

21.2.1 Fogalmak és szabályok

Definíció. Az *API Guidelines* a Rust közösség által karbantartott, idiomatikus könyvtár-API-kra vonatkozó ajánlásgyűjtemény. Nem nyelvi szabály, de a `cargo` ökoszisztéma, a dokumentáció és a hívók elvárása erre épül. A legfontosabb területek: elnevezés, konverziók, paramétertípusok, hibakezelés, jövőbiztosság.

Szabály. Az elnevezési konvenciók: - típusok és trait-ek: `UpperCamelCase` (`Store`, `Storage`, `IntoIterator`); - függvények, metódusok, változók, modulok, crate-ek: `snake_case` (`get_value`, `parse_command`); - konstansok és statikusok: `SCREAMING_SNAKE_CASE` (`MAX_CAPACITY`); - a `get_` előtagot kerüljük, ha a metódus egyszerű mező-hozzáférést ad; a `get` C++-os szokás, Rustban inkább `key()` vagy `value()` használatos, a `get` fenntartott a `Option`-t visszaadó indexelésre.

Szabály. A konverziós metódusok előtagjai: - `as_` olcsó, kölcsönzött nézetet ad vissza: `as_str()`, `as_bytes()`, `as_ref()`. Nem fogyasztja a hívót. - `to_` hagyományosan tulajdonjogot adó, gyakran drágább konverziót jelöl: `to_string()`, `to_owned()`, `to_vec()`. - `into_` a hívót elfogyasztó, tulajdonjogot adó konverzió: `into_bytes()`, `into_iter()`. A C++-ban megszokott `get`/`to` keveréke helyett ez a három előtag egyértelműen jelzi a költségmodellt. A `to_` azonban **konvenció, nem visszatérési típusgarancia**: a `Path::to_str()` például kölcsönzött `Option<&str>`-t ad (történelmi kivétel), a `str::to_lowercase()` viszont allokál, a `String::into_bytes()` pedig a hívót fogyasztva ad tulajdonolt `Vec<u8>`-ot. A tényleges költséget és tulajdonviszonyt mindig a szignatúra és a dokumentáció mondja meg, az előtag csak a szándékot jelzi.

Definíció. A konverziós trait-ek: - `From<T> for U`: párhuzamos az implicit C++-konverzióval, de explicit hívást igényel (`U::from(t)` vagy `t.into()`). Implementálása nem okozhat hibát és nem allokálhat indokolatlanul. Ha `From<T>`-t implementáljuk, az `Into<U>` automatikusan rendelkezésre áll. - `AsRef<T>`: kölcsönzött referenciává alakítás olcsón, például `String` → `&str`, `Vec<u8>` → `&[u8]`. Függvényparaméterben `impl AsRef<str>` elfogad `&str`, `String`, `Cow<str>` értékeket is. - `Borrow<T>`: hasonló, de elsősorban gyűjtemény-kulcsokhoz (`HashMap`-keresés) és a `Cow`-hoz; a `Borrow<str>` a `String`-re garantálja, hogy a hash és az egyenlőség megegyezik.

Szabály. Paramétertípusok: - szöveg esetén `&str`, bájtsorozat esetén `&[T]`, hacsak a függvény nem szerzi meg a tulajdonjogot; - tulajdonjog megszerzésekor `String`, `Vec<T>` vagy `impl Into<String>` — ez utóbbi a hívónak rugalmasságot ad; - az `&String` és `&Vec<T>` paramétert általában kerüljük (a `clippy::ptr_arg` is ezt jelzi): az `&str` és `&[T]` szigorúan általánosabb, mert a deref coercion miatt a hívó mindkettőből átadhatja a sajátját. Ez irányelv, nem nyelvi tiltás, és a célja a hívók rugalmassága; kivétel, ha a függvénynek a tároló-specifikus API-ra van szüksége (például `s.capacity()`, ami a `String`-en létezik, a `&str`-en nem: `fn spare(s: &String) -> usize { s.capacity() - s.len() }`, a clippy ezt nem jelzi). A `&Option<T>` is gyanús, mert a hívónak `Option<&T>`-t kellene építenie (`opt.as_ref()`), de itt is a hívási minta dönt; - gyűjteményt fogyasztó függvénynél `impl IntoIterator<Item = T>` a legáltalánosabb.

Definíció. A *builder minta* olyan típus, amely lépésenként gyűjti a konfigurációt, majd egy `build()` metódussal hozza létre a céltípust. Két változata: - *consuming builder*: minden setter `self`-et fogyaszt és `Self`-et ad vissza, így láncolható, és a `build(self)` az értékeket kimozgathatja (nincs klónozás); a builder egyszer használható. Ilyen az rkv `ServerConfigBuilder`-e; - *`&mut` builder*: a setterek `&mut self`-et vesznek fel és `&mut Self`-et adnak vissza, tehát ez is láncolható, a builder pedig a `build(&self)` után újrahasználható; cserébe a `build` klónoz, ha nem mozgathat. A standard könyvtár `std::process::Command`-ja ilyen: `Command::new("echo").arg("hi").arg("there").output()`. Mindkettő idiomatikus; a választás szempontja a használat: az egyszeri összeállítást a consuming, az újrafelhasznált konfigurációt a `&mut` változat szolgálja jobban. A `&mut` builderre egy ismert csapda: a `let c = Command::new("echo").arg("x");` nem fordul le (`E0716: temporary value dropped while borrowed`), mert az ideiglenes builder a kifejezés végén megszűnik, miközben `c` még kölcsönzi; a láncot egy kifejezésben kell lezárni, vagy a buildert előbb változóba kell tenni.

Definíció. A `#[must_use]` attribútum arra figyelmezteti a hívót, hogy az érték eldobása valószínűleg hiba. Két helyre tehető, és a kettő különbözik:

- *típusra*: a típus minden előfordulása figyelmeztet, ha eldobják. A standard könyvtár a `Result`-ot így jelöli, ezért **minden `Result`-ot visszaadó függvény eldobott eredménye külön attribútum nélkül is `unused_must_use` figyelmeztetést ad**. Az `Option` típusa viszont *nincs* megjelölve, ezért az eldobott `Option` csendben marad;
- *függvényre*: csak az adott függvény eredményét védi. Erre ott van szükség, ahol a visszatérési típus nem must_use (`Option`, `bool`, `Vec`, egy tiszta számítás eredménye), vagy ahol az eldobásnak egyedi magyarázata van: `#[must_use = "indok"]`.

A függvény-attribútum tehát nem a `Result` védelmét pótolja, hanem azt, amit a típus nem fed le. A kifejezett eldobás (`let _ = f();`) mindkét esetben némán elfogadott.

Definíció. A `#[non_exhaustive]` attribútum egy enumon, structon vagy enumvariánson jelzi, hogy a típus a jövőben új variánsokkal vagy mezőkkel bővülhet. A hatása csak a **más crate-ben** lévő kódban érvényesül, és típusonként eltér:

Jelölés	Más crate-beli létrehozás	Más crate-beli minta
`#[non_exhaustive] enum E`	új variáns létrehozható, de a `match` nem lehet kimerítő	`match` kötelezően `_` (wildcard) ágat tartalmaz
`#[non_exhaustive] struct S`	`S { … }` struct-literál **tiltott**; csak konstruktoron át hozható létre	`S { … , .. }` kötelező, a mezők olvashatók
`#[non_exhaustive]` enumvariáns `E::V { … }`	maga a variáns **nem hozható létre** struct/tuple literállal	`E::V { .. }` kötelező a variáns mintájában

Ugyanabban a crate-ben a típus kimerítően illeszthető és literállal létrehozható; a korlát a workspace többi tagjára és minden külső crate-re vonatkozik. Az enum wildcard-kényszere és a struct literál-tilalma tehát két külön szabály, nem ugyanaz a mechanizmus.

Szabály. A szemantikus verziózás (semver) `X.Y.Z` alakú verziót használ. A kompatibilitási határ három esetre bomlik; a `0.0.Z` sor nem olvasztható be a `0.Y.Z` sorba:

	`X.Y.Z`, `X  1`	`0.Y.Z`, `Y > 0`	`0.0.Z`
törő változás	`X` emelése	`Y` emelése	`Z` emelése

	`X.Y.Z`, `X  1`	`0.Y.Z`, `Y > 0`	`0.0.Z`
kompatibilis bővítés (új függvény, új típus, új variáns egy `#[non_exhaustive]` enumban)	`Y` emelése	`Z` emelése	nincs külön kompatibilis bővítés: minden API-változás új `Z`
hibajavítás	`Z` emelése	`Z` emelése	`Z` emelése

A Cargo a `^`-kompatibilitást így értelmezi: a `^1.2.3` a `2.0.0`-t, a `^0.2.3` a `0.3.0`-t zárja ki, a `^0.0.3` pedig csak a `0.0.3`-at fogadja el (a `0.0.4` már nem kompatibilis vele). A semver specifikáció a `0.y.z` sort eleve fejlesztés alatt állónak tekinti, ahol bármi változhat; a Cargo ennél szigorúbb, de a `0.0.z`-nél a patch-emelést is törőnek veszi. A **projekt saját release-policyja** ettől eltérhet (például egy `0.x` sorban is vállalhat stabil API-t), de az a döntés a projekt dokumentációjában él, nem a Cargo feloldási szabályában. Törő változás például egy típus eltávolítása, a szignatúra megváltoztatása (a visszatérési típus is), és a `#[non_exhaustive]` **hozzáadása** egy már publikált, kimerítően illeszthető típushoz. Nem törő bővítés viszont egy már `#[non_exhaustive]` enum új variánsa. A `cargo-semver-checks` eszköz a publikált és az aktuális API összehasonlításával jelzi a törő változásokat; új trait-implementáció vagy egy új, alapértelmezett törzsű trait-metódus sem kivétel nélkül kompatibilis (névütközés, blanket impl ütközés, eddig egyértelmű metódushívás elromlása); az esetek nem egyszerű táblázat-keresések, hanem a konkrét hívók függvényei (például a `&String` → `&str` paramétercsere a legtöbb hívónak kompatibilis, a függvénypointerként használóknak nem).

21.2.2 C++ és Rust összevetés

C++-ban egy függvény rugalmasan fogadhat stringet:

```
#include <string>
#include <string_view>

void save(const std::string& path) { /* ... */ }
void save(std::string_view path) { /* ... */ }
void save(const char* path) { /* ... */ }
```

A túlterhelések a hívó kényelmét szolgálják, de a három változat fenntartása ismétlés, és a `std::string_view` élettartama a hívó felelőssége. Rustban nincs túlterhelés, ezért a paramétert úgy választjuk meg, hogy a lehető legtöbb hívási formát elfogadja:

```
use std::path::Path;

pub fn save(path: impl AsRef<Path>) -> std::io::Result<()> {
    let path = path.as_ref();
    // ...
    Ok(())
}
```

A hívó átadhat `&str`-et, `String`-et, `Path`-ot vagy `PathBuf`-ot, a konverzió explicit és olcsó. A különbség: a Rust az `impl AsRef`-fel egyetlen függvényben fejezi ki azt, amit C++-ban három túlterheléssel kell.

C++-ban az implicit konverzió gyakran meglepi a fejlesztőt:

```
void process(int value);
process(3.14);  // double -> int, esetleg figyelmeztetéssel
```

Rustban az implicit konverzió nem létezik, a `From`/`Into` explicit:

```rust
fn process(value: i32) {
    let _ = value;
}

fn main() {
    let x = 3.14_f64;
    process(x as i32);      // explicit cast
    process(x.round() as i32);
}
```

C++-ban a default paramétereket builder minta helyettesíti:

```cpp
void connect(const std::string& host, int port = 6379, int timeout_ms = 1000);

struct Connection;

struct ConnectionBuilder {
    port: u16,
    timeout_ms: u64,
}

impl Connection {
    fn builder() -> ConnectionBuilder {
        ConnectionBuilder { port: 6379, timeout_ms: 1000 }
    }
}

impl ConnectionBuilder {
    fn port(mut self, port: u16) -> Self {
        self.port = port;
        self
    }
    fn timeout(mut self, timeout_ms: u64) -> Self {
        self.timeout_ms = timeout_ms;
        self
    }
    fn build(self) -> Connection {
        let _ = (self.port, self.timeout_ms);
        Connection
    }
}

fn main() {
    let conn = Connection::builder()
        .port(6380)
        .timeout(500)
        .build();
    let _ = conn;
}
```

A builder explicit lépésekre bontja a konfigurációt, és a `build()` hibát is visszaadhat, ha az állapot érvénytelen.

21.2.3 Kidolgozott példa: az rkv-core nyilvános API-jának felülvizsgálata

Munkakönyvtár: `~/rust-labor/rkv/` — a workspace. A labor az eddig elkészült kód nyilvános API-ját veszi sorra az irányelvek szerint; a példák a valódi `rkv-core` és `rkv-server` kódból valók.

Az átvizsgálás három visszatérési szerződést talál a kulcs lekérdezésére, és mindegyiknek más a helyes alakja:

Típus	Aláírás	Miért
`Store`, `FileStore` (tiszta memóriabeli tábla)	`get(&self, &str) -> Option<&[u8]>`	nincs I/O-hiba, az érték a táblában él: kölcsönzés, nincs másolás
`Storage` trait (hibázhat, több implementáció)	`get(&self, &str) -> Result<Option<Vec<u8>>>`	az I/O hibázhat, a hátterek (memória, fájl, csak olvasható, később távoli) eltérően tárolnak, és a `SharedStore` zárhatára miatt az önálló (owned) eredmény a kényelmes közös szerződés. (Egy `fn get(&self, &str) -> Result<Option<&[u8]>>` is dyn-kompatibilis volna, az élettartam az elisióval a `self`-hez kötődne; a döntés tehát tervezési, nem nyelvi lehetetlenség.)
`SharedStore` (zár mögött)	`get(&self, &str) -> Result<Option<Vec<u8>>>`	a zár elengedése után a hivatkozás érvénytelen volna: owned eredmény kell

A hiányzó kulcs mindhárom helyen `None`, nem hiba (`Ok(None)`); a `KeyNotFound` hibát a hídrétegek (például a cxx-híd) képezik belőle.

A felülvizsgálat eredménye (a labor előtti állapotból):

1. `Store::get` és `FileStore::get` `Option<&Vec<u8>>`-t ad: a `&Vec<u8>` a tárolás módjához köti a hívót, a `&[u8]` nem. Átírjuk `Option<&[u8]>`-ra. Ez **törő**: a hívók `.cloned()`-ja és a tesztek `Some(&vec![..])` összehasonlításai nem fordulnak le.
2. A `ServerConfig` nyilvános mezői és `Default`-ja mellé builder kell, amely a `build()`-ben ellenőrzi, hogy a korlátok nem nullák és nem túl nagyok (a nulla `max_connections` használhatatlan szervert adna; a `usize::MAX` `max_frame` túlcsordítaná a `max_frame as u64 + 1` kifejezést); a nyilvános mezők a builder megkerülésével is beállíthatók, ezért ugyanezt a `ServerConfig::validate()` a szerver belépési pontján is ellenőrzi.
3. A `Store::get` eredménye nem `Result`, ezért a típus nem véd az eldobás ellen; `#[must_use]` kell rá.
4. Az `Error` és a `Command` bővíthető, ezért `#[non_exhaustive]`-re jelöljük. Ez a más crate-beli `match`-eket érinti.

Elkészített kód — az átírt `Store::get` és `get_by`:

Kódblokk — fájl: `rkv-core/src/store/mod.rs` · művelet: a `get` és a `get_by` cseréje · ellenőrzés: `cargo check -p rkv-core`

```
/// Egy kulcs értékének lekérdezése. `None`, ha nincs ilyen kulcs.
#[must_use = "a keresés eredménye nélkül a hívás hatástalan"]
pub fn get(&self, key: &str) -> Option<&[u8]> {
    self.data.get(key).map(Vec::as_slice)
}
```

```rust
/// Kényelmi keresés: a kulcs `&str`, `String`, `&String` vagy `Cow<str>` is lehet.
#[must_use = "a keresés eredménye nélkül a hívás hatástalan"]
pub fn get_by<K: AsRef<str>>(&self, key: K) -> Option<&[u8]> {
    self.get(key.as_ref())
}
```

A `FileStore::get` ugyanígy `Option<&[u8]>`-t ad (`self.data.get(key).map(Vec::as_slice)`).

A `ServerConfig` builder az `rkv-server`-ben. A setterek `#[must_use]`-ak, mert a `self`-et fogyasztják: az eldobott builder semmit sem állít be.

Kódblokk — fájl: `rkv-server/src/lib.rs` · művelet: új kód a `ServerConfig` `Default` implementációja után · ellenőrzés: `cargo test -p rkv-server --lib config`

```rust
impl ServerConfig {
    /// Builder: csak a megadott korlátok változnak, a többi az alapérték marad.
    pub fn builder() -> ServerConfigBuilder {
        ServerConfigBuilder::default()
    }
}

/// A `ServerConfig` lépésenkénti összeállítása; a `build` ellenőrzi az értékeket.
#[derive(Debug, Default)]
pub struct ServerConfigBuilder {
    max_frame: Option<usize>,
    max_connections: Option<usize>,
    shutdown_grace: Option<Duration>,
    max_response: Option<usize>,
}

impl ServerConfigBuilder {
    #[must_use]
    pub fn max_frame(mut self, bytes: usize) -> Self {
        self.max_frame = Some(bytes);
        self
    }

    #[must_use]
    pub fn max_connections(mut self, count: usize) -> Self {
        self.max_connections = Some(count);
        self
    }

    #[must_use]
    pub fn max_response(mut self, bytes: usize) -> Self {
        self.max_response = Some(bytes);
        self
    }

    #[must_use]
    pub fn shutdown_grace(mut self, grace: Duration) -> Self {
        self.shutdown_grace = Some(grace);
        self
    }

    /// Érvénytelen (nulla vagy túl nagy) korlátra hibát ad, nem indít használhatatlan szervert.
    pub fn build(self) -> Result<ServerConfig, ConfigError> {
        let default = ServerConfig::default();
        let config = ServerConfig {
            max_frame: self.max_frame.unwrap_or(default.max_frame),
            max_connections: self.max_connections.unwrap_or(default.max_connections),
            shutdown_grace: self.shutdown_grace.unwrap_or(default.shutdown_grace),
```

```
            max_response: self.max_response.unwrap_or(default.max_response),
        };
        // Ugyanaz az ellenőrzés fut a builderben és a `run_server_with` belépési pontján:
        // a nyilvános mezők a builder megkerülésével is beállíthatók.
        config.validate()?;
        Ok(config)
    }
}
```

A `ConfigError` típust és a `ServerConfig::validate` metódust a Az rkv hálózati szerver: TCP, protokoll, kecses leállás alfejezet kódja már tartalmazza (nulla és túl nagy korlát, például `max_frame = usize::MAX` esetén `ConfigError::TooLarge`), mert a szerver belépési pontja is érvényesíti.

21.2.4 Labor: rkv-core API-review checklista alapján

Munkakönyvtár: `~/rust-labor/rkv/` — a workspace.

Az ellenőrzőlista:

1. **Elnevezések:** minden nyilvános elem `UpperCamelCase` vagy `snake_case`; nincs `get_` előtag egyszerű mező-hozzáférésnél.
2. **Paramétertípusok:** az rkv nyilvános API-jában nincs `&String` vagy `&Vec<T>` paraméter, mert egyik függvénynek sincs szüksége tároló-specifikus metódusra; szöveg `&str`, bájtsorozat `&[u8]`. A visszatérésben is `&[u8]`, nem `&Vec<u8>`.
3. **Konverziók:** `From`/`Into` ott, ahol van értelme; `AsRef` a paraméterekben.
4. **Builder:** többopciós konfigurációhoz builder, `build()`-ben ellenőrzéssel.
5. **`#[must_use]`:** a `Result`-ot visszaadó függvények típusuk révén védettek; függvényszintű attribútum a nem védett típusokra (`Option`, `bool`, builder-`Self`) és egyedi indoklásra.
6. **`#[non_exhaustive]`:** a bővíthető nyilvános enumokra; a más crate-beli `match`-ek javítása és az ismeretlen eset meghatározott kezelése.
7. **Semver:** melyik változás törő; a `cargo-semver-checks` (publikált crate esetén).

Lépések:

1. Az `rkv-core/src/store/mod.rs`-ben és a `file_store.rs`-ben cseréljük a `get` és `get_by` visszatérési típusát `Option<&[u8]>`-ra a fenti kód szerint, és adjunk rájuk `#[must_use]`-t.
2. Migráljuk a hívókat. Három hely nem fordul le, mindegyik javítása egyszerű:

Kódblokk — fájl: `rkv-core/src/storage.rs` · művelet: a `.cloned()` hívások cseréje (a `Storage` trait owned `Vec<u8>`-t ad) · ellenőrzés: `cargo check -p rkv-core`

```
// MemStore és FileStore Storage-implementációja
Ok(MemStore::get(self, key).map(<[u8]>::to_vec))
Ok(FileStore::get(self, key).map(<[u8]>::to_vec))
// a list() szűrőjében
.filter_map(|k| MemStore::get(self, k).map(|v| (k.to_owned(), v.to_vec())))
```

Kódblokk — fájl: `rkv-cli/src/main.rs` · művelet: a másolás helyett kölcsönzött UTF-8 ellenőrzés · ellenőrzés: `cargo check -p rkv-cli`

```
Some(bytes) => match std::str::from_utf8(bytes) {
    Ok(text) => format!("VALUE {key} {text}"),
    Err(_) => format!("ERR value of {key} is not valid UTF-8"),
},
```

A tesztekben az összehasonlítás szeletre vált: `assert_eq!(store.get("kulcs"), Some(&b"ertek"[..]))` és `Some(&[1, 9][..])` a korábbi `Some(&b"ertek".to_vec())` helyett (a `rkv-core` `store/mod.rs` és `tests/update_prefix.rs` fájljaiban). A `FileStore` tesztsegédjében a `store.get(key).cloned()` `store.get(key).map(<[u8]>::to_vec)` lesz.

3. Vegyük fel a `ServerConfig::builder()`-t a fenti kóddal, és írjuk meg a két tesztjét (a megadott korlát érvényesül, a többi alapérték; a nulla korlát hiba).
4. Az `Error` enumra és a `Command` enumra tegyünk `#[non_exhaustive]`-t:

Kódblokk — fájl: `rkv-core/src/error.rs` és `rkv-core/src/command.rs` · művelet: attribútum a `pub enum Error`, illetve a `pub enum Command` elé · ellenőrzés: `cargo check --workspace --all-targets`

```
#[derive(Debug, Error)]
#[non_exhaustive]
pub enum Error { /* ... */ }
```

5. Futtassuk a teljes workspace ellenőrzését. A `#[non_exhaustive]` miatt **egyetlen** külső `match` nem fordul le: az `rkv-cli` `execute` függvénye, mert a `Command`-ot egy másik crate-ből illeszti. A többi fogyasztó már kezeli a bővíthetőséget: a `rkv-server` a `store.apply(&command)`-ot hívja (az `apply` az `rkv-core`-ban él, ahol a `match` kimerítő), az `Error` illesztései (`rkv-cli`) pedig eleve tartalmaznak `other =>` ágat. Az ismeretlen (jövőbeli) parancsra határozott választ adunk:

Kódblokk — fájl: `rkv-cli/src/main.rs` · művelet: új utolsó ág az `execute` `match command` blokkjában · ellenőrzés: `cargo check --workspace --all-targets`

```
        // A `Command` `#[non_exhaustive]`: a jövőbeli variánsra határozott válasz jár.
        _ => "ERR unsupported command".to_string(),
```

6. Futtassuk az ellenőrzéseket:

Kódblokk — fájl: terminál (workspace gyökér) · művelet: teljes ellenőrzés · ellenőrzés: nincs hiba és figyelmeztetés

```
cd ~/rust-labor/rkv
cargo test --workspace
cargo clippy --workspace --all-targets -- -D warnings
```

Ellenőrzési pont. A teljes workspace (25 tesztcsomag) zöld és a clippy figyelmeztetés nélkül lefut; a `rkv-server` és a `rkv-cxx` változtatás nélkül fordul, a `rkv-derive` által generált `match` az `rkv-core`-on belül kimerítő marad. Az eldobott `Result` és a megjelölt `Option` viselkedése:

Kódblokk — fájl: `src/main.rs` (önálló projekt) · művelet: teljes csere · ellenőrzés: `cargo build` → figyelmeztetések a lenti listában

```
#[derive(Debug)]
struct E;

fn plain() -> Result<(), E> { Ok(()) }                  // nincs attribútum
#[must_use = "a hiányzó kulcsot kezelni kell"]
fn with_message() -> Result<(), E> { Ok(()) }           // egyéni üzenet
fn option() -> Option<u8> { Some(1) }                   // az Option típusa nem must_use
#[must_use]
fn option_marked() -> Option<u8> { Some(1) }

fn main() {
    plain();           // figyelmeztet: unused `Result` that must be used
    with_message();    // két figyelmeztetés: a típusé és az egyedi üzenetes függvényé
```

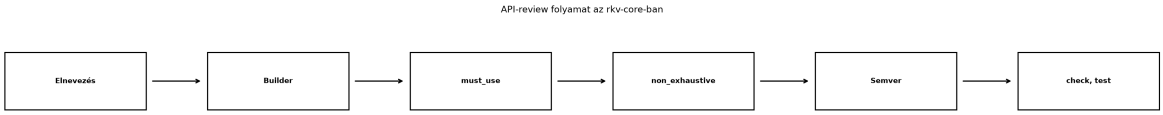
```
    option();          // NEM figyelmeztet
    option_marked();   // figyelmeztet: unused return value of `option_marked`
    let _ = plain();   // kifejezett eldobás: nincs figyelmeztetés
}
```

Az ellenőrzött kimenet négy figyelmeztetés: a `plain()`, a `with_message()` (kettő) és az `option_marked()` hívásra; az `option()` és a `let _ =` nem figyelmeztet.

Az átdolgozott API-t a Záróprojekt: az rkv teljes rendszerként használja tovább.

Az ábra lépései szövegesen:

1. **Elnevezés**: elnevezések és paramétertípusok (`&str`, `&[u8]`).
2. **Builder**: a `ServerConfig` builderrel épül.
3. **must_use**: a nem védett visszatérési típusok jelölése.
4. **non_exhaustive**: a bővíthető `Command` és `Error` jelölése.
5. **Semver**: a változtatások törő jellegének elemzése.
6. **check, test**: `cargo check`, `cargo test --workspace`, `clippy`.



21.2. ábra: API-review folyamat az rkv-core-ban - Az ellenőrzőlista lépései sorrendben; a semver-ellenőrzés az utolsó.

21.2.5 Tipikus hibák és átállási csapdák

1. **`&String` paraméter a `&str` helyett.** A C++-ban megszokott `const std::string&` mintát sokan `&String`-ként ültetik át, de ez feleslegesen szűkíti a hívót, és a `&String` dupla indirekciót is jelent. A `&str` elfogad `String`-ből deref koercióval, és a kölcsönzési szabályok is egyszerűbbek.

2. **`unwrap/expect` a nyilvános API-ban.** A hívó helyett döntünk a hibáról, és a C++ kivételektől eltérően a `panic!` a teljes szálat lebontja. Minden hibát `Result`-ként adjunk vissza; a `Result` típusa már `#[must_use]`, ezért a hívónak a visszatérési értékkel kezdenie kell valamit (`?`, `match`, `let _ =` kifejezett eldobás). A `panic!` csak invariáns-sértés esetén elfogadható.

3. **A két builder-forma összekeverése.** A `&mut self`-es builder is láncolható (lásd `Command`), nem hibás; az eltérés a költségmodellben van: a consuming builder egyszer használható és mozgat, a `&mut` builder újrahasználható, de a `build(&self)` klónoz. Hiba akkor van, ha a `&mut` builder láncát változóba mentjük (`E0716`), vagy ha a consuming builder setterét `#[must_use]` nélkül hagyjuk, és az eldobott eredmény csendben elveszik.

4. **`#[non_exhaustive]` elhagyása nyilvános enumoknál.** A `Command` és az `Error` bővíthető; ha nem jelöljük, minden új variáns törő változás. A C++-ban az `enum class` bővítése is törő lehet, de ott a hiba csendes (egy `switch` default nélkül figyelmeztetés), Rustban a semver ezt explicit módon kezeli. A jelölés *utólagos* hozzáadása viszont maga is törő, ezért az első publikálás előtt érdemes eldönteni.

5. **A `#[must_use]` felesleges halmozása.** A `Result`-os függvényekre külön attribútumot tenni zaj; az attribútumot az `Option`-t, `bool`-t vagy tiszta számítást adó függvényekre és az egyedi indoklásra tartsuk fenn. Ahol az eldobás legitim (például `Store::delete` visszaadott régi értéke), ne jelöljük.

21.2.6 Feladatok

1. **Fogalmi kérdés.** Mi a különbség az `as_`, `to_` és `into_` előtagok között? Adj mindegyikre egy-egy példát a standard könyvtárból, és mondd meg, melyik mikor allokálhat.
2. **C++-kód átírása.** Adott a C++ függvény:

```cpp
void process(const std::string& name, const std::vector<int>& values);
```

Írd át idiomatikus Rust-függvénnyé, amely nem szerez tulajdonjogot, és magyarázd el a paraméterválasztást.

3. **Implementációs feladat.** Bővítsd a `ServerConfigBuilder`-t úgy, hogy a `build()` elutasítsa a 16 MiB-nál nagyobb `max_frame`-et is (`ConfigError::TooLarge`). Mivel a `ConfigError` `#[non_exhaustive]`, a változás kompatibilis: igazold ezt, és írj tesztet a hibára.
4. **Tervezési döntés.** Az `rkv-core` `Error` enumja `#[non_exhaustive]` lett. Indokold meg, miért érdemes, és mondd ki, hogy a jelölés hozzáadása és egy új variáns hozzáadása törő-e a semver szerint `1.x`, illetve `0.x` verzió esetén. Mit tegyünk, ha a crate még nem publikált?

21.2.7 Megoldások

1. Az `as_` olcsó, kölcsönzött nézetet ad: `str::as_bytes()` → `&[u8]`, nem allokál. A `to_` tulajdonjogot adó, általában allokáló konverzió: `"x".to_string()` új `String`-et hoz létre, másol. Az `into_` a hívót elfogyasztó, tulajdonjogot adó: `String::into_bytes()` `Vec<u8>`-at ad, a `String` megsemmisül; nem másol, mert a puffer átkerül az új tulajdonoshoz.
2. Az idiomatikus Rust-függvény:

```rust
fn process(name: &str, values: &[i32]) {
    // ...
}
```

A `name` `&str`-ként elfogad `String`-ből is deref koercióval, a `values` `&[i32]` pedig `Vec<i32>`-ből. A függvény nem szerez tulajdonjogot, a hívó dönt a másolásról.

3. A hibatípus bővítése és az ellenőrzés:

```rust
#[derive(Debug, PartialEq, Eq)]
#[non_exhaustive]
pub enum ConfigError {
    Zero(&'static str),
    TooLarge(&'static str),
}

const MAX_FRAME_LIMIT: usize = 16 * 1024 * 1024;

// a build()-ben, a nulla-ellenőrzések után
if config.max_frame > MAX_FRAME_LIMIT {
    return Err(ConfigError::TooLarge("max_frame"));
}
```

```
// a Display-ben új ág
ConfigError::TooLarge(field) => write!(f, "a(z) {field} korlát túl nagy"),
```

Mivel a `ConfigError` `#[non_exhaustive]`, az új variáns a más crate-beli hívóknak nem törő: a `match`-eknek eleve volt wildcard ágat kellett tartalmazniuk. A teszt: `ServerConfig::builder().max_frame(MAX_FRAME_LIMIT + 1).build()` → `Err(ConfigError::TooLarge("` (A `Display` implementáció a crate-ben van, ezért ott a `match` kimerítő, és az új ágat fel kell venni.)

4. Az `Error` enum bővíthető (később hálózati hibák, FFI-hibák), ezért `#[non_exhaustive]`-re jelöljük. A hatás a más crate-beli `match`-ekre vonatkozik, ott wildcard ágat követel. A semver-hatás:

Változás	`1.x`	`0.x`
`#[non_exhaustive]` hozzáadása meglévő enumhoz	törő: `2.0.0`	törő: `0.(Y+1).0`
új variáns egy már `#[non_exhaustive]` enumban	kompatibilis: `1.(Y+1).0`	kompatibilis: `0.Y.(Z+1)`
új variáns jelölés nélküli enumban	törő	törő

Ha a crate még nem publikált, a jelölés az első kiadás előtt, verzióemelés nélkül hozzáadható; ez a kiindulópont az rkv-core-ban.

21.2.8 Rövid összefoglaló

Az rkv-core és az rkv-server nyilvános API-ját az API Guidelines szerint dolgoztuk át. A kulcslekérdezésnek három, egymástól eltérő szerződése van: a `Store`/`FileStore` `get` `Option<&[u8]>`-t ad kölcsönzéssel, a `Storage` trait és a `SharedStore` `Result<Option<Vec<u8>>>`-t owned értékkel, a hiányzó kulcs mindenhol `None`/`Ok(None)`. A `ServerConfig` builderrel épül, és a `build()` ellenőrzi a korlátokat. A `#[must_use]` a típuson (`Result`) és a függvényen különbözik: a `Result` típusa már véd, a függvény-attribútum az `Option`-ra és egyedi indoklásra való. A `Command` és az `Error` `#[non_exhaustive]`; ez csak a más crate-beli `match`-eket érinti (itt az `rkv-cli` `execute` függvényét, amely az ismeretlen parancsra `ERR unsupported command`-ot ad). A semver-határ `1.x` és `0.x` esetén különbözik. **Továbbvitt eredmény:** az átdolgozott `Store::get`/`FileStore::get` (`Option<&[u8]>`), a `ServerConfig::builder()`, a `non_exhaustive` `Command`/`Error` és az `rkv-server` `session` modulja a Záróprojekt: az rkv teljes rendszerként alapja.

21.2.9 Források és ellenőrzési hiányok

- Ellenőrizve (Rust 1.99.1): a teljes workspace `cargo test` (25 tesztcsomag) és `clippy -D warnings`; a migrációs hibák (`cloned`, szeletes összehasonlítás, `non_exhaustive` wildcard); a `must_use` figyelmeztetések (négy a lab programjában); a `ServerConfig` builder tesztjei.
- A Rust API Guidelines a hivatalos oldalon érhető el: https://rust-lang.github.io/api-guidelines/.
- A semver-értelmezés a Cargo dokumentációjának SemVer-fejezetén alapul; a `cargo-semver-checks` pontos telepítése és használata a https://crates.io/crates/cargo-semver-checks oldalon ellenőrizendő, ezt a fejezet nem futtatta.

22 Teljesítmény

22.1 Teljesítmény: mérés, profilozás, allokáció, release-beállítások

22.1.1 Motiváció

A C++-fejlesztő ösztönösen tudja, hogy a teljesítményoptimalizálás nem hit kérdése: előbb mérünk, aztán profilozunk, és csak utána nyúlunk a kódhoz. A Rust ugyanezt a mérnöki fegyelmet kínálja, de a nyelv adottságai – a borrow checker, a generikusok monomorfizálása, az iterátorok és a release-profil – más optimalizálási utakat nyitnak, mint a C++. Ebben az alfejezetben a `rkv` projektet vesszük elő, és a méréstől a kézzelfogható gyorsításig vezetjük végig a folyamatot. Kiindulópontunk a Tesztelés és dokumentáció: property-alapú tesztelés és benchmark alfejezetben létrehozott criterion benchmark. A fejezet mérőszáma az **allokációszám**: ezzel azonosítjuk a forró pontot, megszüntetjük a felesleges másolatokat, és a release-profil finomhangolásával további nyereséget érünk el. A CPU-profil (perf, flamegraph) bemutatása kitekintés marad; a laborok nem futtatnak perf-et, és nem ígérnek checksum-benchmarkot.

22.1.2 Fogalmak és szabályok

Definíció. A **profilozás** (profiling) a program futás közbeni megfigyelése annak érdekében, hogy megtudjuk, mely függvények, kódsorok vagy erőforrás-műveletek fogyasztják a legtöbb időt. A **forró pont** (hot spot) a profilban kiemelkedő, a futási idő jelentős részét kitevő kódrész. A **flamegraph** a profilozás vizualizációja, amely a hívási láncokat színes sávokkal, a szélességet pedig időaránnyal ábrázolja.

Definíció. Az **allokáció** (allocation) futás közbeni memóriafoglalás a kupacon, például `Vec::push` kapacitásbővítés, `String` növekedés, `clone`, vagy `Box` létrehozása. Egy allokáció költsége (az allokátor hívása, a cache-hatás, a későbbi felszabadítás) gyakran nagyobb, mint néhány aritmetikai utasításé, ezért az allokációk számának csökkentése sokszor többet hoz, mint az utasításszámé. Hogy mennyit, az az adott programtól függ: ezt mérni kell, nem kitalálni.

Szabály. A mérés-vezérelt optimalizálás ciklusa: (1) fogalmazz meg egy hipotézist, (2) mérj egy determinisztikus benchmarkkal, (3) profilozz, (4) végezd el a módosítást, (5) mérd újra ugyanazzal a benchmarkkal, (6) ha a javulás elhanyagolható, lépj tovább. Soha ne optimalizálj mérés nélkül, és soha ne fogadd el a javulást visszamérés nélkül.

Szabály. Az indexelés (`v[i]`) a nyelv szemantikája szerint ellenőrzött: ha `i >= v.len()`, a program pánikol (nem a memóriához fér hozzá). A `get(i)` ugyanezt `Option`-nel fejezi ki: `None` jelzi a határon kívüli indexet, panic nélkül. Ez *szemantika*, nem kötelező futási költség: az optimalizáló gyakran **bizonyítani tudja**, hogy az index érvényes (például egy `0..v.len()` ciklusban), és ilyenkor az ellenőrzést eltávolítja. Az iterátor (`for x in &v`) nem indexel, így a kérdés fel sem merül. Az ellenőrzés megmaradása vagy eltűnése konkrét kódon és fordítási beállításon múlik, ezért egy költségállítást mindig ugyanazzal a toolchainnel és profillal kell igazolni.

Szabály. A generikus függvények és típusok **monomorfizálódnak**: minden konkrét típusparaméterre a fordító külön kódot generál. Ez lehetővé teszi az inline-t és a statikus diszpécs miatti devirtualizálást, de növeli a bináris méretét. A `#[inline]` attribútum a fordítót a függvény beillesztésére kéri; a `lto` és a `codegen-units` ezt a lehetőséget erősíti.

22.1.3 C++ és Rust összevetés

C++-ban a Google Benchmark és a `std::chrono` a mérés, a `perf` és a `gprof` a profilozás eszköze. Rustban a criterion ugyanazt a szerepet tölti be, mint a Google Benchmark: statisztikai elemzéssel, konfidenciaintervallummal és `black_box`-szal véd a kihagyástól. A profilozáshoz Linuxon ugyanaz a `perf` működik, a `cargo flamegraph` pedig a perf-et és a flamegraph generálást csomagolja egy parancsba; ezek a fejezetben **kitekintésként** szerepelnek, a laborok mérőszáma az allokációszám.

A legfontosabb nyelvi különbség a C++-hoz képest a **noalias** információnál jelenik meg. A Rust `&mut` referencia szavatolja, hogy nincs más élő alias ugyanarra a memóriára. Ez a fordító számára olyan optimalizálási lehetőséget ad, amit C++-ban a `restrict` kulcsszóval (ami nem szabványos, bár a GCC és a Clang támogatja) csak körülményesen érhetünk el. A Rust minden `&mut` paraméternél automatikusan érvényesíti a noalias-t, így a vektorizálás, a cache-optimalizálás és a konstansok előrehozása agresszívebb lehet.

A C++ `std::vector::operator[]` nem végez határellenőrzést (a határon kívüli index definiálatlan viselkedés), míg a Rust `v[i]` igen (pánik). C++-ban az ellenőrzött hozzáférés az `at()` (kivétel), Rustban a `get()` (`Option`) vagy az indexelés (pánik). A Rust tehát alapból a biztonságosabb szemantikát adja, és az optimalizáló dolga, hogy ahol tudja, a költséget eltüntesse; ahol nem tudja, ott az ellenőrzés megmarad.

22.1.4 Kidolgozott példa: határellenőrzés a gépi kódban

Vizsgáljunk meg négy függvényt. Az első kettő ugyanazt számolja (egy szelet bájtjainak XOR-ját) indexeléssel, illetve iterátorral; a harmadik és a negyedik külső indexeket használ, amelyekről a fordító semmit sem tud. (Ez csak a határellenőrzés bemutatása, **nem** az rkv ellenőrzőösszege; azt az Adatpárhuzamosság rayonnal alfejezet kétszintű hash-e adja.)

Kódblokk — fájl: `src/lib.rs` (önálló `cargo new --lib bc` projekt) · művelet: teljes csere · ellenőrzés: `cargo test`

```rust
#[inline(never)]
pub fn xor_fold_indexed(data: &[u8]) -> u8 {
    let mut result = 0u8;
    for i in 0..data.len() {
        result ^= data[i];
    }
    result
}

#[inline(never)]
pub fn xor_fold(data: &[u8]) -> u8 {
    data.iter().fold(0u8, |acc, &b| acc ^ b)
}

/// Az index kívülről jön: a fordító nem bizonyíthatja, hogy érvényes.
#[inline(never)]
pub fn xor_picked(data: &[u8], idx: &[usize]) -> u8 {
    let mut result = 0u8;
```

```
    for &i in idx {
        result ^= data[i];
    }
    result
}

#[inline(never)]
pub fn xor_picked_get(data: &[u8], idx: &[usize]) -> Option<u8> {
    let mut result = 0u8;
    for &i in idx {
        result ^= *data.get(i)?;
    }
    Some(result)
}

#[cfg(test)]
mod tests {
    use super::*;
    #[test]
    fn same_value() {
        let v: Vec<u8> = (0..=255).collect();
        assert_eq!(xor_fold_indexed(&v), xor_fold(&v));
        assert_eq!(xor_picked(&v, &[1, 2, 3]), xor_picked_get(&v, &[1, 2, 3]).unwrap());
        assert_eq!(xor_picked_get(&v, &[999]), None);
    }
    #[test]
    #[should_panic]
    fn index_panics() {
        xor_picked(&[1, 2], &[5]);
    }
}
```

A C++-ban az indexelés ellenőrzés nélküli; a Rust `xor_fold_indexed` forráskódja ugyanezt írja, csak a szemantika ellenőrzött. A két fogalom különválasztása a kulcs: **mit ír elő a nyelv** és **mit tartalmaz az optimalizált gépi kód**.

Ellenőrzés ugyanazzal a toolchainnel (Rust 1.99.1, `x86_64`, `--release`): az optimalizált assemblyt `cargo rustc --release --lib -- --emit asm` állítja elő, és a függvényenként megszámoljuk az utasításokat és a `panic_bounds_check` hívásokat:

Függvény	Utasítás	`panic_bounds_check` hívás
`xor_fold_indexed` (`0..data.len()` ciklus, `data[i]`)	67	0
`xor_fold` (iterátor)	67	0
`xor_picked` (külső indexek, `data[i]`)	19	1
`xor_picked_get` (külső indexek, `get(i)?`)	21	0 (az `Option` ág kezeli)

Az első két függvény gépi kódja ugyanannyi utasítás, és egyik sem hív határellenőrző rutint: az optimalizáló a `0..data.len()` ciklusban bizonyította az index érvényességét, és eltávolította az ellenőrzést. (A `cargo test` ellenőrzi, hogy a két változat ugyanazt az értéket adja.) A `xor_picked`-ben viszont az index kívülről jön, ezért az ellenőrzés megmarad, és egy érvénytelen index pánikot okoz (`index_panics` teszt); a `xor_picked_get` ugyanezt `Option`-nel kezeli. Tanulság: *ne állítsuk, hogy az indexelés futási többlet*, de azt se, hogy mindig ingyenes. Debug módban (`opt-level = 0`) az ellenőrzések megmaradnak.

22.1.5 Labor: az rkv `FileStore` optimalizálása mérés alapján

Munkakönyvtár: `~/rust-labor/rkv/`

A labor az **előző alfejezetekben elkészült, ellenőrzött rkv-kódon** dolgozik (a `rkv-core` a `Store`/`FileStore` típusokkal). Az optimalizálás célpontját nem találgatjuk: allokációszámlálóval mérjük meg, hol foglal a kód feleslegesen. A fejezet a *profilozás* alatt itt ezt az allokációszámlálós mérést érti; a CPU-profil (perf, flamegraph) kitekintés, nem része a labornak. A `FileStore::set` jelenlegi változata:

```
pub fn set(&mut self, key: String, value: Vec<u8>) -> Result<()> {
    let entry = LogEntry::Set { key: key.clone(), value: value.clone() };
    self.append_entry(&entry)?;
    self.data.insert(key, value);
    Ok(())
}
```

A naplórekord *tulajdonolt* `LogEntry`, ezért a kulcsot és az értéket le kell másolni, hogy a másolatot serde kiírja, az eredetiek pedig a térképbe kerüljenek. A `save_snapshot` pedig a teljes térképet másolja le egy `Snapshot`-ba, csak azért, hogy kiírja. Mindkettő olyan allokáció, amelyet a nyelv nem kényszerít ki.

22.1.5.1 1. lépés: allokációszámláló (alapállapot)

A számláló egy globális allokátor-burkoló. Az allokációszám kevésbé zajos, mint az időmérés, de **nem univerzálisan gépfüggetlen**: a standard könyvtár és a `HashMap` belső növekedési stratégiája, valamint a fordító verziója befolyásolja. A fejezet számait ezért egy rögzített környezet (Rust 1.99.1, Linux x86-64, a könyv `Cargo.lock`-ja) eredményeként kell olvasni. A stabil követelmény a felesleges másolatok megszűnése, nem az, hogy minden gépen pontosan ugyanaz a szám jöjjön ki; a saját mérésnél az előtte/utána különbséget és a működés változatlanságát kell ellenőrizni.

Kódblokk — fájl: `rkv-core/examples/alloc_count.rs` · művelet: új fájl · ellenőrzés: `cargo run --release -p rkv-core --example alloc_count`

```
//! Allokációszámláló: hány kupacfoglalást végez 1000 `FileStore::set`, illetve egy
↪  snapshot-mentés.
use std::alloc::{GlobalAlloc, Layout, System};
use std::sync::atomic::{AtomicUsize, Ordering::Relaxed};

use rkv_core::FileStore;

struct Counting;
static ALLOCS: AtomicUsize = AtomicUsize::new(0);
static BYTES: AtomicUsize = AtomicUsize::new(0);

unsafe impl GlobalAlloc for Counting {
    unsafe fn alloc(&self, layout: Layout) -> *mut u8 {
        ALLOCS.fetch_add(1, Relaxed);
        BYTES.fetch_add(layout.size(), Relaxed);
        unsafe { System.alloc(layout) }
    }
    unsafe fn dealloc(&self, ptr: *mut u8, layout: Layout) {
        unsafe { System.dealloc(ptr, layout) }
    }
    unsafe fn realloc(&self, ptr: *mut u8, layout: Layout, new_size: usize) -> *mut u8 {
        ALLOCS.fetch_add(1, Relaxed);
        BYTES.fetch_add(new_size, Relaxed);
```

```
        unsafe { System.realloc(ptr, layout, new_size) }
    }
}

#[global_allocator]
static GLOBAL: Counting = Counting;

fn measure<T>(label: &str, f: impl FnOnce() -> T) -> T {
    let (a0, b0) = (ALLOCS.load(Relaxed), BYTES.load(Relaxed));
    let out = f();
    println!(
        "{label}: {} allokáció, {} bájt",
        ALLOCS.load(Relaxed) - a0,
        BYTES.load(Relaxed) - b0
    );
    out
}

fn main() {
    let dir = std::env::temp_dir().join(format!("rkv-alloc-{}", std::process::id()));
    std::fs::create_dir_all(&dir).unwrap();
    let mut store = FileStore::open(dir.join("state.json"), dir.join("append.log")).unwrap();
    // Az előkészítést (kulcsok, értékek) a mérésen kívül végezzük.
    let items: Vec<(String, Vec<u8>)> =
        (0..1000).map(|i| (format!("kulcs-{i:04}"), vec![i as u8; 64])).collect();
    // A riportban szereplő kulcshossz ellenőrzése: "kulcs-" (6 bájt) + 4 jegy = 10 bájt.
    assert!(items.iter().all(|(k, _)| k.len() == 10));
    measure("1000 x FileStore::set", || {
        for (key, value) in items {
            store.set(key, value).unwrap();
        }
    });
    measure("FileStore::save_snapshot (1000 elem)", || store.save_snapshot().unwrap());
    drop(store);
    std::fs::remove_dir_all(&dir).ok();
}
```

Alapállapot (ellenőrzött futás, 1000 elem, 10 bájtos kulcs, 64 bájtos érték):

```
1000 x FileStore::set: 4610 allokáció, 965868 bájt
FileStore::save_snapshot (1000 elem): 2003 allokáció, 182596 bájt
```

22.1.5.2 2. lépés: kölcsönzött naplórekord és snapshot

A serde képes kölcsönzött adatot szerializálni. A `LogEntryRef` és a `SnapshotRef` ugyanazt a JSON-t írja, mint a tulajdonolt `LogEntry`, illetve `Snapshot`, ezért a napló- és snapshot-formátum (és a visszaolvasás) **nem változik**.

Kódblokk — fájl: `rkv-core/src/store/file_store.rs` · művelet: két új típus a `LogEntry` mellé; a `set`, `delete`, `append_entry` és a `save_snapshot` egy-egy sorának cseréje · ellenőrzés: `cargo test -p rkv-core`

```
/// Kölcsönzött naplórekord: ugyanazt a JSON-t írja, mint a `LogEntry`, de nem másol.
#[derive(Serialize)]
enum LogEntryRef<'a> {
    Set { key: &'a str, value: &'a [u8] },
    Delete { key: &'a str },
}

/// Kölcsönzött snapshot: a `Snapshot` JSON-alakja, a térkép másolása nélkül.
#[derive(Serialize)]
```

```rust
struct SnapshotRef<'a> {
    data: &'a HashMap<String, Vec<u8>>,
}

pub fn set(&mut self, key: String, value: Vec<u8>) -> Result<()> {
    self.append_entry(&LogEntryRef::Set { key: &key, value: &value })?;
    // Csak a tartós naplórekord után módosul a memóriabeli állapot.
    self.data.insert(key, value);
    Ok(())
}

// a delete-ben:   self.append_entry(&LogEntryRef::Delete { key })?;
// az append_entry aláírása:   fn append_entry(&mut self, entry: &LogEntryRef<'_>) -> Result<()>
// a save_snapshot-ban:        serde_json::to_writer(&mut writer, &SnapshotRef { data:
↪  &self.data })?;
```

A commit-szerződés (teljes rekord kiírása, flush, `sync_data`, csak utána a memória módosítása) érintetlen. Két új teszt védi a formátumot: a kölcsönzött és a tulajdonolt rekord **bájtra azonos** JSON-t ad, és a szóközös kulcs, a sortörést tartalmazó kulcs, valamint minden bájtérték (`0..=255`) veszteség nélkül visszaolvasható a naplóból és a snapshotból.

Kódblokk — fájl: `rkv-core/src/store/file_store.rs` · művelet: két teszt a `tests` modul végére · ellenőrzés: `cargo test -p rkv-core`

```rust
#[test]
fn borrowed_records_serialize_like_owned_ones() {
    let owned = LogEntry::Set { key: "a b".into(), value: vec![0, 255, 10] };
    let borrowed = LogEntryRef::Set { key: "a b", value: &[0, 255, 10] };
    assert_eq!(serde_json::to_vec(&owned).unwrap(), serde_json::to_vec(&borrowed).unwrap());
    let owned = LogEntry::Delete { key: "k".into() };
    let borrowed = LogEntryRef::Delete { key: "k" };
    assert_eq!(serde_json::to_vec(&owned).unwrap(), serde_json::to_vec(&borrowed).unwrap());
}

#[test]
fn awkward_keys_and_values_survive_log_and_snapshot() {
    let (snap, log) = fresh_dir("awkward");
    let all_bytes: Vec<u8> = (0..=255).collect();
    {
        let mut store = FileStore::open(&snap, &log).unwrap();
        store.set("kulcs szóközzel".into(), all_bytes.clone()).unwrap();
        store.set("sor\ntörés".into(), b"\n\r\0".to_vec()).unwrap();
        store.save_snapshot().unwrap(); // snapshot: a kölcsönzött SnapshotRef írja
        store.set("a napló".into(), vec![]).unwrap(); // napló: a kölcsönzött LogEntryRef írja
    }
    let store = FileStore::open(&snap, &log).unwrap();
    assert_eq!(get(&store, "kulcs szóközzel"), Some(all_bytes));
    assert_eq!(get(&store, "sor\ntörés"), Some(b"\n\r\0".to_vec()));
    assert_eq!(get(&store, "a napló"), Some(vec![]));
}
```

22.1.5.3 3. lépés: az eredmény mérése

Ugyanaz a számláló az átírt kódon:

```
1000 x FileStore::set: 2610 allokáció, 891868 bájt
FileStore::save_snapshot (1000 elem): 2 allokáció, 8228 bájt
```

A diff **pontosan** a megnevezett allokációkat szüntette meg: a `set` hívásonként kettőt (a kulcs és az érték másolatát: $4610 - 2610 = 2000 = 2 \times 1000$), a `save_snapshot` pedig a teljes térkép másolatát ($2003 \to 2$). A maradék 2,6 allokáció `set`-enként a serde kimeneti pufferének (`to_vec`)

növekedése és a térkép bővülése; az előbbit a 3. feladat szünteti meg. Az időnyereség ennél kisebb: a `set` ideje a `sync_data`-n (lemezszinkron) múlik, nem a másoláson, ezért itt az allokációszám a jó mérőszám, az idő nem.

22.1.5.4 4. lépés: a `set` benchmark pontos leírása

A criterion-benchmark mindig egy konkrét műveletet mér. A `set` két különböző esete két külön névvel szerepel, a nevek leírják, mit mérnek:

Kódblokk — fájl: `rkv-core/benches/benchmark.rs` · művelet: a `bench_set` cseréje · ellenőrzés: `cargo bench -p rkv-core --bench benchmark`

```
/// Mért művelet: egyetlen `Storage::set` hívás. Két külön eset, két külön névvel.
fn bench_set(c: &mut Criterion) {
    // 1. Új kulcs egy üres tárolóban. Előkészítés (nem mért): friss tároló, tulajdonolt kulcs
    ↪  és érték.
    // A tároló eldobása (Drop) a mért függvény visszatérési értékén keresztül kívül esik a
    ↪  mérésen.
    c.bench_function("MemStore::set (új kulcs, üres tároló)", |b| {
        b.iter_batched(
            || (MemStore::new(), "key".to_string(), b"value".to_vec()),
            |(mut store, key, value)| {
                Storage::set(&mut store, black_box(key), black_box(value)).unwrap();
                store
            },
            BatchSize::SmallInput,
        );
    });

    // 2. Meglévő kulcs cseréje egy 1000 elemű tárolóban. Az előkészítés (nem mért) feltölti a
    // tárolót; a mért hívás a "kulcs-0500" értékét cseréli, tehát nincs új csomópont és kulcs.
    c.bench_function("MemStore::set (csere, 1000 elemű tároló)", |b| {
        b.iter_batched(
            || {
                let mut store = MemStore::new();
                for i in 0..1000 {
                    Storage::set(&mut store, format!("kulcs-{i:04}"), vec![0u8; 16]).unwrap();
                }
                (store, "kulcs-0500".to_string(), b"value".to_vec())
            },
            |(mut store, key, value)| {
                Storage::set(&mut store, black_box(key), black_box(value)).unwrap();
                store
            },
            BatchSize::LargeInput,
        );
    });
}
```

Rekonstruálható leírás: a mért művelet egyetlen `Storage::set` hívás; az inputméret az első esetben 0 meglévő elem, a másodikban 1000; az első eset **új** kulcsot szúr be, a második egy **meglévő** kulcsot cserél; a tároló összeállítása az `iter_batched` előkészítő lépésében történik (nem mért), a tároló eldobása pedig a mért függvény visszatérési értékén keresztül kívül esik a mérésen. Egy ellenőrzött futás eredménye (gépfüggő, egyetlen futás): `[26,6 ns 26,9 ns 27,2 ns]` az új kulcsra; a cserére a lenti profiltáblázat alapsora (74,7 ns) érvényes. Az egymás utáni futások között ±10% eltérés is előfordul.

22.1.5.5 5. lépés: release-profil, egyszerre egy változtatás

A profil beállításai kölcsönhatásban vannak, ezért **egyszerre egy** változtatást mérünk az alapértelmezett `release` ellen. A profilt kizárólag a workspace gyökerében lévő `Cargo.toml` veszi figyelembe (a tagcrate-ek `[profile]` szakaszát a Cargo figyelmen kívül hagyja). A mérés környezeti változókkal történik, hogy a `Cargo.toml` ne módosuljon: `CARGO_PROFILE_RELEASE_LTO=thin cargo build --release -p rkv-cli`, és `CARGO_PROFILE_RELEASE_LTO=thin cargo bench -p rkv-core --bench benchmark -- csere`. A két metrika külön van: a bináris mérete (`rkv-cli`) és a `MemStore::set (csere, 1000 elemű tároló)` ideje.

Változtatás (egyedül, az alaphoz képest)	`rkv-cli` mérete	`set (csere)` ideje
alap `release`	1 234 680 bájt	74,7 ns
`codegen-units = 1`	1 032 984 bájt	69,8 ns
`lto = "thin"`	1 163 608 bájt	73,8 ns
`lto = "fat"`	937 928 bájt	74,2 ns
`panic = "abort"`	1 049 600 bájt	(nem értelmezhető, lásd lent)
`lto = "fat"` + `codegen-units = 1`	874 184 bájt	81,5 ns

Egyetlen futás, egy gép: az időkülönbségek (±10%) a zaj tartományában vannak, a méretcsökkenés viszont egyértelmű (`lto = "fat"` + `codegen-units = 1`: −29%). A tanulság: a profilbeállítás itt a *bináris méretét* és a linkelés minőségét javítja, ezt a mikrobenchmark nem látja. A `[profile.release]`-be ezért azt vesszük fel, amit a mérés indokol:

Kódblokk — fájl: `Cargo.toml` (workspace gyökér) · művelet: új szakasz a fájl végére · ellenőrzés: `cargo build --release -p rkv-cli` → kisebb bináris

```
[profile.release]
lto = "fat"
codegen-units = 1
```

A `panic = "abort"` külön kérdés, nem teljesítménybeállítás. A cargo a teszt- és benchmark-célpontokat mindig unwinding-gel fordítja, ezért a Criterion-idő nem mutathatja a hatását, és a `cargo test --release -p rkv-ffi` zöld marad `panic = "abort"` mellett is: a tesztek nem a termék viselkedését igazolják. A termék viselkedése megváltozik: az `rkv-ffi` a Rust-pánikot `catch_unwind`-del fogja el és `RKV_PANIC` státuszkóddá alakítja, a szerver a kliensfeladat pánikját a taskra korlátozza. Abort módban mindkettő megszűnik. Önálló bizonyíték (egy `catch_unwind`-es program): `panic = "unwind"` mellett a kimenet `elkapva: true`, `panic = "abort"` mellett a folyamat megszakad, és a sor ki sem íródik. Ezért az rkv workspace-ben a `panic = "abort"` **nincs** a profilban; ha a CLI-hez kis binárisra van szükség, azt külön csomagolt, `catch_unwind`-et nem használó célpontnál lehet megfontolni.

Ellenőrzési pont. Az `alloc_count` példa a 3. lépés számait adja (`4610 → 2610`, illetve `2003 → 2` allokáció), a `cargo test -p rkv-core` zöld (a két formátumvédő teszttel együtt), a `cargo bench -p rkv-core --bench benchmark` a két külön nevű `set` esetet futtatja, a profiltáblázat minden sorához tartozik parancs és profil, és a `panic = "abort"` nem kerül a `[profile.release]`-be.

Továbbvitt eredmény. A kölcsönzött `LogEntryRef`/`SnapshotRef`, az allokációszámláló, a két `set` benchmark és a `[profile.release]` (`lto = "fat"`, `codegen-units = 1`, **abort nélkül**) a Záróprojekt: az rkv teljes rendszerként alfejezetben a teljes rendszer összeállításakor érvényesül.

22.1.6 Tipikus hibák és átállási csapdák

1. **Klónozás a borrow checker „legyőzésére”.** Amikor a kölcsönzési szabályok miatt nem férünk hozzá egy értékhez, a gyors munka kedvéért `clone`-t hívunk. Ez elrejti az allokációt, és a forró ponton teljesítményromlást okozhat. A megoldás: vizsgáljuk meg, hogy a `clone` valóban szükséges-e, vagy áthelyezhetjük a tulajdonlást, átadhatunk `&str`-et `String` helyett, vagy használhatunk `Cow`-t. A laborban a `FileStore` tulajdonolt `LogEntry`-je volt ilyen: kölcsönzött rekorddal a másolat felesleges.

2. **A hibakezelés módját a sebességből választani.** Az `unwrap`/`expect`, a `?` és a `match` közül a *szerződés* dönt: `unwrap` ott, ahol a hiba programozói hiba (invariánssértés), `Result` és `?` ott, ahol a hívó kezelheti. A teljesítménykülönbség a legtöbb kódban elhanyagolható, és nem általánosítható: a `?` használata önmagában nem gyorsít. Ha a hot path hibakezelése számít, azonos feladatot megoldó két konkrét változatot kell lemérni, nem általános szabályt alkalmazni.

3. **`Rc<RefCell<T>>` és `Arc<Mutex<T>>` összekeverése.** Négy különböző költség van. Az `Rc` referenciaszámlálása *nem* atomi (olcsó, de `!Send`); az `Arc`-é atomi. A `RefCell` futásidejű kölcsönzés-ellenőrzése *nem* atomi, egy számláló vizsgálata és módosítása; a `Mutex` zárolása atomi művelet, szálak közötti kizárással. Egyszálú megosztásra `Rc<RefCell<T>>` elegendő, szálak között `Arc<Mutex<T>>` vagy `Arc<RwLock<T>>` kell; a forró ponton viszont gyakran el lehet kerülni a megosztást tulajdonjog-átadással vagy `&mut`-tal.

4. **Generikus monomorfizálás kiteregetése.** A túlzottan generikus kód minden típusra külön példányt hoz létre, ami a bináris méretét és a fordítási időt növeli. Ha a típusparaméterek csak kevés konkrét típussal fordulnak elő, érdemes megfontolni a trait-objektumot (`Box<dyn Storage>`) a forró ponton kívül, ahol a dinamikus diszpécs költsége elhanyagolható. C++-ban ugyanez a sablon-kiteregetés problémája, de a Rustban a trait-objektumok expliciten jelzik a dinamikus diszpécs szándékát.

5. **Több profilbeállítás egyszerre, mérés nélkül.** Ha az `lto`, a `codegen-units` és a `panic` egyszerre változik, nem tudjuk, melyik tett jót vagy rosszat. Egyszerre egyet mérjünk, és külön a bináris méretét az idejétől.

22.1.7 Feladatok

1. Feladat (fogalmi). Miért nem elegendő a debug módú `cargo run` a teljesítmény mérésére? Magyarázd el, miért használunk release módot, `black_box`-ot és konfidenciaintervallumot a criterionban. Miért jobb mérőszám az allokációk száma, mint az idő, ha a művelet lemezszinkronra vár?

2. Feladat (C++ → Rust átírás). Adott a következő C++ függvény, amely egy `std::vector<std::string>` elemeit fűzi össze szóközökkel elválasztva:

```
std::string join(const std::vector<std::string>& words) {
    std::string result;
    for (size_t i = 0; i < words.size(); ++i) {
        if (i > 0) result += " ";
        result += words[i];
    }
    return result;
}
```

Írd át a függvényt idiomatikus Rustra, `&[String]` paraméterrel, iterátorokkal, és legalább `with_capacity` használatával, hogy csökkentsd az allokációk számát. A kód ne klónozza a szavakat.

3. Feladat (implementáció az rkv-ban). A labor a `FileStore::set` és a `save_snapshot` másolatait szüntette meg. Keress a `FileStore`-ban egy további, szükségtelen allokációt az allokációszámláló segítségével (tipp: a naplóbejegyzés `serde_json::to_vec` + `push(b'\n')` párosa), és szüntesd meg úgy, hogy a naplóformátum bájtra azonos maradjon. Mutasd meg az előtte/utána számokat, és védd tesztel, hogy minden engedélyezett bájtérték, szóközös kulcs és sortörés veszteség nélkül visszaolvasható.

4. Feladat (tervezési döntés). A `Storage` trait jelenleg `fn set(&mut self, key: String, value: Vec<u8>) -> Result<()>`. Tételezd fel, hogy a hálózati szerver forró hívási láncában a `set` a legtöbbször kis kulcsokkal és értékekkel dolgozik. Válaszd ki, hogy a `key` paramétert `String`-ről `&str`-re vagy `impl Into<String>`-re változtatod-e. Két konkrét hívóhelyen (a hívó `String`-et birtokol / a hívó csak `&str`-t lát) számold össze az allokációkat, és különítsd el az új kulcs beszúrását a meglévő frissítésétől. Hogyan érinti ez a `FileStore` implementációt?

22.1.8 Megoldások

1. Megoldás. A debug módban a fordító nem végez optimalizálást (`opt-level = 0`), a határellenőrzések megmaradnak, és a generált kód lassabb. A release mód (`opt-level = 3`) a fordítói optimalizálások teljes eszköztárát bekapcsolja, így a mért adatok a valós használatot tükrözik. A `black_box` arra szolgál, hogy a fordító ne hajtsa ki a konstans kifejezéseket, ne inline-olja el a teljes hívást, és ne vegye észre, hogy az eredmény nem használt. A konfidenciaintervallum a mérési zaj miatt fontos: a criterion újra és újra lefuttatja a benchmarkot, és statisztikai elemzéssel ad megbízható tartományt az átlag körül. Ha a művelet `fsync`-re vár, az idő a lemez sebességét méri; az allokációszám determinisztikus, ezért a kód változását zajmentesen mutatja.

2. Megoldás.

```rust
fn join(words: &[String]) -> String {
    // A kapacitás bájtokban számít (nem karakterekben): a szavak bájthosszának összege
    // plusz az elválasztók száma. Az összeg túlcsordulását ellenőrizzük.
    let mut total = words.len().saturating_sub(1);
    for w in words {
        total = total.checked_add(w.len()).expect("capacity overflow");
    }
    let mut result = String::with_capacity(total);
    for (i, word) in words.iter().enumerate() {
        if i > 0 {
            result.push(' ');
        }
        result.push_str(word);
    }
    result
}
```

A reallokációmentesség **csak** azért igaz, mert az előre lefoglalt kapacitás (`total`) pontosan elég: a `push` és a `push_str` egyszer sem lépi túl. Egy becsült kapacitásnál (például átlagos szóhosszal) lehetnek reallokációk. Az üres lista `total = 0`-t ad (a `saturating_sub` megakadályozza az `0 - 1` alulcsordulást), az eredmény üres `String`. A hossz bájtban értendő: az `"árvíz"` 7 bájt, a `" "` 4 bájt, ezért `chars().count()`-tal számolt kapacitás kevés lenne. A túlcsordulásra a `checked_add` panicol (`capacity overflow`), ahogy a standard könyvtár is teszi, valódi szavakból álló szeleten ez gyakorlatilag nem fordulhat elő.

A szabványos megoldás ugyanezt kész eszközzel adja: `words.join(" ")`, ami szintén előre kiszámolja a méretet, ezért a saját `join` ritkán indokolt. Futtatva (release, számláló-allokátorral, 1000 szó): a saját `join` 1 allokáció, a `words.join(" ")` 1 allokáció, az `String::new()`-ból induló naiv változat 13 (a kapacitás duplázódik); az eredmény bájtra azonos, üres, egyelemű és többbájtos (`árvíz`, `tűrő`,) bemeneten is.

3. Megoldás. A `save_snapshot` másolata a labor 2. lépésében már megszűnt. A maradék allokációt (2610 / 1000 `set`) az `append_entry` okozza: minden híváskor új `Vec<u8>`-et készít (`serde_json::to_vec`), majd egy `\n`-nel tovább növeli. Megoldás: a rekordot közvetlenül a `BufWriter`-be írjuk, ugyanazzal a sorvéggel, a commit-szerződés (flush + `sync_data`, hiba esetén a tároló „megmérgezése") érintetlen marad:

Kódblokk — fájl: `rkv-core/src/store/file_store.rs` · művelet: az `append_entry` rekordkiírásának cseréje · ellenőrzés: `cargo test -p rkv-core` és az allokációszámláló

```
fn append_entry(&mut self, entry: &LogEntryRef<'_>) -> Result<()> {
    self.ensure_healthy()?;
    let res = serde_json::to_writer(&mut self.log_writer, entry)
        .map_err(io::Error::from)
        .and_then(|()| self.log_writer.write_all(b"\n"))
        .and_then(|()| self.log_writer.flush())
        .and_then(|()| self.log_writer.get_ref().sync_data());
    if let Err(e) = res {
        // A pufferben vagy a fájlban félkész rekord maradhatott:
        // vakon nem folytatjuk a naplózást.
        self.poisoned = true;
        return Err(e.into());
    }
    Ok(())
}
```

Az eredmény (ellenőrzött futás, a fejezet rögzített környezetében: Rust 1.99.1, Linux x86-64): `1000 x FileStore::set: 10 allokáció, 200668 bájt` (korábban 2610 és 891868 bájt); a maradék a térkép növekedése. A hordozható követelmény nem az abszolút érték, hanem a felesleges másolatok eltűnése és a bájtra azonos formátum; a számok gépenként és toolchainenként eltérhetnek. A formátum nem változik, mert ugyanazt a JSON-t írjuk ugyanazzal a sorvéggel; a labor `borrowed_records_serialize_like_owned_ones` és `awkward_keys_and_values_survive_log_and_snapshot` tesztjei ezt védik, és a teljes `rkv-core` tesztkészlet változtatás nélkül zöld. Egy mellékhatás: a szerializációs hiba is „megmérgezi" a tárolót, mert a rekord egy része már a pufferben lehet; a `LogEntryRef` típusokra ez nem fordulhat elő. Nem szabad másik, „szebb" formátumot (például `from_utf8_lossy`-val szövegesített értéket) bevezetni: az nem alakítja vissza a nem UTF-8 bájtokat, így adatot veszítene.

4. Megoldás. A döntés a hívók adatformáján és a tárolóművelet jellegén múlik, nem általános szabály. Lemérve (release, számláló-allokátor, előre méretezett `HashMap<String, Vec<u8>>`, üres értékkel):

Hívóhely	`key: String`	`key: &str` (`get_mut`, különben `to_owned`)	`impl Into<String>`
(a) a hívó már birtokol `String`-et, új kulcs	0 allokáció (átmozgatjuk)	1	0
(b) a hívó csak `&str`-t lát, **meglévő** kulcs frissítése	1 (a kulcs másolata, amit a `insert` eldob)	0	1

Hívóhely	`key: String`	`key: &str` (`get_mut`, különben `to_owned`)	`impl Into<String>`
(c) a hívó csak `&str`-t lát, **új** kulcs	1	1	1

Az `owned` paraméter tehát akkor jó, ha a tipikus hívó már `String`-et épített (például a parancssor szétszedése után), mert a tulajdonjog átadása másolatmentes, és a `FileStore::set` a kulcsot kölcsönzi a naplóhoz (`LogEntryRef`), majd a térképbe mozgatja. Ha viszont a hívó csak `&str`-t lát (egy sorpuffer része), és a szerver gyakran **frissít** meglévő kulcsot, akkor a `&str`-es szignatúra spórol, mert a `get_mut`-tal megtalált kulcshoz nem kell `String`. A `&str` esetén a hívónak **nem kötelező** `String`-et készítenie: a tároló készíti el, de csak új kulcsnál. Az `impl Into<String>` ergonomikus középút (a hívó `&str`-t és `String`-et is átadhat), de generikus, ezért trait-objektumon (`dyn Storage`) át nem hívható, így a `Storage` trait szintjén nem alkalmazható; konkrét típus (`FileStore`) kényelmi metódusán igen. Az rkv-ban a `String` marad, mert a szerver a parancs szétszedésekor úgyis birtokolt kulcsot készít, a mérés pedig az (a) esetet mutatja tipikusnak; ha a profil sok frissítést és `&str` hívót mutatna, érdemes lenne újragondolni. A `FileStore` esetén is ugyanez a helyzet: a naplórekordhoz a kulcsot kölcsönözzük (`LogEntryRef`), utána pedig a térképbe mozgatjuk, másolat nélkül.

22.1.9 Rövid összefoglaló

A mérés-vezérelt optimalizálás a hipotézis, a mérés, a profilozás, a módosítás és az újramérés ciklusára épül. A határellenőrzés a nyelv szemantikája, nem garantált futási költség: a labor gépi kódja szerint a `0..len` ciklus indexelése és az iterátor ugyanazt a (67 utasításos, ellenőrzés nélküli) kódot adja, külső index esetén viszont az ellenőrzés megmarad. A laborban az allokációszámláló megmutatta, hogy a `FileStore::set` hívásonként két, a `save_snapshot` a teljes térképnyi felesleges másolatot készített; kölcsönzött `LogEntryRef`/`SnapshotRef` típusokkal, a naplóformátum megváltoztatása nélkül ezek megszűntek (a fejezet rögzített környezetében `4610 → 2610`, illetve `2003 → 2` allokáció; a 3. feladat megoldása a `set`-et tovább csökkenti 10-re). A `set` benchmark két, külön nevű esetet mér (új kulcs üres tárolóban; csere 1000 elemű tárolóban). A release-profilnál egyszerre egy beállítást mértünk: az `lto = "fat"` és a `codegen-units = 1` a bináris méretét 29%-kal csökkentette, a `panic = "abort"` viszont nem teljesítmény-, hanem viselkedésváltozás, és az `rkv-ffi` `catch_unwind`-je miatt nem kerül a profilba. **Továbbvitt eredmény.** A kölcsönzött naplórekord és snapshot, az allokációszámláló, a benchmarkok és a `[profile.release]` a Záróprojekt: az rkv teljes rendszerként alfejezet alapja.

22.1.10 Források és ellenőrzési hiányok

- Ellenőrizve (Rust 1.99.1, `x86_64` Linux, egyetlen gép): a `xor_*` függvények assembly-elemzése (`--emit asm`), az allokációszámlálós futások, a két `set` benchmark, a profilmátrix (méret és idő), a `catch_unwind` viselkedése `panic = "abort"` és `"unwind"` mellett, valamint a `rkv-core` tesztjei és a `clippy -D warnings`.
- A mért időértékek gépfüggők és egyetlen futásból származnak; a ±10%-os eltérések zajnak tekintendők. A méretek (bájt) ugyanezen a toolchainen reprodukálhatók.
- A `cargo flamegraph` és a `perf` ebben a fejezetben nem lett futtatva; a pontos parancssori kapcsolók verziófüggők. A flamegraph a következő lépés lehet, ha az allokációszám nem magyarázza meg az időt.
- A `criterion` főverzióját a `Cargo.toml`, a pontos alverziót a `Cargo.lock` rögzíti.

23 Beágyazott Rust kitekintés

23.1 no_std, embedded-hal és Embassy: Rust mikrovezérlőn

23.1.1 Motiváció

Egy tapasztalt C++ beágyazott fejlesztő pontosan tudja, milyen érzés, amikor a hibakeresés nem a logikáról, hanem arról szól, hogy valamelyik megszakításkezelő túlírta a másik regiszterét. A C++11–C++20 hiába ad RAII-t, okosmutatókat és erős típusokat, a tipikus beágyazott C/C++ kód mégis gyakran makrókba, `volatile` pointerekbe, kézi regisztercímekbe és globális perifériaobjektumokba fut. A `GPIOB->ODR |= (1 << 5)` stílusú sorok nem típusbiztosak, semmi nem akadályozza meg, hogy egy bemenetre konfigurált lábon feszültséget írjunk, és az ownership hiánya mindenféle rejtett aliasolási hibát enged.

A Rust válasza erre nem csupán „más szintaxis”, hanem a típusrendszer és az ownership bevonása a hardver erőforrásainak modellezésébe. Ugyanaz a Rust, amit a host-oldali rkv szerverben megismertünk, `std` nélkül is működik mikrovezérlőn. Amit a Pin, Unpin, async trait-ek és Send-határok és a Newtype, PhantomData, typestate és const generikusok alfejezetekben tanultunk, most a fizikai hardveren kap szerepet.

23.1.2 Fogalmak és szabályok

Definíció. A `no_std` környezet olyan Rust-környezet, amelyben a `std` könyvtár nem érhető el. A `std` a `core` és az `alloc` fölé épül, és olyan szolgáltatásokat ad, mint a thread, a fájlrendszer, a dinamikus időzítés vagy a standard I/O. A `no_std` tehát nem azt jelenti, hogy semmilyen könyvtár nincs.

Szabály. `#![no_std]` crateszintű attribútummal a crate lemond a `std`-ről. Ekkor a `core` könyvtár automatikusan elérhető: ez tartalmazza a legalapvetőbb elemeket, például az `Option`, `Result`, `Iterator`, `Clone`, `Copy`, `Send`, `Sync`, `Sized`, a `slice` és a `str` típusokat, valamint a `core::ptr`, `core::mem`, `core::ops`, `core::fmt` modulokat. Az `alloc` könyvtár a `Vec`, `String`, `Box`, `Arc` és hasonló, kupacot igénylő típusokat biztosítja. Egy **könyvtárban** ehhez elég az `extern crate alloc;` és az `alloc::…` import; a globális allokátort (`#[global_allocator]`) a **végső binárisnak** kell biztosítania, nem a könyvtárnak (lásd a háromszintű táblázatot lentebb).

Szabály. A `std`-től megfosztott bináris programban nincs normál `main`; helyette `#![no_main]` attribútumot használunk, és a belépési pontot a platform adja: Cortex-M-en a `cortex-m-rt` `#[entry]` makrója (és a reset-vektor), más célon saját, `#[unsafe(no_mangle)] pub extern "C"` belépési függvény (a 2024-es editionben a `no_mangle` `unsafe(...)` alakú). `#![no_std]` *binárisban* kötelező egy `#[panic_handler]` függvény, amely `fn(&PanicInfo) -> !` szignatúrájú, és soha nem tér vissza.

Három szint, három követelményhalmaz. A „no_std” szó három különböző dolgot takarhat, és csak a harmadikhoz kell minden:

Szint	Mit használ	Mit igényel	Ellenőrzés
1. core-only könyvtár	csak `core`	semmit a fordításhoz	`cargo check --lib` (host és célplatform)
2. alloc-ot importáló könyvtár	`extern crate alloc;` + `use alloc::vec::Vec;`	a *könyvtárhoz* nem kell allokátor, a használóhoz igen	`cargo check --lib --target thumbv7em-none-eabihf`
3. végső no_std bináris	az egész lánc	`#[global_allocator]` (ha van `alloc`), `#[panic_handler]`, belépési pont, linker-szkript és `memory.x`	célplatformos `cargo build`, majd hardveres vagy emulált futtatás

A hostteszt (`cargo test` a fejlesztői gépen) a `std` tesztfuttatóját használja, ezért *nem* bizonyítja, hogy a kód célplatformon fordul vagy fut; hardveres validációnak sem számít. Az `alloc`-os könyvtár célplatformos `cargo check --lib` lépése allokátor nélkül sikeres (kipróbáltuk `thumbv7em-none-eabihf`-re), a belőle linkelt bináris viszont `error: no global memory allocator found but one is required; link to std or add #[global_allocator] to a static item that implements the GlobalAlloc trait` hibával áll meg, amíg nincs allokátor.

Definíció. A beágyazott Rust-ökoszisztéma szokásos rétegződése: **PAC** (*Peripheral Access Crate*), **HAL** (*Hardware Abstraction Layer*), **embedded-hal traitek**, majd **driverek** és alkalmazás. A PAC a gyártó által definiált regiszterek szintjét adja, a HAL típusbiztos, kényelmes periféria-API-t, az embedded-hal traitek pedig gyártófüggetlen interfészeket.

Szabály. A perifériák szingletonok: minden fizikai periféria — például egy GPIO-blokk vagy egy lába — egyetlen Rust-értékként jelenik meg, amelynek nincs `Clone` és `Copy` implementációja, a konstruktora pedig privát. A perifériák együttes birtokosa a `Peripherals` struktúra, amelyet a `take()` ad vissza **legfeljebb egyszer**: ez *futásidejű* egyszeriség (egy statikus jelzőbit atomi cseréje; a második hívás `None`), nem fordítási idejű. A fordító ezután a már megszerzett értékek tulajdonjogát követi: egy lábat egy tulajdonos birtokol, az üzemmód-átalakítás elfogyasztja a régit. A safe API invariánsa tehát: *egy fizikai lábhoz egyszerre legfeljebb egy érték létezik*, amíg senki nem kerüli meg a `take()`-et (valós PAC-okban erre `unsafe steal()` való).

Definíció. A **typestate** minta a periféria üzemmódját típusparaméterként ábrázolja: például `Pin<P, Input<Floating>>` és `Pin<P, Output>` különböző típusok. A konfiguráció tehát fordítási időben rögzül, és a `Pin<P, Input<Floating>>` típuson egyszerűen nincs `set_high` metódus.

23.1.3 C++ és Rust összevetés

A tipikus C++ beágyazott GPIO-kezelő osztály így nézhet ki:

```
// cpp
class GpioPin {
  public:
    enum class Mode { Input, Output, InputPullUp };

    void configure(Mode mode) {
      mode_ = mode;
      // ... registers ...
```

```
    }

    void set_high() {
      // Feltételezi, hogy a láb Output módban van!
      // ... registers ...
    }

  private:
    Mode mode_;
};
```

A C++ megoldásban az üzemmód futási idejű adattag, a `set_high()` hívás helyessége a fejlesztő fegyelmén múlik. Rossz üzemmód esetén a hiba legfeljebb futás közben derül ki, vagy egyáltalán nem.

Rustban a typestate ezt a hibát fordítási időben zárja ki. Az alábbi `rkv-embedded` crate **teljes, lefordított és tesztelt** példa; a hardvert egy szimulált regiszter (`AtomicU16`) helyettesíti, így gép nélkül is futtatható. Ez önálló oktatópélda, nem az rkv része.

Kódblokk — fájl: `~/rust-labor/r050/rkv-embedded/Cargo.toml` · művelet: teljes csere · ellenőrzés: `cargo test`

```
[package]
name = "rkv-embedded"
version = "0.1.0"
edition = "2024"
rust-version = "1.89"

[dependencies]
[workspace]
```

Kódblokk — fájl: `~/rust-labor/r050/rkv-embedded/src/lib.rs` · művelet: teljes csere · ellenőrzés: `cargo test` (a `compile_fail` doctestek is)

```
//! Szimulált GPIO typestate-tel: önálló oktatópélda, nem az rkv része.
#![no_std]

use core::marker::PhantomData;
use core::sync::atomic::{AtomicBool, AtomicU16, Ordering};

/// Üzemmódok (típusszinten).
pub struct Input<MODE>(PhantomData<MODE>);
pub struct Floating;
pub struct PullUp;
pub struct Output;

/// Egy láb. `P` a lábszám a porton belül; csak a `Peripherals` hozhatja létre.
pub struct Pin<const P: u8, MODE> {
    _mode: PhantomData<MODE>,
}

/// A szimulált port kimeneti regisztere (ODR). Valós hardveren ez egy memóriára leképezett,
/// a lábak között megosztott regiszter; ezért a bitműveletek atomiak (hardveren: BSRR).
static ODR: AtomicU16 = AtomicU16::new(0);
/// Futásidejű egyszeriség: a `take()` legfeljebb egyszer sikerül.
static TAKEN: AtomicBool = AtomicBool::new(false);

/// A port összes lába. Nem `Clone`, nem `Copy`; a konstruktora privát.
pub struct Peripherals {
    pub pin5: Pin<5, Input<Floating>>,
    pub pin6: Pin<6, Input<Floating>>,
}
```

```
impl Peripherals {
    /// Első híváskor `Some`, utána mindig `None`.
    pub fn take() -> Option<Self> {
        if TAKEN.swap(true, Ordering::AcqRel) {
            None
        } else {
            Some(Peripherals { pin5: Pin::new(), pin6: Pin::new() })
        }
    }
}

impl<const P: u8, MODE> Pin<P, MODE> {
    /// Csak a modulon belül hívható; a lábszám a port tartományában van.
    fn new() -> Self {
        const { assert!(P < 16, "a portnak 16 lába van") };
        Pin { _mode: PhantomData }
    }
}

impl<const P: u8> Pin<P, Input<Floating>> {
    pub fn into_pull_up_input(self) -> Pin<P, Input<PullUp>> {
        Pin::new()
    }

    pub fn into_push_pull_output(self) -> Pin<P, Output> {
        // A kimenet alapállapota alacsony.
        ODR.fetch_and(!(1 << P), Ordering::AcqRel);
        Pin::new()
    }
}

impl<const P: u8> Pin<P, Output> {
    pub fn set_high(&mut self) {
        ODR.fetch_or(1 << P, Ordering::AcqRel);
    }

    pub fn set_low(&mut self) {
        ODR.fetch_and(!(1 << P), Ordering::AcqRel);
    }

    pub fn is_set_high(&self) -> bool {
        ODR.load(Ordering::Acquire) & (1 << P) != 0
    }
}

/// ```compile_fail,E0599
/// use rkv_embedded::Peripherals;
/// let p = Peripherals::take().unwrap();
/// let mut input = p.pin5;
/// input.set_high(); // bemeneti lábon nincs set_high
/// ```
///
/// ```
/// use rkv_embedded::Peripherals;
/// let p = Peripherals::take().unwrap();
/// let mut out = p.pin5.into_push_pull_output();
/// out.set_high();
/// assert!(out.is_set_high());
/// ```
///
/// ```compile_fail
/// use rkv_embedded::Peripherals;
```

```
/// let p = Peripherals::take().unwrap();
/// let _out = p.pin5.into_push_pull_output();
/// let _use_after_move = p.pin5; // az átalakítás elfogyasztotta a lábat
/// ```
#[cfg(doctest)]
struct _Doctests;
```

A külső (integrációs) teszt csak a nyilvános, safe API-t használja: egy lábat kér a `take()`-ből, a második `take()` sikertelen, a bemeneti láb `set_high`-t nem kap, az outputtá alakítás után a hívás fordul, a másik láb ugyanazon a megosztott regiszteren független bit.

Kódblokk — fájl: `~/rust-labor/r050/rkv-embedded/tests/gpio.rs` · művelet: új fájl · ellenőrzés: `cargo test`

```
use rkv_embedded::Peripherals;

#[test]
fn take_is_single_use_and_pins_follow_their_mode() {
    let p = Peripherals::take().expect("az első take sikeres");
    assert!(Peripherals::take().is_none(), "a második take sikertelen");

    let mut led = p.pin5.into_push_pull_output();
    assert!(!led.is_set_high());
    led.set_high();
    assert!(led.is_set_high());

    // A másik láb ugyanazon a megosztott regiszteren független bit.
    let mut other = p.pin6.into_push_pull_output();
    other.set_high();
    led.set_low();
    assert!(other.is_set_high() && !led.is_set_high());
}
```

Ellenőrzési pont. `cargo test`: az integrációs teszt és a három doctest (köztük a két `compile_fail`) zöld. A hibás használat ellenőrzött fordítói kimenete: bemeneti lábon `set_high` → `E0599` (no method named `set_high`); a privát konstruktor kívülről → `E0624` (`associated functionnewis private`); `Pin<20, _>` → `error[E0080]: evaluation panicked: a portnak 16 lába van` (a lábszám tartománya fordítási időben ellenőrzött).

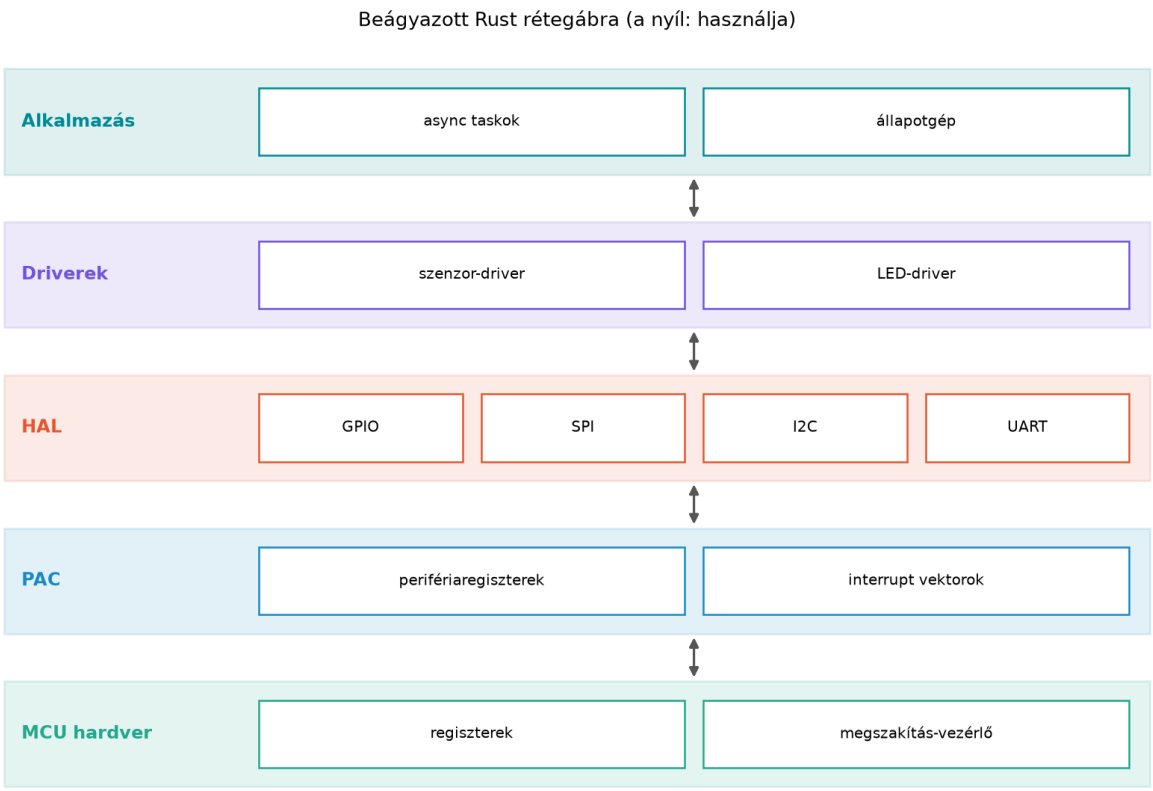
Három szabály, amelyet a példa mutat, valós hardveren is érvényes:

- **Támogatott `P` értékek.** A `Pin<const P: u8, _>` const generikus általános; a tartomány (itt `P < 16`) a konstruktorban `const { assert!(..) }`-tel fordítási időben kényszerített, hogy `Pin<20, _>` ne létezhessen.
- **Közös regiszter.** A port kimeneti regisztere a lábak között megosztott. A `set_high`/`set_low` ezért *atomi* bitművelet (hardveren a BSRR set/reset regiszter, nem olvasás–módosítás–írás), különben két láb kezelése megszakítással keresztezve elveszíthetné egymás bitjét.
- **Hol van az `unsafe`.** Valós PAC fölött a regiszterírás `unsafe`, és csak a legalsó rétegben él, `// SAFETY:` megjegyzéssel; a példa szimulált regisztere atomi, ezért safe.

A `Pin<P, Input<Floating>>` értéken nincs `set_high`: a metódus csak a `Pin<P, Output>` típuson létezik. Az `into_` konverziók felemésztik a régi értéket, és az új üzemmódnak megfelelő típust adják vissza. Ez a destruktív mozgatás pontosan azt az ownership-gondolkodást használja, amit az Ownership: minden értéknek egy tulajdonosa van alapozott meg.

23.1.4 A no_std könyvtárak rétegződése

Az alábbi ábra a beágyazott Rust tipikus rétegződését mutatja:



23.1. ábra: Beágyazott Rust rétegábra (a nyíl: használja) - A felső réteg az alatta lévőt használja. Az embedded-hal traitek a driverek és a HAL közötti hordozható illesztést adják.

A rétegek felülről lefelé haladva: az alkalmazás csak a driverek és a HAL típusait látja. A PAC a gyártói regiszterek elrendezését írja le, gyakran `unsafe` hozzáféréssel. A HAL ebből a nyers rétegből biztonságos absztrakciót épít, és az Unsafe Rust: szuperképességek és felelősség alfejezetben tanult elv érvényesül: az `unsafe` blokkok a legbelső rétegbe kerülnek, a kívülről látható API `safe`. Az embedded-hal traitek e fölé a gyártóspecifikus HAL fölé helyeznek egy közös interfész-réteget. Egy I2C-szenzor driver például nem az STM32 vagy RP2040 konkrét típusát, hanem az `embedded_hal::i2c::I2c` trait implementációját fogadja el, így a driver újrahasznosítható bármely támogatott MCU-n.

23.1.5 embedded-hal traitek: a hordozható illesztő

Az **embedded-hal 1.0.0** (a `Cargo.lock` szerinti verzió) a gyártófüggetlen illesztőket trait-csoportokban adja (`digital`, `i2c`, `spi`, `serial` stb.). A driverek nem konkrét HAL-típust, hanem ezeket a traiteket fogadják el. Egy kimeneti láb vezérlése és a hoston futtatható mock implementáció:

```
use embedded_hal::digital::{ErrorType, OutputPin, StatefulOutputPin};

/// Bármely embedded-hal kimeneti láb, a konkrét HAL-tól függetlenül.
```

```rust
fn blink_once<P: OutputPin>(pin: &mut P) -> Result<(), P::Error> {
    pin.set_high()?;
    pin.set_low()?;
    Ok(())
}

/// Mock láb a hosttesthez: nem hardver, csak a trait szerződését valósítja meg.
struct MockPin {
    high: bool,
}

impl ErrorType for MockPin {
    type Error = core::convert::Infallible;
}

impl OutputPin for MockPin {
    fn set_low(&mut self) -> Result<(), Self::Error> {
        self.high = false;
        Ok(())
    }
    fn set_high(&mut self) -> Result<(), Self::Error> {
        self.high = true;
        Ok(())
    }
}

impl StatefulOutputPin for MockPin {
    fn is_set_high(&mut self) -> Result<bool, Self::Error> {
        Ok(self.high)
    }
    fn is_set_low(&mut self) -> Result<bool, Self::Error> {
        Ok(!self.high)
    }
}

fn main() {
    let mut pin = MockPin { high: false };
    blink_once(&mut pin).unwrap();
    assert!(pin.is_set_low().unwrap());
}
```

A `blink_once` generikus, ezért ugyanaz a függvény fut az STM32 HAL-ján, az RP2040-en és a `MockPin`on; a mock hoston is lefut, és a trait-szerződést teszteli. A valódi hardveren a `set_high`/`set_low` a HAL implementációjában a regiszterírást végzi; a hiba típusa HAL-onként eltér, ezt köti össze a `P::Error`. A saját `rkv-embedded` `Pin` típusára `impl OutputPin` adható, és akkor egy erre épülő driver változtatás nélkül használja. A példát a rögzített toolchainnel (`embedded-hal 1.0.0`) lefordítottuk és hoston futtattuk.

23.1.6 Embassy: a megszakításokra épülő async feladatmodell

A hagyományos beágyazott C++ kódban a nem blokkoló várakozást gyakran megszakításokkal, callbackekkel vagy eseményzászlókkal oldjuk meg. A Rust async a várakozási logikát lineáris kódban teszi írhatóvá, az Embassy pedig ezt a modellt viszi beágyazott környezetbe: az executor együttműködő (kooperatív) ütemezéssel futtatja a taskokat, a task akkor alszik el, amikor `.await`-en vár, és a hardveres megszakítás (időzítő, GPIO-él, UART) a `Waker`-en keresztül ébreszti fel. Nincs operációs rendszer, nincs szál; a taskok statikusan foglalódnak, kupac nem kell. A `Future` pollozását a Future-ök és async/await alfejezetben ismertük meg.

Két példát adunk, és **külön jelöljük a státuszukat**:

1. *Host-példa* (lefordítva és lefuttatva): ugyanazok az Embassy-crate-ek (executor, time, sync) a gépünkön futnak, a LED szimulált. A megosztási minta itt ellenőrizhető.
2. *Célplatform-példa* (lefordítva és linkelve `thumbv7em-none-eabihf`-re, hardveren nem futtatva): megmutatja, honnan jön a `#[panic_handler]`, és milyen feature-ök kellenek.

Használt verziók és feature-ök (a `Cargo.lock` szerint): `embassy-executor 0.10.0`, `embassy-time 0.5.1`, `embassy-sync 0.8.0`, `critical-section 1.2.0`, `embassy-futures 0.1.2`, `cortex-m-rt 0.7.7`, `panic-halt 1.0.0`.

23.1.6.1 Host-példa: két task ugyanazt a LED-et használja

Kódblokk — fájl: `embassy-led/Cargo.toml` · művelet: `cargo new embassy-led`, majd a fájl cseréje · ellenőrzés: `cargo run`

```
[package]
name = "embassy-led"
version = "0.1.0"
edition = "2024"

[dependencies]
critical-section = { version = "1", features = ["std"] }
embassy-executor = { version = "0.10", features = ["platform-std", "executor-thread"] }
embassy-sync = "0.8"
embassy-time = { version = "0.5", features = ["std"] }

[workspace]
```

Kódblokk — fájl: `embassy-led/src/main.rs` · művelet: teljes csere · ellenőrzés: `cargo run`

```
//! Két Embassy-task közös LED-je, hoston futtatva (a LED szimulált).
use embassy_executor::Spawner;
use embassy_sync::blocking_mutex::raw::CriticalSectionRawMutex;
use embassy_sync::mutex::Mutex;
use embassy_time::{Duration, Timer};

/// Szimulált LED: a hardveres változatban egy kimeneti láb.
struct Led {
    on: bool,
}

impl Led {
    const fn new() -> Self {
        Led { on: false }
    }

    fn toggle(&mut self, who: &str) {
        self.on = !self.on;
        println!("{who}: LED {}", if self.on { "be" } else { "ki" });
    }
}

/// Statikus tároló: a taskok `'static` hivatkozást kapnak; nincs heap.
static LED: Mutex<CriticalSectionRawMutex, Led> = Mutex::new(Led::new());

#[embassy_executor::task]
async fn fast_blink() {
    loop {
        {
            // A zár csak a szinkron művelet idejére él...
            let mut led = LED.lock().await;
```

```
            led.toggle("gyors");
        } // ...itt elengedődik, mielőtt várakoznánk.
        Timer::after(Duration::from_millis(300)).await;
    }
}

#[embassy_executor::task]
async fn slow_blink() {
    loop {
        LED.lock().await.toggle("lassú");
        Timer::after(Duration::from_millis(500)).await;
    }
}

#[embassy_executor::main]
async fn main(spawner: Spawner) {
    spawner.spawn(fast_blink().unwrap());
    spawner.spawn(slow_blink().unwrap());
    Timer::after(Duration::from_millis(1100)).await;
    println!("kész");
    std::process::exit(0);
}
```

A megosztott LED egy `static Mutex<CriticalSectionRawMutex, Led>`: a taskok `'static` hivatkozást kapnak, kupac és `Arc` nélkül. A két `#[embassy_executor::task]` függvény a `Spawner`-ből indul (`spawner.spawn(task().unwrap())`). A mutex-őr (`guard`) csak a szinkron `toggle` idejéig él: a `fast_blink` belső blokkban engedi el, mielőtt `Timer::after(..).await`-re vár, így várakozás közben a másik task hozzáfér a LED-hez. Egy őr, amelyet a `.await` fölött tartanánk, a másik taskot kiéheztetné.

Ellenőrzött futás (a sorrend az időzítéstől függ):

```
lassú: LED be
gyors: LED ki
gyors: LED be
lassú: LED ki
gyors: LED be
gyors: LED ki
lassú: LED be
kész
```

23.1.6.2 Melyik megosztási mód mikor?

A host-oldali `Arc<Mutex<T>>` (Mutex, RwLock, csatornák és a megosztott rkv) beágyazott környezetben nem vihető át változtatás nélkül: kupacot, operációs rendszert és blokkoló zárat feltételez, a megszakításkezelő pedig nem várakozhat. A választást az dönti el, *ki fér hozzá*, *kell-e heap vagy atomika*, és *blokkolhat-e az ISR*:

Helyzet	Eszköz	Ki fér hozzá	Heap / atomika	Blokkolhat az ISR?
két task, ugyanazon az executoron	`Mutex<NoopRawMutex, T>` vagy `Channel<NoopRawMutex, ..>`	csak taskok, egy executor; **nem** `Sync`, ezért nem lehet `static` (`StaticCell` kell)	nincs heap, nincs atomika	nem használható ISR-ből

Helyzet	Eszköz	Ki fér hozzá	Heap / atomika	Blokkolhat az ISR?
task–task, több executor vagy ISR is érinti	`Mutex<CriticalSectionRawMutex, T>` (a példa)	task-ok és (a `blocking_mutex` formával) ISR-ek	nincs heap; a kritikus szakasz a megszakításokat letiltja	az ISR nem várhat `.await`-en; a kritikus szakasz alatt a megszakítások késnek
ISR → task jelzés vagy üzenet	`Signal::signal`, `Channel::try_send`	az ISR küld, a task `.await`-tel fogad	nincs heap	**nem blokkol**: `signal` azonnali, a tele csatorna `Err`-t ad
ISR és task közös számláló vagy zászló	`AtomicU32`, `AtomicBool`	mindkettő	atomika (a MCU támogatásától függ)	nem blokkol

A táblázat `Signal`/`Channel` sora lefordított és futtatott példán alapul: egy másik szál játssza az ISR-t (hoston nincs megszakítás), és `signal`-lel, illetve `try_send`-del jelez a task felé:

Kódblokk — fájl: `embassy-led/examples/isr_to_task.rs` · művelet: új fájl · ellenőrzés: `cargo run --example isr_to_task` → `jelzés: 7, esemény: 7`

```
//! ISR-szerű jelzés: a küldő oldal nem várakozhat (`signal`, `try_send`), a task igen.
use embassy_executor::Spawner;
use embassy_sync::blocking_mutex::raw::CriticalSectionRawMutex;
use embassy_sync::channel::Channel;
use embassy_sync::signal::Signal;
use embassy_time::{Duration, Timer};

static BUTTON: Signal<CriticalSectionRawMutex, u32> = Signal::new();
static EVENTS: Channel<CriticalSectionRawMutex, u32, 4> = Channel::new();

/// Megszakításkezelő helyett egy szál játssza az ISR-t (hoston nincs megszakítás).
fn fake_isr(n: u32) {
    BUTTON.signal(n); // nem blokkol
    let _ = EVENTS.try_send(n); // tele csatornánál Err, nem várakozás
}

#[embassy_executor::task]
async fn consumer() {
    let first = BUTTON.wait().await;
    let second = EVENTS.receive().await;
    println!("jelzés: {first}, esemény: {second}");
    std::process::exit(0);
}

#[embassy_executor::main]
async fn main(spawner: Spawner) {
    spawner.spawn(consumer().unwrap());
    std::thread::spawn(|| fake_isr(7));
    Timer::after(Duration::from_millis(500)).await;
}
```

A negatív eset is ellenőrzött: ha a statikus `Mutex<CriticalSectionRawMutex, Led>`-et `Mutex<NoopRawMutex, Led>`-re cseréljük, a fordító `error[E0277]:*mut ()cannot be shared between threads safely` hibával leáll, mert a `NoopRawMutex` nem `Sync`.

23.1.6.3 Célplatform-példa: a panic handler külön crate-ből jön

A `#[embassy_executor::main]` makró az executor indítását és a belépési pontot hozza létre. **Nem generál panic handlert.** Egy `#![no_std]` binárisban a `#[panic_handler]`-t külön kell megadni: `panic-halt` (megáll), `panic-probe`, vagy saját `#[panic_handler]`. Hiányában a linkelés előtt már a fordító leáll.

Kódblokk — fájl: `embassy-target/Cargo.toml` · művelet: `cargo new embassy-target`, majd a fájl cseréje · ellenőrzés: `cargo build`

```
[package]
name = "embassy-target"
version = "0.1.0"
edition = "2024"

[dependencies]
cortex-m-rt = "0.7"
embassy-executor = { version = "0.10", features = ["platform-cortex-m", "executor-thread"] }
embassy-futures = "0.1.2"
panic-halt = "1"

[workspace]
```

Kódblokk — fájl: `embassy-target/.cargo/config.toml` és `embassy-target/memory.x` · művelet: új fájlok (a `memory.x` egy általános Cortex-M4 memóriatérkép; a valós MCU-hoz a lapkaleírás szerint igazítandó) · ellenőrzés: `cargo build`

```
[build]
target = "thumbv7em-none-eabihf"

[target.thumbv7em-none-eabihf]
rustflags = ["-C", "link-arg=-Tlink.x"]
```

```
MEMORY
{
  FLASH : ORIGIN = 0x08000000, LENGTH = 256K
  RAM   : ORIGIN = 0x20000000, LENGTH = 64K
}
```

Kódblokk — fájl: `embassy-target/src/main.rs` · művelet: teljes csere · ellenőrzés: `rustup target add thumbv7em-none-eabihf && cargo build` → létrejön a `target/thumbv7em-none-eabihf/debug/embassy-target`

```
#![no_std]
#![no_main]

use core::sync::atomic::{AtomicU32, Ordering};

use embassy_executor::Spawner;
use embassy_futures::yield_now;
use panic_halt as _; // a #[panic_handler]-t ez a crate adja

static TICKS: AtomicU32 = AtomicU32::new(0);

#[embassy_executor::task]
async fn counter() {
    loop {
        TICKS.fetch_add(1, Ordering::Relaxed);
        yield_now().await;
    }
}
```

```rust
#[embassy_executor::main]
async fn main(spawner: Spawner) {
    spawner.spawn(counter().unwrap());
}
```

Ellenőrzött eredmény: a `use panic_halt as _;` sorral a bináris lefordul és linkelődik (a debug ELF mérete 1 065 440 bájt); a sor nélkül `error:#[panic_handler]function required, but not found`. A példa **nem** használ időzítőt: az `embassy-time` a célon egy time drivert igényel, amelyet a választott HAL ad (például az `embassy-stm32` vagy az `embassy-rp`), és amelyet itt nem választottunk. Ezért ez a példa a fordítás/linkelés és a panic-kezelés igazolása, nem működő firmware.

23.1.7 Labor: no_std kísérlet

Munkakönyvtár: `~/rust-labor/r050/`

A labor célja, hogy a `no_std` korlátait kis léptékben megtapasztaljuk, mielőtt valódi mikrokontrolleres targetre váltanánk. Készíts egy `no_std` könyvtárcrate-et, amely csak a `core`-t használja:

```bash
mkdir -p ~/rust-labor/r050
cd ~/rust-labor/r050
cargo new --lib nostd-core --vcs none
cd nostd-core
```

A `src/lib.rs` tartalma:

```rust
// rust
#![no_std]

pub fn clamp(value: i32, min: i32, max: i32) -> i32 {
    if value < min {
        min
    } else if value > max {
        max
    } else {
        value
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn clamp_bounds() {
        assert_eq!(clamp(5, 0, 10), 5);
        assert_eq!(clamp(-3, 0, 10), 0);
        assert_eq!(clamp(11, 0, 10), 10);
    }
}
```

Fordítsd le és futtasd a teszteket:

```
cargo test
```

A könyvtárcrate (1. szint) `no_std` módban is gond nélkül fordul, mert nem használ `std`-t; a `cargo test` itt hostteszt, nem célplatformos igazolás. Célplatformon: `rustup target add thumbv7em-none-eabihf && cargo check --lib --target thumbv7em-none-eabihf`. Próbáld ki, mi történik, ha `Vec`-et akarsz létrehozni a `clamp` függvényben: `Vec` import nélkül a fordító `cannot find type` hibát ad, mert a `no_std` prelúdiumban nincs `Vec`. Az `extern crate alloc;` és a `use alloc::vec::Vec;` sorokkal (2. szint) a könyvtár lefordul allokátor nélkül is; allokátort csak a végső binárisnak kell adni (3. szint).

Opcionális labor. A valós beágyazott futtatás QEMU-ban vagy fizikai kártyán ennek a fejezetnek a keretein túlmutat, de a teljesség kedvéért érdemes megemlíteni. Egy ARM Cortex-M target esetén például a `thumbv7em-none-eabihf` target telepítése után egy hasonló vázlat fordítható:

```
rustup target add thumbv7em-none-eabihf
```

A teljes bináris futtatásához linker-szkript, boot-kód és a célhardverhez illeszkedő `memory.x` is szükséges; ezek tartalma a választott MCU-tól függ (a fejezet későbbi `embassy-target` példája egy általános Cortex-M4 memóriatérképet használ). A QEMU-s emuláció beállításait a `cortex-m` és `cortex-m-rt` crate-ek dokumentációja írja le.

23.1.8 Tipikus hibák és átállási csapdák

1. **`Vec` és `String` használata `no_std` kódban automatikusan.** A `std`-hez kötött típusok nem érhetők el. Egy no_std **könyvtárban** az `alloc` engedélyezése az `extern crate alloc;` sorral történik, és a könyvtár fordításához **nem** kell allokátor; a **végső binárisban** ezen felül `#[global_allocator]` is kell a futtatáshoz. A Box, Rc, Arc és Weak alfejezetben tanult `Box` is csak akkor érhető el, ha az `alloc` jelen van; a lefoglalás futásidejű feltétele a bináris allokátora.

2. **A `panic_handler` hiánya.** A `no_std` binárisban a `panic!` makró futási idejű hibát jelez, de ennek kezelése nincs beépítve. Fordítói/linkelési hiba keletkezik, ha a program nem definiál `#[panic_handler]`-t. A host-oldali `std` automatikusan ad panichandler-t, a beágyazott világban ezt kézzel kell megadni.

3. **A periféria-singleton megkerülése és a host-minták átvétele.** A `take()` által visszaadott periféria nem `Clone` és nem `Copy`. A vágy, hogy két helyen is hozzáférjünk ugyanahhoz a perifériához, `unsafe` pointerekhez vagy `steal()`-hez csábít. A helyes út: a perifériát egyetlen helyen birtokoljuk, és a megosztást a környezethez illő eszközzel oldjuk meg (lásd a fenti táblázatot). A host-oldali `Arc<Mutex<..>>` itt nem jó kiindulás: kupacot és blokkoló zárat feltételez, a megszakításkezelő pedig nem várakozhat.

4. **A typestate kikerülése.** Az `into_push_pull_output` utáni `Pin<P, Output>` típus nem engedi a bemeneti konfigurációs metódust, de ha a fejlesztő a hardverregisztert `unsafe` közvetlenül írja, akkor a typestate által adott garancia elveszik. A szabály itt is: az `unsafe` csak a legalsó rétegben, világos `// SAFETY:` indoklással.

23.1.9 Feladatok

1. **Fogalmi kérdés.** Mondd meg, melyik típus érhető el `#![no_std]` környezetben `alloc` nélkül: `Option`, `String`, `Vec`, `Result`, `Arc`, `core::iter::Map`. Indokold.

2. **C++-kód átírása idiomatikus Rustra.** Adott egy C++ generátorfüggvény, amely egy `std::vector<int>`-et ad vissza, és egy beágyazott hőmérséklet-szenzor kalibrációs értékeit olvassa. Írd át a logikát `no_std`-kompatibilis Rustra, amely statikusan lefoglalt puffert vagy a `heapless::Vec` típust használ. (A `heapless` útvonalat ellenőrizni kell a docs.rs-en.)

3. **Typestate GPIO implementáció.** Hozz létre egy önálló `rkv-embedded` gyakorló crate-et a `~/rust-labor/r050/rkv-embedded/` könyvtárban (nem az rkv workspace része). Valósíts meg `Pin<const P: u8, MODE>` típust `Input<Floating>`, `Input<PullUp>` és `Output` módokkal, `into_pull_up_input`, `into_push_pull_output`, `set_high`, `set_low` metódusokkal, a szimulált regiszterrel, a `Peripherals::take()` egyszeri factoryval. Írj külső integrációs tesztet a safe API-val: a második `take()` sikertelen, bemeneti láb `set_high` hívása `compile_fail` doctesttel bukik, az outputtá alakítás utáni hívás fordul.

4. **Megosztási döntés.** Az Embassy async LED-villogtatóban a LED-et két taskból is vezérelni szeretnénk. Milyen típusú megosztást választanál, és miért? Add meg a teljes kódot a `Spawner`, a statikus `Mutex` és a két `#[task]` felhasználásával, a szükséges crate-ekkel és feature-ökkel, és futtasd hoston.

23.1.10 Megoldások

1. Az `Option`, `Result` és a `core::iter::Map` elérhetők, mert a `core`-ban vannak. A `String`, `Vec` és `Arc` nem elérhetők `alloc` nélkül, mert kupacfoglalást igényelnek. Az `alloc` ezeket az `extern crate alloc;` sorból adja; a **könyvtár** fordításához nem kell allokátor, a futtatáshoz a **végső bináris** `#[global_allocator]`-a kell.

2. A C++ `std::vector<int>` helyett statikus puffer vagy `heapless::Vec` használható. Egy lehetséges Rust-változat:

```
// rust
#![no_std]

// A heapless::Vec használatakor a crate a Cargo.toml-ban:
// heapless = "0.8" (ellenőrizendő a főverzió)
use heapless::Vec;

pub fn read_calibration_values<const N: usize>(sensor: &dyn Sensor)
    -> Vec<i32, N>
{
    let mut values = Vec::new();
    for i in 0..N {
        if let Some(v) = sensor.read_calibration(i) {
            // A push itt is hibát ad, ha a puffer tele van.
            let _ = values.push(v);
        }
    }
    values
}

pub trait Sensor {
    fn read_calibration(&self, index: usize) -> Option<i32>;
}
```

A `Vec<i32, N>` a `heapless` crate típusa, amely statikus kapacitással rendelkezik. A `no_std` ezt tökéletesen támogatja, mert nincs dinamikus allokáció.

3. A megoldás a fejezetben közölt `rkv-embedded` crate (`src/lib.rs`, `tests/gpio.rs`, `Cargo.toml`): a `Peripherals::take()` a futásidejű egyszeriséget, a privát `Pin::new()`

és az átalakításokat elfogyasztó `into_*` függvények a safe API invariánsát tartják fenn. Ellenőrzés: `cargo test`.

4. A két task közös LED-jéhez `embassy_sync::mutex::Mutex<CriticalSectionRawMutex, Led>` jó választás: nem blokkoló, async-barát (a `lock().await` a taskot altatja, nem a szálat), nincs heap, és két executor vagy ISR-érintettség esetén is megfelel. Egyetlen executor esetén `NoopRawMutex` is elég volna, de az nem `Sync`, ezért `StaticCell` kell hozzá. A teljes, lefordított és futtatott megoldás a fejezet `embassy-led` példája: helyes import (`embassy_sync::mutex::Mutex` és `embassy_sync::blocking_mutex::raw::CriticalSectionRawMutex`), statikus inicializálás (`static LED: Mutex<..> = Mutex::new(Led::new())`), két `#[embassy_executor::task]` függvény és két `spawner.spawn(..)` hívás. A mutex-őr csak a szinkron `toggle` idejére marad meg, az időzített várakozás előtt elengedjük. Szükséges crate-ek és feature-ök: `embassy-executor 0.10` (`platform-std`, `executor-thread`), `embassy-time 0.5` (`std`), `embassy-sync 0.8`, `critical-section 1` (`std`).

23.1.11 Rövid összefoglaló

A `no_std` környezet a `std` könyvtár elhagyását jelenti, de a `core` mindig elérhető, az `alloc` opcionális. A beágyazott Rust-ökoszisztéma rétegei: PAC → HAL → embedded-hal traitek → driverek. A perifériák szingletonok: a `take()` futásidejűleg egyszeri, utána a fordító követi a tulajdonjogot. A typestate minta a GPIO-láb üzemmódját típusparaméterként kezeli, így a `Pin<P, Output>` rendelkezik `set_high`-jal, a `Pin<P, Input<Floating>>` nem; a lábszám tartománya fordítási időben ellenőrzött, a közös regiszter atomi bitművelettel védett. Az Embassy a Rust async modellt viszi beágyazott környezetbe: a taskok megszakítások által ébresztett, kooperatívan ütemezett futurek. A megosztást a környezethez kell igazítani (`NoopRawMutex`, `CriticalSectionRawMutex`, `Signal`/`Channel` az ISR–task határon, atomikák), nem a host `Arc<Mutex>`-ét kell átvenni. A `#![no_std]` binárisban `#![no_main]` és `#[panic_handler]` szükséges, utóbbit külön crate (például `panic-halt`) adja, nem az `#[embassy_executor::main]`.
Továbbvitt eredmény. A Összefoglalás: C++→Rust fordítási kézikönyv és továbbtanulás ezt a rétegződést és a no_std-elméletet használja tovább: a beágyazott és host-oldali Rust ugyanazokra a típusrendszeri alapokra épül, és a C++-ból hozott minták itt is az ownership, a generikusok és a trait-ek nyelvére fordíthatók.

23.1.12 Források és ellenőrzési hiányok

- Ellenőrizve (Rust 1.99.1; hoston és a `thumbv7em-none-eabihf` célon):
  - az `rkv-embedded` crate tesztjei és doctestjei;
  - a privát konstruktor és a tartomány-ellenőrzés hibái;
  - az `embassy-led` host-példa és az `isr_to_task` példa futása;
  - a `NoopRawMutex` statikus hibája;
  - az `embassy-target` linkelése `panic-halt`-tal, a `#[panic_handler]` hiányának hibája.
- Nem futtatott: bármely példa valódi mikrovezérlőn vagy QEMU-ban. Az `embassy-target` nem tartalmaz time drivert, ezért nem működő firmware; a `memory.x` általános, a valós MCU-hoz igazítandó.
- Nyelvi források: Rust Reference 1.99.1 (`no_std`, `no_main`, `panic_handler`) és az embedded-hal 1.0.0 dokumentáció (`OutputPin`, `StatefulOutputPin`, `ErrorType`). A fejezet `embedded-hal 1.0.0`-val fordított mock példáját hoston futtattuk.

- Az Embassy API-ja gyorsan fejlődik (a feature-nevek is változtak, például `platform-std`, `platform-cortex-m`); a közölt verziók a példákat rögzítik, újabb verzióhoz a docs.rs/embassy-executor a forrás.
- A `heapless::Vec` leírása: docs.rs/heapless (a `heapless` verzióját a labor `Cargo.lock`-ja rögzíti).

VII. RÉSZ – Záróprojekt és összefoglalás

24 Záróprojekt és összefoglalás

24.1 Záróprojekt: az rkv teljes rendszerként

24.1.1 Motiváció: a rendszer mint egész

Egy C++-projekt zárásakor az átadás általában így néz ki: a CMake-listák, a tesztek, a CI-konfiguráció és a dokumentáció külön-külön rendben van, de a teljes rendszer integrációja az utolsó héten derül ki. Az rkv esetén ugyanezt a pillanatot éljük át Rustban, azzal a különbséggel, hogy a cargo workspace és a beépített tesztinfrastruktúra az integráció nagy részét már lefedi. A cél most nem új nyelvi elem megtanítása, hanem a korábbi alfejezetek eredményeinek egyetlen, átadható rendszerré gyúrása, **egy** API-készlettel és **egy** adatmodellel minden rétegen.

A korábbi laborokban elkészült a `Store` típus, a `Storage` trait, a `MemStore` és `FileStore` implementáció, a `SharedStore`, a CLI, a tokio-szerver, a C ABI-s FFI, a proc-macro crate és a cxx-hídon keresztül elérhető C++-kliens. A záróprojekt ezeket a darabokat egységesíti, teszteli, méri és dokumentálja. C++-ban a Catch2/GoogleTest + CMake + CTest + clang-tidy kombináció a szokásos; Rustban ezt a `cargo test`, `clippy`, `fmt` és adott esetben a Miri váltja ki, szorosabban a nyelvbe integrálva.

24.1.2 Követelménylista és architektúra

Az rkv átvételi követelményei konkrét, mérhető feltételek. A listát a Tesztelés és dokumentáció, a C++-együttműködés és a Naplózás és megfigyelhetőség alfejezetek eredményeiből állítjuk össze:

1. **Funkcionális**: a `Storage` traites `MemStore` és `FileStore` átmegy a generikus szerződés-teszten; a `FileStore` adatai SET/DEL, újranyitás és kompaktálás után is ugyanabban az állapotban állnak helyre.
2. **Protokoll**: a szöveges protokoll `GET`, `SET`, `DEL`, `LIST [<prefix>]` parancsai működnek a szerveren és a CLI-n keresztül; a `LIST` keretezése minden rétegen azonos.
3. **Párhuzamos hozzáférés**: a megosztott `SharedStore` szálbiztos, a terheléses teszt nem okoz holtpontot vagy adatvesztést.
4. **FFI**: a C ABI-s és a cxx-hidas C++-kliens egyaránt eléri a tárolót; a C++ átvételi teszt a ténylegesen generált hídra épül.
5. **Minőség**: `cargo fmt --check`, `cargo clippy -- -D warnings`, `cargo test`, `cargo doc --no-deps`, valamint az `rkv-ffi` unsafe részeire a Miri futtatása sikeres.
6. **Dokumentáció**: a rustdoc figyelmeztetés nélkül generálódik (`RUSTDOCFLAGS="-D warnings" cargo doc --no-deps --workspace`: törött intra-doc hivatkozás hiba), és a doc-tesztek lefutnak. A `missing_docs` lintet **nem** kényszerítjük ki: bekapcsolva az `rkv-core` 51, az `rkv-server` 9 hiányzó dokumentációs figyelmeztetést ad, ezért a „minden publikus elem dokumentált” állítás nincs igazolva; ez nyitott, kézi vizsgálatot igénylő pont.

Követhetőségi mátrix. Minden követelményhez nevesített ellenőrzés tartozik. A „Típus” oszlop szerint *automatikus* az, amit a `scripts/accept.sh` (lásd lent) futtat, a nevesített tesztek és a minőségkapuk (fmt, clippy, test, doctest, doc) egyaránt; *kézi* az, ami emberi vizsgálat

vagy külön eszköz. Egy tesztnév a `cargo test … -- --exact` szűrővel pontosan **1** tesztet kell futtasson, mert a nulla teszt is `ok`-val és 0-s kilépési kóddal végződik (`running 0 tests`, `1 filtered out`), vagyis egy elírt névvel a követelmény látszólag teljesülne.

#	Követelmény	Ellenőrzés (csomag · teszt)	Típus
1	`MemStore` a szerződésen	`rkv-core` · `memstore_satisfies_storage_contract`	automatikus
1	`FileStore` a szerződésen	`rkv-core` · `filestore_satisfies_storage_contract`	automatikus
1	Újranyitás és kompaktálás után azonos állapot	`rkv-core` · `state_survives_reopen_and_compaction` (`restore_state`)	automatikus
2	`LIST` szűrés és keretezés a szerveren	`rkv-server` · `server_list_prefix_returns_filtered_pairs` (`list_prefix`)	automatikus
3	Szálbiztos `SharedStore`	`rkv-core` · `shared_store_works_across_threads`, `file_backend_survives_concurrent_writers`	automatikus
4	Híd Rust-oldali szerződése	`rkv-cxx` · `tests::contract_holds_on_the_rust_side`	automatikus
4	Kapcsolatlimit a szerveren	`rkv-server` · `connection_limit_rejects_the_excess_client`	automatikus
4	C++ kliens a generált hídon	`acceptance.cpp` fordítása és futtatása (g++, lásd a C++ átvételi teszt)	kézi / külön parancs
5	fmt, clippy, test, doc	a 4. lépés parancsai, kilépési kód 0	automatikus (parancs)
5	Miri az `rkv-ffi`-n	`cargo +nightly miri test -p rkv-ffi`	kézi / külön parancs
6	rustdoc, doctestek	`RUSTDOCFLAGS="-D warnings" cargo doc --no-deps --workspace`; `cargo test --doc --workspace`	automatikus (parancs)
6	Teljes dokumentáltság	nincs kikényszerítve (`missing_docs` figyelmeztetések)	kézi, nyitott

Kódblokk — fájl: `scripts/accept.sh` · művelet: új fájl a workspace gyökerében · ellenőrzés: `bash scripts/accept.sh` → minden sor `OK`, kilépési kód 0

```
#!/usr/bin/env bash
# Átvételi mátrix: minden sor egy nevesített teszt (pontosan 1 futott és 1 sikeres), a
# minőségkapuk pedig külön parancsok. A cargo kilépési kódját is vizsgáljuk.
set -u
fail=0
check() { # <követelmény> <csomag> <tesztcél: --lib | --test NÉV> <teszt teljes neve>
  local req=$1 pkg=$2 target=$3 name=$4 out status
```

```
  # shellcheck disable=SC2086
  out=$(cargo test -p "$pkg" $target -- --exact "$name" 2>&1); status=$?
  if [ "$status" -eq 0 ] && [ "$(grep -c '^test result: ok\. 1 passed; 0 failed' <<<"$out")"
   ↪  -eq 1 ]; then
    echo "OK   [$req] $pkg $name"
  else
    echo "FAIL [$req] $pkg $name (kilépési kód $status, nem pontosan 1 sikeres teszt)"
    echo "$out" | tail -n 40   # a teljes releváns diagnosztika megmarad
    fail=1
  fi
}
gate() { # <név> <parancs...>
  local name=$1; shift
  if "$@" >/tmp/accept-gate.log 2>&1; then echo "OK   [gate] $name"
  else echo "FAIL [gate] $name"; tail -n 40 /tmp/accept-gate.log; fail=1; fi
}
check 1 rkv-core "--test memstore"  memstore_satisfies_storage_contract
check 1 rkv-core "--test filestore" filestore_satisfies_storage_contract
check 1 rkv-core "--test restore_state" state_survives_reopen_and_compaction
check 2 rkv-server "--test list_prefix" server_list_prefix_returns_filtered_pairs
check 3 rkv-core "--test shared" shared_store_works_across_threads
check 3 rkv-core "--test shared_backends" file_backend_survives_concurrent_writers
check 4 rkv-cxx "--lib" tests::contract_holds_on_the_rust_side
check 4 rkv-server "--test server" connection_limit_rejects_the_excess_client
gate fmt     cargo fmt --all --check
gate clippy  cargo clippy --workspace --all-targets -- -D warnings
gate test    cargo test --workspace
gate doctest cargo test --doc --workspace
gate doc     env RUSTDOCFLAGS="-D warnings" cargo doc --no-deps --workspace
exit $fail
```

Ellenőrzött eredmény (Rust 1.99.1): a szkript a nyolc tesztsorra és az öt minőségkapura (fmt, clippy, test, doctest, doc) `OK`-t ír és 0-val lép ki; a `RUSTDOCFLAGS="-D warnings" cargo doc --no-deps --workspace` figyelmeztetés nélkül lefut. A segédmodul (`tests/common/mod.rs`, `assert_storage_contract`) hívói a `memstore.rs` és a `filestore.rs` (lásd Tesztelés és dokumentáció); az 1. követelmény két első sora ezen át fut. A Miri és a C++ teszt ebben a szkriptben nincs, azokat külön, a lenti lépésekben futtatjuk.

A rendszer rétegei:

```
Cargo-függőségek (a nyíl: "függ tőle", egyirányú):

  rkv-cli      rkv-core
  rkv-server   rkv-core
  rkv-ffi      rkv-core
  rkv-cxx      rkv-core
  rkv-core     rkv-derive          (path-függőség)

Nem Cargo-él (külön build):

  cpp-client ·· rkv-ffi / rkv-cxx   (a staticlib-re és a generált fejlécre épül)
```

A nyilak a **Cargo-függőség irányát** jelölik: a felső réteg a közvetlenül alatta lévőtől függ (`[dependencies]`). A `Cargo.toml` függőséglistája ebből egyértelműen felírható: `rkv-cli` → `rkv-core`; `rkv-server` → `rkv-core`; `rkv-ffi` → `rkv-core`; `rkv-cxx` → `rkv-core`; `rkv-core` → `rkv-derive` (path-függőség). A makró crate (`rkv-derive`) nem függ az `rkv-core`-tól: a derive által generált kód az `::rkv_core::CommandName` útvonalat nevezi meg, de ez a használó crate-ben oldódik fel, nem Cargo-függőség. A fizikai mappaszerkezet (a tagok egymás melletti könyvtárai) nem azonos ezzel a függőségi gráffal.

A rétegábra azt mutatja, hogy az `rkv-core` a rendszer magja: minden más crate ezt használja, egyetlen kivétellel a `rkv-derive`, amely a `Command` levezetett `CommandName` implementációját adja az `rkv-core`-nak. A C++-kliens az `rkv-cxx` crate-ből generált bridge-en keresztül beszél a Rust tárolóval – ez az a határ, ahol a C++-os kényelem és a Rust-biztonság találkozik.

24.1.3 Az adatmodell és a formátumok rögzítése

A záróprojekt egyetlen adatmodellt használ minden rétegen. A korábbi vázlatok szöveges (`String`) értékeket és tabulátoros snapshotot említettek; **ezek nem részei a végleges rendszernek**.

Réteg	Döntés
kulcs	`String` (a tárolóban tetszőleges UTF-8, a protokollban nem üres, szóköz- és vezérlőkarakter-mentes)
érték	`Vec<u8>`: tetszőleges bájtsorozat, a tárolás veszteségmentes
`Storage` trait	`get(&str) -> Result<Option<Vec<u8>>>`, `set(String, Vec<u8>) -> Result<()>`, `delete(&str) -> Result<bool>`, `list(Option<&str>) -> Result<Vec<(String, Vec<u8>)>>` (kulcs szerint rendezve)
tárolási formátum, v1	snapshot: egyetlen JSON-objektum `{"data":{kulcs:[bájtok...]}}`; napló: soronként egy JSON-rekord (`{"Set":{"key":..,"value":[..]}}` vagy `{"Delete":{"key":..}}`), `\n`-nel lezárva
hálózati protokoll	soronként egy parancs. Szöveges réteg: `GET k`, `SET k v`, `DEL k`, `LIST [prefix]`; bináris-biztos réteg: `SETX <hexkulcs> <hexérték|->`, `GETX <hexkulcs>`
`LIST` keretezés	soronként `VALUE <kulcs> <érték>` (szöveg-biztos bejegyzés) vagy `VALUEX <hexkulcs> <hexérték>` (egyéb), a végén egy `OK` sor; a CLI, a szerver és a C++ átvételi teszt ugyanezt a keretezést használja
értékek a protokollban	a szöveges rétegben nem üres UTF-8 szöveg vezérlőkarakterek (sortörés) nélkül; tetszőleges bájtsorhoz (üres értékhez is) a `SETX`/`GETX` hexa rétege (az üres érték jele `-`); a nem szöveg-biztos érték `GET`-re `ERR value is not text-safe, use GETX` választ ad

Formátumverzió és kompatibilitás. A fenti tárolási formátum a **1-es verzió**, amelyet a fájlokban nem jelölünk külön mezővel. Régebbi fájlok nincsenek: a tabulátoros snapshot, amelyet egy korai vázlat említett, soha nem készült el. Ha a formátum változik, a snapshot kap egy `version` mezőt (`#[serde(default)]`, hiányában 1-es), a napló pedig új fájlnevet; az ismeretlen verziót a `FileStore::open` hibával utasítja el, és nem próbálja kitalálni. A protokollban a keretezés szabálya: a kliens a `VALUE` sorokat a lezáró `OK` sorig olvassa; a szerver *nem* zárja le a kapcsolatot egy parancs után.

24.1.4 A perzisztencia véglegesítése

A tartósági modell ugyanaz, mint az I/O és szerializáció: fájlok, pufferelés, serde alfejezetben: Linux, helyi POSIX-fájlrendszer (ext4, XFS), a gyorsítótár-ürítést betartó háttértár. A `FileStore` snapshot + hozzáfűzéses naplóval tárol (a párhuzamos ellenőrzőösszeg önálló segéd, a `FileStore` nem hívja). Az alábbi invariánsok adják a szerződést; a teljes, végleges forráskód lejjebb következik.

- **Commit-sorrend (WAL).** Egy művelet akkor commitolt, ha a teljes naplórekord (JSON + `\n`) kiíródott, és a `flush` és a `sync_data` sikerült. A memóriabeli állapotot **csak ezután** módosítjuk (`self.data.insert`/`remove`).
- **Hibaszerződés.** Ha a naplóírás hibát ad, a memória változatlan marad, a tároló `poisoned` jelzőt kap, és minden további írást elutasít, amíg újra nem nyitjuk: félkész rekord mögé nem írunk vakon. A hiba `Error::Io`.
- **Replay-formátum.** Újranyitáskor a snapshot után a napló rekordjait játsszuk vissza. Egy sorvégjel nélküli, csonka utolsó rekord sosem kapott `Ok`-t, ezért eldobjuk és a fájlt levágjuk; egy sorvégjellel lezárt, de nem értelmezhető rekord valódi sérülés, hibát adunk.
- **Kompaktálás.** A snapshotot ideiglenes fájlba írjuk, `sync_all`-lal lemezre kényszerítjük, `rename`-mel atomikusan lecseréljük, a szülőkönyvtárat szinkronizáljuk, és **csak ezután** csonkoljuk a naplót. Összeomlás után a lemezen vagy a régi snapshot és a teljes napló, vagy az új snapshot és a teljes vagy üres napló marad; a napló rekordjainak újrajátszása az új snapshotra a végállapotot nem változtatja meg.
- **Tulajdonlás és szinkronizáció.** A `Store`/`FileStore` *belül nem szinkronizál*: a módosító metódusok `&mut self`-et kapnak, az alkalmazás kizárólagos tulajdonosa az adatnak. A szálak közti megosztást egyetlen helyen, a `SharedStore` külső `Mutex`-e adja; a lekérdezések **owned** (`Vec<u8>`) eredményt adnak, mert a zár elengedése után hivatkozás nem élhetne tovább. A kizárólagos fájlmegnyitást a naplófájlra tett `try_lock` kényszeríti ki.

Kódblokk — fájl: `rkv-core/src/store/file_store.rs` · művelet: a `file_store.rs` **teszt-modulon kívüli** (az első `#[cfg(test)]` sor előtti) teljes tartalmának cseréje; a fájl végén lévő `#[cfg(test)] mod tests { ... }` blokkot (az I/O és szerializáció: fájlok, pufferelés, serde és a Teljesítmény: mérés, profilozás, allokáció, release-beállítások tesztjei) érintetlenül meg kell tartani, tehát ez nem a teljes fájl cseréje · ellenőrzés: `cargo test -p rkv-core`

```
use serde::{Deserialize, Serialize};
use std::collections::HashMap;
use std::fs::{self, File, OpenOptions, TryLockError};
use std::io::{self, BufRead, BufReader, BufWriter, Write};
use std::path::{Path, PathBuf};

use crate::{Error, Result};

#[derive(Debug, Clone, Serialize, Deserialize)]
struct Snapshot {
    data: HashMap<String, Vec<u8>>,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
enum LogEntry {
    Set { key: String, value: Vec<u8> },
    Delete { key: String },
}

/// Kölcsönzött naplórekord: ugyanazt a JSON-t írja, mint a `LogEntry`, de nem másol.
#[derive(Serialize)]
enum LogEntryRef<'a> {
    Set { key: &'a str, value: &'a [u8] },
```

```
    Delete { key: &'a str },
}

/// Kölcsönzött snapshot: a `Snapshot` JSON-alakja, a térkép másolása nélkül.
#[derive(Serialize)]
struct SnapshotRef<'a> {
    data: &'a HashMap<String, Vec<u8>>,
}

pub struct FileStore {
    data: HashMap<String, Vec<u8>>,
    log_writer: BufWriter<File>,
    snapshot_path: PathBuf,
    /// A snapshotra tett kizárólagos zár (külön `.lock` fájl); a FileStore eldobásakor oldódik.
    _lock: File,
    /// Egy sikertelen naplóírás után igaz: a tároló nem fogad több írást.
    poisoned: bool,
}

impl FileStore {
    pub fn open(snapshot_path: impl AsRef<Path>, log_path: impl AsRef<Path>) -> Result<Self> {
        let snapshot_path = snapshot_path.as_ref().to_path_buf();
        let log_path = log_path.as_ref();

        // A három segédfájl neve a snapshot nevéből származik, ezért a snapshot,
        // a napló, az ideiglenes fájl és a zárfájl útvonalai nem eshetnek egybe.
        let tmp_path = sibling(&snapshot_path, ".tmp");
        let lock_path = sibling(&snapshot_path, ".lock");
        for (a, name) in [(&tmp_path, "ideiglenes"), (&lock_path, "zár")] {
            if same_path(log_path, a) {
                return Err(Error::Io(io::Error::new(
                    io::ErrorKind::InvalidInput,
                    format!("a napló útvonala ütközik a snapshot {name} fájljával"),
                )));
            }
        }
        if same_path(log_path, &snapshot_path) {
            return Err(Error::Io(io::Error::new(
                io::ErrorKind::InvalidInput,
                "a napló és a snapshot útvonala nem lehet azonos",
            )));
        }

        // Kizárólagos használat: a *snapshotra* (nem a naplóra) tett zár miatt
        // egy snapshotot egyszerre csak egy FileStore nyithat meg, akárhány
        // naplóútvonallal. A zár a FileStore eldobásakor oldódik fel.
        let lock_file = OpenOptions::new()
            .create(true)
            .write(true)
            .truncate(false)
            .open(&lock_path)?;
        match lock_file.try_lock() {
            Ok(()) => {}
            Err(TryLockError::WouldBlock) => {
                return Err(Error::Io(io::Error::new(
                    io::ErrorKind::WouldBlock,
                    "a tárolót már egy másik FileStore használja",
                )));
            }
            Err(TryLockError::Error(e)) => return Err(e.into()),
        }

        let mut data = HashMap::new();
```

```
    if snapshot_path.exists() {
        let file = File::open(&snapshot_path)?;
        let reader = BufReader::new(file);
        let snap: Snapshot = serde_json::from_reader(reader)?;
        data = snap.data;
    }

    let log_file = OpenOptions::new()
        .create(true)
        .append(true)
        .open(log_path)?;
    // Az újonnan létrehozott napló fájlneve is tartós legyen (nem csak a tartalma).
    sync_parent_dir(log_path)?;
    let valid_len = replay_log(log_path, &mut data)?;
    if log_file.metadata()?.len() > valid_len {
        // Csonka (nem commitolt) utolsó rekord levágása, hogy az új
        // bejegyzések ne félkész sor mögé kerüljenek.
        log_file.set_len(valid_len)?;
        log_file.sync_all()?;
    }
    let log_writer = BufWriter::new(log_file);

    Ok(Self {
        data,
        log_writer,
        snapshot_path,
        _lock: lock_file,
        poisoned: false,
    })
}

#[must_use = "a keresés eredménye nélkül a hívás hatástalan"]
pub fn get(&self, key: &str) -> Option<&[u8]> {
    self.data.get(key).map(Vec::as_slice)
}

pub fn set(&mut self, key: String, value: Vec<u8>) -> Result<()> {
    self.append_entry(&LogEntryRef::Set {
        key: &key,
        value: &value,
    })?;
    // Csak a tartós naplórekord után módosul a memóriabeli állapot.
    self.data.insert(key, value);
    Ok(())
}

pub fn delete(&mut self, key: &str) -> Result<()> {
    // A mérgezett tároló minden további írást elutasít, hiányzó kulcs törlését is.
    self.ensure_healthy()?;
    if !self.data.contains_key(key) {
        return Ok(());
    }
    self.append_entry(&LogEntryRef::Delete { key })?;
    self.data.remove(key);
    Ok(())
}

fn ensure_healthy(&self) -> Result<()> {
    if self.poisoned {
        return Err(std::io::Error::other(
            "a napló egy korábbi írási hiba óta sérült; nyisd meg újra a tárolót",
        )
        .into());
```

```
        }
        Ok(())
    }

    /// Commit-szerződés: a teljes rekord (JSON + '\n') kiírása, flush és
    /// sync_data. Csak Ok esetén szabad a memóriabeli állapotot módosítani.
    fn append_entry(&mut self, entry: &LogEntryRef<'_>) -> Result<()> {
        self.ensure_healthy()?;
        let mut line = serde_json::to_vec(entry)?;
        line.push(b'\n');
        let res = self
            .log_writer
            .write_all(&line)
            .and_then(|()| self.log_writer.flush())
            .and_then(|()| self.log_writer.get_ref().sync_data());
        if let Err(e) = res {
            // A pufferben vagy a fájlban félkész rekord maradhatott:
            // vakon nem folytatjuk a naplózást.
            self.poisoned = true;
            return Err(e.into());
        }
        Ok(())
    }

    pub fn list(&self, prefix: Option<&str>) -> Vec<(String, Vec<u8>)> {
        let mut items: Vec<_> = self
            .data
            .iter()
            .filter(|(k, _)| prefix.is_none_or(|p| k.starts_with(p)))
            .map(|(k, v)| (k.clone(), v.clone()))
            .collect();
        items.sort_by(|a, b| a.0.cmp(&b.0));
        items
    }

    pub fn save_snapshot(&mut self) -> Result<()> {
        self.ensure_healthy()?;
        let tmp_path = sibling(&self.snapshot_path, ".tmp");
        // A File::create az ideiglenes fájlt nullázza le, a régi snapshotot nem.
        let file = File::create(&tmp_path)?;
        let mut writer = BufWriter::new(file);
        serde_json::to_writer(&mut writer, &SnapshotRef { data: &self.data })?;
        writer.flush()?;
        writer.get_ref().sync_all()?; // 1. az új tartalom a lemezen van
        drop(writer);
        fs::rename(&tmp_path, &self.snapshot_path)?; // 2. atomikus csere
        sync_parent_dir(&self.snapshot_path)?; // 3. maga a csere is tartós
        Ok(())
    }

    pub fn compact(&mut self) -> Result<()> {
        self.save_snapshot()?; // 1-3. lépés: az új snapshot tartós
        // 4. Csak ezután dobható el a napló.
        let log = self.log_writer.get_mut();
        log.set_len(0)?;
        log.sync_all()?;
        Ok(())
    }
}

/// A snapshot mellé, a nevéhez fűzött utótaggal képzett útvonal (`state.json` →
↪  `state.json.tmp`).
fn sibling(path: &Path, suffix: &str) -> PathBuf {
```

```
    let mut name = path.file_name().unwrap_or_default().to_os_string();
    name.push(suffix);
    path.with_file_name(name)
}

/// Két útvonal azonos-e (létező fájlnál a kanonikus alak, különben a szülő kanonikus alakja +
↪  név).
fn same_path(a: &Path, b: &Path) -> bool {
    fn norm(p: &Path) -> PathBuf {
        let parent = p
            .parent()
            .filter(|d| !d.as_os_str().is_empty())
            .unwrap_or(Path::new("."));
        match (fs::canonicalize(parent), p.file_name()) {
            (Ok(d), Some(n)) => d.join(n),
            _ => p.to_path_buf(),
        }
    }
    norm(a) == norm(b)
}

/// POSIX-on a rename a szülőkönyvtár szinkronizálása után tartós.
#[cfg(unix)]
fn sync_parent_dir(path: &Path) -> Result<()> {
    let dir = match path.parent() {
        Some(p) if !p.as_os_str().is_empty() => p,
        _ => Path::new("."),
    };
    File::open(dir)?.sync_all()?;
    Ok(())
}

#[cfg(not(unix))]
fn sync_parent_dir(_path: &Path) -> Result<()> {
    // Nem POSIX-platformon könyvtár így nem szinkronizálható; lásd a
    // tartóssági modellt a labor után.
    Ok(())
}

/// Visszajátssza a naplót, és visszaadja az utolsó teljes rekord végének
/// bájtpozícióját.
fn replay_log(path: &Path, data: &mut HashMap<String, Vec<u8>>) -> Result<u64> {
    let file = match File::open(path) {
        Ok(f) => f,
        Err(e) if e.kind() == std::io::ErrorKind::NotFound => return Ok(0),
        Err(e) => return Err(e.into()),
    };
    let mut reader = BufReader::new(file);
    let mut line = Vec::new();
    let mut valid_len = 0u64;
    loop {
        line.clear();
        let n = reader.read_until(b'\n', &mut line)?;
        if n == 0 || line.last() != Some(&b'\n') {
            // Fájl vége, vagy sorvégjel nélküli csonka rekord: ez utóbbi
            // sosem kapott Ok-t, mert az append_entry a teljes sort írja.
            break;
        }
        valid_len += n as u64;
        if line.trim_ascii().is_empty() {
            continue;
        }
        // A csak whitespace-ből álló sort tudatosan átugorjuk (a naplóíró ilyet nem állít elő).
```

```
        // A sorvégjellel lezárt, de hibás rekord valódi sérülés: hiba.
        let entry: LogEntry = serde_json::from_slice(&line)?;
        match entry {
            LogEntry::Set { key, value } => {
                data.insert(key, value);
            }
            LogEntry::Delete { key } => {
                data.remove(&key);
            }
        }
    }
    Ok(valid_len)
}
```

A `Storage` trait, amelyet a `MemStore` és a `FileStore` egyaránt implementál:

Kódblokk — fájl: `rkv-core/src/storage.rs` · művelet: a trait definíciója (a fájl eleje) · ellenőrzés: `cargo check -p rkv-core`

```
use std::path::Path;

use crate::{Command, Error, FileStore, MemStore, Result};

/// Kulcs-érték tároló absztrakció: a parancsok végrehajtásához szükséges
/// műveletek. A kulcs `String`, az érték `Vec<u8>` (r007).
pub trait Storage {
    fn get(&self, key: &str) -> Result<Option<Vec<u8>>>;
    fn set(&mut self, key: String, value: Vec<u8>) -> Result<()>;
    /// `true`, ha a kulcs létezett.
    fn delete(&mut self, key: &str) -> Result<bool>;
    /// A `prefix`-szel kezdődő párok kulcs szerint rendezve.
    fn list(&self, prefix: Option<&str>) -> Result<Vec<(String, Vec<u8>)>>;
}
```

A **szándékosan hibás sorrend** gyakori hiba: a napló törlése a snapshot előtt.

```
// Szándékosan HIBÁS: a napló elveszik, ha a snapshot írása közben hiba lép fel.
fn bad_compact(&mut self) -> Result<()> {
    self.log_writer.get_mut().set_len(0)?; // a napló már üres...
    self.save_snapshot()?; // ...a snapshot pedig még nem tartós
    Ok(())
}
```

Ha a `save_snapshot` hibával tér vissza, a napló már üres, a snapshot pedig nem frissült: adatvesztés. A javítás a sorrend megfordítása: először tartós snapshot, utána napló-csonkolás, ahogy a `compact` teszi. A Rust `?` operátora a hibát explicit teszi, de a logikai sorrendet a fordító nem ellenőrzi; ez a tervező felelőssége, mint C++-ban a kivételbiztossági garanciáké.

24.1.4.1 LIST tartomány-lekérdezés

A `LIST [<prefix>]` a prefixszel kezdődő kulcsokat adja vissza, a C++ `std::map::lower_bound`-jához hasonlóan. **Definíció.** A prefix-lekérdezés a kulcsok rendezett halmazában a megadott string-prefixszel kezdődő kulcsok halmaza.

A `BTreeMap<String, _>` a `std::map` Rust-megfelelője: rendezett, önkiegyensúlyozó fa. A prefixszel kezdődő kulcsok a rendezett sorrendben egy összefüggő sávot alkotnak, amely a prefix helyén kezdődik. A helyes lekérdezés alulról zárt, felül nyitott tartománnyal (`prefix..`) indul, és addig olvas (`take_while`), amíg a kulcsok a prefixszel kezdődnek. Felső határt számolni itt szükségtelen (a helyes lexikografikus utód megvalósítható, de bonyolultabb, a naiv `prefix +`

`char::MAX` pedig hibás). Egy kis kulcshalmazon végigvezetve (a kulcsok bájtonkénti rendezése: `a` < `ab` < `abc` < `az` < `a{` < `a\u{10FFFF}` < `a\u{10FFFF}x` < `b`, mert a `{` kódja `0x7B`, nagyobb, mint a `z` kódja `0x7A`):

Kódblokk — fájl: `src/main.rs` (önálló `cargo new` projekt) · művelet: teljes csere · ellenőrzés: `cargo run`

```
use std::collections::BTreeMap;
use std::ops::Bound::{Excluded, Included, Unbounded};

fn main() {
    let mut map = BTreeMap::new();
    for key in ["a", "ab", "abc", "az", "a{", "a\u{10FFFF}", "a\u{10FFFF}x", "b"] {
        map.insert(key.to_string(), ());
    }
    let keys = |it: &mut dyn Iterator<Item = &String>| -> Vec<String> { it.cloned().collect() };

    println!("rendezés: {:?}", map.keys().collect::<Vec<_>>());
    // Helyes: alulról zárt, felül nyitott, és addig olvas, amíg a prefix egyezik.
    let ok = keys(&mut map.range("a".to_string()..).map(|(k, _)| k).take_while(|k|
    ↪  k.starts_with("a")));
    println!("range(\"a\"..) + take_while: {ok:?}");
    // Alsó határ: Included tartalmazza a prefix-szel egyenlő kulcsot, Excluded nem.
    let incl = keys(&mut map.range((Included("a".to_string()), Unbounded)).map(|(k, _)|
    ↪  k).take_while(|k| k.starts_with("a")));
    let excl = keys(&mut map.range((Excluded("a".to_string()), Unbounded)).map(|(k, _)|
    ↪  k).take_while(|k| k.starts_with("a")));
    println!("alsó Included: {incl:?}\nalsó Excluded: {excl:?}");
    // Rossz felső korlát: prefix + 'z'
    let bad_z = keys(&mut map.range("a".to_string()..="az".to_string()).map(|(k, _)| k));
    println!("rossz: \"a\"..=\"az\": {bad_z:?}");
    let bad_z_excl = keys(&mut map.range("a".to_string().."az".to_string()).map(|(k, _)| k));
    println!("rossz: \"a\"..\"az\": {bad_z_excl:?}");
    // Rossz felső korlát: prefix + char::MAX
    let max = format!("a{}", char::MAX);
    let bad_max = keys(&mut map.range("a".to_string()..=max.clone()).map(|(k, _)| k));
    println!("rossz: \"a\"..=\"a\\u{{10FFFF}}\": {bad_max:?}");
    let bad_max_excl = keys(&mut map.range("a".to_string()..max).map(|(k, _)| k));
    println!("rossz: \"a\"..\"a\\u{{10FFFF}}\": {bad_max_excl:?}");
}
```

Ellenőrzött kimenet:

```
rendezés: ["a", "ab", "abc", "az", "a{", "a\u{10ffff}", "a\u{10ffff}x", "b"]
range("a"..) + take_while: ["a", "ab", "abc", "az", "a{", "a\u{10ffff}", "a\u{10ffff}x"]
alsó Included: ["a", "ab", "abc", "az", "a{", "a\u{10ffff}", "a\u{10ffff}x"]
alsó Excluded: ["ab", "abc", "az", "a{", "a\u{10ffff}", "a\u{10ffff}x"]
rossz: "a"..="az": ["a", "ab", "abc", "az"]
rossz: "a".."az": ["a", "ab", "abc"]
rossz: "a"..="a\u{10FFFF}": ["a", "ab", "abc", "az", "a{", "a\u{10ffff}"]
rossz: "a".."a\u{10FFFF}": ["a", "ab", "abc", "az", "a{"]
```

Tanulságok: (1) a helyes `range("a"..)` mind a hét `a`-val kezdődő kulcsot megtalálja, a `b`-t nem; minden kulcs pontosan egyszer szerepel, az iterátor nem duplikál, mert a térkép kulcsai egyediek. (2) Az alsó határ inkluzivitása számít: a `Included("a")` a magát a prefixet (a `"a"` kulcsot) is tartalmazza, az `Excluded("a")` nem. (3) A felső határ viszont mindig hibás: a `"a"..="az"` kihagyja az `a{`, az `a\u{10FFFF}` és az `a\u{10FFFF}x` kulcsot; a `"a".."az"` az `az`-t is; a `char::MAX`-os felső határ (`prefix + '\u{10FFFF}'`) a `…x` farkú kulcsot hagyja ki, és az exkluzív változat a `char::MAX`-ot tartalmazót is. Ezért a felső határt a `starts_with` ellenőrzés adja, nem egy kiszámított kulcs. Az rkv `Store::range_prefix` ezt valósítja meg (a részletes indoklás és a tesztek a Vec, HashMap, BTreeMap és társaik; az Entry API laborjában vannak).

A `Storage::list` és a `range_prefix` viszonya. A `MemStore` a `Store` aliasa, amely Vec, HashMap, BTreeMap és társaik; az Entry API óta `BTreeMap`-et tart, ezért a végleges `Storage::list` prefix esetén a `range_prefix`-et használja a korábbi `keys()` + `filter` + `sort` helyett; így a prefix-sávot nem kell az összes kulcson végigszűrni. A `FileStore` viszont `HashMap`-et tart, és a naplóból/snapshotból állítja vissza az állapotot; ott nincs rendezett index, ezért a `list` lineárisan szűr és rendez. Az optimalizált `range_prefix` tehát nem univerzális `Storage::list`-megvalósítás, hanem a BTreeMap-alapú `MemStore` sajátja; a `FileStore` költségmodellje lineáris szűrés + rendezés, és a prefix-lekérdezés optimalizálása ott külön indexet igényelne.

A végleges `MemStore`-implementáció (a Trait-ek és generikusok: sablonok és concepts helyett alfejezetbeli `list` helyett):

```
impl Storage for MemStore {
    fn list(&self, prefix: Option<&str>) -> Result<Vec<(String, Vec<u8>)>> {
        // A BTreeMap rendezett: a prefix-sávot range_prefix adja. Az üres
        // prefix minden kulcsot visszaad, szintén rendezve.
        let prefix = prefix.unwrap_or("");
        Ok(MemStore::range_prefix(self, prefix)
            .map(|(k, v)| (k.to_owned(), v.to_vec()))
            .collect())
    }
}
```

A szerver válasza a `LIST app` parancsra:

```
LIST app
VALUE app::1 x
VALUE app::2 y
OK
```

Ha a prefix üres, az összes kulcsot listázza. A keretezés minden rétegen ugyanez: a `OK` sor mindig lezárja a választ (üres találatnál egyedül áll).

24.1.5 Labor: a teljes átadás

Munkakönyvtár: `~/rust-labor/rkv/`

A labor a korábbi alfejezetek eredményeire támaszkodik, és egyetlen, végleges API-készletre hozza őket. Ha bármelyik crate hiányzik, a `cargo new` parancsokkal pótolható, de a cél most az ellenőrzés és a véglegesítés.

24.1.5.1 1. lépés: a workspace manifest

A közös `edition`, `rust-version` és `version` a `[workspace.package]` táblába kerül, a tagok örökölik. Az `edition` **nem** lehet a `[workspace]` táblában: ott a Cargo ismeretlen kulcsként ignorálja (`warning: unused manifest key: workspace.edition`), és a tagok editionje a hiányzó mező miatt 2015 lenne. A virtuális workspace-nek nincs saját editionje, ezért a függőségfeloldót (`resolver = "3"`, a 2024-es edition alapértelmezése) explicit adjuk meg.

Kódblokk — fájl: `Cargo.toml` · művelet: teljes csere (workspace gyökér) · ellenőrzés: `cargo metadata --no-deps` → minden tag `edition: 2024`, figyelmeztetés nélkül

```toml
[workspace]
resolver = "3"
members = [
    "rkv-core",
    "rkv-cli",
    "rkv-server",
    "rkv-ffi",
    "rkv-cxx",
    "rkv-derive",
]

[workspace.package]
version = "0.1.0"
edition = "2024"
rust-version = "1.89"
license = "MIT OR Apache-2.0"
publish = false   # tanulóprojekt: nem kerül a crates.io-ra
```

Kódblokk — fájl: `rkv-core/Cargo.toml`, `rkv-cli/Cargo.toml`, `rkv-server/Cargo.toml`, `rkv-ffi/Cargo.toml`, `rkv-cxx/Cargo.toml`, `rkv-derive/Cargo.toml` · művelet: a `[package]` tábla `version`, `edition`, `rust-version`, `license` és `publish` soraiban az értékek helyett örökítés · ellenőrzés: `cargo metadata --no-deps`

```toml
[package]
name = "rkv-core"
version.workspace = true
edition.workspace = true
rust-version.workspace = true
license.workspace = true
publish.workspace = true
```

(Ha valaki nem akarja az örökítést, az alternatíva: minden tag `[package]` táblájában külön `edition = "2024"`; a mellékelt `r002`/`r003` laborok ezt használták.) Az `rkv-derive` proc-macro crate, az `rkv-cxx` a `cxx-build` miatt, az `rkv-ffi` a `cbindgen` miatt speciális; a `rkv-cli` a `clap`-et, a `rkv-server` a `tokio`-t és a `tracing`-et használja.

24.1.5.2 2. lépés: a migráció összefoglalója (melyik réteg mit kapott)

A záróprojekt nem ír át semmit önkényesen: az alábbi változások mind a korábbi alfejezetek labor-lépései. Ha az olvasó az előző állapotból indul, ezeket kell alkalmaznia.

Réteg	Végleges állapot	Hol készült
`Store`, `FileStore`	`get`/`get_by` → `Option<&[u8]>`, `#[must_use]`; `FileStore` kölcsönzött naplórekorddal és snapshottal	API-tervezés, Teljesítmény
`Storage`, `SharedStore`	owned `Result<Option<Vec<u8>>>`; a szinkronizáció a `SharedStore` `Mutex`-e	Mutex, RwLock, csatornák és a megosztott rkv
`Command`, `Error`	`#[non_exhaustive]`; a `Command` protokollnevét a `CommandName` derive adja (`Delete` → `DEL`)	Makrók, API-tervezés Rustban: irányelvek és ergonómia

Réteg	Végleges állapot	Hol készült
`rkv-cli`	`execute: _ => "ERR unsupported command"` ág a `non_exhaustive` miatt; `std::str::from_utf8` másolás nélkül	API-tervezés Rustban: irányelvek és ergonómia
`rkv-server`	`run_server(listener, SharedStore, token)`, `ServerConfig::builder()`, `session` modul	Az rkv hálózati szerver: TCP, protokoll, kecses leállás–API-tervezés Rustban: irányelvek és ergonómia
`rkv-ffi`	`RkvStore` (benne `SharedStore`), `catch_unwind` (unwind profil)	C ABI: extern “C”, repr(C), bindgen és cbindgen
`rkv-cxx`	`rkv::Db` híd (`open_file`, `get`, `list`, …), `Logger`	C++-együttműködés a cxx crate-tel és a fokozatos migráció
profil	`[profile.release]`: `lto = "fat"`, `codegen-units = 1`, **`panic` marad `unwind`**	Teljesítmény: mérés, profilozás, allokáció, release-beállítások

24.1.5.3 3. lépés: az átvételi tesztek

Állapot helyreállítása. Ez a teszt a perzisztencia-követelményt ellenőrzi: SET/DEL, újranyitás, kompaktálás, újabb írás és újabb újranyitás után ugyanaz az állapot álljon helyre.

Kódblokk — fájl: `rkv-core/tests/restore_state.rs` · művelet: új fájl · ellenőrzés: `cargo test -p rkv-core --test restore_state` → 1 sikeres teszt

```
//! Átvételi teszt: SET/DEL, újranyitás és kompaktálás után ugyanaz az állapot áll helyre.
use rkv_core::{FileStore, Storage};

fn snapshot_of(store: &FileStore) -> Vec<(String, Vec<u8>)> {
    Storage::list(store, None).unwrap()
}

#[test]
fn state_survives_reopen_and_compaction() {
    let dir = tempfile::tempdir().unwrap();
    let (snap, log) = (dir.path().join("state.json"), dir.path().join("append.log"));

    let expected = {
        let mut store = FileStore::open(&snap, &log).unwrap();
        store.set("app::1".into(), b"x".to_vec()).unwrap();
        store.set("app::2".into(), vec![0, 255, 10]).unwrap(); // tetszőleges bájtok
        store.set("szóköz kulcs".into(), b"y".to_vec()).unwrap();
        store.set("torolt".into(), b"z".to_vec()).unwrap();
        store.delete("torolt").unwrap();
        store.set("app::1".into(), b"x2".to_vec()).unwrap(); // csere
        snapshot_of(&store)
    };
    assert_eq!(expected.len(), 3);

    // 1. Újranyitás: a naplóból (snapshot nélkül) ugyanaz az állapot áll helyre.
    let mut store = FileStore::open(&snap, &log).unwrap();
    assert_eq!(snapshot_of(&store), expected);
```

```
    // 2. Kompaktálás: snapshot + üres napló; a tartalom változatlan.
    store.compact().unwrap();
    assert_eq!(snapshot_of(&store), expected);
    drop(store);

    // 3. Újranyitás a snapshotból, majd új írás a kompaktált napló mögé, és megint újranyitás.
    let mut store = FileStore::open(&snap, &log).unwrap();
    assert_eq!(snapshot_of(&store), expected);
    store.set("uj".into(), b"w".to_vec()).unwrap();
    drop(store);
    let store = FileStore::open(&snap, &log).unwrap();
    let mut with_new = expected.clone();
    with_new.push(("uj".into(), b"w".to_vec()));
    with_new.sort();
    assert_eq!(snapshot_of(&store), with_new);
}
```

Szerver-integráció. A teszt az `rkv-server` átadott `run_server(listener, store, token)` függvényét hívja változtatás nélkül; a `tempfile` a `[dev-dependencies]` része, a `tokio` és a `tokio-util` függőség.

Kódblokk — fájl: `rkv-server/tests/list_prefix.rs` · művelet: új fájl · ellenőrzés: `cargo test -p rkv-server --test list_prefix` → 1 sikeres teszt

```
use std::io;
use std::time::Duration;

use rkv_core::{FileStore, SharedStore};
use rkv_server::run_server;
use tokio::io::{AsyncBufReadExt, AsyncWriteExt, BufReader};
use tokio::net::{TcpListener, TcpStream};
use tokio::time::timeout;
use tokio_util::sync::CancellationToken;

#[tokio::test]
async fn server_list_prefix_returns_filtered_pairs() {
    let dir = tempfile::tempdir().unwrap();
    let mut file_store =
        FileStore::open(dir.path().join("state.json"), dir.path().join("append.log")).unwrap();
    file_store.set("app::1".into(), b"x".to_vec()).unwrap();
    file_store.set("app::2".into(), b"y".to_vec()).unwrap();
    file_store.set("other".into(), b"z".to_vec()).unwrap();
    let store = SharedStore::new(file_store);

    let listener = TcpListener::bind("127.0.0.1:0").await.unwrap();
    let addr = listener.local_addr().unwrap();
    let token = CancellationToken::new();
    // Az átadott `run_server` változtatás nélkül: (listener, tároló, token).
    let server = tokio::spawn(run_server(listener, store, token.clone()));

    // A szerver egy kapcsolaton több parancsot kezel, ezért nem EOF-ig olvasunk:
    // a VALUE sorokat a lezáró OK sorig gyűjtjük.
    let exchange = async {
        let stream = TcpStream::connect(addr).await?;
        let (read_half, mut write_half) = stream.into_split();
        write_half.write_all(b"LIST app\n").await?;
        let mut lines = BufReader::new(read_half).lines();
        let mut values = Vec::new();
        while let Some(line) = lines.next_line().await? {
            if line == "OK" {
                return Ok(values);
            }
            values.push(line);
```

```
        }
        Err(io::Error::new(
            io::ErrorKind::UnexpectedEof,
            "a LIST válasza OK nélkül ért véget",
        ))
    };
    let result = timeout(Duration::from_secs(5), exchange).await;

    // A szervert minden kimenetelnél leállítjuk és megvárjuk, csak utána értékelünk.
    token.cancel();
    timeout(Duration::from_secs(10), server)
        .await
        .expect("a szerver nem állt le határidőn belül")
        .expect("a szervertask pánikolt");

    let values = result
        .expect("a LIST válasza 5 s alatt nem érkezett meg")
        .expect("I/O-hiba a LIST közben");
    assert_eq!(values, vec!["VALUE app::1 x", "VALUE app::2 y"]);
}
```

A teszt külső időkorlát nélkül sem akadhat el: a hálózati csere és a szerver leállása is határidős, a `server.await` pedig garantálja, hogy a teszt végén nem fut háttértask.

C++ átvételi teszt. A teszt a *ténylegesen generált* hídra épül: az `rkv-cxx` bridge `rkv::Db`, `rkv::Entry` és `rust::Error` típusait használja, ugyanazt a `lib.rs.h` fejlécet, amelyet a C++-együttműködés a cxx crate-tel és a fokozatos migráció labor `cargo build`-je előállít. A kapcsolat: a `build.rs` a `src/lib.rs` bridge-éből generálja a `target/cxxbridge/rkv-cxx/src/lib.rs.h` fejlécet és a `lib.rs.cc` forrást (a `logger.cc`-vel együtt a `rkv-cxx-glue` könyvtárba), a Cargo a `staticlib` típusból `target/release/librkv_cxx.a`-t állít elő, a C++ fordító pedig a `-I target/cxxbridge -I .` útvonalakkal találja meg a fejlécet és a `rkv-cxx/include/logger.h`-t.

Kódblokk — fájl: `cpp-client/acceptance.cpp` · művelet: új fájl · ellenőrzés: az alábbi parancsok

```
// C++ átvételi teszt: a rkv-cxx generált hídján keresztül ellenőrzi a közös szerződést.
#include "rkv-cxx/src/lib.rs.h"

#include <cassert>
#include <cstdint>
#include <iostream>
#include <string>

namespace {

rust::Slice<const std::uint8_t> bytes(const std::string& s) {
    return {reinterpret_cast<const std::uint8_t*>(s.data()), s.size()};
}

std::string text(const rust::Vec<std::uint8_t>& v) {
    return std::string(v.begin(), v.end());
}

// A szerver és a CLI LIST-keretezése: soronként `VALUE <kulcs> <érték>`, a végén `OK`.
std::string list_frame(rkv::Db& db, const std::string& prefix) {
    std::string out;
    for (const rkv::Entry& e : db.list(prefix)) {
        out += "VALUE " + std::string(e.key) + " " + text(e.value) + "\n";
    }
    return out + "OK\n";
}
```

```
} // namespace

int main(int argc, char** argv) {
    assert(argc == 2 && "használat: acceptance <üres adatkönyvtár>");
    const std::string dir = argv[1];

    {
        rust::Box<rkv::Db> db = rkv::open_file(dir);
        db->set("app::1", bytes("x"));
        db->set("app::2", bytes("y"));
        db->set("other", bytes("z"));

        assert(text(db->get("app::1")) == "x");
        assert(db->count("app") == 2);
        assert(db->count("") == 3);
        assert(list_frame(*db, "app") == "VALUE app::1 x\nVALUE app::2 y\nOK\n");

        // Hiányzó kulcs: rust::Error kivétel, nem üres érték.
        bool threw = false;
        try {
            (void)db->get("nincs");
        } catch (const rust::Error& e) {
            threw = std::string(e.what()).find("nem található") != std::string::npos;
        }
        assert(threw);

        db->del("other");
        assert(db->count("") == 2);
    } // a Box eldobása felszabadítja a tárolót (és a fájlzárat)

    // Újranyitás: az állapot a folyamaton belül is megmarad a FileStore-ban.
    rust::Box<rkv::Db> reopened = rkv::open_file(dir);
    assert(reopened->count("") == 2);
    assert(text(reopened->get("app::2")) == "y");

    std::cout << "C++ client acceptance test passed\n";
    return 0;
}
```

Kódblokk — fájl: terminál (workspace gyökér) · művelet: tiszta környezetben generálás, fordítás és futtatás · ellenőrzés: `C++ client acceptance test passed`, kilépési kód 0

```
cargo build -p rkv-cxx --release
g++ -std=c++17 -Wall -Wextra -I target/cxxbridge -I . \
    cpp-client/acceptance.cpp target/release/librkv_cxx.a \
    -lpthread -ldl -lm -o cpp-client/acceptance
./cpp-client/acceptance "$(mktemp -d)"
```

(Az `assert`-ek az `NDEBUG` nélkül aktívak; ne fordítsd `-DNDEBUG`-gel.) A CMake/corrosion változat a C++-együttműködés alfejezet vázlata, amelyet nem futtattunk: a szerzői környezetben nincs CMake. A fenti `g++` parancsok viszont végig lefutottak.

24.1.5.4 4. lépés: a minőségkapuk

A workspace gyökeréből:

```
cargo fmt --all -- --check
cargo clippy --workspace --all-targets --all-features -- -D warnings
cargo test --workspace
cargo doc --no-deps --workspace
```

Ha a `fmt --check` az első futtatáskor a kézzel másolt laborkódon eltérést jelez, a `cargo fmt --all` igazítja azt; a kapu lényege, hogy a végleges fában tiszta. A `cargo test --workspace` futtatja a `rkv-core` generikus szerződéstesztjét és perzisztenciatesztjeit, a szerver protokoll- és leállási tesztjeit, a `session` doctesteket és a szálbiztossági teszteket.

Ellenőrzött eredmény (Rust 1.99.1): `cargo fmt --all -- --check` tiszta; `cargo clippy --workspace --all-targets --all-features -- -D warnings` figyelmeztetés nélkül lefut; `cargo test --workspace` 93 tesztet futtat hiba nélkül; `cargo doc --no-deps --workspace` figyelmeztetés nélkül lefut.

24.1.5.5 5. lépés: Miri az unsafe részekre

Az `rkv-ffi` unsafe kódot tartalmaz. A Miri futtatása a C ABI-nál:

```
cargo +nightly miri test -p rkv-ffi
```

Ellenőrzött eredmény: a két teszt zöld, undefined behavior nélkül. A Miri az `rkv-cxx` crate-re nem használható értelmesen: a C++ oldalra átlépő hívásokat (idegen függvényeket) nem tudja értelmezni; ott a C++ átvételi teszt és a szokásos sanitizerek (ASan/UBSan a C++ oldalon) a megfelelő eszközök. A Miri a nightly toolchaint igényli. Az `unsafe` blokkokban a `// SAFETY:` megjegyzéseknek jelen kell lenniük, és a Miri által talált hibákat javítani kell.

Ellenőrzési pont. A workspace-en a `cargo fmt --all -- --check`, a `cargo clippy --workspace --all-targets --all-features -- -D warnings`, a `cargo test --workspace` és a `cargo doc --no-deps --workspace` hibátlanul lefut; a `restore_state`, a `list_prefix` és a C++ `acceptance` teszt zöld; a `cargo +nightly miri test -p rkv-ffi` undefined behavior nélkül fut; a `cargo metadata --no-deps` minden tagra 2024-es editiont mutat.

24.1.5.6 Készültségi állapot és átadási feltételek

A zöld minőségkapuk a **jelenlegi workspace** állapotát igazolják, nem azt, hogy a kurzus minden fejezete és minden nyilvános API-dokumentáció elkészült. A kettőt külön kell kezelni:

Állítás	Mit igazol	Állapot
`cargo test --workspace` zöld (93 teszt)	a jelenlegi kód viselkedése	igazolt
`clippy -D warnings`, `fmt --check`, `doc --no-deps`	a jelenlegi kód minősége	igazolt
minden publikus elem dokumentált	a nyilvános API szerződése	**nincs igazolva** (51 + 9 hiányzó dokumentáció)
minden fejezet projektállapota fordul	a könyv lépésenkénti építhetősége	**nincs CI-ben igazolva**
CMake/Corrosion build	a másodlagos buildút	**nem ellenőrzött kitekintés**
hardveres/QEMU futtatás	a beágyazott példák	**nem futtatott**

Az átadási feltételek a zöld kapukon felül:

1. **Publikus API-dokumentáció.** A `missing_docs` figyelmeztetések megszüntetése, és a publikus elemek szerződésének (paraméter, hiba, pánik, `# Safety`) dokumentálása. Ez külön kapu: a `RUSTDOCFLAGS="-D warnings" cargo doc` a *hiányzó* dokumentációt nem jelzi, csak a törött hivatkozásokat.

2. **Fejezetenkénti build.** A könyv által ajánlott `git tag rNNN` állapotok mindegyikét CI-ben kell építeni; a végső kód zöld tesztjei nem bizonyítják, hogy a korábbi fejezet önállóan is fordul. Például:

```yaml
# .github/workflows/chapters.yml (vázlat)
jobs:
  chapter:
    strategy:
      matrix:
        tag: [r003, r024, r030, r034, r040, r045, r051]
    steps:
      - uses: actions/checkout@v4
      - run: git checkout ${{ matrix.tag }}
      - run: cargo test --workspace
```

3. **Átvételi script.** A `bash scripts/accept.sh` zöld a végleges workspace-en.

A kurzus akkor tekinthető teljesen átadottnak, ha mind a három feltétel teljesül; a jelenlegi állapot ezek közül az első kettőt nem igazolja.

24.1.6 Átvételi tesztek és minőségkapuk

A minőségkapuk a fenti parancsok. A **tipikus hibák és átállási csapdák** ebben a fázisban:

- **A `Drop`-ban akarunk I/O-t végezni.** C++-ban a destruktorok gyakran írnak fájlt vagy zárnak kapcsolatot; Rustban a `Drop` nem adhat vissza `Result`-ot, és a pánikolás a `Drop`-ban abortához vezethet. A mentést explicitté kell tenni (`save_snapshot`, `compact`), a `Drop` csak erőforrás-lezárásra való.
- **A Miri-t kihagytuk, mert „a tesztek zöldek”.** A Miri az undefined behavior-t és a hibás mutatóaritmetikát keresi, amit a `cargo test` nem fedez fel. C++-ban a sanitizereknek felel meg; Rustban ez az egyik legerősebb eszköz az unsafe kód ellenőrzésére.
- **A clippy `-D warnings` túl szigorúnak tűnik.** A clippy lintjei gyakran konkrét Rust-idiómát javasolnak; a `-D warnings` arra kényszerít, hogy az API-tervezés és a kódbeli minták a API-tervezés Rustban alfejezetben tanult elveket kövessék.
- **A prefix felső határát számolni.** A `prefix..=prefix + z` vagy `+ char::MAX` kulcsok kihagynak prefixszel kezdődő kulcsokat; a `take_while(starts_with)` az egyetlen megbízható határ.
- **Az `edition` a `[workspace]` táblában.** Néma hiba: a Cargo figyelmeztet, de a tagok editionje 2015 lesz.

24.1.7 Kiterjesztési ötletek

Az átvett rendszer további irányai:

- **Replikáció:** a naplóbejegyzéseket a vezérlő szerver elküldi a követőknek; a Rust `async` modell és a tokio csatornák erre jól illenek.
- **TLS:** a `tokio-rustls` crate-tel a TCP-szerver TLS-t kapna.
- **Beágyazott kliens:** a `no_std`-s `rkv-core` változat a Beágyazott Rust kitekintés alfejezetben bemutatott `embedded-hal` és `Embassy` eszközökkel élne.

24.1.8 Feladatok

1. **Fogalmi kérdés.** A `compact` metódus sorrendje: tartós snapshot, csak utána a napló csonkolása. Sorolj fel három lehetséges összeomlási pontot, és mondd meg mindegyiknél, mit lát a következő `FileStore::open`. Miért nem működik a fenti `bad_compact`?

2. **C++-kód átírása Rustra.** Adott egy C++ függvény, amely egy `std::map<std::string, std::string>` elemei közül a megadott prefixszel kezdődő kulcsokat gyűjti ki. Valósítsd meg idiomatikus Rustban a `BTreeMap` `range` metódusával, kulcsok klónozása nélkül, amíg lehet.

3. **Implementációs feladat az rkv-ban.** Valósítsd meg az automatikus kompaktálást: a `FileStore` kapjon egy `set_compact_threshold(bytes: u64)` metódust (a küszöb mértékegysége a commitolt napló mérete **bájtban**), és ha a napló ezt átlépi, a következő sikeres `set`/`delete` hívja a `compact`-ot. Írj tesztet kicsi küszöbbel: lépd át a küszöböt, vizsgáld meg, hogy a snapshot létrejött, hogy a napló sosem nőtt a küszöb és egy rekord összege fölé, és hogy az újranyitás után az állapot változatlan. A korlát csak sikeres kompaktálás mellett igaz: ha a snapshotírás tartósan hibázik, a napló a küszöb fölé nőhet, ezért a hibát a `last_compact_error()` felszínre hozza.

4. **Hibajavítás, refaktorálás.** A `SharedStore::lock` a mérgezett zárra hibát ad. Mikor volna helyes a `PoisonError::into_inner`-rel továbbfolytatni, és mikor nem? Mit kell a tárolónak garantálnia, és hogyan védi ezt a `FileStore`?

24.1.9 Megoldások

24.1.9.1 1. feladat megoldása

Három pont: (a) a snapshot ideiglenes fájlba írása közben: a lemezen a régi snapshot és a teljes napló van, az ideiglenes fájl kósza; a nyitás a régi állapotot adja; (b) a `rename` után, de a naplócsonkolás előtt: új snapshot és teljes napló; a napló rekordjainak újrajátszása az új snapshotra a végállapotot nem változtatja (a `Set`/`Delete` idempotens), így ugyanaz az állapot áll helyre; (c) a csonkolás után: új snapshot és üres napló. A `bad_compact` fordított sorrendje a (b)-nél rosszabb helyzetet engedne: a napló törlése után, a snapshot tartóssága előtti összeomláskor a régi snapshot és a (már üres) napló maradna, és a legutóbbi írások elvesznének.

24.1.9.2 2. feladat megoldása

A C++:

```
std::vector<std::pair<std::string, std::string>>
list_prefix(const std::map<std::string, std::string>& map, const std::string& prefix) {
    std::vector<std::pair<std::string, std::string>> result;
    auto it = map.lower_bound(prefix);
    while (it != map.end() && it->first.rfind(prefix, 0) == 0) {
        result.emplace_back(*it);
        ++it;
    }
    return result;
}
```

A Rust-megfelelő:

```rust
use std::collections::BTreeMap;

fn list_prefix<'a>(
    map: &'a BTreeMap<String, String>,
    prefix: &str,
) -> impl Iterator<Item = (&'a str, &'a str)> {
    let prefix = prefix.to_owned();
    map.range(prefix.clone()..)
        .take_while(move |(k, _)| k.starts_with(&prefix))
        .map(|(k, v)| (k.as_str(), v.as_str()))
}
```

Az iterátor kölcsönzött `&str` párokat ad, nem klónoz; a hívó dönt arról, kell-e tulajdonolt másolat.

24.1.9.3 3. feladat megoldása

A `FileStore` két új mezőt kap, a küszöböt és a napló aktuális méretét; az `append_entry` a commitolt rekord hosszával növeli, a `compact` nullázza. A `set` és a `delete` a memória módosítása **után** hívja a `maybe_compact`-ot, tehát a kompaktálás sosem előzi meg a commitot.

Kódblokk — fájl: `rkv-core/src/store/file_store.rs` · művelet: változtatások az alábbi helyeken · ellenőrzés: `cargo test -p rkv-core --test auto_compact`

```rust
// a struktúrában
/// A napló jelenlegi mérete bájtban (a commitolt rekordok összege).
log_len: u64,
/// Ennél nagyobb napló esetén a `set`/`delete` után automatikus kompaktálás (bájt).
compact_threshold: Option<u64>,
/// Az automatikus kompaktálás legutóbbi hibája (best-effort: az írás attól még tartós).
last_compact_error: Option<String>,

// az open() végén: log_len: valid_len, compact_threshold: None, last_compact_error: None

/// Kompaktálási küszöb bájtban: ha a commitolt napló ennél nagyobb, a következő
/// sikeres `set`/`delete` kompaktál. Ha a kompaktálás hibázik, az iménti írás
/// attól még tartós; a hiba a következő írásnál újra próbálkozik.
pub fn set_compact_threshold(&mut self, bytes: u64) {
    self.compact_threshold = Some(bytes);
}

fn maybe_compact(&mut self) {
    if self.compact_threshold.is_some_and(|limit| self.log_len > limit) {
        // A hiba nem nyelődik el: eltároljuk, és a hívó lekérdezheti.
        self.last_compact_error = self.compact().err().map(|e| e.to_string());
    }
}

/// A legutóbbi automatikus kompaktálás hibája, ha volt. Tartós snapshotírási hiba esetén a
/// napló mérete a küszöb fölé nőhet: a korlát csak sikeres kompaktálás mellett érvényes.
pub fn last_compact_error(&self) -> Option<&str> {
    self.last_compact_error.as_deref()
}

// set()/delete(): a memória módosítása után: self.maybe_compact();
// append_entry(): a sikeres kiírás után:      self.log_len += line.len() as u64;
// compact(): a naplócsonkolás után:           self.log_len = 0;
```

A `let _ = self.compact()` tudatos döntés: a kompaktálás hibája nem teszi érvénytelenné az éppen commitolt írást (a napló továbbra is a teljes igazság), ezért az írás `Ok`-t ad, a következő írás pedig újra próbálkozik. Ha a hívó látni akarja a hibát, a `compact()` explicit hívható.

Kódblokk — fájl: `rkv-core/tests/auto_compact.rs` · művelet: új fájl · ellenőrzés: `cargo test -p rkv-core --test auto_compact` → 1 sikeres teszt

```
use rkv_core::{FileStore, Storage};

#[test]
fn compaction_triggers_at_the_threshold_and_keeps_the_state() {
    let dir = tempfile::tempdir().unwrap();
    let (snap, log) = (dir.path().join("state.json"), dir.path().join("append.log"));
    let log_len = || std::fs::metadata(&log).unwrap().len();

    let mut store = FileStore::open(&snap, &log).unwrap();
    store.set_compact_threshold(200); // bájt: néhány rekord után átlépjük
    assert!(!snap.exists());

    let mut max_len = 0;
    for i in 0..20u8 {
        store.set(format!("k{i}"), vec![i]).unwrap();
        max_len = max_len.max(log_len());
    }
    store.delete("k3").unwrap();

    // Volt kompaktálás: snapshot létrejött, és a napló sosem nőtt a küszöb + egy rekord fölé.
    assert!(snap.exists());
    assert!(max_len <= 200 + 64, "a napló túl nagyra nőtt: {max_len}");
    assert!(log_len() < max_len);

    // A visszaállított állapot a kompaktálás nélküli elvárással egyezik.
    let expected: Vec<(String, Vec<u8>)> = {
        let mut v: Vec<_> = (0..20u8).filter(|&i| i != 3).map(|i| (format!("k{i}"),
        ↪  vec![i])).collect();
        v.sort();
        v
    };
    assert_eq!(Storage::list(&store, None).unwrap(), expected);
    drop(store);
    let store = FileStore::open(&snap, &log).unwrap();
    assert_eq!(Storage::list(&store, None).unwrap(), expected);
}
```

A teszt a feladat összes pontját ellenőrzi: kicsi, bájtban megadott küszöb (200), a küszöb átlépése (20 rekord), a snapshot létrejötte, a napló korlátos mérete és az újranyitás utáni azonos állapot.

24.1.9.4 4. feladat megoldása

A mérgezett `Mutex` azt jelzi, hogy egy szál a zár birtokában pánikolt. Folytatni akkor helyes, ha a védett adat invariánsa a pánik után is érvényes, vagy ha a tároló maga védi azt. A `SharedStore` ezért alapértelmezésben hibát ad (nem engedi tovább a belső értéket), mert tetszőleges `Storage` panic-biztonságát nem ismeri. A `FileStore` ettől független védelmei: a sikertelen naplóírás után `poisoned` jelzőt állít, és minden további írást elutasít (a hiányzó kulcs törlését is), a memóriát pedig csak a tartós rekord után módosítja; újranyitás után a napló visszajátszása építi vissza az állapotot. Ez nem ad védelmet minden pánikra (például egy félig lefutott, nem naplózott módosításra egy saját `Storage`-ban); ott a hiba propagálása a biztonságosabb választás.

A hibák protokollválaszra képezése a szerverben ténylegesen így működik: a `process_line` a tárolóhibát `ERR <üzenet>` sorrá alakítja (egysorosra szűrve), a kapcsolatkezelő a választ kiírja és üríti (`reply`), majd a kapcsolat **nyitva marad**, és a következő parancsot dolgozza fel. Ha a blokkoló feladat maga pánikol (`JoinError`), a válasz `ERR internal error`, a kapcsolat ekkor is

folytatódik. A zárat a `SharedStore::apply` tartja, kizárólag a parancs végrehajtásának idejére, a `spawn_blocking` feladatban; hálózati I/O alatt (`.await` fölött) soha nem él őr.

24.1.10 Rövid összefoglaló

A záróprojekt az rkv workspace-t teljes rendszerként adja át, egyetlen adatmodellel (`String` kulcs, `Vec<u8>` érték), veszteségmentes JSON-alapú tárolási formátummal (v1) és egyetlen szöveges protokollal, egységes `LIST` keretezéssel (`VALUE …` sorok, záró `OK`). Az `rkv-core` `Storage` traites `MemStore` és `FileStore` típusai a snapshot + hozzáfűzéses napló mechanizmussal perzisztálnak (WAL-sorrend: tartós rekord, utána memória; kompaktálás: tartós snapshot, utána naplócsonkolás). A `LIST` prefix-lekérdezés a `BTreeMap::range` alulról zárt tartományát és a `take_while(starts_with)` felső határt használja. A workspace közös `edition`/`rust-version` értékét a `[workspace.package]` táblából örökli. Az átvételi tesztek az állapot-helyreállítási tesztre, a szerver `run_server`-t változtatás nélkül hívó integrációs tesztjére és a ténylegesen generált cxx-hídra épülő C++ tesztre támaszkodnak. A minőségkapuk: `cargo fmt --check`, `cargo clippy -- -D warnings`, `cargo test --workspace`, `cargo doc --no-deps`, valamint az `rkv-ffi` unsafe részeire a Miri nightly alatt. **Továbbvitt eredmény:** az Összefoglalás: C++→Rust fordítási kézikönyv és továbbtanulás összefoglaló ezt az átvételi állapotot és a minőségkapuk eredményét használja kiindulópontként a C++→Rust fordítási kézikönyvhöz.

24.1.11 Források és ellenőrzési hiányok

- Ellenőrizve (Rust 1.99.1, g++ 13, Linux): a teljes workspace (`cargo fmt --check`, `clippy -D warnings`, `cargo test --workspace`: 93 teszt, `cargo doc --no-deps`), a három átvételi teszt (`restore_state`, `list_prefix`, a C++ `acceptance`), a 3. feladat automatikus kompaktálása (`auto_compact`), a `cargo metadata` editionjei, a `[workspace]`-beli `edition` figyelmeztetése, a `BTreeMap::range` kimenete és a Miri az `rkv-ffi`-n.
- Nem futtatott: a CMake/corrosion build (nincs CMake), a Miri az `rkv-cxx`-en (idegen C++ hívások), hosszú ideig futó terheléses és összeomlás-injektáló tesztek.
- A `cargo miri` parancs pontos szintaxisa és a Miri által ellenőrzött garanciák a current nightly dokumentációban ellenőrizendők; a stabil toolchainen nem érhető el.
- A `cxx-build` által generált header elérési útja (`target/cxxbridge/<crate>/src/lib.rs.h`) a crate verziójától függhet.
- A `tempfile` crate 3-as főverzióját használjuk; a pontos alverziót a `Cargo.lock` rögzíti.

24.2 Összefoglalás: C++→Rust fordítási kézikönyv és továbbtanulás

A könyv utolsó fejezetéhez érkeztünk: az előző alfejezetben átvetted az rkv teljes rendszert, most pedig egy olyan kézikönyvet kapsz, amely a C++-ból érkező fejlesztő számára sűríti össze a döntő fordítási mintákat, a tipikus csapdákat és a továbbtanulási utakat. Ez az alfejezet nem új nyelvi anyagot tanít, hanem a korábbi huszonegy fejezet tudását rendezi referenciává. A cél az, hogy amikor egy C++-kódrészletet Rustba írsz át, ne csak szintaktikailag forduljon le, hanem idiomatikus, a Rust típusrendszerét kihasználó megoldás szülessen.

24.2.1 Motiváció

A C++-ból Rustba való átállás legnagyobb nehézsége nem a szintaxis, hanem az, hogy ugyanaz a tervezési reflex más következményekkel jár. A C++-ban bevált `std::shared_ptr`-alapú

gráfmodell például Rustban `Rc<RefCell<T>>`-halommá válhat, ami ugyanúgy nem skálázódik, mint a C++-os elődje, csak éppen a borrow checker és a futásidejű kölcsönzési pánik még kellemetlenebbé teszi. Ez a fejezet azokat a döntési pontokat foglalja össze, amelyekkel a könyv során találkoztál: mikor érdemes másolni, mikor kölcsönözni, mikor generikus és mikor dinamikus diszpécst használni, és hogyan válassz szál és async között.

Szabály. Mielőtt egy C++-mintát Rustra ültetsz, először keresd meg a Rust-idiómát; ha a C++-minta nem illik az ownership- és traitrendszerbe, ne erőltesd, hanem alakítsd át a tervezést.

24.2.2 Fogalmak és szabályok

Definíció (fordítási kézikönyv). A C++→Rust fordítási kézikönyv olyan rendezett idiómapárok gyűjteménye, amely egy adott C++-konstrukcióhoz a neki megfelelő, idiomatikus Rust-konstrukciót rendeli.

Definíció (döntési térkép). A döntési térkép olyan irányított döntéssorozat, amely a fő tervezési választásokban (tulajdonlás, absztrakció, párhuzamosság) a konkrét követelményekből vezeti le az ajánlott Rust-eszközt.

24.2.3 C++ és Rust összevetés: a fordítási kézikönyv

Az alábbi táblázat a leggyakoribb C++-idiómák Rust-megfelelőjét foglalja össze. A második oszlopban szereplő választás nem mindig egyetlen típus, hanem egy gondolkodási irány: a döntést a konkrét követelményekre kell lebontani.

C++-idióma	Rust-megfelelő	Megjegyzés
RAII, destruktor	`Drop` trait	Ugyanaz a determinisztikus felszabadítás, de a `drop` nem hívható explicit módon általános kódból: `drop(x)` a `Drop::drop` hívása után az értéket is érvényteleníti.

C++-idióma	Rust-megfelelő	Megjegyzés
`std::unique_ptr<T>`	`Box<T>` vagy közvetlen `T`	Rustban a kupacallokáció explicit: a `Box<T>` mindig a kupacra tesz, a közvetlen `T` pedig ott él, ahol a tartalmazó érték (lokális változó esetén a veremen, egy struct mezőjeként a tartalmazóban). A `String`, `Vec` és `HashMap` a saját puffereiket a kupacra teszik, ezért a „veremtárolás” nem a típus nevéhez, hanem a mezők elhelyezéséhez kötődik; egy `String` mezős enum például allokál. A `Box` akkor indokolt, ha a méretet a típus nem ismeri fordítási időben (rekurzív típus, trait-objektum), vagy ha nagy értéket mozgatunk gyakran; a `Shape` példában a `Box<dyn Shape>` a heterogén elemek miatt kell, nem azért, mert minden Rust-érték a veremen él.
`std::shared_ptr<T>` / `std::weak_ptr<T>`	`Rc<T>` / `Weak<T>` (egyszálú), `Arc<T>` / `Weak<T>` (többszálú)	Az `Arc` csak a *tulajdont* osztja meg, a módosítást nem szinkronizálja. Két különböző szerep: `Arc<Mutex<T>>` = közösen módosított `T` (`let s = Arc::clone(&a); thread::spawn(move || s.lock().unwrap().push(2));`); `Mutex<Arc<T>>` = lecserélhető, megosztott *handle*: az olvasó `Arc::clone(&*m.lock().unwrap())`-nal pillanatképet kap, az író `*m.lock().unwrap() = Arc::new(új)`-val kicseréli, a régi pillanatkép tovább él. Nem felcserélhető megoldások.
`std::optional<T>`	`Option<T>`	Nincs null; a hiányt a típusrendszer kényszeríti ki.
`std::expected<T, E>` (C++23) vagy kivételek	`Result<T, E>` + `?` operátor	A hibák értékként terjednek; a `?` a korai visszatérést valósítja meg.

C++-idióma	Rust-megfelelő	Megjegyzés
Osztályöröklés és virtuális függvények	`trait` + implementáció, trait-objektum (`dyn Trait`), generikus paraméter vagy `enum`	Nincs implementációs öröklés. A választás két tengelyen dől el: bővíthetőség (zárt halmaz → `enum`, nyílt → trait) és futásidejű heterogenitás (egy típus fordítási időben → generikus, vegyes elemek futásidőben → `dyn Trait`).
Sablonok és C++20 concepts	Generikusok `impl Trait` / `where` boundokkal	A generikus kód fordítási időben monomorfizálódik; a boundok a conceptekhez hasonlóan korlátozzák a típusparamétert.
`std::variant<T, U>`	`enum`	A Rust `enum` az összeg-típus; a variánsokhoz adat társítható, és a `match` kimerítő.
Lambda függvények	Closure-ök (`Fn`, `FnMut`, `FnOnce`)	A capture alapból implicit: a fordító a törzs használata alapján kikövetkezteti, hogy referenciát, módosítható referenciát vagy értéket fog meg; explicit csak a `move` kulcsszó (mindent érték szerint fog meg). A `Fn*` trait-megvalósítás is automatikus.
Iterátorok és `<algorithm>`	Iterator trait és adapterek (`Iterator`, `IntoIterator`)	A lusta adapterek önmagukban nem allokálnak; a gyűjtemények `iter()`, `iter_mut()`, `into_iter()` metódusokat adnak. Az iterátor nem indexel, ezért nincs határellenőrzés; az indexelt (`v[i]`) ciklusnál a nyelv ellenőriz, az optimalizáló pedig ott távolítja el az ellenőrzést, ahol bizonyítani tudja az index érvényességét (lásd Teljesítmény: mérés, profilozás, allokáció, release-beállítások).
`std::thread` + mutex	`std::thread` + `Mutex` / `RwLock` / csatornák	A `Send` és `Sync` határok garantálják a data race kizárását fordítási időben.
`std::async` / `std::future`	`async` / `await` + tokio (vagy más runtime)	A Rust future-ök lusták: pollolás alapúak, és a runtime futtatja őket.

C++-idióma	Rust-megfelelő	Megjegyzés
`reinterpret_cast`, nyers pointerek	`unsafe` blokk, `*const T` / `*mut T`, `MaybeUninit<T>`	A nem biztonságos műveletek explicit blokkba kerülnek, és a `// SAFETY:` komment dokumentálja az invariánst.

A táblázat kulcsa az a felismerés, hogy a Rust nem egyszerűen „jobb C++”, hanem más kompromisszumokat köt: a biztonságért cserébe a fordító több információt követel, és ez gyakran tisztább, de másképp strukturált kódot eredményez.

24.2.4 Kidolgozott példa: egy C++ Shape-osztály Rustra ültetése

Vegyünk egy tipikus C++-példát, amely öröklést és virtuális függvényt használ:

```
#include <iostream>
#include <memory>
#include <vector>

struct Shape {
    virtual ~Shape() = default;
    virtual double area() const = 0;
};

struct Circle : Shape {
    double r;
    explicit Circle(double radius) : r(radius) {}
    double area() const override { return 3.14159 * r * r; }
};

struct Square : Shape {
    double a;
    explicit Square(double side) : a(side) {}
    double area() const override { return a * a; }
};

double total_area(const std::vector<std::unique_ptr<Shape>>& shapes) {
    double sum = 0.0;
    for (const auto& s : shapes)
        sum += s->area();
    return sum;
}
```

A Rust-megfelelő két fő alakot ölthet. Ha a variánsok zártak, az `enum` a természetes választás:

```
enum Shape {
    Circle { r: f64 },
    Square { a: f64 },
}

impl Shape {
    fn area(&self) -> f64 {
        match self {
            Shape::Circle { r } => std::f64::consts::PI * r * r,
            Shape::Square { a } => a * a,
        }
    }
}
```

```rust
fn total_area(shapes: &[Shape]) -> f64 {
    shapes.iter().map(|s| s.area()).sum()
}
```

Ha viszont bővíthető hierarchiára van szükség, a trait-objektum a megfelelő:

```rust
trait Shape {
    fn area(&self) -> f64;
}

struct Circle { r: f64 }
impl Shape for Circle {
    fn area(&self) -> f64 {
        std::f64::consts::PI * self.r * self.r
    }
}

struct Square { a: f64 }
impl Shape for Square {
    fn area(&self) -> f64 {
        self.a * self.a
    }
}

fn total_area(shapes: &[Box<dyn Shape>]) -> f64 {
    shapes.iter().map(|s| s.area()).sum()
}
```

Különbség. A C++-ban a `std::unique_ptr<Shape>` és a virtuális hívás a szokásos nyílt hierarchia-minta. Rustban ugyanerre a `Box<dyn Shape>` használható, de csak akkor, ha a viselkedés valóban nyílt és heterogén. Ha a variánsok ismertek, az `enum` + `match` a jobb választás: nincs vtable-indirekció, nincs kupacallokáció, és a fordító ellenőrzi, hogy minden esetet kezeltél.

24.2.5 Tipikus átállási hibák

1. **`Rc<RefCell<T>>` túlhasználata.** A C++-os `shared_ptr`-reflex könnyen vezet `Rc<RefCell<T>>`-hálókhoz, ahol a kölcsönzés futásidőben bukik. A Rust azt kényszeríti ki, hogy a tulajdonviszonyokat előre átgondold: ki birtokolja az adatot, és ki kérdezheti le? Gyakran a megoldás a `Weak` használata ciklusok megtörésére, az `Rc` helyett egyetlen tulajdonos bevezetése, vagy a struktúra átdolgozása indexekkel/azonosítókkal.

2. **Klónozás a borrow checker legyőzésére.** Amikor a fordítási hiba E0507 (kimozgatás kölcsönzött vagy indexelt tartalomból: `cannot move out of index of Vec<String>`) vagy E0382 (használat mozgatás után: `borrow of moved value`), sok kezdő a `.clone()`-t szúrja be. Ez ideiglenesen elnémítja a hibát, de felesleges másolatokat és rejtett teljesítményveszteséget okoz. A helyes válasz gyakran a kölcsönzés (`&`), a `Cow` (copy-on-write), vagy az adatáramlás átszervezése úgy, hogy az érték ne kerüljön többszörös tulajdonba.

3. **`unwrap()` és `expect()` mindenhol.** A `Result` és `Option` feltárása nélkül a kód hamarosan tele lesz pánikokkal, amelyek a C++-os kivételeket idézik vissza. A könyvtári kódban a `?` operátor és a saját hibatípus a fenntartható út; az `expect` csak olyan invariánsokra való, amelyek megsértése programhibát jelez.

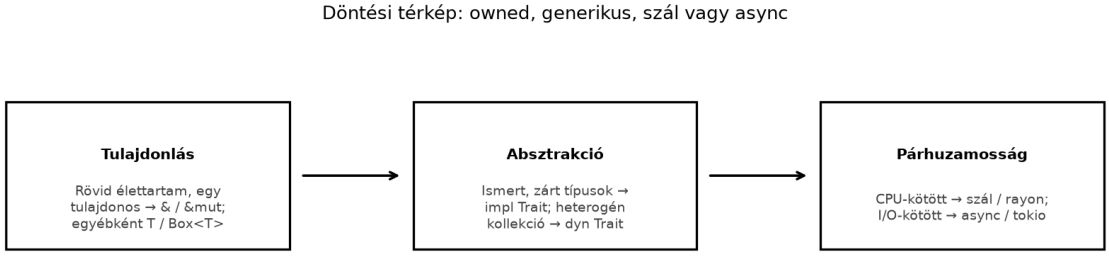
4. **Öröklés-szimuláció `Deref`-fel.** A `Deref` trait arra való, hogy egy típus úgy viselkedjen, mint egy másik típusú érték referenciája, nem pedig arra, hogy „alosztályozást” építs. A `Deref`-fel “örökölt” metódusok csak automatikus derefeléssel találhatók meg: a célértéken definiált trait-ek nem teljesítik a generikus határt a burkoló típusra, nincs altípusosság

(a `Wrapper` nem helyettesíthető `Target`-tel ott, ahol típust várnak), és a metódusfeloldás meglepő lehet, ha a burkoló és a cél metódusneve ütközik. Ha közös viselkedés kell, használj traitet; ha zárt variánsok, használj `enum`-ot.

5. **`unsafe` blokk indokolatlan használata.** A `unsafe` nem a C++-os „kapcsold ki az ellenőrzést” eszköz, hanem arra való, hogy olyan invariánsokat írj le, amelyeket a típusrendszer nem tud kifejezni. Minden `unsafe` blokkhoz `// SAFETY:` komment tartozik, és a blokk köré biztonságos absztrakció épül.

24.2.6 Döntési térkép

Az alábbi folyamatábra a leggyakoribb tervezési döntéseket foglalja össze. A döntés mindig a konkrét használatból indul ki, nem a C++-os beidegződésből.



24.1. ábra: Döntési térkép: owned, generikus, szál vagy async - A nyilak a döntések javasolt sorrendjét mutatják; a végső választás az élettartamtól, a heterogenitástól és a párhuzamossági modelltől függ.

Az ábra üzenete: először tisztázd a tulajdonlást, azután válassz absztrakciós mechanizmust, végül a párhuzamossági modellt. A három döntés független, de a rosszul megválasztott tulajdonlás később megbosszulja magát a párhuzamos kódban.

- **Owned vs. borrowed:** ha az adat élettartama rövid, és a hívó továbbra is birtokolja, kölcsönözz (`&T` vagy `&mut T`). Ha a kalauznak birtokolnia kell az adatot, használj `T`-t, nagy vagy rekurzív adatnál `Box<T>`-t.
- **Generikus vs. `dyn Trait`:** ha a típusok fordítási időben ismertek, és a kód nem okoz kódfúvódást, használj generikust. Ha heterogén kollekcióra vagy bővíthető plugin-rendszerre van szükséged, használj `dyn Trait`-et.
- **Szál vs. async:** CPU-kötött párhuzamossághoz szálak és `rayon`; sok párhuzamos I/O-hoz async és tokio. A kettő kombinálható, de egy projektben érdemes egy domináns modellt választani.

24.2.7 Továbbtanulás

A könyv végére értél, de a Rust ökoszisztéma ennél jóval mélyebb. A következő lépéseket a céljaid szerint válaszd:

Az alábbi források mindegyike megnyitható hivatkozás; az elérhetőséget és a tartalmat a szerzői ellenőrzés napján (2026-10-28) a rögzített toolchainhez (Rust 1.99.1) viszonyítva néztük meg.

Cél	Forrás	Hivatkozás	Kapcsolódó fejezet
Alapok, ownership	The Rust Programming Language (Rust Book)	https://doc.rust-lang.org/book/	Ownership
Nyelvi szabályok	The Rust Reference	https://doc.rust-lang.org/reference/	Unsafe Rust
Standard könyvtár	`std` API-dokumentáció	https://doc.rust-lang.org/std/	minden fejezet
Unsafe, belső működés	The Rustonomicon	https://doc.rust-lang.org/1.99.1/nomicon/meet-safe-and-unsafe.html	Unsafe Rust, Biztonságos absztrakció
Async	Asynchronous Programming in Rust	https://rust-lang.github.io/async-book/	Future-ök és async/await, Pin, Unpin, async trait-ek
Tokio	Tokio-útmutató és API	https://tokio.rs/tokio/tutorial; https://docs.rs/tokio/1.47.1/tokio/	Tokio
Beágyazott	The Embedded Rust Book	https://docs.rust-embedded.org/book/	Beágyazott Rust kitekintés
C++-interop	`cxx` dokumentáció	https://cxx.rs/	C++-együttműködés a cxx crate-tel
API-tervezés	Rust API Guidelines	https://rust-lang.github.io/api-guidelines/	API-tervezés Rustban
Hibakódok	Compiler Error Index	https://doc.rust-lang.org/error_codes/	minden fejezet
Cargo, workspace	The Cargo Book	https://doc.rust-lang.org/1.99.1/cargo/	Függőségek, feature-ök, workspace-ek
Edition	Rust 2024 Edition Guide	https://doc.rust-lang.org/edition-guide/rust-2024/index.html	Eszközlánc
Fordító belső működése	rustc-dev-guide	https://rustc-dev-guide.rust-lang.org/	A borrow checker működése
UB-ellenőrzés	Miri	https://github.com/rust-lang/miri	Záróprojekt

Crate-ek API-ját a docs.rs-en a verzió megadásával érdemes nyitni (például `https://docs.rs/embassy-executor/0.10.0`), mert az API verziónként változik.

24.2.8 Fogalomtár (angol–magyar)

Angol	Magyar	Hol
ownership	tulajdonjog	Ownership

Angol	Magyar	Hol
borrowing / borrow checker	kölcsönzés / kölcsönzés-ellenőr	Referenciák, A borrow checker működése
lifetime	élettartam	Élettartam-annotációk
move (destructive move)	mozgatás (destruktív mozgatás)	Ownership
trait / trait object	trait / trait-objektum	Trait-ek és generikusok, Trait objektumok
monomorphization	monomorfizálás	Trait-ek és generikusok
interior mutability	belső mutabilitás	Belső mutabilitás
smart pointer	okos mutató	Box, Rc, Arc és Weak
data race	adatversenyhelyzet	Szálak, Send és Sync
panic	pánik	Result, a ? operátor és a panic helye
zero-cost abstraction	nulla költségű absztrakció	Miért Rust?
undefined behavior (UB)	nem definiált viselkedés	Unsafe Rust
FFI / ABI	idegen függvény-interfész / bináris interfész	C ABI
macro hygiene	makróhigiénia	Deklaratív és procedurális makrók
variance	variancia	Newtype, PhantomData, typestate
typestate	típusállapot-minta	Newtype, PhantomData, typestate
future / executor / waker	future / végrehajtó / ébresztő	Future-ök és async/await
pinning	rögzítés (pin)	Pin, Unpin, async trait-ek
workspace / crate	munkaterület / csomag-egység	Függőségek, feature-ök, workspace-ek

24.2.9 Önellenőrző kérdéssor

Az alábbi kérdések a könyv kulcspontjait tesztelik. Válaszold meg őket a fejezet elolvasása után, majd ellenőrizd a megoldásokat. A kérdések nehézsége a sorrendben nő (1. könnyű, 2. közepes, 3. nehéz, 4. közepes, 5. nehéz); a megoldás megnyitása előtt fogalmazd meg a saját válaszodat, és csak utána hasonlítsd össze. A válasz akkor helyes, ha a megoldás kulcselemét és annak indokát is tartalmazza, nem csak a végeredményt.

1. **Fogalmi kérdés.** Melyik Rust-konstrukció felel meg a C++ `std::variant`-nak, és miért jobb annál a Rust `enum` mintaillesztéssel?
2. **Kódolási feladat.** Írj C++-kódot, amely `std::map<std::string, int>`-et tölt fel, majd írd át idiomatikus Rustra `HashMap`-mel és az `Entry` API-val. Miért nem a `[]` operátort használod?
3. **Tervezési döntés.** Egy rkv-bővítményben többféle háttértárolót szeretnél támogatni (`MemStore`, `FileStore`, és egy jövőbeli `RemoteStore`). Melyik Rust-absztrakciót választod: generikus paramétert vagy `dyn Storage` trait-objektumot? Indokold a választást két tengelyen: bővíthetőség (zárt/nyílt halmaz) és futásidejű heterogenitás (egy vagy több tárolótípus egyszerre, fordítási vagy futásidőben választva).
4. **Hibajavítás.** Az alábbi kód nem fordul le. Azonosítsd a hibát, és add meg a javítást úgy, hogy ne használj felesleges klónozást.

```rust
let mut data = vec![1, 2, 3];
let first = &data[0];
data.push(4);
println!("{}", first);
```

5. **Döntési térkép.** Egy nagy fájlt kell feldolgoznod, és minden sorhoz több független hálózati kérést küldesz. Melyik párhuzamossági modellt választod: szálakat vagy async-ot? Milyen crate-et használnál?

24.2.10 Megoldások

1. A C++ `std::variant` megfelelője a Rust `enum`. Az `enum` jobb, mert a `match` kimerítőségét a fordító ellenőrzi: ha új variánst adsz hozzá, a fordítási hiba felsorolja az érintett `match`-eket. A C++ `std::visit` hasonló célt szolgál, de a Rust beépített nyelvi támogatása egyszerűbbé és biztonságosabbá teszi a mintaillesztést.

2. C++:

```cpp
#include <iostream>
#include <map>
#include <string>

int main() {
    std::map<std::string, int> m;
    m["a"] = 1;
    m["b"] += 2;
    for (auto& [k, v] : m)
        std::cout << k << " -> " << v << '\n';
}
```

Rust:

```rust
use std::collections::HashMap;

fn main() {
    let mut m: HashMap<String, i32> = HashMap::new();
    m.entry("a".to_string()).or_insert(1);
    *m.entry("b".to_string()).or_insert(0) += 2;
    for (k, v) in &m {
        println!("{k} -> {v}");
    }
}
```

A `[]` operátor Rustban a `Index` trait, amely panikkel, ha a kulcs nem létezik. Az `Entry` API egyszerre ad hozzáférést a meglévő értékhez vagy hoz létre újat, és nem igényel kettős keresést.

3. A két tengely: *bővíthetőség* és *futásidejű heterogenitás*. Ha egy futásban egyetlen tárolótípus él, és azt fordítási időben ismerjük, a generikus paraméter (`Store<S: Storage>`) monomorfizál, nincs vtable-indirekció. Ha a típust futásidőben (konfigurációból) választjuk, vagy vegyesen kell tárolni őket, `dyn Storage` kell, akkor is, ha a halmaz zárt. Ha a halmaz zárt és a variánsok adatait közösen akarjuk kezelni, az `enum` + `match` is jó; nyílt halmaznál (bővítmények, jövőbeli `RemoteStore`) trait kell, mert az `enum`-ot minden új tárolónál módosítani kellene. A vtable-hívás költsége általában kicsi a lemezes tárolóhoz képest, de mérni kell; ne állítsunk általános sebességkülönbséget mérés nélkül. Az rkv `SharedStore`-ja `Arc<Mutex<Box<dyn Storage + Send>>>`-t használ, így a szerver futásidőben választhat háttértárolót (Mutex, RwLock, csatornák és a megosztott rkv).

4. A hiba: E0502 – nem lehet `&mut` kölcsönzés (`data.push(4)`) akkor, amikor egy `&` kölcsönzés (`first`) él. A javítás: kerüld a kölcsönzés és a módosítás átfedését. Például:

```
let mut data = vec![1, 2, 3];
let first = data[0]; // másolás, mert i32: Copy
data.push(4);
println!("{}", first);
```

Ha az érték nem `Copy`, a feladat viselkedését (a `push` után is az első elemet írjuk ki) kétféleképp őrizhetjük meg. Az első: a `push` után újra indexelünk, így nem él kölcsönzés a módosítás alatt:

```
let mut data = vec![String::from("a"), String::from("b")];
data.push(String::from("c"));
println!("{}", data[0]); // a
```

A második: ha a *push előtti* állapotról kell tulajdonolt pillanatkép, akkor a másolás indokolt, és a `clone` ezt jelzi:

```
let mut data = vec![String::from("a"), String::from("b")];
let first = data[0].clone(); // szükséges másolat: a Vec átméretezése miatt a &data[0] nem
↪  élhetne tovább
data.push(String::from("c"));
println!("{}", first); // a
```

Mindkét változat ugyanazt a `String`-et (`a`) írja ki, mint az eredeti szándék, és a `Vec` bővítésével együtt fordul.

5. Sok független hálózati kérést async modellel érdemes megvalósítani, mert az I/O-kötött feladatok várakozás közben nem foglalnak OS-szálat. A kérések számát korlátozni kell (visszanyomás), különben a nagy fájl minden sora egyszerre indítana kérést. Fontos részlet: a `tokio::io::AsyncBufReadExt::lines()` **nem** `futures` `Stream`, hanem `tokio::io::Lines`; streammé a `tokio-stream` `LinesStream` wrapperével alakítható. A `buffer_unordered` és a `try_fold` a `futures-util` crate-ből jön. A teljes, önálló program (a `fetch` itt a sor hosszát adja vissza; valódi hálózati hívásnál a HTTP-kliens hibáját `io::Error`-rá kell alakítani):

```
# Cargo.toml
[dependencies]
tokio = { version = "1", features = ["fs", "io-util", "macros", "rt-multi-thread"] }
tokio-stream = { version = "0.1", features = ["io-util"] }
futures-util = "0.3"

use std::io;

use futures_util::{StreamExt, TryStreamExt};
use tokio::fs::File;
use tokio::io::{AsyncBufReadExt, BufReader};
use tokio_stream::wrappers::LinesStream;

const MAX_IN_FLIGHT: usize = 4;

/// Egy sor feldolgozása: a példában a sor hossza a "válasz".
/// Valódi hálózati hívásnál itt egy HTTP-kliens hívása állna,
/// amelynek hibája `io::Error`-rá alakul.
async fn fetch(line: String) -> io::Result<usize> {
    Ok(line.trim().len())
}

#[tokio::main]
async fn main() -> io::Result<()> {
    let path = std::env::args().nth(1).unwrap_or_else(|| "urls.txt".to_string());
```

```
    let file = File::open(path).await?;
    let reader = BufReader::new(file);
    let lines = LinesStream::new(reader.lines());

    let total: usize = lines
        .map(|line| async move {
            let line = line?;
            fetch(line).await
        })
        .buffer_unordered(MAX_IN_FLIGHT)
        .try_fold(0usize, |acc, n| async move { Ok(acc + n) })
        .await?;

    println!("összeg: {total}");
    Ok(())
}
```

A sorolvasás hibája `io::Error` (`LinesStream Item = io::Result<String>`), a `fetch` hibája szintén `io::Error`; a `try_fold` az első hibánál leáll. A programot a rögzített toolchainnel (tokio 1.54.1, tokio-stream 0.1.19, futures-util 0.3.34) lefordítottuk és egy háromsoros fájlon futtattuk (`összeg: 9`). Tokio-alternatíva: `JoinSet` + `Arc<Semaphore>`; a `sem.clone().acquire_owned().await` a ciklusban vár, amíg felszabadul egy hely (ez a visszanyomás), a `set.join_next().await` gyűjti az eredményeket. Mindkét változatot lefuttattuk 50, illetve 20 elemre: a mért csúcs-egyidejűség 4 volt, a limit nem sérült. A szálmodell különbsége pontosan: egy blokkoló hívás *leköt egy OS-szálat* (a szál alszik, nem terheli a CPU-t, de a szálak száma és memóriája korlátos); CPU-terhelés esetén viszont a mag dolgozik, és az async nem gyorsít: a CPU-igényes munkát `spawn_blocking`-ba vagy `rayon`-ra kell tenni, különben az executor szálát foglalja le.

24.2.11 Rövid összefoglaló

Ez a záró alfejezet a C++→Rust átállás kézikönyvét adja: a fő C++-idiómák (`unique_ptr`, `shared_ptr`, öröklés, sablonok, kivételek, iterátorok, szálak) Rust-megfelelőit táblázatba foglalja, és kiemeli a döntő különbségeket. A gyakori átállási hibák – `Rc<RefCell<T>>` túlhasználat, klónozás a borrow checker ellen, `unwrap` mindenhol, öröklés-szimuláció `Deref`-fel, indokolatlan `unsafe` – mind a C++-os reflexek gátló hatását mutatják. A döntési térkép három fő választást rendszerez: owned vs. borrowed, generikus vs. `dyn Trait`, szál vs. async. A továbbtanulási út a Rustonomicon, az async dokumentáció, az Embedded Rust Book és a compiler internals felé mutat. Az önellenőrző kérdéssor a kulcspontokat teszteli. Ezzel az rkv projekt teljes rendszerként elkészült, és a könyv anyaga lezárult.

24.2.12 Források és ellenőrzési hiányok

Az alfejezet a könyv korábbi fejezeteire épít, különösen a Ownership: minden értéknek egy tulajdonosa van, a Trait-ek és generikusok: sablonok és concepts helyett, a Szálak, Send és Sync és a Tokio részekre. A továbbtanulási források közül a verziózható hivatalos dokumentáció a rögzített toolchain (Rust 1.99.1) alatt érhető el: a Rustonomicon „Meet Safe and Unsafe” fejezete (https://doc.rust-lang.org/1.99.1/nomicon/meet-safe-and-unsafe.html), a Rust Reference (https://doc.rust-lang.org/1.99.1/reference/) és a Cargo Book (https://doc.rust-lang.org/1.99.1/cargo/). Az Asynchronous Programming in Rust (https://rust-lang.github.io/async-book/01_getting_started/01_chapter.html), az Embedded Rust Book (https://docs.rust-embedded.org/book/intro/index.html) és a rustc-dev-guide (https://rustc-dev-guide.rust-lang.org/) élő, nem verziózott dokumentum; ezeknél a hivatkozás a 2026. októberi állapotot tükrözi, és a fejezet címére, nem a

gyűjtőoldalra mutat. A nyelvi állítások a stabil Rust-szabályokon alapulnak; a crate-verziók és a dokumentációk elérhetősége idővel változhat.

Függelékek

Fogalomtár (magyar–angol)

Ez a függelék a könyv kulcsfogalmait rögzíti egységes magyar és angol névvel, alfabetikus sorrendben. A nyelvi szabályokat és a számítási képleteket a Képlet- és jelölésgyűjtemény függelék tartalmazza.

Magyar	English	Jelentés / megkülönböztetés
`Arc`	atomically reference counted	szálbiztos, megosztott tulajdonlású okos pointer
async / await	asynchronous programming	felfüggeszthető jövőbeli értékre váró végrehajtás
borrow checker	borrow checker	a kölcsönzések és élettartamok fordítási ellenőrzője
borrowing	borrowing	hozzáférés átadása tulajdon átruházása nélkül
`Box`	heap allocation	egyetlen értéket a heapre tesz, egyedi tulajdonlással
`Cell`	interior mutability cell	másolható érték belső mutabilitása
`Clone`	explicit deep copy	explicit, felhasználó által definiált másolat
closure	closure	környezetét befogó névtelen függvény
`Copy`	implicit bitwise copy	bájtonként másolható, move helyett copy viselkedés
crate	crate	fordítási és csomagolási egység
`Drop`	destructor	az érték élettartamának végén futó tisztítás
`dyn Trait`	trait object	dinamikus diszpécsű, típus-erased trait-referencia
enum	enum	címkézett unió, mintázatillesztéssel
FFI	foreign function interface	C/C++ vagy más nyelvű bináris interfész
`Future`	future	aszinkron számítás állapotgépe
generikus	generic	típus- és lifetime-paraméteres absztrakció
`HashMap`	hash map	kulcs–érték tároló, átlagos O(1) kereséssel
interior mutability	interior mutability	mutáció megosztott referencián keresztül

Magyar	English	Jelentés / megkülönböztetés
iterator	iterator	lusta, láncolható elemsorozat
láthatóság	visibility	`pub` és modulhatárok által szabályozott elérés
lifetime	lifetime	referencia érvényességi tartománya
lifetime-elhagyás	lifetime elision	a fordító által kikövetkeztetett lifetime-paraméterek
makró	macro	kódot generáló nyelvi eszköz
`match`	pattern matching	értékek szerkezetére illesztő vezérlés
`Miri`	Miri interpreter	UB-t és memóriahibákat kereső interpreter
monomorfizáció	monomorphization	generikus kód típusonkénti példányosítása
move	move semantics	tulajdon átadása, az eredeti érték érvénytelenné válik
`Mutex`	mutual exclusion	szálak közötti kölcsönös kizárás
`Option`	optional value	`Some` vagy `None` érték
ownership	ownership	minden érték egyetlen tulajdonoshoz kötése
`panic` / unwind	panic / unwinding	nem várt hiba esetén történő leállás vagy kicsomagolás
`Pin`	pinned value	az érték memóriacímének rögzítése
procedurális makró	procedural macro	tokenfán dolgozó, fordítási idejű kódgenerátor
`Rc`	reference counted	nem szálbiztos, megosztott tulajdonlás
`RefCell`	interior mutability cell	futásidejű kölcsönzési szabályokat betartató cella
referencia	reference	kölcsönzött, nem tulajdonló hozzáférés
`repr(C)`	C layout	a C ABI-nak megfelelő memórialayout
`Result`	result type	`Ok` vagy `Err` hibaérték
`Send`	sendable across threads	az érték átadható más szálnak
slice	slice	tömb vagy `Vec` nem tulajdonló nézete
smart pointer	smart pointer	tulajdonlást és élettartamot kifejező pointer
`str`	string slice	UTF-8 szöveg nem tulajdonló szelete
`String`	growable UTF-8 string	heapen tárolt, növelhető szöveg
`Sync`	shared across threads	az érték megosztható hivatkozással szálak között
trait	trait	közös viselkedést leíró absztrakció
`trait object`	trait object	lásd `dyn Trait`

Magyar	English	Jelentés / megkülönböztetés
unsafe	unsafe block	a fordító által nem ellenőrzött invariánsok vállalása
`Vec`	growable array	heapen tárolt, növelhető tömb
zero-cost abstraction	zero-cost abstraction	fordítás után költségmentes absztrakció

Képlet- és jelölésgyűjtemény

Ez a függelék a könyv fontosabb memóriamodell-szabályait, számítási képleteit és jelöléseit gyűjti egy helyre. A Rust nyelvi garanciák pontos feltételeit mindig az adott fejezet és a hivatkozott dokumentáció rögzíti.

1. Méret, igazítás és memórialayout

Egy `T` típus mérete `size_of::<T>()`, igazítása `align_of::<T>()`. Egy mező kezdő offsetje a megelőző mezők méretétől és igazításától függ:

$$\text{offset}_{i+1} = \text{alignup}(\text{offset}_i + \text{size}_i,\ \text{align}_{i+1}).$$

A teljes struct mérete az utolsó mező után is igazított:

$$\text{sizeof}(S) \equiv 0 \pmod{\text{alignof}(S)}.$$

A `repr(C)` a C ABI szerinti mezősorrendet és igazítást kéri; a `repr(Rust)` layout nem garantált, ezért FFI-hez és bináris formátumhoz `repr(C)` kell. A `#[repr(transparent)]` egyetlen nem nulla méretű mezőt enged átlátszóan.

2. Ownership és kölcsönzés

Egy értéknek egyszerre egyetlen tulajdonosa van. Kölcsönzésre:

- egyszerre akár több megosztható (`&T`) referencia, **vagy**
- pontosan egy módosítható (`&mut T`) referencia,

de a kettő nem élhet együtt ugyanarra az értékre. A referenciának mindaddig érvényesnek kell maradnia, amíg használják; ezt a borrow checker ellenőrzi. A move után az eredeti változó nem használható, kivéve, ha a típus `Copy`.

Lifetime-elhagyás (lifetime elision) esetén a fordítási szabályok: 1. minden referencia-paraméter saját lifetime-ot kap; 2. egyetlen bemeneti lifetime esetén az összes kimeneti referencia azt kapja; 3. több bemeneti lifetime és `&self`/`&mut self` esetén a kimenet a self lifetime-ját kapja.

3. Kollekciók és komplexitás

Amortizált és legrosszabb eseti költségek:

Művelet	Átlagos	Legrosszabb
`Vec::push`	$O(1)$ amortizált	$O(n)$ újrafoglaláskor
`Vec` indexelés	$O(1)$	$O(1)$
`HashMap` keresés	$O(1)$ átlagos	$O(n)$ ütközés esetén
rendezés	$O(n \log n)$	$O(n \log n)$
lineáris keresés	$O(n)$	$O(n)$

A $O(f(n))$ jelölés jelentése: létezik $c > 0$ és n_0, hogy $T(n) \le c f(n)$ minden $n \ge n_0$ esetén. A Big-O mellett a konstansokat, az allokációt és a cache-viselkedést is figyelembe kell venni.

4. Hibakezelés és algebrai típusok

A `Result<T, E>` és az `Option<T>` zárt algebrai típus:

$$\text{Result} = \text{Ok}(T) \mid \text{Err}(E), \qquad \text{Option} = \text{Some}(T) \mid \text{None}\,.$$

A `?` operátor a `From` konverzióval terjeszti tovább a hibát; `Option` esetén `None`-t ad tovább. A `match` teljes illesztést kér: minden konstruktort le kell fedni, vagy `_` ágat kell adni.

5. Trait-ek, generikusok és dinamikus diszpécs

A generikus függvény monomorfizálódik: minden konkrét típusra külön példány keletkezik, ezért futásidőben nincs dinamikus diszpécs. A `dyn Trait` trait object virtual táblán keresztül hív, ezért futásidejű költséggel és típus-erasure-rel jár. A `where` feltétel és a trait bound a fordítási idejű szerződés.

A `Send` azt jelenti, hogy az érték átadható más szálnak; a `Sync` azt, hogy megosztható hivatkozással több szál között. Ezek auto trait-ek, és a negatív implementáció (`!Send`, `!Sync`) is lehetséges.

6. Egyidejűség és aszinkron

A közös állapotot `Mutex`/`RwLock` védi; a zár megőrzése alatt tartott referencia élettartama a zárhatárig tart. A `Arc<Mutex<T>>` a szálak közötti megosztott tulajdonlás tipikus mintája. Az atomikus műveletek `Ordering`-ja (Relaxed, Acquire, Release, AcqRel, SeqCst) határozza meg a láthatóságot; a `Relaxed` nem ad sorrendi garanciát.

Az `async fn` egy `Future`-t ad vissza; a `.await` a future pollozását végzi. A future nem fut magától: executor (például Tokio) szükséges. A `Pin` azokat az önreferenciális future-öket védi, amelyek címét nem szabad megváltoztatni.

7. Unsafe, FFI és ABI

Az `unsafe` blokk nem kapcsolja ki a borrow checkert; a programozó vállalja a nem ellenőrzött invariánsokat (pointer-érvényesség, aliasing, initialization). Az FFI-hez `extern "C"`, `repr(C)` és a C ABI-nak megfelelő típusok kellenek. A `*const T`/`*mut T` nyers pointer lehet null vagy érvénytelen; dereferálása csak `unsafe` blokkban és bizonyított invariánsok mellett történhet.

8. Numerika és hibakezelés

Egész túlcsordulás debug buildben panic, release buildben wraparound; a hordozható szándékhoz `checked_*`, `wrapping_*` vagy `saturating_*` kell. A lebegőpontos műveletekre az IEEE 754 szabályai érvényesek; a NaN nem egyenlő önmagával. A `f32`/`f64` közötti konverzió és a kerekítés platformfüggetlen, de a numerikus stabilitás a műveletsorrendtől függ.

Kompetenciatérkép és tanulási útvonal

A könyv célprofilja: gyakorlott C++-fejlesztő, aki a Rust tulajdonlási, kölcsönzési és típusrendszeri modelljét a nyelv filozófiájával együtt akarja elsajátítani, és a záróprojektben egy valós CLI/kv-tároló rendszert épít. A térkép tíz kompetenciát és tíz ellenőrző pontot rögzít.

Belépési és kilépési feltételek

Belépés. C++-gyakorlat (RAII, STL, template-ek, move szemantika), Linux-parancssor és alapvető build/ismeret. Nem szükséges előzetes Rust-, async- vagy FFI-tapasztalat.

Kilépés. A tanuló önállóan tervez ownership-helyes Rust API-t, megírja a `rkv` tároló core-, CLI-, kliens–szerver- és FFI-rétegét, kezeli az élettartamokat és a hibákat, valamint méréssel igazolja a teljesítményét.

A tíz kompetencia és ellenőrző pontjai

#	Kompetencia	Mit tud a tanuló?	Ellenőrző pont	Bizonyíték
1	Eszközlánc és filozófia	Használja a cargo/rustup/clippy eszközöket, és megkülönbözteti a Rust garanciáit a C++ gyakorlattól	`r003`	működő `rkv` workspace, clippy-tiszta build
2	Szintaxis, típusok, struct/enum	Ír típushelyes Rust kódot, mintaillesztéssel és impl-blokkokkal	`r009`	szintaxis- és enum-feladat, fordítási tesztek
3	Modulok, crate-ek, láthatóság	Rendezi a kódot modulokba és crate-ekbe, helyes `pub` határokkal	`r011`	több-crate workspace, API-dokumentáció

#	Kompetencia	Mit tud a tanuló?	Ellenőrző pont	Bizonyíték
4	Ownership, borrowing, lifetime	Megtervezi a tulajdonviszonyt, elkerüli a borrow-hibákat, érti a lifetime-következtetést	`r018`	borrow-checker-tiszta modul, lifetime-magyarázat
5	Hibakezelés, trait-ek, generikusok	`Result`/`Option` alapú API-t, trait boundokat és generikus absztrakciót ír	`r023`	hibakezelő könyvtár, trait-alapú absztrakció
6	Kollekciók, iteratorok, closure-ök	Hatékonyan használja a `Vec`/`HashMap`/iterator láncokat, ismeri a költségeket	`r030`	iteratoros feldolgozó, komplexitási jegyzőkönyv
7	Async, konkurencia, üzenetátadás	Szálbiztos és async kódot ír, érti a `Send`/`Sync` és a záró-élettartam szabályait	`r036`	tokio-szerver, konkurencia-teszt
8	Unsafe, FFI, makrók	Indokolja az `unsafe` invariánsokat, C ABI-t készít, makrót ír	`r046`	C-ből hívható FFI, makró-teszt
9	Haladó típusrendszer és teljesítmény	Használ advanced trait-mintákat, méri és optimalizálja a kódot	`r049`	benchmark-jegyzőkönyv, API-terv
10	Beágyazott kitekintés és záróprojekt	Elhelyezi a Rustot beágyazott környezetben, és átadja a teljes `rkv` rendszert	`r052`	záróprojekt, átvételi teszt, dokumentáció

Ajánlott útvonal

1. **Alapok (1–3. kompetencia):** `r000`–`r011`. Az eszközlánc, a szintaxis és a modulrendszer nélkül a ownership nem gyakorolható biztonságosan.
2. **Ownership és absztrakció (4–5. kompetencia):** `r012`–`r023`. A kölcsönzés, az élettartamok, a trait-ek és a hibakezelés a nyelv magja.
3. **Adat és egyidejűség (6–7. kompetencia):** `r024`–`r036`. Az iteratorok, a kollekciók és az async/konkurencia adják a valós alkalmazásokat.

4. **Rendszerszint és projekt (8–10. kompetencia):** `r037`–`r052`. Az unsafe, az FFI, a makrók, a teljesítmény és a záróprojekt zárja a könyvet.

Útmutató a haladáshoz

- Minden ownership-hibánál először a tulajdonviszonyt rajzold fel; a fordító hibaüzenete a szándékot és az élettartamot is ellenőrzi.
- Az `unsafe` blokkhoz írj invariánslistát, és a blokkon kívül teszteld a viselkedést; az unsafe nem a borrow checker megkerülése.
- Az FFI-hez `repr(C)`, dokumentált ABI és C-oldali teszt kell; a Rust-oldali típuslayout nem garantált.
- Teljesítménynél a mért és a levezetett értéket különítsd el, és rögzítsd a build profilt, a crate-verziókat és a hardvert.

Bibliográfia és forráskulcsok

Ezek a tervben szereplő kiindulópontok, nem ebben a tervben ellenőrzött források.

Fejezetenként közös forráscsomag. A fejezet-leírás fókusza és URL-jei az elsődlegesek; az alábbi lista referencia-térkép, nem minden fejezetben feldolgozandó teljes lista. Minimum három releváns forrás szükséges, olvasott szakasz/szabály/API szerint. A Rust-nyelv dokumentációja, a std- és crate-dokumentáció (docs.rs, verzióval), az RFC-k és a közösségi könyvek külön kezeltek. Edition-, stabilizálás- és crate-verziófüggő részleteknél a verziót és az ellenőrzés dátumát rögzíteni kell. Továbbtanulás further_reading; hiány gaps; verzió vagy mért érték nem található ki.

Hivatalos nyelvi és eszköz-dokumentáció:

- https://doc.rust-lang.org/book/ (The Rust Programming Language)
- https://doc.rust-lang.org/reference/ (The Rust Reference)
- https://doc.rust-lang.org/std/ (Rust standard könyvtár dokumentáció)
- https://doc.rust-lang.org/cargo/ (The Cargo Book)
- https://rust-lang.github.io/rustup/ (rustup könyv)
- https://doc.rust-lang.org/edition-guide/ (Rust Edition Guide)
- https://doc.rust-lang.org/clippy/ (Clippy dokumentáció)
- https://doc.rust-lang.org/rustdoc/ (The rustdoc book)
- https://doc.rust-lang.org/rust-by-example/ (Rust by Example)
- https://rust-lang.github.io/api-guidelines/ (Rust API Guidelines)

Haladó és unsafe:

- https://doc.rust-lang.org/nomicon/ (The Rustonomicon)
- https://rust-lang.github.io/unsafe-code-guidelines/ (Unsafe Code Guidelines)
- https://github.com/rust-lang/miri (Miri)
- https://doc.rust-lang.org/nomicon/ffi.html (Rustonomicon: FFI)
- https://marabos.nl/atomics/ (Rust Atomics and Locks (M. Bos))
- https://nnethercote.github.io/perf-book/ (The Rust Performance Book (N. Nethercote))
- https://veykril.github.io/tlborm/ (The Little Book of Rust Macros)
- https://doc.rust-lang.org/reference/procedural-macros.html (Rust Reference: procedurális makrók)

Async:

- https://rust-lang.github.io/async-book/ (Asynchronous Programming in Rust)
- https://tokio.rs/tokio/tutorial (Tokio tutorial)

Crate-ek (docs.rs):

- https://docs.rs/clap/ (clap)
- https://serde.rs/ (serde)
- https://docs.rs/thiserror/ (thiserror)
- https://docs.rs/anyhow/ (anyhow)
- https://docs.rs/rayon/ (rayon)
- https://docs.rs/proptest/ (proptest)
- https://docs.rs/criterion/ (criterion)
- https://docs.rs/tracing/ (tracing)
- https://rust-lang.github.io/rust-bindgen/ (bindgen útmutató)
- https://github.com/mozilla/cbindgen (cbindgen)
- https://cxx.rs/ (cxx — biztonságos C++/Rust FFI)

Beágyazott:

- https://docs.rust-embedded.org/book/ (The Embedded Rust Book)
- https://docs.rs/embedded-hal/ (embedded-hal)
- https://embassy.dev/ (Embassy)

C++ összevetéshez:

- https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines (C++ Core Guidelines (összevetésekhez))